NixOS

K19G

2026-11-02

Table of contents

Notes on Nix & NixOS **100**
What "done" looks like . . . 100
How the book is organized . . . 100
Recommended path if your daily driver is Fedora . . . 101
Lab 0 (recommended before first host work) . . . 102
How I intend to study . . . 102
Prerequisites . . . 103
Related books . . . 103
Formats . . . 103
Acknowledgments . . . 103

Front Matter **104**

Syllabus **105**

Syllabus — expert path for Nix & NixOS **106**

Why this order (expert rationale) **107**
How source notes were used . . . 107
Other languages (including Go) . . . 108

Modern stack (defaults for this book) **109**

Part map (topic book) **110**

Front matter — Foundations & Lab 0 (before Concepts) **113**

Concepts — Foundations **114**

Nix on Linux — Fedora bridge (recommended before host cutover) **115**

NixOS host **116**

Home Manager and flake layout **117**

Services and security **118**

Packaging craft **119**

Ops and fleet **120**

Internals and currency **121**

Capstone and recovery **122**
Capstone A — required . . . 122
Capstone B — stretch . . . 123

Projects (optional depth track) **124**

Review habit **125**

Completeness audit (sources → parts) **126**

Explicitly out of scope **127**

Resources (primary, not blogs-of-2022) **128**

Writing rule for future chapters **129**

Lab hardware default (this book) **130**

Introduction: The Reproducibility Crisis **131**

Introduction: The Reproducibility Crisis **132**
The Heisenbugs That Cost Teams . . . 132
Why Traditional Tooling Fails . . . 133
The Container Illusion . . . 134
A Different Approach: Declarative, Immutable Infrastructure . . . 134
The Three Tenets of the Nix Model . . . 135
The Shift from Imperative Ops to Mathematical Determinism . . . 135
What This Book Will Teach You . . . 136
Key Milestones on Your Path . . . 136

Lab 0 — Overview: roles before the Why Nix exists chapter **137**

Lab 0 — Overview: roles before the Why Nix exists chapter **138**
Why a lab host exists . . . 138
Philosophy: host is not an appliance . . . 138
Three roles (do not collapse them) . . . 139
Chapters in this section . . . 139
Recommended topology for this curriculum . . . 140
Decision: host OS . . . 140
Hypervisor choice (libvirt vs Incus) . . . 141
What "ready for the Why Nix exists chapter" means . . . 141
Time budget . . . 141
Further reading . . . 142
Next . . . 142

Lab 0A — Journey single-server (KVM + containers) **143**

Lab 0A — Journey single-server (KVM + containers) **144**
What this lab optimizes for . . . 144
Architecture . . . 144

Part 1 — Host OS install (metal) . . . 145
Option H1 (recommended): NixOS host . . . 145
Option H2: Debian/Ubuntu host . . . 146
Part 2 — Storage plan (critical on 16 GB RAM laptops) . . . 146
If the Elitebook still has a spinning HDD . . . 146
Thin provision . . . 146
Part 3 — KVM: create journey VMs . . . 147
Resource template (16 GB host) . . . 147
Example: `virt-install` (adjust ISO path) . . . 147
Snapshots = courage . . . 147
Networking for "remote" deploy . . . 147
Part 4 — Containers (Podman) . . . 148
Part 4b — Optional hypervisor: Incus (CLI-first VMs + system containers) . . . 148
When to consider Incus on the lab host . . . 148
NixOS host sketch (Incus) . . . 149
Simulate the **host** before you reinstall metal . . . 149
If you already run Proxmox . . . 149
Part 5 — Nested virtualization? . . . 150
Part 6 — Services the host should run (boring is good) . . . 150
Part 7 — Daily workflow for the journey . . . 150
Debugging without drift . . . 151
Part 8 — Declarative hardware quirks (host flake) . . . 151
Part 9 — Failure modes checklist . . . 151
Checkpoint — Lab 0A complete . . . 152
Next . . . 152

Lab 0B — NixOS development machine **153**

Lab 0B — NixOS development machine **154**
Dev machine vs journey guest . . . 154
Part 1 — Install Nix (if host is not NixOS) . . . 154
Part 2 — Essential packages . . . 155
NixOS host module sketch . . . 155
Non-NixOS host . . . 155
Part 3 — Editor + LSP . . . 155
Part 4 — Git layout for the whole journey . . . 156
Part 5 — Build ergonomics . . . 156
Keep host builds from melting a 2012 laptop . . . 156
Substituters . . . 157
Remote / guest as builder (optional, Stage V+) . . . 157
Part 6 — Access to journey VMs . . . 157
Part 7 — Workflow commands you will live with . . . 157
Part 8 — Home Manager on the **dev** role . . . 158
Part 9 — Binary cache & secrets (preview only) . . . 158
Part 10 — Quality of life on old hardware . . . 158
Dev machine checklist . . . 159
Anti-patterns . . . 159
Next . . . 159

Lab 0C — HP Elitebook 2570p (16 GB) profile **160**

Lab 0C — HP Elitebook 2570p · 16 GB RAM lab profile **161**
Hardware snapshot (typical 2570p) . . . 161
BIOS checklist (do this first) . . . 162
Disk strategy for *this* laptop . . . 162
Best . . . 162
Acceptable . . . 162
Partition sketch (NixOS host, single disk) . . . 162
RAM budget (16 GB fixed) . . . 162
Suggested virsh memory . . . 163
CPU budget . . . 163
What this machine is good for (curriculum map) . . . 163
Networking on the Elitebook . . . 164
Power & thermals . . . 164
Firmware & drivers notes . . . 164
Host is fully yours (but still a lab) . . . 164
Concrete "day zero" procedure (you) . . . 165
Optional extras that fit this chassis . . . 165
What not to run on the 2570p . . . 166
Elitebook readiness checklist . . . 166
After this page . . . 166

Lab 0D — Ops cheatsheet **167**

Lab 0D — Ops cheatsheet (single-server lab) **168**
Daily . . . 168
Snapshots / restore points . . . 168
Disk pressure . . . 169
Guest rebuild (from host flake) . . . 169
When the Elitebook sweats . . . 169
Drift control . . . 169
Emergency . . . 169
Inventory template (fill once) . . . 170

Concepts **171**

Concepts — Nix language, store, CLI, flakes **172**

Concepts — Nix language, store, CLI, flakes **173**
What this part is for . . . 173
Baseline (this book) . . . 173
Learning outcomes . . . 174
Prerequisites . . . 174
Chapter map . . . 174
Suggested learning paths . . . 175
How each chapter is structured . . . 175
Lab workspace convention . . . 176

Mental model map (part-level) . . . 176
What you will *not* do in this part . . . 177
Suggested checkpoints (part gate) . . . 177
Further depth (this book) . . . 177
How to use this part as a reference . . . 178

Why Nix exists **179**

Why Nix exists **180**
Why this chapter matters . . . 180
Theory 1 — Three faces of the same ecosystem . . . 180
Analogy (imperfect but useful) . . . 181
Naming traps in the wild . . . 181
Theory 2 — Failure modes of traditional systems . . . 182
Dependency hell . . . 182
Snowflake servers . . . 182
Language-level global installs . . . 182
CI and agent "works on my machine" graphs (the three-way drift) . . . 182
What "reproducible" can mean (honesty ladder) . . . 183
Theory 3 — Nix's core bet: pure builds + hashed store . . . 183
Content-addressed intuition . . . 183
Coexistence . . . 184
Environments as compositions . . . 184
What purity buys operations . . . 184
Theory 4 — Evaluation vs realization . . . 185
Substituters (binary caches) . . . 185
Why the split matters for debugging . . . 185
Theory 5 — Installer shapes and multi-user daemon . . . 186
Why a daemon? . . . 186
Config locations (theory) . . . 186
Disk and trust . . . 186
Theory 6 — Flakes as the modern pinning model (preview) . . . 186
Baseline pin you will reuse . . . 187
Theory 7 — CLI eras (so docs don't gaslight you) . . . 187
Theory 8 — What Nix is *not* . . . 187
Worked examples . . . 188
Example A — "Install" without polluting a global profile . . . 188
Example B — Why two machines diverge without locks . . . 188
Example C — What Nix does *not* do . . . 188
Example D — Eval error vs build error (feel the phases) . . . 188
Example E — Minimal REPL tour . . . 188
Example F — Inspect config after enabling features . . . 189
Lab 1 — Install Nix . . . 189
Lab 2 — Enable modern features . . . 189
Lab 3 — First modern commands . . . 190
REPL . . . 190
Lab 4 — Write the model . . . 190
Exercises . . . 190

Common pitfalls . . . 191
Checkpoint . . . 191
Further depth (this book) . . . 192

The Nix store and closures **193**

The Nix store and closures **194**
Why this chapter matters . . . 194
Theory 1 — The store is the database . . . 194
Invariants (practical) . . . 194
What lives beside the store . . . 195
Theory 2 — Path anatomy . . . 195
Examples (illustrative shapes) . . . 195
Input-addressed vs content-addressed (awareness) . . . 196
Name segment is not a version pin . . . 196
Theory 3 — References and the closure . . . 196
Direct references . . . 196
Closure (requisites) . . . 197
Why closures matter — worked scenarios . . . 197
Example narrative . . . 197
Closure size theory . . . 197
Referrers (reverse edges) . . . 198
Theory 4 — Evaluation vs realization (store-centric) . . . 198
Theory 5 — Profiles, generations, and roots . . . 199
Why GC exists . . . 199
Root types (non-exhaustive) . . . 199
The `./result` trap (critical theory) . . . 199
Generations theory (preview) . . . 200
Theory 6 — Substituters, signatures, trust (preview) . . . 200
Theory 7 — What "uninstall" means . . . 200
Operational uninstall checklist . . . 201
Theory 8 — Copying closures (ops preview) . . . 201
Worked examples bank . . . 201
Example A — Build and print path . . . 201
Example B — Direct vs full graph . . . 201
Example C — Tree view . . . 202
Example D — Result root lifecycle . . . 202
Example E — Path info metadata . . . 202
Example F — Compare two packages . . . 202
Lab 1 — Realize `hello` and dissect the path . . . 202
Lab 2 — Closure measurements . . . 203
Lab 3 — Follow one dependency . . . 203
Lab 4 — Roots observation . . . 203
Lab 5 — Teach-back . . . 203
Exercises . . . 204
Common pitfalls . . . 204
Checkpoint . . . 205
Journal prompts . . . 205

Further depth (this book) . . . 205

Nix language: values and attrsets **206**

Nix language: values and attrsets **207**
Why this chapter matters . . . 207
Theory 1 — Values are few; composition is everything . . . 207
Type inspection . . . 208
Theory 2 — Strings: interpolation and two quote forms . . . 208
Double quotes . . . 208
Indented strings (`'' … ''`) . . . 208
Escaping notes . . . 208
Paths vs path-looking strings . . . 209
Theory 3 — Lists . . . 209
List builtins you will reuse . . . 209
Theory 4 — Attribute sets (the main event) . . . 210
Access . . . 210
Nested sets . . . 210
Dynamic keys . . . 210
Update / merge intuition . . . 211
Attrset builtins . . . 211
Theory 5 — `let … in` (local bindings) . . . 211
Rules that matter . . . 211
Nested lets vs one flat let . . . 212
`rec` attrsets . . . 212
Theory 6 — `inherit` (pull names into sets or lets) . . . 212
Into an attrset . . . 212
Rename form . . . 212
Into `let` . . . 213
Theory 7 — `with` (know it; use sparingly) . . . 213
Theory 8 — Conditionals and operators (minimal set) . . . 213
Theory 9 — Config-shaped modeling (bridge to NixOS) . . . 214
Theory 10 — Destructuring patterns you will see . . . 215
Worked examples bank . . . 215
Example A — REPL tour . . . 215
Example B — Deep access and missing attrs . . . 215
Example C — `inherit` package list pattern . . . 216
Example D — Shallow merge trap . . . 216
Example E — Dynamic user keys . . . 216
Lab 1 — Workspace and file evaluation . . . 217
Lab 2 — Nested merge experiments . . . 217
Lab 3 — Attrset utilities in the REPL . . . 218
Lab 4 — Model something real from your life . . . 218
Lab 5 — Teach-back . . . 219
Exercises . . . 219
Common pitfalls . . . 220
Checkpoint . . . 220
Further depth (this book) . . . 221

Nix language: functions **222**

Nix language: functions **223**
Why this chapter matters . . . 223
Theory 1 — Everything callable is a lambda . . . 223
Application is juxtaposition . . . 223
Functions are values . . . 224
Identity and composition (lightweight) . . . 224
Theory 2 — Attrset pattern arguments (the Nix idiom) . . . 224
Destructuring is the point . . . 224
Why not positional for public APIs? . . . 224
Theory 3 — Default arguments with `?` . . . 225
Defaults can be expressions . . . 225
Explicit `null` vs missing key . . . 225
Theory 4 — Ellipsis `...` (allow unknown attributes) . . . 226
When to use ellipsis . . . 226
Theory 5 — `@` binding (name the whole set) . . . 226
Why `@` exists . . . 227
Theory 6 — Composition and higher-order helpers . . . 227
`map` / `filter` (builtins) . . . 227
Function returning function . . . 227
`enableIf`-style helper (preview of module style) . . . 228
Theory 7 — Fixed-point awareness (not mastery) . . . 228
Theory 8 — Call patterns you will see in the wild . . . 228
1. Package "callPackage" shape . . . 228
2. Overlay shape . . . 229
3. Module shape (preview) . . . 229
4. `override` / `overrideAttrs` as functions . . . 229
Theory 9 — Error messages as teachers . . . 229
Worked examples bank . . . 230
Example A — Defaults + ellipsis . . . 230
Example B — `@` forward . . . 230
Example C — User skeleton helper . . . 230
Example D — Package list merge helper . . . 231
Example E — Strict API for typos . . . 231
Lab 1 — Workspace . . . 231
Lab 2 — Strict vs ellipsis . . . 232
Lab 3 — `@` args inventory . . . 232
Lab 4 — Compose three helpers into a "mini host" . . . 233
Lab 5 — Read real shapes (optional but high value) . . . 233
Exercises . . . 233
Common pitfalls . . . 234
Checkpoint . . . 234
Further depth (this book) . . . 235

Laziness, imports, and lib **236**

Laziness, imports, and lib **237**
Why this chapter matters . . . 237
Theory 1 — Laziness: values are promises . . . 237
Why it matters . . . 237
Forcing . . . 238
Mental model . . . 238
Theory 2 — Imports are different: loading is eager (once forced) . . . 238
Mental model . . . 238
Common patterns . . . 238
Paths . . . 239
Import caching . . . 239
Theory 3 — Multi-file composition patterns . . . 239
Pattern A — Data module . . . 239
Pattern B — Function module . . . 239
Pattern C — Directory + `default.nix` . . . 240
Pattern D — Layered host-shaped tree (preview) . . . 240
Theory 4 — Infinite recursion (how you will meet it) . . . 240
Debugging habits . . . 241
Fixed points (recognition only) . . . 241
Theory 5 — `lib` as the standard library (taste, not catalog) . . . 241
Attrsets . . . 241
Lists . . . 242
Strings . . . 242
Example pipeline — users list → attrset . . . 242
Why not memorize all of `lib`? . . . 242
Theory 6 — `NIX_PATH` and angle-bracket paths (literacy) . . . 243
Theory 7 — Evaluation vs runtime (again, file-centric) . . . 243
When does "work" happen? . . . 243
Theory 8 — Project layout habits . . . 243
Worked examples bank . . . 244
Example A — Lazy skip . . . 244
Example B — Import function . . . 244
Example C — `lib.optional` / `optionals` . . . 244
Example D — `mapAttrs` . . . 244
Example E — `recursiveUpdate` vs `//` . . . 245
Lab 1 — Multi-file project skeleton . . . 245
Lab 2 — Trace forcing . . . 246
Lab 3 — Infinite recursion (safe sandbox) . . . 246
Lab 4 — Real `lib` if nixpkgs available . . . 247
Lab 5 — Teach-back . . . 247
Exercises . . . 247
Common pitfalls . . . 248
Checkpoint . . . 248
Further depth (this book) . . . 249

Derivations as an idea **250**

Derivations as an idea **251**
Why this chapter matters . . . 251
Theory 1 — Derivation = build recipe with a promised output path . . . 251
Vocabulary table . . . 252
Theory 2 — Eval vs realize (operational) . . . 252
`.drv` is not the package . . . 252
Pipeline sketch . . . 252
Theory 3 — `mkDerivation` mental model (lite) . . . 253
Other trivial helpers (awareness) . . . 253
Theory 4 — Sandbox: why builds are fenced . . . 253
Fixed-output derivations (FODs) . . . 254
Why FODs matter to operators . . . 254
Theory 5 — Substituters vs local build . . . 254
Theory 6 — Outputs and multiple outputs (preview) . . . 255
Theory 7 — What you inspect after a build . . . 255
Reading a log like an operator . . . 255
Theory 8 — Impure vs pure evaluation (awareness) . . . 256
Worked examples bank . . . 256
Example A — Build hello from nixpkgs . . . 256
Example B — Trivial package via flake . . . 256
Example C — Read derivation metadata . . . 256
Example D — Sandbox failure story (conceptual) . . . 256
Example E — `runCommand` sketch . . . 257
Example F — Wrong FOD hash . . . 257
Lab 1 — Workspace + flake for trivial builds . . . 257
Lab 2 — Inspect plan and log . . . 258
Lab 3 — Compare to nixpkgs hello closure . . . 258
Lab 4 — Break a FOD on purpose (optional) . . . 259
Lab 5 — Teach-back . . . 259
Exercises . . . 259
Common pitfalls . . . 260
Checkpoint . . . 260
Further depth (this book) . . . 261

Modern Nix CLI **262**

Modern Nix CLI **263**
Why this chapter matters . . . 263
Theory 1 — One CLI, many intents . . . 263
Theory 2 — Installables: how you *name* what you want . . . 264
`#` splits flake ref from attribute . . . 264
Ambiguity and defaults . . . 264
Theory 3 — `nix build` deep enough for daily use . . . 264
Theory 4 — `nix run` vs `nix shell` vs `nix develop` . . . 265
`nix run` . . . 265
`nix shell` . . . 265
`nix develop` . . . 265

Theory 5 — Flake subcommands . . . 266
`show` vs `metadata` . . . 266
JSON for scripting . . . 266
Theory 6 — Eval and debugging eval . . . 266
Theory 7 — Store queries you will reuse . . . 267
Theory 8 — Profiles without making them a lifestyle . . . 267
Theory 9 — Search and discovery . . . 268
Theory 10 — Error class → command checklist . . . 268
Worked examples bank . . . 268
Example A — Same package, three ways . . . 268
Example B — Flake introspection . . . 268
Example C — Logs streaming . . . 269
Example D — JSON for scripting . . . 269
Example E — Non-interactive develop . . . 269
Lab 1 — Cheat-sheet construction . . . 269
Lab 2 — Operate a local flake with modern CLI only . . . 270
Lab 3 — Ad-hoc `nix shell` workflow . . . 270
Lab 4 — Search + run loop . . . 270
Lab 5 — Intent mapping drill (closed book) . . . 271
Exercises . . . 271
Common pitfalls . . . 272
Checkpoint . . . 272
Further depth (this book) . . . 272

Flakes and project devShells **273**

Flakes and project devShells **274**
Why this chapter matters . . . 274
Theory 1 — What a flake is . . . 274
Evaluation model . . . 274
Baseline for this book . . . 275
Theory 2 — `inputs` . . . 275
Common URL forms . . . 275
Following transitive inputs . . . 275
Input types (awareness) . . . 275
Theory 3 — `outputs` . . . 276
Output schema (common) . . . 276
`self` . . . 276
Naming `default` . . . 276
Theory 4 — Systems and portability . . . 277
`genAttrs` pattern . . . 277
Theory 5 — `mkShell` / `mkShellNoCC` . . . 277
`mkShellNoCC` . . . 278
Theory 6 — Lock discipline (intro) . . . 278
Theory 7 — Flakes and git . . . 278
Dirty tree semantics . . . 278
Theory 8 — Registries (light awareness) . . . 279
Theory 9 — Apps vs packages (optional clarity) . . . 279

Theory 10 — What not to put in a first flake . . . 279
Worked examples bank . . . 280
Example A — Minimal full flake . . . 280
Example B — Enter and verify . . . 280
Example C — Metadata . . . 280
Example D — Non-interactive CI style . . . 280
Example E — Package + shell together . . . 281
Lab 1 — Create the project . . . 281
Lab 2 — Make it *your* project . . . 281
Lab 3 — `shellHook` and env . . . 282
Lab 4 — Offline-capable after populate (preview) . . . 282
Lab 5 — Teach-back . . . 282
Exercises . . . 282
Common pitfalls . . . 283
Checkpoint . . . 283
Further depth (this book) . . . 284

Classic CLI literacy **285**

Classic CLI literacy **286**
Why this chapter matters . . . 286
Theory 1 — Same store, two UIs . . . 286
Theory 2 — Command map (core) . . . 287
Intent-first map . . . 287
Theory 3 — Channels as history . . . 287
Channel model . . . 287
Flake model (preferred) . . . 288
When channels still appear . . . 288
Theory 4 — `shell.nix` / `default.nix` legacy layouts . . . 288
Classic project shell . . . 288
Modern . . . 289
Dual support pattern (optional) . . . 289
Theory 5 — `nix-env` and imperative profiles . . . 289
Theory 6 — Purity and flakes . . . 289
Theory 7 — Reading old error messages . . . 290
Theory 8 — GC and store tools (both eras) . . . 290
Theory 9 — Decision policy for this book . . . 290
Worked examples bank . . . 291
Example A — Build hello both ways . . . 291
Example B — Shell both ways . . . 291
Example C — Query store both ways . . . 291
Example D — Profile vs run . . . 291
Example E — Channel list observation . . . 292
Lab 1 — Build the translation table . . . 292
Lab 2 — Dual path for a project shell . . . 292
Lab 3 — Channel inspection (optional, disposable) . . . 293
Lab 4 — Doc archaeology . . . 293
Lab 5 — Commitment statement . . . 293

Exercises . . . 293
Common pitfalls . . . 294
Checkpoint . . . 294
Concepts part — exit criteria . . . 295
Further depth (this book) . . . 295

Nix on Linux **296**

Nix on Linux — Fedora bridge path **297**

Nix on Linux — Fedora bridge path **298**
Why Fedora 44 matters . . . 298
What you will learn here . . . 299
Suggested study path . . . 299
Lab workspace . . . 300
Exit gate (before NixOS host) . . . 300
Further depth (this book) . . . 301

Why Nix on Linux first **302**

Why Nix on Linux first **303**
Why this chapter matters . . . 303
Three faces again (bridge framing) . . . 304
What Nix is good at *on* an existing distro . . . 304
What you should still leave to Fedora . . . 304
Dual-stack anti-patterns . . . 305
Skills map: bridge → NixOS . . . 306
Mental model: evaluation vs realization (ops view) . . . 306
Worked decision: "Should this be Nix or dnf?" . . . 307
Lab . . . 307
Checkpoint . . . 307
Further depth . . . 308

Install Nix on Fedora 44 **309**

Install Nix on Fedora 44 **310**
Prerequisites . . . 310
Install modes (choose one) . . . 310
Multi-user install (Workstation / Server) . . . 311
1. Install packages . . . 311
2. Enable the daemon . . . 311
3. Groups and shell environment . . . 311
4. First smoke test . . . 312
Configure system vs user . . . 312
Daemon / system config . . . 312
User config . . . 313
Single-user mode (optional) . . . 313
Atomic / immutable desktops (Silverblue, Kinoite, bootc) . . . 314

Verify deeply (acceptance checklist) . . . 314
Upstream installer vs Fedora package . . . 315
Uninstall / reset notes . . . 315
Common failures . . . 316
Lab . . . 316
Checkpoint . . . 317
Further depth . . . 317

Store, profiles, and GC on Linux **318**

Store, profiles, and GC on Linux **319**
Why this chapter matters . . . 319
Store anatomy (quick ops view) . . . 319
Profiles: what they are for . . . 320
Inspect . . . 320
Install / remove (modern) . . . 320
Upgrade . . . 320
Other roots you will meet . . . 320
Garbage collection . . . 321
Safe everyday . . . 321
Delete old profile generations . . . 321
Disk pressure playbook . . . 322
Channels vs flakes (on Fedora) . . . 322
Binary caches . . . 322
Worked lab: profile vs shell . . . 323
Checkpoint . . . 323
Further depth . . . 323

Nix without /nix: chroot stores, –store, and why paths bake in /nix/store **324**

Nix without `/nix`: chroot stores, `--store`, and why paths bake in `/nix/store` **325**
Learning goals . . . 325
Thesis . . . 325
Mental model . . . 326
Case 1 — Can this host do it? . . . 327
Case 2 — A chroot store: physical layout, logical paths . . . 327
Case 3 — Running programs from a chroot store . . . 328
Case 4 — Builds in a chroot store are the same builds . . . 329
Case 5 — Relabelling the store: the expensive mistake . . . 330
Case 6 — The host's view: results, profiles and GC roots . . . 331
The trap . . . 332
The boring rule . . . 332
Try this . . . 333
Next . . . 333
Verification . . . 333
Sources . . . 334
Inspiration . . . 334

Daily CLI, flakes, and devShells **335**

Daily CLI, flakes, and devShells **336**
Modern CLI map (muscle memory) . . . 336
First project flake on Fedora . . . 336
`nix develop` vs `nix shell` vs profile . . . 338
direnv + nix-direnv (high leverage) . . . 338
Install tools (profile or HM later) . . . 338
direnvrc . . . 338
Shell hook . . . 338
Project . . . 339
Ergonomic extras (optional) . . . 339
Multi-system flakes (preview) . . . 339
CI-shaped local check . . . 340
Lab . . . 340
Checkpoint . . . 340
Next . . . 341
Further depth . . . 341

Self-contained scripts with #! nix-shell / #! nix **342**

Self-contained scripts with `#! nix-shell` / `#! nix` **343**
Learning goals . . . 343
Thesis . . . 343
Mental model . . . 345
Case 1 — What the kernel and `env` actually pass (run here) . . . 345
Case 2 — Classic form: `nix-shell -i` / `-p` / `--pure` / `-I` (not run here) . . . 347
Case 3 — Modern form: `#!/usr/bin/env nix` + `nix shell` (not run here) . . . 348
Case 4 — Pinning so "reproducible" means something . . . 349
Case 5 — When *not* to use a shebang . . . 350
Promoting a script to a Flake app (`pkgs.writers.writePython3Bin`) . . . 350
The trap . . . 351
The boring rule . . . 351
Try this . . . 352
Next . . . 352
Verification . . . 352
Sources . . . 352
Inspiration . . . 353

Home Manager on Fedora (standalone) **354**

Home Manager on Fedora (standalone) **355**
Why HM on Fedora . . . 355
Standalone bootstrap (flake-based) . . . 355
1. Minimal home flake skeleton . . . 355
2. `home.nix` . . . 356
3. First switch . . . 357
4. Verify . . . 357
What to put in HM vs project flakes . . . 357
Generations and rollback . . . 358

Secrets warning . . . 358
Coexistence with Fedora's bashrc . . . 358
Lab . . . 358
Checkpoint . . . 359
Further depth . . . 359

dnf and Nix coexistence **360**

dnf and Nix coexistence **361**
PATH reality . . . 361
Decision matrix (printable) . . . 362
Avoid dual service managers for the same app . . . 362
Language version managers vs Nix . . . 362
Graphics / Electron / browsers . . . 363
Disk layout awareness . . . 363
Updates: two clocks . . . 363
Lab . . . 363
Checkpoint . . . 364
Further depth . . . 364

SELinux, Atomic, and day-2 ops **365**

SELinux, Atomic, and day-2 ops **366**
SELinux: do not "fix" Nix by disabling it . . . 366
Atomic desktops and bootc . . . 366
Problem . . . 366
Pattern A — bind mount . . . 367
Pattern B — rootless / pure user Nix . . . 367
Pattern C — Nix in a toolbox/distrobox . . . 367
Day-2 operations checklist . . . 367
Daemon health . . . 367
Disk . . . 367
Trust and config . . . 368
Upgrade Nix itself (Fedora) . . . 368
Networking / proxies / corporate TLS . . . 368
Backup thoughts . . . 368
Performance tips (laptop) . . . 369
Lab . . . 369
Checkpoint . . . 369
Further depth . . . 370

Bridge to NixOS **371**

Bridge to NixOS **372**
What stays the same . . . 372
What is new . . . 372
Recommended transition shapes . . . 373
A — Keep Fedora laptop; NixOS server/lab (common) . . . 373
B — Dual boot / spare disk NixOS . . . 373

C — Full cutover . . . 373
Porting checklist (HM → NixOS) . . . 373
First NixOS learning loop (preview) . . . 374
Dual-stack monorepo sketch . . . 374
Skills you should already have (gate) . . . 375
Lab . . . 375
Checkpoint . . . 376
Further depth . . . 376

NixOS Host **377**

NixOS host — install, configure, modules **378**

NixOS host — install, configure, modules **379**
What this part is for . . . 379
Baseline (this book) . . . 379
Learning outcomes . . . 380
Prerequisites . . . 380
Chapter map . . . 381
Suggested learning paths . . . 382
Architecture sketch (part-level) . . . 382
Safety rules for this part . . . 383
Lab workspace convention . . . 383
Part gate (before Home / flake layout) . . . 383
Further depth (this book only) . . . 384
How to use this part as a reference . . . 384

Installing NixOS **385**

Installing NixOS **386**
Why this chapter matters . . . 386
Theory 1 — What you are installing . . . 386
Release identity . . . 387
Theory 2 — Disposable VM emphasis . . . 387
VM resources (practical) . . . 387
Networking modes (hypervisor) . . . 387
Theory 3 — UEFI vs legacy BIOS . . . 388
ESP requirements . . . 388
Theory 4 — Disk layout options . . . 388
Manual (classic installer path) . . . 388
Disko (intro only) . . . 388
Suggested simple layout (UEFI) . . . 389
Why not exotic FS on first install? . . . 389
Theory 5 — `nixos-generate-config` outputs . . . 389
What generate-config is good at . . . 389
Theory 6 — Installer networking and time . . . 390
Theory 7 — First boot mental model . . . 390
Activation vs “files on disk” . . . 390

Theory 8 — Credentials hygiene on a lab . . . 390
Theory 9 — `system.stateVersion` . . . 391
Theory 10 — Minimal install config knobs . . . 391
Worked examples bank . . . 391
Example A — Download image . . . 391
Example B — Partition sketch (sfdisk/parted-level ideas) . . . 392
Example C — Generate + install skeleton . . . 392
Example D — Minimal configuration knobs to set *during* install . . . 392
Example E — Mount checklist before generate-config . . . 393
Lab 1 — Plan and snapshot strategy . . . 393
Lab 2 — Install NixOS 26.05 . . . 393
Lab 3 — Post-install inventory . . . 394
Lab 4 — Preserve hardware config . . . 394
Lab 5 — Break-glass familiarity . . . 395
Exercises . . . 395
Common pitfalls . . . 395
Checkpoint . . . 396
Further depth (this book) . . . 396

Flake-defined hosts **397**

Flake-defined hosts **398**
Why this chapter matters . . . 398
Theory 1 — `nixosConfigurations` output . . . 398
Evaluation sketch . . . 399
Theory 2 — Where the flake lives . . . 399
Sudo and paths . . . 399
Theory 3 — Module list composition . . . 400
`imports` inside configuration.nix . . . 400
Theory 4 — Minimal flake host contents . . . 401
`flake.nix` . . . 401
`hosts/lab/configuration.nix` . . . 401
`hardware-configuration.nix` . . . 402
Theory 5 — First rebuild flow . . . 402
Safe preview builds . . . 402
Theory 6 — `specialArgs` / `extraSpecialArgs` (preview) . . . 402
Theory 7 — Hostname vs flake attribute . . . 403
Theory 8 — Git discipline for hosts . . . 403
Flake purity reminder . . . 403
Theory 9 — Multiple hosts (preview) . . . 403
Theory 10 — Leaving installer `/etc/nixos` behind . . . 404
Worked examples bank . . . 404
Example A — Build without switching (safe) . . . 404
Example B — Switch . . . 404
Example C — Show config value . . . 404
Example D — Show system path . . . 404
Example E — Trace eval errors . . . 404
Lab 1 — Repository layout on the lab host . . . 405

Lab 2 — First flake rebuild . . . 405
Lab 3 — Prove flake is authority . . . 405
Lab 4 — README for future you . . . 406
Lab 5 — Optional: match `/etc/nixos` to flake . . . 406
Exercises . . . 406
Common pitfalls . . . 407
Checkpoint . . . 407
Further depth (this book) . . . 408

Rebuild modes and generations **409**

Rebuild modes and generations **410**
Why this chapter matters . . . 410
Theory 1 — System profile and generations . . . 410
Inspect . . . 410
Theory 2 — Rebuild modes . . . 411
Choosing modes . . . 411
Remote note . . . 411
Theory 3 — Activation vs boot default . . . 411
Decision flowchart . . . 412
Theory 4 — Listing and switching generations . . . 412
Boot menu recovery . . . 412
Theory 5 — What is *not* rolled back . . . 413
Theory 6 — GC and generations . . . 413
Theory 7 — Flake + rebuild flags worth knowing . . . 414
Theory 8 — Activation scripts and failure modes . . . 414
Theory 9 — Naming generations in ops notes . . . 414
Worked examples bank . . . 415
Example A — Safe build . . . 415
Example B — test then switch . . . 415
Example C — rollback . . . 415
Example D — Compare generations (conceptual) . . . 415
Example E — Logs during rebuild . . . 415
Lab 1 — Inventory generations before chaos . . . 415
Lab 2 — Mode drills (harmless package) . . . 416
Lab 3 — Deliberate break → rollback . . . 416
Lab 4 — Boot menu literacy . . . 416
Lab 5 — Incident card . . . 417
Exercises . . . 417
Common pitfalls . . . 418
Checkpoint . . . 418
Further depth (this book) . . . 418

Users and access **419**

Users and access **420**
Why this chapter matters . . . 420

Theory 1 — Declarative users . . . 420
System users (preview) . . . 421
Theory 2 — Root vs wheel admin . . . 421
Order of hardening . . . 421
Theory 3 — Passwords and the store . . . 422
Why the store leaks matter . . . 422
Theory 4 — SSH service basics . . . 422
Host keys vs user keys . . . 422
Theory 5 — Authorized keys on NixOS . . . 423
Client config hygiene . . . 423
Theory 6 — Groups you will actually use . . . 423
Theory 7 — Home directories and mutability . . . 424
`mutableUsers` . . . 424
Theory 8 — Sudo and elevation patterns . . . 424
Theory 9 — Debugging SSH failures . . . 425
Theory 10 — Audit checklist for flakes . . . 425
Worked examples bank . . . 425
Example A — Admin user with key . . . 425
Example B — Disable password SSH . . . 426
Example C — Console password without storing plaintext . . . 426
Example D — System user for a service (preview) . . . 426
Example E — Key file in repo . . . 426
Lab 1 — Generate a lab keypair on the **client** . . . 426
Lab 2 — Wire user + SSH into flake . . . 427
Lab 3 — Verify identity paths . . . 427
Lab 4 — Lockout drill (console required) . . . 427
Lab 5 — Audit flake for secret smells . . . 428
Lab 6 — `users.mutableUsers` experiment (optional, snapshot) . . . 428
Lab 7 — SSH hard fail checklist . . . 428
Exercises . . . 429
Common pitfalls . . . 429
Checkpoint . . . 430
Further depth (this book) . . . 430

Networking and firewall **431**

Networking and firewall **432**
Why this chapter matters . . . 432
Theory 1 — Two common network stacks on NixOS . . . 432
Classic DHCP simplicity . . . 432
networkd file shape (server VM) . . . 433
Theory 2 — Host identity . . . 433
Extra hosts . . . 433
Theory 3 — Firewall model . . . 433
nftables awareness (26.05) . . . 434
Theory 4 — Interface names . . . 434
Theory 5 — Static addressing (optional lab) . . . 435
Theory 6 — DNS and resolution . . . 435

Theory 7 — Firewall vs service modules . . . 435
Theory 8 — Rebuild safety pattern for remote operators . . . 436
Lockout drill ethics . . . 436
Theory 9 — Port audit as continuous practice . . . 436
Theory 10 — Hypervisor port forwards . . . 436
Worked examples bank . . . 437
Example A — Server-like DHCP + SSH . . . 437
Example B — Explicit interface DHCP . . . 437
Example C — Temporary debug open (lab only) . . . 437
Example D — networkd DHCP match . . . 437
Example E — DNS pin . . . 437
Lab 1 — Discover ground truth . . . 437
Lab 2 — Declare firewall explicitly . . . 438
Lab 3 — Port audit . . . 438
Lab 4 — Controlled break (console ready) . . . 438
Lab 5 — Network README snippet . . . 439
Lab 6 — Firewall rule listener map . . . 439
Lab 7 — DNS failure drill (safe) . . . 439
Exercises . . . 440
Common pitfalls . . . 440
Checkpoint . . . 441
Further depth (this book) . . . 441

Packages and programs **442**

Packages and programs **443**
Why this chapter matters . . . 443
Theory 1 — `environment.systemPackages` . . . 443
Organization pattern . . . 444
Theory 2 — `programs.*` modules . . . 444
Discovery . . . 444
Theory 3 — Decision table . . . 445
Theory 4 — Closure discipline (light) . . . 445
Theory 5 — Overlays and pins (awareness only) . . . 445
Theory 6 — Unfree and allow flags . . . 446
Theory 7 — System vs user packages . . . 446
Theory 8 — `environment.defaultPackages` trap . . . 446
Theory 9 — Package name binary name . . . 447
Theory 10 — Layers interaction with generations . . . 447
Worked examples bank . . . 447
Example A — Baseline admin tools . . . 447
Example B — programs integration . . . 448
Example C — Search package attr . . . 448
Example D — Conditional unfree single package . . . 448
Example E — Avoid double install . . . 448
Lab 1 — Declare your daily CLI set . . . 448
Lab 2 — Convert one package to `programs.*` . . . 449
Lab 3 — Anti-profile audit . . . 449

Lab 4 — Size awareness . . . 449
Lab 5 — Split comments in config . . . 449
Lab 6 — `programs.*` option archaeology . . . 450
Lab 7 — Remove one redundant package . . . 450
Exercises . . . 450
Common pitfalls . . . 451
Checkpoint . . . 451
Further depth (this book) . . . 451

systemd via Nix **452**

systemd via Nix **453**
Why this day exists . . . 453
Theory 1 — Module options generate units . . . 453
Theory 2 — Service essentials . . . 453
Absolute paths matter . . . 454
Theory 3 — Timers . . . 454
Theory 4 — State directories and permissions . . . 455
Theory 5 — Inspecting generated units . . . 455
Theory 6 — Relationship to `services.*` high-level modules . . . 455
Theory 7 — Failure and rollback . . . 456
Worked examples bank . . . 456
Example A — oneshot + timer writing a log . . . 456
Example B — long-running simple service . . . 456
Example C — `path` for tools in script . . . 456
Lab 1 — Ship `lab-tick` timer . . . 457
Lab 2 — `systemctl cat` literacy . . . 457
Lab 3 — One long-running service (localhost) . . . 457
Lab 4 — Break ExecStart on purpose . . . 458
Lab 5 — Document units in README . . . 458
Theory 8 — Hardening starters for custom units . . . 458
Theory 9 — `systemd.tmpfiles` for paths units need . . . 459
Theory 10 — Activation vs live `systemctl edit` . . . 459
Worked example D — oneshot with `RemainAfterExit` . . . 459
Lab 6 — Timer calendar form . . . 459
Lab 7 — Unit dependency map . . . 460
Common gotchas . . . 460
Checkpoint . . . 460
Commit . . . 461

Modules as a consumer **462**

Modules as a consumer **463**
Why this day exists . . . 463
Theory 1 — A module is a function (usually) . . . 463
Theory 2 — Options vs config . . . 464
Theory 3 — Merge semantics (intuition) . . . 464
Theory 4 — `lib.mkIf` . . . 465

Theory 5 — `lib.mkDefault` and priority . . . 465
Theory 6 — How to read a nixpkgs module . . . 465
Step path . . . 465
Local source on a NixOS system . . . 466
What to extract mentally . . . 466
Theory 7 — Consumer anti-patterns . . . 466
Theory 8 — `imports` as composition . . . 467
Worked examples bank . . . 467
Example A — Read openssh options . . . 467
Example B — Override a default carefully . . . 467
Example C — When `mkForce` appears in blogs . . . 467
Lab 1 — Study three modules . . . 468
Lab 2 — Change behavior via options only . . . 468
Lab 3 — Trace an option to source . . . 469
Lab 4 — Priority experiment (safe) . . . 469
Lab 5 — Consumer checklist . . . 469
Common gotchas . . . 470
Checkpoint . . . 470
Commit . . . 470

Writing custom modules **471**

Writing custom modules **472**
Why this day exists . . . 472
Theory 1 — Minimal custom module shape . . . 472
Theory 2 — `mkOption` and types (starter set) . . . 473
Theory 3 — `cfg = config.…` pattern . . . 473
Theory 4 — Assertions . . . 474
Theory 5 — Import into the host . . . 474
Theory 6 — Namespace your options . . . 475
Theory 7 — What to extract first . . . 475
Worked examples bank . . . 475
Example A — Enable site . . . 475
Example B — Assertion trip . . . 475
Example C — Package override . . . 475
Lab 1 — Create module file . . . 476
Lab 2 — Add a real assertion . . . 476
Lab 3 — Extract something from your host config . . . 476
Lab 4 — Disable path . . . 477
Lab 5 — Document the module . . . 477
Theory 8 — `mkDefault` vs bare assignment (producer side) . . . 477
Theory 9 — Option descriptions are UX . . . 478
Theory 10 — Module args you receive . . . 478
Worked example D — warnings for soft policy . . . 478
Lab 6 — `nixos-option` on your option . . . 479
Lab 7 — Priority conflict drill . . . 479
Common gotchas . . . 479

Checkpoint . . . 479
Commit . . . 480

Boot and hardware **481**

Boot and hardware **482**
Why this day exists . . . 482
Theory 1 — Boot chain (UEFI lab) . . . 482
Theory 2 — systemd-boot vs GRUB . . . 482
Generation UI . . . 483
Theory 3 — Kernel packages . . . 483
Theory 4 — Firmware . . . 484
Theory 5 — CPU / virt hardware bits . . . 484
Theory 6 — `hardware-configuration.nix` discipline . . . 484
Theory 7 — Temporary boot parameters . . . 485
Worked examples bank . . . 485
Example A — Document current loader . . . 485
Example B — Kernel identity . . . 485
Example C — Safe param experiment . . . 485
Lab 1 — Boot path document . . . 485
Lab 2 — Verify loader config matches reality . . . 486
Lab 3 — Kernel package awareness . . . 486
Lab 4 — Firmware decision . . . 486
Lab 5 — Recovery dry-run (no panic) . . . 487
Theory 8 — `boot.loader.timeout` and generation menu . . . 487
Theory 9 — Initrd and LUKS awareness (preview) . . . 487
Theory 10 — `hardware-configuration.nix` vs flake purity . . . 488
Worked example D — QEMU guest agent (virt) . . . 488
Lab 6 — Loader entries inventory . . . 488
Lab 7 — Kernel param one-shot (snapshot) . . . 488
Common gotchas . . . 489
Checkpoint . . . 489
Commit . . . 489

Filesystems and state **490**

Filesystems and state **491**
Why this day exists . . . 491
Theory 1 — Three layers of “the disk” . . . 491
Theory 2 — `fileSystems` options . . . 491
Theory 3 — Why UUID/LABEL . . . 492
Theory 4 — State directories that matter . . . 492
Theory 5 — Impermanence idea (concept only) . . . 493
Why people want it . . . 493
Costs . . . 493
Theory 6 — Store is not a backup of state . . . 494
Theory 7 — tmpfs and ephemeral mounts (awareness) . . . 494

Worked examples bank . . . 494
Example A — Mount inventory . . . 494
Example B — What's big? . . . 494
Example C — SSH host key paths . . . 494
Lab 1 — Full inventory table . . . 495
Lab 2 — Persist classification . . . 495
Lab 3 — State created by your Stage II work . . . 495
Lab 4 — Impermanence reading (light) . . . 496
Lab 5 — Danger checklist . . . 496
Theory 8 — Btrfs / ZFS awareness without deploying them . . . 496
Theory 9 — `/nix` on its own mount (optional design) . . . 496
Theory 10 — Bind mounts as proto-impermanence . . . 497
Worked example D — `neededForBoot` meaning . . . 497
Lab 6 — What survives `nixos-rebuild switch`? . . . 498
Lab 7 — Size budget sentence . . . 498
Common gotchas . . . 498
Checkpoint . . . 498
Commit . . . 499

Activation Reload/Restart Deprecation **500**

Activation Reload/Restart Deprecation **501**
Mental model . . . 501
Worked examples . . . 502
Case 1: Find legacy restart/reload side channels on the desk . . . 502
Case 2: Restart when a config file changes — use `restartTriggers` . . . 503
Case 3: Prefer reload when only `ExecReload=` changed . . . 504
Case 4: Replace "mkdir + chown" activation with tmpfiles . . . 505
Case 5: Prep that belongs to one service → `ExecStartPre=` . . . 506
Case 6: Work that must re-run on every switch → oneshot + `sysinit-reactivation.target` 506
The trap . . . 507
The boring rule . . . 508
Try this . . . 509

Clevis LUKS Askpass in Initrd **510**

Clevis LUKS Askpass in Initrd **511**
Mental model . . . 511
Worked examples . . . 512
Case 1: Bind Clevis to a LUKS volume (prerequisite, once) . . . 512
Case 2: Enable askpass on a Tang-backed desk . . . 513
Case 3: TPM2-only (no Tang, no initrd network) . . . 514
Case 4: Eval-time assertion and Tang networking warning . . . 514
Case 5: Contrast — older JWE-file `boot.initrd.clevis` . . . 515
Case 6: 26.05 LUKS / Stage 1 operator notes (passphrase timeout) . . . 515
The trap . . . 516
The boring rule . . . 517
Try this . . . 517

Specialisations **518**

Specialisations **519**
Mental model . . . 519
When to use vs containers / second hosts . . . 520
Worked examples . . . 520
Case 1: GPU-off by default, GPU-on specialisation . . . 520
Case 2: Independent specialisation (`inheritParentConfig = false`) . . . 521
Case 3: Parent-only config (not pulled into children) . . . 521
Case 4: Inspect built specialisations . . . 522
Activation cheat sheet . . . 522
The trap . . . 522
Try this . . . 523
Sources . . . 523

Modular Services **524**

Modular Services **525**
Mental model . . . 525
Worked examples . . . 526
Case 1: A portable service module . . . 526
Case 2: Two instances on one host . . . 528
Case 3: Read the units NixOS generated . . . 529
Case 4: A package-exported service as a sub-service . . . 530
Case 5: systemd specifiers and variables (`mainExecStart`, 26.05+) . . . 532
The trap . . . 532
Try this . . . 534
Sources . . . 535

systemd in Stage 1: the 26.05 Initrd Default **536**

systemd in Stage 1: the 26.05 Initrd Default **537**
Mental model . . . 537
Worked examples . . . 537
Case 1: Confirm what you are running . . . 538
Case 2: An old hook fails evaluation (the good failure) . . . 538
Case 3: Rewrite the hook as an initrd unit . . . 539
Case 4: The trap evaluation will not catch: `/dev/root` . . . 540
Case 5: The escape hatch, and its warning . . . 541
The trap . . . 541
The boring rule . . . 541
Try this . . . 541
Sources . . . 542

Locales: the Locale Archive, i18n.extraLocales, and the Missing-Locale Crash **543**

Locales: the Locale Archive, `i18n.extraLocales`, and the Missing-Locale Crash **544**
Mental model . . . 544

Worked examples . . . 545
Case 1: What a default host ships . . . 546
Case 2: Regional formats the supported way . . . 547
Case 3: Asking for a locale glibc does not ship (the good failure) . . . 548
Case 4: The runtime failure: the archive lacks what the environment asks for . . . 549
Case 5: Building a site-local locale with the stock builder . . . 550
The trap . . . 553
The boring rule . . . 554
Try this . . . 555
Sources . . . 555

system.stateVersion: What It Changes, and How to Bump It Without Losing Data **556**

`system.stateVersion`: What It Changes, and How to Bump It Without Losing Data **557**
Mental model . . . 558
Worked examples . . . 558
Case 1: How many modules care? . . . 559
Case 2: The same configuration under five stateVersions . . . 559
Case 3: `stateVersion` left unset . . . 561
Case 4: The floor: stateVersions older than the oldest supported default . . . 562
Case 5: What a bump from 23.11 to 26.05 would actually change . . . 562
Case 6: The safe bump: pin first, then diff again . . . 564
Case 7: A `stateVersion` per module . . . 565
The trap . . . 566
The boring rule . . . 566
Try this . . . 566
Sources . . . 567

/boot Is Full: How systemd-boot Fills the ESP, and How to Empty It **568**

`/boot` Is Full: How systemd-boot Fills the ESP, and How to Empty It **569**
Mental model . . . 569
The code that decides . . . 570
Lab: the real builder against a 96 MiB ESP . . . 571
The trap: garbage collection that forgets the ESP . . . 573
The boring rule . . . 573
Try this (not run here) . . . 573
Sources . . . 574

Home and Flake Layout **575**

Home Manager and flake layout **576**

Home Manager and flake layout **577**
What this part is for . . . 577
Prerequisites . . . 578
Chapter map . . . 578
Mental model — three planes . . . 579

Dependency direction (import DAG) . . . 579
Suggested repo tree (target shape) . . . 580
How to work this part . . . 580
Cross-cutting conventions (26.05) . . . 581
What this part does *not* cover . . . 581
Skills checklist (end of part) . . . 581
How chapters are structured . . . 582
Reading strategies . . . 582
Lab host assumptions . . . 582
Next part . . . 583

Flake repository layout **584**

Flake repository layout **585**
Why this chapter exists . . . 585
Theory 1 — What "layout" means in a flake monorepo . . . 585
Target tree . . . 586
Theory 2 — Dependency direction (prevent circular imports) . . . 587
What belongs where . . . 587
Theory 3 — Host entry pattern . . . 588
`default.nix` vs explicit path . . . 589
`specialArgs` vs module arguments . . . 589
Theory 4 — Module "barrel" files . . . 589
Optional deeper split . . . 590
Theory 5 — Hardware config and purity . . . 590
Theory 6 — Roles vs common vs services . . . 590
Theory 7 — Multi-host readiness (without building the second machine) . . . 591
Theory 8 — Relative paths and flake root . . . 591
Worked example — Minimal SSH + users + nix split . . . 592
Worked example — Thin flake with optional `lib` . . . 593
Exercises . . . 594
Exercise 1 — Inventory the monolith . . . 594
Exercise 2 — Create the directory skeleton . . . 594
Exercise 3 — Split common modules . . . 594
Exercise 4 — Host entry + thin flake . . . 594
Exercise 5 — Build before switch . . . 594
Exercise 6 — Document the tree . . . 595
Exercise 7 — Role extract (optional stretch) . . . 595
Exercise 8 — Path-break drill . . . 595
Exercise 9 — Diff against backup . . . 595
Exercise 10 — Second-host skeleton (docs only) . . . 595
Exercise 11 — Secrets flag list . . . 595
Exercise 12 — Commit hygiene . . . 596
Common gotchas . . . 596
Checkpoint . . . 596
Journal (optional) . . . 597

Lock file discipline **598**

Lock file discipline **599**
Why this chapter exists . . . 599
Theory 1 — What `flake.lock` actually pins . . . 599
Inputs vs follow-ons . . . 600
Anatomy of a lock node (conceptual) . . . 600
Theory 2 — Update operations (precision toolkit) . . . 600
Mental model . . . 601
Theory 3 — How to read a lock diff . . . 601
Nested duplicates . . . 602
Theory 4 — Pin strategies (pick a house rule) . . . 602
Security vs stability . . . 602
Theory 5 — Registries (light literacy) . . . 603
Registry vs input (summary) . . . 603
Theory 6 — Multi-machine and "reproducible" . . . 603
Clone checklist . . . 603
Theory 7 — Commit policy for locks . . . 604
Revert pattern . . . 604
Theory 8 — Inputs that should co-move . . . 604
Worked example — Selective update narrative . . . 605
Worked commands . . . 605
Exercises . . . 605
Exercise 1 — Snapshot current pins . . . 605
Exercise 2 — Read the lock structure . . . 606
Exercise 3 — Selective update of one input . . . 606
Exercise 4 — Prove "update all" is louder . . . 606
Exercise 5 — Document house rules . . . 607
Exercise 6 — Registry awareness (read-only) . . . 607
Exercise 7 — follows fix (if needed) . . . 607
Exercise 8 — Compare URL construction . . . 607
Exercise 9 — Impurity awareness list . . . 607
Exercise 10 — CI anti-pattern note . . . 608
Exercise 11 — Paired restore drill . . . 608
Exercise 12 — Multi-input update plan . . . 608
Common gotchas . . . 608
Checkpoint . . . 609
Journal (optional) . . . 609

Home Manager on NixOS **610**

Home Manager on NixOS **611**
Why this chapter exists . . . 611
Theory 1 — Three ways people run Home Manager . . . 611
Why module mode first . . . 612
Theory 2 — Input wiring (26.05-aligned) . . . 612
Where to put the HM block . . . 613
Theory 3 — System user vs HM user . . . 613
`home.stateVersion` . . . 614
Name and path must match . . . 614

Theory 4 — What belongs in HM vs NixOS . . . 614
Theory 5 — Activation model (what rebuild does) . . . 615
Standalone commands (awareness) . . . 615
Theory 6 — Programs modules vs raw packages . . . 616
Shell source of truth . . . 616
Theory 7 — Splitting the home tree . . . 616
Theory 8 — Backup and collision with existing dots . . . 617
Worked example — Minimal useful user . . . 617
Worked example — HM wiring module (multi-host friendly) . . . 618
Exercises . . . 619
Exercise 1 — Add the input and lock . . . 619
Exercise 2 — Stub → real home module . . . 619
Exercise 3 — Rebuild and verify . . . 619
Exercise 4 — Change only HM, rebuild once . . . 619
Exercise 5 — Deliberate failure drill . . . 620
Exercise 6 — Journal: boundary decisions . . . 620
Exercise 7 — Activation log literacy . . . 620
Exercise 8 — Split programs.nix . . . 620
Exercise 9 — Package placement check . . . 620
Exercise 10 — Shell source of truth . . . 620
Exercise 11 — Backup existing dots . . . 620
Exercise 12 — Version alignment proof . . . 621
Exercise 13 — Negative case: standalone not required . . . 621
Common gotchas . . . 621
Checkpoint . . . 621
Journal (optional) . . . 622

Home files and XDG **623**

Home files and XDG **624**
Why this chapter exists . . . 624
Theory 1 — How HM places files . . . 624
Escape hatches (know, use sparingly) . . . 624
Theory 2 — `home.file` vs `xdg.configFile` . . . 625
Minimal examples . . . 625
Theory 3 — Content sources: `text`, `source`, templates . . . 626
Recursive directories . . . 626
Theory 4 — XDG directories . . . 626
Classic vs XDG . . . 627
Theory 5 — Permissions and clobber behavior . . . 627
Mode options . . . 628
Theory 6 — Theming (optional, bounded) . . . 628
Theory 7 — Multi-machine user splits . . . 628
Theory 8 — `programs.*` vs raw files for the same tool . . . 629
Theory 9 — What never goes in managed files . . . 629
Worked example — Migrate git ignores + editor config . . . 630
Worked example — Structured home tree . . . 630

Exercises . . . 631
Exercise 1 — Inventory live dotfiles . . . 631
Exercise 2 — First managed XDG file . . . 631
Exercise 3 — Migrate via `source` . . . 631
Exercise 4 — Conflict drill . . . 631
Exercise 5 — Secret anti-pattern (fake only) . . . 631
Exercise 6 — Structure the home tree . . . 632
Exercise 7 — Recursive config tree . . . 632
Exercise 8 — Executable home file . . . 632
Exercise 9 — programs vs file for one tool . . . 632
Exercise 10 — Multi-host plan . . . 632
Exercise 11 — Cache vs config . . . 632
Exercise 12 — Activation after migrate . . . 633
Exercise 13 — Documentation . . . 633
Common gotchas . . . 633
Checkpoint . . . 633
Journal (optional) . . . 634

Shell and Editor Configuration **635**

Shell and Editor Configuration **636**
Why Shell and Editor Declarations Matter . . . 636
Declarative Shell Environments . . . 636
Modern Shell Declaration (Bash and Fish) . . . 637
Universal Prompt Engineering with Starship . . . 638
Terminal Multiplexers: Tmux and Zellij . . . 639
Declarative Editor Mastery: Neovim and Helix . . . 640
Declarative Neovim with Pinned Plugins and Built-in LSPs . . . 640
Minimal Modal Editing: Helix . . . 642
Visual Studio Code Declarative Management . . . 643
Common Traps and State Separation . . . 644
Checkpoint . . . 644

Home Manager user services **645**

Home Manager user services **646**
Why this chapter exists . . . 646
Theory 1 — Systemd user vs system . . . 646
Lingering . . . 646
Theory 2 — HM service options . . . 647
HM attrset shape . . . 647
Theory 3 — Mapping system knowledge to user bus . . . 648
Theory 4 — `Simple` vs `oneshot` vs long-running . . . 648
Theory 5 — Paths and packages in units . . . 649
WorkingDirectory and paths . . . 649
Theory 6 — Prefer HM modules when they exist . . . 649
Theory 7 — Targets, wants, and session lifecycle . . . 650
Theory 8 — Debugging workflow . . . 650

Worked example — Notes fetcher timer . . . 650
Worked example — Calendar timer variants . . . 651
Exercises . . . 651
Exercise 1 — User systemd baseline . . . 651
Exercise 2 — Heartbeat oneshot + timer . . . 652
Exercise 3 — Journal debugging . . . 652
Exercise 4 — One service you actually want . . . 652
Exercise 5 — Boundary check . . . 652
Exercise 6 — Import services.nix . . . 652
Exercise 7 — list-unit-files literacy . . . 653
Exercise 8 — Failure restart policy . . . 653
Exercise 9 — OnCalendar experiment . . . 653
Exercise 10 — Wrong bus drill . . . 653
Exercise 11 — Secret audit of unit text . . . 653
Exercise 12 — Graphical vs headless note . . . 653
Exercise 13 — Comparison table in docs . . . 653
Exercise 14 — Clean disable . . . 653
Common gotchas . . . 654
Checkpoint . . . 654
Journal (optional) . . . 654

direnv and project shells **655**

direnv and project shells **656**
Why this chapter exists . . . 656
Theory 1 — `devShells` in a flake . . . 656
Multi-system sketch . . . 657
Why not always put shells in the NixOS flake? . . . 657
Theory 2 — direnv + nix-direnv . . . 658
HM installation pattern . . . 658
Verify hook . . . 658
Theory 3 — `.envrc` with `use flake` . . . 658
Named shells . . . 659
`.envrc` is not for secrets . . . 659
Theory 4 — System packages vs project shells . . . 659
Theory 5 — Pinning discipline for project flakes . . . 660
Theory 6 — `nix develop` vs `nix shell` vs HM packages . . . 660
`inputsFrom` (awareness) . . . 660
Theory 7 — shellHook hygiene . . . 660
Theory 8 — Host flake devShell for Nix ops . . . 661
Worked example — Multi-tool Python shell . . . 661
Worked example — Isolation proof layout . . . 662
Exercises . . . 662
Exercise 1 — Install direnv via HM . . . 662
Exercise 2 — Create a project flake . . . 662
Exercise 3 — Wire direnv . . . 662
Exercise 4 — Prove version isolation . . . 663
Exercise 5 — Thin the profile (careful) . . . 663

Exercise 6 — Document the workflow . . . 663
Exercise 7 — Commit project lock . . . 663
Exercise 8 — Reload after lock bump . . . 663
Exercise 9 — Quiet shellHook . . . 663
Exercise 10 — Host ops shell . . . 663
Exercise 11 — Ad-hoc vs project . . . 664
Exercise 12 — gitignore hygiene . . . 664
Exercise 13 — Failure: allow missing . . . 664
Exercise 14 — Boundary table . . . 664
Common gotchas . . . 664
Checkpoint . . . 665
Journal (optional) . . . 665

Multi-Language Development Environments **666**

Multi-Language Development Environments **667**
The Polyglot Monorepo Reality . . . 667
The Anatomy of a Polyglot `devShell` . . . 667
Production Multi-Language Flake . . . 668
Overcoming Native Library Traps . . . 671
1. The `pkg-config` Resolution Trap . . . 671
2. The `patchelf` and `LD_LIBRARY_PATH` Dilemma . . . 671
Automated Activation with `direnv` . . . 671
Checkpoint . . . 672

Migrating from Docker Desktop to Nix DevShells **673**

Migrating from Docker Desktop to Nix DevShells **674**
The Docker Desktop Trap . . . 674
1. The Virtual Machine Overhead . . . 674
2. Filesystem Latency and Bind Mounts . . . 674
3. File Watcher Failures . . . 674
4. Commercial Licensing and Corporate bloat . . . 675
Architectural Comparison: Containers vs Nix DevShells . . . 675
Step-by-Step Migration: Dockerfile to `flake.nix` . . . 676
The Original Dockerfile . . . 676
The Translated `flake.nix` . . . 677
Bridging External Services: Native Shell + Lightweight OCI . . . 678
Minimal `compose.yml` for Backing Services Only . . . 678
Starting the Environment . . . 679
Performance Benchmark: Inner-Loop Compilation . . . 679
Checkpoint . . . 679

flake-parts (optional structure) **680**

flake-parts (optional structure) **681**
Why this chapter exists . . . 681
Theory 1 — Pain that motivates structure . . . 681

Theory 2 — Hand-rolled discipline (no flake-parts) . . . 682
House rules for hand-rolled . . . 683
Theory 3 — flake-parts approach . . . 683
Costs . . . 684
Split flake-parts modules (optional) . . . 684
Theory 4 — Decision rubric . . . 685
Theory 5 — What not to do . . . 685
Theory 6 — Formatter as a style signal . . . 685
Theory 7 — Overlay injection point . . . 686
Worked example — Split hand-rolled helpers . . . 686
Worked example — Decision record template . . . 686
Exercises . . . 687
Exercise 1 — Audit current outputs . . . 687
Exercise 2 — Pick and write the decision . . . 687
Exercise 3A — If flake-parts: adopt on a branch . . . 687
Exercise 3B — If hand-rolled: extract helpers . . . 687
Exercise 4 — Merge the winner . . . 688
Exercise 5 — Teach the style to "future you" . . . 688
Exercise 6 — Formatter end-to-end . . . 688
Exercise 7 — specialArgs survival . . . 688
Exercise 8 — Show vs build . . . 688
Exercise 9 — follows still holds . . . 688
Exercise 10 — Negative hybrid cleanup . . . 688
Exercise 11 — Systems honesty . . . 689
Exercise 12 — Style PR description . . . 689
Common gotchas . . . 689
Checkpoint . . . 689
Journal (optional) . . . 690

Flake checks **691**

Flake checks **692**
Why this chapter exists . . . 692
Theory 1 — What `checks` are . . . 692
Theory 2 — Hand-rolled checks skeleton . . . 693
Host toplevel (strong, heavy) . . . 693
Lighter host check — eval-oriented . . . 693
Theory 3 — flake-parts checks . . . 694
Theory 4 — Format check that actually fails . . . 694
Alternative: treefmt (awareness) . . . 694
Theory 5 — What belongs in check vs push-time CI only . . . 694
Theory 6 — `nix flake check` UX . . . 695
Common failure modes . . . 695
Theory 7 — Checks and secrets . . . 695
Theory 8 — Wiring consistency with pkgs/overlays . . . 696
Worked example — Practical minimal set . . . 696
Worked example — Script wrapper . . . 696

Exercises . . . 697
Exercise 1 — Baseline without checks . . . 697
Exercise 2 — Add `devShell` as a check . . . 697
Exercise 3 — Add a format check that can fail . . . 697
Exercise 4 — Optional host toplevel check . . . 697
Exercise 5 — Developer loop script (optional) . . . 697
Exercise 6 — Journal: CI shape . . . 697
Exercise 7 — Single-attr debug . . . 698
Exercise 8 — cleanSource impact . . . 698
Exercise 9 — System gate . . . 698
Exercise 10 — Pre-commit habit . . . 698
Exercise 11 — Document check inventory . . . 698
Exercise 12 — Secret scan of check context . . . 698
Exercise 13 — Fail-closed demo . . . 698
Exercise 14 — Overlay consistency . . . 699
Common gotchas . . . 699
Checkpoint . . . 699
Journal (optional) . . . 700

Overlays (light introduction) 701

Overlays (light introduction) 702
Why this chapter exists . . . 702
Theory 1 — What an overlay is . . . 702
Fixed-point intuition . . . 703
Theory 2 — Wiring overlays on NixOS . . . 703
Flake-level pkgs for shells/checks . . . 703
Theory 3 — `overrideAttrs` vs `override` vs new package . . . 704
Universal patching vs. forks . . . 704
Theory 4 — Composition order . . . 705
Theory 5 — When not to overlay . . . 705
Theory 6 — Pinning a package from another nixpkgs (light) . . . 706
Theory 7 — Overlays vs `packageOverrides` (classic) . . . 706
Theory 8 — Infinite recursion patterns . . . 706
Worked example — Shared overlay module . . . 707
Worked example — HM sees the overlay . . . 707
Exercises . . . 708
Exercise 1 — Create overlay file . . . 708
Exercise 2 — Attach to NixOS . . . 708
Exercise 3 — Attach to devShell / checks pkgs . . . 708
Exercise 4 — Prove HM sees it (if useGlobalPkgs) . . . 708
Exercise 5 — Intentional override discipline . . . 708
Exercise 6 — Negative space . . . 709
Exercise 7 — Composition list . . . 709
Exercise 8 — Failure: overlay not wired . . . 709
Exercise 9 — Document overlays . . . 709
Exercise 10 — Check integration . . . 709
Exercise 11 — Prefer wrapper over override . . . 709

Exercise 12 — Search before overlay . . . 709
Exercise 13 — Recursion caution note . . . 709
Exercise 14 — Upgrade readiness . . . 709
Common gotchas . . . 710
Checkpoint . . . 710
Journal (optional) . . . 710
Part close-out . . . 710

Determinate Nix: Wasm in Expressions and Flake Schemas **712**

Determinate Nix: Wasm in Expressions and Flake Schemas **713**
Mental model . . . 713
Worked examples . . . 714
Case 1: Enable and probe `builtins.wasm` . . . 714
Case 2: YAML without IFD (sketch) . . . 715
Case 3: A minimal custom flake schema . . . 715
Case 4: Roles — teach `nix build` about a custom output . . . 716
Case 5: Exporting schemas for other flakes . . . 716
The trap . . . 716
The boring rule . . . 717
Try this . . . 717
Sources . . . 717

A Multi-Arch Flake Template that Stays Boring **718**

A Multi-Arch Flake Template that Stays Boring **719**
Thesis . . . 719
Mental model . . . 719
Worked examples . . . 719
Trap . . . 719
Boring rule . . . 720
Try this . . . 720
Sources . . . 720

Services and Security **721**

Services and security patterns **722**

Services and security patterns **723**
What this part is for . . . 723
Prerequisites . . . 724
Chapter map . . . 724
Mental model — service slice . . . 724
Suggested module layout (extends previous part) . . . 725
How to work this part . . . 725
Cross-cutting conventions (26.05) . . . 726
Security non-goals (this part) . . . 726
Skills checklist (end of part) . . . 727

Reading strategies . . . 727
Relationship to other parts . . . 727
Lab host assumptions . . . 728
Next parts . . . 728

The secrets problem **729**

The secrets problem **730**
Why this chapter exists . . . 730
Theory 1 — The store is not a vault . . . 730
Evaluation vs runtime . . . 731
Theory 2 — Common anti-patterns catalog . . . 731
`builtins.readFile` trap . . . 732
Theory 3 — What "good" looks like (preview) . . . 732
Theory 4 — Generations and GC . . . 732
Theory 5 — Multi-user threat on a "lab laptop" . . . 733
Theory 6 — Distinguishing config from secret . . . 733
Theory 7 — Process listings and logs . . . 734
Theory 8 — Git history is forever (enough) . . . 734
Worked example — Finding secrets with search . . . 734
Worked example — Safe store readability demo . . . 735
Exercises . . . 735
Exercise 1 — Threat model paragraph . . . 735
Exercise 2 — Audit the flake for anti-patterns . . . 735
Exercise 3 — Prove store readability (safe demo) . . . 735
Exercise 4 — User password hygiene check . . . 735
Exercise 5 — Remove anti-patterns and rebuild . . . 736
Exercise 6 — Incident checklist card . . . 736
Exercise 7 — HM file audit . . . 736
Exercise 8 — Environment variable audit . . . 736
Exercise 9 — Generation awareness . . . 736
Exercise 10 — Backup story note . . . 736
Exercise 11 — Public vs private key check . . . 737
Exercise 12 — Document TODO list for sops . . . 737
Exercise 13 — Peer review simulation . . . 737
Exercise 14 — CI log risk . . . 737
Common gotchas . . . 737
Checkpoint . . . 738
Journal (optional) . . . 738

sops-nix **739**

sops-nix **740**
Why this chapter exists . . . 740
Theory 1 — Components . . . 740
Flow . . . 741
Theory 2 — Age keys (host-centric pattern) . . . 741
Key backup . . . 741

Theory 3 — Flake input . . . 742
Theory 4 — Minimal NixOS sops config . . . 743
Theory 5 — Creating encrypted files . . . 743
Adding a secret later . . . 744
Theory 6 — HM secrets (awareness) . . . 744
Theory 7 — Bootstrapping chicken-and-egg . . . 744
SSH host key method (awareness) . . . 745
Theory 8 — systemd integration patterns . . . 745
Theory 9 — Multi-host recipients . . . 745
Worked example — Secret for a custom script service . . . 746
Exercises . . . 746
Exercise 1 — Add input and module . . . 746
Exercise 2 — Age keys and `.sops.yaml` . . . 746
Exercise 3 — First encrypted secret file . . . 747
Exercise 4 — Declare `sops.secrets` and rebuild . . . 747
Exercise 5 — Runtime consumer . . . 747
Exercise 6 — Failure drill . . . 747
Exercise 7 — Docs . . . 747
Exercise 8 — Selective lock update . . . 748
Exercise 9 — Permission check . . . 748
Exercise 10 — Second secret . . . 748
Exercise 11 — No readFile audit . . . 748
Exercise 12 — Journal hygiene . . . 748
Exercise 13 — Recipient add rehearsal (docs) . . . 748
Exercise 14 — Bootstrap checklist in git . . . 748
Common gotchas . . . 748
Checkpoint . . . 749
Journal (optional) . . . 749

agenix (comparison) 750

agenix (comparison) 751
Why this chapter exists . . . 751
Theory 1 — agenix mental model . . . 751
Flow . . . 752
Theory 2 — Flake input sketch . . . 752
Theory 3 — secrets.nix shape . . . 752
Identity reuse . . . 753
Theory 4 — NixOS declaration . . . 753
Theory 5 — Side-by-side comparison . . . 753
When agenix might still win for you . . . 754
When sops-nix wins for this curriculum . . . 754
Theory 6 — Dual-running risks . . . 754
Theory 7 — Declaring house default . . . 754
Theory 8 — Re-encryption and rotation . . . 755
Theory 9 — Path API discipline (both tools) . . . 755
Worked example — Same placeholder secret both ways . . . 755

Exercises . . . 756
Exercise 1 — Add agenix input . . . 756
Exercise 2 — Public keys and secrets.nix . . . 756
Exercise 3 — Encrypt one age secret . . . 756
Exercise 4 — NixOS module + rebuild . . . 756
Exercise 5 — Consumer parity . . . 756
Exercise 6 — Comparison write-up . . . 757
Exercise 7 — Decommission the non-default . . . 757
Exercise 8 — docs/secrets.md house default section . . . 757
Exercise 9 — follows check . . . 757
Exercise 10 — Incident note . . . 757
Exercise 11 — Team rationale . . . 757
Exercise 12 — Path-only audit . . . 757
Exercise 13 — Rebuild with single stack . . . 758
Exercise 14 — Future secret list . . . 758
Common gotchas . . . 758
Checkpoint . . . 758
Journal (optional) . . . 759

Choosing between sops-nix and agenix **760**

Choosing between sops-nix and agenix **761**
Thesis . . . 761
Mental model . . . 761
Worked examples . . . 761
Trap . . . 761
Boring rule . . . 761
Try this . . . 762
Sources . . . 762

HashiCorp Vault Integration **763**

HashiCorp Vault Integration **764**
The Enterprise Secret Boundary . . . 764
Vault Architecture on NixOS . . . 765
Declarative Vault Agent Configuration . . . 765
Consuming Dynamic Secrets in systemd Services . . . 767
Pattern A: `EnvironmentFile` with File ACLs . . . 767
Pattern B: Modern systemd `LoadCredential` . . . 768
Architectural Comparison: SOPS vs Agenix vs Vault . . . 768
Best Practice: The Hybrid Vault + SOPS Pattern . . . 769
Checkpoint . . . 769

Host hardening patterns **770**

Host hardening patterns **771**
Why this chapter exists . . . 771
Theory 1 — Threat model for this lab . . . 771
Role split . . . 772

Theory 2 — SSH baseline (NixOS) . . . 772
Optional allowlists . . . 772
Theory 3 — sudo and wheel . . . 773
Theory 4 — Disable what you do not use . . . 773
Theory 5 — Kernel / sysctl care . . . 774
Theory 6 — Automatic updates? (judgment) . . . 774
Theory 7 — Module shape . . . 774
Theory 8 — Assertions as guardrails . . . 775
Theory 9 — Safe apply order . . . 775
Worked example — Recovery card . . . 775
Exercises . . . 776
Exercise 1 — Inventory surface . . . 776
Exercise 2 — Write hardening module . . . 776
Exercise 3 — Apply carefully . . . 776
Exercise 4 — Lockout recovery drill (document) . . . 776
Exercise 5 — Assertions (optional polish) . . . 776
Exercise 6 — Diff and explain . . . 776
Exercise 7 — Key-before-password-off proof . . . 776
Exercise 8 — wheel policy document . . . 777
Exercise 9 — Role wiring . . . 777
Exercise 10 — PermitRootLogin check . . . 777
Exercise 11 — Disable list rationale . . . 777
Exercise 12 — Sysctl minimalism . . . 777
Exercise 13 — Workstation exception note . . . 777
Exercise 14 — Generation rollback rehearsal (console available) . . . 777
Common gotchas . . . 778
Checkpoint . . . 778
Journal (optional) . . . 778

Firewall discipline **779**

Firewall discipline **780**
Why this chapter exists . . . 780
Theory 1 — NixOS firewall model . . . 780
Prefer service options . . . 780
Theory 2 — Deny by default . . . 781
Theory 3 — Per-service matrix (living document) . . . 781
Theory 4 — Localhost vs LAN vs world . . . 781
Theory 5 — Temporary debug holes . . . 782
Theory 6 — Interaction with containers (preview) . . . 782
Theory 7 — Interfaces and trusted nets . . . 782
Cloud security groups . . . 783
Theory 8 — Connection tracking . . . 783
Theory 9 — Co-locate opens with services . . . 783
Worked example — Tight SSH + HTTPS owned by proxy . . . 783
Worked example — Service openFirewall pattern . . . 784
Exercises . . . 784
Exercise 1 — Audit current opens . . . 784

Exercise 2 — Enable firewall explicitly . . . 784
Exercise 3 — Remove unjustified ports . . . 784
Exercise 4 — Service-tied open (stub) . . . 784
Exercise 5 — Localhost service experiment . . . 785
Exercise 6 — ports.md committed . . . 785
Exercise 7 — Diff firewall after change . . . 785
Exercise 8 — Ping policy decision . . . 785
Exercise 9 — SSH tunnel habit . . . 785
Exercise 10 — Cloud SG note . . . 785
Exercise 11 — Orphan port hunt . . . 785
Exercise 12 — UDP inventory . . . 786
Exercise 13 — test generation safety . . . 786
Exercise 14 — Pre-proxy checklist . . . 786
Common gotchas . . . 786
Checkpoint . . . 786
Journal (optional) . . . 787

TLS and reverse-proxy front door **788**

TLS and reverse-proxy front door **789**
Why this chapter exists . . . 789
Theory 1 — Choose one proxy . . . 789
Theory 2 — ACME on NixOS . . . 790
Secrets . . . 790
Theory 3 — Caddy pattern (NixOS) . . . 790
Theory 4 — Nginx + ACME pattern . . . 791
Theory 5 — Backend for the lab . . . 791
Theory 6 — Failure modes . . . 792
Theory 7 — Certificate storage and renewal . . . 792
Theory 8 — HTTP → HTTPS redirect . . . 793
Theory 9 — Internal TLS without public ACME . . . 793
Worked example — ports.md dual-write . . . 793
Exercises . . . 793
Exercise 1 — Pick proxy and document . . . 793
Exercise 2 — Localhost backend . . . 794
Exercise 3 — Enable proxy module . . . 794
Exercise 4 — TLS path . . . 794
Exercise 5 — Headers and hygiene (light) . . . 794
Exercise 6 — Break/fix . . . 794
Exercise 7 — ports.md + proxy.md dual-write . . . 794
Exercise 8 — Certificate expiry awareness . . . 794
Exercise 9 — ss audit . . . 795
Exercise 10 — ACME journal . . . 795
Exercise 11 — Staging note . . . 795
Exercise 12 — WebSocket flag (if needed) . . . 795
Exercise 13 — Single proxy enforcement . . . 795
Exercise 14 — Secret path for DNS-01 (plan) . . . 795
Exercise 15 — Smoke from remote . . . 795

Common gotchas . . . 796
Checkpoint . . . 796
Journal (optional) . . . 796

A data service pattern **797**

A data service pattern **798**
Why this chapter exists . . . 798
Theory 1 — One DB only (pattern depth) . . . 798
Theory 2 — Minimal PostgreSQL module . . . 799
Theory 3 — State directories and disk . . . 799
Theory 4 — Credentials . . . 800
Theory 5 — App connection story . . . 800
Theory 6 — Backup thought (required cognitive work) . . . 800
Theory 7 — Upgrades . . . 801
Theory 8 — `ensureDatabases` vs migrations . . . 801
Theory 9 — Socket vs TCP local connections . . . 801
Worked example — Service + docs . . . 802
Worked example — Timer sketch for dumps (optional) . . . 802
Exercises . . . 802
Exercise 1 — Enable PostgreSQL . . . 802
Exercise 2 — Database and role . . . 803
Exercise 3 — Wire a secret password path . . . 803
Exercise 4 — Network posture . . . 803
Exercise 5 — Manual dump . . . 803
Exercise 6 — Failure narrative . . . 803
Exercise 7 — Schema independence proof . . . 803
Exercise 8 — Document restore dry-run . . . 804
Exercise 9 — dataDir documentation . . . 804
Exercise 10 — Peer auth admin path . . . 804
Exercise 11 — Package pin note . . . 804
Exercise 12 — Backup location permissions . . . 804
Exercise 13 — App connection string template . . . 804
Exercise 14 — ports.md final for DB . . . 804
Common gotchas . . . 805
Checkpoint . . . 805
Journal (optional) . . . 805

NixOS containers **806**

NixOS containers **807**
Why this chapter exists . . . 807
Theory 1 — When containers help on NixOS . . . 807
Decision micro-script . . . 808
Theory 2 — `containers.<name>` (NixOS nspawn) . . . 808
Pros / cons . . . 809
Theory 3 — OCI awareness (without next-chapter depth) . . . 809

Theory 4 — Security boundaries (honest) . . . 809
Hardening checklist (guest) . . . 810
Theory 5 — State and containers . . . 810
Theory 6 — Relation to proxy / firewall . . . 810
Theory 7 — Rebuild UX and nested eval . . . 811
Theory 8 — Feature flag for demos . . . 811
Worked example — Prefer native, demo container optional . . . 811
Exercises . . . 811
Exercise 1 — Decision record . . . 811
Exercise 2 — Explore host tools . . . 812
Exercise 3 — One declarative container **or** dry config . . . 812
Exercise 4 — Reach the guest service . . . 812
Exercise 5 — Compare rebuild UX . . . 812
Exercise 6 — When you would use OCI instead . . . 812
Exercise 7 — Firewall / ports doc . . . 813
Exercise 8 — Nested stateVersion . . . 813
Exercise 9 — Journal path . . . 813
Exercise 10 — Bind mount caution note . . . 813
Exercise 11 — Disable cleanly . . . 813
Exercise 12 — Eval time measurement . . . 813
Exercise 13 — Diagram . . . 813
Exercise 14 — Threat model honesty . . . 813
Common gotchas . . . 813
Checkpoint . . . 814
Journal (optional) . . . 814

Podman and Docker patterns **815**

Podman and Docker patterns **816**
Why this chapter exists . . . 816
Theory 1 — Podman vs Docker on NixOS . . . 816
Theory 2 — Rootless vs rootful . . . 817
Theory 3 — Declarative containers (OCI) . . . 817
Theory 4 — Images and supply chain (awareness) . . . 818
Theory 5 — Compare to nspawn . . . 818
Theory 6 — Firewall and proxy integration . . . 819
Theory 7 — Stateful volumes . . . 819
Theory 8 — Rootless paths and user namespaces . . . 819
Theory 9 — Imperative snowflakes vs flake . . . 820
Worked example — Full small stack slice . . . 820
Exercises . . . 820
Exercise 1 — Enable Podman (or Docker) . . . 820
Exercise 2 — Declarative OCI container . . . 820
Exercise 3 — Imperative contrast (then undo) . . . 821
Exercise 4 — Wire to proxy (optional but valuable) . . . 821
Exercise 5 — Secret anti-pattern check . . . 821
Exercise 6 — Comparison table filled . . . 821
Exercise 7 — Volume inventory . . . 821

Exercise 8 — Update story one-pager . . . 821
Exercise 9 — autoStart reboot proof . . . 821
Exercise 10 — Port conflict check . . . 821
Exercise 11 — docker group risk note . . . 822
Exercise 12 — Image pin upgrade . . . 822
Exercise 13 — Disable cleanly . . . 822
Exercise 14 — Isolation.md final . . . 822
Common gotchas . . . 822
Checkpoint . . . 822
Journal (optional) . . . 823

Observability patterns **824**

Observability patterns **825**
Why this chapter exists . . . 825
Theory 1 — Three pillars (light) . . . 825
Theory 2 — Logs-first pattern (valid) . . . 825
Theory 3 — Prometheus pattern . . . 826
Theory 4 — Grafana (optional layer) . . . 827
Theory 5 — What to scrape for *your* slice . . . 827
Theory 6 — Firewall and multi-tenant risk . . . 828
Theory 7 — Alerting (awareness only) . . . 828
Theory 8 — Retention and disk . . . 828
Theory 9 — Authn in front of Grafana . . . 828
Worked example — Minimal metrics module set . . . 829
Worked example — Runbook skeleton . . . 829
Exercises . . . 829
Exercise 1 — Choose path . . . 829
Exercise 2A — Metrics path . . . 829
Exercise 2B — Logs-first path . . . 830
Exercise 3 — One service deep dive . . . 830
Exercise 4 — Access path . . . 830
Exercise 5 — Update ports and isolation docs . . . 830
Exercise 6 — Capstone foreshadow . . . 830
Exercise 7 — One dashboard or one journal recipe . . . 830
Exercise 8 — Disk use snapshot . . . 831
Exercise 9 — PromQL basics (metrics path) . . . 831
Exercise 10 — Password hygiene for Grafana . . . 831
Exercise 11 — Unit list for the slice . . . 831
Exercise 12 — Error priority journal . . . 831
Exercise 13 — Tunnel muscle memory . . . 831
Exercise 14 — Docs cross-link . . . 831
Common gotchas . . . 832
Checkpoint . . . 832
Journal (optional) . . . 832

k3s (optional awareness) **833**

k3s (optional awareness) **834**
Why this chapter exists . . . 834
Theory 1 — Decision rubric . . . 834
Theory 2 — What k3s is . . . 835
Theory 3 — NixOS vs k8s configuration . . . 835
Theory 4 — Single-node honesty . . . 836
Theory 5 — Minimal literacy tasks (if enabled) . . . 836
Theory 6 — Skip path is first-class . . . 836
Theory 7 — When NixOS modules beat k8s on a single node . . . 837
Theory 8 — Resource floor honesty . . . 837
Theory 9 — Feature flag module . . . 837
Worked example — Disable / enable toggles . . . 838
Worked example — Tiny workload (if enabled) . . . 838
Exercises . . . 839
Exercise S — Skip path (most readers) . . . 839
Exercise S2 — Skip artifact quality bar . . . 839
Exercise K1 — Opt-in: enable k3s . . . 839
Exercise K2 — Tiny workload . . . 839
Exercise K3 — Tear-down plan . . . 839
Exercise K4 — Compare control planes . . . 840
Exercise K5 — API surface inventory (if enabled) . . . 840
Exercise 6 — Resource estimate . . . 840
Exercise 7 — Secrets boundary note . . . 840
Exercise 8 — Proxy interaction . . . 840
Exercise 9 — Decision commit . . . 840
Exercise 10 — FOMO check . . . 840
Exercise 11 — Revisit calendar . . . 840
Exercise 12 — ports.md consistency . . . 840
Exercise 13 — Disable half-install . . . 841
Exercise 14 — Part retrospective . . . 841
Common gotchas . . . 841
Checkpoint . . . 841
If skipped . . . 841
If enabled . . . 841
Journal (optional) . . . 842
Part close-out . . . 842

nixos-vex: configuration-aware VEX for vulnerability triage **843**

nixos-vex: configuration-aware VEX for vulnerability triage **844**
Learning goals . . . 844
Thesis . . . 844
Mental model . . . 845
Case 1 — Why SBOMs alone drown ops . . . 845
Case 2 — Minimal conditional VEX from a NixOS module . . . 846
Case 3 — Using nixos-vex on a flake host . . . 847
The trap . . . 848
The boring rule . . . 848

Try this . . . 848
Sources . . . 849
Inspiration . . . 849
Verification . . . 849

Vulnerability Inventory across Profiles and Images **850**

Vulnerability Inventory across Profiles and Images **851**
Thesis . . . 851
Mental model . . . 851
Worked example . . . 851
Trap . . . 852
Boring rule . . . 852
Try this . . . 852
Sources . . . 852

Packaging 853

Packaging craft **854**

Packaging craft **855**
What this part is for . . . 855
Prerequisites . . . 856
Chapter map . . . 856
Mental model — three layers of "a package" . . . 857
stdenv is the spine . . . 857
FODs vs normal builds . . . 857
Caches vs builders (preview) . . . 858
Lab workspace conventions . . . 858
How chapters are structured . . . 859
Reading paths . . . 859
Path A — "I need one internal CLI in nixpkgs style" . . . 859
Path B — "CI is compiling the world" . . . 860
Path C — "We forked half of nixpkgs" . . . 860
Glossary (part-local) . . . 860
Quality bar for "done packaging" . . . 861
Relationship to other parts . . . 861
Anti-patterns to avoid while studying . . . 861
Part checkpoint (before you leave packaging) . . . 862
Suggested lab commit series . . . 862
How to use this part as reference . . . 863
Next . . . 863

stdenv and build phases **864**

stdenv and build phases **865**
Why it matters . . . 865

Theory 1 — What `stdenv.mkDerivation` actually is . . . 865
`buildInputs` vs `nativeBuildInputs` (cross-aware) . . . 866
Evaluation vs realization . . . 866
Theory 2 — Default phase order . . . 866
Phase responsibilities . . . 867
Hooks API (muscle memory) . . . 867
Theory 3 — Important environment variables in the sandbox . . . 868
Flags that skip work . . . 868
Theory 4 — Minimal patterns you will reuse forever . . . 869
Shell-only "package" (no compile) . . . 869
Autotools / simple C (upstream tarball) . . . 869
Single C file without autotools . . . 870
Flat single-file source (unpack quirks) . . . 870
Theory 5 — `finalAttrs` vs `rec` . . . 871
Theory 6 — What fixup actually does (short list) . . . 871
Worked example — flake that exports packages . . . 872
Host consumption (optional) . . . 873
Worked example — intentional phase map document . . . 873
Lab — multi-step . . . 874
Step 1 — Scaffold . . . 874
Step 2 — Write three packages . . . 874
Step 3 — Break phases on purpose . . . 875
Step 4 — Flip input kinds . . . 875
Step 5 — Phase introspection . . . 875
Step 6 — Drop `runHook` once . . . 875
Step 7 — Parallel build flag . . . 875
Step 8 — Host or shell consumption . . . 876
Exercises . . . 876
Common gotchas . . . 876
Checkpoint . . . 877
Commit . . . 877

Fetchers and fixed-output derivations **878**

Fetchers and fixed-output derivations **879**
Why it matters . . . 879
Theory 1 — Fixed-output vs normal derivations . . . 879
Why the store path is special . . . 880
Theory 2 — Common fetchers (2026 mental map) . . . 880
`fetchFromGitHub` pattern . . . 880
Submodules and deep clones . . . 881
Theory 3 — The hash mismatch workflow . . . 881
Correct response . . . 881
Prefetch helpers . . . 882
SRI vs base32 . . . 882
Theory 4 — Impure traps and purity . . . 882
Theory 5 — Flake inputs vs package fetchers . . . 883
Theory 6 — `outputHashMode` (awareness) . . . 884

Theory 7 — Private sources and credentials . . . 884
Worked examples . . . 885
Example A — tarball package skeleton . . . 885
Example B — GitHub + patch FOD . . . 885
Example C — intentional mismatch . . . 886
Example D — flake wiring . . . 886
Lab — multi-step . . . 886
Step 1 — Prefetch a known tarball . . . 886
Step 2 — Package from that tarball . . . 887
Step 3 — Force a mismatch . . . 887
Step 4 — GitHub fetcher . . . 887
Step 5 — Patch as FOD and as path . . . 887
Step 6 — Document the trust boundary . . . 887
Step 7 — Offline check . . . 888
Step 8 — Submodules experiment (optional) . . . 888
Exercises . . . 888
Common gotchas . . . 888
Checkpoint . . . 889
Commit . . . 889

Language ecosystems (part A) 890

Language ecosystems (part A) 891
Why it matters . . . 891
Theory 1 — How language builders relate to stdenv . . . 891
Shared skeleton . . . 892
Theory 2 — Go path (`buildGoModule`) . . . 892
Local module (lab-friendly) . . . 893
`vendorHash` workflow . . . 893
Useful knobs . . . 893
Example `src/hello-go` . . . 894
Theory 3 — Rust path (`buildRustPackage`) . . . 894
Git dependencies in Cargo . . . 895
Useful knobs . . . 895
Theory 4 — Python path (application vs library) . . . 895
Standalone scripts with `pkgs.writers.writePython3Bin` . . . 896
pytest pattern (sketch) . . . 898
Theory 5 — Node (`buildNpmPackage`) sketch . . . 898
Theory 6 — Version injection and `meta.mainProgram` . . . 898
Theory 7 — Tests and purity . . . 899
Worked example — flake with Go tool + check . . . 899
Lab — multi-step . . . 900
Step 1 — Choose primary language . . . 900
Step 2 — Minimal upstream project . . . 900
Step 3 — First package expression . . . 901
Step 4 — Wire `meta.mainProgram` and run . . . 901
Step 5 — Version injection . . . 901
Step 6 — Tests . . . 901

Step 7 — Consume from a devShell . . . 901
Step 8 — Break the vendor/cargo/npm hash . . . 901
Step 9 — Closure glance . . . 902
Exercises . . . 902
Common gotchas . . . 902
Checkpoint . . . 903
Commit . . . 903

Language ecosystems (part B) 904

Language ecosystems (part B) 905
Why it matters . . . 905
Theory 1 — Pattern map across ecosystems . . . 905
Theory 2 — Choose B deliberately . . . 906
Transfer questions (answer in notes) . . . 906
Theory 3 — Wrapping and runtime deps . . . 906
High-level script writers: `pkgs.writers` . . . 907
Theory 4 — Overrides preview . . . 908
Theory 5 — Cross-ecosystem FOD discipline . . . 909
Theory 6 — Runtime vs build-time closure . . . 910
Theory 7 — Dual interface pattern . . . 910
Worked example — dual packages in one flake . . . 911
Tiny Python app (if A was compiled) . . . 911
Tiny Node CLI (alternative) . . . 912
Tiny Go (if A was Python) . . . 913
Lab — multi-step . . . 913
Step 1 — Import or copy the part A package . . . 913
Step 2 — Scaffold smaller B project . . . 913
Step 3 — Package with the B builder . . . 913
Step 4 — Comparison table (required) . . . 913
Step 5 — Unified check . . . 914
Step 6 — Optional same interface . . . 914
Step 7 — Closure glance . . . 914
Step 8 — Hash bump drill . . . 914
Step 9 — Light override . . . 914
Exercises . . . 915
Common gotchas . . . 915
Checkpoint . . . 915
Commit . . . 916

Patching and overrides 917

Patching and overrides 918
Why it matters . . . 918
Theory 1 — Three layers of change . . . 918
Theory 2 — `overrideAttrs` . . . 919
`override` vs `overrideAttrs` . . . 919
Composing safely . . . 919

Theory 3 — Patches as first-class inputs . . . 920
Generating a patch . . . 920
Theory 4 — Overlay composition . . . 920
Order matters . . . 921
Infinite recursion patterns . . . 921
NixOS / HM consumption . . . 921
Theory 5 — Pinning and "don't rebuild the world" . . . 922
Theory 6 — Scoped injection without global overlays . . . 922
Theory 7 — Language-specific override helpers . . . 923
Worked examples . . . 923
Example A — local patch on `hello` . . . 923
Example B — overlay adding your ecosystem A package . . . 923
Example C — `packageOverrides` legacy note . . . 924
Example D — flake export . . . 924
Lab — multi-step . . . 924
Step 1 — Baseline . . . 924
Step 2 — Write a patch . . . 924
Step 3 — `overrideAttrs` only (no overlay) . . . 925
Step 4 — Promote to overlay . . . 925
Step 5 — Stack two overlays . . . 925
Step 6 — Dependency override . . . 925
Step 7 — Maintenance note . . . 925
Step 8 — Intentional drop of upstream patches . . . 925
Exercises . . . 926
Common gotchas . . . 926
Checkpoint . . . 926
Commit . . . 927

Multiple outputs **928**

Multiple outputs **929**
Why it matters . . . 929
Theory 1 — Outputs are separate store paths . . . 929
First output is special . . . 930
Theory 2 — Conventional output roles . . . 930
Environment variables in the build . . . 930
Theory 3 — Why dependents get smaller . . . 931
Reference leaks . . . 931
Theory 4 — `moveToOutput` and helpers . . . 931
Theory 5 — Consuming multi-output packages . . . 932
Multiple outputs and `nix shell` . . . 932
Theory 6 — `separateDebugInfo` . . . 932
Theory 7 — When single-output is fine . . . 933
Worked example — toy multi-output C library + app . . . 933
Lab — multi-step . . . 935
Step 1 — Inspect an existing multi-out package . . . 935
Step 2 — Build libgreet with three outputs . . . 935
Step 3 — Verify separate paths . . . 936

Step 4 — Build greetapp against libgreet . . . 936
Step 5 — Deliberate mistake . . . 936
Step 6 — Document policy . . . 936
Step 7 — Optional `separateDebugInfo` . . . 936
Step 8 — pkg-config break/fix . . . 936
Exercises . . . 937
Common gotchas . . . 937
Checkpoint . . . 937
Commit . . . 938

Debugging builds **939**

Debugging builds **940**
Why it matters . . . 940
Theory 1 — Where information lives . . . 940
Theory 2 — Eval failure vs build failure . . . 940
Theory 3 — Keep failed trees . . . 941
Theory 4 — `breakpointHook` . . . 941
Theory 5 — Common failure classes . . . 942
Reading a log quickly . . . 942
Theory 6 — Sandbox: don't disable casually . . . 942
Theory 7 — Reproducing CI failures locally . . . 943
Bisect mental model . . . 943
Theory 8 — Save the drv path . . . 944
Worked example — scripted break/fix series . . . 944
Failure dossier template . . . 945
Worked example — personal runbook skeleton . . . 945
Lab — multi-step . . . 945
Step 1 — Baseline success package . . . 945
Step 2 — Inject a compile failure . . . 946
Step 3 — `--keep-failed` . . . 946
Step 4 — Inject an install failure . . . 946
Step 5 — FOD failure recall . . . 946
Step 6 — `breakpointHook` (Linux recommended) . . . 946
Step 7 — Write a personal runbook . . . 946
Step 8 — Force local rebuild . . . 946
Step 9 — Complete failure dossier . . . 946
Step 10 — Sandbox proof note . . . 947
Exercises . . . 947
Common gotchas . . . 947
Checkpoint . . . 948
Commit . . . 948

Navigating nixpkgs **949**

Navigating nixpkgs **950**
Why it matters . . . 950

Theory 1 — Mental map of nixpkgs (2026) . . . 950
by-name rules of thumb . . . 950
Theory 2 — Anatomy of a PR-shaped package.nix . . . 951
Modern habits . . . 952
Theory 3 — callPackage and fixed-points . . . 952
Why function heads matter . . . 952
Theory 4 — Reading existing packages as literature . . . 953
What to look for . . . 953
Theory 5 — Tests and passthru (awareness) . . . 953
Theory 6 — Category tree vs by-name coexistence . . . 954
Theory 7 — Contribution hygiene (even if you never PR) . . . 954
Worked example — flake package laid out like by-name . . . 955
Lab — multi-step . . . 955
Step 1 — Locate packages . . . 956
Step 2 — Reshape your best packaging package . . . 956
Step 3 — Overlay + flake export . . . 956
Step 4 — Meta completeness pass . . . 956
Step 5 — PR-shaped checklist . . . 956
Step 6 — Optional: clone nixpkgs and dry-place the file . . . 956
Step 7 — Compare category tree vs by-name . . . 957
Step 8 — Literature notes . . . 957
Exercises . . . 957
Common gotchas . . . 957
Checkpoint . . . 958
Commit . . . 958

Binary caches **959**

Binary caches **960**
Why it matters . . . 960
Theory 1 — Substituters vs builders . . . 960
Theory 2 — Trust model (non-negotiable) . . . 961
Theory 3 — Cachix (hosted, popular) . . . 961
Push full runtime closure . . . 962
Theory 4 — Self-hosted: Attic & Harmonia . . . 962
Harmonia sketch (NixOS) . . . 962
Attic sketch (conceptual) . . . 962
Theory 5 — Push what matters . . . 963
Theory 6 — Substituter order and priority . . . 963
Theory 7 — What “cache miss” really means . . . 964
Force local build (contrast) . . . 964
Theory 8 — Daemon vs user config . . . 964
Worked examples . . . 965
NixOS module fragment for team cache . . . 965
Local file NAR cache (always available lab path) . . . 965
Query whether a path would substitute . . . 965
Lab — multi-step . . . 965
Step 1 — Observe default cache . . . 965

Step 2 — Build your package cold-ish . . . 966
Step 3 — Choose push target . . . 966
Step 4 — Second machine / second user story . . . 966
Step 5 — Trust write-up . . . 966
Step 6 — CI note . . . 966
Step 7 — Negative test . . . 966
Step 8 — Document lab cache policy . . . 967
Exercises . . . 967
Common gotchas . . . 967
Checkpoint . . . 968
Commit . . . 968

Remote builders **969**

Remote builders **970**
Why it matters . . . 970
Theory 1 — Cache vs remote builder . . . 970
Theory 2 — Configuration surfaces (2026) . . . 971
NixOS (`nix.buildMachines`) . . . 971
Ad-hoc one-shot . . . 971
Features . . . 971
Theory 3 — SSH and store talk . . . 972
Theory 4 — Cross and multi-system . . . 972
Theory 5 — Security notes . . . 973
Decision card — do I need a builder? . . . 973
Theory 6 — Scheduling knobs that matter . . . 973
Upload storms . . . 974
Theory 7 — Builder host minimal config . . . 974
Topology diagram (keep in notes) . . . 974
Worked example — two-VM lab checklist . . . 974
Prove which machine built . . . 975
Lab — multi-step . . . 975
Step 1 — Inventory local capacity . . . 975
Step 2 — Provision builder . . . 975
Step 3 — SSH from daemon-relevant user . . . 976
Step 4 — One-shot remote build . . . 976
Step 5 — Persist config . . . 976
Step 6 — Feature flag experiment . . . 976
Step 7 — Combine with cache (optional) . . . 976
Step 8 — Write topology diagram . . . 976
Step 9 — Failure injection . . . 976
Step 10 — Disk pressure note . . . 977
Exercises . . . 977
Common gotchas . . . 977
Checkpoint . . . 978
Commit . . . 978

Reproducibility (honest view) **979**

Reproducibility (honest view) **980**
Why it matters . . . 980
Theory 1 — Strength ladder . . . 980
How to talk about this in a runbook . . . 981
Theory 2 — Input-addressed sameness . . . 981
Path sameness vs byte sameness . . . 981
Theory 3 — Why bit-for-bit fails . . . 981
Fixed-output derivations (FODs) . . . 982
Theory 4 — Tools for comparison . . . 982
Theory 5 — Operational reproducibility vs academic . . . 983
Theory 6 — What Nix does *not* make reproducible . . . 983
Theory 7 — Cache vs reproducibility . . . 983
Worked example — measurement script . . . 984
Worked example — deliberate nondeterminism . . . 984
Worked example — REPRO.md template . . . 985
Two-machine log template . . . 985
Lab — multi-step . . . 985
Step 1 — Environment record . . . 985
Step 2 — Pick three artifacts . . . 986
Step 3 — L1 measurements . . . 986
Step 4 — L2 if you have two environments . . . 986
Step 5 — Substitute false control . . . 986
Step 6 — Deliberate nondeterminism (lab only) . . . 986
Step 7 — Closure drift story . . . 986
Step 8 — Honesty statement . . . 986
Step 9 — Optional diffoscope . . . 987
Step 10 — Cache trust paragraph . . . 987
Step 11 — README claim scrub . . . 987
Exercises . . . 987
Common gotchas . . . 987
Checkpoint . . . 988
Commit . . . 988

ROS 2 robotics with nix-ros-overlay **989**

ROS 2 robotics with nix-ros-overlay **990**
Why this chapter exists . . . 990
Mental model . . . 991
Status and expectations (be honest) . . . 991
Binary caches (do this first) . . . 992
Official community cache (Cachix) . . . 992
Experimental Attic cache (when Cachix is thin) . . . 992
Fast path: try upstream examples (no project yet) . . . 993
Flake develop (recommended) . . . 993
Classic nix-shell form . . . 993
Project flake: ROS 2 underlay in a devShell . . . 994
Minimal `flake.nix` . . . 994
direnv . . . 996

Choosing a ROS 2 distro attr . . . 996
Workspace workflow (overlay underlay + your packages) . . . 997
Why not put *everything* in Nix immediately? . . . 997
Custom ROS packages (direction of travel) . . . 997
Graphics: RViz, Gz, tools on non-NixOS . . . 998
NixOS host integration (what belongs where) . . . 998
Fedora dual-stack note . . . 999
Troubleshooting . . . 999
Security and supply chain . . . 1000
Lab A — Hello underlay . . . 1000
Lab B — Project flake . . . 1000
Lab C — Distro compare (optional) . . . 1001
Lab D — Robot appliance sketch (optional) . . . 1001
Checkpoint . . . 1001
Further depth (this book) . . . 1001
Upstream . . . 1002

Running a Nixpkgs team **1003**

Running a Nixpkgs team **1004**
Thesis . . . 1004
Mental model . . . 1004
Worked examples . . . 1005
Case 1: Draft the charter as a Nixpkgs issue first . . . 1005
Case 2: Create the GitHub team and let the tree sync . . . 1005
Case 3: Make review load visible . . . 1006
Case 4: What "success" looked like for Tweag . . . 1006
Case 5: Maintainer metadata shape (sketch) . . . 1006
The trap . . . 1006
The boring rule . . . 1007
Try this . . . 1007
Sources . . . 1007

Fixing Packages with Overlays **1008**

Fixing Packages with Overlays **1009**
Thesis . . . 1009
Mental model . . . 1009
Worked example . . . 1009
Trap . . . 1010
Boring rule . . . 1010
Try this . . . 1010
Sources . . . 1010

Cross-Compilation for Multiple Platforms **1011**

Cross-Compilation for Multiple Platforms **1012**
The Cross-Compilation Problem: Why Emulation Fails . . . 1012
The Nixpkgs Platform Model: Triples and System Types . . . 1012

Instant Cross-Toolchains with `pkgsCross` . . . 1013
Classifying Inputs: `nativeBuildInputs` vs `buildInputs` . . . 1014
Language Ecosystems in Cross-Compilation . . . 1014
1. Cross-Compiling C/C++ Projects (CMake) . . . 1014
2. Cross-Compiling Rust with Cargo . . . 1015
3. Cross-Compiling Go Modules . . . 1016
Flake Pattern for Multi-Architecture Outputs . . . 1016
Common Cross-Compilation Pitfalls . . . 1017
Checkpoint . . . 1018

Ops and Fleet **1019**

Operations and fleet **1020**

Operations and fleet **1021**
What this part is for . . . 1021
Prerequisites . . . 1022
Chapter map . . . 1022
Mental model — fleet pipeline . . . 1023
Disk and identity . . . 1023
Deploy tools at a glance . . . 1023
Lab topology suggestions . . . 1024
How chapters are structured . . . 1025
Reading paths . . . 1025
Path A — "Homelab single server → clean reinstall" . . . 1025
Path B — "Three nodes with roles" . . . 1025
Path C — "Cloud image factory" . . . 1026
Glossary (part-local) . . . 1026
Safety rules (non-negotiable) . . . 1026
Quality bar for "fleet ready" . . . 1027
Relationship to other parts . . . 1027
Anti-patterns . . . 1027
Part checkpoint (before you leave ops) . . . 1028
Suggested lab commit series . . . 1028
How to use this part as reference . . . 1029
Next . . . 1029

Disko **1030**

Disko **1031**
Why it matters . . . 1031
Theory 1 — What Disko is . . . 1031
Theory 2 — Minimal GPT + ext4 root (UEFI) . . . 1032
Prefer stable device paths . . . 1033
Theory 3 — Flake integration . . . 1033
Evaluate mounts without wiping . . . 1034
Theory 4 — LUKS sketch (common homelab) . . . 1034

Theory 5 — btrfs subvolumes (impermanence-ready) . . . 1035
ZFS note . . . 1035
Theory 6 — Host-specific devices without copy-paste hell . . . 1036
Theory 7 — Swap, BIOS, and multi-disk . . . 1036
Swap . . . 1036
Legacy BIOS / hybrid . . . 1037
Multi-disk . . . 1037
Theory 8 — Interaction with hardware-configuration.nix . . . 1037
Worked example — VM lab checklist . . . 1037
DISK.md template . . . 1038
Lab — multi-step . . . 1038
Step 1 — Snapshot safety . . . 1038
Step 2 — Add Disko input . . . 1038
Step 3 — Write simplest UEFI layout . . . 1038
Step 4 — Dry evaluation . . . 1038
Step 5 — Apply on VM . . . 1039
Step 6 — Reinstall proof (core) . . . 1039
Step 7 — Optional LUKS or btrfs . . . 1039
Step 8 — Document recovery . . . 1039
Step 9 — Wrong-disk tabletop . . . 1039
Exercises . . . 1039
Common gotchas . . . 1039
Checkpoint . . . 1040
Commit . . . 1040

Impermanence (idea and patterns) 1041

Impermanence (idea and patterns) 1042
Why it matters . . . 1042
Theory 1 — Model . . . 1042
What “disappears” means . . . 1043
Theory 2 — `impermanence` module (nix-community) . . . 1043
`neededForBoot` . . . 1044
Theory 3 — btrfs rollback-on-boot pattern . . . 1044
Theory 4 — What breaks first (expect it) . . . 1044
Theory 5 — Soft impermanence . . . 1045
Theory 6 — Interaction with generations and secrets . . . 1045
Theory 7 — Home Manager alignment . . . 1046
Services checklist (starting set) . . . 1046
Worked example — minimal VM proof . . . 1047
Worked example — IMPERMANENCE.md template . . . 1048
Lab — multi-step . . . 1048
Step 1 — Decide commit level . . . 1048
Step 2 — State inventory . . . 1049
Step 3 — Implement level A or B . . . 1049
Step 4 — Proof reboot . . . 1049
Step 5 — Secrets path . . . 1049
Step 6 — Rollback story interaction . . . 1049

Step 7 — If Level C . . . 1049
Step 8 — Persist list review . . . 1049
Step 9 — Second surprise reboot . . . 1049
Exercises . . . 1050
Common gotchas . . . 1050
Checkpoint . . . 1050
Commit . . . 1051

deploy-rs **1052**

deploy-rs **1053**
Why it matters . . . 1053
Theory 1 — Deploy pipeline . . . 1053
Theory 2 — Flake skeleton with deploy-rs . . . 1054
Theory 3 — SSH prerequisites . . . 1055
Theory 4 — Magic rollback / confirmation . . . 1056
Theory 5 — Build locality . . . 1056
Cache interaction . . . 1056
Theory 6 — Profiles beyond `system` . . . 1057
Theory 7 — vs `nixos-rebuild --target-host` . . . 1057
Worked example — trivial visible change . . . 1057
Generation awareness . . . 1058
Worked example — RUNBOOK seed . . . 1058
Lab — multi-step . . . 1058
Step 1 — Target ready . . . 1058
Step 2 — Add deploy-rs input . . . 1058
Step 3 — Wire `deploy.nodes` . . . 1059
Step 4 — First deploy . . . 1059
Step 5 — Prove activation . . . 1059
Step 6 — Second deploy delta . . . 1059
Step 7 — Failure drill (safe) . . . 1059
Step 8 — Compare one-liner . . . 1059
Step 9 — Document RUNBOOK seed . . . 1059
Step 10 — Generation awareness after deploy . . . 1059
Step 11 — Cache interaction . . . 1060
Step 12 — Multi-node sketch (optional) . . . 1060
Exercises . . . 1060
Common gotchas . . . 1060
Checkpoint . . . 1061
Commit . . . 1061

Colmena **1062**

Colmena **1063**
Why it matters . . . 1063
Theory 1 — Hive mental model . . . 1063
Theory 2 — Flake-based Colmena (modern) . . . 1064

Theory 3 — Tags and blast radius . . . 1065
Parallelism . . . 1065
Theory 4 — Evaluation vs activation . . . 1065
Theory 5 — Relationship to `nixosConfigurations` . . . 1066
Theory 6 — When Colmena vs deploy-rs . . . 1066
Theory 7 — Secrets and specialArgs . . . 1066
Worked example — two-node hive (one local) . . . 1067
Lab — multi-step . . . 1068
Step 1 — Install Colmena in a shell . . . 1068
Step 2 — Define hive with defaults . . . 1068
Step 3 — Two nodes . . . 1068
Step 4 — Build only . . . 1068
Step 5 — Apply tagged subset . . . 1068
Step 6 — Tag experiment . . . 1068
Step 7 — Blast-radius note . . . 1068
Step 8 — Decision card . . . 1068
Step 9 — Optional parallel apply . . . 1069
Step 10 — DRY modules . . . 1069
Exercises . . . 1069
Common gotchas . . . 1069
Checkpoint . . . 1070
Commit . . . 1070

Shared modules across hosts **1071**

Shared modules across hosts **1072**
Why it matters . . . 1072
Theory 1 — Layering . . . 1072
Theory 2 — Role module pattern . . . 1072
Theory 3 — Profile module pattern . . . 1073
Theory 4 — Options API design . . . 1074
Theory 5 — Directory layout that scales . . . 1074
Theory 6 — SpecialArgs and lib helpers . . . 1075
Theory 7 — Testing shared modules . . . 1075
Worked example — before/after host thinness . . . 1076
Before (bad) . . . 1076
After (good) . . . 1076
Worked example — flake wiring both deploy tools . . . 1077
Lab — multi-step . . . 1077
Step 1 — Inventory duplication . . . 1077
Step 2 — Extract `profiles/baseline.nix` . . . 1077
Step 3 — Extract one role . . . 1077
Step 4 — Thin hosts . . . 1077
Step 5 — Override test . . . 1077
Step 6 — Wire into Colmena/deploy-rs . . . 1077
Step 7 — Option docs . . . 1078
Step 8 — Negative test . . . 1078
Step 9 — README map . . . 1078

Exercises . . . 1078
Common gotchas . . . 1078
Checkpoint . . . 1079
Commit . . . 1079

Images with nixos-generators **1080**

Images with nixos-generators **1081**
Why it matters . . . 1081
Theory 1 — Image = NixOS config + format . . . 1081
Mental model . . . 1082
Theory 2 — nixos-generators integration shapes . . . 1082
Flake app / package patterns . . . 1082
Module specializations . . . 1082
Theory 3 — Same modules, different entrypoints . . . 1083
Theory 4 — Cloud-init and metadata boundary . . . 1083
Theory 5 — Build performance and caches . . . 1084
Theory 6 — ISO as recovery . . . 1084
Worked example — qcow lab image (shape) . . . 1084
Worked example — consistency checklist . . . 1085
Lab — multi-step . . . 1085
Step 1 — Pick a format . . . 1086
Step 2 — Wire generators input . . . 1086
Step 3 — Image module entry . . . 1086
Step 4 — Build artifact . . . 1086
Step 5 — Boot once . . . 1086
Step 6 — Consistency proof . . . 1086
Step 7 — Size diet (optional) . . . 1086
Step 8 — Document CI policy . . . 1086
Step 9 — Stretch cloud . . . 1086
Exercises . . . 1087
Common gotchas . . . 1087
Checkpoint . . . 1087
Commit . . . 1088

Terraform boundary **1089**

Terraform boundary **1090**
Why it matters . . . 1090
Theory 1 — Responsibility split . . . 1090
Theory 2 — Handoff patterns . . . 1091
Pattern A — Image + launch . . . 1091
Pattern B — Generic distro image + first deploy . . . 1091
Pattern C — NixOS on metal (no TF) . . . 1091
Theory 3 — What to put in TF state vs flake lock . . . 1091
Theory 4 — OpenTofu vs Terraform . . . 1092
Theory 5 — Minimal resource graph (sketch) . . . 1092
Theory 6 — Secrets across the boundary . . . 1093

Theory 7 — Anti-patterns gallery . . . 1093
Worked example — BOUNDARY.md template . . . 1094
Worked example — inventory bridge . . . 1094
Lab — multi-step . . . 1095
Step 1 — Write BOUNDARY.md . . . 1095
Step 2 — Draw resource graph . . . 1095
Step 3 — Choose pattern A/B/C . . . 1095
Step 4 — Optional implement . . . 1095
Step 5 — User-data audit . . . 1095
Step 6 — State backend note . . . 1095
Step 7 — Failure story . . . 1095
Step 8 — Stretch destroy . . . 1096
Exercises . . . 1096
Common gotchas . . . 1096
Checkpoint . . . 1096
Commit . . . 1097

nixosTest **1098**

nixosTest **1099**
Why this day exists . . . 1099
Theory 1 — What a nixosTest is . . . 1099
Theory 2 — Minimal single-node test . . . 1100
Theory 3 — testScript essentials . . . 1101
Theory 4 — What to test (high value) . . . 1101
Theory 5 — Running tests . . . 1102
Worked example — web role smoke test . . . 1102
Lab — multi-step . . . 1103
Step 1 — Confirm virtualization . . . 1103
Step 2 — Hello test . . . 1103
Step 3 — Port your module . . . 1103
Step 4 — Make it fail once . . . 1103
Step 5 — Wire `checks` . . . 1103
Step 6 — Document runtime . . . 1103
Step 7 — Multi-node stretch . . . 1103
Theory 6 — Tests as flake checks . . . 1104
Theory 7 — Interactive vs pure test runs . . . 1104
Worked example — wait for unit + port . . . 1104
Lab — one negative assertion . . . 1104
Lab — time the test . . . 1105
Common gotchas . . . 1105
Checkpoint . . . 1105
Commit . . . 1105

More testing patterns **1106**

More testing patterns **1107**
Why this day exists . . . 1107

Theory 1 — Test pyramid for NixOS fleets . . . 1107
Theory 2 — Three checks minimum (template) . . . 1108
Theory 3 — Lightweight non-VM checks . . . 1108
Format / lint . . . 1108
Eval / build host configuration . . . 1109
Structured eval assertions . . . 1109
Theory 4 — Anti-flake tactics . . . 1109
Theory 5 — Mapping tests to modules . . . 1110
Theory 6 — Suite layout and CI shape . . . 1110
Worked example — fail-inject once . . . 1111
Lab — multi-step . . . 1111
Step 1 — Inventory today's checks . . . 1112
Step 2 — Add check #1 (package) . . . 1112
Step 3 — Add check #2 (nixosTest) . . . 1112
Step 4 — Add check #3 (config) . . . 1112
Step 5 — Fail inject matrix . . . 1112
Step 6 — Timing budget . . . 1112
Step 7 — README table . . . 1112
Step 8 — Stability . . . 1112
Step 9 — Secrets policy for tests . . . 1113
Common gotchas . . . 1113
Checkpoint . . . 1113
Commit . . . 1113

CI pipelines for flakes **1114**

CI pipelines for flakes **1115**
Why this day exists . . . 1115
Theory 1 — Minimal trustworthy pipeline . . . 1115
Trust boundaries . . . 1115
Theory 2 — GitHub Actions workflow (2026-shaped) . . . 1116
Theory 3 — Cachix (and friends) in CI . . . 1117
House rules . . . 1117
Alternatives . . . 1118
Theory 4 — What not to run on free runners . . . 1118
Timeouts and disks . . . 1119
Theory 5 — GitLab CI sketch . . . 1119
Theory 6 — CI vs deploy . . . 1119
Worked example — selected checks job . . . 1120
Worked example — intentional red PR . . . 1120
Lab — multi-step . . . 1120
Step 1 — Add workflow file . . . 1121
Step 2 — First green CI . . . 1121
Step 3 — Wire cache . . . 1121
Step 4 — PR simulation . . . 1121
Step 5 — Secrets hygiene . . . 1121
Step 6 — Consumer config . . . 1121
Step 7 — Document runner needs . . . 1121

Step 8 — Status badge (optional) . . . 1121
Step 9 — Deploy from CI? (decide) . . . 1122
Common gotchas . . . 1122
Checkpoint . . . 1122
Commit . . . 1122

GC and store hygiene **1123**

GC and store hygiene **1124**
Why this day exists . . . 1124
Theory 1 — What GC deletes . . . 1124
Mental model . . . 1124
Theory 2 — Generations vs store GC . . . 1125
User vs system profiles . . . 1125
Theory 3 — Policy template . . . 1125
Theory 4 — Finding what holds space . . . 1126
CI-specific roots . . . 1127
Theory 5 — optimise (hardlinks) . . . 1127
Theory 6 — Incident patterns . . . 1127
Worked example — hygiene report script . . . 1128
Lab — multi-step . . . 1128
Step 1 — Baseline metrics . . . 1128
Step 2 — Map roots . . . 1128
Step 3 — Safe cleanup . . . 1128
Step 4 — Generation policy . . . 1129
Step 5 — Enable automatic GC on lab host . . . 1129
Step 6 — optimise . . . 1129
Step 7 — CI runner note . . . 1129
Step 8 — Policy doc . . . 1129
Step 9 — Rollback still possible? . . . 1129
Step 10 — Profile audit (user + HM) . . . 1129
Step 11 — Dry-run before aggressive delete . . . 1130
Worked example — min-free incident story . . . 1130
Common gotchas . . . 1130
Checkpoint . . . 1131
Commit . . . 1131

Secure Boot and TPM (awareness) **1132**

Secure Boot and TPM (awareness) **1133**
Why this day exists . . . 1133
Theory 1 — Problem statement . . . 1133
Theory 2 — Lanzaboote overview (2026) . . . 1134
Theory 3 — TPM2 + LUKS . . . 1134
PCR trade-offs . . . 1135
Theory 4 — VM lab limits . . . 1135
QEMU/swtpm note (advanced lab) . . . 1136
Theory 5 — Recovery & break-glass . . . 1136

Theory 6 — Relation to Disko, deploy, and images . . . 1136
Worked example — dossier template . . . 1137
Lab — multi-step . . . 1138
Step 1 — Probe hardware . . . 1138
Step 2 — Choose track . . . 1138
Step 3 — Read upstream (required even for Track C) . . . 1138
Step 4 — Recovery assets first . . . 1138
Step 5 — Implement or simulate . . . 1139
Step 6 — Failure plan drill (written) . . . 1139
Step 7 — Link Disko / LUKS . . . 1139
Step 8 — Fleet policy paragraph . . . 1139
Step 9 — Tabletop: evil maid + firmware update . . . 1139
Step 10 — Capstone A vs B honesty line . . . 1139
Worked example — recovery USB checklist . . . 1140
Common gotchas . . . 1140
Checkpoint . . . 1140
Commit . . . 1141

Backup story **1142**

Backup story **1143**
Why this day exists . . . 1143
Theory 1 — What to back up . . . 1143
Inventory questions . . . 1143
Theory 2 — Tool choice (pick one) . . . 1144
3-2-1 reminder . . . 1144
Theory 3 — Restic on NixOS (shape) . . . 1144
Borgmatic sketch . . . 1145
ZFS . . . 1145
Theory 4 — Consistency . . . 1145
Theory 5 — Restore is the product . . . 1146
Success criteria . . . 1146
Theory 6 — Secrets and backup encryption . . . 1146
Worked example — local restic lab (no cloud) . . . 1147
Worked example — RTO/RPO honesty . . . 1147
Lab — multi-step . . . 1148
Step 1 — Choose tool . . . 1148
Step 2 — State inventory . . . 1148
Step 3 — Declare backup job in Nix . . . 1148
Step 4 — First backup . . . 1148
Step 5 — Restore drill (required) . . . 1148
Step 6 — DB drill (if you have the data service pattern chapter DB) . . . 1148
Step 7 — Offsite note . . . 1148
Step 8 — Runbook . . . 1149
Step 9 — Negative cases table . . . 1149
Common gotchas . . . 1149
Checkpoint . . . 1150
Commit . . . 1150

Unattended upgrades (system.autoUpgrade) 1151

Unattended upgrades (system.autoUpgrade) 1152
Thesis . . . 1152
Mental model . . . 1152
Worked examples (Cases) . . . 1153
Case 1: A server tracking a reviewed branch . . . 1153
Case 2: Switch userland now, reboot only for kernel and initrd . . . 1156
Case 3: Rings: canary, early, prod from one module . . . 1157
Case 4: Hear about failures . . . 1158
Case 5: Operate it on the host . . . 1159
Case 6: Laptops and desktops . . . 1160
The trap . . . 1160
Pitfalls . . . 1161
The boring rule . . . 1162
Try this . . . 1162
Checkpoint . . . 1162
Sources . . . 1162

Read an Update Before You Switch: diff-closures, path-info and why-depends 1164

Read an Update Before You Switch: diff-closures, path-info and why-depends 1165
Thesis . . . 1165
Mental model . . . 1165
Concepts . . . 1166
What diff-closures prints . . . 1166
Reading it for risk . . . 1166
why-depends –precise . . . 1167
Worked example: a real update (run here) . . . 1167
Reading this output . . . 1169
On a NixOS host (from the manuals, not run here) . . . 1169
The trap . . . 1170
The boring rule . . . 1170
Try this . . . 1170
Verification . . . 1171
Sources . . . 1171

Is My flake.lock Stale? Lock Age, Branch Distance and End-of-Life Branches 1172

Is My flake.lock Stale? Lock Age, Branch Distance and End-of-Life Branches 1173
Thesis . . . 1174
The flake under test . . . 1174
Check 1: how old is each pin? . . . 1175
Check 2: is each pin behind its branch? . . . 1176
Check 3: is the branch still alive? . . . 1177
The fourth problem: a second nixpkgs . . . 1178
The fix, and what it changes . . . 1178
Does the new lock carry the fix? . . . 1179
Put it in CI . . . 1180

Pitfalls . . . 1180
Sources . . . 1180

Internals **1181**

Internals and currency **1182**

Internals and currency **1183**
Learning outcomes . . . 1183
Chapter map . . . 1184
How this part connects . . . 1185
Suggested study modes . . . 1186
Lab workspace convention . . . 1186
Acceptance bar for the part . . . 1187
Baseline and policy . . . 1188
Common traps in this part . . . 1188
Next . . . 1188

The evaluator **1189**

The evaluator **1190**
Why this chapter exists . . . 1190
Theory 1 — Evaluation is a graph of thunks . . . 1190
Values vs thunks . . . 1190
Why modules love laziness . . . 1191
Eager edges you already know . . . 1191
Theory 2 — Infinite recursion patterns . . . 1192
Pattern A — Direct self-reference . . . 1192
Pattern B — Mutual recursion without a base case . . . 1192
Pattern C — Attrset self-reference (legal *shape*, dangerous force) . . . 1192
Pattern D — Module / option cycles (production shape) . . . 1192
Pattern E — `with` + shadowing confusion . . . 1193
Pattern F — `lib.fix` / fixed points gone wrong . . . 1193
Theory 3 — Debugging toolkit . . . 1193
`builtins.trace` . . . 1193
`builtins.tryEval` . . . 1194
`seq` vs `deepSeq` . . . 1194
`lib.debug.traceVal` / `traceSeq` / `traceValSeq` (nixpkgs) . . . 1194
`NIX_SHOW_STATS` and evaluator metrics . . . 1194
`nix-instantiate --eval` / `nix eval` . . . 1194
Bisect strategy for real flakes . . . 1195
Theory 4 — When to force (and when not to) . . . 1195
Theory 5 — Eval vs build vs activation (where time goes) . . . 1195
Worked example — Construct, diagnose, fix . . . 1196
Broken . . . 1196
Fixed . . . 1196
Module-shaped fix . . . 1196
Force a leaf, not the world . . . 1197

Lab 0 — Workspace . . . 1197
Lab 1 — Watch memoization . . . 1197
Lab 2 — Infinite recursion zoo . . . 1198
Lab 3 — `seq` vs `deepSeq` . . . 1198
Lab 4 — Trace a tricky real eval (your flake) . . . 1198
Lab 5 — Intentional soft failure with `tryEval` . . . 1199
Lab 6 — Module-shaped cycle (optional, high value) . . . 1199
Lab 7 — Stats snapshot . . . 1200
Failure modes (quick reference) . . . 1200
Common gotchas . . . 1200
Checkpoint . . . 1201
Commit . . . 1201
Next . . . 1201

One language, several evaluators **1202**

One language, several evaluators **1203**
Thesis . . . 1204
Mental model . . . 1204
Worked examples . . . 1206
Case 1 — Addition, and the float that prints like an int . . . 1206
Case 2 — Ordinary float arithmetic, then the extreme range . . . 1206
Case 3 — `eval-fail` means "refuse", and `|>` is still experimental . . . 1207
Case 4 — String context is part of the value . . . 1208
Case 5 — A flake the suite can check without the network . . . 1209
How to write a portable expression . . . 1209
Pinning more than one evaluator . . . 1210
The trap . . . 1212
The boring rule . . . 1212
Try this . . . 1212
Sources . . . 1213

Module system in depth **1214**

Module system in depth **1215**
Why this chapter exists . . . 1215
Theory 1 — Anatomy of a module (recall with precision) . . . 1215
Freeform modules (awareness) . . . 1216
Theory 2 — Type catalog (practical reference) . . . 1216
Scalars & paths . . . 1216
Collections . . . 1217
Structure . . . 1217
Function-ish / paths to callables . . . 1217
Useful helpers . . . 1217
Choosing types under pressure . . . 1218
Theory 3 — Definitions: `mkIf`, `mkMerge`, priorities . . . 1218
`mkIf` . . . 1218
`mkMerge` . . . 1218

Priority ladder (high level) . . . 1219
Example conflict . . . 1219
Debugging silent loss . . . 1219
Theory 4 — Assertions & warnings . . . 1220
Theory 5 — Submodules & `attrsOf` services . . . 1220
Theory 6 — Import graphs and role modules . . . 1221
Worked example — Redesign sketch . . . 1221
Before (weak) . . . 1221
After (typed) . . . 1222
Bad configs that must fail . . . 1223
Lab 0 — Workspace . . . 1223
Lab 1 — Type catalog cheat sheet (you write it) . . . 1223
Lab 2 — Redesign one module . . . 1224
Lab 3 — Priority drill . . . 1224
Lab 4 — Intentional bad config . . . 1225
Lab 5 — Multi-instance sketch (optional) . . . 1225
Lab 6 — Description quality audit . . . 1225
Failure modes . . . 1225
Common gotchas . . . 1226
Checkpoint . . . 1226
Commit . . . 1226
Next . . . 1226

Store and daemon **1227**

Store and daemon **1228**
Why this chapter exists . . . 1228
Theory 1 — Single-user vs multi-user . . . 1228
Trusted users . . . 1229
Allowed users . . . 1229
Threat sketches . . . 1229
Theory 2 — Store paths, validity, and roots . . . 1229
Path anatomy (recall) . . . 1229
Validity . . . 1229
GC roots . . . 1230
Disk full playbook (operator) . . . 1230
Theory 3 — Sandbox (Linux) . . . 1230
Why sandbox . . . 1230
Mechanisms (Linux) . . . 1231
Config knobs . . . 1231
Fixed-output derivations (FOD) . . . 1231
Nested virt / KVM note . . . 1232
Theory 4 — Substituters and signatures . . . 1232
Flow . . . 1232
Trust policy . . . 1232
Signing your own cache (awareness) . . . 1233
Theory 5 — Reading build logs . . . 1233
Theory 6 — Daemon lifecycle on NixOS . . . 1233

Worked example — Trust interview (self) . . . 1234
Lab 0 — Workspace . . . 1234
Lab 1 — Map live roots . . . 1234
Lab 2 — Daemon & trust surface . . . 1235
Lab 3 — Sandbox log literacy . . . 1235
Lab 4 — Optional impurity demo (VM only) . . . 1236
Lab 5 — Signature refusal mental model . . . 1236
Lab 6 — Elitebook / KVM lab acceptance . . . 1236
Common gotchas . . . 1237
Checkpoint . . . 1237
Commit . . . 1238
Next . . . 1238

What evaluation sends to the daemon **1239**

What evaluation sends to the daemon **1240**
Why this chapter exists . . . 1240
Mental model . . . 1240
The lab: a counting proxy . . . 1241
Case 1: cold store, three runs . . . 1242
Case 2: what the bytes were . . . 1243
Case 3: what that costs . . . 1243
The three switches . . . 1244
The trap . . . 1244
The boring rule . . . 1244
Try this . . . 1245
Sources . . . 1245

builtins.readFile: file bytes become Nix strings (and pay at eval) **1246**

builtins.readFile: file bytes become Nix strings (and pay at eval) **1247**
Thesis . . . 1247
Mental model . . . 1247
Worked examples . . . 1248
Case 1: The contract (from the manual) . . . 1248
Case 2: NUL is the hard stop . . . 1248
Case 3: Eval cost and string context . . . 1249
Case 4: When `readFile` *is* the right tool . . . 1249
The trap . . . 1249
The boring rule . . . 1249
Try this . . . 1250
Sources . . . 1250

How Nix computes a store path **1251**

How Nix computes a store path **1252**
Thesis . . . 1252
Mental model . . . 1252
The code . . . 1253

Worked examples . . . 1257
Case 1: `builtins.toFile`, one step at a time . . . 1257
Case 2: A directory: NAR bytes, NAR hash, `source` path . . . 1258
Case 3: One file, three addresses . . . 1259
Case 4: An input-addressed derivation, byte for byte . . . 1260
Case 5: The "modulo": two mirrors, one output path . . . 1261
The trap . . . 1263
The boring rule . . . 1264
Try this . . . 1264
Sources . . . 1265

Nix performance **1266**

Nix performance **1267**
Why this chapter exists . . . 1267
Theory 1 — The rebuild time budget . . . 1267
Elitebook 16 GB reality . . . 1268
Theory 2 — Jobs vs cores . . . 1268
Definitions . . . 1268
Choosing numbers (rules of thumb) . . . 1268
Remote builders . . . 1269
Theory 3 — Eval performance . . . 1269
Eval cache . . . 1269
IFD cost . . . 1269
Module count . . . 1270
Memory . . . 1270
Theory 4 — Download & store IO . . . 1270
Theory 5 — What *not* to do . . . 1270
Theory 6 — CI budget design . . . 1271
Worked example — Measurement protocol . . . 1271
Interpreting results . . . 1272
Lab 0 — Workspace . . . 1272
Lab 1 — Baseline timings . . . 1272
Lab 2 — Tune jobs/cores . . . 1273
Lab 3 — Cache hit awareness . . . 1273
Lab 4 — Eval hygiene checklist . . . 1273
Lab 5 — Remote builder decision (write-only if no hardware) . . . 1274
Lab 6 — Memory pressure drill (lab only) . . . 1274
Common gotchas . . . 1274
Checkpoint . . . 1274
Commit . . . 1275
Next . . . 1275

Content-addressed derivations (awareness) **1276**

Content-addressed derivations (awareness) **1277**
Why this chapter exists . . . 1277

Theory 1 — Input-addressed world (today's default) . . . 1277
FODs vs CA (do not conflate) . . . 1278
Theory 2 — Content-addressed idea . . . 1278
Theory 3 — Experimental feature surface . . . 1278
Reading status (primary sources) . . . 1279
Theory 4 — What CA does *not* magically fix . . . 1279
Theory 5 — Relationship to Capstone and 26.05 . . . 1280
Theory 6 — Trust and substitutes under CA thinking . . . 1280
Theory 7 — Operational decision template . . . 1281
Worked example — Conceptual comparison card . . . 1281
Lab 0 — Workspace . . . 1281
Lab 1 — Version & feature inventory (required) . . . 1282
Lab 2 — Choose a track . . . 1282
Track A — Optional tiny experiment (if feature available) . . . 1282
Track B — Status memo without enabling (always acceptable) . . . 1282
Lab 3 — Teach-back diagram . . . 1283
Lab 4 — Nixpkgs awareness scan . . . 1283
Lab 5 — Explicit non-adoption . . . 1283
Lab 6 — Analogy card (teach others) . . . 1284
Interaction with other internals chapters . . . 1284
Common gotchas . . . 1284
Checkpoint . . . 1285
Commit . . . 1285
Next . . . 1285

nixosTest mastery **1286**

nixosTest mastery **1287**
Why this chapter exists . . . 1287
Theory 1 — What nixosTest really runs . . . 1287
Theory 2 — Flake (flaky) sources . . . 1288
Theory 3 — Driver API patterns (Python DSL) . . . 1288
Anti-pattern . . . 1289
Multi-node . . . 1289
Theory 4 — Systemd-aware testing . . . 1289
Theory 5 — Secrets and tests . . . 1290
Theory 6 — CI integration . . . 1290
Elitebook / KVM lab acceptance . . . 1290
Theory 7 — Machine objects and multi-node patterns . . . 1291
Theory 8 — Snapshotting vs rebuild cost . . . 1291
Worked example — systemd journal assertion . . . 1291
Worked example — regression that fails when firewall wrong . . . 1291
Lab 0 — Workspace . . . 1292
Lab 1 — Inventory existing tests . . . 1292
Lab 2 — Eliminate sleep-only waits . . . 1292
Lab 3 — Add one regression that would have bitten you . . . 1292
Lab 4 — Multi-node or client/server (stretch) . . . 1293
Lab 5 — Failure UX . . . 1293

Lab 6 — Flake check wiring audit . . . 1293
Lab 7 — Failure screenshot discipline . . . 1293
Common gotchas . . . 1294
Checkpoint . . . 1294
Commit . . . 1294
Next . . . 1294

Ecosystem map **1295**

Ecosystem map **1296**
Why this chapter exists . . . 1296
Theory 1 — Layers of "ecosystem" . . . 1296
Theory 2 — devenv (project environments) . . . 1297
Compare to house default . . . 1297
Theory 3 — Snowfall (flake library / structure) . . . 1298
Compare to flake-parts . . . 1298
Theory 4 — nix-darwin . . . 1298
Theory 5 — Decision framework (keep/drop) . . . 1299
Theory 6 — Ecosystem postcard (vocabulary) . . . 1299
Theory 7 — Integration risks matrix . . . 1299
Worked example — COMPARE.md skeleton . . . 1300
Lab 0 — Workspace . . . 1301
Lab 1 — Map your current layout (30 min) . . . 1301
Lab 2 — Pick one tool and trial . . . 1301
If devenv . . . 1301
If Snowfall . . . 1302
If nix-darwin . . . 1302
Lab 3 — Comparison table (required) . . . 1302
Lab 4 — Ecosystem postcard . . . 1302
Lab 5 — Capstone protection check . . . 1303
Lab 6 — Teach-back (5 bullets) . . . 1303
Common gotchas . . . 1303
Checkpoint . . . 1304
Commit . . . 1304
Next . . . 1304

Comparative landscape **1305**

Comparative landscape **1306**
Why this chapter exists . . . 1306
Theory 1 — Comparison dimensions . . . 1306
Theory 2 — NixOS (your baseline) . . . 1307
Theory 3 — Guix System . . . 1307
Theory 4 — Image-based / immutable distros . . . 1308
Theory 5 — Ansible (and friends) . . . 1309
Theory 6 — Honesty checklist (avoid tribal takes) . . . 1309
Theory 7 — Decision time-bounds . . . 1310
Worked example — weighted scorecard fragment . . . 1310

Lab 0 — Workspace . . . 1310
Lab 1 — Primary sources (skimming with intent) . . . 1311
Lab 2 — Scorecard for *your* context . . . 1311
Lab 3 — Essay (required) . . . 1311
Lab 4 — Elevator version . . . 1312
Lab 5 — Capstone implication . . . 1312
Lab 6 — Scenario cards (forced choice) . . . 1312
Card A — Homelab "weird topology" . . . 1312
Card B — Employer fleet . . . 1312
Card C — Kubernetes worker pool . . . 1312
Lab 7 — Quote hygiene . . . 1313
Worked example — elevator closer . . . 1313
Common gotchas . . . 1313
Checkpoint . . . 1314
Commit . . . 1314
Next . . . 1314

Reading release notes (26.05) 1315

Reading release notes (26.05) 1316
Why this chapter exists . . . 1316
Theory 1 — What "a NixOS release" actually is . . . 1316
Theory 2 — How to read release notes like an operator . . . 1317
Pass A — Inventory (30–45 min) . . . 1317
Pass B — Personal filter (45–60 min) . . . 1317
Pass C — Prove with eval/build . . . 1318
Pass D — Warnings are future breakages . . . 1318
Theory 3 — `stateVersion` is not a fashion number . . . 1318
Theory 4 — Cross-check Nix itself . . . 1318
Theory 5 — Module renames and aliases (kinds of change) . . . 1319
Theory 6 — Inputs beyond nixpkgs . . . 1319
Theory 7 — Severity and scheduling . . . 1320
Worked example — IMPACT.md row . . . 1320
Lab 0 — Workspace . . . 1321
Lab 1 — Capture current pin . . . 1321
Lab 2 — Official notes intake . . . 1321
Lab 3 — Repo audit scripts . . . 1322
Lab 4 — Build verification . . . 1322
Lab 5 — Personal "gotchas from 26.05" . . . 1322
Lab 6 — Feed upgrade planning . . . 1323
Common gotchas . . . 1323
Checkpoint . . . 1323
Commit . . . 1324
Next . . . 1324

Upgrade planning 1325

Upgrade planning **1326**
Why this chapter exists . . . 1326
Theory 1 — What actually changes in a flake upgrade . . . 1326
Theory 2 — Staging strategy . . . 1327
Environments . . . 1327
Git strategy . . . 1327
Companion inputs . . . 1327
Theory 3 — Tests-first gate . . . 1327
Theory 4 — Rollback is part of the plan . . . 1328
Concrete rollback commands (fill for your path) . . . 1328
Theory 5 — `stateVersion` and stateful services . . . 1329
Theory 6 — Communication & windows . . . 1329
Abort criteria template . . . 1329
Theory 7 — Elitebook / small-lab constraints . . . 1330
Runbook template (you will fill this) . . . 1330
0. Metadata . . . 1330
1. Preconditions . . . 1330
2. Release notes . . . 1331
3. Technical steps (lab first) . . . 1331
4. Health verification . . . 1331
5. Rollback . . . 1331
6. Risks . . . 1331
7. Decision . . . 1331
Lab 0 — Workspace . . . 1332
Lab 1 — Inventory pins . . . 1332
Lab 2 — Write the full runbook . . . 1332
Lab 3 — Dry-run mechanics on lab (optional execution) . . . 1332
Lab 4 — Test plan matrix . . . 1333
Lab 5 — Abort & rollback tabletop . . . 1333
Lab 6 — Disk space math . . . 1333
Common gotchas . . . 1334
Checkpoint . . . 1334
Commit . . . 1334
Next . . . 1334

Troubleshooting drills **1335**

Troubleshooting drills **1336**
Why this chapter exists . . . 1336
Theory 1 — Failure class matrix . . . 1336
Theory 2 — Golden command palette . . . 1337
Remote when SSH dies . . . 1337
Elitebook lab specifics . . . 1337
Theory 3 — Injection catalog (choose carefully) . . . 1338
Theory 4 — Time discipline . . . 1338
Theory 5 — Playbook entry format . . . 1338
Theory 6 — Rollback as a diagnostic tool . . . 1339
Theory 7 — Layered isolation order . . . 1339

Worked example — playbook stub . . . 1340
Lab 0 — Snapshot . . . 1340
Lab 1 — Baseline health . . . 1340
Lab 2 — Inject & recover #1 (eval or build) . . . 1341
Lab 3 — Inject & recover #2 (service or secrets) . . . 1341
Lab 4 — Inject & recover #3 (network) — console required . . . 1341
Lab 5 — Compile master index . . . 1342
Lab 6 — Timeboxed chaos #4 (state) . . . 1342
Lab 7 — Master index cross-links . . . 1342
Lab 8 — End-state health . . . 1342
Common gotchas . . . 1342
Checkpoint . . . 1343
Commit . . . 1343
Next . . . 1343

Architecture review **1344**

Architecture review **1345**
Why this chapter exists . . . 1345
Theory 1 — Capstone A bar (recall) . . . 1345
Theory 2 — Coverage matrix (topic parts) . . . 1346
Theory 3 — Architecture views (write these) . . . 1347
A. Context . . . 1347
B. Containers of code . . . 1347
C. Runtime view . . . 1347
D. Trust view . . . 1347
E. Failure view . . . 1347
F. Data view . . . 1348
Theory 4 — Prioritization . . . 1348
Theory 5 — Non-goals for Capstone A . . . 1349
Theory 6 — Evidence standards . . . 1349
Theory 7 — Dossier structure (feeds Capstone) . . . 1349
Lab 0 — Workspace . . . 1350
Lab 1 — Automated inventory . . . 1350
Lab 2 — Coverage matrix . . . 1350
Lab 3 — `ARCHITECTURE.md` (required) . . . 1351
Lab 4 — Prioritized backlog . . . 1351
Lab 5 — Checklist freeze seed . . . 1352
Lab 6 — Risk register (short) . . . 1352
Lab 7 — Sequence plan . . . 1352
Common gotchas . . . 1353
Checkpoint . . . 1353
Commit . . . 1353
Next . . . 1353

How Nix Normalizes Metadata for Reproducibility **1354**

How Nix Normalizes Metadata for Reproducibility **1355**
Thesis . . . 1355
Mental model . . . 1355
Worked example . . . 1355
Trap . . . 1356
Boring rule . . . 1356
Try this . . . 1356
Sources . . . 1356

Capstone **1357**

Capstone and recovery **1358**

Capstone and recovery **1359**
Capstone A vs optional bar . . . 1359
Learning outcomes . . . 1360
Chapter map . . . 1360
Suggested sequence . . . 1360
Dossier structure (canonical) . . . 1361
Live Capstone A checklist (copy into repo) . . . 1362
Lab workspace convention . . . 1362
Acceptance bar for Capstone A . . . 1362
Elitebook / KVM lab notes . . . 1363
Policy reminders . . . 1363
Next . . . 1363

Capstone: scope and plan **1364**

Capstone: scope and plan **1365**
Why this chapter exists . . . 1365
Theory — Working agreements for Capstone . . . 1365
Suggested repo shape (adapt, don't dogma) . . . 1365
Capstone A checklist (live) . . . 1366
Target scope for this chapter . . . 1366
Explicit non-goals (this chapter) . . . 1367
Worked example — Minimal host flake skeleton . . . 1367
Theory — Capstone A success definition (this chapter) . . . 1368
Theory — Branch discipline . . . 1368
Worked example — smoke script . . . 1368
Rebuild proof steps (skeleton → switch) . . . 1369
Lab 0 — Branch & checklist . . . 1369
Lab 1 — Build green host . . . 1369
Lab 2 — Users story . . . 1370
Track HM (workstation-ish) . . . 1370
Track service users (server-ish) . . . 1370
Lab 3 — Module hygiene . . . 1370
Lab 4 — Update architecture & gaps . . . 1371
Lab 5 — Smoke . . . 1371

Lab 6 — Gap register freeze for next chapter . . . 1371
Lab 7 — Teach-back outline (5 bullets) . . . 1371
Failure modes . . . 1372
Common gotchas . . . 1372
Checkpoint . . . 1372
Commit . . . 1373
Next . . . 1373

Capstone: implement **1374**

Capstone: implement **1375**
Why this chapter exists . . . 1375
Theory 1 — Secrets E2E definition of done . . . 1375
Threat model (short) . . . 1375
Theory 2 — sops-nix sketch (preferred) . . . 1376
`.sops.yaml` creation rules (idea) . . . 1376
Theory 3 — agenix sketch (alternative) . . . 1377
Theory 4 — Proxy E2E definition of done . . . 1377
nginx sketch . . . 1377
TLS modes (pick and document in `docs/TLS.md`) . . . 1378
Theory 5 — Ordering and dependencies . . . 1378
Store discipline reminder . . . 1378
Theory 6 — Module shape for Capstone . . . 1378
Concrete checklists . . . 1379
Secrets checklist . . . 1379
Proxy checklist . . . 1379
Worked example — Proof commands . . . 1380
Lab 0 — Branch . . . 1380
Lab 1 — Secrets path E2E . . . 1380
Lab 2 — Proxy path E2E . . . 1381
Lab 3 — Failure injection (mini) . . . 1381
Lab 4 — Tests stub toward prove-rebuild chapter . . . 1381
Lab 5 — Docs & checklist . . . 1381
Lab 6 — Offline key custody proof . . . 1382
Common gotchas . . . 1382
Checkpoint . . . 1382
Commit . . . 1383
Next . . . 1383

Capstone: harden and document **1384**

Capstone: harden and document **1385**
Why this chapter exists . . . 1385
Theory 1 — Deploy definition of done . . . 1385
deploy-rs sketch . . . 1385
Colmena sketch . . . 1386
Theory 2 — CI definition of done . . . 1386
Minimal workflow sketch . . . 1387

Theory 3 — Cache push value . . . 1387
Theory 4 — SSH deploy auth checklist . . . 1387
Theory 5 — RUNBOOK / DEPLOY docs minimum . . . 1388
Concrete checklists . . . 1388
Deploy checklist . . . 1388
CI checklist . . . 1389
Lab 0 — Workspace . . . 1389
Lab 1 — Wire deploy tool . . . 1389
Lab 2 — CI workflow . . . 1390
Lab 3 — Hardening pass (light) . . . 1390
Lab 4 — Docs . . . 1390
Lab 5 — Failure tabletop . . . 1391
Lab 6 — Elitebook admin path . . . 1391
Common gotchas . . . 1391
Checkpoint . . . 1392
Commit . . . 1392
Next . . . 1392

Capstone: prove rebuildability **1393**

Capstone: prove rebuildability **1394**
Why this chapter exists . . . 1394
Theory 1 — "Sufficient" tests for Capstone A . . . 1394
Flake outputs shape . . . 1395
Theory 2 — RUNBOOK audience . . . 1395
Theory 3 — Diagrams that match reality . . . 1395
Theory 4 — Rebuild proof steps (pre-wipe) . . . 1396
Theory 5 — Assertion fixtures . . . 1396
Concrete checklists . . . 1396
Tests checklist . . . 1396
Docs checklist . . . 1397
Lab 0 — Workspace . . . 1397
Lab 1 — Harden tests . . . 1397
Lab 2 — RUNBOOK . . . 1397
Lab 3 — Diagrams . . . 1398
Lab 4 — Paper DR draft . . . 1398
Lab 5 — Freeze readiness review . . . 1398
Lab 6 — Negative eval fixture . . . 1398
Common gotchas . . . 1399
Checkpoint . . . 1399
Commit . . . 1399
Next . . . 1399

Capstone: optional multi-node bar **1400**

Capstone: optional multi-node bar **1401**
Why this chapter exists . . . 1401

Theory 1 — What “freeze” means . . . 1401
Freeze is a social contract with yourself . . . 1401
Tag shape . . . 1402
Theory 2 — Waiver register . . . 1402
Theory 3 — Optional multi-node bar . . . 1402
Elitebook 16 GB guidance . . . 1403
Shared modules pattern . . . 1403
Theory 4 — Freeze gate commands . . . 1403
Concrete freeze checklist . . . 1404
Lab 0 — Workspace . . . 1404
Lab 1 — Honesty pass on CAPSTONE-A . . . 1404
Lab 2 — Green suite at freeze . . . 1404
Lab 3 — Tag . . . 1405
Lab 4 — Optional second node (only if A green) . . . 1405
Lab 5 — Snapshot for DR . . . 1405
Lab 6 — Freeze announcement (to yourself) . . . 1405
Common gotchas . . . 1406
Checkpoint . . . 1406
Commit . . . 1406
Next . . . 1406

Disaster recovery notes **1407**

Disaster recovery notes **1408**
Why this chapter exists . . . 1408
Theory 1 — What “from flake + keys alone” means . . . 1408
Theory 2 — Recovery paths (pick what you documented) . . . 1409
Path A — Interactive installer + flake . . . 1409
Path B — nixos-anywhere / remote install . . . 1409
Path C — Image rehydrate . . . 1409
Theory 3 — Timing and RTO . . . 1409
Theory 4 — What wipe proves / does not prove . . . 1410
Theory 5 — Data vs system . . . 1410
Concrete DR checklist . . . 1410
Pre-wipe (mandatory) . . . 1410
Post-rebuild health . . . 1411
Lab 0 — Confirm freeze and keys . . . 1411
Lab 1 — Execute wipe per DR-PLAN . . . 1411
Lab 2 — Time the recovery . . . 1411
Lab 3 — Secrets rehydrate . . . 1412
Lab 4 — Health + deploy proof . . . 1412
Lab 5 — Fix docs (the real deliverable) . . . 1412
Lab 6 — Optional: refuse snapshot . . . 1412
Common gotchas . . . 1413
Checkpoint . . . 1413
Commit . . . 1413
Next . . . 1413

Rollback and tests **1414**

Rollback and tests **1415**
Why this chapter exists . . . 1415
Theory 1 — Rollback layers . . . 1415
Commands (lab) . . . 1416
Theory 2 — Bad change protocol . . . 1416
Theory 3 — Full suite definition . . . 1417
Theory 4 — Troubleshooting checklist (finalize) . . . 1417
Theory 5 — State vs generation (again) . . . 1417
Concrete checklists . . . 1418
Rollback drill . . . 1418
Suite . . . 1418
Lab 0 — Workspace . . . 1418
Lab 1 — Record baseline generation . . . 1418
Lab 2 — Intentional break . . . 1419
Lab 3 — Rollback . . . 1419
Lab 4 — Git hygiene . . . 1419
Lab 5 — Full suite green . . . 1419
Lab 6 — Finalize troubleshooting checklist . . . 1420
Lab 7 — Bootloader path (optional) . . . 1420
Common gotchas . . . 1420
Checkpoint . . . 1420
Commit . . . 1421
Next . . . 1421

Retrospective **1422**

Retrospective **1423**
Why this chapter exists . . . 1423
Theory 1 — What you should now be able to do . . . 1423
Theory 2 — Retrospective structure (blameless) . . . 1424
Theory 3 — Teach plan (1 session) . . . 1424
Theory 4 — Roadmap options (pick few) . . . 1425
Theory 5 — What not to do today . . . 1425
Lab 0 — Workspace . . . 1425
Lab 1 — Skills matrix . . . 1425
Lab 2 — Full retrospective writeup . . . 1426
Lab 3 — Teach plan . . . 1426
Lab 4 — Roadmap . . . 1426
Lab 5 — Capstone A final sign-off . . . 1426
Lab 6 — Celebrate without thrash . . . 1427
Common gotchas . . . 1427
Checkpoint . . . 1427
Commit . . . 1427
Next . . . 1428

CI/CD **1429**

Setting Up Binary Caches **1430**

Setting Up Binary Caches **1431**
Mental model . . . 1431
Worked examples . . . 1432
Case 1: Consume a team cache on NixOS . . . 1432
Case 2: Push from a workstation (once) . . . 1432
Case 3: Verify a hit versus a miss . . . 1433
Case 4: Serve a tiny cache with `nix copy` . . . 1433
Case 5: Generate a signing key for a self-hosted cache . . . 1433
The trap . . . 1434
The boring rule . . . 1434
Try this . . . 1434

GitHub Actions with Nix **1435**

GitHub Actions with Nix **1436**
Mental model . . . 1436
Worked examples . . . 1436
Case 1: Check and build, push to Cachix . . . 1436
Case 2: `nix flake check` is the gate . . . 1437
Case 3: Magic Nix Cache when you do not have Cachix yet . . . 1438
Case 4: Pin the installer, pin nixpkgs . . . 1438
Case 5: Free disk on the runner . . . 1438
The trap . . . 1439
The boring rule . . . 1439
Try this . . . 1439

GitLab CI Integration **1440**

GitLab CI Integration **1441**
Mental model . . . 1441
Worked examples . . . 1441
Case 1: Official Nix image, flakes on . . . 1441
Case 2: Cachix from GitLab . . . 1442
Case 3: Cache `/nix` on a self-hosted runner . . . 1442
Case 4: `rules` so forks do not push . . . 1443
Case 5: Show what you shipped . . . 1443
The trap . . . 1444
The boring rule . . . 1444
Try this . . . 1444

Build Caching Strategies **1445**

Build Caching Strategies **1446**
Mental model . . . 1446

Worked examples . . . 1446
Case 1: Team cache is the real cache . . . 1446
Case 2: magic-nix-cache as a spare tyre . . . 1447
Case 3: `lib.cleanSource` / gitignore . . . 1447
Case 4: Vendor once, compile often . . . 1448
Case 5: See what would rebuild . . . 1448
The trap . . . 1449
The boring rule . . . 1449
Try this . . . 1449

Testing in CI Pipelines **1450**

Testing in CI Pipelines **1451**
Mental model . . . 1451
Worked examples . . . 1451
Case 1: Hermetic Go tests . . . 1451
Case 2: Flake checks . . . 1452
Case 3: CI workflow . . . 1452
Case 4: VM test job (KVM) . . . 1453
Case 5: Fail the gate . . . 1453
The trap . . . 1454
The boring rule . . . 1454
Try this . . . 1454

Secrets Management in CI **1455**

Secrets Management in CI **1456**
Mental model . . . 1456
Worked examples . . . 1456
Case 1: Private flake inputs without putting the PAT in Nix . . . 1456
Case 2: Eval sops without decrypting values into the store . . . 1457
Case 3: The leak, on purpose, once . . . 1458
Case 4: Cachix write only on main . . . 1458
Case 5: `nix flake check` without credentials . . . 1459
The trap . . . 1459
The boring rule . . . 1460
Try this . . . 1460

Remote Builders **1461**

Remote Builders **1462**
Mental model . . . 1462
Worked examples . . . 1463
Case 1: NixOS client configuration . . . 1463
Case 2: One-shot without a module . . . 1463
Case 3: Builder host — NixOS . . . 1463
Case 4: Darwin → Linux . . . 1464
Case 5: Features — why tests ran locally . . . 1465
The trap . . . 1465

The boring rule . . . 1465
Try this . . . 1465

systemd-nspawn Test Backend **1466**

systemd-nspawn Test Backend **1467**
Mental model . . . 1467
Worked examples . . . 1468
Case 1: Host daemon settings for container tests . . . 1468
Case 2: Minimal two-machine ping with `containers` . . . 1468
Case 3: Keep QEMU `nodes` when the claim needs a real VM . . . 1470
Case 4: Mixed containers + VMs sharing a VLAN need `/dev/net` . . . 1471
Case 5: Bind-mount a host device (GPU sketch) — still not a kernel test . . . 1472
The trap . . . 1472
The boring rule . . . 1473
Try this . . . 1473

Containers and Kubernetes **1474**

Creating Container Images with Nix (dockerTools) **1475**

Creating Container Images with Nix (dockerTools) **1476**
Mental model . . . 1476
Worked examples . . . 1476
Case 1: Layered image . . . 1476
Case 2: `vendorHash` for a real Go module . . . 1478
Case 3: Inspect layers . . . 1478
Case 4: Flake package . . . 1478
Case 5: Nobody is home in the image . . . 1480
The trap . . . 1480
The boring rule . . . 1480
Try this . . . 1480

Kubernetes Manifests as Nix Modules **1481**

Kubernetes Manifests as Nix Modules **1482**
Mental model . . . 1482
Worked examples . . . 1482
Case 1: A Deployment as JSON . . . 1482
Case 2: Same flake as the image . . . 1483
Case 3: Apply . . . 1484
Case 4: ConfigMap from Nix, not from kubectl create . . . 1484
Case 5: When the cluster cannot run Nix . . . 1485
The trap . . . 1485
The boring rule . . . 1485
Try this . . . 1485

Helm Charts Generated from Nix **1486**

Helm Charts Generated from Nix **1487**
Mental model . . . 1487
Worked examples . . . 1487
Case 1: Vendor a chart tarball . . . 1487
Case 2: Render values from Nix . . . 1488
Case 3: Wrapper script in a flake . . . 1488
Case 4: Apply the bits you reviewed . . . 1489
Case 5: Diff in CI . . . 1489
The trap . . . 1490
The boring rule . . . 1490
Try this . . . 1490

GitOps with Argo CD and Flux **1491**

GitOps with Argo CD and Flux **1492**
Mental model . . . 1492
Worked examples . . . 1492
Case 1: Generated tree layout . . . 1492
Case 2: Flux GitRepository (illustrative) . . . 1493
Case 3: Kustomization pointing at generated YAML . . . 1493
Case 4: CI gate . . . 1494
Case 5: Argo Application (if the org standardised on Argo) . . . 1494
The trap . . . 1495
The boring rule . . . 1495
Try this . . . 1495

Service Mesh and Observability **1497**

Service Mesh and Observability **1498**
Mental model . . . 1498
Worked examples . . . 1498
Case 1: App metrics without a mesh . . . 1498
Case 2: Probe the endpoint . . . 1499
Case 3: NetworkPolicy before sidecar . . . 1500
Case 4: If you must mesh . . . 1500
Case 5: Logs on NixOS already exist . . . 1500
The trap . . . 1501
The boring rule . . . 1501
Try this . . . 1501

IaC and Cloud **1502**

Managing Terraform Providers with Nix **1503**

Managing Terraform Providers with Nix **1504**
Mental model . . . 1504
Worked examples . . . 1504
Case 1: Shell with pinned providers . . . 1504

Case 2: Flake devShell . . . 1505
Case 3: Offline init . . . 1505
Case 4: Tiny plan . . . 1506
Case 5: CI without `setup-terraform` . . . 1506
The trap . . . 1507
The boring rule . . . 1507
Try this . . . 1507

OpenTofu Integration **1508**

OpenTofu Integration **1509**
Mental model . . . 1509
Worked examples . . . 1509
Case 1: devShell with pinned plugins . . . 1509
Case 2: Same `main.tf` as Terraform . . . 1510
Case 3: Backend still not in Nix . . . 1511
Case 4: CI splits plan and apply . . . 1511
Case 5: Do not mix CLIs . . . 1511
The trap . . . 1512
The boring rule . . . 1512
Try this . . . 1512

Ansible Provisioning with Nix **1513**

Ansible Provisioning with Nix **1514**
Mental model . . . 1514
Worked examples . . . 1514
Case 1: Pinned ansible CLI . . . 1514
Case 2: Inventory in git . . . 1515
Case 3: Collections pinned, not Galaxy at runtime . . . 1515
Case 4: Playbook that only talks to leftover Ubuntu . . . 1516
Case 5: Refuse NixOS in the inventory . . . 1516
The trap . . . 1517
The boring rule . . . 1517
Try this . . . 1517

Cloud Provider Configuration **1518**

Cloud Provider Configuration **1519**
Mental model . . . 1519
Worked examples . . . 1519
Case 1: Official AMI lookup (OpenTofu) . . . 1519
Case 2: Image you built . . . 1520
Case 3: user-data is SSH keys, not packages . . . 1520
Case 4: Security group matches the firewall . . . 1521
Case 5: Tags are how you find the box . . . 1521
The trap . . . 1522
The boring rule . . . 1522
Try this . . . 1522

Multi-Cloud Deployment Patterns **1523**

Multi-Cloud Deployment Patterns **1524**
Mental model . . . 1524
Worked examples . . . 1524
Case 1: Shared module, two hosts . . . 1524
Case 2: Same birth, two clouds . . . 1525
Case 3: Images from the generator, not a conversion script . . . 1525
Case 4: Separate tofu directories . . . 1526
Case 5: WireGuard between clouds, not a service mesh . . . 1526
The trap . . . 1526
The boring rule . . . 1527
Try this . . . 1527

Platform Engineering **1528**

The Internal Developer Platform Pattern **1529**

The Internal Developer Platform Pattern **1530**
Mental model . . . 1530
Worked examples . . . 1530
Case 1: Platform flake . . . 1530
Case 2: Init from template . . . 1531
Case 3: Consumer lock bump . . . 1532
Case 4: Baseline NixOS module . . . 1532
Case 5: Version the platform . . . 1533
The trap . . . 1533
The boring rule . . . 1533
Try this . . . 1533

Monorepo Strategies with Nix **1534**

Monorepo Strategies with Nix **1535**
Mental model . . . 1535
Worked examples . . . 1535
Case 1: fileset `src` per leaf . . . 1535
Case 2: One flake, many packages . . . 1536
Case 3: PR path filter (CI) . . . 1536
Case 4: `linkFarm` for the nightly smoke set . . . 1537
Case 5: Do not use recursive flakes . . . 1537
The trap . . . 1538
The boring rule . . . 1538
Try this . . . 1538

Multi-Platform Development and Distribution **1539**

Multi-Platform Development and Distribution **1540**
Mental model . . . 1540

Worked examples . . . 1540
Case 1: Enumerate systems without flake-utils . . . 1540
Case 2: Native remote builder, not qemu . . . 1541
Case 3: CI matrix for the OS humans use . . . 1542
Case 4: Release two closures . . . 1542
Case 5: `file` the toolchain, not your hopes . . . 1542
The trap . . . 1543
The boring rule . . . 1543
Try this . . . 1543

Build Optimization and Performance Tuning **1544**

Build Optimization and Performance Tuning **1545**
Mental model . . . 1545
Worked examples . . . 1545
Case 1: NixOS 26.05 knobs . . . 1545
Case 2: No IFD on the hot path . . . 1546
Case 3: Measure eval . . . 1547
Case 4: `--dry-run` after a docs edit . . . 1547
Case 5: Heavy compiles belong on the builder . . . 1547
The trap . . . 1547
The boring rule . . . 1548
Try this . . . 1548

Debugging and Troubleshooting Production Issues **1549**

Debugging and Troubleshooting Production Issues **1550**
Mental model . . . 1550
Worked examples . . . 1550
Case 1: Build log . . . 1550
Case 2: Eval trace . . . 1551
Case 3: Unit after switch . . . 1551
Case 4: Unexpected python in the OS closure . . . 1551
Case 5: Rollback then revert git . . . 1552
The trap . . . 1552
The boring rule . . . 1553
Try this . . . 1553

Testing Strategies for Nix Configurations **1554**

Testing Strategies for Nix Configurations **1555**
Mental model . . . 1555
Worked examples . . . 1555
Case 1: Cheap flake checks . . . 1555
Case 2: NixOS VM test . . . 1556
Case 3: CI job . . . 1557
Case 4: Hermetic `checkPhase` . . . 1557
Case 5: What not to run on every PR . . . 1557
The trap . . . 1558

The boring rule . . . 1558
Try this . . . 1558

Maintenance, Upgrades, and Migration **1559**

Maintenance, Upgrades, and Migration **1560**
Mental model . . . 1560
Worked examples . . . 1560
Case 1: Lock bump inside 26.05 . . . 1560
Case 2: Release upgrade . . . 1561
Case 3: `stateVersion` stays . . . 1561
Case 4: Home Manager follows the same train . . . 1561
Case 5: Record the lock . . . 1562
The trap . . . 1562
The boring rule . . . 1562
Try this . . . 1563

Enterprise Operational Checklists **1564**

Enterprise Operational Checklists **1565**
Mental model . . . 1565
Worked examples . . . 1565
Case 1: Daily . . . 1565
Case 2: Weekly disk . . . 1566
Case 3: Before fleet apply . . . 1566
Case 4: Advisory day . . . 1567
Case 5: New host (checklist that is also git) . . . 1567
The trap . . . 1567
The boring rule . . . 1568
Try this . . . 1568

Fleet Deployments with Colmena **1569**

Fleet Deployments with Colmena **1570**
Mental model . . . 1570
Worked examples . . . 1570
Case 1: Hive in the flake . . . 1570
Case 2: Canary, then the rest of the tag . . . 1572
Case 3: Dry-run before activate . . . 1572
Case 4: Build where the CPU is . . . 1572
Case 5: Activation failure is a generation, not a brick . . . 1572
The trap . . . 1573
The boring rule . . . 1573
Try this . . . 1573

Lightweight MicroVMs with microvm.nix **1574**

Lightweight MicroVMs with microvm.nix **1575**
Mental model . . . 1575

Worked examples . . . 1575
Case 1: Guest in the flake . . . 1575
Case 2: Run the declared runner . . . 1576
Case 3: Persist nothing unless listed . . . 1577
Case 4: Host virtualisation (workstation) . . . 1577
Case 5: microVM vs `runNixOSTest` . . . 1577
The trap . . . 1578
The boring rule . . . 1578
Try this . . . 1578

Content-Addressed Nix Internals **1579**

Content-Addressed Nix Internals **1580**
Mental model . . . 1580
Worked examples . . . 1580
Case 1: FOD — identity is the hash . . . 1580
Case 2: Comment rebuilds an input-addressed path . . . 1581
Case 3: `nix show-derivation` . . . 1581
Case 4: Disk without CA — optimise . . . 1582
Case 5: Do not enable CA on the desk . . . 1582
The trap . . . 1583
The boring rule . . . 1583
Try this . . . 1583

AI Agent Sandboxes, Loop Engineering, and Universal Overlays **1584**

AI Agent Sandboxes, Loop Engineering, and Universal Overlays **1585**
Mental model . . . 1585
The three-way drift crisis . . . 1585
Why the operating system matters for agents . . . 1586
Universal overlays: "Fix everything at the root" . . . 1586
Worked examples . . . 1586
Case 1: Declarative agent sandbox with devenv and backpressure . . . 1586
Case 2: Minimal layered container from the same package expression . . . 1588
Case 3: Universal overlay for capability truncation (preventing agent force-pushes) . . . 1589
Case 4: Universal security hotfix across the entire dependency graph . . . 1590
Case 5: Loop engineering — putting the entire operating system under test . . . 1590
Bare metal fleet economics in the AI era . . . 1593
Antipatterns . . . 1593
The boring rule . . . 1594
Verification checklist . . . 1594

Production Labs **1595**

Reading NixOS.org Infra as a Pattern Source **1596**

Reading NixOS.org Infra as a Pattern Source **1597**
Mental model . . . 1597

Worked examples . . . 1598
Case 1: Map the live flake, ignore the stale README . . . 1598
Case 2: Extra substituter on the flake, daemon still needs the key . . . 1598
Case 3: `follows` is how a 20-input flake stays on one nixpkgs . . . 1599
Case 4: Desk flake that *looks* like infra without flake-parts . . . 1599
Case 5: What we will not copy . . . 1600
The trap . . . 1600
The boring rule . . . 1601
Try this . . . 1601

Operator Keys and Groups **1602**

Operator Keys and Groups **1603**
Mental model . . . 1603
Worked examples . . . 1603
Case 1: Desk `keys.nix` . . . 1603
Case 2: Root and a named admin from the group . . . 1604
Case 3: `knownHosts` from `ssh.machines` . . . 1605
Case 4: Drop a human in one commit . . . 1606
Case 5: sops creation rules share the same people . . . 1606
The trap . . . 1607
The boring rule . . . 1607
Try this . . . 1607

Critical vs Non-Critical Hives **1608**

Critical vs Non-Critical Hives **1609**
Mental model . . . 1609
Worked examples . . . 1609
Case 1: Tags on the existing hive . . . 1609
Case 2: Two sops trees . . . 1611
Case 3: Named devShells, not one kitchen sink . . . 1611
Case 4: Staging is a host, not a branch of secrets . . . 1612
Case 5: `colmena.sh` is a one-liner, not a platform . . . 1612
The trap . . . 1613
The boring rule . . . 1613
Try this . . . 1613

Builder Factory and Restricted Store SSH **1614**

Builder Factory and Restricted Store SSH **1615**
Mental model . . . 1615
Worked examples . . . 1615
Case 1: `mkHost` factory with common modules . . . 1615
Case 2: Forced `nix-store --serve --write` . . . 1617
Case 3: Nix daemon knobs the factory always sets . . . 1618
Case 4: Client `buildMachines` matches the forced user . . . 1618
Case 5: Instance file is disks + identity, not a second OS . . . 1619
The trap . . . 1619

The boring rule . . . 1620
Try this . . . 1620

Channel Promotion and CI Host Groups **1621**

Channel Promotion and CI Host Groups **1622**
Mental model . . . 1622
Worked examples . . . 1622
Case 1: A desk channel table . . . 1622
Case 2: `tested` as an explicit package . . . 1623
Case 3: Group host toplevels by arch (no surprise Darwin eval) . . . 1624
Case 4: OpenTofu envelope, NixOS OS — two shells . . . 1624
Case 5: Promote, then deprecate . . . 1625
The trap . . . 1626
The boring rule . . . 1626
Try this . . . 1626

Production Capstones **1627**

Production Architecture and Threat Model **1628**

Production Architecture and Threat Model **1629**
Mental model . . . 1629
Worked examples . . . 1630
Case 1: Inventory in the flake . . . 1630
Case 2: Trust boundary in git . . . 1631
Case 3: Secret inventory . . . 1631
Case 4: Failure drill list . . . 1632
Case 5: What this capstone is not . . . 1632
The trap . . . 1632
The boring rule . . . 1633
Try this . . . 1633

Disko Storage and Ephemeral Base System **1634**

Disko Storage and Ephemeral Base System **1635**
Mental model . . . 1635
Worked examples . . . 1635
Case 1: The Production Disko Module . . . 1635
Case 2: The Impermanence Persistent State Contract . . . 1637
Case 3: `neededForBoot` is a Stage-1 mount, not a comment . . . 1638
Case 4: Wrong disk, wrong mapper . . . 1639
Case 5: Format is once; mounts are every boot . . . 1639
The trap . . . 1639
The boring rule . . . 1639
Try this . . . 1640

Hardened Networking and WireGuard Mesh **1641**

Hardened Networking and WireGuard Mesh **1642**
Mental model . . . 1642
Worked examples . . . 1642
Case 1: Shared mesh module . . . 1642
Case 2: Database host — Postgres only on `wg0` . . . 1643
Case 3: Gateway — HTTP public, API only over mesh . . . 1644
Case 4: App node does not open 8080 publicly . . . 1644
Case 5: Prove the path . . . 1644
The trap . . . 1645
The boring rule . . . 1645
Try this . . . 1645

Distributed Application Stack and Secrets **1646**

Distributed Application Stack and Secrets **1647**
Mental model . . . 1647
Worked examples . . . 1647
Case 1: Caddy on the gateway . . . 1647
Case 2: Sandboxed API . . . 1648
Case 3: Postgres on `db-01` . . . 1649
Case 4: Eval the unit path . . . 1651
Case 5: Security score . . . 1651
The trap . . . 1651
The boring rule . . . 1651
Try this . . . 1652

Automated Fleet Rollout and Disaster Recovery **1653**

Automated Fleet Rollout and Disaster Recovery **1654**
Mental model . . . 1654
Worked examples . . . 1654
Case 1: Hive for the five-host fleet . . . 1654
Case 2: Canary, probe, then the tag . . . 1655
Case 3: Zero-touch replace of `app-02` . . . 1656
Case 4: Restore `/persist` on `db-01` (not the store) . . . 1657
Case 5: Quarterly drill is the playbook . . . 1657
The trap . . . 1657
The boring rule . . . 1658
Try this . . . 1658

Conclusion: The Path Forward **1659**

Conclusion: The Path Forward **1660**
The Synthesis of Reproducible Systems . . . 1660
The Production Mindset: From Pet Servers to Deterministic Graphs . . . 1661
Overcoming Skepticism and Leading Cultural Adoption . . . 1661
1. Lead with Developer Pain Points (The Trojan Horse) . . . 1661
2. Provide High-Level Abstractions . . . 1662
3. Maintain Absolute Lockfile Discipline . . . 1662

Frontiers of the Nix Ecosystem . . . 1662
1. Content-Addressed (CA) Derivations . . . 1662
2. Flake Schemas and Modern Standards . . . 1662
3. Native MicroVM and Lightweight Virtualization . . . 1662
4. AI Coding Agents and Hermetic Toolchains . . . 1663
The Path Forward: Your Continuing Roadmap . . . 1663

Projects **1665**

Projects index **1666**

Projects: packaging and your own distro **1667**
How to use this part . . . 1667
Learning loop . . . 1667
Lab environment checklist . . . 1668
How chapters are structured . . . 1668
Project path . . . 1668
Track A — Derivations & packaging . . . 1668
Track B — Custom NixOS-based distro . . . 1669
Suggested order . . . 1669
Time and difficulty (rough) . . . 1670
Repository hygiene (all projects) . . . 1670
Cross-links into the main book . . . 1671
Done means (global bar) . . . 1671
Trademark and ethics (read once) . . . 1671
Quick start: create the lab root . . . 1672

Project 1: Hello CLI derivation **1673**

Project 1 — Package a tiny CLI with `stdenv.mkDerivation` **1674**
Why this project exists . . . 1674
Prerequisites . . . 1674
Theory . . . 1675
Derivation vs package vs app . . . 1675
Phases you will skip on purpose . . . 1675
`$out` is the contract . . . 1675
`callPackage` dependency injection . . . 1675
Purity / sandbox . . . 1675
Lab tree . . . 1676
Step 1 — Source: shell tool . . . 1676
Step 2 — Derivation . . . 1676
Optional: clean the source filter . . . 1677
Step 3 — Flake . . . 1677
Step 4 — Build, run, inspect . . . 1679
Step 5 — C variant (stretch, same flake) . . . 1679
Theory checkpoints (self-test) . . . 1679
Pitfalls . . . 1680
Acceptance criteria . . . 1680

Stretch goals . . . 1681
Checkpoint commit message (example) . . . 1681

Project 2: Script tool derivation **1682**

Project 2 — Real script tool with deps and wrap **1683**
Why this project exists . . . 1683
Prerequisites . . . 1683
Theory . . . 1684
Ways to ship script tools with dependencies . . . 1684
What the wrapper actually does . . . 1684
Closure vs references . . . 1684
Build-time vs run-time (again) . . . 1684
Lab tree . . . 1685
Step 1 — Preferred path: `writeShellApplication` . . . 1685
Flake . . . 1686
Build and inspect . . . 1687
Step 2 — Fail closed (required demo) . . . 1687
Step 3 — Alternative: `stdenv` + `wrapProgram` . . . 1687
Step 4 — Optional live mode (network at **runtime**) . . . 1688
Lab tasks . . . 1689
Pitfalls . . . 1689
Acceptance criteria . . . 1689
Stretch goals . . . 1690
Theory recap . . . 1690

Project 3: Language package derivation **1691**

Project 3 — Package a Go or Python app with language helpers **1692**
Why this project exists . . . 1692
Prerequisites . . . 1693
Theory map . . . 1693
Why helpers beat raw stdenv here . . . 1693
Hash discipline . . . 1693
Lab tree (shared idea) . . . 1694
Option A — Go (`buildGoModule`) . . . 1694
A1. Scaffold module . . . 1694
A2. Package expression . . . 1695
A3. Flake . . . 1695
A4. Hash fix loop . . . 1696
A5. Vendor / tests (optional) . . . 1696
Option B — Python (`buildPythonApplication`) . . . 1696
Shared finish . . . 1698
REPORT.md (required) . . . 1698
Pitfalls . . . 1698
Acceptance criteria . . . 1699
Stretch goals . . . 1699
Self-check . . . 1699

Project 4: Overlay pin and patch **1701**

Project 4 — Overlay: pin or patch a package **1702**
Why this project exists . . . 1702
Prerequisites . . . 1703
Theory . . . 1703
Overlay shape . . . 1703
Three layers of change (recap) . . . 1703
Overlay vs separate flake input . . . 1704
Infinite recursion classic . . . 1704
Lab tree . . . 1704
Step 1 — Flake wiring (overlay applied once) . . . 1705
Step 2 — Lab A: message patch on `hello` . . . 1706
A1. Discover the string to patch . . . 1706
A2. Lab-fast path: `postPatch` (allowed for green) . . . 1706
A3. Proper patch path (preferred write-up) . . . 1706
A4. Prove it . . . 1707
Step 3 — Lab B: pin via **extra input** (document, implement lightly) . . . 1707
Step 4 — Optional NixOS consumer + upgrade drill . . . 1708
Gotchas . . . 1708
Acceptance criteria . . . 1709
Stretch goals . . . 1709
Self-check . . . 1709

Project 5: Distro identity and branding **1710**

Project 5 — Name your distro: identity, versioning, branding assets **1711**
Why this project exists . . . 1711
Prerequisites . . . 1712
Theory . . . 1712
Identity surfaces . . . 1712
Version scheme for derivatives . . . 1712
Modules as product API . . . 1713
os-release merge reality . . . 1713
Product sheet (`PRODUCT.md`) . . . 1713
Repo layout . . . 1714
Copy sample branding from this book . . . 1714
Step 1 — Identity module . . . 1714
Step 2 — Lab host + flake . . . 1716
Step 3 — Verify identity . . . 1717
Step 4 — Artwork + consistency . . . 1717
Pitfalls . . . 1718
Acceptance criteria . . . 1718
Stretch goals . . . 1719
Self-check . . . 1719

Project 6: Distro ISO and themes **1720**

Project 6 — Installer ISO with themes (Plymouth / greeter / console) **1721**
Why this project exists . . . 1721
Prerequisites . . . 1722
Theory / architecture . . . 1722
Theme layers (cheap → expensive) . . . 1722
Lab tree (extends Project 5) . . . 1723
Step 1 — ISO host configuration . . . 1723
Step 2 — Themes module . . . 1724
Step 3 — Flake output . . . 1725
Build . . . 1725
Alternative: nixos-generators . . . 1726
Step 4 — Boot test (QEMU) . . . 1726
VirtualBox / UTM . . . 1726
Verification checklist (live session) . . . 1726
Step 5 — Branding matrix . . . 1727
Step 6 — Iterate without tears . . . 1727
Plymouth (optional after minimal ISO is green) . . . 1728
Pitfalls . . . 1728
Acceptance criteria . . . 1728
Stretch goals . . . 1729
Self-check . . . 1729

Project 7: Complete custom distro product **1730**

Project 7 — Ship a complete NixOS-based distro flake **1731**
Why this project exists . . . 1731
Product definition of done . . . 1732
Target tree . . . 1732
Theory: editions vs hosts . . . 1733
Step 1 — Defaults module (flakes-first) . . . 1733
Step 2 — Edition modules . . . 1734
Minimal . . . 1734
Desktop (or server) . . . 1734
Step 3 — Hosts . . . 1735
Demo VM / installed profile . . . 1735
ISO host . . . 1736
Step 4 — Flake product surface . . . 1736
Step 5 — Optional metapackage (toolkit) . . . 1737
Step 6 — Install story (document, then test once) . . . 1737
Step 7 — README + DEMO.md (required) . . . 1738
Step 8 — Channels of truth . . . 1738
Pitfalls . . . 1739
Acceptance criteria . . . 1739
Stretch goals . . . 1739
Capstone reflection (½ page) . . . 1740
Self-check . . . 1740
Done with the projects track . . . 1740

Branding **1741**

Sample distro branding assets (lab only) **1742**
License . . . 1742
Trademark . . . 1742
Wire into a flake . . . 1742

Plymouth / greeter theme notes (lab) **1743**
Goal . . . 1743
Practical path on 26.05 . . . 1743
Custom Plymouth (stretch) . . . 1743
Assets in this folder . . . 1743

Appendices **1744**

Glossary of Terms **1745**

Glossary of Terms **1746**
Derivation (`.drv`) . . . 1746
Nix Store (`/nix/store`) . . . 1746
Closure . . . 1746
Reference scanning . . . 1746
Flake . . . 1746
DevShell . . . 1746
Binary cache / substituter . . . 1746
NAR . . . 1747
GC root . . . 1747
Home Manager . . . 1747
Overlay . . . 1747
Module system . . . 1747
Fixed-output derivation (FOD) . . . 1747
RUNPATH . . . 1747
nix-ld . . . 1747
FHS env (`buildFHSEnv`) . . . 1747
Remote builder . . . 1748
stateVersion . . . 1748
Generation . . . 1748
Impermanence . . . 1748

Command Cheat Sheet **1749**

Command Cheat Sheet **1750**
Flakes and development . . . 1750
Build, run, eval . . . 1750
Inspect closures . . . 1750
Fetchers . . . 1751
Copy and caches . . . 1751
Remote builders . . . 1751

NixOS . . . 1751
Home Manager . . . 1752
GC and disk . . . 1752
Backup (restic) . . . 1752

Nix and NixOS Version Reference **1753**

Nix and NixOS Version Reference **1754**
Nix CLI Releases . . . 1754
Nix 2.18 . . . 1754
Nix 2.19 . . . 1755
Nix 2.20 . . . 1756
Nix 2.21 . . . 1756
Nix 2.22 . . . 1757
Nix 2.23 . . . 1758
Nix 2.24 . . . 1759
NixOS Releases . . . 1760
NixOS 23.05 — Stoat . . . 1760
NixOS 23.11 — Tapir . . . 1761
NixOS 24.05 — Uakari . . . 1762
NixOS 24.11 — Vicuna . . . 1764
Nix 2.28 – 2.34 (bridge) . . . 1765
Nix 2.35 . . . 1765
NixOS 25.05 / 25.11 . . . 1766
NixOS 26.05 — Yarara . . . 1766
Quick-Reference Matrix . . . 1767

Nix and NixOS Version Reference **1768**
Nix CLI Releases . . . 1768
Nix 2.18 . . . 1768
Nix 2.19 . . . 1769
Nix 2.20 . . . 1770
Nix 2.21 . . . 1770
Nix 2.22 . . . 1771
Nix 2.23 . . . 1772
Nix 2.24 . . . 1773

NixOS Releases **1775**
NixOS 23.05 — Stoat . . . 1775
NixOS 23.11 — Tapir . . . 1776
NixOS 24.05 — Uakari . . . 1777
NixOS 24.11 — Vicuna . . . 1778

Nix 2.28 – 2.34 (bridge) **1780**
Nix 2.35 . . . 1780
NixOS 25.05 / 25.11 . . . 1781
NixOS 26.05 — Yarara . . . 1781
Quick-Reference Matrix . . . 1782

Notes on Nix & NixOS

Personal **journey notes** for learning **Nix** and **NixOS** the way practitioners who already live in this ecosystem would sequence the work: early feedback loops, modern defaults, depth when you have something real to debug.

Baseline: Nix **2.35.x** · NixOS / nixpkgs **26.05 "Yarara"** (as of mid-2026).

This is a **normal topic-oriented book** (parts and chapters), not a day-calendar or certification track. Chapter contract: **Syllabus**.

What "done" looks like

- Explain store, derivations, evaluation vs realization, generations, and GC roots in plain language
- Run **one** flake-defined NixOS 26.05 host; break it, roll back, recover
- Prefer **modern CLI + flakes** as the default; recognize classic commands when you meet them
- Separate **system** (NixOS) and **user** (Home Manager) concerns cleanly
- Package something you actually use; write modules with real options
- Keep secrets out of the store; layout disks with Disko when you reinstall
- Deploy beyond one box, use a binary cache and CI, harden without cargo-cult

How the book is organized

Part	Path	Focus
Front matter	`00-front-matter/`	Syllabus, Reproducibility Crisis + **Lab 0**
Concepts	`01-concepts/`	Why Nix, store, language, CLI, flakes
Nix on Linux	`02-nix-on-linux/`	**Fedora 44+** official Nix; dual-stack bridge to NixOS
NixOS host	`03-nixos-host/`	Install, rebuild, users, net, modules, boot

Part	Path	Focus
Home & layout	`04-home-and-flake-layout/`	Flake layout, locks, Home Manager, shells/editors, polyglot devShells, Docker migration
Services & security	`05-services-and-security/`	Secrets (SOPS/Agenix/Vault), hardening, TLS front door, containers
Packaging	`06-packaging/`	stdenv, overlays, caches, language ecosystems, cross-compilation, **ROS 2**
Ops & fleet	`07-ops-and-fleet/`	Disko, deploy, tests, CI, images, backups
Internals	`08-internals/`	Evaluator, module system, upgrades, troubleshooting
Capstone	`09-capstone/`	Rebuildability proof and recovery notes
CI/CD	`10-cicd/`	Binary caches, GitHub Actions, GitLab CI, remote builders, caching
Containers & K8s	`11-containers-and-kubernetes/`	Nix-built container images, K8s modules, Helm charts, Argo/Flux GitOps
IaC & Cloud	`12-iac-and-cloud/`	Terraform/OpenTofu with Nix, Ansible provisioning, multi-cloud
Platform Engineering	`13-platform-engineering/`	Internal developer platform, monorepos, Colmena fleet deploys, MicroVMs
Production Labs	`14-production-labs/`	Infrastructure audits, operator keys, builder factories, channel promotion
Production Capstones	`15-production-capstones/`	Multi-region production infrastructure deployment & Conclusion
Projects	`99-projects/`	Derivations + branded distro (ISO, themes, editions)
Appendices	`100-appendices/`	Master glossary, CLI cheatsheets, Nix language syntax reference

Chapters are **topics**, not "Day 1…Day 90". Read in order when learning; use the sidebar as a reference index when you already know where to jump.

Recommended path if your daily driver is Fedora

1. **Concepts** — store, language, flakes

2. **Nix on Linux** — `dnf install nix` on **Fedora 44+**, profiles, devShells, standalone Home Manager, dnf coexistence

3. **NixOS host** — when ready, same skills apply to a full OS flake

See Nix on Linux — Fedora bridge path.

Lab 0 (recommended before first host work)

If you have a spare machine (example profile: **HP Elitebook 2570p · 16 GB**), set up the lab first:

1. Roles & topology — `01a-lab-overview-and-roles.qmd`

2. Journey single-server (KVM, VMs, containers) — `01b-lab-journey-single-server.qmd`

3. NixOS **dev** machine — `01c-lab-nixos-dev-machine.qmd`

4. Elitebook hardware profile — `01d-lab-elitebook-2570p.qmd`

5. Ops cheatsheet — `01e-lab-ops-cheatsheet.qmd`

How I intend to study

Slice	Share	Purpose
Concept	~25%	Manuals, mental models, *why*
Hands-on	~55%	Configs and builds that can fail
Review	~20%	Refactor modules, commit, write gotchas

Rules that match expert practice:

- **Disposable VM** (or spare disk) for breakage; keep a daily driver safe
- **Flakes + modern CLI by default** from early chapters
- **One change → rebuild → observe** — avoid huge untested diffs
- Budget disk for `/nix/store`
- Pin **26.05** deliberately; upgrade only with a written plan

Prerequisites

- Comfortable Unix shell
- Processes, files, SSH, basic networking
- Willingness to reinstall a VM

Related books

Book	Overlap
Linux	Classic ops literacy before (and beside) NixOS
VCS	Flake repos, CI, and signed history
Networking	Lab networks you may declare on NixOS hosts

Formats

HTML (primary), PDF, and EPUB via the monorepo library portal when built.

Acknowledgments

Path designed as an expert learning sequence for Nix/NixOS. Topic inventory drawn from personal study notes and production practice.

Official docs (nixos.org, nix.dev, release notes) win when notes and reality disagree. No exam-vendor affiliation.

Front Matter

Syllabus

Syllabus — expert path for Nix & NixOS

Status: Canonical outline for this **standalone NixOS book**.
Baseline: Nix **2.34.x** · NixOS / nixpkgs **26.05 "Yarara"**.
Voice: Topic-oriented journey notebook. **Ordering:** expert pedagogy by **theme**—not a day calendar and not a 90-day program.

> **!** Important
>
> Chapters follow **this** topic path. Source documents supply topics and exercises; they do not override the sequence below. There is **no** required joint schedule with any other book or series.

Why this order (expert rationale)

Nix rewards **tight feedback loops**. Long theory blocks before a real rebuild create fragile knowledge. Production NixOS work in 2026 is overwhelmingly **flake-pinned, module-composed, and generation-rollback based**. The path below matches that world.

Principle	What we do	What we avoid
Feedback early	Install Nix early; first NixOS host soon after	Weeks of pure language before any OS
Modern defaults	Flakes + `nix` CLI as default; classic CLI as literacy	Building muscle memory on `nix-env` / channels as primary
Host before packaging craft	Configure and consume packages on a real system first	Deep `stdenv` before you have felt closure size and rebuilds
System then user	NixOS host stable → Home Manager for the user layer	Dotfiles-only Nix that never owns the machine
Just-in-time depth	Module *types* catalog after you have written real modules	A full type catalog before `services.openssh.enable`
Patterns over product zoo	Proxy, secrets, one DB, CI, observability as *patterns*	Required chapters for every self-hosted app
Internals last	Evaluator, daemon, CA derivations after you can operate	Sandbox namespaces before you can roll back a bad config
Prove rebuildability	Capstone = wipe + rebuild from flake	"It worked once on my laptop"

How source notes were used

Source	Role
Personal lab notes	Mini-projects, 26.05 notes, deploy/secrets tool names
Depth checklists	Language, store, module types, HM, troubleshooting
Production syllabus notes	Stack defaults, hardening, multi-node capstone bar

Rejected as primary structure: day calendars; flakes-before-any-OS schedules; long language-only front loads; "install every service" tables; cert/exam framing; cross-links to series volumes.

Other languages (including Go)

This book does not teach Go or any application language. At packaging stages, any real binary you care about is a valid target. Do not stall NixOS waiting for another curriculum.

Modern stack (defaults for this book)

Layer	Default choice
Pinning	**Flakes** + `flake.lock`
CLI	Modern `nix` (`build`, `run`, `develop`, `flake`, `shell`, `profile`)
System	NixOS modules + `nixos-rebuild` / `nixos-rebuild --flake`
User	**Home Manager** as NixOS module (standalone when on non-NixOS)
Disks	**Disko** when reinstalling or provisioning
Secrets	**sops-nix** (primary); **agenix** compared once
Deploy	**deploy-rs** (single/simple) then **Colmena** (fleet)
Images	**nixos-generators**
Cache	cache.nixos.org + **Cachix** or self-hosted **Attic**/**Harmonia**
Boot security	**Lanzaboote** / Secure Boot when hardware allows
Tests	**nixosTest** + flake `checks`

Classic `nix-*` commands appear for reading older docs and mental models—not as the preferred daily driver.

Part map (topic book)

This **standalone NixOS book** is a **normal topic-oriented** manuscript. Parts and chapters are **not** a day calendar. Study pacing is yours.

Part	Directory	Theme	You can leave when…
Front matter	`00-front-matter/`	Syllabus + Introduction + Lab 0	Lab topology is clear; Elitebook/KVM ready; reproducibility crisis framed
Concepts	`01-concepts/`	Why Nix, store, language, CLI, flakes	You can explain a store path and use a flake devShell
Nix on Linux	`02-nix-on-linux/`	**Fedora 44+** packaged Nix; dual-stack; HM standalone; bridge	You run flakes/HM on an existing distro and know what NixOS still adds
NixOS host	`03-nixos-host/`	Install, rebuild, users, net, modules	You rebuild and roll back a flake host
Home & layout	`04-home-and-flake-layout/`	Layout, locks, Home Manager, shells/editors, polyglot devShells	System + user config + dev environments live cleanly in flakes
Services & security	`05-services-and-security/`	Secrets (SOPS, Agenix, Vault), hardening, proxy, containers	A small server slice is declarative, secret-safe, and audited
Packaging	`06-packaging/`	stdenv, overlays, ecosystems, cross-compilation, caches	You ship native and cross-compiled packages from source
Ops & fleet	`07-ops-and-fleet/`	Disko, deploy, tests, CI	You can deploy and test beyond one laptop
Internals	`08-internals/`	Evaluator, modules, upgrades	You can debug eval/store issues and plan upgrades

Part	Directory	Theme	You can leave when…
Capstone	`09-capstone/`	Rebuildability	Wipe/rebuild proof (+ optional multi-node bar)
CI/CD	`10-cicd/`	Binary caches, GitHub Actions, GitLab CI, remote builders, caching	Automated hermetic build & test pipelines push trusted caches
Containers & K8s	`11-containers-and-kubernetes/`	Nixhelm, container images, K8s modules, Helm charts, Argo/Flux GitOps	Declarative containers & GitOps pipelines without Dockerfile drift
IaC & Cloud	`12-iac-and-cloud/`	Terraform/OpenTofu with Nix, Ansible provisioning, multi-cloud	Provisioning and cloud state managed through deterministic definitions
Platform Engineering	`13-platform-engineering/`	Internal developer platform, monorepos, Colmena fleet deploys, MicroVMs	Enterprise scale platforms operate with automated maintenance & checklists
Production Labs	`14-production-labs/`	Infrastructure audits, operator keys, builder factories, channel promotion	Real-world infra patterns extracted and hardened
Production Capstones	`15-production-capstones/`	Multi-region production infrastructure deployment & conclusion	Automated fleet rollout, disaster recovery drills & the path forward
Projects	`99-projects/`	Derivations + distro branding	Hands-on artifacts exist
Appendices	`100-appendices/`	Master glossary, CLI cheatsheets, version reference	Reference tables at hand

Suggested mix when learning: ~25% concept / ~55% hands-on / ~20% review. Skip freely when using the book as a reference.

Front matter — Foundations & Lab 0 (before Concepts)

Chapter	Purpose
Introduction: The Reproducibility Crisis	The Heisenbugs that cost teams, why traditional tooling fails, and the declarative immutable alternative
Lab overview & roles	Host vs journey guest vs dev role
Journey single-server	KVM/libvirt (default) and optional Incus
NixOS dev machine	Editor, flakes, SSH, caches, drift discipline
Elitebook 2570p profile	Concrete 16 GB RAM/CPU/disk/BIOS plan
Ops cheatsheet	Daily commands

Exit criteria for Lab 0: spare machine boots; virt enabled; at least one NixOS VM; wipe boundaries known; git holds host/guest intent.

Concepts — Foundations

Goal: Correct mental model + modern toolchain before owning a machine.

Focus	Expert emphasis	Hands-on
Why Nix exists	Dependency hell, snowflake servers, honesty about limits; language vs package manager vs NixOS	Install Nix (multi-user); enable flakes + new CLI; first `nix repl`
Store & closures	Content addressing, store paths, GC roots at a glance	Inspect a package closure; find what keeps it alive
Language: values & attrsets	Types, `let`, nested sets, `inherit` — **only what you need to read nixpkgs**	Model a small config as an attrset
Language: functions	`{ a, b ? d }:`, `@`, ellipsis, composition	Write helpers used later in a flake
Laziness, imports, `lib`	Imports are eager, values lazy; `lib` as standard library	Multi-file expression; one `lib` pipeline
Derivations as idea	Evaluation vs realization; `.drv` vs outputs; sandbox *why*	`nix build` something tiny; read `nix log`
Modern CLI fluency	`build` / `run` / `develop` / `shell` / `flake show`	Prefer modern CLI for all tasks
First flake (project)	`inputs` / `outputs` / lock; **devShell before OS**	Flake with `devShells.default` for a real project
Classic CLI literacy	Map old → new (`nix-build`, `nix-shell`, channels as history)	Same task once classic, once modern
Gate	Foundations review	Pin nixpkgs; recreate shell on a clean machine/VM user

Depth to weave in (do not front-load a long language-only block): conditionals/operators, `with` pitfalls, string interpolation, type checks, fixed-point *awareness*, FOD concept, sandbox/substitute vocabulary.

Exit criteria: Explain store path + lock file; `nix develop` works offline after first populate; no dependency on channels for your project.

Nix on Linux — Fedora bridge (recommended before host cutover)

Goal: Master the **package manager on a normal Linux daily driver**—especially **Fedora 44+**, which ships Nix officially (`dnf install nix`, flakes OOTB, `/nix` + SELinux story)—so NixOS later is "machine modules + boot generations," not a total reboot of your mental model.

Focus	Expert emphasis	Hands-on
Why Nix-on-Linux first	Split OS owner (dnf) vs Nix owner (store/flakes/HM)	Write a dual-stack charter
Fedora 44 install	Multi-user daemon; single-user/rootless/Atomic notes	`dnf install nix`; `systemctl enable nix-daemon`; `nix run nixpkgs#hello`
Store, profiles, GC	Same roots model as NixOS minus bootloader	Thin profile; `nix store gc`; inspect roots
CLI + devShells + direnv	Project flakes as SoT	Real repo with `flake.lock` + `use flake`
Home Manager standalone	User generations without NixOS	`home-manager switch --flake`
dnf coexistence	PATH policy; no dual services	Label tools dnf vs Nix
SELinux / Atomic / ops	Stay enforcing; bind-mount `/nix` when needed	Day-2 daemon + disk checklist
Bridge to NixOS	Port HM; add `nixosConfigurations`	VM plan + monorepo sketch
Gate	Fedora path solid	Ready for `03-nixos-host` without relearning flakes

Exit criteria: Flakes + HM work on Fedora after reboot; you can explain what NixOS still adds (modules, system generations, boot).

Skip only if you already live on NixOS or will only use disposable NixOS VMs and never a foreign distro—still skim the bridge chapter for dual-machine layouts.

NixOS host

Goal: Generations, rollback, and "the module system as a user" before authoring a framework.

> Tip
>
> **Expert default:** Install and manage the first host **with flakes from the first NixOS install**. Do not invest weeks in a non-flake `configuration.nix` only to rewrite it.

Focus	Expert emphasis	Hands-on
Install	VM/spare disk; UEFI; **Disko intro** or manual once; generate hardware config	Bootable NixOS 26.05 lab
Flake host skeleton	`nixosConfigurations`; `nixos-rebuild switch --flake .#host`	First flake-defined rebuild
Rebuild modes & generations	switch / test / boot / build; boot menu rollback	Deliberate break → roll back
Users & access	Declarative users; **SSH keys only** for remote	Non-root admin pattern
Networking & firewall	networkd vs NM; open only what you need	Static or known DHCP + SSH
Packages & programs	`environment.systemPackages` vs `programs.*` modules	Install daily tools declaratively
systemd via Nix	One custom service + timer	Module snippet that generates units
Modules as consumer	Read `options` + source for 2–3 nixpkgs modules; `mkIf` / `mkDefault`	Change behavior via options, not forks
First custom module	`options` + `config`, imports, one assertion	Extract something into a module
Boot & hardware	systemd-boot vs GRUB; kernel packages; firmware	Document your boot path
Filesystems intro	Mounts; state vs store; impermanence *idea* only	Label what must persist
Gate	Minimal production-shaped host	Users, SSH, firewall, one service, one custom module — all in flake

Defer later: full Disko redesign, multi-host, secrets machinery, deep type catalogs.

Exit criteria: From cold boot you can roll back; flake is the only source of truth for the lab host.

Home Manager and flake layout

Goal: Structure that scales past one file; user environment without polluting system closure carelessly.

Focus	Expert emphasis	Hands-on
Flake layout	Split `hosts/` / `modules/` / `homes/`; avoid 2k-line `flake.nix`	Refactor host work into layout
Lock discipline	Selective `flake update`; pin strategy; registries lightly	Update one input only; explain lock diff
Home Manager on NixOS	**HM as NixOS module** first (one activation story)	User programs: shell, git, ssh
HM files & XDG	`home.file`, `xdg.configFile`; theming only if you care	Migrate real dotfiles
Shell & editor configuration	Shells (bash/fish), starship, tmux/zellij, neovim/helix LSP	Zero-state declarative terminal & editor
HM services	User systemd timers/services	One user service you actually want
Dev shells + direnv	Project shells *outside* systemPackages	`use flake` on a repo
Multi-language dev environments	Polyglot Go, Rust, Python, Node, C native headers in mkShell	Isolated toolchains without global managers
Docker Desktop to devShells	VM overhead vs native store isolation; decoupled OCI services	Native execution speed with zero idle RAM tax
flake-parts (or discipline)	Reduce boilerplate **or** hand-roll cleanly — pick one style	Consistent multi-output flake
Checks	`nix flake check`; format/lint as checks	Fail CI-shaped check locally
Overlays (light)	Pin or patch one package for the host	Overlay consumed by host + shell
Gate	Workstation/server skeleton	Host + HM + devShell + checks in one flake

HM depth (across HM chapters): programs modules, file permissions/templating, XDG, multi-machine user splits as *pattern*.

Exit criteria: New machine of same role = clone flake + hardware config + rebuild.

Services and security

Goal: Operate services like production without leaking secrets into the store.

Focus	Expert emphasis	Hands-on
Secret problem	Why strings in Nix are dangerous; store world-readable truth	Find a "secret" anti-pattern and remove it
sops-nix	Encrypted-in-repo, decrypt at activation	One secret end-to-end
agenix comparison	Age keys; trade-offs; pick a **house default**	Same secret second way once
HashiCorp Vault	AppRole auth, dynamic short-lived DB leases, LoadCredential	Centralized audit logs & zero-downtime rotation
Hardening baseline	SSH, sudo, unused services off, kernel defaults carefully	Apply a small hardening module
Firewall discipline	Per-service ports; deny by default	Audit open ports
TLS front door	ACME + Caddy **or** Nginx — one reverse-proxy pattern	HTTPS to a static or app backend
Data service pattern	PostgreSQL **or** other **one** DB — modules, state dirs, backups *thought*	Declarative DB + app user
nixos-containers / OCI	When containers help on NixOS; not "Docker first"	One declarative container or nspawn
Podman/Docker modules	Rootless notes; prefer OCI module over snowflake	Compare to NixOS containers
Observability pattern	Prometheus + Grafana **or** minimal logs first	Metrics for one service
Optional: k3s lite	Only if you need k8s; single-node	Skip cleanly if not
Gate	Lab slice	Proxy + secret + one app/data path + firewall story

Not required: Matrix, Nextcloud, Jitsi, full "homelab app of the day" lists. Revisit after Capstone if you want.

Exit criteria: Repo has no live secrets in plaintext; slice rebuilds on a fresh VM from flake + keys.

Packaging craft

Goal: Produce software, not only consume it—after you know how packages feel on a host.

Focus	Expert emphasis	Hands-on
stdenv phases	unpack→install→fixup; hooks; build vs native inputs	Package a small C/shell tool properly
Fetchers & FODs	Hash mismatch workflow; `fetchFromGitHub`	Package from a tag
Language ecosystem A	Your primary language helper (`buildGoModule`, etc.)	Package a tool you use
Language ecosystem B	Second ecosystem for pattern transfer	Smaller package
Patching & overrides	`overrideAttrs`, patches, overlays composition	Fix or pin something broken
Multiple outputs	Closure size discipline	Split out/dev/doc
Debug builds	`nix log`, `--keep-failed`, `breakpointHook`	Break and recover
nixpkgs layout	`by-name`, conventions, meta	PR-shaped package (submit optional)
Binary caches	Push/pull; trust/signatures awareness	Private or Cachix push
Remote builders	Offload when useful	Second VM as builder
Cross-compilation	`buildPlatform` vs `hostPlatform`, `pkgsCross`	Compile ARM64/RISC-V from x86 without QEMU
ROS 2 / nix-ros-overlay (elective)	Overlay pin, `rosPackages.*`, underlay `buildEnv`, caches	Flake `devShell` + talker/listener; colcon workspace
Reproducibility honesty	What is and is not bit-for-bit	Clean rebuild comparison
Gate	Package + cache	Flake output used by your host or CI

Exit criteria: Someone else (or future you) builds your package from lock alone.

Ops and fleet

Goal: Provision and deploy like infra, not like a pet SSH session.

Focus	Expert emphasis	Hands-on
Disko for real	Declarative disks on reinstall/cloud	Install/reinstall with Disko
Impermanence (optional commit)	Persist deliberately	Prove reboot drops unpersisted state **or** document why you skip
deploy-rs	Simple push deploy	Deploy lab to remote VM
Colmena	Tags, multi-host, parallel	Two hosts (one may be local)
Shared modules	Roles vs hosts; no copy-paste drift	`roles/web.nix` pattern
Images & generators	Cloud image / custom ISO	Boot generated image
Terraform boundary	TF provisions, Nix configures	Sketch only is enough
nixosTest	VM tests for *your* modules	Test proxy or hardened SSH module
More tests	Expand suite; flake checks in CI shape	Several meaningful tests
CI pipeline	GHA/GitLab + Nix installer; cache write	PR build of flake
GC & store hygiene	Roots, retention, optimise	Policy + measured store size
Secure Boot / TPM path	Lanzaboote / LUKS unlock — hardware permitting	Document supported path or lab limits
Backup story	Borg/Restic/ZFS — pick one; declare it	Restore drill of *state*
Gate	Remote lab	Flake deploy + CI + cache + test green

Exit criteria: New host of an existing role is boring.

Internals and currency

Goal: Debug and upgrade without superstition.

Focus	Expert emphasis	Hands-on
Evaluator	Thunks, infinite recursion patterns	Trace a tricky eval
Module system deep	Full **type catalog**; `mkMerge` / priorities	Redesign one module with proper types
Store & daemon	Substituters, signatures, sandbox namespaces	Explain roots + one sandbox log
Performance	`max-jobs` / cores / eval cost	Speed a full rebuild with measurement
CA derivations	Experimental awareness	Optional experiment or status memo
nixosTest mastery	Flaky tests, networking in tests	Harden test suite
Ecosystem map	devenv, Snowfall, nix-darwin — **try one**, don't collect all	Written comparison to your layout
Comparative landscape	Guix / image-based distros / Ansible — trade-offs	Honest scorecard + essay
26.05 release notes	Primary source, not blogs	Annotate impact on *your* flake
Upgrade planning	26.05 → next release dry-run	Written upgrade runbook
Troubleshooting drill	Boot, build, service, network matrix	Fix injected failures
Architecture review	Map prior parts onto design	Gap list before Capstone

Exit criteria: You can diagnose eval/store issues and produce IMPACT + upgrade artifacts.

Capstone and recovery

Capstone A — required

Build (or finish) a **flake-defined lab** that includes:

1. At least one NixOS host role you understand end-to-end
2. Home Manager for your user (if workstation) **or** minimal service user story (if server-only)
3. Secrets via sops-nix **or** agenix (house default)
4. One reverse-proxy or equivalent edge pattern
5. Deploy path (deploy-rs and/or Colmena)
6. At least one `nixosTest` or flake check that would catch a real regression
7. CI that builds and preferably pushes a cache
8. **Disaster recovery:** wipe lab disk/VM; rebuild from flake + keys alone
9. Rollback drill + full test suite + troubleshooting checklist
10. Retrospective: teach plan + roadmap (next release, nixpkgs PR, optional Capstone B)

Capstone chapter theme	Focus
Scope and plan	Close structural P0s; host skeleton
Implement	Secrets + proxy E2E
Harden and document	Deploy + CI + cache
Prove rebuildability	Tests + runbook + diagrams
Optional multi-node / freeze	Feature freeze tag; optional second node
Disaster recovery	Timed wipe + rebuild
Rollback and tests	Generation rollback + suite green
Retrospective	Teach plan + roadmap; no rewrite thrash

Capstone B — stretch

Only after A:

- 3 nodes, Colmena + Disko
- LUKS (+ TPM if available)
- Secure Boot / Lanzaboote if feasible
- Fleet observability
- CI → cache → deploy automation

Projects (optional depth track)

Track	Projects
Packaging	Hello CLI → script wrap → language package → overlay
Distro product	Identity → ISO/themes → complete multi-edition flake

Projects sit after packaging theory (or alongside late packaging). They force **demoable artifacts**, not only reading.

Review habit

Every substantial study block:

1. Refactor modules (DRY, naming, options)
2. Re-read one official manual page you used
3. Write 5 “gotchas” in your own words
4. Delete or quarantine experiments that taught something but should not ship

---

Completeness audit (sources → parts)

Theme	Part(s)
Philosophy, reproducibility crisis, store, GC, sandbox	Front matter, Concepts, Internals
Language depth	Concepts (JIT), Internals types revisit
Modern + classic CLI	Concepts
Flakes, locks, checks	Concepts, Home & layout
First host, rebuild, boot, net, users	NixOS host
Module system practice → deep types	Host, Internals
Home Manager depth, shell/editor configs	Home & layout
Polyglot dev shells, direnv, Docker migration	Concepts, Home & layout
Secrets (SOPS, Agenix, Vault), hardening, proxy, DB pattern	Services & security
Containers / optional k3s	Services & security
Packaging, ecosystems, cross-compilation, overlays, caches	Packaging
Disko, deploy, images, CI, tests	Ops & fleet
Secure Boot / TPM / Lanzaboote	Ops, Capstone stretch
Internals, upgrades, ecosystem	Internals
Capstone rebuild proof	Capstone
CI/CD automation & cache distribution	CI/CD
Container images (dockerTools) & Kubernetes GitOps	Containers & Kubernetes
Terraform/OpenTofu integration & Ansible provisioning	IaC & Cloud
Internal developer platform, monorepos, MicroVMs	Platform Engineering
Production audits, operator keys, builder factories	Production Labs
Multi-region fleet capstones, recovery & path forward	Production Capstones
Custom distro product & derivations	Projects
App product zoo	**Out** (optional post-book)
Cert tracks	**Out**
Day calendars / series cross-links	**Out**

Explicitly out of scope

- Go (or other language) full curriculum — separate books
- Certification branding or exam prep
- Exhaustive self-hosted app catalog as required chapters
- Replacing nixos.org / nix.dev
- Required pacing as “Day N”

Resources (primary, not blogs-of-2022)

- *Nix Pills* — store/derivation intuition (Concepts)
- *nix.dev* — maintained tutorials (Concepts–layout)
- Official NixOS & Nix manuals + **26.05 release notes**
- *NixOS & Flakes Book* (community) — companion for flake layouts
- Upstream READMEs: sops-nix, agenix, Disko, deploy-rs, Colmena, Lanzaboote

---

Writing rule for future chapters

1. Follow **parts in pedagogical order** when learning linearly.
2. Each chapter: mental model → minimal working flake/config → break it → gotchas.
3. Prefer the **default stack** table unless hardware forces a fork (document the fork).
4. After content moves: `bash scripts/update-index.sh` only — never hand-edit `_quarto.yml`.
5. Syllabus changes first if scope changes.
6. **Topic book language only**—no day-calendar framing.

Lab hardware default (this book)

Item	Default
Primary lab chassis	HP Elitebook 2570p · 16 GB (or similar spare laptop)
Hypervisor	libvirt/KVM default; Incus optional
Journey guest	`nixlab` 4–6 GB
Deploy target	`deploy-a` ~2 GB optional
Host OS	NixOS 26.05 preferred

See Lab 0 chapters for concrete commands, failure modes, and acceptance criteria.

Introduction: The Reproducibility Crisis

Introduction: The Reproducibility Crisis

Goal: Establish the foundational rationale for reproducible infrastructure, diagnose the systemic failures of traditional imperative tooling, explain the declarative immutable model of Nix and NixOS, and outline the end-to-end journey from zero to production systems at scale.

> **Note**
>
> This chapter forms the conceptual introduction to **From Zero to Production — Reproducible Systems at Scale**. It frames the engineering challenges that plague modern software teams and sets the standard for how Nix eliminates entire classes of infrastructure failure.

The Heisenbugs That Cost Teams

Every engineering organization has paid the reproducibility tax. It manifests not as a single dramatic outage, but as an endless accumulation of invisible friction, wasted sprints, and mysterious bugs that vanish when observed in isolation:

1. **"It Works On My Machine":** A developer builds a backend service locally on macOS. It passes unit tests and runs cleanly. When pushed to the staging CI runner running Ubuntu, it crashes with a cryptic dynamic linker error: a transitive C dependency (`libssl` or `libsqlite3`) resolved to a newer patch release on the developer workstation via a global Homebrew install, while the CI container had an unpatched distribution library.
2. **The Snowflake Production Node:** A critical production server has been running for 412 days. It has received dozens of ad-hoc hotfixes, imperative security patches (`apt-get install --only-upgrade`), manual configuration edits in `/etc`, and temporary environment variables set during past incidents. When the instance fails or needs to be scaled horizontally, no automation script can reproduce its exact state. The machine is a snowflake: precious, fragile, and impossible to duplicate.
3. **The Silent Upstream Deprecation:** A build pipeline that ran flawlessly on Friday fails on Monday morning without any code changes in the repository. An unpinned transitive dependency in a language ecosystem (npm, pip, or crates.io) released a minor version that silently broke backward compatibility or changed dynamic resolution flags.
4. **Agentic Sandboxing Collisions:** In the era of autonomous AI coding agents and automated build loops, non-hermetic environments lead to catastrophic drift. When an agent attempts to resolve missing dependencies by executing `sudo apt-get install` or `pip install --break-system-packages` inside an ephemeral container, it pollutes global state, creates non-deterministic side effects, and produces builds that fail outside its isolated session.

These failures are classic **Heisenbugs**—software anomalies that alter their behavior or disappear entirely when investigated under different environmental conditions. They consume up to 30% of engineering bandwidth across industry teams, turning operations into forensic archeology.

```
Traditional Imperative Drift:
Host A [Arch Linux]   --> gcc 14.1 + glibc 2.39 + libssl 3.2
Host B [Ubuntu 24.04] --> gcc 13.2 + glibc 2.39 + libssl 3.0    > Non-deterministic outcomes
Host C [macOS Sonoma] --> clang 16  + libSystem + Homebrew ssl    > runtime linker crashes,
Host D [CI Runner]    --> gcc 12.3 + glibc 2.35 + libssl 3.0      and silent build divergenc
```

Why Traditional Tooling Fails

The software industry has attempted to solve the reproducibility crisis with successive waves of tooling: virtual machines, configuration management engines, and container images. While each provided incremental isolation, none addressed the fundamental root cause: **imperative mutation of shared state**.

Tooling Class	Representative Tools	Underlying Mechanism	Why It Ultimately Fails
Configuration Management	Ansible, Puppet, Chef, Salt	Sequential execution of stateful tasks against a running host	Order-dependent side effects; non-idempotent package managers; cannot guarantee unmanaged files are removed; drift between runs
Containers (Docker/OCI)	Docker, Podman, Buildah	Layered filesystem snapshots on top of a mutable base image	`RUN apt-get update && apt-get install` fetches whatever happens to be current on upstream mirrors at build time; non-hermetic timestamps; layer caching obscures dependency changes
Virtual Machine Images	Packer, Vagrant, AMI builders	Golden image baking of virtual disk files	High storage cost; slow feedback loops; opaque binary artifacts where tracing a single binary back to its exact source and compiler flags is nearly impossible

Tooling Class	Representative Tools	Underlying Mechanism	Why It Ultimately Fails
Language Package Managers	Pip, Npm, Cargo, Go Modules	Local lockfiles resolving packages to language directories	Only manage application dependencies; completely blind to system-level C libraries (`glibc`, `openssl`, `pkg-config`, `cuda`, `libz`)

The Container Illusion

Containers are often heralded as the ultimate solution for reproducibility. In practice, a standard `Dockerfile` is merely a shell script executing imperative mutations inside a Linux namespace:

```
# A standard Dockerfile: non-reproducible by design
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y \
    curl \
    python3 \
    python3-pip \
    libpq-dev
RUN pip install -r requirements.txt
COPY . /app
CMD ["python3", "/app/server.py"]
```

Building this Dockerfile on January 1st and rebuilding it on June 1st yields two fundamentally different binary artifacts: - The package mirrors have updated `ubuntu:24.04` packages with security releases and new dynamic libraries. - The `pip` resolver may pull different sub-dependencies if wheels or source distributions were modified upstream. - File modification timestamps (`mtime`) and inode orders vary across machines, breaking cryptographic hash verification. - The container shares the host kernel ABI; differences in kernel capabilities, seccomp filters, and cgroups can alter runtime behavior.

Containers solve **packaging and distribution**, but they do **not** solve **hermetic, bit-reproducible construction**.

A Different Approach: Declarative, Immutable Infrastructure

Nix takes a radical departure from the imperative tradition. Rather than treating a computer as a stateful machine to be altered through a sequence of imperative steps, Nix treats software construction and system configuration as a **purely functional evaluation**.

The Three Tenets of the Nix Model

1. **Complete Declarative Purity:** Every package, service, user account, firewall rule, and kernel parameter is declared in a purely functional, lazy domain-specific language (the Nix expression language). The entire configuration of an operating system is an explicit data structure evaluated from source code.
2. **Cryptographic Immutability (The `/nix/store` Model):** Packages are never installed into global directories like `/usr/bin` or `/usr/lib`. Instead, every build output lives in a globally unique, read-only path in `/nix/store` prefixed by a cryptographic hash of all its inputs:

   ```
   /nix/store/7v0g18d451yq8b3k4c6z8w9a0b1c2d3e-nginx-1.26.1/
   ```

   If any input changes—a source patch, a compiler flag, a dynamic library dependency, or an environment variable—the cryptographic hash changes. Different versions of the exact same software coexist on disk without conflict.
3. **Atomic State Transitions and Instant Rollbacks:** Because store paths are immutable and never overwritten in place, upgrading a system or application simply creates a new generation of symlinks pointing to newly realized store paths. An update is an atomic symlink swap. If an update fails, a single command or boot menu selection reverts to the previous generation instantly:

```
Generation 41 (Active):
/run/current-system  > /nix/store/a1b2c3d4-nixos-system-prod-26.05/

Generation 42 (Candidate Rebuild):
[Atomic Switch]
/run/current-system  > /nix/store/e5f6g7h8-nixos-system-prod-26.05/

                              [Failure detected! Instant Rollback to Gen 41]
```

The Shift from Imperative Ops to Mathematical Determinism

In the Nix paradigm: - **Builds are Pure Functions:** $Output = f(Inputs)$. A build runs in an isolated network-isolated sandbox namespace where it cannot access arbitrary files or the internet. - **Dependency Graphs are Complete:** If a dependency is not explicitly declared as an input, it does not exist in the build environment. There are no undeclared ambient dependencies. - **Single Source of Truth:** Flakes (`flake.nix` and `flake.lock`) provide bit-for-bit cryptographic pinning of all inputs across developer machines, CI/CD pipelines, and cloud production servers.

What This Book Will Teach You

This book is an engineering guide to mastering Nix and NixOS from zero to production systems operating at enterprise scale. You will not find toys, ephemeral hacks, or superficial blog recipes here. We build real, production-hardened infrastructure using modern tools: **Nix 2.35+**, **Flakes**, and **NixOS 26.05**.

Key Milestones on Your Path

- **Part 1 & 2: Foundations and Modern Tooling**
  Deconstruct the Nix store, cryptographic hashes, closure graphs, and the Nix language from basic syntax to recursive attribute sets and functional abstractions.
- **Part 3 & 4: Reproducible Workstations and Developer Experience**
  Build hermetic, multi-language developer shells (`devShells`), automate directory-based activation with `direnv`, manage dotfiles declaratively with Home Manager, and migrate away from bloated local container VMs.
- **Part 5 & 6: Production Packaging and Cross-Compilation**
  Master `stdenv` phases, fixed-output derivations (FODs), language builders for Go, Rust, Python, and C/C++, and cross-compile binaries for foreign architectures (AArch64, RISC-V) without hardware emulation.
- **Part 7 & 8: Declarative System Engineering (NixOS)**
  Architect production NixOS hosts from scratch using the module system, manage systemd services, configure secure networking, and handle disk provisioning with Disko and ephemeral roots.
- **Part 9 & 10: CI/CD, Enterprise Secrets, and Cloud Platforms**
  Deploy binary caches (Harmonia, Attic, Cachix), build automated GitHub Actions and GitLab CI pipelines, integrate HashiCorp Vault and SOPS for zero-leak secrets management, and declare Terraform/OpenTofu configurations.
- **Part 11 & 12: Distributed Infrastructure and Scale**
  Generate minimal, unbloated OCI container images with `dockerTools`, manage Kubernetes manifests and GitOps pipelines with Argo CD, deploy fleets with Colmena, and architect internal developer platforms.
- **Production Capstones:**
  Prove rebuildability by wiping machines to bare metal and restoring them completely from a single Git repository and cryptographic keys.

By the end of this book, you will no longer debug snowflake servers or battle "works on my machine" anomalies. You will build and operate mathematical, verifiable, and rock-solid systems that scale effortlessly.

Lab 0 — Overview: roles before the Why Nix exists chapter

Lab 0 — Overview: roles before the Why Nix exists chapter

Before the Why Nix exists chapter. Do **not** skip this if you will follow the full journey on real hardware. The curriculum assumes you can **break and rebuild** machines without fear. That only works if lab roles are clear.

Why a lab host exists

NixOS learning fails when:

- The only machine is your daily driver and every `nixos-rebuild` is scary
- Disk fills because `/nix/store` and VM images share a tiny root
- Nested "VM inside VM inside laptop on battery" becomes the whole evening

A **single spare machine** (your case: HP Elitebook 2570p · 16 GB RAM) is enough for the entire path—if you treat it as a **lab server**, not a casual browser laptop.

Philosophy: host is not an appliance

Homelab hypervisors like Proxmox are excellent—and often **GUI-first**. You *can* automate them (CLI, API, Terraform, Ansible), but day-to-day debugging still tempts **one-off imperative fixes** that never return to git. That drift is annoying for humans and messy if an agent runs dozens of shell "fixes" that leave the machine unreproducible.

This curriculum optimizes for a different contract, in the spirit of operators who moved **Proxmox → NixOS + Incus** (see Bas Nijholt, *I've gone full Nix*):

Idea	What it means for Lab 0
Git is the source of truth	Host networking quirks, hypervisor packages, firewall ports live in the flake—not only in shell history
Host is fully yours	NixOS-as-host is not a locked appliance: you *may* run tools on metal, but **journey breakage stays in guests**
CLI-first ops	Prefer `virsh` / `incus` / `nixos-rebuild` over click-only workflows so every step is copy-pasteable
Simulate before bare metal	Prove a config in a VM that mirrors the host role *before* you reinstall the Elitebook

Idea	What it means for Lab 0
Hardware fixes are modules	NIC offload bugs, firmware flags, microcode → commented Nix + systemd, not "I ran ethtool once in 2019"

Proxmox remains fine software. For **this** guide we default to **NixOS on metal** so one language describes host + guests.

Three roles (do not collapse them)

Role	What it is	Mutability
Lab host (hypervisor)	Metal OS: KVM/libvirt and/or **Incus**, storage, networking, maybe Podman	Stable; rare rebuilds; **fully declarative**
Journey guest(s)	NixOS VMs (or Incus VMs) you rebuild, break, roll back (the installing NixOS chapter+)	**Disposable**
Dev environment	Where you edit flakes, run `nix` CLI, commit git	Stable tools; can be host *or* a dedicated VM

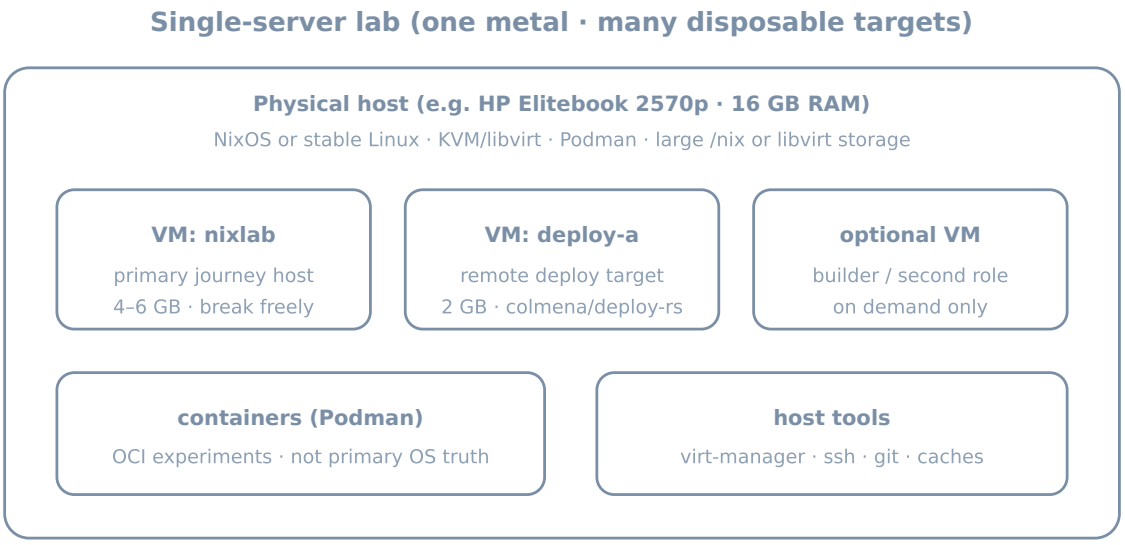


Figure 1: Single-server lab layout

Chapters in this section

Chapter	Purpose
Lab 0A — Journey single-server	One metal box: **libvirt and/or Incus**, containers, storage, simulate-before-metal
Lab 0B — NixOS dev machine	Editor, flakes, SSH, caches, drift discipline
Lab 0C — Elitebook 2570p profile	Concrete RAM/CPU/disk/BIOS plan for **your** spare HP
Lab 0D — Ops cheatsheet	Daily commands

You can run **0A and 0B on the same laptop** (recommended for 16 GB): host = stable NixOS with hypervisor; guests = journey; host also has Nix + editor for flakes.

Recommended topology for this curriculum

```
Elitebook 2570p (16 GB)
  Host OS (NixOS 26.05 preferred)     ← fully yours; flake-managed
    Hypervisor (pick one primary)
      libvirt/KVM  (default for this guide)
      and/or Incus (CLI-first VMs + system containers; optional)
        nixlab     4-6 GB   ← primary journey machine
        deploy-a   2 GB     ← deploy-rs / Colmena target
        (optional builder)  on demand
    Podman (rootless)             ← OCI days
    ~/src/nix-config              ← git = source of truth
    optional: Attic/Cachix
  Do NOT: run k3s multi-node + desktop + 3 heavy VMs all day
```

Decision: host OS

Host choice	Pros	Cons
NixOS host	One mental model; host reclaimable via generations; great for Lab 0B	Early days need care not to smash host
Debian/Ubuntu host	Familiar if you are not ready	Two worlds (apt host + NixOS guests); more drift risk

Expert default: NixOS as host with strong separation—host flake boring and pinned; guests are the playground.

Hypervisor choice (libvirt vs Incus)

Stack	Strength	Use when
libvirt + virt-manager	Familiar, great snapshots, virt-install simple	**Default** for this curriculum on 16 GB
Incus (LXD community fork)	CLI-first VMs + system containers; strong for pets + automation	After host is solid, if you want one CLI surface
Both	Possible but heavy	Avoid on Elitebook unless you know why

Lab 0A documents **libvirt fully** and **Incus as an optional path** (aligned with Proxmox→NixOS+Incus style migrations).

What "ready for the Why Nix exists chapter" means

- ☐ Spare machine boots reliably; disk has free space (see Lab 0C)
- ☐ Virtualization enabled in BIOS; at least one NixOS VM can boot
- ☐ You can SSH into the VM from the host (or use console)
- ☐ You know which disk/VM you are allowed to wipe
- ☐ Dev tools path chosen (Lab 0B): editor + `nix` + git on host or dev VM
- ☐ Host (and guest) config lives in **git**, not only live state

Time budget

Task	Estimate
BIOS + disk + host install	2–4 h
KVM/Incus + first guest	1–2 h
Dev tooling (Lab 0B)	1–2 h
Elitebook tuning (Lab 0C)	30–60 min

Do this **once** before the Why Nix exists chapter; revisit when Stage II and Stage VI need second VMs.

Further reading

- I've gone full Nix: Proxmox to NixOS + Incus — Bas Nijholt (why declarative host + Incus; simulate-before-metal; hardware quirks in git)

Next

→ **Lab 0A — Journey single-server**

Lab 0A — Journey single-server (KVM + containers)

Lab 0A — Journey single-server (KVM + containers)

Goal: Turn **one physical machine** into a lab server that can run the entire Nix & NixOS journey: disposable NixOS VMs under **KVM/libvirt** (default) or **Incus** (optional CLI-first path), light **containers**, enough disk for `/nix` and disk images, and a network layout that makes deploy-rs/Colmena feel real.

Audience: anyone on a single spare box (including HP Elitebook 2570p · 16 GB—see Lab 0C).

> **Note**
>
> Several operational ideas below are aligned with Bas Nijholt's write-up of leaving Proxmox for NixOS + Incus (2025–2026): git as source of truth, CLI-first hypervisor ops, simulate host configs in VMs before reinstalling metal, and capturing hardware quirks as Nix—not shell history. Proxmox itself remains fine software; the point is **reproducible host state**, not "GUI bad."

What this lab optimizes for

Curriculum need	Lab feature
the Why Nix exists chapter–10 Nix on host or VM	`nix` available; flakes on
the installing NixOS chapter–22 break/rebuild NixOS	**VM snapshots** + generations
the secrets problem chapter–44 services	One primary guest with RAM for DB/proxy
the Disko chapter–70 deploy / multi-host	Second small VM on same lab network
Containers days	Podman and/or Incus system containers
Capstone wipe/rebuild	Guest disks you can delete without guilt
Host recoverability	Host flake + generations; optional "simulate host in VM" first

Architecture

```
libvirt default or lab bridge
192.168.122.0/24 (example NAT)
```

```
nixlab              ssh                  deploy-a
4-6G RAM                          2G RAM
journey primary                          deploy target


          edit flakes from host

host (metal)
KVM · git · nix
```

NAT is enough for the journey. Bridge to LAN only if you need LAN access from VMs and understand your network.

Part 1 — Host OS install (metal)

Option H1 (recommended): NixOS host

1. Backup anything on the Elitebook you still care about.
2. Install **NixOS 26.05** minimal or graphical ISO on metal.
3. Use **whole-disk** install on internal drive (or external SSD if you want zero risk to another OS).
4. Enable flakes from first config (Lab 0B expands this).

Minimal host modules (concept)—**all of this belongs in git**, including later hardware quirks:

```
{
  virtualisation.libvirtd.enable = true;
  virtualisation.libvirtd.qemu = {
    package = pkgs.qemu_kvm;
    swtpm.enable = true; # optional
  };
  programs.virt-manager.enable = true; # GUI optional; CLI remains primary
  users.users.YOU.extraGroups = [ "libvirtd" "kvm" "wheel" ];
  networking.firewall.allowedTCPPorts = [ 22 ];
  services.openssh.enable = true;

  # Disk: leave large room for /var/lib/libvirt and /nix
  nix.settings.experimental-features = [ "nix-command" "flakes" ];

  # Example pattern: document WHY a host tweak exists (see Part 9)
```

```
  # systemd.services.fix-nic-offload = { ... };
}
```

Host is not a locked appliance. Unlike "don't touch the hypervisor OS" products, NixOS-as-host *allows* you to run tools on metal (editor, caches, even a media app). Still: **curriculum breakage stays in guests.** If you change the host, change the **flake**, not only a live command.

Option H2: Debian/Ubuntu host

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients \
  bridge-utils virt-manager
sudo usermod -aG libvirt,kvm "$USER"
# log out/in
virsh list --all
```

Install **Nix multi-user** on the host for Lab 0B (editing flakes without only working inside guests):

```
# official installer - follow nixos.org/download
nix --version
```

Part 2 — Storage plan (critical on 16 GB RAM laptops)

RAM is fine; **disk** is usually the failure mode.

Path	Role	Guidance
`/nix`	Host store (if NixOS/Nix on host)	40–80+ GB free if possible
`/var/lib/libvirt/images`	VM disks	40–100 GB free
`~/src`	Flake git repos	SSD strongly preferred

If the Elitebook still has a spinning HDD

- Prefer a **used SATA SSD** (even 256–512 GB) before investing more evenings.
- Or use **external USB3 SSD** for libvirt images + large store (acceptable for lab).

Thin provision

Use `qcow2` with modest virtual size (e.g. 32–40 GB per guest) so you do not preallocate everything.

Part 3 — KVM: create journey VMs

Resource template (16 GB host)

VM	vCPU	RAM	Disk	Purpose
nixlab	2	**4096–6144** MiB	32–40 G	Main curriculum
deploy-a	1–2	**2048** MiB	20–25 G	Remote deploy target
builder (optional)	2	2048	20 G	Offload builds (Stage V+)

Leave **4 GB** for the host if you run a graphical desktop on metal.

Example: `virt-install` (adjust ISO path)

```
# Download NixOS 26.05 minimal ISO to ~/iso/
ISO=~/iso/nixos-minimal-26.05-x86_64-linux.iso

virt-install \
  --name nixlab \
  --memory 4096 \
  --vcpus 2 \
  --disk path=/var/lib/libvirt/images/nixlab.qcow2,size=40,format=qcow2 \
  --cdrom "$ISO" \
  --os-variant nixos-unknown \
  --network network=default \
  --graphics spice \
  --noautoconsole
```

Use `virt-manager` GUI if you prefer.

Snapshots = courage

Before risky Stage II/IV experiments:

```
virsh snapshot-create-as nixlab pre-stage-II "clean before break"
# restore:
# virsh snapshot-revert nixlab pre-stage-II
```

Also learn **NixOS generations** inside the guest—both matter.

Networking for “remote” deploy

On the host:

```
virsh net-dhcp-leases default
# note nixlab and deploy-a IPs
ssh YOU@192.168.122.XX
```

Put stable hostnames in host `/etc/hosts` or libvirt network XML if you get tired of changing IPs.

Part 4 — Containers (Podman)

Use containers for **OCI days**, not as a substitute for learning NixOS modules.

On host (NixOS):

```
virtualisation.podman.enable = true;
virtualisation.containers.enable = true;
```

Rootless Podman preferred for daily experiments.

```
podman run --rm -it docker.io/library/alpine:latest uname -a
```

When the curriculum says "declarative OCI on NixOS," prefer defining that **inside `nixlab`**, so the journey guest remains the system under study.

Many operators eventually run **Compose-style "pets"** inside a VM while the **host** stays pure NixOS. That split (declarative host + stateful guest workloads) is valid later; early stages should still teach **NixOS modules** before outsourcing everything to containers.

Part 4b — Optional hypervisor: Incus (CLI-first VMs + system containers)

Incus (community fork of LXD) is a strong fit when you want **one CLI** for VMs and system containers, and when you care about agentic/automation workflows that hate opaque UIs. It is **optional** for this curriculum; libvirt remains the default on 16 GB laptops.

When to consider Incus on the lab host

Choose Incus if…	Prefer libvirt if…
You want `incus launch` / profiles as daily API	You already know virt-manager and want fastest start
You may import Proxmox LXC/VM images later	You only need 1–2 simple NixOS guests
You like image profiles + clean CLI	You want minimal host packages

Do not run full Incus + full libvirt stacks + desktop + heavy VMs on the Elitebook without a reason—pick a **primary**.

NixOS host sketch (Incus)

```
{
  virtualisation.incus.enable = true;
  # networking: follow current nixpkgs Incus docs for bridge (incusbr0) vs custom
  users.users.YOU.extraGroups = [ "incus-admin" ]; # group name may vary by package
  networking.firewall.trustedInterfaces = [ "incusbr0" ]; # only if you use default bridge
}
```

Verify against **nixpkgs 26.05** option names (`man configuration.nix` / search.nixos.org)—Incus module knobs evolve.

```
incus admin init   # first-time interactive; then encode stable choices into docs/flake note
incus list
# VM example (image names change - use `incus image list images:`):
# incus launch images:nixos/unstable nixlab --vm -c limits.memory=4GiB
```

Simulate the host before you reinstall metal

Powerful idea from the Proxmox→NixOS migration write-up: run an Incus (or libvirt) VM that imports **almost the same modules** as the physical host, with a small **overrides** file (disk device names, interface names, disable real-metal-only services).

```
modules/host-common.nix     ← ssh, libvirt/incus, users, nix settings
hosts/elitebook.nix         ← real hardware-configuration + product
hosts/elitebook-sim.nix     ← imports common + overrides (virtio NICs, qcow root)
```

Workflow:

1. Get `elitebook-sim` healthy in a VM.
2. Only then install/reinstall the physical Elitebook from the same flake.
3. Keep `incus-overrides.nix`-style diffs tiny and commented.

This removes "nuking the only lab machine" fear—especially valuable once the host itself is NixOS.

If you already run Proxmox

You do **not** need to migrate mid-curriculum. If you later consolidate:

Workload	Typical move
Proxmox LXC	`vzdump` → import into Incus (community scripts exist; treat as advanced)
Proxmox VM disk	`qemu-img convert` → qcow2 → Incus or libvirt import

Workload	Typical move
NixOS journey guests	Prefer **fresh** NixOS VMs for learning; migrate pets only

Keep migration notes in your flake repo (inventory of CT/VM IDs, disk backends). Optional reading: migration notes linked from the same article's references.

Part 5 — Nested virtualization?

Elitebook Ivy Bridge: **VT-x** yes (enable in BIOS). Nested KVM (VM inside VM) is **optional** and often slow on this generation.

Need nested?	When
No	Default path: guests are the NixOS systems under test
Maybe	You insist on running libvirt *inside* nixlab

Recommendation: **do not** nest until you have a specific reason.

Part 6 — Services the host should run (boring is good)

Service	Why
`sshd`	Access from your main laptop if Elitebook is headless-ish
`libvirtd` and/or `incus`	Hypervisor
`chrony`/`systemd-timesyncd`	TLS and logs make sense
Optional `cockpit` / virt-manager	GUI if local—still commit CLI equivalents

Do **not** turn the host into the Stage IV app zoo. Guests carry curriculum apps.
(Exception later: a *deliberate* host service that is in the flake—e.g. a binary cache—never "I apt installed it at 2am.")

Part 7 — Daily workflow for the journey

```
1. Open host terminal / editor on ~/src/nix-config
2. Edit flake modules for nixlab / deploy-a  (and host/ only when intentional)
3. Prefer: nix build / nixos-rebuild build-vm / --target-host
4. Apply inside guest; avoid unexplained one-off guest shell config
5. Snapshot before scary changes
6. Commit flake.lock + config  ← if it isn't in git, it didn't happen
```

Alternative: `nixos-rebuild --target-host` / deploy-rs from host into guests (Stage VI).

Debugging without drift

When something breaks, the order is:

1. Reproduce
2. Fix in **Nix** (or Incus profile / cloud-init you track)
3. `switch` / redeploy
4. Commit with a comment that cites *why* (forum link, hardware quirk)

Avoid: fix live → forget → six months of mystery. That is the same class of failure as "checkbox in a web UI three years ago."

Part 8 — Declarative hardware quirks (host flake)

Older Intel NICs and laptops often need one-line kernel/driver workarounds. Example pattern (adapt; **do not** copy blindly):

```
# hosts/elitebook/networking.nix - illustrative
{
  # Document: link to bug/forum in a comment above the service.
  systemd.services.disable-nic-offload = {
    description = "Disable problematic NIC offloads on this hardware";
    wantedBy = [ "multi-user.target" ];
    after = [ "network-pre.target" ];
    serviceConfig.Type = "oneshot";
    serviceConfig.RemainAfterExit = true;
    script = ''
      # Example only - identify YOUR interface with `ip link`
      # ethtool -K enp0s25 tso off gso off gro off || true
    '';
    path = [ pkgs.ethtool ];
  };
}
```

On the Elitebook, when WiFi/Ethernet misbehaves: capture the fix **here**, not only in a successful interactive shell. Same for microcode, `boot.kernelParams`, and firmware.

Part 9 — Failure modes checklist

Symptom	Fix
VMs won't start	BIOS VT-x; user in `libvirtd`/`kvm` / `incus-admin` groups
Host OOM killer	Lower guest RAM; shut deploy-a when idle
Disk full	GC carefully; delete old snapshots; move images to external SSD
Nested rebuild hell	You nested by accident—flatten topology
WiFi-only flaky SSH	Prefer Ethernet dock/USB ethernet for lab sessions
"Works after I typed stuff" unreproducible	Re-apply from flake; delete untracked live changes
Incus + libvirt both fighting bridges	Pick one primary network stack

Checkpoint — Lab 0A complete

- ☐ Host installed and updated; **host flake** exists in git
- ☐ `virsh list --all` **or** `incus list` works (primary stack chosen)
- ☐ **nixlab** VM created (ISO install can finish on the installing NixOS chapter if needed)
- ☐ Snapshot (libvirt) or publish/restore strategy (Incus) tested once
- ☐ Optional: host-sim VM validates host modules before metal reinstall
- ☐ Podman (or Incus container) smoke test optional
- ☐ Written note: IPs/hostnames of guests
- ☐ Know wipe boundary: **guests + lab disks**; host only via generations + git
- ☐ At least one hardware or network quirk documented **in Nix** if you hit it

Next

→ **Lab 0B — NixOS dev machine** (tooling on this same host)
→ **Lab 0C — Elitebook 2570p** (your hardware numbers)

Lab 0B — NixOS development machine

Lab 0B — NixOS development machine

Goal: Configure a **comfortable, stable** environment for writing flakes, reviewing nixpkgs, running builds, and pushing configs to lab VMs—without turning your editor session into the system under test.

On a single Elitebook, **Lab 0B usually lives on the host** (same metal as Lab 0A). The split is **role**, not necessarily second hardware.

> **Tip**
>
> Text-first infrastructure (flakes, modules, `virsh`/`incus` CLI) is also what makes **human + AI** co-maintenance safe: agents can read and propose diffs against git. GUI-only state and untracked shell fixes cannot. Keep Lab 0B disciplined: **if it matters, it is in the repo.**

Dev machine vs journey guest

	Dev (this lab)	Journey guest (`nixlab`)
Purpose	Author configs, run `nix` tools	Be the OS you break
Rebuild frequency	Low	High
Secrets	Your age keys, git ssh	Lab secrets only
Editor	Yes	Optional
"Does this need to be pretty?"	Yes (you live here)	No

Part 1 — Install Nix (if host is not NixOS)

Multi-user install; enable flakes:

```
# ~/.config/nix/nix.conf
experimental-features = nix-command flakes
```

On **NixOS host**, put the same under `nix.settings.experimental-features`.

Part 2 — Essential packages

NixOS host module sketch

```
{ pkgs, ... }: {
  environment.systemPackages = with pkgs; [
    git
    git-lfs
    vim
    neovim
    ripgrep
    fd
    jq
    tree
    htop
    tmux
    direnv
    nix-direnv
    nixfmt-rfc-style   # or nixfmt - pick one and stick to it
    nil                # LSP for Nix
    # optional:
    # vscode / emacs - your preference
  ];

  programs.direnv.enable = true;
  programs.direnv.nix-direnv.enable = true;

  # nice for rebuilds against VMs
  environment.systemPackages = with pkgs; [
    # already have git etc.
  ];
}
```

Non-NixOS host

```
nix profile install nixpkgs#git nixpkgs#neovim nixpkgs#direnv nixpkgs#nix-direnv nixpkgs#nil
```

Part 3 — Editor + LSP

Editor	Nix LSP
Neovim/VS Code	`nil` or `nixd`
Emacs	same + envrc

Minimal checks:

```
nil --help   # or nixd
rg "mkOption" ~/src/nix-config -n | head
```

Part 4 — Git layout for the whole journey

```
~/src/nix-config/          # or ~/lab/nixos/nixos-config
  flake.nix
  flake.lock
  hosts/
    nixlab/
    deploy-a/
  modules/
  homes/                   # from Stage III
  secrets/                 # encrypted only (Stage IV)
  .envrc                   # use flake
  README.md
```

```
cd ~/src/nix-config
git init
echo ".direnv" >> .gitignore
echo "result" >> .gitignore
echo "result-*" >> .gitignore
```

.envrc:

```
use flake
```

```
direnv allow
```

Part 5 — Build ergonomics

Keep host builds from melting a 2012 laptop

```
nix.settings = {
  max-jobs = 2;          # Elitebook: start low
  cores = 2;
  # auto-optimise-store = true; # optional
};
```

Substituters

Default `cache.nixos.org` is fine. Later stages may add Cachix/Attic—document tokens only in sops, never in world-readable nix.

Remote / guest as builder (optional, Stage V+)

When `nixlab` is stronger or you want isolation:

```
# conceptual - configure nix.buildMachines later in curriculum
```

Early on: **build on host, deploy to guest** is simplest.

Part 6 — Access to journey VMs

```
# ssh config ~/.ssh/config
Host nixlab
  HostName 192.168.122.10
  User YOU
  IdentityFile ~/.ssh/id_ed25519_lab

Host deploy-a
  HostName 192.168.122.11
  User YOU
  IdentityFile ~/.ssh/id_ed25519_lab
```

Generate a **lab-only** key:

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_lab -C "lab-elitebook"
```

Do not reuse your primary GitHub key as the only lab key if you prefer blast-radius control.

Part 7 — Workflow commands you will live with

```
# evaluate / build a guest config from the flake (host)
nix build .#nixosConfigurations.nixlab.config.system.build.toplevel

# apply (from host, once SSH works)
nixos-rebuild switch --flake .#nixlab --target-host YOU@nixlab --use-remote-sudo

# or: ssh and rebuild inside guest
ssh nixlab 'sudo nixos-rebuild switch --flake /path/to/config#nixlab'
```

```
# format
nix fmt   # if configured
```

Part 8 — Home Manager on the dev role

Optional but nice on the host:

- Shell (fish/zsh), git, ssh config
- Keep HM on host **separate** from guest HM experiments

When Stage III teaches HM, practice on `nixlab` first if you want isolation; port habits back to host later.

Part 9 — Binary cache & secrets (preview only)

Later stage	Dev machine prep
sops-nix	Install `sops`, `age`; create age key **backup** offline
CI	GitHub/GitLab SSH deploy key for the flake repo
Cachix	Account ready; don't put signing key in git

Create age key early if you want:

```
mkdir -p ~/.config/sops/age
age-keygen -o ~/.config/sops/age/keys.txt
# back up keys.txt to password manager / encrypted USB
```

Part 10 — Quality of life on old hardware

Tip	Why
External monitor + keyboard	2570p is a server that happens to fold
`tmux`/`zellij`	Long builds survive SSH drops
Disable heavy desktop effects on host	Free RAM for VMs
`nix-store --optimise` occasionally	Disk
Don't run browser + 2 VMs + big compile	Sequentialize

Dev machine checklist

- ☐ `nix flake metadata` works in `~/src/nix-config` (even empty flake)
- ☐ direnv loads a simple `devShell`
- ☐ Editor can jump/lint Nix files
- ☐ SSH to `nixlab` (or console) documented
- ☐ Lab SSH key created; age key backup plan written
- ☐ `max-jobs`/`cores` set conservatively
- ☐ `.gitignore` includes `result` and `.direnv`

Anti-patterns

Anti-pattern	Prefer
Editing production-like host to "try modules"	Try on `nixlab` (or host-sim VM)
One 2k-line `configuration.nix` on host	Layout from Stage III early for guests
Storing secrets in world-readable nix	sops/agenix later
Building with `max-jobs = auto` until OOM	Cap jobs on Elitebook
"YOLO" imperative fix left uncommitted	Encode fix in flake; commit with *why*
Relying on virt-manager clicks you cannot replay	Keep a `virt-install` / `incus` snippet in README

Next

→ **Lab 0C — HP Elitebook 2570p profile**
→ Then **Why Nix exists** (first topic chapter)

Lab 0C — HP Elitebook 2570p (16 GB) profile

Lab 0C — HP Elitebook 2570p · 16 GB RAM lab profile

Concrete profile for a spare **HP Elitebook 2570p** with **16 GB RAM**, used as the single-server lab for this NixOS volume (Labs 0A + 0B on the same chassis).

> **Note**
>
> The 2570p is ~2012 Ivy Bridge hardware. It is **excellent** as a quiet lab server for learning NixOS. It is **not** a modern workstation: respect thermals, disk, and CPU age.

Hardware snapshot (typical 2570p)

Component	Typical	Lab implication
CPU	Intel Core i5/i7-3xxxM (Ivy Bridge)	2 cores / 4 threads class; VT-x yes
RAM	Up to 16 GB DDR3 (you have 16 GB)	Comfortable for 1 heavy + 1 light VM
GPU	Intel HD 4000	Fine for console/light desktop host; not GPU passthrough playground
Disk	Factory HDD or aftermarket SATA SSD	**SSD strongly recommended**
Display	12.5”	Treat as headless-ish; external monitor helps
NIC	Intel wired + WiFi	Prefer **Ethernet** for long SSH sessions
Firmware	HP BIOS	Enable Virtualization Technology

Exact CPU SKU varies (`lscpu` after boot). Confirm:

```
lscpu | rg -i 'Model name|Virtualization|CPU\\(s\\)|Thread'
free -h
lsblk
```

BIOS checklist (do this first)

1. **Virtualization Technology (VTx)** — Enabled
2. **VT-d** — Enable if present (not required for basic KVM)
3. Boot mode — UEFI preferred if disk allows; legacy OK for lab
4. Disable unused boot devices if they slow POST
5. Set fans/thermal to a sane default; keep vents clear

Save & reboot.

Disk strategy for *this* laptop

Best

Disk	Use
Internal **SATA SSD** 512 GB+	Host root + `/nix` + libvirt images
Optional USB3 SSD	Overflow images / backups

Acceptable

Disk	Use
Internal SSD 256 GB	Host + one VM; external for second VM images
HDD only	Usable but painful; prioritize SSD upgrade

Partition sketch (NixOS host, single disk)

```
ESP          512 MB   FAT32
root         rest     ext4 or btrfs
  /nix                (same FS is fine; optional subvol on btrfs)
```

If dual-booting is tempting: **don't** for this spare. Dedicate the machine to lab.

RAM budget (16 GB fixed)

Consumer	Allocation	Notes
Host OS + desktop/ssh	**4–5 GB**	Prefer lightweight DE or none
nixlab VM	**4–6 GB**	Primary journey
deploy-a VM	**2 GB**	Stop when idle
Headroom / browser	**2–3 GB**	Close browser during big builds
Total	16 GB	Never overcommit hard on this chassis

Suggested virsh memory

```
# examples
# nixlab:  4096 or 6144 MiB
# deploy-a: 2048 MiB
```

During Stage IV (DB + proxy + observability): prefer **6 GB nixlab** and **powered-off deploy-a** unless needed.

CPU budget

Setting	Value
nixlab vCPUs	2
deploy-a vCPUs	1–2
Host `nix.settings.max-jobs`	**1–2**
Host `nix.settings.cores`	**2**

Ivy Bridge will thermal-throttle under sustained compile. That is normal—not a broken Nix.

What this machine is good for (curriculum map)

Stage	On 2570p?
I Foundations	Yes
II One NixOS host (in VM)	Yes
III HM + layout	Yes
IV Secrets + small services	Yes (watch RAM)
V Packaging	Yes (be patient; use caches)
VI Deploy two VMs	Yes
VI k3s / heavy observability	Optional/light only
VII Internals	Yes
VIII Capstone	Yes (scope to hardware)
Nested libvirt	Avoid
Multi-node k8s + desktop	No

Networking on the Elitebook

Mode	Recommendation
WiFi only	OK for browsing; flaky for long rebuilds
USB Ethernet / dock	**Preferred** for lab evenings
Bridged VM to LAN	Optional; NAT default is safer

Port-forwarding example (if you must reach a guest service from another PC): use host SSH tunnel rather than exposing VMs broadly.

```
ssh -L 8080:192.168.122.10:80 user@elitebook-host
```

Power & thermals

- Use **AC power** for builds and VM sessions
- Elevate rear for airflow if temps high
- `tlp` / power-profiles: on host, prefer performance when labbing on AC
- Expect fan noise under `nix build`—plan quiet hours

Firmware & drivers notes

Area	Note
WiFi	Prefer `linux-firmware`; some cards need `allowUnfree` for redistributable firmware
CPU microcode	`hardware.cpu.intel.updateMicrocode = true;` on NixOS host
GPU	Modesetting is enough; no NVIDIA complexity on stock 2570p
Fingerprint / WWAN	Ignore for lab
Ethernet	Older Intel NICs sometimes need offload disabled (`ethtool -K …`); **put that in the host flake** with a comment (pattern in Lab 0A Part 8), not only in a one-off root shell

Host is fully yours (but still a lab)

On appliance hypervisors you are often discouraged from "messing with the host." On **NixOS host**, you *can* install an editor, run a local cache, or attach a display—and still recover via generations.

That does **not** mean the Elitebook should become your Stage IV app zoo. Keep metal useful; keep **journey risk in VMs**.

Concrete "day zero" procedure (you)

1. **Backup** anything left on the 2570p.
2. Install **SSD** if still on HDD (worth it).
3. BIOS: enable VT-x.
4. *(Optional but wise)* On another machine or live USB workflow: draft `hosts/elitebook-sim.nix` and boot it as a VM first (Lab 0A "simulate host").
5. Install **NixOS 26.05** as host (Lab 0A Option H1).
6. Apply host config from git: libvirt **or** Incus (pick one primary), ssh, flakes, microcode, your user.
7. Create **nixlab** (4–6 GB) + **deploy-a** (2 GB).
8. Snapshot blank/fresh installs.
9. Clone/create `~/src/nix-config` flake (Lab 0B); commit host + guest stubs.
10. SSH from your daily laptop into Elitebook; from Elitebook into VMs.
11. Only then start **the Why Nix exists chapter**.

Optional extras that fit this chassis

Extra	Why
Automatic libvirt start	`nixlab` available after host boot
External USB SSD labeled `nix-lab`	Images + `nix copy` backups
UPS not required	Laptop battery is a mini-UPS for short blips
Second-hand 8 GB→16 GB already done	Good—don't bother 32 GB (max 16 on this platform)
Replace thermal paste	Only if throttling is extreme

What not to run on the 2570p

- Desktop + gaming + three 8 GB VMs
- Full Observability stack + k3s + browser on host simultaneously
- Treating host as wipe-target for Stage II installs (use **nixlab**)
- Expecting modern Secure Boot/TPM2 demos (Stage VI Secure Boot day may be **document-only** on this hardware)

For Lanzaboote/TPM chapters: complete theory + skip or use a newer machine if you have one later.

Elitebook readiness checklist

- ☐ 16 GB confirmed in `free -h`
- ☐ VT-x enabled; `kvm-ok` or equivalent happy
- ☐ SSD or external fast disk for images
- ☐ Host NixOS (or Debian+Nix) installed
- ☐ libvirt works; **nixlab** defined
- ☐ RAM split written down and enforced
- ☐ `max-jobs  2`
- ☐ Ethernet or stable WiFi plan
- ☐ Lab 0A + 0B checklists done
- ☐ Wipe boundary: guests only

After this page

You are ready for **Why Nix exists** (first topic chapter after Lab 0).

When the installing NixOS chapter says "install NixOS," install into **nixlab** (or replace its disk), not by reinstalling the Elitebook host unless you intentionally redesign Lab 0A.

Lab 0D — Ops cheatsheet

Lab 0D — Ops cheatsheet (single-server lab)

Quick reference while studying. Details live in Labs 0A–0C.

Daily

```
# host - libvirt (default path)
virsh list --all
virsh start nixlab
virsh snapshot-list nixlab

# host - Incus (if that is your primary)
incus list
incus start nixlab
incus info nixlab

# network (libvirt NAT example)
virsh net-dhcp-leases default

# ssh
ssh nixlab
ssh deploy-a
```

Snapshots / restore points

```
# libvirt
virsh snapshot-create-as nixlab "label" "notes"
virsh snapshot-revert nixlab "label"
virsh snapshot-delete nixlab "label"

# Incus (publish image / snapshot features - see `incus snapshot` help for your version)
incus snapshot create nixlab pre-change
incus snapshot restore nixlab pre-change
```

Disk pressure

```
df -h / /nix /var/lib/libvirt/images
nix store gc          # careful: know generations
nix store optimise
du -sh /var/lib/libvirt/images/*
```

Guest rebuild (from host flake)

```
cd ~/src/nix-config
nix build .#nixosConfigurations.nixlab.config.system.build.toplevel
nixos-rebuild switch --flake .#nixlab --target-host USER@nixlab --use-remote-sudo
```

When the Elitebook sweats

1. Shut down `deploy-a`
2. Set `max-jobs = 1`
3. Close browser
4. Prefer substitutes over local compiles when possible
5. Pause nested experiments

Drift control

```
cd ~/src/nix-config
git status          # uncommitted host/guest edits?
git diff
# After a live debug: encode the fix in *.nix, rebuild, then commit
```

Emergency

Problem	Action
Host unresponsive	REISUB if enabled; else hard power; check disk full

Problem	Action
Guest won't boot	libvirt/Incus snapshot **or** NixOS generation from boot menu
Locked out of guest	virt-manager / `virsh console` / `incus console`
Store full	Delete old snapshots; `nix store gc` with eyes open
"Fixed it in the shell, lost it on reboot"	That was drift—put it in the flake

Inventory template (fill once)

```
Host hostname: ________
Host IP: ________
nixlab IP: ________
deploy-a IP: ________
Flake path: ________
Lab SSH key: ________
Age key backup: ________
Wipe-allowed disks: ________
```

Concepts

Concepts — Nix language, store, CLI, flakes

Concepts — Nix language, store, CLI, flakes

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

What this part is for

Concepts builds the mental models every later part assumes: what Nix is, how the store works, how the language composes config-shaped data, how derivations become store paths, and how the modern flakes CLI pins work.

You will not install NixOS here. You will install the **package manager**, enable **flakes**, and practice evaluation, builds, and project shells on any supported host (Linux or macOS for most drills; some store demos prefer Linux).

This part	Next part
Language + store + flakes literacy	**Nix on Linux** (`02-nix-on-linux/`) — Fedora 44+ dual-stack bridge (recommended)
Disposable experiments on any Nix install	Then **NixOS host** — machine as code, generations, modules, boot

---

Baseline (this book)

Piece	Baseline
Nix	**2.34.x** (modern CLI + flakes)
nixpkgs channel pin	**nixos-26.05** (Yarara) when you pin flakes
Default workflow	Flakes + `nix build` / `nix run` / `nix develop`
Classic CLI	Literacy only (last chapter of this part)

Version numbers drift in the wild; when in doubt, check `nix --version` and pin **26.05** in your own flakes so examples stay comparable.

Learning outcomes

By the end of this part you should be able to:

1. Separate the **three faces** of Nix: language, package manager, NixOS
2. Read a `/nix/store/…` path and explain **closures** and **GC roots**
3. Write nested **attrsets**, `let`/`inherit`, and config-shaped data
4. Write **functions** with attr-pattern args, defaults, ellipsis, and `@`
5. Split multi-file Nix with **laziness** and a few `lib` pipelines
6. Explain a **derivation** as a plan → `.drv` → realized output (including FODs at awareness level)
7. Use the **modern CLI** for build/run/shell/develop/eval/path-info
8. Author a **project flake** with lockfile + `devShells.default`
9. Translate **classic** commands when reading old docs—without living there

Prerequisites

You should have	Nice to have
Comfortable shell (bash/zsh)	Prior package-manager pain (apt/pip/npm)
Text editor + git basics	A disposable machine or VM for disk-heavy store labs
~10–20 GB free for store growth	Linux (closer to later NixOS host work)

No prior NixOS install required for this part.

Chapter map

Chapter	Title	Focus
Why Nix exists	Problem statement, three faces, install Nix, first REPL	
Store and closures	Path anatomy, references, closures, GC roots	
Language: attrsets	Values, strings, lists, sets, `let`, `inherit`, `//`	
Language: functions	Lambdas, pattern args, `?`, `...`, `@`, call shapes	
Laziness, imports, lib	Lazy force, multi-file, infinite recursion, `lib`	
Derivations as an idea	Plans, sandbox, FODs, trivial builds, logs	
Modern Nix CLI	Installables, build/run/shell/develop, flake subcommands	
Flakes and devShells	inputs/outputs/lock, `mkShell`, git discipline	
Classic CLI literacy	Classic modern map, channels vs locks	

Suggested learning paths

Path	Order	Notes
Linear (recommended first pass)	01 → 09	Builds one model on the next
Language-first	03 → 05, then 01–02, 06–09	If you already installed Nix
Ops-curious skim	01, 02, 07, 08	Then return for language depth
Reference	Any chapter standalone	Use checkpoints + gotchas tables

---

How each chapter is structured

Topic chapters in this book generally share:

1. **Goal** — what competence you leave with
2. **Why it matters** — systems/ops motivation

3. **Theory** — tables, mental models, diagrams when present
4. **Worked examples** — adaptablesnippets (Nix 2.34 / flakes / 26.05 baseline)
5. **Labs / exercises** — hands-on practice
6. **Common pitfalls** — symptom → theory table
7. **Checkpoint** — self-check before moving on
8. **Further depth** — related chapters **in this book only**

Lab workspace convention

Prefer a single lab tree on your workstation (not tied to any calendar series):

```
mkdir -p ~/lab/nixos-book/concepts
cd ~/lab/nixos-book/concepts
```

Per-chapter subdirs (`why-nix`, `store`, `attrsets`, …) keep experiments reproducible. Commit flakes and notes; do not commit private keys or machine secrets.

Mental model map (part-level)

```
Expression (Nix language)
      evaluate

Derivation plan (.drv)   realize    Store path(s)

                                              references → closure
                                              GC roots keep live

Modern CLI / flake outputs name installables


devShell / package / (later) nixosConfigurations
```

Keep this sketch in mind when a command “does nothing” (eval) versus “downloads forever” (realize) versus “fills the disk” (roots).

What you will *not* do in this part

Out of scope here	Covered later in this book
Install NixOS / UEFI / generations	Part **NixOS host**
Home Manager, fleet, Disko	Later parts
Deep `stdenv` packaging	**Packaging** part
Module system authoring	Host + **Internals**
Cross-links to any 90-day series	Not used in this book

Suggested checkpoints (part gate)

Before starting **Nix on Linux** (or NixOS host, if you skip the bridge), confirm:

- ☐ `nix --version` reports a 2.34.x-class Nix with flakes enabled
- ☐ You can explain language vs package manager vs NixOS in one sentence each
- ☐ You can define store path, closure, and GC root
- ☐ You can evaluate multi-file Nix with functions and attrsets
- ☐ You can `nix build` / `nix run` / `nix develop` against a personal flake
- ☐ `flake.lock` is committed for that project
- ☐ You can translate at least five classic commands to modern ones

---

Further depth (this book)

After Concepts…	Go to
Fedora / foreign distro daily driver	Part **Nix on Linux** (`02-nix-on-linux`) — official Fedora Nix, HM standalone, dual-stack
Own a machine as code	Part **NixOS host** (`03-nixos-host`)
Project layout, locks, HM	Part **Home and flake layout**

After Concepts…	Go to
Secrets, services, TLS	Part **Services and security**
Real packaging	Part **Packaging**
Evaluator / module deep dive	Part **Internals**

How to use this part as a reference

- **Sidebar titles** come from each chapter's `title:` frontmatter—search by topic name
- **Gotchas tables** at chapter ends are optimized for "I hit this error" jumps
- **Worked examples** are meant to be copied and adapted, not treated as sacred
- Prefer **pinning nixpkgs to 26.05** in examples so your lock matches the book's baseline

When a later chapter assumes "you know closures," it means this part—not a calendar day number.

Why Nix exists

Why Nix exists

Goal: Separate the three meanings of "Nix," understand the *problem statement* honestly, install the package manager with **flakes + modern CLI**, and evaluate first expressions in a REPL.

> **Note**
>
> This chapter installs **Nix language + package manager**, not NixOS. The OS host arrives in the **NixOS host** part of this book.
> **On Fedora 44+:** prefer the **official distro package** (`sudo dnf install nix`) and the full dual-stack bridge in **Nix on Linux** rather than only a generic upstream installer—SELinux and `/nix` integration are why Fedora packaged it.

Why this chapter matters

People fail Nix for social reasons as much as technical ones: they read a blog that mixes **language**, **CLI**, and **NixOS modules**, then paste commands from three eras of tooling. Theory first reduces that thrash.

Ops motivation: every later skill—closures, flakes, rebuilds, deployments—assumes you know *what problem Nix claims to solve* and *what it deliberately does not*. Without that frame you will treat Nix as "yet another package manager with weird paths" and fight the model.

Theory 1 — Three faces of the same ecosystem

Layer	Question it answers	You touch it in this chapter?
Language	How do we *describe* packages and configs?	Yes — REPL
Package manager	How do we *build/store/compose* software?	Yes — install + `nix run`
NixOS	How do we describe an entire *OS*?	No — host part later

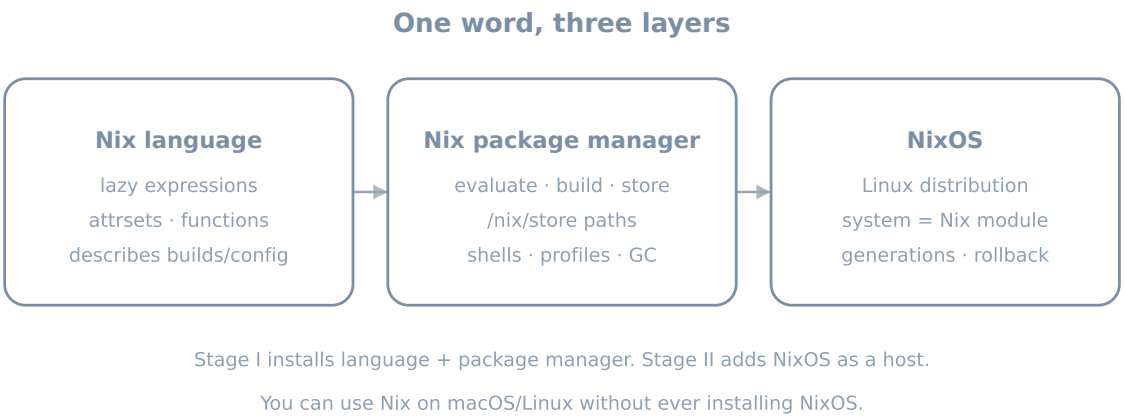


Figure 1: Language, package manager, and NixOS

Analogy (imperfect but useful)

Idea	Rough analogy
Nix expression	A pure function recipe
Derivation	A build plan (inputs → output path)
Store path	The baked loaf with a serial number
Profile / shell	A lunch tray pointing at selected loaves
NixOS system	The whole kitchen layout as code

Analogies break; the table is orientation, not law.

Naming traps in the wild

Someone says…	They might mean…
“Install Nix”	Package manager only
“Install NixOS”	Full Linux distribution
“Write some Nix”	Language expressions
“Nix config”	Could be any of the three

When reading docs, force yourself to label which face is speaking.

Theory 2 — Failure modes of traditional systems

Dependency hell

Classic package managers often assume **one** version of a library in a global prefix (`/usr/lib/...`). Applications A and B that need different versions conflict.

Example story

- App A needs `libssl.so.1.1`
- App B needs `libssl.so.3`
- Distro upgrade breaks A, or pinning freezes security updates for B

Snowflake servers

Imperative drift:

```
ssh prod
apt install foo
vim /etc/foo.conf
# six months later: nobody knows the full story
```

Language-level global installs

```
pip install --user something
npm install -g other
cargo install another
```

Each ecosystem invents isolation. None composes cleanly with the others or with the OS.

CI and agent "works on my machine" graphs (the three-way drift)

Symptom	Typical cause
Green CI, red laptop	Different tool versions
Green laptop, red prod	Hidden system libraries
Two developers disagree	Floating "latest" pins
Agent breaks sandbox with `apt-get`	Ephemeral AI agent lacks hermetic toolchain

Traditional teams struggled with two-way drift between developer laptops and CI runners. Autonomous AI coding agents introduce a **third vector of divergence**: ephemeral agent sandboxes.

When an agent encounters missing dependencies, it attempts destructive, imperative installations (`sudo apt-get`, `pip install --break-system-packages`), destroying reproducibility.

Nix's bet is: **make the graph explicit and hashed, define one single source of truth**, and share it across human laptops, CI/CD runners, and AI agent sandboxes alike.

> **The AI era paradigm shift**
>
> Nix historically had a reputation for a steep, intimidating learning curve. In the era of LLMs, that dynamic inverts: modern AI models excel at reading and generating declarative Nix expressions. Developers can prompt for outcomes while Nix provides the mathematical determinism, hermetic sandboxing, and atomic rollbacks that prevent autonomous agents from hallucinating or corrupting host state.

What "reproducible" can mean (honesty ladder)

Claim	Strength
"Same machine rebuilds the same store paths"	Often achievable with locks
"Same flake lock on two Linux machines → same paths"	Common with pure fixed-output inputs
"Bit-for-bit identical across OS/CPU forever"	Strong; not always true
"My *data* is reproducible"	Not Nix's job unless you design state

Nix optimizes for **controlled, rebuildable software graphs**. It does not delete ops.

Theory 3 — Nix's core bet: pure builds + hashed store

Content-addressed intuition

If a build is a function of its inputs, then informally:

$$\textbf{output path} = f(\text{build recipe, input paths, system, …})$$

In the common **input-addressed** world, the hash reflects the derivation inputs. (Content-addressed derivations are an advanced evolution—see the **Internals** part of this book.)

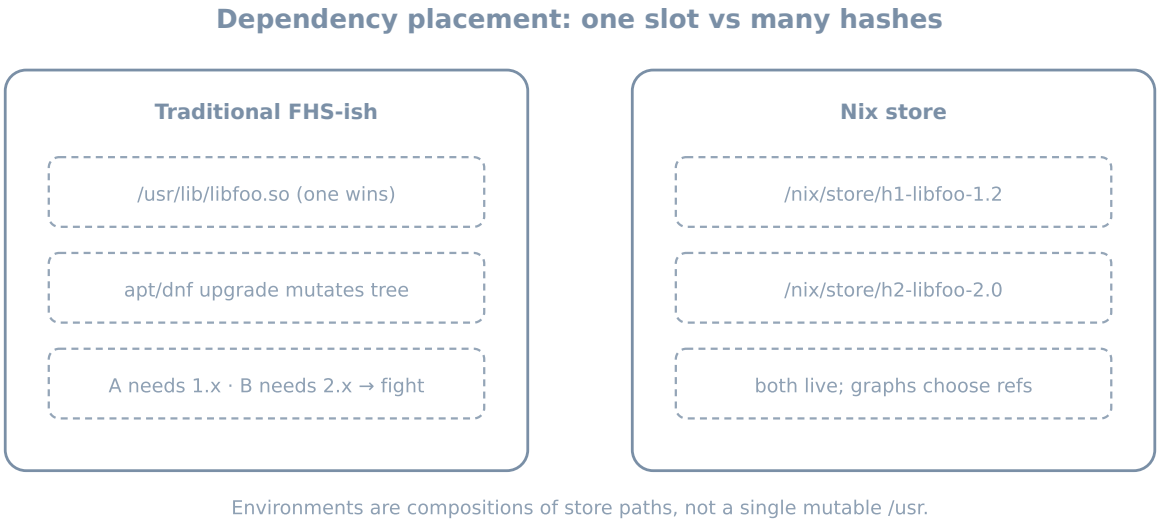


Figure 2: Traditional FHS vs Nix store coexistence

Coexistence

```
/nix/store/hAAAAA-libfoo-1.2.3
/nix/store/hBBBBB-libfoo-2.0.0
```

Both can exist. Package A references `hAAAAA`; package B references `hBBBBB`. No single `/usr/lib/libfoo.so` gladiator match.

Environments as compositions

A "shell" or "profile" is not a pile of files dumped into `/usr`. It is a **set of store paths** arranged on `PATH`, `LIB`, etc.

Example mental model

```
devShell = { go_1_26, git, hello }
         → PATH entries under those store paths
```

What purity buys operations

Property	Operational win
Immutable store paths	Roll forward/back by re-pointing roots
Explicit references	Copy a complete closure to another host
Sandboxed builds	Fewer "works on my machine" build deps
Binary caches	Realize often means download, not compile

Theory 4 — Evaluation vs realization

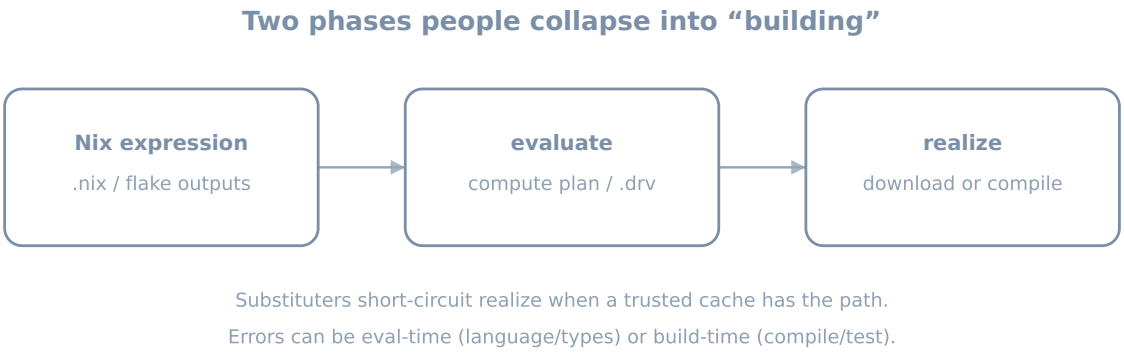


Figure 3: Evaluate then realize

Phase	What happens	Failure examples
Evaluate	Nix language runs; produces derivation plans	Infinite recursion, missing attr, type errors
Realize	Build or download outputs into the store	Compile error, hash mismatch, network/cache miss

Substituters (binary caches)

If a trusted cache already has the output path, realization becomes **download**, not compile. That is why first `nix run nixpkgs#hello` can be fast *or* slow depending on cache/network.

Example

```
eval:  nixpkgs.hello → derivation → planned out path P
realize: P exists on cache.nixos.org? download : build
```

Why the split matters for debugging

You see…	Likely phase
`error: undefined variable`	Evaluate
`error: attribute 'foo' missing`	Evaluate
`builder failed with exit code`	Realize
`hash mismatch in fixed-output`	Realize (fetcher)
Long silence then download progress	Realize via substituter

Theory 5 — Installer shapes and multi-user daemon

Shape	Idea
Multi-user + nix-daemon	Builds run as daemon; shared store; recommended default
Single-user	Store owned by your user; simpler, less isolation
Determinate installer	UX wrapper; still Nix—**record that you used it**

Why a daemon?

Sandbox builds need privileges to set up namespaces/mounts cleanly. Multi-user installs also protect the store from accidental mutation.

Config locations (theory)

File	Role
`~/.config/nix/nix.conf`	User Nix settings
`/etc/nix/nix.conf`	System-wide (multi-user)

Experimental features (`nix-command`, `flakes`) must be visible to the Nix that actually runs.

Disk and trust

Leave **tens of GB** free. The store grows with every experiment. Trust binary caches only when signatures match configured public keys—random HTTP mirrors are not enough.

Theory 6 — Flakes as the modern pinning model (preview)

You enable flakes in this chapter; deep flake authoring is the **Flakes and project devShells** chapter.

Concept	Meaning
`flake.nix`	Declarative inputs + outputs
`flake.lock`	Pinned revisions/hashes of inputs
`nixpkgs#hello`	Indirect flake reference to package `hello`

Contrast with channels

Channels	Flakes
Mutable "what is nixpkgs today?"	Lock file pins *exactly*
`NIX_PATH` / `<nixpkgs>`	Explicit inputs
Easy to drift	Designed for collaboration

This book: **flakes default**, channels as literacy only (classic CLI chapter).

Baseline pin you will reuse

```
inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
```

Theory 7 — CLI eras (so docs don't gaslight you)

Era	Examples	Status in this book
Classic	`nix-env`, `nix-build`, `nix-shell`	Literacy later (classic CLI chapter)
Modern	`nix build`, `nix run`, `nix develop`, `nix flake`	**Default**

Same underlying store; different UX and defaults. Prefer modern unless you are translating an old post.

Theory 8 — What Nix is *not*

Myth	Reality
"Nix replaces backups"	State (`/var`, DBs) needs its own policy
"Nix encrypts secrets"	Store is often world-readable; secrets need tools later
"One hash forever everywhere"	OS/CPU/impure inputs still matter
"No ops needed"	You still design networks, access, upgrades

Honesty keeps adoption durable.

Worked examples

Example A — "Install" without polluting a global profile

```
nix run nixpkgs#hello
```

Runs `hello` from a realized store path without committing to a permanent user profile.

Example B — Why two machines diverge without locks

```
Machine A: nixpkgs as of Monday
Machine B: nixpkgs as of Friday
→ different hashes for "hello", different closures
```

Flake locks exist to end that story.

Example C — What Nix does *not* do

```
Declarative Postgres service  automatic migration of your prod data
Encrypted secrets in git  "Nix encrypts the store"
```

Example D — Eval error vs build error (feel the phases)

```
# Eval failure (missing attribute) - no build starts
nix eval --expr 'let pkgs = {}; in pkgs.doesNotExist'

# Realization path for a known package (may download)
nix build nixpkgs#hello --print-out-paths
```

Example E — Minimal REPL tour

```
nix repl
```

```
1 + 1
{ a = 1; b = "x"; }.a
builtins.typeOf { x = 1; }
builtins.typeOf (x: x + 1)
```

Example F — Inspect config after enabling features

```
nix config show | grep -i experimental
nix --version
```

Lab 1 — Install Nix

Use current docs: https://nixos.org/download/
Or Determinate installer if you choose—**write which**.

```
# new shell after install
nix --version
```

Disk: leave tens of GB free for later chapters and parts.

Record in notes:

- Installer name/version
- Multi-user or single-user
- OS and CPU (`uname -m`)

Lab 2 — Enable modern features

`~/.config/nix/nix.conf` (or system `nix.conf` if that is what your install uses):

```
experimental-features = nix-command flakes
```

Verify:

```
nix config show | grep -i experimental
nix flake --help | head
```

If settings do not apply, check whether the daemon needs a restart and whether user vs system config is the one being read.

Lab 3 — First modern commands

```
nix search nixpkgs hello | head
nix run nixpkgs#hello
```

REPL

```
nix repl
```

```
1 + 1
{ a = 1; b = "x"; }.a
builtins.typeOf { x = 1; }
```

Load nixpkgs when available (`:lf nixpkgs` or version-appropriate equivalent) and inspect `pkgs.hello`.

Lab 4 — Write the model

In your journal, answer without looking up:

1. Language vs PM vs OS—one sentence each
2. Why two `libfoo` versions can coexist
3. Eval failure vs build failure—one example each
4. Which installer you used

Exercises

1. Draw the three faces of Nix from memory; check against the diagram.
2. Name three traditional failure modes Nix targets and one it does not.
3. Run `nix run nixpkgs#hello` twice; note whether the second is faster (cache).
4. Force an eval error and a realize-related error; label each.

5. Locate which `nix.conf` your install actually reads; quote the path.
6. Enable flakes only in user config vs system config—document which worked.
7. Compare `nix --version` with a teammate; note drift risk without locks.
8. Write a 5-line "why Nix" pitch for a skeptic who loves apt.
9. List five blog phrases that mix language/PM/OS; rewrite them precisely.
10. Create `~/lab/nixos-book/concepts/why-nix/NOTES.md` with installer + baseline.
11. Run `nix repl` and evaluate nested attrsets until muscle memory sticks.
12. Explain input-addressed hashing in one paragraph without the word "magic."

Common pitfalls

Symptom	Theory link
`nix` not found	Install/PATH; shell not reloaded
experimental feature errors	`nix.conf` not read by that Nix
Slow first command	Cold substituter / eval of nixpkgs
Permission errors	Single vs multi-user mismatch
Old blog uses only `nix-env -i`	Classic era; map later
"Nix broke my OS" after PM install	You installed PM, not NixOS—different surface
Expecting data reproducibility	Wrong layer of the honesty ladder
Mixing three installers	Record and stick to one story

Checkpoint

- ☐ Can explain three faces of Nix
- ☐ Can state two problems Nix targets and two it does not
- ☐ `nix run nixpkgs#hello` works
- ☐ Flakes + nix-command enabled

- ☐ REPL evaluated an attrset
- ☐ Installer name recorded
- ☐ Eval vs realize distinction written in own words

Further depth (this book)

Topic	Chapter / part
Store paths, closures, GC roots	The Nix store and closures
Language data model	Nix language: values and attrsets
Flake authoring	Flakes and project devShells
Classic CLI translation	Classic CLI literacy
Full OS as modules	Part **NixOS host**
Content-addressed derivations	Part **Internals**

The Nix store and closures

The Nix store and closures

Goal: Read `/nix/store` paths as a graph database of software, define **closures** precisely, and explain **GC roots**—with labs that only *observe* (no aggressive GC yet).

Why this chapter matters

Every advanced Nix topic—binary caches, deploy, Docker-from-Nix, NixOS activations—is a story about **store paths and references**. Without that theory, later tools feel like superstition.

Ops motivation: disk fills, deploys miss a library, "I deleted the project but used 40 GB"—all are root/closure mistakes. If you can *see* the graph, you can operate it.

Theory 1 — The store is the database

The directory `/nix/store` is not a random cache folder. It is the **authoritative object store** for:

- Package outputs
- Derivation files (`.drv`)
- Fixed-output download results
- Sometimes source mirrors

Invariants (practical)

Invariant	Meaning
Paths are immutable	You do not `vim` a store path to "fix prod"
Names include hashes	Collision-resistant identity
References are explicit	Runtime deps encoded as string refs inside outputs
Mutation is via new paths	"Upgrade" means new hash, not overwrite

Hand-editing the store voids the model (and often permissions prevent it).

What lives beside the store

Path (typical)	Role
`/nix/var/nix/db`	Nix database (references, validity)
`/nix/var/nix/profiles`	Profile generations (roots)
`/nix/var/nix/gcroots`	Explicit GC roots
`/nix/var/nix/daemon-socket`	Multi-user daemon

You rarely edit these by hand; you query them.

Theory 2 — Path anatomy

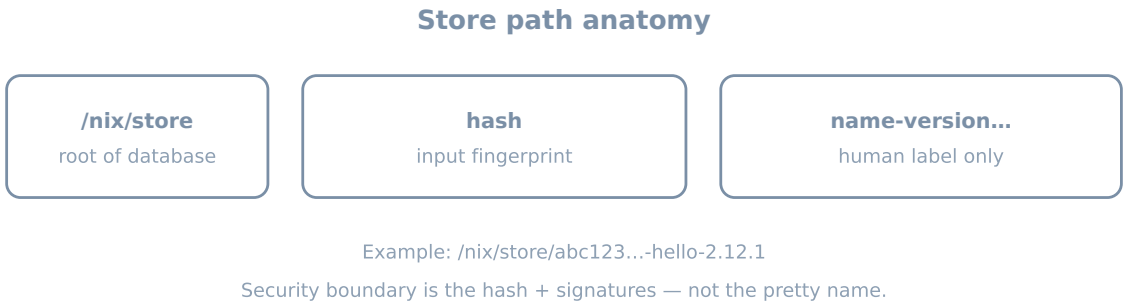


Figure 1: Store path anatomy

```
/nix/store/<hash>-<name>-<version>

                        convenience label (not trust)
                cryptographic fingerprint of inputs (typical case)
       global namespace of this Nix installation
```

Examples (illustrative shapes)

```
/nix/store/abc…-hello-2.12.1
/nix/store/def…-glibc-2.40-36
/nix/store/ghi…-bash-5.2-p37
```

Theory punchline: two paths with the same pretty name and different hashes are **different objects**. Tools that only show names are lying by omission.

Input-addressed vs content-addressed (awareness)

Model	Hash depends on	Note
Input-addressed (classic default)	Build inputs / derivation	Rebuild if inputs change
Content-addressed (experimental/advanced)	Output content	Same bytes → same hash potential

You will live mostly in input-addressed land until the **Internals** part.

Name segment is not a version pin

The `-2.12.1` label is human hinting. Trust the **hash**. Two `hello-2.12.1` builds from different nixpkgs revisions are different paths.

Theory 3 — References and the closure

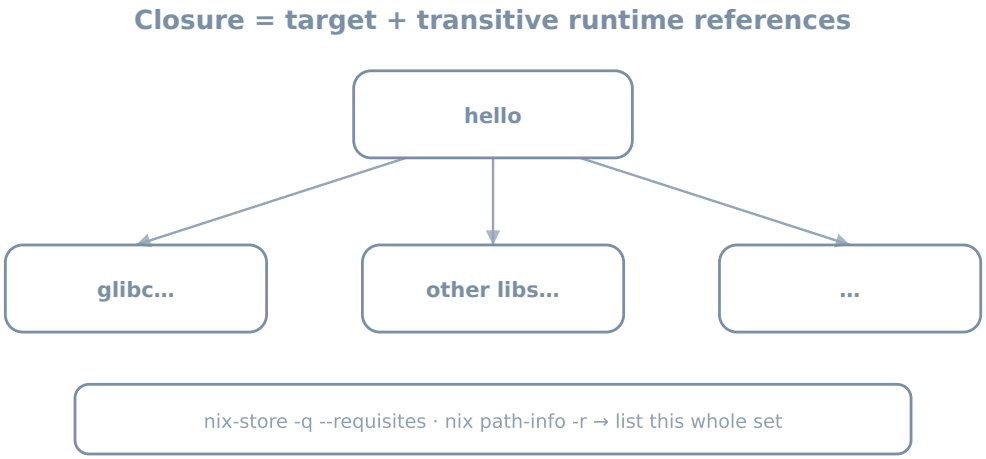


Figure 2: Closure as a reference graph

Direct references

A store path's **references** are other store paths it mentions (dynamic linker paths, wrapped scripts, metadata). Query:

```
nix-store -q --references "$P"
```

Closure (requisites)

The **closure** of a path **P** is the smallest set **S** such that:

1. **P** is in **S**
2. If **x** is in **S** and **x** references **y**, then **y** is in **S**

Operationally:

```
nix-store -q --requisites "$P"
nix path-info -r "$P"
```

Why closures matter — worked scenarios

Scenario	Why closure is the unit
Copy software to another host	Need all runtime deps
Binary cache upload	Serve complete graphs
Disk usage of "hello"	Sum of requisites, not one path
Container image from Nix	Pack a closure
NixOS system activation	Whole system is a giant closure

Example narrative

```
hello
    glibc
        …
    (other runtime inputs)
```

`hello` is a thin node; **glibc** often dominates size. That is normal.

Closure size theory

```
nix path-info -S "$P"      # approx size of one path
nix path-info -rS "$P"     # recursive / closure-oriented sizing
```

Compare single vs recursive. The gap is the theory lesson.

Referrers (reverse edges)

```
nix-store -q --referrers "$P"
```

Useful when asking: “who still holds this library?”

Theory 4 — Evaluation vs realization (store-centric)

Recall the Why Nix exists chapter:

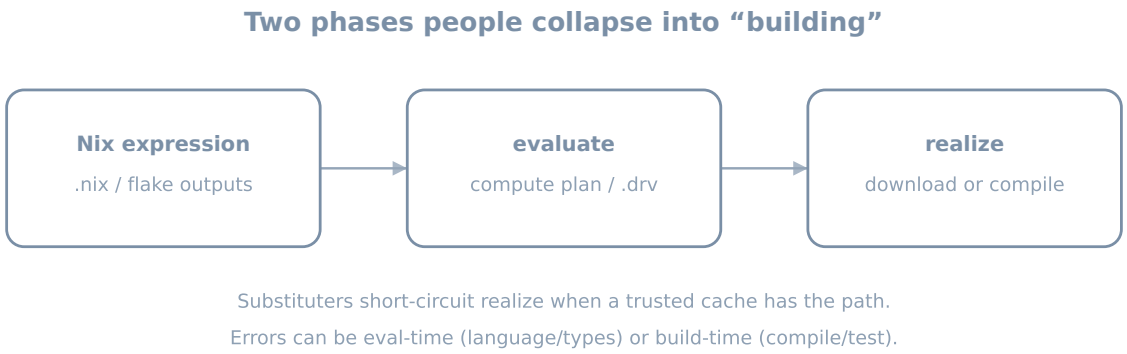


Figure 3: Evaluate then realize

Artifact	Typical location / form
Expression	`.nix` / flake
Derivation	`.drv` in store (build plan)
Output path	realized directory in store

Example commands

```
# realize outputs (modern)
nix build nixpkgs#hello --print-out-paths

# classic query tools still educational
nix-store -q --tree "$HELLO" | head
```

You can evaluate without realizing in some workflows; beginners usually realize first because they want a runnable binary.

Theory 5 — Profiles, generations, and roots

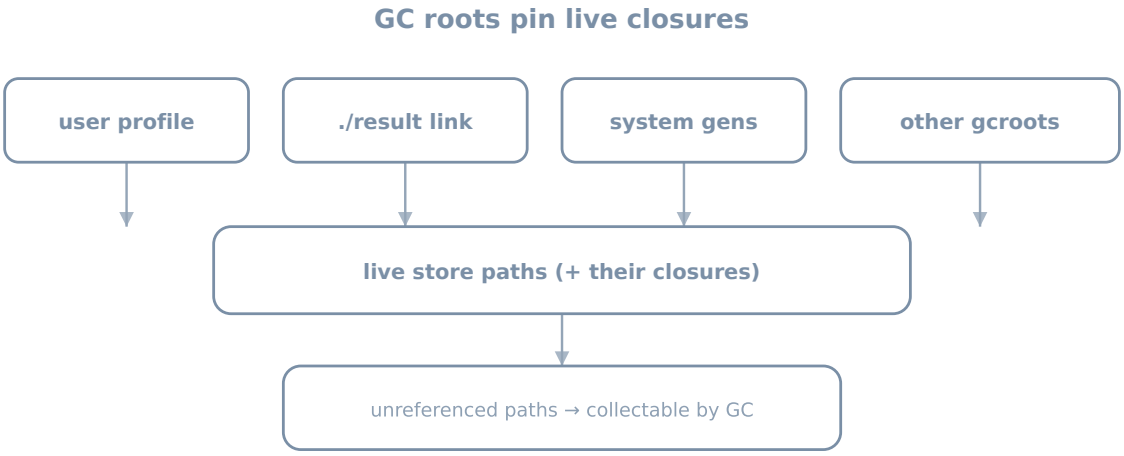


Figure 4: GC roots keep paths live

Why GC exists

Every experiment can add paths. Without collection, disks fill. Collection must not delete paths still **needed**.

Root types (non-exhaustive)

Root kind	Example
User profile generations	`~/.nix-profile`, state profiles
Result symlinks	`./result` from `nix build`
NixOS system profiles	`/nix/var/nix/profiles/system*` (host part)
Indirect roots under gcroots	Registered links
Running processes	Open files can keep paths live temporarily

The `./result` trap (critical theory)

```
nix build nixpkgs#hello
# creates ./result -> /nix/store/…-hello-…
```

That symlink is a **GC root**. Leave hundreds of `result` links in temp dirs → permanent pins → "mystery" disk use.

Discipline: `rm result` when finished experimenting, or use `--no-link`.

Generations theory (preview)

Profiles are versioned. Rolling back a profile generation re-points the user environment. Deleting old generations **makes previously live closures collectable** if nothing else roots them.

> **!** Important
>
> Do **not** run `nix-collect-garbage -d` on a machine you care about until you understand generations. Observation only in this chapter.

Theory 6 — Substituters, signatures, trust (preview)

When realizing path **P**:

1. Ask configured **substituters** (e.g. `cache.nixos.org`)
2. Verify **signatures** against trusted keys
3. Else build locally

Trust model punchline: a random HTTP server offering store paths is not enough; signatures matter. Self-hosted caches (later packaging/ops chapters) require key management.

```
nix config show | grep -i substituter || true
nix path-info --json nixpkgs#hello 2>/dev/null | head -c 400
```

Theory 7 — What "uninstall" means

Action	Effect
Remove package from profile	Drop root edge; paths may remain
Delete project directory	Does not delete store
`rm result`	Drop one root
GC	Delete unreferenced paths

Example misconception

```
"I deleted ~/code/foo, so Nix freed 2GB"
→ False unless GC ran and nothing else rooted those paths
```

Operational uninstall checklist

1. Remove declarative/profile references
2. Remove `result` and other roots
3. Optionally drop old generations
4. Run GC deliberately

Theory 8 — Copying closures (ops preview)

```
# conceptual modern/classic tools
nix copy --to ssh://host "$P"
# or nix-copy-closure in classic literacy
```

Deploy systems (later parts) are sophisticated closure movers. The unit of transfer is still the **closure**.

Worked examples bank

Example A — Build and print path

```
HELLO=$(nix build nixpkgs#hello --print-out-paths --no-link)
echo "$HELLO"
"$HELLO/bin/hello"
```

Example B — Direct vs full graph

```
nix-store -q --references "$HELLO"
nix-store -q --requisites "$HELLO" | wc -l
```

Expect requisites count references count.

Example C — Tree view

```
nix-store -q --tree "$HELLO" | head -60
```

Example D — Result root lifecycle

```
cd /tmp
rm -f result
nix build nixpkgs#hello
readlink result
# inspect roots (command variants exist)
nix-store --gc --print-roots 2>/dev/null | grep -i result || true
rm result
```

Example E — Path info metadata

```
nix path-info -sh "$HELLO"
nix path-info -rS "$HELLO" | tail
```

Example F — Compare two packages

```
H=$(nix build nixpkgs#hello --print-out-paths --no-link)
G=$(nix build nixpkgs#git --print-out-paths --no-link)
echo "hello reqs: $(nix-store -q --requisites "$H" | wc -l)"
echo "git reqs:   $(nix-store -q --requisites "$G" | wc -l)"
```

Lab 1 — Realize `hello` and dissect the path

```
mkdir -p ~/lab/nixos-book/concepts/store
cd ~/lab/nixos-book/concepts/store
HELLO=$(nix build nixpkgs#hello --print-out-paths --no-link)
echo "$HELLO"
ls -la "$HELLO"
ls -la "$HELLO/bin"
```

Journal: full path, name segment, hash length (approx).

Lab 2 — Closure measurements

```
echo "direct refs: $(nix-store -q --references "$HELLO" | wc -l)"
echo "requisites:  $(nix-store -q --requisites "$HELLO" | wc -l)"
nix path-info -S "$HELLO"
nix path-info -rS "$HELLO" | tail -5
```

Write one sentence: *What dominates the closure?*

Lab 3 — Follow one dependency

```
nix-store -q --references "$HELLO"
# pick a non-hello path, ls it
```

Observe: another complete package prefix, not a classic `/usr` layout.

Lab 4 — Roots observation

```
ls -la /nix/var/nix/gcroots 2>/dev/null | head
ls -la ~/.nix-profile 2>/dev/null
nix profile list 2>/dev/null || true
```

Perform Example D (`result` create/remove). Note difference.

Lab 5 — Teach-back

Without notes, define:

1. Store path
2. Reference
3. Closure
4. GC root

5. Why `rm -rf project` free disk

Exercises

1. Parse five real store paths into hash + name; ignore the pretty version for trust decisions.
2. Compute requisites count for `hello` and one larger package (`git` or `python3`).
3. Find the single largest path in `hello`'s closure via `path-info`.
4. Create three `result` links in different temp dirs; list them via print-roots; clean up.
5. Explain why binary cache upload granularity is closure-shaped.
6. Diagram references vs referrers for a tiny graph (3 nodes).
7. Measure disk before/after realizing a package **without** GC (observe growth).
8. Document every GC root kind you can observe on your machine.
9. Use `nix-store -q --tree` and annotate three levels of the tree.
10. Write a one-page "store operator card" for future-you.
11. Compare `--no-link` vs default `result` workflow for lab hygiene.
12. Predict whether deleting `~/.cache` frees store space (then verify: it does not).

Common pitfalls

Symptom	Theory
Disk full after demos	Roots + caches + no GC
`result` broken link	GC collected after root removed
"Uninstalled but still huge"	Roots remain; GC not run
Different hash on friend machine	Different nixpkgs revision
Permission denied writing store	Correct — immutability
Trusting package *name* only	Hash is identity
Blind `nix-collect-garbage -d`	Drops generations/roots you may need

Symptom	Theory
Thinking `/nix/store` is a cache you can wipe casually	It is the object database

---

Checkpoint

- ☐ Parse a store path into hash + name
- ☐ Define closure as a transitive reference set
- ☐ Show requisites count for `hello`
- ☐ Explain `./result` as a GC root
- ☐ List 2 root kinds
- ☐ Agree not to blind `-d` GC yet
- ☐ Single-path vs closure size surprise written down

Journal prompts

1. Closure vs single-path size surprise
2. Where *you* will accidentally create roots
3. One deploy implication of closures

Further depth (this book)

Topic	Chapter / part
Derivations producing paths	Derivations as an idea
Modern queries (`path-info`)	Modern Nix CLI
System generations as roots	Host: Rebuild and generations
GC hygiene in ops	Part **Ops and fleet**
Store daemon internals	Part **Internals**

Nix language: values and attrsets

Nix language: values and attrsets

Goal: Read and write Nix **values**, **attrsets**, `let…in`, and `inherit` well enough to model config-shaped data and follow nixpkgs snippets.

> **Note**
>
> This chapter is language only—no flakes, no NixOS rebuilds. Everything runs in `nix repl` or `nix eval`.

Why this chapter matters

NixOS configuration *is* nested attribute sets with types and merge rules layered on top. If attrsets feel alien, every later chapter becomes copy-paste. Master the data model once; modules will make sense.

Ops motivation: reading an error about a missing attribute, writing a host model, and reviewing a teammate's module all require fluent set/list/string literacy—not memorizing every option.

Theory 1 — Values are few; composition is everything

Nix has a small value inventory:

Kind	Examples	Notes
Integer	`1`, `-3`	Arbitrary precision integers
Float	`1.5`	Prefer integers unless you need floats
Boolean	`true`, `false`	
String	`"hello"`, `''multi''`	Two quoting styles
Path	`./foo.nix`, `/etc/hosts`	Distinct from strings
Null	`null`	Often "unset" in APIs
List	`[ 1 2 3 ]`	Heterogeneous allowed; spaces separate elements
Attrset	`{ a = 1; b = "x"; }`	The backbone
Function	`x: x + 1`	the language functions chapter

Type inspection

```
builtins.typeOf 1           # "int"
builtins.typeOf "x"         # "string"
builtins.typeOf ./foo       # "path"
builtins.typeOf { a = 1; } # "set"
builtins.typeOf [ 1 2 ]     # "list"
builtins.typeOf (x: x)      # "lambda"
```

Punchline: almost all "config" is **attrsets of attrsets**, with lists of packages/strings hanging off leaves.

Theory 2 — Strings: interpolation and two quote forms

Double quotes

```
let name = "yarara"; in "NixOS ${name}"
# → "NixOS yarara"
```

`${…}` interpolates an expression that stringifies cleanly (strings, paths, numbers, …).

Indented strings (`'' … ''`)

```
''
  line one
  line two
''
```

Common for shell scripts and multi-line text. Leading indentation is stripped relative to the least-indented line.

Escaping notes

Need	Approach
Literal `${` in `''` strings	`''${`
Nested quotes	Prefer indented strings for scripts
Path into string	`"${./file}"` copies/context rules apply carefully

Paths vs path-looking strings

```
./config.nix     # path - can be imported, copied into store
"./config.nix"   # string - just characters
```

Import expects a **path** (or something that resolves to one). Confusing the two is a classic beginner bug.

Theory 3 — Lists

```
[ "git" "vim" "curl" ]
[ 1 2 (3 + 4) ]
```

- **No commas** between elements
- Parenthesize subexpressions when needed: `[ (f x) y ]`
- Concatenate with `++`: `[ 1 2 ] ++ [ 3 ]`

Lists are **ordered**. Attrsets are **keyed**. Use the right shape:

Need	Prefer
Ordered packages on `PATH`	list
Named options / config tree	attrset
Set of users by name	attrset of attrsets

List builtins you will reuse

```
builtins.length [ 1 2 3 ]
builtins.head [ 1 2 3 ]
builtins.tail [ 1 2 3 ]
builtins.elemAt [ 10 20 30 ] 1
builtins.concatLists [ [ 1 ] [ 2 3 ] ]
builtins.filter (x: x > 1) [ 0 1 2 ]
builtins.map (x: x * 2) [ 1 2 3 ]
```

Theory 4 — Attribute sets (the main event)

```
{
  hostName = "lab";
  enable = true;
  ports = [ 22 80 ];
}
```

Access

```
let cfg = { hostName = "lab"; enable = true; };
in cfg.hostName
# or
cfg.${"hostName"}
```

Nested sets

```
{
  networking = {
    hostName = "lab";
    firewall = {
      enable = true;
      allowedTCPPorts = [ 22 ];
    };
  };
}
```

This is the *shape* of NixOS config, even before the module system:

```
config
   networking
        hostName
        firewall
             enable
             allowedTCPPorts
```

Dynamic keys

```
let key = "hostName";
in { ${key} = "lab"; }
```

Quoted keys for unusual names: `{ "my-key" = 1; }`.

Update / merge intuition

```
# shallow "override" via //
{ a = 1; b = 2; } // { b = 9; c = 3; }
# → { a = 1; b = 9; c = 3; }
```

`//` is **shallow**. Nested sets are replaced wholesale unless you merge carefully (later: `lib.recursiveUpdate`, module merge).

```
{ n = { x = 1; }; } // { n = { y = 2; }; }
# → { n = { y = 2; }; }   # x is gone!
```

Attrset builtins

```
builtins.attrNames { z = 1; a = 2; }
builtins.attrValues { a = 10; b = 20; }
builtins.hasAttr "a" { a = 1; }
builtins.getAttr "a" { a = 1; }
builtins.removeAttrs { a = 1; b = 2; } [ "a" ]
```

Theory 5 — `let … in` (local bindings)

```
let
  a = 1;
  b = a + 2;
in
  a + b
# → 4
```

Rules that matter

1. Bindings in one `let` can refer to each other (order-independent among pure values).
2. Scope ends at the `in` expression.
3. Prefer `let` over repeating huge expressions.
4. Infinite recursion is possible if a binding depends on itself without a fixed point.

```
let x = x + 1; in x   # boom: infinite recursion
```

Nested lets vs one flat let

Either works; flat is often clearer for small configs:

```
let
  host = "lab";
  user = "alice";
  packages = [ "git" "vim" ];
in {
  networking.hostName = host;
  users.users.${user}.isNormalUser = true;
  environment.systemPackages = packages; # shape only - not real pkgs yet
}
```

`rec` attrsets

```
rec {
  a = 1;
  b = a + 1; # refers to a in the same set
}
```

Useful; easy to create cycles. Prefer `let` when clarity wins.

Theory 6 — `inherit` (pull names into sets or lets)

Into an attrset

```
let
  pkgs = { hello = "h"; git = "g"; };
  system = "x86_64-linux";
in {
  inherit pkgs system;
  # equivalent to: pkgs = pkgs; system = system;
}
```

Rename form

```
let hello = "world"; in { inherit ( { x = hello; } ) x; }
```

Common real form:

```nix
{ inherit (pkgs) git vim curl; }
# → { git = pkgs.git; vim = pkgs.vim; curl = pkgs.curl; }
```

Into `let`

```nix
let
  inherit (builtins) typeOf attrNames;
in
  typeOf { a = 1; }
```

When to inherit: the name in the outer scope *is* the name you want. When names differ, write the binding explicitly.

Theory 7 — `with` (know it; use sparingly)

```nix
with builtins; typeOf [ 1 2 ]
```

`with set; expr` injects `set`'s attributes into scope. Powerful and **easy to shadow silently**.

```nix
let
  lib = { id = x: x; };
  id = "oops";
in
  with lib; id   # which id?
```

House rule for this book: prefer `let inherit (lib) …` or qualified `lib.foo` over broad `with lib;`. You will still *read* `with` in older code.

Theory 8 — Conditionals and operators (minimal set)

```nix
if true then "a" else "b"

true && false
true || false
!true

1 == 1
1 != 2
```

```
# ordering on numbers/strings exists; avoid over-cleverness
```

`assert cond; value` fails evaluation if `cond` is false—useful later for module assertions; try once in this chapter:

```
let x = 2; in assert x > 0; x
```

Theory 9 — Config-shaped modeling (bridge to NixOS)

You are not writing a full module yet. You are practicing the **data shape**:

```
let
  hostName = "lab-vm";
  admin = "alice";
  sshPort = 22;
in {
  networking = {
    inherit hostName;
    firewall = {
      enable = true;
      allowedTCPPorts = [ sshPort ];
    };
  };
  users.users.${admin} = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
  };
  environment.systemPackages = [
    "git"   # stand-ins; real pkgs are derivations later
    "vim"
  ];
}
```

Notice:

- Nested attrsets mirror option paths
- `${admin}` dynamic key for username
- `inherit hostName` avoids repetition
- Lists for multi-valued options

Theory 10 — Destructuring patterns you will see

```
let
  set = { a = 1; b = 2; };
  inherit (set) a b;
in
  a + b
```

```
# nested access sugar people write in modules
config.networking.hostName
```

Reading real nixpkgs becomes pattern matching on these shapes—not learning a second language.

Worked examples bank

Example A — REPL tour

```
nix repl
```

```
1 + 1
{ a = 1; b = { c = 2; }; }.b.c
let x = "nix"; in "hello ${x}"
[ 1 2 ] ++ [ 3 ]
{ a = 1; } // { b = 2; }
builtins.attrNames { z = 1; a = 2; }
```

Example B — Deep access and missing attrs

```
let
  cfg = { networking.firewall.enable = true; };
in
  cfg.networking.firewall.enable
```

Missing attribute → evaluation error. That is good: fail early.

Example C — `inherit` package list pattern

```
let
  tools = {
    git = "git-placeholder";
    vim = "vim-placeholder";
    curl = "curl-placeholder";
  };
in {
  environment.systemPackages = with tools; [ git vim curl ];
  # or better without with:
  # environment.systemPackages = [ tools.git tools.vim tools.curl ];
}
```

Example D — Shallow merge trap

```
let
  base = { services.foo = { enable = true; port = 80; }; };
  overlay = { services.foo = { port = 8080; }; };
in
  base // overlay
# services.foo.enable disappeared - shallow //
```

Journal this; module system merges differently later (host modules chapters).

Example E — Dynamic user keys

```
let
  mkUser = name: {
    users.users.${name} = {
      isNormalUser = true;
    };
  };
in
  mkUser "alice"
```

(Functions deepen in the next chapter; the attrset shape is the point here.)

Lab 1 — Workspace and file evaluation

```
mkdir -p ~/lab/nixos-book/concepts/attrsets
cd ~/lab/nixos-book/concepts/attrsets
```

Create `host-model.nix`:

```
let
  hostName = "lab";
  adminUser = "alice";
  packages = [ "git" "vim" "htop" ];
in {
  networking = {
    inherit hostName;
    firewall.enable = true;
    firewall.allowedTCPPorts = [ 22 ];
  };
  users.users.${adminUser} = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
  };
  environment.systemPackages = packages;
}
```

Evaluate:

```
nix eval --file ./host-model.nix
nix eval --file ./host-model.nix --apply 'cfg: cfg.networking.hostName'
```

Lab 2 — Nested merge experiments

In `merge-trap.nix`, reproduce Example D. Also try nested manual merge:

```
let
  base = { services.foo = { enable = true; port = 80; }; };
  overlay = { services.foo = { port = 8080; }; };
  deep =
    base
    // {
      services = base.services // {
        foo = base.services.foo // overlay.services.foo;
      };
    };
```

```
in
  deep
```

Confirm `enable` survives. Write one sentence on why `//` alone is not "config merge."

Lab 3 — Attrset utilities in the REPL

```
builtins.attrNames { c = 1; a = 2; b = 3; }
builtins.attrValues { a = 10; b = 20; }
builtins.hasAttr "a" { a = 1; }
builtins.getAttr "a" { a = 1; }
```

Build a tiny "user directory":

```
let
  users = {
    alice = { uid = 1000; shell = "bash"; };
    bob = { uid = 1001; shell = "zsh"; };
  };
in {
  names = builtins.attrNames users;
  aliceShell = users.alice.shell;
}
```

Lab 4 — Model something real from your life

Expand `host-model.nix` (or a second file) to include *fictional but realistic* keys for:

1. Hostname + domain
2. Two users (one admin)
3. Three "packages" as strings
4. Firewall ports for SSH + one app
5. A `services.myapp = { enable = true; … }` nest

Use `let` + `inherit` at least twice. No need for real NixOS options names to validate—accuracy of *shape* matters.

Lab 5 — Teach-back

Without looking up, write:

1. Difference between path and string
2. Why lists use spaces not commas
3. What `//` does to nested sets
4. When you would use `inherit`
5. Why broad `with` is risky

Exercises

1. Write attrsets for: web service config, user map, package list—choose list vs set deliberately.
2. Convert a JSON-like mental model of your laptop hostname/firewall into Nix.
3. Break evaluation with a missing attribute; fix with a default in `let`.
4. Demonstrate `rec` vs `let` for the same binding graph.
5. Use indented strings to draft a tiny shell script body (string only).
6. Build a deep nest 4 levels deep; access the leaf two ways.
7. Implement shallow vs manual deep merge; document the difference.
8. Prefer `inherit (builtins) …` in a file and evaluate it.
9. Find three places `with` would hurt readability; rewrite without it.
10. Model two hosts in one attrset keyed by hostname.
11. Use `assert` to enforce `sshPort > 0`.
12. Read a random small snippet from nixpkgs (online) and label each value kind.

13. Explain why NixOS options look like nested attrsets even though types exist.

14. Commit your `~/lab/nixos-book/concepts/attrsets` experiments.

Common pitfalls

Symptom	Theory
`error: undefined variable`	Scope / missing `let` binding
`unexpected ','` in list	No commas in Nix lists
Path vs string import confusion	Paths are a distinct type
Nested keys "disappeared" after `//`	Shallow merge
`with` silently shadows	Prefer explicit / `inherit`
Dynamic key without `${}`	Use `{ ${name} = …; }`
Infinite recursion in `let`	Self-reference without base
Semicolon missing in attrset	Every binding ends with `;`
Treating lists as sets	Wrong shape for named config

Checkpoint

☐ Named all major value kinds

☐ Wrote nested attrset for host-shaped config

☐ Used `let…in` and `inherit` deliberately

☐ Demonstrated shallow `//` pitfall

☐ Evaluated a `.nix` file with `nix eval`

☐ Personal notes include 3 gotchas

☐ Can choose list vs attrset for a new field

Further depth (this book)

Topic	Chapter
Functions over these values	Nix language: functions
Multi-file + lib helpers	Laziness, imports, and lib
Module merge (not `//`)	Host: Modules as a consumer
Real host config trees	Host: Flake-defined hosts

Nix language: functions

Nix language: functions

Goal: Write and call Nix functions fluently—especially **attribute-set pattern arguments**, defaults, `@`-binding, and ellipsis—so flake and module snippets stop looking like magic.

Why this chapter matters

In Nix, packages, shells, overlays, and modules are **functions returning values**. If you only know `x: x + 1`, you cannot read nixpkgs. This chapter is the real functional core.

Ops motivation: every flake `outputs = { self, nixpkgs, ... }: …` and every module `{ config, lib, pkgs, ... }: …` is the same skill. Fluency here turns "paste and pray" into "read and adapt."

Theory 1 — Everything callable is a lambda

```
let
  inc = x: x + 1;
  add = x: y: x + y;   # curried
in
  add 2 3              # 5
```

Application is juxtaposition

```
f x        # call f with x
f x y      # (f x) y if curried
f { a = 1; }  # call with one attrset arg
```

Parenthesize when needed: `(f x) + 1`, `map (x: x + 1) list`.

Functions are values

```
let
  ops = {
    inc = x: x + 1;
    double = x: x * 2;
  };
in
  ops.double (ops.inc 3)  # 8
```

Identity and composition (lightweight)

```
let
  id = x: x;
  compose = f: g: x: f (g x);
in
  compose (x: x + 1) (x: x * 2) 3  # 7
```

Theory 2 — Attrset pattern arguments (the Nix idiom)

Almost every public Nix API looks like:

```
{ a, b }: a + b
```

Call site:

```
({ a, b }: a + b) { a = 1; b = 2; }
```

Destructuring is the point

```
{ hostName, user }: {
  networking.hostName = hostName;
  users.users.${user}.isNormalUser = true;
}
```

You **name** the fields you need. Extra fields are an error unless you allow them (ellipsis—below).

Why not positional for public APIs?

Positional	Attr pattern
Order fragile	Names document call sites
Hard to default	`?` defaults natural
Hard to extend	Ellipsis / optional fields

Theory 3 — Default arguments with `?`

```
{ force ? false, retries ? 3 }: {
  inherit force retries;
}
```

```
f { }                    # force=false, retries=3
f { force = true; }      # force=true, retries=3
f { retries = 10; }      # force=false, retries=10
```

Order of attributes in the pattern does not matter at the call site; names bind by key.

Defaults can be expressions

```
{ pkgs, system ? builtins.currentSystem }:
  # system falls back if omitted
  { inherit pkgs system; }
```

Be careful: defaults that force heavy evaluation run when the binding is **used** (laziness—the next chapter), but bad defaults can still surprise you.

Explicit `null` vs missing key

```
# if you pass force = null, the default is NOT used
# defaults apply only when the attribute is absent
```

Theory 4 — Ellipsis ... (allow unknown attributes)

```
{ a, b, ... }: a + b
```

Call with extras:

```
({ a, b, ... }: a + b) { a = 1; b = 2; c = 99; }  # 3 - c ignored
```

Without ...:

```
({ a, b }: a + b) { a = 1; b = 2; c = 99; }
# error: unsupported argument 'c'
```

When to use ellipsis

Situation	Ellipsis?
Strict public API	Often **no**—catch typos
Pass-through / forward kwargs	**Yes**
Module-style "rest of config"	Sometimes

House rule: use ... when forwarding; avoid it when you want misspelled option names to fail loudly.

Theory 5 — @ binding (name the whole set)

```
args@{ a, b, ... }: {
  inherit a b;
  allKeys = builtins.attrNames args;
}
```

Or:

```
{ a, b, ... }@args: …  # same idea, different sugar position
```

Why `@` exists

1. Forward the full attrset to another function
2. Inspect extras
3. Merge overrides

```
let
  f = { a, b ? 0, ... }@args:
    g (args // { b = b + 1; });
in
  f
```

Classic flake-shaped pattern:

```
outputs = inputs@{ self, nixpkgs, ... }:
  let
    system = "x86_64-linux";
    pkgs = import nixpkgs { inherit system; };
  in {
    packages.${system}.default = pkgs.hello;
  };
```

You will write this shape in the flakes chapter; understand the `inputs@{ … }` part **now**.

Theory 6 — Composition and higher-order helpers

`map` / `filter` (builtins)

```
map (x: x * 2) [ 1 2 3 ]
# [ 2 4 6 ]

builtins.filter (x: x > 1) [ 0 1 2 3 ]
# [ 2 3 ]
```

Function returning function

```
let
  multiply = n: x: n * x;
  triple = multiply 3;
in
```

```
  triple 4  # 12
```

`enableIf`-style helper (preview of module style)

```
let
  enableIf = cond: value: if cond then value else { };
in
  enableIf true { services.foo.enable = true; }
```

Real modules use `lib.mkIf`—same *idea*, better merge semantics (host modules chapters).

Theory 7 — Fixed-point awareness (not mastery)

You will see:

```
lib.fixpoint (self: {
  a = 1;
  b = self.a + 1;
})
```

Idea: define an attrset in terms of its final form. Overlays and package sets use this heavily. In this chapter: recognize it; do not invent fixed points for homework.

Informal:

```
self → { a = 1; b = self.a + 1; } → { a = 1; b = 2; }
```

Theory 8 — Call patterns you will see in the wild

1. Package "callPackage" shape

```
{ lib, stdenv, fetchurl }:
stdenv.mkDerivation { … }
```

Someone injects dependencies by calling the function with an attrset of packages.

2. Overlay shape

```
final: prev: {
  myPkg = prev.hello.overrideAttrs (old: { … });
}
```

Two arguments, both attrsets of packages (final = after overlays, prev = before this overlay).

3. Module shape (preview)

```
{ config, lib, pkgs, ... }: {
  options = { … };
  config = { … };
}
```

Ellipsis is standard—modules receive many arguments.

4. `override` / `overrideAttrs` as functions

Packages often expose functions that take functions:

```
pkgs.hello.overrideAttrs (old: {
  pname = old.pname;
})
```

You are passing a function that receives `old` attrs—same language skill.

Theory 9 — Error messages as teachers

Error fragment	Likely cause
`called without required argument 'x'`	Missing key in attr call
`unsupported argument 'y'`	Extra key; need `...` or remove
`cannot coerce a function to …`	Forgot to call `f` / `f args`
`value is a function while a set was expected`	Passed lambda where attrset needed
Infinite recursion	Self application without base

Worked examples bank

Example A — Defaults + ellipsis

```
let
  mkService = { name, port ? 8080, enable ? true, ... }: {
    inherit name port enable;
  };
in
  mkService { name = "api"; host = "ignored-extra"; }
```

Example B — `@` forward

```
let
  core = { a, b }: a + b;
  wrap = args@{ a, b ? 0, ... }:
    core { inherit a; b = b + 1; };
in
  wrap { a = 10; b = 1; extra = true; }
# → 12
```

Example C — User skeleton helper

```
let
  mkUser = {
    name,
    groups ? [ "wheel" ],
    keys ? [ ],
  }: {
    users.users.${name} = {
      isNormalUser = true;
      extraGroups = groups;
      openssh.authorizedKeys.keys = keys;
    };
  };
in
  mkUser {
    name = "alice";
    keys = [ "ssh-ed25519 AAAA… alice@lab" ];
  }
```

Example D — Package list merge helper

```
let
  mergePkgs = lists: builtins.concatLists lists;
in
  mergePkgs [
    [ "git" "vim" ]
    [ "curl" ]
    [ "htop" ]
  ]
```

Example E — Strict API for typos

```
let
  # no ellipsis - typo "hostNme" fails loudly
  mkHost = { hostName, user }: {
    inherit hostName user;
  };
in
  mkHost { hostName = "lab"; user = "alice"; }
```

Lab 1 — Workspace

```
mkdir -p ~/lab/nixos-book/concepts/functions
cd ~/lab/nixos-book/concepts/functions
```

Create `helpers.nix`:

```
rec {
  inc = x: x + 1;

  add = { a, b ? 0 }: a + b;

  mkHost = { hostName, user, sshPort ? 22, ... }: {
    networking = {
      inherit hostName;
      firewall.allowedTCPPorts = [ sshPort ];
    };
    users.users.${user}.isNormalUser = true;
  };
```

```
  enableIf = cond: attrs: if cond then attrs else { };

  mergeLists = builtins.concatLists;
}
```

Evaluate pieces:

```
nix eval --file ./helpers.nix --apply 'h: h.add { a = 2; b = 3; }'
nix eval --file ./helpers.nix --apply 'h: h.mkHost { hostName = "lab"; user = "alice"; }'
```

Lab 2 — Strict vs ellipsis

Create `strict.nix` and `loose.nix`:

```
# strict.nix
{ a, b }: a + b
```

```
# loose.nix
{ a, b, ... }: a + b
```

```
nix eval --file ./strict.nix --apply 'f: f { a = 1; b = 2; c = 3; }'   # should fail
nix eval --file ./loose.nix  --apply 'f: f { a = 1; b = 2; c = 3; }'   # 3
```

Journal: when would failure be *better*?

Lab 3 — `@` args inventory

```
# inspect.nix
args@{ a, b ? 1, ... }: {
  sum = a + b;
  keys = builtins.attrNames args;
  raw = args;
}
```

```
nix eval --file ./inspect.nix --apply 'f: f { a = 5; c = "extra"; }'
```

Confirm `keys` includes `c` and default `b` behavior matches your mental model.

Lab 4 — Compose three helpers into a "mini host"

In `mini-host.nix`, import/use helpers to produce one attrset that includes:

1. Hostname + admin user (via `mkHost`)
2. Conditionally enabled `services.demo` with `enableIf`
3. Merged package string lists

```
nix eval --file ./mini-host.nix
```

Lab 5 — Read real shapes (optional but high value)

If network/cache available:

```
nix repl
```

```
:lf nixpkgs
# skim types only - do not deep dive packaging yet
builtins.typeOf pkgs.hello
```

Open any small module online or local nixpkgs checkout later—note the header:

```
{ config, lib, pkgs, ... }:
```

Map each binder to this chapter's theory.

Exercises

1. Write curried `add` and attr-pattern `add`; call both.
2. Implement `mkUser` with defaults for shell and groups.
3. Show that passing `retries = null` does not use the `?` default.
4. Build a strict validator function that rejects unknown keys (no `...`).
5. Use `@` to log (via returned attrset) all keys passed to a function.

6. Write `map`/`filter` pipeline over a list of host attrsets.
7. Sketch an overlay `final: prev: …` that renames nothing—just returns `{}`.
8. Translate a Python `def f(a, b=1, **kw)` mental model into Nix patterns.
9. Break and fix parentheses around lambdas in a list: `[ (x: x) ]`.
10. Read `outputs = { self, nixpkgs, ... }:` and rewrite with `@inputs`.
11. Implement `enableIf` and contrast with raw `if` for empty attrsets.
12. Document three error messages you triggered and their causes.
13. Commit the functions lab directory.
14. Preview: write the module header line from memory.

Common pitfalls

Symptom	Theory
`called without required argument`	Missing pattern field
`unsupported argument '…'`	Need `...` or remove extra
Defaults not applied	You passed `null` explicitly—defaults only for *missing* keys
`f x y` vs `f {…}` confusion	Know arity/shape of f
Forgetting parentheses around lambdas in lists	`[ (x: x) ]` not `[ x: x ]`
Trying defaults with positional args	Defaults are for **attr** patterns
Infinite recursion	Self-call without base case
Ellipsis hides typos forever	Prefer strict public APIs

Checkpoint

- ☐ Write a curried and an attr-pattern function
- ☐ Use `?` defaults correctly
- ☐ Explain ellipsis trade-off (strict vs loose)

- ☐ Use `@` to inspect or forward args
- ☐ Build `mkHost` / `enableIf` style helpers
- ☐ Recognize module/overlay/flake call shapes

Further depth (this book)

Topic	Chapter
Lazy evaluation of args	Laziness, imports, and lib
Flake `outputs` function	Flakes and project devShells
Module functions in production	Host: Modules as a consumer, Writing custom modules
`callPackage` packaging	Part **Packaging**

Laziness, imports, and lib

Laziness, imports, and lib

Goal: Split expressions across files, understand **lazy values vs eager imports**, and run useful `lib` pipelines without memorizing the whole standard library.

Why this chapter matters

Real Nix is multi-file. People hit "infinite recursion," mysteriously slow eval, or "why did that download?" without a model of **when** work happens. This chapter installs that model and introduces `lib` as the toolbox you will touch forever.

Ops motivation: large host configs, shared modules, and nixpkgs itself only evaluate tractably because of laziness. Knowing what *forces* a value is how you debug slow evals and cycles.

Theory 1 — Laziness: values are promises

Nix does not compute a binding until something **needs** its value.

```
let
  a = builtins.trace "forcing a" (1 + 1);
  b = builtins.trace "forcing b" (2 + 2);
in
  a
```

Only `"forcing a"` prints—`b` is never forced.

Why it matters

Benefit	Example
Huge package sets	Unused packages not fully evaluated
Optional config	Disabled modules can avoid heavy work
Recursive structures	Attrsets can refer to each other carefully

Forcing

Anything that **inspects** the value forces it: arithmetic, string interpolation of it, `builtins.length`, taking an attribute that needs it, building a derivation that lists it as input, etc.

```
let
  expensive = builtins.trace "boom" (import ./big.nix);
in
  if false then expensive else 0
# "boom" should not appear - branch not taken
```

Mental model

```
binding  =  thunk (unevaluated)
use in expr  →  force thunk  →  value (cached for later forces)
```

Theory 2 — Imports are different: loading is eager (once forced)

```
import ./other.nix
```

`import` reads and parses the file and evaluates the root expression of that file **when the import expression is forced**. People say "imports are eager" meaning: **once you import, that file's top-level expression is evaluated** (subject to laziness *inside* that file's bindings).

Mental model

```
import path
  → read file
  → parse
  → evaluate root expression of that file
  → return the resulting value (function, attrset, …)
```

Common patterns

```
# other.nix returns an attrset
import ./other.nix

# other.nix returns a function
import ./mkHost.nix { hostName = "lab"; user = "alice"; }
```

```
# relative to current file
import ./modules/ssh.nix
```

Paths

Prefer path literals so imports resolve relative to the **file**, not the process cwd:

```
import ./lib.nix          # good
import "./lib.nix"        # string - different rules; avoid casually
```

Import caching

Nix caches imports of the same path in an evaluation—knowing this helps reason about "why didn't my edit apply?" (usually: different evaluation / flake copy / not saved).

Theory 3 — Multi-file composition patterns

Pattern A — Data module

`users.nix`:

```
{
  alice = { groups = [ "wheel" ]; };
  bob = { groups = [ ]; };
}
```

`default.nix`:

```
let
  users = import ./users.nix;
in {
  names = builtins.attrNames users;
  inherit users;
}
```

Pattern B — Function module

`mkUser.nix`:

```
{ name, groups ? [ ] }: {
  users.users.${name} = {
    isNormalUser = true;
    extraGroups = groups;
  };
}
```

`default.nix`:

```
import ./mkUser.nix { name = "alice"; groups = [ "wheel" ]; }
```

Pattern C — Directory + `default.nix`

Nix allows `import ./dir` when `./dir/default.nix` exists. Useful for tiny libraries.

Pattern D — Layered host-shaped tree (preview)

```
host.nix
  imports data/users.nix
  imports lib/mkFirewall.nix
  returns nested config attrset
```

Same idea later as NixOS `imports = [ … ]`, with module merge on top.

Theory 4 — Infinite recursion (how you will meet it)

```
let
  x = y;
  y = x;
in
  x
```

```
let
  pkgs = import nixpkgs { inherit system overlays; };
  overlays = [ (final: prev: { x = pkgs.hello; }) ]; # pkgs not ready
in
  pkgs
```

Debugging habits

1. Shrink the expression
2. `builtins.trace "here" value`
3. Break cycles by not referencing the final fixed point too early
4. Remember: printing an attrset forces attributes (in some UIs)
5. Bisect files: comment imports until eval succeeds

Fixed points (recognition only)

```
lib.fixpoint (self: {
  a = 1;
  b = self.a + 1;
})
```

Cycles are intentional *only* via structured fixed points—not accidental `let` loops.

Theory 5 — `lib` as the standard library (taste, not catalog)

When you have nixpkgs:

```
lib = pkgs.lib;
# or
lib = import <nixpkgs/lib>;  # classic; flakes prefer pkgs.lib
```

Attrsets

Function	Role
`lib.mapAttrs`	Map over name/value
`lib.filterAttrs`	Filter by predicate
`lib.attrNames` / `lib.attrValues`	Keys / values
`lib.recursiveUpdate`	Deep-ish merge
`lib.optionalAttrs`	Conditionally include an attrset
`lib.nameValuePair`	Build `{ name, value }` for listToAttrs

Lists

Function	Role
`lib.concatLists`	Flatten one level
`lib.unique`	Dedup
`lib.toList`	Wrap non-list
`lib.optional`	`[ x ]` or `[]` from bool
`lib.optionals`	list or `[]` from bool
`lib.flatten`	Deeper flatten

Strings

Function	Role
`lib.concatStringsSep`	Join
`lib.hasPrefix` / `hasSuffix`	Tests
`lib.toLower`	Case
`lib.splitString`	Split

Example pipeline — users list → attrset

```
{ lib }:
let
  userList = [
    { name = "alice"; admin = true; }
    { name = "bob"; admin = false; }
  ];
in
  lib.listToAttrs (
    map (u: {
      name = u.name;
      value = {
        isNormalUser = true;
        extraGroups = lib.optional u.admin "wheel";
      };
    }) userList
  )
```

Why not memorize all of `lib`?

Learn **10–15** functions deeply. Search nixpkgs/`lib` when you need more. Module authors live in `lib.mkOption`, `lib.mkIf`, `lib.types.*` (host modules chapters).

Theory 6 — `NIX_PATH` and angle-bracket paths (literacy)

```
import <nixpkgs> { }
```

`<nixpkgs>` looks up `NIX_PATH`. This book's **default** is flakes—not channels—but you must recognize angle brackets in docs.

Style	Pinning
`<nixpkgs>`	Ambient, drift-prone
`import ./pinned.nix`	File-relative
Flake `inputs.nixpkgs`	Lock file (preferred)

Labs in this chapter should use **relative imports** between your files; if you pull nixpkgs, prefer a flake or a known pin over bare `<nixpkgs>` when you can.

Theory 7 — Evaluation vs runtime (again, file-centric)

```
let
  data = import ./data.nix;          # eval time: load nix
  # building a package would realize store paths - the derivations chapter
in
  data
```

Importing a Nix file does **not** compile C. It only produces Nix values (which might *describe* builds).

When does "work" happen?

Action	Phase
`import ./x.nix` forced	Eval
`lib.mapAttrs …`	Eval
`mkDerivation` resulting build	Realize
`fetchurl` network	Realize (FOD)

Theory 8 — Project layout habits

Habit	Why
One concern per file	Diffs and reuse
`default.nix` as facade	Clean imports
Data vs functions separated	Test data without logic
Avoid 2k-line single files	Reviewability

Later flake layouts expand this (Home and flake layout part).

Worked examples bank

Example A — Lazy skip

```nix
let
  boom = builtins.abort "should not run";
in
  if true then 1 else boom
```

Example B — Import function

`add.nix`:

```nix
{ a, b }: a + b
```

```nix
import ./add.nix { a = 2; b = 40; }
```

Example C — `lib.optional` / `optionals`

```nix
lib.optional true "wheel"          # [ "wheel" ]
lib.optional false "wheel"         # [ ]
lib.optionals true [ "a" "b" ]     # [ "a" "b" ]
```

Example D — `mapAttrs`

```nix
lib.mapAttrs (name: cfg: cfg // { inherit name; }) {
  alice = { uid = 1000; };
  bob = { uid = 1001; };
}
```

Example E — `recursiveUpdate` vs `//`

```
lib.recursiveUpdate
  { services.foo = { enable = true; port = 80; }; }
  { services.foo = { port = 8080; }; }
# enable preserved (deep merge behavior of recursiveUpdate)
```

Compare to shallow `//` from the attrsets chapter.

Lab 1 — Multi-file project skeleton

```
mkdir -p ~/lab/nixos-book/concepts/laziness/{lib,data}
cd ~/lab/nixos-book/concepts/laziness
```

`data/users.nix`:

```
[
  { name = "alice"; admin = true;  shell = "bash"; }
  { name = "bob";   admin = false; shell = "zsh"; }
  { name = "carol"; admin = true;  shell = "bash"; }
]
```

`lib/users.nix`:

```
{ lib }:
userList:
lib.listToAttrs (
  map (u: {
    name = u.name;
    value = {
      isNormalUser = true;
      extraGroups = lib.optional u.admin "wheel";
      shellName = u.shell;
    };
  }) userList
)
```

`default.nix` (standalone `lib` stub so you need no nixpkgs for this lab):

```
let
  lib = {
    optional = cond: x: if cond then [ x ] else [ ];
    listToAttrs = builtins.listToAttrs;
```

```
  };
  userList = import ./data/users.nix;
  toUsers = import ./lib/users.nix { inherit lib; };
  users = toUsers userList;
in {
  inherit users;
  admins =
    map (u: u.name) (builtins.filter (u: u.admin) userList);
}
```

```
nix eval --file ./default.nix
nix eval --file ./default.nix --apply 'v: v.admins'
```

Lab 2 — Trace forcing

Create `trace.nix`:

```
let
  a = builtins.trace "force-a" 1;
  b = builtins.trace "force-b" 2;
  c = a + 10;
in
  c
```

```
nix eval --file ./trace.nix
```

Edit so the result uses `b` instead; confirm traces change. Record observations.

Lab 3 — Infinite recursion (safe sandbox)

```
# cycle.nix - expected to fail
let x = y; y = x; in x
```

```
nix eval --file ./cycle.nix || true
```

Write a one-line description of the error. Then fix a *near-cycle* by breaking the dependency.

Lab 4 — Real `lib` if nixpkgs available

```
nix repl
```

```
:lf nixpkgs
lib = pkgs.lib
lib.id 42
lib.concatStringsSep "," [ "a" "b" "c" ]
lib.filterAttrs (n: v: v > 1) { a = 1; b = 2; c = 3; }
lib.recursiveUpdate { a.b = 1; } { a.c = 2; }
```

Port Lab 1's `toUsers` to use `pkgs.lib` instead of the stub (optional stretch).

Lab 5 — Teach-back

1. Lazy value vs import of a file
2. Why multi-file beats one 2k-line expression
3. One `lib` function you will actually reuse
4. Why `<nixpkgs>` is ambient state

Exercises

1. Prove with `trace` that an unused `let` binding is not forced.
2. Force a binding via string interpolation and via `builtins.length`.
3. Split a host model into `data.nix` + `logic.nix` + `default.nix`.
4. Import a directory via `default.nix` facade.
5. Rebuild the shallow merge trap using `lib.recursiveUpdate`.
6. Write `lib.optionalAttrs` equivalents by hand; then use real `lib`.
7. Trigger and explain one infinite recursion from overlays-shaped code.

8. Map a list of services into an attrset with `listToAttrs`.
9. Document five force points you hit while exploring.
10. Compare eval time of a tiny file vs importing a heavy stub (subjective notes).
11. Replace all angle-bracket mental habits with relative imports in your lab.
12. List ten `lib.*` functions and one-line each (search as needed).
13. Commit the multi-file lab; ensure paths are file-relative.
14. Preview module `imports = [ ./a.nix ./b.nix ]` as the same composition idea.
15. Explain why disabled flakes inputs still may or may not evaluate (depends on forcing).

Common pitfalls

Symptom	Theory
Infinite recursion	Cycle in let/fixed-point
Import path not found	Relative to file vs cwd
Unexpected work during eval	Something forced a heavy binding
`cannot coerce a function`	Forgot to *call* imported function
Angle-bracket surprise versions	Unpinned `NIX_PATH`
`lib` undefined	Never imported pkgs/lib
Edit not picked up	Evaluating different file/copy
`//` where recursive merge needed	Wrong merge tool

Checkpoint

- ☐ Split data + logic across 2 files
- ☐ Explained laziness with a trace experiment
- ☐ Imported a function file and called it
- ☐ Used map/filter or `lib` pipeline on users

- ☐ Documented one recursion footgun
- ☐ Prefer relative imports over `<nixpkgs>` in new work

Further depth (this book)

Topic	Chapter / part
Derivations (when eval becomes build)	Derivations as an idea
Flake multi-file projects	Flakes and project devShells
Module imports + mkIf	Host modules chapters
Evaluator performance	Part **Internals**
Shared module libraries	Part **Ops and fleet**

Derivations as an idea

Derivations as an idea

Goal: Treat a **derivation** as a pure build plan: relate evaluation → `.drv` → store outputs; run a tiny build; read logs; explain sandbox and **fixed-output derivations (FODs)** at a conceptual level.

> **Note**
>
> Deep `stdenv` phase mastery lives in the **Packaging** part of this book. This chapter is the **idea** and a successful build loop.

Why this chapter matters

Without "derivation," the store is a pile of directories and flakes are just JSON with extra steps. Derivations are the **contracts** that produce store paths.

Ops motivation: cache misses, hash mismatches, sandbox failures, and "why did CI compile?" all sit on this abstraction. You cannot operate Nix without naming the plan.

Theory 1 — Derivation = build recipe with a promised output path

A derivation answers:

1. What **inputs** (other store paths, sources)?
2. What **builder** runs (usually bash + stdenv)?
3. What **environment** (env vars, sandbox)?
4. What **output path(s)** will hold results?

Informally:

```
derivation D
  inputs:  I1, I2, …
  builder: B
  → output path P = hash(inputs, builder, args, system, …)
```

In the common **input-addressed** model, the hash is about the plan, not the bytes of the result (contrast content-addressed—Internals part).

Vocabulary table

Term	Meaning
Derivation	Build plan object / `.drv`
Output	Realized store directory
Builder	Program that performs the build
Fixed-output derivation	Special case: hash of *content* declared
Dependent derivation	Normal build; hash of *inputs*

Theory 2 — Eval vs realize (operational)

Phase	Artifact	Command vibes
Evaluate	Nix values + derivation objects	`nix eval`, instantiate
Low-level plan	`.drv` file in the store	`nix derivation show`
Realize	Output path directories	`nix build`
Run	Binaries inside outputs	`nix run`, `./result/bin/…`

`.drv` is not the package

```
/nix/store/xxxx-hello-2.12.drv   ← plan
/nix/store/yyyy-hello-2.12       ← output (realized)
```

You ship and cache **outputs**; `.drv` explains how they were supposed to be built.

Pipeline sketch

```
flake/expression
     eval

derivation graph (.drv files)
     realize (build or substitute)

output paths in /nix/store
     references
```

```
closure
```

Theory 3 — `mkDerivation` mental model (lite)

Most software uses `stdenv.mkDerivation`:

```
stdenv.mkDerivation {
  pname = "mytool";
  version = "0.1.0";
  src = ./src;
  buildInputs = [ /* runtime/build deps */ ];
  # phases: unpack, patch, configure, build, install, fixup, …
}
```

For learning in this chapter, prefer **trivial builders** that avoid compilers:

```
pkgs.writeText "hello.txt" "hello from nix\n"

pkgs.writeShellScriptBin "greet" ''
  echo "hello from store script"
''
```

These still produce **real store paths** and teach the loop without a C toolchain fight.

Other trivial helpers (awareness)

Helper	Result
`writeText`	File in store
`writeTextFile`	More options
`writeShellScript`	Script path
`writeShellScriptBin`	`$out/bin/name`
`runCommand`	Tiny custom builder

Theory 4 — Sandbox: why builds are fenced

The build sandbox aims to make builds **pure**:

Restriction (typical Linux sandbox)	Why
Limited filesystem view	No sneaking `/usr/lib` deps
No network (usually)	Downloads must be fixed-output derivations
Clean env	Fewer "works on my machine" env vars
Only declared inputs visible	Graph stays honest

Fixed-output derivations (FODs)

Fetchers (`fetchurl`, `fetchFromGitHub`) are special: they **may** use network but must declare a **hash** of the result. Wrong hash → fail. That is how purity reconciles with downloading sources.

```
fetchurl {
  url = "https://example.com/foo.tar.gz";
  sha256 = "…";  # wrong → error
}
```

Property	Normal derivation	FOD
Network	Usually blocked	Allowed
Output hash	From inputs (plan)	Declared (content)
Failure mode	Build error	Hash mismatch / fetch fail

Why FODs matter to operators

- Pinning sources makes rebuilds reproducible
- Hash mismatch is a security *and* integrity signal
- "Works with wrong hash" is not a feature

Theory 5 — Substituters vs local build

Realization of path `P`:

```
1. Is P already in the local store? → done
2. Can a trusted substituter provide P? → download
3. Else build from .drv locally
```

Implications

- First build can be "download a lot from cache.nixos.org"

- Changing one input changes the hash → cache miss → compile
- Trust is about **signatures**, not "HTTP has a file"

Observation	Interpretation
Instant `hello`	Substituted
Long compile after one-line change	Input hash changed
Hash mismatch on fetch	FOD integrity failure

Theory 6 — Outputs and multiple outputs (preview)

A derivation can have several outputs (`out`, `dev`, `doc`, …):

```
P-out, P-dev, P-man
```

Closure size discipline later depends on not pulling `dev` into runtime. In this chapter: know `out` is the default.

```
nix derivation show nixpkgs#hello | head -c 1500
# look for output names in the JSON-ish structure
```

Theory 7 — What you inspect after a build

Tool	Use
`nix build … --print-out-paths`	Output path
`ls result`	Symlink to output
`nix log <path or installable>`	Build log
`nix derivation show`	JSON-ish plan
`nix path-info -rS`	Closure size

Reading a log like an operator

1. Was it a substitute or a local build?
2. Which phase failed (if stdenv)?
3. Missing dependency vs sandbox network vs hash?

Theory 8 — Impure vs pure evaluation (awareness)

Mode	Risk
Pure flake eval	Preferred; limited filesystem
`--impure`	Can see ambient state; less reproducible
`builtins.currentSystem`	Convenient; pin `system` when sharing

Prefer pure flakes in project work (next chapters).

Worked examples bank

Example A — Build hello from nixpkgs

```
nix build nixpkgs#hello --print-out-paths
./result/bin/hello
```

Example B — Trivial package via flake

See Lab 1. Prefer flakes over one-off impure exprs for cleanliness.

Example C — Read derivation metadata

```
OUT=$(nix build nixpkgs#hello --print-out-paths --no-link)
nix derivation show nixpkgs#hello | head -c 2000
nix log "$OUT" 2>/dev/null || nix log nixpkgs#hello | tail
```

Example D — Sandbox failure story (conceptual)

```
Builder does: curl http://example.com/dep.so
→ network blocked (non-FOD)
→ build fails
→ declare dep as input or use fetchurl with hash
```

Example E — `runCommand` sketch

```
pkgs.runCommand "note" { } ''
  mkdir -p $out
  echo "built at eval-defined plan" > $out/note.txt
''
```

Example F — Wrong FOD hash

Expect failure when `sha256` does not match downloaded bytes—note the error class for packaging later.

Lab 1 — Workspace + flake for trivial builds

```
mkdir -p ~/lab/nixos-book/concepts/derivations
cd ~/lab/nixos-book/concepts/derivations
git init
```

Create `flake.nix`:

```
{
  description = "trivial derivations lab";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux"; # change if needed: aarch64-linux, aarch64-darwin, …
      pkgs = import nixpkgs { inherit system; };
    in {
      packages.${system} = {
        greet = pkgs.writeShellScriptBin "greet" ''
          echo "hello from a derivation"
        '';
        note = pkgs.writeText "note.txt" ''
          Derivations plan store paths.
        '';
        default = self.packages.${system}.greet;
      };
    };
}
```

If your system is not `x86_64-linux`, edit `system`. Discover with:

```
nix eval --impure --expr 'builtins.currentSystem'
```

```
nix flake lock
nix build .#greet
./result/bin/greet
nix build .#note
cat result
```

Commit `flake.lock` when generated.

Lab 2 — Inspect plan and log

```
nix build .#greet --print-out-paths
nix derivation show .#greet | head -n 80
nix log .#greet | tail -n 40
```

Journal:

1. Output path hash/name
2. Whether the "build" was essentially a write to the store
3. One field you saw in `derivation show` that looks like an input

Lab 3 — Compare to nixpkgs hello closure

```
H=$(nix build nixpkgs#hello --print-out-paths --no-link)
G=$(nix build .#greet --print-out-paths --no-link)
echo "hello requisites: $(nix-store -q --requisites "$H" | wc -l)"
echo "greet requisites: $(nix-store -q --requisites "$G" | wc -l)"
nix path-info -rS "$H" | tail -3
nix path-info -rS "$G" | tail -3
```

Explain size difference in one sentence (runtime interpreter/libs vs tiny script).

Lab 4 — Break a FOD on purpose (optional)

```
nix build --impure --expr 'let pkgs = import (builtins.getFlake "nixpkgs") { system = builti
```

You want a **hash mismatch** or fetch failure—not a success. Note the error class for later packaging chapters.

Lab 5 — Teach-back

Define without notes:

1. Derivation
2. `.drv` vs output path
3. Why sandbox blocks network by default
4. Substituter vs local build
5. Why `writeShellScriptBin` is still a "real" package

Exercises

1. Add a second script package to the flake; make it `default`.
2. Use `--no-link` and explain GC-root impact.
3. Diff `derivation show` for `greet` vs `hello` (structural notes).
4. Force a rebuild (when applicable) and compare to substitute path.
5. Write `runCommand` output containing your username string (careful: purity).
6. Map each stdenv phase name to one sentence (even if you skip implementing).
7. Document sandbox vs FOD in a two-column table from memory.
8. Measure closure size before/after adding `pkgs.git` to a wrapper (optional).

9. Explain multi-output `dev` risk for runtime closures.
10. Commit flake + lock; clone to a second directory and rebuild offline if possible.
11. Trigger `system` mismatch by asking for the wrong triple; fix it.
12. List three realize failures and which layer they belong to.

Common pitfalls

Symptom	Theory
Wrong `system` in flake	Outputs empty / wrong attr path
`result` pins GC root	`rm result` when done experimenting
Edited files in store	Immutable—change expression, rebuild
"Build" always compiles	Substituters often download
Impure path surprises	Unqualified imports / currentSystem
Darwin vs Linux scripts	shebangs and system must match host
FOD wrong hash ignored	Never; must match
Confusing `.drv` with runnable output	Plan vs product

Checkpoint

- ☐ Built a local trivial package via flake output
- ☐ Ran the resulting binary/script
- ☐ Inspected derivation show + log
- ☐ Compared closure size to `hello`
- ☐ Explained sandbox + FOD at awareness level
- ☐ `flake.lock` present
- ☐ Eval → drv → output story told in own words

Further depth (this book)

Topic	Chapter / part
Modern CLI around builds	Modern Nix CLI
Project shells	Flakes and project devShells
stdenv phases, FODs deep	Part **Packaging**
CA derivations	Part **Internals**
Remote builders / caches	Packaging + Ops parts

Modern Nix CLI

Modern Nix CLI

Goal: Prefer the **modern `nix` CLI** for all daily tasks: build, run, develop, shell, flake inspection, store queries—and know which command matches which intent.

Why this chapter matters

Docs and blog posts mix eras. Muscle memory on the wrong CLI makes flakes feel harder than they are. This chapter is deliberate fluency so flake authoring and host work feel like product work, not archaeology.

Ops motivation: incident response on a Nix machine is mostly "pick the right verb." `build` vs `run` vs `develop` vs `eval` is how you avoid thrash under pressure.

Theory 1 — One CLI, many intents

Intent	Modern command	Rough meaning
Build installable → `result`	`nix build`	Realize outputs
Run app without installing	`nix run`	Build/realize + exec
Project dev environment	`nix develop`	Shell with build inputs (flake devShell)
Ad-hoc package shell	`nix shell`	Temporary PATH with packages
Explore flake outputs	`nix flake show`	Schema of outputs
Update lock	`nix flake update`	Refresh inputs (careful)
Metadata / eval	`nix eval`	Evaluate expressions
Store facts	`nix path-info`	Size, refs, sigs
Search	`nix search`	Find packages
Profile (user env)	`nix profile`	Generational user installs
Format / check (later)	`nix fmt`, `nix flake check`	Quality gates
Store GC	`nix store gc`	Collect unreferenced paths

Classic cousins (`nix-build`, `nix-shell`, …) wait until the classic CLI chapter.

Theory 2 — Installables: how you *name* what you want

Modern CLI takes **installables**:

Form	Example	Meaning
Flake output	`.#greet`	Current flake, package `greet`
Flake + attr	`.#packages.x86_64-linux.greet`	Fully spelled
Indirect flake	`nixpkgs#hello`	Registry/indirect nixpkgs
Git flake	`github:NixOS/nixpkgs/nixos-26.05#hello`	Explicit
Path flake	`./myproj#default`	Local
File / expr	`-f file.nix`, `--expr '…'`	Non-flake (still useful)

splits flake ref from attribute

```
nixpkgs#hello

            attribute path inside outputs (with defaults)
     flake reference
```

Ambiguity and defaults

`nix build .#` often means default package. If show is empty, check `system` keys under `packages.<system>`.

Theory 3 — `nix build` deep enough for daily use

```
nix build .#greet
nix build .#greet --print-out-paths
nix build .#greet -o greet-result
nix build nixpkgs#hello
```

Flag	Role
`--print-out-paths`	Print store paths
`-o name`	Name the result symlink

Flag	Role
`--no-link`	Realize without `result`
`-L` / `--print-build-logs`	Stream logs
`--rebuild`	Force rebuild (when applicable)
`--show-trace`	Eval error detail

Discipline: temporary builds → `--no-link` or `rm result` to avoid GC root clutter (store chapter).

Theory 4 — `nix run` vs `nix shell` vs `nix develop`

`nix run`

```
nix run nixpkgs#hello
nix run .#greet
nix run nixpkgs#cowsay -- "moo"
```

Builds/downloads if needed, runs the package's main program, does **not** require a permanent profile entry. Args after `--` go to the program.

`nix shell`

```
nix shell nixpkgs#git nixpkgs#jq
# temporary shell with git + jq on PATH
nix shell nixpkgs#jq -c 'jq --version'
```

Ad-hoc tools. Great for "I need `jq` once."

`nix develop`

```
nix develop                 # .#devShells.default or similar
nix develop .#name
nix develop -c cargo --version
```

Enters the flake's **devShell**—meant for project dependencies, compilers, env vars. The flakes chapter makes one.

Command	Sticky?	Typical use
`run`	No	One-shot CLI tools
`shell`	Until exit	Ad-hoc package mix

Command	Sticky?	Typical use
`develop`	Until exit	Project toolchain
`profile install`	Yes	User-global tools (use sparingly early)

Theory 5 — Flake subcommands

```
nix flake init          # scaffold (optional)
nix flake new mod       # template
nix flake show          # outputs tree
nix flake metadata      # resolved inputs, locks
nix flake check         # run checks (later quality)
nix flake update        # update all inputs
nix flake update nixpkgs  # selective update
nix flake lock          # ensure lock matches urls
```

`show` vs `metadata`

Command	Answers
`show`	What outputs exist?
`metadata`	What are inputs pinned to?

JSON for scripting

```
nix flake metadata --json | head -c 400
nix flake show --json 2>/dev/null | head -c 400
```

Theory 6 — Eval and debugging eval

```
nix eval .#packages.x86_64-linux.greet.name
nix eval --raw .#packages.x86_64-linux.greet
nix eval --json nixpkgs#hello.meta | head
nix eval --show-trace --expr '1 + "x"'  # intentional fail
```

```
nix repl
:lf .
:lf nixpkgs
```

REPL remains the best theory laboratory.

Goal	Tool
Print a string attr	`nix eval --raw`
Explore interactively	`nix repl`
Deep stack on error	`--show-trace`

Theory 7 — Store queries you will reuse

```
nix path-info -Sh ./result
nix path-info -rS ./result | tail
nix store gc --dry-run   # careful; prefer dry concepts early
# nix store delete …     # advanced; skip casually
```

`nix-store -q` still works; modern `path-info` is friendlier.

Query	Command family
Size	`path-info -S` / `-rS`
Closure	`path-info -r`
Roots (classic)	`nix-store --gc --print-roots`

Theory 8 — Profiles without making them a lifestyle

```
nix profile list
nix profile install nixpkgs#jq
nix profile remove …
```

This book's preference: project flakes + devShells over growing a huge imperative profile. Profiles are fine for a few personal tools; they are not a substitute for declarative hosts (NixOS host part).

Theory 9 — Search and discovery

```
nix search nixpkgs ripgrep
nix search nixpkgs#legacyPackages.x86_64-linux hello  # more explicit forms exist
```

First search can be slow (indexing/eval). Prefer known attribute names when you have them.

Online companion: https://search.nixos.org/packages (select 26.05 when available).

Theory 10 — Error class → command checklist

Error class	First commands
experimental features	`nix config show`
attr missing	`nix flake show`
pure eval path missing	`git status` / flake files
build failed	`nix build -L` / `nix log`
wrong system	`builtins.currentSystem` / show packages

Worked examples bank

Example A — Same package, three ways

```
nix run nixpkgs#hello
nix shell nixpkgs#hello -c hello
nix build nixpkgs#hello --print-out-paths --no-link
```

Example B — Flake introspection

```
cd ~/lab/nixos-book/concepts/derivations   # or your flake
nix flake show
nix flake metadata
```

Example C — Logs streaming

```
nix build nixpkgs#hello -L
```

Example D — JSON for scripting

```
nix flake metadata --json | head -c 400
nix path-info --json nixpkgs#hello | head -c 400
```

Example E — Non-interactive develop

```
nix develop -c bash -lc 'git --version; echo $name'
```

Lab 1 — Cheat-sheet construction

```
mkdir -p ~/lab/nixos-book/concepts/modern-cli
cd ~/lab/nixos-book/concepts/modern-cli
```

Create `CHEATSHEET.md` and fill by *running* each command at least once:

```
build:    …
run:      …
shell:    …
develop:  … (may wait until flakes chapter if no shell yet)
show:     …
metadata: …
eval:     …
path-info:…
search:   …
```

Paste the exact successful command lines you used.

Lab 2 — Operate a local flake with modern CLI only

Reuse the derivations flake (or recreate a minimal package flake).

Tasks:

1. `nix flake show`
2. `nix build .#greet --print-out-paths`
3. `nix run .#greet`
4. `nix path-info -rS ./result | tail` (if result exists)
5. `rm -f result` and rebuild with `--no-link`

Lab 3 — Ad-hoc `nix shell` workflow

```
nix shell nixpkgs#jq nixpkgs#ripgrep -c bash -c 'jq --version; rg --version'
```

Solve a tiny task: query a JSON blob with `jq` **without** installing jq into a profile permanently.

```
echo '{"ok":true}' | nix shell nixpkgs#jq -c 'jq .'
```

Lab 4 — Search + run loop

```
nix search nixpkgs cowsay | head
nix run nixpkgs#cowsay -- "modern cli"
```

Record the attribute name you used.

Lab 5 — Intent mapping drill (closed book)

Write the best command for each:

1. Temporarily need `ffmpeg`
2. See outputs of current flake
3. Build package without leaving `result`
4. Run project's default app
5. See what nixpkgs revision lock pins

Exercises

1. Build with `-o custom-result` and remove it; confirm root behavior.
2. Compare `nix run` vs `nix shell -c` for a multi-arg program.
3. Export `nix flake metadata --json` and extract locked rev (manual or `jq`).
4. Use `nix eval` to print `hello.name` from nixpkgs.
5. Intentionally pass a bad attr; read the error; fix via `flake show`.
6. Time a cold vs warm `nix run nixpkgs#hello` (subjective notes OK).
7. List profile generations; avoid installing anything permanent if possible.
8. Write a 15-line operator runbook: "package missing on PATH."
9. Use `--show-trace` on a deliberate eval error.
10. Document registry vs project lock difference for `nixpkgs#`.
11. Practice `nix build` + `path-info -rS` closure report for two packages.
12. Map five classic commands (from memory) to modern ones; verify next chapter.

Common pitfalls

Symptom	Theory
`experimental features` errors	`nix-command` / `flakes` not enabled
Attribute not found	Wrong system triple / output layout
`nix run` can't find binary	Package has no main program meta
Mixing `nix-env -i` habits	Prefer run/shell/develop
`flake show` empty packages	system key mismatch
Slow `nix search`	First-time indexing/eval
Sudo changes flake path cwd	Use absolute flake refs when needed
Profile becomes SoT	Move tools into flakes/HM later

Checkpoint

- ☐ Cheat-sheet written from real command output
- ☐ Can choose run vs shell vs develop correctly
- ☐ Built with and without `result` link
- ☐ Used `flake show` + `metadata`
- ☐ Used `path-info` on a real path
- ☐ No new dependency on classic CLI for daily work

Further depth (this book)

Topic	Chapter
Author flakes + devShells	Flakes and project devShells
Classic translation table	Classic CLI literacy
Host rebuild CLI	Host: Rebuild and generations
`nix flake check` quality	Home/flake layout part

Flakes and project devShells

Flakes and project devShells

Goal: Author a real **project flake** with pinned `inputs`, clear `outputs`, committed `flake.lock`, and a working `devShells.default`—before any NixOS host work.

Why this chapter matters

Flakes are the modern collaboration unit: reproducible inputs + named outputs. A **devShell** is the highest-value first output—shared tools without polluting a global profile. OS configuration comes in the NixOS host part; project pinning comes **now**.

Ops motivation: "works on my machine" dies when `flake.lock` is the shared truth. Teams that skip locks reinvent channel drift with extra steps.

Theory 1 — What a flake is

A flake is a directory (usually a git tree) with:

File	Role
`flake.nix`	Declarative inputs + outputs function
`flake.lock`	Exact pinned revisions/hashes of inputs

Evaluation model

```
flake.nix outputs = inputs: { … }
                  ↓
        lock pins input trees
                  ↓
     outputs provide packages, shells, apps, nixosConfigurations, …
```

Punchline: without the lock file, "same flake.nix" is not the same system.

Baseline for this book

```
inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
```

Nix **2.34.x** + flakes experimental features enabled (Why Nix exists chapter).

Theory 2 — `inputs`

```
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    # optional:
    # flake-utils.url = "github:numtide/flake-utils";
  };
}
```

Common URL forms

URL	Meaning
`github:Owner/Repo`	Default branch (avoid for pins—use lock)
`github:Owner/Repo/ref`	Branch/tag/rev
`git+https://…`	Arbitrary git
`path:./vendor/x`	Local path input

Following transitive inputs

```
inputs.foo.inputs.nixpkgs.follows = "nixpkgs";
```

Keeps a single nixpkgs when dependencies would otherwise multiply.

Input types (awareness)

Flakes can depend on other flakes, non-flake sources (with care), and path inputs. Prefer flake-native inputs for clarity.

Theory 3 — `outputs`

```
outputs = { self, nixpkgs, ... }@inputs:
  let
    system = "x86_64-linux";
    pkgs = import nixpkgs { inherit system; };
  in {
    packages.${system}.default = pkgs.hello;
    devShells.${system}.default = pkgs.mkShell {
      packages = [ pkgs.git pkgs.hello pkgs.jq ];
    };
  };
```

Output schema (common)

Output	Purpose
`packages.<system>.<name>`	Buildable packages
`devShells.<system>.<name>`	`nix develop` environments
`apps.<system>.<name>`	`nix run` apps
`formatter.<system>`	`nix fmt`
`checks.<system>.<name>`	`nix flake check`
`nixosConfigurations.<name>`	NixOS hosts (host part)
`overlays.default`	Package overlays (later)

`self`

`self` is the flake's own source (after copy to store). Used for `src = self;` packaging and referring to other outputs: `self.packages.${system}.foo`.

Naming `default`

Output	Default behavior
`packages.<sys>.default`	`nix build` / `nix run` without attr
`devShells.<sys>.default`	`nix develop` without attr

Theory 4 — Systems and portability

Hard-coding one `system` is fine for a first lab. Better patterns later: `flake-utils`, `flake-parts`, or explicit map over systems.

```
systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
```

In this chapter: pick **your** system correctly or multi-define the same shell for two systems if you dual-boot.

Discover:

```
nix eval --impure --expr 'builtins.currentSystem'
```

`genAttrs` pattern

```
forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f system);
```

Theory 5 — `mkShell` / `mkShellNoCC`

```
pkgs.mkShell {
  packages = [ pkgs.go pkgs.git ];
  shellHook = ''
    echo "welcome to the dev shell"
    export DEMO=1
  '';
}
```

Field	Role
`packages` / `nativeBuildInputs`	Tools on PATH
`buildInputs`	Libraries (more package-oriented)
`shellHook`	Runs on enter

Prefer putting **project** tools here, not every CLI you like globally.

`mkShellNoCC`

When you do not need a C toolchain, `mkShellNoCC` can shrink closure—useful for pure scripting shells.

Theory 6 — Lock discipline (intro)

```
nix flake lock          # ensure lock exists/up to date with urls
nix flake update        # bump inputs (all)
nix flake update nixpkgs  # bump one input
```

Rule	Why
Commit `flake.lock`	Reproducibility
Review lock diffs	Security + surprise upgrades
Selective update	Smaller change sets
CI uses locked revs	Bit-identical intent

The lock discipline chapter (Home and flake layout part) goes deeper; here: **lock must exist and be committed**.

Theory 7 — Flakes and git

Flakes generally want a **git** repository. Untracked files may be invisible to evaluation depending on setup.

```
git init
git add flake.nix
# after first lock:
git add flake.lock
```

If something "missing," check `git status`—not only the filesystem.

Dirty tree semantics

Flakes may copy a clean view of tracked files. Untracked `extra.nix` will not exist in eval—by design for purity.

Theory 8 — Registries (light awareness)

```
nix registry list
```

`nixpkgs#hello` resolves via registry → often a floating ref **until** you pin your project's own input. Project flakes should **not** rely on registry drift; they declare `inputs.nixpkgs.url`.

Reference	Pin quality
Registry `nixpkgs#`	Convenience; can drift
Project input + lock	Collaboration default

Theory 9 — Apps vs packages (optional clarity)

```
apps.${system}.default = {
  type = "app";
  program = "${self.packages.${system}.greet}/bin/greet";
};
```

Often `nix run` on a package works via `meta.mainProgram`. Apps make the entrypoint explicit.

Theory 10 — What not to put in a first flake

Avoid early	Why
Entire NixOS config	Host part
Secrets	Store readability
Unrelated global tools dump	Closure bloat
Unlocked floating inputs	Drift

Worked examples bank

Example A — Minimal full flake

```
{
  description = "project shell";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f system);
    in {
      devShells = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; };
        in {
          default = pkgs.mkShell {
            packages = with pkgs; [ git hello jq ];
          };
        });
    };
}
```

Example B — Enter and verify

```
nix develop
hello
git --version
jq --version
exit
```

Example C — Metadata

```
nix flake metadata
nix flake show
```

Example D — Non-interactive CI style

```
nix develop -c bash -lc 'git --version && jq --version'
```

Example E — Package + shell together

```
packages.${system}.greet = pkgs.writeShellScriptBin "greet" ''echo hi'';
devShells.${system}.default = pkgs.mkShell {
  packages = [ self.packages.${system}.greet pkgs.git ];
};
```

Lab 1 — Create the project

```
mkdir -p ~/lab/nixos-book/concepts/flakes/myproject
cd ~/lab/nixos-book/concepts/flakes/myproject
git init
```

Write `flake.nix` (start from Example A; trim systems if you want).

```
nix flake lock
nix flake show
nix develop -c hello
```

```
git add flake.nix flake.lock
git commit -m "initial project flake with devShell"
```

Lab 2 — Make it *your* project

Add tools you actually want for a small project (pick a lane):

Lane	Example packages
Go	`go`, `gopls`, `git`
Rust	`rustc`, `cargo`, `rust-analyzer`
Python	`python3`, `uv` or `poetry`
Ops	`jq`, `ripgrep`, `kubectl` (if desired)

Edit `packages = [ … ]`, re-enter shell, verify versions.

Add a trivial project file `README.md` describing how to enter the shell.

Lab 3 — `shellHook` and env

```
shellHook = ''
  export PROJECT_ROOT="$PWD"
  echo "devShell ready for $PROJECT_ROOT"
'';
```

```
nix develop -c bash -c 'echo $PROJECT_ROOT'
```

Lab 4 — Offline-capable after populate (preview)

1. Enter `nix develop` once online (populate store).
2. Note `nix flake metadata` resolved revisions.
3. Optional: airplane mode / disconnect and `nix develop --offline` (or equivalent) if supported in your setup.

Journal what still requires network (usually nothing if all paths realized).

Lab 5 — Teach-back

1. Why `flake.lock` is mandatory in git
2. Difference between `packages` and `devShells`
3. What `nix develop` uses by default
4. Why registry `nixpkgs#` your flake's pinned nixpkgs

Exercises

1. Add a second named devShell (`devShells.${system}.tiny`) with fewer packages.
2. Use `follows` with a second input (even a path stub) to practice wiring.

3. Bump only `nixpkgs` with selective update; read the lock diff.
4. Break purity: leave a needed file untracked; observe failure; `git add`.
5. Export `metadata --json` and record locked `narHash`/`rev`.
6. Add a trivial `packages.default` and `nix run`.
7. Document system portability: does a friend on another arch work?
8. Compare closure of fat vs thin shell (`path-info`).
9. Write CONTRIBUTING.md: "always commit lock."
10. Replace registry experiments with project inputs only.
11. Try `nix flake check` if you add a simple check later (optional).
12. Sketch how `nixosConfigurations` will hang off the same outputs function (host part).

Common pitfalls

Symptom	Theory
File not found in flake eval	Not git-added
Wrong system attr	`devShells.<system>` mismatch
`nix develop` empty	No `default` shell
Divergent tools across machines	Lock not committed / not pulled
Huge shell closure	Too many packages; split shells later
Dirty tree warnings	Commit or understand copy semantics
Floating input without lock	Non-reproducible
Secrets in flake source	Store may expose them

Checkpoint

- ☐ `flake.nix` with nixpkgs **26.05** input
- ☐ `flake.lock` committed

☐ `nix develop` provides 2 real tools

☐ `nix flake show` lists devShell

☐ README explains entry

☐ Understand lock vs registry

☐ Multi-system or correctly single-system for your machine

Further depth (this book)

Topic	Chapter / part
Classic modern map	Classic CLI literacy
Host flakes	Host: Flake-defined hosts
Lock discipline deep	Home and flake layout part
flake-parts, checks	Home and flake layout part
Packaging `src = self`	Packaging part

Classic CLI literacy

Classic CLI literacy

Goal: Map **classic** Nix commands and channel habits to the **modern** CLI—once—so legacy docs are readable without making classic tools your daily driver.

> **!** Important
>
> This chapter is **literacy**, not a lifestyle change. After it, default back to modern `nix` + flakes.

Why this chapter matters

Half the internet still says `nix-env -iA nixpkgs.hello` and `nix-shell -p`. If you cannot translate, you thrash. If you only learn classic, you fight 2026 defaults. Translation table + one dual run is enough.

Ops motivation: production runbooks, ancient Stack Overflow answers, and coworker muscle memory will keep showing classic commands. Translation skill is cheaper than rewriting the internet.

Theory 1 — Same store, two UIs

Layer	Shared	Differs
Store	`/nix/store`, derivations, GC roots	—
Eval	Nix language	Entry points, purity defaults
UX	—	Flag names, flake awareness, UX polish

Classic tools predate flakes. They still work. They are not the pedagogy default of this book.

```
                /nix/store



classic CLI                modern `nix`
(nix-build, …)           (nix build, …)
```

Theory 2 — Command map (core)

Classic	Modern analogue	Notes
`nix-build -A hello`	`nix build nixpkgs#hello`	Attr selection differs
`nix-build` on `default.nix`	`nix build -f .` / flake packages	
`nix-shell -p hello`	`nix shell nixpkgs#hello`	Ad-hoc tools
`nix-shell` (dev `shell.nix`)	`nix develop` (flake) / `nix-shell` still	Project shells
`nix-env -iA nixpkgs.hello`	`nix profile install nixpkgs#hello`	Prefer not to
`nix-env -q`	`nix profile list`	
`nix-env -e`	`nix profile remove`	
`nix-instantiate`	`nix eval` / derivation show	Different shapes
`nix-store -q --requisites`	`nix path-info -r`	Both fine
`nix-store -q --tree`	(still useful) / path-info	Keep classic for trees
`nix-channel --update`	`nix flake update` (project)	Different model
`nix-collect-garbage`	`nix store gc` / `nix-collect-garbage`	Still common
`nix-prefetch-url`	`nix store prefetch-file` etc.	Variants evolve

Intent-first map

Intent	Prefer modern	Classic you may see
Run once	`nix run`	`nix-shell -p … --run`
Project tools	`nix develop`	`nix-shell`
Realize output	`nix build`	`nix-build`
Pin project	flake.lock	channel + NIX_PATH
Query closure	`nix path-info -r`	`nix-store -q --requisites`

Theory 3 — Channels as history

Channel model

```
nix-channel --add https://nixos.org/channels/nixos-26.05 nixpkgs
nix-channel --update
nix-env -iA nixpkgs.hello
```

Property	Effect
Mutable “latest on channel”	Drift between machines/days
`NIX_PATH` / `<nixpkgs>`	Ambient dependency
User or root channels	Confusing multi-user stories

Flake model (preferred)

```
inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
# flake.lock pins exact rev
```

Property	Effect
Lock file	Shared truth
Explicit inputs	No ambient channel required
Per-project pins	Different projects can differ safely

This book: channels = read-only literacy. Do not build new projects on channels.

When channels still appear

Context	Why
Old tutorials	Written pre-flakes default
Some NixOS install defaults	Historical paths
Angle-bracket imports	`import <nixpkgs> {}`

Theory 4 — `shell.nix` / `default.nix` legacy layouts

Classic project shell

`shell.nix`:

```
{ pkgs ? import <nixpkgs> { } }:
pkgs.mkShell {
  packages = [ pkgs.hello pkgs.git ];
}
```

```
nix-shell   # enters shell.nix
```

Modern

`flake.nix` → `devShells.default` → `nix develop`

You may keep a thin `shell.nix` for non-flake users, but **your** path is flakes.

Dual support pattern (optional)

Some repos provide both: flake for modern users, `shell.nix` that re-exports for classic. Only do this if you must support both audiences.

Theory 5 — `nix-env` and imperative profiles

```
nix-env -iA nixpkgs.hello
nix-env -e hello
nix-env --list-generations
```

Problems as a primary workflow:

1. Not recorded in project git
2. Easy to create unreproducible laptops
3. Diverges from NixOS declarative model

`nix profile` is the modern imperative cousin—same caution.

Imperative tool	Declarative alternative
`nix-env -i`	flake packages / NixOS systemPackages / HM
Random `nix-shell -p` forever	`devShells`
Channel update as SoT	`flake.lock` review

Theory 6 — Purity and flakes

Classic ad-hoc eval often sees your cwd and `NIX_PATH` freely. Flake evaluation is **more pure** by default (limited access to paths outside the flake). That is why git-tracked files matter (flakes chapter).

When classic examples "just import `./.`", flake equivalents may need `self` or explicit file inclusion.

Classic habit	Flake analogue
`import ./default.nix`	outputs / packages
Ambient `<nixpkgs>`	`inputs.nixpkgs`
Untracked helper file	Must be added to git

Theory 7 — Reading old error messages

Old-doc symptom	Translation
`attribute 'foo' missing` in `-A`	Wrong attr path / channel content
`file 'nixpkgs' was not found`	`NIX_PATH` / channel not set
`nix-env` permission errors	multi-user profile ownership
Blog says enable flakes differently	Check current Nix 2.34 docs
`error: experimental Nix feature`	modern CLI without features

Theory 8 — GC and store tools (both eras)

Task	Classic	Modern
GC	`nix-collect-garbage`	`nix store gc`
Delete gens aggressively	`nix-collect-garbage -d`	same caution
Query refs	`nix-store -q --references`	`nix path-info`
Roots	`nix-store --gc --print-roots`	still common

Do not GC aggressively on machines with important generations until the host generations chapter.

Theory 9 — Decision policy for this book

Situation	Policy
New personal project	Flakes + modern CLI only
Reading a 2019 blog	Translate; do not adopt channels
Coworker pastes `nix-env`	Offer modern equivalent once

Situation	Policy
Debugging store graphs	Either store tool family is fine
NixOS host management	`nixos-rebuild --flake` (host part)

Worked examples bank

Example A — Build hello both ways

```
# modern
nix build nixpkgs#hello --print-out-paths --no-link

# classic-ish
nix-build -E 'with import <nixpkgs> { }; hello' --no-out-link
```

(If `<nixpkgs>` missing, skip classic path or install channel only for the drill.)

Example B — Shell both ways

```
nix shell nixpkgs#hello -c hello
nix-shell -p hello --run hello
```

Example C — Query store both ways

```
P=$(nix build nixpkgs#hello --print-out-paths --no-link)
nix-store -q --references "$P"
nix path-info -r "$P" | head
```

Example D — Profile vs run

```
# avoid if possible
# nix-env -iA nixpkgs.jq
# prefer
nix run nixpkgs#jq -- --version
```

Example E — Channel list observation

```
nix-channel --list || true
```

Empty can be good on a flakes-first machine.

Lab 1 — Build the translation table

```
mkdir -p ~/lab/nixos-book/concepts/classic-literacy
cd ~/lab/nixos-book/concepts/classic-literacy
```

Create `CLASSIC-MODERN.md` with at least **10 rows** you personally verified (or attempted):

Task	Classic command	Modern command	Result notes
run hello			
…			

Lab 2 — Dual path for a project shell

Take your flakes chapter project (or recreate):

Modern (already works):

```
cd ~/lab/nixos-book/concepts/flakes/myproject
nix develop -c git --version
```

Classic parallel: write `shell.nix` that mirrors the tool list using `<nixpkgs>` *or* a pinned fetch if you know how—only for the drill:

```
{ pkgs ? import <nixpkgs> { } }:
pkgs.mkShell {
  packages = [ pkgs.git pkgs.hello pkgs.jq ];
}
```

```
nix-shell --run 'git --version'
```

Journal: which is easier to share with a teammate on another machine?

Lab 3 — Channel inspection (optional, disposable)

Only if you are willing to touch channels on a **lab account**:

```
nix-channel --list
# do not --update production machines casually
```

If empty: good—note "no channels; flakes only."

Lab 4 — Doc archaeology

Find one external tutorial that uses classic CLI (search your history or web). Rewrite its first three commands into modern equivalents in your notes **without running random untrusted scripts as root**.

Lab 5 — Commitment statement

Write three sentences:

1. When I will still use classic tools
2. When I will refuse to (prefer modern)
3. How I will handle future blog posts that only show `nix-env`

Exercises

1. Expand your table to 15 rows including GC and store queries.
2. Translate `nix-env -iA nixpkgs.hello` into three modern alternatives (`run`/`shell`/`profile`).
3. Explain why `nix-shell -p` is not a project SoT.
4. Convert a classic `default.nix` package build to a flake `packages` output (sketch).
5. Document `NIX_PATH` on your machine; plan to ignore it for new work.

6. Pair-read an old tutorial with a highlighter: classic vs still-valid concepts.
7. Practice `nix-store -q --tree` vs modern path-info on the same path.
8. Write a "please use flakes" snippet for your team README.
9. List risks of `nix-collect-garbage -d` in classic blogs.
10. Map `nixos-rebuild` classic channel workflows to `--flake` (preview host part).
11. Deliberately avoid installing channels even if a blog demands it—note the workaround.
12. Quiz yourself: flashcard classic → modern for one week of reading.

Common pitfalls

Symptom	Theory
Following classic blog on NixOS host	May fight flakes/module setup
Installing channels "to fix" flakes	Usually wrong direction
`nix-shell` vs `nix develop` confusion	Different project conventions
Profile pollution from drills	Remove packages / use disposable user
Assuming channels == 26.05 pin	Channel lag vs flake lock rev
Mixing both as SoT	Pick flakes for new work
Treating literacy chapter as daily driver	Revert to modern defaults

---

Checkpoint

- ☐ Personal 10-row translation table
- ☐ Same task run modern + classic once
- ☐ Can explain channels vs flake locks
- ☐ Can explain why `nix-env` is not the goal
- ☐ Default workflow remains modern
- ☐ Commitment statement written

Concepts part — exit criteria

Before the **NixOS host** part:

- ☐ Three faces of Nix clear
- ☐ Closures and GC roots clear
- ☐ Attrsets + functions + multi-file Nix workable
- ☐ Derivation idea + trivial build done
- ☐ Modern CLI fluent
- ☐ Project flake + lock + devShell done
- ☐ Classic docs are readable, not authoritative

Further depth (this book)

Topic	Chapter / part
Host management CLI	Part **NixOS host**
Packaging classic attrs	Part **Packaging**
Deeper flake layout	Part **Home and flake layout**
Restart Concepts map	Concepts index

Nix on Linux

Nix on Linux — Fedora bridge path

Nix on Linux — Fedora bridge path

This part is the **on-ramp**: run the full **Nix package manager** on a normal Linux distribution—especially **Fedora 44+**, which now ships Nix officially—so you learn store, flakes, profiles, Home Manager, and project shells **without** replacing your OS yet.

When you later install **NixOS**, almost nothing you practice here is wasted. You already speak store paths, locks, `nix develop`, GC roots, and declarative user config. NixOS then "only" adds: module system for the *machine*, generations for the *bootloader*, and system-wide rebuilds.

Tip

Who this part is for

- Daily driver is **Fedora 44** (or nearby: F43+, EPEL packaging where available)
- You want Nix **skills** before a full distro switch
- You already finished (or skimmed) **Concepts** (`01-concepts/`)
- You are *not* ready to wipe the host for NixOS this week

If you only have a spare disk/VM and want the OS immediately, you can jump to **NixOS host**—but this bridge still pays off for multi-machine workflows (Fedora laptop + NixOS server is a common end state).

Why Fedora 44 matters

Historically, Nix on Fedora meant the upstream multi-user installer, SELinux friction, and awkward ownership of `/nix`.

Fedora Change: Nix package tool (feature for Fedora 44) plus approval of **`/nix`** for the distro package changed that:

Before (typical)	Fedora 44+ packaged path
`curl | sh` Determinate/upstream installer	`sudo dnf install nix`
Manual SELinux babysitting	Policy-integrated Fedora build
Flakes often left experimental	**Flakes enabled out of the box** (Fedora packaging)

Before (typical)	Fedora 44+ packaged path
Atomic desktops painful	Documented **rootless / bind-mount** patterns

Upstream discourse and Fedora packaging notes also describe **nix-daemon multi-user**, **single-user**, and **pure rootless** modes (especially useful on Atomic/bootc-style systems). Always confirm with `/usr/share/doc/nix/README.fedora.md` on your machine—Fedora packaging details evolve faster than this notebook.

Baseline for this book still applies: treat **Nix 2.34-class** tooling and **nixpkgs 26.05** pins as the comparison point when you write flakes.

What you will learn here

Chapter	You leave able to…
Why Nix on Linux	Map package manager vs OS; design a dual-stack life (dnf + Nix)
Fedora 44 install	Install daemon or single-user; verify; read Fedora docs on disk
Store, profiles, GC	Own profiles without snowflaking; GC safely; see roots
Nix without /nix	Run a rootless chroot store (`--store DIR`); know why relabelling the store dir kills the cache
CLI + devShells	Daily `nix build/run/develop/shell`; flake projects; direnv
Shebang scripts	Self-contained `#! nix-shell` / `#! nix` scripts with pinned deps
Home Manager standalone	Declarative user environment **on Fedora**
dnf + Nix coexistence	Decide what belongs where; avoid PATH wars
SELinux, Atomic, ops	Survive enforcing SELinux; Atomic `/nix` bind; disk ops
Bridge to NixOS	Port habits into first `nixosConfigurations` host

Suggested study path

```
Concepts (store, language, flakes)


  Nix on Linux (you are here:
  Fedora daily driver)

                same flake skills

      NixOS host (machine as code)
```

```
Home Manager as NixOS module
(you already know standalone HM)
```

Path	Order
Recommended	Concepts → this part → NixOS host → Home layout part (skim what you already know)
Already installed Nix via curl	Start at install chapter for **Fedora packaging migration**, then continue
Atomic desktop only	Install + SELinux/Atomic chapter early

Lab workspace

```
mkdir -p ~/lab/nixos-book/nix-on-linux
cd ~/lab/nixos-book/nix-on-linux
git init   # optional but recommended
```

Keep experiments **outside** critical personal data. The store under `/nix` will grow—budget **20–50+ GB** if you build many shells.

Exit gate (before NixOS host)

- ☐ `nix --version` works after reboot
- ☐ Multi-user **or** intentional single-user/rootless mode documented in your notes
- ☐ Flakes work: `nix flake metadata nixpkgs` (or a personal flake)
- ☐ You can enter a project `devShell` and have teammates reproduce it via `flake.lock`
- ☐ You understand **profile** vs **devShell** vs **system packages** (NixOS later)
- ☐ Home Manager standalone rebuild succeeds for at least shell + git + editor snippet
- ☐ You can explain what moves to NixOS later (modules, generations, boot) without panic

---

Further depth (this book)

Next	Part
Install and own a machine	`03-nixos-host/`
Flake layout + HM as system module	`04-home-and-flake-layout/`
Packaging craft	`06-packaging/`
ROS 2 robotics env (overlay + flakes)	ROS 2 with nix-ros-overlay

Official docs win when notes disagree: nix.dev, NixOS manual, Fedora `/usr/share/doc/nix/`, Fedora Nix Change.

Why Nix on Linux first

Why Nix on Linux first

Goal: Decide *why* to run the Nix package manager on Fedora (or another Linux) **before** NixOS, and design a dual-stack workflow that makes the later OS transition easy—not confusing.

Why this chapter matters

Switching distros and learning Nix **at the same time** multiplies failure modes: bootloader issues look like "Nix is hard," and store GC looks like "Fedora broke."

Nix-on-Linux separates concerns:

Layer	Who owns it on Fedora + Nix	Who owns it on NixOS
Kernel, bootloader, base OS	Fedora (dnf, rpm-ostree, bootc…)	NixOS modules + generations
`/nix/store` software	**Nix**	**Nix** (same mental model)
Project toolchains	**Nix flakes / devShells**	Same
User dotfiles	Home Manager standalone *or* imperative	Home Manager (often as NixOS module)
System services (sshd, nginx)	systemd units / Fedora packages	NixOS `services.*`

Everything in the middle rows **transfers**. Only the "OS owner" row changes later.

Three faces again (bridge framing)

From Concepts you know:

1. **Language** — expressions
2. **Package manager** — store, builds, profiles
3. **NixOS** — OS as a module system

On Fedora you run **(1) + (2)** every day. **(3)** waits until you want the machine itself as a flake output (`nixosConfigurations`).

```
Today (Fedora 44 + Nix)
  dnf         OS packages, kernel, desktop, SELinux policy
  nix         toolchains, CLIs, project shells, optional HM

Later (NixOS)
  nixos-rebuild   OS + services + kernel modules as code
  nix         same store / same flakes for packages & shells
  home-manager    often folded into the host flake
```

What Nix is good at *on* an existing distro

Strength	Concrete use on Fedora
Reproducible project envs	`flake.nix` + `direnv` → same Node/Go/Rust/Python as CI
Version coexistence	Old `glibc`-sensitive tools without breaking desktop
Non-root software	Install tools without waiting for RPM or COPR
Atomic user rollback (HM)	Generations for *your* home config
Cross-machine sameness	Same flake on Fedora laptop and NixOS server
Try packages safely	`nix shell nixpkgs#foo` — no global pollution

What you should still leave to Fedora

Keep on dnf / Atomic image	Why
Kernel, firmware, mesa, GPU stacks	Closest to hardware; SELinux-integrated
Desktop session (GNOME/KDE), PipeWire	Distro integration and portals
SELinux policy, firewalld defaults	Security baseline of the OS
Flatpaks for sandboxed GUI apps (optional)	Complements Nix; different trust model
System daemons you didn't declare in Nix	Avoid two sources of truth for sshd/nginx

Rule of thumb: OS and *machine identity* → Fedora for now. *Development surface and portable CLI life* → Nix.

Dual-stack anti-patterns

Anti-pattern	What goes wrong	Better
Install *everything* with `nix profile`	Huge profile, slow GC, PATH chaos	Flakes per project + small profile
Mix `nix-env -i` and flakes	Two eras of tooling	Modern CLI + flakes only
Put secrets in the store via HM	World-readable store paths	sops/age later; never commit secrets
Disable SELinux "to make Nix work"	Trains bad ops habits	Use Fedora packaging + docs
Skip locks	"Works on my machine" returns	Commit `flake.lock`
Expect Nix to replace dnf for Firefox+kernel	Wrong layer	Keep browser/kernel on Fedora

---

Skills map: bridge → NixOS

Skill on Fedora + Nix	Same skill on NixOS	New on NixOS
`nix build` / `nix run`	Yes	—
Flake inputs + lock	Yes	`nixosConfigurations` output
`nix develop` + direnv	Yes	Same
Store, closures, GC roots	Yes	System profile is also a root
`nix profile`	Optional	Prefer declarative system/HM
Home Manager standalone	Yes	Often `home-manager.users.*` module
systemd user services via HM	Partial	Full system units via modules
Roll back user env	HM generations	**Boot generations** + HM

You are not "wasting time" on Fedora—you are doing **package-manager literacy** with a normal desktop underneath.

Mental model: evaluation vs realization (ops view)

```
flake.nix + flake.lock
          nix develop / nix build

   evaluate (pure-ish)


   derivation plans (.drv)
          nix-daemon realizes

   /nix/store/<hash>-name

           ephemeral shell (devShell) - dies when shell exits (roots may linger with direnv)
           profile generation - GC root until removed
```

On NixOS, add another root: **system generation** → bootloader entry.

Worked decision: "Should this be Nix or dnf?"

Want…	Choose
`ripgrep`, `fd`, latest `helix` for *you*	Nix profile **or** HM
Project needs Go 1.26 + exact lib versions	Flake `devShell`
System upgrade of GNOME	`dnf` / Software
Kernel module for VPN	Fedora packages
nginx on a lab VM you will reinstall as NixOS soon	Either; prefer learning **NixOS module** on a VM, not production Fedora host
CI-identical toolchain	Flake (Nix)

Lab

1. Write a short **charter** in `~/lab/nixos-book/nix-on-linux/CHARTER.md`:

```
# Nix on Fedora charter

## OS owner
Fedora 44 - dnf handles: ...

## Nix owner
Nix handles: ...

## Out of scope for Nix (for 90 days)
...

## Success metrics
- [ ] One real project uses nix develop + direnv
- [ ] HM manages shell + git
- [ ] Disk budget for /nix: __ GB
```

2. List five tools you currently install via raw binaries or language version managers; mark each **Nix candidate** or **keep distro**.

Checkpoint

☐ You can explain package manager vs NixOS in one breath

☐ You have a written split of responsibilities (dnf vs Nix)

☐ You know which skills transfer to NixOS unchanged

Further depth

- Install on Fedora 44: next chapter
- Concepts recap: Why Nix exists, Store and closures
- Eventually: NixOS install

Install Nix on Fedora 44

Install Nix on Fedora 44

Goal: Install Nix from **Fedora's official packages**, enable the daemon (or choose single-user/rootless deliberately), verify flakes, and know where Fedora documents live on disk.

> **!** Important
>
> Prefer **`dnf install nix`** on Fedora 44+ over the classic **`curl | sh`** upstream installer unless you have a specific reason (custom builds, non-Fedora policy). The distro package is the supported path for SELinux and **`/nix`** integration.

Prerequisites

Need	Check
Fedora 44 (or current release with **`nix`** in repos)	**`cat /etc/os-release`**
sudo	Wheel user
Disk	Prefer **20 GB** free on the filesystem that will hold **`/nix`**
Network	For first substitutes from cache.nixos.org

```
free -h
df -h /
# or wherever /nix will live
```

Install modes (choose one)

Fedora packaging supports several setups (see package docs):

Mode	Install sketch	Who builds	Best for
Multi-user + daemon (recommended)	`dnf install nix` + enable `nix-daemon`	`nixbld` users via daemon	Desktops, shared machines, matches NixOS mental model
Single-user	install excluding daemon / singleuser subpackage	Your UID	Solo laptops if daemon undesired
Rootless / Atomic	special `/nix` storage (bind mount)	Your UID / constrained	Silverblue, Kinoite, bootc images

This chapter emphasizes **multi-user daemon** on classic Workstation/Server, then notes Atomic.

Multi-user install (Workstation / Server)

1. Install packages

```
sudo dnf install -y nix
```

On some releases you may see related packages (`nix-daemon`, docs). Let dnf pull the recommended set.

```
rpm -q nix
rpm -ql nix | head
ls /usr/share/doc/nix/ 2>/dev/null || true
```

Read the Fedora-specific README if present:

```
less /usr/share/doc/nix/README.fedora.md
# or
rpm -qd nix
```

2. Enable the daemon

```
sudo systemctl enable --now nix-daemon.service
systemctl status nix-daemon --no-pager
```

3. Groups and shell environment

Multi-user installs typically put builders in `nixbld` and may require your user in a group to talk to the daemon. Fedora's README is authoritative; common pattern:

```
# If docs say so:
getent group nixbld
groups   # ensure you can use nix after re-login
```

Log out/in or open a **new** login shell so profile snippets from `/etc/profile.d/` or `/nix/...` apply.

```
# Common: bash/zsh pick up nix from profile.d
bash -l -c 'command -v nix && nix --version'
```

4. First smoke test

```
nix --version
nix doctor || true

# Classic hello (works even if you still learn modern CLI)
nix-shell -p hello --run hello

# Modern CLI + flakes (Fedora enables flakes in packaging - verify):
nix show-config | rg -n 'experimental-features|flakes|nix-command' || true
nix run nixpkgs#hello
```

If `nix run nixpkgs#hello` works, flakes + substitutes are alive.

Configure system vs user

Daemon / system config

Multi-user security options often live in **`/etc/nix/nix.conf`** (read by the daemon). Examples operators care about:

```
# Inspect effective config
nix show-config | rg -n 'sandbox|experimental|substituter|trusted'

# Example: ensure sandbox (security) - only if not already set by package defaults
# sudo tee -a /etc/nix/nix.conf <<'EOF'
# sandbox = true
# EOF
# sudo systemctl restart nix-daemon
```

> ⚠ Warning
>
> Do not casually set `trusted-users = *` or disable sandbox "to go faster" on a daily driver. Prefer package defaults; change with intent.

User config

```
mkdir -p ~/.config/nix
```

Fedora may already enable flakes system-wide. If `nix flake` fails, set:

```
cat >> ~/.config/nix/nix.conf <<'EOF'
experimental-features = nix-command flakes
EOF
```

Re-check:

```
nix show-config | rg experimental-features
```

Single-user mode (optional)

When you intentionally avoid the daemon (read Fedora docs for exact package names):

```
# Illustrative - confirm package set on your release:
# sudo dnf install nix --exclude nix-daemon
# or: sudo dnf install nix-singleuser
```

Typical consequences:

- Builds run as your user
- Socket/daemon workflow differs
- Closer to "my laptop only," farther from NixOS's always-daemon world

If you plan NixOS soon, **prefer multi-user** so daemon habits match.

Atomic / immutable desktops (Silverblue, Kinoite, bootc)

Immutable systems often make **`/nix` not writable** as a normal directory. Community patterns:

1. Store data under `/var/nix` (writable)
2. **Bind-mount** `/var/nix` → `/nix` (Nix rejects a mere symlink for the store root in many setups)

Example systemd mount unit (adapt paths; verify against current Atomic guidance):

```
# /etc/systemd/system/nix.mount  (illustrative)
[Unit]
Description=Bind mount /var/nix to /nix

[Mount]
What=/var/nix
Where=/nix
Type=none
Options=bind

[Install]
WantedBy=local-fs.target
```

```
sudo mkdir -p /var/nix /nix
sudo systemctl daemon-reload
sudo systemctl enable --now nix.mount
# then install/enable nix packages per Atomic docs
```

Also expect **SELinux** labeling steps on some images. Prefer Fedora documentation and the `nix` package README over blog posts when they disagree.

Verify deeply (acceptance checklist)

```
# Binary + version
nix --version

# Daemon (multi-user)
systemctl is-active nix-daemon

# Store exists and is a directory (not a broken symlink)
findmnt /nix || ls -ld /nix
df -h /nix
```

```
# Flake metadata (network)
nix flake metadata nixpkgs --json | head -c 200; echo

# Build something small and inspect store path
nix build nixpkgs#hello --print-out-paths
nix path-info -r $(nix build nixpkgs#hello --print-out-paths) | head
```

Record versions in your lab notes:

```
mkdir -p ~/lab/nixos-book/nix-on-linux
{
  echo "date: $(date -Iseconds)"
  echo "os: $(. /etc/os-release; echo $PRETTY_NAME)"
  nix --version
  rpm -q nix
  systemctl is-active nix-daemon || echo "daemon inactive"
} | tee ~/lab/nixos-book/nix-on-linux/install-record.txt
```

Upstream installer vs Fedora package

	Fedora `dnf` package	Upstream multi-user script
SELinux	Integrated for Fedora	Often needs extra steps
Updates	`dnf upgrade`	Separate Nix upgrade story
Flakes	Packaged defaults (enabled OOTB historically)	You enable experimental features
Support story	Fedora + Nix communities	Nix upstream
Custom Nix builds	Harder	Easier

You can still use **nixpkgs** the same way either way. This book assumes **Fedora packaging** on F44+.

Uninstall / reset notes

If you need a clean slate (lab machine):

```
# Stop daemon first
sudo systemctl disable --now nix-daemon 2>/dev/null || true
# Remove packages
sudo dnf remove nix
# WARNING: deleting the store removes all Nix software
# sudo rm -rf /nix
```

Only wipe `/nix` if you accept redownloading everything.

Common failures

Symptom	Likely cause	Direction
`nix: command not found`	No login shell / PATH	New login; check profile.d
Cannot connect to daemon	`nix-daemon` down; permissions	`systemctl status nix-daemon`; groups
Flakes experimental error	features not enabled	`~/.config/nix/nix.conf` or `/etc/nix/nix.conf`
SELinux denials	Mislabel / non-Fedora install	`ausearch -m avc -ts recent`; prefer dnf package
`/nix` read-only	Atomic without bind mount	Atomic section above
Disk full during build	Store on small root	Move store / free space / GC later chapter

---

Lab

1. Complete multi-user install on Fedora 44.
2. Capture `install-record.txt` as above.
3. Run `nix run nixpkgs#cowsay -- "fedora+nix"`.
4. Skim `/usr/share/doc/nix/` and note three Fedora-specific warnings in your lab journal.

Checkpoint

- ☐ `nix run nixpkgs#hello` succeeds
- ☐ Daemon mode intentional (or single-user/rootless documented)
- ☐ You know where **system** vs **user** `nix.conf` live
- ☐ Free space plan for `/nix` written down

Further depth

- Store, profiles, GC
- Concepts: Modern CLI, Flakes

Store, profiles, and GC on Linux

Store, profiles, and GC on Linux

Goal: Operate `/nix/store` confidently on Fedora: see what a **profile** is, what keeps paths alive (**GC roots**), and how to garbage-collect without deleting tomorrow's work.

This is the same store model as NixOS—only the set of roots differs (no bootloader generations yet).

Why this chapter matters

On Fedora, nothing forces you to be declarative. The fastest way to recreate "npm global hell" is:

```
nix profile install nixpkgs#tool-a
nix profile install nixpkgs#tool-b
# …six months, no locks, no notes…
```

Learn **roots** early and you will treat profiles as a **small** personal toolkit and flakes as the real project SoT—exactly the habit NixOS rewards.

Store anatomy (quick ops view)

```
ls /nix/store | head
nix path-info nixpkgs#hello
nix path-info -r $(nix build nixpkgs#hello --print-out-paths) | wc -l
```

Idea	Meaning
Store path	`/nix/store/<hash>-name` content-addressed build result
Closure	Path + all runtime references
GC root	A pointer that says "do not delete this closure"
Profile	User (or system) generation of linked apps—**a GC root**

Diagram:

```
~/.nix-profile     /nix/var/nix/profiles/per-user/$USER/profile


                     generation-42-link     store paths
```

Profiles: what they are for

Inspect

```
nix profile list
readlink -f ~/.nix-profile
ls -l /nix/var/nix/profiles/per-user/$USER/ | tail
```

Install / remove (modern)

```
nix profile install nixpkgs#ripgrep nixpkgs#fd
rg --version
fd --version

nix profile list
nix profile remove …   # use the index/name from list - check `nix profile remove --help`
```

Upgrade

```
nix profile upgrade
# or reinstall from a pinned flake input instead of floating nixpkgs#
```

> Tip
>
> **Bridge habit:** keep the imperative profile **thin** (editor, rg, direnv). Put project tools in **devShells**. On NixOS you will barely need `nix profile` at all.

Other roots you will meet

Root kind	Example	Created by
User profile	`~/.nix-profile`	`nix profile`
Result symlink	`./result`	`nix build` in a directory
direnv	nix-direnv GC roots	`use flake` + allow
Home Manager	HM profile generations	`home-manager switch`
NixOS system	`/run/current-system`	`nixos-rebuild` (**later**)

```
# List roots (classic tool still useful)
sudo nix-store --gc --print-roots 2>/dev/null | head -50
# or
nix-store --gc --print-roots 2>/dev/null | head -50
```

Find who pins a path:

```
# After building hello to ./result
nix build nixpkgs#hello
nix-store --query --roots $(readlink -f result) || true
```

Garbage collection

Safe everyday

```
# Delete unreachable paths
nix store gc
# or
nix-collect-garbage
```

Delete old profile generations

```
# User profiles / HM generations (read manpages for exact flags on your version)
nix-collect-garbage -d
# Older alias many still use:
# nix-env --delete-generations old
```

> ⚠ Warning
>
> `nix-collect-garbage -d` is aggressive on **generations**. Fine on a lab account; think twice if you rely on rolling back HM quickly.

Disk pressure playbook

```
df -h /nix
nix path-info -Sh /nix/var/nix/profiles/per-user/$USER/profile 2>/dev/null || true
# Find large paths (nix-tree if installed)
nix run nixpkgs#nix-tree
```

Optional: set free-space goals in `nix.conf` (daemon may need system file):

```
min-free = 1073741824
max-free = 5368709120
```

Channels vs flakes (on Fedora)

Fedora users sometimes enable channels by habit. **This book's default is flakes + locks.**

Approach	Pinning	Team repro
Channels + `nix-env`	Floating	Weak
`nix profile install nixpkgs#foo` without lock	Floating nixpkgs	Weak
Project `flake.lock`	Strong	Strong
HM flake input lock	Strong	Strong

```
# Prefer explicit flake refs in projects
nix flake metadata .
```

You may still see `nix-channel` on old blog posts—literacy only (Classic CLI).

Binary caches

Default substitutes (cache.nixos.org) keep Fedora+Nix usable.

```
nix show-config | rg substituter
# First download of a big stdenv is slow; later hits cache
```

Later: Cachix/Attic in packaging/ops parts. On a laptop, don't disable substitutes "for purity cosplay" unless studying FODs.

Worked lab: profile vs shell

```
cd ~/lab/nixos-book/nix-on-linux
mkdir -p store-lab && cd store-lab

# A) Ephemeral - disappears when shell exits (no profile pollution)
nix shell nixpkgs#jq --command jq --version

# B) Build root in cwd
nix build nixpkgs#jq
ls -l result
./result/bin/jq --version

# C) Profile (persistent)
nix profile install nixpkgs#jq
command -v jq

# Clean lab build root
rm -f result
nix store gc
```

Write three sentences in your journal: what remained after each of A/B/C.

Checkpoint

- ☐ You can define store path, closure, GC root
- ☐ `nix profile list` is intentional and small
- ☐ You can find roots and run `nix store gc` safely
- ☐ You prefer flake locks over channels for projects

Further depth

- Store and closures
- Daily CLI and devShells
- NixOS-side roots: Rebuild and generations

Nix without /nix: chroot stores, –store, and why paths bake in /nix/store

Nix without `/nix`: chroot stores, `--store`, and why paths bake in `/nix/store`

Sooner or later you meet a Linux machine where you cannot create `/nix`: a shared HPC login node, a locked-down CI runner, a container without root, a colleague's laptop. The tempting fix is "put the store somewhere else", and Nix lets you say exactly that in two very different ways. One keeps every store path identical to the rest of the world and only moves the bytes (a **chroot store**). The other changes the store's *name* and quietly throws away the binary cache. This chapter runs both on a host with no `/nix`, no root and no daemon, measures the difference, and shows the traps that appear when the host itself tries to use paths that only exist inside a namespace.

Everything below ran on this book's lab box: Debian 13, Linux 6.12, user `uid=1000`, **no `/nix` directory**, and the statically linked **Nix 2.35.3** binary on `PATH` as `nix`. nixpkgs is pinned to the `nixos-26.05` tip of 2026-10-18 (`8ce73368d1ef…`). Versions for this book: **Nix 2.35.3** (upstream tag; the nix.dev "latest" manual is 2.35.2), **NixOS 26.05**.

Learning goals

- Explain the difference between the **logical** store directory (`/nix/store`, inside every path and binary) and the **physical** location of the bytes
- Create and use a chroot store with `--store DIR` (`local?root=DIR`): build, substitute, run, copy and garbage-collect without root
- Run programs from a chroot store on the host with `nix shell --store` or `bwrap`, and know why `./result` and profiles dangle
- Predict what changing the logical store directory (`local?store=/opt/nix/store`) costs, and why `nix copy` between such stores is wrong
- Check whether a host can run chroot stores at all (unprivileged user and mount namespaces)

Thesis

> Move the bytes, never the name. A chroot store keeps `/nix/store` as the logical path and puts the files anywhere you own; a user namespace makes them appear at `/nix/store` whenever something runs. Change the name instead and every hash changes, the binary cache is gone, and you are bootstrapping a compiler from a hex seed.

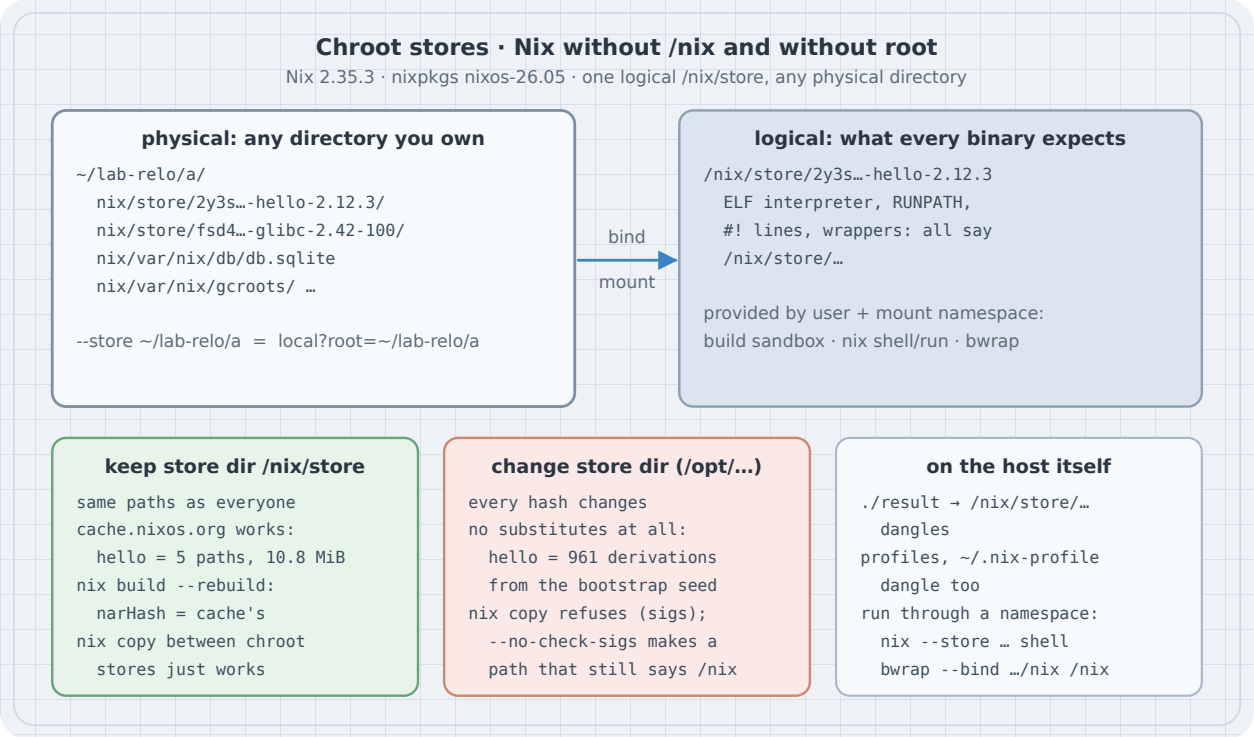


Figure 1: Chroot stores: one logical /nix/store, any physical directory

Mental model

```
logical path (in hashes, ELF headers, scripts)      physical bytes

/nix/store/2y3s…-hello-2.12.3            bind    ~/lab-relo/a/nix/store/2y3s…-hello-2.12.3
                                        mount in    ~/lab-relo/a/nix/var/nix/db/db.sqlite
                                        a user +
                                        mount ns
chroot store :  --store ~/lab-relo/a   =  local?root=~/lab-relo/a       (store dir stays /
relabelled   :  local?store=/opt/nix/store&real=…&state=…&log=…        (store dir changes:
```

Three facts carry the chapter:

1. **The store directory is part of every store path's hash** (see How Nix computes a store path). It is also written into the binaries themselves: the ELF interpreter, `RUNPATH`, `#!` lines and wrapper scripts all contain `/nix/store/…`. Moving files does not move those strings.
2. **A chroot store changes only where the bytes live.** The store directory stays `/nix/store`, so paths, hashes and signatures are the ones `cache.nixos.org` serves. To *use* the files, something must make them appear at `/nix/store`: Nix's own build sandbox, `nix shell`/`nix run` with `--store`, or any tool that can bind-mount in a user namespace.
3. **"Auto" already does this.** With the default `store = auto`, Nix uses `/nix/store` if `/nix/var/nix` is writable, else the daemon socket if it exists, else (Linux only) the chroot store `~/.local/share/nix/root`. A rootless Nix that "just works" on a machine with no `/nix` is a chroot store.

Case 1 — Can this host do it?

Chroot stores need unprivileged **user** and **mount** namespaces. Check before anything else.

```
nix --version
ls -d /nix
nix config show store
cat /proc/sys/user/max_user_namespaces
unshare --user --mount true && echo "unprivileged user+mount namespaces: ok"
```

```
nix (Nix) 2.35.3
ls: cannot access '/nix': No such file or directory
auto
63991
unprivileged user+mount namespaces: ok
```

No `/nix`, the store setting is `auto`, and an unprivileged process may create user and mount namespaces. Places where this fails, all checked from documentation rather than run here: `user.max_user_namespaces = 0`; Debian/Ubuntu kernels with `kernel.unprivileged_userns_clone = 0`; Ubuntu 23.10 and later, where `kernel.apparmor_restrict_unprivileged_userns = 1` lets only AppArmor-profiled programs create user namespaces with full capabilities; and container runtimes whose seccomp profile blocks `unshare`/`clone` with `CLONE_NEWUSER`. On those hosts a chroot store cannot build or run anything; ask for a real `/nix` (or a bind mount at `/nix`) instead of relabelling the store (Case 5).

Case 2 — A chroot store: physical layout, logical paths

```
P=github:NixOS/nixpkgs/8ce73368d1efd4d657e9064d29b144a52046b4e8
H=/nix/store/2y3s86m1yj9xnypd44vivfz3mwybzlki-hello-2.12.3
nix --store ~/lab-relo/a build --no-link --print-out-paths "$P#hello"
ls ~/lab-relo/a ~/lab-relo/a/nix ~/lab-relo/a/nix/var/nix
nix --store ~/lab-relo/a path-info -r "$H"
readelf -l ~/lab-relo/a$H/bin/hello | grep -o 'interpreter: [^]]*'
~/lab-relo/a$H/bin/hello; echo "exit=$?"
```

```
/nix/store/2y3s86m1yj9xnypd44vivfz3mwybzlki-hello-2.12.3
/home/box/lab-relo/a:
nix

/home/box/lab-relo/a/nix:
store
var

/home/box/lab-relo/a/nix/var/nix:
```

```
db
gc.lock
gcroots
profiles
temproots
/nix/store/l1n6wi2lyzglvgzbwzw9n9w2pzkcj41i-xgcc-15.3.0-libgcc
/nix/store/g6mvl9ns7k52zfv9pcfw3dbh65z3j7id-libunistring-1.4.2
/nix/store/l5y42bimjvvn8h3v5y40w13vv7y2p3k5-libidn2-2.3.8
/nix/store/fsd4vbp9zjd9iaicq2fb9lpc1h5j973y-glibc-2.42-100
/nix/store/2y3s86m1yj9xnypd44vivfz3mwybzlki-hello-2.12.3
interpreter: /nix/store/fsd4vbp9zjd9iaicq2fb9lpc1h5j973y-glibc-2.42-100/lib/ld-linux-x86-64.
/tmp/relo2/runall.sh: line 19: /home/box/lab-relo/a/nix/store/2y3s86m1yj9xnypd44vivfz3mwybzl
exit=127
```

`--store ~/lab-relo/a` is shorthand for `local?root=/home/box/lab-relo/a`: an ordinary local store whose every directory (`store`, `state`, `log`) is prefixed with that root. The printed path is the logical one, `/nix/store/2y3s…`, byte for byte what a NixOS machine prints. The bytes are at `~/lab-relo/a/nix/store/2y3s…`, with the SQLite database and GC roots under `~/lab-relo/a/nix/var/nix`.

The last two lines are the whole problem in one error (the `runall.sh` prefix is bash naming the lab script that ran the command). The binary exists, but its ELF interpreter is `/nix/store/…-glibc-2.42-100/lib/ld-linux-x86-64.so.2`, which does not exist on this host. The kernel cannot start it ("required file not found" is `ENOENT` for the interpreter, not for `hello`).

Case 3 — Running programs from a chroot store

Two ways to make the logical path real, both unprivileged:

```
nix --store ~/lab-relo/a shell "$H" --command sh -c \
  'hello; command -v hello; grep " /nix/store " /proc/self/mountinfo | cut -d" " -f4,5; id -
bwrap --bind ~/lab-relo/a/nix /nix "$H/bin/hello" --greeting="hello via bwrap"
```

```
Hello, world!
/nix/store/2y3s86m1yj9xnypd44vivfz3mwybzlki-hello-2.12.3/bin/hello
/home/box/lab-relo/a/nix/store /nix/store
1000
hello via bwrap
```

`nix shell --store …` creates a user and mount namespace, bind-mounts `~/lab-relo/a/nix/store` at `/nix/store` (the `mountinfo` line shows source root and mount point) and runs the command there as the same uid. `nix run --store …` uses the same mechanism (not run here). Outside Nix, `bwrap` (bubblewrap) does it with one bind mount; because bubblewrap starts from an empty root, `hello`'s closure is all the program sees, which is exactly what a Nix closure promises it needs. `nix-user-chroot` and `nix-portable` are community tools built on the same idea (nix-portable

falls back to `proot`, which uses `ptrace` and is much slower, when user namespaces are unavailable); neither was run here.

Case 4 — Builds in a chroot store are the same builds

The build sandbox already runs every builder in its own namespaces with inputs bind-mounted at `/nix/store/…`. A chroot store only changes the source of those bind mounts. To prove it, rebuild `hello` from source in the chroot store and compare with the binary cache:

```
nix --store ~/lab-relo/a build --rebuild --no-link "$P#hello" 2>&1 | grep -v '^copying path'
echo "exit=${PIPESTATUS[0]}"
for s in ~/lab-relo/a https://cache.nixos.org; do
  printf '%-28s %s\n' "$s" "$(nix path-info --store "$s" --json --json-format 1 "$H" | jq -r
done
```

```
checking outputs of '/nix/store/v4c5h94di88ccdd9isyzhivdw84k81nh-hello-2.12.3.drv'...
exit=0
/home/box/lab-relo/a         sha256-OuTV3GyEZLx2+V1V4xaqEYxdvzrsMuymxdvL4WWaWyE=
https://cache.nixos.org      sha256-OuTV3GyEZLx2+V1V4xaqEYxdvzrsMuymxdvL4WWaWyE=
```

`--rebuild` substituted the build inputs (43 paths, filtered out above), compiled GNU hello again under `~/lab-relo/a`, and compared the result with the copy already in the store. Exit 0 and no "may not be deterministic" error: bit-identical. The NAR hash matches what `cache.nixos.org` serves for the same path. A user on a cluster with no `/nix` produces the same bits as Hydra.

The sandbox hides the physical location from *paths*, not from a builder that goes looking. This deliberately nosy derivation records its mounts:

```
# where.nix - a derivation that records where it thinks it is being built
let
  sh = "${builtins.storePath /nix/store/fl6zyj3yfpa973icc6ca3zbm55izmm9r-busybox-1.37.0}/bin
in
derivation {
  name = "where-am-i";
  system = "x86_64-linux";
  builder = sh;
  args = [ "-c" ''
    echo "out   = $out" > $out
    echo "shell = ${sh}" >> $out
    while read -r id parent dev root mnt rest; do
      case $mnt in /nix/store/*) echo "mount = $root -> $mnt" >> $out ;; esac
    done < /proc/self/mountinfo
  '' ];
}
```

```
BB=/nix/store/fl6zyj3yfpa973icc6ca3zbm55izmm9r-busybox-1.37.0
nix --store ~/lab-relo/a build --no-link "$P#busybox-sandbox-shell"
mkdir -p ~/lab-relo/elsewhere
nix copy --from ~/lab-relo/a --to ~/lab-relo/elsewhere/b "$BB"
for s in ~/lab-relo/a ~/lab-relo/elsewhere/b; do
  o=$(nix --store "$s" build -f where.nix --no-link --print-out-paths)
  echo "== $s"; cat "$s$o"
  nix --store "$s" path-info --json --json-format 1 "$o" | jq -r '.[].narHash'
done
```

```
== /home/box/lab-relo/a
out    = /nix/store/95p76pxpqgy0n1pkyrghvcrnq0zwy6kq-where-am-i
shell = /nix/store/fl6zyj3yfpa973icc6ca3zbm55izmm9r-busybox-1.37.0/bin/sh
mount = /home/box/lab-relo/a/nix/store/fl6zyj3yfpa973icc6ca3zbm55izmm9r-busybox-1.37.0 -> /n
sha256-kAo3I6hwz5QYxc8ij4medwDO2Rr8H99Xg2Ez23/kaa0=
== /home/box/lab-relo/elsewhere/b
out    = /nix/store/95p76pxpqgy0n1pkyrghvcrnq0zwy6kq-where-am-i
shell = /nix/store/fl6zyj3yfpa973icc6ca3zbm55izmm9r-busybox-1.37.0/bin/sh
mount = /home/box/lab-relo/elsewhere/b/nix/store/fl6zyj3yfpa973icc6ca3zbm55izmm9r-busybox-1.
sha256-82ArCXEVkU/AH39Xy7f5tNRYNOObKzSP9vj2FAPoek8=
```

Same derivation, same output path (input-addressed: the path depends on the recipe, not on the result), two different NAR hashes. `/proc/self/mountinfo` inside the sandbox names the real source of each bind mount, and this builder wrote it into `$out`. Nothing in Nix flags this: each store built and registered its own bytes. Real packages do not read `mountinfo`, which is why `hello` matched above; but it is the reason "same path" never *proves* "same bytes". Use `nix build --rebuild` (or `--check`) and compare NAR hashes when it matters.

Case 5 — Relabelling the store: the expensive mistake

Now the other way: keep the files in a home directory and also *call* the store `/opt/nix/store`.

```
S='local?store=/opt/nix/store&state=/home/box/lab-relo/c/state&log=/home/box/lab-relo/c/log&
nix --store "$S" eval --no-eval-cache --raw "$P#hello.outPath"; echo
nix --store "$S" build --no-eval-cache --dry-run "$P#hello" 2>&1 | grep 'will be'
nix --store ~/lab-relo/d build --no-eval-cache --dry-run "$P#hello" 2>&1 | grep 'will be'
```

```
/opt/nix/store/qj1hpmrfji1r7lfv1pby3lizgbx856mx-hello-2.12.3
these 961 derivations will be built:
these 5 paths will be fetched (10.8 MiB download, 36.2 MiB unpacked):
```

Same nixpkgs commit, same `hello`, new hash (`qj1h…` instead of `2y3s…`), because the store directory is inside the fingerprint. Nothing under `/opt/nix/store` exists in any public cache, so the relabelled store must build **961 derivations**, starting from nixpkgs 26.05's minimal bootstrap (a `hex0` seed, then `stage0-posix`, the Mes C library and up to GCC). The chroot store `d`, with the store directory

left alone, needs **5 downloads, 10.8 MiB**. `--no-eval-cache` matters here: the flake evaluation cache is keyed by the flake, not by the store directory, and without it the second store can get a cached `/nix/store` answer.

And you cannot convert between the two by copying:

```
nix copy --from ~/lab-relo/a --to "$S" "$H" 2>&1 | grep '^error:'
nix copy --no-check-sigs --from ~/lab-relo/a --to "$S" "$H" && echo "copy with --no-check-si
ls ~/lab-relo/c/store | grep 'hello-2.12.3$'
readelf -l ~/lab-relo/c/store/${H#/nix/store/}/bin/hello | grep -o 'interpreter: [^]]*'
```

```
error: cannot add path '/opt/nix/store/g6mvl9ns7k52zfv9pcfw3dbh65z3j7id-libunistring-1.4.2'
copy with --no-check-sigs: exit 0
2y3s86m1yj9xnypd44vivfz3mwybzlki-hello-2.12.3
interpreter: /nix/store/fsd4vbp9zjd9iaicq2fb9lpc1h5j973y-glibc-2.42-100/lib/ld-linux-x86-64.
```

The first copy is refused: the signatures from `cache.nixos.org` are for `/nix/store/…` names. Forcing it with `--no-check-sigs` "works", and produces `/opt/nix/store/2y3s…-hello-2.12.3`: a path whose hash belongs to a *different* store directory and whose binary still asks for an interpreter in `/nix/store`. The manual is explicit that paths can only be copied between stores with the same `store` setting; the tool does not stop you once you disable the check.

Case 6 — The host's view: results, profiles and GC roots

Anything Nix leaves on the host side (`./result`, profile links, `~/.nix-profile`) is a symlink to a logical path.

```
nix --store ~/lab-relo/a profile add --profile ~/lab-relo/prof-new "$H" 2>&1 | tail -1
nix-env --store ~/lab-relo/a -p ~/lab-relo/prof -i "$H" 2>&1
find ~/lab-relo -maxdepth 1 -name 'prof*' -printf '%f -> %l\n' | sort
test -x ~/lab-relo/prof/bin/hello && echo "prof/bin/hello runs on the host" || echo "prof/bi
bwrap --bind ~/lab-relo/a/nix /nix --ro-bind ~/lab-relo ~/lab-relo ~/lab-relo/prof/bin/hello
nix-store --store ~/lab-relo/a --gc --print-roots 2>/dev/null
nix --store ~/lab-relo/a store gc 2>&1 | tail -1
nix --store ~/lab-relo/a path-info "$H" && echo "hello survived: the profile is a GC root"
```

```
error: cannot read directory "/nix/store/2y3s86m1yj9xnypd44vivfz3mwybzlki-hello-2.12.3": No
installing 'hello-2.12.3'
building '/nix/store/a9m9hffbk96vi1inwnrvx9c3l247r408-user-environment.drv'...
prof -> prof-1-link
prof-1-link -> /nix/store/cxa0nmp5aa22rnxjpn0gibqm7q10yn6a-user-environment
prof/bin/hello: dangling on the host
Hello, world!
"/home/box/lab-relo/prof-1-link" -> /nix/store/cxa0nmp5aa22rnxjpn0gibqm7q10yn6a-user-environ
116 store paths deleted, 358.7 MiB freed
```

```
/nix/store/2y3s86m1yj9xnypd44vivfz3mwybzlki-hello-2.12.3
hello survived: the profile is a GC root
```

What happened:

- **`nix profile add` failed** with Nix 2.35.3 and a chroot store: it assembles the profile by reading package directories at their logical path on the host, which does not exist. Observed here; treat it as a current limitation, not a rule.
- **`nix-env -i` worked**, because classic `nix-env` builds the profile as a derivation (`user-environment`, using the built-in `buildenv` builder) inside the sandbox, where `/nix/store` exists.
- The profile link points at `/nix/store/…-user-environment`, so `~/lab-relo/prof/bin/hello` **dangles on the host**. Putting it on `PATH` does nothing. It runs once the store is mounted (`bwrap`), and the profile directory is bind-mounted read-only so the link can be followed.
- The profile is registered as an indirect **GC root** in the chroot store's `gcroots/auto`, so `nix store gc` (which removed 116 paths, mostly the build inputs from Case 4) kept `hello`.

The trap

Trap: "set `store = /opt/nix/store` and keep using the cache." The store directory is in every hash. A relabelled store gets no substitutes and rebuilds the world from the bootstrap seed (Case 5). If `/nix` cannot exist, use a chroot store; if `/nix` can exist but the disk is elsewhere, bind-mount or mount that storage *at* `/nix`.

Trap: "`./result` works, so I can put the chroot store's bins on `PATH`." Every link Nix creates points at the logical `/nix/store/…`. On a host without `/nix` they dangle (Cases 2 and 6). Run things through `nix shell --store …`, `nix run --store …` or a bind-mounting wrapper, or wrap your shell session in one.

Trap: "same store path means same bytes." Input-addressed paths are a promise about the *recipe*. A builder that reads `/proc/self/mountinfo`, the host name or the time produces different bytes at the same path in different stores, and nothing complains (Case 4). Check with `--rebuild` and NAR hashes.

Trap: "`rm -rf` the lab when done." Store paths are read-only (`0555` directories), so a plain `rm -rf ~/lab-relo` fails with "Permission denied" on every file (it did, on the first run of this lab). Use `nix store gc` on the store, or `chmod -R u+w` the directory first.

Trap: "it worked on my laptop, so it works on the cluster." Chroot stores live or die by unprivileged user namespaces. Check `unshare --user --mount true` on the target before promising anyone a rootless Nix (Case 1).

The boring rule

1. Never change the logical store directory. Keep `/nix/store`.

2. Without root and without `/nix`, use a chroot store: `--store ~/some/dir` (or let `store = auto` pick `~/.local/share/nix/root`), and run programs through `nix shell`/`nix run --store` or `bwrap`.
3. With root once, prefer a real `/nix` (a dedicated filesystem or bind mount at `/nix`) and the daemon; the chroot store is the fallback, not the goal.
4. Treat everything Nix writes on the host side of a chroot store (results, profiles) as meaningful only inside the namespace.

Try this

1. Run `nix --store ~/lab-relo/a run "$P#cowsay" -- moo` and find, in `~/lab-relo/a/nix/var/nix/gcro` whether anything now keeps `cowsay` alive.
2. Point `--store` at a directory on an NFS or Lustre home on a cluster you use. Does SQLite locking under `nix/var/nix/db` behave? (The manual's `read-only` store setting exists for read-only database filesystems.)
3. Make a binary cache from the chroot store with `nix copy --to file:///tmp/cache "$H"`, read `nix-cache-info`, and substitute from it into a fresh chroot store with `--substituters file:///tmp/cache --no-require-sigs`.
4. Change `where.nix` to stop reading `mountinfo`, rebuild it in both stores and compare NAR hashes again.
5. On a test VM where you are root, set `kernel.apparmor_restrict_unprivileged_userns=1` (Ubuntu) or `user.max_user_namespaces=0`, and watch Case 3 fail. Revert afterwards.

Next

- Store, profiles, and GC on Linux for the normal, daemon-backed store.
- How Nix computes a store path for why the store directory is inside every hash.

Verification

- **Run here, unprivileged:** Debian 13, Linux 6.12.94+, `uid=1000`, no `/nix`, no daemon, no root. `nix` is the statically linked **Nix 2.35.3** binary already on the box (nothing installed). `bwrap` 0.12.0 and `jq` were already present. nixpkgs pinned to `nixos-26.05` at `8ce73368d1efd4d657e9064d29b144a52046b4e8` (branch tip 2026-10-18, GitHub API).
- One script recreated the lab from an empty `~/lab-relo` and captured every output block above; only the 43 `copying path` lines of Case 4 and the stderr of `nix-store --print-roots` (stale temp-root notices) are filtered, as the listed commands show. The first run also produced the `rm -rf` "Permission denied" flood described in the trap; the script now does `chmod -R u+w` first.
- Versions checked 2026-10-19: Nix upstream tag **2.35.3**; nix.dev "latest" manual **2.35.2**; NixOS **26.05** current stable.
- `store = auto` semantics quoted from `nix help-stores` (Nix 2.35.3). The `local` store settings (`root`, `store`, `real`, `state`, `log`, `read-only`) and the "same `store` setting" rule for copying come from the Local Store page of the manual.

- **Not run:** hosts where user namespaces are disabled or restricted (sysctls need root; written from kernel and Ubuntu documentation), `nix-user-chroot` and `nix-portable` (not installed), and Try-this item 2 (no NFS/Lustre here).

Sources

- Nix manual — Local Store (chroot stores, `root`, `store`, `real`, `state`, `log`, `read-only`; "you can only copy store paths between stores if they have the same `store` setting"), `store` setting and the `auto` store, `nix store gc`, `nix build --rebuild`, `nix copy`, Store path specification
- Linux — `user_namespaces(7)`, `mount_namespaces(7)`, `/proc/sys/user/max_user_namespaces`
- Ubuntu — Restricted unprivileged user namespaces (`kernel.apparmor_restrict_unprivileged_userns`)
- bubblewrap — https://github.com/containers/bubblewrap
- nixpkgs minimal bootstrap — `pkgs/os-specific/linux/minimal-bootstrap` in https://github.com/NixOS/nixpkgs/tree/nixos-26.05
- Community tools (not run): nix-user-chroot, nix-portable

Inspiration

X post by @bereknyei (2026-08-07): https://x.com/bereknyei/status/2085834505126879588 — topic signal only (a pointer to the Nix Vegas / DEF CON talk on relocatable Nix stores). The chapter cites only the primary docs above.

Daily CLI, flakes, and devShells

Daily CLI, flakes, and devShells

Goal: Make Nix on Fedora your **default way to enter project toolchains**—with flakes, locks, `nix develop`, and direnv—so the same workflow continues unchanged on NixOS.

Modern CLI map (muscle memory)

Task	Command
Build package	`nix build .#pkg` / `nix build nixpkgs#hello`
Run without installing	`nix run nixpkgs#cowsay -- hi`
Temporary tools	`nix shell nixpkgs#jq nixpkgs#yq-go`
Project shell	`nix develop`
Search	`nix search nixpkgs ripgrep`
Eval	`nix eval .#foo`
Flake show	`nix flake show`
Update inputs	`nix flake update`
Metadata	`nix flake metadata`

Always prefer **`nix`** over `nix-env` / `nix-channel` for new work.

First project flake on Fedora

```
mkdir -p ~/lab/nixos-book/nix-on-linux/demo-flake
cd ~/lab/nixos-book/nix-on-linux/demo-flake
git init
```

`flake.nix` (pin philosophy: track **nixos-26.05** to match this book, or `nixpkgs-unstable` if you accept drift—**commit the lock either way**):

```
{
  description = "Fedora bridge demo flake";
```

```
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux"; # or: builtins.currentSystem cautiously; prefer explicit
      pkgs = import nixpkgs { inherit system; };
    in {
      packages.${system}.default = pkgs.hello;

      devShells.${system}.default = pkgs.mkShell {
        packages = with pkgs; [
          git
          jq
          ripgrep
          # language example - pick what you actually use:
          go_1_24 # adjust attribute to what exists on your pin
          # python3
          # nodejs_22
        ];
        shellHook = ''
          echo "devShell ready on $(uname -sr) - Nix on Linux bridge"
        '';
      };
    };
}
```

> **Note**
>
> Attribute names like `go_1_24` change across nixpkgs pins. Use `nix search nixpkgs go` or `nix repl` to confirm on **your** lock. The pattern matters more than the exact attribute.

```
git add flake.nix
nix flake lock
git add flake.lock
nix develop
# inside shell:
jq --version
rg --version
exit
```

Reproducibility check (second machine / second user):

```
nix develop --offline   # after first populate, often works if store warm
```

`nix develop` vs `nix shell` vs profile

	Lifetime	Pinned by	Use
`nix shell nixpkgs#…`	Shell only	Floating unless flake ref	Quick experiments
`nix develop`	Shell only	**flake.lock**	Real projects
`nix profile install`	Until removed	Weak unless you pin	Global personal CLIs

---

direnv + nix-direnv (high leverage)

Auto-load shells when you `cd` into a project—same on Fedora and NixOS.

Install tools (profile or HM later)

```
nix profile install nixpkgs#direnv nixpkgs#nix-direnv
```

direnvrc

```
mkdir -p ~/.config/direnv
# Prefer path from nix-direnv package; common pattern:
echo 'source $HOME/.nix-profile/share/nix-direnv/direnvrc' >> ~/.config/direnv/direnvrc
```

Shell hook

bash (`~/.bashrc`):

```
eval "$(direnv hook bash)"
```

zsh (`~/.zshrc`):

```
eval "$(direnv hook zsh)"
```

Project

```
cd ~/lab/nixos-book/nix-on-linux/demo-flake
echo 'use flake' > .envrc
direnv allow
cd ..
cd demo-flake   # should load shellHook / PATH
```

nix-direnv creates **GC roots** so `nix store gc` won't delete your active shell deps mid-week. Review roots periodically.

Ergonomic extras (optional)

```
nix profile install \
  nixpkgs#nix-output-monitor \
  nixpkgs#nix-tree \
  nixpkgs#nix-your-shell
```

Tool	Role
`nom` (nix-output-monitor)	Readable build logs (`nom develop`, etc.)
`nix-tree`	Why is the store huge?
`nix-your-shell`	Keep zsh/fish inside `nix develop`

Example zsh after omz/init:

```
# if nix-your-shell installed
command -v nix-your-shell >/dev/null && nix-your-shell zsh | source /dev/stdin
```

Multi-system flakes (preview)

Laptops vary (`x86_64-linux`, `aarch64-linux`). Pattern:

```
outputs = { self, nixpkgs }:
  let
    systems = [ "x86_64-linux" "aarch64-linux" ];
    forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f system);
  in {
```

```
    devShells = forAllSystems (system:
      let pkgs = nixpkgs.legacyPackages.${system};
      in {
        default = pkgs.mkShell { packages = [ pkgs.jq ]; };
      });
  };
```

Same flake later grows `nixosConfigurations.myhost = …` without changing how `devShells` work.

CI-shaped local check

```
# Fail if lock missing or eval broken
nix flake check || nix develop -c true
```

When you add packaging later, `checks` become richer (flake checks).

Lab

1. Convert **one real project** you maintain to `flake.nix` + `use flake` direnv.
2. Commit `flake.lock`.
3. On a clean shell, `cd` into the project and confirm tools appear via direnv.
4. Run `nix flake metadata` and paste the lock refreshed time into your journal.

Checkpoint

☐ You use `nix develop` daily for at least one repo

☐ `flake.lock` is committed

☐ direnv loads without manual `nix develop`

☐ Profile remains small

Next

- Deep dive: Self-contained scripts with `#! nix-shell` / `#! nix`

Further depth

- Flakes and devShells
- Home Manager standalone
- direnv chapter in Home layout

Self-contained scripts with #! nix-shell / #! nix

Self-contained scripts with `#! nix-shell` / `#! nix`

A script that starts `curl | xml2json | jq` works on your laptop and fails on a colleague's, on CI, and on the server that has only `curl`. The usual fixes are "install the deps", a container, or a Go/Rust rewrite. **Nix shebang scripts** keep the script as a script and declare the interpreter and every tool it needs in the file itself. Anyone with Nix can run it; without Nix they cannot, which is the honesty the shebang encodes. This chapter covers both forms still in use: classic `nix-shell -i … -p …` and the modern `nix shell … --command` shebang, how the kernel and `env` actually start them, how to pin so tomorrow's run is the same as today's, and the traps that make a shebang silently wrong.

Nix is **not installed on this box**. Every shebang script below is written from the Nix 2.35 and nix.dev manuals and is marked **not run here**. What *was* run is the kernel/`env` behaviour that decides whether Nix ever gets a chance: how a multi-line shebang is handed off, what `env` does with spaces, and how `BINPRM_BUF_SIZE` truncates a long first line. Versions for this book: **Nix 2.35.3** (upstream tags), **NixOS 26.05** (stable "Yarara").

Learning goals

- Write a classic `#!/usr/bin/env nix-shell` script with `-i`, `-p` / `--packages`, `--pure` and a pinned `-I nixpkgs=…`
- Write the modern `#!/usr/bin/env nix` form with `nix shell` and `--command`
- Explain why options must live on extra `#! nix-shell` / `#! nix` lines, and when `env -S` is an alternative
- Pin a revision (tarball, flake lock, or `--override-input`) so the script is reproducible
- Say when a shebang script is the right tool versus a `devShell`, a flake app (`nix run`) or a packaged derivation

Thesis

> A shebang script is a tiny, portable derivation of *one* run. Pin the inputs, declare the packages, and the script carries its own toolchain. Leave the pin floating and you have reinvented "works on my machine" with extra steps.

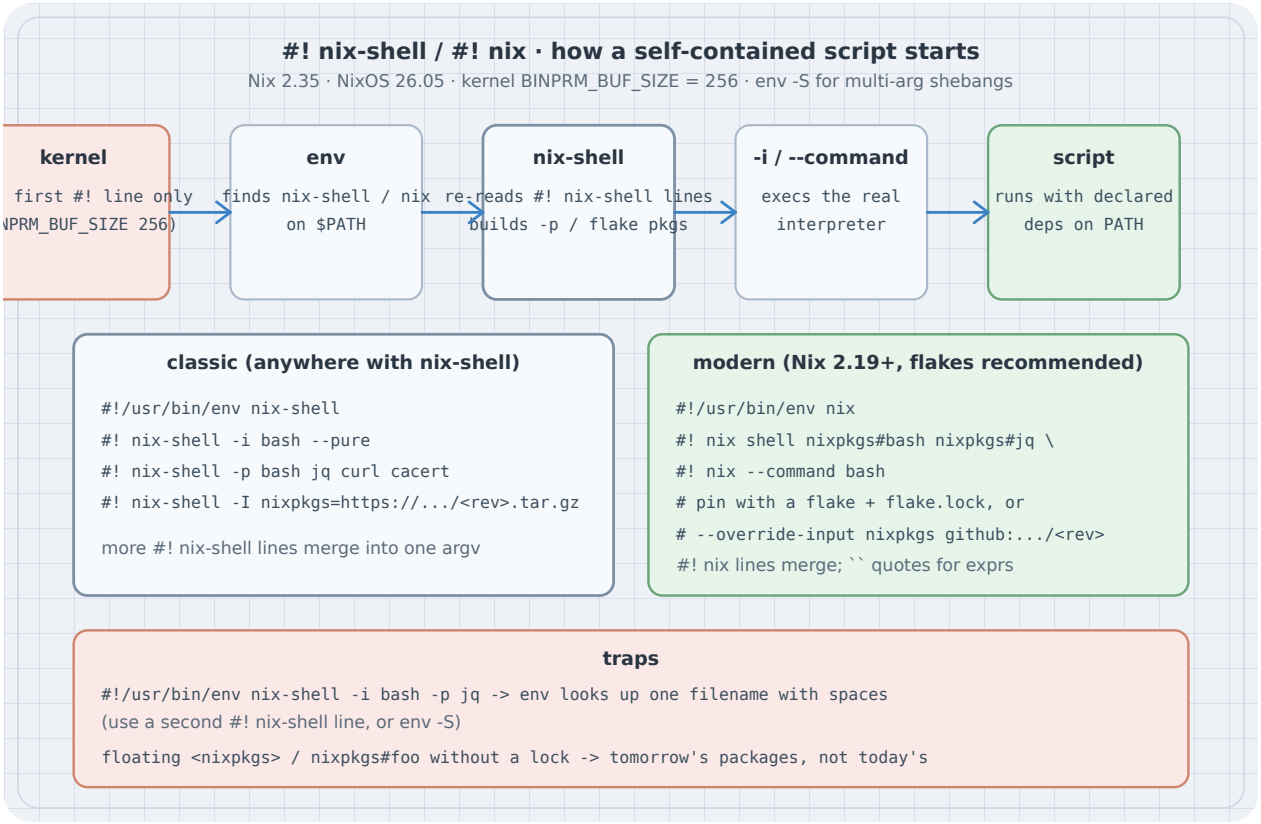


Figure 1: How a nix-shell / nix shebang starts

Mental model

```
script file

  #!/usr/bin/env nix-shell     ← only this line is a
  #! nix-shell -i bash --pure     kernel shebang
  #! nix-shell -p bash jq …    ← Nix re-reads these
  #! nix-shell -I nixpkgs=…/<rev>.tar.gz
  … body …


          kernel: execve("/usr/bin/env", ["nix-shell", script, args…])
          env:     find "nix-shell" on $PATH
          nix-shell: parse #! nix-shell lines, realise packages, exec -i
```

Two facts matter before any Nix option:

1. **The kernel sees only the first `#!` line**, and at most `BINPRM_BUF_SIZE` (256) bytes of it. Everything after the first space on that line is one argument to the interpreter (unless you use `env -S`). Extra `#! nix-shell` / `#! nix` lines are ordinary text to the kernel; Nix opens the file and parses them itself.
2. **`/usr/bin/env` looks up a *name*.** `#!/usr/bin/env nix-shell -i bash` makes `env` search for a file literally called `nix-shell -i bash`. That is why the options go on subsequent lines, or why you write `#!/usr/bin/env -S nix-shell -i bash …` (`env -S` splits).

Case 1 — What the kernel and `env` actually pass (run here)

A stand-in `nix-shell` that only prints its argv shows the hand-off. Nothing below needs Nix.

```
# fakebin/nix-shell - a STAND-IN, not Nix: print exactly what env hands over
#!/bin/sh
printf '$0 = %s\n' "$0"
i=1
for a in "$@"; do printf '$%d = %s\n' "$i" "$a"; i=$((i + 1)); done
```

```
# report.sh - what a real script looks like to the stand-in
#!/usr/bin/env nix-shell
#! nix-shell -i bash --pure
#! nix-shell -p bash jq
echo "this line is for bash, not for the stand-in"
```

```
chmod +x fakebin/nix-shell report.sh
PATH="$PWD/fakebin:$PATH" ./report.sh --since 2026-10-01 "two words"
```

```
$0 = /workspace/lab-1018am-shebang/fakebin/nix-shell
$1 = ./report.sh
$2 = --since
$3 = 2026-10-01
$4 = two words
```

`env` found `nix-shell` on `PATH` and gave it the **script path** and then the user arguments. The `#! nix-shell …` lines never appear in argv: a real `nix-shell` reopens `$1` and reads them. That is also why you must not put options on the first line:

```
# wrong.sh - options on the first line
#!/usr/bin/env nix-shell -i bash -p jq
echo unreachable
```

```
PATH="$PWD/fakebin:$PATH" ./wrong.sh; echo exit:$?
```

```
env: ‘nix-shell -i bash -p jq’: No such file or directory
env: use -[v]S to pass options in shebang lines
exit:127
```

GNU `env` even tells you the fix. With `env -S` the first line *can* carry options, and the stand-in then sees them as separate argv entries before the script path:

```
# split.sh
#!/usr/bin/env -S nix-shell -i bash -p jq
echo unreachable with the stand-in
```

```
PATH="$PWD/fakebin:$PATH" ./split.sh
```

```
$0 = /workspace/lab-1018am-shebang/fakebin/nix-shell
$1 = -i
$2 = bash
$3 = -p
$4 = jq
$5 = ./split.sh
```

Prefer the multi-line `#! nix-shell` form: it is what the Nix manuals show, it works on `env` without `-S`, and it keeps the first line short.

The first line also has a hard length limit. On Linux the kernel copies at most `BINPRM_BUF_SIZE` (256) bytes of the shebang, including the `#!`:

```
# long.py - how much of a long #! line survives?
python3 - <<'PY'
import os, subprocess
for n in (200, 240, 300):
```

```
    path = f"/tmp/long-shebang-{n}.sh"
    with open(path, "w") as f:
        f.write("#!/usr/bin/env -S printf %s\\n " + "x" * n + "\necho never\n")
    os.chmod(path, 0o755)
    r = subprocess.run([path], capture_output=True, text=True)
    got = len(r.stdout.splitlines()[0]) if r.stdout else 0
    print(f"argument of {n:3} x's -> printf received {got:3} x's"
          f"  (shebang line {len('#!/usr/bin/env -S printf %s\\n ') + n} bytes)")
    os.unlink(path)
PY
```

```
argument of 200 x's -> printf received 200 x's  (shebang line 230 bytes)
argument of 240 x's -> printf received 225 x's  (shebang line 270 bytes)
argument of 300 x's -> printf received 225 x's  (shebang line 330 bytes)
```

Past 256 bytes the argument is silently truncated. A long `-I nixpkgs=https://…/<64-char-sha>.tar.gz` on the first line is how people hit this; put pins on later `#!` lines, which Nix reads from the file and which are not limited by `BINPRM_BUF_SIZE`.

Case 2 — Classic form: `nix-shell -i / -p / --pure / -I` (not run here)

This is the form in the Nix 2.35 `nix-shell` manual and the nix.dev tutorial *Reproducible interpreted scripts*. It works with any Nix that provides `nix-shell`, including non-flake installs.

```
# nixpkgs-releases.sh - declare bash, curl, jq, xml2json; pin nixpkgs
#!/usr/bin/env nix-shell
#! nix-shell -i bash --pure
#! nix-shell -p bash cacert curl jq python3Packages.xmljson
#! nix-shell -I nixpkgs=https://github.com/NixOS/nixpkgs/archive/26f4107e47e3ad0ffd0db220469
curl -fsSL https://github.com/NixOS/nixpkgs/releases.atom | xml2json | jq .
```

```
# not run here - Nix is not installed on this box
# chmod +x nixpkgs-releases.sh && ./nixpkgs-releases.sh
```

What each line does:

Piece	Meaning
`#!/usr/bin/env nix-shell`	Kernel shebang. `env` finds `nix-shell` on `PATH` (so Nix must be on `PATH` for the user who runs it).
`-i bash`	After the environment is ready, exec `bash` as the real interpreter; the script path is passed as `$1` to it.

Piece	Meaning
`--pure`	Clear almost all of the ambient environment before starting. Retains a few variables (`HOME`, `USER`, `DISPLAY`, …). Forces the script to use only the packages you listed.
`-p` / `--packages`	Attribute paths in nixpkgs that should be on `PATH` (and in `buildInputs` for the temporary derivation). Multiple `-p` lines merge.
`-I nixpkgs=https://…/<rev>.tar.gz`	Pin. Without this, `nix-shell` uses whatever `<nixpkgs>` resolves to in `NIX_PATH` / the flake registry today.
`cacert`	Needed under `--pure` so TLS trusts work (`curl` https).

A language other than bash is the same pattern: `-i python3` with `python3` and the libraries on `-p`, `-i runghc` with a GHC-with-packages expression, and so on. For languages where `#` is not a comment (JavaScript, Ruby, PHP, Erlang), wrap the `#! nix-shell` lines in a block comment; Nix still finds them, the real interpreter ignores them.

Relative paths in the shebang (Nix 2.24+). A `#! nix-shell --expr 'import ./shell.nix'` or `#! nix-shell ./deps.nix -i python3` resolves `./` **relative to the script's directory**, not the caller's cwd. That is what makes `~/bin/say-hi` that imports `~/bin/shell.nix` work when you run it from `$HOME`. Opt out with `nix-shell-shebang-arguments-relative-to-script = false` in `nix.conf` if you must keep the old behaviour; that option is scheduled to go away.

Case 3 — Modern form: `#!/usr/bin/env nix` + `nix shell` (not run here)

Nix 2.19 made the `nix` CLI itself a shebang interpreter. Subsequent `#! nix …` lines are concatenated into one `nix` invocation. The usual shape puts `nix shell` installables on those lines and ends with `--command real-interpreter`:

```
# hello-cowsay.sh - flake installables, no -I tarball
#!/usr/bin/env nix
#! nix shell nixpkgs#bash nixpkgs#hello nixpkgs#cowsay --command bash
hello | cowsay
```

```
# pinned-with-override.sh - pin the nixpkgs input without a local flake
#!/usr/bin/env nix
#! nix shell --override-input nixpkgs github:NixOS/nixpkgs/26f4107e47e3ad0ffd0db220469ba83b6
#! nix      nixpkgs#bash nixpkgs#jq --command bash
jq --version
```

```
# not run here - Nix is not installed on this box
# chmod +x hello-cowsay.sh pinned-with-override.sh
# ./hello-cowsay.sh
# ./pinned-with-override.sh
```

Notes that bite in practice:

- **Double backticks** (a pair of backtick characters, as in the Nix manual's shebang section) quote a multi-line or special Nix expression inside a `#! nix` line. Single or double quotes are what the classic `nix-shell` shebang uses for `-p` expressions; do not mix the two styles up.
- **`--command bash` replaces `-i bash`.** There is no `-i` in the new CLI.
- **`nixpkgs#foo` without a pin** follows the flake registry. On a machine whose registry tracks `nixpkgs-unstable`, tomorrow's `jq` may not be today's. Prefer a project flake that the script references (`#! nix shell path:.#jq --command bash` next to a `flake.lock`), or `--override-input` / a locked `github:NixOS/nixpkgs/<rev>` as above.
- The new form needs a Nix with the `nix-command` (and for flakes, `flakes`) experimental feature, which Fedora 44's packaged Nix and current Determinate installs enable by default. Classic `nix-shell` remains the lowest-common-denominator for mixed fleets.

Case 4 — Pinning so "reproducible" means something

Pin style	Example	Pros	Cons
Floating channel / registry	`-p jq` or `nixpkgs#jq` alone	Short	Changes under you
nixpkgs tarball URL	`-I nixpkgs=https://github.com/NixOS/nixpkgs/archive/<rev>.tar.gz`	One line, works with classic `nix-shell`	URL length; first hit downloads the tar
Flake + `flake.lock` next to the script	`#! nix shell path:.#tools --command bash`	Best UX, updates via `nix flake update`	Needs flakes; the lock must be committed
`--override-input nixpkgs github:…/<rev>`	See Case 3	No local flake required	Verbose; easy to leave floating by mistake
`fetchTarball` inside a `deps.nix`	`#! nix-shell deps.nix -i python3` with a fixed `sha256`	Classic, auditable	Two files to ship

The commit used in the examples above, `26f4107e47e3ad0ffd0db220469ba83b6f6d96e8`, was the tip of `nixos-26.05` on 2026-10-15. For a script you ship, pick a revision on purpose and record why (a release, a CVE fix, a known-good CI pin). Do not paste "whatever `nix flake metadata` said this morning" into a shebang and call it pinned.

Case 5 — When *not* to use a shebang

Need	Prefer
An interactive project toolchain (compilers, linters, formatters)	A flake `devShell` + `nix develop` / direnv (the previous chapter)
A command other people run as `nix run github:org/repo#cmd`	A flake `apps` / `packages` output
Something installed on a system or in a profile	A real derivation (`writeShellApplication`, `stdenv.mkDerivation`) so GC and signatures apply
A dependency graph shared by many scripts	One flake, many apps; not twenty shebangs each pinning nixpkgs
Running on hosts without Nix	A container image or a static binary; a shebang cannot help

A shebang shines for **glue**: a one-file report, a webhook receiver, a bootstrap that must not assume the host's `jq`. Once the file grows a test suite or a second entry point, promote it to a flake app.

Promoting a script to a Flake app (`pkgs.writers.writePython3Bin`)

On non-NixOS Linux distributions (Fedora, Ubuntu, Debian), scripts that rely on `pip` or ambient package managers frequently break: 1. Distro Python updates break virtualenv symlinks or `pipx` environments. 2. Scripts requiring external system utilities (e.g. `aplay` from `alsa-utils`, `ffmpeg`, or `sox` for media scripts) fail whenever the host lacks those system binaries—a failure mode Python package managers cannot address.

While a `#! nix-shell -i python3` shebang solves this for a single local file, promoting the script to a Flake app using `pkgs.writers.writePython3Bin` packages it into a hermetic, multi-architecture command that anyone with Nix can execute via `nix run`:

```
# flake.nix (co-located with readarticle.py)
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs, ... }:
    let
      forAllSystems = f: nixpkgs.lib.genAttrs [ "x86_64-linux" "aarch64-linux" ] (system:
        f nixpkgs.legacyPackages.${system}
      );
    in {
      packages = forAllSystems (pkgs: {
        default = pkgs.writers.writePython3Bin "readarticle" {
          # System tools placed on PATH
          makeWrapperArgs = [
            "--prefix" "PATH" ":" (pkgs.lib.makeBinPath [ pkgs.alsa-utils ])
```

```
          ];
          # Python modules placed on PYTHONPATH
          libraries = with pkgs.python3Packages; [
            requests
            trafilatura
          ];
        } (builtins.readFile ./readarticle.py);
      });
    };
}
```

```
# Run locally or from git on any Linux machine with Nix
nix run . -- https://example.com/article
```

Inside `readarticle.py`, the Python code invokes `subprocess.run(["aplay", ...])` without caring where `aplay` is installed. Nix wraps the interpreter, injects dependencies onto `PATH` and `PYTHONPATH`, and even runs a build-time lint check (`pyflakes`) that catches syntax errors before runtime.

The trap

Trap: "options on the first line." Case 1. `env` looks up one filename. Use a second `#!` line, or `env -S`.

Trap: "floating `<nixpkgs>` is fine for a demo." It is fine for a demo you throw away. It is how last quarter's "reproducible" script started needing a different Python. Pin.

Trap: "`--pure` and then call `git`/`ssh`/`docker`." They are gone from `PATH`. Either add them to `-p`, or drop `--pure` and accept host tooling (and document that). `--keep VAR` retains named ambient variables under `--pure` when you need a token or a socket.

Trap: "the shebang is a sandbox." It is an environment. Network, home directory, and (without a user namespace) the rest of the machine are still there. `--pure` only cleans the environment block.

Trap: "`writeShellApplication` in nixpkgs already does this." For packages, yes: it rewrites the shebang to a store bash and puts deps on `PATH`. That is the right end state for something you *install*. A `#! nix-shell` script is the right form for something you *copy as a file* to a machine that has Nix but not your package set.

Trap: "long `-I` on the first line." Case 1: silent truncation at 256 bytes. Put pins on later lines.

The boring rule

Put `#!/usr/bin/env nix-shell` (or `nix`) on line one, every option on the following `#!` lines, and a **pinned** nixpkgs (tarball revision, flake lock, or `--override-input`). Use `--pure` unless you have

a named reason not to. When the script grows up, promote it to a flake app.

Try this

1. On a machine with Nix 2.35+, save Case 2's script, `chmod +x` it, and run it. Then change the pinned revision to another `nixos-26.05` commit and confirm `jq --version` (add a `jq --version` line) is unchanged or moves, depending on what that commit changed.
2. Convert Case 2 to Case 3 with `nixpkgs#…` installables and `--override-input`. Which form's first run downloaded more?
3. Drop `--pure` and see whether a host `curl` that is *not* the nixpkgs one still runs. Then put `--pure` back and add `cacert`.
4. Write a Python script that imports `requests` with `#! nix-shell -i python3 -p python3Packages.requests` and a pin. Run it from a different directory than the one you wrote it in.
5. On Nix 2.24, put `deps.nix` next to a script that does `#! nix-shell ./deps.nix -i bash` and run the script from `$HOME`. Confirm it still finds `./deps.nix`.

Next

- Back to Daily CLI, flakes, and devShells
- Home Manager standalone

Verification

- **Run here (no Nix):** Case 1. Debian 13, Linux **6.12.94+**, GNU coreutils **env 9.7**, `BINPRM_BUF_SIZE` 256 from `/usr/include/linux/binfmts.h`. The stand-in `nix-shell`, the `env` error with spaces, `env -S` splitting, and the truncation probe were all executed; outputs above are pasted from those runs.
- **Not run here:** Cases 2–3 and every real `nix-shell` / `nix shell` invocation. `/nix` does not exist on this box, and installing Nix is out of bounds for this run. Those listings follow the Nix 2.35 `nix-shell` manual, the `nix` shebang documentation, the 2.19 / 2.24 release notes, and the nix.dev *Reproducible interpreted scripts* tutorial.
- Versions checked 2026-10-18: Nix **2.35.3** (upstream tag), NixOS **26.05** current on nixos.org, `nixos-26.05` tip `26f4107e47e…` dated 2026-10-15.
- No packages installed, no root, no `sudo`, no AI/coding CLIs.

Sources

- Nix 2.35 manual — `nix-shell` (including *Use as a `#!`-interpreter*)
- Nix 2.35 manual — `nix` shebang interpreter and `nix shell`
- Nix release notes — 2.19 (shebang via `nix`), 2.24 (`nix-shell` shebang paths relative to the script)

- nix.dev tutorial — Reproducible interpreted scripts
- Linux `BINPRM_BUF_SIZE` — `include/uapi/linux/binfmts.h` (256); GNU coreutils `env --help` (`-S` / `--split-str`)
- nixos.org download page (Nix 2.35.x, NixOS 26.05) and the `nixos-26.05` branch tip used as the example pin

Inspiration

X post by @wereness (2026-06-20): https://x.com/wereness/status/2068459592216920440 — topic signal only ("force throwaway scripts onto `#! nix-shell` so they never crash on missing tools"). The chapter cites only the primary docs above.

Home Manager on Fedora (standalone)

Home Manager on Fedora (standalone)

Goal: Manage your **user** environment declaratively with Home Manager **without NixOS**, so later you only change *how* HM is attached (standalone → NixOS module), not the idea of declarative dots.

Why HM on Fedora

Imperative home	Home Manager
Edit `~/.bashrc` in place	Generate from Nix; generations
"Which machine has my aliases?"	Same flake → same home
Drift	`home-manager switch` is the write path

You still use **Fedora** for the desktop session; HM owns *your* shell, git, editor configs, user packages, and optional user systemd units.

Standalone bootstrap (flake-based)

There are multiple install methods; prefer a **flake** so locks exist.

1. Minimal home flake skeleton

```
mkdir -p ~/src/home-config
cd ~/src/home-config
git init
```

Example `flake.nix`:

```
{
  description = "Standalone Home Manager on Fedora";

  inputs = {
```

```
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    home-manager.url = "github:nix-community/home-manager/release-26.05";
    home-manager.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, home-manager }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in {
      homeConfigurations."alice" = home-manager.lib.homeManagerConfiguration {
        inherit pkgs;
        modules = [ ./home.nix ];
      };
    };
}
```

Replace `alice` with your username (`whoami`). Match **home-manager release** to nixpkgs branch (26.05 with 26.05).

2. `home.nix`

```
{ pkgs, ... }: {
  home.username = "alice";
  home.homeDirectory = "/home/alice";
  home.stateVersion = "26.05"; # read HM release notes when bumping

  home.packages = with pkgs; [
    ripgrep
    fd
    bat
  ];

  programs.home-manager.enable = true;

  programs.git = {
    enable = true;
    userName = "Alice Example";
    userEmail = "alice@example.com";
  };

  programs.bash = {
    enable = true;
    enableCompletion = true;
    shellAliases = {
      ll = "ls -la";
```

```
    };
  };

  # programs.zsh.enable = true;  # if you live in zsh
}
```

3. First switch

```
nix flake lock
# Modern invocation (HM  versions vary slightly - check `home-manager --help`):
nix run home-manager/release-26.05 -- switch --flake .#alice

# After HM is on PATH via profile:
home-manager switch --flake .#$(whoami)
```

If the attribute is `homeConfigurations.alice`, the flake ref is `.#alice`.

4. Verify

```
home-manager generations
rg --version
git config --get user.email
```

What to put in HM vs project flakes

HM (user global)	Project flake
Shell, prompt, aliases	Language SDKs for *that* repo
git, gh, editor user config	Formatters/linters pinned to project
Personal CLI (rg, fd, bat)	Heavy toolchains (JDK, Android SDK)
User systemd timers	CI-only tools

Generations and rollback

```
home-manager generations
# switch to older generation if a bad config lands
home-manager switch --flake .#alice -n   # dry-run when supported
```

This is the **user-level** analogue of NixOS boot generations. Learning it now removes fear later.

Secrets warning

HM can write files into the store. **Never**:

```
home.file.".ssh/id_rsa".text = "-----BEGIN …"; # DO NOT
```

Use live files outside the store, or sops/age patterns (later book parts). Prefer `home.file.*.source` only for non-secret text you accept in the store.

Coexistence with Fedora's bashrc

When `programs.bash.enable = true`, HM manages your bash config. Avoid fighting both:

- Either HM owns `~/.bashrc`
- Or use `programs.bash.initExtra` to append and stop hand-editing

Same for zsh.

Lab

1. Create `~/src/home-config` with flake + `home.nix`.
2. Manage git + three CLI tools via HM.
3. Break an alias on purpose, `switch`, then roll back a generation.
4. Write a note: "On NixOS I will attach this module via `home-manager.users.…`."

Checkpoint

- ☐ `home-manager switch --flake …` succeeds on Fedora
- ☐ Generations list is non-empty
- ☐ No secrets in the store from HM
- ☐ You can explain standalone vs NixOS module HM

Further depth

- Home Manager (book part)
- HM files and XDG
- Bridge to NixOS

dnf and Nix coexistence

dnf and Nix coexistence

Goal: Run a calm dual-stack system: Fedora's **dnf** (or rpm-ostree) owns the OS; Nix owns portable toolchains—without PATH wars, duplicate daemons, or split-brain config.

PATH reality

Order matters:

```
echo "$PATH" | tr ':' '\n' | head -20
type -a rg
type -a git
```

Typical conflict:

Tool	Fedora provides	Nix provides	Risk
`git`	`/usr/bin/git`	profile/HM	Different configs/behavior
`python3`	system Python	devShell Python	Shebangs
`podman`	OS integration	rare via nix	Prefer OS for containers on Fedora

Policy examples:

```
# Conservative dual-stack
OS: kernel, desktop, browsers (flatpak/dnf), podman, firmware
Nix: language toolchains, specialized CLIs, project shells
HM: dotfiles + personal CLIs that don't need OS integration
```

Decision matrix (printable)

Question	If yes →
Needs kernel module / udev / SELinux policy?	**dnf**
Needed identically on CI and laptop?	**Nix flake**
GUI app with portals/Flatpak better?	Flatpak or dnf
Only you need it in `$HOME`?	HM or profile
System service on this Fedora host long-term?	dnf/systemd for now; NixOS modules later
Experiment for a weekend?	`nix shell`

Avoid dual service managers for the same app

Bad: nginx from dnf **and** experiments with Nix-built nginx on same ports.

```
ss -lntp | rg ':80|:443' || true
systemctl status nginx 2>/dev/null || true
```

Pick **one** owner per service on a given host.

Language version managers vs Nix

Tool	Overlap with Nix
`nvm` / `fnm`	devShell `nodejs`
`pyenv`	devShell `python3`
`sdkman`	devShell JDK
`rustup`	Often **keep rustup** *or* use nixpkgs rust—pick one story per project
`goenv`	devShell `go`

Migration pattern per repo:

1. Add flake `devShell` with tools
2. Document in README: `direnv allow`
3. Remove project-local `.nvmrc` only when team agrees

Graphics / Electron / browsers

Nix can build browsers; on Fedora daily drivers many people still use **Fedora/Flatpak** browsers for VA-API, Widevine, and portal integration. That's fine. Use Nix where reproducibility beats integration.

Disk layout awareness

```
df -h / /nix /home
```

If `/` is small (Atomic, small root), ensure `/nix` lives on a large filesystem (bind mount from `/var` or secondary disk). dnf and Nix both cache—budget both.

Updates: two clocks

Stream	Command
Fedora	`sudo dnf upgrade`
Nix profile	`nix profile upgrade` / rebuild from flake
HM	`home-manager switch --flake …`
Flake inputs	`nix flake update` + PR/review lock

Stagger upgrades when debugging ("did dnf or flake break me?").

Lab

1. Produce a table of 15 tools on your PATH and label each **dnf / flatpak / nix / other**.
2. Move two development tools from global installers into a project flake.
3. Ensure `type -a` shows the intended binary first.

Checkpoint

- ☐ Written ownership policy for OS vs Nix
- ☐ No accidental duplicate core services
- ☐ PATH inspected and understood
- ☐ Update rituals documented

Further depth

- SELinux, Atomic, ops
- Packages on NixOS (system layer later)

SELinux, Atomic, and day-2 ops

SELinux, Atomic, and day-2 ops

Goal: Keep Fedora's **security and immutability** model while running Nix—without disabling SELinux—and operate the store like an adult (disk, daemon, upgrades).

SELinux: do not "fix" Nix by disabling it

Fedora's packaged Nix is built to work under **enforcing** SELinux. Prefer:

```
getenforce
sudo ausearch -m avc -ts recent 2>/dev/null | tail -20
```

If you see denials after a non-Fedora install method, **switch to the Fedora package** before opening policy holes.

Do	Don't
`dnf install nix`	`setenforce 0` as permanent fix
Read `/usr/share/doc/nix/`	Copy random `chcon -R` from old blog posts without understanding
File bugs against Fedora nix package when policy is wrong	Disable SELinux system-wide

---

Atomic desktops and bootc

Problem

Image-based systems often make `/usr` immutable and restrict creating `/nix` on root.

Pattern A — bind mount

```
sudo mkdir -p /var/nix
# ensure /nix exists as mount point
sudo mkdir -p /nix
```

systemd `.mount` unit bind `/var/nix` → `/nix` (see Install chapter). Then install Nix packages and enable daemon **if** supported on that image.

Pattern B — rootless / pure user Nix

Fedora packaging notes mention **pure rootless** mode useful on Atomic. Follow current `README.fedora.md`—flags and subpackages change.

Pattern C — Nix in a toolbox/distrobox

Some Atomic users keep the host minimal and run a **Fedora toolbox** with classic `/nix` for heavy dev. Tradeoff: another layer; still teaches Nix well.

```
# illustrative
toolbox create -i fedora-toolbox:44 nixdev
toolbox enter nixdev
# then dnf install nix inside toolbox if appropriate
```

Day-2 operations checklist

Daemon health

```
systemctl status nix-daemon --no-pager
journalctl -u nix-daemon -b --no-pager | tail -40
```

Disk

```
df -h /nix
nix store gc
# optional deep clean of old generations
# nix-collect-garbage -d
```

Trust and config

```
nix show-config | rg -n 'substituter|trusted|sandbox|experimental'
```

Upgrade Nix itself (Fedora)

```
sudo dnf upgrade nix
sudo systemctl restart nix-daemon
nix --version
```

Flake-locked **nixpkgs** upgrades are separate from upgrading the **Nix binary**.

Networking / proxies / corporate TLS

If substitutes fail:

```
curl -I https://cache.nixos.org
# corporate MITM: may need custom CA trust for Nix - advanced; document carefully
```

Backup thoughts

Path	Back up?
`flake.nix` / `flake.lock` / HM repo	**Yes** — this is the real config
`/nix/store`	Optional; can rebuild from locks (slow)
Profile generation list	Nice; not required if flakes exist

On NixOS you will also back up **host flake** + secrets—not the entire store.

Performance tips (laptop)

- Keep direnv projects few and GC occasionally
- Use `nix-output-monitor` for readability, not speed
- Prefer substitutes; local compiles of browsers hurt battery
- `nix settings` / `max-jobs` and `cores` can be tuned in `nix.conf`

```
# example user conf - adjust to CPU
# max-jobs = auto
# cores = 0
```

Daemon may need matching system conf for multi-user.

Lab

1. With SELinux enforcing, run `nix run nixpkgs#hello` and confirm no new AVCs.
2. Measure `df -h /nix` before/after `nix develop` on a heavy shell; run `nix store gc`.
3. If on Atomic, document your `/nix` strategy in the lab charter.

Checkpoint

☐ SELinux remains enforcing

☐ Daemon healthy after reboot

☐ Disk budget monitored

☐ Atomic path documented if applicable

Further depth

- GC hygiene (fleet part)
- Troubleshooting

Bridge to NixOS

Bridge to NixOS

Goal: Convert Fedora+Nix habits into a first **NixOS** host with minimal conceptual shock—reuse flakes, Home Manager knowledge, and store literacy.

What stays the same

You already do on Fedora	On NixOS
`/nix/store`, closures, GC roots	Identical store model
`nix build` / `run` / `develop`	Identical
`flake.lock` discipline	Identical (+ host config in same monorepo often)
Home Manager modules for git/shell	Same options; different wiring
direnv + project flakes	Identical

What is new

New concept	Meaning
`nixosConfigurations`	Flake output describing whole machines
Module system	`services.openssh.enable = true;` merges options
`nixos-rebuild switch --flake`	Activate system generation
Bootloader generations	Roll back from GRUB/systemd-boot
`environment.systemPackages`	System-wide packages (not just profile)
Systemd **system** units via Nix	Beyond user HM services
Disko / fileSystems	Disk layout as code

```
Fedora era                          NixOS era
```

```
dnf upgrade                          nixos-rebuild (flake pin)
/etc hand edits                      modules in git
~/.config via HM standalone          HM standalone OR nixos module
nix profile thin toolkit             systemPackages + HM
```

Recommended transition shapes

A — Keep Fedora laptop; NixOS server/lab (common)

- Laptop: this whole **Nix on Linux** part forever
- Lab VM/metal: NixOS host part
- Share one git repo:

```
flake.nix
    devShells
    homeConfigurations.alice      # Fedora laptop
    nixosConfigurations.labbox    # NixOS
```

B — Dual boot / spare disk NixOS

- Practice rebuild/rollback risk-free
- Port HM modules first; add system modules gradually

C — Full cutover

- Only after: flake host builds in VM, secrets story exists, backup restore tested

---

Porting checklist (HM → NixOS)

1. **Keep** `programs.*` and `home.packages` modules almost as-is.
2. **Move** OS concerns out of HM: sound, printing, kernel, networking—use NixOS modules.
3. **Wire** HM as NixOS module:

```
# sketch - details in Home layout / host parts
{
  inputs.home-manager.nixosModules.home-manager
  {
    home-manager.useGlobalPkgs = true;
    home-manager.useUserPackages = true;
    home-manager.users.alice = import ./home.nix;
  }
}
```

4. **Replace** `nix profile` tools with `environment.systemPackages` or HM packages deliberately.

5. **Retire** dnf habits for software you now declare—keep mental note of firmware/kernel still "special."

First NixOS learning loop (preview)

Do this on a **VM** (Lab 0), not your only laptop:

1. Install NixOS (graphical or minimal) — Install

2. Put host in a flake — Flake host

3. `nixos-rebuild switch --flake .#hostname`

4. Break sshd/firewall intentionally; roll back a generation — Rebuild and generations

5. Import your HM config

Success metric: **wipe VM disk, rebuild from flake + backup secrets, services return.**

Dual-stack monorepo sketch

```
{
  description = "Machines and homes";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    home-manager.url = "github:nix-community/home-manager/release-26.05";
```

```
    home-manager.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, home-manager, ... }: {
    homeConfigurations.alice = /* Fedora path */;
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [ ./hosts/lab/configuration.nix ];
    };
    devShells.x86_64-linux.default = /* shared */;
  };
}
```

One mental model: **outputs** are different products from the same factory.

Skills you should already have (gate)

Before NixOS host part, from this bridge path:

- ☐ Nix installed via Fedora (or documented equivalent)
- ☐ Flakes + lock for a real project
- ☐ direnv workflow
- ☐ HM standalone generation/rollback
- ☐ Store GC literacy
- ☐ Written dnf vs Nix policy

Lab

1. Draft `hosts/lab/configuration.nix` **on paper or in git** with only: hostname, user, ssh, firewall—**without** installing yet.
2. List HM modules you will import unchanged.
3. Schedule a VM install using Lab 0 chapters.

Checkpoint

- ☐ You can list what changes and what does not when moving to NixOS
- ☐ You have a transition shape (A/B/C) written down
- ☐ You are ready for part `03-nixos-host/` without relearning store/flakes

---

Further depth

- NixOS host index
- Flake layout
- Home Manager
- Capstone rebuild proof later: `09-capstone/`

NixOS Host

NixOS host — install, configure, modules

NixOS host — install, configure, modules

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

What this part is for

NixOS host turns the Concepts toolkit into **machine ownership**: install a disposable NixOS system, manage it with a flake (`nixosConfigurations`), rebuild and roll back safely, then configure users, network, packages, systemd, modules, boot, and filesystems.

You leave this part with a lab host whose **source of truth is a git flake**, not ad-hoc edits under `/etc/nixos` alone.

Previous part	This part	Next part
Concepts — language, store, flakes	Install + configure one host	Home Manager, flake layout, lock discipline

---

Baseline (this book)

Piece	Baseline
NixOS / nixpkgs	**26.05** "Yarara"
Nix	**2.35+** with `nix-command` + `flakes` (stock 26.05 ships 2.34.x; pin `nix.package = pkgs.nixVersions.latest`)
Host management	`sudo nixos-rebuild switch --flake .#<host>`
Install target	**Disposable VM preferred** (not daily-driver only disk)

Learning outcomes

By the end of this part you should be able to:

1. Install NixOS 26.05 (UEFI-oriented) and document hardware/boot layout
2. Define `nixosConfigurations.<name>` and rebuild from a flake repo
3. Choose `switch` / `test` / `boot` / `build` and **rollback** generations
4. Declare users, SSH keys, and safe remote access habits
5. Configure networking and an explicit firewall that still allows admin access
6. Prefer `environment.systemPackages` vs `programs.*` deliberately
7. Ship custom **systemd** services and timers via Nix
8. Consume modules via options (`mkIf` / `mkDefault` literacy)
9. Write a small **custom module** with options, `mkIf`, and assertions
10. Document boot loader, kernel, and firmware choices
11. Inventory filesystems and classify **store vs state** (impermanence *idea*)
12. Declare every locale the host loads, and diagnose a missing-locale crash
13. Explain what `system.stateVersion` changes on a host, and move it only after diffing and pinning

---

Prerequisites

Required	Strongly recommended
Concepts part (or equivalent fluency)	Working flake + modern CLI habits
Hypervisor or spare machine	VM snapshots before boot/network experiments
Git for the host config repo	Console access for lockout recovery drills

If you skipped Concepts, at least complete: store/closures, functions, flakes + devShell, modern CLI.

Chapter map

Chapter	Title	Focus
Installing NixOS	ISO, UEFI, partition, first boot, snapshots	
Flake-defined hosts	`nixosConfigurations`, repo layout, first switch	
Rebuild and generations	Modes, profile gens, rollback card	
Users and access	wheel, SSH keys, password hygiene	
Networking and firewall	networkd/NM, ports, DNS, safety patterns	
Packages and programs	systemPackages vs programs.*, closure discipline	
systemd via Nix	units, timers, absolute paths, inspect	
Modules as consumer	options, merge, mkIf/mkDefault, reading source	
Custom modules	options + config, assertions, namespaces	
Boot and hardware	loader, kernel, firmware, recovery	
Filesystems and state	mounts, persist lists, store data	
Activation reload/restart deprecation	leave activation lists; unit restartTriggers / reloadIfChanged	
Clevis LUKS askpass	Header tokens, Tang/TPM2, systemd Stage 1 unlock	
Specialisations	boot-menu alternate configs; activate vs reboot; vs containers	
Modular services	`system.services.<name>`, portable `process.argv`/`configData`, sub-services, package-exported modules, traps	
systemd in Stage 1	26.05 initrd default; porting `*Commands` hooks; `/dev/root` and LUKS traps	

Chapter	Title	Focus
Locales and the locale archive	`i18n.extraLocales`/`extraLocaleSettings`, `LOCALE_ARCHIVE`, missing-locale crash, site-local locales without an overlay	
system.stateVersion	what it selects (defaults only), diffing a bump, pin-then-bump, the empty-PostgreSQL trap, per-module `stateVersion`	

Suggested learning paths

Path	Order	Notes
Linear (recommended)	01 → 18	Each chapter assumes the flake host exists
Already installed	Start at 02; skim 01 for inventory	Still document hardware
Ops recovery focus	03, 04, 05, 10, 11	After a minimal flake host works
Module author track	08 → 09 after 02–07	Needs a living host config

---

Architecture sketch (part-level)

```
git repo: ~/nixos-config
  flake.nix  →  nixosConfigurations.lab = nixosSystem { modules = [ … ] }
  hosts/lab/configuration.nix
  hosts/lab/hardware-configuration.nix
  modules/*.nix

sudo nixos-rebuild switch --flake .#lab


  evaluate modules → system derivation → realize → activate


  /nix/var/nix/profiles/system  (generations + boot entries)
```

Safety rules for this part

1. Prefer a **VM** with snapshots before firewall, boot, or SSH lockout drills
2. Keep **console** access until key-based SSH is proven
3. Use `nixos-rebuild **test**` for risky network/firewall changes, then `switch`
4. Never put private keys or live passwords in the flake
5. Treat `hardware-configuration.nix` UUIDs as machine-specific truth
6. Remember: **rollback restores system closure, not `/var` databases**

Lab workspace convention

On the lab host (or mirrored on your workstation):

```
mkdir -p ~/nixos-config/{hosts/lab,modules,docs}
cd ~/nixos-config
git init
```

Rebuild command of record:

```
sudo nixos-rebuild switch --flake .#lab
```

Document hostname, flake attr, and hypervisor network mode in `README.md`.

Part gate (before Home / flake layout)

- ☐ Disposable NixOS **26.05** lab boots
- ☐ Flake `nixosConfigurations.<host>` is source of truth
- ☐ Successful `switch --flake` and a documented **rollback** path
- ☐ SSH key admin user; no private keys in git
- ☐ Firewall enabled with explicit SSH allow

☐ Daily tools declared (not only `nix profile`)

☐ At least one custom systemd unit and one custom module

☐ Boot + filesystem docs exist; store vs state explained

Further depth (this book only)

After this part…	Go to
Home Manager, direnv, flake-parts	Part **Home and flake layout**
sops, hardening, services, TLS	Part **Services and security**
Disko, impermanence, deploy, tests	Part **Ops and fleet**
Module system internals	Part **Internals**
Capstone multi-service host	Part **Capstone / Projects**

How to use this part as a reference

- Jump via chapter titles when debugging (“firewall”, “generations”, “custom modules”)
- Keep a **rollback card** (`docs/rollback.md`) updated as you learn boot and SSH
- Prefer options on search.nixos.org with **26.05** selected
- Do not link or depend on any external day-numbered curriculum; this book is self-contained

When later parts say “on your lab host,” they mean the machine and flake built here.

Installing NixOS

Installing NixOS

Goal: Install a **disposable** NixOS **26.05 "Yarara"** lab system (VM strongly preferred), boot it, capture hardware config notes, and prepare for flake-based management in the next chapter.

> **!** Important
>
> Do **not** experiment on your only daily-driver disk. Use a VM, spare machine, or spare disk. Reinstall without drama is a feature.

Why this chapter matters

Generations, rollback, modules, and firewall only become real on a host. Concepts was the package manager + language. This part is **owning a machine** as code—starting with a clean install.

Ops motivation: every production NixOS story begins with a first generation that boots. Install literacy (UEFI, ESP, generate-config, credentials) is how you recover when experiments go wrong.

Theory 1 — What you are installing

Piece	Role
NixOS	Linux distribution built from Nix modules
nixpkgs 26.05 "Yarara"	Package + module set baseline for this book
Installer image	Live environment to partition + install
First generation	Initial system closure after install

After install you have a bootable system. **Flake ownership** is the flake hosts chapter—this chapter can use installer defaults, but mentally prepare to migrate immediately.

Release identity

```
# after install
nixos-version
cat /etc/os-release
```

Confirm **26.05**, not accidental unstable, when following this book's baseline.

Theory 2 — Disposable VM emphasis

Host choice	Recommendation
VM (QEMU/KVM, UTM, VirtualBox, VMware, …)	**Preferred**
Spare laptop/NUC	Good
Dual-boot daily driver	Risky for learning
Cloud VM	Fine if you accept networking/SSH from the start

VM resources (practical)

Resource	Starting point
RAM	4–8 GB
Disk	40–64 GB+
CPU	2+ cores
Firmware	**UEFI** if available
Snapshot	Take one after first successful boot

Snapshots turn "I broke GRUB" into "restore 30 seconds."

Networking modes (hypervisor)

Mode	Use
NAT	Simple outbound; port-forward for SSH
Bridged	LAN IP like a physical host
Host-only	Isolated lab segment

Record the mode in notes—the networking chapter needs it.

Theory 3 — UEFI vs legacy BIOS

Mode	Common loader on NixOS
UEFI	**systemd-boot** (often default) or GRUB EFI
BIOS/CSM	GRUB

Prefer UEFI for new labs. Note which you used—the boot and hardware chapter returns to boot loaders.

ESP requirements

Need	Typical
ESP size	512 MB–1 GB FAT32
Mount during install	`/mnt/boot`
Variables	`canTouchEfiVariables` often true on real UEFI

Theory 4 — Disk layout options

Manual (classic installer path)

1. Partition (ESP + root, optional swap)
2. Format
3. Mount at `/mnt`, ESP at `/mnt/boot`
4. `nixos-generate-config --root /mnt`
5. Edit `configuration.nix`
6. `nixos-install`

Disko (intro only)

Disko declares disks as code. Full Disko redesign is in the **Ops and fleet** part. In this chapter:

- Know Disko exists
- Optionally follow a **simple** Disko example if you already know it

- Otherwise **manual once** is fine and educational

Do not block install on learning Disko deeply here.

Suggested simple layout (UEFI)

```
ESP  (vfat)  512MB-1GB   → /boot
root (ext4 or btrfs)     → /
swap (optional)
```

Why not exotic FS on first install?

ZFS/btrfs fancy layouts are powerful and easy to misconfigure. Prefer **boring** for the first bootable lab; redesign later with Disko when you understand state.

Theory 5 — `nixos-generate-config` outputs

File	Role
`hardware-configuration.nix`	Detected devices, fileSystems, initrd modules
`configuration.nix`	High-level system config (you own this)

Rules of thumb

- Treat `hardware-configuration.nix` as machine-specific; rarely hand-merge casually
- Put policy (users, services, packages) in `configuration.nix` / modules
- Next chapter both get imported from a flake

What generate-config is good at

Captures well	Does not invent for you
Disk UUIDs	Hostname policy
Initrd modules for disks	Firewall policy
Basic boot bits	SSH keys

Theory 6 — Installer networking and time

Install needs:

- Network for substitutes (unless offline media strategy)
- Correct clock helps TLS

```
# live environment examples (names vary)
ip a
ping -c 2 1.1.1.1
timedatectl 2>/dev/null || date
```

Broken DNS in the live environment looks like "NixOS install is broken."

Theory 7 — First boot mental model

After `nixos-install` and reboot:

```
bootloader → kernel+initrd → stage-1/2 → systemd → multi-user
system profile: /nix/var/nix/profiles/system
generation N is current
```

You can log in as root or the user you created. **Prefer SSH key setup ASAP** (users chapter formalizes); avoid building habits around password-only remote root.

Activation vs "files on disk"

The installed system is a **closure** activated into place—not a traditional unpack of a rootfs tarball only. Generations will make that concrete in the rebuild chapter.

Theory 8 — Credentials hygiene on a lab

Practice	Why
Unique lab password	Compromise isolation
Note in password manager	Labs still get attacked if exposed
Prefer host-only networking until ready	Reduce drive-by SSH noise
Snapshot before experiments	Fast recovery
Temporary `initialPassword` only	Rotate; prefer keys

Practice	Why

Theory 9 — `system.stateVersion`

```
system.stateVersion = "26.05";
```

This is **not** "current release tracking." It tells NixOS which stateful defaults to preserve for compatibility. Set it to the release you **installed**; read release notes before ever changing it.

Theory 10 — Minimal install config knobs

Focus during first install:

Knob	Why
Boot loader enable	Must boot
Hostname	Identity
User + wheel	Admin path
Optional SSH	Remote lab convenience
Timezone	Sanity
Experimental features	Smoother flake migration

Defer fancy services until the flake is source of truth.

Worked examples bank

Example A — Download image

Use official docs for current ISO:

- https://nixos.org/download/

Pick **26.05** graphical or minimal ISO matching your arch (`x86_64`, `aarch64`).

Verify checksums when provided.

Example B — Partition sketch (sfdisk/parted-level ideas)

```
/dev/vda1  EFI System  FAT32  /boot
/dev/vda2  Linux root  ext4   /
```

Device names: `vda` (virtio), `sda`, `nvme0n1`—use what the live system sees (`lsblk`).

Example C — Generate + install skeleton

```
# after mounts ready
sudo nixos-generate-config --root /mnt
sudo nano /mnt/etc/nixos/configuration.nix
sudo nixos-install
# set root password when prompted
```

Example D — Minimal configuration knobs to set *during* install

```
{
  imports = [ ./hardware-configuration.nix ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  networking.hostName = "lab";
  time.timeZone = "UTC"; # set yours

  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    # initialPassword only for offline lab convenience - rotate later
    initialPassword = "changeme";
  };

  services.openssh.enable = true; # if you want SSH into VM
  nix.settings.experimental-features = [ "nix-command" "flakes" ];

  system.stateVersion = "26.05"; # match release; do not casually change later
}
```

Exact options must match your installer generation—adapt from generated file rather than blind paste.

Example E — Mount checklist before generate-config

```
lsblk -f
findmnt /mnt
findmnt /mnt/boot
# ESP must be mounted or boot will fail later
```

Lab 1 — Plan and snapshot strategy

Write in `~/lab/nixos-book/host/install/NOTES.md` (on the **workstation**):

1. Hypervisor / hardware choice
2. UEFI or BIOS
3. Disk size
4. Hostname
5. Username
6. Networking mode (NAT/bridge)
7. Snapshot plan

Lab 2 — Install NixOS 26.05

1. Boot installer ISO
2. Partition + format + mount
3. `nixos-generate-config --root /mnt`
4. Edit config: hostname, user, boot loader, optional SSH
5. `nixos-install`
6. Reboot into installed system
7. Take a **VM snapshot** labeled `nixos-26.05-fresh-install`

If install fails, journal the error class (bootloader, mount, network, disk).

Lab 3 — Post-install inventory

On the new system:

```
nixos-version
uname -a
lsblk -f
cat /etc/os-release
ls /etc/nixos/
```

Copy off-box (screenshot/scp) or retype into host `NOTES.md`:

- `nixos-version` output
- Disk layout
- Boot loader in use
- IP address if any

Lab 4 — Preserve hardware config

```
# on NixOS lab
sudo cp -a /etc/nixos /root/nixos-etc-backup-$(date +%F)
```

On workstation notes, record that the flake hosts chapter will create a **flake** and import `hardware-configuration.nix`.

Optional: already enable flakes in system config for a smoother next chapter:

```
nix.settings.experimental-features = [ "nix-command" "flakes" ];
```

Then `sudo nixos-rebuild switch` once if you edit on the running system.

Lab 5 — Break-glass familiarity

Without destroying the snapshot casually:

1. Locate boot menu / how to pick previous generation (may only have gen 1)
2. Confirm you can log in on console
3. If SSH enabled, log in from host once

Exercises

1. Compare minimal vs graphical ISO trade-offs for your lab.
2. Document exact `lsblk` before and after partitioning.
3. Practice mounting ESP incorrectly in a throwaway VM; see the failure mode (snapshot first).
4. List every password/key created; store only in a password manager.
5. Write a 10-step reinstall runbook from your real commands.
6. Note substitute traffic during install (why network mattered).
7. Identify `system.stateVersion` in your config; explain it to a peer.
8. Capture `nixos-generate-config` diffs if you re-run after disk changes (careful).
9. Time a full install; note where you waited on downloads.
10. Sketch Disko-shaped layout for the *same* partitions (no implement required).
11. Verify checksum of the ISO you downloaded.
12. Confirm experimental features enabled before the flake chapter.

Common pitfalls

Symptom	Theory
No bootable device	ESP not mounted at `/mnt/boot` or wrong firmware mode
Install can't reach cache	Network/DNS in live env
Forgot root password	Live ISO recovery / reinstall—why snapshots matter
`hardware-configuration` wrong root UUID	Generated before final mounts
Using unstable ISO by mistake	Check 26.05 label
Daily driver install "for fun"	Restore from backup; prefer VM next time
Both GRUB and systemd-boot enabled	Loader conflict
Tiny disk	Store fills during first real use

Checkpoint

- ☐ Disposable NixOS 26.05 lab boots
- ☐ User + privilege path known (root/wheel)
- ☐ Disk + boot mode documented
- ☐ `/etc/nixos` inspected
- ☐ VM snapshot taken (if VM)
- ☐ Notes stored on workstation
- ☐ Ready to migrate to a flake repo next

Further depth (this book)

Topic	Chapter
Flake as SoT	Flake-defined hosts
Generations	Rebuild modes and generations
Boot details	Boot and hardware
Disks as code	Ops part: Disko
Filesystems / state	Filesystems and state

Flake-defined hosts

Flake-defined hosts

Goal: Manage the lab host with a **flake** (`nixosConfigurations`) from first ownership—`nixos-rebuild switch --flake .#<host>` is the source of truth.

> Tip
>
> **Do not** invest weeks in non-flake `/etc/nixos` only to rewrite. Migrate now while the config is small.

Why this chapter matters

A NixOS system without a flake pin drifts like a channel laptop. Flakes give you:

- Locked nixpkgs (and later inputs)
- Named hosts in one repo
- A rebuild command you can document and automate

Ops motivation: the host becomes a git review unit. Incident response is "what changed in the flake?" not "what did someone type on the box?"

Theory 1 — `nixosConfigurations` output

```
outputs = { self, nixpkgs, ... }: {
  nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
    system = "x86_64-linux";
    modules = [
      ./hosts/lab/configuration.nix
      ./hosts/lab/hardware-configuration.nix
    ];
  };
};
```

Piece	Role
`nixosSystem`	Module system entry for a host
`system`	CPU-OS triple for packages
`modules`	List of NixOS modules (files/functions)
attr name `lab`	Host id in `.#lab`

Rebuild:

```
sudo nixos-rebuild switch --flake /path/to/repo#lab
# or from repo:
sudo nixos-rebuild switch --flake .#lab
```

Evaluation sketch

```
flake inputs (locked nixpkgs)


nixosSystem { modules }


merged config → system derivation (toplevel)


realize + activate → new generation
```

Theory 2 — Where the flake lives

Location	Pros	Cons
`/etc/nixos/flake.nix`	Simple path	Mixed with generated hardware; git harder
`/home/you/nixos-config` git repo	Proper history	Need path on rebuild
Separate git + copy	Flexible	Sync discipline

Recommended for this book: git repo in your home (or shared path), rebuild with explicit flake path. Optionally symlink strategy later.

Sudo and paths

Under `sudo`, `.` may not be where you think. Prefer:

```
sudo nixos-rebuild switch --flake /home/alice/nixos-config#lab
```

or always `cd` to the repo with a documented absolute path.

Theory 3 — Module list composition

```
modules = [
  ./hardware-configuration.nix
  ./configuration.nix
  ./modules/ssh.nix      # later
];
```

Each module is typically:

```
{ config, lib, pkgs, ... }: {
  # options / config
}
```

Or a plain attrset in simple files. Modules-as-consumer and custom-modules chapters deepen this; here: **imports work**.

`imports` inside configuration.nix

```
{
  imports = [
    ./hardware-configuration.nix
    ../../modules/common.nix
  ];
}
```

Either list modules in `flake.nix` or nest via `imports`—pick a convention and stick to it.

Theory 4 — Minimal flake host contents

flake.nix

```
{
  description = "lab flake host";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux"; # match your VM
      modules = [
        ./hosts/lab/configuration.nix
        ./hosts/lab/hardware-configuration.nix
      ];
    };
  };
}
```

hosts/lab/configuration.nix

```
{ config, lib, pkgs, ... }: {
  networking.hostName = "lab";

  nix.settings.experimental-features = [ "nix-command" "flakes" ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;

  time.timeZone = "UTC";

  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
  };

  environment.systemPackages = with pkgs; [ vim git ];

  services.openssh.enable = true;

  system.stateVersion = "26.05";
}
```

hardware-configuration.nix

Copy from the machine's generated file—**do not invent UUIDs**.

Theory 5 — First rebuild flow

```
edit flake modules
    ↓
nixos-rebuild switch --flake .#lab
    ↓
eval modules → system derivation → realize → activate
    ↓
new system generation
```

Outcome	Meaning
Success	New generation is current
Eval error	Fix Nix before build
Build error	Package/build failure
Activation error	System may be partially switched—read output carefully

Safe preview builds

```
nixos-rebuild build --flake .#lab
nix build .#nixosConfigurations.lab.config.system.build.toplevel
```

Build without activating when you want a cheap check.

Theory 6 — `specialArgs` / `extraSpecialArgs` (preview)

```
nixpkgs.lib.nixosSystem {
  specialArgs = { inherit inputs; };
  modules = [ … ];
}
```

Used later to pass flake inputs into modules. Not required for the minimal host.

Theory 7 — Hostname vs flake attribute

Name	Where
`networking.hostName`	OS hostname
`nixosConfigurations.lab`	Flake attribute

They **may** match but are not required to. Document both in README.

Theory 8 — Git discipline for hosts

Commit	Why
`flake.nix` + modules	Policy history
`flake.lock`	Pin truth
`hardware-configuration.nix`	Machine facts (lab OK to commit)

Do **not** commit secrets. Public SSH keys are fine; private keys are not.

Flake purity reminder

Untracked module files may not appear in evaluation—`git add` them.

Theory 9 — Multiple hosts (preview)

```
nixosConfigurations = {
  lab = nixpkgs.lib.nixosSystem { … };
  # laptop = nixpkgs.lib.nixosSystem { … };
};
```

Share modules via `modules/common.nix`. Fleet tooling comes in Ops part; structure starts now.

Theory 10 — Leaving installer `/etc/nixos` behind

Approach	Notes
Ignore `/etc/nixos` after migration	Flake repo is SoT
README warning	Prevent future-you editing wrong tree
Optional pointer file	Comment-only reminder

Avoid maintaining two competing configs.

Worked examples bank

Example A — Build without switching (safe)

```
nixos-rebuild build --flake .#lab
# or
nix build .#nixosConfigurations.lab.config.system.build.toplevel
```

Example B — Switch

```
sudo nixos-rebuild switch --flake .#lab
```

Example C — Show config value

```
nix eval .#nixosConfigurations.lab.config.networking.hostName
```

Example D — Show system path

```
nix eval --raw .#nixosConfigurations.lab.config.system.build.toplevel
```

Example E — Trace eval errors

```
nixos-rebuild build --flake .#lab --show-trace
```

Lab 1 — Repository layout on the lab host

```
mkdir -p ~/nixos-config/hosts/lab
cd ~/nixos-config
git init
```

```
sudo cp /etc/nixos/hardware-configuration.nix ~/nixos-config/hosts/lab/
sudo chown "$USER":"$USER" ~/nixos-config/hosts/lab/hardware-configuration.nix
```

Write `flake.nix` + `hosts/lab/configuration.nix` (adapt user, system, boot).

```
git add .
nix flake lock
git add flake.lock
git commit -m "initial nixosConfigurations.lab"
```

Lab 2 — First flake rebuild

```
cd ~/nixos-config
sudo nixos-rebuild switch --flake .#lab
```

Verify:

```
nixos-version
hostname
nix eval .#nixosConfigurations.lab.config.networking.hostName
```

Lab 3 — Prove flake is authority

1. Change a harmless package in `environment.systemPackages` (add `htop`).
2. Rebuild switch.
3. Confirm `htop` available.
4. Commit.

Avoid editing `/etc/nixos/configuration.nix` as the long-term path—if installer files remain, leave a comment in README: **flake repo is SoT**.

Lab 4 — README for future you

`README.md`:

```
# nixos-config

## Rebuild
sudo nixos-rebuild switch --flake .#lab

## Host
- flake attr: lab
- hostname: lab
- nixpkgs: 26.05 (see flake.lock)
```

Lab 5 — Optional: match `/etc/nixos` to flake

Some prefer keeping only a stub under `/etc/nixos`. Simplest reliable approach:

```
cd ~/nixos-config && sudo nixos-rebuild switch --flake .#lab
```

Do **not** create circular confusion between two live configs.

Exercises

1. Eval three config values with `nix eval` (hostname, stateVersion, openssh enable).
2. Break a module path on purpose; fix via `--show-trace`.
3. Add a second module file for packages only; import it.
4. Record locked nixpkgs rev in README from `nix flake metadata`.
5. Rebuild with absolute flake path under sudo.
6. Compare installer config vs flake config—list deltas.

7. Sketch multi-host layout for lab + future laptop.
8. Measure toplevel path size with `nix path-info -Sh` after build.
9. Ensure `flake.lock` is never gitignored.
10. Write a one-line alias `nrs` for rebuild (shell config later via HM).
11. Deliberately leave a file untracked; observe flake missing-file behavior.
12. Document `system` triple and why it must match the VM.

Common pitfalls

Symptom	Theory
Absolute import paths broken	Prefer relative module paths in repo
Wrong `system`	Eval/build target mismatch
hardware config not updated after disk change	Regenerate carefully
`experimental-features` missing	Rebuild cannot use flakes cleanly
Dirty git / missing files	Flake file inclusion
`stateVersion` folklore	Set to install release; read release notes before changing
Sudo cwd confusion	Absolute flake URI
Editing `/etc/nixos` by habit	Flake is SoT

Checkpoint

- ☐ `nixosConfigurations.<host>` exists
- ☐ `hardware-configuration.nix` imported from flake repo
- ☐ `switch --flake .#host` succeeded
- ☐ Lock committed
- ☐ README documents rebuild
- ☐ Small declarative change proven

Further depth (this book)

Topic	Chapter
Rebuild modes / rollback	Rebuild modes and generations
Users/SSH	Users and access
Module literacy	Modules as a consumer
Custom modules	Writing custom modules
Advanced flake layout	Home and flake layout part

Rebuild modes and generations

Rebuild modes and generations

Goal: Master `nixos-rebuild` **modes**, **system generations**, and **rollback**—including a deliberate break and recovery on the disposable lab.

Why this chapter matters

NixOS's famous safety is not automatic—it is **generational profiles + boot entries + knowing which rebuild verb you used**. Without that theory, "rollback" is folklore.

Ops motivation: production incidents need a rehearsed rollback card. This chapter builds that muscle on a disposable VM so production is not your first practice.

Theory 1 — System profile and generations

NixOS points the running system at a profile:

```
/nix/var/nix/profiles/system → system-N-link → store path
```

Each successful deployment can add a **generation** `N`. Boot loaders list generations so you can boot an older one when the new world is wrong.

Concept	Meaning
Generation	A complete system closure you can activate/boot
Current	What the profile points at now
Rollback	Point profile (and/or boot default) at older generation

Inspect

```
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
readlink -f /nix/var/nix/profiles/system
ls -l /nix/var/nix/profiles/system*
```

Theory 2 — Rebuild modes

```
sudo nixos-rebuild <mode> --flake .#lab
```

Mode	Effect
`switch`	Build + activate **now** + set boot default to new gen
`boot`	Build + set boot default; **do not** activate until reboot
`test`	Build + activate **now**; **do not** change boot default
`build`	Build only; no activation
`dry-activate`	Show activation plan (when supported/useful)
`build-vm`	Build a VM of the config (extra workflows)

Choosing modes

Situation	Prefer
Normal daily change	`switch`
Risky change; want easy reboot escape	`test` first or `boot`
Remote SSH-only host	Extra care—bad `switch` can lock you out
CI / check	`build`
Kernel/boot experiments	`boot` then reboot deliberately

Remote note

If you break networking/SSH on a remote machine with `switch`, you may need console. Lab VMs have consoles—use them.

Theory 3 — Activation vs boot default

```
test:   running config changes; reboot may return to previous default
switch: running + default updated
boot:   default updated; running stays old until reboot
```

Worked story

1. `test` a firewall change → SSH still works →
2. `switch` to commit it to boot default

Or:

1. `boot` a kernel change → reboot when ready →
2. If unbootable, select previous generation in boot menu

Decision flowchart

```
Is the change network/SSH sensitive?
  yes → prefer test (with console available) → switch
Is the change kernel/bootloader?
  yes → consider boot → reboot → verify
Otherwise → switch
Want no activation? → build
```

Theory 4 — Listing and switching generations

```
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
# or
sudo nixos-rebuild list-generations  # when available in your version
```

Rollback helpers:

```
sudo nixos-rebuild switch --rollback
# or boot older generation from boot menu
```

Also:

```
sudo /nix/var/nix/profiles/system-<N>-link/bin/switch-to-configuration switch
```

Prefer documented `nixos-rebuild` interfaces when possible.

Boot menu recovery

When userland is too broken to roll back from a shell:

1. Reboot
2. Open boot loader menu (systemd-boot/GRUB key)
3. Select previous generation
4. Fix flake; `switch` a good config

Document **your** hypervisor key chords in the rollback card.

Theory 5 — What is *not* rolled back

Rolled back with system gen	Not rolled back
Packages, units, users declared in config	Your data in `/home`, `/var`
Most `/etc` files managed by NixOS	Databases, untracked files
Kernel (for that gen)	Out-of-band disk edits
Firewall rules from config	Manual live nft edits as SoT (don't)

Punchline: rollback restores **system closure**, not time-travel for application data.

Theory 6 — GC and generations

Old generations **root** large closures. Cleaning:

```
sudo nix-collect-garbage -d   # deletes old profile gens - careful
sudo nixos-rebuild …          # after understanding
```

> **!** Important
>
> Do not delete generations until you have a known good current system and understand you are dropping rollback points.

Action	Risk
Keep last N gens	Disk vs safety trade
GC after known-good deploy	Frees space
GC before verifying	Removes escape hatches

Theory 7 — Flake + rebuild flags worth knowing

```
sudo nixos-rebuild switch --flake .#lab
sudo nixos-rebuild switch --flake .#lab --show-trace
sudo nixos-rebuild switch --flake .#lab -L
sudo nixos-rebuild switch --flake path:/home/you/nixos-config#lab
```

Flag	Use
`--show-trace`	Eval errors
`-L`	Build logs
`--impure`	Avoid unless you know why
`--upgrade` / channel habits	Prefer flake lock updates instead

Lock updates are a separate concern (Home and flake layout part).

Theory 8 — Activation scripts and failure modes

Activation runs scripts that update bootloader entries, restart units, rewrite `/etc`, etc.

Failure class	Response
Eval	No new gen; fix Nix
Build	No activation; fix package
Activation partial	Read output; may need rollback/console
Unit failed	`systemctl`, journal; gen may still be current

`test` is valuable because reboot can abandon a bad experiment if boot default never moved.

Theory 9 — Naming generations in ops notes

Track in `docs/rollback.md`:

Field	Example
Last known good gen	12
Last change summary	“opened ssh + htop”
Snapshot name	`pre-firewall`

Generations numbers alone are not memory—annotate them.

Worked examples bank

Example A — Safe build

```
cd ~/nixos-config
nixos-rebuild build --flake .#lab
```

Example B — test then switch

```
sudo nixos-rebuild test --flake .#lab
# exercise the change
sudo nixos-rebuild switch --flake .#lab
```

Example C — rollback

```
sudo nixos-rebuild switch --rollback
```

Example D — Compare generations (conceptual)

```
ls -l /nix/var/nix/profiles/system-*-link | tail
# note store paths differ per generation
```

Example E — Logs during rebuild

```
sudo nixos-rebuild switch --flake .#lab -L --show-trace
```

Lab 1 — Inventory generations before chaos

```
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
readlink -f /nix/var/nix/profiles/system
```

Record current generation number in notes.

Lab 2 — Mode drills (harmless package)

1. Add a tiny package with `boot` mode only.
2. Confirm it is **not** on PATH until reboot (or document actual behavior you observe).
3. Reboot (VM snapshot first).
4. Confirm package present.

Then:

1. Use `test` for another package.
2. Confirm available immediately.
3. Reboot without `switch` and observe whether it persists as default—**write what happened**.

Lab 3 — Deliberate break → rollback

VM snapshot first.

Pedagogical break:

1. `switch` a config that installs a known package `cowsay`.
2. Then `switch` a config that removes it and sets a harmless env marker.
3. `rollback` and show package/env difference.

Document generation numbers before/after.

Safer annoying-but-not-fatal changes beat eval failures (eval never creates a "bad generation").

Lab 4 — Boot menu literacy

On next reboot:

1. Enter boot loader menu (often hold space / show menu for systemd-boot).
2. Identify generation entries.
3. Boot current known-good.

Write how **your** hypervisor + loader expose this.

Lab 5 — Incident card

Create `~/nixos-config/docs/rollback.md`:

```
# Rollback card
1. Console access method: …
2. List generations: …
3. Rollback command: …
4. Boot menu steps: …
5. Last known good generation: …
```

Exercises

1. Fill a table: mode × (activates now?) × (changes boot default?).
2. Perform `build` then inspect toplevel path without switching.
3. Simulate remote-only caution: list changes you would only `test` first.
4. After three switches, list gens and mark which you could GC later.
5. Break SSH in a snapshotted VM; recover via console + rollback.
6. Time a no-op rebuild vs a package-adding rebuild.
7. Document difference between profile rollback and boot-menu boot.
8. Add a "last good gen" field to your git commit message habit.
9. Explain to a peer why DB data survives rollback (or doesn't get restored).
10. Practice absolute flake path rebuild under sudo.
11. Read activation output for one switch; annotate three steps.
12. Update rollback card after the boot chapter for loader keys.

Common pitfalls

Symptom	Theory
Rollback “did nothing” for app data	Data system gen
Locked out via SSH	Use console; prefer `test` for net changes
No older generations	GC’d or first install
`test` change vanished after reboot	Expected if never `switch`ed
Flake path wrong under sudo	Use absolute path or root’s cwd discipline
Eval error mistaken for rollback drill	Need a successful bad generation
GC before verification	Lost escape hatches

Checkpoint

- ☐ Explain switch/test/boot/build
- ☐ List system generations
- ☐ Performed a rollback successfully
- ☐ Boot menu generation selection documented
- ☐ Incident card written
- ☐ VM snapshot restored option still available

Further depth (this book)

Topic	Chapter
Firewall safety with test	Networking and firewall
Boot loader entries	Boot and hardware
State vs generations	Filesystems and state
GC hygiene fleet-wide	Ops part

Users and access

Users and access

Goal: Declare users and remote access properly—**SSH keys** as the default remote path, non-root admin via `wheel`, and no long-lived plaintext password habits in the flake.

Why this chapter matters

Most early NixOS grief is lockouts and sloppy auth: root password only, keys never deployed, secrets pasted into world-readable store paths. Fix access patterns before opening the host to a network.

Ops motivation: access policy is production policy. A lab that practices key-only SSH and console recovery trains the same habits you need on real hosts.

Theory 1 — Declarative users

```
users.users.alice = {
  isNormalUser = true;
  extraGroups = [ "wheel" "networkmanager" ];
  openssh.authorizedKeys.keys = [
    "ssh-ed25519 AAAA… comment"
  ];
};
```

Field	Role
`isNormalUser`	Normal account defaults (home, shell group, …)
`isSystemUser`	Service accounts
`extraGroups`	Group membership
`uid` / `group`	Pin when needed
`shell`	e.g. `pkgs.zsh`
`hashedPassword` / `initialHashedPassword`	Password auth
`openssh.authorizedKeys.keys`	SSH public keys
`description`	GECOS / human label

System users (preview)

```
users.users.labbot = {
  isSystemUser = true;
  group = "labbot";
};
users.groups.labbot = {};
```

Service bots do not belong in `wheel`.

Theory 2 — Root vs wheel admin

Pattern	Notes
Root login over SSH	Discourage; often disable
User in `wheel` + sudo	Preferred admin path
Passwordless sudo	Convenient lab; tighten later

```
security.sudo.wheelNeedsPassword = true; # or false for lab convenience
```

```
services.openssh.settings = {
  PermitRootLogin = "no";
  PasswordAuthentication = false;
};
```

Exact option paths can vary slightly by release—prefer `services.openssh.settings.*` on modern NixOS; verify with docs/`man configuration.nix` / `nixos-option` / search.nixos.org **26.05**.

Order of hardening

1. Key login works for wheel user
2. Disable password SSH
3. Disable root SSH
4. Consider `mutableUsers = false`

Never reverse the order on a remote-only host without console.

Theory 3 — Passwords and the store

Approach	Store risk
`initialPassword = "secret"`	**Password may land in store/world-readable eval** — lab only, rotate
`hashedPassword = "…"`	Hash still visible in store; better than plaintext but not a secret manager
SSH keys only	Public keys are public; private keys never in flake
sops-nix / agenix later	Services and security part

House rule now: SSH public keys in git are fine; **private keys and live passwords are not**.

Generate hash on the machine if you need password console login:

```
mkpasswd -m sha-512
# or openssl passwd -6
```

Why the store leaks matter

NixOS evaluation materializes values into the store. Anything in the flake that is a secret string is a secret management failure—even if file mode looks fine in git.

Theory 4 — SSH service basics

```
services.openssh = {
  enable = true;
  ports = [ 22 ];
  settings = {
    PasswordAuthentication = false;
    KbdInteractiveAuthentication = false;
    PermitRootLogin = "no";
  };
};
```

Firewall must allow the port (networking chapter)—if SSH dies after firewall work, check both.

Host keys vs user keys

Key type	Purpose
SSH **host** keys	Identify the machine to clients
User **authorized** keys	Authenticate users to the machine
Your laptop private key	Stays on client

Host keys live under `/etc/ssh` and are part of machine identity—backup/restore stories later (filesystems chapter).

Theory 5 — Authorized keys on NixOS

```
users.users.alice.openssh.authorizedKeys.keys = [ "ssh-ed25519 AAAA… alice@laptop" ];
# or
# users.users.alice.openssh.authorizedKeys.keyFiles = [ ./keys/alice.pub ];
```

Prefer **one key per line**, full public key material, no wrapping. A truncated key fails as "Permission denied (publickey)."

NixOS often manages authorized keys under `/etc/ssh/authorized_keys.d` rather than only `~/.ssh/authorized_keys`—verify on your host.

Client config hygiene

```
Host nixlab
  HostName 192.0.2.10
  User alice
  IdentityFile ~/.ssh/id_ed25519_nixlab
  IdentitiesOnly yes
```

`IdentitiesOnly yes` avoids offering the wrong key and hitting `MaxAuthTries`.

Theory 6 — Groups you will actually use

Group	Common reason
`wheel`	sudo
`networkmanager`	NM control (if NM)
`docker` / `podman`	Containers later—grant carefully
`video` / `input`	Desktop labs

Group	Common reason

Least privilege: do not dump every group on the admin user “just in case.”

Theory 7 — Home directories and mutability

Declarative users create homes per policy. Files inside homes are **state**. Rolling back system generations does not reset `~/.ssh` edits. Home Manager (later part) manages dotfiles; this chapter is raw user access only.

`mutableUsers`

```
users.mutableUsers = false; # strong: only flake defines users/passwords
# users.mutableUsers = true;  # imperative passwd may persist
```

Setting	Effect
`true`	Imperative drift possible
`false`	Users/passwords from config (hashed)

For a lab that teaches flakes as SoT, consider `false` once SSH keys work—know that password changes then require a rebuild.

Theory 8 — Sudo and elevation patterns

Pattern	Lab use
`wheelNeedsPassword = true`	Safer habit
passwordless sudo	Faster drills; note risk
`sudo -u` service users	Later service chapters

Prefer proving `sudo -v` works before disabling other access methods.

Theory 9 — Debugging SSH failures

From client:

```
ssh -vvv -i ~/.ssh/id_ed25519_nixlab alice@<vm-ip> 2>&1 | tail -40
```

On host:

```
sudo journalctl -u sshd -n 50 --no-pager
ss -ltn | grep 22
```

Symptom	Checks
Connection refused	sshd down / wrong IP / firewall
Permission denied (publickey)	key material, user, identities
Auth succeeds, sudo fails	wheel membership

Theory 10 — Audit checklist for flakes

```
cd ~/nixos-config
rg -n "initialPassword|password\s*=" || true
rg -n "BEGIN OPENSSH PRIVATE KEY" || true
rg -n "PRIVATE KEY" || true
```

Rotate anything that ever landed in git history.

Worked examples bank

Example A — Admin user with key

```
users.users.alice = {
  isNormalUser = true;
  extraGroups = [ "wheel" ];
  openssh.authorizedKeys.keys = [
    "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKeyMaterial alice@laptop"
  ];
};
```

Example B — Disable password SSH

```
services.openssh.enable = true;
services.openssh.settings.PasswordAuthentication = false;
```

Example C — Console password without storing plaintext

```
users.users.alice.hashedPassword = "$6$…"; # from mkpasswd
```

Example D — System user for a service (preview)

```
users.users.labbot = {
  isSystemUser = true;
  group = "labbot";
  description = "Lab automation user";
};
users.groups.labbot = {};
```

Example E — Key file in repo

```
users.users.alice.openssh.authorizedKeys.keyFiles = [
  ./keys/alice.pub
];
```

Ensure `./keys/alice.pub` is git-tracked (public only).

Lab 1 — Generate a lab keypair on the client

On your workstation (not necessarily the VM):

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_nixlab -C "nixlab"
cat ~/.ssh/id_ed25519_nixlab.pub
```

Copy the **public** key into the flake. Never commit the private key.

Lab 2 — Wire user + SSH into flake

Edit `hosts/lab/configuration.nix`:

1. Admin user with `wheel`
2. `openssh.authorizedKeys.keys`
3. SSH enabled
4. Prefer `PasswordAuthentication = false` once key login works
5. `PermitRootLogin = "no"` when ready

```
cd ~/nixos-config
sudo nixos-rebuild switch --flake .#lab
```

From client:

```
ssh -i ~/.ssh/id_ed25519_nixlab alice@<vm-ip>
```

Lab 3 — Verify identity paths

On host:

```
whoami
groups
sudo -v
ss -ltn | grep -E ':22'
```

On client: confirm you did **not** need a password for SSH (if disabled).

Lab 4 — Lockout drill (console required)

Snapshot first.

1. Misconfigure authorized key (truncate key).
2. Rebuild.

3. Confirm SSH fails.
4. Fix via **console**, rebuild.

Write the recovery path on your rollback card.

Lab 5 — Audit flake for secret smells

```
cd ~/nixos-config
rg -n "initialPassword|password\s*=" || true
rg -n "BEGIN OPENSSH PRIVATE KEY" || true
```

Remove plaintext habits. Document any temporary lab exceptions.

Lab 6 — `users.mutableUsers` experiment (optional, snapshot)

1. Note current `mutableUsers`
2. Set explicitly in flake
3. Rebuild
4. Document whether `passwd` changes survive the next rebuild

Revert if the policy is too strict for console-only recovery yet.

Lab 7 — SSH hard fail checklist

Run the verbose client + journalctl flow from Theory 9. Write the actual authorized_keys path you observed.

Exercises

1. Add a second non-admin user with a different key.
2. Move public keys to `keyFiles` and rebuild.
3. Toggle `wheelNeedsPassword` and document UX impact.
4. Attempt root SSH after `PermitRootLogin = "no"`; confirm failure.
5. Create a system user; prove it has no login shell access as intended.
6. Document host key fingerprints for your SSH `known_hosts` process.
7. Practice recovery: break sudo group membership; fix on console.
8. Write an access policy paragraph for this lab (who, how, exceptions).
9. Scan git history for accidental secrets if you ever committed them.
10. Align firewall chapter: ensure port 22 allowed after hardening.
11. Compare password hash approaches; choose one documented method.
12. Update `docs/rollback.md` with SSH lockout recovery steps.
13. Explain store visibility of `hashedPassword` to a peer.
14. Prepare for Home Manager later: list dotfiles you will *not* put in the system flake.

Common pitfalls

Symptom	Theory
Key in flake but SSH password still works	PasswordAuthentication still on
Permission denied (publickey)	Wrong user, wrong key, typo in key line
Locked out	Console + rollback
Password in git history	Rotate; rewrite history if needed; prefer never
Root SSH only	Add wheel user before disabling root
Private key committed	Emergency rotation
Wrong authorized_keys path assumptions	NixOS managed paths
`mutableUsers` surprise	Policy mismatch with passwd

Checkpoint

- ☐ Declarative non-root admin user
- ☐ SSH key login works
- ☐ Password SSH disabled (or explicitly justified)
- ☐ Root SSH disabled or justified
- ☐ No private keys in repo
- ☐ Lockout recovery tested on console
- ☐ Secret audit clean

Further depth (this book)

Topic	Chapter / part
Firewall for SSH	Networking and firewall
Host keys as state	Filesystems and state
Secrets management	Services and security: sops/agenix
User dotfiles	Home Manager chapters

Networking and firewall

Networking and firewall

Goal: Configure lab networking deliberately—pick **networkd or NetworkManager**, know your address, and run a **default-deny-ish firewall** that still allows SSH.

Why this chapter matters

A flake host that "has SSH in config" but blocked by firewall, or random DHCP with no notes, is not operable. Networking is part of the system closure story.

Ops motivation: most "NixOS is down" tickets are network, DNS, or firewall mistakes. Declare them, test them, document them.

Theory 1 — Two common network stacks on NixOS

Stack	Typical use	Module vibes
systemd-networkd	Servers, VMs, simple static	`systemd.network`, `networking.useNetworkd`
NetworkManager	Laptops, Wi-Fi, desktop	`networking.networkmanager.enable`

Do not enable both casually without understanding conflicts. For a **server-like VM**, networkd or simple DHCP via `networking.*` is enough.

Classic DHCP simplicity

```
networking.useDHCP = false;
networking.interfaces.ens3.useDHCP = true; # interface name from `ip a`
```

Or global patterns depending on release defaults—**inspect generated hardware and `ip a`**.

networkd file shape (server VM)

```
{
  networking.useNetworkd = true;
  networking.useDHCP = false;
  systemd.network.networks."10-wan" = {
    matchConfig.Name = "en*";
    networkConfig.DHCP = "yes";
  };
}
```

Interface match patterns reduce rename pain vs hard-coding `ens3` only—verify on **your** virt NIC names.

Theory 2 — Host identity

```
networking.hostName = "lab";
# networking.domain = "example.local";
```

Hostname affects prompts, certificates later, and mental ops. Keep flake attr and hostname documented (flake hosts chapter).

Extra hosts

```
networking.extraHosts = ''
  127.0.0.1 lab.local
  10.0.0.5  sibling.lab
'';
```

Useful for multi-VM labs; real DNS arrives with TLS/ACME later.

Theory 3 — Firewall model

NixOS firewall (iptables/nftables backend depending on config) is commonly:

```
networking.firewall = {
  enable = true;
  allowedTCPPorts = [ 22 ];
  allowedUDPPorts = [ ];
```

```
  # allowedTCPPortRanges = [ { from = 8000; to = 8010; } ];
};
```

Idea	Practice
Default deny inbound	Enable firewall
Open only needed ports	List explicitly
Per-interface rules	Advanced; later
Service modules open ports	Some set firewall rules when enabled

Always ensure SSH port is allowed before testing remote-only access.

nftables awareness (26.05)

Modern NixOS often prefers **nftables** as the firewall backend. You rarely write raw rules by hand; you set module options and **inspect**:

```
sudo nft list ruleset | head -80
sudo iptables -L -n 2>/dev/null | head || true
```

Practice	Avoid
Module options + rebuild	Hand-editing live nft rules as SoT
Document backend in `docs/network.md`	Assuming iptables forever

Theory 4 — Interface names

Predictable names (`enp0s3`, `ens18`, `eth0`) depend on virt hardware.

```
ip -br a
networkctl status   # if networkd
nmcli device        # if NM
```

Hard-coding the wrong interface → no network after switch. Prefer DHCP-on-all or verified names.

Theory 5 — Static addressing (optional lab)

```
networking.interfaces.ens3.ipv4.addresses = [{
  address = "192.168.56.10";
  prefixLength = 24;
}];
networking.defaultGateway = "192.168.56.1";
networking.nameservers = [ "1.1.1.1" "9.9.9.9" ];
```

Use only when you understand the hypervisor network. Wrong gateway = silence.

Theory 6 — DNS and resolution

```
resolvectl status  # systemd-resolved often in play
getent hosts cache.nixos.org
```

Broken DNS looks like “Nix is broken” during rebuilds (substituters).

Failure	Symptom
No route	Timeouts
Bad DNS	NXDOMAIN / resolve fail mid-rebuild
Captive / filtered net	Partial substituter failures

Theory 7 — Firewall vs service modules

Example: enabling `services.openssh` does not always mean you thought about firewall—on many configs you still list port 22 (or rely on module defaults). **Verify**:

```
sudo nft list ruleset | head
ss -ltn
```

Hardening deep-dives continue in the Services and security part; this chapter is correctness + minimal surface.

Theory 8 — Rebuild safety pattern for remote operators

```
sudo nixos-rebuild test --flake .#lab
# new shell: ssh still works?
sudo nixos-rebuild switch --flake .#lab
```

`test` activates without making the generation the boot default—slightly safer while iterating firewall. Console access remains the real safety net.

Lockout drill ethics

Only lock yourself out on a VM with console + snapshot. Production requires out-of-band access plans.

Theory 9 — Port audit as continuous practice

Listener	Firewall	Config source	Keep?
`:22`	open	`allowedTCPPorts` / ssh	yes
localhost-only	n/a	bind address	ok
mystery	?	?	investigate

Any public listener without a story is a future ticket.

Theory 10 — Hypervisor port forwards

NAT mode often needs host port → guest 22 forward. Document:

```
host localhost:2222 → guest :22
ssh -p 2222 alice@127.0.0.1
```

Firewall on guest still applies to forwarded traffic.

Worked examples bank

Example A — Server-like DHCP + SSH

```
{
  networking.hostName = "lab";
  networking.firewall.enable = true;
  networking.firewall.allowedTCPPorts = [ 22 ];
  services.openssh.enable = true;
}
```

Example B — Explicit interface DHCP

```
{
  networking.useDHCP = false;
  networking.interfaces.ens3.useDHCP = true;
}
```

Example C — Temporary debug open (lab only)

```
# networking.firewall.allowedTCPPorts = [ 22 80 443 ];
```

Remove ports you do not need.

Example D — networkd DHCP match

See Theory 1 networkd snippet.

Example E — DNS pin

```
networking.nameservers = [ "1.1.1.1" "9.9.9.9" ];
```

Lab 1 — Discover ground truth

On the lab host:

```
ip -br a
ip route
cat /etc/hostname
resolvectl status 2>/dev/null | head
```

Record interface name(s), IPv4, default route.

Lab 2 — Declare firewall explicitly

In flake config:

1. `networking.firewall.enable = true`
2. Allow SSH port only
3. Rebuild (`test` recommended if remote)

```
sudo nixos-rebuild test --flake .#lab
# verify ssh still works
sudo nixos-rebuild switch --flake .#lab
```

Lab 3 — Port audit

```
ss -ltnup
# map each listener to a reason
```

Fill table in `docs/network.md`.

Lab 4 — Controlled break (console ready)

Snapshot.

1. Remove 22 from `allowedTCPPorts` while SSH is your only remote path.
2. `switch` (expect lockout if no console).
3. Fix via console; restore port.

If you are console-only already, simulate by checking `nft` counters and documenting the mistake.

Lab 5 — Network README snippet

Add to `docs/network.md`:

- Hypervisor network mode (NAT/bridge/host-only)
- How to find VM IP from host
- Firewall policy summary
- DNS notes
- Backend (nftables/iptables observation)

Lab 6 — Firewall rule listener map

Build a three-column table:

Listener (`ss`)	Firewall open?	Config source
`:22`	yes	`allowedTCPPorts` / ssh module

Lab 7 — DNS failure drill (safe)

```
getent hosts cache.nixos.org
```

Document: rebuilds that need substitutes fail hard when DNS dies mid-flight—order network fixes carefully.

Exercises

1. Draw your hypervisor network path (host → NAT → guest).
2. Switch from interface-hardcoded DHCP to a match pattern (if networkd).
3. Add a second allowed port for a localhost service only if intentional.
4. Compare `test` vs `switch` after a firewall edit.
5. Capture `nft list ruleset` before/after enabling firewall.
6. Break DNS with a bad nameserver in a snapshotted VM; recover.
7. Document IPv6 status on your lab (on/off/partial).
8. Map each `ss` listener to a Nix option or "unknown."
9. Write a "no inbound except SSH" policy in one paragraph.
10. Practice reconnect scripts from the workstation.
11. Note whether openssh module opened ports for you automatically.
12. Prepare for TLS chapter: pick a future lab hostname.

Common pitfalls

Symptom	Theory
Rebuild can't fetch	DNS/route broken mid-change
SSH timeout after firewall	Port not allowed
Interface rename after hardware change	Update config
Both NM and networkd fighting	Pick one model
`useDHCP` defaults surprise	Read options for 26.05
Live nft edits as SoT	Lost on rebuild
NAT without port forward	"VM has no SSH" false alarm

Checkpoint

- ☐ Interface + IP documented
- ☐ Firewall enabled with explicit SSH allow
- ☐ Port audit table written
- ☐ Recovery from firewall mistake understood
- ☐ Committed network docs
- ☐ Prefer `test` for risky net changes

Further depth (this book)

Topic	Chapter / part
SSH users	Users and access
Service ports	systemd via Nix
Hardening / TLS	Services and security part
Firewall discipline deep	Services and security part

Packages and programs

Packages and programs

Goal: Install software **declaratively** on the host—know when to use `environment.systemPackages` versus `programs.*` modules, and keep ad-hoc profiles from becoming the real SoT.

Why this chapter matters

New NixOS users `nix-env` / `nix profile install` a zoo, then wonder why rebuilds do not reproduce the machine. Declarative packages are how the host stays honest.

Ops motivation: reproducible admin tooling is part of incident response. The box you SSH into should carry `jq` and friends because the flake says so—not because someone remembered to install them once.

Theory 1 — `environment.systemPackages`

```
environment.systemPackages = with pkgs; [
  vim
  git
  curl
  htop
  jq
];
```

Property	Meaning
On PATH system-wide	For all users (typical)
Closure in system gen	Rolled back with system
No service config	Just binaries/libs

This is the blunt instrument—and it is correct for many CLI tools.

Organization pattern

```
environment.systemPackages = with pkgs; [
  # editors
  vim
  # net
  curl
  # observability
  htop
];
```

Readable policy beats an unsorted dump.

Theory 2 — `programs.*` modules

Many packages have first-class modules:

```
programs.git.enable = true;
programs.zsh.enable = true;
programs.tmux.enable = true;
```

Modules may:

- Install the package
- Drop config snippets
- Set shells in `/etc/shells`
- Enable related services

Prefer `programs.*` when you need integration, not only a binary.

Discovery

```
man configuration.nix  # huge
nixos-option programs.git 2>/dev/null
```

Online: https://search.nixos.org/options (select **26.05** when available).

Theory 3 — Decision table

Need	Prefer
Just a CLI on PATH	`environment.systemPackages`
Shell as login shell + vendor config	`programs.zsh` / `programs.fish` / `programs.bash`
Git system-wide with config hooks	`programs.git` and/or HM later
Language toolchains for *one project*	**devShell**, not systemPackages
Desktop app	packages or specialized modules
Always-on service	`services.*` / systemd chapter

Theory 4 — Closure discipline (light)

Every package on the system increases:

- Download size
- Eval/build time
- Attack surface
- Cognitive load

Smell	Fix
Entire language toolchains system-wide	devShell / HM
Three editors "just in case"	Pick one for system
Random `nix profile` duplicates	Remove; declare in flake
Debug tools forever	Keep a small admin set

```
nix path-info -Sh /run/current-system
nix path-info -rS /run/current-system | sort -n | tail
```

Theory 5 — Overlays and pins (awareness only)

You might see:

```
nixpkgs.overlays = [ … ];
```

or pinned packages. Later parts cover overlays properly. Here: stick to nixpkgs **26.05** attribute names from your lock.

Theory 6 — Unfree and allow flags

Some packages need:

```
nixpkgs.config.allowUnfree = true;
```

Or per-package predicates:

```
nixpkgs.config.allowUnfreePredicate = pkg:
  builtins.elem (pkgs.lib.getName pkg) [ "vscode" ]; # example only
```

Only enable when you understand licensing implications for **your** lab. Narrow predicates beat global allow when possible.

Theory 7 — System vs user packages

Layer	Mechanism
System	`environment.systemPackages`, `programs.*`
User (later)	Home Manager `home.packages`
Project	`nix develop` shells
Imperative	`nix profile` — minimize

Stage-shaped later work moves dotfiles and user CLIs to HM; this chapter is system layer only.

Theory 8 — `environment.defaultPackages` trap

NixOS may install a small default package set. Know it exists so you do not double-declare or fight removals:

```
# advanced / intentional only
# environment.defaultPackages = [ ];
```

Prefer adding what you need over emptying defaults on day one unless you understand the fallout (e.g. `nano`/`perl` expectations in scripts).

Theory 9 — Package name binary name

pkgs attr	Binary might be
`ripgrep`	`rg`
`neovim`	`nvim`
some wrappers	different `meta.mainProgram`

```
nix search nixpkgs ripgrep
command -v rg
```

Theory 10 — Layers interaction with generations

Adding/removing system packages creates new generations. Rollback brings the old package set back—another reason not to rely on imperative profiles for "real" tools.

Worked examples bank

Example A — Baseline admin tools

```
{ pkgs, ... }: {
  environment.systemPackages = with pkgs; [
    vim
    git
    curl
    wget
    htop
    tmux
    jq
    ripgrep
    tree
  ];
```

```
}
```

Example B — programs integration

```
{
  programs.zsh.enable = true;
  users.users.alice.shell = pkgs.zsh;
}
```

Example C — Search package attr

```
nix search nixpkgs ripgrep
```

Example D — Conditional unfree single package

```
{
  nixpkgs.config.allowUnfreePredicate = pkg:
    builtins.elem (pkgs.lib.getName pkg) [ "vscode" ];
}
```

Example E — Avoid double install

If `programs.git.enable = true` already installs git, do not also list `git` unless you have a reason—collapse to one path.

Lab 1 — Declare your daily CLI set

List 8–15 tools you truly want on the **lab host**. Add them via `environment.systemPackages`. Rebuild switch. Verify each:

```
command -v rg
command -v jq
# …
```

Lab 2 — Convert one package to `programs.*`

Pick something with a module (git, zsh, tmux, …):

1. Remove from `systemPackages` if redundant
2. Enable `programs.<name>`
3. Rebuild
4. Note extra behavior (completions, config files)

Lab 3 — Anti-profile audit

```
nix profile list 2>/dev/null || true
```

If tools exist only imperatively, either migrate into flake or remove them. Document: **flake owns system tools**.

Lab 4 — Size awareness

```
nix path-info -Sh /run/current-system
nix path-info -rS /run/current-system | sort -n | tail
```

Write one sentence on the cost of your package list.

Lab 5 — Split comments in config

Organize packages with section comments. Commit as readable policy.

Lab 6 — `programs.*` option archaeology

```
nixos-option programs.zsh 2>/dev/null | head -40
```

List three options under a `programs.*` module that are **not** just "install the binary."

Lab 7 — Remove one redundant package

Find a package present both via `systemPackages` and a `programs.*` enable (or profile). Collapse to one path. Rebuild. Confirm still on PATH.

Exercises

1. Build a "minimal rescue" package set vs "comfortable admin" set; compare sizes.
2. Move a language toolchain out of systemPackages into a project devShell.
3. Enable `programs.tmux` and note config paths created.
4. Document unfree policy for the lab in README.
5. Find three packages where attr binary name.
6. Use `nix-tree` or path-info to find heavy dependencies (if available).
7. Prove rollback restores a removed package.
8. Write a PR-style commit message for package policy changes.
9. Avoid `with pkgs;` for a section; use explicit `pkgs.` and compare readability.
10. List tools that should never be system-wide on a shared host.
11. Prepare HM split: which packages will leave system later?
12. Search options for a program you use; enable one non-obvious toggle.

Common pitfalls

Symptom	Theory
Package name binary name	Check `pkgs` attr vs `meta.mainProgram`
Unfree errors	allowUnfree policy
Tool missing after rebuild	Not in config; was profile-only
Giant system closure	Too many packages / wrong layer
Shell not changing	`users.users.*.shell` not set
Double-declared tools	programs + systemPackages
Emptying defaultPackages casually	Surprising script breakage

Checkpoint

- ☐ Daily tools declared in flake
- ☐ At least one `programs.*` used
- ☐ Imperative profile not required for lab tools
- ☐ Rough system path size noted
- ☐ Config comments organized
- ☐ Layering (system vs project) explained

Further depth (this book)

Topic	Chapter / part
Custom services using pkgs	systemd via Nix
Module options	Modules as a consumer
Home packages	Home Manager chapters
Overlays / packaging	Packaging part

systemd via Nix

systemd via Nix

Goal: Express **systemd services and timers** in NixOS modules—ship one custom service + one timer, and verify units with standard systemd tools.

Why this day exists

NixOS does not replace systemd; it **generates** units. If you only know `systemctl` on mutable distros, you will fight activation. Today connects module options → unit files → runtime.

Theory 1 — Module options generate units

```
systemd.services.hello-lab = {
  description = "Day17 hello service";
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    Type = "oneshot";
    ExecStart = "${pkgs.coreutils}/bin/echo hello-from-nix";
  };
};
```

On rebuild, NixOS materializes unit files under the system closure and activates them.

You write	systemd sees
`systemd.services.<name>`	`name.service`
`systemd.timers.<name>`	`name.timer`
`systemd.user.services…`	user units (later/HM)

Theory 2 — Service essentials

Field	Role
`description`	Human text
`wantedBy`	Targets that pull this in
`after` / `requires` / `wants`	Ordering/deps
`serviceConfig.Type`	simple, oneshot, notify, …
`serviceConfig.ExecStart`	Command (prefer **absolute store paths**)
`path`	Extra PATH packages for the unit
`script`	Convenience instead of ExecStart in some cases
`enable` / `wantedBy`	Whether it starts

Absolute paths matter

```
ExecStart = "${pkgs.hello}/bin/hello";
```

Not bare `hello`—units may not share your interactive PATH.

Theory 3 — Timers

```
systemd.services.lab-tick = {
  serviceConfig.Type = "oneshot";
  script = ''
    echo "tick $(date -Is)" >> /var/log/lab-tick.log
  '';
};

systemd.timers.lab-tick = {
  wantedBy = [ "timers.target" ];
  timerConfig = {
    OnBootSec = "1min";
    OnUnitActiveSec = "5min";
    Unit = "lab-tick.service";
  };
};
```

Timer idea	Example
Calendar	`OnCalendar = "daily"`
After boot	`OnBootSec`
Periodic	`OnUnitActiveSec`

Prefer timers over classic cron on NixOS unless you have a reason.

Theory 4 — State directories and permissions

Services that write state need:

- Writable paths (`/var/lib/…`)
- Correct `User=` / `Group=`
- Sometimes `StateDirectory=` in `serviceConfig`

```
serviceConfig = {
  User = "labbot";
  StateDirectory = "labbot"; # → /var/lib/labbot
};
```

Do not write into the Nix store.

Theory 5 — Inspecting generated units

```
systemctl status lab-tick.timer
systemctl cat lab-tick.service
journalctl -u lab-tick.service -n 50
systemctl list-timers --all | head
```

`systemctl cat` shows the unit text—great for verifying Nix rendered what you meant.

Theory 6 — Relationship to `services.*` high-level modules

High-level modules (`services.nginx`, `services.postgresql`) **wrap** systemd units with options, hardening, and firewall hooks. Custom `systemd.services` is the escape hatch and learning tool.

Pattern:

```
prefer services.<known> when it exists
else systemd.services.<custom>
```

Theory 7 — Failure and rollback

A bad unit can fail activation. Generations still help:

1. Fix config
2. Or rollback generation
3. `journalctl -xe` for clues

`test` mode is your friend for unit experiments.

Worked examples bank

Example A — oneshot + timer writing a log

See Theory 3.

Example B — long-running simple service

```
systemd.services.lab-http-static = {
  wantedBy = [ "multi-user.target" ];
  after = [ "network.target" ];
  serviceConfig = {
    Type = "simple";
    ExecStart = "${pkgs.python3}/bin/python3 -m http.server 8000 --bind 127.0.0.1";
    Restart = "on-failure";
  };
};
# open firewall only if you intentionally expose it
```

Bind to localhost for lab safety unless you mean to expose.

Example C — `path` for tools in script

```
systemd.services.lab-path-demo = {
  path = [ pkgs.jq pkgs.coreutils ];
  script = ''
    echo '{"ok":true}' | jq .
```

```
    '';
    serviceConfig.Type = "oneshot";
  };
```

Lab 1 — Ship `lab-tick` timer

Add service+timer from Theory 3 (adjust log path permissions—use `StateDirectory` or a path you own).

```
sudo nixos-rebuild switch --flake .#lab
systemctl status lab-tick.timer
systemctl start lab-tick.service
journalctl -u lab-tick.service -n 20
```

Lab 2 — `systemctl cat` literacy

```
systemctl cat lab-tick.service
systemctl cat lab-tick.timer
```

Annotate in notes: which lines came from your Nix options.

Lab 3 — One long-running service (localhost)

Deploy Example B or simpler. Verify:

```
curl -sS http://127.0.0.1:8000/ | head
systemctl status lab-http-static
```

Ensure firewall does **not** expose 8000 unless intentional.

Lab 4 — Break ExecStart on purpose

Point `ExecStart` at a missing path, rebuild, watch failure, fix, rebuild. Note whether generation advanced.

Lab 5 — Document units in README

```
## Custom units
- lab-tick.timer: …
- lab-http-static.service: …
```

Theory 8 — Hardening starters for custom units

Even lab units benefit from a few defaults:

```
systemd.services.lab-http-static.serviceConfig = {
  Type = "simple";
  ExecStart = "${pkgs.python3}/bin/python3 -m http.server 8000 --bind 127.0.0.1";
  Restart = "on-failure";
  # hardening starters (tune per workload)
  NoNewPrivileges = true;
  PrivateTmp = true;
  ProtectSystem = "strict";
  ProtectHome = true;
  # ReadWritePaths = [ "/var/lib/lab-http" ]; # if needed
};
```

Do not cargo-cult every sandbox flag onto a unit that must write state—pair with `StateDirectory` / `ReadWritePaths`.

Theory 9 — `systemd.tmpfiles` for paths units need

```
systemd.tmpfiles.rules = [
  "d /var/log/lab-tick 0755 root root -"
];
```

Creates directories at boot/activation. Prefer `StateDirectory=` when the unit's runtime user owns the state under `/var/lib`.

Theory 10 — Activation vs live `systemctl edit`

Path	SoT?
Flake `systemd.services.*`	Yes
`systemctl edit --full` drop-ins	Drift; vanishes from mental model
Hand unit in `/etc/systemd/system`	Fights NixOS activation

If you must debug live, capture the working unit into the flake before the next rebuild.

Worked example D — oneshot with `RemainAfterExit`

```
systemd.services.lab-seed = {
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    Type = "oneshot";
    RemainAfterExit = true;
    ExecStart = "${pkgs.coreutils}/bin/touch /var/lib/lab-seed/ok";
    StateDirectory = "lab-seed";
  };
};
```

Useful for "run once to prepare state" patterns without a long-running daemon.

Lab 6 — Timer calendar form

Replace interval timer with:

```
timerConfig.OnCalendar = "*-*-* *:0/15:00"; # every 15 minutes - verify syntax
```

```
systemctl cat lab-tick.timer
systemd-analyze calendar '*-*-* *:0/15:00'
```

Document which calendar expression you settled on.

Lab 7 — Unit dependency map

For your long-running service, write:

```
network.target → lab-http-static.service → (curl clients)
```

Add `after` / `wants` only when real. Over-`requires` makes boot fragile.

Common gotchas

Symptom	Theory
`Executable not found`	Need store absolute path or `path`
Timer never fires	not `wantedBy` timers.target
Permission denied writing log	state dir / user
Service works manually not under systemd	PATH / env differences
Forgotten firewall	the networking chapter + service ports

Checkpoint

- ☐ Custom service in flake
- ☐ Custom timer in flake
- ☐ `systemctl cat` inspected
- ☐ journalctl read for your unit
- ☐ Localhost service verified

Commit

```
cd ~/nixos-config
git add .
git commit -m "day17: systemd services and timers"
```

Modules as a consumer

Modules as a consumer

Goal: Use the NixOS module system **as a consumer**—read options and source for real modules, apply `mkIf` / `mkDefault` concepts correctly, and change behavior via **options** rather than forking nixpkgs.

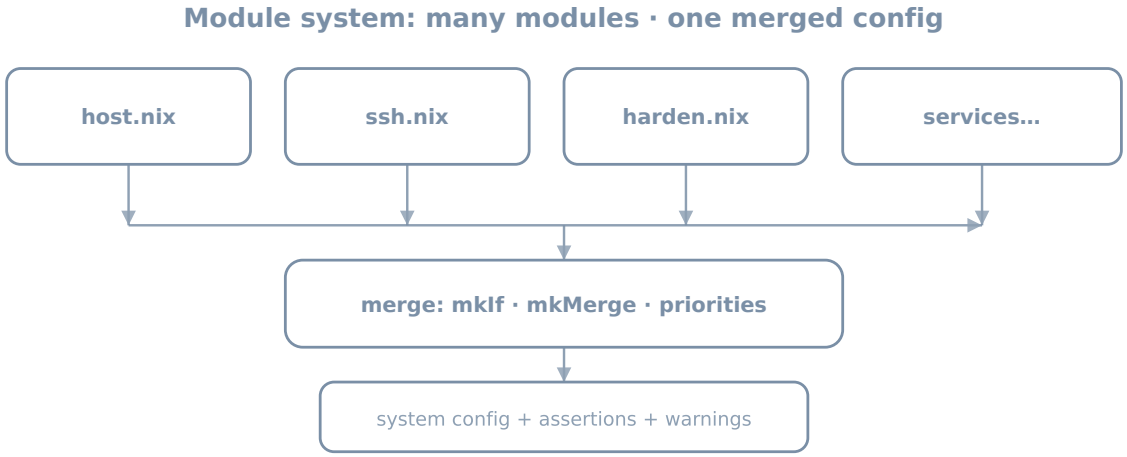


Figure 1: Module merge diagram

Why this day exists

Copy-pasting entire module files into your repo is a maintenance trap. Power users set options, import small local modules, and only override when necessary. That skill starts with **reading**.

Theory 1 — A module is a function (usually)

```
{ config, lib, pkgs, ... }: {
  options = { /* declarations */ };
  config = { /* implementations */ };
}
```

Argument	Role
`config`	Final merged configuration (lazy)
`lib`	Library + `mkOption`, `mkIf`, …
`pkgs`	Package set for this system
…	Other module args

Some files are plain attrsets; most nixpkgs modules use the function form.

Theory 2 — Options vs config

Side	Meaning
options	Schema: what *may* be set, types, defaults, descriptions
config	Values and implementations that produce system artifacts

As a consumer you mostly **set option values**:

```
{
  services.openssh.enable = true;
  networking.firewall.allowedTCPPorts = [ 22 ];
}
```

You are writing `config` fragments that merge into the global config.

Theory 3 — Merge semantics (intuition)

Multiple modules set the same option; the module system **merges** by type:

Type vibe	Merge idea
Boolean	Priority rules / any true patterns depend on option
List	Concatenate
Attrset	Deep merge of attributes
Exclusive values	Conflict error or priority

This is why shallow `//` (the language attrsets chapter) is not how NixOS config works.

Theory 4 — `lib.mkIf`

```
{ config, lib, ... }: {
  config = lib.mkIf config.services.foo.enable {
    environment.systemPackages = [ /* … */ ];
    networking.firewall.allowedTCPPorts = [ 8080 ];
  };
}
```

`mkIf cond value` includes `value` only when `cond` is true—**without** breaking merge when false (unlike a raw `if` that might remove structure incorrectly in some patterns).

Consumer pattern: you rarely write `mkIf` until the custom modules chapter; you **benefit** from modules that use it when you set `enable = true`.

Theory 5 — `lib.mkDefault` and priority

```
services.openssh.enable = lib.mkDefault true;
```

Helper	Idea
`mkDefault`	Low priority default; easy to override
`mkForce`	High priority; stomps others
`mkOverride n`	Numeric priority control

Consumer rule: prefer ordinary assignment first. Use `mkForce` only when you understand why a default wins.

```
# last resort
networking.hostName = lib.mkForce "lab";
```

Theory 6 — How to read a nixpkgs module

Step path

1. Find option on search.nixos.org/options

2. Note default and type
3. Open “Declared in” source file
4. Skim `options` then `config = mkIf cfg.enable`

Local source on a NixOS system

```
# approximate - paths vary by version
nix eval --raw 'nixpkgs#path'
# then browse nixos/modules/...
```

Or:

```
nix repl
:lf nixpkgs
# explore; or use ripgrep in a checkout
```

What to extract mentally

Question	Why
What does `enable` turn on?	Side effects
Which ports open?	Firewall
Which user created?	Permissions
What state dir?	Backup/persist
Hardening defaults?	Security

Theory 7 — Consumer anti-patterns

Anti-pattern	Prefer
Vendor-copy whole module	Set options; small local module
Patch store files	Declarative options / overlays later
`mkForce` everything	Understand merge
Ignore option descriptions	Read them
Edit generated unit files in `/etc`	Change Nix; rebuild

Theory 8 — `imports` as composition

```
{
  imports = [
    ./hardware-configuration.nix
    ./modules/ssh.nix
    ./modules/packages.nix
  ];
}
```

Each import is a module. Order rarely matters for merges (priorities do). the custom modules chapter authors one.

Worked examples bank

Example A — Read openssh options

Conceptually you will find:

```
options.services.openssh.enable = mkEnableOption "…";
config = mkIf cfg.enable { … };
```

Set as consumer:

```
services.openssh.enable = true;
services.openssh.settings.PasswordAuthentication = false;
```

Example B — Override a default carefully

```
# module might mkDefault some package set
# you add:
environment.systemPackages = [ pkgs.htop ];
# lists merge - usually no mkForce needed
```

Example C — When `mkForce` appears in blogs

Often silencing a conflict:

```
boot.kernelPackages = lib.mkForce pkgs.linuxPackages_latest;
```

Use sparingly; document why.

Lab 1 — Study three modules

Pick **three** from:

- `services.openssh`
- `services.timesyncd` or `time`
- `programs.git`
- `networking.firewall`
- `services.nginx` (even if unused)

For each, fill:

Module	enable option	2 interesting options	One side effect

Keep notes in `~/nixos-config/docs/modules-study.md`.

Lab 2 — Change behavior via options only

Without forking module source:

1. Tighten or loosen one SSH setting
2. Change timezone or NTP-related option
3. Adjust firewall ports

Rebuild; verify with `systemctl` / `ss` / `ssh`.

Lab 3 — Trace an option to source

Using search.nixos.org or local nixpkgs:

1. Choose `networking.firewall.enable`
2. Find declaring file
3. Read how firewall backend is selected

Write five lines on what `enable = true` actually does.

Lab 4 — Priority experiment (safe)

In a throwaway module snippet:

```
{ lib, ... }: {
  networking.hostName = lib.mkDefault "default-name";
}
```

And in main config:

```
networking.hostName = "lab";
```

Confirm final hostname is `lab`. Then reverse priorities with `mkForce` experimentally and reset to clean state.

Lab 5 — Consumer checklist

Before inventing a custom module tomorrow, ask:

1. Does an option already exist?
2. Can two options combine?
3. Am I fighting a default that `mkDefault` set?

Common gotchas

Symptom	Theory
Option does not exist	Typos / wrong release
Infinite recursion	`config` referring badly in custom modules
Silent override	Priorities / mkForce elsewhere
Copied outdated module	Diverges from nixpkgs
`mkIf` confusion	Study enable patterns first

Checkpoint

- ☐ Three modules studied with notes
- ☐ Behavior changed via options only
- ☐ Can explain options vs config
- ☐ Can explain mkIf / mkDefault roles
- ☐ Avoided forking nixpkgs modules

Commit

```
cd ~/nixos-config
git add .
git commit -m "day18: modules as consumer mkIf mkDefault literacy"
```

Writing custom modules

Writing custom modules

Goal: Author a **small local module** with `options` + `config`, import it into the host, gate behavior with `mkIf`, and fail early with an **assertion**.

Why this day exists

Stage II ends with a production-shaped host—not a 400-line `configuration.nix`. Custom modules are how you name intent (`myLab.monitoring.enable`) instead of scattering unrelated assignments.

Theory 1 — Minimal custom module shape

`modules/hello-lab.nix`:

```
{ config, lib, pkgs, ... }:
let
  cfg = config.myLab.hello;
in {
  options.myLab.hello = {
    enable = lib.mkEnableOption "hello lab demo module";

    package = lib.mkOption {
      type = lib.types.package;
      default = pkgs.hello;
      description = "Package providing the hello binary";
    };

    greetInterval = lib.mkOption {
      type = lib.types.str;
      default = "hourly";
      description = "systemd OnCalendar expression for greeting timer";
    };
  };

  config = lib.mkIf cfg.enable {
    environment.systemPackages = [ cfg.package ];
```

```
    systemd.services.my-lab-hello = {
      serviceConfig = {
        Type = "oneshot";
        ExecStart = "${cfg.package}/bin/hello";
      };
    };

    systemd.timers.my-lab-hello = {
      wantedBy = [ "timers.target" ];
      timerConfig = {
        OnCalendar = cfg.greetInterval;
        Persistent = true;
        Unit = "my-lab-hello.service";
      };
    };
  };
}
```

Theory 2 — `mkOption` and types (starter set)

Type	Use
`lib.types.bool`	Flags
`lib.types.str`	Strings
`lib.types.int`	Integers
`lib.types.package`	Packages
`lib.types.listOf lib.types.str`	Lists
`lib.types.attrsOf …`	Maps
`lib.types.port`	Ports
`lib.types.nullOr …`	Optional

Deep type catalogs come later. Today: **bool** + **package** + **str** is enough.

`mkEnableOption "doc"` → boolean option with standard description pattern.

Theory 3 — `cfg = config.…` pattern

Always bind:

```
let cfg = config.myLab.hello; in …
```

Then `mkIf cfg.enable` and read `cfg.package`. Avoid long repeated paths; avoid accidental recursion by not defining options in terms of themselves carelessly.

Theory 4 — Assertions

```
config = lib.mkIf cfg.enable {
  assertions = [
    {
      assertion = cfg.greetInterval != "";
      message = "myLab.hello.greetInterval must be non-empty";
    }
  ];
};
```

Assertions fail **evaluation** with a clear message—better than a mysterious empty timer.

Also available: `warnings = [ "…" ];` for non-fatal nags.

Theory 5 — Import into the host

`hosts/lab/configuration.nix`:

```
{
  imports = [
    ./hardware-configuration.nix
    ../../modules/hello-lab.nix
  ];

  myLab.hello.enable = true;
  # myLab.hello.greetInterval = "daily";
}
```

Or list modules in `flake.nix` `modules = [ … ]`.

Theory 6 — Namespace your options

Good	Risky
`myLab.*` / `myOrg.*`	Top-level inventing `services.foo` that collides
Clear enable flags	Global options without prefix

Use a personal/org prefix until you contribute upstream.

Theory 7 — What to extract first

Extract **cohesive features**, not random lines:

Good extraction	Weak extraction
"lab SSH hardening bundle"	One unrelated package
"tick timer feature"	Half a firewall + half a user
"admin tools profile"	Entire host config as one module forever

God modules become tomorrow's spaghetti.

Worked examples bank

Example A — Enable site

```
myLab.hello.enable = true;
```

Example B — Assertion trip

Set an invalid option deliberately; rebuild; read error; fix.

Example C — Package override

```
myLab.hello.package = pkgs.hello; # or another pkg with similar interface carefully
```

Lab 1 — Create module file

```
mkdir -p ~/nixos-config/modules
# write modules/hello-lab.nix (Theory 1)
```

Import + enable. Rebuild:

```
cd ~/nixos-config
sudo nixos-rebuild switch --flake .#lab
hello
systemctl list-timers | grep my-lab-hello || true
```

Lab 2 — Add a real assertion

Extend options with something checkable, e.g.:

```
port = lib.mkOption {
  type = lib.types.port;
  default = 22;
};
```

Assert `port != 0` or that firewall contains the port when a hypothetical service is enabled—keep it simple but real.

Lab 3 — Extract something from your host config

Identify 10–40 lines in `configuration.nix` (packages bundle, SSH settings, timer) and move into `modules/<feature>.nix` with an enable flag.

Before/after: host file should get shorter and more intentional.

Lab 4 — Disable path

```
myLab.hello.enable = false;
```

Rebuild; confirm packages/units from the module disappear (or stop being enabled). Re-enable.

Lab 5 — Document the module

`modules/README.md`:

```
# modules
## hello-lab
Options: myLab.hello.*
Purpose: …
```

Theory 8 — `mkDefault` vs bare assignment (producer side)

Inside your module's `config`, prefer leaving room for host overrides:

```
config = lib.mkIf cfg.enable {
  environment.systemPackages = [ cfg.package ];
  # If you set a global-ish value, consider mkDefault so hosts can override:
  # networking.firewall.allowPing = lib.mkDefault true;
};
```

Helper	Intent
bare `=`	Force (subject to merge priority rules)
`mkDefault`	Soft default
`mkForce`	Rare; win conflicts deliberately

the modules as consumer chapter covered consumer-side `mkDefault`; today you **emit** values carefully.

Theory 9 — Option descriptions are UX

```
greetInterval = lib.mkOption {
  type = lib.types.str;
  default = "hourly";
  example = "daily";
  description = ''
    systemd OnCalendar expression for the greeting timer.
    Prefer named calendars like hourly/daily while learning.
  '';
};
```

Future you reads `nixos-option` output under pressure—write for that moment.

Theory 10 — Module args you receive

```
{ config, lib, pkgs, modulesPath, ... }:
```

Arg	Use
`config`	Read final/merged values (`cfg = config.…`)
`lib`	`mkOption`, `mkIf`, `types`, …
`pkgs`	Packages for defaults and units
`modulesPath`	Import nixpkgs NixOS modules by path (advanced)

Do not add custom top-level args unless you wire `specialArgs` / `_module.args` intentionally (later).

Worked example D — warnings for soft policy

```
config = lib.mkIf cfg.enable {
  warnings =
    lib.optional (cfg.greetInterval == "minutely")
      "myLab.hello.greetInterval = minutely will spam logs on the lab host";
};
```

Lab 6 — `nixos-option` on your option

```
nixos-option myLab.hello.enable
nixos-option myLab.hello.greetInterval
```

If the option is missing, the module is not imported into the evaluated system.

Lab 7 — Priority conflict drill

Set the same option in host config and inside the module (with/without `mkDefault`). Rebuild and note which wins. Write one sentence on the result in `modules/README.md`.

Common gotchas

Symptom	Theory
Infinite recursion	options/config cycle
Option not found	Not imported
Type error	Wrong types.*
Module always on	Forgot mkIf
Colliding option names	Namespace
Assertion silent	Not inside merged config properly

Checkpoint

- ☐ Custom module with options + config
- ☐ `mkIf` on enable
- ☐ At least one assertion
- ☐ Imported into host flake
- ☐ Extracted real config into a feature module
- ☐ Enable/disable verified

Commit

```
cd ~/nixos-config
git add .
git commit -m "day19: first custom module options assertions"
```

Boot and hardware

Boot and hardware

Goal: Document and intentionally configure the lab's **boot loader**, **kernel packages**, and **firmware**—so recovery and hardware quirks are not mysteries.

> **!** Important
>
> Boot experiments can brick a bad disk layout. Prefer the **disposable VM**, snapshot first, and change one variable at a time.

Why this day exists

When generations fail to boot, the difference between "NixOS is broken" and "I know how to pick last generation / fix EFI" is this literacy. Hardware config is part of ownership.

Theory 1 — Boot chain (UEFI lab)

```
UEFI firmware
  → EFI system partition (ESP) files
  → boot loader (systemd-boot or GRUB)
  → kernel + initrd (from a generation)
  → stage-1 (find root, unlock, …)
  → stage-2 / systemd
```

NixOS generations typically install loader entries pointing at store paths for kernel/initrd.

Theory 2 — systemd-boot vs GRUB

Loader	Common on	Notes
systemd-boot	UEFI VMs/laptops	Simple, generation entries
GRUB	BIOS + UEFI	More knobs; legacy friendly

```
# systemd-boot (UEFI)
boot.loader.systemd-boot.enable = true;
boot.loader.efi.canTouchEfiVariables = true;

# GRUB example (do not enable both casually)
# boot.loader.grub.enable = true;
# boot.loader.grub.device = "/dev/vda";
```

Pick one model. Mixing without care is a classic footgun.

Generation UI

- systemd-boot: machine-id entries, often need key to show menu
- GRUB: menu with generation list

Document how **your** VM displays the menu (ESC, hold space, etc.).

Theory 3 — Kernel packages

```
boot.kernelPackages = pkgs.linuxPackages;          # default channel kernel
# boot.kernelPackages = pkgs.linuxPackages_latest;
# boot.kernelPackages = pkgs.linuxPackages_6_12; # example versioned attr - check 26.05
```

Choice	Trade-off
Default	Matches nixpkgs expectations
`_latest`	Newer hardware; more churn
Pinned series	Stability / specific needs

Kernel modules:

```
boot.kernelModules = [ ];
boot.initrd.availableKernelModules = [ /* often from hardware-config */ ];
boot.extraModulePackages = [ ];
```

`hardware-configuration.nix` usually fills initrd modules for disks/filesystems—do not delete blindly.

Theory 4 — Firmware

```
hardware.enableRedistributableFirmware = true;
# hardware.enableAllFirmware = true; # broader; licensing awareness
```

VMs often need little firmware; bare metal Wi-Fi/GPU might need redistributable firmware enabled.

Theory 5 — CPU / virt hardware bits

Common NixOS knobs:

```
# virtualisation-friendly examples (enable only if true for you)
# services.qemuGuest.enable = true;
# virtualisation.hypervGuest.enable = true;
```

Also: microcode updates on real Intel/AMD machines (`hardware.cpu.*.updateMicrocode`)—relevant on bare metal, less so on many VMs.

Theory 6 — `hardware-configuration.nix` discipline

Do	Don't
Import it	Hand-edit UUIDs casually
Regenerate after disk changes	Copy another machine's hardware file
Review initrd modules	Assume it never needs refresh

Regenerate carefully (live media or docs)—wrong root UUID → unbootable.

Theory 7 — Temporary boot parameters

Kernel params:

```
boot.kernelParams = [ "console=ttyS0" ]; # example serial console for some VMs
```

Useful for debugging; remove noise when done. One change per rebuild when experimenting.

Worked examples bank

Example A — Document current loader

```
bootctl status 2>/dev/null | head -40
ls /boot 2>/dev/null | head
ls /boot/loader/entries 2>/dev/null | head
```

Example B — Kernel identity

```
uname -r
nixos-version
ls /run/current-system/kernel
```

Example C — Safe param experiment

Add a harmless `boot.kernelParams` comment-level change only after snapshot; or set a documented console param if your hypervisor uses serial.

Lab 1 — Boot path document

Create `docs/boot.md` in the flake repo:

```
# Boot path - lab
- Firmware: UEFI / BIOS
- Loader: systemd-boot / GRUB
- ESP mount: /boot (options)
- Kernel package option: …
- uname -r: …
```

```
- How to open generation menu: …
- Snapshot name: …
```

Fill with real commands' output.

Lab 2 — Verify loader config matches reality

Compare flake `boot.loader.*` with `bootctl status` / GRUB presence. Fix mismatches.

Lab 3 — Kernel package awareness

1. Record current `boot.kernelPackages` (implicit default if unset).
2. Optionally try `linuxPackages_latest` **in a VM with snapshot**.
3. Reboot; `uname -r`; note difference.
4. Revert if not needed.

If you skip the change, still write *how* you would do it.

Lab 4 — Firmware decision

State in docs:

- VM only → firmware settings chosen
- Bare metal → enable redistributable firmware if devices need it

Rebuild if you change it.

Lab 5 — Recovery dry-run (no panic)

Without breaking the system:

1. List generations in bootloader docs
2. Rehearse verbally: "If black screen after upgrade, I will…"
3. Update the rebuild and generations chapter rollback card with boot-menu specifics

---

Theory 8 — `boot.loader.timeout` and generation menu

```
boot.loader.timeout = 3; # seconds; 0 may hide menu unless key held
```

For labs that practice rollback, a short non-zero timeout is friendlier than zero. Document the key to force the menu on your firmware (often Esc or hold Space for systemd-boot).

Theory 9 — Initrd and LUKS awareness (preview)

Encrypted roots need initrd unlock configuration. Even if your VM is plaintext:

Concept	Why note it
`boot.initrd.luks.devices`	Maps UUID → mapped name
Prompt vs keyfile	Physical vs automated unlock
Wrong UUID	Emergency shell at stage-1

Disko/impermanence (Stage VI) will revisit layout; today only name the risk.

Theory 10 — `hardware-configuration.nix` vs flake purity

`hardware-configuration.nix` is **machine-specific generated state**. It belongs in git for the lab host **or** is generated on first install and kept private per machine.

Approach	Trade-off
Commit lab VM hardware file	Easy clone of *this* VM
Generate per machine	Correct for fleet; clone drill needs care

Do not copy UUIDs between bare metal and VM casually.

Worked example D — QEMU guest agent (virt)

```
services.qemuGuest.enable = true; # if hypervisor is QEMU/KVM
```

Helps shutdown/reboots from host tooling. Enable only when accurate.

Lab 6 — Loader entries inventory

```
bootctl list 2>/dev/null | head -60
ls -la /boot/loader/entries 2>/dev/null | head
```

Map one entry to a NixOS generation number in `docs/boot.md`.

Lab 7 — Kernel param one-shot (snapshot)

With a VM snapshot:

```
boot.kernelParams = [ "loglevel=7" ]; # noisier logs; lab only
```

Rebuild, reboot, confirm `cat /proc/cmdline`, revert.

Common gotchas

Symptom	Theory
Two boot loaders enabled	Conflicts
`canTouchEfiVariables` false in some virt	May need alternate install path
Removed initrd modules	Disk not found
Latest kernel + guest tools mismatch	Virt drivers
Editing `/boot` by hand	Lost on rebuild

Checkpoint

- ☐ `docs/boot.md` complete
- ☐ Loader in flake matches reality
- ☐ Kernel identity recorded
- ☐ Firmware policy chosen
- ☐ Rollback card updated with boot menu
- ☐ Snapshot available before any kernel play

Commit

```
cd ~/nixos-config
git add .
git commit -m "day20: boot hardware documentation"
```

Filesystems and state

Filesystems and state

Goal: Inventory mounts and partitions; separate **store**, **state**, and **ephemeral** paths; understand the **impermanence idea** (persist deliberately) without implementing full impermanence yet.

Why this day exists

Nix makes the system closure rebuildable; it does **not** automatically make your data policy sane. Ops failures often come from "I rolled back the system but the database is gone" or the reverse. Label what must persist.

Theory 1 — Three layers of "the disk"

Layer	Examples	Rebuild replaces?
Store	`/nix/store`	Via new paths; GC collects dead
Config activation	much of `/etc`	Yes, from generation
State	`/var`, `/home`, databases	**No** (unless you design it)

```
/home  /var  (state)          ← backup & persist policy

/etc (mixed; mostly managed)

/nix/store (immutable pkgs)
```

Theory 2 — `fileSystems` options

From `hardware-configuration.nix` typically:

```
fileSystems."/" = {
  device = "/dev/disk/by-uuid/…";
  fsType = "ext4";
};

fileSystems."/boot" = {
  device = "/dev/disk/by-uuid/…";
  fsType = "vfat";
};
```

Field	Role
`device`	UUID/LABEL preferred over raw `/dev/sdX`
`fsType`	ext4, btrfs, xfs, vfat, zfs, …
`options`	mount flags (`noatime`, …)
`neededForBoot`	initrd critical mounts

Swap:

```
swapDevices = [{ device = "/dev/disk/by-uuid/…"; }];
```

Theory 3 — Why UUID/LABEL

Device names (`vda2`, `sda1`) can reorder. By-uuid is stable:

```
ls -l /dev/disk/by-uuid
lsblk -f
```

Theory 4 — State directories that matter

Path	Typical content
`/var/lib`	Service state (postgres, docker, …)
`/var/log`	Logs (sometimes ephemeral OK)
`/var/cache`	Regenerable caches
`/home`	User data, secrets, keys
`/root`	Root's home
`/etc/ssh` host keys	Machine identity (special)

Path	Typical content
`/nix`	Store + nix state database

When you enable services (Stage IV+), each service's state dir is part of your **persist list**.

Theory 5 — Impermanence idea (concept only)

Impermanence (pattern/tooling, the impermanence chapter deep dive): treat root as mostly ephemeral; **bind-mount or symlink** only chosen paths from a persistent volume.

```
boot → empty-ish root
     → restore/persist: /home, /var/lib/postgres, /etc/ssh, …
     → everything else regenerated by NixOS activation
```

Why people want it

Benefit	Meaning
Force declarative truth	Untracked snowflakes disappear
Cleaner systems	Less drift
Clear backup targets	Persist volume = backup unit

Costs

Cost	Meaning
Design work	Must list persist paths correctly
Host keys / secrets	Easy to forget; painful
Debugging	"Where did my file go?"

Today: you do **not** need to enable impermanence. You **do** need a written list of what *would* be persisted.

Theory 6 — Store is not a backup of state

```
nixos-rebuild switch
  → new system closure
    snapshot of /var/lib/postgresql
```

Backups are a separate story (the backups chapter). Generations help **software**; not **data**.

Theory 7 — tmpfs and ephemeral mounts (awareness)

```
# conceptual examples - do not apply casually
# boot.tmp.useTmpfs = true;
# fileSystems."/tmp" = { device = "tmpfs"; fsType = "tmpfs"; };
```

Useful later with impermanence. Understanding: not every path needs disk durability.

Worked examples bank

Example A — Mount inventory

```
findmnt -R /
lsblk -f
df -hT
```

Example B — What's big?

```
sudo du -xhd1 / 2>/dev/null | sort -h
sudo du -sh /nix/store /var /home 2>/dev/null
```

Example C — SSH host key paths

```
ls -l /etc/ssh/ssh_host_*
```

These are state even though under `/etc`.

Lab 1 — Full inventory table

In `docs/filesystems.md`:

Mountpoint	device/UUID	fsType	options	role (root/boot/state/…)

Fill from `findmnt` + `lsblk -f`.

Lab 2 — Persist classification

Label each:

Path	Persist?	Why	Backup?
`/nix/store`	rebuildable		no (or cache strategy)
`/home/alice`	yes		yes
`/var/log`	maybe		optional
`/var/lib`	mostly yes		yes
`/etc/ssh` host keys	yes	identity	yes
`/etc/nixos` if still used	prefer git flake		git

Be specific to **your** lab services.

Lab 3 — State created by your Stage II work

List state produced so far:

- user homes
- SSH host keys
- timer logs you wrote
- any DB? (probably not yet)

Lab 4 — Impermanence reading (light)

Skim what the impermanence chapter will cover (impermanence module concepts) from memory of Theory 5. Write five sentences: *Would I want impermanence on this lab VM? Why/why not?*

No implementation required.

Lab 5 — Danger checklist

Confirm you will **never**:

- `rm -rf /nix/store` as "cleanup"
- Format ESP casually
- Delete `/var/lib` because "Nix will regenerate" (it will not for app data)

Theory 8 — Btrfs / ZFS awareness without deploying them

You may see labs using btrfs snapshots or ZFS datasets. For Stage II:

FS	Persist story
ext4 default VM	Simple; backups = copy/dump
btrfs	Subvolumes + snapshots possible
ZFS	Datasets; often external to vanilla hardware-config

Do not switch root FS casually mid-lab. Understand that snapshot tools are about **state**, not store purity.

Theory 9 — `/nix` on its own mount (optional design)

Some installs put `/nix` on a dedicated volume:

Benefit	Cost
Reinstall root without losing store cache	More mount points to get right
Clear size accounting	Hardware-config complexity

Record whether your lab is single-root or split.

Theory 10 — Bind mounts as proto-impermanence

```
# conceptual - only if you knowingly add a data disk
# fileSystems."/var/lib/postgres" = {
#   device = "/data/postgres";
#   options = [ "bind" ];
# };
```

Bind mounts are how many impermanence setups pin state onto a durable dataset. Note the idea; implement later (the impermanence chapter).

Worked example D — `neededForBoot` meaning

```
fileSystems."/" = {
  device = "/dev/disk/by-uuid/…";
  fsType = "ext4";
  neededForBoot = true; # default true for /
};
```

Extra mounts required to reach multi-user (e.g. separate `/var`) may need `neededForBoot = true` or the system reaches emergency targets.

Lab 6 — What survives `nixos-rebuild switch`?

Create a marker:

```
echo "state-$(date -Is)" | sudo tee /var/tmp/day21-marker
echo "etc-try" | sudo tee /etc/day21-hand-edit 2>/dev/null || true
```

Rebuild switch. Check which markers remain. Note: hand-edited `/etc` files may be replaced by activation—another reason flake is SoT.

Lab 7 — Size budget sentence

From Lab 2 `du` output, write one budget sentence:

> "If I wipe the VM disk, I lose X (state) but can rebuild Y (system) from git+flake.lock."

Common gotchas

Symptom	Theory
Rolled back OS, data weird	Data generation
Copied hardware-config UUIDs wrong	Unbootable
Thought `/etc` fully ephemeral	Host keys, some static files
No backup of home	Lab loss on disk wipe
Impermanence without persist list	Instant footgun

Checkpoint

- ☐ Mount inventory table complete
- ☐ Persist classification written
- ☐ Store vs state explained in own words
- ☐ Impermanence idea summarized
- ☐ Host keys noted as state
- ☐ Docs committed

Commit

```
cd ~/nixos-config
git add .
git commit -m "day21: filesystems inventory persist vs store"
```

Activation Reload/Restart Deprecation

Activation Reload/Restart Deprecation

NixOS 26.05 deprecated **reloading or restarting systemd units from the NixOS activation script**, with removal targeted for **26.11**. The boring default is: **stop writing unit names into `/run/nixos/activation-restart-list` or `/run/nixos/activation-reload-list` from `system.activationScripts`; declare restart/reload behavior on the unit itself (`restartTriggers`, `reloadIfChanged`, `restartIfChanged`), and migrate other activation work to `systemd.tmpfiles.settings`, `ExecStartPre=`, or a oneshot service ordered with `sysinit-reactivation.target`.**

Activation scripts used to be the catch-all "run shell after every switch." That model serializes boot, fights parallelism, and is on a multi-release path toward removal (see the nixpkgs tracking effort). The 26.05 release notes call out the reload/restart side channel first because nixpkgs itself no longer uses it. Desk configs that still poke those list files will keep working through 26.05 with deprecation pressure, then break on 26.11.

Mental model

```
nixos-rebuild switch / boot


 activation script fragments     ← system.activationScripts.*
   (legacy: echo unit >> /run/nixos/activation-restart-list)


 switch-to-configuration
   reads restart/reload lists    ← DEPRECATED path (gone in 26.11)
   OR applies unit metadata      ← preferred: restartTriggers / reloadIfChanged


 systemd starts / reloads / restarts units
```

Piece	Role
`system.activationScripts`	Shell fragments run on activate (boot + switch); being phased out over several releases
`/run/nixos/activation-restart-list`	Legacy side channel: activation script asks STC to restart units
`/run/nixos/activation-reload-list`	Legacy side channel: activation script asks STC to reload units

Piece	Role
`systemd.services.<name>.restartTriggers`	Declared restart when listed store paths / strings change
`systemd.services.<name>.reloadIfChanged`	Prefer reload over restart when the unit file changes
`systemd.services.<name>.restartIfChanged`	Default-ish restart policy for unit file changes
`systemd.tmpfiles.settings`	Create dirs/files/permissions without an activation script
`serviceConfig.ExecStartPre`	Prep work bound to the service that needs it
`sysinit-reactivation.target`	Re-run oneshots on every activation (switch), not only cold boot
`switch-to-configuration` (26.05+)	Reloads (not restarts) when the only unit change is `ExecReload=`

Two timelines, one desk:

1. **Now (26.05):** stop using the activation restart/reload lists. Prefer unit options and systemd-native prep.
2. **Soon (multi-release):** shrink `system.activationScripts` itself — same migration ladder as the wiki: **tmpfiles → ExecStartPre → dedicated oneshot**.

Worked examples

Case 1: Find legacy restart/reload side channels on the desk

On a NixOS 26.05 host (or a flake eval against 26.05), hunt for the deprecated pattern before you bump toward 26.11:

```
# Search the desk flake / config tree
rg -n 'activation-restart-list|activation-reload-list|dry-activation-restart-list|dry-activa
rg -n 'system\.activationScripts' --glob '*.nix' .
```

Representative hit (anti-pattern — do not keep):

```
# legacy_restart_from_activation.nix  - remove before 26.11
{ pkgs, ... }:

{
  system.activationScripts.deskTicketReload = ''
    # Ask switch-to-configuration to restart a unit after activation.
    # DEPRECATED in 26.05; REMOVED in 26.11.
    if [ -d /run/nixos ]; then
      echo "desk-ticket.service" >> /run/nixos/activation-restart-list
    fi
  '';
```

```
}
```

Also inspect whether anything still echoes reload lists:

```
# After a dry run on a lab VM that still has the legacy fragment
sudo nixos-rebuild dry-activate --flake .#desk-workstation
# If present (legacy only):
sudo cat /run/nixos/dry-activation-restart-list 2>/dev/null || true
sudo cat /run/nixos/dry-activation-reload-list 2>/dev/null || true
```

Empty files / missing paths after migration are success. Treat any remaining echo into those lists as a 26.11 blocker ticket.

Case 2: Restart when a config file changes — use `restartTriggers`

Desk microservice whose unit should restart when its generated config store path changes. Save as `desk_ticket_restart.nix`:

```
# desk_ticket_restart.nix
{ config, pkgs, lib, ... }:

let
  ticketConf = pkgs.writeText "desk-ticket.conf" ''
    listen = 127.0.0.1:9471
    desk = family-lab
  '';
in
{
  systemd.services.desk-ticket = {
    description = "Desk ticket stub API";
    wantedBy = [ "multi-user.target" ];
    after = [ "network.target" ];

    # When the store path of ticketConf changes, STC restarts this unit.
    # No activation-script side channel required.
    restartTriggers = [ ticketConf ];

    serviceConfig = {
      Type = "simple";
      ExecStart = "${pkgs.python3}/bin/python3 -m http.server 9471 --bind 127.0.0.1";
      # Real desks pin WorkingDirectory / a real binary; stub keeps the lab cheap.
    };
  };

  # Ship the file somewhere inspectable (optional for the stub).
  environment.etc."desk-ticket/desk-ticket.conf".source = ticketConf;
```

```
}
```

Apply and verify the trigger is wired:

```
sudo nixos-rebuild switch --flake .#desk-workstation
systemctl show desk-ticket.service -p FragmentPath -p DropInPaths
# Bump the conf contents → rebuild → expect a restart of desk-ticket.service
sudo nixos-rebuild switch --flake .#desk-workstation
systemctl status desk-ticket.service --no-pager
```

Representative status after a conf-triggered restart:

```
desk-ticket.service - Desk ticket stub API
    Loaded: loaded (/etc/systemd/system/desk-ticket.service; enabled)
    Active: active (running) since …
```

`restartTriggers` is the boring replacement for "activation script noticed a file moved, please restart unit X."

Case 3: Prefer reload when only `ExecReload=` changed

NixOS 26.05 also tightened switch-to-configuration itself: when the **only** change to a unit is `ExecReload=`, STC **reloads** instead of restarting; removing `ExecReload=` alone no longer forces a restart. Declare reload-friendly units explicitly:

```
# desk_proxy_reload.nix
{ pkgs, ... }:

{
  systemd.services.desk-proxy = {
    description = "Desk reverse-proxy stub";
    wantedBy = [ "multi-user.target" ];
    reloadIfChanged = true;

    serviceConfig = {
      Type = "simple";
      ExecStart = "${pkgs.nginx}/bin/nginx -g 'daemon off;' -c /etc/desk-proxy/nginx.conf";
      ExecReload = "${pkgs.nginx}/bin/nginx -s reload";
      # Keep a minimal conf out of band or via environment.etc - lab sketch only.
    };
  };
}
```

Operator check after editing only the reload command (or a drop-in that only touches `ExecReload=`):

```
sudo nixos-rebuild dry-activate --flake .#desk-workstation
# Expect reload planning for desk-proxy, not a full stop/start, when that is the sole diff.
sudo nixos-rebuild switch --flake .#desk-workstation
journalctl -u desk-proxy.service -n 20 --no-pager
```

Do not paper over "I need a restart" by writing the unit name into an activation reload list. Set `reloadIfChanged` / `restartTriggers` / `restartIfChanged` on the service.

Case 4: Replace "mkdir + chown" activation with tmpfiles

Classic activation fragment that only prepares directories:

```
# legacy_cache_dir.nix  - anti-pattern
{
  system.activationScripts.deskCache = ''
    mkdir -p /var/cache/desk-ticket
    chown desk-ticket:desk-ticket /var/cache/desk-ticket
    chmod 0750 /var/cache/desk-ticket
  '';
}
```

Boring replacement (`desk_cache_tmpfiles.nix`):

```
# desk_cache_tmpfiles.nix
{
  users.users.desk-ticket = {
    isSystemUser = true;
    group = "desk-ticket";
  };
  users.groups.desk-ticket = { };

  systemd.tmpfiles.settings."desk-ticket" = {
    "/var/cache/desk-ticket".d = {
      mode = "0750";
      user = "desk-ticket";
      group = "desk-ticket";
    };
  };
}
```

Verify:

```
sudo nixos-rebuild switch --flake .#desk-workstation
systemd-tmpfiles --cat-config | rg desk-ticket
ls -ld /var/cache/desk-ticket
```

Representative listing:

```
drwxr-x--- 2 desk-ticket desk-ticket 4096 … /var/cache/desk-ticket
```

Case 5: Prep that belongs to one service → `ExecStartPre=`

When the script only exists so `some-service` can start cleanly, attach it to that service:

```nix
# desk_ticket_prestart.nix
{ pkgs, ... }:

let
  seed = pkgs.writeShellApplication {
    name = "desk-ticket-seed";
    runtimeInputs = [ pkgs.coreutils ];
    text = ''
      set -euo pipefail
      install -d -m 0750 /var/lib/desk-ticket
      if [ ! -f /var/lib/desk-ticket/seeded ]; then
        date -u +"seeded at %Y-%m-%dT%H:%M:%SZ" > /var/lib/desk-ticket/seeded
      fi
    '';
  };
in
{
  systemd.services.desk-ticket = {
    description = "Desk ticket stub with seed preStart";
    wantedBy = [ "multi-user.target" ];
    serviceConfig = {
      Type = "simple";
      ExecStartPre = [ "${seed}/bin/desk-ticket-seed" ];
      ExecStart = "${pkgs.python3}/bin/python3 -m http.server 9471 --bind 127.0.0.1";
      StateDirectory = "desk-ticket";  # prefer this when a plain state dir is enough
    };
  };
}
```

Prefer `StateDirectory=` / `CacheDirectory=` / `RuntimeDirectory=` when they already express the prep. Use `ExecStartPre=` for custom seed logic that still must run immediately before that unit.

Case 6: Work that must re-run on every switch → oneshot + `sysinit-reactivation.target`

Activation scripts run on boot **and** on `nixos-rebuild switch`. A oneshot wanted only by `multi-user.target` runs on boot, not necessarily again on every activation. For "run again after each switch," pull in `sysinit-reactivation.target` (desk pattern from current NixOS practice):

```
# desk_sync_oneshot.nix
{ pkgs, ... }:

{
  systemd.services.desk-lab-sync = {
    description = "Re-sync desk lab marker on every activation";
    wantedBy = [ "sysinit-reactivation.target" ];
    after = [ "local-fs.target" ];
    path = [ pkgs.coreutils ];

    serviceConfig = {
      Type = "oneshot";
      RemainAfterExit = true;
    };

    script = ''
      set -euo pipefail
      mkdir -p /var/lib/desk-lab
      date -u +"activated %Y-%m-%dT%H:%M:%SZ" >> /var/lib/desk-lab/activations.log
    '';
  };
}
```

Exercise:

```
sudo nixos-rebuild switch --flake .#desk-workstation
systemctl status desk-lab-sync.service --no-pager
wc -l /var/lib/desk-lab/activations.log
sudo nixos-rebuild switch --flake .#desk-workstation
wc -l /var/lib/desk-lab/activations.log   # expect +1 line
```

Representative log growth:

```
1 /var/lib/desk-lab/activations.log
2 /var/lib/desk-lab/activations.log
```

Order relative to user/group managers when needed (`after = [ "userborn.service" ];` or the equivalent on your pin) instead of inventing `activationScripts.*.deps` strings.

The trap

Trap A — keep echoing into `/run/nixos/activation-restart-list` "until 26.11." The list is already deprecated on 26.05; waiting until the removal release is how you get a surprise outage during an otherwise calm channel bump. Migrate on 26.05 while the old path still warns instead of fails.

Trap B — replace every activation fragment with a oneshot, skipping tmpfiles and `ExecStartPre=`. You pad the unit graph without gaining clarity. Follow the wiki ladder: **tmpfiles → ExecStartPre → dedicated service**.

Trap C — assume a oneshot `wantedBy = [ "multi-user.target" ]` re-runs on every `nixos-rebuild switch`. Cold boot activation. Use `sysinit-reactivation.target` when you need switch-time re-entry.

Trap D — disable `reloadIfChanged` / ignore STC's smarter reload behavior, then force restarts via activation lists "to be safe." Prefer declarative reload/restart metadata on the unit. Let 26.05 STC reload when only `ExecReload=` changed.

```
# Anti-pattern - side-channel restart
{
  system.activationScripts.forceNginx = ''
    echo "nginx.service" >> /run/nixos/activation-restart-list
  '';
}
```

```
# Boring fix - declare it on the unit
{
  systemd.services.nginx = {
    reloadIfChanged = true;
    restartTriggers = [ /* conf store paths */ ];
  };
}
```

The boring rule

- On NixOS **26.05**, treat activation-script **reload/restart lists** as deprecated; plan for **removal in 26.11**.
- Never add new `echo … >> /run/nixos/activation-*-list` fragments. Grep the desk flake and delete what you find.
- Express "restart when X changes" with `restartTriggers` / `reloadIfChanged` / `restartIfChanged` on the systemd service.
- Migrate other activation work in order: **`systemd.tmpfiles.settings` → `ExecStartPre=` → oneshot service**.
- For "run again on every switch," use a oneshot `wantedBy = [ "sysinit-reactivation.target" ]`, not a new activation script.
- Expect STC on 26.05 to **reload** (not restart) when the only unit change is `ExecReload=`.
- Track the broader activation-script deprecation (nixpkgs tracking + wiki); this chapter is the reload/restart cut that lands first.

Try this

1. On a disposable 26.05 VM flake, `rg` for `activation-restart-list`, `activation-reload-list`, and `system.activationScripts`. File a ticket for every out-of-tree hit.
2. Import Case 2's `desk_ticket_restart.nix`, switch, change the conf string, switch again, and confirm `desk-ticket.service` restarted via `journalctl -u desk-ticket`.
3. Convert one real `mkdir`/`chown` activation fragment on the lab host to `systemd.tmpfiles.settings` (Case 4). Diff `ls -ld` before/after.
4. Add Case 6's oneshot, run `nixos-rebuild switch` twice, and confirm `/var/lib/desk-lab/activations.log` gains a line each time.
5. Skim the Replacing activationScripts wiki page and the 26.05 release-notes bullet; paste the three-step ladder into the desk runbook.

Clevis LUKS Askpass in Initrd

Clevis LUKS Askpass in Initrd

NixOS **26.05** adds `boot.initrd.clevisLuksAskpass`: automatic LUKS unlock in the **systemd Stage 1** initrd by answering systemd password requests with **Clevis tokens already bound into the LUKS headers**. The boring default for a systemd-stage1 desk is: **bind Clevis (Tang and/or TPM2) on the LUKS device once, enable the module (and initrd networking if Tang), keep an interactive passphrase as fallback, and point `fileSystems."/".device` at `/dev/mapper/<name>` matching `boot.initrd.luks.devices`.** Prefer this header-token path over the older `boot.initrd.clevis.devices.*.secretFile` JWE-file approach for LUKS on systemd stage1.

Clevis (latchset/clevis) is a pluggable automated-decryption framework. Pins such as Tang (network server) and TPM2 unseal a secret without typing a passphrase when the pin conditions hold. The 26.05 module ships upstream `clevis-luks-askpass.path` / `.service` into the initrd so those bindings can answer `systemd-cryptsetup` prompts in parallel with a human typing at the console.

Mental model

```
boot (systemd Stage 1 initrd)


 cryptsetup.target
   wantedBy: clevis-luks-askpass.path


 systemd password request for LUKS



 clevis-luks-askpass.service     interactive passphrase prompt
   (reads Clevis token in        (always available if pin fails)
    LUKS header; Tang/TPM2…)


 /dev/mapper/<name> ready


 fileSystems."/".device = "/dev/mapper/<name>"
```

Piece	Role
`boot.initrd.clevisLuksAskpass.enable`	Install askpass path/service into systemd initrd
`boot.initrd.clevisLuksAskpass.package`	Clevis package (default `pkgs.clevis`)
`boot.initrd.clevisLuksAskpass.useTang`	Order askpass after/wants `network-online.target`
`boot.initrd.systemd.enable`	**Required** (assertion); default `true` on 26.05
`boot.initrd.network.enable` / `boot.initrd.systemd.network.enable`	Needed when Tang must be reached from initrd
`clevis luks bind -d /dev/… tang '…'` / `tpm2`	One-time bind: token lives in the LUKS header
Older `boot.initrd.clevis` + `devices.*.secretFile`	JWE file shipped into initrd secrets — different path
`fileSystems."/".device = "/dev/mapper/<name>"`	Avoid systemd passphrase/device timeout on 26.05 stage1
`systemctl default`	Replaces gone `cryptsetup-askpass` in stage1 emergency

Two desks, one rule:

1. **Header tokens + askpass (26.05+ systemd stage1 LUKS):** bind Clevis into the LUKS header; enable `clevisLuksAskpass`; networking only if Tang.
2. **JWE files (`boot.initrd.clevis`):** still valid for some ZFS/bcachefs and legacy layouts, but you manage a secret file path per device — not the boring default for plain LUKS on systemd stage1.

Worked examples

Case 1: Bind Clevis to a LUKS volume (prerequisite, once)

Clevis must already be bound **on the device** before the module can unlock it. From a live system (or installer) with the LUKS partition unlocked or with passphrase access:

```
# Identify the encrypted partition (lab example - use your real by-uuid / by-id)
lsblk -f
# Bind a Tang pin (server must be reachable from this host for the bind)
sudo clevis luks bind -d /dev/disk/by-uuid/AAAAAAAA-BBBB-CCCC-DDDD-EEEEEEEEEEEE \
  tang '{"url":"http://tang.lab.example:80"}'
# Optional second pin: TPM2 on the same slot set (machine-local)
sudo clevis luks bind -d /dev/disk/by-uuid/AAAAAAAA-BBBB-CCCC-DDDD-EEEEEEEEEEEE tpm2 '{}'
# Inspect bindings
sudo clevis luks list -d /dev/disk/by-uuid/AAAAAAAA-BBBB-CCCC-DDDD-EEEEEEEEEEEE
```

Representative listing:

```
1: tang '{"url":"http://tang.lab.example:80"}'
2: tpm2 '{}'
```

The token lives in the LUKS header (via LUKSMeta / Clevis metadata). There is **no** JWE file to copy into the Nix store for `clevisLuksAskpass`.

Case 2: Enable askpass on a Tang-backed desk

Save as `desk_clevis_tang.nix` and import from the host flake:

```nix
# desk_clevis_tang.nix
{ lib, ... }:

{
  # 26.05 default is true; keep it explicit on encrypted desks.
  boot.initrd.systemd.enable = true;

  boot.initrd.luks.devices.cryptroot = {
    device = "/dev/disk/by-uuid/AAAAAAAA-BBBB-CCCC-DDDD-EEEEEEEEEEEE";
  };

  # Mapper name must match the LUKS device attr name above.
  fileSystems."/" = {
    device = "/dev/mapper/cryptroot";
    fsType = "ext4";
  };

  boot.initrd.clevisLuksAskpass = {
    enable = true;
    useTang = true; # after/wants network-online.target in initrd
  };

  # Tang needs a route in Stage 1.
  boot.initrd.systemd.network.enable = true;
  # Or: boot.initrd.network.enable = true;  # legacy/scripted-adjacent path
}
```

Apply and confirm the units land in the initrd closure:

```
sudo nixos-rebuild boot --flake .#desk-workstation
# Inspect generated initrd systemd units (lab VM with systemd stage1)
nixos-rebuild build-vm --flake .#desk-workstation   # optional disposable check
systemctl --no-pager cat clevis-luks-askpass.path 2>/dev/null || true
# After reboot into the generation, from an emergency shell if needed:
# journalctl -b -u clevis-luks-askpass.service --no-pager
```

With Tang up, Stage 1 should unlock without typing. With Tang down, the **interactive passphrase prompt still works** — askpass runs in parallel; it does not replace the human path.

Case 3: TPM2-only (no Tang, no initrd network)

When the only pin is TPM2, leave `useTang = false` (default) and skip initrd networking:

```
# desk_clevis_tpm2.nix
{
  boot.initrd.systemd.enable = true;

  boot.initrd.luks.devices.cryptroot.device =
    "/dev/disk/by-uuid/AAAAAAAA-BBBB-CCCC-DDDD-EEEEEEEEEEEE";

  fileSystems."/".device = "/dev/mapper/cryptroot";

  boot.initrd.clevisLuksAskpass.enable = true;
  # useTang defaults to false - no network-online ordering, no Tang warning.
}
```

Bind first (`clevis luks bind … tpm2 '{}'`). Expect unlock on the same hardware; expect fallback passphrase if the TPM state no longer matches the binding (PCR policy changes, board swap).

Case 4: Eval-time assertion and Tang networking warning

The module **asserts** systemd initrd:

```
# Anti-pattern - will fail evaluation
{
  boot.initrd.systemd.enable = false; # scripted stage1 (deprecated; removal 26.11)
  boot.initrd.clevisLuksAskpass.enable = true;
}
```

Representative eval failure:

```
error: clevis-luks-askpass requires boot.initrd.systemd.enable = true
```

Tang without initrd networking emits a **warning** (does not hard-fail):

```
# Warns at eval: configure networking in initrd for Tang
{
  boot.initrd.systemd.enable = true;
  boot.initrd.clevisLuksAskpass = {
    enable = true;
    useTang = true;
```

```
  };
  # missing: boot.initrd.network.enable or boot.initrd.systemd.network.enable
}
```

Representative warning text from the module:

```
In order to use a Tang pinned secret you must configure networking in initrd
```

Case 5: Contrast — older JWE-file `boot.initrd.clevis`

Pre-askpass desks (and ZFS/bcachefs Clevis paths) often used a **JWE file** declared per device:

```
# legacy_clevis_jwe.nix  - still valid, not the 26.05 LUKS-header default
{
  boot.initrd.clevis = {
    enable = true;
    useTang = true;
    devices.cryptroot.secretFile = ./secrets/cryptroot.jwe;
  };
  boot.initrd.network.enable = true;
}
```

Approach	Where the secret lives	Stage 1 fit
`clevisLuksAskpass` (26.05)	Clevis token **in LUKS header** after `clevis luks bind`	systemd askpass watches password requests
`boot.initrd.clevis` + `secretFile`	JWE file in initrd secrets (`/etc/clevis/<name>.jwe`)	Extra utils / storePaths; device-keyed map

For a plain LUKS root on **systemd Stage 1**, pick **header bind + `clevisLuksAskpass`**. Keep the JWE module when your unlock story is still file-based or filesystem-module-driven.

Case 6: 26.05 LUKS / Stage 1 operator notes (passphrase timeout)

Independently of Clevis, NixOS 26.05 systemd stage1 LUKS desks must:

1. Set `fileSystems."/".device` to **`/dev/mapper/<name>`** matching `boot.initrd.luks.devices.<name>` (or add `"x-systemd.device-timeout=infinity"` on complex LVM-on-LUKS layouts).
2. Remember **`cryptsetup-askpass` is gone** — from an initrd emergency shell use **`systemctl default`** so passphrase prompts return; for SSH unlock helpers that pipe stdin, use **`ssh -o RequestTTY=force`**.

```
# Emergency Stage 1 shell (e.g. after rd.systemd.debug_shell)
systemctl default
```

```
# Not: cryptsetup-askpass
```

Kernel cmdline escape hatch before you can rebuild:

```
systemd.default_device_timeout_sec=infinity
```

The trap

Trap A — enable `clevisLuksAskpass` without `clevis luks bind`. The module only answers requests; it does not create bindings. Unbound headers → interactive passphrase every boot (or hang until timeout if mapper/root paths are also wrong).

Trap B — `useTang = true` without initrd networking. Eval warns; Tang is unreachable at unlock time; you type the passphrase forever and wonder why "automatic unlock" is documented. Set `boot.initrd.systemd.network.enable` (or `boot.initrd.network.enable`) and verify Tang from Stage 1.

Trap C — point `/` at the encrypted UUID while expecting mapper unlock. Systemd stage1 waits on a device unit that never becomes the mounted root. Align mapper name and `fileSystems."/".device`.

Trap D — ship a JWE via `boot.initrd.clevis.devices.*.secretFile` and also enable askpass "to be safe" without understanding which path owns unlock. Pick one story per device. Header-token askpass is the boring 26.05 LUKS path.

Trap E — treat askpass as removing the passphrase. Pins fail (Tang down, TPM PCR drift). Keep a strong passphrase slot; practice typing it once per lab cycle.

```
# Anti-pattern - Tang askpass, no initrd net, wrong root device
{
  boot.initrd.clevisLuksAskpass = { enable = true; useTang = true; };
  fileSystems."/".device = "/dev/disk/by-uuid/ENCRYPTED-PART-UUID"; # wrong for stage1
}
```

```
# Boring fix
{
  boot.initrd.systemd.enable = true;
  boot.initrd.systemd.network.enable = true;
  boot.initrd.luks.devices.cryptroot.device = "/dev/disk/by-uuid/ENCRYPTED-PART-UUID";
  fileSystems."/".device = "/dev/mapper/cryptroot";
  boot.initrd.clevisLuksAskpass = { enable = true; useTang = true; };
}
```

The boring rule

- On NixOS **26.05**, unlock LUKS in systemd Stage 1 with `boot.initrd.clevisLuksAskpass` after `clevis luks bind` (Tang and/or TPM2) on the real device.
- Require `boot.initrd.systemd.enable = true` (default); do not pair this module with scripted stage1.
- Set `useTang = true` **only when Tang is a pin**, and enable **initrd networking** (`boot.initrd.systemd.network.enable` or `boot.initrd.network.enable`).
- Keep the interactive passphrase; askpass runs **in parallel**, it does not delete the human path.
- Prefer header tokens over `boot.initrd.clevis` + `secretFile` for plain LUKS systemd-stage1 desks.
- Set `fileSystems."/".device = "/dev/mapper/<name>"` to match `boot.initrd.luks.devices.<name>`; use `systemctl default` instead of removed `cryptsetup-askpass`.
- Pin desks to current stable (**Nix 2.35+ / NixOS 26.05**) and re-check release notes when bumping channels.

Try this

1. On a disposable encrypted 26.05 VM, bind Tang (`Case 1`), import `desk_clevis_tang.nix` (`Case 2`), reboot with Tang up — confirm unlock without typing (`journalctl -b -u clevis-luks-askpass.service`).
2. Stop the Tang server (or unplug the lab NIC), reboot — confirm the interactive passphrase still unlocks.
3. Force-eval Case 4's assertion (`systemd.enable = false`) and read the error; restore systemd stage1.
4. Deliberately set `fileSystems."/".device` to the encrypted UUID, boot once, observe the timeout; fix to `/dev/mapper/cryptroot`.
5. Skim the 26.05 announcement, release notes (clevis-luks-askpass + LUKS stage1 bullets), and the module source `clevis-luks-askpass.nix`; paste the boring rule into the desk runbook.

Specialisations

Specialisations

Goal: Ship **boot-menu alternate configurations** from one flake host using NixOS `specialisation.<name>.configuration`, know how to activate them at runtime vs reboot, and when a specialisation beats a second container or VM.

NixOS **26.05** ("Yarara") keeps specialisations as first-class boot entries built beside the parent system. Channel Nix is typically **2.34.x**; many desks run Nix CLI **2.35+** via `nix.package` — specialisations do not depend on that split, but pin your notes to `nixos-version` / `nix --version` on the lab.

Mental model

```
nixosConfigurations.lab


 parent system closure  (default boot entry)

          specialisation.nvidia.configuration  →  extra closure + boot submenu entry
          specialisation.rescue.configuration →  …
          …


 /run/current-system/specialisation/<name>/bin/switch-to-configuration
```

Piece	Role
`specialisation.<name>.configuration`	Nested NixOS module(s) for the variant
`inheritParentConfig` (default `true`)	Merge on top of the parent; `false` = build from scratch
Boot loader entries	Choose the specialisation at boot (kernel/initrd differences need reboot)
`nixos-rebuild switch --specialisation <name>`	Activate that variant after a rebuild
`/run/current-system/specialisation/<name>/…`	Switch without a full rebuild when the closure already exists
Nested specialisations	**Ignored** — do not nest

Names must **not** contain `/`.

When to use vs containers / second hosts

Need	Prefer
Same machine, same disks, alternate kernel / drivers / desktop / "laptop on battery vs docked GPU"	**Specialisation**
Strong isolation, different OS, throwaway rootfs	**Container** or VM
Long-lived second identity with different networking policy	Separate `nixosConfigurations` host or VM
One-off package test	`nix shell` / vm test — not a specialisation

Specialisations still share the machine's hardware and usually much of the store; they are **configuration variants**, not security boundaries.

Worked examples

Case 1: GPU-off by default, GPU-on specialisation

Parent keeps heavy drivers off with `mkDefault` so the child can flip them:

```
# hosts/lab/specialisations-gpu.nix
{ lib, pkgs, ... }:

{
  # Parent desk: drivers off unless a specialisation overrides
  hardware.nvidia.modesetting.enable = lib.mkDefault false;

  specialisation.nvidia = {
    configuration = {
      system.nixos.tags = [ "nvidia" ];
      hardware.nvidia.modesetting.enable = true;
      # Add the rest of your NVIDIA module stack here as you already do on a GPU host
    };
  };
}
```

Import that file from the host module list. Rebuild:

```
sudo nixos-rebuild switch --flake .#lab
# Boot menu should offer a specialised entry tagged/named for nvidia
```

Runtime activate (userspace switch — **does not reload the kernel**):

```
sudo nixos-rebuild switch --flake .#lab --specialisation nvidia
# Or, if already built into the current system:
sudo /run/current-system/specialisation/nvidia/bin/switch-to-configuration switch
```

If the specialisation changes the **kernel** or early initrd modules, reboot and pick the boot entry; a userspace switch will not magically load the other kernel.

Case 2: Independent specialisation (`inheritParentConfig = false`)

```
# hosts/lab/specialisation-kiosk.nix
{ pkgs, ... }:

{
  specialisation.kiosk = {
    inheritParentConfig = false;
    configuration = {
      system.nixos.tags = [ "kiosk" ];
      imports = [ ./kiosk-baseline.nix ];
      users.users.kiosk = {
        isNormalUser = true;
        extraGroups = [ "video" ];
      };
      services.displayManager.autoLogin = {
        enable = true;
        user = "kiosk";
      };
      environment.systemPackages = with pkgs; [ chromium ];
    };
  };
}
```

Use this when the variant should **not** inherit desk services, VPN, or extra users from the parent. You own the full module closure — easy to under-specify boot/networking; test on a disposable VM first.

Case 3: Parent-only config (not pulled into children)

When `specialisation != {}`, you can attach options that apply to the **default** system only:

```
# modules/parent-only-opengl.nix
{ lib, config, pkgs, ... }:

{
  config = lib.mkIf (config.specialisation != { }) {
    # Example: keep a parent-only package set out of specialised closures
```

```
    environment.systemPackages = lib.mkBefore [ pkgs.pciutils ];
  };
}
```

If **no** specialisations exist, `config.specialisation != {}` is false — do not rely on this pattern to mean "always on the default host" in a config with zero specialisations.

Case 4: Inspect built specialisations

```
ls -la /run/current-system/specialisation/
readlink -f /run/current-system/specialisation/nvidia
nixos-version
nix --version
```

After `switch`, confirm tags:

```
cat /run/current-system/nixos-version
# specialised systems often carry system.nixos.tags in the banner / labels you set
```

Activation cheat sheet

Action	Command
Build parent + all specialisations	`sudo nixos-rebuild switch --flake .#lab`
Switch into one by name	`sudo nixos-rebuild switch --flake .#lab --specialisation <name>`
Switch using already-built path	`sudo /run/current-system/specialisation/<name>/bin/switch-to switch`
Test without adding a boot generation the same way you usually `test`	Prefer `switch-to-configuration test` on that path when experimenting
Kernel / initrd divergent	**Reboot** into the boot-menu entry

Plain `nixos-rebuild switch` without `--specialisation` returns you to the **parent** default — a common surprise after you had activated a child.

The trap

Trap A — treat specialisations like containers. They do not isolate failures or untrusted workloads.

Trap B — expect `switch --specialisation` to change the running kernel. Boot-menu reboot required for kernel/initrd.

Trap C — override parent booleans without `mkDefault` on the parent. The merge may not let the child win; set parent defaults with `lib.mkDefault`.

Trap D — nest specialisations. Nested entries are ignored; flatten the matrix or use separate hosts.

Trap E — huge Cartesian products. Each specialisation is another system closure to build and store; prefer two or three named modes, not a dozen.

Try this

1. Add a `specialisation.demo.configuration` that only sets `environment.variables.DEMO = "1";` and `system.nixos.tags`.
2. `nixos-rebuild switch`, then activate with `--specialisation demo` and `echo $DEMO` in a new login.
3. Change a kernel package in the specialisation on a **VM**, reboot into the entry, confirm `uname -r`.
4. Document on the rollback card: how to get back to the parent (`switch` without `--specialisation` + reboot if needed).

Sources

- NixOS Wiki — Specialisation
- nixpkgs `specialisation` option (select **26.05**)
- nixpkgs `specialisation.nix` module

Modular Services

Modular Services

Goal: Define a service **once, as a module**, and run several named instances of it from `system.services.<name>`. Ship a portable core (`process.argv`, `configData`) that a non-systemd service manager could also consume, and keep systemd-specific settings behind a guard. Compose services from other services, including modules that packages export as `pkgs.<pkg>.services.default`. Know which parts are still moving, and the four traps that bite first.

Version note: modular services first shipped in **NixOS 25.11**. The 26.05 manual still marks them **"Status: in development ... significant changes should be expected"**, and **user services are still "TBD"**. Everything below was evaluated against **nixos-26.05** (`26.05.20261008.7c8764b`) with **Nix 2.35.2**, and every unit file and error shown is real output from that evaluation. 26.05 adds two systemd-side options that 25.11 lacks, `systemd.mainExecStart` and `systemd.lib.escapeSystemdExecArgs`. Case 5 depends on them.

Mental model

Classic NixOS services are **options in a module**: `services.nginx.enable = true` gives you exactly one nginx, configured through a fixed option tree. A modular service turns that inside out: the **service is the module**. You import it into a named slot as many times as you like, and each slot becomes its own unit.

```
system.services.<name>  :: attrsOf (submodule, class "service")

          portable core (lib/services)          ← any service manager
             process.argv        command line, as a list (no shell)
             configData.<file>   text/source → path the manager exposes
             services.<sub>      sub-services this one owns
             meta.maintainers

          systemd layer (nixos/modules/system/service/systemd)   ← NixOS only
             systemd.service      → systemd.services."<name>"
             systemd.services.<x> → extra units, prefixed with <name>
             systemd.socket(s)
             systemd.mainExecStart, systemd.lib.*   (26.05+)

NixOS realises it as:
  systemd.services.<name>                ExecStart from process.argv
  environment.etc."system-services/<name>/<file>"   from configData
```

```
systemd.services.<name>-<sub>          for each sub-service
```

Piece	What it means in practice
`_class = "service"`	The module declares which evaluation it belongs to. Importing it anywhere else is an **error**, not a silent no-op (see The trap)
`process.argv`	A list, **not** a shell string. NixOS escapes it for systemd, so `%` and `$` reach the program literally
`configData.<file>.text` / `.source`	Files the service reads. On NixOS, `.path` resolves to `/etc/system-services/<name>/<file>`
`options ? systemd`	The portability guard. The systemd layer is present on NixOS and absent elsewhere
`services.<sub>`	Ownership, not ordering. A sub-service unit gets **no `After=`** on its parent unless you add it
NixOS defaults	`Type=simple`, `Restart=always`, `RestartSec=5`, `WantedBy=multi-user.target`, all overridable
Package-exported modules	`pkgs.<pkg>.services.default`, imported **inside** a `system.services.<name>` slot

When to use what (26.05):

Need	Prefer
One instance of a mature service with a big option tree (nginx, postgres)	Classic `services.<x>` module
Several instances of the same thing with different settings	**Modular service**, one slot per instance
Your own small daemon, written once and reused across hosts or managers	**Modular service** with a portable core
A unit with no reuse value	Plain `systemd.services.<x>` (see *systemd via Nix*)

Worked examples

Case 1: A portable service module

A status page: one static HTML file, served by `darkhttpd`. The page text and port are options. Nothing systemd-specific appears outside the guard.

```nix
# status-page.nix - a portable modular service: one static page, served by darkhttpd
# Non-module dependencies, passed with lib.modules.importApply
{ darkhttpd }:

# The service module
{ lib, config, options, ... }:
let
```

```
  cfg = config.statusPage;
in
{
  _class = "service";

  options.statusPage = {
    package = lib.mkOption {
      type = lib.types.package;
      default = darkhttpd;
      description = "Web server package.";
    };
    port = lib.mkOption {
      type = lib.types.port;
      description = "TCP port on 127.0.0.1.";
    };
    message = lib.mkOption {
      type = lib.types.str;
      default = "all systems nominal";
      description = "Text shown on the page.";
    };
  };

  config = {
    configData."index.html".text = ''
      <h1>${cfg.message}</h1>
    '';

    process.argv = [
      (lib.getExe cfg.package)
      (dirOf config.configData."index.html".path)
      "--port"
      (toString cfg.port)
      "--addr"
      "127.0.0.1"
    ];
  }
  // lib.optionalAttrs (options ? systemd) {
    # Only when the service manager is systemd (NixOS); ignored elsewhere.
    systemd.service.serviceConfig = {
      DynamicUser = true;
      Restart = "on-failure";       # the NixOS default is "always"
    };
  };
}
```

Three conventions here come straight from the nixpkgs contributor rules for modular services:

- **`importApply` for dependencies.** The file is a function that takes its packages (`{ darkhttpd`

`}:`) and returns the module. The module itself never sees `pkgs`, because a service manager other than NixOS might not have one.

- **`configData` instead of `pkgs.writeText` plus a hard-coded path.** The service asks for a file, and the manager decides where it lives. `config.configData."index.html".path` is how the module finds out.
- **`lib.optionalAttrs (options ? systemd)` instead of `mkIf`.** `mkIf` would still need the `systemd` option to *exist*. The attribute-set guard avoids even mentioning it when it doesn't.

Case 2: Two instances on one host

```nix
# flake.nix - one host, two instances of the same modular service
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { nixpkgs, ... }: {
    nixosConfigurations.box = nixpkgs.lib.nixosSystem {
      modules = [
        ({ lib, pkgs, ... }:
          let
            statusPage = lib.modules.importApply ./status-page.nix {
              inherit (pkgs) darkhttpd;
            };
          in
          {
            nixpkgs.hostPlatform = "x86_64-linux";
            boot.isContainer = true;          # no bootloader/kernel needed for eval
            system.stateVersion = "26.05";

            system.services.status-public = {
              imports = [ statusPage ];
              statusPage.port = 8080;

              # a sub-service: TLS in front of the page, owned by status-public
              services.tls = {
                imports = [ pkgs.ghostunnel.services.default ];
                ghostunnel = {
                  listen = "0.0.0.0:8443";
                  target = "127.0.0.1:8080";
                  keystore = "/run/keys/status-public.pem";
                  disableAuthentication = true;
                };
              };
            };

            system.services.status-ops = {
              imports = [ statusPage ];
```

```
            statusPage.port = 8081;
            statusPage.message = "maintenance at 22:00 UTC";
          };
        })
    ];
  };
};
}
```

`boot.isContainer = true` keeps this example evaluable without hardware configuration. On your lab host, add the two `system.services` blocks to the existing host module instead.

```
git add -A        # flakes only see tracked files
nix --version
nix eval --raw .#nixosConfigurations.box.config.system.nixos.version; echo
nix eval --json .#nixosConfigurations.box.config.systemd.services \
  --apply 's: builtins.filter (n: builtins.match "status-.*" n != null) (builtins.attrNames
nix eval --json .#nixosConfigurations.box.config.environment.etc \
  --apply 'e: builtins.filter (n: builtins.match "system-services/.*" n != null) (builtins.a
```

```
nix (Nix) 2.35.2
26.05.20261008.7c8764b
["status-ops","status-public","status-public-tls"]
["system-services/status-ops/index.html","system-services/status-public/index.html"]
```

One module gave **two** services plus a **third** unit for the sub-service, named `<parent>-<sub>`. Each instance has its own copy of `index.html` under `/etc/system-services/<name>/`.

Case 3: Read the units NixOS generated

Evaluate the unit file without switching anything:

```
out=$(nix build --no-link --print-out-paths \
  '.#nixosConfigurations.box.config.systemd.units."status-ops.service".unit')
cat "$out/status-ops.service"
```

```
[Unit]

[Service]
Environment="LOCALE_ARCHIVE=/nix/store/dc427sjs33dci797apv4lvsg1a69szn0-glibc-locales-2.42-1
Environment="PATH=/nix/store/bhv6xyvqw6gjs3j488gi1z6wdv18bdpa-coreutils-9.11/bin:/nix/store/
Environment="TZDIR=/nix/store/mkbbamycmw838fwxk21jz0nw3mhnmk16-tzdata-2026c/share/zoneinfo"
DynamicUser=true
ExecStart="/nix/store/qc2lzh2vk30sg00q6v08fd4gdl7mikmz-darkhttpd-1.17/bin/darkhttpd" "/etc/s
Restart=on-failure
```

```
RestartSec=5
Type=simple

[Install]
WantedBy=multi-user.target
```

Read it against the module:

- `ExecStart` is `process.argv`, with every element quoted. The `configData` directory resolved to `/etc/system-services/status-ops`.
- `DynamicUser=true` and `Restart=on-failure` came from the guarded systemd block. Our `Restart` beat the NixOS default (`always`) without `mkForce`, because the systemd layer sets `Type`, `Restart`, `RestartSec` and `wantedBy` with `lib.mkDefault`.
- `RestartSec=5`, `Type=simple` and `WantedBy=multi-user.target` are NixOS's modular-service defaults.

Check the generated file, and smoke-test the server outside systemd:

```
page=$(nix build --no-link --print-out-paths \
  '.#nixosConfigurations.box.config.environment.etc."system-services/status-ops/index.html".
srv=$(nix build --no-link --print-out-paths \
  '.#nixosConfigurations.box.config.system.services.status-ops.statusPage.package')
mkdir -p /tmp/status-ops && install -m 644 "$page" /tmp/status-ops/index.html
"$srv/bin/darkhttpd" /tmp/status-ops --port 18081 --addr 127.0.0.1 >/dev/null & sleep 0.5
curl -s http://127.0.0.1:18081/; kill %1
```

```
<h1>maintenance at 22:00 UTC</h1>
```

Note that you can reach into an instance's options (`system.services.status-ops.statusPage.package`) like any other option path. That is handy in tests and `nix repl`.

Case 4: A package-exported service as a sub-service

Packages can ship their own service module as `passthru.services.default`. In 26.05 the ones that do are `ghostunnel`, `git-pages`, `ktls-utils`, `php` and `snid` (25.11 had only `ghostunnel` and `php`). In Case 2, `status-public` imports ghostunnel as a **sub-service** to put TLS in front of the page:

```
out=$(nix build --no-link --print-out-paths \
  '.#nixosConfigurations.box.config.systemd.units."status-public-tls.service".unit')
cat "$out/status-public-tls.service"
```

```
[Unit]
After=network.target
Wants=network.target
```

```
[Service]
Environment="LOCALE_ARCHIVE=/nix/store/dc427sjs33dci797apv4lvsg1a69szn0-glibc-locales-2.42-1
Environment="PATH=/nix/store/bhv6xyvqw6gjs3j488gi1z6wdv18bdpa-coreutils-9.11/bin:/nix/store/
Environment="TZDIR=/nix/store/mkbbamycmw838fwxk21jz0nw3mhnmk16-tzdata-2026c/share/zoneinfo"
AmbientCapabilities=CAP_NET_BIND_SERVICE
DynamicUser=true
ExecStart="/nix/store/70zzr769qsi8jijkwjp2560f59p3fgg3-ghostunnel-1.8.4/bin/ghostunnel" "ser
LoadCredential=keystore:/run/keys/status-public.pem
Restart=always
RestartSec=5
Type=simple

[Install]
WantedBy=multi-user.target
```

The ghostunnel module did real systemd work: it loads the key through `LoadCredential=` and appends `--keystore=${CREDENTIALS_DIRECTORY}/…` **unquoted**, so systemd expands the variable. That is the `mainExecStart` pattern from Case 5. Look at what is **missing**, though: there is no `After=status-public.service`. "Sub-service" means *owned by*, not *ordered after*. The manual says a sub-service relationship "does not automatically create any other relationship between services". Add the ordering yourself if the proxy should not start before its backend:

```
# order-fix.nix - make the TLS sub-service start after, and stop with, its parent
let
  flake = builtins.getFlake (toString ./.);
  sys = flake.nixosConfigurations.box.extendModules {
    modules = [{
      system.services.status-public.services.tls.systemd.service = {
        after = [ "status-public.service" ];
        bindsTo = [ "status-public.service" ];
      };
    }];
  };
  unit = sys.config.systemd.services.status-public-tls;
in { inherit (unit) after bindsTo; }
```

```
nix eval --impure --json --file order-fix.nix
```

```
{"after":["status-public.service","network.target"],"bindsTo":["status-public.service"]}
```

In a real host, put those two lines directly into the `services.tls` block in `flake.nix`. `extendModules` is used here only so that the example can amend Case 2 without editing it.

Case 5: systemd specifiers and variables (`mainExecStart`, 26.05+)

`process.argv` is portable, so NixOS escapes it completely: `%` becomes `%%` and `$` becomes `$$`. When you *want* systemd to substitute something (the unit name `%n`, a credentials directory, an environment variable), build `systemd.mainExecStart` from the escaped argv and append the unescaped part:

```nix
# trap-escape.nix - what happens to % and $ in process.argv
let
  flake = builtins.getFlake (toString ./.);
  sys = flake.nixosConfigurations.box.extendModules {
    modules = [
      ({ pkgs, ... }: {
        system.services.hello = {
          process.argv = [ "${pkgs.coreutils}/bin/echo" "unit=%n" "home=$HOME" ];
        };
        system.services.hello2 = { config, ... }: {
          process.argv = [ "${pkgs.coreutils}/bin/echo" ];
          systemd.mainExecStart =
            config.systemd.lib.escapeSystemdExecArgs config.process.argv
            + " unit=%n home=$HOME";
        };
      })
    ];
  };
in
map (n: "${n}: ${builtins.head sys.config.systemd.services.${n}.serviceConfig.ExecStart}")
  [ "hello" "hello2" ]
```

```
nix eval --impure --json --file trap-escape.nix | python3 -m json.tool
```

```
[
    "hello: \"/nix/store/bhv6xyvqw6gjs3j488gi1z6wdv18bdpa-coreutils-9.11/bin/echo\" \"unit=%
    "hello2: \"/nix/store/bhv6xyvqw6gjs3j488gi1z6wdv18bdpa-coreutils-9.11/bin/echo\" unit=%n
]
```

`hello` will print the literal text `unit=%n home=$HOME`. In `hello2`, systemd itself substitutes `%n` with the unit name (`hello2.service`) and expands `$HOME` from the service's environment when the unit starts. The first form is right for arguments that come from users or settings, because it can never be misinterpreted. Use the second only for the specific parts that need substitution, and only behind the `options ? systemd` guard, since other service managers have no `%n`.

The trap

Four ways this goes wrong on a first attempt. All four were reproduced on 26.05.

1. Importing a service module as a NixOS module. Service modules go *inside* a `system.services.<name>` slot. Put one in the host's top-level `imports` (or `modules = [ … ]`) and the module system refuses:

```
# trap-class.nix - importing a service module as if it were a NixOS module
let
  flake = builtins.getFlake (toString ./.);
  pkgs = flake.inputs.nixpkgs.legacyPackages.x86_64-linux;
  sys = flake.nixosConfigurations.box.extendModules {
    modules = [ pkgs.ghostunnel.services.default ];
  };
in
sys.config.system.build.toplevel.drvPath
```

```
nix eval --impure --raw --file trap-class.nix 2>&1 | tail -n 6
```

```
       error: The module `/nix/store/1ffpr0yhnxfkyj467rjjxfzsmc9564qb-source/pkgs/by-name/gh

       Help:
       - Ensure that you are importing the correct module.
       - Verify that the module's `_class`, "service" matches the expected `class` "nixos".
       - If you are using a custom class, make sure it is correctly defined and used consist
```

That error is `_class` doing its job. Set `_class = "service"` in your own service modules too, so the same mistake fails loudly instead of quietly defining nonsense options.

2. Expecting `%` or `$` in `process.argv` to be expanded. Case 5: they are escaped. Use `systemd.mainExecStart` (26.05+). On 25.11 that option does not exist, and the workaround is a wrapper script.

3. Overriding the package and getting the original binary. A package-exported module may pin the package it came from in a way that ignores your override. In 26.05, `ghostunnel`'s `passthru.services.default` sets `ghostunnel.package = ghostunnel; # FIXME: finalAttrs.finalPackage`:

```
# trap-override.nix - a modular service from an overridden package
let
  flake = builtins.getFlake (toString ./.);
  sys = flake.nixosConfigurations.box.extendModules {
    modules = [
      ({ pkgs, ... }:
        let
          patched = pkgs.ghostunnel.overrideAttrs (old: {
            postInstall = (old.postInstall or "") + "\n# local patch\n";
          });
        in {
          system.services.gt = {
```

```
            imports = [ patched.services.default ];
            ghostunnel = {
              listen = "0.0.0.0:9443";
              target = "127.0.0.1:9000";
              keystore = "/run/keys/gt.pem";
              disableAuthentication = true;
            };
          };
          environment.etc."patched-ghostunnel".text = "${patched}";
        })
    ];
  };
in {
  patched = sys.config.environment.etc."patched-ghostunnel".text;
  used = sys.config.system.services.gt.ghostunnel.package.outPath;
}
```

```
nix eval --impure --json --file trap-override.nix | python3 -m json.tool
```

```
{
    "patched": "/nix/store/bxmg1sxzfpbix6qh7fd9j9j4rdm5zig8-ghostunnel-1.8.4",
    "used": "/nix/store/70zzr769qsi8jijkwjp2560f59p3fgg3-ghostunnel-1.8.4"
}
```

The service imported from `patched` still runs the **unpatched** store path. Setting `ghostunnel.package = patched;` next to the import fails with "defined multiple times while it's expected to be unique", because the package's definition is a normal-priority value, not a default. You need `ghostunnel.package = lib.mkForce patched;`. With that line (and `lib` added to the module arguments), the same evaluation reports `"used": "/nix/store/bxmg1sxzfpbix6qh7fd9j9j4rdm5zig8-ghostunnel` the patched build. In your own packages, export services with `finalAttrs.finalPackage` so overrides flow through.

4. Assuming sub-services are ordered. Case 4: `status-public-tls` has no `After=` on `status-public`. Add `after`/`bindsTo` (or `requires`) explicitly when the relationship matters.

Try this

1. Add a third instance, `status-staging`, on port 8082, with only two lines of new configuration. Confirm the new unit and `/etc` entry with the `nix eval` commands from Case 2.
2. Give `status-page.nix` a `statusPage.extraArgs` list option, append it to `process.argv`, and pass an argument containing `%`. Check the generated `ExecStart`, and explain why it is safe.
3. Rewrite `status-ops` to log its unit name at start-up using `systemd.mainExecStart` and `%n`. Keep it behind `options ? systemd`.
4. Write a NixOS VM test (`pkgs.testers.runNixOSTest`) that boots a node with `status-public` and `status-ops`, waits for both units, and `curl`s ports 8080 and 8081.

The nixpkgs contributor guide requires such a test for every modular service in nixpkgs.

5. Port one of your own single-instance `systemd.services.<x>` definitions (from *systemd via Nix*) to a modular service. Which settings stayed portable, and which had to move behind the guard?

Sources

- NixOS manual (26.05), *Modular Services*: status, portability, composition and ownership, the portable and systemd-specific option references: https://nixos.org/manual/nixos/stable/#modular-services
- nixpkgs contributor guide, `nixos/README-modular-services.md` (VM test, `meta.maintainers`, `options ? systemd`, `_class`, `finalAttrs.finalPackage`): https://github.com/NixOS/nixpkgs/blob/nixos-26.05/nixos/README-modular-services.md
- Portable option declarations: https://github.com/NixOS/nixpkgs/blob/nixos-26.05/lib/services/service.nix; systemd layer: https://github.com/NixOS/nixpkgs/blob/nixos-26.05/nixos/modules/system/service/systemd/service.nix
- Original design and feedback thread, NixOS/nixpkgs PR #372170: https://github.com/NixOS/nixpkgs/pull/372170
- `ghostunnel` service module and package export (26.05): https://github.com/NixOS/nixpkgs/blob/nixos-26.05/pkgs/by-name/gh/ghostunnel/service.nix
- systemd: `systemd.service(5)` *Command lines* and `systemd.unit(5)` *Specifiers*: https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html

systemd in Stage 1: the 26.05 Initrd Default

systemd in Stage 1: the 26.05 Initrd Default

Goal: Know what changed when NixOS 26.05 made the **systemd-based initrd** the default for stage 1. Move old `boot.initrd.*Commands` shell hooks into real initrd units, and catch the two migration traps that evaluation *cannot* catch for you (`/dev/root` and LUKS device timeouts). The scripted stage 1 still exists behind `boot.initrd.systemd.enable = false`, but it is deprecated and **scheduled for removal in 26.11**. Treat this chapter as an upgrade item for this release cycle, not a someday item.

Version note: everything below was evaluated against **nixos-26.05** (`26.05.20261010.6fcc160`) with **Nix 2.35.3**. All evaluation output is pasted from those runs. The initrd's systemd is **260.5** on this channel.

Mental model

Stage 1 is the small root filesystem the kernel unpacks from the initrd. It has one job: find the real root, mount it at `/sysroot`, and hand over to stage 2 (the system you log into). NixOS has had two implementations of that job:

```
scripted stage 1 (  25.11 default)          systemd stage 1 (26.05 default)

/init = one generated bash script           /init = systemd, PID 1 in the initrd
  preLVMCommands                              units ordered by bootup(7):
  preDeviceCommands                             systemd-udevd → cryptsetup → *.device
  postDeviceCommands         your hooks        → sysroot.mount → initrd-fs.target
  postMountCommands            are spliced       → initrd-nixos-activation
  (switch_root by hand)        into the script   → initrd-switch-root
                                                 your code = boot.initrd.systemd.services.*
```

The scripted initrd ran your strings *in sequence* inside one script. The systemd initrd has no sequence. It has a **dependency graph**, and your code becomes a unit with `Before=` / `After=` edges inside that graph. Most migration work is choosing the right edge.

Worked examples

The lab is one flake with several host variants. None of them is booted. Every case is proved at evaluation time, which is where NixOS reports most of its stage-1 decisions.

```
# flake.nix - minimal host to inspect the stage-1 initrd that nixos-26.05 builds
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { nixpkgs, ... }: {
    nixosConfigurations.box = nixpkgs.lib.nixosSystem {
      modules = [ ./host.nix ];
    };
    nixosConfigurations.trap = nixpkgs.lib.nixosSystem { modules = [ ./host.nix ./trap-scrip
    nixosConfigurations.fixed = nixpkgs.lib.nixosSystem { modules = [ ./host.nix ./initrd-se
    nixosConfigurations.devroot = nixpkgs.lib.nixosSystem { modules = [ ./host.nix ./trap-de
    nixosConfigurations.scripted = nixpkgs.lib.nixosSystem { modules = [ ./host.nix ./opt-ou
  };
}
```

```
# host.nix - the smallest bootable-looking host; nothing about initrd is set here
{ ... }: {
  nixpkgs.hostPlatform = "x86_64-linux";
  boot.loader.systemd-boot.enable = true;
  fileSystems."/" = { device = "/dev/disk/by-label/nixos"; fsType = "ext4"; };
  system.stateVersion = "26.05";
}
```

Case 1: Confirm what you are running

Do not assume the default from memory. Ask the configuration:

```
$ nix eval .#nixosConfigurations.box.config.system.nixos.label
"26.05.20261010.6fcc160"
$ nix eval .#nixosConfigurations.box.config.boot.initrd.systemd.enable
true
$ nix eval .#nixosConfigurations.box.config.boot.initrd.systemd.services --apply builtins.at
[ "dbus" "initrd-find-nixos-closure" "initrd-nixos-activation" "initrd-parse-etc" "initrd-sw
```

The `initrd-*` services are the NixOS-specific part. They locate the system closure on the new root, run activation against `/sysroot`, and switch root. Everything else in the boot is upstream systemd behaviour, as documented in `bootup(7)`, and that is the point of the change. Mounting, LUKS prompts, udev and timeouts now follow the same rules as on any systemd distribution.

Case 2: An old hook fails evaluation (the good failure)

A config written for the scripted initrd often carries a hook like this:

```
# trap-scripted.nix - a scripted-stage-1 hook carried over from an older config
{ ... }: {
  boot.initrd.postDeviceCommands = ''
```

```
    echo "waiting for slow disk"; sleep 2
  '';
}
```

On 26.05 it does not silently vanish. It stops the build:

```
$ nix eval .#nixosConfigurations.trap.config.system.build.toplevel.drvPath
error:
Failed assertions:
- systemd stage 1 does not support `boot.initrd.postDeviceCommands`. Instead, create systemd

    Definitions:
    - /nix/store/…-source/trap-scripted.nix
```

The same assertion covers `preFailCommands`, `preDeviceCommands`, `preLVMCommands`, `postDeviceCommands` and `postMountCommands`. Grep for all of them before you upgrade:

```
$ grep -rnE 'boot\.initrd\.(preFail|preDevice|preLVM|postDevice|postMount)Commands' /etc/nix
```

Case 3: Rewrite the hook as an initrd unit

The replacement uses the stage-2 `systemd.services` API under `boot.initrd.systemd.services`. The work is choosing edges. "After devices, before root is mounted" becomes `Before=sysroot.mount`:

```
# initrd-service.nix - the same hook as a real unit inside the systemd initrd
{ pkgs, ... }: {
  boot.initrd.systemd.services.slow-disk-settle = {
    description = "Give the slow disk two seconds before root is mounted";
    wantedBy = [ "initrd.target" ];
    before = [ "sysroot.mount" ];
    after = [ "systemd-udev-settle.service" ];
    unitConfig.DefaultDependencies = false;
    serviceConfig.Type = "oneshot";
    script = ''
      echo "waiting for slow disk"; sleep 2
    '';
  };
}
```

Read back the unit NixOS will put into the initrd:

```
$ nix eval --raw '.#nixosConfigurations.fixed.config.boot.initrd.systemd.units."slow-disk-se
[Unit]
After=systemd-udev-settle.service
Before=sysroot.mount
DefaultDependencies=false
```

```
Description=Give the slow disk two seconds before root is mounted

[Service]
Environment="LOCALE_ARCHIVE=/nix/store/dc427sjs…-glibc-locales-2.42-100/lib/locale/locale-ar
Environment="TZDIR=/nix/store/mkbbamyc…-tzdata-2026c/share/zoneinfo"
ExecStart=/nix/store/yb7a6i88…-unit-script-slow-disk-settle-start/bin/slow-disk-settle-start
Type=oneshot

[Install]
WantedBy=initrd.target
```

Two facts make this safe. First, the generated script is copied into the initrd automatically. Second, `sleep` resolves, because coreutils is in the initrd's default `PATH`:

```
$ nix eval .#nixosConfigurations.fixed.config.boot.initrd.systemd.storePaths \
    --apply 'ps: builtins.filter (p: builtins.match ".*slow-disk.*" (toString (p.source or p
[ { dlopen = { features = [ ]; usePriority = "recommended"; }; enable = true; source = "/nix
$ nix eval .#nixosConfigurations.fixed.config.boot.initrd.systemd.initrdBin --apply 'map (p:
[ "lvm2-2.03.39" "e2fsprogs-1.47.4" "coreutils-9.11" "systemd-260.5" "kmod-31" "bash-interac
```

Any *other* binary your hook calls must be listed in `boot.initrd.systemd.initrdBin` (on `PATH`) or `boot.initrd.systemd.storePaths` (copied only). The initrd is not your system closure.

`DefaultDependencies=false` matters. Without it, systemd adds `After=sysinit.target basic.target` and the implicit shutdown conflicts. In the initrd those can create an ordering cycle with mounts that `basic.target` waits for. Early-boot units almost always want it off.

Case 4: The trap evaluation will not catch: `/dev/root`

The scripted initrd created a `/dev/root` symlink from `root=` on the kernel command line. The systemd initrd does not. A config that names it still evaluates cleanly:

```
# trap-devroot.nix - a root filesystem named by the scripted-initrd symlink
{ lib, ... }: { fileSystems."/".device = lib.mkForce "/dev/root"; }
```

```
$ nix eval .#nixosConfigurations.devroot.config.system.build.toplevel.drvPath
"/nix/store/nc5n9p8y…-nixos-system-nixos-26.05.20261010.6fcc160.drv"
```

No warning and no assertion, and on the next boot `sysroot.mount` waits for a device that never appears. Name the root with `/dev/disk/by-label/…`, `/dev/disk/by-uuid/…` or `/dev/mapper/…`, then check:

```
$ nix eval --json .#nixosConfigurations.box.config.fileSystems --apply 'fs: builtins.mapAttr
{"/":"/dev/disk/by-label/nixos"}
```

The LUKS case is in the same family. The release notes say `fileSystems."/".device` should be `/dev/mapper/<name>`, matching `boot.initrd.luks.devices.<name>`. Otherwise systemd's default device timeout can expire while you are still typing the passphrase. For LVM on LUKS, add `"x-systemd.device-timeout=infinity"` to the root's `options`. If you are already locked out, `systemd.default_device_timeout_sec=infinity` on the kernel command line gets you in.

Case 5: The escape hatch, and its warning

```
# opt-out.nix - reverting to the scripted stage 1 (deprecated on 26.05)
{ ... }: { boot.initrd.systemd.enable = false; }
```

```
$ nix eval .#nixosConfigurations.scripted.config.system.build.toplevel.drvPath
evaluation warning: Scripted initrd is deprecated and scheduled for removal in 26.11. See th
"/nix/store/cr5jdspc…-nixos-system-nixos-26.05.20261010.6fcc160.drv"
```

Use it to get a machine booting tonight. Do not use it as a destination. If you keep it, give the line a dated comment and open an issue to remove it before you move to 26.11.

The trap

The dangerous migrations are the ones that **evaluate fine**. Hooks fail loudly (Case 2), and that is the easy part. `/dev/root`, a LUKS root named by UUID instead of `/dev/mapper/<name>`, and remote unlock scripts that call `cryptsetup-askpass` all build successfully and fail at the console. (`cryptsetup-askpass` no longer exists. Use `systemctl default`, and `ssh -o RequestTTY=force` if you pipe the passphrase over SSH.) Your rollback is the previous generation in the boot menu, so keep it until the new initrd has booted at least once *on that hardware*.

The boring rule

Before switching a host to 26.05: grep for `*Commands` hooks and port them to `boot.initrd.systemd.services` with explicit `Before=`/`After=` and `DefaultDependencies=false`. Name every root and LUKS device by a stable path. Boot once with the old generation still in the menu. Do not ship `boot.initrd.systemd.enable = false` without a removal date.

Try this

1. Change Case 3 to `before = [ "initrd-root-device.target" ]` and read the unit again. Which `bootup(7)` milestone does that correspond to, and does it still run before `sysroot.mount`?
2. Add `pkgs.util-linux` to `boot.initrd.systemd.initrdBin`, call `blkid` from the hook, and confirm the binary appears in the initrd's `PATH`.

3. Build `config.system.build.initialRamdisk` for `box` and `scripted` and compare their sizes with `du -h`.
4. In a NixOS VM test (`nixosTest`), boot `fixed` and grep `journalctl -b -u slow-disk-settle` from the test script.

Sources

- NixOS 26.05 release notes, "Stage 1 (a.k.a. initrd) is now based on systemd by default": `nixos/doc/manual/release-notes/rl-2605.section.md` in nixpkgs `nixos-26.05`, https://nixos.org/manual/nixos/stable/release-notes
- nixpkgs `nixos/modules/system/boot/systemd/initrd.nix` (default and assertions), branch `nixos-26.05`
- `bootup(7)`, "Bootup in the Initrd": https://www.freedesktop.org/software/systemd/man/latest/bootup.html
- `systemd.unit(5)`, `DefaultDependencies=`: https://www.freedesktop.org/software/systemd/man/latest/systemd.unit.html

Locales: the Locale Archive, i18n.extraLocales, and the Missing-Locale Crash

Locales: the Locale Archive, `i18n.extraLocales`, and the Missing-Locale Crash

Goal: Know exactly which locales a NixOS host can load and where they come from. Add regional formats without guessing at option names. Diagnose the crash a program hits when it is told to use a locale the system never built. And when the locale you need does not exist in glibc at all (here an invented-but-reasonable `en_AT`: English text, Austrian formats), compile it yourself the way nixpkgs does, without rebuilding half the package set.

Version note: everything below was evaluated and built against **nixos-26.05** (`26.05.20261013.a033e66`, glibc **2.42**) with **Nix 2.35.3**. All evaluation, build, and probe output is pasted from those runs. The probe binaries ran from the store on an x86-64 Linux box. No VM was booted, so the "on a running host" commands in the trap section are the standard checks, not captured output.

Mental model

On most distributions, locales are files under `/usr/lib/locale`, created by `locale-gen`. NixOS has no such directory. glibc in nixpkgs is patched (`nix-locale-archive.patch`) to open its locale archive from a list of places, in order:

```
glibc setlocale(LC_ALL, "")  →  which archive?   (nixpkgs glibc, open_locale_archive)
  1. $LOCALE_ARCHIVE_2_27
  2. $LOCALE_ARCHIVE                  NixOS sets this for services and for sessions
  3. /run/current-system/sw/lib/locale/locale-archive
  4. /usr/lib/locale/locale-archive
  5. glibc's compiled-in path

the archive itself = i18n.glibcLocales
  default: pkgs.glibcLocales.override { locales = config.i18n.supportedLocales; }
  supportedLocales (hidden, computed) =
      "C.UTF-8/UTF-8" + "en_US.UTF-8/UTF-8"
    + i18n.defaultLocale               /UTF-8
    + every value in i18n.extraLocaleSettings (LC_TIME, LC_NUMERIC, …)
    + i18n.extraLocales                (anything else a program needs)
```

So the archive holds the locales the configuration *names*, and nothing else. The environment variables (`LANG`, `LC_*`) are a separate thing. They are a *request*, and a process can carry any value: from `/etc/locale.conf`, from a desktop's region settings, from a wrapper, from an SSH client if `AcceptEnv` forwards it. When the request names a locale the archive lacks, glibc does not

fail the process. It fails `setlocale` and leaves the process in the `C` locale. The rest of this chapter is about what happens next.

Two 26.05 details are worth knowing. `i18n.extraLocales` (since 25.05) is the supported way to add locales. `i18n.supportedLocales` is now hidden and deprecated, and setting it by hand produces a warning if you forget anything (Case 2). New in 26.05 is `i18n.imperativeLocale`: when enabled, `localectl set-locale` owns `/etc/locale.conf` and NixOS stops exporting `LANG`/`LC_*` itself. It does **not** add locales to the archive, so the same rule applies: a locale `localectl` points at must still be in `i18n.extraLocales`.

Worked examples

The lab is one flake. None of the hosts is booted. Each case evaluates or builds one part of a host, and a small C++ probe runs against the archives those hosts would ship.

```
# flake.nix - five hosts that differ only in their i18n settings, plus a probe
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { nixpkgs, ... }:
    let
      host = extra: nixpkgs.lib.nixosSystem { modules = [ ./host.nix extra ]; };
    in {
      nixosConfigurations = {
        plain   = host { };
        austria = host ./austria.nix;
        trap    = host ./trap-en-at.nix;
        en-at   = host ./en-at.nix;
        en-at-overlay = host ./en-at-overlay.nix;
      };
      # A tiny C/C++ probe built with the same toolchain as the system
      packages.x86_64-linux.probe =
        let pkgs = nixpkgs.legacyPackages.x86_64-linux; in
        pkgs.runCommandCC "locale-probe" { } ''
          mkdir -p $out/bin
          $CXX -O2 -o $out/bin/locale-probe ${./probe/locale-probe.cc}
        '';
    };
}
```

```
# host.nix - a minimal host; nothing about locales is set here
{ ... }: {
  nixpkgs.hostPlatform = "x86_64-linux";
  boot.loader.systemd-boot.enable = true;
  fileSystems."/" = { device = "/dev/disk/by-label/nixos"; fsType = "ext4"; };
  system.stateVersion = "26.05";
}
```

The probe does what a GUI toolkit does at startup: apply the user's locale, decode a filename to wide characters, format a date and a number, then build a C++ `std::locale` from the environment.

```
// probe/locale-probe.cc - what a GUI app does with $LANG and a filename
#include <clocale>
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <locale>

int main(int argc, char **argv) {
  const char *file = argc > 1 ? argv[1] : "Bericht_März.pdf";
  std::setvbuf(stdout, nullptr, _IOLBF, 0);  // keep lines that precede an abort

  // 1. The C layer: failure is a NULL return, and the process stays in "C".
  const char *got = std::setlocale(LC_ALL, "");
  std::printf("setlocale(LC_ALL, \"\") = %s\n", got ? got : "NULL -> still \"C\"");

  // 2. Decode a filename the way toolkits do before showing or sending it.
  wchar_t wide[256];
  std::size_t n = std::mbstowcs(wide, file, 256);
  if (n == static_cast<std::size_t>(-1))
    std::printf("mbstowcs(\"%s\") = -1 (invalid multibyte sequence)\n", file);
  else
    std::printf("mbstowcs(\"%s\") = %zu characters\n", file, n);

  // 3. Formats from the locale, when there is one.
  char buf[64];
  std::time_t t = 1792022400;  // 2026-10-15 00:00 UTC
  std::strftime(buf, sizeof buf, "%x  %A", std::gmtime(&t));
  std::printf("date: %s   number: %'.2f\n", buf, 1234567.891);

  // 4. The C++ layer: the same failure is an exception.
  std::locale user("");
  std::cout << "std::locale(\"\") = " << user.name() << "\n";
}
```

Case 1: What a default host ships

```
$ nix eval --json .#nixosConfigurations.plain.config.i18n.supportedLocales
["C.UTF-8/UTF-8","en_US.UTF-8/UTF-8"]
$ nix eval --json .#nixosConfigurations.plain.config.systemd.globalEnvironment
{"LOCALE_ARCHIVE":"/nix/store/dc427sjs33dci797apv4lvsg1a69szn0-glibc-locales-2.42-100/lib/lo
```

Two locales, about 3 MB of archive, and this exact store path is in `cache.nixos.org`, so a default

host downloads it instead of building it. systemd services get the archive by **store path**. Login sessions get `LOCALE_ARCHIVE=/run/current-system/sw/lib/locale/locale-archive` from `environment.sessionVariables`, which follows the current generation. Every service unit NixOS renders carries that `LOCALE_ARCHIVE=` line, which is why it appears in the unit dumps elsewhere in this part.

To run the probe against the archives, build everything once and keep the paths in variables:

```
$ b() { nix build --no-link --print-out-paths "$@"; }
$ PLAIN=$(b .#nixosConfigurations.plain.config.i18n.glibcLocales)/lib/locale/locale-archive
$ PROBE=$(b .#probe)/bin/locale-probe
$ LOCALE=$(b .#nixosConfigurations.plain.pkgs.glibc.bin)/bin/locale
$ env -i LOCALE_ARCHIVE=$PLAIN LANG=en_US.UTF-8 $PROBE
setlocale(LC_ALL, "") = en_US.UTF-8
mbstowcs("Bericht_März.pdf") = 16 characters
date: 10/15/2026  Thursday   number: 1,234,567.89
std::locale("") = en_US.UTF-8
```

`env -i` starts from an empty environment, so the output depends only on the two variables shown. Use absolute paths after `env -i`. With no `PATH`, a bare `locale` silently resolves to whatever is in `/usr/bin`, which on a non-NixOS build box is the distribution's own glibc, and that one ignores `LOCALE_ARCHIVE`.

Case 2: Regional formats the supported way

A common desk setup is English messages with local formats:

```
# austria.nix - English messages, Austrian formats, one extra locale for a tool
{ ... }: {
  i18n.defaultLocale = "en_US.UTF-8";
  i18n.extraLocaleSettings = {
    LC_TIME = "de_AT.UTF-8";
    LC_NUMERIC = "de_AT.UTF-8";
    LC_MONETARY = "de_AT.UTF-8";
    LC_PAPER = "de_AT.UTF-8";
    LC_MEASUREMENT = "de_AT.UTF-8";
  };
  i18n.extraLocales = [ "en_GB.UTF-8/UTF-8" ];
}
```

```
$ nix eval --json .#nixosConfigurations.austria.config.i18n.supportedLocales
["C.UTF-8/UTF-8","en_US.UTF-8/UTF-8","de_AT.UTF-8/UTF-8","en_GB.UTF-8/UTF-8"]
$ cat $(b '.#nixosConfigurations.austria.config.environment.etc."locale.conf".source')
LANG=en_US.UTF-8
LC_MEASUREMENT=de_AT.UTF-8
LC_MONETARY=de_AT.UTF-8
LC_NUMERIC=de_AT.UTF-8
```

```
LC_PAPER=de_AT.UTF-8
LC_TIME=de_AT.UTF-8
```

Every value in `extraLocaleSettings` is added to the archive automatically, deduplicated, with the `/UTF-8` charset appended (`i18n.localeCharsets` changes that per category). `extraLocales` is only for locales nothing in the config names, for example one a single tool needs. This archive is not in the binary cache, so the host builds it locally. On the lab box that took 15 seconds: the builder unpacks glibc, configures it, and runs `localedef` for just these four locales.

```
$ AUSTRIA=$(b .#nixosConfigurations.austria.config.i18n.glibcLocales)/lib/locale/locale-arch
$ env -i LOCALE_ARCHIVE=$AUSTRIA LANG=en_US.UTF-8 LC_TIME=de_AT.UTF-8 LC_NUMERIC=de_AT.UTF-8
date: 2026-10-15  Donnerstag   number: 1.234.567,89
```

That line is the reason `en_AT` comes up at all: glibc's `de_AT` uses ISO dates and **German** day and month names. If you want Austrian numbers *and* English names, glibc has no such locale.

If you still set the hidden option yourself, NixOS tells you what you forgot:

```
$ nix eval --json .#nixosConfigurations.austria --apply 'c: (c.extendModules { modules = [ {
["`i18n.supportedLocales` is deprecated in favor of `i18n.extraLocales`,\nand it seems you a
```

Case 3: Asking for a locale glibc does not ship (the good failure)

```
# trap-en-at.nix - asks for a locale glibc does not ship
{ ... }: {
  i18n.defaultLocale = "en_AT.UTF-8";
}
```

Evaluation is happy. It just appends `en_AT.UTF-8/UTF-8` to the list:

```
$ nix eval --json .#nixosConfigurations.trap.config.i18n.supportedLocales
["C.UTF-8/UTF-8","en_US.UTF-8/UTF-8","en_AT.UTF-8/UTF-8"]
$ nix build .#nixosConfigurations.trap.config.i18n.glibcLocales
…
glibc-locales> zu_ZA.UTF-8/UTF-8 \
glibc-locales> zu_ZA/ISO-8859-1 \
glibc-locales> Error: unsupported locales detected:
glibc-locales> en_AT.UTF-8/UTF-8 \
glibc-locales> You should choose from the list above the error.
error: Cannot build '/nix/store/0p9vwnax8swaypzma2dnyy6c75nn6dik-glibc-locales-2.42-100.drv'
       Reason: builder failed with exit code 1.
```

The builder compares your list with glibc's `localedata/SUPPORTED` (501 entries in 2.42) before compiling anything, and refuses. This is the failure you want. It happens at `nixos-rebuild` time, on your terminal, not at runtime on somebody's desktop. The list printed above the error is the authoritative spelling of every locale you *can* ask for.

Case 4: The runtime failure: the archive lacks what the environment asks for

The dangerous version never touches the build. The host ships the default archive, and something in a user's session sets `LANG=en_AT.UTF-8`. A single category is enough: `LANG=en_US.UTF-8 LC_TIME=de_AT.UTF-8` against the same archive aborts in exactly the same way, because `setlocale(LC_ALL, "")` fails if any one category cannot be loaded. Run the probe:

```
$ env -i LOCALE_ARCHIVE=$PLAIN LANG=en_AT.UTF-8 $PROBE; echo "exit=$?"
setlocale(LC_ALL, "") = NULL -> still "C"
mbstowcs("Bericht_März.pdf") = -1 (invalid multibyte sequence)
date: 10/15/26  Thursday   number: 1234567.89
terminate called after throwing an instance of 'std::runtime_error'
  what():  locale::facet::_S_create_c_locale name not valid
exit=134
```

(Your shell also prints its own `Aborted` job line, omitted here.) Read it top to bottom, because this is how such bugs show up in the wild:

1. **The C layer fails quietly.** `setlocale` returns `NULL` and the process stays in `C`. Most C programs never check the return value.
2. **ASCII works and non-ASCII does not.** In the `C` locale the multibyte charset is ASCII, so `März` is an invalid sequence. A program that converts filenames to wide strings works perfectly until the first file with an umlaut, an accent, or a CJK name. "It crashes only for some files" usually starts here.
3. **The formats degrade silently.** The date and number fall back to US-style `C` formats. Nothing reports an error.
4. **The C++ layer throws.** `std::locale("")` turns the same condition into `std::runtime_error`. If nothing catches it, the process aborts (exit 134, SIGABRT). A toolkit that builds a `std::locale` or a `codecvt` facet on demand, for example when a file-transfer dialog first converts a filename, crashes at that moment instead of at startup. That is why the crash looks related to the feature, not to the locale.

`locale` itself shows the condition before any application does:

```
$ env -i LOCALE_ARCHIVE=$PLAIN LANG=en_AT.UTF-8 $LOCALE 2>&1 >/dev/null
/nix/store/kgd806ssw99qaxwiislhfx5dwnc8k893-glibc-2.42-100-bin/bin/locale: Cannot set LC_CTY
/nix/store/kgd806ssw99qaxwiislhfx5dwnc8k893-glibc-2.42-100-bin/bin/locale: Cannot set LC_MES
/nix/store/kgd806ssw99qaxwiislhfx5dwnc8k893-glibc-2.42-100-bin/bin/locale: Cannot set LC_ALL
```

Overriding the request is enough to stop a single program crashing, which is useful while you fix the host:

```
$ env -i LOCALE_ARCHIVE=$PLAIN LANG=en_AT.UTF-8 LC_ALL=en_US.UTF-8 $PROBE | tail -1
std::locale("") = en_US.UTF-8
```

Case 5: Building a site-local locale with the stock builder

If a locale really does not exist in glibc, write it. A glibc locale source is a text file with one section per category. Most sections can `copy` an existing locale, so a new one is mostly decisions, not data:

```
comment_char %
escape_char /
% locales/en_AT - English language, Austrian conventions.
% Site-local: glibc does not ship this locale. Build it, don't wish for it.

LC_IDENTIFICATION
title      "English locale for Austria (site-local)"
source     "example.org IT"
address    ""
contact    ""
email      ""
tel        ""
fax        ""
language   "English"
territory  "Austria"
revision   "1.0"
date       "2026-10-15"
category "i18n:2012";LC_IDENTIFICATION
category "i18n:2012";LC_CTYPE
category "i18n:2012";LC_COLLATE
category "i18n:2012";LC_TIME
category "i18n:2012";LC_NUMERIC
category "i18n:2012";LC_MONETARY
category "i18n:2012";LC_MESSAGES
category "i18n:2012";LC_PAPER
category "i18n:2012";LC_NAME
category "i18n:2012";LC_ADDRESS
category "i18n:2012";LC_TELEPHONE
category "i18n:2012";LC_MEASUREMENT
END LC_IDENTIFICATION

LC_CTYPE
copy "en_GB"
END LC_CTYPE

LC_COLLATE
copy "iso14651_t1"
END LC_COLLATE

LC_MESSAGES
copy "en_GB"
```

```
END LC_MESSAGES

% Money and numbers the Austrian way: € 1.234,56
LC_MONETARY
copy "de_AT"
END LC_MONETARY

LC_NUMERIC
copy "de_AT"
END LC_NUMERIC

% English names, Austrian day-first date, Monday first
LC_TIME
abday   "Sun";"Mon";"Tue";"Wed";"Thu";"Fri";"Sat"
day     "Sunday";"Monday";"Tuesday";"Wednesday";"Thursday";"Friday";"Saturday"
abmon   "Jan";"Feb";"Mar";"Apr";"May";"Jun";"Jul";"Aug";"Sep";"Oct";"Nov";"Dec"
mon     "January";"February";"March";"April";"May";"June";/
        "July";"August";"September";"October";"November";"December"
d_t_fmt "%a %d %b %Y %T"
date_fmt "%a %d %b %Y %T %Z"
d_fmt   "%d.%m.%Y"
t_fmt   "%T"
am_pm   "";""
t_fmt_ampm ""
week    7;19971130;4
first_weekday 2
END LC_TIME

LC_PAPER
copy "de_AT"
END LC_PAPER

LC_TELEPHONE
copy "de_AT"
END LC_TELEPHONE

LC_MEASUREMENT
copy "de_AT"
END LC_MEASUREMENT

LC_NAME
copy "en_GB"
END LC_NAME

LC_ADDRESS
postal_fmt   "%f%N%a%N%d%N%b%N%s %h %e %r%N%z %T%N%c%N"
country_name "Austria"
```

```
country_ab2  "AT"
country_ab3  "AUT"
country_num  40
country_car  "A"
lang_name    "English"
lang_ab      "en"
lang_term    "eng"
lang_lib     "eng"
END LC_ADDRESS
```

Wire it in by extending the derivation the module would have built anyway. The source file goes into glibc's `localedata/locales`, and one line goes into `SUPPORTED` before the builder runs its check from Case 3:

```
# en-at.nix - a site-local locale, compiled by the stock glibcLocales builder
{ pkgs, ... }:
{
  i18n.defaultLocale = "en_AT.UTF-8";

  # Same derivation the module would build, plus one source file and one
  # SUPPORTED line, added before the builder's "is it supported?" check.
  i18n.glibcLocales =
    (pkgs.glibcLocales.override {
      allLocales = false;
      locales = [ "C.UTF-8/UTF-8" "en_US.UTF-8/UTF-8" "en_AT.UTF-8/UTF-8" ];
    }).overrideAttrs (old: {
      preBuild = ''
        cp ${./locales/en_AT} ../localedata/locales/en_AT
        echo 'en_AT.UTF-8/UTF-8 \' >> ../localedata/SUPPORTED
      '' + old.preBuild;
    });
}
```

Setting `i18n.glibcLocales` replaces the default archive completely, so the list must contain every locale the host needs, not just the new one. The module's own description warns about this. Keep `C.UTF-8` in the list: the module's default always includes it, and the builder compiles only what you list.

```
$ nix eval --json .#nixosConfigurations.en-at.config.warnings
[]
$ nix build -L .#nixosConfigurations.en-at.config.i18n.glibcLocales 2>&1 | grep -E 'UTF-8\.+
glibc-locales> C.UTF-8en_AT.UTF-8en_US.UTF-8......... done
glibc-locales> Adding /build//nix/store/fsd4vbp9zjd9iaicq2fb9lpc1h5j973y-glibc-2.42-100/lib/
$ ENAT=$(b .#nixosConfigurations.en-at.config.i18n.glibcLocales)/lib/locale/locale-archive
$ for a in PLAIN AUSTRIA ENAT; do echo "$a: $(LOCALE_ARCHIVE=${!a} $LOCALE -a | tr '\n' ' ')
PLAIN: C C.utf8 POSIX en_US.utf8
AUSTRIA: C C.utf8 POSIX de_AT.utf8 en_GB.utf8 en_US.utf8
```

```
ENAT: C C.utf8 POSIX en_AT.utf8 en_US.utf8
$ env -i LOCALE_ARCHIVE=$ENAT LANG=en_AT.UTF-8 $PROBE; echo "exit=$?"
setlocale(LC_ALL, "") = en_AT.UTF-8
mbstowcs("Bericht_März.pdf") = 16 characters
date: 15.10.2026  Thursday   number: 1.234.567,89
std::locale("") = en_AT.UTF-8
exit=0
$ env -i LOCALE_ARCHIVE=$ENAT LANG=en_AT.UTF-8 $LOCALE -k currency_symbol mon_decimal_point
currency_symbol="€"
mon_decimal_point=","
d_fmt="%d.%m.%Y"
first_weekday=2
country_name="Austria"
lang_name="English"
```

English day names, an Austrian day-first date, euro amounts with a decimal comma, and a week that starts on Monday. The same probe that aborted in Case 4 now runs cleanly. The host wiring follows from the module: `/etc/locale.conf` says `LANG=en_AT.UTF-8`, and services get the new archive by store path:

```
$ nix eval --json .#nixosConfigurations.en-at.config.systemd.globalEnvironment.LOCALE_ARCHIV
"/nix/store/qrw04749s4scz8fka96arprnibbrl1qq-glibc-locales-2.42-100/lib/locale/locale-archiv
```

Two notes on the source file. The monetary thousands separator copied from `de_AT` is U+202F, a narrow no-break space (`locale mon_thousands_sep | od -An -tx1` prints `e2 80 af 0a`). Formatted amounts that look like `€ 1 234,56` are not ASCII, and code that parses them back will notice. And keep the file in the flake next to the module that uses it. It is configuration, and a reviewer should see it in the same diff.

The trap

The tempting shortcut for Case 5 is an overlay: patch `glibcLocales` once in `nixpkgs.overlays`, and the module's default (`pkgs.glibcLocales.override { … }`) picks it up with no `i18n.glibcLocales` line. It evaluates, and it would even work. Now look at what it costs, compared on the full system closure:

```
# en-at-overlay.nix - the same locale, injected with an overlay instead
{ ... }:
{
  i18n.defaultLocale = "en_AT.UTF-8";
  nixpkgs.overlays = [
    (final: prev: {
      glibcLocales = prev.glibcLocales.overrideAttrs (old: {
        preBuild = ''
          cp ${./locales/en_AT} ../localedata/locales/en_AT
```

```
          echo 'en_AT.UTF-8/UTF-8 \' >> ../localedata/SUPPORTED
        '' + old.preBuild;
      });
    })
  ];
}
```

```
$ for h in plain en-at en-at-overlay; do echo "== $h"; nix build --dry-run .#nixosConfigurat
== plain
these 254 derivations will be built:
these 367 paths will be fetched (384.5 MiB download, 916.8 MiB unpacked):
== en-at
these 254 derivations will be built:
these 367 paths will be fetched (384.5 MiB download, 916.8 MiB unpacked):
== en-at-overlay
these 1421 derivations will be built:
these 1174 paths will be fetched (3.5 GiB download, 7.5 GiB unpacked):
```

(The 254 local derivations for `plain` are this lab's own configuration files and units, which no cache has. `en-at` matches it because its archive is already built.) The overlay changes `pkgs.glibcLocales` for **every derivation that has it anywhere in its build-time graph**, and for everything downstream of those. Test suites and documentation builds often pull it in to get a UTF-8 locale, so the change spreads a long way. In this closure the rebuild list includes `gcc-15.3.0`, `glib-2.88.3`, `coreutils-full-9.11`, `dbus-broker-37` and the `linux-6.18.56` kernel. They all get new store paths, none of them is in the cache, and your laptop compiles a compiler because you wanted English month names. The `i18n.glibcLocales` option changes one runtime file and nothing that is built from source.

The second half of the trap is the runtime failure from Case 4, because no build ever sees it. A host can be perfectly configured and still have users whose session asks for a locale it does not have. When a desktop app crashes only on certain files, or a C++ service dies with `_S_create_c_locale name not valid`, check the request and the archive before you read the application's code:

```
$ tr '\0' '\n' < /proc/$(pidof the-app)/environ | grep -E '^(LANG|LC_|LOCALE_ARCHIVE)'
$ locale -a                       # what the current generation's archive holds
$ locale 2>&1 >/dev/null          # "Cannot set LC_… to default locale" = mismatch
```

The boring rule

List every locale the host should load in configuration: `i18n.defaultLocale`, `i18n.extraLocaleSettings` for per-category formats, `i18n.extraLocales` for anything else a person or program will ask for. Do not set `i18n.supportedLocales` yourself. If glibc does not ship a locale, write the source file, keep it in the flake, and attach it through `i18n.glibcLocales` with the complete list. Never patch `glibcLocales` in an overlay. When a program crashes on non-ASCII names, run `locale` in its environment first.

Try this

1. Add `LC_TIME = "en_IE.UTF-8";` to `austria.nix` and confirm, without building, that `en_IE.UTF-8/UTF-8` joins `supportedLocales`. Then compare the probe's date line with `de_AT`.
2. Change `d_fmt` in `locales/en_AT` to `"%Y-%m-%d"`, rebuild, and watch which store path changes and which does not.
3. Set `i18n.imperativeLocale = true;` on `en-at`, and diff `environment.sessionVariables` and `environment.etc` against the declarative host. Which file stops being managed?
4. Catch the exception in the probe (`try { std::locale(""); } catch (const std::runtime_error &) { … }`) and fall back to `std::locale::classic()`. Decide whether the program should then refuse non-ASCII filenames or decode them as UTF-8 regardless.
5. Wrap a single program so it always runs in a known locale: `pkgs.symlinkJoin` plus `makeWrapper … --set LC_ALL en_US.UTF-8`. When is that better than fixing the host?

Sources

- nixpkgs `nixos/modules/config/i18n.nix` on branch `nixos-26.05` (rev `a033e66`): `defaultLocale`, `extraLocaleSettings`, `extraLocales`, `localeCharsets`, hidden `supportedLocales` and its deprecation warning, `imperativeLocale`, `LOCALE_ARCHIVE` wiring: https://github.com/NixOS/nixpkgs/blob/nixos-26.05/nixos/modules/config/i18n.nix
- nixpkgs `pkgs/development/libraries/glibc/locales.nix` (the `SUPPORTED` check and `localedef` build) and `nix-locale-archive.patch` (archive lookup order): https://github.com/NixOS/nixpkgs/tree/nixos-26.05/pkgs/development/libraries/glibc
- NixOS manual, Internationalisation options (select 26.05): https://search.nixos.org/options?channel=26.05&query=i18n.
- glibc `localedata/SUPPORTED` and locale sources: https://sourceware.org/git/?p=glibc.git;a=blob;f=localedata/SUPPORTED
- `locale(5)` (locale definition file format) and `localedef(1)`: https://man7.org/linux/man-pages/man5/locale.5.html, https://man7.org/linux/man-pages/man1/localedef.1.html
- `setlocale(3)` (return value on failure): https://man7.org/linux/man-pages/man3/setlocale.3.html
- C++ `std::locale` constructor (throws `std::runtime_error` for an invalid name): https://en.cppreference.com/w/cpp/locale/locale/locale

system.stateVersion: What It Changes, and How to Bump It Without Losing Data

`system.stateVersion`: What It Changes, and How to Bump It Without Losing Data

Goal: Know exactly what the one line `system.stateVersion = "…";` does on a NixOS host and what it does not do. See which defaults it selects on *your* configuration, and why changing it "to match the release" can start a database server on an empty data directory. Learn the safe way to move a host to newer defaults: diff, pin or migrate, diff again. Recognise the newer per-module `stateVersion` that lets one service migrate on its own schedule.

Version note: everything below was evaluated against **nixos-26.05** (`26.05.20261018.8ce7336`) with **Nix 2.35.3**. All evaluation output is pasted from those runs. Evaluation does not need a NixOS machine, so nothing was booted. The data-migration steps in Case 6 and *The trap* (`pg_upgrade`, moving a data folder) need a running host with real data. They are written from the NixOS manual and marked as not run.

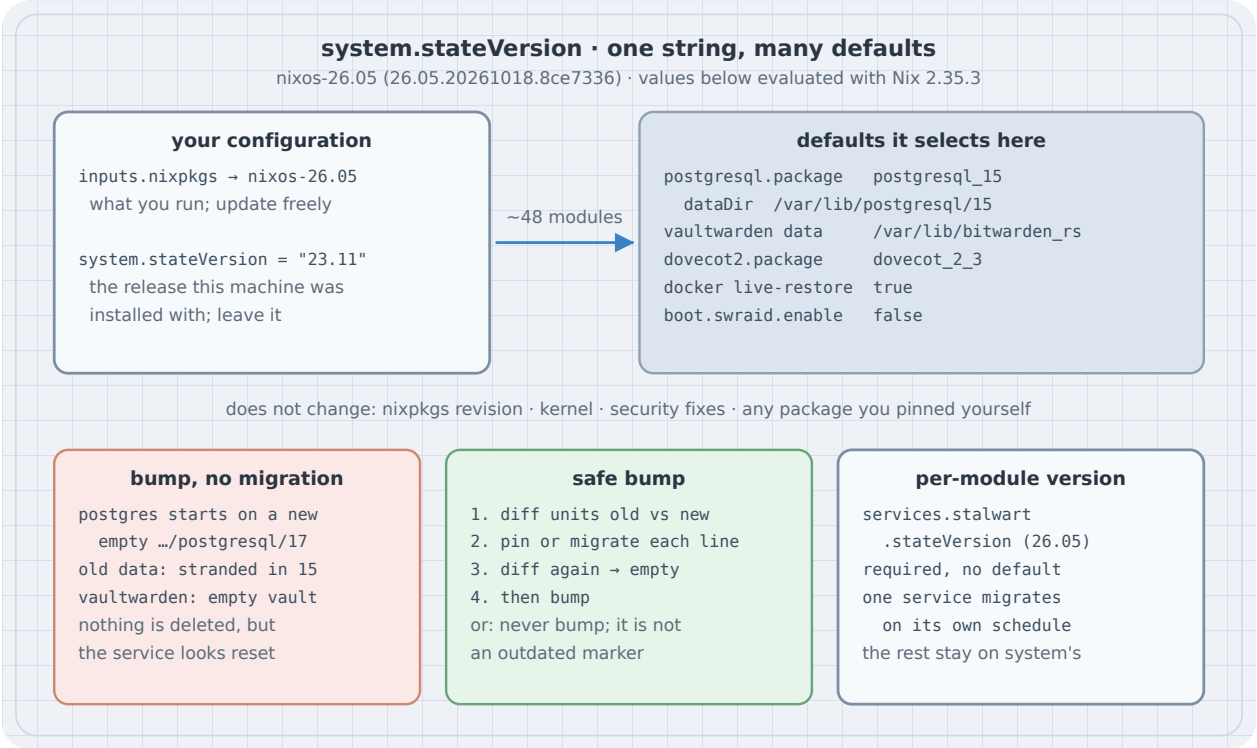


Figure 1: system.stateVersion: one string, many defaults

Mental model

A NixOS host has two different "versions", and people mix them up:

```
what you RUN                                    what your DATA was created with

inputs.nixpkgs → nixos-26.05 (rev 8ce7336)      system.stateVersion = "23.11"
  · packages, kernel, systemd, security fixes     · set once, by nixos-generate-config,
  · you update it every week, freely                to the release you INSTALLED
  · nixos-version prints it                       · read by module DEFAULTS only:
                                                    if versionAtLeast stateVersion "25.11"
                                                    then pkgs.postgresql_17 else …
```

Modules use `stateVersion` only where a newer default would break **existing state**: a database major version whose on-disk format needs a manual upgrade, a data directory that was renamed, a daemon setting whose default flipped. Code that can migrate its own data does not need it, and the option description says so. Everything else follows nixpkgs, whatever `stateVersion` says.

The consequences are simple:

- A low `stateVersion` is **not** outdated, unsupported or insecure. A host installed on 23.11 and updated to nixos-26.05 runs 26.05's PostgreSQL 15 build, with 26.05's patches. It just has not been moved to PostgreSQL 17.
- Changing `stateVersion` changes **defaults**, and nothing else. Options you set explicitly are not affected. Defaults that depend on it switch to the new value on the next `switch`, and the data on disk stays where it is.
- If you leave it unset, it defaults to the current release and evaluation prints a warning (Case 3). The next release upgrade then moves your defaults, with only that warning as notice. That is the one way to get the dangerous behaviour without editing anything.

`nixos-generate-config` writes the line for you, with a warning comment (`nixos/modules/installer/tools/too`

```
system.stateVersion = "${config.system.nixos.release}"; # Did you read the comment?
```

Worked examples

No host is booted. Each case evaluates a minimal NixOS configuration with `nixpkgs.lib`'s `eval-config.nix` and reads back option values or the generated systemd units. Everything runs as an ordinary user. The `nixpkgs` revision is the locked nixos-26.05 commit from the version note.

```
$ nix --version; nix eval --raw github:NixOS/nixpkgs/8ce73368d1efd4d657e9064d29b144a52046b4e
nix (Nix) 2.35.3
26.05.20261018.8ce7336
```

Case 1: How many modules care?

A census of the module tree at that revision:

```
# census.sh - which NixOS modules read system.stateVersion, and at which release thresholds
cd "$1/nixos/modules"
echo "modules mentioning system.stateVersion: $(rg -l -g '*.nix' 'system\.stateVersion|inher
echo "comparisons per threshold, code + defaultText:"
rg -o --no-filename -g '*.nix' 'version(Older|AtLeast) (config\.system\.|cfg\.)?stateVersion
  | grep -o '"[0-9.]*"' | tr -d '"' | sort -V | uniq -c | awk '{printf "  %s:%s", $2, $1} EN
echo "thresholds 26.05 and later:"
rg -l -g '*.nix' 'stateVersion "26\.(05|11)"' . | sed 's|^\./|  |' | sort
```

```
$ bash census.sh ./nixpkgs     # an unpacked checkout of rev 8ce7336
modules mentioning system.stateVersion: 48
comparisons per threshold, code + defaultText:
  17.09:5  18.03:2  19.03:1  19.09:2  20.03:2  20.09:4  21.03:1  21.11:1  22.05:6  23.05:1
thresholds 26.05 and later:
  services/databases/mysql.nix
  services/mail/dovecot.nix
  services/mail/stalwart.nix
  services/misc/seerr.nix
  services/misc/tandoor-recipes.nix
  services/misc/taskchampion-sync-server.nix
  services/networking/sabnzbd/default.nix
  services/web-apps/lauti.nix
  services/web-apps/netbox.nix
  services/web-apps/nextcloud.nix
  services/web-apps/olivetin.nix
  tasks/filesystems/zfs.nix
```

48 of the 2,421 `.nix` files under `nixos/modules` mention the option. A few only *set* it (the installer's generated configuration, test and builder-VM profiles), and the rest read it. Most of the readers are services that own a data directory: databases, mail, web apps, media servers. Thresholds as old as 17.09 stay in the code. When the package an old branch would select is removed from nixpkgs, that branch turns into an evaluation error (PostgreSQL 13 and older, Case 4). Note the three `26.11` comparisons that are already on the 26.05 branch, for example `boot.zfs.forceImportRoot` and a MySQL first-start change. A host installed with the coming 26.11 release will get the new default, and every host installed earlier keeps the old one.

Case 2: The same configuration under five stateVersions

```
# probe.nix - evaluate one minimal NixOS configuration per stateVersion and read back defaul
let
  nixpkgs = builtins.getFlake "github:NixOS/nixpkgs/8ce73368d1efd4d657e9064d29b144a52046b4e8
```

```
  host = stateVersion: (import "${nixpkgs}/nixos/lib/eval-config.nix" {
    system = "x86_64-linux";
    modules = [{
      boot.isContainer = true;                  # no bootloader / fileSystems needed for eval
      system.stateVersion = stateVersion;
      services.postgresql.enable = true;        # enabled, but nothing pinned
      services.vaultwarden.enable = true;
      services.dovecot2.enable = true;
    }];
  }).config;
  probe = sv: let c = host sv; in {
    "postgresql.package"          = c.services.postgresql.package.name;
    "postgresql.dataDir"          = c.services.postgresql.dataDir;
    "vaultwarden StateDirectory"  = c.systemd.services.vaultwarden.serviceConfig.StateDirect
    "dovecot2.package"            = c.services.dovecot2.package.name;
    "oci-containers.backend"      = c.virtualisation.oci-containers.backend;
    "docker live-restore"         = c.virtualisation.docker.daemon.settings.live-restore;
    "boot.swraid.enable"          = c.boot.swraid.enable;
    "zfs.forceImportRoot"         = c.boot.zfs.forceImportRoot;
  };
in builtins.listToAttrs (map (sv: { name = sv; value = probe sv; }) [ "22.11" "23.11" "24.11
```

```
$ nix eval --impure --json -f probe.nix > probe.json && python3 table.py probe.json
stateVersion →              22.11                   23.11                   24.11
boot.swraid.enable          true                    false                   false
docker live-restore         true                    true                    false
dovecot2.package            dovecot-2.3.21.1        dovecot-2.3.21.1        dovecot-2.3.21.1
oci-containers.backend      podman                  podman                  podman
postgresql.dataDir          /var/lib/postgresql/14  /var/lib/postgresql/15  /var/lib/postgre
postgresql.package          postgresql-14.24        postgresql-15.19        postgresql-16.15
vaultwarden StateDirectory  bitwarden_rs           bitwarden_rs            vaultwarden
zfs.forceImportRoot         true                    true                    true
```

`table.py` only turns the JSON into rows:

```
# table.py - print probe.json as rows = options, columns = stateVersions
import json, sys
d = json.load(open(sys.argv[1]))
svs = sorted(d, key=lambda v: tuple(map(int, v.split("."))))
opts = list(d[svs[0]])
cell = lambda v: str(v).lower() if isinstance(v, bool) else str(v)
rows = [["stateVersion →"] + svs] + [[o] + [cell(d[s][o]) for s in svs] for o in opts]
w = [max(len(r[i]) for r in rows) for i in range(len(rows[0]))]
for r in rows:
    print("  ".join(c.ljust(w[i]) for i, c in enumerate(r)).rstrip())
```

Read it column by column: these are the defaults *one* nixpkgs revision gives each host. Every

column runs the same 26.05 packages. What differs is which *variant* the module chooses for state it cannot migrate itself:

- **PostgreSQL** follows a staircase: 14 → 15 → 16 → 17. The data directory has the major version in its name (`/var/lib/postgresql/${psqlSchema}`). That detail decides what happens on a bump (Case 5).
- **Vaultwarden** keeps the pre-rename data folder `bitwarden_rs` for hosts older than 24.11.
- **Dovecot 2.4** changed its configuration format, so only hosts installed with 26.05 get it by default.
- **Docker `live-restore`** was a NixOS-specific `true` until 24.11. Newer hosts follow upstream (`false`).
- **`boot.swraid.enable`** was implicitly on before 23.11. Newer hosts must enable it, as `nixos-generate-config` does when it finds MD arrays.
- **`oci-containers.backend`** and **`zfs.forceImportRoot`** do not change within these columns. Their thresholds are 22.05 (Podman) and 26.11 (not yet released).

Case 3: `stateVersion` left unset

```nix
# edge.nix - edge cases: stateVersion unset, very old, and a module with its own stateVersio
let
  nixpkgs = builtins.getFlake "github:NixOS/nixpkgs/8ce73368d1efd4d657e9064d29b144a52046b4e8
  eval = m: (import "${nixpkgs}/nixos/lib/eval-config.nix" {
    system = "x86_64-linux";
    modules = [ { boot.isContainer = true; } m ];
  }).config;
in {
  unset = (eval { }).system.stateVersion;
  old-pg = (eval { system.stateVersion = "21.11"; services.postgresql.enable = true; })
    .services.postgresql.package.name;
  stalwart = (eval { system.stateVersion = "26.05"; services.stalwart.enable = true; })
    .services.stalwart.dataDir;
  stalwart-own = builtins.mapAttrs (_: sv: (eval {
    system.stateVersion = "26.05";                    # the host was installed on 26.05 …
    services.stalwart = { enable = true; stateVersion = sv; };   # … the mail server earlier
  }).services.stalwart.dataDir) { "25.11" = "25.11"; "26.05" = "26.05"; };
}
```

```
$ nix eval --impure --raw -f edge.nix unset; echo
evaluation warning: system.stateVersion is not set, defaulting to 26.05. Read why this matte
26.05
```

The default is `config.system.nixos.release`, the release of the nixpkgs you evaluate. Today that is harmless, because the host gets today's defaults. On the day you move the flake input to nixos-26.11, the default becomes `26.11` and every module from Case 1 that has a 26.11 threshold switches over at once. The warning is the only notice you get. Treat it as an error. Copy the value it prints into the configuration and commit it.

Case 4: The floor: stateVersions older than the oldest supported default

```
$ nix eval --impure --raw -f edge.nix old-pg 2>&1 | grep -E '^\s*error: [a-zA-Z]'
       error: postgresql_13 was removed, please upgrade your postgresql version.
```

A 21.11 host that left PostgreSQL unpinned would get `postgresql_13`, which nixpkgs no longer ships. Rather than silently starting a newer major version on old data, the module **refuses to evaluate**. The same will happen to `postgresql_14` once it is removed. Upstream support for PostgreSQL 14 ends in November 2026, and the module already carries a warning helper for "the oldest version available and thus the next that will be removed". If your host's `stateVersion` is between 22.05 and 23.05, pin the package and plan the `pg_upgrade` now, while you choose the time.

Case 5: What a bump from 23.11 to 26.05 would actually change

The useful question is not "what is the newest stateVersion?" but "what would change on *my* host?". Generate the units for both values and diff them:

```
# host.nix - a small server installed on 23.11 and upgraded release by release since
{ ... }: {
  boot.isContainer = true;
  services.postgresql.enable = true;
  services.vaultwarden.enable = true;
  services.dovecot2.enable = true;
  virtualisation.docker.enable = true;
}
```

```
# units.nix - the systemd units host.nix generates under a given stateVersion
{ stateVersion, hostModule ? ./host.nix }:
let
  nixpkgs = builtins.getFlake "github:NixOS/nixpkgs/8ce73368d1efd4d657e9064d29b144a52046b4e8
  cfg = (import "${nixpkgs}/nixos/lib/eval-config.nix" {
    system = "x86_64-linux";
    modules = [ hostModule { system.stateVersion = stateVersion; } ];
  }).config;
  wanted = [ "postgresql" "vaultwarden" "dovecot" "docker" ];
  pick = s: { inherit (s) environment serviceConfig; preStart = s.preStart; };
in builtins.listToAttrs (map (n: { name = n; value = pick cfg.systemd.services.${n}; })
  (builtins.filter (n: cfg.systemd.services ? ${n}) wanted))
// { "docker daemon.json" = cfg.virtualisation.docker.daemon.settings; }
```

```
# diff.py - flatten two JSON documents and print every leaf that differs
import json, sys

def flat(x, p=""):
```

```
    if isinstance(x, dict):
        for k, v in x.items():
            yield from flat(v, f"{p}.{k}" if p else k)
    elif isinstance(x, list):
        for i, v in enumerate(x):
            yield from flat(v, f"{p}[{i}]")
    else:
        yield p, x

def short(v, n=96):
    s = json.dumps(v) if not isinstance(v, str) else v
    s = "  ".join(l.strip() for l in s.splitlines() if l.strip())
    return s if len(s) <= n else s[: n - 1] + "…"

def window(x, y, n=88):
    """Show both strings from a little before the first character where they differ."""
    x, y = short(x, 10**6), short(y, 10**6)
    i = next((i for i, (p, q) in enumerate(zip(x, y)) if p != q), min(len(x), len(y)))
    start = max(0, i - 24)
    cut = lambda s: ("…" if start else "") + (s[start:start + n] + ("…" if len(s) > start +
    return cut(x), cut(y)

a, b = (dict(flat(json.load(open(f)))) for f in sys.argv[1:3])
for k in sorted(a.keys() | b.keys()):
    if a.get(k) != b.get(k):
        print(k)
        old, new = window(a.get(k, "(absent)"), b.get(k, "(absent)"))
        print("  -", old)
        print("  +", new)
```

```
$ for v in 23.11 26.05; do nix eval --impure --json --expr "import ./units.nix { stateVersio
$ python3 diff.py units-23.11.json units-26.05.json
docker daemon.json.live-restore
  - true
  + false
docker.serviceConfig.ExecStart[1]
  - …-config-file=/nix/store/84qkcv0j9v7vj8f9wymbx0s26mc7pz0b-daemon.json \
  + …-config-file=/nix/store/r1c783pjbja7as7qyjzqf6r62vbkv0ai-daemon.json \
dovecot.serviceConfig.ExecReload
  - /nix/store/0jf59pcd4sqbgrx6swq356ynzad7lk53-dovecot-2.3.21.1/bin/doveadm reload
  + /nix/store/4n0shh682cd47yb71sjkhx6vr22l328m-dovecot-2.4.5/bin/doveadm reload
dovecot.serviceConfig.ExecStart
  - /nix/store/0jf59pcd4sqbgrx6swq356ynzad7lk53-dovecot-2.3.21.1/bin/dovecot -F
  + /nix/store/4n0shh682cd47yb71sjkhx6vr22l328m-dovecot-2.4.5/bin/dovecot -F
postgresql.environment.PATH
  - /nix/store/fhwggl31v8v7yxlkg1fvq5c2m89h0jsv-postgresql-and-plugins-15.19/bin:/nix/store/
```

```
  + /nix/store/apxhlnhqwj588xzrhrm2bqi6jw8h25xk-postgresql-and-plugins-17.11/bin:/nix/store/
postgresql.environment.PGDATA
  - /var/lib/postgresql/15
  + /var/lib/postgresql/17
postgresql.preStart
  - …-e /var/lib/postgresql/15/PG_VERSION; then   # Cleanup the data directory.   rm -f /var/
  + …-e /var/lib/postgresql/17/PG_VERSION; then   # Cleanup the data directory.   rm -f /var/
postgresql.serviceConfig.ExecStart
  - /nix/store/fhwggl31v8v7yxlkg1fvq5c2m89h0jsv-postgresql-and-plugins-15.19/bin/postgres
  + /nix/store/apxhlnhqwj588xzrhrm2bqi6jw8h25xk-postgresql-and-plugins-17.11/bin/postgres
postgresql.serviceConfig.ExecStartPre[0]
  - /nix/store/px983wkhz9vlc9ss8ksc0b0a12flpgyg-unit-script-postgresql-pre-start/bin/postgre
  + /nix/store/v3c2ba8kssw9l803c93kzzk76b105b39-unit-script-postgresql-pre-start/bin/postgre
postgresql.serviceConfig.StateDirectory
  - postgresql postgresql/15
  + postgresql postgresql/17
vaultwarden.serviceConfig.EnvironmentFile[0]
  - /nix/store/7vc2iqrapfga7jic8wmcyay6z5wwb932-vaultwarden.env
  + /nix/store/lq210axv4v22lk56zp1fj41nqrqikab1-vaultwarden.env
vaultwarden.serviceConfig.StateDirectory
  - bitwarden_rs
  + vaultwarden
```

That list is the real meaning of the one-line change for this host:

- **PostgreSQL:** new binaries (`17.11` instead of `15.19`), and `PGDATA`, `StateDirectory` and the `preStart` check all move to `/var/lib/postgresql/17`. `preStart` runs `initdb` when `PG_VERSION` is missing, and there is no `PG_VERSION` in a directory that has never existed. So the first start after the bump creates a **new, empty cluster**, and the service comes up "healthy" with no databases. The old cluster is still in `/var/lib/postgresql/15`. Nothing was deleted, but every application now sees an empty database.
- **Vaultwarden:** `StateDirectory` (and `DATA_FOLDER` in the environment file) move from `bitwarden_rs` to `vaultwarden`. It starts with an empty data folder: no users, no vault.
- **Dovecot:** 2.3 → 2.4, a major version whose configuration syntax changed. A hand-written `extraConfig` from the 2.3 days may no longer parse.
- **Docker:** `live-restore` flips to `false`. Restarting `dockerd` (for example during the next `switch` that updates Docker) now stops the running containers.

A diff of the units catches what changes in services. It does not catch defaults outside `systemd.services`, such as `boot.swraid.enable` in Case 2. For a complete answer, build both system closures and compare them with `nix store diff-closures`, or `nvd diff`, before switching. That needs both systems built and was not run here.

Case 6: The safe bump: pin first, then diff again

Write down every default you rely on, then bump:

```
# host-pinned.nix - host.nix with every stateVersion-dependent default it relies on written
{ pkgs, ... }: {
  imports = [ ./host.nix ];
  services.postgresql.package = pkgs.postgresql_15;            # was the 23.11 default
  services.dovecot2.package = pkgs.dovecot_2_3;               # was the default before 26.05
  virtualisation.docker.daemon.settings.live-restore = true;  # was the default before 24.11
}
```

```
$ nix eval --impure --json --expr 'import ./units.nix { stateVersion = "26.05"; hostModule =
$ python3 diff.py units-23.11.json units-26.05-pinned.json
vaultwarden.serviceConfig.EnvironmentFile[0]
  - /nix/store/7vc2iqrapfga7jic8wmcyay6z5wwb932-vaultwarden.env
  + /nix/store/lq210axv4v22lk56zp1fj41nqrqikab1-vaultwarden.env
vaultwarden.serviceConfig.StateDirectory
  - bitwarden_rs
  + vaultwarden
```

Three of the four services are now identical to the 23.11 host. Vaultwarden has **no option** for its data folder: it is a `let` binding inside the module. You either keep `stateVersion` below 24.11 or migrate the data yourself: stop the service, move `/var/lib/bitwarden_rs` to `/var/lib/vaultwarden`, bump and start. That was **not run** here. Repeat the diff until it is empty or every remaining line is one you have migrated. At that point the bump is a no-op for this host, which is the only kind of bump worth making. After that, upgrading PostgreSQL is a separate, deliberate change: follow the NixOS manual's `upgrade-pg-cluster`/`pg_upgrade` procedure, then raise `services.postgresql.package` (not run here).

Case 7: A `stateVersion` per module

Stalwart (mail and collaboration server) got its own `services.stalwart.stateVersion` in 26.05. It has **no default**, and the 26.05 release notes say existing installations must set it to their `system.stateVersion`:

```
$ nix eval --impure --raw -f edge.nix stalwart 2>&1 | grep -E '^\s*error: [a-zA-Z]'
       error: The option `services.stalwart.stateVersion' was accessed but has no value defi
```

With it set, the host and the service move independently. The host below was installed with 26.05, but its mail data came from 25.11:

```
$ nix eval --impure --json -f edge.nix stalwart-own; echo
{"25.11":"/var/lib/stalwart-mail","26.05":"/var/lib/stalwart"}
```

At this revision Stalwart's option is the only module-level `stateVersion` in the module tree (`rg 'stateVersion = (lib\.)?mkOption' nixos/modules` finds it and `version.nix`, nothing else). It shows the alternative to one global switch: each service carries its own migration state. Until more modules adopt it, `system.stateVersion` remains the default that every other module reads.

The trap

"My stateVersion is three releases old, so I should update it." That is exactly what the option description tells you not to do:

> Most users should **never** change this value after the initial install, for any reason, even if you've upgraded your system to a new NixOS release.
>
> This value does **not** affect the Nixpkgs version your packages and OS are pulled from, so changing it will **not** upgrade your system.

The failure looks harmless: evaluation succeeds, `switch` succeeds and the services report active. What is lost is the connection between services and their data (Case 5): PostgreSQL on an empty cluster, Vaultwarden with an empty vault. If you have already done it, **do not write new data**. Set `system.stateVersion` back (or pin the old defaults as in Case 6) and `switch`. The old directories are still there and are used again. Anything written to the new, empty directories in the meantime has to be merged by hand.

The mirror-image trap is copying a configuration to a **new** machine together with its old `stateVersion`. The new host then starts with the oldest defaults (PostgreSQL 15 instead of 17, a legacy data folder) and walks into Case 4 one release earlier. A fresh install gets the release it was installed with, even when the rest of the configuration is shared. In a fleet, set `stateVersion` per host, not in a shared module.

The boring rule

Set `system.stateVersion` once, to the release you installed. Commit it per host, and never move it as part of a release upgrade. Upgrade services by setting their package or option explicitly (`services.postgresql.package`), using the service's own migration procedure. If you ever want newer defaults, diff the generated units (or closures) for the old and new value, pin or migrate every difference, and bump only when the diff is empty. For modules with their own `stateVersion`, set it the same way and move it per service.

Try this

1. Add your own host's enabled services to `host.nix` (or point `units.nix` at your real host module). Run the 23.11 → 26.05 diff. Which lines are package changes, and which are path changes?
2. Extend `probe.nix` with `services.nextcloud.package` and `services.mattermost.dataDir` (both need `enable = true` and, for Nextcloud, a `hostName`). At which thresholds do they move?
3. Find the `stateVersion` comparisons on the `nixos-unstable` branch and compare the 26.11 list with Case 1. Which defaults will a host installed on 26.11 get that yours never will?
4. Build two toplevels that differ only in `stateVersion` with `nix build .#nixosConfigurations.<a>.config.` (and `<b>`), then run `nix store diff-closures ./result-a ./result-b`. What does the closure diff show that the unit diff missed?

5. Set `system.stateVersion` to `"26.11"` while still on nixos-26.05. Does anything check the value against the release? Which defaults from Case 1 change?

Lab files, all shown above: `census.sh`, `probe.nix`, `table.py`, `edge.nix`, `host.nix`, `host-pinned.nix`, `units.nix`, `diff.py`.

Sources

- nixpkgs `nixos/modules/misc/version.nix` on branch `nixos-26.05` (rev `8ce7336`): the `system.stateVersion` option, its default, the "not set" warning and the full description quoted above: https://github.com/NixOS/nixpkgs/blob/nixos-26.05/nixos/modules/misc/version.nix
- NixOS options search, `system.stateVersion` (26.05): https://search.nixos.org/options?channel=26.05&show=system.stateVersion
- nixpkgs modules read in this chapter (same revision): `services/databases/postgresql.nix` and `postgresql.md` (version staircase, `dataDir`, upgrade procedure), `services/security/vaultwarden/def` `services/mail/dovecot.nix`, `virtualisation/docker.nix`, `tasks/swraid.nix`, `tasks/filesystems/zfs` `services/mail/stalwart.nix`, `installer/tools/tools.nix`: https://github.com/NixOS/nixpkgs/tree/nixos-26.05/nixos/modules
- NixOS 26.05 release notes (`nixos/doc/manual/release-notes/rl-2605.section.md`): Stalwart's module-level `stateVersion`, Dovecot 2.4 and Nextcloud defaults: https://nixos.org/manual/nixos/stable/release-notes
- NixOS manual, PostgreSQL, "Upgrading": https://nixos.org/manual/nixos/stable/#module-services-postgres-upgrading
- PostgreSQL versioning policy (14 EOL November 2026): https://www.postgresql.org/support/versioning/
- Nix manual, `nix store diff-closures`: https://nix.dev/manual/nix/latest/command-ref/new-cli/nix3-store-diff-closures

/boot Is Full: How systemd-boot Fills the ESP, and How to Empty It

`/boot` Is Full: How systemd-boot Fills the ESP, and How to Empty It

Goal: Understand what NixOS copies onto the EFI system partition (ESP) for every generation and why a 512 MiB partition can fill up. Learn why `nix-collect-garbage -d` frees nothing on `/boot` until the boot loader is installed again, and how `boot.loader.systemd-boot.configurationLimit` caps the menu. Get a host whose `nixos-rebuild` stops with *No space left on device* booting new generations again.

Version note: everything below comes from the nixpkgs systemd-boot module and its installer script, `systemd-boot-builder.py`, at nixpkgs commit `1c89083` (nixos-unstable, October 2026). The lab runs that **real script**, unchanged apart from its build-time placeholders, against a fake store and a 96 MiB tmpfs standing in for the ESP. Two calls are replaced: the `nix-env --list-generations` call and the path of the system profile. Nothing was booted, and no firmware variables were touched. The host commands in *Try this* are marked as not run.

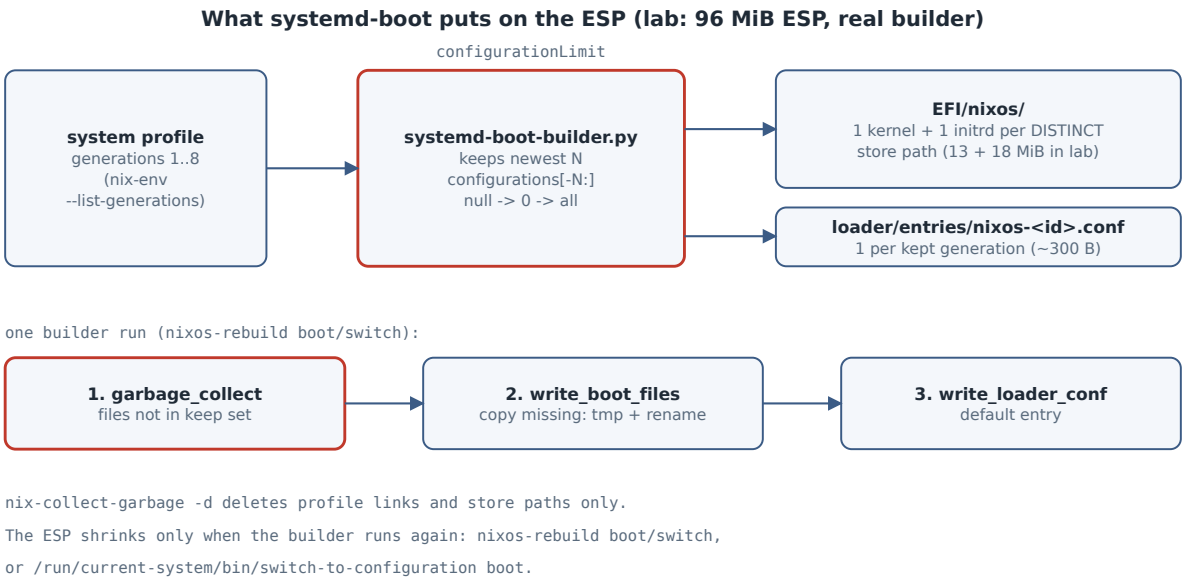


Figure 1: What systemd-boot puts on the ESP

Mental model

The kernel and initrd of every NixOS generation live in `/nix/store`. Firmware cannot read `/nix/store`, because it is usually on ext4, btrfs or ZFS, and sometimes encrypted. So on each

`nixos-rebuild boot` or `switch`, the systemd-boot installer **copies** them onto the FAT-formatted ESP:

```
/boot                                          (the ESP, FAT32, often 512 MiB)
  EFI/nixos/<store-hash>-linux-6.12.58-bzImage.efi        ← one per DISTINCT kernel
  EFI/nixos/<store-hash>-initrd-linux-6.12.58-initrd.efi  ← one per DISTINCT initrd
  loader/entries/nixos-<id>.conf                          ← one per generation, ~300 bytes
```

Three facts follow from the builder's code:

1. **The cost is the distinct kernels and initrds, not the generations.** `boot_path()` names each copy after its store path. Ten generations that share one kernel and one initrd cost one copy of each, but each kernel update or initrd change adds another 30 MiB or so.
2. **Which generations get files is decided by the profile, filtered by `configurationLimit`.** The builder lists `/nix/var/nix/profiles/system` and keeps the newest N.
3. **The ESP is only cleaned when the builder runs.** It deletes ESP files that are not in the keep set *before* copying new ones. Deleting generations, by hand or with `nix-collect-garbage -d`, changes the profile. It does not touch `/boot`.

The code that decides

`get_generations()` ends like this (source):

```
    configurationLimit = CONFIGURATION_LIMIT
    ...
    return configurations[-configurationLimit:]
```

The module passes the option through with `configurationLimit = if cfg.configurationLimit == null then 0 else cfg.configurationLimit;`. With the default `null`, the builder computes `configurations[-0:]`, and since `-0 == 0` in Python, that is the **whole list**. The option's own description says it is "Useful to prevent boot partition running out of disk space", and its default is no limit at all:

Boot loader	Option	Default
systemd-boot	`boot.loader.systemd-boot.configurationLimit`	`null` (all generations)
GRUB	`boot.loader.grub.configurationLimit`	`100`
extlinux (`generic-extlinux-compatible`)	`boot.loader.generic-extlinux-compatible.configurationLimit`	`20`

In `install_bootloader()` the order is:

```
    garbage_collect(boot_files)
    ...
    write_boot_files(boot_files, critical_paths)
    ...
```

```
    write_loader_conf(default_entry_id)
    ...
    remove_extra_files()
```

`garbage_collect` removes every file in `EFI/nixos/` and every `loader/entries/nixos-*.conf` that the current list of generations does not need. `write_boot_files` deduplicates by destination path, and its `CopyWriter` skips any file that already exists. Otherwise it copies into a temporary file in the same directory, `fsync`s it and `rename`s it into place. Before any of this, the script refuses to run when the profile lists no generations at all (`error: no system generations found ..., refusing to remove all boot loader entries`).

Lab: the real builder against a 96 MiB ESP

The NixOS manual's install guide creates a 512 MiB ESP (`mkpart ESP fat32 1MB 512MB`). To keep the lab fast, everything is scaled down: a 96 MiB tmpfs ESP, a 13 MiB "kernel" and an 18 MiB "initrd" of random bytes. That gives the same ratio: about three kernel and initrd pairs fit, and a fourth does not.

The driver loads `systemd-boot-builder.py` with its `@placeholders@` filled in. It replaces two things: `run()`, so that `nix-env --list-generations` prints the lab's generation numbers, and `generation_dir()`, so that profiles live under `/tmp/bootlab`. Then it calls the builder's own `get_generations`, `boot_file`, `garbage_collect` and `write_boot_files` in the same order as `install_bootloader()`. It leaves out bootctl and `loader.conf`.

```
# esp_lab.py (excerpt) - the rest builds the fake store and prints the ESP state
src = Path("builder.py").read_text()
for k, v in {"@efiSysMountPoint@": str(ESP), "@bootMountPoint@": str(ESP), "@nixosDir@": "/E
             "@configurationLimit@": "0", "@storeDir@": str(STORE), "@distroName@": "NixOS",
             "@timeout@": "5", "@bootCountingTries@": "3", "@bootCounting@": "False"}.items(
    src = src.replace(k, v)
b = types.ModuleType("builder"); exec(compile(src, "systemd-boot-builder.py", "exec"), b.__d

b.run = fake_run                                    # prints GENS like nix-env --list-generat
b.generation_dir = lambda profile, gen: PROFILES / f"system-{gen}-link"

def install(limit):
    """The core of install_bootloader(), minus bootctl and loader.conf."""
    b.CONFIGURATION_LIMIT = limit
    gens = b.get_generations()
    files = []
    for g in gens:
        files.extend(b.boot_file(*g, None, b.get_bootspec(g.profile, g.generation))[0])
    b.garbage_collect(files)
    b.write_boot_files(files, set())
    return [g.generation for g in gens]
```

```
# Eight generations; the kernel changes three times, the initrd once more on its own.
plan = [(1, "6.12.50", 1), (2, "6.12.50", 1), (3, "6.12.50", 1), (4, "6.12.54", 1),
        (5, "6.12.54", 2), (6, "6.12.54", 2), (7, "6.12.58", 1), (8, "6.12.58", 1)]
```

It runs as an unprivileged user inside a private mount namespace:

```
unshare -rm sh -c 'mkdir -p /tmp/bootlab/esp &&
  mount -t tmpfs -o size=96m lab-esp /tmp/bootlab/esp && python3 esp_lab.py'
```

```
1) configurationLimit = null (the builder sees 0), generations [1, 2, 3, 4]
  ESP  62.0 of 96 MiB used, 2 kernels, 2 initrds, 4 entries
2) four more generations, still no limit: OSError: [Errno 28] No space left on device
  ESP  96.0 of 96 MiB used, 3 kernels, 3 initrds, 6 entries, leftover ['iiiiiiii61258r1-init
3) no generation deleted, configurationLimit = 3: [6, 7, 8]
  ESP  62.0 of 96 MiB used, 2 kernels, 2 initrds, 3 entries
   profile still lists [1, 2, 3, 4, 5, 6, 7, 8] -- the store keeps all of them until GC
4) what is on the ESP now:
   EFI/nixos/iiiiiiii61254r2-initrd-linux-6.12.54-initrd.efi  18 MiB
   EFI/nixos/iiiiiiii61258r1-initrd-linux-6.12.58-initrd.efi  18 MiB
   EFI/nixos/kkkkkkkk61254-linux-6.12.54-bzImage.efi  13 MiB
   EFI/nixos/kkkkkkkk61258-linux-6.12.58-bzImage.efi  13 MiB
   loader/entries/nixos-352e28cdc428c9072ddc293e607a848b9ae72319ad54a966a9d45cc4fef4f95f.con
   loader/entries/nixos-b1572c14203fc5ede08d627cc292ce38c8ea27b39d77136c20fb24affdbd32be.con
   loader/entries/nixos-c3a35b9bc97e0cc4c3e51138b42086d607308cdba91b07d59fd44ecbaf4e0114.con
5) one entry (nixos-c3a35b9bc97e...):
   title NixOS
   version Generation 6 26.05 (Linux 6.12.54), built on 2026-10-31
   linux /EFI/nixos/kkkkkkkk61254-linux-6.12.54-bzImage.efi
   initrd /EFI/nixos/iiiiiiii61254r2-initrd-linux-6.12.54-initrd.efi
   options init=/tmp/bootlab/store/00000006-nixos-system-lab-26.05/init quiet
   sort-key nixos
```

The store paths in the lab are fake (`kkkkkkkk…`, `iiiiiiii…`), so the file names are short. On a real host they start with the 32-character store hash. Reading the run:

- **Step 1.** Four generations cost two kernels and two initrds (62 MiB), not four of each. Generations 1 to 3 share one pair, and generation 4 brought kernel 6.12.54.
- **Step 2.** Generation 5 changed only the initrd, which is another 18 MiB. Generation 7 brought kernel 6.12.58. The builder copied that kernel, then ran out of space part-way through the matching initrd. It raised `ENOSPC` and left a partial `….tmp` file behind. Entries for generations 7 and 8 were never written, because a generation's entry comes after its files in the builder's list. A real `nixos-rebuild` stops at this point with the same Python `OSError`.
- **Step 3.** With `configurationLimit = 3` and the **same** profile, the next builder run succeeded. Its garbage collection runs first, so it removed the 6.12.50 pair, the generation-4 initrd and the leftover `.tmp` before copying the 6.12.58 initrd. That ordering is why lowering the limit can fix a full ESP without any manual deletion, as long as the generations you keep fit.
- **Step 5.** The entry's `options` line points `init=` into `/nix/store`. The ESP holds only the

kernel and initrd. Everything after the initrd comes from the store.

The trap: garbage collection that forgets the ESP

`nix-collect-garbage -d` deletes old profile generations and then the store paths that nothing refers to any more (Nix manual). It does not run the boot loader installer, so the ESP keeps:

- the kernels and initrds of the deleted generations, which wastes space that the next rebuild may need;
- menu entries for the deleted generations. Their `init=` points at a store path that no longer exists, so picking one in the menu loads the kernel and initrd and then fails to find `init`.

The ESP catches up the next time the builder runs: `nixos-rebuild boot` or `switch`, or, with no rebuild at all, `/run/current-system/bin/switch-to-configuration boot`. `nix.gc.automatic` has the same gap. It frees the store on a timer, and `/boot` waits for your next rebuild.

The boring rule

Set `boot.loader.systemd-boot.configurationLimit` to the number of generations you would actually roll back to. Ten is plenty for most hosts, and it caps the ESP at ten distinct kernel and initrd pairs. After pruning generations, run `switch-to-configuration boot` so `/boot` matches the profile. If the ESP is still too small for the kernels you keep, put the copies on a larger partition with `boot.loader.systemd-boot.xbootldrMountPoint`. Its description says it stores "boot loader entries and extra files instead of the EFI partition".

Try this (not run here)

On a systemd-boot NixOS host:

```
df -h /boot                                                  # how full is the ESP?
ls /boot/EFI/nixos | wc -l; du -sh /boot/EFI/nixos           # copies currently held
ls /boot/loader/entries | wc -l                              # menu entries
nix-env --list-generations -p /nix/var/nix/profiles/system # what the builder will see
```

To recover a host whose rebuild failed with *No space left on device*:

```
boot.loader.systemd-boot.configurationLimit = 10;
```

```
sudo nix-collect-garbage --delete-older-than 14d   # optional: free the store as well
sudo nixos-rebuild boot                            # builder GCs the ESP first, then copies
df -h /boot
```

If even the kept generations do not fit, delete more generations first (`sudo nix-env --delete-generations +3 -p /nix/var/nix/profiles/system`), then run `sudo /run/current-system/bin/switch-to-configuratio`

`boot`. Deleting `*.efi` files from `/boot/EFI/nixos` by hand also works, but leave the files that belong to the running generation and the default entry alone. The builder will copy anything that is missing on its next run.

Sources

- nixpkgs, `systemd-boot-builder.py` at `1c89083`: https://github.com/NixOS/nixpkgs/blob/1c89083e451e891152365b799595cf7790a43325/nixos/modules/system/boot/loader/systemd-boot/systemd-boot-builder.py
- nixpkgs, `systemd-boot.nix` (options `configurationLimit`, `xbootldrMountPoint`) at `1c89083`: https://github.com/NixOS/nixpkgs/blob/1c89083e451e891152365b799595cf7790a43325/nixos/modules/system/boot/loader/systemd-boot/systemd-boot.nix
- NixOS options, `boot.loader.systemd-boot.configurationLimit`: https://nixos.org/manual/nixos/stable/options#opt-boot.loader.systemd-boot.configurationLimit
- nixpkgs, GRUB and extlinux `configurationLimit` defaults: `nixos/modules/system/boot/loader/grub/gr` `nixos/modules/system/boot/loader/generic-extlinux-compatible/default.nix` at `1c89083`
- NixOS manual, installation (512 MiB ESP): https://github.com/NixOS/nixpkgs/blob/1c89083e451e891152365b799595cf7790a43325/nixos/doc/manual/installation/installing.chapter.md
- Nix manual, `nix-collect-garbage`: https://nix.dev/manual/nix/latest/command-ref/nix-collect-garbage
- UAPI Group, Boot Loader Specification (type #1 entries): https://uapi-group.org/specifications/specs/boot_loader_specification/

Home and Flake Layout

Home Manager and flake layout

Home Manager and flake layout

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

Baseline: NixOS / nixpkgs **26.05**, flakes, modern `nix` CLI, Home Manager as a **NixOS module**.

What this part is for

You already own a host from a flake (`nixosConfigurations`). That is necessary and insufficient for a maintainable personal platform. This part covers the **shape** of a multi-concern flake and the **user plane**:

Concern	Chapters
Repo layout that scales past one file	Flake repository layout
Pins you can explain and revert	Lock file discipline
User environment as NixOS module	Home Manager on NixOS
Dotfiles, XDG, store-backed files	Home files and XDG
User systemd units and timers	Home Manager user services
Project toolchains without profile bloat	direnv and project shells
Multi-output flake structure	flake-parts (optional structure)
Local quality gates	Flake checks
Shared package tweaks	Overlays (light introduction)

After this part you should be able to hand a second host or a second user a **predictable import graph**, activate system + home with one rebuild, and keep project compilers out of the always-on profile.

Prerequisites

From earlier parts of this book you should already be comfortable with:

- Store paths, closures, and why the store is not private
- Attrsets, modules, and `nixosSystem`
- A working flake host: `sudo nixos-rebuild switch --flake .#<host>`
- Generations and rollback
- Basic SSH/user config on the host

If those are shaky, return to **concepts** and **NixOS host** before deepening layout and Home Manager.

Chapter map

Chapter	Focus	Outcome
Flake repository layout	`hosts/` · `modules/` · `homes/` DAG	Thin `flake.nix`; rebuild-equivalent split
Lock file discipline	Selective updates, `follows`, registries	Written lock policy; revert skill
Home Manager on NixOS	HM as NixOS module, 26.05 pin	One rebuild activates system + user
Home files and XDG	`home.file`, `xdg.configFile`	Real configs migrated; secrets out
Home Manager user services	`systemd.user.*`	Timers/services that survive reinstall
direnv and project shells	`devShells` + `use flake`	Toolchains load on `cd`
flake-parts (optional structure)	Framework vs hand-rolled helpers	House style documented
Flake checks	`checks` + `nix flake check`	Format/shell/host gates
Overlays (light introduction)	Small wrappers / overrides	Same overlay on host + shell

---

Mental model — three planes

```
flake.nix  (wiring only)
  inputs → nixosConfigurations / shells


hosts/       modules/      homes/
(machine)    (policy)      (user / HM)
```

Plane	Truth lives in	Activated by
Host	`hosts/<name>/`, hardware	`nixos-rebuild`
Policy	`modules/common`, roles, services	imported by hosts
User	`homes/<user>/` via Home Manager	same rebuild (module mode)

Rule: host never imported by a common module; user config does not replace system daemons; project compilers do not live forever in `home.packages`.

Dependency direction (import DAG)

```
flake.nix
  → hosts/lab
       → modules/common/*
       → modules/roles/*
       → modules/services/*
       → hardware-configuration
       → home-manager.users.<name> → homes/<name>
```

Allowed	Forbidden
Host imports modules	Module imports a specific host
Role imports common	Common imports role
Service module self-contained	Service reaches into `hosts/lab`
Flake wires HM users	Homes import host hardware

Circular graphs fail as infinite recursion or missing attributes. Design the DAG **before** splitting files.

Suggested repo tree (target shape)

```
flake.nix
flake.lock
hosts/
  lab/
    default.nix
    hardware-configuration.nix
modules/
  common/
    default.nix
    nix.nix
    users.nix
    ssh.nix
  roles/
    server.nix
    workstation.nix
  services/
homes/
  alice/
    default.nix
    programs.nix
    files.nix
    services.nix
overlays/
  default.nix
lib/
docs/
  layout.md
  lock-policy.md
  flake-style.md
```

Names are conventions; **layering** is the invariant.

How to work this part

1. **Layout first** — behavior-preserving split of an existing host flake.
2. **Lock policy** — before adding HM, sops, or flake-parts inputs casually.
3. **HM module mode** — one activation path with the host.
4. **Files and services** — migrate real user state deliberately.
5. **direnv** — project shells, not profile bloat.
6. **Structure + checks + light overlays** — multi-output hygiene.

Prefer small commits: move-only, then feature. Do not mix layout refactors with hardening or new network services.

Cross-cutting conventions (26.05)

Topic	Convention in this book
nixpkgs	`github:NixOS/nixpkgs/nixos-26.05`
Home Manager	`github:nix-community/home-manager/release-26.05` + `follows`
HM activation	**NixOS module** (`home-manager.nixosModules.home-manager`)
`useGlobalPkgs`	`true` so overlays reach HM
Secrets in homes	**Never** plaintext tokens in `home.file` / `text`
Project tools	flake `devShells` + direnv, not eternal `home.packages`

What this part does *not* cover

Topic	Where it lives
sops-nix / agenix	Services and security patterns
Fleet deploy, Disko, CI	Ops and fleet
Full packaging / overlays mastery	Packaging
Day-calendar curriculum	Not this book's model

Skills checklist (end of part)

- ☐ Thin flake wiring; hosts / modules / homes tree
- ☐ Import DAG has no reverse host deps
- ☐ Lock policy written; selective update + revert practiced
- ☐ HM module mode: git + shell for at least one user
- ☐ Two real configs via XDG / `home.file` or `programs.*`
- ☐ One user timer or service via HM
- ☐ One project with `use flake` + direnv
- ☐ House flake style (hand-rolled or flake-parts) documented
- ☐ `nix flake check` fails on intentional format break
- ☐ One light overlay or wrapper visible on host and shell

How chapters are structured

Each topic chapter typically includes:

1. **Goal** and callouts (baseline, pitfalls)
2. **Why this chapter exists** — problem framing
3. **Theory** sections with tables and full Nix snippets
4. **Worked examples** you can paste and adapt
5. **Exercises** (8–15) with verify commands
6. **Common gotchas**
7. **Checkpoint** for self-assessment

Indexes (this file and part openers) orient; topic chapters go deep.

Reading strategies

Mode	Approach
First pass	Layout → lock → HM → files → services → direnv; then style/checks/overlays
Reference	Jump to gotchas + worked example for the tool you need
Audit existing flake	Layout + lock + HM boundary chapters first
Laptop vs server	Roles chapter patterns; lighter HM on servers

---

Lab host assumptions

Examples use:

- Host attr / hostname: `lab`
- User: `alice` (replace with yours)
- System: `x86_64-linux` (swap for `aarch64-linux` as needed)
- Rebuild: `sudo nixos-rebuild switch --flake .#lab`

Paths under `~/lab/...` are suggestions only—use your real flake repo.

Next part

Services and security patterns builds on this shape: secrets (sops-nix), hardening, firewall, TLS front door, data services, containers, observability. Layout and HM must be boring before those topics get loud.

Flake repository layout

Flake repository layout

Goal: Refactor a single-host flake into a **hosts / modules / homes** layout that stays rebuildable, readable, and ready for multi-host growth—without a 2 000-line `flake.nix`.

> **Note**
>
> **Baseline:** NixOS / nixpkgs **26.05**, flakes, modern `nix` CLI. Home Manager is wired in the Home Manager chapter; in this chapter only **create the `homes/` slot** so later imports have a home.

Why this chapter exists

Owning one machine from a flake proves control. **Shape** is what keeps that control after a second host, a second user, or the first "extract this SSH policy" refactor.

A common failure mode:

```
flake.nix          # 1800 lines of mixed host, packages, users, networking
hardware-configuration.nix
```

That works until the import graph becomes a ball of yarn. Layout is not aesthetics; it is **dependency direction and change-surface control**.

Theory 1 — What "layout" means in a flake monorepo

A flake has two layers:

Layer	Role
`flake.nix` outputs	Thin wiring: inputs → `nixosConfigurations`, later `devShells` / `checks` / `packages`
Module tree	Almost all real config: options + config fragments imported by hosts

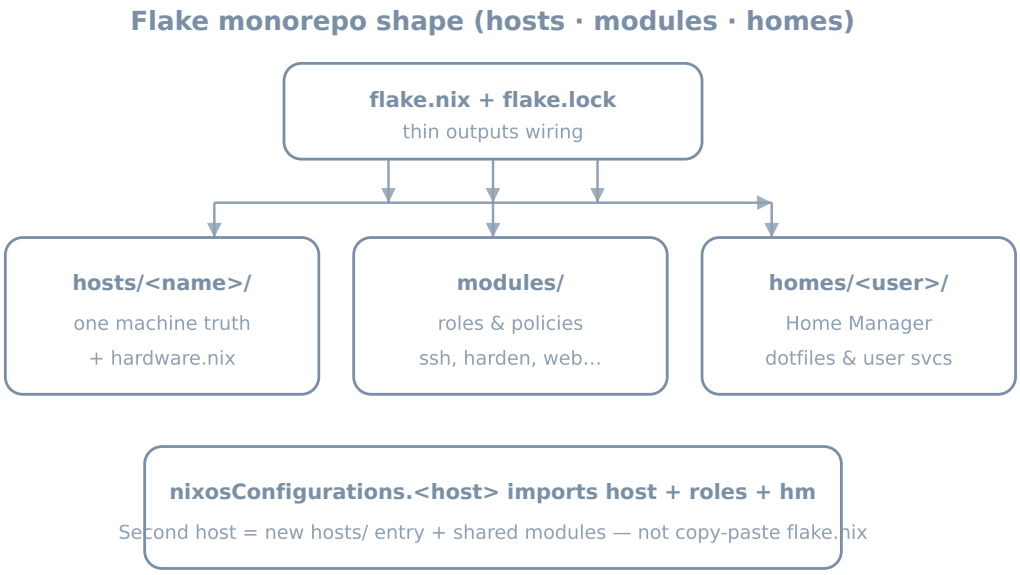


Figure 1: Flake hosts modules homes layout

Rule of thumb: if a file is only true for *one machine*, it lives under `hosts/<name>/`. If it is true for *a role or policy*, it lives under `modules/`. If it is true for *a user*, it will live under `homes/` (Home Manager starts in the next chapters).

Target tree

```
flake.nix
flake.lock
hosts/
  lab/
    default.nix              # host entry: imports modules + hardware
    hardware-configuration.nix
modules/
  common/
    default.nix              # baseline shared by all hosts
    nix.nix
    users.nix
    ssh.nix
    networking.nix
  roles/
    workstation.nix
    server.nix
  services/
    # empty or one tiny service extracted from the monolith
homes/
```

```
  alice/
    default.nix              # stub until HM chapter
lib/
  default.nix                # optional helpers (mkHost, …)
docs/
  layout.md
```

Names are conventions, not law. **The invariant is layering**, not exact folder names.

Theory 2 — Dependency direction (prevent circular imports)

Think of imports as a DAG:

```
flake.nix
  → hosts/lab
       → modules/common/*
       → modules/roles/*
       → modules/services/*
       → hardware-configuration
       → (later) homes via HM module
```

Allowed	Forbidden
Host imports modules	Module imports a *specific* host
Role imports common modules	Common imports role
Service module is self-contained	Service module reaches into `hosts/lab` paths
`lib` pure helpers used by modules	`lib` that evaluates full NixOS configs

Circular imports fail evaluation with confusing "infinite recursion" or "attribute missing" symptoms. Design the graph **before** you split files blindly.

What belongs where

Concern	Place
`networking.hostName`, hardware, disk labels	`hosts/<name>/`
SSH policy, firewall defaults, time zone, locale	`modules/common/`
"This is a headless server" package set	`modules/roles/server.nix`
One service option bundle	`modules/services/<name>.nix`
User shell, git, editors	`homes/<user>/`
`specialArgs`, input plumbing	`flake.nix` only

Theory 3 — Host entry pattern

A host file is a **module**, not a second flake:

```
# hosts/lab/default.nix
{ config, pkgs, lib, ... }:
{
  imports = [
    ./hardware-configuration.nix
    ../../modules/common
    ../../modules/roles/server.nix
    # ../../modules/services/my-timer.nix
  ];

  networking.hostName = "lab";

  # Host-only overrides (mkForce / mkDefault as needed)
  # services.openssh.openFirewall = true;
}
```

And the flake stays thin:

```
# flake.nix
{
  description = "NixOS lab - hosts/modules/homes layout";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  };

  outputs = { self, nixpkgs, ... }:
  let
    system = "x86_64-linux"; # or aarch64-linux
  in {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      inherit system;
      modules = [
        ./hosts/lab
      ];
      # specialArgs = { inherit inputs; }; # when modules need inputs
    };
  };
}
```

`default.nix` vs explicit path

Importing `./hosts/lab` loads `./hosts/lab/default.nix`. Prefer that over listing every leaf in `flake.nix`.

`specialArgs` vs module arguments

Mechanism	Use
`specialArgs = { inherit inputs; }`	Pass flake inputs into all host modules
`_module.args`	More advanced; avoid until needed
`import` of pure `lib/`	Helpers that need no `config`

Do not thread half the world through `specialArgs` "just in case." Add when a module actually needs an input (Home Manager, sops-nix, …).

Theory 4 — Module "barrel" files

`modules/common/default.nix` re-exports fragments:

```nix
# modules/common/default.nix
{
  imports = [
    ./nix.nix
    ./users.nix
    ./ssh.nix
    ./networking.nix
  ];
}
```

Benefits:

- Hosts import **one** common entry
- You can still open a single policy file for review
- Diffs stay small when only SSH changes

Refactor **behavior-preserving** first: cut → import → rebuild → only then change options. Never mix layout moves with hardening or new services in one commit.

Optional deeper split

```
modules/common/
  default.nix
  nix.nix
  users.nix
  ssh.nix
  networking.nix
  locale.nix
  time.nix
```

Split when a file exceeds ~150 lines or two people edit conflicting concerns—not on principle alone.

Theory 5 — Hardware config and purity

`hardware-configuration.nix` is **host-local** (UUIDs, firmware). Do **not** share it across machines.

Practice	Why
Keep under `hosts/<name>/`	Machine-specific
Generated by installer / `nixos-generate-config`	Not hand-invented blindly
Import only from that host	Avoid wrong disk UUIDs

Optional `lib/mkHost` helpers can wait until you have three hosts—prefer an obvious explicit `nixosSystem` until then.

Before → after: a monolith `configuration.nix` pointed from `flake.nix`; after, `flake.nix` points at `./hosts/lab`, which imports hardware + `modules/common` (+ roles/services). Same behavior; smaller change surface.

Theory 6 — Roles vs common vs services

Kind	Semantics	Example
common	True for nearly every host	flakes enabled, base SSH policy
role	Class of machine	`server.nix` disables Bluetooth; `workstation.nix` enables NetworkManager
service	One workload option bundle	`modules/services/caddy.nix`

```
# modules/roles/server.nix
{
  imports = [ ../common ];
  # server-lean defaults
  # hardware.bluetooth.enable = false;
  # services.printing.enable = false;
}
```

Hosts compose roles; roles compose common. Services are imported by hosts or roles when that workload is on.

Theory 7 — Multi-host readiness (without building the second machine)

Sketch without wiring a non-bootable config:

```
hosts/
  lab/
  spare/          # optional empty skeleton
    default.nix   # imports common + own hardware later
```

```
# flake.nix (future)
nixosConfigurations = {
  lab = nixpkgs.lib.nixosSystem { … modules = [ ./hosts/lab ]; };
  # spare = nixpkgs.lib.nixosSystem { … modules = [ ./hosts/spare ]; };
};
```

Do not invent fake hardware UUIDs "for symmetry." Document the intended second host in `docs/layout.md` instead.

Theory 8 — Relative paths and flake root

After moves, relative imports break silently until eval:

From	Import
`hosts/lab/default.nix`	`../../modules/common`
`modules/common/ssh.nix`	no host paths
`flake.nix`	`./hosts/lab` only

Avoid absolute `/home/you/...` imports. Flake evaluation should work from any clone path.

Worked example — Minimal SSH + users + nix split

```
# modules/common/ssh.nix
{ lib, ... }:
{
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "no";
    };
  };
}
```

```
# modules/common/users.nix
{ pkgs, ... }:
{
  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    openssh.authorizedKeys.keys = [
      "ssh-ed25519 AAAA...REPLACE_WITH_YOUR_PUBLIC_KEY alice@lab"
    ];
  };
}
```

```
# modules/common/nix.nix
{
  nix.settings = {
    experimental-features = [ "nix-command" "flakes" ];
  };
  nixpkgs.config.allowUnfree = false;
}
```

```
# modules/common/networking.nix
{
  # hostName stays on the host; put only shared defaults here
  networking.firewall.enable = true;
  time.timeZone = "UTC"; # or your zone
```

```
}
```

```
# hosts/lab/default.nix
{ ... }:
{
  imports = [
    ./hardware-configuration.nix
    ../../modules/common
  ];

  networking.hostName = "lab";
  system.stateVersion = "26.05"; # keep your real stateVersion
}
```

Worked example — Thin flake with optional `lib`

```
# lib/systems.nix
{
  defaultSystem = "x86_64-linux";
}
```

```
# flake.nix
{
  description = "NixOS lab";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs, ... }:
  let
    inherit (import ./lib/systems.nix) defaultSystem;
  in {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = defaultSystem;
      modules = [ ./hosts/lab ];
    };
  };
}
```

Keep `lib/` free of full system evaluation until you know you need it.

Exercises

Exercise 1 — Inventory the monolith

```
cd /path/to/your-flake
find . -name '*.nix' | sort
wc -l flake.nix configuration.nix hosts/*/*.nix 2>/dev/null || true
```

Journal:

1. List every *concern* in the big file (users, SSH, packages, networking, …)
2. Mark each as **host / common / role / service / user**
3. Note any absolute paths or secrets (flag secrets for the secrets chapters—do not leave plaintext passwords in git)

Exercise 2 — Create the directory skeleton

```
mkdir -p hosts/lab modules/common modules/roles modules/services homes/alice lib docs
```

Move hardware config:

```
# adjust names to your layout
mv hardware-configuration.nix hosts/lab/ 2>/dev/null || true
cp configuration.nix /tmp/configuration.nix.bak 2>/dev/null || true
```

Exercise 3 — Split common modules

Create `ssh.nix`, `users.nix`, `nix.nix`, `networking.nix`, and the barrel `default.nix` from the worked examples. Replace keys and usernames with yours—**public keys only**.

Exercise 4 — Host entry + thin flake

Write `hosts/lab/default.nix`. Point `flake.nix` at `./hosts/lab` only. Remove or stop importing the old monolith path.

Exercise 5 — Build before switch

```
nixos-rebuild build --flake .#lab
sudo nixos-rebuild switch --flake .#lab
```

Verify:

```
hostname
systemctl is-active sshd
nixos-version
```

If something broke: roll back, fix one module, retry.

Exercise 6 — Document the tree

Write `docs/layout.md`:

```
flake.nix          → outputs only
hosts/lab          → hostname, hardware, host overrides
modules/common     → shared policy
modules/roles      → (empty or one role)
modules/services   → extracted service if any
homes/alice        → Home Manager chapters
```

Exercise 7 — Role extract (optional stretch)

Move 2–3 server-lean settings into `modules/roles/server.nix` and import from the host. Rebuild. Confirm behavior unchanged.

Exercise 8 — Path-break drill

Temporarily wrong a relative import (`../common` instead of `../../modules/common`). Run `nixos-rebuild build`, read the error, fix. Note the message for future you.

Exercise 9 — Diff against backup

```
# conceptual: compare options that matter
diff -u /tmp/configuration.nix.bak hosts/lab/default.nix | head -80 || true
```

List any intentional deltas; there should be none for a pure layout move.

Exercise 10 — Second-host skeleton (docs only)

Write five lines in `docs/layout.md` about what a second host would import. Do not invent hardware.

Exercise 11 — Secrets flag list

From Exercise 1, list every sensitive-looking string. Either remove now or ticket for secrets chapters. No live tokens in git.

Exercise 12 — Commit hygiene

```
git add hosts modules homes lib docs flake.nix flake.lock
git status
git commit -m "flake: hosts/modules/homes layout (behavior-preserving)"
```

Prefer one commit for layout-only moves.

Common gotchas

Symptom / mistake	What to do
Infinite recursion after split	Module imports a host or `config` cycle; straighten the DAG
`error: path '…' does not exist`	Relative paths wrong after move; re-check `../../modules`
Flake still points at old `configuration.nix`	Update `modules = [ ./hosts/lab ];`
Hardware UUIDs missing	Moved hardware file but forgot to import it
"Works on eval, fails on boot"	Bootloader/filesystem only in hardware host file—verify
Mixed absolute `/home/you/...` imports	Use flake-relative paths only
Giant PR of layout + features	Split commits: move-only first
`stateVersion` changed "to match release"	Do not casual-bump; keep original
Barrel forgets a file	Missing policy after split—add to `imports`
Role imports host	Invert: host imports role

Checkpoint

- ☐ Directory tree matches hosts / modules / homes (homes may be stub)
- ☐ `flake.nix` is thin wiring, not the bulk of config
- ☐ `nixos-rebuild switch --flake .#lab` (or your hostname) succeeds
- ☐ Import graph has no host→module reverse deps
- ☐ Layout documented in `docs/layout.md`
- ☐ Behavior roughly matches pre-refactor host
- ☐ No plaintext secrets introduced during the move
- ☐ stateVersion unchanged unless you know why

Journal (optional)

Write three personal gotchas before continuing—especially path errors and anything you almost put in the wrong plane.

Lock file discipline

Lock file discipline

Goal: Treat `flake.lock` as a **deliberate pin**, not a mysterious generated blob—update **one input at a time**, read diffs, and explain how locks interact with registries and multi-machine rebuilds.

> **!** Important
>
> Blind `nix flake update` on a production host is how "nothing changed except everything." Prefer **staged, selective updates**.

Why this chapter exists

Flakes solve channel drift **only if** you respect the lock file. Teams fail when:

- Someone updates all inputs "because CI was green last week"
- Lock files are gitignored (never do this for apps/hosts)
- Two machines use different locks and argue about "Nix being unreproducible"
- Registries silently redirect `nixpkgs` away from the flake's pin

Lock hygiene is foundational for every later input (Home Manager, sops-nix, flake-parts, …).

Theory 1 — What `flake.lock` actually pins

For each flake input, the lock records roughly:

Field	Meaning
`type` / `owner` / `repo` / `url`	Where the input came from
`rev`	Exact git commit (or equivalent)
`narHash`	Content hash of the fetched tree
`lastModified`	Metadata for humans/tools

Implication: two clones with the **same lock** should fetch the **same input trees** (modulo substituted vs rebuilt outputs later). Without the lock, `nixos-26.05` is a **moving branch tip**.

Inputs vs follow-ons

```
inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  home-manager.url = "github:nix-community/home-manager/release-26.05";
  home-manager.inputs.nixpkgs.follows = "nixpkgs";
};
```

`follows` means Home Manager reuses **your** nixpkgs pin instead of its own nested lock entry as a separate copy. That reduces "two nixpkgs versions in one eval" confusion and often saves eval work.

Anatomy of a lock node (conceptual)

```
"nixpkgs": {
  "locked": {
    "lastModified": 1700000000,
    "narHash": "sha256-…",
    "owner": "NixOS",
    "repo": "nixpkgs",
    "rev": "abcdef…",
    "type": "github"
  },
  "original": {
    "owner": "NixOS",
    "ref": "nixos-26.05",
    "repo": "nixpkgs",
    "type": "github"
  }
}
```

`original` is policy (track `nixos-26.05`); `locked` is the decision (this commit).

Theory 2 — Update operations (precision toolkit)

Command	Effect
`nix flake lock`	Refresh lock metadata without changing intended refs (often no-op if already locked)
`nix flake update`	Update **all** inputs to latest matching their `url` refs
`nix flake update nixpkgs`	Update **only** the `nixpkgs` input

Command	Effect
`nix flake update home-manager`	Update only that input
`nix flake metadata`	Show resolved pins / last modified

Modern CLI also supports updating multiple named inputs; check `nix flake update --help` on your Nix version.

Mental model

```
flake.nix   →  policy ("track nixos-26.05")
flake.lock  →  decision ("this commit of that policy")
```

Changing `flake.nix` URLs without updating lock can error or keep stale locks depending on change type—always re-lock intentionally and commit both files together when inputs change.

Theory 3 — How to read a lock diff

```
git diff flake.lock | head -200
```

Look for:

1. **Which input** changed (`"nixpkgs": {` block)
2. **Old `rev` vs new `rev`**
3. **`narHash` change** (expected when rev changes)
4. Nested inputs (if not `follows`)

Optional deep inspection:

```
nix flake metadata
nix flake metadata --json | jq '.locks.nodes.nixpkgs'
```

Map `rev` to a GitHub compare URL when investigating regressions:

```
https://github.com/NixOS/nixpkgs/compare/<old>...<new>
```

Nested duplicates

If you see `nixpkgs_2` style nodes, a dependency brought its own nixpkgs. Prefer:

```
inputs.foo.inputs.nixpkgs.follows = "nixpkgs";
```

Then re-lock and confirm the duplicate disappears or shrinks.

Theory 4 — Pin strategies (pick a house rule)

Strategy	When	Risk
Track release branch (`nixos-26.05`) + lock commits	Default for this book	Branch tip moves; you choose when to update
Pin exact rev in `flake.nix`	Freeze hard for a milestone	Manual bumps; easy to forget security updates
Track `nixos-unstable`	Bleeding edge lab	Breakage rate higher
Multiple flakes (host vs tools)	Advanced separation	Operational complexity

Recommended for lab hosts: track `nixos-26.05` (and matching HM release), **commit lock**, update **one input per change window**, rebuild + smoke-test before updating the next.

Security vs stability

Updating nixpkgs is how you receive package and module fixes. Never freeze a public-facing host forever "because it works." Discipline is **when and how**, not **whether**.

Cadence idea	Fit
Weekly selective nixpkgs on lab	Learning hosts
PR-based lock bumps in CI	Teams
Point release freezes for demos	Talks / workshops
Unattended full `flake update` nightly	Usually a bad default for hosts

Theory 5 — Registries (light literacy)

Flake **registries** map short names (`nixpkgs`) to URLs for ad-hoc commands like `nix run nixpkgs#hello`.

Concern	Practice
Global registry	Convenient for exploration
Your host flake	Should use **explicit inputs**, not hope the registry matches
CI	Prefer locked flake in repo over registry luck

```
nix registry list
```

If global `nixpkgs` points at unstable while your flake pins 26.05, **that is fine** for one-off `nix run`—as long as you never assume those are the same pin.

Registry vs input (summary)

```
nix run nixpkgs#hello     → registry resolution (may float)
nix build .#packages…     → your flake.lock
nixos-rebuild --flake .#  → your flake.lock
```

Theory 6 — Multi-machine and "reproducible"

Same lock + same system (`x86_64-linux`) → same **evaluation** of inputs. Realization may still:

- Download substitutes vs build locally
- Differ if impure eval sneaks in (`builtins.currentSystem` misuse, unpinned `fetchTarball`, IFD surprises)
- Differ across architectures

Lock discipline is **necessary, not always sufficient**, for bit-identical everything—but it is the main lever you control daily.

Clone checklist

```
git clone <repo>
cd <repo>
git status   # flake.lock must be present
nix flake metadata
```

```
sudo nixos-rebuild switch --flake .#lab
```

Missing lock → everyone invents their own "26.05 tip."

Theory 7 — Commit policy for locks

Do	Don't
Commit `flake.lock` with the change that needs it	Gitignore lock for NixOS host flakes
One logical update per commit when possible	Mix lock bumps with huge unrelated refactors
Note *why* you updated in commit body	"chore: update" with no smoke test
Keep `flake.nix` + `flake.lock` paired in reverts	Restore only one of the pair

Example commit body:

```
flake: update nixpkgs (26.05 tip)

Smoke: sshd active, rebuild switch OK, hello from devShell OK.
```

Revert pattern

```
git checkout HEAD~1 -- flake.lock
# if flake.nix inputs also changed, restore that too:
# git checkout HEAD~1 -- flake.nix flake.lock
sudo nixos-rebuild switch --flake .#lab
```

Theory 8 — Inputs that should co-move

Pair	Rule of thumb
nixpkgs `nixos-26.05` + HM `release-26.05`	Keep major/minor aligned
nixpkgs + sops-nix	`follows` nixpkgs; update sops after nixpkgs if needed

Pair	Rule of thumb
flake-parts + nixpkgs	Framework usually tracks independently; still `follows` when offered

Updating HM without nixpkgs (or the reverse) is a common source of obscure option errors. Prefer: update nixpkgs → rebuild → update HM → rebuild.

Worked example — Selective update narrative

```
Session A:  update nixpkgs only → rebuild lab → OK
Session B:  add home-manager input → lock appears → rebuild
Session C:  update home-manager only → HM activation issue → revert lock hunk
```

Reverting a bad lock is often faster than debugging three simultaneous input moves.

Worked commands

```
cp flake.lock /tmp/flake.lock.bak
nix flake update nixpkgs
git diff --stat flake.lock
nixos-rebuild build --flake .#lab
# if OK:
sudo nixos-rebuild switch --flake .#lab
# if bad:
cp /tmp/flake.lock.bak flake.lock
sudo nixos-rebuild switch --flake .#lab
```

Exercises

Exercise 1 — Snapshot current pins

```
cd /path/to/your-flake
nix flake metadata
cp flake.lock /tmp/flake.lock.bak
git status
```

Journal:

1. List every input name and its locked `rev` (short hash is fine)
2. Confirm `flake.lock` is tracked by git
3. Note whether any input uses `follows`

Exercise 2 — Read the lock structure

```
nix flake metadata
nix flake metadata --json | jq '.locks.nodes | keys' 2>/dev/null || true
```

Open `flake.lock` and identify:

- The `root` node's inputs
- The `nixpkgs` node's `rev` and `narHash`
- Any nested `nixpkgs_2` style duplicates (candidate for `follows`)

Exercise 3 — Selective update of one input

Only if you accept rebuild churn. On a disposable lab host:

```
nix flake update nixpkgs
git diff --stat flake.lock
git diff flake.lock | head -120
```

Build before switch:

```
nixos-rebuild build --flake .#lab
sudo nixos-rebuild switch --flake .#lab
```

Smoke checklist:

```
systemctl is-system-running || true
systemctl is-active sshd
nix --version
```

If break: restore backup lock and re-switch.

Exercise 4 — Prove "update all" is louder

On a **throwaway branch**:

```
git checkout -b experiment/flake-update-all
nix flake update
git diff --stat flake.lock
```

Count how many inputs / revs moved. Then restore:

```
git checkout main   # or your real branch name
git branch -D experiment/flake-update-all
```

Write one sentence: *why selective update is the default for hosts.*

Exercise 5 — Document house rules

Create `docs/lock-policy.md`:

```
# Lock policy (lab)

- Track nixpkgs nixos-26.05
- Commit flake.lock always
- Update one input per change window
- Smoke: rebuild + SSH + one app path
- Revert lock on failure before deep debug
- Align HM release-* with nixos-*
```

Exercise 6 — Registry awareness (read-only)

```
nix registry list
nix flake metadata
```

Note if the registry's nixpkgs rev differs from the flake's. No need to change registries.

Exercise 7 — follows fix (if needed)

If you have (or add) a second input with its own nixpkgs:

```
inputs.foo.inputs.nixpkgs.follows = "nixpkgs";
```

```
nix flake lock
nix flake metadata --json | jq '.locks.nodes | keys'
```

Exercise 8 — Compare URL construction

Write the GitHub compare URL for your last nixpkgs bump (from Exercise 3) or a synthetic old→new pair from metadata history. Bookmark the pattern.

Exercise 9 — Impurity awareness list

List three things that can still differ with the same lock (substituters, arch, impure builtins). One sentence each.

Exercise 10 — CI anti-pattern note

In `docs/lock-policy.md`, add: "CI must not run `nix flake update` on every job; lock bumps are deliberate PRs."

Exercise 11 — Paired restore drill

```
# simulate bad partial restore understanding
git show HEAD:flake.lock >/dev/null
# document: always restore flake.nix + flake.lock together when input URLs change
```

Exercise 12 — Multi-input update plan

Write the order you will use when adding Home Manager (next chapters): lock HM → `follows` → rebuild → only then update nixpkgs again.

Common gotchas

Symptom / mistake	What to do
Everyone's host differs after clone	Missing or uncommitted `flake.lock`
`follows` not set → two nixpkgs	Add `inputs.foo.inputs.nixpkgs.follows = "nixpkgs"`
Updated lock, forgot rebuild	`switch` / `boot` to activate
CI uses `nix flake update` every run	Pin lock in repo; update in deliberate PRs
"I'm on 26.05" but lock is months old	Expected until you update; schedule updates
Restored `flake.nix` but not lock	Always restore the pair when debugging inputs
Private input auth fails after update	Token/SSH access issue, not lock format
HM release nixpkgs release	Align `release-26.05` with `nixos-26.05`
Reviewed only `flake.nix` in PR	Always review lock diff too
Massive lock + feature PR	Split: lock bump PR vs feature PR

Checkpoint

- ☐ Can explain what `rev` + `narHash` buy you
- ☐ Can update **one** named input and read the lock diff
- ☐ Have a written house lock policy
- ☐ Know how to revert `flake.lock` quickly
- ☐ Understand registry flake pin
- ☐ `follows` planned or applied for companion inputs
- ☐ Know why full `nix flake update` is loud on hosts
- ☐ Commit policy treats lock as first-class

Journal (optional)

Write three personal gotchas before continuing—especially any surprise nested nixpkgs or registry mismatch.

Home Manager on NixOS

Home Manager on NixOS

Goal: Wire **Home Manager as a NixOS module** (not standalone-first), manage one user's shell/git/ssh client config, and rebuild with a single `nixos-rebuild switch --flake` activation story.

> **Note**
>
> **House default for this book:** Home Manager **NixOS module** on NixOS hosts. Standalone HM is for non-NixOS machines or special cases—know it exists; do not make it your lab's primary path.

Why this chapter exists

Host modules put packages on the **system** (`environment.systemPackages`). That is correct for daemons and multi-user tools—and wrong as the only place for **personal** developer config. Home Manager (HM) owns:

- User programs (`programs.git`, `programs.bash` / `zsh` / `fish`, …)
- Dotfiles and XDG (next chapter)
- User systemd services (HM services chapter)

Doing HM as a NixOS module means: **one flake, one rebuild, one generation story** for system + user.

Theory 1 — Three ways people run Home Manager

Mode	How it activates	When to use
NixOS module	Part of `nixos-rebuild`	**Default on NixOS (this book)**
Standalone	`home-manager switch --flake ...`	Other distros; or isolated user experiments
nix-darwin module	macOS analog	Out of scope here

Why module mode first

- Aligns HM with system generations (easier mental model for labs)
- `users.users.<name>` and `home-manager.users.<name>` stay in one repo
- Avoids "system switched but HM forgotten" dual-track ops

Standalone still matters if you develop on Ubuntu with HM-only—but your **NixOS lab** should not require two tools for every change.

Theory 2 — Input wiring (26.05-aligned)

```nix
# flake.nix (merge with your layout)
{
  description = "NixOS lab with Home Manager";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

    home-manager.url = "github:nix-community/home-manager/release-26.05";
    home-manager.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, home-manager, ... }@inputs: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      specialArgs = { inherit inputs; };
      modules = [
        ./hosts/lab
        home-manager.nixosModules.home-manager
        {
          home-manager.useGlobalPkgs = true;
          home-manager.useUserPackages = true;
          # home-manager.extraSpecialArgs = { inherit inputs; };
          home-manager.users.alice = import ./homes/alice;
        }
      ];
    };
  };
}
```

Option	Meaning
`useGlobalPkgs = true`	HM uses the same `pkgs` as the system (consistent overlays later)
`useUserPackages = true`	User packages install into user profile via HM
`home-manager.users.alice = ...`	That user's HM module tree
`extraSpecialArgs`	Pass flake inputs into HM modules when needed

Version match: HM `release-26.05` with nixpkgs `nixos-26.05`. Mismatched releases are a common source of obscure option errors.

Where to put the HM block

Style	Pros
Inline in `flake.nix` as above	Obvious for one host
Dedicated module `modules/common/home-manager.nix`	Cleaner multi-host
Per-host in `hosts/lab/default.nix`	Different users per machine

All are valid; pick one and document it.

Theory 3 — System user vs HM user

You still need a **NixOS user**:

```
# modules/common/users.nix
{
  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    # shell can be set here OR via HM programs.* - pick one source of truth
  };
}
```

HM configures that user's home:

```
# homes/alice/default.nix
{ pkgs, ... }:
{
  home.username = "alice";
  home.homeDirectory = "/home/alice";
```

```
  home.stateVersion = "26.05"; # set once; do not casually bump

  programs.home-manager.enable = true;

  programs.git = {
    enable = true;
    userName = "Alice Example";
    userEmail = "alice@example.invalid";
  };

  programs.bash = {
    enable = true;
    enableCompletion = true;
  };

  home.packages = with pkgs; [
    ripgrep
    fd
  ];
}
```

`home.stateVersion`

Like NixOS `system.stateVersion`, this is a **compatibility pin** for HM stateful defaults—not "current release marketing." Set it to the HM release you first adopt for that user; read release notes before bumping.

Name and path must match

Field	Must agree with
`home-manager.users.alice`	attr name
`home.username`	`"alice"`
`home.homeDirectory`	actual home, usually `/home/alice`
`users.users.alice`	system user exists

Mismatches produce activation failures or configs landing in the wrong place.

Theory 4 — What belongs in HM vs NixOS

Concern	Prefer
`sshd`, firewall, kernel, system users	NixOS modules
Interactive shell, prompt, git, editor config	HM
CLI tools only you use	HM `home.packages`
Libraries/services shared by system units	NixOS
Secrets for user apps	Secrets chapters (not plaintext in HM)
Network daemons (nginx, postgres)	NixOS
Project compilers	Project `devShells` + direnv (later chapter)

Anti-pattern: dumping everything into `environment.systemPackages` because HM felt optional.
Opposite anti-pattern: putting `nginx` only in HM.

Theory 5 — Activation model (what rebuild does)

On `nixos-rebuild switch` with HM module:

1. System config builds
2. HM config for each defined user builds
3. Activation scripts run: system switch + HM activation for users

Failures can be **system** or **HM**. Read the traceback: `home-manager-alice.service` style units often appear in logs.

```
journalctl -u home-manager-alice.service -b --no-pager | tail -80
```

(Username substitution as appropriate.)

Standalone commands (awareness)

Even in module mode you may see:

```
home-manager generations 2>/dev/null || true
ls -la ~/.local/state/nix/profiles 2>/dev/null || true
```

You should **not** need `home-manager switch` for normal changes if module mode is correct.

Theory 6 — Programs modules vs raw packages

HM `programs.*` modules often:

- Install the package
- Generate config files
- Wire shell integration

Example: `programs.git.enable = true` vs only `home.packages = [ pkgs.git ]` (no managed config). Prefer **programs modules** when they exist for tools you configure.

Discover options:

- Home Manager option search
- Or eval/docs from your locked HM input

Shell source of truth

Approach	Note
`users.users.alice.shell = pkgs.bashInteractive`	System login shell
`programs.bash.enable = true`	HM manages bash config
Both fighting	Pick login shell once; let HM own rc files

Theory 7 — Splitting the home tree

```
homes/alice/
  default.nix      # imports + identity (username, stateVersion)
  programs.nix     # git, bash, ssh client
  files.nix        # later: xdg / home.file
  services.nix     # later: user systemd
```

```nix
# homes/alice/default.nix
{
  imports = [
    ./programs.nix
    # ./files.nix
    # ./services.nix
  ];

  home.username = "alice";
  home.homeDirectory = "/home/alice";
```

```
  home.stateVersion = "26.05";
  programs.home-manager.enable = true;
}
```

Theory 8 — Backup and collision with existing dots

HM activation may refuse to overwrite an existing regular file with a symlink. Before first activation of a path:

```
ls -la ~/.bashrc ~/.config/git 2>/dev/null || true
# backup and move aside if needed
```

Situation	Approach
Imperative bashrc	Backup; let HM own it
Mixed partial HM	Finish migration; avoid dual management
One-off experiment file	Keep outside HM paths

Worked example — Minimal useful user

```
# homes/alice/default.nix
{ pkgs, lib, ... }:
{
  home.username = "alice";
  home.homeDirectory = "/home/alice";
  home.stateVersion = "26.05";

  programs.home-manager.enable = true;

  programs.git = {
    enable = true;
    userName = "Alice";
    userEmail = "alice@example.invalid";
    extraConfig = {
      init.defaultBranch = "main";
      pull.rebase = true;
    };
  };
```

```
  programs.ssh = {
    enable = true;
    # Do not put private keys in the flake. Config only:
    matchBlocks = {
      "lab-remote" = {
        hostname = "192.0.2.10"; # documentation range example
        user = "alice";
      };
    };
  };

  programs.bash = {
    enable = true;
    shellAliases = {
      ll = "ls -la";
      ".." = "cd ..";
    };
  };

  home.packages = with pkgs; [ htop ];
}
```

Private keys stay in `~/.ssh/` with correct permissions—**never** in git, never as world-readable store strings (secrets chapters).

Worked example — HM wiring module (multi-host friendly)

```
# modules/common/home-manager.nix
{ inputs, ... }:
{
  imports = [
    inputs.home-manager.nixosModules.home-manager
  ];

  home-manager = {
    useGlobalPkgs = true;
    useUserPackages = true;
    users.alice = import ../../homes/alice;
  };
}
```

Requires `specialArgs = { inherit inputs; };` on `nixosSystem` (or equivalent) so `inputs` is available.

Exercises

Exercise 1 — Add the input and lock

```
cd /path/to/your-flake
# edit flake.nix inputs + outputs as in Theory 2
nix flake lock
# or: nix flake update home-manager
git diff flake.lock | head -80
```

Confirm `follows` for nixpkgs.

Exercise 2 — Stub → real home module

```
mkdir -p homes/alice
```

Write `homes/alice/default.nix` with git + bash + one package. Wire HM NixOS module and `home-manager.users.<you>`.

Ensure `users.users.<you>` exists on the system side with matching name and home directory.

Exercise 3 — Rebuild and verify

```
sudo nixos-rebuild switch --flake .#lab
```

As the target user:

```
git config --global --list | head
which rg || which htop
echo "$HOME"
home-manager generations 2>/dev/null || true
```

Exercise 4 — Change only HM, rebuild once

Edit `programs.git.userName`, rebuild, confirm new value. You should **not** need a separate `home-manager switch` if module mode is correct.

Exercise 5 — Deliberate failure drill

```
programs.git.enabl = true; # intentional typo
```

Rebuild, read the error, fix. Note whether the failure is clearly HM evaluation.

Exercise 6 — Journal: boundary decisions

List 10 tools you use daily. Mark each **NixOS / HM / either / project shell**. Resolve at least the ambiguous ones with a one-line reason.

Exercise 7 — Activation log literacy

```
journalctl -u home-manager-$USER.service -b --no-pager | tail -100
```

Save one successful and (from Exercise 5) one failed snippet in notes (redact secrets).

Exercise 8 — Split programs.nix

Move git/bash/ssh into `homes/alice/programs.nix` and import it. Rebuild; behavior unchanged.

Exercise 9 — Package placement check

Add a tool to `home.packages`, rebuild, verify `command -v`. Confirm it is **not** required for other users of the host.

Exercise 10 — Shell source of truth

Document in `docs/layout.md` (or home notes): where login shell is set, where aliases live.

Exercise 11 — Backup existing dots

Before enabling a `programs.*` that manages a file you already have, backup:

```
cp -a ~/.gitconfig /tmp/gitconfig.bak 2>/dev/null || true
```

Exercise 12 — Version alignment proof

```
nix flake metadata | tee /tmp/flake-meta.txt
```

Confirm HM input tracks `release-26.05` and nixpkgs `nixos-26.05`.

Exercise 13 — Negative case: standalone not required

Write two sentences: when you would use standalone HM; why the lab does not.

Common gotchas

Symptom / mistake	What to do
HM release ≠ nixpkgs release	Align `release-26.05` with `nixos-26.05`
User name mismatch	`users.users.x` must match `home-manager.users.x`
`home.homeDirectory` wrong	Must match actual home path
Infinite recursion with `useGlobalPkgs`	Check for pkgs/HM cycles; simplify modules
Expecting standalone CLI only	Module mode activates via nixos-rebuild
Private keys in flake	Remove immediately; rotate if committed
Bumping `home.stateVersion` casually	Read HM release notes first
Shell set in three places	One source of truth (NixOS *or* HM for login shell)
Activation blocked by existing file	Backup/remove blocker
Forgot `follows`	Two nixpkgs; fix and re-lock
HM user without system user	Create `users.users.*` first
Putting sshd in HM	System service → NixOS module

Checkpoint

- ☐ HM input pinned with `follows` nixpkgs
- ☐ HM NixOS module imported in flake
- ☐ At least one user has git + shell via HM
- ☐ Single `nixos-rebuild switch --flake` activates system + HM

- ☐ No private key material in the repo
- ☐ `home.stateVersion` set consciously
- ☐ Boundary list (NixOS vs HM vs project) started
- ☐ Know where to read HM activation failures

Journal (optional)

Write three personal gotchas before continuing.

Home files and XDG

Home files and XDG

Goal: Manage real user files with Home Manager—`home.file`, `xdg.configFile`, and XDG directories—while understanding **store-backed** links, permissions, and what must **never** be checked into the flake as plaintext secrets.

Why this chapter exists

The Home Manager chapter enabled programs modules. This chapter is the **file layer**: most of a personal environment is text under `~/.config`, `~/.local`, and a few classic dotfiles. HM makes those files **declarative and rebuildable**—or, if misused, makes secrets **world-readable in the Nix store**.

Theory 1 — How HM places files

Typical mechanism:

1. File contents (or a path) enter the Nix store as part of the HM generation
2. Activation creates **symlinks** from the home directory into the store (or manages mutable copies depending on options)
3. Next activation may replace links when content changes

Implications:

Property	Consequence
Store paths are world-readable	**No passwords, tokens, private keys** in HM file content
Links point at immutable content	Editing the live file in `~` may be wrong layer—edit the flake
Rebuild required for changes	Dotfile tweak → rebuild (or temporary escape hatch)

Escape hatches (know, use sparingly)

- Manual files outside HM for truly mutable scratch
- `mkOutOfStoreSymlink` patterns (advanced; can reintroduce impurity)
- Secrets tooling (services/security part) for sensitive values

```
# advanced awareness only - impure path outside store
# home.file."somewhere".source =
#   config.lib.file.mkOutOfStoreSymlink "/home/alice/mutable/config";
```

Prefer store-backed management for anything you want rebuildable and reviewable in git.

Theory 2 — `home.file` vs `xdg.configFile`

API	Default target idea	Example use
`home.file."<path>"`	Relative to `$HOME`	`home.file.".npmrc"`
`xdg.configFile."<path>"`	Under XDG config home (usually `~/.config`)	`xdg.configFile."foot/foot.ini"`
`xdg.dataFile`	Under data home	Desktop entries, shared data
`xdg.mimeApps`	MIME defaults	Default apps

Prefer **`xdg.configFile`** for modern tools that honor XDG.

Minimal examples

```
# homes/alice/files.nix
{ pkgs, ... }:
{
  xdg.enable = true;

  xdg.configFile."demo/hello.txt".text = ''
    managed by home-manager
    rebuilt with the host flake
  '';

  home.file.".config/demo/from-home-file.txt".text = ''
    same idea via home.file
  '';
}
```

Import from `homes/alice/default.nix`:

```
{
  imports = [ ./files.nix ];
}
```

Theory 3 — Content sources: `text`, `source`, templates

Field	Use
`.text`	Inline small configs
`.source`	Path to a file in the repo (`./config/gitconfig` sibling)
`.source = pkgs.writeText ...`	Generated in Nix
`.executable = true`	Scripts that need +x

Repo layout option:

```
homes/alice/
  default.nix
  files.nix
  config/
    starship.toml
    htoprc
    nvim/init.lua
```

```
xdg.configFile."starship.toml".source = ./config/starship.toml;
xdg.configFile."nvim/init.lua".source = ./config/nvim/init.lua;
```

Benefit of `source`: normal editor diffs on real files; Nix only wires them.

Recursive directories

```
xdg.configFile."nvim" = {
  source = ./config/nvim;
  recursive = true;
};
```

Use when a tool expects a tree. Keep secrets out of that tree.

Theory 4 — XDG directories

```
xdg.enable = true;
# xdg.configHome / dataHome / cacheHome / stateHome can be customized
# defaults are usually correct - only override with a reason
```

Variable	Typical path
`XDG_CONFIG_HOME`	`~/.config`
`XDG_DATA_HOME`	`~/.local/share`
`XDG_CACHE_HOME`	`~/.cache`
`XDG_STATE_HOME`	`~/.local/state`

Tools differ in XDG discipline. Your job: **put managed files where the tool reads them**, not where tradition says.

Classic vs XDG

Classic path	Prefer if tool supports XDG
`~/.gitconfig`	`programs.git` (HM) or XDG git config
`~/.bashrc`	`programs.bash`
`~/.config/foo`	`xdg.configFile."foo/..."`

Theory 5 — Permissions and clobber behavior

HM will not always happily overwrite unexpected pre-existing files—activation may fail if a real file blocks a symlink.

Situation	Approach
Old imperative dotfile in the way	Backup and remove, then rebuild
You need a secret file mode 0600	Do not put secret content in store; use runtime secret install
Public config	Store-backed symlink is fine

Check links:

```
ls -la ~/.config/demo
readlink -f ~/.config/demo/hello.txt
stat "$(readlink -f ~/.config/demo/hello.txt)"
```

Mode options

```
home.file.".local/bin/lab-tool" = {
  text = ''
    #!/usr/bin/env bash
    echo lab
  '';
  executable = true;
};
```

Remember: store object readability still applies to content; `executable` does not make content private.

Theory 6 — Theming (optional, bounded)

Theming (GTK, fonts, pointer cursors) is an **infinite rabbit hole**. If you care:

- Prefer HM modules that exist (`gtk`, `qt`, fonts at system level, etc.)
- Bound the experiment: one terminal theme + one font—not a full desktop redesign

If you run a server-only lab: **skip theming**; migrate shell/git configs instead.

Theory 7 — Multi-machine user splits

Same user, different hosts:

```
# homes/alice/default.nix
{
  imports = [
    ./common.nix
    # ./hosts/lab.nix   # optional per-host HM slice
  ];
}
```

Or select in the flake:

```
home-manager.users.alice = {
  imports = [
    ./homes/alice/common.nix
    ./homes/alice/hosts/lab.nix
  ];
```

```
};
```

Do not copy entire homes per host if 90% is shared—**common + overlay modules**.

Theory 8 — `programs.*` vs raw files for the same tool

Prefer	When
`programs.git`	Structured options exist
`xdg.configFile`	Tool has no HM module or you want raw file fidelity
Both for same path	Conflict risk—pick one

```
# Good: structured
programs.git.ignores = [ "result" ".direnv/" ];

# Also good: raw when complex
# xdg.configFile."git/ignore".source = ./config/git-ignore-global;
```

Theory 9 — What never goes in managed files

Never in flake/HM text	Why
API tokens	Store world-readable
Private SSH keys	Same + git leak
Cloud provider secrets	Same
DB passwords	Same
`.env` production values	History forever if committed

Gray area: non-secret feature flags, public keys, preferred editor settings—OK.

Worked example — Migrate git ignores + editor config

```
# homes/alice/git.nix
{
  programs.git = {
    enable = true;
    userName = "Alice";
    userEmail = "alice@example.invalid";
    ignores = [
      "*.env"
      ".direnv/"
      "result"
      "result-*"
    ];
  };

  xdg.configFile."nvim/init.lua".text = ''
    -- minimal managed nvim config
    vim.opt.number = true
    vim.opt.relativenumber = true
  '';
}
```

No API tokens. No `extraConfig` with `github.token`.

Worked example — Structured home tree

```
homes/alice/
  default.nix
  programs.nix
  files.nix
  config/
    starship.toml
    htop/htoprc
```

```
# homes/alice/files.nix
{
  xdg.enable = true;
  xdg.configFile."starship.toml".source = ./config/starship.toml;
  xdg.configFile."htop/htoprc".source = ./config/htop/htoprc;
}
```

Exercises

Exercise 1 — Inventory live dotfiles

As your lab user:

```
ls -la ~ | head -50
ls -la ~/.config 2>/dev/null | head -50
```

Pick **3–5** real configs to migrate (git, shell rc fragments, one tool). Skip secrets and browser profiles.

Journal: path → HM attribute choice (`programs.*` vs `xdg.configFile` vs `home.file`).

Exercise 2 — First managed XDG file

Add `xdg.enable = true` and a demo `xdg.configFile`. Rebuild. Verify symlink into store:

```
ls -la ~/.config/demo
readlink -f ~/.config/demo/hello.txt
stat "$(readlink -f ~/.config/demo/hello.txt)"
```

Note permissions on the store path (world-readable). **That observation is the point.**

Exercise 3 — Migrate via `source`

Place a real config file next to the HM module and set `.source`. Rebuild. Confirm tool behavior.

Exercise 4 — Conflict drill

While HM manages a path: backup, replace symlink with a regular file, rebuild, observe activation error. Restore by removing the blocker and rebuilding. Document the error text.

Exercise 5 — Secret anti-pattern (fake only)

On a local throwaway branch (do not push real secrets):

```
# BAD - educational only, use fake value
# home.file.".config/acme/token".text = "super-secret-placeholder";
```

If you activate a fake value, show yourself:

```
# cat "$(readlink -f ~/.config/acme/token)"
```

Then **delete** the pattern, rebuild, and write: "secrets need secrets tooling, not HM file content."

Never commit real tokens. If you did: rotate them.

Exercise 6 — Structure the home tree

```
homes/alice/
  default.nix
  programs.nix
  files.nix
  config/
```

Rebuild once more to prove the split.

Exercise 7 — Recursive config tree

If you use nvim or similar, try `recursive = true` on a config directory. Confirm links.

Exercise 8 — Executable home file

Install a tiny script under `~/.local/bin` via `home.file` + `executable = true`. Ensure `~/.local/bin` is on PATH (HM `home.sessionPath` or shell config).

Exercise 9 — programs vs file for one tool

For git **or** another tool, write two sentences: why you chose `programs.*` vs raw file.

Exercise 10 — Multi-host plan

Sketch `common.nix` + optional `hosts/lab.nix` even if you only have one machine.

Exercise 11 — Cache vs config

List one path that should stay **unmanaged** (cache, browser profile). Confirm it is not in the flake.

Exercise 12 — Activation after migrate

```
sudo nixos-rebuild switch --flake .#lab
journalctl -u home-manager-$USER.service -b --no-pager | tail -40
```

Exercise 13 — Documentation

Add a short "Home files" section to `docs/layout.md`: which configs are managed, which are intentionally not.

Common gotchas

Symptom / mistake	What to do
Activation fails on existing file	Backup/remove blocker; rebuild
Edited `~` file, change vanished	You edited a symlink or lost it on rebuild—edit flake
Secret in store	Remove from Nix; rotate credential
Wrong path (`~/.foo` vs XDG)	Check tool docs; fix attribute
Huge binary blobs in HM	Don't; keep media out of the flake
`text` with nested quotes hell	Prefer `.source` files
Assuming 0600 store file is private	Store is not a private vault
Dual management of same path	Pick programs.* **or** file API
Recursive source includes `.git` junk	Clean source tree
Theming rabbit hole on server	Skip; ship shell/git first

Checkpoint

- ☐ `xdg.enable` (or conscious decision not to)
- ☐ At least two real configs managed via HM files or programs
- ☐ Can show a managed path is store-backed
- ☐ Home tree split into programs/files modules
- ☐ Written note: no secrets in HM file content
- ☐ Conflict + recovery once
- ☐ At least one `.source` migration
- ☐ Know when to use escape hatches vs store links

Journal (optional)

Write three personal gotchas before continuing.

Shell and Editor Configuration

Shell and Editor Configuration

Goal: Declare complete user workstation environments—interactive shells (Bash, Zsh, Fish), modern prompt engines (Starship), terminal multiplexers (Tmux, Zellij), and code editors (Neovim, Helix, VS Code)—entirely within Home Manager, wiring Language Server Protocol (LSP) binaries and treesitter parsers without mutable runtime state.

> **Note**
>
> This chapter builds upon `03-home-manager.qmd` and `04-hm-files-xdg.qmd`. While previous chapters focused on raw files and user account creation, this chapter covers the deep configuration of daily interactive developer tooling using structured `programs.*` modules.

Why Shell and Editor Declarations Matter

Traditional workstation setup relies on mutable imperative managers: `curl | sh` scripts, Oh-My-Zsh plugins, Lazy.nvim downloading git repositories dynamically into `~/.local/share`, or Visual Studio Code extensions updating automatically.

This imperative approach breaks in three ways: 1. **Machine Synchronization Lag:** When setting up a second laptop or pairing station, plugins, themes, and keybindings are inevitably forgotten or drift across machines. 2. **Missing System Binaries:** An editor configuration that assumes `ripgrep`, `fd`, `clangd`, `pyright`, or `gopls` are installed globally in the system `$PATH` breaks when opened on a minimal host. 3. **Non-Hermetic Plugin Caching:** A plugin manager that pulls directly from GitHub master branches at startup introduces breaking changes and breaks offline work.

By declaring shells, multiplexers, and editors in Home Manager, all binaries, plugins, themes, and configurations become immutable store paths pinned in your flake lockfile.

Declarative Shell Environments

Home Manager provides dedicated modules for all major interactive shells. Rather than managing complex nested `.bashrc` or `.zshrc` files with raw strings, use typed options.

Modern Shell Declaration (Bash and Fish)

```
# homes/developer/programs/shell.nix
{ pkgs, ... }: {
  # Bash configuration
  programs.bash = {
    enable = true;
    enableCompletion = true;
    historySize = 100000;
    historyFileSize = 200000;
    historyControl = [ "ignoredups" "ignorespace" ];

    shellAliases = {
      ll = "ls -lah --color=auto";
      g = "git";
      nr = "nixos-rebuild switch --use-remote-sudo --flake .";
      nd = "nix develop -c $SHELL";
    };

    initExtra = ''
      # Fast vi mode and custom keybindings
      set -o vi
      bind '"\e[A": history-search-backward'
      bind '"\e[B": history-search-forward'
    '';
  };

  # Fish configuration (alternative modern interactive shell)
  programs.fish = {
    enable = true;
    interactiveShellInit = ''
      set -g fish_greeting "" # suppress greeting banner
      fish_vi_key_bindings
    '';
    shellAliases = {
      cat = "bat --paging=never";
      find = "fd";
    };
    plugins = [
      {
        name = "z";
        src = pkgs.fetchFromGitHub {
          owner = "jethrokuan";
          repo = "z";
          rev = "e0e1b9deb977e84c8298f1caf7c6d467e5b4d460";
          hash = "sha256-j28y4O9h/P4b41eyvdQdo286q13E8k7i0/U74zQzZ80=";
```

```
        };
      }
    ];
  };
}
```

Universal Prompt Engineering with Starship

The `starship` prompt provides a fast, cross-shell prompt written in Rust. Home Manager integrates it directly into Bash, Zsh, and Fish with zero shell-specific boilerplate:

```
# homes/developer/programs/starship.nix
{ ... }: {
  programs.starship = {
    enable = true;
    enableBashIntegration = true;
    enableFishIntegration = true;
    enableZshIntegration = true;

    settings = {
      add_newline = false;
      character = {
        success_symbol = "[ ](bold green)";
        error_symbol = "[ ](bold red)";
      };
      nix_shell = {
        symbol = "  ";
        format = "via [$symbol\\($name\\)]($style) ";
        style = "bold blue";
      };
      git_branch = {
        symbol = "  ";
        truncation_length = 24;
      };
      directory = {
        truncation_length = 5;
        truncate_to_repo = true;
      };
      aws.disabled = true;
      gcloud.disabled = true;
    };
  };
}
```

Terminal Multiplexers: Tmux and Zellij

Multiplexers preserve developer sessions across SSH connections and provide split-pane workspaces. Home Manager can configure both traditional `tmux` and modern `zellij`:

```
# homes/developer/programs/multiplexers.nix
{ pkgs, ... }: {
  # Traditional tmux with declarative plugins
  programs.tmux = {
    enable = true;
    shortcut = "a"; # Ctrl-a prefix
    baseIndex = 1;
    escapeTime = 0;
    terminal = "screen-256color";
    mouse = true;
    keyMode = "vi";

    plugins = with pkgs.tmuxPlugins; [
      sensible
      pain-control
      yank
      {
        plugin = catppuccin;
        extraConfig = ''
          set -g @catppuccin_flavor 'mocha'
          set -g @catppuccin_window_tabs_enabled on
        '';
      }
    ];

    extraConfig = ''
      # Split panes using | and -
      bind | split-window -h -c "#{pane_current_path}"
      bind - split-window -v -c "#{pane_current_path}"
    '';
  };

  # Modern Rust-based Zellij
  programs.zellij = {
    enable = true;
    settings = {
      theme = "catppuccin-mocha";
      default_layout = "compact";
      pane_frames = false;
```

```
      copy_command = "wl-copy";
    };
  };
}
```

Declarative Editor Mastery: Neovim and Helix

Declarative Neovim with Pinned Plugins and Built-in LSPs

Rather than letting Neovim execute git clones at runtime, Home Manager's `programs.neovim` builds a hermetic Neovim derivation. It wraps `nvim` with a generated `init.lua` and injects exact language server binaries into its private execution environment:

```
# homes/developer/programs/neovim.nix
{ pkgs, ... }: {
  programs.neovim = {
    enable = true;
    defaultEditor = true;
    viAlias = true;
    vimAlias = true;

    # System binaries made available specifically to Neovim
    extraPackages = with pkgs; [
      # Language servers (LSP)
      nil                # Nix LSP
      gopls              # Go LSP
      rust-analyzer      # Rust LSP
      pyright            # Python LSP
      lua-language-server

      # Formatters & Linters
      alejandra          # Opinionated Nix formatter
      nixpkgs-fmt
      rustfmt
      black
      stylua

      # Core tools for fuzzy finding
      ripgrep
      fd
      git
    ];
```

```
# Immutable plugins fetched through Nix
plugins = with pkgs.vimPlugins; [
  # Syntax & Highlighting
  nvim-treesitter.withAllGrammars

  # UI & Theming
  catppuccin-nvim
  lualine-nvim
  nvim-web-devicons

  # Navigation & Fuzzy Search
  telescope-nvim
  plenary-nvim

  # LSP & Completion
  nvim-lspconfig
  nvim-cmp
  cmp-nvim-lsp
  cmp-buffer
  cmp-path
  luasnip
];

# Pure Lua configuration pointing to the immutable paths
extraLuaConfig = ''
  vim.opt.number = true
  vim.opt.relativenumber = true
  vim.opt.expandtab = true
  vim.opt.shiftwidth = 2
  vim.opt.tabstop = 2
  vim.opt.termguicolors = true

  -- Set color scheme
  vim.cmd.colorscheme "catppuccin-mocha"

  -- Configure LSP through nvim-lspconfig
  local lspconfig = require('lspconfig')
  local capabilities = require('cmp_nvim_lsp').default_capabilities()

  -- Nix LSP (nil)
  lspconfig.nil_ls.setup {
    capabilities = capabilities,
    settings = {
      ['nil'] = {
        formatting = { command = { "alejandra" } },
      },
    },
```

```
      }

      -- Go LSP
      lspconfig.gopls.setup { capabilities = capabilities }

      -- Rust Analyzer
      lspconfig.rust_analyzer.setup { capabilities = capabilities }

      -- Telescope Keybindings
      local builtin = require('telescope.builtin')
      vim.keymap.set('n', '<leader>ff', builtin.find_files, {})
      vim.keymap.set('n', '<leader>fg', builtin.live_grep, {})
      vim.keymap.set('n', '<leader>fb', builtin.buffers, {})
    '';
  };
}
```

Minimal Modal Editing: Helix

For developers seeking a zero-plugin modal editor with built-in LSP, Treesitter, and fuzzy-finding, `helix` provides a clean declarative configuration:

```
# homes/developer/programs/helix.nix
{ pkgs, ... }: {
  programs.helix = {
    enable = true;
    settings = {
      theme = "catppuccin_mocha";
      editor = {
        line-number = "relative";
        cursorline = true;
        color-modes = true;
        cursor-shape = {
          insert = "bar";
          normal = "block";
          select = "underline";
        };
        lsp = {
          display-messages = true;
          display-inlay-hints = true;
        };
      };
    };
    languages = {
      language = [
        {
```

```
          name = "nix";
          auto-format = true;
          formatter = { command = "${pkgs.alejandra}/bin/alejandra"; };
          language-servers = [ "nil" ];
        }
        {
          name = "rust";
          auto-format = true;
          language-servers = [ "rust-analyzer" ];
        }
      ];
    };
  };
}
```

Visual Studio Code Declarative Management

When a graphical IDE is required, Home Manager can manage VS Code, its settings, and its extensions without manual marketplace installations:

```
# homes/developer/programs/vscode.nix
{ pkgs, ... }: {
  programs.vscode = {
    enable = true;
    package = pkgs.vscode;
    extensions = with pkgs.vscode-extensions; [
      bbenoist.nix
      jnoortheen.nix-ide
      rust-lang.rust-analyzer
      golang.go
      catppuccin.catppuccin-vsc
      eamodio.gitlens
    ];
    userSettings = {
      "workbench.colorTheme" = "Catppuccin Mocha";
      "editor.formatOnSave" = true;
      "nix.enableLanguageServer" = true;
      "nix.serverPath" = "${pkgs.nil}/bin/nil";
      "nix.serverSettings" = {
        "nil" = {
          "formatting" = { "command" = [ "alejandra" ]; };
        };
      };
```

```
      "telemetry.telemetryLevel" = "off";
    };
  };
}
```

Common Traps and State Separation

Trap	Root Cause	Declarative Correction
`lazy.nvim` or Vundle trying to clone Git repos	Running a mutable plugin manager inside an immutable Nix Neovim build	Declare plugins via `pkgs.vimPlugins.*` in `plugins = [ ... ]`.
LSP binary not found in editor	Relying on host `$PATH` instead of injecting binaries	Use `extraPackages = with pkgs; [ nil gopls ... ]` in Home Manager Neovim.
History or cache permission errors	Attempting to link mutable cache files (e.g. `.bash_history`) to `/nix/store`	Keep history paths dynamic in user `$HOME` or `$XDG_STATE_HOME`. Only config is read-only.
Treesitter compilation failure	Neovim trying to run local `gcc` to compile C grammars	Use `nvim-treesitter.withAllGrammars` from `pkgs.vimPlugins`.

Checkpoint

- ☐ Shells (Bash, Zsh, or Fish) are managed via `programs.*` with declarative aliases and options.
- ☐ Starship prompt is integrated uniformly across all configured shells.
- ☐ Multiplexers (Tmux/Zellij) are declared with plugins pulled from `pkgs`.
- ☐ Editor (Neovim/Helix/VS Code) bundles language servers directly without polluting system-wide packages.
- ☐ Rebuilding your flake on a completely clean machine yields an identical interactive terminal environment.

Home Manager user services

Home Manager user services

Goal: Run **user-level** systemd services and timers through Home Manager—work that belongs to a login user, not to the system-wide daemon set.

Why this chapter exists

System systemd via NixOS covers host daemons. Not everything should be system-wide:

Better as **user** service	Better as **system** service
Personal backup of `~/docs`	PostgreSQL, Caddy, sshd
Wait-for-graphical clipboard tool	Network-facing APIs
User-specific sync agent	Multi-user shared daemons
Periodic `git fetch` of notes	CI runners for a team

HM exposes a structured way to declare user units so they survive reinstalls and rebuilds.

Theory 1 — Systemd user vs system

Bus	Unit location idea	Starts when
System	`/etc/systemd/system` (generated)	Boot / system targets
User	`~/.config/systemd/user` (generated by HM)	User session / lingering

Lingering

If you need user services **without** an interactive login (headless user timers):

```
sudo loginctl enable-linger alice
loginctl show-user alice | grep Linger
```

Without linger, user services may only run while logged in (depending on setup). For a server lab admin user, lingering is often desirable; for a laptop, maybe not—**document the choice**.

Theory 2 — HM service options

Home Manager provides:

- `systemd.user.services.<name>`
- `systemd.user.timers.<name>`
- `services.<hm-module>` for popular user apps (where modules exist)

Raw unit example:

```
# homes/alice/services.nix
{ pkgs, ... }:
{
  systemd.user.services.lab-heartbeat = {
    Unit = {
      Description = "Write a heartbeat timestamp";
    };
    Service = {
      Type = "oneshot";
      ExecStart = "${pkgs.bash}/bin/bash -c 'date -Is >> %h/lab-heartbeat.log'";
    };
  };

  systemd.user.timers.lab-heartbeat = {
    Unit = {
      Description = "Run lab-heartbeat every 15 minutes";
    };
    Timer = {
      OnBootSec = "2min";
      OnUnitActiveSec = "15min";
      Persistent = true;
    };
    Install = {
      WantedBy = [ "timers.target" ];
    };
  };
}
```

`%h` is systemd specifier for the user home directory.

Import from `homes/alice/default.nix`.

HM attrset shape

HM's user unit options often mirror unit file sections as nested attrsets (`Unit`, `Service`, `Install`). Exact schema can vary slightly by HM version—when in doubt, check HM option docs for your **26.05-aligned** pin.

Theory 3 — Mapping system knowledge to user bus

Concept	System (NixOS)	User (HM)
Service attrset	`systemd.services.*`	`systemd.user.services.*`
Timer	`systemd.timers.*`	`systemd.user.timers.*`
journal	`journalctl -u foo`	`journalctl --user -u foo`
restart	`systemctl restart`	`systemctl --user restart`
list timers	`systemctl list-timers`	`systemctl --user list-timers`

Permissions: user services **cannot** bind privileged ports (<1024) without capabilities setup; they should not replace system network daemons.

Theory 4 — `Simple` vs `oneshot` vs long-running

Type	Use
`oneshot`	Task that exits; good with timers
`simple` / default	Long-running process
`exec`	systemd newer behavior; fine when you know it

For a periodic script: **oneshot service + timer**.

For a personal agent: long-running service with `Restart = "on-failure"`.

```
systemd.user.services.notes-agent = {
  Unit.Description = "Long-running personal agent";
  Service = {
    ExecStart = "${pkgs.coreutils}/bin/sleep infinity"; # placeholder
    Restart = "on-failure";
    RestartSec = "5s";
  };
  Install.WantedBy = [ "default.target" ];
};
```

(Replace the placeholder with a real agent binary from `pkgs`.)

Theory 5 — Paths and packages in units

Always use **store paths** from `pkgs`, not assumed global binaries:

```
ExecStart = "${pkgs.restic}/bin/restic snapshots";
# not: ExecStart = "restic snapshots";
```

Environment:

```
Service = {
  Environment = [
    "FOO=bar"
  ];
  # EnvironmentFile must not point at secrets in the flake/store
};
```

Sensitive env files belong to secret deployment paths with tight permissions—not `home.file` with tokens.

WorkingDirectory and paths

```
Service = {
  WorkingDirectory = "%h/notes";
  ExecStart = "${pkgs.git}/bin/git fetch --all --prune";
};
```

Ensure the directory exists (create once imperatively or with a oneshot).

Theory 6 — Prefer HM modules when they exist

Examples (availability depends on HM version):

- `services.syncthing` (user)
- `services.gpg-agent`
- Various desktop helpers

Pattern: search HM options first; fall back to raw `systemd.user.services` for custom scripts.

```
services.gpg-agent = {
  enable = true;
  enableSshSupport = true;
};
```

Theory 7 — Targets, wants, and session lifecycle

Target	Role
`default.target`	User session default
`timers.target`	Timers
`graphical-session.target`	Needs graphical session

```
Install.WantedBy = [ "default.target" ];
# or timers.target for timers
```

Graphical-only tools should not use headless linger assumptions without testing.

Theory 8 — Debugging workflow

```
systemctl --user daemon-reload   # usually activation handles this
systemctl --user status lab-heartbeat.service
systemctl --user start lab-heartbeat.service
journalctl --user -u lab-heartbeat.service -n 50 --no-pager
systemctl --user list-timers --all
```

Break → read journal → fix flake → rebuild → retest. Avoid editing generated unit files under `~/.config/systemd/user` as source of truth.

Worked example — Notes fetcher timer

```
{ pkgs, ... }:
{
  home.packages = [ pkgs.git ];

  systemd.user.services.fetch-notes = {
    Unit.Description = "Fetch personal notes repo";
    Service = {
      Type = "oneshot";
      WorkingDirectory = "%h/notes";
      ExecStart = "${pkgs.git}/bin/git fetch --all --prune";
    };
  };
```

```
  systemd.user.timers.fetch-notes = {
    Unit.Description = "Timer for fetch-notes";
    Timer = {
      OnCalendar = "hourly";
      Persistent = true;
    };
    Install.WantedBy = [ "timers.target" ];
  };
}
```

Requires `~/notes` to be a git repo. Create it if missing (empty repo is fine).

Worked example — Calendar timer variants

```
Timer = {
  OnCalendar = "daily";
  # OnCalendar = "Mon..Fri *-*-* 09:00:00";
  Persistent = true;
  RandomizedDelaySec = "10m";
};
```

`Persistent = true` catches up missed runs after downtime (within reason).

Exercises

Exercise 1 — User systemd baseline

```
systemctl --user status
systemd-analyze --user blame 2>/dev/null | head || true
loginctl show-user "$USER" | grep -E 'Linger|State'
```

Decide whether to enable linger for your lab user; document the choice.

Exercise 2 — Heartbeat oneshot + timer

Add the heartbeat service from Theory 2. Rebuild:

```
sudo nixos-rebuild switch --flake .#lab
```

As user:

```
systemctl --user list-timers --all | grep -i heart || systemctl --user list-timers --all
systemctl --user start lab-heartbeat.service
systemctl --user status lab-heartbeat.service
cat ~/lab-heartbeat.log
systemctl --user enable --now lab-heartbeat.timer
```

Exercise 3 — Journal debugging

```
journalctl --user -u lab-heartbeat.service -n 50 --no-pager
```

Break `ExecStart` deliberately, rebuild, read the failure, fix.

Exercise 4 — One service you actually want

Ideas:

- `git fetch` on a notes repo
- `rsync` dry-run of a directory to a backup path (no secrets in unit)
- Generate a dated file in `~/lab/`

Implement, enable, prove timer or manual start works twice.

Exercise 5 — Boundary check

Write four lines:

1. Why this is not a system service
2. What user it runs as
3. Whether linger is required
4. What happens on logout without linger

Exercise 6 — Import services.nix

Ensure `homes/alice/services.nix` is imported from `default.nix`. Rebuild once after the split.

Exercise 7 — list-unit-files literacy

```
systemctl --user list-unit-files | head -40
systemctl --user cat lab-heartbeat.service
```

Confirm the unit content matches your flake (store paths visible).

Exercise 8 — Failure restart policy

Add a long-running service with `Restart = "on-failure"` (even a simple script). Kill it, observe restart behavior, then disable if you do not need it.

Exercise 9 — OnCalendar experiment

Change heartbeat to `OnCalendar = "*:0/10"` (every 10 minutes) or similar; reload; `list-timers`.

Exercise 10 — Wrong bus drill

Run `systemctl status lab-heartbeat` **without** `--user`. Note the confusion. Always use `--user` for HM units.

Exercise 11 — Secret audit of unit text

Read your service modules: no passwords in `Environment=`. Plan secrets paths if needed later.

Exercise 12 — Graphical vs headless note

If this host is headless, list any user services that would need a desktop session—and do not enable them.

Exercise 13 — Comparison table in docs

Add to `docs/layout.md`: one system service vs one user service you run, with rationale.

Exercise 14 — Clean disable

Disable and remove a throwaway timer after testing:

```
systemctl --user disable --now lab-heartbeat.timer 2>/dev/null || true
# remove from flake; rebuild
```

Common gotchas

Symptom / mistake	What to do
Timer never fires while logged out	Enable linger or accept session-bound life
`systemctl` without `--user`	You queried the system bus
Binary not found	Use `${pkgs.foo}/bin/foo`
Service works manually, not on timer	Check `WantedBy`, enable timer, `list-timers`
Putting nginx in user systemd	Use NixOS system module instead
Secrets in `Environment=` in flake	Secrets chapters; remove from git
Activation OK but unit missing	Confirm HM user matches logged-in user
Edited generated unit by hand	Revert; change flake only
WorkingDirectory missing	Create dir or unit fails
Port 80 in user service	Use system proxy/service
Linger enabled globally without thought	Document per-user choice

---

Checkpoint

- ☐ At least one user service defined via HM
- ☐ At least one timer or a long-running user service
- ☐ Can use `systemctl --user` and `journalctl --user`
- ☐ Linger decision documented
- ☐ No secrets in unit text
- ☐ Clear system-vs-user boundary write-up
- ☐ Units imported from `services.nix` (or equivalent split)
- ☐ Know how to read a failed user unit journal

Journal (optional)

Write three personal gotchas before continuing.

---

direnv and project shells

direnv and project shells

Goal: Keep project toolchains in **flake `devShells`**, load them automatically with **direnv** (`use flake`), and stop stuffing every language toolchain into system or HM profiles.

Why this chapter exists

Two failure modes dominate Nix workstations:

1. **Profile bloat:** Go, Node, Rust, Python, JDK all in `home.packages` forever
2. **Snowball shells:** random `nix-shell -p` with no lock, unreproducible on Monday

The modern fix:

Layer	Holds
NixOS / HM	OS, editors, generic CLI, user services
Project flake `devShells`	Compilers, linters, project CLIs
direnv	Enters/exits the shell when you `cd`

Project pinning is a first-class skill—independent of host configuration.

Theory 1 — `devShells` in a flake

Minimal project flake (can be **separate** from the NixOS flake):

```
# ~/lab/demo-app/flake.nix
{
  description = "demo app toolchain";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
  let
    system = "x86_64-linux";
    pkgs = nixpkgs.legacyPackages.${system};
  in {
```

```
    devShells.${system}.default = pkgs.mkShell {
      packages = with pkgs; [
        go_1_24  # adjust to versions available on your pin
        gopls
        git
      ];
      shellHook = ''
        export DEMO_APP=1
        echo "entered demo-app devShell"
      '';
    };
  };
}
```

Enter manually:

```
cd ~/lab/demo-app
nix develop
```

Multi-system sketch

```
let
  systems = [ "x86_64-linux" "aarch64-linux" ];
  forAllSystems = nixpkgs.lib.genAttrs systems;
in {
  devShells = forAllSystems (system:
    let pkgs = nixpkgs.legacyPackages.${system}; in {
      default = pkgs.mkShell {
        packages = with pkgs; [ git jq ];
      };
    });
}
```

Why not always put shells in the NixOS flake?

You *can* expose `devShells` from the host flake, but **project repos** usually own their own flake so clones on other machines work without your entire OS config. Host flake remains for `nixosConfigurations` (+ checks). Host-level shells for ops tools (nil, nixpkgs-fmt) are still fine.

Theory 2 — direnv + nix-direnv

direnv loads environment variables when you enter a directory with `.envrc`.

nix-direnv (recommended) caches Nix flake shells efficiently so every `cd` is not a full re-eval disaster.

HM installation pattern

```
# homes/alice/direnv.nix
{ pkgs, ... }:
{
  programs.direnv = {
    enable = true;
    nix-direnv.enable = true;
    enableBashIntegration = true;
    # enableZshIntegration / enableFishIntegration as needed
  };
}
```

Rebuild host/HM, then new shells pick up the hook (`eval "$(direnv hook bash)"` is usually wired by the module).

Verify hook

```
command -v direnv
type direnv
# open a new login shell if needed
```

Theory 3 — `.envrc` with `use flake`

In the project:

```
# ~/lab/demo-app/.envrc
use flake
```

Allow once:

```
direnv allow
```

Behavior:

Action	Result
`cd` into project	Loads `devShell` env
`cd` out	Unloads
flake/lock change	direnv reloads (may rebuild)

Named shells

```
# use flake .#ci
use flake
```

If you define `devShells.${system}.ci`, you can select it—check current nix-direnv / direnv-stdlib docs for exact syntax on your pin.

`.envrc` is not for secrets

Do not put API tokens in `.envrc` committed to git. Use:

- Untracked `.envrc.local` patterns (with care)
- Secret managers / sops (services/security part)
- Runtime private files excluded by git

```
.envrc.local
.env
```

Theory 4 — System packages vs project shells

Put in HM/system	Put in devShell
`git`, `neovim`, `htop`	`go`, `cargo`, `node`, `python` venv tools
direnv itself	project linters pinned to repo
SSH client	DB client only for one service's repo

Test: if two projects need different major versions, it belongs in the project shell.

Theory 5 — Pinning discipline for project flakes

Project `flake.lock` should be committed for apps you ship or share. For private throwaways, still locking saves future you.

Update project nixpkgs **independently** of the host flake—on purpose. Host OS pin every app pin.

Host flake	Project flake
Stability for OS modules	May need newer compiler
Selective update cadence	Can track unstable carefully
Shared by all services	Local to one repo

Theory 6 — `nix develop` vs `nix shell` vs HM packages

Command	Role
`nix develop`	Flake devShell (hooks, packages, inputsFrom)
`nix shell nixpkgs#foo`	Ad-hoc package on PATH
`home.packages`	Always-on user profile
direnv `use flake`	Automates develop-class env

`inputsFrom` (awareness)

```
pkgs.mkShell {
  inputsFrom = [ self.packages.${system}.default ];
  packages = [ pkgs.jq ];
};
```

Pulls build inputs from a package—useful when packaging and developing the same project.

Theory 7 — shellHook hygiene

Do	Don't
Export needed `PROJECT_ROOT`	Print banners that annoy CI
Set `LANG` if required	Embed secrets
Keep hooks idempotent	Assume interactive-only blindly

```
  shellHook = ''
    export PROJECT_ROOT="$PWD"
    # echo only if interactive:
    # [ -n "$PS1" ] && echo "devShell ready"
  '';
```

Theory 8 — Host flake devShell for Nix ops

Optional host-level shell (not a substitute for project shells):

```
# in host flake outputs
devShells.${system}.default = pkgs.mkShell {
  packages = with pkgs; [ nixpkgs-fmt nil git jq ];
};
```

```
cd /path/to/nixos-config
nix develop
```

Use for editing the OS flake; keep language toolchains in app repos.

Worked example — Multi-tool Python shell

```
devShells.${system}.default = pkgs.mkShell {
  packages = with pkgs; [
    python3
    python3Packages.pytest
    ruff
  ];
  shellHook = ''
    export PYTHONPATH="$PWD/src''${PYTHONPATH:+:$PYTHONPATH}"
  '';
};
```

```
# .envrc
use flake
```

Worked example — Isolation proof layout

```
~/lab/proj-a/   # flake with cowsay only
~/lab/proj-b/   # flake with hello only
```

```
cd ~/lab/proj-a && command -v cowsay
cd ~/lab/proj-b && command -v hello
cd ~ && command -v cowsay; command -v hello  # both missing if thinned
```

Exercises

Exercise 1 — Install direnv via HM

Add `programs.direnv` with `nix-direnv.enable = true`. Rebuild. Verify:

```
command -v direnv
direnv version
```

Open a new login shell if hooks are missing.

Exercise 2 — Create a project flake

```
mkdir -p ~/lab/demo-app
cd ~/lab/demo-app
git init
# write flake.nix with mkShell and 2-3 tools
nix flake lock
nix develop -c which git
```

Exercise 3 — Wire direnv

```
echo 'use flake' > .envrc
direnv allow
cd ..
cd demo-app
echo "PATH head:"; echo "$PATH" | tr ':' '\n' | head
type python3 2>/dev/null || type go 2>/dev/null || type hello 2>/dev/null || true
```

Confirm leaving the directory drops the env.

Exercise 4 — Prove version isolation

Make **two** tiny projects with different tools (e.g. one with `hello` only, one with `cowsay` only). `cd` between them; show different PATH entries.

Exercise 5 — Thin the profile (careful)

List `home.packages` / system packages. Move **one** language toolchain from always-on profile into a project shell. Rebuild HM. Confirm the tool is gone from bare shells but present under direnv.

Do not remove your editor without a plan.

Exercise 6 — Document the workflow

Journal template:

```
cd project → direnv loads flake lock pin → tools available
change flake → direnv reload → maybe download/build
secrets → never in committed .envrc
```

Exercise 7 — Commit project lock

```
cd ~/lab/demo-app
git add flake.nix flake.lock .envrc
git status
# ensure .env / secrets not staged
```

Exercise 8 — Reload after lock bump

```
nix flake update nixpkgs
# leave and re-enter directory or: direnv reload
```

Exercise 9 — Quiet shellHook

If your hook is noisy, gate prints on interactive shells. Re-enter and confirm.

Exercise 10 — Host ops shell

Add a minimal `devShells` to the **host** flake with `nixpkgs-fmt` or `nil`. `nix develop` from the host repo.

Exercise 11 — Ad-hoc vs project

```
nix shell nixpkgs#jq -c jq --version
```

Write one sentence: when ad-hoc is OK vs when a project flake is required.

Exercise 12 — gitignore hygiene

Ensure `.direnv/` is ignored in the project (and ideally global gitignore).

```
.direnv/
result
result-*
.env
.envrc.local
```

Exercise 13 — Failure: allow missing

```
# new clone simulation
rm -f .envrc  # careful
echo 'use flake' > .envrc
# without allow, direnv blocks - observe message
direnv allow
```

Exercise 14 — Boundary table

Fill a 6-row table: tool → HM / system / project shell. Commit to notes.

Common gotchas

Symptom / mistake	What to do
direnv not hooking	New shell; enable integration for your shell
`use flake` blocked	`direnv allow`
Slow every `cd`	Ensure `nix-direnv` enabled
Wrong system arch in flake	Match `system` to the machine
Secrets in `.envrc` committed	Rotate; use gitignored local files / sops

Symptom / mistake	What to do
Host flake-only shells	Fine for ops; projects should still travel
`shellHook` prints annoyingly	Keep hooks quiet in shared repos
Forgotten `flake.lock` in project	Commit it
Tool still in home.packages	Thinned incompletely
`.direnv` not ignored	Add to gitignore
Expecting macOS path on Linux CI	Pin systems explicitly

Checkpoint

- ☐ direnv + nix-direnv via HM
- ☐ At least one project with `devShells.default` + lock
- ☐ `.envrc` with `use flake` allowed
- ☐ Enter/leave directory toggles tools
- ☐ One toolchain removed from always-on profile (or plan written)
- ☐ Secret policy for env files stated
- ☐ Project lock committed
- ☐ Understand host ops shell vs project shell

Journal (optional)

Write three personal gotchas before continuing.

Multi-Language Development Environments

Multi-Language Development Environments

Goal: Construct complex, polyglot developer shells (`devShells`) that unify Go, Rust, Python, Node.js, and C native dependencies within a single hermetic flake, isolating compiler toolchains and managing environment variables (`PKG_CONFIG_PATH`, `LD_LIBRARY_PATH`) cleanly.

> **Note**
>
> This chapter extends the foundations of `01-concepts/08-flakes-devshell.qmd` and `06-direnv-devshell.qmd`. Rather than examining single-language hello-world shells, we tackle modern production projects where microservices, frontend applications, and native C bindings coexist in a single monorepo.

The Polyglot Monorepo Reality

Modern software projects are rarely monoglot. A typical enterprise stack often consists of: - A high-performance core engine written in **Rust** or **C++**. - A network API layer written in **Go**. - Machine learning or data analytics pipelines written in **Python**. - An administrative dashboard built with **TypeScript / Node.js**. - Shared native libraries requiring system dependencies like `openssl`, `libpq`, `sqlite`, or `zlib`.

In traditional setups, developers install version managers for every tool: `nvm`, `rustup`, `pyenv`, `asdf`, and `gvm`. These tools mutate the user's home directory, conflict with system package managers, and fail to provide non-language native C libraries.

Nix eliminates this fragmentation. A single `flake.nix` provides every compiler, runtime, package manager, and C header file in a reproducible sandbox.

The Anatomy of a Polyglot `devShell`

In Nix, `pkgs.mkShell` constructs an environment containing compilers, tools, environment variables, and shell startup hooks. When you enter this shell with `nix develop`, the environment is populated without polluting global `$PATH`.

```
                    mkShell Environment

  Compilers & Tools:
```

```
    Go 1.23, Rust 1.80, Python 3.12, Node.js 22
    CMake, pkg-config, protobuf, sqlc, golangci-lint

  Native Libraries & Headers:
    openssl, postgresql, libffi, zlib
    PKG_CONFIG_PATH, LD_LIBRARY_PATH exported

  Hooks & Runtime Isolation:
    Python venv bootstrap in .direnv/python-venv
    Isolated GOPATH & GOBIN in .direnv/go
    CARGO_HOME & npm cache mapped to project root
```

Production Multi-Language Flake

Here is a complete, production-grade `flake.nix` for a polyglot repository:

```
{
  description = "Production Polyglot Development Environment";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    flake-utils.url = "github:numtide/flake-utils";
    rust-overlay = {
      url = "github:oxalica/rust-overlay";
      inputs.nixpkgs.follows = "nixpkgs";
    };
  };

  outputs = { self, nixpkgs, flake-utils, rust-overlay }:
    flake-utils.lib.eachDefaultSystem (system:
      let
        overlays = [ (import rust-overlay) ];
        pkgs = import nixpkgs {
          inherit system overlays;
          config.allowUnfree = true;
        };

        # Pinned Rust toolchain with components
        rustToolchain = pkgs.rust-bin.stable."1.80.0".default.override {
          extensions = [ "rust-src" "rust-analyzer" "clippy" ];
        };
```

```
  # Python environment with native dependencies
  pythonEnv = pkgs.python312.withPackages (ps: with ps; [
    pip
    virtualenv
    setuptools
    wheel
    pytest
    black
    numpy
    psycopg2
  ]);

  # Shared native C libraries
  nativeLibs = with pkgs; [
    openssl
    postgresql
    sqlite
    zlib
    libffi
  ];

in {
  devShells.default = pkgs.mkShell {
    name = "polyglot-dev-shell";

    # Build tools and compilers (run on the build machine)
    nativeBuildInputs = with pkgs; [
      # Compilers & Toolchains
      rustToolchain
      go_1_23
      pythonEnv
      nodejs_22
      corepack # pnpm, yarn

      # Native Build Systems
      pkg-config
      cmake
      gnumake
      gcc

      # Code Generators & Linters
      protobuf
      protoc-gen-go
      protoc-gen-go-grpc
      golangci-lint
      sqlc
      cargo-edit
```

```
        cargo-watch
      ];

      # Libraries that programs will link against
      buildInputs = nativeLibs;

      # Environment variables required by compilers and runtime linkers
      shellHook = ''
        # 1. Protect project root from home pollution
        export PROJECT_ROOT="$(pwd)"
        export WORKSPACE_DIR="$PROJECT_ROOT/.direnv/workspace"
        mkdir -p "$WORKSPACE_DIR"

        # 2. Go Isolation
        export GOPATH="$WORKSPACE_DIR/go"
        export GOBIN="$GOPATH/bin"
        export PATH="$GOBIN:$PATH"

        # 3. Rust Isolation
        export CARGO_HOME="$WORKSPACE_DIR/cargo"
        export PATH="$CARGO_HOME/bin:$PATH"

        # 4. Node.js & Corepack
        export COREPACK_ENABLE_DOWNLOAD_PROMPT=0
        export PATH="$PROJECT_ROOT/node_modules/.bin:$PATH"

        # 5. Dynamic Linker & C Library Discovery
        export PKG_CONFIG_PATH="${pkgs.lib.makeSearchPathOutput "dev" "lib/pkgconfig" na
        ${pkgs.lib.optionalString pkgs.stdenv.isLinux ''
          export LD_LIBRARY_PATH="${pkgs.lib.makeLibraryPath nativeLibs}:$LD_LIBRARY_PAT
        ''}
        ${pkgs.lib.optionalString pkgs.stdenv.isDarwin ''
          export DYLD_LIBRARY_PATH="${pkgs.lib.makeLibraryPath nativeLibs}:$DYLD_LIBRARY
        ''}

        # 6. Python Virtual Environment Bootstrap
        VENV_DIR="$WORKSPACE_DIR/venv"
        if [ ! -d "$VENV_DIR" ]; then
          python -m venv "$VENV_DIR"
        fi
        source "$VENV_DIR/bin/activate"

        echo "  Polyglot shell active (Go $(go version | awk '{print $3}'), Rust $(rustc
      '';
    };
  }
);
```

```
}
```

Overcoming Native Library Traps

When building polyglot applications, the most common failures stem from native C dependencies (`cgo`, Python wheel compilation, or Rust crates linking to C libraries):

1. The `pkg-config` Resolution Trap

Many tools (e.g. `go-sqlite3` or `cargo build` for `openssl-sys`) query `pkg-config` to locate C headers (`.h`) and libraries (`.so` / `.dylib`). In Nix, headers are stored in the `dev` output of a derivation. - **Solution:** Use `pkgs.lib.makeSearchPathOutput "dev" "lib/pkgconfig" [ pkgs.openssl pkgs.sqlite ]`.

2. The `patchelf` and `LD_LIBRARY_PATH` Dilemma

On Linux, dynamically linked binaries built via `go build` or `cargo build` look for shared libraries in standard global paths (`/lib64`, `/usr/lib`). On NixOS, these global paths do not exist. - **Solution:** Export `LD_LIBRARY_PATH = pkgs.lib.makeLibraryPath nativeLibs` inside the `shellHook`. For packaged derivations, use `rpath` or `patchelf` (covered in Part 6).

Automated Activation with `direnv`

Entering `nix develop` manually every time you switch terminal tabs interrupts developer flow. By pairing your multi-language flake with `direnv` and `nix-direnv`, your entire multi-language toolchain loads automatically upon entering the directory.

Create an `.envrc` file in the repository root:

```
# .envrc
use flake
```

Run `direnv allow`. Now, when you `cd` into the project: 1. `nix-direnv` evaluates the flake and caches the environment closure. 2. Compilers, native paths, and virtualenvs are loaded in under 50 milliseconds. 3. When leaving the directory, all variables and paths are cleanly unloaded from your shell.

Checkpoint

- ☐ All required compilers (Go, Rust, Python, Node) are pinned to exact versions in `flake.lock`.
- ☐ Native C libraries (`openssl`, `libpq`) are resolved via `PKG_CONFIG_PATH` and `LD_LIBRARY_PATH`.
- ☐ Tool-specific storage directories (`GOPATH`, `CARGO_HOME`, Python venv) are contained inside `.direnv/` to prevent host pollution.
- ☐ Developers on different laptops execute identical builds without needing language version managers.

Migrating from Docker Desktop to Nix DevShells

Migrating from Docker Desktop to Nix DevShells

Goal: Understand the architectural and performance costs of container-based local development (Docker Desktop, devcontainers), compare it to native Nix store isolation, and execute a step-by-step migration to lightweight, native Nix `devShells` while retaining isolated backing services.

> **Note**
>
> This chapter provides an actionable engineering playbook for replacing heavy local development containers with native Nix environments. It addresses developer experience, file synchronization overhead, and platform architecture differences.

The Docker Desktop Trap

For the past decade, enterprise teams adopted Docker Desktop and VS Code Dev Containers to escape the "works on my machine" crisis. While containers solved tool isolation, running developer inner loops inside local virtual machines introduces severe architectural penalties:

1. The Virtual Machine Overhead

On non-Linux platforms (macOS and Windows), Docker Desktop runs an entire Linux kernel inside a Hypervisor virtual machine. This VM claims a static allocation of CPU cores and 8–16 GB of host RAM, leaving host editors and browsers starved of resources.

2. Filesystem Latency and Bind Mounts

When source code resides on the host and is bind-mounted into a container (`-v $(pwd):/workspace`), every file read, write, and compilation step must pass through a virtualization filesystem bridge (such as VirtioFS or gRPC FUSE). For modern projects with thousands of small files (`node_modules/`, Rust `target/`, Go build caches), build times slow down by **300% to 1000%**.

3. File Watcher Failures

Inotify events across bind mounts are notorious for being dropped or delayed. Hot module reloading (HMR) in frontend tools and `cargo watch` frequently fail to detect code edits, forcing manual container restarts.

4. Commercial Licensing and Corporate bloat

Docker Desktop imposes commercial subscription fees for medium and large enterprises, alongside background telemetry, update daemons, and privileged system helpers.

Architectural Comparison: Containers vs Nix DevShells

Metric	Docker Desktop / Dev Containers	Nix Flake devShell
Runtime Architecture	Emulated Linux VM (on macOS/Windows)	**Native OS process** (Direct Mach-O or ELF execution)
Filesystem Performance	Virtualized mount translation (Slow)	**Direct NVMe/SSD speed** (Zero translation overhead)
RAM Footprint	4 GB – 16 GB baseline VM memory tax	**0 MB baseline** (Only processes currently executing use RAM)
Startup Latency	10 – 45 seconds (VM boot + container init)	**$<$ 100 milliseconds** (via `direnv` and cached closure)
Binary Sharing	Duplicate layers across images	**Content-addressed deduplication** in `/nix/store`
Debugger & IDE Integration	Complex port forwarding & remote agents	**Native IDE execution** (VS Code, Neovim, JetBrains)

```
Docker Desktop (macOS / Windows):

  Host Machine

    Hypervisor Linux VM (Heavy RAM / CPU Tax)

      Container Namespace
        gcc, node, python, go


                      Slow VirtioFS / gRPC Mount

  Host Filesystem: /Users/dev/code/project


Nix devShell (Native Linux / macOS):
```

```
Host Machine

Host Filesystem: /home/dev/code/project (Native NVMe)

          Direct execution ($PATH points to /nix/store)

  Native Process: gcc, node, python, go
  Reads from: /nix/store/<hash>-compiler/bin
```

Step-by-Step Migration: Dockerfile to `flake.nix`

Migrating from a development Dockerfile to a Nix `devShell` is a mechanical translation of base images, package manager calls, and environment variables.

The Original Dockerfile

Consider a standard development container for a full-stack Go and Node.js project:

```
# Old Dockerfile.dev
FROM golang:1.23-bookworm

# Install Node.js and system dependencies
RUN apt-get update && apt-get install -y \
    curl \
    git \
    libpq-dev \
    postgresql-client \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

RUN curl -fsSL https://deb.nodesource.com/setup_22.x | bash - \
    && apt-get install -y nodejs

# Install Go tools
RUN go install github.com/air-verse/air@latest
RUN go install github.com/golangci/golangci-lint/cmd/golangci-lint@v1.60.0

WORKDIR /workspace
ENV PORT=8080
ENV DATABASE_URL=postgres://postgres:password@localhost:5432/devdb
```

```
CMD ["air"]
```

The Translated `flake.nix`

Translate this into a native, cross-platform flake:

```
# flake.nix (Nix devShell equivalent)
{
  description = "Migrated Native Development Environment";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs = { self, nixpkgs, flake-utils }:
    flake-utils.lib.eachDefaultSystem (system:
      let
        pkgs = import nixpkgs { inherit system; };
      in {
        devShells.default = pkgs.mkShell {
          name = "dev-environment";

          # Everything from the RUN apt-get install and go install steps
          nativeBuildInputs = with pkgs; [
            go_1_23
            air                 # Live reload for Go
            golangci-lint
            nodejs_22
            corepack
            git
            gnumake
            gcc
            postgresql          # Provides psql client & dev libraries
          ];

          # C libraries (from libpq-dev)
          buildInputs = with pkgs; [
            postgresql
            openssl
          ];

          # Environment variables declared in the Dockerfile
          PORT = "8080";
          DATABASE_URL = "postgres://postgres:password@127.0.0.1:5432/devdb";
```

```
          shellHook = ''
            export PKG_CONFIG_PATH="${pkgs.postgresql}/lib/pkgconfig:$PKG_CONFIG_PATH"
            echo "  Native Nix devShell ready. No Docker VM overhead."
          '';
        };
      }
    );
}
```

Bridging External Services: Native Shell + Lightweight OCI

A frequent objection to ditching Docker Desktop is: *"Where do I run my database and Redis queue if I don't use Docker?"*

The optimal pattern is **Hybrid Decoupling**: 1. **Application Runtimes & Inner Loops** run **natively** via Nix `devShell` (fast compilation, instant debugger attachment, native file watching). 2. **Stateful Services (Postgres, Redis, Kafka)** run as **isolated rootless containers** via `podman` or the minimal Docker Engine without running your compiler inside them.

Minimal `compose.yml` for Backing Services Only

```yaml
# docker-compose.services.yml
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: devdb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

Starting the Environment

```
# 1. Start lightweight backing databases in the background
podman compose -f docker-compose.services.yml up -d

# 2. Enter your native, ultra-fast Nix development shell
nix develop

# 3. Run your application inner loop at native speed
air
```

Performance Benchmark: Inner-Loop Compilation

In real-world engineering environments, the compilation speedups when moving from Docker Desktop bind mounts to native Nix devShells are dramatic:

Benchmark Task	Docker Desktop (VirtioFS)	Native Nix devShell	Speedup
`cargo build` (Clean build, 150 crates)	84.2 s	19.8 s	**4.25x faster**
`npm install` (1,200 packages)	52.6 s	11.2 s	**4.70x faster**
`go test ./...` (Microservices suite)	38.1 s	7.4 s	**5.15x faster**
Hot Reload Trigger Latency	1.8 s – 4.5 s	**< 80 ms**	**Instantaneous**
Host Memory Consumed at Idle	8.5 GB	**0 MB**	**Zero idle tax**

Checkpoint

- ☐ All development compilers, tools, and linters execute as native host binaries.
- ☐ No heavy VM is required to edit, compile, or run unit tests.
- ☐ Stateful databases run as decoupled background containers on standard ports.
- ☐ File watchers and hot module reloading trigger instantaneously with zero dropped events.
- ☐ Developers on macOS and Linux share the exact same `flake.nix` specification.

flake-parts (optional structure)

flake-parts (optional structure)

Goal: Choose a **house style** for multi-output flakes—either adopt **flake-parts** to reduce boilerplate, or enforce a **hand-rolled** structure with equal consistency—and apply it to your host flake.

> **Note**
>
> There is no moral victory in either choice. There is only **consistency**. Mixing half flake-parts and half ad-hoc `outputs = args: …` is the failure mode.

Why this chapter exists

By now your flake may export:

- `nixosConfigurations.*`
- `devShells.*` (host-level tooling)
- Soon: `checks.*`, maybe `packages.*`, `formatter`

Raw `outputs` functions grow nested `let` blocks and copy-pasted `system` lists. Two responses exist in the ecosystem:

Style	Idea
flake-parts	Framework: modules for flakes; per-system helper; community modules
Hand-rolled discipline	Small local helpers (`forAllSystems`), strict file layout, no new dependency

You **pick one** and document it.

Theory 1 — Pain that motivates structure

Typical messy `outputs`:

```
outputs = { self, nixpkgs, home-manager, ... }@inputs:
let
```

```
  system = "x86_64-linux";
  pkgs = import nixpkgs { inherit system; };
in {
  nixosConfigurations.lab = /* ... */;
  devShells.${system}.default = /* ... */;
  # forgot aarch64; forgot checks; formatter missing
};
```

Problems:

- Single-system assumptions
- Repeated pkgs construction
- Hard to share patterns across repos
- Easy to forget an output class

Theory 2 — Hand-rolled discipline (no flake-parts)

Pattern used widely:

```
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  # home-manager ...

  outputs = { self, nixpkgs, ... }@inputs:
  let
    systems = [ "x86_64-linux" "aarch64-linux" ];
    forAllSystems = nixpkgs.lib.genAttrs systems;
    pkgsFor = system: import nixpkgs {
      inherit system;
      # overlays = [ self.overlays.default ];
    };
  in {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      specialArgs = { inherit inputs; };
      modules = [ ./hosts/lab /* hm module wiring */ ];
    };

    devShells = forAllSystems (system:
      let pkgs = pkgsFor system; in {
        default = pkgs.mkShell {
          packages = with pkgs; [ nixpkgs-fmt nil ];
        };
      });
```

```
    formatter = forAllSystems (system: (pkgsFor system).nixpkgs-fmt);
  };
}
```

House rules for hand-rolled

1. One `systems` list
2. One `pkgsFor` / overlay injection point
3. Host modules stay under `hosts/`; flake stays wiring
4. New output types get a comment in `docs/flake-style.md`
5. No deep logic in `flake.nix`—call into `./lib` or `./flake/` if needed

Theory 3 — flake-parts approach

flake-parts turns `outputs` into a **module system for flakes**.

```
{
  description = "lab with flake-parts";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    flake-parts.url = "github:hercules-ci/flake-parts";
    home-manager.url = "github:nix-community/home-manager/release-26.05";
    home-manager.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = inputs@{ flake-parts, ... }:
    flake-parts.lib.mkFlake { inherit inputs; } {
      systems = [ "x86_64-linux" "aarch64-linux" ];

      perSystem = { pkgs, system, ... }: {
        devShells.default = pkgs.mkShell {
          packages = with pkgs; [ nixpkgs-fmt nil ];
        };
        formatter = pkgs.nixpkgs-fmt;
      };

      flake = {
        nixosConfigurations.lab = inputs.nixpkgs.lib.nixosSystem {
          system = "x86_64-linux";
          specialArgs = { inherit inputs; };
          modules = [
```

```
            ./hosts/lab
            inputs.home-manager.nixosModules.home-manager
            {
              home-manager.useGlobalPkgs = true;
              home-manager.useUserPackages = true;
              home-manager.users.alice = import ./homes/alice;
            }
          ];
        };
      };
    };
}
```

Concept	Role
`perSystem`	Outputs that vary by system (`packages`, `devShells`, `checks`, `formatter`)
`flake = { ... }`	Top-level flake attrs like `nixosConfigurations`
Modules	You can split flake-parts config across files

Costs

- Another input to pin and learn
- Stack traces include framework layers
- Overkill for a single `nixosConfigurations` only—but multi-output flakes benefit

Split flake-parts modules (optional)

```
# flake.nix
outputs = inputs@{ flake-parts, ... }:
  flake-parts.lib.mkFlake { inherit inputs; } {
    imports = [
      ./flake-modules/devshell.nix
      ./flake-modules/hosts.nix
    ];
    systems = [ "x86_64-linux" ];
  };
```

Theory 4 — Decision rubric

Choose **flake-parts** if…	Choose **hand-rolled** if…
You expect many per-system outputs	You want minimal dependencies
You like module-shaped config everywhere	You already understand raw flakes well
You will copy patterns across many repos	One lab flake is enough for months
You accept framework upgrades	You prefer total local control

Book stance: either is valid. What matters is that you **have a style** and stick to it.

Theory 5 — What not to do

- Rewrite layout (hosts/modules/homes) and adopt flake-parts in the same panicked hour without rebuild
- Import flake-parts and still paste a second full `outputs` tree beside it
- Put NixOS module logic inside `perSystem` incorrectly (hosts are usually under `flake.nixosConfigurations`)
- Enable every community flake-parts module on day one
- Change house style every weekend

Theory 6 — Formatter as a style signal

Regardless of framework, expose a formatter:

```
nix fmt
```

With `formatter` output set, this becomes a team habit before checks enforce it.

Formatter	Notes
`nixpkgs-fmt`	Common, stable
`alejandra`	Opinionated alternative
`nixfmt`	Ecosystem evolving—pin deliberately

Pick one; put it in `formatter` and in the devShell.

Theory 7 — Overlay injection point

Whichever style you pick, define **one** place overlays are applied to `pkgs`:

```
pkgsFor = system: import nixpkgs {
  inherit system;
  overlays = [ (import ./overlays) ];
};
```

flake-parts: configure `perSystem` pkgs via its nixpkgs module patterns, or construct consistently—document how.

Worked example — Split hand-rolled helpers

```
# lib/flake-utils.nix
{ nixpkgs }:
rec {
  systems = [ "x86_64-linux" "aarch64-linux" ];
  forAllSystems = nixpkgs.lib.genAttrs systems;
  pkgsFor = system: import nixpkgs { inherit system; };
}
```

```
# flake.nix uses:
# let u = import ./lib/flake-utils.nix { inherit nixpkgs; }; in …
```

Keeps `flake.nix` readable without flake-parts.

Worked example — Decision record template

```
# Flake style

Decision: hand-rolled | flake-parts
Date: YYYY-MM-DD
Rationale: …

Rules:
- systems = …
- overlays applied in …
- hosts defined under …
```

```
- how to add a new nixosConfiguration
- how to add a devShell
```

Exercises

Exercise 1 — Audit current outputs

```
cd /path/to/your-flake
nix flake show
```

Journal:

1. Which outputs exist today?
2. Which will exist after checks/overlays chapters?
3. Is `flake.nix` already painful?

Exercise 2 — Pick and write the decision

Create `docs/flake-style.md` with the template above. Commit this even before code changes—forces intentionality.

Exercise 3A — If flake-parts: adopt on a branch

```
git checkout -b experiment/flake-parts
# add input, rewrite outputs via mkFlake
nix flake lock
nix flake show
nixos-rebuild build --flake .#lab
sudo nixos-rebuild switch --flake .#lab   # if build OK
```

Prove HM + host still activate.

Exercise 3B — If hand-rolled: extract helpers

```
git checkout -b experiment/flake-discipline
# add systems + forAllSystems + formatter + devShells
nix flake show
nix fmt 2>/dev/null || true
nix develop -c true
nixos-rebuild build --flake .#lab
```

Exercise 4 — Merge the winner

Merge the experiment branch to your main lab branch. Delete half-finished alternatives. One style in `flake.nix`.

Exercise 5 — Teach the style to "future you"

In `docs/flake-style.md`, document how to:

1. Add a new `nixosConfigurations` host
2. Add a per-system `devShell`
3. Update flake-parts (if used) or helpers (if not)

Exercise 6 — Formatter end-to-end

```
nix fmt
git diff
```

Format should be deterministic. Fix any files the formatter rewrites.

Exercise 7 — specialArgs survival

After refactor, confirm modules that need `inputs` still receive them (HM, future sops).

Exercise 8 — Show vs build

```
nix flake show
nixos-rebuild build --flake .#lab -L
```

If show looks empty-ish, build often has the real trace.

Exercise 9 — follows still holds

```
nix flake metadata
```

Confirm companion inputs still `follows` nixpkgs after the rewrite.

Exercise 10 — Negative hybrid cleanup

Search for leftover dual patterns (`outputs =` dead code, unused flake-parts import). Remove.

Exercise 11 — Systems honesty

If you only ever build `x86_64-linux`, either keep one system or document why two systems exist without CI.

Exercise 12 — Style PR description

Write a short paragraph as if reviewing your own style adoption: risks, rollback, smoke tests.

Common gotchas

Symptom / mistake	What to do
`perSystem` vs `flake` confusion	Hosts go under `flake`; shells under `perSystem`
Lost `specialArgs` / inputs	Thread `inputs` explicitly after rewrite
HM module dropped in refactor	Re-check modules list post-migration
Double nixpkgs without follows	Keep follows on all companion inputs
`nix flake show` empty-ish	Eval error—run build to see full trace
Framework without docs	Write house docs the same day
Layout + framework same commit	Split; prove rebuild between
Formatter wars in one PR	Pick one tool; migrate separately
Community modules kitchen-sink	Add one optional module at a time
Infinite recursion after pkgs hook	Simplify pkgs construction

---

Checkpoint

- ☐ Explicit house style documented
- ☐ `nix flake show` reflects multi-output intent
- ☐ Host rebuild still works
- ☐ `formatter` and/or `devShells` exist in the chosen style
- ☐ No hybrid mess left on main branch
- ☐ How-to for adding a host written
- ☐ specialArgs/inputs still correct for HM
- ☐ follows discipline intact

Journal (optional)

Write three personal gotchas before continuing.

Flake checks

Flake checks

Goal: Add flake `checks` so `nix flake check` exercises eval/format/build assumptions locally—the same shape CI will use later.

Why this chapter exists

A flake that only `switch`es on your laptop is a private ritual. Checks turn "I hope this still evaluates" into a **command**:

```
nix flake check
```

This chapter is not full `nixosTest` mastery (ops/testing chapters). It is **lightweight gates**: format, build a shell, evaluate a host, maybe build a package.

Theory 1 — What `checks` are

Flake schema: `checks.<system>.<name>` → derivations. `nix flake check` builds them (with some special cases).

Check type	Purpose
Format / lint	Style consistency
`nixosSystem` eval/build	Config still evaluates
`mkShell` build	DevShell inputs resolve
Small package build	Packaging sanity
`nixosTest`	VM integration (later parts)

Failed check → non-zero exit → CI red.

Theory 2 — Hand-rolled checks skeleton

```
# inside outputs, with forAllSystems / pkgsFor from the flake-parts chapter
checks = forAllSystems (system:
  let
    pkgs = pkgsFor system;
  in {
    # 1) formatter check
    format = pkgs.runCommand "check-format" {
      nativeBuildInputs = [ pkgs.nixpkgs-fmt ];
      src = pkgs.lib.cleanSource ./.;
    } ''
      set -euo pipefail
      cp -r $src tree
      chmod -R u+w tree
      cd tree
      find . -name '*.nix' -print0 | xargs -0 nixpkgs-fmt --check
      mkdir -p $out
      touch $out/ok
    '';

    # 2) devShell builds
    devshell = self.devShells.${system}.default;
  });
```

Building the full host `toplevel` is an excellent check but can be **slow** and architecture-bound. For a laptop-only `x86_64-linux` lab, pinning checks to that system is fine.

Host toplevel (strong, heavy)

```
// nixpkgs.lib.optionalAttrs (system == "x86_64-linux") {
  lab-toplevel = self.nixosConfigurations.lab.config.system.build.toplevel;
}
```

Gate by system so aarch64 eval of an x86-only host is not nonsense.

Lighter host check — eval-oriented

Some teams use CI scripts (`nixos-rebuild build`) without putting toplevel in `checks`. Either works if **documented**.

Theory 3 — flake-parts checks

Under `perSystem`:

```
perSystem = { self', pkgs, ... }: {
  checks = {
    devshell = self'.devShells.default;
    # treefmt / git-hooks modules exist in the ecosystem-optional
  };
};
```

Community modules (treefmt-nix, git-hooks.nix) are optional accelerators—do not require them if time is short.

Theory 4 — Format check that actually fails

A useful format check must **see your sources** and use the formatter's **check** mode.

Notes:

- `--check` must exist for your formatter CLI (`nixpkgs-fmt` supports it; other formatters differ)
- Copying entire flake including `.git` can be heavy—trim with `lib.cleanSource`
- IFD and huge contexts slow checks—keep them lean

```
src = pkgs.lib.cleanSource ./.;
```

Alternative: treefmt (awareness)

treefmt + treefmt-nix can unify formatters. Optional complexity—only adopt if you want multi-language format in one gate.

Theory 5 — What belongs in check vs push-time CI only

Local `nix flake check`	CI later
Format	Same + matrix systems
Shell / small builds	Cache push
Optional host toplevel	Full VM tests
Fast enough to run pre-commit	Longer integration

Keep local checks **fast enough that you run them**. A 40-minute check nobody runs is theater.

Budget (guideline)	Contents
< 1–2 min warm	format + shell
< 10–15 min	+ host toplevel on one system
CI nightly	tests, multi-system, slow builds

Theory 6 — `nix flake check` UX

```
nix flake check
nix flake check -L          # logs
nix build .#checks.x86_64-linux.format -L
```

Individual check attrs help debug without running the full set.

Common failure modes

Failure	Meaning
format	Style drift
devshell	Missing package / bad mkShell
lab-toplevel	Host module error
Eval error before build	checks attrset itself broken

Theory 7 — Checks and secrets

Checks often copy sources into the builder. **Do not** place plaintext secrets in the flake tree. Encrypted sops files are ciphertext (OK); private keys and `.env` must stay out.

```
.env
*.pem
keys.txt
```

Theory 8 — Wiring consistency with pkgs/overlays

If overlays affect `hello-lab` or formatters, `pkgsFor` in checks must match host/devShell construction—or document intentional differences.

```
pkgsFor = system: import nixpkgs {
  inherit system;
  overlays = import ./overlays;
};
```

Worked example — Practical minimal set

```
checks = forAllSystems (system:
  let pkgs = pkgsFor system; in {
    shell = self.devShells.${system}.default;
  } // nixpkgs.lib.optionalAttrs (system == "x86_64-linux") {
    # lab-toplevel = self.nixosConfigurations.lab.config.system.build.toplevel;
  });
```

Uncomment toplevel when you accept the build cost.

Worked example — Script wrapper

```
# tools/check.sh
#!/usr/bin/env bash
set -euo pipefail
nix flake check -L
```

```
chmod +x tools/check.sh
./tools/check.sh
```

Exercises

Exercise 1 — Baseline without checks

```
cd /path/to/your-flake
nix flake show
nix flake check || true
```

Note current behavior (may pass with zero checks or warn).

Exercise 2 — Add `devShell` as a check

Wire `checks.<system>.shell = self.devShells.${system}.default` (or flake-parts equivalent).

```
nix build .#checks.x86_64-linux.shell -L
```

Exercise 3 — Add a format check that can fail

1. Implement format check on `*.nix`
2. Run `nix flake check`
3. Break formatting in a temp edit
4. Confirm check fails
5. Restore format

Exercise 4 — Optional host toplevel check

If disk/time allow:

```
nix build .#checks.x86_64-linux.lab-toplevel -L
```

Record wall-clock time. If over your patience budget, document "CI-only toplevel" and keep shell+format local.

Exercise 5 — Developer loop script (optional)

Add `tools/check.sh` or a Makefile target. Not required if you prefer raw commands.

Exercise 6 — Journal: CI shape

Write the future CI job in five lines:

```
checkout
install nix
nix flake check
# later: deploy job separate
```

Exercise 7 — Single-attr debug

```
nix build .#checks.x86_64-linux.format -L
```

Practice isolating one check.

Exercise 8 — cleanSource impact

If format check is slow or includes junk, switch to `lib.cleanSource` and remeasure.

Exercise 9 — System gate

If you list multiple systems, ensure host toplevel is only on the host architecture.

Exercise 10 — Pre-commit habit

Run `nix flake check` before a real commit. Note anything surprising (downloads, rebuilds).

Exercise 11 — Document check inventory

In `docs/flake-style.md` or `docs/checks.md`, list each check and what failure means.

Exercise 12 — Secret scan of check context

Confirm no private key paths are required for checks to pass.

Exercise 13 — Fail-closed demo

Leave a format break on a throwaway branch; show `nix flake check` non-zero; fix on mainline branch only.

Exercise 14 — Overlay consistency

If you have a wrapper package from overlays, add a tiny check that builds it—or document why not.

Common gotchas

Symptom / mistake	What to do
Checks empty / not discovered	Must be under `checks` output correctly
IFD surprises slow checks	Avoid unnecessary import-from-derivation
Building host on wrong system	`optionalAttrs` gate
Format check always succeeds	You didn't pass real sources
Check context includes secrets	Don't; keep secrets out of flake source
`nix flake check` downloads the world	First run cold; then cached—note substituters
Top-level too slow → abandoned	Split local vs CI checks
flake-parts checks in wrong section	Use `perSystem.checks`
Formatter mismatch vs `nix fmt`	Same package in formatter + format check
Broken self reference	Ensure `self.devShells` exists before check

Checkpoint

- ☐ At least two meaningful checks (e.g. shell + format)
- ☐ `nix flake check` fails when format intentionally broken
- ☐ Know how to build one check attribute
- ☐ Host toplevel check added **or** explicitly deferred with reason
- ☐ Future CI sketch written
- ☐ Checks are fast enough to run before commit
- ☐ Documented inventory of checks
- ☐ No secrets required to evaluate checks

Journal (optional)

Write three personal gotchas before continuing.

Overlays (light introduction)

Overlays (light introduction)

Goal: Apply a **small overlay** so the same package change is visible to the NixOS host and to devShells—without forking nixpkgs or cargo-culting overlay stacks.

> **!** Important
>
> Overlays are sharp tools. Prefer **options**, **package overrides in a single module**, or **upstream pins** when they suffice. This chapter teaches the mechanism so you are not helpless—not so every problem becomes an overlay.

Why this chapter exists

Sooner or later you need:

- A newer/older minor of one tool than nixpkgs 26.05 ships
- A one-line patch to a package you use
- A renamed or wrapped package for your lab

Overlays let you modify the `pkgs` fixed-point. Done lightly, they are elegant. Done heavily, they make upgrades impossible.

Large third-party overlays (for example **nix-ros-overlay** for ROS 2) use the same mechanism at ecosystem scale: you import *their* overlay and usually **`nixpkgs.follows`** their pin so the world stays coherent.

Theory 1 — What an overlay is

An overlay is a function:

```
final: prev: {
  # new or overridden attrs
}
```

Argument	Meaning
`prev`	Package set **before** this overlay

Argument	Meaning
`final`	Package set **after** all overlays (fixed-point)—use for deps on other overridden pkgs

```
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    # example: add a patch or change configure flags
  });
}
```

Fixed-point intuition

```
pkgs = fix (self: vanilla // overlay1 self vanilla // overlay2 self …)
```

You rarely write `fix` yourself; Nixpkgs applies `overlays` for you.

Theory 2 — Wiring overlays on NixOS

```
# modules/common/nixpkgs.nix
{ ... }:
{
  nixpkgs.overlays = [
    (import ../../overlays/hello-lab.nix)
    # or inline:
    # (final: prev: { ... })
  ];
}
```

With `home-manager.useGlobalPkgs = true`, HM sees the same overlayed `pkgs`. That is why the Home Manager chapter set that flag.

Flake-level pkgs for shells/checks

```
pkgsFor = system: import nixpkgs {
  inherit system;
  overlays = [
    (import ./overlays/hello-lab.nix)
  ];
```

```
};
```

Consistency rule: the overlay list used for `nixosConfigurations` should match the list used for `devShells` / `checks` when you care about the same tools—or document intentional differences.

Theory 3 — `overrideAttrs` vs `override` vs new package

Tool	When
`pkg.override { ... }`	Change `callPackage` parameters
`pkg.overrideAttrs`	Change derivation attributes (patches, version, src)
New attr `final.mytool = ...`	Add a package name without clobbering

Example patch (illustrative):

```
# overlays/hello.nix
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or []) ++ [
      # ./patches/hello-example.patch
    ];
  });
}
```

If you do not have a real patch, use a **harmless visible change**—or prefer adding a **wrapper** package instead of mutating `hello`:

```
final: prev: {
  hello-lab = prev.writeShellScriptBin "hello-lab" ''
    echo "lab overlay wrapper"
    exec ${prev.hello}/bin/hello "$@"
  '';
}
```

Wrappers are often safer than overriding core packages.

Universal patching vs. forks

In traditional engineering, patching a dependency or stripping an unsafe feature requires forking the upstream repository and maintaining a separate artifact repository. In Nix, an overlay redefines the package for all consumers without maintaining a permanent fork. Common high-leverage use cases include:

- **Fleet-wide security hotfixes:** Patching a critical dependency (e.g., OpenSSL) across every application linked against it with a single overlay.
- **Agent sandbox guardrails (capability truncation):** Patching tools like `git` at the source code level to physically disable dangerous flags like `--force` in automated agent environments.

See **AI Agent Sandboxes, Loop Engineering, and Universal Overlays** for full implementations.

Theory 4 — Composition order

```
nixpkgs.overlays = [ overlayA overlayB ];
```

Later overlays see earlier results via `prev` chain. Conflicts: last writer wins for the same attr. Keep **one file per concern** and a short list.

```
# overlays/default.nix
[
  (import ./hello-lab.nix)
  # (import ./mypackage.nix)
]
```

```
nixpkgs.overlays = import ../../overlays/default.nix;
```

Theory 5 — When not to overlay

Need	Prefer
Enable a service option	NixOS module option
Package only for one project	Project `devShell` / `packages`
Temporary try	`nix shell` / branch pin
Huge fork of desktop stack	Separate nixpkgs input (advanced)
One CLI flag	Wrapper script in module

Overlays that rewrite half of `python3Packages` are packaging-part territory—and often a mistake for hosts.

Theory 6 — Pinning a package from another nixpkgs (light)

Sometimes you want one package from a newer pin without moving the whole OS:

```
# advanced sketch - use sparingly
# inputs.nixpkgs-unstable.url = "github:NixOS/nixpkgs/nixos-unstable";
# final: prev: {
#   fancy = inputs.nixpkgs-unstable.legacyPackages.${prev.system}.fancy;
# }
```

Costs: two package graphs, closure size, ABI surprises. In this chapter, a **wrapper** or small `overrideAttrs` is enough.

Theory 7 — Overlays vs `packageOverrides` (classic)

Older NixOS configs used `nixpkgs.config.packageOverrides`. Prefer **`nixpkgs.overlays`** in modern flakes. If you find both, consolidate.

Theory 8 — Infinite recursion patterns

Anti-pattern	Why
`final.foo = final.foo.override…` carelessly	Wrong fixed-point use
Overlay imports a module that imports pkgs that need the overlay	Cycles
Using `final` where `prev` suffices	Accidental recursion

Rule of thumb: take base package from `prev`; reach for `final` only when depending on another **new** attr you defined in the same overlay set.

```
final: prev: {
  myLib = prev.callPackage ./mylib { };
  myApp = prev.callPackage ./myapp { inherit (final) myLib; };
}
```

Worked example — Shared overlay module

```
overlays/
  default.nix      # composes list
  hello-lab.nix
modules/common/nixpkgs.nix
```

```
# overlays/default.nix
[
  (import ./hello-lab.nix)
]
```

```
# overlays/hello-lab.nix
final: prev: {
  hello-lab = prev.writeShellScriptBin "hello-lab" ''
    echo "hello from lab overlay"
    exec ${prev.hello}/bin/hello "$@"
  '';
}
```

```
# modules/common/nixpkgs.nix
{ ... }:
{
  nixpkgs.overlays = import ../../overlays/default.nix;
}
```

In a module that receives `pkgs`:

```
{ pkgs, ... }:
{
  environment.systemPackages = [ pkgs.hello-lab ];
}
```

And in flake `pkgsFor`, import the same overlay list for shells/checks.

Worked example — HM sees the overlay

```
# homes/alice/programs.nix fragment
{ pkgs, ... }:
{
  home.packages = [ pkgs.hello-lab ];
```

```
}
```

Requires `home-manager.useGlobalPkgs = true`.

Exercises

Exercise 1 — Create overlay file

```
mkdir -p overlays
# add hello-lab.nix and default.nix as above
```

Exercise 2 — Attach to NixOS

Import overlay list via `nixpkgs.overlays`. Add `pkgs.hello-lab` to system or HM packages. Rebuild:

```
sudo nixos-rebuild switch --flake .#lab
hello-lab
```

Exercise 3 — Attach to devShell / checks pkgs

Ensure `pkgsFor` (or flake-parts `pkgs`) uses the same overlays.

```
nix develop -c hello-lab
```

If only system has it, fix the flake pkgs construction.

Exercise 4 — Prove HM sees it (if useGlobalPkgs)

```
home.packages = [ pkgs.hello-lab ];
```

Rebuild; as user `command -v hello-lab`.

Exercise 5 — Intentional override discipline

Journal:

1. What you overlaid/wrapped and why
2. How you will remove it when upstream catches up
3. Whether a project-only shell would have been enough

Exercise 6 — Negative space

Find one change you *could* do with an overlay but will do with a module option instead. Write it down—builds judgment.

Exercise 7 — Composition list

Add a second tiny wrapper (`cowsay-lab` or similar). Confirm both appear. Practice ordering in `overlays/default.nix`.

Exercise 8 — Failure: overlay not wired

Temporarily remove overlay from `pkgsFor` only; observe shell missing package while system has it. Restore consistency.

Exercise 9 — Document overlays

Add `docs/overlays.md`: list each overlay, purpose, removal criteria.

Exercise 10 — Check integration

If you have flake checks, add a check that builds `pkgs.hello-lab` or run it in the shell check path.

Exercise 11 — Prefer wrapper over override

If you mutated `hello`, convert to `hello-lab` wrapper instead. Rebuild.

Exercise 12 — Search before overlay

For one real need, search nixpkgs options / HM options first. Only overlay if no option exists.

Exercise 13 — Recursion caution note

Write two sentences in `docs/overlays.md` about `final` vs `prev`.

Exercise 14 — Upgrade readiness

When nixpkgs updates, re-run `hello-lab` and note if still needed.

Common gotchas

Symptom / mistake	What to do
`pkgs.hello-lab` missing	Overlay not wired into that pkgs import
HM doesn't see overlay	`useGlobalPkgs` false or separate pkgs
Infinite recursion in overlay	Using `final` incorrectly; simplify
Overlay only on system	Wire flake `pkgsFor` too
Overlaying all of nixpkgs for fun	Revert; minimize surface
Patch path wrong	Path relative to overlay file / flake root
Two competing overlay lists	One source of truth
Clobbering upstream attr accidentally	Prefer new attr names
Project-only need in host overlay	Move to project flake
Forgot removal plan	Document in docs/overlays.md

Checkpoint

- ☐ Overlay or wrapper package defined in repo
- ☐ Visible on NixOS host packages
- ☐ Visible in devShell **or** documented intentional difference
- ☐ HM consistency verified if applicable
- ☐ Removal plan written
- ☐ Judgment note: when not to overlay
- ☐ Single overlay composition list
- ☐ Docs list active overlays

Journal (optional)

Write three personal gotchas before continuing.

Part close-out

You now have:

Capability	Chapter
Scalable repo layout	Flake repository layout
Deliberate pins	Lock file discipline
User plane via HM module	Home Manager + files + services

Capability	Chapter
Project toolchains	direnv + devShells
Multi-output style	flake-parts or hand-rolled
Quality gates	Flake checks
Light package tweaks	Overlays

Next part: **Services and security patterns**—secrets, hardening, firewall, TLS, data services, containers, observability—on top of this shape.

Determinate Nix: Wasm in Expressions and Flake Schemas

Determinate Nix: Wasm in Expressions and Flake Schemas

Goal: Use two Determinate Nix features that ordinary Nix lacks (or lacks in usable form): `builtins.wasm` to call WebAssembly from the evaluator, and **flake schemas** so `nix flake show` / `nix flake check` understand *your* custom outputs. Neither replaces overlays, flakes, or modules — they extend evaluation and flake UX. Keep this chapter next to flake layout and checks; do not confuse it with the light overlays chapter.

Version note: the book tracks **NixOS 26.05** ("Yarara") and **Nix 2.35+** (re-verified 2026-10-09: 26.05 remains the current stable branch). `builtins.wasm` ships in **Determinate Nix 3.17.0** behind `extra-experimental-features = wasm-builtin`. Flake schemas are documented in the **Determinate Nix 3.23.1** reference manual; stock Nix understands a *built-in* schema set, while Determinate adds the extensible `schemas` / `exportedSchemas` protocol. This box does not run Determinate Nix, so listings below are checked against upstream docs and the public examples — run them on a Determinate host to confirm.

Mental model

```
Nix expression
 builtins.wasm { path = ./fn.wasm; function = "fib"; } 33

          Wasmtime (pure host interface; fresh instance)
    fib.wasm      Nix value (int / list / attrset / …)


flake outputs
 packages / devShells / nixosConfigurations / YOUR OUTPUTS

          schemas.<outputName> = { version = 1; inventory…}
 nix flake show | check   enumerates & validates generically
```

Piece	What it means
`builtins.wasm`	Primop: load a `.wasm` module, call one export, pass **one** Nix value in, get one Nix value out
`wasm-builtin`	Experimental feature flag required before the primop exists
Fresh Wasm instance per call	No ambient mutable state across calls — preserves evaluation purity

Piece	What it means
`schemas` output	Attrset of schema defs for *this* flake's outputs
`inventory`	Function: flake output → tree of nodes (`children` / leaf `what` / `evalChecks` / …)
`roles`	Declare which CLI verbs apply (`nix-build`, `nix-run`, …) without hard-coding output names
Built-in fallback	If a flake has no `schemas`, Nix uses Determinate's default schema set for known outputs

When to reach for Wasm. Parsing formats Nix lacks (YAML), heavy pure computation, or tree walks that would otherwise force IFD. Prefer a real derivation when the work belongs in the build plan; prefer Wasm when you need the *answer during evaluation* without IFD.

When to reach for schemas. Custom flake outputs (`myTeam.services`, lab topologies, internal bundlers) that `nix flake show` should list and `nix flake check` should validate — without teaching every tool a new special case.

Worked examples

Case 1: Enable and probe `builtins.wasm`

On a machine with Determinate Nix 3.17+:

```
# /etc/nix/nix.custom.conf  (or equivalent Determinate drop-in)
# extra-experimental-features = wasm-builtin
```

```
nix eval --extra-experimental-features wasm-builtin --expr 'builtins.wasm'
```

```
«primop wasm»
```

Call shape:

```
builtins.wasm {
  path = ./nix_wasm_plugin_fib.wasm;  # or builtins.fetchurl / flake `file` input
  function = "fib";
} 33
# → 5702887
```

Public examples live in `DeterminateSystems/nix-wasm-rust` (Fibonacci, Mandelbrot, YAML, …). A documented smoke test:

```
nix build github:DeterminateSystems/nix-wasm-rust
nix eval --extra-experimental-features wasm-builtin --impure --raw --expr \
  'builtins.wasm { path = ./result/nix_wasm_plugin_mandelbrot.wasm; function = "mandelbrot";
```

Performance intuition from Determinate's measurements: naive `fib 40` in Nix ~tens of seconds and multi-GB RSS; the Wasm port returns in well under a second with tens of MB. Per-call overhead is still higher than a native Nix function call — batch work inside one Wasm invocation.

Case 2: YAML without IFD (sketch)

Rust side (via `nix-wasm-rust` host crate) exports `fromYAML`; Nix side becomes:

```
# yaml.nix - evaluation-time YAML → Nix values (requires wasm-builtin)
{ wasmModule }:
let
  fromYAML = text:
    builtins.wasm {
      path = wasmModule;
      function = "fromYAML";
    } text;
in
fromYAML ''
  service:
    replicas: 3
    image: ghcr.io/example/app:1.2.3
''
```

Compared to IFD (`runCommand` + `builtins.fromJSON`), this keeps `nix flake show` / eval-only CI from spawning builders just to read config. Compared to a C++ plugin, the `.wasm` can live in-tree or as a `file` flake input — flakes stay self-contained.

Case 3: A minimal custom flake schema

From the Determinate 3.23.1 manual — teach `nix flake show` about `nixosConfigurations`-shaped output without special-casing in the CLI:

```
# flake.nix - schemas output (excerpt)
{
  outputs = { self, ... }: {
    nixosConfigurations.edge = { /* … */ };

    schemas.nixosConfigurations = {
      version = 1;
      doc = ''
        The `nixosConfigurations` flake output defines NixOS system configurations.
      '';
      inventory = output: {
        children = builtins.mapAttrs (configName: machine: {
          what = "NixOS configuration";
          derivationAttrPath = [ "config" "system" "build" "toplevel" ];
```

```
        }) output;
      };
    };
  };
}
```

Leaf nodes may also set `evalChecks` (attrset of booleans for `nix flake check`), `shortDescription`, `forSystems`, `isLegacy`, and `isFlakeCheck`. Non-leaves nest via `children`.

Case 4: Roles — teach `nix build` about a custom output

```
schemas.labImages = {
  version = 1;
  doc = "Disk images for the teaching lab.";
  roles.nix-build = { };   # nix build / nix shell / nix develop may consume these
  inventory = output: {
    children = builtins.mapAttrs (_: drv: {
      what = "lab disk image";
      derivationAttrPath = [ ];  # leaf is the derivation itself
    }) output;
  };
};
```

Stock roles include `nix-build`, `nix-run`, `nix-develop`, `nix-fmt`, `nix-bundler`, `nix-search`, `nix-template`. Tools may define more (for example a future `nixos-configuration` role instead of hard-coding `nixosConfigurations`).

Case 5: Exporting schemas for other flakes

Determinate's later schema revision distinguishes schemas you consume locally from schemas you **export** for others to import (`exportedSchemas`). Prefer documenting shared schemas in one flake and importing them, rather than copy-pasting `inventory` functions into every repo. Check the current flake schemas protocol and the "interface change" blog post before pinning a team convention — the attribute names moved once already.

The trap

1. **Expecting `builtins.wasm` on upstream Nix.** As of this writing it is Determinate-first (with an upstream PR in flight). Flakes that *require* it are not portable to stock `nix` without a feature flag and binary that provide the primop.
2. **Using Wasm for impure work.** No RNG, no ambient FS writes: that is the point. If you punch host functions that read the ambient machine, you break evaluation reproducibility.
3. **Chatty per-item Wasm calls.** Instance setup dominates tiny calls. Parse a whole file once; do not call Wasm once per list element in a tight eval loop.

4. **IFD-building the `.wasm` on every eval.** Fetch or vendor the module; compiling it via IFD reintroduces the cost Wasm was meant to avoid.
5. **Schemas that lie.** An `inventory` that always returns `evalChecks = { ok = true; }` makes `nix flake check` green while hiding broken outputs. Put real predicates in `evalChecks`.
6. **Confusing schemas with overlays.** Overlays rewrite `pkgs`. Schemas only describe flake *outputs* for tooling. Different layer (see the overlays chapter for the fixed-point story).

The boring rule

On Determinate Nix: enable `wasm-builtin` only when evaluation-time pure compute beats IFD; vendor or `file`-input the `.wasm`; keep flakes that must stay upstream-Nix-portable free of `builtins.wasm`. Add a `schemas` (and, when sharing, `exportedSchemas`) entry for every custom output you expect humans and CI to `show`/`check`, with an honest `inventory`.

Try this

1. On a Determinate host, `nix eval --extra-experimental-features wasm-builtin --expr 'builtins.wasm'` and run the Mandelbrot example from `nix-wasm-rust`.
2. Add a `schemas` stub for one custom output in a lab flake; compare `nix flake show` before and after.
3. Time an IFD YAML parse vs a Wasm `fromYAML` on a multi-thousand-line file; keep the winner, delete the loser.

Sources

- WebAssembly | Determinate Nix docs (Wasm since 3.17.0; docs updated 2026-09-21)
- A new chapter for the Nix language, courtesy of WebAssembly — Eelco Dolstra, 2026-03-05
- Flake Schemas — Determinate Nix 3.23.1 Reference Manual
- Extensible flake outputs with flake schemas; An interface change for flake schemas
- NixOS 26.05 release notes / channel (stable as of 2026-10-09)

A Multi-Arch Flake Template that Stays Boring

A Multi-Arch Flake Template that Stays Boring

Thesis

A flake that only works on `x86_64-linux` surprises Darwin CI. List **systems**, loop with `genAttrs`, keep packages and shells inside. Not evaluated here (`nix` missing).

Mental model

```
forEach system → packages.${system} / devShells.${system}
```

Worked examples

```
{
  description = "boring multi-arch template";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f {
        pkgs = import nixpkgs { inherit system; };
      });
    in {
      packages = forAllSystems ({ pkgs }: { default = pkgs.hello; });
      devShells = forAllSystems ({ pkgs }: {
        default = pkgs.mkShell { packages = [ pkgs.hello ]; };
      });
    };
}
```

Re-verify current nixpkgs pins when you use this.

Trap

Hard-coding `system = "x86_64-linux"` at the top level.

Boring rule

Enumerate systems once; put every package and shell inside the loop.

Try this

1. Add formatter/checks per system.
2. Compare `nix flake init` templates.
3. Grow into flake-parts `perSystem` if needed.

Sources

- Nix flakes manual

Services and Security

Services and security patterns

Services and security patterns

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

Baseline: NixOS / nixpkgs **26.05**, flakes, Home Manager as NixOS module (from the previous part), deliberate firewall and secrets discipline.

What this part is for

Layout and Home Manager made the repo operable. This part makes a **service slice** operable **and** defensible:

Theme	Chapters
Secrets threat model	The secrets problem
Primary secrets tooling	sops-nix
Comparison alternative	agenix
Host attack surface	Host hardening patterns
Network policy	Firewall discipline
HTTPS front door	TLS and reverse-proxy
Durable state	A data service pattern (PostgreSQL)
Isolation literacy	NixOS containers
OCI industry path	Podman and Docker patterns
See what is running	Observability patterns
Optional k8s awareness	k3s (optional)

After this part you should be able to run a small app path—proxy → backend → database—with encrypted secrets, documented ports, and a monitoring or logs pattern—without secrets in the Nix store.

Prerequisites

- Working flake host with `hosts/` · `modules/` · `homes/` layout (or equivalent discipline)
- Lock policy and selective updates
- Comfort with `systemd` units and `journalctl`
- Firewall basics from the host networking chapter
- No reliance on plaintext passwords in the flake

Chapter map

Chapter	Focus	Outcome
The secrets problem	Store vault	Audit + remove anti-patterns
sops-nix	Encrypt in git; path at runtime	End-to-end secret consumer
agenix (comparison)	Same job, different UX	House default declared
Host hardening	SSH, sudo, surface	Hardening module + recovery notes
Firewall discipline	Deny by default	`docs/ports.md` living matrix
TLS front door	Caddy or Nginx + ACME/lab TLS	443 front; localhost backends
Data service	PostgreSQL pattern	State, auth, backup story
NixOS containers	nspawn `containers.*`	Isolation decision record
Podman/Docker	Declarative OCI	Localhost publish + comparison
Observability	Metrics and/or logs	Runbook or Prometheus path
k3s optional	Skip or lite install	Explicit decision artifact

Mental model — service slice

```
clients

:443 reverse proxy (TLS)    modules/services/proxy-*.nix

127.0.0.1:app               backend unit / OCI / nspawn

127.0.0.1:5432              PostgreSQL (localhost)
```

```
secrets via sops-nix        /run/secrets/... paths only
firewall                    22, 80, 443 (documented)
observability               journald and/or Prometheus localhost
```

Layer	Default posture
Edge	TLS terminate; minimal open ports
App	Bind localhost; no public DB
Secrets	Ciphertext in git; decrypt at activation
State	Mutable dirs outside store; backup plan
Observe	Localhost metrics or excellent logs

Suggested module layout (extends previous part)

```
modules/
  common/
    hardening/
    firewall.nix
    sops.nix
  services/
    proxy-caddy.nix   # or proxy-nginx.nix
    postgres.nix
    observability.nix
    podman.nix
    oci-hello.nix
secrets/
  lab.yaml            # sops-encrypted
.sops.yaml
docs/
  secrets.md
  ports.md
  proxy.md
  data-service.md
  isolation.md
  observability.md
```

How to work this part

1. **Threat model and cleanup** before tooling

2. **sops-nix end-to-end** before real service passwords
3. **agenix once** for comparison; pick a house default
4. **Hardening + firewall** before public exposure
5. **Proxy + one data service** as the spine
6. **Containers literacy** without containerizing everything
7. **Observability** so the slice is inspectable
8. **k3s** only if justified—or skip with a written decision

Prefer small commits: secrets infra, then hardening, then each service.

Cross-cutting conventions (26.05)

Topic	Convention
Secrets primary	**sops-nix** (agenix compared once)
Secret consumption	Runtime **paths**, never Nix strings of secret values
DB / metrics / backends	Listen on **127.0.0.1** unless documented otherwise
Proxy	**One of** Caddy or Nginx
OCI	Prefer **Podman**; localhost port publishes
Ports	Living `docs/ports.md`
ACME tokens / DNS API	sops only

Security non-goals (this part)

Non-goal	Note
Full CIS benchmark automation	Patterns over checklists
Multi-AZ HA databases	Single-node honesty
Corporate SSO platform	Mentions only
“Containers make it secure”	Kernel shared; be honest
Required Kubernetes	Optional chapter

Skills checklist (end of part)

- ☐ Can explain why store vault in one minute
- ☐ sops-nix: encrypt, activate, consume path
- ☐ House secrets default documented; dual-stack cleaned up
- ☐ Hardening module: no password SSH, recovery path written
- ☐ Firewall on; every open port in `docs/ports.md`
- ☐ TLS (or documented lab alternative) reverse proxy
- ☐ PostgreSQL (or one DB) with backup/restore story
- ☐ Isolation decision (native vs nspawn vs OCI)
- ☐ One declarative OCI container **or** justified refusal
- ☐ Observability path: metrics and/or logs runbook
- ☐ k3s decision recorded (skip is first-class)

Reading strategies

Mode	Approach
Spine first	secrets → sops → harden → firewall → TLS → postgres → observability
Compare tools	sops vs agenix; nspawn vs OCI; metrics vs logs
Audit existing host	secrets problem + firewall + ports.md first
Resource-constrained	skip k3s; logs-first observability; native modules over k8s

---

Relationship to other parts

Earlier	This part uses
Host networking / firewall intro	Deepens discipline
Systemd via Nix	Service units, ordering
Flake layout / HM / locks	Where modules and inputs live
Later: ops, packaging, capstone	Deploy, tests, multi-node, wipe proofs

---

Lab host assumptions

Examples use host `lab`, user `alice`, system `x86_64-linux`, domains like `app.example.invalid` (replace with real DNS or lab TLS). **Never** paste production secrets into the book examples or your git history.

Next parts

Packaging, **ops and fleet**, and **capstone** assume you can keep secrets out of the store, open only justified ports, and rebuild a service slice from the flake. If this part is thin, those parts become theater.

The secrets problem

The secrets problem

Goal: Internalize why **secrets-as-Nix-strings** fail on NixOS: the store is a world-readable object database. Find and remove secret anti-patterns from your lab flake before wiring sops-nix.

> ! Important
>
> **Never put live passwords, API tokens, or private keys in `flake.nix`, modules, or HM `text` files.** Examples below use obvious placeholders only.

Why this chapter exists

Real services need credentials. The naive NixOS instinct:

```
# CATASTROPHIC PATTERN - do not use
services.someApp.password = "hunter2";
```

or:

```
# ALSO BAD
environment.variables.API_TOKEN = "sk-live-...";
```

Those values enter the **Nix store**, typically **world-readable**, often copied to binary caches if you ever push paths, and immortalized in old generations until GC.

This chapter is pure threat model + cleanup. Implementation tools: **sops-nix** (next) and **agenix** (comparison).

Theory 1 — The store is not a vault

Recall the store and closures chapter: store paths are immutable objects. Permissions are generally:

```
/nix/store/...  readable by all users on the machine
```

So:

Belief	Reality
“It’s in my private git repo”	Still world-readable on every machine that builds/activates
“Only root rebuilds”	Any local user can `cat` the store path
“I’ll delete the file later”	Old generations may retain the path
“I’ll push to my cache”	You may distribute the secret to the cache’s consumers
“HM mode 600 symlink”	Target in store is still world-readable

Evaluation vs runtime

Nix evaluation **sees strings**. If a module option takes a string password, that string becomes part of a derivation input or config file in the store.

```
Nix string secret
  → appears in .drv / generated config
  → realized under /nix/store
  → readable by local users (and caches if pushed)
```

Good secrets appear as **paths to runtime files** (tmpfs `/run/secrets/...`) that are not Nix-string content of the secret itself.

Theory 2 — Common anti-patterns catalog

Pattern	Why it fails
Password options in nixpkgs modules set to plaintext	Store + process listing risk
`writeText "secret" "value"`	Explicit store file with secret
HM `home.file` with tokens	Store-backed home links
Embedding TLS private keys in module `text`	Private key in store
`builtins.readFile ./secret.key` into config	File content becomes string in eval graph
Committing `.env` with production secrets	Git history forever
`environment.sessionVariables` with tokens	Often ends up visible
Embedding secrets in `shellHook`	Project flakes leak too

`builtins.readFile` trap

```
# STILL BAD if the result is interpolated into world-readable config
services.foo.token = builtins.readFile ./token.txt;
```

Reading a non-store private file at eval time can also break purity and CI; when it "works," it often still **copies content into the store** via generated configs.

Theory 3 — What "good" looks like (preview)

Approach	Idea
sops-nix (primary this book)	Encrypted files in git; decrypt on activation to root-owned paths
agenix	Age-encrypted secrets; similar activation story
External secret inject	Cloud SM, Vault—out of band at runtime
Password hashes only	`hashedPassword` for users—not reversible plaintext
SSH keys	Public in git; private only on disk/agent
LoadCredential / EnvironmentFile	systemd reads path at start

Good secrets:

1. Appear in git **only encrypted** (or not at all)
2. Land on disk at **activation/runtime** with tight permissions
3. Are **not** Nix string inputs to public derivations

```
# GOOD shape (conceptual)
# service reads config.sops.secrets."app/db_password".path
# NEVER builtins.readFile that path into another derivation
```

Theory 4 — Generations and GC

Even after you remove a secret from the flake:

- Previous system generations may still reference store paths containing it

- `nix-collect-garbage` eventually drops unreachable paths—but not if a generation roots them

Incident response sketch:

1. **Rotate** the credential (assume compromise)
2. Remove from config
3. Delete old generations once new system confirmed
4. GC
5. If paths were pushed to a cache: treat cache as compromised for that secret

```
sudo nix-env --list-generations -p /nix/var/nix/profiles/system
# sudo nix-collect-garbage -d   # only when you accept dropping old gens
```

Theory 5 — Multi-user threat on a "lab laptop"

If your lab has only one human user, risk feels abstract—until:

- You enable a network service
- A second local account exists
- A container escapes
- You copy a store path off-machine
- A backup tarball includes `/nix/store`

Design as if **local multi-user** and **backup tarballs of /nix/store** exist.

Theory 6 — Distinguishing config from secret

Config (OK in git)	Secret (not OK plaintext)
Port numbers	DB passwords
Public hostnames	API tokens
Public SSH keys	Private SSH keys
Non-secret feature flags	Session signing keys
ACME email (usually)	ACME account private material (tool-managed)
Package names	Registry pull tokens

Gray areas (document decisions): internal-only hostnames, non-production lab passwords that still teach bad habits—**prefer good habits even in lab**.

Theory 7 — Process listings and logs

Even outside the store:

Risk	Mitigation
`ps` shows command-line secrets	Prefer env files / credentials; never `cmd --password x`
journald logs echo secrets	Never `echo "$TOKEN"` in units
Core dumps	Limit where appropriate; avoid secrets in argv

Theory 8 — Git history is forever (enough)

Force-push rewriting history is incomplete if:

- Forks and clones exist
- CI caches retained logs
- Chat paste included the secret

Rotation is mandatory after exposure. History scrubbing is extra, not a substitute.

Worked example — Finding secrets with search

```
cd /path/to/your-flake
rg -n -i 'password\s*=\s*\"' .
rg -n -i 'token\s*=\s*\"' .
rg -n -i 'api[_-]?key' .
rg -n -i 'BEGIN OPENSSH PRIVATE KEY|BEGIN RSA PRIVATE KEY|BEGIN SECRET' .
rg -n 'hashedPassword|initialHashedPassword' .
rg -n -i 'secret|passwd|private_key' --glob '*.nix' .
```

`hashedPassword` is OK if it is a **hash**, not a plaintext password field.

Worked example — Safe store readability demo

```
# DO NOT MERGE - educational package in flake outputs temporarily
# packages.x86_64-linux.fakeSecretDemo =
#   pkgs.writeText "fake-secret" "PLACEHOLDER_ONLY";
```

```
# nix build .#fakeSecretDemo
# cat result
# ls -la result
```

Then **delete** the demo. Point: content is just a file in the store.

Exercises

Exercise 1 — Threat model paragraph

Write 10–15 lines:

- Who can read `/nix/store` on your lab host?
- Who can read your git remote?
- What is the worst secret currently at risk (even if lab-only)?

Exercise 2 — Audit the flake for anti-patterns

Run searches from the worked example. Open every hit. Classify:

Hit	OK / Bad / Investigate

Fix **Bad** by removing or replacing with placeholders and a comment `// TODO sops`.

Exercise 3 — Prove store readability (safe demo)

Build a **fake** `writeText` or temporary package; show world-readable content; delete.

Exercise 4 — User password hygiene check

If any user still has plaintext password configuration, migrate to:

- SSH keys only for remote, and/or
- `hashedPassword` generated via `mkpasswd`

```
mkpasswd -m yescrypt
```

Do not commit the interactive password itself.

Exercise 5 — Remove anti-patterns and rebuild

```
sudo nixos-rebuild switch --flake .#lab
```

Confirm services you care about still run **without** embedding secrets yet.

Exercise 6 — Incident checklist card

Write a 5-step card for "secret committed to git":

1. Rotate
2. Rewrite history or accept leak + rotate (policy)
3. Remove from flake
4. Drop generations / GC
5. Audit caches

Exercise 7 — HM file audit

```
rg -n 'home\.file|xdg\.configFile' homes/ || true
```

Ensure no token-like `.text` values.

Exercise 8 — Environment variable audit

```
rg -n 'environment\.(variables|sessionVariables)|Environment\s*=' .
```

Exercise 9 — Generation awareness

```
sudo nix-env --list-generations -p /nix/var/nix/profiles/system | tail
```

Note how many generations might hold old secrets if any ever existed.

Exercise 10 — Backup story note

If you back up the host, does the backup include store paths? Write one sentence on secret risk in backups.

Exercise 11 — Public vs private key check

Confirm only **public** SSH keys live in the flake; private keys only on disk/agent.

Exercise 12 — Document TODO list for sops

List every secret the next service chapters will need (DB password, optional Grafana admin, DNS token). No values—names only.

Exercise 13 — Peer review simulation

Read your threat model as if onboarding a coworker. Fix any "lab only, nobody cares" handwaves.

Exercise 14 — CI log risk

If CI evaluates the flake, confirm logs would not print secrets (they should not be in eval). Note any impure readFile risks.

Common gotchas

Symptom / mistake	What to do
"Lab only, nobody cares"	Habits transfer to prod; practice correctly
Secret in old generation	Rotate + GC old gens
HM file "mode 600" but store-backed	Still world-readable store object
`readFile` feels private	Content still often ends up in store configs
Binary cache push after leak	Rotate; scrub if possible; assume leak
Focusing on encryption theater without rotation	Rotation is mandatory after exposure
hashedPassword confused with password	Hash only in git
Secrets in commit messages	Rotate; amend is not enough if pushed
Demo package left merged	Remove educational writeText
Thinking sops alone without path discipline	Still don't readFile into store

---

Checkpoint

- ☐ Can explain why store vault in one minute
- ☐ Repo audited with secret heuristics
- ☐ Anti-patterns removed or ticketed for sops-nix
- ☐ Fake store demo understood and deleted
- ☐ User access uses keys/hashes, not plaintext passwords in git
- ☐ Incident rotation checklist written
- ☐ Named list of future secrets (no values)
- ☐ HM and environment audits clean

Journal (optional)

Write three personal gotchas before continuing.

---

sops-nix

sops-nix

Goal: Wire **sops-nix** end-to-end: encrypt a secret in-repo, decrypt at activation, consume it from a NixOS service or timed script—**without** plaintext secrets in the Nix store evaluation graph.

> **Note**
>
> **House primary:** sops-nix. The agenix chapter compares agenix; do not run both as permanent dual systems unless you have a reason.

> **Important**
>
> Use **placeholder values** and lab-only credentials. Never paste production tokens into examples or commits.

Why this chapter exists

The secrets problem chapter established the threat model. This chapter installs the **default tooling** for this book:

Property	sops-nix approach
At rest in git	Encrypted YAML/JSON/ENV (SOPS)
Encryption keys	Age and/or GPG; **age recommended**
On host	Decrypted at activation to a path like `/run/secrets/...`
Nix modules	Reference secret **paths**, not secret **strings**

Theory 1 — Components

Piece	Role
SOPS	Encrypts/decrypts structured files
age	Modern encryption tool; identities (private) + recipients (public)
sops-nix	NixOS/HM modules: decrypt during activation; systemd credentials patterns

Piece	Role
`.sops.yaml`	Creation rules: which keys encrypt which files

Flow

```
edit plaintext locally (editor via `sops file`)
    → encrypted file committed
    → nixos-rebuild
    → sops-nix activation decrypts with host/user age key
    → secret file on tmpfs/run with tight perms
    → service reads path
```

Theory 2 — Age keys (host-centric pattern)

Common NixOS pattern: **host age key** on the machine, public key in `.sops.yaml` creation rules.

Generate key (on the lab host; protect private key):

```
nix shell nixpkgs#age -c age-keygen -o "$HOME/.config/sops/age/keys.txt"
# private key file - NEVER commit
# public key printed as age1...
```

For unattended host decryption, sops-nix docs describe placing keys where the activation can read them (often `/var/lib/sops-nix/key.txt` or configured path)—**follow current sops-nix README for 26.05-compatible setup**. Private key stays on host disk with root-only permissions—not in git.

> **Tip**
>
> Also encrypt to **your admin age key** so you can decrypt on a workstation for editing. Creation rules can list multiple recipients.

Key backup

Losing all private keys without other recipients = **cannot decrypt**. Keep admin recipient + offline backup of identities.

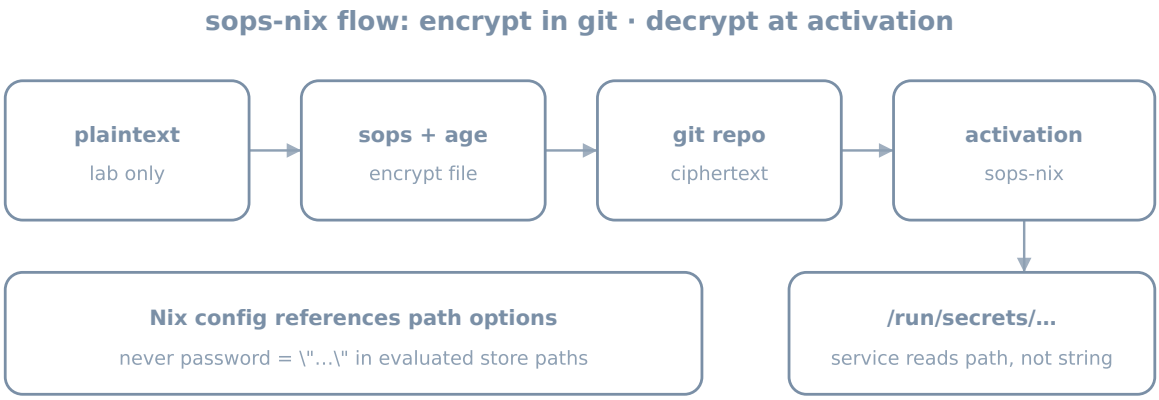


Figure 1: sops-nix encrypt decrypt flow

Theory 3 — Flake input

```
inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  sops-nix.url = "github:Mic92/sops-nix";
  sops-nix.inputs.nixpkgs.follows = "nixpkgs";
};

# modules list includes:
# inputs.sops-nix.nixosModules.sops
```

```
# flake.nix sketch
outputs = { self, nixpkgs, sops-nix, ... }@inputs: {
  nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
    system = "x86_64-linux";
    specialArgs = { inherit inputs; };
    modules = [
      ./hosts/lab
      sops-nix.nixosModules.sops
      # or import modules/common/sops.nix that uses inputs
    ];
  };
};
```

Update only this input when bumping:

```
nix flake update sops-nix
```

Theory 4 — Minimal NixOS sops config

```
# modules/common/sops.nix
{ config, ... }:
{
  sops.defaultSopsFile = ../../secrets/lab.yaml;
  # sops.age.keyFile = "/var/lib/sops-nix/key.txt"; # per upstream guidance
  # sops.age.sshKeyPaths = [ ... ]; # alternative: derive from SSH host key

  sops.secrets."lab/demo_token" = {
    mode = "0400";
    owner = "root";
    # restartUnits = [ "some-service.service" ];
  };
}
```

Consume **path**:

```
systemd.services.sops-demo-check = {
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    Type = "oneshot";
    RemainAfterExit = true;
    ExecStart = "/bin/sh -c 'test -f /run/secrets/lab/demo_token && echo ok > /var/lib/sops-
  };
};
```

Exact secret path layout depends on sops-nix version and naming—inspect after activation:

```
sudo ls -la /run/secrets
```

Never do `builtins.readFile config.sops.secrets."lab/demo_token".path` into another world-readable derivation if that re-embeds content. Prefer systemd `LoadCredential`, `EnvironmentFile` pointing at the secret path, or scripts that read at runtime.

Theory 5 — Creating encrypted files

`.sops.yaml` example shape:

```
keys:
  - &admin age1AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
  - &host age1BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
creation_rules:
```

```
  - path_regex: secrets/.*\.yaml$
    key_groups:
      - age:
          - *admin
          - *host
```

Edit:

```
nix shell nixpkgs#sops nixpkgs#age -c sops secrets/lab.yaml
```

Example structure **before** encryption (editor buffer only):

```
lab:
  demo_token: PLACEHOLDER_LAB_TOKEN_ROTATE_ME
```

After save, file on disk is encrypted ciphertext suitable for git.

Adding a secret later

1. `sops secrets/lab.yaml`
2. Add key under structured YAML
3. Declare `sops.secrets."…"` in Nix
4. Rebuild
5. Point service at `.path`

Theory 6 — HM secrets (awareness)

sops-nix also integrates with Home Manager for user secrets. Prefer **system** secrets for host services first; user secrets when a user-only app needs them.

Theory 7 — Bootstrapping chicken-and-egg

First host key setup is the awkward part:

1. Generate age key on host
2. Put **public** key in `.sops.yaml`
3. Encrypt secrets to that recipient
4. Place **private** key where sops-nix expects
5. Rebuild

Document bootstrap in `docs/secrets.md`.

SSH host key method (awareness)

Some setups convert or use SSH host keys as age identities. Follow upstream docs if you choose that—document which method you used.

Theory 8 — systemd integration patterns

Pattern	Use
`config.sops.secrets.*.path` in `ExecStart` script	Read file at runtime
`LoadCredential=`	systemd credentials API
`EnvironmentFile=`	If file is `KEY=value` format
`restartUnits`	Restart service when secret changes

```
sops.secrets."lab/demo_token" = {
  mode = "0400";
  restartUnits = [ "lab-token-touch.service" ];
};
```

Theory 9 — Multi-host recipients

```
creation_rules:
  - path_regex: secrets/lab\.yaml$
    key_groups:
      - age: [*admin, *host_lab]
  - path_regex: secrets/prod\.yaml$
    key_groups:
      - age: [*admin, *host_prod]
```

Do not encrypt prod secrets only to a laptop key if the server must decrypt unattended.

Worked example — Secret for a custom script service

```
{ config, pkgs, ... }:
{
  sops.secrets."lab/demo_token" = {
    mode = "0400";
    owner = "root";
  };

  systemd.services.lab-token-touch = {
    description = "Demonstrate runtime read of sops secret path";
    wantedBy = [ "multi-user.target" ];
    serviceConfig = {
      Type = "oneshot";
      RemainAfterExit = true;
      ExecStart = pkgs.writeShellScript "lab-token-touch" ''
        set -euo pipefail
        token_path="${config.sops.secrets."lab/demo_token".path}"
        test -r "$token_path"
        # Do NOT echo the token to logs
        wc -c < "$token_path" > /var/lib/lab-token-bytes
      '';
    };
  };
}
```

Note: `writeShellScript` embeds the **path string**, not the secret content—good.

Exercises

Exercise 1 — Add input and module

```
# edit flake inputs; nix flake update sops-nix
# import sops-nix.nixosModules.sops
nix flake metadata
```

Exercise 2 — Age keys and `.sops.yaml`

1. Generate admin age identity (laptop)
2. Generate or configure host identity per sops-nix docs
3. Write `.sops.yaml` with both public keys
4. Confirm private keys are gitignored

```
keys.txt
*.agekey
```

Exercise 3 — First encrypted secret file

```
mkdir -p secrets
# create encrypted secrets/lab.yaml via sops
git add secrets/lab.yaml .sops.yaml
# ensure plaintext never staged
```

Exercise 4 — Declare `sops.secrets` and rebuild

```
sudo nixos-rebuild switch --flake .#lab
sudo ls -la /run/secrets
sudo wc -c /run/secrets/...   # do not cat into scrollback you paste publicly
```

Exercise 5 — Runtime consumer

Add the oneshot that **reads path** without logging secret contents. Prove `/var/lib/lab-token-bytes` or similar.

Exercise 6 — Failure drill

1. Break key access (rename key file temporarily)
2. Rebuild or restart activation path
3. Observe failure mode
4. Restore key

Journal: how the host fails closed.

Exercise 7 — Docs

`docs/secrets.md`:

- Key locations
- How to add a secret
- How to add a recipient
- Rotation sketch

Exercise 8 — Selective lock update

```
nix flake update sops-nix
git diff flake.lock | head -80
```

Exercise 9 — Permission check

```
sudo ls -la /run/secrets
# confirm mode 0400 (or intended) and owner
```

Exercise 10 — Second secret

Add another key in the YAML; declare second `sops.secrets` entry; rebuild; list paths.

Exercise 11 — No readFile audit

```
rg -n 'builtins.readFile.*sops|sops\.secrets' .
```

Ensure no anti-pattern re-embedding.

Exercise 12 — Journal hygiene

```
journalctl -u lab-token-touch.service -b --no-pager | tail
```

Confirm secret values never appear.

Exercise 13 — Recipient add rehearsal (docs)

Write steps to add a second admin age public key without performing a risky prod op.

Exercise 14 — Bootstrap checklist in git

Commit `docs/secrets.md` with bootstrap steps a future you can follow on a blank host.

Common gotchas

Symptom / mistake	What to do
Cannot decrypt on host	Host public key not in creation rules / wrong private key path
Cannot edit on laptop	Admin age key missing from recipients
Secret path empty	Activation order; sops config; check journal
Plaintext committed	Rotate; purge if needed; fix
`readFile` secret into store	Use runtime path only
Logged secret via `echo $TOKEN`	Never log secrets
GPG-only complexity	Prefer age for new labs
Private key in git	Rotate immediately; purge
Wrong defaultSopsFile path	Fix relative path from module file
Dual sops+agenix forever	Pick default after comparison chapter
follows missing on sops-nix	Align nixpkgs

Checkpoint

- ☐ sops-nix input pinned with follows
- ☐ `.sops.yaml` with host + admin recipients
- ☐ Encrypted secret file in git
- ☐ Private keys not in git
- ☐ `sops.secrets` declared; appears under `/run/secrets`
- ☐ Service/script consumes **path** only
- ☐ Bootstrap docs written
- ☐ Failure-closed drill done once

Journal (optional)

Write three personal gotchas before continuing.

---

agenix (comparison)

agenix (comparison)

Goal: Implement the **same lab secret** once with **agenix**, compare trade-offs to sops-nix, and declare a **house default** (this book: **sops-nix** primary).

> **Note**
>
> Comparison chapter dual-stack forever. After the lab, remove or disable the non-default tool so you maintain one system.

Why this chapter exists

Both sops-nix and agenix solve "encrypted secrets in git + decrypt on activation." Teams pick based on:

Factor	Why it matters
File format UX	YAML multi-secret files vs one age file per secret
Tooling	`sops` editor vs `agenix` CLI
Ecosystem fit	Docs you read; modules you copy
Complexity	Bootstrap and key distribution

You should **not** choose from social-media vibes alone. Touch both once.

Theory 1 — agenix mental model

agenix (and compatible community usage) roughly:

Concept	Meaning
`secrets.nix`	Maps secret files → public keys allowed to decrypt
`*.age` files	Ciphertext in repo
`age` identities	Private keys on machines/admins
NixOS module	`age.secrets.<name>.file = ./secret.age;` → runtime path

Flow

```
echo 'PLACEHOLDER' | agenix -e secret.age
git add secret.age secrets.nix
nixos-rebuild → decrypt to /run/agenix/... (path scheme per version)
service reads path
```

Theory 2 — Flake input sketch

```
inputs = {
  nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  agenix.url = "github:ryantm/agenix";
  agenix.inputs.nixpkgs.follows = "nixpkgs";
};

# modules list:
# inputs.agenix.nixosModules.default
```

Pin and `follows` like any other input (lock discipline).

```
nix flake update agenix
```

Theory 3 — secrets.nix shape

```
# secrets/secrets.nix - public keys only
let
  # replace with your real age public keys
  admin = "age1AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA";
  host  = "age1BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB";
in
{
  "lab-demo-token.age".publicKeys = [ admin host ];
}
```

No private keys in this file.

Identity reuse

Reuse **public** age keys from the sops-nix setup when possible so you are not juggling unrelated key material during comparison.

Theory 4 — NixOS declaration

```
{ config, inputs, ... }:
{
  imports = [ inputs.agenix.nixosModules.default ];

  age.secrets.lab-demo-token = {
    file = ../../secrets/lab-demo-token.age;
    # mode / owner as needed
  };

  # config.age.secrets.lab-demo-token.path → runtime path
}
```

Consumer pattern matches sops-nix: **path at runtime**, never embed content in derivations.

```
systemd.services.agenix-demo = {
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    Type = "oneshot";
    RemainAfterExit = true;
    ExecStart = "/bin/sh -c 'test -r ${config.age.secrets.lab-demo-token.path}'";
  };
};
```

Theory 5 — Side-by-side comparison

Dimension	**sops-nix**	**agenix**
Multi-key structured file	Natural (YAML tree)	Usually one file per secret
Edit UX	`sops file` in-place decrypt editor	`agenix -e`
Creation rules	`.sops.yaml`	`secrets.nix` publicKeys
HM integration	Strong story	Available patterns
Learning transfer	SOPS used outside Nix too	Age-centric, Nix-flavored

Dimension	**sops-nix**	**agenix**
House default (this book)	**Primary**	Compare once

When agenix might still win for you

- You want the simplest possible "one secret, one file" model
- Your team already standardized on agenix
- You dislike SOPS YAML tooling

When sops-nix wins for this curriculum

- Multiple secrets per environment file
- Same SOPS skills in non-Nix contexts
- Book consistency for later examples

Theory 6 — Dual-running risks

Running both modules on one host:

- Two key bootstraps
- Two runtime directories
- Cognitive load on incident response

Acceptable **for one comparison lab**. Not acceptable as permanent undefined state—pick a default before the next service chapters pile on.

Theory 7 — Declaring house default

Write in `docs/secrets.md`:

```
## House default

**sops-nix** for all new secrets.

agenix: lab comparison only; module removed after comparison
(or: we chose agenix because … - only if you override the book).
```

If you override the book, do so **explicitly** so later sops-shaped examples are translated consciously.

Theory 8 — Re-encryption and rotation

Event	sops-nix	agenix
Add recipient	update `.sops.yaml`, re-encrypt	update `secrets.nix`, re-encrypt
Rotate value	edit via tool	re-encrypt file
Rotate keys	re-encrypt all files	re-encrypt all `.age`

Practice rotation on a **placeholder** secret, not production.

Theory 9 — Path API discipline (both tools)

```
# GOOD: path reference
environment.etc."demo-path".text = config.age.secrets.lab-demo-token.path;
# (still careful: only the path string)

# BAD: content into store
# environment.etc."demo".text = builtins.readFile config.age.secrets....path;
```

Same rule as sops: runtime consumers, not eval-time content embedding.

Worked example — Same placeholder secret both ways

Tool	Artifact
sops	`secrets/lab.yaml` key `lab.demo_token`
agenix	`secrets/lab-demo-token.age`

Same placeholder value during local edit; different ciphertext formats in git. Runtime paths differ; consumer scripts should use module-provided `.path`.

Exercises

Exercise 1 — Add agenix input

```
# flake input + lock update agenix only
nix flake update agenix
nix flake metadata
```

Exercise 2 — Public keys and secrets.nix

Reuse **public** age keys from sops-nix if possible. Write `secrets/secrets.nix` with publicKeys only.

Exercise 3 — Encrypt one age secret

Using current agenix/age CLI docs for your pin:

```
nix shell github:ryantm/agenix#agenix nixpkgs#age
# agenix -e secrets/lab-demo-token.age
```

Put only a **lab placeholder** value inside.

Exercise 4 — NixOS module + rebuild

Declare `age.secrets.…`, rebuild, list runtime secrets dir for agenix:

```
sudo nixos-rebuild switch --flake .#lab
sudo ls -la /run/agenix 2>/dev/null || sudo ls -la /run/secrets 2>/dev/null || true
```

Confirm path exists; do not paste secret contents into notes.

Exercise 5 — Consumer parity

Point a oneshot at `config.age.secrets.lab-demo-token.path` similar to sops-nix. Prove readable by the intended user.

Exercise 6 — Comparison write-up

One page in the journal:

1. Bootstrap friction: sops vs agenix
2. Day-2 secret add friction (ongoing maintenance)
3. Multi-secret ergonomics
4. Failure modes observed
5. **Decision:** house default = …

Exercise 7 — Decommission the non-default

If house default is sops-nix:

1. Remove agenix module from active host imports
2. Optionally leave encrypted `.age` file as historical artifact or delete
3. Remove agenix input **or** keep input unused (prefer remove to simplify lock)
4. Rebuild clean

Opposite steps if you chose agenix (then translate later chapters).

Exercise 8 — docs/secrets.md house default section

Commit the decision explicitly.

Exercise 9 — follows check

Confirm agenix input used `follows` while present; clean lock after removal.

Exercise 10 — Incident note

Add one paragraph: how rotation differs if dual tools had been left enabled.

Exercise 11 — Team rationale

Write two sentences you would tell a teammate who prefers the other tool.

Exercise 12 — Path-only audit

```
rg -n 'age\.secrets|sops\.secrets' modules/ || true
```

Ensure consumers use `.path`.

Exercise 13 — Rebuild with single stack

```
sudo nixos-rebuild switch --flake .#lab
nix flake metadata  # no unexpected dual inputs if removed
```

Exercise 14 — Future secret list

Map upcoming secrets (DB, proxy DNS token, Grafana) to the **chosen** tool only.

Common gotchas

Symptom / mistake	What to do
Private key in secrets.nix	Remove; rotate
Wrong public key list	Re-encrypt with correct recipients
Both tools half-configured	Finish decommission exercise
Assuming identical paths	Always use `.path` from the module
Editing ciphertext as plaintext	Use tool entrypoints only
Forgetting follows on agenix input	Fix lock discipline
Leaving dual forever “for flexibility”	Pick one; document override
Comparison without write-up	Decision is the deliverable
Production secrets in comparison	Placeholders only

Checkpoint

- ☐ agenix encrypt + decrypt-on-activate worked once
- ☐ Comparison table completed in your words
- ☐ House default declared in `docs/secrets.md`
- ☐ Non-default tool removed or clearly disabled
- ☐ Still no plaintext secrets in git
- ☐ Rebuild succeeds with chosen stack only
- ☐ Future secrets mapped to one tool
- ☐ follows/lock cleaned up

Journal (optional)

Write three personal gotchas before continuing.

Choosing between sops-nix and agenix

Choosing between sops-nix and agenix

Thesis

Both **sops-nix** and **agenix** decrypt secrets at activation. Pick **whichever your team already encrypts with**. Comparison after `02-sops-nix` and `03-agenix`. `nix` is not on this writer host — re-verify flake input versions locally.

Mental model

```
sops-nix: structured SOPS → age/GPG/KMS
agenix:   age files → small module surface
```

Concern	sops-nix	agenix
Format	SOPS YAML/JSON	age files
Recipients	age, GPG, KMS	age / SSH keys
Fit	existing SOPS shops	age-only minimal setups

Worked examples

Checklist: existing SOPS+KMS → sops-nix; age-only + small audit surface → agenix; need one system of record → never both.

Trap

Running both modules on one host. Two decryption paths, two failure modes.

Boring rule

One secret system per fleet; document recipients in the flake README.

Try this

1. Encrypt one secret both ways in a scratch flake.
2. Read the sibling how-to chapters side by side.
3. Check current flake input versions before copying snippets.

Sources

- https://github.com/Mic92/sops-nix
- https://github.com/ryantm/agenix

HashiCorp Vault Integration

HashiCorp Vault Integration

Goal: Integrate HashiCorp Vault with NixOS to inject dynamic secrets, short-lived database credentials, and rotated TLS certificates into systemd services at runtime without ever exposing plaintext secrets to `/nix/store` or git history.

> **Note**
>
> This chapter complements `02-sops-nix.qmd` and `03-agenix.qmd`. While SOPS and Agenix excel at repository-committed static secrets (bootstrapping SSH keys, WireGuard preshared keys), enterprise platforms require centralized audit logs, ephemeral credentials, and automated rotation provided by HashiCorp Vault.

The Enterprise Secret Boundary

In small homelabs or single-node servers, static encrypted secrets stored in git (`sops-nix` or `agenix`) are often sufficient. However, at enterprise scale, static repository secrets hit severe operational boundaries:

1. **No Ephemeral Leases:** A compromised database password remains valid indefinitely until an engineer manually re-encrypts the secret file and triggers a full fleet rebuild.
2. **Absence of Centralized Auditing:** Statically decrypted secrets at boot cannot produce audit logs answering *"Which service queried the Stripe API key at 14:02:11 UTC?"*
3. **No Automatic PKI Rotation:** Internal mTLS certificates expire, requiring manual renewals and deployment coordination.
4. **Static Credentials vs Dynamic Credentials:** Modern security compliance (SOC 2, ISO 27001) mandates that applications never share a static master database credential; each service instance must request a unique, short-lived database role that auto-revokes when the container or service dies.

```
Static Secrets (sops-nix / agenix):
[Git Repository]  > Encrypted File  > Host Private Key  > Decrypted into /run/secrets at Boo

Dynamic Secrets (HashiCorp Vault):
[NixOS Host]  > AppRole Auth  > [Vault Server]  > Issues Short-Lived Token (1 hr lease)


                                Generates Unique Database User (auto-expires)
                                Renders into volatile RAM (/run/vault/db.env)
                                Auto-renews & triggers systemd reload on rotation
```

Vault Architecture on NixOS

To maintain pure declarative reproducibility while consuming dynamic secrets, NixOS separates **infrastructure definition** from **runtime secret resolution**:

1. **NixOS declares the system state:** It installs the `vault-agent` daemon, provisions systemd units, defines authentication parameters, and sets up template rendering directives.
2. **Vault Agent manages runtime tokens:** Running as an unprivileged service, `vault-agent` authenticates to the remote Vault cluster using the **AppRole** method (or AWS/GCP instance identity), auto-renews leases, and writes decrypted secrets to an in-memory `tmpfs` directory (`/run/vault/`).
3. **systemd isolates service access:** Target application services read secrets either via systemd `LoadCredential` directives or via protected environment files in `/run/vault/`.

Declarative Vault Agent Configuration

NixOS provides a first-class module for Vault Agent via `services.vault-agent`. Here is a complete production configuration:

```
# modules/security/vault-agent.nix
{ config, pkgs, ... }: {
  # 1. Ensure Vault client tools are available
  environment.systemPackages = [ pkgs.vault ];

  # 2. Vault Agent systemd service
  services.vault-agent = {
    enable = true;
    settings = {
      vault = {
        address = "https://vault.internal.corp:8200";
        ca_cert = "/etc/ssl/certs/internal-ca.pem";
      };

      auto_auth = {
        method = {
          type = "approle";
          config = {
            role_id_file_path = "/var/lib/vault/role-id";
            secret_id_file_path = "/var/lib/vault/secret-id"; # Bootstrap secret ID
            remove_secret_id_file_after_reading = false;
          };
```

```
      };

      sink = {
        type = "file";
        config = {
          path = "/run/vault/vault-token";
          mode = 384; # 0600
        };
      };
    };

    # Templates render dynamic secrets to volatile tmpfs
    template = [
      {
        destination = "/run/vault/app-db.env";
        perms = "0400";
        # Template written in Consul/Vault template syntax
        contents = ''
          {{ with secret "database/creds/backend-role" }}
          DATABASE_USER="{{ .Data.username }}"
          DATABASE_PASSWORD="{{ .Data.password }}"
          DATABASE_URL="postgres://{{ .Data.username }}:{{ .Data.password }}@db.internal.c
          {{ end }}
        '';
        # Command executed by systemd when secrets rotate
        command = "${pkgs.systemd}/bin/systemctl reload-or-restart backend-api.service";
      }
    ];
  };
};

# 3. Create volatile runtime directory in tmpfs (RAM only)
systemd.services.vault-agent.serviceConfig = {
  RuntimeDirectory = "vault";
  RuntimeDirectoryMode = "0750";
  StateDirectory = "vault";
  StateDirectoryMode = "0700";
  User = "vault";
  Group = "vault";
  Restart = "always";
  RestartSec = "5s";
};

# Define system user and group for vault agent
users.users.vault = {
  isSystemUser = true;
```

```
    group = "vault";
  };
  users.groups.vault = {};
}
```

Consuming Dynamic Secrets in systemd Services

Once Vault Agent renders dynamic secrets into `/run/vault/app-db.env`, the consumer service must read them securely without creating world-readable files:

Pattern A: `EnvironmentFile` with File ACLs

```
# modules/services/backend-api.nix
{ config, pkgs, ... }: {
  # Add backend user to vault group to read rendered secrets
  users.users.backend = {
    isSystemUser = true;
    group = "backend";
    extraGroups = [ "vault" ];
  };
  users.groups.backend = {};

  systemd.services.backend-api = {
    description = "Production Backend API";
    after = [ "network.target" "vault-agent.service" ];
    wants = [ "vault-agent.service" ];
    wantedBy = [ "multi-user.target" ];

    serviceConfig = {
      Type = "notify";
      User = "backend";
      Group = "backend";
      ExecStart = "${pkgs.backend-api}/bin/server";

      # Load dynamic credentials rendered by Vault Agent
      EnvironmentFile = "/run/vault/app-db.env";

      # Security Hardening
      ProtectSystem = "strict";
      ProtectHome = true;
      PrivateTmp = true;
      NoNewPrivileges = true;
```

```
      Restart = "on-failure";
      RestartSec = "3s";
    };
  };
}
```

Pattern B: Modern systemd `LoadCredential`

For maximum isolation, use systemd's built-in credential management. systemd mounts credentials into a private, encrypted directory per service under `$CREDENTIALS_DIRECTORY`:

```
systemd.services.backend-api.serviceConfig = {
  # systemd safely provisions the credential into the service namespace
  LoadCredential = [
    "db_password:/run/vault/db_password.txt"
    "api_token:/run/vault/api_token.txt"
  ];
  ExecStart = "${pkgs.backend-api}/bin/server --credentials-dir=\${CREDENTIALS_DIRECTORY}";
};
```

Architectural Comparison: SOPS vs Agenix vs Vault

Feature	`sops-nix`	`agenix`	HashiCorp Vault
Storage Location	Git repository (Encrypted)	Git repository (Encrypted)	Centralized External Cluster
Decryption Key	Host Age/SSH Key or KMS	Host SSH Key	AppRole / Cloud Instance Identity
Decryption Timing	NixOS System Activation	NixOS System Activation	Continuous runtime daemon
Secret Lifetime	Static (until manual re-key)	Static (until manual re-key)	**Dynamic leases (minutes to hours)**
Audit Trails	Git commit logs only	Git commit logs only	**Comprehensive real-time API logs**
Auto-Rotation	Manual rebuild required	Manual rebuild required	**Automated zero-downtime reloads**
Primary Use Case	Node bootstrapping, static tokens	Node bootstrapping, lightweight setups	**Multi-tenant platforms, enterprise compliance**

Best Practice: The Hybrid Vault + SOPS Pattern

In production architectures, combine SOPS and Vault: 1. Use `sops-nix` to store the **bootstrap AppRole Secret ID** in git. 2. When the NixOS machine boots, `sops-nix` decrypts the Secret ID into `/var/lib/vault/secret-id`. 3. `vault-agent` reads the Secret ID, authenticates to HashiCorp Vault, and manages all dynamic database passwords, cloud API tokens, and TLS certificates dynamically.

Checkpoint

- ☐ Vault Agent is declared as a managed systemd service.
- ☐ Secrets are written only to volatile `tmpfs` (`/run/vault/`) and never touch `/nix/store` or persistent disks.
- ☐ Application services depend on `vault-agent.service` and read credentials via `EnvironmentFile` or `LoadCredential`.
- ☐ Dynamic database leases auto-renew and trigger automatic service reloads on expiration.

Host hardening patterns

Host hardening patterns

Goal: Apply a **small, documented hardening module** for a lab server: SSH posture, sudo policy, disable unused services, and careful sysctl/kernel defaults—without cargo-culting every CIS line item.

> ⚠ Warning
>
> Hardening can lock you out of a remote VM. Keep a **console/VNC path** or test with `nixos-rebuild test` / boot generation rollback ready.

Why this chapter exists

Secrets tooling protects credentials. Hardening reduces **attack surface** and enforces least privilege. Production-shaped labs should not run with:

- Password SSH
- Root SSH login
- Unnecessary network services
- Empty sudo policies that surprise you later

This chapter builds a **module you can import** on server roles—not a security theater checklist of 200 toggles.

Theory 1 — Threat model for this lab

Assume:

Asset	Threat
SSH on a public IP	Password spray, key theft
Local users	Accidental privilege escalation
Services you enable next	Extra open ports
Laptop multi-boot lab	Lower remote risk, still practice

Non-goals: full SELinux policy authoring, corporate MDM, compliance certifications.

Role split

Role	Hardening intensity
`server`	Strict SSH, fewer conveniences
`workstation`	Keys still; keep desktop UX

Theory 2 — SSH baseline (NixOS)

```nix
# modules/common/hardening/ssh.nix
{
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "no";
      X11Forwarding = false;
      # MaxAuthTries = 4;
      # AllowUsers = [ "alice" ];
    };
  };
}
```

Setting	Intent
No passwords	Keys only
No root login	Use `sudo` from admin user
No X11 forward	Reduce attack surface on servers

Ensure your user has `openssh.authorizedKeys.keys` **before** applying password-off if you ever had passwords.

Optional allowlists

```nix
services.openssh.settings.AllowUsers = [ "alice" ];
# services.openssh.listenAddresses = [ { addr = "0.0.0.0"; port = 22; } ];
```

Theory 3 — sudo and wheel

```
# modules/common/hardening/sudo.nix
{
  security.sudo = {
    enable = true;
    wheelNeedsPassword = true; # or false for lab convenience - document choice
    execWheelOnly = true;
  };
  # users.users.alice.extraGroups = [ "wheel" ];
}
```

Policy	Trade-off
`wheelNeedsPassword = true`	Safer; needs password/TTY
`= false`	Convenient lab; weaker if key stolen + local

Prefer **true** on anything network-exposed.

Theory 4 — Disable what you do not use

Examples (only if present/enabled):

```
{
  services.avahi.enable = false;
  services.printing.enable = false;
  hardware.bluetooth.enable = false;
}
```

Do not randomly disable things you depend on (NetworkManager on a laptop role). Use **roles**:

```
# modules/roles/server.nix
{
  imports = [ ../common/hardening ];
}
```

```
# modules/roles/workstation.nix
{
  # lighter hardening; keep user conveniences
}
```

Theory 5 — Kernel / sysctl care

NixOS exposes many `boot.kernel.sysctl` knobs. Apply **few**, understand each:

```
{
  boot.kernel.sysctl = {
    "net.ipv4.conf.all.rp_filter" = 1;
    "net.ipv4.tcp_syncookies" = 1;
    # Do not paste a 50-line blog dump without reading
  };
}
```

Kernel package pinning and hardened profiles exist in the ecosystem; treat full "hardened kernel" as optional advanced work—not required for this part.

Theory 6 — Automatic updates? (judgment)

Unattended upgrades on NixOS are a product decision:

- Flake pins mean "latest" is not automatic unless you automate lock bumps + rebuilds
- Auto-reboot policies can surprise

For this book: **manual, deliberate updates** (lock discipline) unless you later build CI-driven update PRs.

Theory 7 — Module shape

```
modules/common/hardening/
  default.nix    # imports ssh.nix sudo.nix misc.nix
  ssh.nix
  sudo.nix
  misc.nix
```

```
# default.nix
{
  imports = [ ./ssh.nix ./sudo.nix ./misc.nix ];
}
```

Import from server role or common—**document** if workstations share it.

Theory 8 — Assertions as guardrails

```
{ config, lib, ... }:
{
  assertions = [
    {
      assertion = config.services.openssh.settings.PasswordAuthentication == false;
      message = "PasswordAuthentication must be false on this host role";
    }
  ];
}
```

Assertions fail evaluation early—better than silent drift.

Theory 9 — Safe apply order

1. Confirm console access
2. Confirm admin key SSH works
3. `nixos-rebuild test` **or** `boot`
4. Open new SSH session before closing old
5. Only then `switch` if using switch and tests pass

```
sudo nixos-rebuild test --flake .#lab
# new ssh session:
ssh lab
sudo -v
```

Worked example — Recovery card

```
# Recovery if SSH rejects

1. Hypervisor / cloud serial console
2. Login locally
3. sudo nixos-rebuild switch --rollback
   # or pick previous generation at bootloader
4. Fix flake; re-apply carefully
```

Keep this in `docs/hardening.md`.

Exercises

Exercise 1 — Inventory surface

```
ss -tulpn | sort
systemctl list-unit-files --state=enabled | head -80
```

Journal: what is listening, what you do not recognize.

Exercise 2 — Write hardening module

Create `modules/common/hardening/` with SSH + sudo + 2–3 sensible disables for **your** role.

Exercise 3 — Apply carefully

```
sudo nixos-rebuild test --flake .#lab
# new ssh session
ssh lab
sudo -v
```

Then `switch` if satisfied.

Exercise 4 — Lockout recovery drill (document)

Write exact steps to recover if SSH rejects you. Do not skip writing this.

Exercise 5 — Assertions (optional polish)

Add an assertion for PasswordAuthentication false (or your critical policy).

Exercise 6 — Diff and explain

```
ss -tulpn
```

Explain each remaining listener in one line (sshd, etc.).

Exercise 7 — Key-before-password-off proof

Confirm `authorizedKeys` present for your admin user before disabling passwords.

Exercise 8 — wheel policy document

Write one sentence in `docs/hardening.md` for `wheelNeedsPassword` choice.

Exercise 9 — Role wiring

Import hardening from `modules/roles/server.nix` (or equivalent). Rebuild.

Exercise 10 — PermitRootLogin check

```
sudo sshd -T | grep -i -E 'passwordauthentication|permitrootlogin'
```

Exercise 11 — Disable list rationale

For each disabled service, one-line reason in docs.

Exercise 12 — Sysctl minimalism

If you added sysctls, write what each does. Remove any you cannot explain.

Exercise 13 — Workstation exception note

If you also use this flake on a laptop role, document what differs.

Exercise 14 — Generation rollback rehearsal (console available)

From console, practice listing generations; do not necessarily roll back if healthy.

Symptom / mistake	What to do

Common gotchas

Symptom / mistake	What to do
Locked out via SSH	Console + previous generation
PasswordAuth off before keys	Fix via console; add keys
Hardening module on laptop breaks UX	Split server/workstation roles
Blind sysctl paste	Remove unknowns
`PermitRootLogin yes` “for debug” left on	Remove before network exposure
No recovery notes	Write recovery card before remote travel
test vs switch confusion	Prefer test when changing SSH
AllowUsers typo lockout	Console fix
Firewall not yet disciplined	Next chapters—but do not open random ports here

---

Checkpoint

- ☐ Hardening module exists and is imported on the lab role
- ☐ Password SSH off; root SSH off
- ☐ sudo policy explicit
- ☐ Unnecessary services disabled with reasons
- ☐ Recovery path documented
- ☐ Rebuild + new SSH session verified
- ☐ Listeners explained
- ☐ Assertions or checklist for critical SSH settings

Journal (optional)

Write three personal gotchas before continuing.

Firewall discipline

Firewall discipline

Goal: Treat the NixOS firewall as a **deny-by-default** policy: open only ports you can name, tie openings to services, and audit with tools—not hope.

Why this chapter exists

Host networking introduced firewall basics. Services (proxy, DB, metrics) tempt:

```
networking.firewall.enable = false; # "lab"
```

or:

```
networking.firewall.allowedTCPPorts = [ 22 80 443 5432 3000 9090 9100 /* … */ ];
```

Without a story. This chapter is **discipline**: every open port has an owner service and a reason.

Theory 1 — NixOS firewall model

With `networking.firewall.enable = true` (default on many installs):

Mechanism	Role
nftables/iptables backend	Implementation (prefer not hand-editing)
`allowedTCPPorts` / `UDP`	Host-wide opens
`allowedTCPPortRanges`	Ranges when unavoidable
Interface-specific options	Limit exposure to LAN interfaces
Service modules	Often open ports via `openFirewall` options

Prefer service options

```
services.openssh.enable = true;
# many services offer:
# services.foo.openFirewall = true;
```

That keeps port + service definition adjacent. Global lists become a last resort.

Theory 2 — Deny by default

```
{
  networking.firewall = {
    enable = true;
    allowPing = true; # or false on hardened internet hosts
    allowedTCPPorts = [ ];
    allowedUDPPorts = [ ];
  };
}
```

Then add ports **only** with services. SSH often opened by `services.openssh` + related settings depending on config—**verify** rather than assume.

Check:

```
sudo nixos-option networking.firewall.allowedTCPPorts
sudo nft list ruleset 2>/dev/null | head -100
# or iptables-save depending on backend
```

Theory 3 — Per-service matrix (living document)

Maintain `docs/ports.md`:

Port	Proto	Service	Source scope	Module
22	TCP	sshd	admin IPs / world (lab)	hardening/ssh
443	TCP	caddy/nginx	world	reverse-proxy
5432	TCP	postgres	**localhost only**	data service

If Postgres is localhost-only, **do not** open 5432 on the firewall.

Theory 4 — Localhost vs LAN vs world

Scope	Pattern
Localhost only	Bind service to `127.0.0.1`; no firewall open
LAN only	Firewall interface restrictions / not exposing WAN NIC
World	80/443 typically; auth/TLS required

```
# postgres listen addresses - keep lab default local
services.postgresql.enable = true;
# ensure listen_addresses = 'localhost' for lab default
```

Exact options depend on module—read search.nixos.org for your pin.

Theory 5 — Temporary debug holes

Bad habit: leave `allowedTCPPorts = [ 3000 ]` forever for a dev server.

Better:

- SSH tunnel: `ssh -L 3000:127.0.0.1:3000 lab`
- Or short-lived config on a branch
- Or VPN/Tailscale patterns (document if you use them)

Theory 6 — Interaction with containers (preview)

Containers and port publishes can bypass your mental model if you open host ports carelessly. Treat published ports as first-class firewall entries in `docs/ports.md`.

Theory 7 — Interfaces and trusted nets

```
networking.firewall = {
  enable = true;
  # trustedInterfaces = [ "tailscale0" ]; # only if real
  # interfaces."eth0".allowedTCPPorts = [ 22 ]; # verify option shape on 26.05
};
```

Prefer **least exposure**: admin SSH on a management interface if you have one; never open DB ports “because the app is in the same VPC narrative” without proof.

Cloud security groups

NixOS firewall is **not** a substitute for cloud SG/NSG rules—and vice versa. Document both layers in `docs/ports.md` when the lab is a VPS.

Theory 8 — Connection tracking

Allowing TCP 22 does not require opening random high ports for SSH. Related/established traffic is handled by the firewall state machine. Do not "open 1024–65535 for good measure."

Theory 9 — Co-locate opens with services

```
# modules/services/proxy.nix
{
  networking.firewall.allowedTCPPorts = [ 80 443 ];
  services.caddy.enable = true;
  # ...
}
```

Deleting the service module should remove the hole—review PRs for orphaned ports.

Worked example — Tight SSH + HTTPS owned by proxy

```
# modules/common/firewall.nix
{ lib, ... }:
{
  networking.firewall = {
    enable = true;
    allowPing = true;
    allowedTCPPorts = [
      # 80 443 added with the proxy module, not here forever empty-handed
    ];
  };
}
```

When proxy lands, open 80/443 in the **proxy** module file.

Worked example — Service openFirewall pattern

```
services.nginx.enable = true;
# services.nginx... openFirewall-style options - check 26.05 module

# Fallback explicit ownership in the same module file:
# networking.firewall.allowedTCPPorts = [ 80 443 ];
```

Exercises

Exercise 1 — Audit current opens

```
ss -tulpn
sudo nft list ruleset 2>/dev/null | head -150
```

Fill the ports table for **current** state.

Exercise 2 — Enable firewall explicitly

Ensure `networking.firewall.enable = true`. Rebuild carefully (console ready).

```
sudo nixos-rebuild test --flake .#lab
```

Confirm SSH still works from a **new** session.

Exercise 3 — Remove unjustified ports

Delete any "temporary" ports. Rebuild. Re-audit `ss` and ruleset.

Exercise 4 — Service-tied open (stub)

```
# modules/services/proxy-firewall.nix
# TLS chapter will open 80/443 here when proxy is enabled
```

Wire the idea that **proxy owns those ports**.

Exercise 5 — Localhost service experiment

Run a quick local listener **or** use an existing local-only service; confirm it is not reachable from another machine without a firewall hole and public bind.

```
# on lab: nc -l 127.0.0.1 9999
# from another host: should fail unless you opened the port AND bound 0.0.0.0
```

Exercise 6 — ports.md committed

Commit `docs/ports.md` with the matrix. Update as services arrive.

Exercise 7 — Diff firewall after change

```
sudo nft list ruleset > /tmp/fw-after.txt
# compare mentally to docs/ports.md
```

Exercise 8 — Ping policy decision

```
networking.firewall.allowPing = true; # or false
```

Write one sentence: who needs ICMP on this host?

Exercise 9 — SSH tunnel habit

Practice one tunnel instead of opening a port:

```
ssh -L 9090:127.0.0.1:9090 lab
```

Exercise 10 — Cloud SG note

If on a VPS, paste (redacted) SG rules into ports.md as a second table.

Exercise 11 — Orphan port hunt

Find any `allowedTCPPorts` entry without a matching enabled service. Remove or justify.

Exercise 12 — UDP inventory

```
ss -ulpn
```

Document any unexpected UDP listeners.

Exercise 13 — test generation safety

Use `nixos-rebuild test` when tightening rules so reboot returns to previous if you disconnect.

Exercise 14 — Pre-proxy checklist

List ports you expect after TLS + postgres chapters; mark which are localhost-only.

Common gotchas

Symptom / mistake	What to do
Locked out of SSH	Console; previous generation; re-open 22
Firewall on but service bound wrong	Fix listen address
Docker-ish publishes confuse audit	Inventory publish flags (Podman chapter)
`enable = false` "for lab"	Re-enable; use tunnels for debug
Open 5432 to world	Bind localhost; proxy/app only
Ruleset empty surprise	Backend differ; verify with `ss` too
Orphan ports after service delete	Review ports.md + module
Opened high port ranges	Close; use tunnels
Trusted interface too broad	Narrow trust

Checkpoint

- ☐ Firewall enabled
- ☐ Every open port documented in `docs/ports.md`
- ☐ No unjustified temporary holes
- ☐ SSH verified after firewall changes
- ☐ Localhost vs world distinction understood
- ☐ Plan for proxy ports owned by proxy module

- ☐ Ping policy decided
- ☐ Tunnel practiced once

Journal (optional)

Write three personal gotchas before continuing.

TLS and reverse-proxy front door

TLS and reverse-proxy front door

Goal: Put a **TLS reverse proxy** in front of a simple backend using **ACME** (Let's Encrypt or lab alternative)—pick **Caddy or Nginx**, not both, as the house reverse-proxy pattern.

> **Note**
>
> **Public ACME** needs a public DNS name pointing at your lab. If you cannot, use **internal TLS**, **mkcert**, or Caddy/Nginx self-signed lab modes—and document the deviation. The pattern still counts.

Why this chapter exists

Services should not all expose their own TLS stacks. The production pattern:

```
Internet/LAN clients
    → :443 reverse proxy (TLS terminates)
        → 127.0.0.1:app-port (HTTP plain on localhost)
```

One certificate story, one cipher/policy place, one set of firewall holes (80/443).

Theory 1 — Choose one proxy

Proxy	Strengths	Notes
Caddy	Automatic HTTPS UX, simple config	Great default for labs
Nginx	Ubiquitous, explicit	ACME via `security.acme` + nginx module

Pick one for the rest of this part. Write the choice in `docs/proxy.md`.

Do not run both on 443.

Theory 2 — ACME on NixOS

NixOS `security.acme` (used heavily with nginx) or Caddy's built-in ACME:

Concern	Practice
Email	Real email for Let's Encrypt notices
Domains	You control DNS
HTTP-01	Port 80 reachable from internet
DNS-01	Provider token via **sops** (not plaintext)
Staging	Use LE staging while debugging rate limits

Secrets

ACME account keys are managed on disk by the tooling—do not invent plaintext private keys in the flake. DNS provider tokens: **sops-nix**.

Theory 3 — Caddy pattern (NixOS)

```
# modules/services/proxy-caddy.nix
{ pkgs, ... }:
{
  networking.firewall.allowedTCPPorts = [ 80 443 ];

  services.caddy = {
    enable = true;
    virtualHosts."app.example.invalid".extraConfig = ''
      encode gzip
      reverse_proxy 127.0.0.1:8080
    '';
  };
}
```

Caddy often obtains certs automatically when DNS is real. For invalid/lab domains, configure accordingly or use a local-only site block for HTTP lab testing.

Static site alternative:

```
services.caddy.virtualHosts."static.example.invalid".extraConfig = ''
  root * ${pkgs.writeTextDir "index.html" "<h1>lab</h1>"}
  file_server
'';
```

(Adjust to current Caddy module options on 26.05 if attribute names differ—verify with option search.)

Theory 4 — Nginx + ACME pattern

```
# modules/services/proxy-nginx.nix
{
  networking.firewall.allowedTCPPorts = [ 80 443 ];

  security.acme = {
    acceptTerms = true;
    defaults.email = "you@example.invalid";
  };

  services.nginx = {
    enable = true;
    recommendedProxySettings = true;
    recommendedTlsSettings = true;
    virtualHosts."app.example.invalid" = {
      enableACME = true;
      forceSSL = true;
      locations."/" = {
        proxyPass = "http://127.0.0.1:8080";
        proxyWebsockets = true;
      };
    };
  };
}
```

Theory 5 — Backend for the lab

Minimal backend options:

1. **Static files** via the proxy itself
2. **Tiny declarative service**:

```
systemd.services.lab-backend = {
  wantedBy = [ "multi-user.target" ];
  serviceConfig = {
    DynamicUser = true;
```

```
    ExecStart = "${pkgs.python3}/bin/python3 -m http.server 8080 --bind 127.0.0.1";
    WorkingDirectory = "${pkgs.writeTextDir "index.html" "<h1>backend</h1>"}";
  };
};
```

Bind **127.0.0.1** so the firewall need not open 8080.

Theory 6 — Failure modes

Symptom	Checks
ACME fail	DNS A/AAAA, port 80, rate limits, email/terms
502 bad gateway	Backend down; wrong port
Cert works, app not	`proxyPass` URL
Works on host, not remote	Firewall 80/443; cloud SG

```
curl -vI https://app.example.invalid
journalctl -u caddy -b --no-pager | tail -50
# or nginx / acme services
```

Theory 7 — Certificate storage and renewal

ACME clients store account/certs under state dirs (often under `/var/lib`). That state is **not** in the Nix store.

Event	Result
System generation rollback	Software/config may roll back; **certs usually remain**
Wipe `/var`	Certificates gone; re-issue (rate limits!)
Domain change	New cert path; update proxy vhost

Back up ACME state on anything beyond a disposable lab, or accept re-issuance cost.

Theory 8 — HTTP → HTTPS redirect

```
curl -I http://app.example.invalid
curl -I https://app.example.invalid
```

Expect redirect or secure connection—not a permanent plain HTTP production path.

Theory 9 — Internal TLS without public ACME

```
# Conceptual lab path - exact options differ by proxy:
# - generate mkcert/local CA on admin machine
# - place cert/key via sops-nix into /run/secrets
# - point proxy ssl_certificate paths at those files
```

Document production delta: "In prod we would use ACME DNS-01 / HTTP-01; lab uses internal CA."

Worked example — ports.md dual-write

Port	Service
80	ACME HTTP-01 / redirect
443	HTTPS proxy
8080	localhost backend only

`docs/proxy.md` and `docs/ports.md` must agree.

Exercises

Exercise 1 — Pick proxy and document

Write `docs/proxy.md` with Caddy **or** Nginx and domain plan (public / LAN / self-signed).

Exercise 2 — Localhost backend

Add declarative backend on `127.0.0.1:8080`. Prove:

```
curl -sS http://127.0.0.1:8080/ | head
```

From another machine, connection should fail if not firewalled open and not public-bound.

Exercise 3 — Enable proxy module

Import proxy module; open 80/443; rebuild.

```
sudo nixos-rebuild switch --flake .#lab
```

Exercise 4 — TLS path

If public DNS: complete ACME; `curl -I https://your.domain`.

If not: configure self-signed or internal CA approach; `curl -k` for lab proof; document how production would differ.

Exercise 5 — Headers and hygiene (light)

Enable recommended TLS/proxy settings (nginx flags above, or Caddy defaults). Confirm you do not open backend port publicly.

Exercise 6 — Break/fix

Stop backend; observe 502; start backend; recover. Journal the unit names.

Exercise 7 — ports.md + proxy.md dual-write

Ensure both docs agree on engine, public ports, backend, TLS mode.

Exercise 8 — Certificate expiry awareness

```
echo | openssl s_client -connect 127.0.0.1:443 -servername app.example.invalid 2>/dev/null |
```

Exercise 9 — ss audit

```
ss -tlnp | grep -E ':80|:443|:8080'
```

Confirm 8080 is 127.0.0.1 only.

Exercise 10 — ACME journal

```
journalctl -u acme-app.example.invalid.service -b --no-pager 2>/dev/null | tail -40 || true
journalctl -u caddy -b --no-pager | tail -40
```

Exercise 11 — Staging note

If you hit LE rate limits, document staging usage for next attempt.

Exercise 12 — WebSocket flag (if needed)

If your app needs websockets, confirm proxy config (`proxyWebsockets` or Caddy defaults).

Exercise 13 — Single proxy enforcement

```
# ensure only one of caddy/nginx enabled
systemctl is-active caddy nginx 2>/dev/null || true
```

Exercise 14 — Secret path for DNS-01 (plan)

If using DNS-01, name the sops secret key in docs (no value).

Exercise 15 — Smoke from remote

From another machine (or phone on data): hit `https://…` if public. Record result.

Symptom / mistake	What to do

Common gotchas

Symptom / mistake	What to do
LE rate limited	Use staging; wait; fix DNS first
Both Caddy and Nginx on 443	Pick one
Backend on 0.0.0.0:8080	Bind localhost
Secrets for DNS-01 in git	sops
IPv6 DNS mismatch	Align AAAA or disable
Forgot port 80 for HTTP-01	Open 80 or use DNS-01
502 ignored	Fix backend unit first
Certs wiped with `/var`	Backup ACME state
docs disagree	Dual-write exercise

Checkpoint

- ☐ One proxy chosen and documented
- ☐ Backend on localhost
- ☐ 80/443 in firewall intentionally
- ☐ HTTPS or documented lab TLS alternative works
- ☐ `docs/ports.md` updated
- ☐ 502 break/fix once
- ☐ 8080 not world-open
- ☐ Cert dates inspected once

Journal (optional)

Write three personal gotchas before continuing.

A data service pattern

A data service pattern

Goal: Run **one** data service—**PostgreSQL** by default—declaratively on NixOS: module options, state directories, app role/auth via secrets paths, and a written backup story (even if automation is minimal).

> **!** Important
>
> DB passwords go through **sops-nix paths**, not Nix strings. Examples use placeholders and path references only.

Why this chapter exists

Application servers without state are easy. Real systems need **durable data**:

Concern	NixOS angle
Package + unit	`services.postgresql` module
State location	`services.postgresql.dataDir` / ensure on persistent disk
Auth	peer/scram; password files from secrets
Network exposure	localhost first (firewall discipline)
Backups	Not automatic just because Nix is declarative

Nix rebuilds the **software and config**; it does not magically version your rows.

Theory 1 — One DB only (pattern depth)

Depth > zoo. We use PostgreSQL. If you already run another engine at work, the pattern transfers: module → state dir → credentials → firewall → backups.

Theory 2 — Minimal PostgreSQL module

```
# modules/services/postgres.nix
{ config, pkgs, ... }:
{
  services.postgresql = {
    enable = true;
    package = pkgs.postgresql_16; # pick a version available on 26.05 pin
    ensureDatabases = [ "labapp" ];
    ensureUsers = [
      {
        name = "labapp";
        ensureDBOwnership = true;
      }
    ];
    settings = {
      # listen_addresses = "localhost"; # confirm option name on your version
    };
  };

  # Do NOT open 5432 on the host firewall for world access in the default lab.
}
```

Verify option names against search.nixos.org for 26.05.

Theory 3 — State directories and disk

PostgreSQL data lives **outside** the Nix store (mutable).

Question	Answer you must know
Where is `dataDir`?	Document path
Is it on the disk that survives reinstall?	Ops/impermanence later
What happens on rollback of *system* generation?	Software rolls back; **data may not**

Impermanence hosts need explicit persistence for DB dirs—note the footgun now.

```
sudo -u postgres psql -c 'SHOW data_directory;'
```

Theory 4 — Credentials

Patterns:

1. **Peer auth** for local system users (good for admin)
2. **SCRAM password** for app users—password from sops at runtime
3. **Socket-only** access for local apps

Sketch with sops (illustrative—align with module capabilities):

```
sops.secrets."postgres/labapp_password" = {
  owner = "postgres";
  mode = "0400";
};

# Prefer module-supported passwordFile style options when available.
# If you script role passwords, run a oneshot that reads the secret path
# and does not print it.
```

Never:

```
# BAD
# services.postgresql.ensureUsers = [{ name = "labapp"; password = "secret"; }];
```

Theory 5 — App connection story

App config should receive:

- Host: `127.0.0.1` or socket
- DB name: `labapp`
- User: `labapp`
- Password: from env file populated at activation from sops

Your TLS front door proxy does **not** replace DB auth. Backend talks to DB on localhost; users talk to proxy on 443.

Theory 6 — Backup thought (required cognitive work)

Even a one-node lab needs a plan:

Element	Example
What	`pg_dump` of `labapp`
When	nightly timer (system or user)
Where	`/var/backups/postgres/` on another disk or remote
Encryption	restic/age to remote (secrets via sops)
Restore test	once per quarter / before capstone

```
sudo -u postgres pg_dump labapp > /tmp/labapp-dump.sql
# store off-box in real life; scrub /tmp later
```

Theory 7 — Upgrades

Postgres major upgrades are **not** "change package attr and forget." They need dump/restore or `pg_upgrade` procedures. Pin a major version intentionally; schedule upgrades.

Theory 8 — `ensureDatabases` vs migrations

`ensureDatabases` / `ensureUsers` create **empty** roles/DBs. Application schema is **not** Nix's job by default.

Layer	Owner
Package + unit + listen policy	NixOS module
Roles/DBs existence	ensure* options / oneshots
Schema / migrations	App tooling (flyway, django, …)
Data	Backups

Theory 9 — Socket vs TCP local connections

Postgres often exposes a Unix socket under `/run/postgresql`. Local admin via `sudo -u postgres psql` uses peer auth. Apps may use:

```
host=127.0.0.1 port=5432
# or host=/run/postgresql
```

Pick one, document in `docs/data-service.md`, match app secret env.

Worked example — Service + docs

```
modules/services/postgres.nix
docs/data-service.md
  - version
  - dataDir
  - DB/user names
  - auth method
  - backup sketch
  - restore sketch
```

Worked example — Timer sketch for dumps (optional)

```
{ pkgs, ... }:
{
  systemd.services.labapp-pg-dump = {
    serviceConfig = {
      Type = "oneshot";
      User = "postgres";
      ExecStart = "${pkgs.bash}/bin/bash -c '${pkgs.postgresql}/bin/pg_dump labapp | ${pkgs.
    };
  };
  # + timer + StateDirectory/tmpfiles for /var/backups
}
```

Wire carefully (permissions, disk space). Even a commented sketch in the flake beats “we’ll back up later.”

Exercises

Exercise 1 — Enable PostgreSQL

Add module; rebuild; verify unit:

```
sudo systemctl status postgresql
sudo -u postgres psql -c '\l'
```

Exercise 2 — Database and role

Confirm `labapp` DB and user via `ensure*` options. Connect:

```
sudo -u postgres psql -d labapp -c 'SELECT 1;'
```

Exercise 3 — Wire a secret password path

Create sops secret placeholder for app password. Configure the safest supported method for your module version. Prove app user can authenticate **without** password living in the flake as plaintext.

Exercise 4 — Network posture

```
ss -tulpn | grep 5432 || true
```

Confirm listening on localhost (or document intentional LAN bind). Ensure firewall does not expose 5432 publicly.

Update `docs/ports.md`.

Exercise 5 — Manual dump

```
sudo -u postgres pg_dump labapp | head
```

Write restore commands in `docs/data-service.md`.

Exercise 6 — Failure narrative

Answer:

1. System generation rollback after a bad app deploy—does data revert?
2. Disk failure—what is lost without backups?
3. Secret rotation steps for DB password

Exercise 7 — Schema independence proof

```
sudo -u postgres psql -d labapp -c "CREATE TABLE lab_demo(id int);"
sudo -u postgres psql -d labapp -c '\dt'
sudo nixos-rebuild switch --flake .#lab
sudo -u postgres psql -d labapp -c '\dt'
```

Table should still exist—state generation.

Exercise 8 — Document restore dry-run

```
# restore sketch
# sudo -u postgres psql -d labapp < labapp-dump.sql
```

Exercise 9 — dataDir documentation

```
sudo -u postgres psql -c 'SHOW data_directory;'
```

Record in docs.

Exercise 10 — Peer auth admin path

Document how you admin without password in git (`sudo -u postgres psql`).

Exercise 11 — Package pin note

Write which `postgresql_*` package attr you chose and why you will not casual-bump majors.

Exercise 12 — Backup location permissions

```
sudo mkdir -p /var/backups/postgres
# ownership suitable for dump user
```

Exercise 13 — App connection string template

Write a template with **path** reference to password file—not the password.

Exercise 14 — ports.md final for DB

Confirm 5432 is localhost-only in docs and reality.

Symptom / mistake	What to do

Common gotchas

Symptom / mistake	What to do
Data in store myth	Data is on mutable disk
Open 5432 to world	Bind localhost; close firewall
Password in nix	sops path / peer auth
Major version bump casual	Plan dump/restore
No backup thought	Exercises 5–6 required
Proxying Postgres over HTTP	Wrong layer; don't
ensure* expected to migrate schema	Use app migrations
Rollback "restored" DB	Generations data
Dump committed with secrets	Scrub; don't commit dumps

Checkpoint

- ☐ PostgreSQL enabled declaratively
- ☐ App DB + user exist
- ☐ Auth without plaintext secret in git
- ☐ Localhost exposure default
- ☐ Manual dump performed once
- ☐ Backup/restore story written
- ☐ ports.md updated
- ☐ Schema-vs-generation understood

Journal (optional)

Write three personal gotchas before continuing.

NixOS containers

NixOS containers

Goal: Understand **isolation options** on NixOS—especially declarative **`containers.*` (nspawn-style)** and light OCI awareness—and run **one** declarative container (or clearly document why your lab uses a pure NixOS service instead).

> **Note**
>
> This chapter is **not** "Docker first." Prefer a native NixOS module when it exists. Containers help for upstream images, multi-version isolation, or hostile workloads—not as a default personality. Pin remains **NixOS 26.05**.

Why this chapter exists

Ecosystem pressure says "everything in containers." NixOS offers stronger default stories for many services: one store, one firewall story, one secrets story. Still, you need literacy so you can refuse containers for good reasons—or adopt them deliberately.

Tool	Isolation style
NixOS systemd services	Same host, units, store
`containers.<name>`	systemd-nspawn guest with NixOS config
OCI (Podman/Docker)	Image-based; next chapter
VMs / k3s	Heavier; optional k3s chapter

Theory 1 — When containers help on NixOS

Use container	Prefer native module
Vendor only ships image	PostgreSQL, Caddy, Prometheus in nixpkgs
Need older/newer app than nixpkgs	Package it when feasible
Separate failure domain lightly	One more systemd service is enough
Multi-tenant sketch	Single-user lab
Hostile untrusted binary	Still not a full security boundary—see Theory 4

Decision micro-script

```
Is there a maintained NixOS module?
  yes → use it
  no  → Can I package it?
          yes → package + native service
          no  → OCI (next chapter) or nspawn NixOS guest (this chapter)
```

Theory 2 — `containers.<name>` (NixOS nspawn)

NixOS can define ephemeral/declarative containers. The guest is a **nested NixOS module graph**—not a Dockerfile.

```nix
# modules/services/demo-container.nix
{
  containers.demo = {
    autoStart = true;
    privateNetwork = true;
    hostAddress = "192.168.100.1";
    localAddress = "192.168.100.2";
    config = { config, pkgs, ... }: {
      system.stateVersion = "26.05";
      networking.firewall.allowedTCPPorts = [ 80 ];
      services.nginx = {
        enable = true;
        virtualHosts."localhost" = {
          root = pkgs.writeTextDir "index.html" "<h1>container</h1>";
        };
      };
    };
  };
}
```

Option idea	Meaning
`autoStart`	Start at boot
`privateNetwork`	Separate network namespace
`hostAddress` / `localAddress`	veth pair addressing
`bindMounts`	Host paths into guest
`config`	Nested NixOS module system
`ephemeral`	Lose rootfs state on stop (when supported)

Exact networking options evolve—verify against **26.05** options docs if attributes differ.

Pros / cons

Pros	Cons
Declarative guest config	Nested complexity / longer eval
Shares host store often	Not full VM isolation
Good teaching tool	Debug requires container tooling
Rebuild updates guest like host	Nested `stateVersion` still required

```
sudo machinectl list
sudo nixos-container status demo 2>/dev/null || true
sudo nixos-container root-login demo 2>/dev/null || true
```

Theory 3 — OCI awareness (without next-chapter depth)

OCI images: layers + config + entrypoint. On NixOS you may:

- Pull images with Podman/Docker (next chapter)
- Build images from Nix (`dockerTools`—packaging awareness later)
- Run rootless vs rootful

An **image is not a NixOS module**. It is a blob with its own update story, CVE surface, and entrypoint that may ignore your host hardening.

NixOS guest (`containers.*`)	OCI image
Config is Nix	Config is image + run flags
Updates via host rebuild	Updates via pull/tag discipline
Same module options as host	Ad-hoc env vars / volumes
Fits flake locks naturally	Needs digest pins for honesty

Theory 4 — Security boundaries (honest)

Boundary	Strength
Process on host	Weak isolation
systemd hardening (`ProtectSystem`, …)	Better, still host kernel
nspawn container	Stronger namespaces
OCI with user namespaces	Better if configured; easy to misconfigure

Boundary	Strength
VM	Stronger (hardware virt)
Separate machine	Operational isolation

Do not claim “container = secure” without profile hardening and a patch process for images.

Hardening checklist (guest)

- Minimal packages in guest
- Firewall on guest *and* publish policy on host
- No unnecessary bind mounts of `/` or secret dirs
- No privileged flags unless justified in writing

Theory 5 — State and containers

Pattern	Use
Ephemeral guest	Stateless demos, rebuild-friendly
Bind `/var/lib/app` from host	Persist data; backup host path
Named volume (OCI)	Next chapter territory
Store-only content	Ideal for pure static sites

Mounting database dirs into random containers without care is a good way to corrupt data.

Theory 6 — Relation to proxy / firewall

```
[client] → host:443 (caddy/nginx)
              ↓
         192.168.100.2:80 (container)
```

Every publish updates `docs/ports.md`.

Mistake	Fix
Open guest port on public interface	Proxy only; firewall deny direct
Forget `ports.md`	Fix immediately
Double NAT confusion	Draw the path once

Theory 7 — Rebuild UX and nested eval

```
sudo nixos-rebuild build --flake .#lab -L
```

Friction vs native service	Why
Longer eval	Nested module system
Harder journal navigation	Machine names / nspawn units
Networking options maze	Worth a diagram

Theory 8 — Feature flag for demos

```
{ config, lib, ... }:
let cfg = config.myLab.demoContainer; in {
  options.myLab.demoContainer.enable = lib.mkEnableOption "demo nspawn container";
  config = lib.mkIf cfg.enable {
    containers.demo = { /* ... */ };
  };
}
```

Default off on constrained hosts.

Worked example — Prefer native, demo container optional

Path A (valid): skip long-lived containers; write a decision record "native modules suffice for the service slice."

Path B: run the `containers.demo` nginx guest and curl via host networking setup you chose.

Both paths require **written judgment**. Path A is not a skip of the chapter—it is an evidence-backed refusal.

Exercises

Exercise 1 — Decision record

In `docs/isolation.md`:

```
# Isolation choices

Service slice:
- proxy: native module
- postgres: native module
- container: yes/no because …

Boundary strength needed:
Threat model (1 paragraph):
When we would revisit containers:
```

Exercise 2 — Explore host tools

```
command -v nixos-container || true
command -v machinectl || true
systemctl list-units 'systemd-nspawn@*' 2>/dev/null | head || true
```

Exercise 3 — One declarative container or dry config

Implement `containers.demo` **or** write the full Nix config with enable false if you truly cannot run nspawn. Prefer running when possible.

```
sudo nixos-rebuild switch --flake .#lab
sudo nixos-container list 2>/dev/null || sudo machinectl list
```

Exercise 4 — Reach the guest service

Curl from host to guest IP or port. Document the path in `docs/isolation.md`.

```
curl -sS http://192.168.100.2/ | head
```

Exercise 5 — Compare rebuild UX

Change guest `index.html` content via Nix; rebuild; note friction vs native `services.nginx` on host.

Exercise 6 — When you would use OCI instead

Write three bullets for the Podman chapter: cases where an upstream image beats nspawn NixOS guest.

Exercise 7 — Firewall / ports doc

Update `docs/ports.md` for any published path. If Path A, document that no extra ports were opened.

Exercise 8 — Nested stateVersion

Confirm guest sets `system.stateVersion = "26.05"` (or your real pin).

Exercise 9 — Journal path

```
journalctl -M demo -b --no-pager 2>/dev/null | tail || true
```

Exercise 10 — Bind mount caution note

Write two sentences: what you would never bind-mount into a guest.

Exercise 11 — Disable cleanly

If demo only, disable container; rebuild; confirm resources freed.

Exercise 12 — Eval time measurement

```
time nixos-rebuild build --flake .#lab
```

Note cost of nested modules.

Exercise 13 — Diagram

ASCII diagram of client → proxy → container (or N/A for Path A).

Exercise 14 — Threat model honesty

One paragraph: nspawn is not a VM; when you would use a real VM instead.

Common gotchas

Symptom / mistake	What to do
Nested `stateVersion` missing	Set in guest config
Networking black hole	Start simpler; read options
Assuming Docker API	That's the Podman chapter
Container as default for Postgres	Prefer native module
Forgotten port documentation	Update ports.md
Nested eval errors	`nixos-rebuild build`; read trace
Bind-mounting secrets world-readable	Fix modes; sops on host only
Expecting VM-level isolation	Adjust threat model
Path A without write-up	Decision record required

Checkpoint

- ☐ Isolation decision record written
- ☐ Understand nspawn-style `containers.*` vs OCI
- ☐ One container ran **or** justified skip with config draft
- ☐ Networking/ports documented
- ☐ Comparison note vs native services
- ☐ Preview of OCI use cases written
- ☐ Nested stateVersion correct if enabled
- ☐ Honest security boundary language

Journal (optional)

Write three personal gotchas before continuing.

Podman and Docker patterns

Podman and Docker patterns

Goal: Run **OCI containers** on NixOS via **declarative modules** (prefer **Podman**), understand rootless vs rootful trade-offs, and compare honestly to NixOS nspawn—without turning the host into an imperative `docker run` snowflake.

Why this chapter exists

The previous chapter covered NixOS-native containers. The industry default many colleagues know is **Docker/Podman + images**. On NixOS you should still:

1. Prefer a nixpkgs service module when it exists
2. When you need an image, declare it in the flake/host modules
3. Track ports, volumes, and update cadence

Theory 1 — Podman vs Docker on NixOS

Engine	Notes
Podman	Daemonless roots; good rootless story; often preferred on NixOS labs
Docker	Familiar Compose ecosystem; `virtualisation.docker` module

House suggestion: Podman unless you must match a Docker-only workflow.

```
# modules/services/podman.nix
{
  virtualisation.podman = {
    enable = true;
    dockerCompat = true; # optional: docker CLI alias - document if used
  };
}
```

Docker alternative sketch:

```
{
  virtualisation.docker.enable = true;
  # users.users.alice.extraGroups = [ "docker" ]; # rootful socket risk
}
```

`docker` group membership is effectively **near-root**. Prefer rootless or rootful with care.

Theory 2 — Rootless vs rootful

Mode	Pros	Cons
Rootless	Better default security	Port <1024 hard; storage quirks
Rootful	Simpler networking/ports	Higher blast radius

Rootless Podman for user workloads; rootful only when needed—or put proxy on host and map high ports.

Theory 3 — Declarative containers (OCI)

Common patterns include `virtualisation.oci-containers.containers` (backend docker/podman).

```
# modules/services/oci-hello.nix
{
  virtualisation.oci-containers = {
    backend = "podman";
    containers = {
      hello-lab = {
        image = "docker.io/library/nginx:alpine";
        ports = [ "127.0.0.1:8081:80" ];
        # volumes = [ "/var/lib/hello-lab:/usr/share/nginx/html:ro" ];
        autoStart = true;
      };
    };
  };
}
```

Field	Discipline
`image`	Pin tags carefully; digests better for prod

Field	Discipline
`ports`	Prefer `127.0.0.1:` binds
`volumes`	Host paths must be managed (tmpfiles/persist)
`environment`	**No secrets in plaintext**—use secret files / sops-generated env

Exact option schema: confirm on search.nixos.org for 26.05.

Theory 4 — Images and supply chain (awareness)

Practice	Why
Pin by digest	Tags move
Prefer known registries	Reduce surprise
Update deliberately	Same as flake lock discipline
Scan when available	Optional tooling

Nix can *build* images from packages (`dockerTools`)—packaging mindset later. Here: consume one official image declaratively.

```
# Prefer tags you understand; better: digest pins when available
image = "docker.io/library/nginx:1.27";
# image = "nginx@sha256:…";
```

Floating `:latest` is the OCI equivalent of an unlocked flake input.

Theory 5 — Compare to nspawn

Dimension	nspawn `containers.*`	OCI Podman
Guest config	NixOS modules	Image + env/cmd
Updates	nixpkgs guest	Image pulls
Good for	Nix-first isolation	Vendor images
Store integration	Strong	Looser

Write your own two-sentence preference for the service slice.

Theory 6 — Firewall and proxy integration

```
clients → host :443 Caddy/Nginx → 127.0.0.1:8081 → OCI nginx
```

Do not publish OCI ports on `0.0.0.0` unless `docs/ports.md` says so and auth exists.

```
# proxy snippet idea
# reverse_proxy 127.0.0.1:8081
```

Theory 7 — Stateful volumes

If the container needs state:

1. Create host directory with tmpfiles or activation
2. Bind mount
3. Include in backup story
4. Never put secrets into world-readable volume without mode care

---

Theory 8 — Rootless paths and user namespaces

Rootless Podman stores containers under the user data dir. On NixOS:

Concern	Note
subuid/subgid	User must be mapped
Lingering	User services may need lingering for boot-start
Ports <1024	Often need rootful or redirect

```
podman info | head
id alice  # check subuids if rootless fails
```

Theory 9 — Imperative snowflakes vs flake

```
podman run --rm -p 127.0.0.1:8082:80 docker.io/library/nginx:alpine
# stop/remove; do NOT leave unmanaged containers as the real setup
```

Imperative is fine for five-minute experiments. The rebuildable truth lives in `virtualisation.oci-containers`.

Worked example — Full small stack slice

```
{ ... }:
{
  imports = [ ./podman.nix ];

  virtualisation.oci-containers.backend = "podman";
  virtualisation.oci-containers.containers.webish = {
    image = "docker.io/library/nginx:1.27";
    ports = [ "127.0.0.1:8081:80" ];
    autoStart = true;
  };
}
```

```
curl -sS http://127.0.0.1:8081/ | head
podman ps
```

Exercises

Exercise 1 — Enable Podman (or Docker)

```
# add module, rebuild
podman info 2>/dev/null || docker info 2>/dev/null
```

Document rootless vs rootful choice.

Exercise 2 — Declarative OCI container

Define one container with **localhost** port publish. Rebuild. Verify:

```
podman ps
curl -sS http://127.0.0.1:8081/ | head
```

Exercise 3 — Imperative contrast (then undo)

Short imperative run; stop/remove; journal why the flake definition wins.

Exercise 4 — Wire to proxy (optional but valuable)

Add a reverse_proxy path to the OCI port. Update ports.md.

Exercise 5 — Secret anti-pattern check

Confirm no `environment = { PASSWORD = "..." }` in nix. If the image needs a password, plan sops-backed env file mount.

Exercise 6 — Comparison table filled

Complete nspawn vs OCI table in `docs/isolation.md` with *your* results.

Exercise 7 — Volume inventory

```
podman volume ls 2>/dev/null || docker volume ls 2>/dev/null || true
```

Map each volume to a backup/persist decision.

Exercise 8 — Update story one-pager

Write five lines: how you will refresh images, who tests, whether flake lock/image digest changes ship together.

Exercise 9 — autoStart reboot proof

Reboot or restart the oci unit; confirm container returns.

Exercise 10 — Port conflict check

```
ss -tlnp | grep 8081
```

Avoid clashing with TLS chapter backend on 8080.

Exercise 11 — docker group risk note

If you used Docker + group, write why it is near-root.

Exercise 12 — Image pin upgrade

Change tag deliberately; rebuild; observe pull; document.

Exercise 13 — Disable cleanly

Remove container from config; rebuild; `podman ps` clean.

Exercise 14 — Isolation.md final

State house preference: native / nspawn / OCI for each service in the slice.

Common gotchas

Symptom / mistake	What to do
Port already in use	Conflict with TLS backend—choose free port
Image pull fails	Network/registry; mirror; pin
Rootless port 80 bind fails	Use high ports + host proxy
`docker` group casual	Avoid; understand privilege
State lost on recreate	Bind volume on host
Forgot autoStart	Container missing after reboot
Secrets in environment=	sops file mounts
`:latest` forever	Pin tag/digest
Imperative left running	Remove snowflake

Checkpoint

- ☐ Podman or Docker enabled via NixOS module
- ☐ One OCI container declarative in flake
- ☐ Localhost publish; ports documented
- ☐ Imperative snowflake cleaned up
- ☐ Isolation docs updated vs nspawn
- ☐ No plaintext secrets in container env

- ☐ Image update story written
- ☐ autoStart verified or documented

Journal (optional)

Write three personal gotchas before continuing.

Observability patterns

Observability patterns

Goal: Add an **observability pattern** for the lab slice—either **Prometheus + Grafana** (or Prometheus alone) **or** a deliberate **logs-first** minimal stack—so one service is inspectable without SSH archaeology alone.

Why this chapter exists

Services without observability fail quietly:

- Disk fills
- Proxy 502s for hours
- Postgres connections exhaust
- ACME renewals fail

You do not need a full SRE platform. You need a **repeatable pattern**: what you measure, where it lives, who can access it, and how it is firewalled.

Theory 1 — Three pillars (light)

Pillar	Lab approach
Logs	journald (`journalctl`) first-class
Metrics	Prometheus node exporter + app exporters optional
Traces	Optional later; not required

Pick a **primary**: metrics stack **or** logs discipline. Doing both shallowly is fine; doing neither is not.

Theory 2 — Logs-first pattern (valid)

If resources are tight:

```
journalctl -u caddy -b --no-pager | tail
journalctl -u postgresql -b --no-pager | tail
journalctl -p err -b --no-pager | tail -50
```

Declarative helpers:

```
{
  services.journald.extraConfig = ''
    SystemMaxUse=500M
  '';
}
```

Document a **debug runbook** for proxy, DB, and sops activation units. That can satisfy the pattern if you explicitly choose logs-first in `docs/observability.md`.

Theory 3 — Prometheus pattern

```
# modules/services/prometheus.nix
{
  services.prometheus = {
    enable = true;
    port = 9090;
    listenAddress = "127.0.0.1";
    scrapeConfigs = [
      {
        job_name = "node";
        static_configs = [{
          targets = [ "127.0.0.1:9100" ];
        }];
      }
    ];
  };

  services.prometheus.exporters.node = {
    enable = true;
    port = 9100;
    listenAddress = "127.0.0.1";
    openFirewall = false;
  };
}
```

Verify option names for 26.05. Access via SSH tunnel:

```
ssh -L 9090:127.0.0.1:9090 lab
# browse localhost:9090
```

Or reverse-proxy with **authentication**—never raw public Prometheus without auth.

Theory 4 — Grafana (optional layer)

```
{
  services.grafana = {
    enable = true;
    settings = {
      server = {
        http_addr = "127.0.0.1";
        http_port = 3000;
      };
    };
  };
}
```

Initial admin password: use sops / first-boot procedures—**not** plaintext in git. Prefer password-file options when available.

Theory 5 — What to scrape for *your* slice

Target	Why
node exporter	Host health
Caddy/Nginx metrics	If exporter/module available
Postgres exporter	Optional; more moving parts
Blackbox / probes	Optional

Minimum bar: node exporter + Prometheus **or** excellent journald runbooks + one automated log check.

Sample queries:

```
up
node_memory_MemAvailable_bytes
node_load1
```

Theory 6 — Firewall and multi-tenant risk

Metrics endpoints often expose sensitive operational detail. Defaults:

- Listen on `127.0.0.1`
- SSH tunnel or authenticated proxy
- Update `docs/ports.md`

Theory 7 — Alerting (awareness only)

Alertmanager is real production work. Optional: record "what would page me." Do not block on paging infrastructure.

Theory 8 — Retention and disk

Metrics and journal logs fill disks.

```
services.journald.extraConfig = ''
  SystemMaxUse=500M
'';
```

Prometheus local TSDB also needs a size budget. A lab that fills the disk mid-capstone is a self-own.

```
journalctl --disk-usage
du -sh /var/lib/prometheus2 2>/dev/null || du -sh /var/lib/prometheus 2>/dev/null || true
```

Theory 9 — Authn in front of Grafana

Never expose Grafana `3000` to the world bare. Patterns:

1. Localhost + SSH tunnel
2. Reverse proxy + SSO/basic auth
3. VPN-only interface

Worked example — Minimal metrics module set

```
modules/services/observability.nix
  - prometheus listen localhost
  - node exporter localhost
docs/observability.md
  - how to tunnel
  - example PromQL: up, node_load1
  - log commands for proxy/db
```

Worked example — Runbook skeleton

```
# docs/runbooks/service-down.md

## Proxy 502
journalctl -u caddy -b --no-pager | tail -100
systemctl status lab-backend

## Postgres
journalctl -u postgresql -b --no-pager | tail -100
sudo -u postgres psql -c 'SELECT 1;'

## Secrets activation
journalctl -b --no-pager | rg -i sops | tail
```

Exercises

Exercise 1 — Choose path

Write top of `docs/observability.md`:

```
Primary path: metrics | logs-first | both
Rationale: …
```

Exercise 2A — Metrics path

Enable Prometheus + node exporter (Grafana optional). Rebuild.

```
curl -sS http://127.0.0.1:9090/-/ready
curl -sS http://127.0.0.1:9100/metrics | head
```

Exercise 2B — Logs-first path

Create a runbook with copy-pasteable `journalctl` lines for:

1. Proxy
2. PostgreSQL
3. sops-related activation
4. OCI container unit if any

Add journald retention config if useful.

Exercise 3 — One service deep dive

Pick proxy **or** postgres **or** node. Answer:

- How do I know it is up?
- How do I see last error?
- What metric or log line changes when I break it deliberately?

Perform a controlled break/fix (stop unit; observe; start).

Exercise 4 — Access path

Document SSH tunnel **or** authenticated proxy. Prove you can view metrics/logs without opening world-readable admin UIs.

Exercise 5 — Update ports and isolation docs

Mark 9090/9100/3000 as localhost-only (if used).

Exercise 6 — Capstone foreshadow

List three signals you would want before a wipe/rebuild proof (e.g. free disk, proxy up, DB accepting connections).

Exercise 7 — One dashboard or one journal recipe

Either export a minimal Grafana dashboard path **or** ship `docs/runbooks/service-down.md`.

Exercise 8 — Disk use snapshot

```
journalctl --disk-usage
du -sh /var/lib/prometheus2 2>/dev/null || true
```

Record numbers in observability notes.

Exercise 9 — PromQL basics (metrics path)

In Prometheus UI: run `up` and one node metric. Screenshot optional; notes required.

Exercise 10 — Password hygiene for Grafana

If Grafana enabled, confirm no admin password in git.

Exercise 11 — Unit list for the slice

```
systemctl list-units --type=service --state=running | rg -i 'caddy|nginx|postgres|prometheus
```

Exercise 12 — Error priority journal

```
journalctl -p err -b --no-pager | tail -30
```

Exercise 13 — Tunnel muscle memory

```
ssh -L 9090:127.0.0.1:9090 lab
```

Exercise 14 — Docs cross-link

From `docs/observability.md`, link to `ports.md` and `proxy.md` paths (relative notes).

Symptom / mistake	What to do

Common gotchas

Symptom / mistake	What to do
Grafana/Prometheus on 0.0.0.0 public	Bind localhost; add auth
Scrape target wrong	Check listen addresses
Huge cardinality later	Keep lab exporters few
Logs-only with no runbook	Write runbook
Password in flake for Grafana	sops / setup procedure
Thinking Nix replaces monitoring	It does not
Disk full of journals/TSDB	Cap retention
No break/fix	Do Exercise 3
Metrics without ports.md	Update docs

Checkpoint

- ☐ Observability path chosen and documented
- ☐ Metrics stack running **or** logs runbook complete
- ☐ One controlled break/fix observed
- ☐ Admin interfaces not world-open
- ☐ ports.md updated
- ☐ Signals list for future ops
- ☐ Disk usage noted
- ☐ Access path (tunnel/auth) documented

Journal (optional)

Write three personal gotchas before continuing.

---

k3s (optional awareness)

k3s (optional awareness)

Goal: Either stand up a **single-node k3s** for literacy **or skip cleanly** with a written decision—Kubernetes is not required for this part's service slice.

> **!** Important
>
> If you do not need k8s for work or curiosity right now, **skip**. A forced k3s install can destabilize a small lab (CPU, disk, conceptual overload).

Why this chapter exists

Some learners finish a proxy + Postgres slice and ask: "Should everything move to Kubernetes?" On NixOS the honest answer is usually **no**—not for a one-node lab with Caddy + Postgres.

This chapter exists to:

1. Prevent FOMO-driven architecture
2. Give a **safe optional on-ramp** if you truly need k8s APIs
3. Practice **skipping** as a professional skill

Theory 1 — Decision rubric

Choose **skip** if…	Choose **k3s lite** if…
Service slice already works natively	You must learn kubectl for a job soon
Host is small on RAM/disk	You have 8–16 GB RAM lab resources
You have not finished secrets/proxy/DB solidly	Those foundations are already solid
Curiosity only, no time	You will actually deploy one manifest

Default recommendation: skip, write the decision, move on.

Theory 2 — What k3s is

k3s is a lightweight Kubernetes distribution. On NixOS:

```
# modules/services/k3s.nix - only if you opt in
{
  services.k3s = {
    enable = true;
    role = "server";
    # extraFlags = [ "--write-kubeconfig-mode=0644" ]; # understand security implications
  };
}
```

Options and defaults change—read current module docs for 26.05 before enabling.

Implications:

Area	Effect
Ports	API server, kubelet, etc.—firewall carefully
CPU/RAM	Non-trivial idle cost
Storage	Container images + etcd/sqlite data
Mental model	YAML/Helm vs NixOS modules

Theory 3 — NixOS vs k8s configuration

Concern	NixOS native	k3s
System packages	flake modules	images inside cluster
Host firewall	`networking.firewall`	+ k8s Services/NodePorts
Secrets	sops-nix on host	also k8s Secrets (different!)
Rollback	generations	cluster state is separate

Running k3s **does not replace** host flake discipline. You now have **two** control planes: NixOS + cluster.

Theory 4 — Single-node honesty

Single-node k3s is fine for learning APIs. It is **not**:

- Multi-AZ HA
- A reason to abandon PostgreSQL module for "cloud native" on one VPS without backups
- Automatically more secure

Theory 5 — Minimal literacy tasks (if enabled)

1. `kubectl get nodes`
2. Deploy a tiny `Deployment` + `Service`
3. Expose via host proxy or port-forward—not random NodePort to the world
4. Delete the workload
5. Document how to disable `services.k3s` and reclaim resources

Theory 6 — Skip path is first-class

Skipping is documented output:

```
# docs/k3s-decision.md
Decision: SKIP
Date: …
Reasons:
- Service slice does not require k8s
- Host resources: …
- Will revisit if: job requirement / multi-service mesh need
```

That file **passes** the chapter.

Workload	Prefer

Theory 7 — When NixOS modules beat k8s on a single node

Workload	Prefer
One reverse proxy + one DB	NixOS modules
Many identical microservices with k8s API dependency	k3s/k8s
GPU training cluster	Often specialized schedulers
Homelab “because CNCF”	Usually modules + compose/podman

NixOS is already a declarative control plane for **one machine**. k3s adds a second control plane—justify it.

Theory 8 — Resource floor honesty

Single-node k3s wants real RAM/CPU. On a 2 GB VM:

```
k3s + postgres + caddy + prometheus = thrash
```

Skip is the professional choice when resources are tight.

Theory 9 — Feature flag module

```nix
# modules/services/k3s.nix
{ config, lib, ... }:
let cfg = config.myLab.k3s; in {
  options.myLab.k3s.enable = lib.mkEnableOption "optional single-node k3s";
  config = lib.mkIf cfg.enable {
    services.k3s = {
      enable = true;
      role = "server";
      # extraFlags = [ "--disable traefik" ]; # if you keep host proxy as edge
    };
    # networking.firewall.allowedTCPPorts = [ 6443 ]; # admin API - lock down
  };
}
```

Default `enable = false` on the lab host.

Worked example — Disable / enable toggles

Prefer a role flag or simply do not import k3s module. Avoid half-enabled clusters.

Worked example — Tiny workload (if enabled)

```yaml
# manifests/hello.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
spec:
  replicas: 1
  selector:
    matchLabels: { app: hello }
  template:
    metadata:
      labels: { app: hello }
    spec:
      containers:
        - name: hello
          image: docker.io/library/nginx:alpine
          ports:
            - containerPort: 80
```

```
kubectl apply -f manifests/hello.yaml
kubectl get pods
kubectl port-forward deploy/hello 8088:80
curl -sS http://127.0.0.1:8088/ | head
```

Exercises

Exercise S — Skip path (most readers)

1. Write `docs/k3s-decision.md` with SKIP + reasons
2. Skim what k3s would open (ports, disk) from docs—no install
3. List two future triggers to revisit
4. Commit and proceed

Time box: ~30 minutes. Do not guilt-install.

Exercise S2 — Skip artifact quality bar

Your skip note must include:

1. Decision (skip)
2. Resource or complexity reason
3. What you use instead (NixOS modules / podman)
4. Revisit criteria ("if job requires CKA lab…")

A one-word "skipped" is not enough.

Exercise K1 — Opt-in: enable k3s

Only if rubric says go:

```
sudo nixos-rebuild switch --flake .#lab
sudo systemctl status k3s
sudo k3s kubectl get nodes
```

Kubeconfig location per module—follow module notes. Restrict access modes.

Exercise K2 — Tiny workload

Apply the hello Deployment; port-forward; curl; delete or keep intentionally.

Exercise K3 — Tear-down plan

Document:

```
kubectl delete -f manifests/hello.yaml
# disable services.k3s; rebuild; confirm resources freed
```

Update firewall if you opened API ports. Prefer API on localhost / VPN only.

Exercise K4 — Compare control planes

Write a short note: what still belongs in the NixOS flake vs what you put in the cluster.

Exercise K5 — API surface inventory (if enabled)

```
sudo k3s kubectl get nodes
sudo k3s kubectl get pods -A | head
ss -tlnp | grep -E '6443|10250' || true
```

Add every k3s port to `docs/ports.md` with "cluster-internal / admin-only" labels.

Exercise 6 — Resource estimate

Write RAM/disk numbers before and (if enabled) after k3s.

Exercise 7 — Secrets boundary note

One paragraph: host sops vs k8s Secrets—both needed if cluster is on.

Exercise 8 — Proxy interaction

If Traefik conflicts with Caddy/Nginx, disable one; document edge story.

Exercise 9 — Decision commit

```
git add docs/k3s-decision.md
git commit -m "docs: k3s optional decision recorded"
```

Exercise 10 — FOMO check

Write two sentences: what pressure almost made you install k3s, and why you did or did not.

Exercise 11 — Revisit calendar

If skip: pick a future review date or trigger (job change, multi-node need).

Exercise 12 — ports.md consistency

Whether skip or enable, ports.md should not claim k3s ports that do not exist.

Exercise 13 — Disable half-install

If you started and regret: disable module, rebuild, free disk images.

Exercise 14 — Part retrospective

List the service slice components you trust without k8s: proxy, DB, secrets, firewall, observability.

Common gotchas

Symptom / mistake	What to do
Guilt install before readiness	Use skip path
Opened k8s API to world	Close; bind/firewall
Disk full of images	Prune; disable k3s
Secrets only in k8s, host ignored	Host sops still matters
Two proxies fighting	One ingress story
Undocumented cluster	Decision doc mandatory either way
Thrashing small VM	Skip; free resources
"Everything in cluster"	Push back; host still owns kernel/disks

---

Checkpoint

If skipped

- ☐ `docs/k3s-decision.md` with SKIP + reasons
- ☐ Future revisit triggers listed
- ☐ No half-installed k3s units left
- ☐ ports.md consistent

If enabled

- ☐ Single-node ready; `kubectl get nodes` works
- ☐ One workload applied and removed or still intentional
- ☐ API not casually public
- ☐ Disable/tear-down plan written
- ☐ ports.md updated
- ☐ Resource impact noted

Journal (optional)

Write three personal gotchas before continuing.

Part close-out

You should now have a **service and security spine**:

Capability	Chapter
Store vault	Secrets problem
Encrypt-in-git secrets	sops-nix (+ agenix compared)
Reduced attack surface	Hardening
Deny-by-default network	Firewall + ports.md
TLS edge	Reverse proxy
Durable data	PostgreSQL pattern
Isolation literacy	nspawn + OCI
Inspectability	Observability
k8s FOMO managed	Optional k3s decision

Later parts (packaging, ops/fleet, capstone) assume this discipline. If secrets still live as Nix strings or 5432 is world-open, fix that before scaling out.

---

nixos-vex: configuration-aware VEX for vulnerability triage

nixos-vex: configuration-aware VEX for vulnerability triage

SBOM scanners are blunt. They see package names and versions, not whether your OpenSSH build ever hands a login prompt to PAM, whether OPIE is in the closure, or whether keyboard-interactive authentication is off. The result is a flood of CVEs that do not apply to *this* machine. **VEX** (Vulnerability Exploitability eXchange) is the machine-readable answer: a statement that a product is `not_affected` (or still affected) for a named vulnerability, with a justification scanners can honor.

NixOS makes that statement cheap to *re-check*. The system configuration is data. A module can evaluate predicates against `config.*` and emit an OpenVEX document that only keeps statements still true for your host. **nixos-vex** packages that idea: a shared feed of unconditional and conditional triage decisions, plus a NixOS module that builds `system.build.vexFeed` for your configuration.

This chapter teaches the VEX mental model, how NixOS turns config into exploitability predicates, how to wire nixos-vex + sbomnix + grype, and where the trust boundaries are. Versions: **NixOS 26.05** (stable "Yarara" as of 2026-10) / **nixos-unstable** for the feed's current baseline; OpenVEX `v0.2.0` context. No local NixOS rebuild was run on this box—module shapes and commands follow the project's README and the October 2026 walkthrough that introduced the technique.

Learning goals

By the end of this chapter you can:

- Explain what an OpenVEX statement claims (`status`, `justification`, product `@id`)
- Contrast SBOM scanning with config-aware triage on NixOS
- Import `nixos-vex.nixosModules.default` and build `system.build.vexFeed`
- Pass the feed to `grype --vex` next to an SBOM
- State the project's disclaimer: zero *untriaged* is not zero vulnerabilities

Thesis

> On NixOS, exploitability is often a property of the *module graph*, not only of the package version. Codify the triage once as a conditional OpenVEX statement; re-evaluate it on every rebuild so scanners stop re-asking questions the config already answered.

Mental model

```
SBOM (sbomnix / syft)          OpenVEX feed (nixos-vex)

  packages in                    unconditional: always true
  system closure     grype     conditional: config.services
                      + VEX         .openssh.settings.…
                        only true statements kept
```

Idea	Meaning
SBOM	Inventory of components (what is installed)
VEX	Asserted exploitability status for a CVE × product
Unconditional statement	True for every NixOS system in the feed's scope (bad upstream data, wrong distro patch, …)
Conditional statement	True only if predicates on *your* `config` hold; dropped (with a warning) when they fail
`system.build.vexFeed`	Derivation output: OpenVEX JSON tailored to this configuration

OpenVEX statuses you will see most often: `not_affected` with justifications such as `vulnerable_code_not_present`, `vulnerable_code_not_in_execute_path`, or `component_not_present` (see the OpenVEX specification).

Case 1 — Why SBOMs alone drown ops

A typical pipeline:

1. Generate a CycloneDX SBOM for a system or container image.
2. Run Grype (or another scanner) against that SBOM.
3. Triage every hit by hand.

For language ecosystems with strong reachability tools (Go's `govulncheck`), many hits evaporate. For an OS image, the same CVE may depend on PAM modules you never install, filesystems you never mount, or SSH options you already disabled. Repeating that analysis on every config change does not scale.

NixOS's advantage is not "fewer CVEs in nixpkgs." It is that **the same evaluation that builds the system can answer "does this knob still hold?"**

Case 2 — Minimal conditional VEX from a NixOS module

The technique (illustrated in Fabian Kammel's October 2026 post, and the shape nixos-vex automates) looks like this: a boolean derived from `config` and `pkgs`, then `lib.optional` into an OpenVEX `statements` list.

```
# illustrative-module.nix - pattern only; prefer nixos-vex's feed in production
{
  config,
  lib,
  pkgs,
  ...
}:
let
  # CVE-2007-2768: username enumeration via OPIE through PAM / keyboard-interactive
  notAffected =
    !(pkgs ? opie)
    && config.services.openssh.settings.KbdInteractiveAuthentication == false
    && !(lib.any (rule: rule.enable && lib.hasInfix "opie" rule.modulePath) (
      lib.attrValues config.security.pam.services.sshd.rules.auth
    ));
in
{
  system.build.vex = pkgs.writeText "vex.json" (
    builtins.toJSON {
      "@context" = "https://openvex.dev/ns/v0.2.0";
      "@id" = "https://example.invalid/vex/sshd";
      author = "lab";
      timestamp = "2026-10-16T00:00:00Z";
      version = 1;
      statements = lib.optional notAffected {
        vulnerability.name = "CVE-2007-2768";
        products = [ { "@id" = "pkg:nix/openssh"; } ];
        status = "not_affected";
        justification = "vulnerable_code_not_present";
      };
    }
  );
}
```

Wire a host that enables SSH with keyboard-interactive off (NixOS 26.05+ `system.stateVersion` style):

```
# configuration.nix fragment
{
  system.stateVersion = "26.05";
  services.openssh.enable = true;
```

```
  services.openssh.settings.KbdInteractiveAuthentication = false;
}
```

When `notAffected` is true, Grype can ignore that CVE for the OpenSSH product id. When someone flips keyboard-interactive back on, the statement disappears from the document—the scanner noise returns, which is the correct failure mode.

> **!** Important
>
> nixos-vex's author explicitly warns: treat the feed as a triage aid, not a high-trust attestation. Statements may be wrong; LLM-assisted drafts exist in the project history and are human-signed-off, not oracle-perfect. Conditions see **configuration**, not imperative runtime changes.

Case 3 — Using nixos-vex on a flake host

From the nixos-vex README:

```
# flake.nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    nixos-vex.url = "github:datosh/nixos-vex";
  };

  outputs = { nixpkgs, nixos-vex, ... }: {
    nixosConfigurations.myhost = nixpkgs.lib.nixosSystem {
      modules = [
        ./configuration.nix
        nixos-vex.nixosModules.default # adds system.build.vexFeed
      ];
    };
  };
}
```

Build the feed and scan:

```
# SBOM for the system closure (sbomnix elsewhere in your CI)
# sbomnix .#nixosConfigurations.myhost.config.system.build.toplevel

nix build .#nixosConfigurations.myhost.config.system.build.vexFeed -o vex-feed.json
grype sbom:sbom.cdx.json --vex vex-feed.json
```

Artifact	Role
Baseline host in the repo	Defines which packages the shared feed has triaged
Unconditional statements	Apply to every consumer of the feed
Conditional statements	Filtered by the module against *your* `config`
Failed condition	Warning + triage handed back to you (`status_notes` may carry `[nixos-vex:condition=<id>]`)

Demos in the repository (`demo/desktop`, `demo/server`) show a soft desktop profile (more failed conditions) versus a hardened server (strict mode when every statement holds).

The trap

- **Equating "ignored by VEX" with "safe."** The feed's goal is zero *untriaged* findings for packages it covers—not a clean bill of health.
- **Assuming the feed covers packages you added.** Only the reference baseline's packages are triaged; your extra services are on you.
- **Trusting conditions against a drifted runtime.** Loading a kernel module by hand or starting a unit outside the config is invisible to `config.*` predicates.
- **Shipping a hand-written VEX without CI.** The whole point is re-evaluation on rebuild; a stale JSON file in git is worse than honest scanner noise.

The boring rule

Pair every SBOM scan of a NixOS closure with a **config-evaluated** OpenVEX document. Prefer a maintained feed module (nixos-vex or an in-house equivalent) over one-off JSON. Re-check failed conditions in review the same way you review `services.*` diffs.

Try this

1. Add `nixos-vex` to a lab flake and build `system.build.vexFeed`.
2. Generate an SBOM with sbomnix for the same `toplevel`.
3. Run Grype twice: once without `--vex`, once with it. Diff the ignored count.
4. Flip one hardening knob that a conditional statement depends on; rebuild the feed; confirm the statement drops and the CVE reappears.

Sources

- nixos-vex — module, feed, demos, disclaimer
- The Unreasonable Effectiveness of VEX in NixOS (Fabian Kammel / DaTosh, 2026-10-11) — walkthrough motivating the tool
- OpenVEX / OpenVEX spec
- sbomnix, Grype VEX support
- NixOS 26.05 announcement — current stable channel context

Inspiration

- Topic signal: discussion of configuration-aware VEX on NixOS (HN / X threads around the Kammel post, October 2026). Chapter cites the blog and repository above, not social posts.

Verification

Docs-based against nixos-vex README and the Kammel post (retrieved 2026-10-16). No `nixos-rebuild` / grype run on this shared box (no NixOS host, no package installs for scanners). NixOS **26.05** confirmed current stable via nixos.org announcement; nixos-vex itself currently targets **nixos-unstable** x86_64 for its baseline feed—call that out in CI if you pin 26.05.

Vulnerability Inventory across Profiles and Images

Vulnerability Inventory across Profiles and Images

Thesis

A clean `nixos-rebuild` does **not** mean every closure you run is patched. User profiles, old generations, `nix-shell` GC roots, and container images keep **old store paths** alive. Inventory must walk those roots, not only the booted system.

Mental model

```
live software = union of closures reachable from:
  - current system generation
  - previous generations (until GC)
  - per-user profiles (~/.nix-profile, home-manager)
  - explicit GC roots (docker layers, result symlinks)
  - ephemeral shells pinned by direnv / nix-direnv
```

CVE scanners that only look at `/run/current-system` under-report.

Worked example

Docs-based inventory checklist (`nix` absent here):

```
# booted system
readlink -f /run/current-system

# generations still rooted
nix-env --list-generations -p /nix/var/nix/profiles/system

# user profile
nix profile list   # or nix-env -q

# GC roots that surprise you
ls -l /nix/var/nix/gcroots

# container image built with dockerTools / nix2container
# → scan the image's closure, not just the host
```

Wire a scanner (e.g. `vulnix`, VEX/NixOS VEX workflows, or SBOM export) against **each** root class. Delete stale generations and `result` links when patches land.

Trap

"We rebuilt yesterday" as a security statement. Old generations remain bootable and runnable. CI images cached on a registry may pin a vulnerable `bash` for months.

Boring rule

Track roots explicitly, GC on a schedule, and fail deploys when a scanner finds a CVE in any still-rooted closure you ship — not only in `current-system`.

Try this

1. List GC roots before/after removing a `result` symlink.
2. Compare `nix path-info -r /run/current-system` vs a home-manager profile.
3. Export an SBOM for a `dockerTools` image and scan it offline.

Sources

- Nix manual: garbage collection / profiles
- NixOS VEX / vulnerability tracking discussions in nixpkgs
- Software supply-chain SBOM practices for Nix closures

Packaging

Packaging craft

Packaging craft

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

Baseline: Nix **2.34.x**, flakes-first workflows, nixpkgs / NixOS **26.05**. Language builders, FOD hash fields, and CLI flags evolve—prefer patterns that match current nixpkgs examples when a helper renames an attribute.

What this part is for

You already run hosts, compose modules, and pin flakes. **Packaging craft** is how you:

1. Turn source into store paths with deliberate `stdenv` phases
2. Pin bytes with fixed-output derivations (FODs)
3. Use language ecosystems without cargo-culting random blogs
4. Patch and override without forking nixpkgs forever
5. Split outputs, debug sandboxes, navigate nixpkgs layout
6. Feed fleets with **binary caches** and **remote builders**
7. Speak honestly about **reproducibility**
8. (Elective) Run **ROS 2** via **nix-ros-overlay** without Ubuntu-deb lock-in

Without this part, "we use Nix" means only consuming `environment.systemPackages`. With it, you **produce** packages, own supply-chain pins, and debug CI the same way you debug a laptop.

Prerequisites

Skill	Where it came from
Derivations, store paths, closures	Concepts part
Flakes, `callPackage`, overlays (light)	Home and flake layout
Host consumption of packages	NixOS host part
Optional: secrets for cache tokens	Services and security

You do **not** need prior C packaging expertise. You do need willingness to read build logs.

Chapter map

Chapter	File	Focus
stdenv and build phases	`01-stdenv-phases`	`genericBuild`, hooks, inputs kinds
Fetchers and FODs	`02-fetchers-fod`	Hash workflow, trust, flake inputs vs `src`
Language ecosystems A	`03-language-ecosystems-a`	One deep language builder
Language ecosystems B	`04-language-ecosystems-b`	Second ecosystem; pattern transfer
Patching and overrides	`05-patching-overrides`	`overrideAttrs`, overlays, patches
Multiple outputs	`06-multiple-outputs`	`out`/`dev`/`man`, closure size
Debugging builds	`07-debug-builds`	Logs, keep-failed, breakpointHook
Navigating nixpkgs	`08-nixpkgs-layout`	`by-name`, PR-shaped packages
Binary caches	`09-binary-caches`	Substituters, Cachix/Attic/Harmonia, trust
Remote builders	`10-remote-builders`	Offload realization; features
Reproducibility	`11-reproducibility`	L0–L5 honesty, measurement
ROS 2 / nix-ros-overlay	`12-ros2-nix-ros-overlay`	Robotics underlays, flakes, caches, workspaces

Suggested linear path: **01** → **02** → **03** → **04** → **05** → **06** → **07** → **08**, then **09** **10** as a pair, finish with **11**.
Robotics elective: after overlays + caches literacy, take **12** (can also run from a Fedora dual-stack install).

Mental model — three layers of “a package”

```
Meta & policy (license, mainProgram, …)

Build plan (phases, hooks, language helper)

Fixed inputs (src FOD, vendor/cargo/npm …)


    /nix/store/…-pname-version
```

Layer	Failure class	First tool
Inputs	Hash mismatch, floating tag	Prefetch / fakeHash
Build	Compile, link, install	`-L`, `nix log`
Meta / consume	Wrong bin name, missing wrap	`nix run`, path-info

---

stdenv is the spine

Almost every language helper is still `stdenv.mkDerivation` plus setup hooks:

```
buildGoModule / buildRustPackage / buildPythonApplication / buildNpmPackage


  stdenv.mkDerivation (+ setup hooks)


  unpack → patch → configure → build → check → install → fixup
```

If you cannot name the default phase order, every “mysterious” failure later is slower than it needs to be. The stdenv chapter is non-skippable for practitioners who only ever used `nix profile install`.

FODs vs normal builds

Kind	Network in sandbox?	Output path depends on
Normal	No (default)	Derivation inputs
Fixed-output	Yes, limited	Declared **output hash**

Package `src` and language dependency graphs are almost always FODs. Updating a hash is a **trust decision**, not a clerical annoyance—covered in the fetchers chapter and again under reproducibility.

Caches vs builders (preview)

These two chapters are confused constantly:

```
[dev] --build--> [builder] --push--> [cache] --substitute--> [fleet]
```

Mechanism	Moves	Needs existing path?
Binary cache	Results (NARs)	Yes
Remote builder	Work (realize drv)	No

Packaging craft produces the paths; ops and CI **distribute** them. Read both packaging chapters before wiring fleet CI.

Lab workspace conventions

Suggested roots (adjust freely):

```
~/lab/nixos/packaging/
  stdenv/          # shell + C toys
  fod/             # fetchurl / GitHub drills
  eco-a/           # primary language tool
  eco-b/           # second ecosystem
  overrides/       # patches + overlays
  multi-out/       # lib + app
  debug/           # intentional breaks
  by-name/         # nixpkgs-shaped layout
  cache/           # push/pull notes
  builders/        # remote build topology
  repro/           # REPRO.md + measurements
```

Each chapter's lab can be a **separate flake** or packages inside one monorepo flake—prefer one flake with `packages.*` attrs if you already have a lab flake.

```
# Recurring commands
nix build .#attr -L
nix run .#attr
nix flake check
nix path-info -rS ./result | sort -k2 -n | tail
nix log .#attr
```

How chapters are structured

Most chapters use the same spine so you can scan as a reference:

1. **Goal** — what "done" looks like
2. **Why it matters** — ops/packaging motivation
3. **Theory** — models, tables, complete Nix snippets
4. **Worked examples** — copyable, 26.05-shaped
5. **Lab** — multi-step exercises
6. **Pitfalls** — symptom → fix tables
7. **Checkpoint** — self-review list
8. **Commit** — suggested message (optional discipline)

Reading paths

Path A — "I need one internal CLI in nixpkgs style"

1. stdenv phases
2. Fetchers / FODs
3. Language ecosystems A (your language)

4. Debugging builds
5. nixpkgs layout (meta + by-name shape)
6. Reproducibility (write claims)

Path B — "CI is compiling the world"

1. Binary caches (trust + push)
2. Remote builders
3. Debugging builds (cache miss vs compile fail)
4. Reproducibility (L1/L2 measurements)

Path C — "We forked half of nixpkgs"

1. Patching and overrides
2. Multiple outputs (if closures bloated)
3. nixpkgs layout (upstream-shaped expressions)
4. Language ecosystems B (stop reinventing builders)

Glossary (part-local)

Term	Meaning
stdenv	Standard environment: toolchain + `genericBuild` driver
Phase	Named step (`buildPhase`, `installPhase`, …)
Hook	`preInstall` / `postFixup` style glue around phases
FOD	Fixed-output derivation; hash pins content
SRI hash	`sha256-…` form preferred in modern nixpkgs
vendorHash / cargoHash / npmDepsHash	Language-dep FODs
overrideAttrs	Change derivation attributes after the fact
Overlay	Function `final: prev: { … }` reshaping `pkgs`
Multiple outputs	Separate store paths (`out`, `dev`, …)
Substituter	Binary cache endpoint Nix may pull from
Remote builder	Machine that realizes derivations for you
L1 / L2 repro	Same path on rebuild / across similar hosts

Quality bar for "done packaging"

A package is not "done" when it builds once on your laptop. Aim for:

Criterion	Check
Builds from flake lock	`nix build .#pkg -L`
Runnable entrypoint	`meta.mainProgram` + `nix run`
Hashes intentional	No `fakeHash` left committed
Logs readable	You can name the failing phase
Meta honest	license, description, platforms
Closure sane	`nix path-info -Sh`; multi-out if library
Claim measured	REPRO note for anything you ship to others

Relationship to other parts

Part	Connection
Concepts	Derivations and store theory become muscle memory here
NixOS host	`environment.systemPackages` and modules *consume* packages
Home / flakes	Overlays, checks, lock discipline
Services / security	Cache tokens and signing keys are secrets
Ops and fleet	Deploy tools ship **closures** you learned to build
Internals	Evaluator, CA derivations, store daemon deepen later
Projects	Hands-on packaging drills reuse this part's patterns

This book does **not** cross-link a parallel day-calendar series. Everything here is topic-organized library material.

Anti-patterns to avoid while studying

Anti-pattern	Prefer
Disable sandbox to "make it work"	Find undeclared dep or FOD
Branch name as `rev` forever	Tag or full commit SHA
Global overlay for a one-off flag	Local `overrideAttrs`

Anti-pattern	Prefer
Single `$out` with headers + docs + bins for shared libs	Multiple outputs
"Fully reproducible" in README without measurement	L1/L2 claims with lock + system
Copying random `default.nix` without reading phases	stdenv + official language helper

---

Part checkpoint (before you leave packaging)

- ☐ Can list default `stdenv` phase order from memory
- ☐ Shipped at least one package with a real FOD hash workflow
- ☐ One language-builder package end-to-end (ecosystem A)
- ☐ Second ecosystem or transfer table completed (ecosystem B)
- ☐ Patch + overlay composition demonstrated
- ☐ Multi-output or closure inspection done deliberately
- ☐ Personal debug runbook exists
- ☐ Package shaped like nixpkgs (`meta`, preferably by-name layout)
- ☐ Cache trust model written (even if only cache.nixos.org)
- ☐ Know cache vs remote builder in one paragraph
- ☐ Honesty note on reproducibility for one attr

Suggested lab commit series

```
git commit -m "packaging: stdenv greet + addone"
git commit -m "packaging: FOD hash workflow"
git commit -m "packaging: language ecosystem A"
git commit -m "packaging: language ecosystem B"
git commit -m "packaging: patches and overlays"
git commit -m "packaging: multi-output lib + app"
git commit -m "packaging: debug runbook"
git commit -m "packaging: by-name shape + meta"
git commit -m "packaging: cache trust notes"
git commit -m "packaging: remote builder topology"
git commit -m "packaging: REPRO.md measurements"
```

How to use this part as reference

When something breaks in six months:

Symptom	Jump to
Unknown phase failure	Debugging builds; stdenv phases
`hash mismatch`	Fetchers and FODs
Language dep network in build	Ecosystems A/B
Need a CVE pin	Patching and overrides
Runtime closure huge	Multiple outputs
Where do I put this for upstream?	nixpkgs layout
CI rebuilds everything	Binary caches; remote builders
Manager asks “is it reproducible?”	Reproducibility

Next

Start with **stdenv and build phases**: name every default phase, package a shell tool and a tiny C tool, and break install on purpose so logs become familiar. Then pin bytes with FODs before you trust any “reproducible” claim.

stdenv and build phases

stdenv and build phases

Goal: Package a small C or shell tool with `stdenv.mkDerivation`, name every default phase, and know when to override hooks vs rewrite the whole build.

> **Note**
>
> **Baseline:** Nix 2.34 / nixpkgs **26.05**, flakes-first. Language helpers (`buildGoModule`, …) wrap the same lifecycle—master phases here first.

Why it matters

Most "how do I package X?" questions collapse into one skill: **drive `stdenv` phases deliberately**. Language helpers are thin wrappers on the same lifecycle. If you cannot read a failing phase log, every later packaging chapter will feel random.

You have already *consumed* packages on a host. Packaging craft is producing them: pure plans, sandboxed builds, store outputs you can reason about.

Theory 1 — What `stdenv.mkDerivation` actually is

A derivation is a pure build plan: inputs → sandbox → outputs under `/nix/store`. `stdenv.mkDerivation` fills that plan with a **bash driver** (`genericBuild`) and a default toolchain (compiler, coreutils, patch, make, …).

```
{ stdenv, fetchurl, lib }:
stdenv.mkDerivation rec {
  pname = "hello";
  version = "2.12.1";
  src = fetchurl {
    url = "mirror://gnu/hello/hello-${version}.tar.gz";
    hash = "sha256-jZkUKv2SV28wsM18tCqNxoCZmLxdYH2Idh9RLibH2yA=";
  };
  meta = {
    description = "GNU Hello prints a friendly greeting";
    license = lib.licenses.gpl3Plus;
    mainProgram = "hello";
```

```
  };
}
```

Concept	Meaning
pname / version	Human labels; feed `name` as `${pname}-${version}` by default
src	Input path (usually a fixed-output fetcher)
buildInputs	Host libraries linked into the result (runtime + link time on native)
nativeBuildInputs	Tools that run **on the build machine** (compilers, cmake, pkg-config)
propagated*	Rare; pollutes dependents—prefer explicit deps
meta	Description, license, platforms, `mainProgram`

`buildInputs` vs `nativeBuildInputs` (cross-aware)

Attr	Role	Cross example
`nativeBuildInputs`	Execute during build	`pkg-config`, `cmake`, host `gcc` when cross-compiling
`buildInputs`	Link / use as target libraries	`openssl`, `zlib` for the target system
`depsBuildBuild`	Advanced bootstrap	Tools for the build→build hop
`depsBuildTarget`	Cross edge cases	See nixpkgs cross docs

For native-only packaging you can get away with sloppy placement; **cross and static** punish it. Prefer correct classification from the first package you write.

Evaluation vs realization

```
nix eval / callPackage  →  derivation (.drv)
nix build / realize     →  /nix/store/… outputs
```

Phase scripts run at **realization**. A typo in a phase string fails the build; a typo in attribute structure may fail at eval. Know which side you are on before staring at compiler errors that never ran.

Theory 2 — Default phase order

`genericBuild` runs roughly:

```
unpackPhase
patchPhase
updateAutotoolsGnuConfigScriptsPhase   # often invisible
configurePhase
buildPhase
checkPhase          # if doCheck = true
installPhase
fixupPhase
installCheckPhase   # if doInstallCheck = true
distPhase           # rare for binary packages
```

Hooks fire **before** and **after** many phases (`preConfigure`, `postInstall`, …). Prefer hooks over replacing an entire phase when you only need a few lines.

Phase responsibilities

Phase	Default intent	Common overrides
unpack	Extract `src` into `$sourceRoot`	`sourceRoot`, `setSourceRoot`, custom `unpackPhase` for flat files
patch	Apply `patches` list	`prePatch` to fix line endings
configure	`./configure --prefix=$out` if script exists	`cmakeConfigurePhase`, meson, or `dontConfigure`
build	`make`	Custom compile; prefer language helpers for Go/Rust/…
check	`make check`	Gate with `doCheck = true`
install	`make install`	Copy binaries into `$out/bin`
fixup	Strip, compress man, move docs, patch shebangs	`dontStrip`, `separateDebugInfo`

Hooks API (muscle memory)

When you **replace** a phase, call hooks so setup hooks from dependencies still run:

```
runHook preBuild
# your commands
runHook postBuild
```

Hook	Fires
`preUnpack` / `postUnpack`	Around unpack
`prePatch` / `postPatch`	Around patches
`preConfigure` / `postConfigure`	Around configure
`preBuild` / `postBuild`	Around build
`preInstall` / `postInstall`	Around install
`preFixup` / `postFixup`	Around fixup

Hook	Fires

Skipping `runHook` is a silent footgun: `wrapProgram` hooks, Python path setup, and many language helpers never run.

Theory 3 — Important environment variables in the sandbox

During the build, the driver exports paths you should treat as API:

```
echo "$out"           # primary output
echo "$src"           # path to source input
echo "$NIX_BUILD_TOP" # temp build directory
echo "$sourceRoot"    # directory after unpack
echo "$PWD"           # usually $NIX_BUILD_TOP/$sourceRoot during build
echo "$CC" "$CXX"     # stdenv compilers
echo "$NIX_CFLAGS_COMPILE"  # injected flags
```

Multiple outputs (later chapter) add `$dev`, `$man`, `$doc`, …. In this chapter, stick to `$out`.

Shebang and RPATH fixups happen in `fixupPhase` / `patchShebangs`. If a script still points at `/usr/bin/env` incorrectly, you usually forgot `nativeBuildInputs` for the interpreter **or** skipped fixup (`dontFixup = true` is rarely what you want).

Flags that skip work

Attr	Effect
`dontUnpack`	Skip unpack (inline sources / copy yourself)
`dontConfigure`	Skip configure
`dontBuild`	Skip build
`dontInstall`	Almost never—you need `$out` populated
`dontFixup`	Skip strip/shebang; debug only
`doCheck`	Enable checkPhase
`enableParallelBuilding`	`make -j` style parallelism

Theory 4 — Minimal patterns you will reuse forever

Shell-only "package" (no compile)

```
{ stdenv, lib, bash }:
stdenv.mkDerivation {
  pname = "greet";
  version = "0.1.0";
  dontUnpack = true;
  installPhase = ''
    runHook preInstall
    mkdir -p $out/bin
    cat > $out/bin/greet <<'EOF'
    #!${bash}/bin/bash
    echo "hello from packaging craft"
    EOF
    chmod +x $out/bin/greet
    runHook postInstall
  '';
  meta = {
    description = "Trivial greet script packaged with stdenv";
    license = lib.licenses.mit;
    mainProgram = "greet";
  };
}
```

Note the **${bash}/bin/bash** in the shebang: absolute store path, pure and fixup-friendly.

Autotools / simple C (upstream tarball)

```
{ stdenv, fetchurl, lib }:
stdenv.mkDerivation rec {
  pname = "hello";
  version = "2.12.1";
  src = fetchurl {
    url = "https://ftp.gnu.org/gnu/hello/hello-${version}.tar.gz";
    hash = "sha256-jZkUKv2SV28wsM18tCqNxoCZmLxdYH2Idh9RLibH2yA=";
  };
  doCheck = true;
  meta = {
    homepage = "https://www.gnu.org/software/hello/";
    license = lib.licenses.gpl3Plus;
    mainProgram = "hello";
  };
}
```

Single C file without autotools

```
{ stdenv, lib }:
stdenv.mkDerivation {
  pname = "addone";
  version = "0.1.0";
  src = ./src; # directory with addone.c
  buildPhase = ''
    runHook preBuild
    $CC addone.c -O2 -o addone
    runHook postBuild
  '';
  installPhase = ''
    runHook preInstall
    install -Dm755 addone $out/bin/addone
    runHook postInstall
  '';
  meta = {
    description = "Adds one to an integer argument";
    license = lib.licenses.mit;
    mainProgram = "addone";
  };
}
```

Flat single-file source (unpack quirks)

```
{ stdenv, lib }:
stdenv.mkDerivation {
  pname = "note";
  version = "0.1.0";
  src = ./note.sh;
  dontUnpack = true;
  installPhase = ''
    runHook preInstall
    install -Dm755 $src $out/bin/note
    runHook postInstall
  '';
  meta = {
    mainProgram = "note";
    license = lib.licenses.mit;
  };
}
```

Or keep unpack and set:

```
unpackPhase = ''
  cp $src ./note.sh
  chmod +x ./note.sh
  sourceRoot=.
'';
```

Theory 5 — `finalAttrs` vs `rec`

Modern nixpkgs prefers **`finalAttrs`** over `rec` for self-reference:

```
stdenv.mkDerivation (finalAttrs: {
  pname = "cooltool";
  version = "1.2.3";
  src = fetchurl {
    url = "https://example.com/${finalAttrs.pname}-${finalAttrs.version}.tar.gz";
    hash = "sha256-…";
  };
})
```

Style	Risk
`rec { version = …; name = "${pname}-${version}"; }`	Easy to shadow; override surprises
`finalAttrs:`	Overrides of `version` update dependent attrs cleanly

Use either in lab; prefer `finalAttrs` when shaping packages for upstream later.

Theory 6 — What fixup actually does (short list)

`fixupPhase` (simplified mental model):

1. Move docs/man if multi-out helpers apply
2. Strip binaries (unless `dontStrip`)
3. `patchShebangs` on scripts under `$out`

4. Compress man pages

5. Record references for the runtime closure

If a binary works in the build directory but fails after install, check:

- Did you install into `$out`?
- Did RPATH / interpreter need `patchelf` (usually automatic for stdenv ELF)?
- Did you set `dontFixup`?

```
file result/bin/addone
ldd result/bin/addone   # Linux
```

Worked example — flake that exports packages

```
# flake.nix
{
  description = "stdenv packaging lab";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f (
        import nixpkgs { inherit system; }
      ));
    in {
      packages = forAllSystems (pkgs: {
        default = pkgs.callPackage ./pkgs/greet.nix { };
        greet = pkgs.callPackage ./pkgs/greet.nix { };
        addone = pkgs.callPackage ./pkgs/addone.nix { };
        hello = pkgs.hello; # control sample from nixpkgs
      });

      checks = forAllSystems (pkgs: {
        greet-runs = pkgs.runCommand "greet-runs" {
          nativeBuildInputs = [ self.packages.${pkgs.system}.greet ];
        } ''
          greet | grep -q "packaging craft"
          touch $out
        '';
```

```
        addone-runs = pkgs.runCommand "addone-runs" {
          nativeBuildInputs = [ self.packages.${pkgs.system}.addone ];
        } ''
          test "$(addone 41)" = "42"
          touch $out
        '';
      });

      devShells = forAllSystems (pkgs: {
        default = pkgs.mkShell {
          packages = [
            self.packages.${pkgs.system}.greet
            self.packages.${pkgs.system}.addone
          ];
        };
      });
    };
}
```

```
nix build .#greet
./result/bin/greet
nix build .#addone
./result/bin/addone 41
nix flake check
nix log ./result     # after a failed build: nix log .#attr
```

Host consumption (optional)

```
# on a NixOS host module
{ pkgs, inputs, ... }:
{
  environment.systemPackages = [
    inputs.self.packages.${pkgs.system}.greet
  ];
}
```

Or without rebuilding the host: `nix shell .#greet`.

Worked example — intentional phase map document

For every package, keep a one-pager:

```
## addone
- unpack: directory src (default)
- patch: none
- configure: skipped (no configure script / default no-op)
- build: $CC addone.c
- check: none (doCheck false)
- install: install -Dm755 → $out/bin/addone
- fixup: strip + shebang (n/a for pure ELF)
```

This document is gold when something fails six months later.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/stdenv`

Step 1 — Scaffold

```
mkdir -p ~/lab/nixos/packaging/stdenv/{pkgs,src}
cd ~/lab/nixos/packaging/stdenv
```

Create `src/addone.c`:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv) {
  if (argc != 2) {
    fprintf(stderr, "usage: %s N\n", argv[0]);
    return 1;
  }
  printf("%d\n", atoi(argv[1]) + 1);
  return 0;
}
```

Step 2 — Write three packages

1. `pkgs/greet.nix` — shell script package
2. `pkgs/addone.nix` — single-file C
3. Optional: package `hello` yourself with `fetchurl` (compare to `pkgs.hello`)

Wire the flake above. Confirm:

```
nix build .#greet -L
nix build .#addone -L
nix run .#greet
```

Step 3 — Break phases on purpose

In `addone.nix`, set temporarily:

```
installPhase = "false"; # fail install
```

Build, read the log, note which phase failed:

```
nix build .#addone --print-build-logs || true
# restore installPhase with runHook wrappers
```

Step 4 — Flip input kinds

Move a tool between `buildInputs` and `nativeBuildInputs` (e.g. add `pkgs.pkg-config` wrongly then correctly). On native Linux both may "work"; write one sentence on **why** native is still correct for compilers and pkg-config.

Step 5 — Phase introspection

```
nix build .#greet -L
nix path-info -r ./result | head
nix derivation show .#greet | head -c 2000
```

Document: unpack → build → install → fixup for each of your packages (what actually ran).

Step 6 — Drop `runHook` once

Remove `runHook preInstall` / `postInstall` from greet, rebuild. Note: may still succeed for trivial packages—document why hooks still matter for non-trivial trees (wrappers, setup hooks).

Step 7 — Parallel build flag

For a make-based package (hello), set `enableParallelBuilding = true` and rebuild with `-L`. Note make job messages.

Step 8 — Host or shell consumption

Wire systemPackages **or** use `nix develop` / `nix shell`. Prove the binary is on `PATH`.

Exercises

1. **Empty `$out`:** Write a derivation that "succeeds" but leaves `$out` empty—does Nix allow it? What does `nix path-info` show? Fix by installing a marker file.

2. **`sourceRoot`:** Unpack a nested tarball layout (or fake one) and set `sourceRoot` correctly.

3. **Check phase:** Enable `doCheck` with a trivial `checkPhase` that greps your binary's help text.

4. **meta.mainProgram:** Break `mainProgram`, run `nix run .#greet`, fix it.

5. **Compare to nixpkgs:** `nix edit nixpkgs#hello` (or equivalent) and list three differences from your toy expressions.

Common gotchas

Symptom / mistake	What to do
`builder for '…' failed with exit code 1` with no clue	Rebuild with `-L` / `--print-build-logs`; jump to last phase name in log
`install: missing destination`	Create `$out/bin` before install; use `install -Dm755`
Binary runs in build sandbox but not after install	Forgot install into `$out`; or RPATH/interpreter wrong—check `ldd` / `file`
`patchShebangs` left bad paths	Ensure interpreter package is a build input; call `patchShebangs $out`
Overrode `buildPhase` without `runHook`	Add `runHook preBuild` / `postBuild`
Used `buildInputs` for `cmake`/`pkg-config`	Prefer `nativeBuildInputs`
`src` is a single file, unpack fails	`dontUnpack` + copy, or custom `unpackPhase`
Name collisions / impure timestamps in C	Avoid embedding timestamps; prefer `-D` defines from version only
`rec` + override surprises	Prefer `finalAttrs` pattern

Symptom / mistake	What to do
Darwin vs Linux `$CC` assumptions	Use stdenv's `$CC`, not hard-coded `gcc`
Forgetting `meta.license`	Set it; habit for nixpkgs-shaped work
Parallel race in custom `buildPhase`	Document; fix races or disable parallel for that package

---

Checkpoint

- ☐ Can list default phase order from memory
- ☐ `greet` and `addone` build with `nix build`
- ☐ At least one intentional phase failure diagnosed from logs
- ☐ `nativeBuildInputs` vs `buildInputs` explained in notes
- ☐ Phases documented for each package
- ☐ Hooks (`runHook`) used in overridden phases
- ☐ Flake `checks` smoke-test at least one package
- ☐ Optional: compared personal `hello` packaging to `pkgs.hello`

Commit

```
git add .
git commit -m "packaging: stdenv phases - greet + addone packages"
```

Write three personal gotchas before continuing to fetchers and FODs.

Fetchers and fixed-output derivations

Fetchers and fixed-output derivations

Goal: Use fixed-output derivations (FODs) correctly—`fetchurl`, `fetchFromGitHub`, vendor hashes—and debug hash mismatches without turning off purity.

> **Note**
>
> **Baseline:** Nix 2.34 / nixpkgs **26.05**. Prefer SRI `hash = "sha256-…"` over legacy `sha256 = "…"` attrs (both still appear in the wild).

Why it matters

Every package starts as **bytes from somewhere**. If those bytes are not fixed-output and hashed, your "reproducible" package is a network role-play. FODs are the contract: *given this hash, the store path is determined by content, and the build may touch the network only to produce those exact bytes.*

Without FOD discipline you cannot trust CI, caches, or offline rebuilds. With it, hash mismatches become a clear signal: either you meant to bump content, or something untrustworthy changed.

Theory 1 — Fixed-output vs normal derivations

Kind	Network in sandbox?	Output path depends on	Failure mode
Normal build	No (default)	Inputs (input-addressed)	Compile errors, missing tools
Fixed-output (FOD)	Yes, limited	Declared **output hash**	Hash mismatch if bytes change

FODs power almost every `src = fetch…` you see in nixpkgs.

```
src = fetchurl {
  url = "https://example.com/foo-1.2.3.tar.gz";
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
};
```

Why the store path is special

For FODs, the output path is a function of the **hash** (and related fixed-output metadata), not of the builder script details the same way normal derivations are. That is why two packages fetching the same bytes can share a store path, and why a wrong hash never "quietly" accepts different content.

```
Normal:  out = f(name, builder, input paths, env, system, …)
FOD:     out = f(outputHash, outputHashMode, …)  # content-addressed flavor
```

(Exact hashing rules are an internals topic; operationally: **trust the hash field**.)

Theory 2 — Common fetchers (2026 mental map)

Fetcher	Use when	Notes
`fetchurl`	Direct tarball/file URL	Mirrors, `curlOpts`, `name`
`fetchTarball`	Eval-time tarball (flakes/inputs style)	Different API; not always for package `src`
`fetchFromGitHub`	GitHub repo + rev/tag	Sets `owner`, `repo`, `rev`, `hash`
`fetchFromGitLab`	GitLab	Similar shape
`fetchgit`	Arbitrary git	Needs `hash`; `leaveDotGit` carefully
`fetchzip`	Zip that should unpack	Unpacks in fetcher
`fetchpatch` / `fetchpatch2`	Single patch files	Great for `patches = [ … ]`
`fetchPypi`	Python sdist/wheel helpers	Via python infra
Language-specific vendor hashes	Go modules, Cargo, npm	See language ecosystem chapters

`fetchFromGitHub` pattern

```
{ stdenv, fetchFromGitHub, lib }:
stdenv.mkDerivation rec {
  pname = "example-src-only";
  version = "14.1.1";
  src = fetchFromGitHub {
    owner = "BurntSushi";
    repo = "ripgrep";
    rev = version;           # tag or full commit
    hash = "sha256-…";      # FOD for the archive Nix downloads
```

```
  };
  # real packaging would use rustPlatform; this is fetcher-focused
  meta = {
    description = "Fetcher demo only";
    license = lib.licenses.unfreeRedistributable; # demo-set real license for real pkgs
  };
}
```

Prefer **tags or full commit SHAs** over branch names (`main`). Branches move; FODs must not.

Submodules and deep clones

```
src = fetchFromGitHub {
  owner = "org";
  repo = "repo";
  rev = "v1.0.0";
  hash = "sha256-…";
  fetchSubmodules = true; # changes the hash vs without
};
```

Sparse / partial checkouts are advanced—prefer release tarballs when upstream publishes them.

Theory 3 — The hash mismatch workflow

You will see this forever:

```
error: hash mismatch in fixed-output derivation '…':
         specified: sha256-AAAA…
            got:    sha256-BBBB…
```

Correct response

1. **Trust "got" only after you trust the URL/rev.** If the project was compromised, blindly updating the hash pins malware.

2. Confirm you intended that `rev` / version.

3. Replace `hash` with the `got` value (or use `lib.fakeHash` once to force a fail).

4. Rebuild; lock the known-good hash in git.

5. Never leave `hash = "";` or `lib.fakeHash` in committed production flakes.

Prefetch helpers

```
# URL
nix-prefetch-url https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz
nix store prefetch-file https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz

# Flake-style prefetch (metadata + hash)
nix flake prefetch github:BurntSushi/ripgrep/14.1.1

# Generic: let the build fail once
# hash = lib.fakeHash;
```

```
# Temporary during packaging
hash = lib.fakeHash; # forces mismatch with a clear "got"
```

SRI vs base32

Modern nixpkgs prints and prefers **SRI** (`sha256-…`). Older docs show base32. Both can refer to the same bytes; do not mix formats carelessly when comparing strings by eye—use the `got:` line Nix prints.

Theory 4 — Impure traps and purity

Trap	Why it hurts	Fix
`builtins.fetchGit` without flake lock discipline	Eval-time drift	Prefer flake inputs or FOD fetchers with hash
Branch `rev = "main"`	Hash churn	Pin commit
`curl` in a normal `buildPhase`	Sandbox network blocked	Use FOD or `pkgs.fetchurl` as input
Post-download patching inside FOD	Changes bytes → hash mismatch	Patch in normal derivation `patches`
Leaving `.git` without care	Non-determinism	Default archives omit `.git`; `leaveDotGit` is advanced
Mirror that rewrites tarballs	Same URL, new bytes	Pin mirrors; notice mismatch as signal

Trap	Why it hurts	Fix
LFS pointers instead of files	Tiny “source”, broken build	Ensure LFS materialization in fetch

Rule: Download and fix bytes in the FOD **or** patch after unpack in the normal build—do not mix casually.

```
[FOD: pure download → exact bytes]


[Normal derivation: patch / compile / install]
```

Theory 5 — Flake inputs vs package fetchers

Mechanism	Pinned by	Good for
`inputs.foo.url = "github:…"`	`flake.lock`	Nix code, whole repos as flakes
`fetchFromGitHub` in a package	`hash` on the fetcher	Upstream **source** of a package
`pkgs.fetchgit` of a monorepo path	hash + sparse options	Subdir projects
`path: / ./src`	Your git tree	Lab-local code

Do not confuse “my flake input updated” with “my package `src` hash updated.” Both can change store paths; only one is your app’s source tarball.

```
# flake input - for Nix modules / another flake
inputs.tooling.url = "github:acme/tooling/v1.2.3";

# package src - for building C/Go/… sources
src = fetchFromGitHub {
  owner = "acme";
  repo = "tooling";
  rev = "v1.2.3";
  hash = "sha256-…";
};
```

Sometimes you use **both**: flake input for Nix files, FOD for the language project’s release tarball. That is normal—document which is which.

Theory 6 — `outputHashMode` (awareness)

FODs declare how content is hashed:

Mode	Typical use
`flat`	Single file
`recursive`	Directory tree (default for many fetchers)

You rarely set this by hand when using `fetchurl` / `fetchFromGitHub`. When writing custom FODs (`stdenv.mkDerivation` with `outputHash`), wrong mode causes confusing mismatches. Prefer high-level fetchers.

```
# Conceptual custom FOD - prefer fetchurl in real packages
stdenv.mkDerivation {
  name = "my-fod";
  builder = …;
  outputHashMode = "flat";
  outputHashAlgo = "sha256";
  outputHash = "sha256-…";
}
```

Theory 7 — Private sources and credentials

Approach	Notes
CI token in netrc / access tokens	Keep out of world-readable store; prefer FODs that do not embed secrets in drv env
SSH `fetchgit`	Agent/daemon SSH complexity; document who can build
Vendor a release tarball internally	Mirror to your cache/object store + `fetchurl`
`builtins.fetchGit` with local credentials	Eval-time; flake lock discipline still required

Never put passwords in `hash` or URL query strings committed to git. Prefer: internal mirror + FOD hash of the mirrored artifact.

Worked examples

Example A — tarball package skeleton

```
# pkgs/hello-fod.nix
{ stdenv, fetchurl, lib }:
stdenv.mkDerivation rec {
  pname = "hello";
  version = "2.12.1";
  src = fetchurl {
    url = "https://ftp.gnu.org/gnu/hello/hello-${version}.tar.gz";
    hash = "sha256-jZkUKv2SV28wsM18tCqNxoCZmLxdYH2Idh9RLibH2yA=";
  };
  doCheck = true;
  meta = {
    description = "GNU Hello via explicit FOD";
    license = lib.licenses.gpl3Plus;
    mainProgram = "hello";
  };
}
```

Example B — GitHub + patch FOD

```
{ stdenv, fetchFromGitHub, fetchpatch, lib }:
stdenv.mkDerivation {
  pname = "example";
  version = "1.0.0";
  src = fetchFromGitHub {
    owner = "example-org";
    repo = "example";
    rev = "v1.0.0";
    hash = "sha256-SRCSOURCESOURCESOURCESOURCESOURCESOURCESO=";
  };
  patches = [
    (fetchpatch {
      url = "https://github.com/example-org/example/commit/abcdef.patch";
      hash = "sha256-PATCHPATCHPATCHPATCHPATCHPATCHPATCHPA=";
    })
  ];
  meta = {
    license = lib.licenses.mit;
  };
}
```

Local patches (`./patches/foo.patch`) are **path inputs**, not FODs. Remote `fetchpatch` is a FOD—second hash to maintain.

Example C — intentional mismatch

```
src = fetchurl {
  url = "https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz";
  hash = lib.fakeHash;
};
```

First build prints the real hash; paste it back.

Example D — flake wiring

```
{
  description = "FOD lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs { inherit system; };
    in {
      packages.${system}.hello-fod = pkgs.callPackage ./pkgs/hello-fod.nix { };
      packages.${system}.default = self.packages.${system}.hello-fod;
    };
}
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/fod`

Step 1 — Prefetch a known tarball

```
mkdir -p ~/lab/nixos/packaging/fod/pkgs
cd ~/lab/nixos/packaging/fod
nix store prefetch-file \
  https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz
```

Record the SRI hash.

Step 2 — Package from that tarball

Write `pkgs/hello-fod.nix` with `fetchurl` + your hash. Export from a flake:

```
nix build .#hello-fod -L
./result/bin/hello
```

Step 3 — Force a mismatch

Change one character of the hash; rebuild; capture the error. Restore the correct hash.

Step 4 — GitHub fetcher

Pick a **tiny** tagged GitHub project (or a single-file tool). Package with `fetchFromGitHub`:

```
src = fetchFromGitHub {
  owner = "…";
  repo = "…";
  rev = "vX.Y.Z";
  hash = lib.fakeHash;
};
```

Iterate until the hash stabilizes. Prefer a project with a clear install (copy binary/script).

Step 5 — Patch as FOD and as path

Add a local patch first:

```
patches = [ ./patches/fix-typo.patch ];
```

Then replace with `fetchpatch` for one upstream commit if available—note the second hash.

Step 6 — Document the trust boundary

In `NOTES.md`:

1. Where did bytes come from?
2. Who could change them?
3. What does updating `hash` mean operationally?
4. Difference between updating `flake.lock` and updating `src.hash`.

Step 7 — Offline check

After populate:

```
nix build .#hello-fod --offline
```

If it works, your FOD is already in the store; purity is about **definition**, not "never used network once."

Step 8 — Submodules experiment (optional)

Find a tiny repo with a submodule **or** skip with a written note on why `fetchSubmodules` changes hashes.

Exercises

1. Prefetch the same URL twice; confirm hash identity.
2. Change only the `name` of a `fetchurl` (if supported) and observe whether the store path changes—document findings.
3. Write a one-paragraph runbook: "upstream re-tagged v1.2.3 with different bytes."
4. Compare `nix flake prefetch` output to `fetchFromGitHub` hash for the same rev—are they interchangeable? (Often related but not always drop-in; note what you observe.)
5. List every FOD in one of your language packages (src + vendor/cargo/npm).

Common gotchas

Symptom / mistake	What to do
Hash mismatch after "nothing changed"	Upstream re-released same tag; pin commit SHA
`fetchFromGitHub` hash differs on Darwin vs Linux	Unusual for pure archives; check `name`, filters, or LFS
Using empty `sha256`	Prefer `lib.fakeHash` for clarity; never commit fakes
Network error inside normal derivation	Move download to fetcher

Symptom / mistake	What to do
Submodules missing	`fetchSubmodules = true` (+ hash changes)
Huge monorepo as `src`	Sparse checkout / release tarball / `sourceRoot`
Private repo fetch	Mirror + FOD; careful with credentials
Patched inside FOD builder	Move patches to normal derivation
Confused flake lock with src hash	Update the pin that actually feeds the package
`leaveDotGit = true` nondeterminism	Avoid unless required; pin carefully

Checkpoint

- ☐ Explain FOD vs normal derivation in one paragraph
- ☐ Packaged something with `fetchurl` and correct SRI hash
- ☐ Packaged something with `fetchFromGitHub`
- ☐ Demonstrated and fixed a hash mismatch deliberately
- ☐ Notes on trust when updating hashes
- ☐ Offline rebuild succeeded for a cached FOD
- ☐ Distinguishes flake input pins from package `src` hashes

Commit

```
git add .
git commit -m "packaging: fetchers and FOD hash workflow"
```

Write three personal gotchas before continuing to language ecosystems.

Language ecosystems (part A)

Language ecosystems (part A)

Goal: Package a real tool in **your primary language** using the nixpkgs language builder (`buildGoModule`, `buildRustPackage`, `buildPythonApplication`, …)—not raw `stdenv` unless the ecosystem forces it.

> Tip
>
> Pick **one** primary ecosystem in this chapter. Part B is a second, smaller transfer exercise. Depth beats collecting half-broken scaffolds for five languages.

Why it matters

`stdenv` taught phases. Production packaging is almost always **ecosystem helpers**: they know how to vendor deps, set `LDFLAGS`, wrap interpreters, and place modules. Mastering one helper end-to-end is how packaging craft becomes useful at work.

Raw `stdenv` for a Go module graph is possible and miserable. Use the builder; understand that it is still phases + FODs underneath.

Theory 1 — How language builders relate to stdenv

```
buildGoModule / buildRustPackage / buildPythonApplication / …

  stdenv.mkDerivation (+ setup hooks)

  phases + FODs (module vendor hashes, cargoLock, …)
```

You still have `pname`, `version`, `meta`, patches, and hooks—but **fetching language deps** is specialized.

Ecosystem	Primary builder (nixpkgs)	Extra hash / lock concept
Go	`buildGoModule`	`vendorHash`
Rust	`rustPlatform.buildRustPackage`	`cargoHash` / `cargoLock`

Ecosystem	Primary builder (nixpkgs)	Extra hash / lock concept
Python	`buildPythonApplication` / `buildPythonPackage`	`pyproject` / `fetchPypi` / overrides
Node	`buildNpmPackage` / `importNpmLock`	`npmDepsHash`
Zig	`buildZigPackage` (where available) / stdenv	Fetch deps carefully
C/C++	`stdenv` + cmake/meson	Usually no vendor hash

Shared skeleton

```
src FOD → language-dep FOD → build/install hooks → $out/bin → meta
```

When stuck, ask first: **which hash is wrong—source or language deps?**

Theory 2 — Go path (`buildGoModule`)

```
{ lib, buildGoModule, fetchFromGitHub }:
buildGoModule rec {
  pname = "hello-go";
  version = "0.1.0";

  src = fetchFromGitHub {
    owner = "you";
    repo = "hello-go";
    rev = "v${version}";
    hash = "sha256-…";
  };

  vendorHash = "sha256-…"; # FOD of the vendored module graph

  ldflags = [
    "-s" "-w"
    "-X main.version=${version}"
  ];

  meta = {
    description = "Tiny Go greeter";
    license = lib.licenses.mit;
    mainProgram = "hello-go";
  };
}
```

Local module (lab-friendly)

```
{ lib, buildGoModule }:
buildGoModule {
  pname = "hello-go";
  version = "0.1.0";
  src = ../src/hello-go;
  vendorHash = null; # only if no external modules
  meta = {
    mainProgram = "hello-go";
    license = lib.licenses.mit;
  };
}
```

If you import modules, use a real `vendorHash`.

`vendorHash` workflow

1. Set `vendorHash = lib.fakeHash;` (or `null` only if truly module-free).
2. Build; read `got:` hash.
3. Paste; rebuild.
4. When `go.mod` / `go.sum` change, **vendorHash changes**.

```
nix build .#hello-go -L
```

Useful knobs

Attr	Purpose
`subPackages`	Build only `./cmd/foo`
`proxyVendor`	Vendor via module proxy inside FOD
`doCheck = false`	Skip tests if network-bound (prefer fixing tests)
`env.CGO_ENABLED = "0"`	Pure Go when appropriate
`ldflags`	Version injection, strip
`tags`	Build tags
`modRoot`	Multi-module monorepos

Example `src/hello-go`

```
// src/hello-go/main.go
package main

import (
  "fmt"
  "os"
)

var version = "dev"

func main() {
  if len(os.Args) > 1 && os.Args[1] == "--version" {
    fmt.Println(version)
    return
  }
  fmt.Println("hello-go")
}
```

```
// src/hello-go/go.mod
module example.com/hello-go

go 1.22
```

Theory 3 — Rust path (`buildRustPackage`)

```
{ lib, rustPlatform, fetchFromGitHub }:
rustPlatform.buildRustPackage rec {
  pname = "hello-rs";
  version = "0.1.0";

  src = fetchFromGitHub {
    owner = "you";
    repo = "hello-rs";
    rev = "v${version}";
    hash = "sha256-…";
  };

  cargoHash = "sha256-…";

  meta = {
```

```
    description = "Tiny Rust greeter";
    license = lib.licenses.mit;
    mainProgram = "hello-rs";
  };
}
```

Alternatively lock from a committed `Cargo.lock`:

```
cargoLock = {
  lockFile = ./Cargo.lock;
  # outputHashes = { "foo-0.1.0" = "sha256-…"; }; # for git deps
};
```

Follow current nixpkgs examples for **26.05** if `cargoHash` vs git deps bit you.

Git dependencies in Cargo

Vendoring git crates often needs `cargoLock.outputHashes` for each git source. Expect deep debugging the first time—same skills as the debug builds chapter.

Useful knobs

Attr	Purpose
`cargoBuildFlags`	Extra flags
`cargoTestFlags`	Test control
`buildAndTestSubdir`	Workspace member
`doCheck`	Run tests
`auditable`	Supply-chain related (when available)

Theory 4 — Python path (application vs library)

```
{ lib, python3Packages, fetchPypi }:
python3Packages.buildPythonApplication rec {
  pname = "hello-py";
  version = "0.1.0";
  pyproject = true;

  src = fetchPypi {
    inherit pname version;
    hash = "sha256-…";
  };
```

```
  build-system = with python3Packages; [ setuptools ];
  dependencies = with python3Packages; [ click ];

  meta = {
    description = "Tiny Click CLI";
    license = lib.licenses.mit;
    mainProgram = "hello-py";
  };
}
```

Local app:

```
{ lib, python3Packages }:
python3Packages.buildPythonApplication {
  pname = "hello-py";
  version = "0.1.0";
  pyproject = true;
  src = ../src/hello-py;
  build-system = with python3Packages; [ setuptools ];
  meta = {
    mainProgram = "hello-py";
    license = lib.licenses.mit;
  };
}
```

Builder	When
`buildPythonApplication`	CLI / end-user tool on PATH
`buildPythonPackage`	Library for others to depend on

Prefer **existing** `python3Packages.*` deps over vendoring wheels by hand. Align interpreters (`python311` vs `python312`) across the graph.

Standalone scripts with `pkgs.writers.writePython3Bin`

Not every Python tool warrants a full project tree with `pyproject.toml`, `setup.py`, and `buildPythonApplication`. For single-file utilities, scrapers, audio processing scripts, or maintenance glue, creating a packaging boilerplate adds friction.

Moreover, real-world scripts frequently have **two distinct layers of dependencies**:

1. **Python packages:** e.g., `requests`, `trafilatura`.
2. **System/host CLI utilities:** external binaries invoked via subprocesses, e.g. `aplay` (from `alsa-utils`), `ffmpeg`, or `ripgrep`.

Traditional Python tools (`pip`, `venv`, `pipx`, `uv`) only manage the first layer. They cannot manage host binaries like `aplay`, leaving scripts broken when run on systems lacking the underlying package. Furthermore, virtualenvs are notoriously fragile across system Python upgrades.

`pkgs.writers.writePython3Bin` solves both layers hermetically in a single Nix derivation:

```
# flake.nix (standalone Python script with Python and system dependencies)
{
  description = "Read articles aloud with Python and aplay";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs, ... }:
    let
      forAllSystems = f: nixpkgs.lib.genAttrs [ "x86_64-linux" "aarch64-linux" ] (system:
        f nixpkgs.legacyPackages.${system}
      );
    in {
      packages = forAllSystems (pkgs: {
        default = pkgs.writers.writePython3Bin "readarticle" {
          # Prefix PATH with system dependencies needed by subprocesses
          makeWrapperArgs = [
            "--prefix" "PATH" ":" (pkgs.lib.makeBinPath [ pkgs.alsa-utils ])
          ];

          # Python library dependencies made available to the interpreter
          libraries = with pkgs.python3Packages; [
            requests
            trafilatura
          ];
        } (builtins.readFile ./readarticle.py);
      });
    };
}
```

What makes `pkgs.writers.writePython3Bin` powerful:

- **Dual-layer closure:** `libraries` adds the Python packages to the script's `sys.path`. `makeWrapperArgs` wraps the resulting executable with `wrapProgram`, prefixing `PATH` with the exact Nix store paths for `alsa-utils` (`aplay`).
- **Unaware script code:** Inside `readarticle.py`, code calls `subprocess.run(["aplay", ...])` and `import trafilatura` as if running in a globally configured machine. No Nix-specific paths need to be hardcoded in Python.
- **Build-time linting:** `writePython3Bin` executes a linting pass (`pyflakes`) over the script during the build. If there are syntax errors or undefined variables, the derivation fails to build before anyone runs it. (Similarly, `writeBashBin` runs `shellcheck`).

- **Zero-install portability:** Anyone on Linux (NixOS, Fedora, Ubuntu, Arch) with Nix can run the script instantly without setting up a virtualenv or installing `alsa-utils`:

  ```
  nix run . -- https://example.com/article
  ```

pytest pattern (sketch)

```
nativeCheckInputs = with python3Packages; [ pytestCheckHook ];
# or pytest in checkPhase via hooks the builder provides
```

Theory 5 — Node (`buildNpmPackage`) sketch

```
{ lib, buildNpmPackage, fetchFromGitHub }:
buildNpmPackage rec {
  pname = "hello-node";
  version = "0.1.0";
  src = fetchFromGitHub {
    owner = "you";
    repo = "hello-node";
    rev = "v${version}";
    hash = "sha256-…";
  };
  npmDepsHash = "sha256-…";
  meta = {
    mainProgram = "hello-node";
    license = lib.licenses.mit;
  };
}
```

`package-lock.json` (or supported lock) must exist; hash tracks the npm dependency FOD. Lock format churn changes hashes—commit the lockfile.

Theory 6 — Version injection and `meta.mainProgram`

Ecosystem	Version injection idea
Go	`ldflags = [ "-X main.version=${version}" ];`
Rust	`env.VERGEN_*` or `clap` long_version via build.rs / env

Ecosystem	Version injection idea
Python	single-source version in package metadata
Node	`package.json` version; avoid drift with Nix `version`

Always set:

```
meta.mainProgram = "hello-go";
```

so `nix run` finds the binary without guessing.

Theory 7 — Tests and purity

Situation	Approach
Unit tests offline	Keep `doCheck = true`
Tests need network	Patch, mock, or disable with a **comment**
Tests need golden files with timestamps	Fix tests, don't disable sandbox
Integration tests need services	Prefer `nixosTest` later; not every unit suite

Disabling checks forever without a ticket is how bitrot lands in production images.

Worked example — flake with Go tool + check

```
{
  description = "language ecosystem A lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" ];
      forAllSystems = nixpkgs.lib.genAttrs systems;
    in {
      packages = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; }; in {
          default = pkgs.callPackage ./pkgs/hello-go.nix { };
          hello-go = pkgs.callPackage ./pkgs/hello-go.nix { };
        });
```

```
      apps = forAllSystems (system: {
        default = {
          type = "app";
          program = "${self.packages.${system}.default}/bin/hello-go";
        };
      });

      checks = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; }; in {
          hello-go-version = pkgs.runCommand "hello-go-version" {
            nativeBuildInputs = [ self.packages.${system}.default ];
          } ''
            hello-go --version | tee "$out"
          '';
        });

      devShells = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; }; in {
          default = pkgs.mkShell {
            packages = [
              pkgs.go
              pkgs.gopls
              self.packages.${system}.default
            ];
          };
        });
    };
}
```

Adapt builders and binary names if your primary language is not Go.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/eco-a`

Step 1 — Choose primary language

Write the choice in `NOTES.md`: Go / Rust / Python / Node / other. Stick to it for all steps.

Step 2 — Minimal upstream project

Either:

- Create a tiny local project under `src/`, **or**
- Fetch a small tagged tool you actually use.

Requirements: builds offline once vendored; one CLI entrypoint; license known.

Step 3 — First package expression

Use the matching builder. Start with `lib.fakeHash` / fake vendor hashes as appropriate.

```
nix build .#default -L
```

Iterate until green.

Step 4 — Wire `meta.mainProgram` and run

```
nix run .#
# or
./result/bin/<name> --help
```

Step 5 — Version injection

Pass version via `ldflags` (Go), env/build.rs (Rust), or package metadata (Python/Node). Prove `tool --version` matches derivation `version` where feasible.

Step 6 — Tests

Enable package checks if feasible. If tests need network, patch or disable **with a comment** explaining why—not silence.

Step 7 — Consume from a devShell

```
nix develop
which <tool>
```

Optional: add to a NixOS `environment.systemPackages` via flake package output.

Step 8 — Break the vendor/cargo/npm hash

Edit a dependency version or lock line; rebuild; observe hash mismatch; restore or update intentionally.

Step 9 — Closure glance

```
nix path-info -rS ./result | sort -k2 -n | tail
nix path-info -Sh ./result
```

Note interpreter vs pure binary closure differences.

Exercises

1. Map your builder's phases back to stdenv names in a table.
2. Produce a second output binary with `subPackages` or workspace members (if applicable).
3. Add a `passthru.tests` or flake `checks` that fails if `--version` drifts.
4. Cross-compile awareness: read whether your builder sets `GOOS`/`CARGO_BUILD_TARGET`—document only.
5. Find the same tool in nixpkgs (`nix edit nixpkgs#…`) and list three techniques they use that you skipped.

Common gotchas

Symptom / mistake	What to do
`vendorHash` / `cargoHash` / `npmDepsHash` mismatch	Update after intentional lock change; never ignore silently
Go build tries network in buildPhase	Modules must be vendored via FOD; check `proxyVendor` / vendor dir
Rust git dependency hash missing	Add `outputHashes` for the git crate
Python module import works in tests but CLI missing	Use `buildPythonApplication` + `meta.mainProgram`
Node package missing lockfile	Generate supported lock; don't hand-wave `node_modules`
`mainProgram` wrong	Fix meta; `nix run` depends on it for default bin
CGO / shared libs surprise	Set `CGO_ENABLED`, add `buildInputs` for libs
Building entire monorepo	`subPackages` / workspace members / flags

Symptom / mistake	What to do
Mixed python3 versions	Align package sets
Disabled checks permanently	Ticket + comment or fix

Checkpoint

- ☐ Primary language chosen and recorded
- ☐ Package builds with official language helper
- ☐ Vendor/cargo/npm hash workflow demonstrated
- ☐ `nix run` or `./result/bin` works
- ☐ Version string controlled from Nix where possible
- ☐ Notes on how this maps back to stdenv phases
- ☐ At least one check or test story
- ☐ Closure size noted

Commit

```
git add .
git commit -m "packaging: language ecosystem A package"
```

Write three personal gotchas before part B.

Language ecosystems (part B)

Language ecosystems (part B)

Goal: Package a **second** language ecosystem tool so the *patterns* transfer: FODs for language deps, helpers vs raw stdenv, and how overrides differ across builders.

Why it matters

At work you will not stay monoglot. The point is not fluency in five builders—it is recognizing the **same skeleton**:

```
src FOD → language-dep FOD → build/install hooks → $out/bin → meta
```

Part A was depth. This chapter is **transfer** with a deliberately smaller scope: one binary, clear version, known license.

Theory 1 — Pattern map across ecosystems

Concern	Go	Rust	Python	Node
Source pin	`src.hash`	`src.hash`	`src.hash` / fetchPypi	`src.hash`
Dep pin	`vendorHash`	`cargoHash` / lock	package set / frozen	`npmDepsHash`
Builder	`buildGoModule`	`buildRustPackage`	`buildPython*`	`buildNpmPackage`
Skip tests	`doCheck`	`doCheck` / cargo flags	pytest hooks	often build-only
Binary name	module main	crate bin	`meta.mainProgram`	`meta.mainProgram`
Cross	`GOOS`/`GOARCH`	target triples care	often pure	often pure JS
Override flavor	`overrideAttrs`	`overrideAttrs`	`overridePythonAttrs`	`overrideAttrs`

When stuck, ask: *which hash is wrong—source or language deps?*

Theory 2 — Choose B deliberately

If part A was…	Good part B pick	Why
Go	Python CLI or Rust	Different dep story
Rust	Go small tool	Faster vendor cycle
Python	Node or Go	Leaves interpreter wrapping
Node	Python or Go	Sees compiled vs interpreted

Avoid starting a second multi-crate workspace. **One binary, clear version, known license.**

Transfer questions (answer in notes)

1. What is the second FOD field named?
2. Where does the binary land under `$out`?
3. What is the fastest hash-mismatch workflow for this builder?
4. Would I pick this ecosystem for internal CLIs at work? Why?

Theory 3 — Wrapping and runtime deps

Interpreted ecosystems often need **wrappers**:

```
{ stdenv, lib, python3, makeWrapper }:
stdenv.mkDerivation {
  pname = "tool";
  version = "0.1.0";
  dontUnpack = true;
  nativeBuildInputs = [ makeWrapper ];
  installPhase = ''
    runHook preInstall
    mkdir -p $out/bin
    cat > $out/bin/tool <<'EOF'
    #!/bin/sh
    echo tool
    EOF
    chmod +x $out/bin/tool
    wrapProgram $out/bin/tool \
      --prefix PATH : ${lib.makeBinPath [ python3 ]}
    runHook postInstall
```

```
  '';
  meta = {
    mainProgram = "tool";
    license = lib.licenses.mit;
  };
}
```

Situation	Prefer
Official language builder exists	Use it (`buildGoModule`, `buildPythonApplication`, etc.)
Single script / glue needing runtime deps	`pkgs.writers.*` (`writePython3Bin`, `writeBashBin`, …)
Need one env var or PATH sibling on custom binary	`wrapProgram` / `makeWrapper`
Need whole tree of modules	Builder's dependency graph
Need different interpreter	Override package set / python version

Python applications usually wrap `PYTHONPATH` via the builder—do not reimplement unless you must.

High-level script writers: `pkgs.writers`

Writing raw `stdenv.mkDerivation` boilerplate with `nativeBuildInputs = [ makeWrapper ]` and manual `wrapProgram` calls is repetitive for standalone tools. nixpkgs provides **`pkgs.writers`** (`pkgs/build-support/writers/` in nixpkgs source) to author self-contained script binaries with wrapped dependencies and built-in validation.

1. Language-specific writers

Pre-packaged writers exist for common scripting and compiled languages:

- `pkgs.writers.writePython3Bin "name" { libraries = [...]; makeWrapperArgs = [...]; } (builtins.readFile ./script.py)`: passes Python packages to `PYTHONPATH`, prefixes `PATH` for system utilities, and runs a build-time syntax check via `pyflakes`.
- `pkgs.writers.writeBashBin "name" { makeWrapperArgs = [...]; } ''...''`: writes an executable Bash script and runs it through `shellcheck` during the build, failing fast on syntax or quoting issues.
- `pkgs.writers.writeJSBin`, `pkgs.writers.writeRustBin`, `pkgs.writers.writeHaskellBin`: corresponding turnkey writers for other languages.

2. Generic interpreted scripts: `pkgs.writers.makeScriptWriter`

When targeting an interpreted language that lacks a ready-made `write<Lang>Bin` helper (such as Emacs Lisp, Guile Scheme, Perl, or R), use the lower-level `makeScriptWriter`. If the interpreter

follows the `withPackages` convention, runtime language dependencies and system binaries compose directly:

```
pkgs.writers.makeScriptWriter {
  # System binary dependencies made available on PATH
  makeWrapperArgs = [
    "--prefix" "PATH" ":" (pkgs.lib.makeBinPath [ pkgs.alsa-utils ])
  ];

  # The interpreter with its language libraries
  interpreter = "${(pkgs.emacs.withPackages (epkgs: [ epkgs.org-mode ]))}/bin/emacs --script
} "/bin/render-notes" (builtins.readFile ./render.el);
```

The output path (`/bin/render-notes`) is package-relative, placed safely under `$out/bin/render-notes`.

3. Generic compiled scripts: `pkgs.writers.makeBinWriter`

For compiled languages without a turnkey helper (e.g., Ada, Pascal, or specialized C tools), `makeBinWriter` takes a compilation script and input file path:

```
pkgs.writers.makeBinWriter {
  makeWrapperArgs = [
    "--prefix" "PATH" ":" (pkgs.lib.makeBinPath [ pkgs.coreutils ])
  ];

  # Nix runs compileScript in the build sandbox
  # $contentPath contains the source code, $out is the destination binary
  compileScript = ''
    ${pkgs.gnat}/bin/gnatmake -o $out $contentPath
  '';
} "/bin/ada-tool" (builtins.readFile ./main.adb);
```

Where to find them

Because `pkgs.writers` documentation in the Nixpkgs manual is minimal, the authoritative reference is the nixpkgs source code itself at `pkgs/build-support/writers/default.nix`. Reading that file reveals every supported argument (`makeWrapperArgs`, `libraries`, linter toggles) and how wrappers are assembled.

Theory 4 — Overrides preview

Deep dive lives in the patching chapter; know these exist:

```
# Change flags without forking the whole expression
helloGo.overrideAttrs (old: {
  ldflags = (old.ldflags or []) ++ [ "-X main.extra=eco-b" ];
})
```

```
# Python package override inside python3.pkgs
python3.pkgs.requests.overridePythonAttrs (old: {
  doCheck = false;
})
```

```
# Node / generic
helloNode.overrideAttrs (old: {
  postPatch = (old.postPatch or "") + ''
    substituteInPlace package.json --replace-fail '"private": true' '"private": false'
  '';
})
```

Composition rules matter later—for now, confirm your ecosystem B package can be lightly tweaked without a full fork.

Theory 5 — Cross-ecosystem FOD discipline

Whatever language B you chose, fetchers obey the same rules as the fetchers chapter:

Rule	Why
Pin URL + hash	Reproducible
Vendor offline where possible	CI without network flakiness
Prefer language-native lock → Nix conversion carefully	Don't invent hashes by hand when tooling exists
Never commit `fakeHash`	CI will fail or worse, someone "fixes" blindly

```
# Always capture both hashes when documenting
echo "src.hash=…"
echo "langDep.hash=…"   # vendorHash / cargoHash / npmDepsHash
```

Theory 6 — Runtime vs build-time closure

```
nativeBuildInputs  → tools for build (compilers, make)
buildInputs        → linked/used at build; may propagate
propagated*        → infect dependents (use sparingly)
```

A package that "works in `nix develop`" but fails as `nix build` often mixed shell conveniences with missing `buildInputs`.

Ecosystem	Common closure weight
Pure Go (`CGO_ENABLED=0`)	Small
Rust release bin	Small–medium
Python app	Interpreter + site-packages
Node CLI	Node runtime + modules (or bundled)

```
nix path-info -Sh ./result
nix path-info -rS ./result | sort -k2 -n | tail -n 20
```

Theory 7 — Dual interface pattern

Make both A and B print something comparable so checks share structure:

```
eco-a: hello from ecosystem A
eco-b: hello from ecosystem B
```

Or both support `--version` with Nix-injected versions. Unified checks then look like:

```
both-run = pkgs.runCommand "both-run" {
  nativeBuildInputs = [ a b ];
} ''
  ${a}/bin/${a.meta.mainProgram} --version
  ${b}/bin/${b.meta.mainProgram} --version
  touch $out
'';
```

Worked example — dual packages in one flake

```
{
  description = "language ecosystems A+B lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" ];
      forAllSystems = nixpkgs.lib.genAttrs systems;
    in {
      packages = forAllSystems (system:
        let pkgs = import nixpkgs { inherit system; }; in {
          lang-a = pkgs.callPackage ./pkgs/lang-a.nix { };
          lang-b = pkgs.callPackage ./pkgs/lang-b.nix { };
          default = self.packages.${system}.lang-b;
        });

      checks = forAllSystems (system:
        let
          pkgs = import nixpkgs { inherit system; };
          a = self.packages.${system}.lang-a;
          b = self.packages.${system}.lang-b;
        in {
          both-run = pkgs.runCommand "both-run" {
            nativeBuildInputs = [ a b ];
          } ''
            "${a}/bin/${a.meta.mainProgram}" --help >/dev/null 2>&1 \
              || "${a}/bin/${a.meta.mainProgram}" >/dev/null
            "${b}/bin/${b.meta.mainProgram}" --help >/dev/null 2>&1 \
              || "${b}/bin/${b.meta.mainProgram}" >/dev/null
            touch $out
          '';
        });
    };
}
```

Tiny Python app (if A was compiled)

```
{ lib, python3Packages }:
python3Packages.buildPythonApplication {
  pname = "eco-b-greet";
  version = "0.1.0";
  pyproject = true;
  src = ../src/eco-b-greet;
```

```
  build-system = with python3Packages; [ setuptools ];
  meta = {
    description = "Ecosystem B greet";
    license = lib.licenses.mit;
    mainProgram = "eco-b-greet";
  };
}
```

```
# src/eco-b-greet/pyproject.toml
[project]
name = "eco-b-greet"
version = "0.1.0"
requires-python = ">=3.11"
[project.scripts]
eco-b-greet = "eco_b_greet:main"
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"
```

```
# src/eco-b-greet/eco_b_greet.py
def main():
    print("hello from ecosystem B")

if __name__ == "__main__":
    main()
```

Tiny Node CLI (alternative)

```
{ lib, buildNpmPackage }:
buildNpmPackage {
  pname = "eco-b-node";
  version = "0.1.0";
  src = ../src/eco-b-node;
  npmDepsHash = "sha256-…";
  meta = {
    mainProgram = "eco-b-node";
    license = lib.licenses.mit;
  };
}
```

Tiny Go (if A was Python)

```
{ lib, buildGoModule }:
buildGoModule {
  pname = "eco-b-go";
  version = "0.1.0";
  src = ../src/eco-b-go;
  vendorHash = null;
  meta = {
    mainProgram = "eco-b-go";
    license = lib.licenses.mit;
  };
}
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/eco-b`
(or extend the ecosystem A flake)

Step 1 — Import or copy the part A package

Keep `lang-a` buildable so you compare logs side by side.

Step 2 — Scaffold smaller B project

Budget: under ~50 lines of application code. No microservices.

Step 3 — Package with the B builder

```
nix build .#lang-b -L
nix run .#lang-b
```

Record every hash you had to set (src + language deps).

Step 4 — Comparison table (required)

In `NOTES.md`:

Question	Lang A	Lang B
Builder name		
Source hash field		
Dep hash field		
Time to first success		
Hardest error		
Where binary lands		
Closure (nar size)		
Would I use this at work for CLIs?		

Step 5 — Unified check

Add a flake `checks.*` that smoke-tests both tools. `nix flake check`.

Step 6 — Optional same interface

Make both print a comparable line so a shell script can assert either.

Step 7 — Closure glance

```
nix path-info -rS ./result | sort -k2 -n | tail
nix path-info -Sh ./result
```

Note which ecosystem pulled a heavier closure and why (interpreter? compiler runtime? libc?).

Step 8 — Hash bump drill

1. Intentionally change a src rev or use `lib.fakeHash`
2. Run build; capture the `got:` hash
3. Paste correct hash; rebuild green
4. Write the exact error class name in notes

Step 9 — Light override

Apply `overrideAttrs` or `overridePythonAttrs` to flip `doCheck` or add an `ldflags` entry. Rebuild once.

Exercises

1. Draw the FOD graph for A and B (boxes and arrows).
2. Time cold build vs warm cache for each (rough wall clock).
3. Document one override API difference between the two builders.
4. Try `nix run` without `mainProgram` set—observe failure mode—then fix.
5. Write a recommendation: "default internal CLI language for our team" with two bullets of rationale from your measurements.

Common gotchas

Symptom / mistake	What to do
Trying to "finish" a second huge app	Shrink scope; transfer patterns, not product
Mixing package sets (python311 vs python312)	Align interpreters
Assuming `vendorHash = null` always works	Only pure module-free trees
`npmDepsHash` churn from lock format	Commit lock; pin npm
Comparing wall times unfairly	Same machine, warm vs cold cache noted
Forgetting `meta.license`	Set it; habit for nixpkgs-shaped work
Unified check hardcodes wrong bin names	Use `meta.mainProgram`
Porting Go ldflags habits to Python	Use each ecosystem's idioms
Skipping comparison table	Required—this chapter is about transfer

---

Checkpoint

- ☐ Second ecosystem package builds and runs
- ☐ Comparison table filled honestly
- ☐ Both A and B reachable from one flake
- ☐ At least one combined check

- ☐ Closure notes captured
- ☐ Hash bump drill completed
- ☐ Light override demonstrated

Commit

```
git add .
git commit -m "packaging: language ecosystem B + comparison"
```

Write three personal gotchas before patching and overrides.

Patching and overrides

Patching and overrides

Goal: Fix or pin packages with `overrideAttrs`, `patches`, and overlays—without maintaining a permanent hard fork of nixpkgs.

Why it matters

Upstream will be wrong for your window of time: a CVE pin, a broken release, a flag your hardware needs. The professional skill is **minimal, reviewable deltas** that compose, not copy-pasting half of nixpkgs into your flake.

Theory 1 — Three layers of change

Mechanism	Scope	Reuse	Maintenance
`override` / `overrideAttrs`	One package instance	Call-site	Low–medium
Overlay	Package set view for a flake/host	Shared modules	Medium
Fork / `fetch` your tree	Anything	High control	High—avoid until needed

```
nixpkgs  →  overlay(s)  →  pkgs.yourTool
                ↑
          overrideAttrs / patches
```

Overlays reshape `pkgs` for everyone who shares that package set. Call-site overrides affect one reference. Choose the smallest scope that works.

Theory 2 — `overrideAttrs`

```
pkgs.hello.overrideAttrs (old: {
  patches = (old.patches or []) ++ [ ./patches/hello-msg.patch ];
  postPatch = (old.postPatch or "") + ''
    echo "packaging-craft" >> README
  '';
})
```

Form	Notes
`overrideAttrs (old: { … })`	Preferred; see previous attrs
`overrideAttrs { … }`	Easy to drop old patches by mistake
`package.override { stdenv = …; }`	Change **callPackage** args (dependencies), not phases

`override` vs `overrideAttrs`

```
# Change dependencies / function args
pkgs.ffmpeg.override { withXcb = true; }

# Change derivation attributes / phases
pkgs.ffmpeg.overrideAttrs (old: {
  configureFlags = (old.configureFlags or []) ++ [ "--enable-something" ];
})
```

Language builders often expose both (e.g. `overridePythonAttrs`).

Composing safely

```
old.patches or []           # never assume patches exists
(old.postPatch or "") + ''  # string concat for hooks
(old.nativeBuildInputs or []) ++ [ pkgs.foo ]
```

Dropping upstream patches by accident is a classic security and build break.

Theory 3 — Patches as first-class inputs

```
stdenv.mkDerivation {
  # …
  patches = [
    ./patches/0001-fix-build-on-musl.patch
    (fetchpatch {
      url = "https://github.com/org/proj/commit/abc123.patch";
      hash = "sha256-…";
    })
  ];
}
```

Practice	Why
Numbered filenames	Apply order is list order
Keep patches small	Rebase pain scales with hunk size
Prefer upstream PRs	Local patches rot on every release
Test without patch once	Confirm the patch is still required
Record expiry	“Remove when nixpkgs rev …”

`patchPhase` applies these automatically; fail logs show hunk rejects.

Generating a patch

```
# Against unpacked sources or a git checkout of upstream
diff -u a/file b/file > patches/0001-msg.patch
# or
git format-patch -1 HEAD --stdout > patches/0001-msg.patch
```

Refresh patches when you bump `src.rev`—do not force with weaker reject settings as policy.

Theory 4 — Overlay composition

```
# overlays/hello-patch.nix
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or []) ++ [ ../patches/hello-msg.patch ];
  });
}
```

```
# flake.nix fragment
pkgs = import nixpkgs {
  inherit system;
  overlays = [
    (import ./overlays/hello-patch.nix)
    (import ./overlays/my-tool.nix)
  ];
};
```

Order matters

```
overlay1 final:prev → overlay2 final:prev → …
```

Later overlays see earlier results via the fixed-point.
Use **`final`** when depending on other packages you also overlay; use **`prev`** to modify the incoming package without recursion traps.

```
final: prev: {
  myApp = prev.myApp.overrideAttrs (_: { /* … */ });
  # depend on overlaid hello:
  myAppWrapped = final.hello;
}
```

Infinite recursion patterns

```
# BAD
final: prev: {
  hello = final.hello.overrideAttrs (…); # often loops
}

# GOOD
final: prev: {
  hello = prev.hello.overrideAttrs (…);
}
```

NixOS / HM consumption

```
nixpkgs.overlays = [
  (import ./overlays/hello-patch.nix)
];
```

Ensure the **same** `pkgs` instance that builds your host sees the overlay. A second `import nixpkgs { }` without overlays silently ignores your work.

Theory 5 — Pinning and “don’t rebuild the world”

Goal	Technique
One package newer than channel	Overlay with `fetchFromGitHub` src bump + hashes
One package older (pin)	Overlay src to known-good rev; note security debt
Only your app sees the change	Don’t put overlay in global NixOS; inject via `callPackage` scope
Temporary CI fix	Overlay in flake only; ticket to drop it

```
final: prev: {
  curl = prev.curl.overrideAttrs (old: rec {
    version = "8.11.1";
    src = prev.fetchurl {
      url = "https://curl.se/download/curl-${version}.tar.bz2";
      hash = "sha256-…";
    };
  });
}
```

Bumping shared libraries can rebuild huge closures—check before celebrating:

```
nix why-depends .#myService .#openssl
nix build .#myService --dry-run 2>&1 | head
```

Theory 6 — Scoped injection without global overlays

```
# Only this package sees a special dependency
pkgs.callPackage ./pkgs/mytool.nix {
  openssl = pkgs.openssl.override { … };
}
```

```
# Overlay adds your package without changing world openssl
final: prev: {
  mytool = final.callPackage ../pkgs/mytool.nix { };
}
```

Prefer **narrow** scope for experimental flags; global overlays for org-wide pins with review.

Theory 7 — Language-specific override helpers

Ecosystem	Helper	Example use
Generic	`overrideAttrs`	phases, patches, env
Generic	`override`	`callPackage` parameters
Python	`overridePythonAttrs`	`doCheck`, deps
Haskell	`overrideCabal` (ecosystem)	when you live there
Node	often `overrideAttrs`	`npmDepsHash`, patches

```
python3.pkgs.numpy.overridePythonAttrs (old: {
  doCheck = false;
})
```

Worked examples

Example A — local patch on `hello`

```
# patches/hello-msg.patch
--- a/src/hello.c
+++ b/src/hello.c
@@ -59,7 +59,7 @@
-  printf ("Hello, world!\n");
+  printf ("Hello, packaging craft!\n");
```

(Exact hunks must match the version you patch—regenerate against real sources.)

```
# overlays/hello-craft.nix
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or []) ++ [ ../patches/hello-msg.patch ];
  });
}
```

Example B — overlay adding your ecosystem A package

```
final: prev: {
  hello-go = prev.callPackage ../pkgs/hello-go.nix { };
}
```

Example C — `packageOverrides` legacy note

Older blogs use `packageOverrides`. Prefer **overlays** in modern flakes; recognize the old name when reading.

Example D — flake export

```
{
  description = "overrides lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs {
        inherit system;
        overlays = [ (import ./overlays/hello-craft.nix) ];
      };
    in {
      packages.${system}.hello = pkgs.hello;
      packages.${system}.hello-callsite =
        (import nixpkgs { inherit system; }).hello.overrideAttrs (old: {
          patches = (old.patches or []) ++ [ ./patches/hello-msg.patch ];
        });
    };
}
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/overrides`

Step 1 — Baseline

```
nix build nixpkgs#hello -o baseline-hello
./baseline-hello/bin/hello
```

Step 2 — Write a patch

Create a minimal patch (message change or README touch). If patching C is painful on your platform, patch a script package you own from the stdenv chapter instead.

Step 3 — `overrideAttrs` only (no overlay)

```
pkgs.hello.overrideAttrs (old: {
  patches = (old.patches or []) ++ [ ./patches/hello-msg.patch ];
})
```

```
nix build .#hello-patched -L
./result/bin/hello
```

Step 4 — Promote to overlay

Move the same change to `overlays/…`. Import overlay into flake `pkgs`. Confirm drv path differs from unpatched when appropriate.

Step 5 — Stack two overlays

1. Overlay patches `hello`
2. Overlay adds `myTool` depending on `final.hello`

Prove `myTool` sees the patched hello if it references it (even a trivial wrap).

Step 6 — Dependency override

Pick a package with a boolean flag (choose something not enormous if you can):

```
pkgs.ffmpeg_7.override { /* one flag - or smaller package */ }
```

Build **or** evaluate and document why you stopped short if closure is huge.

Step 7 — Maintenance note

In `NOTES.md`: expiry conditions for the patch ("remove when nixpkgs rev …" / "upstream PR #").

Step 8 — Intentional drop of upstream patches

Temporarily set `patches = [ ./only-ours.patch ];` without `old.patches`. Observe risk; restore proper composition.

Exercises

1. Write a table: call-site override vs overlay vs nixpkgs PR—when each wins.
2. Refresh a patch against a newer `hello` version; document the conflict resolution.
3. Use `lib.optionals stdenv.isLinux` for a Linux-only patch.
4. Find a package in your flake that should **not** use a global overlay; refactor to call-site.
5. Draft a PR description as if upstreaming your patch (problem, approach, testing).

Common gotchas

Symptom / mistake	What to do
Patch rejects	Refresh patch against actual `src` version
Overlay silently not applied	Ensure same `pkgs` instance reaches the host/package
Infinite recursion in overlay	Use `prev` for the package you modify
Dropped upstream patches	Always `(old.patches or []) ++ [ ours ]`
Rebuilt half of nixpkgs	You overrode a low-level lib; narrow scope
Unmaintainable mega-patch	Split; upstream; or wait for release
Darwin vs Linux patch drift	Conditional patches with `lib.optionals`
Two nixpkgs imports, one overlay	Unify package set construction
Security pin without expiry note	Add ticket date and risk acceptance

---

Checkpoint

- ☐ `overrideAttrs` patch builds and changes behavior
- ☐ Overlay form works from flake
- ☐ Two overlays composed with known order

- ☐ Difference `override` vs `overrideAttrs` written down
- ☐ Patch expiry / upstream plan noted
- ☐ Avoided infinite recursion pattern
- ☐ Understood blast radius of low-level library overrides

Commit

```
git add .
git commit -m "packaging: patches, overrideAttrs, overlays"
```

Write three personal gotchas before multiple outputs.

Multiple outputs

Multiple outputs

Goal: Use multi-output derivations (`out`, `dev`, `man`, `doc`, …) to keep **runtime closures** small while preserving headers and debug assets for dependents.

Why it matters

A single `$out` that ships headers, man pages, static libs, and a 2 MB binary forces every dependent to download and GC the world. Multi-output packaging is how nixpkgs stays usable on laptops and CI runners.

For fleet images, multi-out is a **disk and bandwidth** control: servers need `out`, build machines need `dev`, humans sometimes need `man`.

Theory 1 — Outputs are separate store paths

```
/nix/store/hash-foo-1.0       ← out (runtime)
/nix/store/hash-foo-1.0-dev   ← dev (headers, pkg-config)
/nix/store/hash-foo-1.0-man   ← man
/nix/store/hash-foo-1.0-doc   ← html docs
```

Each is a different path. **References** between them are tracked so GC stays correct: runtime `out` should **not** reference `dev` if you did it right.

```
stdenv.mkDerivation {
  pname = "foo";
  version = "1.0";
  outputs = [ "out" "dev" "man" ];
  # …
}
```

Default if omitted: `outputs = [ "out" ];`.

First output is special

The first element of `outputs` is the default when someone depends on the package without selecting an output. Convention: **`out` first**.

```
outputs = [ "out" "dev" "man" ]; # good
outputs = [ "dev" "out" ];       # surprising defaults
```

Theory 2 — Conventional output roles

Output	Typical content	Who needs it
`out`	Binaries, shared libs, runtime data	Everyone running the tool
`dev`	Headers, `.pc`, static libs, cmake files	Compiling dependents
`lib`	Sometimes shared libs split from bins	Rare patterns / legacy
`man`	Man pages	Humans; optional on servers
`doc`	HTML/PDF docs	Optional
`info`	Texinfo	Optional
`devdoc`	Heavy API docs	Dev machines
`debug`	Separate debuginfo (with `separateDebugInfo`)	Debuggers

Environment variables in the build

Var	Points at
`$out`	First output (usually runtime)
`$dev`	Dev output path
`$man`	Man output
`$outputs`	List of outputs

Install into the correct root:

```
install -Dm755 foo $out/bin/foo
install -Dm644 foo.h $dev/include/foo.h
install -Dm644 foo.pc $dev/lib/pkgconfig/foo.pc
install -Dm644 foo.1 $man/share/man/man1/foo.1
```

Theory 3 — Why dependents get smaller

```
app depends on libfoo
  → needs libfoo.out (and its runtime closure)
  → does NOT need libfoo.dev unless building against headers
```

`nix path-info -r` on an application should **not** pull `*-dev` for every library if packaging is correct.

```
nix path-info -r ./result | rg -- '-dev$' || echo "no dev outputs in closure (good for runti
```

Reference leaks

If a file in `$out` contains the store path of `$dev` (e.g. a script hardcoding include paths), Nix records a **reference**, and GC / closure pull `dev` into runtime. That defeats multi-out.

```
# After build, hunt for leaks (illustrative)
nix path-info -r .#libgreet | rg dev || true
strings result/bin/* | rg 'dev' || true
```

Theory 4 — `moveToOutput` and helpers

stdenv provides helpers used heavily in nixpkgs:

```
moveToOutput share/man "$man"
moveToOutput include "$dev"
moveToOutput lib/pkgconfig "$dev"
```

Hazard	Result	Mitigation
Headers installed to `$out`	Runtime closure grows	Move to `$dev`
`out` references `$dev`	Forces `dev` into runtime closure	Fix install paths / `moveToOutput`
Forgetting `outputs` but installing to `$dev`	Empty/wrong paths	Declare `outputs` first
pkg-config file wrong prefix	Consumers cannot find libs	Check `.pc` `prefix`/`libdir`
Static libs in `out`	Heavier runtime	Prefer static in `dev`

Theory 5 — Consuming multi-output packages

```
nativeBuildInputs = [ pkg-config ];
buildInputs = [ foo ]; # pulls foo.dev for compilation via setup hooks usually
```

Setup hooks from `dev` outputs typically put headers on the search path when `foo` is in `buildInputs`.

Selecting outputs explicitly:

```
foo.dev  # the dev store path attribute
foo.out
foo.man
```

```
nix build nixpkgs#openssl
nix build nixpkgs#openssl.dev -o o-dev
nix build nixpkgs#openssl.man -o o-man
du -sh result o-dev o-man
```

Multiple outputs and `nix shell`

Depending on tooling, you may need the package default output only—headers come when building other packages, not always when "shelling" into a library.

Theory 6 — `separateDebugInfo`

```
stdenv.mkDerivation {
  # …
  separateDebugInfo = true;
}
```

When supported, debug symbols go to a separate output so runtime stays lean while GDB can still find debuginfo via standard mechanisms.

Goal	Approach
Small production closure	strip + multi-out; optional separate debug
Debug a crash on a server	install debug outputs temporarily; don't ship forever
Reproducible debug paths	Prefer separateDebugInfo over ad-hoc unstripped bins

---

Theory 7 — When single-output is fine

Package kind	Multi-out?
Tiny shell script CLI	Usually no
Go/Rust single static-ish binary	Often single `out`
Shared C library used by many pkgs	**Yes** (`out`+`dev`)
Huge docs package	Split `doc`/`man`
Firmware blobs	Case-by-case

Do not multi-out for fashion—multi-out when dependents would otherwise drag weight.

Worked example — toy multi-output C library + app

```c
/* src/libgreet/greet.c */
#include "greet.h"
const char *greet_msg(void) { return "hello multi-out"; }
```

```c
/* src/libgreet/greet.h */
#pragma once
const char *greet_msg(void);
```

```c
/* src/greetapp/main.c */
#include <stdio.h>
#include "greet.h"
int main(void) { puts(greet_msg()); return 0; }
```

```
# pkgs/libgreet.nix
{ stdenv, lib }:
stdenv.mkDerivation {
  pname = "libgreet";
  version = "0.1.0";
  outputs = [ "out" "dev" "man" ];
  src = ../src/libgreet;

  buildPhase = ''
    runHook preBuild
    $CC -shared -fPIC greet.c -o libgreet.so
    runHook postBuild
  '';
```

```
  installPhase = ''
    runHook preInstall
    install -Dm755 libgreet.so $out/lib/libgreet.so
    install -Dm644 greet.h $dev/include/greet.h
    mkdir -p $dev/lib/pkgconfig
    cat > $dev/lib/pkgconfig/libgreet.pc <<EOF
    prefix=$out
    libdir=$out/lib
    includedir=$dev/include
    Name: libgreet
    Version: 0.1.0
    Libs: -L\''${libdir} -lgreet
    Cflags: -I\''${includedir}
    EOF
    mkdir -p $man/share/man/man3
    echo ".TH GREET 3" > $man/share/man/man3/greet.3
    runHook postInstall
  '';

  meta = {
    description = "Tiny greet library (multi-output lab)";
    license = lib.licenses.mit;
  };
}
```

```
# pkgs/greetapp.nix
{ stdenv, lib, libgreet, pkg-config }:
stdenv.mkDerivation {
  pname = "greetapp";
  version = "0.1.0";
  src = ../src/greetapp;
  nativeBuildInputs = [ pkg-config ];
  buildInputs = [ libgreet ];
  buildPhase = ''
    runHook preBuild
    $CC main.c -o greetapp $(pkg-config --cflags --libs libgreet)
    runHook postBuild
  '';
  installPhase = ''
    runHook preInstall
    install -Dm755 greetapp $out/bin/greetapp
    runHook postInstall
  '';
  meta = {
    mainProgram = "greetapp";
    license = lib.licenses.mit;
```

```
  };
}
```

```
# flake fragment
{
  packages = forAllSystems (system:
    let
      pkgs = import nixpkgs { inherit system; };
      libgreet = pkgs.callPackage ./pkgs/libgreet.nix { };
    in {
      libgreet = libgreet;
      greetapp = pkgs.callPackage ./pkgs/greetapp.nix { inherit libgreet; };
      default = self.packages.${system}.greetapp;
    });
}
```

```
nix build .#libgreet
nix build .#libgreet.dev -o result-dev
nix build .#libgreet.man -o result-man
nix build .#greetapp
./result/bin/greetapp
nix path-info -r ./result | rg libgreet
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/multi-out`

Step 1 — Inspect an existing multi-out package

```
nix eval nixpkgs#openssl.outputs
nix build nixpkgs#openssl -o o-out
nix build nixpkgs#openssl.dev -o o-dev
du -sh o-out o-dev
```

Record sizes.

Step 2 — Build libgreet with three outputs

Implement the toy library (or split your stdenv tool into lib + bin).

Step 3 — Verify separate paths

```
nix build .#libgreet
nix build .#libgreet.dev
nix build .#libgreet.man
nix path-info .#libgreet .#libgreet.dev .#libgreet.man
```

Step 4 — Build greetapp against libgreet

Confirm the app runs and its runtime closure includes `libgreet` **out** but ideally not manpages.

```
nix path-info -r ./result | rg 'libgreet'
```

Step 5 — Deliberate mistake

Install a header into `$out/include` and rebuild app. Compare closure size / path-info. Fix it.

Step 6 — Document policy

In `NOTES.md`: when you will use `outputs = [ "out" "dev" ]` for internal packages vs single-output scripts.

Step 7 — Optional `separateDebugInfo`

```
separateDebugInfo = true;
```

Build and note the extra debug output if supported on your platform.

Step 8 — pkg-config break/fix

Corrupt the `.pc` `libdir`; observe consumer failure; restore.

Exercises

1. Compare `du -sh` for `openssl.out` vs `openssl.dev` on your system.
2. Find three multi-out packages in nixpkgs used by your hosts; list their `outputs`.
3. Explain in two sentences how GC roots interact with multi-out (each path separately).
4. Design outputs for a hypothetical SDK: runtime daemon, headers, CLI, docs.
5. Measure greetapp closure before/after the intentional header-in-out mistake.

Common gotchas

Symptom / mistake	What to do
`dev` empty	Not installing into `$dev`; check `outputs` list
Runtime needs `dev`	Cross-reference leak; fix files that embed `$dev` paths
`pkg-config` cannot find package	Ensure `.pc` in `dev` and package in `buildInputs`
Only `result` symlink	Multi-out: use `nix build .#pkg.dev` or `-o` names
Broke library for dependents	Test a dependent package in the same flake
Docs in `out`	`moveToOutput` or install directly to `$doc/$man`
Wrong outputs order	Put `out` first
Static lib only in `out`	Move `.a` to `dev` when possible

Checkpoint

- ☐ Can explain multi-out as multiple store paths
- ☐ Toy (or real) package produces `out` + `dev`
- ☐ Dependent app builds against `dev`, runs against `out`

- ☐ Closure inspection done with `nix path-info`
- ☐ One deliberate mis-install fixed
- ☐ Policy note written for internal packages
- ☐ Understand reference leaks

Commit

```
git add .
git commit -m "packaging: multiple outputs libgreet + greetapp"
```

Write three personal gotchas before debugging builds.

Debugging builds

Debugging builds

Goal: Recover from failed package builds using `nix log`, `-L`, `--keep-failed`, `breakpointHook`, and structured isolation of phase failures—without disabling the sandbox as a first resort.

Why it matters

Packaging skill is half writing expressions and half **reading sandboxes under stress**. Teams lose days because someone only knows "it failed" and rebuilds with random flags. This chapter builds a deliberate debug playbook.

Theory 1 — Where information lives

Tool	Shows	When
`nix build -L` / `--print-build-logs`	Live phase output	First failure
`nix log <drv|path>`	Stored build log	After failure/success
`nix derivation show`	JSON-ish drv env	Input/env inspection
`--keep-failed`	Leaves `tmp` build dir	Interactive inspect
`breakpointHook`	Drops into shell on failure	Deep phase debug
`nix why-depends`	Why a dep is in a closure	Bloat / unexpected refs
`--rebuild`	Force re-realization	Cache hiding issues

```
nix build .#broken -L
nix log .#broken
nix derivation show .#broken | less
```

Modern CLI accepts installables (`.#pkg`); classic `nix-store -l /nix/store/...drv` still works.

Theory 2 — Eval failure vs build failure

Separate these **immediately**:

```
nix eval .#packages.x86_64-linux.foo.drvPath
# works? → build issue. fails? → expression / module issue.
```

Class	Signals	Look at
Eval	`error:` before builder starts	Nix syntax, missing attrs, infinite recursion
Build	Phase logs, compiler errors	Sources, deps, phases
FOD	`hash mismatch`	URL/rev/hash trust
Substitute	download errors	Network, cache keys

Spending an hour in `config.log` for an eval error is pure waste.

Theory 3 — Keep failed trees

```
nix build .#broken --keep-failed -L
```

On failure, Nix prints a path under `/tmp/nix-build-…` (or configured temp). You can:

```
cd /tmp/nix-build-foo-*
ls -la
# inspect source tree after patchPhase, object files, config.log, etc.
```

> ⚠ Warning
>
> The kept tree is **not** the sandbox re-entered with the same env. It is a corpse for forensics. Re-running `make` there may work or lie—prefer `breakpointHook` for a live env.

Never commit keep-failed trees. They are for diagnosis only.

Theory 4 — `breakpointHook`

```
{ stdenv, breakpointHook }:
stdenv.mkDerivation {
  pname = "explode";
  version = "0.0.1";
  dontUnpack = true;
  nativeBuildInputs = [ breakpointHook ];
```

```
  buildPhase = ''
    echo "about to fail"
    false
  '';
}
```

When the build fails, you get a shell **inside** the build environment (with caveats on platforms and interactive settings). Fix hypotheses, then exit and fix the Nix expression permanently.

Also related: temporary extra tools in `nativeBuildInputs` for inspection (`strace`, `file`, `busybox`)—**remove before merge**.

Theory 5 — Common failure classes

Class	Signals	First moves
Hash / FOD	`hash mismatch`	Fetchers chapter workflow
Configure	`config.log`, missing headers	`nativeBuildInputs`, pkg-config
Compile	`error:`, missing symbols	deps, flags, patches
Link	`undefined reference`	`buildInputs`, order, static/shared
Install	empty `$out`, wrong paths	`installPhase`, multi-out
Check	test suite	`doCheck`, sandbox network, golden files
Patch	rejects	refresh patch
Purity	banned network/path	FOD or remove impurity
Eval	before build	fix Nix syntax/attrs

Reading a log quickly

1. Jump to the **last** phase banner before the error
2. Read the first **error:** line, not the last warning
3. Check whether the failing command is yours or a setup hook
4. Re-run with `-L` if the stored log is truncated in your UI

Theory 6 — Sandbox: don't disable casually

Flag / knob	Effect	Use?
`--option sandbox false`	Allows host impurities	Last resort local diagnosis only
`--no-sandbox`	Same family	Never as CI policy
`require-sigs` / substituters	Cache trust	Unrelated to compile fails
`__noChroot` (legacy patterns)	Avoid	Prefer proper FODs

If it only builds with sandbox off, your package is **not done**. Undeclared `/usr` deps will break on the next machine.

```
nix show-config | rg -i sandbox
# or
nix config show | rg -i sandbox
```

Theory 7 — Reproducing CI failures locally

```
# Clean-ish rebuild of one attr
nix build .#foo --rebuild -L

# Ignore binary cache for this attr (force local compile)
nix build .#foo --option substitute false -L

# Show why a dep is present
nix why-depends .#foo nixpkgs#openssl
```

Match CI: same flake lock, same system, same Nix version band (book baseline: **2.34.x**, nixpkgs **26.05**).

Bisect mental model

```
old lock (green)   update input  > new lock (red)
                 narrow: which input?
                 then: which package rebuild?
                 then: which phase?
```

Lock discipline + phase logs = faster bisect.

Theory 8 — Save the drv path

Always save the **drv path** from a failure before GC:

```
nix eval .#packages.x86_64-linux.foo.drvPath
nix log /nix/store/….drv
```

Without it, debugging after garbage collection becomes archaeology.

Worked example — scripted break/fix series

```
# pkgs/debug-lab.nix
{ stdenv, lib }:
stdenv.mkDerivation {
  pname = "debug-lab";
  version = "0.1.0";
  src = ../src/debug-lab;

  buildPhase = ''
    runHook preBuild
    $CC broken.c -o debug-lab
    runHook postBuild
  '';

  installPhase = ''
    runHook preInstall
    install -Dm755 debug-lab $out/bin/debug-lab
    runHook postInstall
  '';

  meta = {
    mainProgram = "debug-lab";
    license = lib.licenses.mit;
  };
}
```

```
/* broken.c - missing semicolon */
#include <stdio.h>
int main(void) {
  puts("debug-lab")
  return 0;
}
```

Fix: add the semicolon. Re-run with `-L` until green.

Failure dossier template

```
## Failure dossier
- Command:
- Eval or build?
- Phase (if build):
- First error line:
- Fix:
- Verification command:
```

Worked example — personal runbook skeleton

```
# DEBUG-RUNBOOK

1. Eval or build? (`nix eval …drvPath`)
2. `-L` first failure phase
3. `nix log`
4. Classify: hash vs compile vs link vs install vs check vs purity
5. `--keep-failed` if need tree
6. `breakpointHook` / extra tools if need live env
7. Only then consider sandbox experiments
8. Port fix into Nix sources; never only /tmp
9. Add a check so it cannot regress silently
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/debug`

Step 1 — Baseline success package

Start from stdenv `addone` or ecosystem A tool. Confirm green build + log:

```
nix build .#addone -L
nix log .#addone | tail -n 50
```

Step 2 — Inject a compile failure

Break a source file. Capture:

```
nix build .#addone -L 2>&1 | tee fail-compile.txt
nix log .#addone | tee fail-compile-log.txt
```

Step 3 — `--keep-failed`

```
nix build .#addone --keep-failed -L || true
# locate printed /tmp/nix-build-* path; list files; find .o / sources
```

Document what you could see vs not see.

Step 4 — Inject an install failure

Fix compile; break `installPhase` (`mkdir` missing, wrong path). Repeat log + keep-failed.

Step 5 — FOD failure recall

Temporarily wrong `src.hash`; confirm you recognize the error class in under 10 seconds.

Step 6 — `breakpointHook` (Linux recommended)

Add hook; force `false` in `buildPhase`; enter the breakpoint if your environment supports it. Type `env | sort | head`, inspect `$CC`, exit.

Step 7 — Write a personal runbook

`DEBUG-RUNBOOK.md` with **your** ordered steps (max one page).

Step 8 — Force local rebuild

```
nix build .#addone --rebuild --option substitute false -L
```

Note time vs cached build.

Step 9 — Complete failure dossier

For one intentional break, fill the dossier template under `notes/debug-dossier.md`.

Step 10 — Sandbox proof note

```
nix show-config 2>/dev/null | rg -i sandbox || nix config show | rg -i sandbox
```

State whether you ever disabled sandbox and why that is dangerous (undeclared deps "work on my machine").

Exercises

1. Given only a CI log snippet ending mid-compile, list three next commands you would run locally.

2. Break `doCheck` with a failing test; identify checkPhase in logs.

3. Use `nix derivation show` to find an env var you did not know was set.

4. Compare `--keep-failed` corpse vs `breakpointHook` live shell for the same failure—write pros/cons.

5. Time how long it takes you to classify five pasted error messages (FOD/eval/compile/link/install).

Common gotchas

Symptom / mistake	What to do
`nix log` says no build log	Build never started (eval) or GC'd; rebuild with `-L`
Kept temp already deleted	Re-run with `--keep-failed` immediately
breakpointHook "does nothing"	Platform/interactive limits; use keep-failed + better logs
Fixed tree by hand in `/tmp` only	Always port fix into Nix sources
Disabled sandbox "to move on"	Track impurity; fix properly
CI-only failure	Align system, lock, substituters, case-sensitive paths
Silent `doCheck` fails	Watch for checkPhase in log; set `doCheck` explicitly
Scrolled past first error	Search for `error:` from top of failure region

---

Checkpoint

- ☐ Used `-L` and `nix log` on a real failure
- ☐ Used `--keep-failed` and inspected the tree
- ☐ Classified at least three failure kinds
- ☐ Tried `breakpointHook` or documented why not
- ☐ Personal debug runbook committed
- ☐ Eval vs build separation practiced
- ☐ Sandbox policy note written

Commit

```
git add .
git commit -m "packaging: debug builds runbook and failure drills"
```

Write three personal gotchas before navigating nixpkgs.

Navigating nixpkgs

Navigating nixpkgs

Goal: Navigate modern nixpkgs package layout—especially **`pkgs/by-name`**—and shape a package so it could become an upstream PR (submission optional).

Why it matters

Copy-pasting random `default.nix` from a blog creates private snowflakes. Knowing where packages live, how they are called, and what reviewers expect makes your flake packages **portable** and your future contributions reviewable.

Theory 1 — Mental map of nixpkgs (2026)

Path	Role
`pkgs/by-name/<p>/<pname>/package.nix`	**Preferred** home for many new single-package expressions
`pkgs/top-level/all-packages.nix`	Historic wiring; less hand-editing for new pkgs
`pkgs/development/…`	Older category tree still holds many packages
`pkgs/applications/…`	Same—legacy category layout
`lib/`	Nix library functions
`nixos/modules/`	NixOS modules (not package expressions)
`nixos/tests/`	nixosTest definitions
`maintainers/maintainer-list.nix`	Maintainer identities
`pkgs/build-support/`	Fetchers, stdenv pieces, helpers

`by-name` rules of thumb

```
pkgs/by-name/he/hello/package.nix

                  package directory = pname
               first two lowercase letters of pname
```

Rule	Detail
File name	Usually `package.nix` (not always `default.nix`)
Shard	`toLower (substring 0 2 pname)`
Scope	One package per dir; keep it self-contained
Call	Auto-wired for eligible packages—follow current nixpkgs docs if adding upstream

Your **flake** can still use `callPackage ./pkgs/foo.nix { }` without sharding; by-name is the **upstream shape**.

Theory 2 — Anatomy of a PR-shaped `package.nix`

```nix
{
  lib,
  stdenv,
  fetchFromGitHub,
  # only real deps
}:

stdenv.mkDerivation (finalAttrs: {
  pname = "cooltool";
  version = "1.2.3";

  src = fetchFromGitHub {
    owner = "acme";
    repo = "cooltool";
    rev = "v${finalAttrs.version}";
    hash = "sha256-…";
  };

  nativeBuildInputs = [ /* … */ ];
  buildInputs = [ /* … */ ];

  meta = {
    description = "One-line description";
    homepage = "https://github.com/acme/cooltool";
    license = lib.licenses.mit;
    maintainers = with lib.maintainers; [ /* your handle if upstream */ ];
    platforms = lib.platforms.unix;
    mainProgram = "cooltool";
  };
})
```

Modern habits

Habit	Why
`finalAttrs:` pattern	Version/src self-reference without `rec` pitfalls
Explicit function head args	`callPackage` injects deps
`meta.mainProgram`	`nix run nixpkgs#cooltool`
Real `license`	Policy and binary caches
No unrelated formatting of huge trees	Reviewer happiness
Tests via `passthru` / nixosTests	"Done upstream" bar

Theory 3 — `callPackage` and fixed-points

```nix
pkgs.callPackage ./package.nix { }
# package.nix is a function of deps
```

```nix
# Override only one dep
pkgs.callPackage ./package.nix {
  openssl = pkgs.openssl_3;
}
```

Overlays often do:

```nix
final: prev: {
  cooltool = final.callPackage ./cooltool/package.nix { };
}
```

Use **`final.callPackage`** so deps see other overlay packages.

Why function heads matter

```nix
# callPackage binds by argument name
{ stdenv, fetchurl, openssl }:  # must match pkgs attributes
stdenv.mkDerivation { … }
```

Unknown args fail; unused args may still be required by callers for override ergonomics—follow local style.

Theory 4 — Reading existing packages as literature

```
# If you have a nixpkgs checkout:
rg -n 'pname = "ripgrep"' -S pkgs | head

# Query without full clone:
nix edit nixpkgs#ripgrep
# opens the file in $EDITOR when configured

nix eval nixpkgs#ripgrep.meta --json | jq .
```

Read three packages in **your** primary language:

1. Tiny CLI
2. One with patches
3. One with multiple outputs

Note patterns: hooks, `finalAttrs`, meta, tests.

What to look for

Signal	Meaning
`finalAttrs`	Modern self-reference
`passthru.tests`	Linked tests
`updateScript`	nix-update support
`meta.changelog`	Release notes link
`separateDebugInfo`	Debug hygiene
Language builder	Ecosystem-native packaging

Theory 5 — Tests and `passthru` (awareness)

Many nixpkgs packages expose:

```
passthru.tests = {
  version = testers.testVersion { package = finalAttrs.finalPackage; };
};
passthru.updateScript = nix-update-script { };
```

Your flake can mirror with `checks`. Upstream PRs often expect *some* test story for non-trivial packages.

Theory 6 — Category tree vs by-name coexistence

Not everything is migrated:

```
pkgs/applications/misc/hello/default.nix   # classic
pkgs/by-name/ri/ripgrep/package.nix      # by-name
```

Aspect	Category tree	by-name
Discovery	Human taxonomy	Automatic shard
New packages	Discouraged for many cases	Preferred
Review tooling	Mature	Mature + automation
Your flake	N/A	Copy the *shape*

Do not rewrite half of nixpkgs for fun. **New** private packages: by-name shape. **Reading** old packages: know both trees.

Theory 7 — Contribution hygiene (even if you never PR)

Practice	Why
One logical change per commit	Review
`cooltool: init at 1.2.3` message style	Convention
No secrets in expressions	Security
Builds on at least one platform claimed	Honesty
`nixpkgs-review` awareness	Community expectation
License correct	Legal

```
# PR-CHECKLIST.md
- [ ] Builds on my system
- [ ] meta complete (description, license, mainProgram, platforms)
- [ ] Hashes pinned (no fakeHash)
- [ ] Naming matches pname
- [ ] No secrets
- [ ] Commit message style ok
- [ ] Optional: nixpkgs-review
```

Worked example — flake package laid out like by-name

```
packaging-by-name/
  flake.nix
  pkgs/
    by-name/
      co/cooltool/package.nix
  overlays/default.nix
  PR-CHECKLIST.md
  NOTES.md
```

```
# overlays/default.nix
final: prev: {
  cooltool = final.callPackage ../pkgs/by-name/co/cooltool/package.nix { };
}
```

```
# flake.nix
{
  description = "by-name shaped package lab";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs {
        inherit system;
        overlays = [ (import ./overlays/default.nix) ];
      };
    in {
      packages.${system}.default = pkgs.cooltool;
      packages.${system}.cooltool = pkgs.cooltool;
      checks.${system}.meta-license =
        assert pkgs.cooltool.meta ? license;
        pkgs.runCommand "meta-ok" { } "touch $out";
    };
}
```

Reuse your best package from earlier chapters as `cooltool`—reshape, don't invent a new product.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/by-name`

Step 1 — Locate packages

```
nix edit nixpkgs#hello || true
nix eval nixpkgs#hello.meta --json | jq .
```

Record: description, license, platforms, maintainers shape.

Step 2 — Reshape your best packaging package

Move it to:

```
pkgs/by-name/<shard>/<pname>/package.nix
```

Use `finalAttrs` if it fits.

Step 3 — Overlay + flake export

`callPackage` through overlay; `nix build .#cooltool`.

Step 4 — Meta completeness pass

Ensure:

- ☐ `description`
- ☐ `homepage` (if any)
- ☐ `license`
- ☐ `mainProgram`
- ☐ `platforms` or sensible default

Step 5 — PR-shaped checklist

Write `PR-CHECKLIST.md` even if not submitting.

Step 6 — Optional: clone nixpkgs and dry-place the file

```
# shallow clone is large; optional
# place file in correct by-name path; do not push unless you intend a real PR
```

Step 7 — Compare category tree vs by-name

Find one package still under `pkgs/applications/…` and one under `by-name`. Note one reviewer-facing difference.

Step 8 — Literature notes

For three packages in your language, fill:

Package	Builder	Patches?	Multi-out?	Tests?

Exercises

1. Compute the by-name shard for `pname = "ripgrep"`, `pname = "Go"`, `pname = "q"`.
2. Convert a `rec` derivation to `finalAttrs`.
3. Add a flake check that fails if `meta.license` is missing.
4. Read `lib.licenses` docs mentally—pick correct license for a real tool you use.
5. Sketch what would block an upstream PR for your package (honest list).

Common gotchas

Symptom / mistake	What to do
Wrong two-letter shard	Use first two chars of **pname**, lowercase
`default.nix` in by-name	Prefer `package.nix` per current convention
Recursive `rec` version bugs	Prefer `finalAttrs`
Giant unrelated deps in head	Only list what the function uses
Meta missing `mainProgram`	Multi-bin packages: set correctly or omit carefully
Assuming auto-discovery in **your** flake	Flakes need explicit `callPackage`/overlay
Editing all-packages by habit	Prefer by-name for new pkgs

Symptom / mistake	What to do
Fake maintainers list entry	Only add yourself upstream if contributing for real

---

Checkpoint

- ☐ Explained `by-name` sharding
- ☐ Package lives in by-name-shaped path
- ☐ Overlay + `callPackage` works
- ☐ Meta filled to nixpkgs-ish bar
- ☐ PR checklist written (submit optional)
- ☐ Read three upstream packages as literature
- ☐ Know category tree still exists

Commit

```
git add .
git commit -m "packaging: by-name shaped package + meta"
```

Write three personal gotchas before binary caches.

Binary caches

Binary caches

Goal: Pull and push substitutes with **cache.nixos.org**, **Cachix**, and/or self-hosted **Attic** / **Harmonia**—with a clear model of trust and signatures.

Why it matters

Without substitutes, packaging and fleet deploys become compile farms. With careless substituters, you outsource trust to whoever can serve a path. Caches are a **performance and supply-chain** topic, not just "make CI fast."

Theory 1 — Substituters vs builders

```
eval → wanted store path P
           trusted substituter has P?  → download (+ verify sigs)
           else                        → build (local or remote builder)
```

Term	Meaning
Substituter	HTTP(S) NAR store endpoint (`cache.nixos.org`)
Binary cache	Popular name for substituter + signing
Signed NAR	Artifact with signature by a cache key
Trusted key	Public keys your Nix is willing to accept
Priority	Order among substituters

```
# NixOS example
nix.settings = {
  substituters = [
    "https://cache.nixos.org"
    "https://myorg.cachix.org"
  ];
  trusted-public-keys = [
    "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
    "myorg.cachix.org-1:BASE64KEY…"
  ];
};
```

Flake/`nix.conf` user settings similar under `~/.config/nix/nix.conf` or daemon config.

Remote builders (next chapter) move *work*. Caches move *results*. You usually want both.

Theory 2 — Trust model (non-negotiable)

Claim	Reality
"HTTPS means safe binaries"	No—HTTPS protects transport, not *who built* the bits
"I added a cache URL only"	Incomplete without matching **public key** (or explicit none)
"Private cache = trusted"	Only if builders and keys are controlled
"cache.nixos.org is magic"	Trusted by default for official paths; still a third party

Rule: Every substituter you add is part of your TCB (trusted computing base) for that machine class.

```
Who can push?  →  Who holds signing keys?  →  Which hosts trust the pubkey?
                          ↓
                What is the revoke plan?
```

Theory 3 — Cachix (hosted, popular)

Cachix hosts per-project caches with simple push UX.

```
nix shell nixpkgs#cachix -c cachix --help

# Auth (token from UI; treat as secret)
export CACHIX_AUTH_TOKEN=…   # prefer secret manager / CI secret

cachix use myorg             # writes substituter + pubkey locally (daemon may need trust)
cachix push myorg ./result
```

CI sketch:

```
- uses: cachix/install-nix-action@v31
- uses: cachix/cachix-action@v16
  with:
    name: myorg
```

```
    authToken: ${{ secrets.CACHIX_AUTH_TOKEN }}
- run: nix build .#default -L
```

Pin action versions to what you verify—follow current Cachix docs for 2026.

Push full runtime closure

```
nix-store -qR ./result | cachix push myorg
# or modern copy patterns when endpoint supports them
```

Theory 4 — Self-hosted: Attic & Harmonia

Project	Role (approx.)	When
Attic	Modern multi-tenant binary cache server	Teams wanting self-host push/pull
Harmonia	cache.nixos.org-compatible server in Rust	Serve local store as substituter
nix-serve / **nix-serve-ng**	Classic signing servers	Legacy blogs

Harmonia sketch (NixOS)

```
services.harmonia = {
  enable = true;
  # signKeyPath = "/var/lib/secrets/harmonia.secret"; # manage via sops-nix
  # settings…
};
# nginx TLS reverse proxy in front in production
```

Attic sketch (conceptual)

```
# server configured with tokens / buckets (see upstream Attic docs)
attic login local http://cache.example.internal:8080 "$TOKEN"
attic push local:lab ./result
```

Exact CLI flags evolve—**read current Attic docs** for 26.05-era packaging; labs may use Cachix **or** local Harmonia/Attic **or** file-store simulation.

Theory 5 — Push what matters

Push	Skip
Your packages & CI deps	Entire nixpkgs rebuilds by accident
Closure of deploy targets	Personal `/tmp` experiments
Signed artifacts from trusted CI	Unsigned ad-hoc laptop builds for prod (policy choice)

Policy question	Example house rule
Who may push?	CI on `main` only
Who may add a substituter?	Module PR review
Laptop pushes?	Dev cache only, not prod keys

Theory 6 — Substituter order and priority

```
nix.settings = {
  substituters = [
    "https://cache.nixos.org"
    "https://myorg.cachix.org"
  ];
  trusted-public-keys = [
    "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
    "myorg.cachix.org-1:…"
  ];
};
```

Idea	Practice
Official cache first	Common default
Extra caches append	Org/project binaries
Keys must match	Failures if mismatch / unsigned

Never add a substituter without its **public key**. Trust is cryptographic, not vibes.

Theory 7 — What “cache miss” really means

```
eval → derive output path hash
  → query substituters for that path
     → hit: download NAR
     → miss: build locally (or remote builder)
```

A miss is normal for private packages. A miss on `hello` from cache.nixos.org means network, config, or exotic platform issues.

```
nix path-info --store https://cache.nixos.org nixpkgs#hello
nix store ping --store https://cache.nixos.org
```

Force local build (contrast)

```
nix build .#cooltool --option substitute false -L
```

Feel the difference; do not leave `substitute false` in user config.

Theory 8 — Daemon vs user config

On multi-user Nix (NixOS default), **daemon** settings govern builds:

Location	Affects
`nix.settings` in NixOS	Daemon (system)
`~/.config/nix/nix.conf`	User; may be ignored for daemon builds
`trusted-users`	Who may set extra substituters

Symptom: “I added a cache to my user conf and nothing happened.” Fix: system module or trusted user policy.

Worked examples

NixOS module fragment for team cache

```
{ lib, ... }:
{
  nix.settings = {
    substituters = lib.mkAfter [
      "https://myorg.cachix.org"
    ];
    trusted-public-keys = lib.mkAfter [
      "myorg.cachix.org-1:REPLACE_ME="
    ];
  };
}
```

Local file NAR cache (always available lab path)

```
nix build .#cooltool -o result
nix copy --to "file://$PWD/nar-cache" ./result
# elsewhere or after GC of path (careful):
nix copy --from "file://$PWD/nar-cache" --to "$NIX_STORE" ./result
```

Query whether a path would substitute

```
OUT=$(nix build nixpkgs#hello --print-out-paths --no-link)
nix path-info --store https://cache.nixos.org --json "$OUT" | jq .
```

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/cache`

Step 1 — Observe default cache

```
nix build nixpkgs#hello -L
# note download lines mentioning cache.nixos.org
nix config show 2>/dev/null | rg -i 'substituter|trusted-public' \
  || nix show-config | rg -i 'substituter|trusted-public'
```

Step 2 — Build your package cold-ish

```
nix build .#cooltool --rebuild -L
```

Time it roughly.

Step 3 — Choose push target

Path A — **Cachix:** create free/personal cache if allowed; `cachix use`; push `./result`.
Path B — **Self-host lab:** enable Harmonia or Attic on a lab VM; configure key; push/copy.
Path C — **Document-only:** if no account/VM, write full config you *would* apply and simulate with `nix copy --to file:///…`.

Step 4 — Second machine / second user story

On another user or VM (or after careful path GC):

```
nix build .#cooltool -L
# should substitute if cache wired and path present
```

Step 5 — Trust write-up

`TRUST.md`:

1. Who can push?
2. Who holds signing keys?
3. What hosts trust the pubkey?
4. What is the revoke plan?

Step 6 — CI note

Add a short paragraph: push from CI only with secrets, not from random laptops, if that is your house rule. Ops CI chapter deepens this.

Step 7 — Negative test

Point at a cache **without** adding the public key (on a disposable Nix config if you can). Observe failure mode; restore.

Step 8 — Document lab cache policy

In `docs/cache.md`:

1. Which substituters are enabled
2. Which keys
3. Whether you push CI builds
4. Who may add a new cache (code review rule)

Exercises

1. Explain why HTTPS alone is insufficient for binary trust (one paragraph).
2. Map your org's (or lab's) TCB for substituters.
3. Compare wall time: substitute vs `--option substitute false` for a medium package.
4. Draft a revoke checklist for a leaked Cachix token.
5. Find the public key string for cache.nixos.org on your machine; confirm it matches docs.

Common gotchas

Symptom / mistake	What to do
"don't know how to trust" / unsigned	Add correct `trusted-public-keys` or sign
User `nix.conf` ignored by daemon	Configure system/daemon NixOS settings
Pushed wrong path	Push derivation outputs you care about; verify with another client
Cache hit but old content	Paths are immutable; rebuild means **new hash**
Leaked Cachix token	Rotate; treat like deploy key
Priority fights	Order substituters; understand fallback to build
Assumed private URL = auth	Many caches need tokens separately from Nix substituter URL

Symptom / mistake	What to do
Pushed entire nixpkgs by accident	Scope push to your closures

---

Checkpoint

- ☐ Explain substituter + signature trust
- ☐ Built with and without substitutes
- ☐ Pushed **or** file-store simulated push
- ☐ Second client pull story demonstrated/documented
- ☐ `TRUST.md` written
- ☐ Daemon vs user config understood
- ☐ Cache miss vs hit explained

Commit

```
git add .
git commit -m "packaging: binary caches push/pull and trust notes"
```

Write three personal gotchas before remote builders.

Remote builders

Remote builders

Goal: Offload realization to a remote Nix machine (Linux builder VM is ideal), understand `builders` / build machine config, and know when remote build beats "just use a cache." Align lab with **nixos-26.05** clients and builders when possible.

> **Note**
>
> Remote builders move **work**; binary caches move **results**. Confusing them wastes weeks. You usually want both: build once on a beefy box, push to Cachix/Attic, everyone substitutes.

Why it matters

Laptops should not compile Chromium. CI runners should not all rebuild the same uncached leaf. **Remote builders** are how Darwin talks to Linux, how small VMs ship large closures, and how fleets keep eval local while realization lives elsewhere.

Theory 1 — Cache vs remote builder

Binary cache	Remote builder
Needs existing path?	Yes	No
Uses your flake inputs	Downloads NARs	Evaluates plan, builds drv remotely
Good for	Repeat installs	First build, exotic systems
Trust	Signing keys	SSH + Nix auth + who runs builder
Network pattern	Pull results	Upload inputs + download outputs

```
[dev] --build--> [builder] --push--> [cache] --substitute--> [fleet]
```

Theory 2 — Configuration surfaces (2026)

NixOS (`nix.buildMachines`)

```
{
  nix.distributedBuilds = true;
  nix.buildMachines = [
    {
      hostName = "builder.lab.internal";
      system = "x86_64-linux";
      protocol = "ssh-ng"; # preferred modern protocol when available
      sshUser = "nixbuilder";
      sshKey = "/root/.ssh/id_build"; # or agent patterns
      maxJobs = 8;
      speedFactor = 4;
      supportedFeatures = [ "nixos-test" "big-parallel" "kvm" ];
      mandatoryFeatures = [ ];
    }
  ];
  nix.settings = {
    builders-use-substitutes = true; # remote may fetch deps itself
  };
}
```

Ad-hoc one-shot

```
nix build .#pkg --builders 'ssh-ng://nixbuilder@builder.lab.internal x86_64-linux' --max-job
# classic:
# --option builders 'ssh://…'
```

Features

Feature	Typical meaning
`big-parallel`	Allow heavy parallel builds
`kvm`	Nested VM tests (`nixosTest`)
`nixos-test`	OS tests scheduling
`benchmark`	Specialization (rare)

If a drv requires `kvm` and the builder lacks it, scheduling skips that machine—often looking like "mysterious local build" or hard failure depending on flags.

Theory 3 — SSH and store talk

Remote build roughly:

1. Local eval (usually) produces derivations
2. Upload input paths the remote lacks
3. Remote builds
4. Download result paths back (unless you only needed remote side effects)

```
# Test SSH as the Nix daemon user (often root on NixOS)
sudo ssh -i /root/.ssh/id_build nixbuilder@builder.lab.internal nix-store --version
```

Daemon context: On multi-user Nix, *root* (nix-daemon) must reach the builder—not only your user ssh config—unless using remote build modes that run as you (know your setup).

Check	Why
Non-interactive SSH	No password prompts in daemon
`known_hosts` stable	Automation vs MITM prompts
Dedicated build key	Least privilege vs your personal key
Builder disk free	Full disk mid-build is common
Builder Nix version band	Avoid obscure protocol skew

Theory 4 — Cross and multi-system

Goal	Approach
Build `aarch64-linux` from `x86_64` laptop	aarch64 builder **or** qemu-binfmt (slower)
Build Linux from Darwin	Linux remote builder almost always
Same-arch offload only	Simplest lab
`nixosTest` needing kvm	Builder with kvm feature + hardware

```
# Prefer real hardware/VM matching system
system = "aarch64-linux";
```

Honest lab goal: **one successful remote same-arch build**. Cross is stretch, not gate.

Theory 5 — Security notes

Risk	Mitigation
Builder can see sources	Treat builder as trusted environment
SSH key sprawl	Dedicated build key; restrict `authorized_keys`
Untrusted builder returns malicious paths	Only use builders you control; CI policies
World-open builder	VPN/Tailscale; no public SSH
Builder as root playground	Separate user; trusted-users carefully

Remote builders are **not** a sandbox that makes untrusted code safe. A compromised builder is a supply-chain incident.

Decision card — do I need a builder?

Situation	Prefer
Package already on cache	Substituter only
First build of large leaf on weak laptop	Remote builder
Darwin needing Linux closure	Linux builder
CI rebuilds identical drv every job	Push cache after one builder run
Untrusted contributor PR	**Do not** give them your builder

Theory 6 — Scheduling knobs that matter

Knob	Effect
`max-jobs 0` (client)	Force offload when supported
`speedFactor`	Prefer faster builders
`builders-use-substitutes`	Remote fetches deps itself
`supportedFeatures`	Eligibility filter
Local `max-jobs` > 0	Local may steal work

```
nix build .#cooltool -L \
  --builders "ssh-ng://nixbuilder@builder x86_64-linux - 8 1 big-parallel" \
  --max-jobs 0
```

Upload storms

If every client uploads the world to the builder, enable `builders-use-substitutes` and share the same binary cache with the builder so it pulls deps itself.

Theory 7 — Builder host minimal config

```
# on builder
users.users.nixbuilder = {
  isNormalUser = true;
  # group membership per current wiki; keep tight
};
nix.settings.trusted-users = [ "root" "nixbuilder" ];
# SSH key auth only; open firewall only to admin net
# Ensure large /nix disk; enable flakes
```

Exact store write permissions and `nix.sshServe` patterns vary by version—verify against **26.05** docs when wiring production.

Topology diagram (keep in notes)

```
[dev laptop / host A] --ssh-ng--> [builder VM B: NixOS 26.05, multi-user Nix]
        |                                  |
        +--------> Cachix/Attic <---------+  (optional push from B or A)
```

Worked example — two-VM lab checklist

```
1. Inventory local capacity (max-jobs, disk)
2. Provision builder VM with free /nix space
3. Create dedicated SSH key; install pubkey for nixbuilder
4. Prove non-interactive SSH as root (daemon) and as user
5. One-shot remote build with --max-jobs 0
```

```
6. Capture log evidence of remote host
7. Persist nix.buildMachines
8. Optional: push results to cache; rebuild substitute-only
9. Failure injection: stop sshd; observe client error; restore
```

Prove which machine built

```
nix path-info -Sh .#cooltool
# keep a snippet of the build log that mentions the remote host
```

If you cannot show evidence of remote work, you may have silently built local—re-check `--max-jobs 0` and builder SSH as daemon user.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/builders`

Step 1 — Inventory local capacity

```
nix config show 2>/dev/null | rg 'max-jobs|cores|builders' \
  || nix show-config | rg 'max-jobs|cores|builders'
nproc
df -h /nix
```

Step 2 — Provision builder

Options:

1. Second NixOS VM on LAN/VPN
2. Same machine via `ssh://localhost` (limited learning value but tests plumbing)
3. Cloud NixOS box

Install Nix multi-user; enable flakes; ensure free disk (tens of GB).

Step 3 — SSH from daemon-relevant user

```
ssh -i ~/.ssh/id_build nixbuilder@builder 'nix-store --version'
sudo ssh -i /root/.ssh/id_build nixbuilder@builder 'nix-store --version'
```

Document which user the daemon uses on your OS.

Step 4 — One-shot remote build

```
nix build .#cooltool -L \
  --builders "ssh-ng://nixbuilder@builder x86_64-linux - 8 1 big-parallel" \
  --max-jobs 0
```

Capture log proving remote participation.

Step 5 — Persist config

Add `nix.buildMachines` on the client host flake **or** user remote builders file. Rebuild; run without CLI flags.

Step 6 — Feature flag experiment

Request a bogus mandatory feature; observe scheduling failure; remove it. Write one sentence on what you saw.

Step 7 — Combine with cache (optional)

On success, push the result (binary caches chapter). Then disable builder and rebuild from substitute-only.

Step 8 — Write topology diagram

ASCII in `NOTES.md`: who evals, who builds, who signs, who deploys, trust boundaries.

Step 9 — Failure injection

1. Stop `sshd` on builder (or block with firewall).
2. Retry build; capture the error class (SSH vs scheduling).
3. Restore builder; confirm recovery.

Document: "client symptoms when builder is down."

Step 10 — Disk pressure note

Fill or simulate low disk on builder (carefully); document monitoring need (`df -h /nix` in runbooks).

Exercises

1. One paragraph: when cache alone is enough vs when you need a builder.
2. Design features list for a "CI builder" vs a "laptop offload" builder.
3. Explain why user SSH working is insufficient on multi-user Nix.
4. Estimate upload size for a clean builder without shared cache (qualitative).
5. Security review: what can a malicious builder do to your laptop store?

Common gotchas

Symptom / mistake	What to do
`cannot build on …`	Features/system mismatch; check `supportedFeatures`
SSH works as user, not as root	Configure root's key or daemon SSH
Huge upload times	Enable `builders-use-substitutes`; share cache with builder
Darwin → Linux confusion	Need Linux builder; not just `--system`
Stale known_hosts / MITM prompts	Non-interactive SSH hardening for automation
Builder disk full	GC on builder; monitor
Nested flake eval differences	Same Nix version band; same inputs
Assuming cache = builder	Re-read Theory 1
Silent local build	`--max-jobs 0`; verify logs

Checkpoint

- ☐ Distinguished cache vs remote builder
- ☐ Completed at least one remote (or localhost SSH) build
- ☐ Knew which SSH identity the builder used
- ☐ Documented persistent config approach
- ☐ Topology notes include trust
- ☐ Failure injection documented
- ☐ Features experiment observed

Commit

```
git add .
git commit -m "packaging: remote builders lab and topology"
```

Write three personal gotchas before reproducibility.

Reproducibility (honest view)

Reproducibility (honest view)

Goal: Measure what "reproducible" means for **your** packages—bit-for-bit where possible, rebuild-equivalent where not—and document limits without marketing language.

> **Note**
>
> **Baseline:** flakes + **nixpkgs 26.05**-class pins. This chapter is measurement and claims hygiene—not a promise that every package is bitwise identical forever.

Why it matters

Nix marketing slides say "reproducible builds." Production engineers say "same flake lock gave the same store paths on two Linux boxes last Tuesday." Both can be true at different strengths. Packaging craft ends with **honesty**, or fleet ops and disaster recovery false confidence will hurt.

Theory 1 — Strength ladder

Level	Claim	How to test
L0	Builds on my laptop once	`nix build`
L1	Rebuilds same path on same machine	`nix build --rebuild` → identical out path
L2	Same lock → same paths on two similar Linux hosts	Compare `nix build --print-out-paths`
L3	Bit-for-bit identical NAR contents	`diffoscope` / hash NARs
L4	Cross-OS / cross-vendor identical	Often **false** for complex pkgs
L5	Reproducible *data* and *uptime state*	Not Nix's job alone

Most flake apps aim for **L1–L2**. Many packages are not fully L3 yet (timestamps, capture order, non-deterministic linking). **L5** is backups, migrations, and ops—not `stdenv`.

How to talk about this in a runbook

Weak claim	Stronger claim
"Fully reproducible"	"With lock X on x86_64-linux, attr Y yields path P"
"Same everywhere"	"Same path on two Linux builders; Darwin not claimed"
"Immutable forever"	"Unchanged until inputs change; updates are deliberate"

Theory 2 — Input-addressed sameness

In the default model, the **output path hash** is a function of the derivation inputs (names, other paths, builder, system, env). If two machines produce the **same out path**, Nix agrees the build plan matched—not always that every byte was independently audited by you.

Content-addressed derivations (internals part) change the story: output hash can track content. Know they exist; **do not** depend on CA here unless you already enabled it deliberately.

```
input-addressed:  out path = f(derivation inputs…)
content-addressed: out path can reflect output hash (advanced)
```

Path sameness vs byte sameness

Observation	Means
Same out path	Same build plan (input-addressed)
Same NAR hash	Same bytes
Cache hit of path P	Someone trusted published P; not necessarily you built it twice

---

Theory 3 — Why bit-for-bit fails

Source	Example
Embedded timestamps	gzip headers, archives, man pages
Unordered maps / readdir	Parallel compression, zip member order
CPU-specific codegen	`-march=native` (avoid in shared packages)

Source	Example
Download nondeterminism	Unpinned floating tags (FODs)
Test data with time	Snapshot tests embedding dates
Build-id / path capture	Binaries recording build paths poorly
RNG in build	Unseeded UUIDs in artifacts

```
# Bad idea in shared packages:
NIX_CFLAGS_COMPILE = "-march=native";
```

Prefer portable flags; put optimized overlays in **explicit local** overlays, not silent defaults.

Fixed-output derivations (FODs)

FODs pin **output hash** of fetched content. They improve input stability but do not magically make compilers deterministic. A wrong hash fails the build; a floating `main` branch fetch is still an honesty bug at the pin layer.

Theory 4 — Tools for comparison

```
# Same path string?
nix build .#cooltool --print-out-paths | tee path-a.txt
# on machine B with same lock:
# nix build .#cooltool --print-out-paths | tee path-b.txt
diff path-a.txt path-b.txt

# Force rebuild and compare path (avoid silent substitute hiding local nondeterminism)
nix build .#cooltool --rebuild --print-out-paths

# Diff two store paths if they differ
nix store diff-closures /nix/store/…-a /nix/store/…-b

# Detailed binary diff
nix shell nixpkgs#diffoscope -c diffoscope ./result-a ./result-b
```

```
# Hash runtime closure (rough fingerprint)
nix path-info -rS ./result | sort | tee closure.txt
nix hash path ./result
```

```
# When you must ignore caches to test local build determinism
nix build .#cooltool --rebuild --option substitute false -L
```

Theory 5 — Operational reproducibility vs academic

Ops need	Nix contribution
Rebuild server from git	Flake lock + Disko + secrets keys
Same CI binary as prod	Shared cache + same installable attr
Legal/compliance bit-identical	Extra tooling; not free with Nix alone
Debug "works on my machine"	Locks + sandboxed builds + L1/L2 tests
Trust binary cache	Signatures + trusted keys

Write claims in runbooks as **testable sentences**, not adjectives.

Theory 6 — What Nix does *not* make reproducible

Domain	Why
Database contents	State; backups
TLS certificates from ACME	Time and CA interaction
VM disk drift	Imperative writes outside store
Hardware firmware	Out of band
"Same UI screenshot"	Fonts, GPU, timing
User home clutter	Not in the flake

Honesty: **system closure** rebuildable **product data** immortal.

Theory 7 — Cache vs reproducibility

```
## Cache vs reproducibility
- Same out path from cache  I audited the build locally.
- Signatures answer "who published this path?" not "is the universe bit-identical?"
- Our claim: trusted substituters may supply P if signatures verify.
- Our non-claim: every path was built twice under our eyes.
```

Substitutes accelerate L2 distribution; they do not replace measurement.

Worked example — measurement script

```
#!/usr/bin/env bash
# scripts/measure-repro.sh
set -euo pipefail
ATTR="${1:-.#cooltool}"
OUT1=$(nix build "$ATTR" --print-out-paths --no-link)
OUT2=$(nix build "$ATTR" --rebuild --print-out-paths --no-link)
echo "first:   $OUT1"
echo "rebuild: $OUT2"
if [[ "$OUT1" == "$OUT2" ]]; then
  echo "L1 path match: YES"
else
  echo "L1 path match: NO"
  nix store diff-closures "$OUT1" "$OUT2" || true
fi
```

```
chmod +x scripts/measure-repro.sh
./scripts/measure-repro.sh .#cooltool | tee ~/lab/nixos/packaging/repro/l1-cooltool.txt
```

Worked example — deliberate nondeterminism

```
# pkgs/bad-stamp.nix - lab only
{ stdenv }:
stdenv.mkDerivation {
  name = "bad-stamp";
  phases = [ "buildPhase" "installPhase" ];
  buildPhase = ''
    date > stamped.txt
  '';
  installPhase = ''
    mkdir -p $out
    cp stamped.txt $out/
    echo ok > $out/ok
  '';
}
```

Rebuild twice; discuss whether **out path** changed (input-addressed plan may still match if recipe identical while **contents** differ—observe carefully with `--rebuild` and content hashes). Then remove `date` and remeasure.

Worked example — REPRO.md template

```
## Claims we make
- With flake.lock revision … on x86_64-linux, `nix build .#cooltool` yields path P.
- We push P to cache C signed by key K (if applicable).

## Claims we do not make
- Bit-for-bit identity on Darwin vs Linux
- Reproducible database contents
- Forever stability without lock updates
- L3 for all transitive nixpkgs deps

## Measured on
- date, nix version, systems, results table

## Residual risks
- …
```

Two-machine log template

```
## L2 attempt
Lock: flake.lock nixpkgs rev …
Attr: .#cooltool
Machine A: hostname, nix version, out path
Machine B: hostname, nix version, out path
Match: YES/NO
If NO: first nix store diff-closures lines / system difference notes
```

Even a failed L2 (no second machine) earns credit if you document the blocker and a next attempt plan.

Lab — multi-step

Suggested workspace: `~/lab/nixos/packaging/repro`

Step 1 — Environment record

```
mkdir -p ~/lab/nixos/packaging/repro
cd /path/to/your/flake
nix flake metadata | tee ~/lab/nixos/packaging/repro/metadata.txt
```

```
nix --version | tee ~/lab/nixos/packaging/repro/nix-version.txt
uname -m | tee ~/lab/nixos/packaging/repro/uname.txt
```

Step 2 — Pick three artifacts

1. Your primary packaging package (e.g. `.#cooltool`)
2. `nixpkgs#hello`
3. One "fat" but sane package you use (e.g. `git`—not a browser)

Step 3 — L1 measurements

For each, run rebuild comparison; record path equality in a table:

Attr	Path1	Path2	L1 match?

Step 4 — L2 if you have two environments

Same `flake.lock`, two machines or two VMs. Compare printed out paths. If only one machine, document the blocker.

Step 5 — Substitute false control

For your package, once with substitutes allowed and once with `--option substitute false --rebuild`. Note time and path.

Step 6 — Deliberate nondeterminism (lab only)

Add `bad-stamp` package; rebuild; fix by removing `date`; note findings.

Step 7 — Closure drift story

Change a dependency pin (or overlay bump) while keeping app source identical. Show out path change. One paragraph on **reproducible inputs**.

Step 8 — Honesty statement

Write `REPRO.md` from the template.

Step 9 — Optional diffoscope

If two outputs differ, run diffoscope and paste the **top** finding into notes—not a 10k-line dump.

Step 10 — Cache trust paragraph

Link binary cache trust to honesty using the Theory 7 bullets.

Step 11 — README claim scrub

If your package README says "fully reproducible," rewrite that sentence using L1/L2 language from `REPRO.md`. Paste before/after into notes.

Exercises

1. Classify five real packages you use into L0–L3 ambition.
2. Argue for or against requiring L3 for an internal Go CLI (one page max).
3. Show that floating `inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable"` without lock discipline breaks L2.
4. Measure L1 for a multi-output package's `out` vs `dev` separately.
5. Write an executive one-liner that is true and useful about Nix reproducibility.

Common gotchas

Symptom / mistake	What to do
Assumed same path across Darwin/Linux	Different `system` → different paths normal
Compared without same lock	Update discipline first
Trusted cache hit hiding local nondeterminism	`--rebuild` / `--option substitute false`
Marketing "fully reproducible" in README	Replace with measured claims
Floating `main` input	Pin; FODs with correct hashes
Compared `result` symlinks without realizing GC	Paths may still exist; note roots
L3 required for every package	Prefer honest L1–L2 for apps

Symptom / mistake	What to do
Confused data durability with package repro	Backups chapter / ops

---

Checkpoint

- ☐ Ladder L0–L5 explained in own words
- ☐ L1 measured for own package
- ☐ Environment metadata captured
- ☐ Deliberate nondeterminism demo + fix
- ☐ `REPRO.md` honesty statement written
- ☐ Cache trust paragraph written
- ☐ README claim scrubbed if needed
- ☐ Know path-sameness vs byte-sameness
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "packaging: reproducibility measurements and honesty notes"
```

You now have the packaging spine: phases, FODs, ecosystems, overrides, multi-out, debug, nixpkgs shape, caches, builders, honesty. Ops and fleet chapters ship the closures you learned to build.

ROS 2 robotics with nix-ros-overlay

ROS 2 robotics with nix-ros-overlay

Goal: Stand up a **reproducible ROS 2 development environment** with `nix-ros-overlay`—flakes, binary caches, distro selection (Humble/Jazzy/…), project `devShells`, and a path toward packaging your own nodes—without being locked to Ubuntu's deb repos.

> **Note**
>
> This chapter is about the **package manager + overlay**, not a full robotics curriculum. It assumes you know (or will learn elsewhere) ROS 2 concepts: nodes, topics, DDS, workspaces, `colcon`. Here you learn how Nix makes those tools **pinable and portable** on NixOS, Fedora, or any Linux with Nix.

Why this chapter exists

Classic ROS 2 setup:

```
Ubuntu LTS → apt ROS repo → /opt/ros/<distro> → source setup.bash
```

That works—and couples your **OS**, **ROS distro**, and **system Python** forever. Teams then invent Docker stacks so CI can still build.

nix-ros-overlay answers a different question:

Want	Approach
Same ROS graph on Fedora / NixOS / CI	Overlay + flake lock
Multiple ROS distros without dual-boot	Separate `devShells`
Honest dependency graph	Nix store + hydra status badges
Avoid `apt` pollution on daily driver	Project shell only

It is an **overlay** on a specific nixpkgs pin (the overlay *follows its own nixpkgs*). Treat it like a specialized ecosystem helper—related to language ecosystems and overlays, not like `environment.systemPackages = [ ros-desktop-full ]` on every laptop forever.

Mental model

```
                    nix-ros-overlay flake


        overlays.default    pinned nixpkgs    rosPackages.<distro>

                   pkgs = import nixpkgs { overlays = [ … ]; }


                    buildEnv { underlay = true; paths = [ ros-core … ]; }


                         mkShell / nix develop  (+ colcon, clang, …)
```

Piece	Role
Overlay	Injects `rosPackages.*` and ROS build helpers into `pkgs`
`rosPackages.<distro>`	Attrset of packages for Humble, Jazzy, Kilted, Lyrical, Rolling, …
`buildEnv { underlay = true; … }`	Composes a ROS underlay environment (AMENT/COLCON paths) for shells
`colcon`	Workspace build tool (non-ROS nixpkgs package, usually)
Binary caches	Avoid compiling the world; still expect occasional local builds

ROS 1 / Gazebo Classic were removed from `master` after EOL (2025). Historical work lives on branches such as `ros1-25.05`—this chapter is **ROS 2 only**.

Status and expectations (be honest)

Upstream tracks build health per distro on Hydra (see the README badge table on lopsided98/nix-ros-overlay).

What generally works:

- Large fraction of distro packages on **x86_64-linux** / **aarch64-linux**
- Functional **dev environments** via `nix-shell` / `nix develop`

- Generated package defs via **superflore**

What still hurts:

- Not every package builds (thousands of packages; 80–90% targets are aspirational and drop as distros age)
- Free **ros.cachix.org** capacity is limited—packages may be missing or evicted
- Graphics (RViz, Gazebo/Gz) on non-NixOS needs **nixGL** or **nix-system-graphics**
- macOS is not a first-class story

> ⚠ Warning
>
> Budget **time and disk**. First entry into a desktop-class shell can pull tens of gigabytes when substitutes miss. Prefer a slim `ros-core` shell before `desktop` / `desktop-full`.

Binary caches (do this first)

Official community cache (Cachix)

```
# Option A
cachix use ros

# Option B - nix.conf (user or system)
# substituters = https://cache.nixos.org https://ros.cachix.org
# trusted-public-keys = cache.nixos.org-1:… ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wC
```

Public key commonly documented by the project:

```
ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL+38Rvo=
```

Experimental Attic cache (when Cachix is thin)

Upstream documents an experimental Hydra-fed cache (availability not guaranteed). Example from project docs (verify current values on the README before trusting):

```
extra-substituters = https://attic.iid.ciirc.cvut.cz/ros
extra-trusted-public-keys = ros:JR95vUYsShSqfA1VTYoFt1Nz6uXasm5QrcOsGry9f6Q=
```

On **NixOS**, prefer declarative:

```
# modules/common/nix-ros-cache.nix (sketch)
{
  nix.settings = {
    substituters = [
      "https://cache.nixos.org"
      "https://ros.cachix.org"
      # "https://attic.iid.ciirc.cvut.cz/ros"  # optional experimental
    ];
    trusted-public-keys = [
      "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
      "ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL+38Rvo="
      # "ros:JR95vUYsShSqfA1VTYoFt1Nz6uXasm5QrcOsGry9f6Q="
    ];
  };
}
```

Multi-user: **trusted** substituters belong in the **daemon** config (system), not only `~/.config/nix/nix.conf`.

Fast path: try upstream examples (no project yet)

Flake develop (recommended)

```
# ROS 2 Jazzy desktop-class example (name may track upstream flake outputs)
nix develop \
  --option extra-substituters 'https://ros.cachix.org' \
  --option extra-trusted-public-keys 'ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL
  github:lopsided98/nix-ros-overlay/master#example-ros2-desktop-jazzy
```

Inside the shell:

```
ros2 --help
# Classic demo (if packages present in that env)
ros2 launch demo_nodes_cpp talker_listener_launch.xml
```

Classic nix-shell form

```
nix-shell \
  -I nix-ros-overlay=https://github.com/lopsided98/nix-ros-overlay/archive/master.tar.gz \
  --option extra-substituters 'https://ros.cachix.org' \
  --option extra-trusted-public-keys 'ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL
  '<nix-ros-overlay/examples/ros2-desktop.nix>' --argstr rosDistro jazzy
```

Prefer **flakes + lock** for anything you keep.

```
# Scaffold from upstream template
mkdir -p ~/lab/nixos-book/ros2-env && cd ~/lab/nixos-book/ros2-env
nix flake init --template github:lopsided98/nix-ros-overlay
```

Project flake: ROS 2 underlay in a devShell

Critical rule: `nixpkgs.follows = "nix-ros-overlay/nixpkgs"`.
If you pin book baseline `nixos-26.05` *independently*, you will fight ABI/version skew. The overlay chooses a nixpkgs it builds against—**follow it**.

Minimal `flake.nix`

```
{
  description = "ROS 2 lab environment via nix-ros-overlay";

  inputs = {
    nix-ros-overlay.url = "github:lopsided98/nix-ros-overlay/master";
    # IMPORTANT: do not replace this with a free-floating nixos-26.05 input
    nixpkgs.follows = "nix-ros-overlay/nixpkgs";
  };

  outputs = { self, nix-ros-overlay, nixpkgs }:
    nix-ros-overlay.inputs.flake-utils.lib.eachDefaultSystem (system:
      let
        pkgs = import nixpkgs {
          inherit system;
          overlays = [ nix-ros-overlay.overlays.default ];
        };

        # Pick ONE distro per shell (or define multiple shells)
        rosDistro = pkgs.rosPackages.jazzy; # or .humble / .kilted / .lyrical / .rolling

        rosEnv = rosDistro.buildEnv {
          underlay = true;
          paths = with rosDistro; [
            ros-core
            # Grow deliberately:
            # ros-base
            # desktop
            # demo-nodes-cpp
            # rviz2
```

```
            # teleop-twist-keyboard
          ];
        };
      in {
        devShells.default = pkgs.mkShell {
          name = "ros2-jazzy-lab";
          packages = [
            pkgs.colcon
            pkgs.cmake
            pkgs.gcc
            pkgs.python3
            rosEnv
            # optional quality-of-life
            # pkgs.rosPackages.jazzy.ros2cli  # often already via env
          ];
          shellHook = ''
            echo "ROS 2 shell (nix-ros-overlay) - distro attrs under pkgs.rosPackages.*"
            # AMENT/COLCON hooks are provided by buildEnv underlay; if a var is missing,
            # check `env | rg -i 'ament|colcon|ros'` and upstream examples.
          '';
        };

        # Optional second shell: minimal CI-ish
        devShells.ci = pkgs.mkShell {
          name = "ros2-ci";
          packages = [
            pkgs.colcon
            (rosDistro.buildEnv {
              underlay = true;
              paths = with rosDistro; [ ros-core ];
            })
          ];
        };
      });

  # Flake-level nixConfig is honored by many CLIs; system trusted-keys still matter for mult
  nixConfig = {
    extra-substituters = [ "https://ros.cachix.org" ];
    extra-trusted-public-keys = [
      "ros.cachix.org-1:dSyZxI8geDCJrwgvCOHDoAfOm5sV1wCPjBkKL+38Rvo="
    ];
  };
}
```

```
cd ~/lab/nixos-book/ros2-env
git init
```

```
# paste flake.nix
nix flake lock
nix develop
ros2 --help
```

direnv

```
echo 'use flake' > .envrc
direnv allow
```

Same direnv discipline as any other project shell—plus larger GC roots. Prune unused roots when disk hurts.

Choosing a ROS 2 distro attr

Names track ROS releases exposed by the overlay (check `pkgs.rosPackages` in `nix repl` when unsure):

Distro attr (typical)	Role
`humble`	Older LTS-class; good for legacy robots
`jazzy`	Common current teaching/desktop target
`kilted` / `lyrical`	Newer releases as overlay adds them
`rolling`	Bleeding edge; expect breakage

```
nix develop -c nix eval --raw --impure --expr \
  'let pkgs = import (builtins.getFlake "git+file://'"$PWD"'").inputs.nixpkgs { system = "x8
# Simpler once inside a shell that has pkgs: use nix repl with the overlay loaded
```

Practical approach:

```
nix repl
# :lf github:lopsided98/nix-ros-overlay
# then explore outputs / pkgs construction from the flake
```

Or open upstream Hydra badges before pinning a distro for a class.

Workspace workflow (overlay underlay + your packages)

Nix supplies the **underlay** (system ROS packages). Your robot code often still uses a **colcon workspace** on top:

```
~/lab/robot_ws/
  flake.nix          # nix-ros-overlay shell
  .envrc
  src/
    my_robot_bringup/
    my_robot_desc/
```

```
nix develop
mkdir -p src && cd src
# git clone your packages …
cd ..
colcon build --symlink-install
source install/setup.bash
ros2 pkg list | head
```

Why not put *everything* in Nix immediately?

Phase	Reasonable default
Week 1	Overlay underlay + colcon for *your* packages
Later	Package stable internal libs with ROS Nix builders / overlays
CI	`nix develop -c colcon build` with lockfile

Building every teammate's WIP branch as pure Nix is optional maturity—not day-one tax.

Custom ROS packages (direction of travel)

Patterns used in the ecosystem (details evolve; check overlay helpers when you implement):

1. **Overlay on the overlay** — add your package set on top of `nix-ros-overlay.overlays.default`
2. **`buildRosPackage` / distro helpers** — when exported by the overlay version you pin
3. **Upstream first** — prefer nixpkgs for non-ROS deps; fix rosdep → nix name mappings when evaluation complains about `_unresolved_<dep>`

Evaluation error pattern from upstream FAQ:

```
Function called without required argument "_unresolved_<dependency>"
```

Usually means rosdep lacks a `nixos` key for that dependency—map it to a nixpkgs package or package the dep.

Do **not** start by forking thousands of generated expressions; override the few packages you need.

Graphics: RViz, Gz, tools on non-NixOS

On **NixOS**, OpenGL/Vulkan integration is usually straightforward if drivers are set up.

On **Fedora / Ubuntu + Nix**, Nix-built GL apps often need a wrapper:

Tool	Idea
nixGL	Wrap `rviz2` with host GL
nix-system-graphics	System-oriented graphics story

```
# Conceptual - install nixGL per its docs, then:
# nixGL rviz2
```

Put the wrapper in `shellHook` or a small script in the flake so students don't invent flags.

NixOS host integration (what belongs where)

Layer	Recommendation
devShell (this chapter)	Daily robot development
HM packages	Maybe `colcon`/`clangd`—not full desktop ROS
`environment.systemPackages`	Avoid multi-GB ROS desktops system-wide
Robot appliance image	Separate `nixosConfigurations.robot` with only runtime nodes

Runtime robot image sketch (conceptual):

```
# hosts/robot/configuration.nix - illustration only
{ pkgs, ... }: {
  # Use same overlayed pkgs in the flake's nixosSystem
  environment.systemPackages = [
    # minimal runtime env or specific nodes-not desktop-full
  ];
```

```
  # networking, users, systemd services that launch ros2 launch …
}
```

Wire the overlay once at flake level so **devShell and robot image share the same pin**.

Fedora dual-stack note

If you followed Nix on Linux:

- Keep **desktop/GPU** on Fedora
- Keep **ROS** in a project flake shell
- Add ROS caches to the **daemon** config
- Do not `dnf install` ROS debs *and* Nix ROS for the same workspace—pick one story per project

---

Troubleshooting

Symptom	Direction
Massive local compiles	Caches not trusted by daemon; check `nix show-config | rg substituter`
`nixpkgs` version chaos	Forgot `nixpkgs.follows = "nix-ros-overlay/nixpkgs"`
`_unresolved_*` eval error	Missing rosdep→nix mapping; package or map dependency
Package fails to build	Check Hydra; open issue; override; try another distro
`ros2` not found in shell	`buildEnv` paths incomplete; compare upstream example env
RViz black window / GL error	nixGL / drivers; try non-NixOS graphics wrappers
Disk full	`nix store gc`; drop `desktop-full` shells; limit direnv roots
DDS / multicast weirdness	Host firewall/network; ROS_DOMAIN_ID; not always a Nix bug

```
# Useful inspections
nix flake metadata
nix path-info -Sh .#devShells.x86_64-linux.default 2>/dev/null || true
```

```
env | rg -i 'ament|colcon|ros|rmw' | head
```

Security and supply chain

- Overlay + ROS sources are a **large** trust surface—pin **flake.lock** and review updates
- Cachix/Attic keys: only add caches you intend to trust for substitutes
- Robot network: treat DDS discovery like any open LAN service
- Don't bake robot secrets into store paths (same rule as HM)

Lab A — Hello underlay

1. Enable `ros.cachix.org` on your Nix install.
2. `nix develop github:lopsided98/nix-ros-overlay/master#example-ros2-desktop-jazzy` (or current equivalent).
3. Run talker/listener launch; capture a screenshot or log snippet in lab notes.
4. Record closure size: how painful was the first download?

Lab B — Project flake

1. `nix flake init --template github:lopsided98/nix-ros-overlay` in `~/lab/nixos-book/ros2-env`.
2. Trim env to `ros-core` (+ one demo package if available).
3. Add `direnv` + commit `flake.lock`.
4. `colcon build` an empty package skeleton; `ros2 pkg list` shows it after source.

Lab C — Distro compare (optional)

1. Define `devShells.humble` and `devShells.jazzy`.
2. Document one package that exists in one and fails in the other.
3. Write a team policy: "default teaching distro = …".

Lab D — Robot appliance sketch (optional)

1. In a monorepo flake, add `nixosConfigurations.ros-bot` sharing the overlay.
2. System packages only runtime nodes; develop still uses `devShells.default`.
3. Explain why the robot image is smaller than the desktop shell.

Checkpoint

- ☐ You can explain why `nixpkgs.follows` the overlay
- ☐ Caches configured for multi-user Nix
- ☐ A project flake enters `ros2` successfully
- ☐ You know underlay vs colcon workspace
- ☐ Graphics plan for RViz on your host OS
- ☐ You did **not** dump `desktop-full` into `environment.systemPackages` without reason

Further depth (this book)

Topic	Chapter
Overlays mechanism	Overlays (light)
Project shells	direnv and devShells
Language packaging patterns	Ecosystems A/B
Caches	Binary caches
Fedora dual-stack	Nix on Linux

Topic	Chapter
Reproducibility honesty	Reproducibility

Upstream

- lopsided98/nix-ros-overlay — README, examples, Hydra badges
- ros.cachix.org — community substitutes
- ROS 2 docs — nodes/launch/DDS semantics

When this notebook and upstream disagree, **trust the overlay README and your `flake.lock`**.

Running a Nixpkgs team

Running a Nixpkgs team

Thesis

Nixpkgs is the largest open-source software distribution in the world, with roughly a thousand active contributors every month. Most of that work is volunteer, and many critical pull requests still get **one** committer glance. A **Nixpkgs team** is how you turn "we care about these packages" into a durable, public review loop: a GitHub team under the NixOS organisation, packages that list that team as maintainers, a board outsiders can see, and a norm that the team **blocks** bad merges rather than racing to press the green button. Tweag's 2026 write-up of their security-minded team is the worked example; the mechanics apply to any topic-oriented group that can meet the community's trust bar.

Version note: packaging workflow here targets **Nix 2.35+** and **NixOS 26.05+** (verified against static Nix **2.35.3** on the authoring host). Team sync automation described by Tweag landed in 2025; policy that **teams are individuals, not corporate brands** is current as of the Tweag post (2026-10-08).

Mental model

```
GitHub team (NixOS org)   sync   in-tree team definition (maintainers-list / teams)

                                  package meta.maintainers includes the team

project board + review script          CODEOWNERS / review requests / notifications

Matrix / chat for standards     private checklist (slightly harder to game)
```

Piece	What it does
GitHub team under NixOS/Nixpkgs	Any Nixpkgs **committer** can create one; membership is the trust boundary
In-code team definitions	Sync from GitHub into the Nixpkgs tree (automation from 2025) so `meta.maintainers` stays honest
Package maintainer entry	How the team gets pinged when those packages move
Public board	Shows what is in flight so others can help or audit

Piece	What it does
Private checklist	Internal review bar (security smells, patch correctness) — not a secret cabal, just not a public scoring rubric attackers can tailor to
Merge posture	Tweag's team **does not merge**; it **blocks** when something is wrong. Adjust to your charter, but be explicit

Policy that surprised Tweag (and then made sense): corporate-affiliated *branded* teams are no longer permitted; members must be **individuals**, with or without a day-job affiliation. You still get org-sponsored capacity; you do not get a logo as a trust root.

Worked examples

Case 1: Draft the charter as a Nixpkgs issue first

Before creating GitHub objects, write a short public issue that states:

1. **Goal** (e.g. "prevent malicious or careless changes from landing on packages we track").
2. **Scope** (exact attribute paths or package sets).
3. **Membership rule** (Tweag required **commit access** so reputation is already established).
4. **Merge rule** (review-and-block vs review-and-merge).
5. **Communication** (Matrix room, meeting cadence, how reminders work).

That issue becomes the onboarding document. Volunteers join because the work is visible; drive-by requests get a clear "yes, if you already have commit bit" answer.

Case 2: Create the GitHub team and let the tree sync

Committers create the team in the NixOS/Nixpkgs GitHub organisation, add initial members, then rely on the **GitHub → Nixpkgs team definition sync** so the repository's team data matches reality. You should not hand-maintain a diverging parallel list in a private wiki.

Checklist:

```
[ ] GitHub team exists under the NixOS org
[ ] Members are individuals with commit access (if that is your bar)
[ ] In-tree team entry appears after sync
[ ] Relevant packages list the team in meta.maintainers
[ ] Notifications actually fire on a test PR (open a no-op against a tracked package)
```

Adding the team as maintainer is the boring step people forget. Without it, you have a chat room with a fancy name.

Case 3: Make review load visible

Tweag paired a **GitHub project board** with a small script that keeps the board current. The point is not the script language; it is that:

- outsiders can see queue depth,
- a vacationing reviewer does not strand a PR invisibly,
- you can measure "250+ PRs in six months" instead of vibes.

Keep the **public** list of tracked packages next to the board. Keep the **private** checklist private enough that it is not a free coaching guide for someone trying to sneak a bad `postInstall` past you.

Case 4: What "success" looked like for Tweag

Between March and September 2026 the team reviewed **everything** that touched their package set — more than **250** pull requests. They did not claim to have caught malware or a live zero-day. They **did** catch real mistakes (for example an incorrectly applied patch) and code-quality issues. That is the honest bar for a new team: continuous presence beats heroic incident response.

Case 5: Maintainer metadata shape (sketch)

Exact attribute names evolve; always copy from a nearby package in the tree you are editing. The *idea* is:

```
# sketch - follow current nixpkgs conventions for the package you touch
meta = {
  # …
  maintainers = with lib.maintainers; [
    someIndividual
    # teams.<yourTeam>  # when the synced team entry exists
  ];
};
```

Run the repository's existing validation (`nixpkgs-lint` / CI maintainer checks — whatever the current `CONTRIBUTING.md` points at) rather than inventing a parallel schema. On this authoring host, `/workspace/nix-static --version` reports **nix-static (Nix) 2.35.3**; we did **not** evaluate nixpkgs in-tree here (no channel pin / no sudo). Treat the sketch as orientation, then verify against `pkgs/by-name/…` on your checkout.

The trap

1. **Branding the team as a company.** Policy wants accountable humans. Sponsor the work; do not make the org name the trust root.

2. **Opening membership to everyone on day one without a trust bar.** "Anyone can join" is fine *after* you define how strangers get evaluated. Tweag leaned on **existing commit access**.
3. **Merging by default because you can.** A security-oriented team that also merges creates conflict-of-interest optics. Prefer **block-on-problem** unless your charter says otherwise.
4. **Publishing the entire checklist.** Transparency of *queue* ≠ transparency of *detection heuristics*. A little friction against gaming is healthy.
5. **Ignoring volunteer intermittency.** Availability fluctuates. Budget gentle reminders and a single facilitator who keeps the board honest — otherwise the team exists only on paper.

The boring rule

Publish a charter issue, create an **individual** GitHub team, sync it into Nixpkgs, attach it to `meta.maintainers` for a **small** package set, and review in public with a board. Block bad changes; do not fetishize merge counts. Rotate people; keep one person responsible for nudging the queue. Re-read Nixpkgs `CONTRIBUTING.md` and the current teams policy before you invent process.

Try this

1. Pick **three** packages you already maintain. Estimate monthly PR volume (GitHub search `is:pr path:pkgs/…`).
2. Draft a one-page charter issue (goal, scope, membership, merge posture).
3. If you have commit access, create a topic team and add yourself; confirm the in-tree sync updates.
4. Add the team to those three packages' `meta.maintainers` on a branch and open a PR that *only* does that — prove notifications work.
5. Stand up a project board with columns `Incoming / Reviewing / Done` and move one real PR through it.

Sources

- How to set up and run a Nixpkgs team (Aleksandr Bantev / Tweag, 2026-10-08)
- Nixpkgs CONTRIBUTING.md (maintainer and contribution norms)
- Nixpkgs teams / maintainers documentation in-tree (follow current `maintainers/` and teams sync docs in the revision you ship against)

Fixing Packages with Overlays

Fixing Packages with Overlays

Thesis

A **Nix overlay** is a function `(final: prev: { … })` that patches the package set: bump a version, swap a dependency, apply a local fix — without forking all of nixpkgs. Overlays compose; later overlays see earlier results via `final`.

Mental model

```
  prev  = package set so far
  final = package set after all overlays (fixed point)

  overlays = [
    (final: prev: { hello = prev.hello.overrideAttrs (...); })
  ]
```

Prefer `final` when referring to packages that might themselves be overlaid; prefer `prev` for the pre-overlay package you are replacing.

Worked example

Docs-based flake fragment (`nix` not on this box):

```
# overlays/default.nix
final: prev: {
  mycurl = prev.curl.overrideAttrs (old: {
    configureFlags = (old.configureFlags or []) ++ [ "--disable-ares" ];
  });
}
```

```
# flake.nix (excerpt)
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-25.11";
  outputs = { nixpkgs, ... }: {
    packages.x86_64-linux.default =
      let
        pkgs = import nixpkgs {
```

```
          system = "x86_64-linux";
          overlays = [ (import ./overlays) ];
        };
      in pkgs.mycurl;
  };
}
```

On a Nix host: `nix build .#default` and `nix why-depends` to confirm the override.

Trap

Infinite recursion — referencing `final.hello` inside the overlay that defines `hello` without going through `prev`, or cycles across overlays. Also: overlays that silently rename packages break dependents expecting upstream attribute names.

Boring rule

Keep overlays small, named, and version-controlled next to the flake. Prefer `overrideAttrs` / `override` over copying whole package expressions. Drop the overlay once nixpkgs absorbs the fix.

Try this

1. Overlay a package to add a `patches = [ ./fix.patch ];`.
2. Compose two overlays and predict `final` vs `prev` for a shared dependency.
3. Compare overlays with `pkgs.extend` and NixOS `nixpkgs.overlays`.

Sources

- Nixpkgs manual: Overlays
- NixOS manual: `nixpkgs.overlays`
- nix.dev tutorials on overlays

Cross-Compilation for Multiple Platforms

Cross-Compilation for Multiple Platforms

Goal: Master native cross-compilation in Nixpkgs, cross-compiling C/C++, Rust, and Go binaries from an `x86_64-linux` host to `aarch64-linux` (ARM64) and `riscv64-linux` targets without resorting to slow QEMU emulation, distinguishing `buildPlatform`, `hostPlatform`, and `targetPlatform` accurately.

> **Note**
>
> This chapter completes **Part 6: Packaging Craft**. While previous chapters focused on compiling for the native host platform, production platform engineering requires building software for distributed fleets of varied architectures—ARM64 cloud servers (AWS Graviton, Ampere), embedded boards (Raspberry Pi), and edge gateways.

The Cross-Compilation Problem: Why Emulation Fails

Traditional CI/CD pipelines build multi-architecture binaries using QEMU user emulation (`binfmt_misc`):

```
x86_64 CI Runner  > [QEMU Software Translation]  > Emulated aarch64 gcc / cargo
```

While emulation is conceptually simple, it incurs severe penalties: 1. **Extreme Performance Penalty:** Emulating an ARM64 instruction set on an x86_64 CPU slows execution down by **5x to 20x**. Large C++ or Rust codebases that compile in 5 minutes natively take over an hour under QEMU. 2. **Emulation Instability:** QEMU user-mode frequently crashes on complex multi-threaded compilers, memory barriers, or signal handlers. 3. **Wasted Cloud Compute:** Engineering organizations burn thousands of dollars in CI compute hours waiting for emulated compilers to finish.

The Solution: True **cross-compilation**. An x86_64 compiler binary runs natively at full x86_64 hardware speed while emitting machine code targeted for ARM64 or RISC-V.

The Nixpkgs Platform Model: Triples and System Types

Nixpkgs implements a rigorous, formalized model of cross-compilation based on the GNU toolchain conventions. Every derivation is evaluated in the context of three platforms:

Platform Concept	Nix Attribute	Definition	Practical Meaning
Build Platform	`stdenv.buildPlatform`	The platform where compilation executes	The CPU and OS running `gcc`, `rustc`, or `go` (e.g. `x86_64-linux`)
Host Platform	`stdenv.hostPlatform`	The platform where the built package will run	The machine where the resulting application binary executes (e.g. `aarch64-linux`)
Target Platform	`stdenv.targetPlatform`	The platform for which a compiler generates code	Only relevant when building compilers (e.g. building an ARM64-targeted compiler)

```
Build Platform: x86_64-linux (Your laptop / CI Runner)

        [Cross-Compiler runs at native x86_64 speed]
Host Platform:  aarch64-linux (Your AWS Graviton or Raspberry Pi server)
```

In standard native builds, `buildPlatform == hostPlatform`. In cross-compilation, `buildPlatform != hostPlatform`.

Instant Cross-Toolchains with `pkgsCross`

Nixpkgs includes pre-configured cross-compilation package sets under `pkgsCross`. You do not need to compile a custom cross-toolchain from scratch; Nix provides them out of the box:

```
# Query available cross sets in the REPL
nix repl '<nixpkgs>'
nix-repl> builtins.attrNames pkgsCross
[ "aarch64-multiplatform" "armv7l-hf-multiplatform" "ppc64le" "riscv64" "x86_64-embedded" ..
```

To cross-compile any existing package from Nixpkgs for ARM64:

```
# Cross-compile GNU Hello from x86_64 to aarch64
nix build nixpkgs#pkgsCross.aarch64-multiplatform.hello

# Verify the resulting ELF binary architecture
file ./result/bin/hello
# Output: ELF 64-bit LSB executable, ARM a-arch64, version 1 (SYSV), dynamically linked ...
```

Notice that `hello` was built without executing a single ARM instruction on your machine.

Classifying Inputs: `nativeBuildInputs` vs `buildInputs`

The most critical rule in cross-compiling Nix derivations is placing dependencies in the correct input list:

Input Attribute	Target Execution Platform	What Belongs Here
`nativeBuildInputs`	**Build Platform** (`buildPlatform`)	Compilers, code generators, build systems (`cmake`, `pkg-config`, `bison`, `protobuf`, `cargo`)
`buildInputs`	**Host Platform** (`hostPlatform`)	Libraries and headers that the binary links against (`openssl`, `glibc`, `zlib`, `sqlite`)
`depsBuildBuild`	**Build Platform** (`buildPlatform`)	Tools required to build tools for the build platform

If you accidentally place `pkg-config` in `buildInputs`, Nix will attempt to run an ARM64 binary on your x86_64 machine, crashing with `Exec format error`.

Language Ecosystems in Cross-Compilation

1. Cross-Compiling C/C++ Projects (CMake)

When using `stdenv` with `cmake`, Nixpkgs automatically generates a CMake toolchain file configured with the target triple:

```
# packages/my-c-service/default.nix
{ stdenv, lib, fetchFromGitHub, cmake, pkg-config, openssl, zlib }:

stdenv.mkDerivation rec {
  pname = "sensor-service";
  version = "1.2.0";

  src = fetchFromGitHub {
    owner = "example-corp";
    repo = pname;
    rev = "v${version}";
    hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  };
```

```
  # Build-time tools running on the builder CPU
  nativeBuildInputs = [ cmake pkg-config ];

  # Target libraries for ARM64
  buildInputs = [ openssl zlib ];

  # CMake automatically receives CMAKE_SYSTEM_NAME and CMAKE_C_COMPILER
  meta = with lib; {
    description = "C++ sensor processing service";
    platforms = platforms.linux;
  };
}
```

2. Cross-Compiling Rust with Cargo

`rustPlatform.buildRustPackage` natively supports cross-compilation when invoked from `pkgsCross`:

```
# packages/rust-service/default.nix
{ lib, rustPlatform, fetchFromGitHub, pkg-config, openssl }:

rustPlatform.buildRustPackage rec {
  pname = "telemetry-agent";
  version = "0.4.1";

  src = fetchFromGitHub {
    owner = "example-corp";
    repo = pname;
    rev = "v${version}";
    hash = "sha256-BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB=";
  };

  cargoHash = "sha256-CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC=";

  nativeBuildInputs = [ pkg-config ];
  buildInputs = [ openssl ];

  # Disable tests during cross-compilation (since host binaries cannot run on build machine)
  doCheck = false;
}
```

Building this via `nix build .#pkgsCross.aarch64-multiplatform.telemetry-agent` uses an x86_64 `rustc` with `--target aarch64-unknown-linux-gnu` and cross-compiles OpenSSL cleanly.

3. Cross-Compiling Go Modules

Go has native cross-compilation built into its compiler via `GOOS` and `GOARCH`. `buildGoModule` handles this seamlessly:

```
# packages/go-api/default.nix
{ lib, buildGoModule, fetchFromGitHub }:

buildGoModule rec {
  pname = "gateway-api";
  version = "2.1.0";

  src = fetchFromGitHub {
    owner = "example-corp";
    repo = pname;
    rev = "v${version}";
    hash = "sha256-DDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDD=";
  };

  vendorHash = "sha256-EEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEE=";

  # If CGO is required, ensure cross-gcc is used
  CGO_ENABLED = 0; # Pure Go cross-compiles instantly

  doCheck = false;
}
```

Flake Pattern for Multi-Architecture Outputs

Here is how to expose cross-compiled packages in your `flake.nix`:

```
{
  description = "Multi-Architecture Cross-Compilation Flake";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  };

  outputs = { self, nixpkgs }:
    let
      # Build platform is x86_64-linux
      pkgsNative = import nixpkgs { system = "x86_64-linux"; };
```

```
      # Cross package sets
      pkgsCrossAarch64 = import nixpkgs {
        system = "x86_64-linux";
        crossSystem = { config = "aarch64-unknown-linux-gnu"; };
      };

      pkgsCrossRiscv64 = import nixpkgs {
        system = "x86_64-linux";
        crossSystem = { config = "riscv64-unknown-linux-gnu"; };
      };
    in {
      packages.x86_64-linux = {
        # Native build
        default = pkgsNative.callPackage ./packages/my-service.nix {};

        # Cross-compiled builds for ARM64 and RISC-V
        arm64 = pkgsCrossAarch64.callPackage ./packages/my-service.nix {};
        riscv = pkgsCrossRiscv64.callPackage ./packages/my-service.nix {};
      };
    };
}
```

Now, from an x86_64 laptop or CI server, simply run:

```
nix build .#arm64
nix build .#riscv
```

Common Cross-Compilation Pitfalls

Failure Symptom	Root Cause	Fix
`Exec format error` during build	A host-architecture binary was executed on the build machine (e.g. running unit tests or a code generator)	Set `doCheck = false;` during cross builds. Ensure code generators are placed in `nativeBuildInputs`.

Failure Symptom	Root Cause	Fix
`Cannot find -lssl`	Dynamic library placed in `nativeBuildInputs` instead of `buildInputs`	Move libraries the application links with into `buildInputs`.
`pkg-config` outputs host paths	Using host `pkg-config` instead of target cross-wrapper	Always pull `pkg-config` from `nativeBuildInputs`; Nix provides the target-prefixed wrapper (e.g. `aarch64-unknown-linu`

Checkpoint

- ☐ Explained the difference between `buildPlatform`, `hostPlatform`, and `targetPlatform`.
- ☐ Cross-compiled a binary using `pkgsCross.aarch64-multiplatform` without running QEMU.
- ☐ Correctly separated host libraries in `buildInputs` from build tools in `nativeBuildInputs`.
- ☐ Exported multi-architecture cross builds cleanly in `flake.nix`.

Ops and Fleet

Operations and fleet

Operations and fleet

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by calendar day.

Read chapters in order within the part when learning; skip or skim freely when using the book as a reference.

Baseline: NixOS **26.05**, flakes-first. Nix **2.34.x** is 26.05's default `pkgs.nix`, and **2.35.x** is `nixVersions.latest` (set `nix.package` to use it). Deploy tool CLIs, Disko modes, and generator format names evolve—prefer current upstream READMEs when a flag renames.

What this part is for

Packaging craft produces store paths. **Operations and fleet** ships them safely:

1. **Disks as code** (Disko) so reinstalls are boring
2. **Impermanence** patterns so roots stop accumulating snowflakes
3. **Deploy tools** (deploy-rs, Colmena) for remote activation
4. **Shared modules** so ten hosts do not fork ten firewalls
5. **Images** (nixos-generators) for cloud and edge
6. **Terraform boundary** so cloud APIs and NixOS stay friends
7. **Tests and CI** so regressions die in PRs
8. **GC hygiene**, **Secure Boot/TPM awareness**, **backups** so the fleet survives Monday
9. **Unattended upgrades** so hosts converge on reviewed commits without a human at 03:00

Without this part, NixOS is a clever single host. With it, NixOS is an operations platform.

Prerequisites

Skill	Source
Flake host, rebuild, generations	NixOS host part
Modules as consumer and author	NixOS host part
Secrets (sops-nix or equivalent)	Services and security
Binary caches / builders vocabulary	Packaging craft
Optional: packaging your own modules' packages	Packaging craft

You need **at least one disposable machine or VM**. Disko and impermanence are unsafe on irreplaceable disks.

Chapter map

Chapter	File	Focus
Disko	`01-disko`	Declarative partitioning / filesystems
Impermanence	`02-impermanence`	Ephemeral root + persist lists
deploy-rs	`03-deploy-rs`	Profile deploy over SSH
Colmena	`04-colmena`	Hive, tags, multi-node
Shared modules	`05-shared-modules`	Roles, profiles, host thinness
Images / generators	`06-images-generators`	ISO, qcow, cloud images
Terraform boundary	`07-terraform-boundary`	TF provisions; Nix configures
nixosTest	`08-nixos-tests`	VM integration tests
More testing	`09-more-tests`	Check suite / pyramid
CI pipelines	`10-ci-pipeline`	Forge CI + flakes + cache
GC hygiene	`11-gc-hygiene`	Roots, generations, optimise
Secure Boot / TPM	`12-secure-boot-tpm`	Lanzaboote awareness, LUKS+TPM
Backups	`13-backups`	Restic/Borg/ZFS + restore drill
Unattended upgrades	`14-auto-upgrade`	`system.autoUpgrade` from a reviewed branch, reboot windows, rings, alerts
Is my flake.lock stale?	`16-is-my-flake-lock-stale`	Pin age, distance to branch tip, end-of-life branches, a second nixpkgs without `follows`

Suggested path: **Disko → Impermanence → deploy-rs → Colmena → shared modules**, then **images terraform**, then **tests → CI**, finish with **GC / SB-TPM / backups**.

Mental model — fleet pipeline

```
modules + host data

           nixosConfigurations / hive

                     deploy-rs / Colmena / nixos-rebuild
                     nixos-generators images
                     nixosTest VMs

           Disko layouts + (optional) impermanence
           secrets activation (sops-nix / agenix)
           binary cache + remote builders
```

Stage	Failure mode	First fix locus
Eval	Module merge errors	Shared modules / host
Build	Compile / missing cache	Packaging / CI cache
Copy	SSH / disk	Deploy tool / network
Activate	Switch scripts, boot	Host modules
Prove	Tests / canaries	nixosTest / live probes
Survive	Disk full / no backup	GC / backups

Disk and identity

```
Disko describes blocks
Impermanence describes what must survive reboot
Backups describe how state returns after disaster
Secure Boot/TPM describes boot trust and unlock
```

These four chapters answer different questions. Confusing “declarative disk” with “backed up database” is a classic Capstone failure mode.

Deploy tools at a glance

Tool	Strength	Typical use
`nixos-rebuild --target-host`	Built-in	One-off / small
deploy-rs	Profiles, magic rollback patterns	Small–medium fleets
Colmena	Hive, tags, parallel apply	Multi-role fleets

You do not need both forever. Leave this part with a **deliberate choice** and one working path.

Lab topology suggestions

```
[admin laptop]
       SSH
        lab-vm-1  (Disko reinstall target)
        lab-vm-2  (second Colmena node or remote builder)
        optional: cache VM (Harmonia/Attic)

Cloud stretch:
[OpenTofu] → VM + disk + SG → [image from nixos-generators] → deploy-rs
```

Suggested workspace root:

```
~/lab/nixos/ops/
  disko/
  impermanence/
  deploy/
  colmena/
  modules/          # may be your main flake
  images/
  terraform/
  tests/
  ci/
  gc/
  secureboot/
  backups/
```

Many labs live **inside one monorepo flake** with `nixosConfigurations`, `deploy`, `colmena`, and `checks`—prefer that once you have more than two hosts.

How chapters are structured

1. **Goal** — done criteria
2. **Why it matters** — ops motivation
3. **Theory** — models + complete Nix fragments
4. **Worked examples** — 26.05-shaped
5. **Lab** — multi-step, disposable infrastructure
6. **Pitfalls** — symptom → fix
7. **Checkpoint** — self-review
8. **Commit** — optional discipline

Reading paths

Path A — "Homelab single server → clean reinstall"

1. Disko
2. Impermanence (or conscious reject)
3. deploy-rs
4. Backups
5. GC hygiene

Path B — "Three nodes with roles"

1. Shared modules
2. Colmena (or deploy-rs multi-node)
3. nixosTest + more tests
4. CI pipeline
5. Binary cache (packaging part)

Path C — "Cloud image factory"

1. Shared modules
2. Images / generators
3. Terraform boundary
4. CI pipeline
5. Secure Boot awareness (if metal/edge)

Glossary (part-local)

Term	Meaning
Disko	Declarative disk partitioning/formatting for NixOS
Impermanence	Ephemeral root; explicit persist paths
Profile	Nix profile activation target (system generation)
Hive	Colmena multi-node definition
Tag	Colmena selection label (`@web`)
nixosTest	QEMU VM integration test harness
nixos-generators	Build disk/ISO/cloud images from modules
Boundary	Clear split TF vs Nix responsibilities
GC root	Reference that keeps store paths alive
Lanzaboote	Secure Boot integration for NixOS generations
Restore drill	Prove backups by restoring, not only by job logs

---

Safety rules (non-negotiable)

Rule	Why
Disposable disks for Disko experiments	Wipes are real
Recovery USB before Secure Boot enrollment	Bricks happen
Never paste production secrets into examples	Supply chain + git history
PR forks often must not get push tokens	Cache/signing TCB
GC without understanding roots	Can remove needed paths; rarely deletes running system if roots correct—but can remove rollback options

---

Quality bar for “fleet ready”

Criterion	Evidence
Storage is code	Disko config in git; reinstall notes
Hosts are thin	Roles/profiles, not copy-paste
Remote activate works	deploy-rs or Colmena success log
Regressions caught	`nix flake check` / CI green
State is honest	Backup + restore drill document
Disk won’t fill silently	GC policy + monitoring note
Boot trust known	SB/TPM dossier or enablement notes

Relationship to other parts

Part	Connection
NixOS host	Single-host skills scale to fleets here
Services / security	Secrets, firewall, TLS feed deploy targets
Packaging	Caches/builders feed CI and deploy speed
Internals	Deeper module/eval performance later
Capstone	Combines ops chapters into a rebuildable system
Projects	Optional drills; this part is the ops spine

No day-calendar language; no parallel series cross-links. This is the standalone library’s operations volume.

Anti-patterns

Anti-pattern	Prefer
Hand-edited `/etc` on prod nodes	Module + deploy
Unique firewall per host by copy-paste	Shared profile
Image modules forked from live hosts	One module graph → image + deploy
TF `user_data` installing Docker via curl	Golden NixOS image + Nix modules
Tests only on laptop	CI `nix flake check`
Backup job green without restore	Restore drill
“We’ll enable Secure Boot later” with no notes	Written path + recovery plan

Anti-pattern	Prefer

Part checkpoint (before you leave ops)

- ☐ Disko layout for at least one disposable host; reinstall or format proof
- ☐ Impermanence implemented **or** written reject with inventory of what must persist
- ☐ One successful remote deploy (deploy-rs and/or Colmena)
- ☐ Roles/profiles refactor started (hosts thinner than before)
- ☐ One image artifact built from shared modules **or** documented stretch plan
- ☐ Terraform/Nix boundary sketch written
- ☐ At least one nixosTest (or documented KVM blocker)
- ☐ Three-check suite shape in flake
- ☐ CI workflow runs checks on the forge you use
- ☐ GC policy + roots understanding demonstrated
- ☐ Secure Boot/TPM awareness dossier
- ☐ Backup tool chosen + **restore drill** documented

Suggested lab commit series

```
git commit -m "ops: disko layout and reinstall notes"
git commit -m "ops: impermanence or reject rationale"
git commit -m "ops: deploy-rs end-to-end"
git commit -m "ops: colmena hive and tags"
git commit -m "ops: shared roles and profiles"
git commit -m "ops: nixos-generators image"
git commit -m "ops: terraform boundary sketch"
git commit -m "ops: nixosTest for role module"
git commit -m "ops: flake check suite"
git commit -m "ops: CI workflow"
git commit -m "ops: GC policy"
git commit -m "ops: secure boot / TPM notes"
```

```
git commit -m "ops: backup + restore drill"
```

How to use this part as reference

Symptom	Jump to
Reinstall tribal knowledge	Disko
Root full of mystery state	Impermanence
Need SSH activate	deploy-rs
Need tags / many hosts	Colmena
Copy-paste hosts	Shared modules
Cloud boot drift	Images; Terraform boundary
"Works on my VM"	nixosTest; more tests; CI
Disk full mid-deploy	GC hygiene
Evil maid / datacenter unlock	Secure Boot / TPM
DB gone after rebuild	Backups

Next

Start with **Disko** on a disposable VM: declare ESP + root, format, install from a flake, and prove a second reinstall from the same code. Everything else in this part assumes storage is no longer folklore.

Disko

Disko

Goal: Describe disks declaratively with **Disko**, install or reinstall a lab host from that description, and wire Disko into your flake so storage stops being tribal knowledge.

> **!** Important
>
> Disko can **wipe disks**. Use a disposable VM, spare drive, or cloud instance. Never experiment on a disk with irreplaceable data.

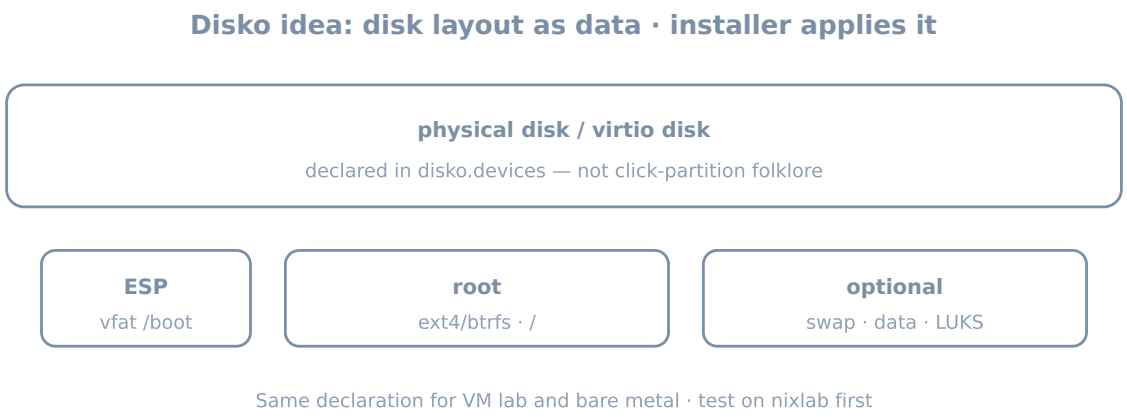


Figure 1: Disko layout

Why it matters

Installer GUIs and one-shot `hardware-configuration.nix` fragments do not scale. Fleet habits require **repeatable storage**: partitions, LVM, btrfs/ZFS layouts, LUKS—as code. Disko is the common 2026 answer in many NixOS fleets.

Without Disko (or equivalent), disaster recovery docs say "partition like last time" and nobody remembers last time.

Theory 1 — What Disko is

Disko turns a Nix attrset into:

1. Partitioning / formatting scripts
2. Mount configuration compatible with NixOS
3. Optionally integrated into `nixos-rebuild` / installer flows

```
disko.devices = { … }

           disko script (destroy/create/mount)
           fileSystems / boot / swap NixOS options
```

Piece	Role
`disko.devices.disk.*`	Physical (or virt) disks
`content.type`	`gpt`, `luks`, `btrfs`, `filesystem`, …
`mountpoint`	Becomes NixOS `fileSystems` entries
CLI modes	destroy / format / mount / disown (names evolve—check `--help`)

Disko is **not** a backup tool. It describes structure; data durability is the backups chapter.

Theory 2 — Minimal GPT + ext4 root (UEFI)

```
# modules/disko-lab.nix
{
  disko.devices = {
    disk = {
      main = {
        type = "disk";
        device = "/dev/vda"; # CHANGE per machine: nvme0n1, sda, …
        content = {
          type = "gpt";
          partitions = {
            ESP = {
              size = "512M";
              type = "EF00";
              content = {
                type = "filesystem";
                format = "vfat";
                mountpoint = "/boot";
                mountOptions = [ "umask=0077" ];
              };
            };
```

```
            root = {
              size = "100%";
              content = {
                type = "filesystem";
                format = "ext4";
                mountpoint = "/";
              };
            };
          };
        };
      };
    };
  };
}
```

Field	Notes
`device`	Host-specific; inject via host module
`type = "gpt"`	UEFI-friendly
ESP `EF00`	EFI system partition type
`mountpoint`	Becomes NixOS `fileSystems`

Prefer stable device paths

```
device = "/dev/disk/by-id/nvme-…"; # better than /dev/nvme0n1 when possible
```

Virt devices (`/dev/vda`) are fine in VMs; metal likes `by-id`.

Theory 3 — Flake integration

```
# flake.nix fragment
{
  inputs.disko.url = "github:nix-community/disko";
  inputs.disko.inputs.nixpkgs.follows = "nixpkgs";

  outputs = { self, nixpkgs, disko, ... }: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        disko.nixosModules.disko
        ./hosts/lab/disko.nix
        ./hosts/lab/configuration.nix
```

```
      ];
    };
  };
}
```

```
# From installer ISO or rescue, after networking + clone:
sudo nix --extra-experimental-features "nix-command flakes" run github:nix-community/disko -
  --mode destroy,format,mount \
  --flake .#lab
# modes/flags evolve-check current Disko CLI help
```

Then install:

```
sudo nixos-install --flake .#lab
```

Or use documented `disko-install` flows when available in your Disko version.

Evaluate mounts without wiping

```
nix eval .#nixosConfigurations.lab.config.fileSystems --json | jq 'keys'
# ensure / and /boot appear as expected after disko module merge
```

Theory 4 — LUKS sketch (common homelab)

```
root = {
  size = "100%";
  content = {
    type = "luks";
    name = "crypted";
    settings.allowDiscards = true; # SSDs; understand TRIM tradeoffs
    content = {
      type = "filesystem";
      format = "ext4";
      mountpoint = "/";
    };
  };
};
```

Unlock method	Notes
Passphrase	Simple lab; interactive boot

Unlock method	Notes
Key file	Automation; protect the key
TPM2	Secure Boot/TPM chapter

Password vs TPM unlock: passphrase in this chapter's lab; TPM path appears later. **Store recovery keys offline** before production encryption.

Theory 5 — btrfs subvolumes (impermanence-ready)

```
content = {
  type = "btrfs";
  subvolumes = {
    "@" = {
      mountpoint = "/";
    };
    "@home" = {
      mountpoint = "/home";
    };
    "@nix" = {
      mountpoint = "/nix";
    };
    "@persist" = {
      mountpoint = "/persist";
    };
    "@log" = {
      mountpoint = "/var/log";
    };
  };
};
```

Impermanence loves `@persist` + wipe `@` on boot. You can create the layout even if you skip impermanence initially—future-you will thank you.

ZFS note

Disko can express ZFS layouts; complexity jumps (pools, datasets, encryption). If you already run ZFS operationally, Disko still helps **repeat** the layout; if you are learning both ZFS and Disko, do ext4/btrfs first.

Theory 6 — Host-specific devices without copy-paste hell

```
# modules/disko-btrfs-template.nix
{
  disko.devices.disk.main = {
    type = "disk";
    # device set per host
    content = { /* shared layout */ };
  };
}
```

```
# hosts/lab/disko.nix
{ lib, ... }:
{
  imports = [ ../../modules/disko-btrfs-template.nix ];
  disko.devices.disk.main.device = lib.mkDefault "/dev/vda";
}
```

```
# hosts/metal/disko.nix
{
  imports = [ ../../modules/disko-btrfs-template.nix ];
  disko.devices.disk.main.device = "/dev/disk/by-id/nvme-…";
}
```

Never commit a fleet-wide wrong `device` path.

Theory 7 — Swap, BIOS, and multi-disk

Swap

```
swap = {
  size = "8G";
  content = {
    type = "swap";
    resumeDevice = true; # if you want hibernation carefully
  };
};
```

Or prefer `zramSwap` in NixOS config and skip disk swap on small VMs—**decide intentionally**.

Legacy BIOS / hybrid

UEFI is the default story here. BIOS/GPT or MBR layouts exist in Disko examples—follow current docs if you must boot old hardware.

Multi-disk

```
disk.main → system
disk.data → /var/lib/postgres bulk (example)
```

Keep system vs data responsibilities clear in `DISK.md`.

Theory 8 — Interaction with hardware-configuration.nix

After Disko-driven install, generated hardware config may still detect NICs and kernel modules. **Filesystem mounts** should come from Disko's merge, not a conflicting hand-maintained `fileSystems` block.

Source of truth	For
Disko	partitions, formats, mountpoints
`hardware-configuration` / `nixos-generate-config`	often CPU microcode, kernel modules, NIC
Host module	hostname, networking, roles

Conflicts: two modules defining different `fileSystems."/"` → merge errors or surprise boots.

Worked example — VM lab checklist

```
1. Create empty qcow2 / virt disk
2. Boot NixOS 26.05 installer ISO
3. Git clone flake (or nix copy)
4. Identify disk: lsblk / by-id
5. Set device in disko config
6. Run disko (destructive)
7. nixos-install --flake .#lab
8. Reboot; verify findmnt / lsblk
```

```
9. Wipe once more; reinstall; time it
```

```
lsblk -f
findmnt /
# source of truth is Nix; /etc/fstab is generated view
cat /etc/fstab
```

DISK.md template

```
# DISK.md
## lab
- device: /dev/vda (virtio)
- layout: GPT ESP 512M + ext4 root
- LUKS: no
- disko command that saved us: …
- recovery: installer ISO + flake URL + this file
```

Lab — multi-step

Suggested workspace: your main lab flake + `~/lab/nixos/ops/disko` notes

Step 1 — Snapshot safety

Confirm target disk is disposable. `lsblk` screenshot/paste into notes.

Step 2 — Add Disko input

Pin via flake; `follows` nixpkgs; update lock.

Step 3 — Write simplest UEFI layout

ext4 root + ESP. Parameterize `device`.

Step 4 — Dry evaluation

```
nix eval .#nixosConfigurations.lab.config.fileSystems --json | jq 'keys'
```

Step 5 — Apply on VM

Run Disko + install **or** rebuild path if reinstalling an already-Disko host per docs.

Step 6 — Reinstall proof (core)

Wipe once more (VM) and reinstall from same flake. Time it. Note pain points.

Step 7 — Optional LUKS or btrfs

Add one complexity only if Step 6 succeeded cleanly.

Step 8 — Document recovery

`DISK.md`: device names per host, LUKS key location, disko command that saved you.

Step 9 — Wrong-disk tabletop

Write a five-line "stop the line" checklist if `lsblk` shows unexpected serials.

Exercises

1. Convert your layout from `device = "/dev/vda"` to a `by-id` path in a metal story (even if hypothetical).
2. Sketch a three-disk server (boot/system/data) as Disko attrset stubs.
3. Explain why Disko does not replace backups (one paragraph).
4. Diff `fileSystems` eval before and after adding a swap partition.
5. Time: first install vs second reinstall—what was slow (download, format, copy)?

Common gotchas

Symptom / mistake	What to do
Wiped wrong disk	Device IDs; prefer `/dev/disk/by-id/…`
No EFI boot	Missing ESP / wrong bootloader module
Device path differs after hardware change	by-id; update host module
Forgot disko module import	`fileSystems` empty/wrong
Swap forgotten	Add swap partition or zram intentionally
Cloud images	Device often `/dev/vda` or `/dev/nvme0n1`; check vendor docs
Conflicting `fileSystems`	One source of truth
LUKS without recovery key	Create and store offline **before** prod
Impermanence without `@persist`	Plan subvolumes early

Checkpoint

- ☐ Disko input wired in flake
- ☐ Declarative layout for lab host committed
- ☐ At least one real format/install or reinstall on disposable media
- ☐ `fileSystems` matches intent after boot
- ☐ `DISK.md` recovery notes written
- ☐ Device path strategy chosen (by-id vs virt path)
- ☐ Optional LUKS/btrfs attempted or deferred with reason

Commit

```
git add .
git commit -m "ops: disko layout and reinstall proof"
```

Write three personal gotchas before impermanence.

Impermanence (idea and patterns)

Impermanence (idea and patterns)

Goal: Implement **or** consciously reject root filesystem impermanence using `impermanence` patterns—proving that unpersisted state disappears on reboot while `/persist` (or equivalent) keeps what you declare.

> Tip
>
> Impermanence is optional as a lifestyle. It is **not** optional to understand. Skipping still requires a written "why." Disk layouts pair with Disko; secrets with sops-nix/agenix. Target: **NixOS 26.05**.

Why it matters

Snowflakes are not only packages—they are **mutated roots**: `/etc` edits, `/var` surprises, leftover packages. Impermanence makes the root a camping tent and `/persist` the shipping container. Combined with Disko btrfs/ZFS layouts, reinstalls become boring—and disaster recovery gets a cleaner story.

Even if you never wipe root on a daily driver, the **inventory habit** (what must survive reboot?) is permanent skill.

Theory 1 — Model

```
boot

    tmpfs or fresh subvolume for /
    bind/mount persisted paths from /persist
    everything else is recreated by activation
```

Path class	Examples	Strategy
Store	`/nix`	Own subvolume/partition; never wipe carelessly
Persist	`/persist/var/lib/postgres`, ssh host keys	Explicit list
Ephemeral	`/var/cache`, junk in `/root`	Accept loss

Path class	Examples	Strategy
Secrets	sops age keys	Persist carefully; permissions matter
Boot	EFI, kernels, generations	Separate story; do not "wipe /boot" casually

What "disappears" means

On reboot, anything not in the persist list is gone *if* you actually wipe/reset root. Soft modes that only document policy without wipe are still useful—but do not claim impermanence until the reboot proof works.

Theory 2 — `impermanence` module (nix-community)

```nix
# flake input
inputs.impermanence.url = "github:nix-community/impermanence";
```

```nix
{
  imports = [ inputs.impermanence.nixosModules.impermanence ];

  environment.persistence."/persist" = {
    hideMounts = true;
    directories = [
      "/var/log"
      "/var/lib/nixos"
      "/var/lib/systemd/coredump"
      { directory = "/var/lib/tailscale"; mode = "0700"; }
    ];
    files = [
      "/etc/machine-id"
      "/etc/ssh/ssh_host_ed25519_key"
      "/etc/ssh/ssh_host_ed25519_key.pub"
    ];
    users.alice = {
      directories = [ "Downloads" ".ssh" ];
      # files = [ ".config/sops/age/keys.txt" ]; # only if you accept this layout
    };
  };
}
```

Exact option paths can track module versions—confirm against current impermanence README when options shift.

neededForBoot

```
fileSystems."/persist".neededForBoot = true;
```

If `/persist` is not available early enough, activation ordering fails in mysterious ways. Treat this as non-negotiable for real setups.

Theory 3 — btrfs rollback-on-boot pattern

Common production pattern:

1. Root subvolume `@`
2. On boot, optionally reset `@` to empty snapshot
3. Mount `@persist` at `/persist`
4. Impermanence binds declared paths

```
# Conceptual boot.initrd or systemd unit logic-use a maintained pattern
# Prefer copying a pattern you understand over opaque one-liners.
# Pair with Disko btrfs subvolumes from the Disko chapter.
```

Layout piece	Role
`@`	Ephemeral root
`@persist`	Declared state
`@nix`	Store (never auto-wipe)
EFI	Bootloader

ZFS native datasets with rollback are an alternative school—pick **one** for the lab. Mixing half-understood btrfs blog posts with half-understood ZFS is how you reinstall for the wrong reason.

Theory 4 — What breaks first (expect it)

Symptom	Cause	Fix
Cannot SSH after reboot	Host keys not persisted	Persist `/etc/ssh/ssh_host_*`

Symptom	Cause	Fix
Machine ID churn	`/etc/machine-id` ephemeral	Persist or accept
sops fails	Age key path wiped	Persist key path; mode `600`
Docker/podman volumes gone	State under `/var/lib`	Persist selected dirs
User linger/services odd	systemd user state	Persist carefully or redesign
Tailscale identity reset	State dir wiped	Persist with tight mode
NetworkManager connections gone	Connection profiles ephemeral	Persist or declarative NM
ACME/certs confusion	Live cert state paths	Persist or use DNS-01 carefully

Iterate: wipe/reboot is a **test suite**, not a one-shot.

Theory 5 — Soft impermanence

Not ready to wipe root? Still gain discipline:

```
# Document-only policy
# - no manual /etc edits
# - all state dirs listed in inventory
# - weekly diff against expected paths
```

Or tmpfs for `/tmp` only—training wheels.

Level	What you prove
A Full	Wipe + persist list + reboot proof
B Partial	Persist mounts without wipe; inventory honest
C Skip	Written architecture why mutable root is OK

All three are valid **if documented**. Silent skip is not.

Theory 6 — Interaction with generations and secrets

Impermanence does **not** replace Nix generations. Store rollbacks and root wipes are different axes:

```
Generation rollback  →  previous system closure
Root wipe            →  empty / except binds from /persist
Secrets              →  must live on persist or offline media
```

Scenario	Tool
Bad module activation	Generation rollback
Unknown files on root	Impermanence wipe
Lost age key	Offline backup; real pain if missing

Theory 7 — Home Manager alignment

If HM manages `~/.config/...` on an impermanent home, either:

- Persist the HM-managed paths you care about, or
- Accept regenerate-from-flake behavior, or
- Keep home on a persistent dataset

Mismatch symptoms: "my HM files vanished" after reboot. Align Home Manager layouts with persist lists.

Services checklist (starting set)

Path	Usually
`/etc/ssh/ssh_host_*`	Persist
`/var/lib/nixos`	Persist
`/var/log`	Persist or ship logs off-box
`/var/lib/postgresql`	Persist + backups
`/var/cache`	Ephemeral
`/tmp`	Ephemeral
sops key	Persist, mode 600
`/var/lib/docker`	Prefer not to; redesign or persist knowingly

Worked example — minimal VM proof

```
# hosts/imperm-lab/configuration.nix fragments
{ inputs, ... }:
{
  imports = [
    ./disko-btrfs.nix
    inputs.impermanence.nixosModules.impermanence
    ./persist.nix
  ];

  fileSystems."/persist".neededForBoot = true;

  users.users.root.openssh.authorizedKeys.keys = [
    "ssh-ed25519 AAAA… lab"
  ];
  services.openssh.enable = true;
  system.stateVersion = "26.05";
}
```

```
# persist.nix
{
  environment.persistence."/persist" = {
    hideMounts = true;
    directories = [
      "/var/log"
      "/var/lib/nixos"
    ];
    files = [
      "/etc/machine-id"
      "/etc/ssh/ssh_host_ed25519_key"
      "/etc/ssh/ssh_host_ed25519_key.pub"
    ];
  };
}
```

Proof protocol:

```
echo "ephemeral-should-die" | sudo tee /root/SCRATCH
echo "persist-me" | sudo tee /persist/PROOF
sudo reboot
# after boot:
cat /root/SCRATCH     # should fail
cat /persist/PROOF    # should work
ssh lab true          # should still work if keys persisted
```

Capture both outcomes in notes. A proof that only tests the success path is incomplete.

Worked example — IMPERMANENCE.md template

```
# IMPERMANENCE.md
## Level
A / B / C

## Inventory (path → persist|regenerate|ignore)
- …

## Wipe mechanism
btrfs reset @ / tmpfs / none

## Proof
- date:
- SCRATCH gone: yes/no
- PROOF present: yes/no
- SSH after reboot: yes/no

## Secrets
- age key path:
- offline backup location:

## Why not full wipe (if B/C)
- …
```

Lab — multi-step

Suggested notes: `~/lab/nixos/ops/impermanence`

Step 1 — Decide commit level

Level	Action
A Full	btrfs/ZFS + impermanence module + wipe proof
B Partial	Persist module without wipe; list future wipe
C Skip	Written architecture review why not (still do path inventory)

Record choice in `IMPERMANENCE.md`.

Step 2 — State inventory

On current lab host:

```
sudo du -xh /var/lib 2>/dev/null | sort -h | tail -n 30
ls /etc/ssh
systemctl --user list-units --failed 2>/dev/null || true
```

Classify each important path: **persist / regenerate / ignore**.

Step 3 — Implement level A or B

Wire flake input; persist SSH keys + one service state dir you actually use. Set modes deliberately.

Step 4 — Proof reboot

Create SCRATCH vs PROOF files; reboot; record outcomes with timestamps.

Step 5 — Secrets path

Confirm sops-nix/agenix key path survives (or document regenerate procedure and offline backup location).

Step 6 — Rollback story interaction

Write 5 lines: store generations vs root wipes.

Step 7 — If Level C

Write one page: threat model, backup story, why mutable root is acceptable for this host class, and what would trigger revisiting (e.g. multi-user, kiosk, fleet sameness).

Step 8 — Persist list review

Print effective list; remove anything that is "comfort mutable root." Goal is intentional survival, not nostalgia.

Step 9 — Second surprise reboot

After a week of use (or a forced second cycle), reboot again; capture new paths that "should have been persisted." Update the list.

Exercises

1. Map every path under `/var/lib` larger than 100 MB to persist/regenerate/ignore.
2. Design persist modes for a postgres host vs a stateless web edge.
3. Explain why wiping `/nix` is not impermanence (one paragraph).
4. Write a failure story: age key on ephemeral root—detection and recovery.
5. Compare tmpfs-root vs btrfs-reset-root (pros/cons table).

Common gotchas

Symptom / mistake	What to do
`/persist` not `neededForBoot`	Boot fails ordering; set flag
Persisted too little	Iterate after each surprise reboot
Persisted too much	You rebuilt a mutable root in disguise
Home Manager vs system paths	Align HM files with persist lists
LUKS + wipe scripts	Order of unlock vs mounts
Forgetting `/nix` separation	Do not wipe the store subvolume
SSH host key regenerate	Clients scream TOFU; persist keys
Age key mode 644	Tighten; treat as root-equivalent
Claiming impermanence without wipe	Call it policy, not impermanence

---

Checkpoint

- ☐ Level A/B/C chosen and documented
- ☐ State inventory exists
- ☐ Reboot proof (or skip rationale) recorded
- ☐ SSH still works after reboot if remote
- ☐ Secrets path understood

- ☐ Generations vs wipe note written
- ☐ Persist list reviewed for “too much”

Commit

```
git add .
git commit -m "ops: impermanence level and reboot proof"
```

Write three personal gotchas before deploy-rs.

deploy-rs

deploy-rs

Goal: Deploy a flake-defined NixOS host to a remote machine with **deploy-rs**, understand activation profiles, and succeed at least once end-to-end over SSH. Target config: **NixOS 26.05**.

> **Note**
>
> `nixos-rebuild --target-host` works; fleets still standardize on deploy tools for profiles, magic rollback, and multi-node later. **deploy-rs** is a common choice for small-to-medium flakes before (or beside) Colmena.

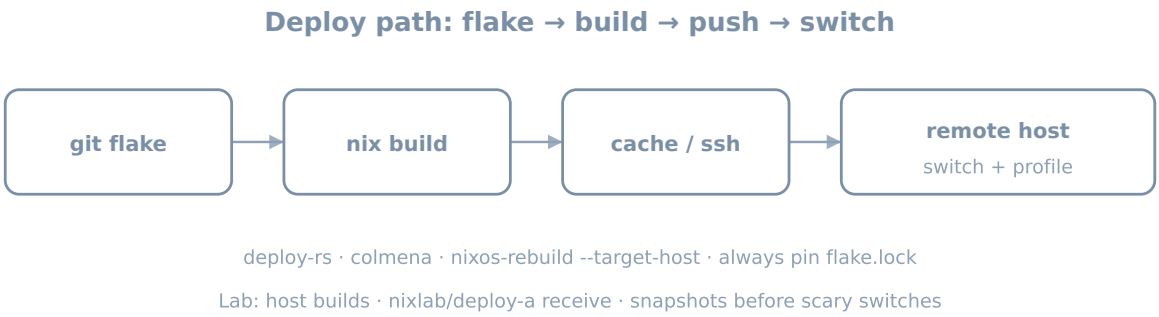


Figure 1: Deploy flow

Why it matters

Pushing a closure and activating it **without** sitting on the machine is the core of fleet ops. This chapter installs the boring muscle memory: SSH auth, profile activation, proof file, second delta, and a safe failure story.

Theory 1 — Deploy pipeline

```
local eval (flake) → build (local/remote) → copy closure → activate → health check → keep/ro
```

Step	Failure mode
Eval	Module errors
Build	Compile / missing cache
Copy	SSH/disk
Activate	Boot/switch scripts fail
Confirm	Custom checks fail → auto rollback (when configured)

Understanding *where* it failed is half the job—read the stage in the log before rewriting modules blindly.

```
deploy-rs
  builds profile path P (NixOS system)
  copies closure of P to target
  activates P (switch-to-configuration)
  optional confirm / magic rollback window
```

Theory 2 — Flake skeleton with deploy-rs

```
{
  description = "deploy-rs lab";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    deploy-rs.url = "github:serokell/deploy-rs";
    deploy-rs.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, deploy-rs, ... }:
    let
      system = "x86_64-linux";
    in {
      nixosConfigurations.lab-vm = nixpkgs.lib.nixosSystem {
        inherit system;
        modules = [ ./hosts/lab-vm/configuration.nix ];
      };

      deploy.nodes.lab-vm = {
        hostname = "lab-vm.example.internal"; # or IP
        profiles.system = {
          user = "root";
          path = deploy-rs.lib.${system}.activate.nixos
            self.nixosConfigurations.lab-vm;
```

```
        };
      };

      checks = builtins.mapAttrs
        (system: deployLib: deployLib.deployChecks self.deploy)
        deploy-rs.lib;
    };
}
```

```
nix shell github:serokell/deploy-rs -c deploy .#lab-vm
# or after packaging deploy-rs in a shell:
deploy .#lab-vm
```

Pin via flake input so CI and laptop match. Exact CLI entrypoints: prefer current serokell docs for your pin.

Theory 3 — SSH prerequisites

```
ssh root@lab-vm.example.internal nix --version
# or deploy user with passwordless sudo / root activation per your policy
```

Requirement	Notes
Nix on target	NixOS has it; confirm version band
Flakes enabled	On target as needed for activation
Disk space	Closures are large first time
Network to substituters	Or pre-pushed cache
Non-interactive auth	Keys, not passwords
Same arch (or builder)	Arch mismatch fails painfully

```
# target snippets
services.openssh.enable = true;
users.users.root.openssh.authorizedKeys.keys = [
  "ssh-ed25519 AAAA… deploy"
];
# Prefer dedicated deploy key; hardening still applies
system.stateVersion = "26.05";
```

Theory 4 — Magic rollback / confirmation

deploy-rs can require confirmation that the new system is healthy; if SSH dies, it rolls back. Read current serokell docs for `confirmTimeout` / magic-rollback options for your version—**enable something** before testing firewall changes remotely.

```
deploy.nodes.lab-vm = {
  hostname = "192.0.2.50";
  # sshOpts = [ "-p" "22" ];
  profiles.system = {
    user = "root";
    path = deploy-rs.lib.x86_64-linux.activate.nixos
      self.nixosConfigurations.lab-vm;
    # magicRollback / confirmTimeout per current docs
  };
};
```

Always keep a **console/VNC** path on the first remote firewall experiment. Magic rollback is not a substitute for out-of-band access.

If this fails…	Look at
Eval on laptop	modules, flake inputs
Build	caches, builders, arch
Copy	SSH, disk, bandwidth
Activate	systemd, scripts, secrets paths
Confirm timeout	network cut by your own firewall

Theory 5 — Build locality

Mode	Idea
Build local, copy	Laptop/CI builds; target only activates
Build on target	Sometimes via remote builder / deploy options
Build on remote builder + cache	Packaging machinery

For lab: local build + copy is fine if architectures match. Cross-arch without a builder is pain—fix remote builders first if needed.

Cache interaction

Deploy once with cold cache (or after GC on target), once with warm substitutes. Record wall times. Deploy without cache is a social incident for fleets.

Theory 6 — Profiles beyond `system`

deploy-rs supports multiple **profiles** per node (system is the common first). Later you might activate user/home profiles; master **one** system profile end-to-end first.

Profile idea	Role
`system`	NixOS activation
Future custom	App-only activations (advanced)

Do not invent multi-profile complexity until single profile is boring.

Theory 7 — vs `nixos-rebuild --target-host`

```
nixos-rebuild switch --flake .#lab-vm --target-host root@lab-vm
```

Concern	rebuild –target-host	deploy-rs
Built-in	Yes	Flake input
Magic rollback ergonomics	Limited	Stronger story
Multi-node	Manual	`deploy.nodes`
Checks	Manual	`deployChecks`
Learning cost	Low	Medium

Write five lines on when you prefer each after you have used both once.

Worked example — trivial visible change

```
# hosts/lab-vm/configuration.nix
{ pkgs, ... }:
{
  networking.hostName = "lab-vm";
  environment.systemPackages = [ pkgs.hello pkgs.git ];
  environment.etc."deploy-proof".text = "deployed-by-deploy-rs\n";
  services.openssh.enable = true;
```

```
  system.stateVersion = "26.05";
}
```

```
deploy .#lab-vm
ssh lab-vm cat /etc/deploy-proof
```

Second deploy: change the text; redeploy; confirm update. Two successful deploys beat one lucky shot.

Generation awareness

```
ssh lab-vm -- sudo nix-env -p /nix/var/nix/profiles/system --list-generations | tail
```

Note generation number before/after deploy. You cannot rollback what you never listed.

Worked example — RUNBOOK seed

```
# Deploy lab-vm
1. VPN/SSH path reachable
2. nix flake lock status known
3. deploy .#lab-vm
4. health: cat /etc/deploy-proof && systemctl is-system-running
5. if fail: console access; list generations; rollback
```

Lab — multi-step

Suggested notes: `~/lab/nixos/ops/deploy`

Step 1 — Target ready

Disposable NixOS VM with SSH key auth. Snapshot the VM **before** first deploy.

Step 2 — Add deploy-rs input

`follows` nixpkgs; lock; commit.

Step 3 — Wire `deploy.nodes`

Match `nixosConfigurations` name; set hostname/IP. Prefer IP in lab DNS chaos.

Step 4 — First deploy

```
nix flake check   # includes deployChecks if wired
deploy .#lab-vm
# -s / skip-checks flags: know them; don't live on skip forever
```

Capture full terminal output in notes.

Step 5 — Prove activation

```
ssh lab-vm cat /etc/deploy-proof
ssh lab-vm nixos-version
ssh lab-vm readlink /run/current-system
```

Step 6 — Second deploy delta

Change `environment.etc` text; redeploy; confirm update. Note wall time vs first deploy (cache effects).

Step 7 — Failure drill (safe)

Deploy a config that adds a harmless broken oneshot **or** bad package, observe failure handling; restore from snapshot if needed. Do **not** lock yourself out on purpose without console.

Step 8 — Compare one-liner

Once, run classic `nixos-rebuild --target-host`. Write comparison notes.

Step 9 — Document RUNBOOK seed

As above.

Step 10 — Generation awareness after deploy

List generations; record numbers.

Step 11 — Cache interaction

Cold vs warm deploy times.

Step 12 — Multi-node sketch (optional)

Add a second node attr in `deploy.nodes` even if only one is real—prove eval shape.

Exercises

1. Draw the pipeline and mark where magic rollback acts.
2. Design a deploy key policy (who has it, rotation).
3. Estimate first-deploy closure size for your lab host (`nix path-info -Sh`).
4. Write a "deploy failed at copy" runbook paragraph.
5. Compare deploy-rs vs Colmena for a 2-node vs 20-node mental model (preview).

Common gotchas

Symptom / mistake	What to do
Arch mismatch	Build on same system or use remote builder
Permission denied	root keys / sudo deploy user
Out of disk on target	GC; grow disk
Hang on activation	Console; magic rollback timeouts
`deploy` not found	`nix shell` the tool
Hostname DNS	Use IP for lab
Checks fail locally	Fix `deployChecks` or understand impure needs
`follows` forgotten	Mixed nixpkgs between deploy-rs and host
Firewall lockout	Console first; enable magic rollback
Secrets missing on target	sops keys / paths must exist before service start

---

Checkpoint

- ☐ deploy-rs in flake
- ☐ Successful remote activation
- ☐ Visible file/package change verified over SSH
- ☐ Second delta deploy succeeded
- ☐ Comparison note vs `nixos-rebuild --target-host`
- ☐ RUNBOOK seed written
- ☐ Generation numbers recorded
- ☐ Magic rollback / confirm awareness

Commit

```
git add .
git commit -m "ops: deploy-rs end-to-end"
```

Write three personal gotchas before Colmena.

Colmena

Colmena

Goal: Express a small hive with **Colmena**, deploy at least two nodes (one may be local), use tags, and know when Colmena wins over deploy-rs for fleet shape. Hosts pin **NixOS 26.05**.

> **Note**
>
> deploy-rs is excellent node/profile oriented glue. **Colmena** shines when you think in **hives**: many hosts, tags (`@web`, `@db`), parallel apply, and a flake hive attribute. Real orgs often evaluate both; leave ops with a deliberate choice, not two half-wired tools forever.

Why it matters

One remote deploy proves the pipe. Fleets need **selection**, **defaults**, and **blast-radius control**. Colmena's mental model maps cleanly onto roles you will build in the shared modules chapter.

Theory 1 — Hive mental model

```
meta (nixpkgs, specialArgs)
  nodes
    host-a → NixOS module stack
    host-b → …
  defaults → shared modules
```

Concept	Role
`meta.nixpkgs`	Package set for evaluation
`defaults`	Applied to every node
`nodes.<name>`	Per-host config + deployment keys
tags	Selection subsets for apply
`specialArgs`	Inject libs/secrets helpers carefully

Theory 2 — Flake-based Colmena (modern)

```
{
  description = "colmena hive lab";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs, ... }: {
    colmena = {
      meta = {
        nixpkgs = import nixpkgs { system = "x86_64-linux"; };
        # specialArgs = { inherit inputs; };
      };

      defaults = { pkgs, ... }: {
        # shared baseline modules
        imports = [ ../modules/profiles/baseline.nix ];
        system.stateVersion = "26.05";
        deployment.tags = [ "all" ];
      };

      edge = { name, nodes, ... }: {
        deployment = {
          targetHost = "edge.lab.internal";
          targetUser = "root";
          tags = [ "web" "edge" ];
        };
        imports = [ ../hosts/edge/configuration.nix ];
      };

      db = {
        deployment = {
          targetHost = "db.lab.internal";
          targetUser = "root";
          tags = [ "db" ];
        };
        imports = [ ../hosts/db/configuration.nix ];
      };
    };
  };
}
```

CLI shapes evolve—typical patterns:

```
nix shell nixpkgs#colmena -c colmena --help
colmena build
```

```
colmena apply --on @web
colmena apply-local --sudo   # local node stories
```

Confirm current Colmena flake attribute and CLI against upstream docs for your pin.

Theory 3 — Tags and blast radius

Tag pattern	Use
`@all` / `all`	Everything (dangerous)
`@web`	Edge nodes
`@db`	Data plane
`@canary`	Single host first

```
# Conceptual - flags vary by version
colmena apply --on @web
colmena apply --on edge
```

Rule: Never default to "apply entire hive" on Friday. Prefer tag + canary.

Parallelism

Colmena can apply many nodes in parallel. Great for speed; terrible for correlated mistakes. Start serial for scary changes (firewall, disk, boot).

Theory 4 — Evaluation vs activation

```
colmena build     → realize all selected system closures
colmena apply     → build + copy + switch on targets
```

Command family	Risk
build/eval only	Low—catches module errors
apply one tag	Medium
apply all	High

Wire flake checks that **eval** the hive even when you do not apply.

Theory 5 — Relationship to `nixosConfigurations`

Many flakes expose both:

```
nixosConfigurations.edge = nixpkgs.lib.nixosSystem { … };
# colmena.nodes.edge imports same modules
```

Approach	Pros
Single module entry per host	DRY
Generate Colmena from nixosConfigurations	Less drift
Separate trees	Fast way to diverge—avoid

Shared modules chapter makes this maintainable.

Theory 6 — When Colmena vs deploy-rs

Situation	Lean
1–3 nodes, profile/rollback focus	deploy-rs
Many nodes, tags, parallel ops	Colmena
Already invested in one	Don't dual-run forever
Need both temporarily	Document primary

Write a decision paragraph in notes after you deploy with both once (or one deeply).

Theory 7 — Secrets and specialArgs

```
meta = {
  nixpkgs = import nixpkgs { system = "x86_64-linux"; };
  specialArgs = { inherit inputs; };
};
```

Inject	Careful of
`inputs`	Large eval; fine when needed
Secret **paths**	OK
Secret **strings**	Never—store contamination

Inject	Careful of

sops-nix still decrypts on the host; Colmena only delivers config that references paths.

Worked example — two-node hive (one local)

```
colmena = {
  meta.nixpkgs = import nixpkgs { system = "x86_64-linux"; };

  defaults = {
    deployment.tags = [ "all" ];
    services.openssh.enable = true;
    system.stateVersion = "26.05";
  };

  laptop = {
    deployment = {
      allowLocalDeployment = true;
      targetHost = null; # local patterns per docs
      tags = [ "local" ];
    };
    environment.etc."hive-proof".text = "laptop\n";
  };

  vm = {
    deployment = {
      targetHost = "192.0.2.50";
      targetUser = "root";
      tags = [ "vm" "web" ];
    };
    environment.etc."hive-proof".text = "vm\n";
    imports = [ ./hosts/vm.nix ];
  };
};
```

```
colmena apply --on @vm
ssh root@192.0.2.50 cat /etc/hive-proof
```

Lab — multi-step

Suggested notes: `~/lab/nixos/ops/colmena`

Step 1 — Install Colmena in a shell

```
nix shell nixpkgs#colmena -c colmena --version
```

Step 2 — Define hive with defaults

Baseline SSH + stateVersion + one shared import.

Step 3 — Two nodes

One may be local; one remote VM. Distinct `hive-proof` texts.

Step 4 — Build only

```
colmena build
# or build --on @vm
```

Fix eval errors before apply.

Step 5 — Apply tagged subset

Apply only `@vm` (or equivalent). Prove file on remote.

Step 6 — Tag experiment

Add `@web`; apply by tag; document selection syntax you used.

Step 7 — Blast-radius note

Write: "scary change procedure" = canary host → tag → all.

Step 8 — Decision card

Colmena vs deploy-rs for *your* lab size.

Step 9 — Optional parallel apply

Apply two nodes; note ordering/logs; decide if parallel is for you.

Step 10 — DRY modules

Point both nodes at a shared profile module (preview of shared modules chapter).

Exercises

1. Design tags for web/db/obs/canary.
2. Eval-only CI job sketch for the hive.
3. Failure mode: wrong `targetHost`—what do logs show?
4. Convert a deploy-rs node list into a Colmena hive sketch.
5. Security: who can run `colmena apply --on @all`?

Common gotchas

Symptom / mistake	What to do
Applied all nodes by accident	Tags; confirm prompts; canary
Hive eval huge/slow	Reduce specialArgs; modularize
Drift vs nixosConfigurations	Share modules
Local deploy confusion	Read allowLocalDeployment docs
SSH user wrong	targetUser + keys
Mixed nixpkgs	Single meta.nixpkgs
Parallel firewall change	Serial + console
Secrets in hive attrset	Paths only

Checkpoint

- ☐ Hive evaluates
- ☐ At least one remote (or local) apply succeeded
- ☐ Tags used for selection
- ☐ Build-only vs apply distinction clear
- ☐ Decision note vs deploy-rs
- ☐ Blast-radius procedure written

Commit

```
git add .
git commit -m "ops: colmena hive and tags"
```

Write three personal gotchas before shared modules.

Shared modules across hosts

Shared modules across hosts

Goal: Refactor fleet config into **roles** and **profiles** so hosts become thin compositions—no copy-paste SSH/firewall/observability blocks.

Why it matters

Two Colmena nodes already tempt duplication. Ten nodes without roles guarantee drift: one host missing a hardening line is an incident. Modules are the compression algorithm of NixOS fleets.

Theory 1 — Layering

```
nixpkgs modules
    your library modules (roles/, profiles/, services/)
          host modules (hosts/edge)
                secrets / hardware / disko (host-specific)
```

Layer	Contains	Avoid
Role	`web`, `db`, `observer` — purpose	Hardware paths
Profile	`baseline`, `hardened`, `workstation`	One-off hacks
Host	hostname, addresses, disk, keys	Recreating baseline
Service module	Single app options	Global side effects silent

Theory 2 — Role module pattern

```
# modules/roles/web.nix
{ config, lib, pkgs, ... }:
let
  cfg = config.roles.web;
in {
  options.roles.web = {
```

```
    enable = lib.mkEnableOption "web edge role";
    domain = lib.mkOption {
      type = lib.types.str;
      example = "service.example.com";
    };
  };

  config = lib.mkIf cfg.enable {
    networking.firewall.allowedTCPPorts = [ 80 443 ];
    services.nginx.enable = true;
    # services.nginx.virtualHosts.${cfg.domain} = { … };
    environment.systemPackages = [ pkgs.curl ];
  };
}
```

```
# hosts/edge/configuration.nix
{
  imports = [
    ../../modules/profiles/baseline.nix
    ../../modules/roles/web.nix
  ];
  roles.web = {
    enable = true;
    domain = "edge.example.com";
  };
  networking.hostName = "edge";
}
```

Hosts should read like a **shopping list**, not a novel.

Theory 3 — Profile module pattern

```
# modules/profiles/baseline.nix
{ lib, pkgs, ... }:
{
  services.openssh = {
    enable = true;
    settings.PasswordAuthentication = false;
  };
  environment.systemPackages = [ pkgs.vim pkgs.git pkgs.htop ];
  time.timeZone = lib.mkDefault "UTC";
  i18n.defaultLocale = lib.mkDefault "en_US.UTF-8";
}
```

```nix
# modules/profiles/hardened.nix
{
  imports = [ ./baseline.nix ];
  # tighter ssh, sysctl, etc.-compose, don't fork baseline forever
}
```

Profile	Intent
`baseline`	Every host
`hardened`	Prod-ish
`workstation`	GUI/dev laptops
`lab`	Relaxed for learning

Theory 4 — Options API design

Practice	Why
`mkEnableOption`	Clear on/off
Typed options	Catch typos at eval
`mkDefault` in profiles	Hosts can override
`mkForce` sparingly	Fight club otherwise
Document `example`	Next human

```nix
options.roles.db = {
  enable = lib.mkEnableOption "database role";
  package = lib.mkPackageOption pkgs "postgresql_16" { };
  listenAddresses = lib.mkOption {
    type = lib.types.listOf lib.types.str;
    default = [ "127.0.0.1" ];
  };
};
```

Theory 5 — Directory layout that scales

```
flake.nix
modules/
  profiles/
    baseline.nix
```

```
    hardened.nix
  roles/
    web.nix
    db.nix
  services/
    myapp.nix
  common/
    sops.nix
    networking-lab.nix
hosts/
  edge/
    configuration.nix
    disko.nix
    hardware.nix
  db/
    configuration.nix
    disko.nix
```

Rule	Detail
No `hosts/*` copying firewalls	Profiles/roles
Disko/hardware only under host	Physical truth
Secrets wiring shared; keys host-specific	sops creation rules

---

Theory 6 — SpecialArgs and lib helpers

```
nixpkgs.lib.nixosSystem {
  specialArgs = { inherit inputs; myLib = import ./lib; };
  modules = [ ./hosts/edge ];
};
```

Keep helpers pure and small. A 2k-line `lib/default.nix` becomes a second language.

Theory 7 — Testing shared modules

Roles without tests drift. Preview:

```
# tests/web-role.nix - nixosTest imports roles.web
nodes.machine = {
```

```
  imports = [ ../modules/roles/web.nix ];
  roles.web.enable = true;
  roles.web.domain = "test.local";
};
```

Full harness in the nixosTest chapter—design modules so tests can import them without host hardware.

Worked example — before/after host thinness

Before (bad)

```
# hosts/edge/configuration.nix - 200 lines of ssh, firewall, nginx, users…
```

After (good)

```
{
  imports = [
    ./hardware.nix
    ./disko.nix
    ../../modules/profiles/hardened.nix
    ../../modules/roles/web.nix
    ../../modules/common/sops.nix
  ];
  networking.hostName = "edge";
  networking.hostId = "deadbeef"; # if needed
  roles.web = {
    enable = true;
    domain = "edge.example.com";
  };
  # host-only networking addresses, etc.
  system.stateVersion = "26.05";
}
```

Measure: host file line count before/after; aim for sharp drop.

Worked example — flake wiring both deploy tools

```
# same modules feed:
# - nixosConfigurations.edge
# - colmena.edge
# - deploy.nodes.edge
# - nixos-generators image for edge
```

One module graph is the entire point of this chapter.

Lab — multi-step

Suggested work: your fleet flake

Step 1 — Inventory duplication

`rg -n "openssh|allowedTCPPorts|nginx" hosts modules | head`

List copy-pasted blocks.

Step 2 — Extract `profiles/baseline.nix`

SSH + packages + stateVersion defaults. Import from two hosts.

Step 3 — Extract one role

`roles.web` or `roles.lab-tooling` with enable flag.

Step 4 — Thin hosts

Move host-specific only. Commit before/after line counts in notes.

Step 5 — Override test

Use `lib.mkForce` or host option once deliberately; document who wins.

Step 6 — Wire into Colmena/deploy-rs

Prove both nodes still deploy/eval.

Step 7 — Option docs

Add `example` and a one-line comment per role option.

Step 8 — Negative test

Enable role without required option; ensure eval fails clearly.

Step 9 — README map

```
# modules/README.md
profiles/ - …
roles/ - …
```

Exercises

1. Design options for a `roles.observer` (metrics scrapes only).
2. Draw merge priority: baseline vs hardened vs host.
3. Find three nixpkgs modules that are good style models; note patterns.
4. Refactor a third host using only imports + three option sets.
5. Write a "no mkForce without comment" team rule.

Common gotchas

Symptom / mistake	What to do
Role enables services globally without `mkIf`	Always gate on `cfg.enable`
Host still 500 lines	Keep extracting
Option name typos	Types + eval tests
Circular imports	Layer strictly
Hardware in roles	Move to host
Silent `mkForce` wars	Prefer defaults + host overrides
specialArgs secret strings	Paths only
Untested roles	nixosTest next

Checkpoint

- ☐ Baseline profile shared by 2 hosts
- ☐ At least one role module with options
- ☐ Hosts thinner (measured)
- ☐ Deploy/hive still evaluates
- ☐ modules README map
- ☐ Clear enable-option gating

Commit

```
git add .
git commit -m "ops: shared roles and profiles"
```

Write three personal gotchas before images.

Images with nixos-generators

Images with nixos-generators

Goal: Build a bootable artifact (cloud image, ISO, or VM format) from your flake modules using **nixos-generators** (and related patterns), then boot it once and prove it is the *same* module graph as a live host—not a snowflake fork.

> **Note**
>
> Images are **first-class deploy targets**, not a side quest. The win is one module tree → install path *and* golden image path. NixOS **26.05** pin stays consistent with the rest of this book.

Why it matters

Not every node is `nixos-install` by hand. Cloud and edge want **images**: qcow2, AMIs (via pipelines), raw disks, installer ISOs. Generating images from the **same modules** as deploy targets closes a huge consistency gap—and catches "works on my VM but not in the AMI" earlier.

Without an image story you either:

- Hand-maintain cloud user-data that reimplements half of NixOS, or
- Drift image hosts away from Colmena/deploy-rs nodes until nobody trusts either.

Theory 1 — Image = NixOS config + format

```
same modules as hosts/edge

nixos-generators / make-disk-image / sd-image

artifact (iso, qcow2, raw, gce, azure, …)
```

Format (examples)	Use
`iso`	Installer / live recovery
`qcow` / `qcow2` / `vmware` / `virtualbox`	Hypervisors

Format (examples)	Use
`amazon` / `gce` / `azure`	Cloud (tooling evolves)
`sd-card` / `raw`	SBC / `dd`

Check the **current** nixos-generators format list for your pin—names and defaults change. Prefer the project README over stale blog posts.

Mental model

An image build is still a NixOS evaluation: modules merge, activation scripts exist, store paths land on a disk layout the *format* owns. You are not "exporting files"; you are **realizing a system derivation into a disk picture**.

Theory 2 — nixos-generators integration shapes

Flake app / package patterns

Community patterns include:

```
# Conceptual - verify against current nixos-generators docs
inputs.nixos-generators.url = "github:nix-community/nixos-generators";
inputs.nixos-generators.inputs.nixpkgs.follows = "nixpkgs";
```

```
nix run github:nix-community/nixos-generators -- --help
# format flags and flake integration evolve
```

Module specializations

Images often need:

Concern	Image-friendly approach
Hostname	Generic or cloud-init later
SSH keys	Inject via cloud-init / metadata; or bake lab keys carefully
Disko	May differ from bare-metal Disko; know format's disk story
Size	Minimal profiles; avoid workstation GUIs on cloud images
First boot	growpart, regenerate machine-id policies

Theory 3 — Same modules, different entrypoints

```
# modules shared
imports = [
  ../modules/profiles/baseline.nix
  ../modules/roles/web.nix
];

# hosts/edge - deploy target
# images/edge - generators entry with format-specific tweaks only
```

Allowed in image entry	Forbidden without reason
Format modules	Entirely different nginx config
Cloud agent packages	Hardcoded prod secrets
Temporary serial console	Divergent firewall policy forever

If image and live host diverge by 200 lines, you have two products.

Theory 4 — Cloud-init and metadata boundary

Terraform/cloud metadata often supplies:

- SSH keys
- Hostname
- Network

NixOS modules should **consume** those mechanisms or disable conflicting agents intentionally.

Smell	Fix
500-line bash user-data	Bake image; tiny user-data
Image with prod secrets	Inject at instance start via sops/metadata carefully
No SSH to new instance	Console + known image user/key story

Theory 5 — Build performance and caches

Image builds are **heavy**. Use:

- Binary caches for the system closure
- Remote builders with disk
- CI only on `main` or nightly for full images; PR builds eval/smaller checks

```
nix build .#images.edge-qcow -L
ls -lh result*
```

Theory 6 — ISO as recovery

A flake-built installer ISO that includes your SSH key and network helpers is a **recovery product**:

```
laptop dies → boot ISO → clone flake → Disko → install
```

Pair with Disko chapter reinstall proofs. Store ISO hashes in runbooks if you distribute them.

Worked example — qcow lab image (shape)

```
# Illustrative structure - adapt to current generators API
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  # inputs.nixos-generators = …;

  outputs = { self, nixpkgs, ... }: {
    nixosConfigurations.edge-image = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        ./modules/profiles/baseline.nix
        ./modules/roles/web.nix
        ./images/edge-image.nix
      ];
    };
    # packages.x86_64-linux.edge-qcow = … generators expression …
  };
```

```
}
```

```
# images/edge-image.nix
{ lib, modulesPath, ... }:
{
  imports = [
    (modulesPath + "/profiles/qemu-guest.nix")
  ];
  services.openssh.enable = true;
  # lab only:
  users.users.root.openssh.authorizedKeys.keys = [ "ssh-ed25519 AAAA… lab" ];
  system.stateVersion = "26.05";
}
```

Boot under qemu:

```
qemu-system-x86_64 -m 2048 -enable-kvm -drive file=./result/nixos.qcow2,if=virtio
# paths vary by generator output layout
```

Prove:

```
ssh root@… cat /etc/os-release
# compare packages/options to live edge host where possible
```

Worked example — consistency checklist

```
# IMAGE-CONSISTENCY.md
- Module list shared with hosts/edge: yes/no
- Divergences (intentional): …
- Format: qcow2 / iso / …
- Build command:
- Boot proof date:
- SSH method:
```

Lab — multi-step

Suggested notes: `~/lab/nixos/ops/images`

Step 1 — Pick a format

qcow2 for local hypervisor is the friendliest lab path. ISO if you care about recovery.

Step 2 — Wire generators input

`follows` nixpkgs; read current README for flake examples.

Step 3 — Image module entry

Import the **same** baseline/role as a live host. Only format-specific imports extra.

Step 4 — Build artifact

```
# per current docs
nix build … -L
du -h result*
```

Step 5 — Boot once

QEMU/libvirt/VirtualBox—whatever you have. Reach SSH or console.

Step 6 — Consistency proof

Compare `nixos-version`, hostname policy, and one role-provided package/service to the live host story.

Step 7 — Size diet (optional)

Remove fat packages; rebuild; note image size delta.

Step 8 — Document CI policy

When do you rebuild images? Who may publish them?

Step 9 — Stretch cloud

If credentials exist, note AMI/GCE path as stretch—do not block on cloud.

Exercises

1. List five things that must never be baked into a published image.
2. Design a versioning scheme for golden images (flake rev + date).
3. Explain why Disko for metal and cloud image root disk may differ.
4. Sketch a pipeline: CI builds qcow → uploads → TF launches.
5. Recovery ISO contents checklist (network, git, ssh, disko).

Common gotchas

Symptom / mistake	What to do
Format name changed	Read current generators docs
Image huge	Minimal profile; multi-out not enough—fewer packages
Cannot SSH	Console; authorizedKeys; cloud-init race
Divergent modules	Re-unify with live host
Build OOM/disk	Builder sizing; GC
Secrets in image	Remove; inject later
Slow every PR	Cache; nightly images
Wrong system arch	Match hypervisor/cloud

Checkpoint

- ☐ Generators (or equivalent) wired
- ☐ Image built from shared modules
- ☐ Booted once with proof
- ☐ Consistency notes vs live host
- ☐ CI/publish policy sketched
- ☐ No secrets baked intentionally

Commit

```
git add .
git commit -m "ops: nixos-generators image from shared modules"
```

Write three personal gotchas before Terraform boundary.

Terraform boundary

Terraform boundary

Goal: Draw a clean boundary—**Terraform (or OpenTofu) provisions cloud resources; NixOS owns system configuration**—and produce a sketch you could implement without turning TF into a second module system. Images from the generators chapter + deploy from deploy-rs/Colmena sit on the Nix side; pin OS stories to **26.05**.

> **Note**
>
> Full multi-cloud automation is out of scope. A **sharp sketch** with one example resource graph is the bar. Implement only if you already have cloud credentials and want the stretch.

Why it matters

Nix people overreach into provisioning; TF people template snowflake user-data forever. The 2026 happy path for many teams:

```
Terraform/OpenTofu: VPC, VM, disks, DNS, IAM, object storage

          outputs: IP, IDs, SSH target

Nix flake: Disko/image/deploy-rs/Colmena configure the OS
```

Without a boundary doc, every new hire reinvents the split—and usually badly.

Theory 1 — Responsibility split

Concern	Owner	Why
Cloud account / IAM	TF	Lifecycle & policy engines
VM size, subnet, SG	TF	Provider API native
Disk attach / snapshots API	TF	Cloud object model
Packages, services, users	Nix	Pure rebuild story
SSH hardening	Nix	Modules
App releases	Nix / CI	Closures
DNS record for service	Often TF	Or Nix + provider with care
Golden AMI/GCE image build	Nix + CI	Images chapter

Concern	Owner	Why
Instance launch from image	TF	Provider resource

Smell: 500-line `user_data` shell scripts installing docker by curl.

Smell: Nix trying to create AWS VPCs with ad-hoc `runCommand` and keys in the store.

Theory 2 — Handoff patterns

Pattern A — Image + launch

```
CI: nix build image → upload AMI/GCE image
TF: instance from image ID
Nix: optional second-phase deploy-rs for fast iteration
```

Pattern B — Generic distro image + first deploy

```
TF: stock NixOS or minimal image + SSH key
deploy-rs/Colmena: converge to desired closure
```

Pattern C — NixOS on metal (no TF)

```
Disko + install + deploy-rs
TF only for adjacent cloud DNS/objects if any
```

Pattern	Best when
A	Immutable cattle, slow-changing base
B	Fast module iteration
C	Homelab / colo

Theory 3 — What to put in TF state vs flake lock

Artifact	System of record
VPC ID, instance ID	TF state (remote backend)
nixpkgs rev, module graph	`flake.lock` + git
Deploy proof file	Nix modules
Object storage bucket for backups	TF create; Nix restic config consumes name
DNS API token	Secret manager; neither plaintext git

Never put TF state in the Nix store. Never put flake lock "truth" only in a random S3 object without git.

Theory 4 — OpenTofu vs Terraform

For this book, treat **OpenTofu** and **Terraform** as interchangeable at the boundary level. Choose based on org license/policy. Examples may say TF generically.

```
# versions.tf - illustrative
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    # provider pins…
  }
}
```

Theory 5 — Minimal resource graph (sketch)

```
[network]
  vpc
  subnet
  security_group (22 from admin CIDR; 80/443 public if web)
[compute]
  instance (ami/image from Nix pipeline or stock)
  eip / dns A record
[data]
  data_volume (optional)
  backup_bucket
```

```
# pseudo-HCL - not a full provider config
resource "aws_instance" "edge" {
  ami           = var.nixos_ami # produced by image pipeline
  instance_type = "t3.small"
  subnet_id     = aws_subnet.main.id
  vpc_security_group_ids = [aws_security_group.edge.id]
  key_name      = var.ssh_key_name # or cloud-init only

  tags = {
    Role = "edge"
    Nix  = "flake-managed"
  }
}

output "edge_ip" {
  value = aws_instance.edge.public_ip
}
```

```
# after apply
deploy .#edge   # hostname from TF output
```

Theory 6 — Secrets across the boundary

Secret	Where
Cloud provider credentials	TF env / CI OIDC—not flake
SSH deploy key	Agent / CI secret—not world-readable
sops age keys	Host persist / secure bootstrap
DB passwords	sops-nix on Nix side after instance exists

TF `sensitive` outputs still land in state—protect state backends.

Theory 7 — Anti-patterns gallery

Anti-pattern	Prefer
TF `remote-exec` apt-get	NixOS image + modules

Anti-pattern	Prefer
Nix `pkgs.writeScript` creating AWS resources with static keys	TF + IAM roles
Duplicating SG rules in nginx modules only	TF SG + Nix firewall both intentional
Mutating instance packages by hand post-TF	redeploy Nix
One mega root module for all envs	workspaces/stacks + clear envs

---

Worked example — BOUNDARY.md template

```
# BOUNDARY.md
## Owners
- Cloud resources: Terraform/OpenTofu in repo `infra/`
- OS/app: Nix flake in repo `nix/`

## Handoff
1. TF apply → outputs IP, instance id, bucket name
2. Update flake host inventory / sops if needed
3. deploy-rs/Colmena apply
4. Health checks

## Image pipeline
- Built by: CI job …
- Consumed by: TF var nixos_ami

## Non-goals
- TF will not install packages
- Nix will not create VPCs

## Secrets
- …
```

Worked example — inventory bridge

```
# hosts/edge/deployment-ip.nix - generated or hand-updated from TF output
{
  # Prefer automation later; honesty first
  # deployment target:
```

```
  # networking…
}
```

```
tofu output -raw edge_ip > /tmp/edge_ip
# feed deploy hostname
```

Automating TF output → flake is optional stretch; **document the manual bridge** first.

Lab — multi-step

Suggested notes: `~/lab/nixos/ops/terraform`

Step 1 — Write BOUNDARY.md

Fill the template even with zero cloud resources.

Step 2 — Draw resource graph

ASCII or mermaid for one environment (lab).

Step 3 — Choose pattern A/B/C

Record why for your context.

Step 4 — Optional implement

If credentials exist: one VM + SG + output IP; deploy Nix once.

Step 5 — User-data audit

If any user-data remains, cap it at <20 lines or delete.

Step 6 — State backend note

Where would state live? Who accesses it?

Step 7 — Failure story

"TF created VM, deploy failed"—recovery steps.

Step 8 — Stretch destroy

`tofu destroy` only for disposable lab; never prod without ceremony.

Exercises

1. Assign 15 concerns to TF vs Nix vs Shared in a table.
2. Rewrite a fictional 80-line user-data into image+modules bullets.
3. Design IAM least privilege for CI that builds AMIs.
4. Explain why flake lock is not TF state (one paragraph).
5. Multi-env: how do you stop lab AMI IDs leaking into prod TF?

Common gotchas

Symptom / mistake	What to do
Drift: hand-edited security group	TF is source; import or recreate
Deploy to wrong IP	Outputs + inventory discipline
Secrets in TF console output logs	sensitive + log hygiene
NixOS firewall vs cloud SG confusion	Both; document layers
AMI stale	Image pipeline cadence
Two teams, no BOUNDARY.md	Write it
`remote-exec` growth	Delete; bake image

Checkpoint

- ☐ BOUNDARY.md written
- ☐ Resource graph sketched
- ☐ Pattern A/B/C chosen

- ☐ Handoff steps listed
- ☐ Optional real TF apply or explicit skip
- ☐ User-data policy stated
- ☐ State backend thought through

Commit

```
git add .
git commit -m "ops: terraform/nix boundary sketch"
```

Write three personal gotchas before nixosTest.

nixosTest

nixosTest

Goal: Write and run a **nixosTest** VM test that exercises **your** module (proxy, hardened SSH, or role from the shared modules chapter)—not only a copy-paste hello world.

Why this day exists

`nixos-rebuild` on a pet proves little about the next pet. **nixosTest** boots one or more QEMU VMs from your modules, runs shell Python/Perl test scripts, and fails CI when regressions land. It is the highest leverage test type in NixOS land for system behavior.

Theory 1 — What a nixosTest is

```
NixOS configs (nodes) → build VM images → boot under QEMU → testScript → pass/fail
```

Piece	Role
`nodes`	Attrset of NixOS module stacks
`testScript`	Python (modern) driving VMs
`name`	Test identity
Driver	`nixos/lib/testing.nix` machinery

Requires **KVM** (or slow emulation) and often builder feature `kvm` / `nixos-test`.

> Loop engineering: the operating system as SUT
>
> In autonomous agent workflows, the highest leverage loop does not test only an application process—it puts the **entire operating system and network topology under test**. With `runNixOSTest`, agents verify multi-node VLAN isolation, firewall drops, and systemd units before deployment, or drop into `.#checks.<name>.driverInteractive` for live inspection. See **AI Agent Sandboxes, Loop Engineering, and Universal Overlays**.

Theory 2 — Minimal single-node test

```
# tests/ssh-hardening.nix
{
  name = "ssh-hardening";
  nodes.machine = { pkgs, ... }: {
    imports = [ ../modules/profiles/hardened.nix ];
    users.users.alice = {
      isNormalUser = true;
      openssh.authorizedKeys.keys = [
        "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKeyForTestOnly not-a-real-key"
      ];
    };
    services.openssh.enable = true;
  };

  testScript = ''
    machine.wait_for_unit("sshd.service")
    machine.succeed("grep -q 'PasswordAuthentication no' /etc/ssh/sshd_config")
    # or sshd -T output depending on version
    machine.succeed("sshd -T | grep -i 'passwordauthentication no'")
  '';
}
```

Wire into flake:

```
checks.x86_64-linux.ssh-hardening =
  nixpkgs.lib.nixos.runTest {
    imports = [ ./tests/ssh-hardening.nix ];
    hostPkgs = import nixpkgs { system = "x86_64-linux"; };
    # API: prefer current nixpkgs `runTest` / `nixosTest` helper for 26.05
  };
```

> **Note**
>
> nixpkgs has migrated helpers (`pkgs.testers.runNixOSTest`, `lib.nixos.runTest`, classic `nixosTest`). Use whatever your **26.05** pin documents; the **shape** (nodes + testScript) remains the skill.

Classic style still seen in blogs:

```
import nixpkgs {
  system = "x86_64-linux";
  config = { allowUnfree = false; };
} .lib.nixosSystem { } # not this
```

```nix
# classic:
pkgs.nixosTest {
  name = "example";
  nodes.machine = { ... };
  testScript = ''...'';
}
```

Theory 3 — testScript essentials

```python
machine.wait_for_unit("multi-user.target")
machine.wait_for_open_port(22)
machine.succeed("true")
machine.fail("false")
machine.copy_from_host("file", "/tmp/file")
print(machine.succeed("uname -a"))
```

Multi-node:

```nix
nodes = {
  server = { ... };
  client = { ... };
};
```

```python
server.wait_for_unit("caddy.service")
client.succeed("curl -f http://server/")
```

Network is usually a test VLAN; hostnames often equal node names.

Theory 4 — What to test (high value)

Good	Weak
Module assertions about sshd config	`true` only
Service reaches active + port open	Only that evaluation succeeds
Client→server HTTP 200	Full browser suites
Firewall rejects unexpected port	Entire Kubernetes

Test **your** contracts from the hardening chapter–38 / 61.

Theory 5 — Running tests

```
nix build .#checks.x86_64-linux.ssh-hardening -L
# or
nix flake check -L
```

Interactive debug (when supported by driver):

```
nix build .#checks… -L
# many setups: ./result/bin/nixos-test-driver with breakpoints in testScript
```

```
machine.shell_interact()  # if available in your nixpkgs test driver
```

Worked example — web role smoke test

```
# tests/web-role.nix
{
  name = "web-role";
  nodes.server = { ... }: {
    imports = [
      ../modules/profiles/baseline.nix
      ../modules/roles/web.nix
    ];
    roles.web = {
      enable = true;
      domain = "server"; # or localhost style per caddy config
    };
  };
  nodes.client = { pkgs, ... }: {
    environment.systemPackages = [ pkgs.curl ];
  };
  testScript = ''
    server.wait_for_unit("caddy.service")
    server.wait_for_open_port(80)
    client.succeed("curl -f --retry 5 --retry-delay 1 http://server/ | grep -q ok")
  '';
}
```

Adapt to your actual reverse proxy module from Stage IV.

Lab — multi-step

Suggested workspace: fleet flake `tests/`; notes in `~/lab/nixos/ch64`

Step 1 — Confirm virtualization

```
ls /dev/kvm && echo kvm-ok || echo "need kvm or nested virt"
```

Step 2 — Hello test

Minimal machine with `environment.etc."t".text = "ok";` and `succeed("test -f /etc/t")`.

Step 3 — Port your module

SSH hardening **or** web role **or** custom Stage IV module.

Step 4 — Make it fail once

Break the module; observe red test; fix.

Step 5 — Wire `checks`

`nix flake check` runs it (may be heavy—ok).

Step 6 — Document runtime

Wall clock; note CI machine requirements (the CI pipelines chapter).

Step 7 — Multi-node stretch

Client/server only if single-node is green.

Theory 6 — Tests as flake checks

```
checks.${system}.lab-ssh = self.nixosConfigurations.lab.config.system.build.toplevel; # heav
# better: dedicated nixosTest derivation
checks.${system}.web-smoke = pkgs.nixosTest { … };
```

Wiring tests into `checks` makes `nix flake check` the Stage VI/CI gate. Keep at least one **fast** smoke and quarantine long tests behind explicit attrnames.

Theory 7 — Interactive vs pure test runs

```
nix build .#checks.x86_64-linux.web-smoke -L
# driver interactively (when supported by your setup):
# nix run .#checks.…driverInteractive
```

Interactive drivers help author tests; CI should run non-interactive builds.

Worked example — wait for unit + port

```
machine.wait_for_unit("caddy.service")
machine.wait_for_open_port(443)
machine.succeed("curl -k -sS https://127.0.0.1 | grep -q lab")
```

Prefer `wait_for_unit` / `wait_for_open_port` over `sleep 30`.

Lab — one negative assertion

```
machine.fail("curl -sS --max-time 2 http://127.0.0.1:5432 || true")
# better: assert postgres not on 0.0.0.0 if that is policy
```

Encode a security/policy property, not only happy path.

Lab — time the test

```
time nix build .#checks.x86_64-linux.web-smoke -L
```

Record duration. Tests that take 20+ minutes will not be run and thus do not protect you—split or slim.

Common gotchas

Symptom / mistake	What to do
No `/dev/kvm`	Nested virt; remote builder with kvm; or accept qemu slow
Hang on boot	Serial logs; simplify modules; wait_for_unit
DNS names	Use node names as hostnames
Testing wrong module version	Same flake inputs as deploy
Over-large configs	Slim test nodes; don't import Desktop
Flaky timing	`wait_for_open_port`, retries

Checkpoint

- ☐ One nixosTest green
- ☐ Tests **your** module behavior
- ☐ Intentional red→green cycle
- ☐ Flake `checks` wiring
- ☐ KVM/runtime notes written

Commit

```
git add .
git commit -m "day64: nixosTest for own module"
```

Write three personal gotchas before continuing.

More testing patterns

More testing patterns

Goal: Grow a **suite** of at least three meaningful flake checks (including nixosTest and lighter eval/build checks) that mirror CI shape without becoming a flaky tar pit.

> **Note**
>
> Builds on the nixosTest chapter (`nixosTest`). Baseline: flakes, **26.05** pins, modules you already own. Diversify **kinds** of checks; do not only clone one VM test three times.

Why this day exists

One demo test rots. A small suite with clear ownership becomes the merge gate for roles and packages. Today you diversify test layers, kill flakes early, and time the suite so the CI pipelines chapter CI has a budget.

Theory 1 — Test pyramid for NixOS fleets

Layer	Examples	Speed	Catches
Eval	`nix flake show`, option asserts, `nix eval`	Fast	Typos, missing options
Build	`nix build .#pkg`, host toplevel	Medium	Compile/package breaks
VM (nixosTest)	Service behavior	Slow	Integration
Deploy dry	deploy-rs checks, colmena eval	Medium	Hive wiring
Live canary	Real host probes	Manual/slow	Reality

Aim: **many fast, few slow**. Capstone and CI die when every PR boots three full VMs for a typo in a comment.

```
        /live canary\
       /  nixosTest   \
      / build / toplevel \
     / eval / lint / fmt  \
```

Theory 2 — Three checks minimum (template)

```
# flake outputs fragment - illustrative
checks = forAllSystems (system:
  let
    pkgs = import nixpkgs {
      inherit system;
      overlays = [ self.overlays.default ];
    };
  in {
    # 1) package as check (build is the test)
    cooltool-build = self.packages.${system}.cooltool;

    # 2) pure command check
    cooltool-runs = pkgs.runCommand "cooltool-runs" {
      nativeBuildInputs = [ self.packages.${system}.cooltool ];
    } ''
      cooltool --help >/dev/null
      touch $out
    '';
  } // nixpkgs.lib.optionalAttrs (system == "x86_64-linux") {
    # 3+) VM tests - Linux only
    ssh-hardening = import ./tests/ssh-hardening.nix { inherit pkgs self; };
    web-role = import ./tests/web-role.nix { inherit pkgs self; };
  }
);
```

```
nix build .#checks.x86_64-linux.cooltool-runs -L
nix build .#checks.x86_64-linux.ssh-hardening -L
nix flake check -L
```

Theory 3 — Lightweight non-VM checks

Format / lint

```
# Prefer your Stage III treefmt/nixfmt wiring if present
fmt = pkgs.runCommand "fmt-check" {
  nativeBuildInputs = [ pkgs.nixfmt-rfc-style ];
} ''
```

```
  # example only - real checks usually run treefmt --ci on src
  touch $out
'';
```

Eval / build host configuration

```
edge-toplevel =
  self.nixosConfigurations.edge.config.system.build.toplevel;
```

```
nix build .#nixosConfigurations.edge.config.system.build.toplevel -L
```

Strong, but heavy—cache it; don't duplicate five slight variants.

Structured eval assertions

```
nix eval .#nixosConfigurations.edge.config.networking.hostName
nix eval --json .#nixosConfigurations.edge.config.services.openssh.enable
```

Wrap in a `runCommand` with `jq` if you want it as a formal check:

```
host-name-ok = pkgs.runCommand "host-name-ok" {
  nativeBuildInputs = [ pkgs.jq ];
} ''
  echo ${pkgs.lib.escapeShellArg (
    builtins.toJSON self.nixosConfigurations.edge.config.networking.hostName
  )} | jq -e '. == "edge"'
  touch $out
'';
```

(Prefer cleaner patterns as your flake matures; the point is **fast** feedback.)

Theory 4 — Anti-flake tactics

Problem	Fix
Sleep-based waits	`wait_for_unit` / `wait_for_open_port`
Order-dependent multi-test	Isolated nodes per test file
Network to internet in tests	Forbid; vendor fixtures
Shared mutable state	Each test pure

Problem	Fix
Over-broad `flake check` on Darwin	`optionalAttrs` for Linux VM tests
30-minute suite	Split “CI fast” vs “nightly full”
Secret decryption in CI	Dummy modules / test keys only

```
# nixosTest script - good waits
machine.wait_for_unit("nginx.service")
machine.wait_for_open_port(80)
machine.succeed("curl -f http://127.0.0.1/")
```

```
# Run one check without the world
nix build .#checks.x86_64-linux.ssh-hardening -L
```

Theory 5 — Mapping tests to modules

Module / concern	Check ideas
`profiles.hardened`	`sshd -T` probes, package absences
`roles.web`	curl client→server via proxy
Stage V package	help/version smoke
Disko	eval `fileSystems` keys (not full wipe)
sops-nix	assert secret **options** configured; decrypt with test key only
firewall	multi-node: client cannot hit closed port
deploy hive	eval nodes exist / `--dry-activate` if available

Every check should name an **owner module** in a README table so rotting tests get a human.

Theory 6 — Suite layout and CI shape

```
tests/
  ssh-hardening.nix
  web-role.nix
  cooltool.nix
  fixtures/
```

```
    test-secrets.nix
  README.md          # table: check → layer → minutes → owner
```

```
# flake fragment
checks.x86_64-linux = {
  inherit (self.packages.x86_64-linux) cooltool;
  cooltool-runs = /* … */;
  ssh-hardening = /* … */;
  web-role = /* … */;
  edge-toplevel =
    self.nixosConfigurations.edge.config.system.build.toplevel;
};
```

the CI pipelines chapter will run these on GitHub Actions. Design attrs you can select:

```
nix build -L \
  .#checks.x86_64-linux.cooltool-runs \
  .#checks.x86_64-linux.ssh-hardening
```

Worked example — fail-inject once

```
# 1) break intentionally
# e.g. change expected hostname assertion or stop nginx in test config
nix build .#checks.x86_64-linux.web-role -L   # expect fail
# 2) restore
# 3) confirm green
# 4) confirm other checks still independent
nix build .#checks.x86_64-linux.cooltool-runs -L
```

If breaking test A always breaks unrelated B, your suite shares too much impure state.

Lab — multi-step

Suggested notes: `~/lab/nixos/ch65`

Step 1 — Inventory today's checks

```
mkdir -p ~/lab/nixos/ch65
cd /path/to/your/flake
nix flake show 2>&1 | tee ~/lab/nixos/ch65/flake-show.txt
```

List current checks and gaps.

Step 2 — Add check #1 (package)

Build + run your Stage V package (`runCommand` or package attr as check).

Step 3 — Add check #2 (nixosTest)

Keep/improve the nixosTest chapter test; ensure waits are unit/port based.

Step 4 — Add check #3 (config)

Either second nixosTest **or** toplevel build for one host **or** eval assertion.

Step 5 — Fail inject matrix

Break each check once; confirm isolation; restore green.

Step 6 — Timing budget

```
/usr/bin/time -p nix flake check -L \
  2>&1 | tee ~/lab/nixos/ch65/timing.txt
```

If over your CI budget, document split: `checks.ci` vs full suite.

Step 7 — README table

Check	Layer	Minutes	Owner module

Step 8 — Stability

Rerun thrice (or twice if time-boxed); note flakiness tickets.

Step 9 — Secrets policy for tests

Write one paragraph: how CI avoids prod sops keys (fixtures, dummy tokens).

Common gotchas

Symptom / mistake	What to do
`flake check` builds everything forever	Select attrs; fix IFD explosions
Darwin CI without KVM	Gate VM tests on Linux
Secret decryption in checks	Dummy modules / mock
Duplicate heavy toplevels	Share less; accept one + cache
Tests not in merge gate	the CI pipelines chapter wires CI
Sleep-only "green"	Replace with readiness waits
Three identical VM tests	Diversify layers

Checkpoint

- ☐ 3 meaningful checks green
- ☐ At least one VM test and one non-VM test
- ☐ Timing measured
- ☐ README/table of suite with owners
- ☐ Fail-inject done once per check
- ☐ Flakiness notes empty or ticketed
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "day65: expanded flake check suite"
```

Write three personal gotchas before continuing.

CI pipelines for flakes

CI pipelines for flakes

Goal: Run your flake checks/builds on **GitHub Actions** (or GitLab) with a modern Nix installer, optional **Cachix/Attic** push, and a PR-shaped workflow that fails on regressions.

> **Note**
>
> If tests only run on your laptop, they are hobbies. 2026 default shape: **forge CI + flakes + binary cache**. Baseline nixpkgs: **26.05**.

Why this day exists

Stage V packages and Stage VI modules become team infrastructure only when a clean runner reproduces them. CI also forces purity: "works on my machine" impurities die in public logs. Capstone the capstone harden chapter reuses this muscle.

Theory 1 — Minimal trustworthy pipeline

```
checkout → install Nix → (auth cache) → nix flake check / build → (push cache)
```

Job / step	Purpose
`checkout`	Source + lockfile
`install Nix`	Known installer; flakes enabled
`cache auth`	Pull/push substitutes
`check`	`nix flake check` or selected checks
`build-pkg`	Stage V package
`build-hosts`	Optional toplevel builds

Trust boundaries

Surface	Rule
PR from fork	Often **no** secret cache write tokens
`main`	May push cache

Surface	Rule
Logs	Never print age keys, Cachix tokens, SSH keys
`trusted-users`	Do not weaken developer laptops for CI

Theory 2 — GitHub Actions workflow (2026-shaped)

```yaml
# .github/workflows/nix.yml
name: nix

on:
  push:
    branches: [ main ]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read
  # id-token: write  # if using OIDC for caches later

concurrency:
  group: nix-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Nix
        uses: DeterminateSystems/nix-installer-action@main
        # pin to a reviewed tag in real repos

      - name: Magic Nix Cache (optional)
        uses: DeterminateSystems/magic-nix-cache-action@main
        # or Cachix below

      # - uses: cachix/cachix-action@v15
      #   with:
      #     name: myorg
      #     authToken: ${{ secrets.CACHIX_AUTH_TOKEN }}
```

```
      - name: Flake metadata
        run: nix flake metadata

      - name: Selected checks
        run: |
          nix build -L \
            .#checks.x86_64-linux.cooltool-runs \
            .#checks.x86_64-linux.ssh-hardening

      - name: Build package
        run: nix build .#cooltool -L

      - name: Build edge toplevel (optional, heavy)
        run: nix build .#nixosConfigurations.edge.config.system.build.toplevel -L
```

> ⚠ Warning
>
> **Pin action versions** to tags or SHAs you have reviewed. Versions above are illustrative—confirm current upstream tags when you wire the repo. Freeze pins harder by the multi-node bar chapter for Capstone.

Theory 3 — Cachix (and friends) in CI

```
- uses: cachix/install-nix-action@v31
  with:
    nix_path: nixpkgs=channel:nixos-26.05
- uses: cachix/cachix-action@v15
  with:
    name: myorg
    authToken: '${{ secrets.CACHIX_AUTH_TOKEN }}'
    # signingKey: '${{ secrets.CACHIX_SIGNING_KEY }}'  # if used
- run: nix build .#cooltool -L
```

House rules

Rule	Why
Push from CI (common)	Shared binaries; less laptop upload chaos
Separate PR vs main push rights	Abuse / token theft mitigation
Public keys on consumers	Lab + admin `substituters` + `trusted-public-keys`

Rule	Why
Never commit private signing keys	Rotate immediately if leaked

Alternatives

Tool	Notes
Cachix	Hosted; simple
Attic	Self-hosted multi-tenant cache
Harmonia	Serve local store as binary cache
Magic Nix Cache	UX-oriented GitHub cache integration

Any one done well beats none.

Theory 4 — What not to run on free runners

Heavy	Strategy
Full nixosTest matrix	Linux job; fewer nodes; path filters
Multiple toplevels	Build one primary host per PR
Nested KVM	May work on GH-hosted; else self-hosted runner
Secret full decrypt	Dummy configs for CI (the more tests chapter)
Unbounded `flake check`	Select attrs until budget known

```yaml
on:
  pull_request:
    paths:
      - 'flake.nix'
      - 'flake.lock'
      - 'modules/**'
      - 'hosts/**'
      - 'pkgs/**'
      - 'tests/**'
      - '.github/workflows/nix.yml'
```

Timeouts and disks

```yaml
jobs:
  check:
    runs-on: ubuntu-latest
    timeout-minutes: 90
    steps:
      # …
```

Out-of-disk: slim checks, GC mid-job only if needed, or larger runners.

Theory 5 — GitLab CI sketch

```yaml
# .gitlab-ci.yml (sketch)
image: debian:bookworm
variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"
before_script:
  - apt-get update && apt-get install -y curl xz-utils
  # install Nix via official/Determinate method - prefer current docs
  - nix flake metadata
check:
  script:
    - nix flake check -L
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Prefer maintained install methods over ancient `nixos/nix` container tags when flakes misbehave.

Theory 6 — CI vs deploy

Pattern	the CI pipelines chapter stance
CI verifies build/check	**Default**
CI pushes cache	Preferred
CI deploys to lab	Optional later; protect with environments
CI holds prod SSH keys	High risk; avoid early

Human or gated CD **deploy** after green checks is enough for Stage VI. Capstone the capstone harden chapter revisits.

Worked example — selected checks job

```
- name: Fast checks
  run: |
    set -euo pipefail
    nix build -L \
      .#checks.x86_64-linux.cooltool-runs \
      .#checks.x86_64-linux.ssh-hardening
- name: Optional full check
  if: github.ref == 'refs/heads/main'
  run: nix flake check -L
```

Split fast PR signal from heavier main-branch verification when needed.

Worked example — intentional red PR

```
# on a branch: break a check assertion or package
git checkout -b ci/red-test
# … introduce fail …
git commit -am "ci: temporary red"
git push -u origin ci/red-test
# open PR → confirm CI red
# fix → confirm green
```

If you cannot get a red PR, the workflow is not protecting you.

Lab — multi-step

Suggested notes: `~/lab/nixos/ch66`

Step 1 — Add workflow file

Commit `.github/workflows/nix.yml` (or GitLab equivalent) to your lab flake repo. Use path filters if monorepo.

Step 2 — First green CI

Push branch; fix until green on a clean runner. Paste run URL into notes.

```
mkdir -p ~/lab/nixos/ch66
echo "CI_URL=…" > ~/lab/nixos/ch66/ci-url.txt
```

Step 3 — Wire cache

Cachix **or** Magic Nix Cache **or** Attic. Confirm second run is faster or shows substitutes.

Step 4 — PR simulation

Open PR that breaks a check; confirm CI red; fix; green.

Step 5 — Secrets hygiene

Ensure tokens only in forge secrets; scan workflow for accidental `echo` of secrets.

Step 6 — Consumer config

Add substituter public keys to lab/admin Nix config; rebuild or set user `nix.conf`.

Step 7 — Document runner needs

KVM? memory? timeout? Write `docs/CI.md` with:

Topic	Content
Triggers	PR/main
Commands	exact `nix build` lines
Cache	name + public key
Limits	known failures without KVM
Non-goals	deploy-from-CI policy

Step 8 — Status badge (optional)

README badge for the workflow—only if green is honest.

Step 9 — Deploy from CI? (decide)

Write the choice: verify-only vs gated deploy. Default for this day: **verify-only**.

Common gotchas

Symptom / mistake	What to do
Flakes disabled	Installer action enables; set `NIX_CONFIG`
Disk full on runner	Slim checks; larger runner; GC carefully
Unpinned actions	Pin majors/SHAs per org policy
Cache not used	Public key / auth; path existence; wrong cache name
VM tests fail without KVM	Self-hosted or reduce tests
Monorepo wrong cwd	`defaults.run.working-directory` / filters
Fork PRs leaking write tokens	Restrict secret access
IFD / eval forever	Fix expressions; select attrs

Checkpoint

- ☐ Workflow exists and runs on PR/push
- ☐ At least one Nix build/check green in CI
- ☐ Intentional red PR observed
- ☐ Cache strategy configured or explicitly deferred with reason
- ☐ `docs/CI.md` notes runner limits and commands
- ☐ Tokens not in git
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "day66: GitHub Actions Nix CI pipeline"
```

Write three personal gotchas before continuing.

GC and store hygiene

GC and store hygiene

Goal: Measure store size, explain **GC roots**, set a retention policy for generations and CI, and run `optimise`/GC safely without deleting live systems.

Why this day exists

Deploy + CI + packaging fill disks silently. Out-of-disk mid-activate is a self-inflicted outage. Hygiene is an operational skill equal to deploy tooling—Capstone hosts die from full `/nix` as often as from bad modules.

Theory 1 — What GC deletes

Garbage collection deletes store paths that are **not reachable from any GC root**.

Root examples	Keeps alive
`/run/current-system`	Running NixOS closure
`/nix/var/nix/profiles/*`	System generations, user profiles
`~/.local/state/nix/profiles`	Modern user profiles
`/nix/var/nix/gcroots/*`	Explicit pins
Result symlinks in cwd	`./result` from `nix build`
Indirect roots / registries	Various tooling pins

```
nix-store --gc --print-roots | head -n 50
nix-collect-garbage --dry-run
nix store gc --dry-run  # modern CLI shape; flags evolve-check --help
```

Mental model

```
GC root → profile/generation/result link → closure of store paths
Anything not reachable is eligible for deletion
```

Deleting a project directory does **not** free the store if roots still point at builds.

Theory 2 — Generations vs store GC

```
# List system generations
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
# or inspect profile links
ls -l /nix/var/nix/profiles/system*

# Delete old generations then GC
sudo nix-collect-garbage --delete-older-than 14d
# aggressive: remove all old gens of profiles
# sudo nix-collect-garbage -d
```

Command	Effect
`nix-collect-garbage`	GC unreachable paths
`--delete-older-than`	Drop old gens then GC
`-d`	Aggressive old gen removal
`nix store gc`	Modern CLI GC
`nix store optimise`	Hardlink identical files

> **!** Important
>
> Deleting generations removes rollback options. Keep enough to survive a bad deploy week—especially before Capstone freeze and DR drills.

User vs system profiles

Home-Manager and user `nix profile` installs have their own generation chains. System GC policy does not automatically equal user hygiene.

```
nix profile list 2>/dev/null || true
ls ~/.local/state/nix/profiles 2>/dev/null || true
```

Theory 3 — Policy template

Host class	Generation retention	Notes
Laptop	7–14 days	Disk pressure
Prod VM	14–30 days + keep last N	Rollback window

Host class	Generation retention	Notes
CI runner	Daily GC; few profiles	Ephemeral preferred
Builder	Weekly GC; keep hot cache set	Balance disk vs rebuild
Capstone lab	14 days through the rollback and tests chapter	Rollback drills need gens

```
# NixOS automatic GC example
{ ... }:
{
  nix.gc = {
    automatic = true;
    dates = "weekly";
    options = "--delete-older-than 14d";
  };
  nix.optimise.automatic = true;
  # proactive free-space GC thresholds (bytes)
  nix.settings.min-free = 10 * 1024 * 1024 * 1024; # 10 GiB
  nix.settings.max-free = 50 * 1024 * 1024 * 1024;
}
```

`min-free` triggers proactive GC when disk is low—excellent on small roots and cloud boxes.

Theory 4 — Finding what holds space

```
df -h /nix
nix path-info -Sh /run/current-system
nix path-info -rS /run/current-system | sort -k2 -n | tail -n 20

# Why is path alive?
nix-store --query --roots /nix/store/…-something
```

```
# Old result links (common disk vampires)
find "$HOME" -maxdepth 3 -type l \( -name result -o -name 'result-*' \) 2>/dev/null
```

```
# Largest store paths (approx; tools vary)
nix path-info -rS /run/current-system | sort -k2 -h | tail
```

CI-specific roots

Runners accumulate `result` links and old profiles if jobs are not ephemeral. Prefer clean workspaces per job; document GC for self-hosted runners in `docs/CI.md`.

Theory 5 — optimise (hardlinks)

```
nix store optimise
# or
sudo nix-store --optimise
```

Identical files across store paths become hardlinks → often saves GBs when many similar closures exist (multiple generations, overlapping devshells). Safe; CPU-heavy first run.

> **Note**
>
> Backup tools that do not understand hardlinks may inflate sizes or behave oddly. Know your backup tool's hardlink policy (the backups chapter).

Theory 6 — Incident patterns

Symptom	First checks
`No space left on device` during build	`df -h /nix /`; roots; old gens; `result` links
Activate failed mid-switch	Free space; incomplete profile; repair with boot generation
GC "deleted my package"	Missing root; rebuild; avoid bare `-d` in prod
Rollback missing	Generations deleted by policy
Disk full only on CI	Self-hosted runner hygiene

Never run destructive GC on prod without:

1. Confirming current-system root
2. Confirming retention window
3. Preferring `--delete-older-than` over blind `-d`

Worked example — hygiene report script

```
#!/usr/bin/env bash
# scripts/store-report.sh
set -euo pipefail
echo "== disk =="
df -h /nix 2>/dev/null || df -h /
echo "== current system closure =="
nix path-info -Sh /run/current-system 2>/dev/null || true
echo "== gc roots (sample) =="
nix-store --gc --print-roots 2>/dev/null | head -n 40 || true
echo "== result symlinks =="
find "$HOME" -maxdepth 2 -type l -name 'result*' 2>/dev/null || true
echo "== system generations =="
sudo nix-env -p /nix/var/nix/profiles/system --list-generations 2>/dev/null | tail -n 15 ||
```

```
chmod +x scripts/store-report.sh
./scripts/store-report.sh | tee ~/lab/nixos/ch67/before.txt
```

Lab — multi-step

Suggested notes: `~/lab/nixos/ch67`

Step 1 — Baseline metrics

Record `df -h /nix` (or `/`) and current-system size via `nix path-info -Sh`.

Step 2 — Map roots

Print roots; identify unexpected `result` links and old profiles. Screenshot or tee a sample.

Step 3 — Safe cleanup

Remove stale `./result` from lab dirs; GC **without** aggressive `-d` first:

```
nix store gc
df -h /nix
```

Step 4 — Generation policy

List generations; delete only those outside your written policy window.

```
sudo nix-collect-garbage --delete-older-than 14d
```

Step 5 — Enable automatic GC on lab host

Commit `nix.gc` (+ optional `min-free` / optimise) settings; deploy or switch.

Step 6 — optimise

Run optimise; record space reclaimed if measurable.

```
df -h /nix | tee ~/lab/nixos/ch67/before-optimise.txt
nix store optimise
df -h /nix | tee ~/lab/nixos/ch67/after-optimise.txt
```

Step 7 — CI runner note

Add GC step for self-hosted runners or rely on ephemeral runners; document in `docs/CI.md`.

Step 8 — Policy doc

Write `docs/STORE-HYGIENE.md` (or section in RUNBOOK):

Host	Retention	Auto GC	Notes
lab	14d	weekly	
laptop	7d	weekly	

Step 9 — Rollback still possible?

Confirm at least one previous generation remains on lab after cleanup. If not, you over-pruned—adjust policy.

Step 10 — Profile audit (user + HM)

```
# User profiles / modern nix profile
nix profile list 2>/dev/null | tee ~/lab/nixos/ch67/user-profile.txt || true
# Home-Manager generations if present
ls -la /nix/var/nix/profiles/per-user/"$USER"/ 2>/dev/null || true
```

```
ls -la ~/.local/state/home-manager 2>/dev/null || true
```

Write one sentence: how user-profile retention differs from system `nix.gc` on this host.

Step 11 — Dry-run before aggressive delete

```
# Prefer dry-run / list before destroy
nix-collect-garbage --delete-older-than 30d --dry-run 2>&1 \
  | tee ~/lab/nixos/ch67/gc-dry-run.txt || true
```

If dry-run is unavailable on your Nix version, list generations explicitly and delete only named old gens.

Worked example — min-free incident story

```
Disk at 98% on small cloud root:
  1) df -h /nix
  2) remove stale result links in ~ and CI workspaces
  3) nix-collect-garbage --delete-older-than 7d   # laptop policy, not prod
  4) enable nix.settings.min-free so next time GC starts earlier
  5) consider separate /nix partition next reprovision (Disko)
```

Record whether you hit this during Stage V/VI. Capstone hosts on 20 GiB disks fail predictably without policy.

Common gotchas

Symptom / mistake	What to do
GC “does nothing”	Roots still hold paths
Deleted all rollbacks	Stop bare `-d` in prod without policy
`/nix` on small partition	Separate partition; `min-free`
optimise confuses backups	Hardlinks; tool configuration
Multi-user permissions	GC as root for system profiles
HM gens forgotten	User profile hygiene
Full disk mid-CI	Ephemeral runners; job GC; slimmer checks

Checkpoint

- ☐ Measured store before/after
- ☐ Explained roots with real examples from your machine
- ☐ Retention policy written and applied on lab
- ☐ optimise run once
- ☐ Auto GC or min-free configured
- ☐ At least one rollback generation still present
- ☐ User/HM profile note written
- ☐ `docs/STORE-HYGIENE.md` (or RUNBOOK section) committed
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "day67: GC policy and store hygiene"
```

Write three personal gotchas before continuing.

Secure Boot and TPM (awareness)

Secure Boot and TPM (awareness)

Goal: Document and, if hardware allows, implement a **Lanzaboote** Secure Boot path and/or TPM-backed LUKS unlock—or produce a lab-limits dossier that is honest about what you cannot do on VMs.

> **!** Important
>
> Copy-paste without the upstream guide bricks boots. Treat this day as **process + documentation** first; enable on a **disposable** machine second. Recovery USB and recovery keys before enrollment.

Why this day exists

Fleet images and metal boxes increasingly expect **Secure Boot** and disk encryption that does not beg for a passphrase at every reboot in a datacenter. NixOS's community answer for SB is primarily **Lanzaboote** (uki/systemd-boot + signed generations). TPM2 unlock pairs with LUKS (the Disko chapter Disko often declares the encrypted volumes).

Capstone B (stretch) lists SB/TPM; Capstone A does not require full enablement—but Stage VI experts must know the **path** and the **failure modes**.

Theory 1 — Problem statement

Threat	Control
Evil maid bootloader swap	Secure Boot + enrolled keys
Disk theft	LUKS (or vendor disk encryption—know the difference)
Passphrase at scale	TPM2 PCR-bound unlock (with enrollment process)
Unsigned kernel modules policies	SB + your key hierarchy
Silent rollback to unsigned attacker kernel	Sign **each** generation you boot

NixOS's generation model must **sign** new boots—not only ship a distro shim forever. That is why Lanzaboote exists relative to "turn on SB and hope."

Theory 2 — Lanzaboote overview (2026)

Lanzaboote integrates Secure Boot with NixOS (systemd-boot oriented flows).

Typical story (always verify against **current** README for your pin):

1. Enable Secure Boot in firmware (setup mode / enroll keys carefully)
2. Generate or enroll machine keys (PK/KEK/db as relevant to the guide)
3. Import Lanzaboote NixOS modules so boots are signed
4. Keep recovery path (USB installer + known recovery keys)
5. Re-test boot after **each** major change before trusting prod

```
# Conceptual only - follow current Lanzaboote README for 26.05-compatible paths
{
  imports = [ lanzaboote.nixosModules.lanzaboote ];
  boot.loader.systemd-boot.enable = lib.mkForce false; # lanzaboote replaces aspects
  boot.lanzaboote = {
    enable = true;
    pkiBundle = "/etc/secureboot"; # path patterns per docs
  };
}
```

Flake input sketch:

```
# inputs.lanzaboote.url = "github:nix-community/lanzaboote/…";
# inputs.lanzaboote.inputs.nixpkgs.follows = "nixpkgs";
```

> ⚠ Warning
>
> Firmware Setup Mode, key enrollment, and Microsoft/3rd-party key politics vary by vendor. Laptop OEM SB is not the same as a server you control end-to-end.

Theory 3 — TPM2 + LUKS

Goals:

- Unlock root (or data) volume without typing a passphrase every boot
- Still retain a **recovery passphrase** or recovery key file offline

- Survive policy: which PCRs bind the sealed key

```
# Sketch - systemd-cryptenroll / auto-unlock patterns evolve
{
  boot.initrd.systemd.enable = true; # often required for modern enroll flows
  # boot.initrd.luks.devices."crypted".crypttabExtraOpts = [ "tpm2-device=auto" ];
  # Exact options: verify on your nixpkgs 26.05 pin and systemd version
}
```

Manual enrollment often looks like:

```
# list TPM
systemd-cryptenroll --tpm2-device=list
ls /sys/class/tpm

# enroll (EXAMPLE - adjust device + PCR set per threat model)
# systemd-cryptenroll --tpm2-device=auto --tpm2-pcrs=7 /dev/nvme0n1p2
```

PCR trade-offs

Choice	Security	Fragility
Fewer PCRs	Weaker binding	Survives more firmware changes
PCR 7 (Secure Boot state) common	Ties to SB	Firmware/SB policy changes may require re-enroll
Many PCRs	Stronger	Breaks after BIOS updates more often

Document which PCRs and the **re-enroll procedure** after BIOS updates. Never TPM-only without recovery credentials.

Theory 4 — VM lab limits

Environment	Secure Boot	TPM
Bare metal modern laptop	Often yes	Often yes
Cloud VM	Sometimes virtual SB/TPM	Vendor-specific
libvirt QEMU	Emulated possible	swtpm possible
Nested cheap VPS	Often no	Often no

QEMU/swtpm note (advanced lab)

You *can* practice pieces with OVMF Secure Boot and swtpm—but it is a project of its own. Stage VI accepts a **document-only track** if hardware refuses you. Do not fake "SB enabled" in notes without evidence.

Theory 5 — Recovery & break-glass

Asset	Storage
Disk recovery key / passphrase	Password manager / sealed paper / offline encrypted store
SB setup mode knowledge	Runbook steps
Unsigned rescue ISO / installer USB	Physical media
Age/sops keys	Separate from disk encryption keys
Old generation unsigned	Know when SB will **reject** boot

```
Failure Friday scenario:
  BIOS update → PCR mismatch → TPM unlock fails
  → boot installer USB
  → unlock with recovery key
  → re-enroll TPM
  → document time-to-recover
```

Never enroll TPM-only without a recovery passphrase backup you have **tested**.

Theory 6 — Relation to Disko, deploy, and images

Concern	Link
LUKS layout	the Disko chapter Disko declares partitions; TPM enroll often **post-install**
Impermanence	the impermanence chapter: encrypted root + persist datasets
Images	the images/generators chapter generators may not equal metal SB story
Deploy	Remote deploy does not replace firmware SB enrollment

Concern	Link
Capstone B	Fleet policy: which classes require SB

Write one paragraph: **which host classes** require SB vs best-effort.

Worked example — dossier template

```
# SECUREBOOT-TPM.md

## Hardware
- Machine:
- Firmware vendor/version:
- Firmware SB state: SetupMode / User / Disabled / Unknown
- TPM: 2.0 visible? (commands + output summary)
- Lab type: metal / libvirt / cloud

## Goals
- [ ] Secure Boot with Lanzaboote
- [ ] LUKS passphrase only
- [ ] LUKS + TPM2
- [ ] Document-only (limits)

## Upstream sources
- Lanzaboote README URL + commit/date consulted:
- NixOS wiki / manual notes:

## Steps performed
1. …
2. …

## Recovery
- Recovery key location (not the key itself):
- USB installer location:
- Re-enroll after BIOS update:
- Estimated RTO if TPM fails:

## Residual risks
- …

## Fleet policy
- Host classes requiring SB:
- Best-effort only:
```

Lab — multi-step

Suggested notes: `~/lab/nixos/ch68`

Step 1 — Probe hardware

```
mkdir -p ~/lab/nixos/ch68
bootctl status 2>/dev/null | tee ~/lab/nixos/ch68/bootctl.txt || true
efi-readvar 2>/dev/null | tee ~/lab/nixos/ch68/efi-vars.txt || true
ls -la /sys/class/tpm 2>&1 | tee ~/lab/nixos/ch68/tpm-sysfs.txt
systemd-cryptenroll --tpm2-device=list 2>&1 | tee ~/lab/nixos/ch68/tpm-list.txt || true
```

Step 2 — Choose track

Track	Criteria
A	Metal + SB available → Lanzaboote guided enable on **disposable**
B	TPM + LUKS available → enroll with recovery key tested
C	VM only → emulate with swtpm **or** document-only dossier

Write the track choice first; do not improvise on your daily driver.

Step 3 — Read upstream (required even for Track C)

Open current Lanzaboote README + Secure Boot notes relevant to your generation. Save URLs in the dossier. Take notes—do not invent key enrollment sequences from memory.

Step 4 — Recovery assets first

Before any enrollment:

1. Confirm installer USB or rescue media boots
2. Confirm LUKS recovery key unlocks (lab disk)
3. Write locations in dossier

Step 5 — Implement or simulate

If A/B: snapshot/backup first; proceed carefully; capture commands and outcomes.
If C: write why cloud images still matter (signed provenance vs local SB) and what you would do on metal.

Step 6 — Failure plan drill (written)

Write the exact steps to boot installer and decrypt disk if TPM fails after firmware update. Include who holds recovery keys.

Step 7 — Link Disko / LUKS

Map where LUKS is declared and whether TPM enroll is post-install only. Note declarative limits honestly.

Step 8 — Fleet policy paragraph

Which host classes require SB? Which are best-effort? Capstone A vs B expectations.

Step 9 — Tabletop: evil maid + firmware update

Write answers in the dossier (no need to re-brick hardware):

1. Attacker replaces bootloader on stolen laptop with SB off historically—what changes with Lanzaboote enrolled?

2. After OEM BIOS update, TPM unlock fails at initrd—ordered recovery steps?

3. How do age/sops keys relate to disk recovery keys (separate custody)?

4. Can deploy-rs fix a machine that will not boot? (Usually no—firmware/recovery path first.)

Step 10 — Capstone A vs B honesty line

One sentence each:

- Capstone A: SB/TPM **not required**—document limits OK.

- Capstone B: which pieces become mandatory for *your* stretch goals?

Worked example — recovery USB checklist

```
## Recovery USB
- [ ] Boots on target hardware (tested date: …)
- [ ] Contains or can fetch flake tools / nix installer as needed
- [ ] Operator knows LUKS recovery unlock command
- [ ] Network plan if flake must be cloned (ethernet? Wi-Fi firmware?)
- [ ] Time estimate from power-on to decrypted root shell: …
```

If you never tested the USB on this machine, Track A/B is incomplete even if enrollment "worked once."

Common gotchas

Symptom / mistake	What to do
Bricked boot	Installer USB; setup mode; recover keys
PCR mismatch after update	Re-enroll; reconsider PCR set
Lost recovery key	Undecryptable disk; prevention only
Enabled SB without signing new gens	Use Lanzaboote properly
Assumed cloud disk encryption = LUKS	Understand vendor encryption separately
Experiment on only laptop	Use disposable or accept data-loss risk consciously
Docs lag code	Date-stamp upstream guide you followed

Checkpoint

- ☐ Hardware probed with saved outputs
- ☐ Track A/B/C completed with artifacts
- ☐ `SECUREBOOT-TPM.md` filled
- ☐ Recovery path written **and** assets exist

- ☐ No production host left without recovery keys
- ☐ Upstream sources recorded
- ☐ Tabletop answers written
- ☐ Recovery USB checklist honest
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "day68: secure boot / TPM path dossier"
```

Write three personal gotchas before continuing.

Backup story

Backup story

Goal: Pick **one** backup system (**Restic**, **Borg**, or **ZFS send**), declare it in NixOS, and complete a **restore drill of state** (not only a successful backup job log).

> **Note**
>
> Flakes rebuild systems; they do not recreate Postgres, SQLite, photos, or `/persist`. Pair with the disaster recovery chapter later: rebuild OS from git+keys, restore **state** from backups.

Why this day exists

Capstones fail when "declarative" is confused with "immortal data." Today is the state half of disaster recovery. A green `restic snapshots` line without a restore is theatre.

Theory 1 — What to back up

Include	Exclude
`/persist` / service state dirs	`/nix/store` (rebuild from flake)
Database dumps or consistent snapshots	Ephemeral caches, `/tmp`
Secrets **materials** you cannot reissue easily	Cloud provider API caches
User home select paths	`result` symlinks, build trees
Backup tool config references	The entire OS image as only strategy

Rule: If wipe+reinstall cannot recreate it from git+keys+cache, it needs a backup story.

Inventory questions

1. What does the impermanence chapter impermanence already send to `/persist`?
2. What does your the data service pattern chapter data service write?
3. What keys exist only once (age, SSH CA, ACME account)?

4. What would embarrass you if lost (homelab photos vs throwaway metrics)?

Theory 2 — Tool choice (pick one)

Tool	Strengths	Notes
Restic	Simple; many backends (S3, B2, SFTP, local)	Encrypted snapshots; prune policies
Borg	Dedup; mature; SSH repos	Borgmatic helper common on NixOS
ZFS send/recv	Excellent for ZFS hosts	Not an application-level portable tool

This book accepts any **one** done well. Switching tools mid-day is thrash—pick and finish.

3-2-1 reminder

Idea	Lab minimum
3 copies	Live + backup + offsite note
2 media	Disk + remote/object
1 offsite	Even “friend’s SFTP” or second VPS

Local-only lab is fine for learning **if** you write the offsite sentence for real systems.

Theory 3 — Restic on NixOS (shape)

```
{ config, pkgs, ... }:
{
  # Prefer sops-nix for repository password / access keys (the sops-nix chapter)
  # sops.secrets.restic-password = { … };

  services.restic.backups.lab = {
    initialize = true;
    passwordFile = config.sops.secrets.restic-password.path;
    repository = "sftp:backup@backup-host:/backups/lab";
    # or "s3:s3.amazonaws.com/bucket/lab"
    paths = [
```

```
      "/persist"
      "/var/lib/myapp"
    ];
    timerConfig = {
      OnCalendar = "daily";
      Persistent = true;
    };
    pruneOpts = [
      "--keep-daily 7"
      "--keep-weekly 4"
      "--keep-monthly 6"
    ];
  };
}
```

Exact module options: check `man configuration.nix` / search.nixos.org for your **26.05** pin if attribute names shift.

Borgmatic sketch

```
{
  services.borgmatic.enable = true;
  # services.borgmatic.settings = { … };
  # or environment.etc."borgmatic.d/lab.yaml".text = …;
}
```

ZFS

```
zfs snapshot tank/persist@day69
zfs send -v tank/persist@day69 | ssh backup zfs recv -F backup/lab/persist
```

Declare snapshot timers via Nix modules or sanoid/syncoid if you go that ecosystem. Still do a **restore drill** (recv to a scratch dataset, mount, verify files).

Theory 4 — Consistency

Data	Technique
Postgres	`backupPrepareCommand` dump or brief stop / native snapshot
SQLite	Stop service or `.backup` API; avoid mid-write copies
VM disks	Hypervisor snapshot or fs freeze

Data	Technique
Mutable logs	Optional; size-cap
Mail / queues	App-native export

```
services.restic.backups.lab.backupPrepareCommand = ''
  mkdir -p /var/backup
  ${pkgs.sudo}/bin/sudo -u postgres \
    ${pkgs.postgresql}/bin/pg_dumpall > /var/backup/pg.sql
'';
# include /var/backup/pg.sql in paths
```

Inconsistent DB files that "restored" but won't start are not a successful drill.

Theory 5 — Restore is the product

```
# Restic example
export RESTIC_REPOSITORY=…
export RESTIC_PASSWORD_FILE=…
restic snapshots
restic restore latest --target /tmp/restore-drill
# verify files; for DB: restore dump into disposable instance
```

Success criteria

Level	Meaning
Files on disk	Necessary but insufficient
Service starts	Better
Application-usable state	**Done** for the backups chapter
Timed RTO/RPO notes	Ops maturity

Write `RESTORE.md` with commands a tired operator can paste.

Theory 6 — Secrets and backup encryption

Concern	Practice
Repo password	sops-nix / agenix path; not world-readable
Encryption keys only on source host	**Fails** disaster recovery—offsite custody
Backing up decrypted `/run/secrets`	Usually wrong; back up encrypted materials + key custody story
Same disk as source	Not a disaster backup

Restic/Borg encrypt client-side; you still need the password after wipe (the disaster recovery chapter).

Worked example — local restic lab (no cloud)

```
{
  services.restic.backups.local = {
    initialize = true;
    passwordFile = "/persist/secrets/restic-password"; # lab only; prefer sops
    repository = "/mnt/backup-disk/restic-lab";
    paths = [ "/persist/PROOF" "/var/lib/myapp" ];
    timerConfig.OnCalendar = "hourly";
  };
}
```

```
echo important-state | sudo tee /persist/PROOF
sudo systemctl start restic-backups-local.service
sudo systemctl status restic-backups-local.service --no-pager
sudo restic -r /mnt/backup-disk/restic-lab snapshots
sudo rm /persist/PROOF
sudo restic -r /mnt/backup-disk/restic-lab restore latest --target /tmp/r
sudo install -D /tmp/r/persist/PROOF /persist/PROOF
cat /persist/PROOF
```

Worked example — RTO/RPO honesty

```
## Lab targets (not production promises)
- RPO: up to 24h (daily timer) - last successful snapshot time: …
- RTO: ~30-60 min to restore PROOF + app dir on empty disk (measured: …)
- Not included: full bare-metal with SB/TPM re-enroll (the Secure Boot/TPM chapter)
```

Lab — multi-step

Suggested notes: `~/lab/nixos/ch69`

Step 1 — Choose tool

Write choice + backend (local disk, SFTP, S3-compatible) in NOTES.

Step 2 — State inventory

List paths tied to services you run (reuse the impermanence chapter inventory). Mark each: rebuildable from git vs must backup.

Step 3 — Declare backup job in Nix

Wire secrets properly if possible (sops). Commit module + docs.

Step 4 — First backup

Run service once; list snapshots; capture logs.

```
mkdir -p ~/lab/nixos/ch69
sudo systemctl start restic-backups-lab.service  # name may vary
journalctl -u restic-backups-lab.service -n 100 --no-pager \
  | tee ~/lab/nixos/ch69/backup.log
```

Step 5 — Restore drill (required)

Delete or move a **lab** file; restore; verify content byte-for-byte or known string.

Step 6 — DB drill (if you have the data service pattern chapter DB)

Dump → backup → drop table/schema in lab → restore dump → query. If no DB, write N/A with reason.

Step 7 — Offsite note

Even local-only lab needs a sentence on offsite/3-2-1 for real systems.

Step 8 — Runbook

`docs/RESTORE.md`:

- Prerequisites (password location, repo URL)
- List snapshots
- Restore commands
- Service restart order
- Lab RTO/RPO numbers

Step 9 — Negative cases table

Failure	Detection	Response
Timer silent fail	monitoring / `systemctl --failed`	
Repo full	prune / expand	
Wrong password after wipe	key custody	
Restored inconsistent DB	prepare hooks	

Common gotchas

Symptom / mistake	What to do
Backed up `/nix` only	Useless for state; fix paths
Password in world-readable file	sops/agenix; modes
Never pruned	Fill disk; set keep policy
Restored files, app still broken	Consistent DB story
Same disk as source	Not a disaster backup
Encryption keys only on source host	Offsite key custody
"Backup works" without restore	Fail the day until restore proves
Hardlink confusion with optimise	Know tool behavior (the GC hygiene chapter)

Checkpoint

- ☐ One tool chosen and declared in NixOS
- ☐ Backup job succeeds with log proof
- ☐ Restore drill completed with proof
- ☐ `RESTORE.md` written with RTO/RPO honesty
- ☐ Store vs state distinction clear in notes
- ☐ Secrets for repo password handled sanely
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "day69: backup + restore drill"
```

Write three personal gotchas before continuing.

Unattended upgrades (system.autoUpgrade)

Unattended upgrades (system.autoUpgrade)

Goal: Let hosts upgrade themselves on a schedule from a **reviewed Git ref**. Reboot only when the kernel or initrd changed, and only inside a window. Stagger the fleet in rings, and page someone when an upgrade fails. Read the unit that NixOS generates, so nothing about the automation is magic.

Baseline: NixOS **26.05** ("Yarara"), where `nixos-rebuild` is the Python rewrite (`nixos-rebuild-ng`) and the Bash version is gone. The 26.05 default `pkgs.nix` is **2.34.x**. `nixVersions.latest` is **2.35.x** if you set `nix.package`. Nothing here depends on that split. Every option below comes from `nixos/modules/tasks/auto-upgrade.nix` on the `nixos-26.05` branch.

Thesis

`system.autoUpgrade` is a systemd timer that runs `nixos-rebuild` as root. That is all it is. Whether it is safe depends entirely on **what it builds from**:

- From a channel, or from a flake whose lock file the host updates itself, it is an **unreviewed production change** at 03:00.
- From a Git branch that CI has already built, tested, and pushed to a cache, it is just the **pull half of GitOps**: every host converges on a commit a human approved.

Use it only in the second mode. Then the remaining questions are scheduling ones: when to run, when a reboot is allowed, and how to hear about failures.

Mental model

```
nixos-upgrade.timer  (OnCalendar=dates, RandomizedDelaySec, FixedRandomDelay, Persistent)


nixos-upgrade.service  (Type=oneshot, root, After/Wants=network-online.target,
                        X-RestartIfChanged=false: a switch never kills its own upgrade)

          allowReboot = false   nixos-rebuild <operation> <flags> [--upgrade]

          allowReboot = true    nixos-rebuild boot <flags>          (new default boot entry)
                                   compare readlink of {initrd,kernel,kernel-modules}
                                   /run/booted-system  vs  /nix/var/nix/profiles/system
                                       same        nixos-rebuild <operation> <flags>
                                       different, outside rebootWindow   "skipping" (nothing
```

```
                                    different, inside rebootWindow    shutdown -r +1

          OnSuccess=nix-gc.service   (only if runGarbageCollection = true)
```

Option (26.05)	Default	What it really does
`enable`	`false`	Creates the timer and the service
`flake`	`null`	Flake URI. Adds `--refresh --flake <uri>` (`--refresh` forces a re-fetch of the branch tip)
`channel`	`null`	Channel URL, passed as `-I nixpkgs=<url>/nixexprs.tar.xz`. Cannot be combined with `flake`
`upgrade`	`true`	Adds `--upgrade` when no `channel` is set. That updates **channels**, not your flake lock. Set it to `false` for flakes
`operation`	`"switch"`	`"switch"` or `"boot"`
`flags`	`[ ]`	Extra `nixos-rebuild` arguments
`dates`	`"04:40"`	`OnCalendar=` expression (`systemd.time(7)`)
`randomizedDelaySec`	`"0"`	Timer jitter, so a fleet does not stampede the Git server and the cache
`fixedRandomDelay`	`false`	Keep the same jitter on every run (stable per host)
`persistent`	`true`	Catch up on a run missed while powered off
`allowReboot`	`false`	Reboot instead of switching when kernel, initrd, or modules changed
`rebootWindow`	`null`	`{ lower = "HH:MM"; upper = "HH:MM"; }`. Crossing midnight is supported. Bounds are **exclusive**
`runGarbageCollection`	`false`	`OnSuccess=nix-gc.service` (needs `nix.enable`). Tune retention with `nix.gc.options`

Worked examples (Cases)

Case 1: A server tracking a reviewed branch

```nix
# auto-upgrade-server.nix - unattended upgrades for a flake-defined server
{ ... }:
{
  system.autoUpgrade = {
    enable = true;
    flake = "git+https://git.example.com/infra/nixos?ref=release/prod#web1";
```

```
    upgrade = false;              # flake: never pass --upgrade; lock comes from Git
    operation = "boot";           # stage the new generation; reboot applies it
    dates = "Wed 03:00";
    randomizedDelaySec = "45min";
    fixedRandomDelay = true;      # same offset every week on this host
    persistent = true;
    allowReboot = true;
    rebootWindow = { lower = "03:00"; upper = "05:30"; };
    runGarbageCollection = true;
  };
}
```

Do not trust the option descriptions. Evaluate the module and read what it generates. This evaluation runs on any machine with Nix, with no NixOS required:

```
# eval.nix - evaluate a minimal NixOS system and print what autoUpgrade generates
let
  sys = import <nixpkgs/nixos> {
    configuration = { ... }: {
      imports = [ ./auto-upgrade-server.nix ];
      boot.loader.grub.enable = false;
      fileSystems."/" = { device = "/dev/disk/by-label/nixos"; fsType = "ext4"; };
      system.stateVersion = "26.05";
    };
  };
  svc = sys.config.systemd.services.nixos-upgrade;
in {
  flags = sys.config.system.autoUpgrade.flags;
  startAt = svc.startAt;
  onSuccess = svc.unitConfig.OnSuccess;
  timer = sys.config.systemd.timers.nixos-upgrade.timerConfig;
  script = svc.script;
}
```

```
nix-instantiate --parse auto-upgrade-server.nix >/dev/null && echo parse ok
nix-instantiate --eval --strict --json -I nixpkgs=channel:nixos-26.05 eval.nix > out.json
jq -c '.flags, .startAt, .onSuccess, .timer' out.json
jq -r .script out.json
```

```
parse ok
["--refresh","--flake git+https://git.example.com/infra/nixos?ref=release/prod#web1"]
["Wed 03:00"]
["nix-gc.service"]
{"FixedRandomDelay":true,"OnCalendar":["Wed 03:00"],"Persistent":true,"RandomizedDelaySec":"
/nix/store/26nvrcq5…-nixos-rebuild-ng-26.05/bin/nixos-rebuild boot --refresh --flake git+htt
booted="$(/nix/store/833invqv…-coreutils-9.11/bin/readlink /run/booted-system/{initrd,kernel
```

```
built="$(/nix/store/833invqv…-coreutils-9.11/bin/readlink /nix/var/nix/profiles/system/{init

current_time="$(/nix/store/833invqv…-coreutils-9.11/bin/date +%H:%M)"

lower="03:00"
upper="05:30"

if [[ "${lower}" < "${upper}" ]]; then
  if [[ "${current_time}" > "${lower}" ]] && \
     [[ "${current_time}" < "${upper}" ]]; then
    do_reboot="true"
  else
    do_reboot="false"
  fi
else
  # lower > upper, so we are crossing midnight (e.g. lower=23h, upper=6h)
  # we want to reboot if cur > 23h or cur < 6h
  if [[ "${current_time}" < "${upper}" ]] || \
     [[ "${current_time}" > "${lower}" ]]; then
    do_reboot="true"
  else
    do_reboot="false"
  fi
fi


if [ "${booted}" = "${built}" ]; then
  /nix/store/26nvrcq5…-nixos-rebuild-ng-26.05/bin/nixos-rebuild boot --refresh --flake git+h
elif [ "${do_reboot}" != true ]; then
  echo "Outside of configured reboot window, skipping."

else
  /nix/store/b4b1x5fh…-systemd-260.4/bin/shutdown -r +1
fi
```

That is real evaluator output against `nixos-26.05` (store hashes shortened). Three facts you can only see here:

1. `--upgrade` is gone (`upgrade = false`), and `--refresh` makes Nix re-fetch `release/prod` on every run instead of trusting its fetcher cache.
2. `current_time` is read **after** the build, and both bounds use strict `>` / `<`.
3. With `operation = "boot"`, the "kernel unchanged" branch runs `nixos-rebuild boot` **again**. Userland-only changes are staged and never activated. Case 2 and the trap below explain why that matters.

Case 2: Switch userland now, reboot only for kernel and initrd

For most servers this is the right shape. Service and package changes go live on the night they land, and a reboot happens only when it is actually required, inside the window:

```
# auto-upgrade-switch.nix - live switch for userland, windowed reboot for kernel/initrd
{ ... }:
{
  system.autoUpgrade = {
    enable = true;
    flake = "git+https://git.example.com/infra/nixos?ref=release/prod#web1";
    upgrade = false;
    operation = "switch";          # userland changes go live now
    dates = "Wed 03:00";
    randomizedDelaySec = "45min";
    fixedRandomDelay = true;
    allowReboot = true;
    rebootWindow = { lower = "03:00"; upper = "05:30"; };
  };
}
```

The generated script now ends with:

```
if [ "${booted}" = "${built}" ]; then
  /nix/store/26nvrcq5…-nixos-rebuild-ng-26.05/bin/nixos-rebuild switch --refresh --flake git
elif [ "${do_reboot}" != true ]; then
  echo "Outside of configured reboot window, skipping."
```

The rendered units, from `config.systemd.units."nixos-upgrade.timer".text` and `."nixos-upgrade.service` with the store-path `Environment=` lines (`PATH`, `LOCALE_ARCHIVE`, `TZDIR`) trimmed:

```
[Timer]
FixedRandomDelay=true
OnCalendar=Wed 03:00
Persistent=true
RandomizedDelaySec=45min

[Install]
WantedBy=timers.target
```

```
[Unit]
After=network-online.target
Description=NixOS Upgrade
Wants=network-online.target
X-StopOnRemoval=false

[Service]
```

```
Environment="HOME=/root"
Environment="NIX_PATH=nixpkgs=/nix/var/nix/profiles/per-user/root/channels/nixos:nixos-confi
X-RestartIfChanged=false
ExecStart=/nix/store/v085579p…-unit-script-nixos-upgrade-start/bin/nixos-upgrade-start
Type=oneshot
```

`X-RestartIfChanged=false` and `X-StopOnRemoval=false` tell `switch-to-configuration` not to restart or stop the upgrade unit in the middle of its own run. `HOME=/root` means credentials for a private flake are **root's**.

Case 3: Rings: canary, early, prod from one module

Run every host off the same module. The ring decides which branch a host tracks and on which day it upgrades:

```nix
# modules/auto-upgrade-rings.nix - one module, every host; the ring decides when and what
{ config, lib, ... }:
let
  rings = {
    canary = { branch = "release/canary"; dates = "Mon 03:00"; };
    early  = { branch = "release/prod";   dates = "Wed 03:00"; };
    prod   = { branch = "release/prod";   dates = "Thu 03:00"; };
  };
  ring = rings.${config.fleet.ring};
in
{
  options.fleet.ring = lib.mkOption {
    type = lib.types.enum (builtins.attrNames rings);
    default = "prod";
    description = "Upgrade ring: decides which Git branch this host tracks and when it upgra
  };

  config.system.autoUpgrade = {
    enable = true;
    flake = "git+https://git.example.com/infra/nixos?ref=${ring.branch}#${config.networking.
    upgrade = false;
    operation = "switch";
    dates = ring.dates;
    randomizedDelaySec = "30min";
    fixedRandomDelay = true;
    allowReboot = true;
    rebootWindow = { lower = "02:30"; upper = "05:30"; };
  };
}
```

A host sets `networking.hostName = "lab1"; fleet.ring = "canary";`. Evaluating three hosts gives:

```
lab1 {"dates": "Mon 03:00", "flake": "git+https://git.example.com/infra/nixos?ref=release/ca
web1 {"dates": "Wed 03:00", "flake": "git+https://git.example.com/infra/nixos?ref=release/pr
db1  {"dates": "Thu 03:00", "flake": "git+https://git.example.com/infra/nixos?ref=release/pr
```

The release process now happens entirely in Git and CI. **Hosts never touch the lock file**:

```
# CI job (weekly): bump inputs, build every host, push to the binary cache, open a PR
nix flake update
nix build .#nixosConfigurations.{lab1,web1,db1}.config.system.build.toplevel
# … run checks / nixosTests, push closures to the cache, merge PR into main

# promotion is a fast-forward, not a rebuild:
git push origin main:release/canary            # Monday: canaries take it
git push origin release/canary:release/prod    # after a clean soak: early + prod rings
```

Because CI has already pushed the closures, hosts **substitute** from the cache instead of compiling, so a run takes minutes. Rolling back the fleet means resetting `release/prod` to the previous commit. Hosts converge back to it on their next run.

Case 4: Hear about failures

A oneshot that fails at 03:00 is invisible unless something reports it:

```
# modules/upgrade-alerts.nix - page someone when an unattended upgrade fails
{ pkgs, ... }:
{
  systemd.services.nixos-upgrade.onFailure = [ "upgrade-alert@%n.service" ];

  systemd.services."upgrade-alert@" = {
    description = "Report failure of %i";
    serviceConfig = {
      Type = "oneshot";
      EnvironmentFile = "/run/secrets/ntfy.env";   # NTFY_URL=https://ntfy.example.com/ops
    };
    scriptArgs = "%i";
    path = [ pkgs.curl pkgs.systemd pkgs.coreutils ];
    script = ''
      unit="$1"
      journalctl -u "$unit" -n 30 --no-pager -o cat \
        | curl -fsS -H "Title: $(hostname): $unit failed" -H "Priority: high" \
            --data-binary @- "$NTFY_URL"
    '';
  };
}
```

Both modules parse (`nix-instantiate --parse`) and evaluate against `nixos-26.05`. The rendered template unit contains `EnvironmentFile=/run/secrets/ntfy.env` and `ExecStart=…/upgrade-alert_-start`

`%i`. Ship `ntfy.env` with your secrets tool (sops-nix or agenix). Never put the URL or token in the Nix store.

A **skipped** reboot is not a failure: the unit exits `0`. If you want to know about kernels waiting for a reboot, alert on the condition directly, for example with a node-exporter textfile metric or a daily check:

```
# pending-reboot.sh - exit 1 if the default boot generation has a different kernel/initrd
for f in initrd kernel kernel-modules; do
  [ "$(readlink /run/booted-system/$f)" = "$(readlink /nix/var/nix/profiles/system/$f)" ] ||
done
echo "booted system is current"
```

Case 5: Operate it on the host

Trigger a run by hand and watch it. The commands are exact. The output below is illustrative, from a 26.05 lab host:

```
systemctl list-timers nixos-upgrade.timer
sudo systemctl start nixos-upgrade.service      # blocks until the oneshot finishes
journalctl -u nixos-upgrade.service -b --no-pager -o cat | tail -n 8
nix store diff-closures /run/booted-system /run/current-system
```

```
NEXT                         LEFT LAST                         PASSED UNIT                AC
Wed 2026-10-07 03:21:44 IST  18h  Wed 2026-09-30 03:21:44 IST  6 days ago nixos-upgrade.time

building the system configuration...
updating GRUB 2 menu...
activating the configuration...
setting up /etc...
reloading user units for root...
restarting the following units: nginx.service
Finished NixOS Upgrade.

nginx: 1.28.0 → 1.28.1, +4.1 KiB
openssl: 3.5.4 → 3.5.5, -12.3 KiB
```

Because of `fixedRandomDelay`, the `NEXT` time lands at the same offset after 03:00 every week. A changing offset means the option is not set.

Case 6: Laptops and desktops

```
# auto-upgrade-laptop.nix - never reboot a user's machine; stage, then let them reboot
{ ... }:
{
  system.autoUpgrade = {
    enable = true;
    flake = "git+https://git.example.com/infra/nixos?ref=release/prod#laptop1";
    upgrade = false;
    operation = "boot";           # nothing changes under the user's open session
    dates = "daily";
    randomizedDelaySec = "1h";
    persistent = true;            # catch up after the lid was closed all night
    allowReboot = false;
  };
}
```

Here `operation = "boot"` is correct. Nothing changes under an open desktop session, and the next reboot the user chooses applies the update. `persistent = true` means a laptop that slept through 03:00 runs the upgrade soon after it wakes. If that is unwelcome on battery, add a condition to the service:

```
# auto-upgrade-ac-only.nix - skip the run (not fail it) when not on AC power
{ ... }:
{
  systemd.services.nixos-upgrade.unitConfig.ConditionACPower = true;
}
```

The trap

`operation = "boot"` together with `allowReboot = true` quietly stops applying changes.

Read the Case 1 script again. When the kernel, initrd, and modules are unchanged (most weeks), the script runs `nixos-rebuild boot` and stops there. The new generation becomes the default boot entry, but the running system stays on the old one: a patched OpenSSH, a new firewall rule, or a rotated certificate path is **built and never activated**. The next reboot, perhaps an unplanned one months later, applies every staged change at once, and nobody knows which change broke the boot.

The reboot window has a second, related failure. `current_time` is checked **after** the build, with exclusive bounds. If a slow build finishes after `upper`, or exactly at `lower`, the host logs `Outside of configured reboot window, skipping.`, exits successfully, and activates **nothing**, userland included, because only the "kernel unchanged" branch ever switches. This repeats every week until a run happens to finish inside the window.

Fixes:

- Servers: `operation = "switch"` with `allowReboot = true` (Case 2). Userland goes live, and a reboot happens only when the kernel requires one.
- Make the window wider than `randomizedDelaySec` plus your worst build time. Better still, keep builds short by having CI push closures to a cache (Case 3).
- Alert on "reboot pending" (Case 4), not just on failure.
- Never set `operation = "boot"` with `allowReboot = true` unless you really want "nothing changes until a kernel changes".

Pitfalls

Symptom	Cause / fix
Fleet drifts: hosts run different nixpkgs commits	Hosts update their own lock (`--recreate-lock-file`, or the deprecated `--update-input` that the module docs still suggest). Update locks in CI only, and set `upgrade = false`
Upgrade "does nothing" on a flake host	The branch tip did not move, or the host tracks the wrong ref. Check `nixos-version --configuration-revision` against `git rev-parse release/prod` (needs `system.configurationRevision` set from `self.rev`)
`error: … Permission denied (publickey)` in the journal	The private repo needs **root's** credentials (`HOME=/root`). Use a read-only deploy key for `git+ssh://`, declare host keys with `programs.ssh.knownHosts`, or put `access-tokens` for `github:` / `gitlab:` refs in `nix.extraOptions = "!include /run/secrets/nix-access-tokens.conf";`
Every host compiles for an hour	Nothing was pushed to a cache before promotion. Build in CI and add the cache to `nix.settings.substituters` and `trusted-public-keys`
Thundering herd on the Git server and cache	`randomizedDelaySec` is unset. Add jitter, and `fixedRandomDelay` for predictability
Out of disk mid-upgrade	Two closures plus the build need space. Use `runGarbageCollection = true` with `nix.gc.options = "--delete-older-than 14d"`, plus `nix.settings.min-free`
`channel` and `flake` both set	Evaluation assertion: pick one. On flake hosts, also delete leftover root channels
Reboot never happens	`rebootWindow` crosses midnight with a typo, or exclusive bounds miss the run. Test the window with the generated script
Switch inhibitor blocks the run	26.05 switch inhibitors (for example a changed `dbus` implementation) refuse `switch`. The unit fails. Schedule a reboot instead of forcing it with `NIXOS_NO_CHECK=1`

The boring rule

Hosts **pull a reviewed branch**; they never update locks. `upgrade = false`, `operation = "switch"`, `allowReboot = true` with a generous window on servers. `operation = "boot"` with no auto-reboot on laptops. Jitter with `randomizedDelaySec` + `fixedRandomDelay`, rings by branch and day, closures pre-pushed to a cache, and an `onFailure` alert plus a "reboot pending" check. Promotion and rollback are both `git push`.

Try this

1. Run `eval.nix` against `nixos-26.05` with your real host module. Read the generated script and write down which branch runs on a week when only userland changed.
2. Change `rebootWindow` to cross midnight (`lower = "23:00"; upper = "02:00";`), re-evaluate, and walk the script by hand for `current_time` values `22:59`, `23:00`, `23:01`, and `01:59`.
3. Add `modules/auto-upgrade-rings.nix` to a two-host lab. Put one host in `canary`, promote a harmless change (a new `environment.etc` file) with `git push origin main:release/canary`, and confirm only the canary picks it up on its next run (`sudo systemctl start nixos-upgrade.service`).
4. Break the flake on `release/canary` (an undefined option). Confirm that `upgrade-alert@nixos-upgrade.se` fires, and that the host keeps running its previous generation.
5. Bump the kernel (`boot.kernelPackages = pkgs.linuxPackages_latest;`) on a lab VM. Trigger the upgrade outside the window and run `pending-reboot.sh`. Then trigger it inside the window and confirm the `shutdown -r +1` path.

Checkpoint

- ☐ Every auto-upgrading host uses `flake = …?ref=<release branch>#<host>` and `upgrade = false`
- ☐ No host updates `flake.lock`. CI does, then builds, tests, and caches
- ☐ Servers use `operation = "switch"`; any `boot` + `allowReboot` combination is written down and intended
- ☐ Reboot window is wider than jitter plus worst build time
- ☐ `onFailure` alert and a pending-reboot check are deployed
- ☐ Promotion and rollback procedures are written as `git push` commands

Sources

- nixpkgs `nixos/modules/tasks/auto-upgrade.nix` (nixos-26.05): option definitions and the generated script
- NixOS manual, Automatic Upgrades, and 26.05 release notes (`nixos-rebuild-ng` only, switch inhibitors)

- systemd.timer(5) (`RandomizedDelaySec=`, `FixedRandomDelay=`, `Persistent=`) and systemd.unit(5) (`OnFailure=`, `OnSuccess=`, `ConditionACPower=`)
- Nix reference manual 2.35: `nix flake update`, `--refresh`; `--update-input` deprecated

Read an Update Before You Switch: diff-closures, path-info and why-depends

Read an Update Before You Switch: diff-closures, path-info and why-depends

Goal: Turn `nix flake update` from a leap of faith into a change you have read. Build the new system without activating it, list exactly which packages changed version or size, explain any surprise down to the file that holds the reference and the nixpkgs line that caused it, and only then pick `switch`, `boot`, or "not today".

Other chapters touch `nix store diff-closures` in passing: after an unattended upgrade, or when chasing reproducibility. This one makes it the main tool, and runs it on a real update between two pinned nixpkgs revisions.

Thesis

A NixOS system is a store path, and so is the next one. Two store paths can be compared *before* either is running. Everything `nixos-rebuild switch` is about to do to your machine is already written in the difference between `/run/current-system` and `./result`. Read that difference and the update stops being a surprise.

Mental model

Three commands, each answering one question:

Question	Command	Answer looks like
What changed?	`nix store diff-closures A B`	`glibc: 2.40-224 → 2.44-57, 5.1 MiB`
How big is it now?	`nix path-info -Sh A B`	closure size per path
Why is *this* in there?	`nix why-depends --precise B <path>`	the chain of references, down to the file and the bytes

`A` and `B` are any two store paths: two system toplevels, two home-manager generations, two dev shells, or two `nix build` results.

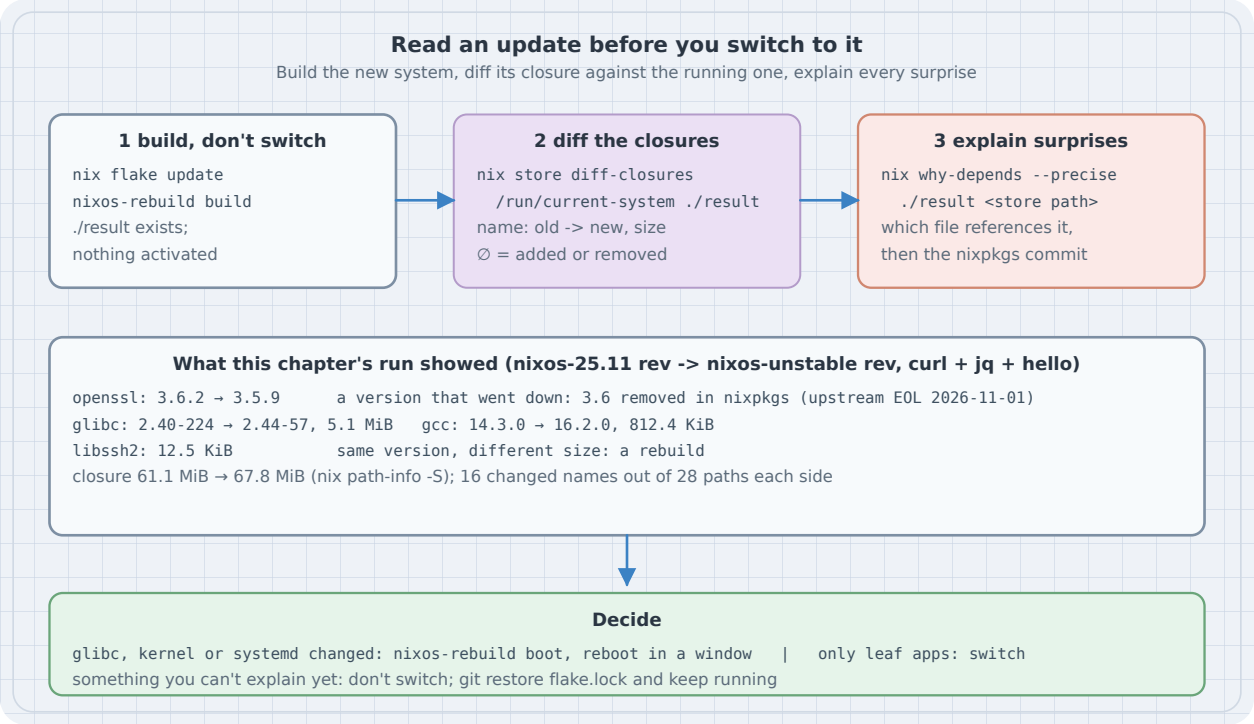


Figure 1: Read an update before you switch: build, diff the closures, explain surprises, then decide

Concepts

What diff-closures prints

`nix store diff-closures` takes the full closure (every path reachable by reference) of each side and **groups store paths by package name**, so `glibc-2.40-224` and `glibc-2.44-57` line up as one entry. For each name it prints:

- `old → new` when the set of versions differs. means "none": `→ 15.2.0` is a package added, `2.12.3 →` a package removed.
- a size delta (red for growth when the terminal has colour), but **only when it is at least 8 KiB**. The rule is `std::abs(sizeDelta) >= 8 * 1024` in `src/nix/diff-closures.cc`. Smaller changes are noise and get dropped.
- a name with a size and **no version change**, like `libssh2: 12.5 KiB`. That's a rebuild: same version, different inputs (a new compiler, a new dependency), so a different path and different bytes.

A name appears only if its versions changed or its size moved by 8 KiB or more. A quiet `diff-closures` means a quiet update.

Reading it for risk

Sort the lines into three groups:

1. **Platform**: `linux`, `systemd`, `glibc`, `gcc` (its `-lib` output is in almost every closure), `openssl`, `nix` itself. Every service links against these. A kernel or initrd change needs a reboot to take effect anyway, so prefer `nixos-rebuild boot` and a reboot in a window.
2. **Your services**: `postgresql`, `nginx`, `openssh`, and so on. Check the release notes, especially for a major version jump (on-disk formats, config syntax).
3. **Surprises**: a version that went *down*, a package you didn't know you had (→ …), a big size jump on a rebuild. Each needs a `why-depends` and a look at nixpkgs before you switch.

why-depends –precise

`nix why-depends A B` prints the shortest reference chain from `A` to `B`. With `--precise`, it also prints **which file** in each path contains the reference, and a snippet of the bytes around the hash. Store references are just hashes inside files, so this is a literal answer: "curl's binary has openssl's path in its RPATH".

Worked example: a real update (run here)

To keep the run small, the "system" is a three-package `buildEnv`. The commands are the same for a full NixOS toplevel, which is just a bigger store path. Both nixpkgs revisions are channel heads, so everything comes from cache.nixos.org and nothing needs compiling.

`toolbox.nix`:

```
# A tiny stand-in for a system closure: three tools from one nixpkgs revision.
{ rev }:
let
  pkgs = import (builtins.fetchTarball
    "https://github.com/NixOS/nixpkgs/archive/${rev}.tar.gz") { };
in
pkgs.buildEnv {
  name = "toolbox";
  paths = [ pkgs.curl pkgs.jq pkgs.hello ];
}
```

`toolbox2.nix` is the same with `[ pkgs.curl pkgs.jq pkgs.ripgrep ]`: a configuration change instead of a nixpkgs change.

`read-update.sh`:

```
#!/bin/sh
# Build the same toolbox from two pinned nixpkgs revisions, then read the difference.
set -e
OLD=b6018f87da91d19d0ab4cf979885689b469cdd41   # nixos-25.11 channel
NEW=1c89083e451e891152365b799595cf7790a43325   # nixos-unstable channel
nix build --impure -f toolbox.nix --argstr rev "$OLD" -o old
nix build --impure -f toolbox.nix --argstr rev "$NEW" -o new
```

```
echo "== what changed";      nix store diff-closures ./old ./new
echo "== closure sizes";     nix path-info -Sh ./old ./new
echo "== paths in each";     nix path-info -r ./old | wc -l; nix path-info -r ./new | wc -l
OSSL=$(nix path-info -r ./new | grep -- '-openssl-3[.0-9]*$')
echo "== why openssl";       nix why-depends --precise ./new "$OSSL"
echo "== why did openssl go down?"
SRC=$(nix eval --impure --raw --expr \
  "builtins.fetchTarball \"https://github.com/NixOS/nixpkgs/archive/$NEW.tar.gz\"")
nix store cat "$SRC/pkgs/top-level/all-packages.nix" | grep -n '^  openssl = '
nix store cat "$SRC/pkgs/top-level/aliases.nix"      | grep -n '^  openssl_3_6 = '
echo "== a configuration change on the same revision"
nix build --impure -f toolbox2.nix --argstr rev "$NEW" -o new2
nix store diff-closures ./new ./new2
```

```
$ sh read-update.sh
== what changed
bash: 5.3p3 → 5.3p20, 36.1 KiB
brotli: 1.1.0 → 1.2.0, 40.4 KiB
curl: 8.20.0 → 8.22.0, 285.2 KiB
gcc: 14.3.0 → 16.2.0, 812.4 KiB
glibc: 2.40-224 → 2.44-57, 5.1 MiB
hello: 2.12.2 → 2.12.3
jq: 1.8.1 → 1.8.2, 25.8 KiB
krb5: 1.22.1 → 1.22.2, 80.3 KiB
libssh2: 12.5 KiB
libunistring: 1.4.1 → 1.4.2
nghttp2: 1.68.1 → 1.70.0, 16.1 KiB
nghttp3: 1.15.0 → 1.18.0, 20.3 KiB
ngtcp2: 1.22.1 → 1.23.0, 77.2 KiB
openssl: 3.6.2 → 3.5.9, 166.5 KiB
publicsuffix-list: 0-unstable-2026-01-25 → 0-unstable-2026-09-02
xgcc: 14.3.0 → 16.2.0
== closure sizes
/nix/store/9ylain3a520r6i57303lh3za99rqpzs8-toolbox   61.1 MiB
/nix/store/xqs9h77yj2abhbv2994g9hlw6pdk9zlj-toolbox   67.8 MiB
== paths in each
28
28
== why openssl
/nix/store/xqs9h77yj2abhbv2994g9hlw6pdk9zlj-toolbox
  bin/curl -> /nix/store/5lpg7j8y8lf8c4zv318yi4rmgyn5yhip-curl-8.22.0-bin/bin/curl
    → /nix/store/5lpg7j8y8lf8c4zv318yi4rmgyn5yhip-curl-8.22.0-bin
      bin/curl: …f4xh2-zstd-1.5.7/lib:/nix/store/lazac5vw20j0pyahzbaxz8z8hyf7m9zf-openssl-3.
        → /nix/store/lazac5vw20j0pyahzbaxz8z8hyf7m9zf-openssl-3.5.9
== why did openssl go down?
6136:  openssl = openssl_3_5;
```

```
2064:  openssl_3_6 = throw "'openssl_3_6' has been removed, due to upstream end of life on 2
== a configuration change on the same revision
hello: 2.12.3 → , -273.0 KiB
pcre2:  → 10.49, 2.0 MiB
ripgrep:  → 15.2.0, 6.7 MiB
```

Reading this output

- **Sixteen names changed, out of 28 paths on each side.** The three packages we asked for (curl, jq, hello) all moved, but so did glibc, gcc, bash and the HTTP/2 and HTTP/3 libraries curl links against. This is normal: a channel bump moves the whole dependency graph, not just the leaves.
- **glibc 2.40 → 2.44 is the biggest item** at 5.1 MiB, and it's a platform change. On a real host it is the line that says "plan a reboot window", together with any `linux` line.
- **`openssl: 3.6.2 → 3.5.9` went down.** That is the surprise to explain. `why-depends --precise` shows the reference sits in the RPATH of `bin/curl`, so curl picked it up from nixpkgs' default `openssl`. The two `grep` lines show why: on the new revision `openssl = openssl_3_5;`, and `openssl_3_6` is now an alias that throws, because the 3.6 branch reaches upstream end of life on 2026-11-01. So the default moved to the 3.5 long-term-support branch. That's a deliberate choice, not a bug, but you only know that because you read it.
- **`libssh2: 12.5 KiB`** has no version arrow: the same libssh2 version, rebuilt against new inputs.
- **hello changed version but shows no size.** Its delta was 5,024 bytes (274,568 → 279,592), under the 8 KiB threshold.
- **The configuration change** (`toolbox2.nix`) shows the forms: hello removed (−273.0 KiB), ripgrep added (6.7 MiB), plus pcre2 (2.0 MiB), which nobody asked for by name. `nix why-depends ./new2 <pcre2 path>` shows it comes in with ripgrep (Try this #1).

On a NixOS host (from the manuals, not run here)

The build box for this book isn't a NixOS machine, so this part follows the `nixos-rebuild(8)` and Nix reference manuals and was not run. The steps are the ones above with real system paths:

```
cd /etc/nixos                                    # your flake
nix flake update                                 # or: nix flake update nixpkgs
nixos-rebuild build --flake .#$(hostname)        # writes ./result; activates nothing, needs
nix store diff-closures /run/current-system ./result
nix path-info -Sh /run/current-system ./result
nix why-depends --precise ./result /nix/store/<hash>-openssl-3.5.9   # for each surprise
```

Then decide:

```
# Only leaf applications changed:
sudo nixos-rebuild switch --flake .#$(hostname)
```

```
# Kernel, initrd, systemd or glibc changed: stage it and reboot in a window
sudo nixos-rebuild boot --flake .#$(hostname)
# Something you can't explain yet: don't switch at all
git restore flake.lock
```

Two more forms from the Nix manual are worth knowing:

- `nix profile diff-closures --profile /nix/var/nix/profiles/system` prints the diff between **every pair of consecutive system generations**: your upgrade history as a changelog.
- `nix store diff-closures /run/booted-system /run/current-system` shows what you've switched to since the last boot. If `linux` or `systemd` is in that list, the running kernel or PID 1 is older than the configuration says.

`nixos-rebuild dry-activate` complements this. It doesn't diff packages: it reports which systemd units an activation *would* start, stop, restart or reload. One tells you *what* changes, the other what *happens* when it does.

The trap

Reading the diff after `switch`. Once you've switched, the diff is just history. Run `build` first. It needs no root, can't break the running system, and the result is the exact path `switch` would activate: same inputs, same hash.

Second trap: trusting that a quiet diff means a safe update. `diff-closures` only sees *packages*. A change to NixOS *module* logic (a new default for an option, a renamed systemd unit setting) can land in `etc`, unit files or the activation script without any package version moving. In the diff, that usually shows up not at all, because those generated paths are tiny and their changes fall under the 8 KiB threshold. Read the NixOS release notes when you cross a release boundary, and use `dry-activate` to see which units would restart.

The boring rule

`nix flake update && nixos-rebuild build && nix store diff-closures /run/current-system ./result`, every time. Explain every version that goes down and every → you didn't ask for. Then `boot` if the platform moved, `switch` if it didn't, and restore `flake.lock` if you can't explain it.

Try this

1. Run `nix why-depends --precise ./new2 <pcre2 path>` and confirm it comes in with ripgrep. Which file in ripgrep holds the reference?
2. Pin `NEW` to an older nixos-unstable revision and re-run. Find the revision range where `openssl` switched from 3.6 to 3.5 (`git log -S 'openssl = openssl_3_5' -- pkgs/top-level/all-packages.nix` in a nixpkgs checkout).

3. Add `pkgs.git` to `toolbox.nix`. How many new names does `diff-closures` show for one package, and which is the biggest?
4. On a NixOS machine, run `nix profile diff-closures --profile /nix/var/nix/profiles/system | less` and find the generation where the kernel last changed.

Verification

- `read-update.sh` was run with Nix 2.35.3 as an unprivileged single-user install with a chroot store, on a Linux box that is not NixOS. Store paths and output are the same as on NixOS, because the store paths depend only on the inputs. Every line of output above is genuine. ANSI colour codes were stripped for print.
- Both nixpkgs revisions are the channel heads at the time of writing: `b6018f87…` (nixos-25.11) and `1c89083e…` (nixos-unstable). Everything was substituted from cache.nixos.org, except the two `buildEnv` derivations, which build locally in a second.
- The 8 KiB threshold was read in `src/nix/diff-closures.cc` on Nix master. The hello sizes (274,568 and 279,592 bytes) come from `nix path-info -s`.
- The NixOS-host section (`nixos-rebuild build/switch/boot`, `nix profile diff-closures`, `dry-activate`) was **not run**. It is written from the manuals linked below.

Sources

- Nix reference manual, `nix store diff-closures`: https://nix.dev/manual/nix/latest/command-ref/new-cli/nix3-store-diff-closures
- Nix reference manual, `nix why-depends`: https://nix.dev/manual/nix/latest/command-ref/new-cli/nix3-why-depends
- Nix reference manual, `nix path-info`: https://nix.dev/manual/nix/latest/command-ref/new-cli/nix3-path-info
- Nix reference manual, `nix profile diff-closures`: https://nix.dev/manual/nix/latest/command-ref/new-cli/nix3-profile-diff-closures
- Nix source, `src/nix/diff-closures.cc`: https://github.com/NixOS/nix/blob/master/src/nix/diff-closures.cc
- NixOS manual, upgrading NixOS and `nixos-rebuild`: https://nixos.org/manual/nixos/stable/#sec-upgrading
- nixpkgs `pkgs/top-level/aliases.nix` and `all-packages.nix` at `1c89083e451e891152365b799595cf7790a` https://github.com/NixOS/nixpkgs/tree/1c89083e451e891152365b799595cf7790a43325/pkgs/top-level

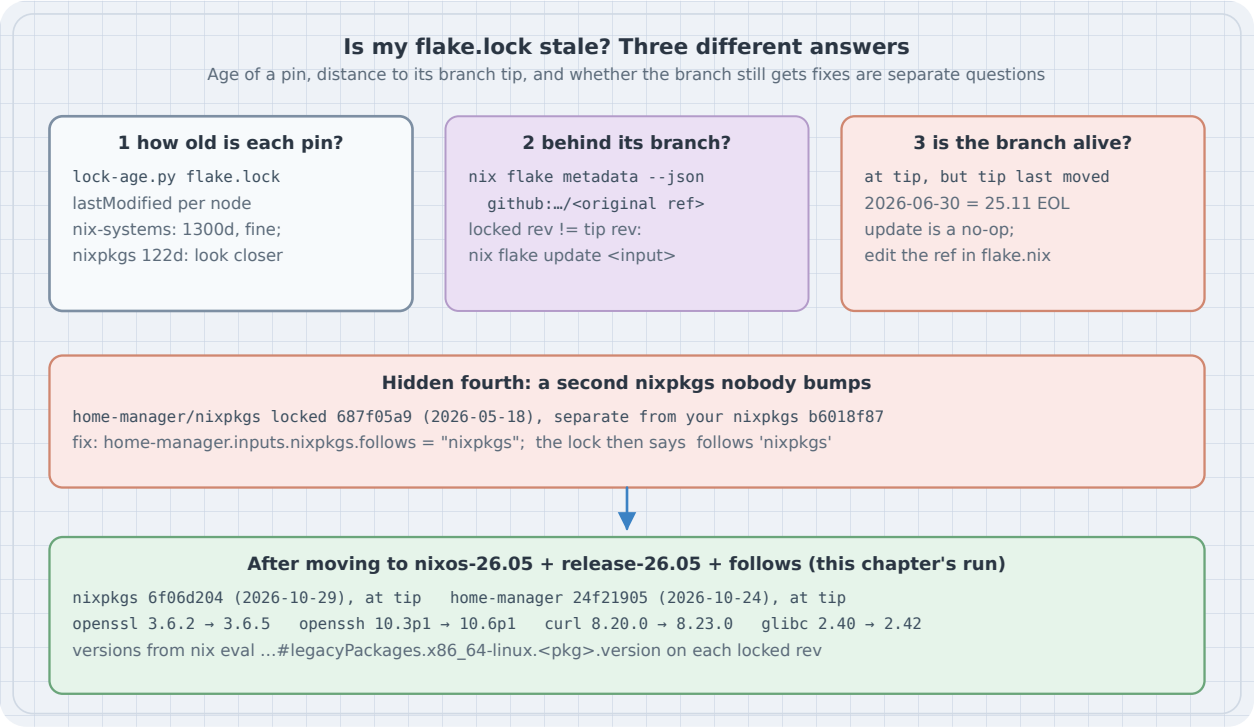
Is My flake.lock Stale? Lock Age, Branch Distance and End-of-Life Branches

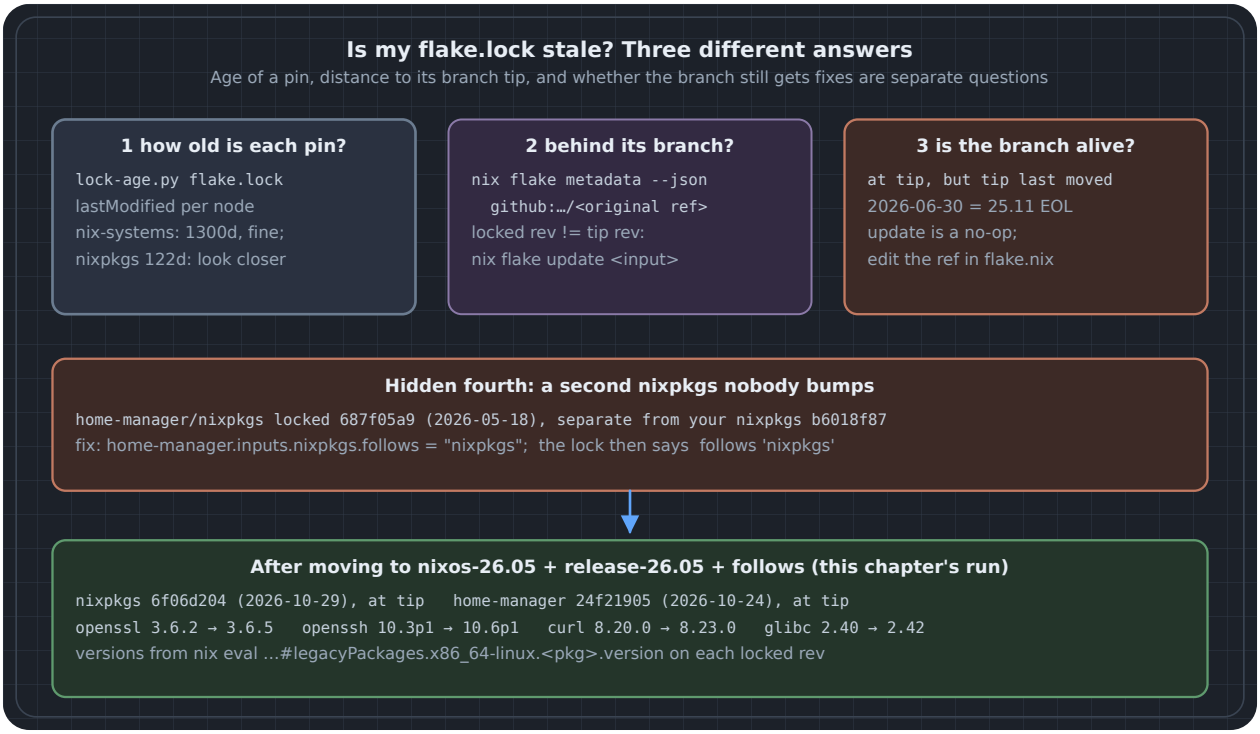
Is My flake.lock Stale? Lock Age, Branch Distance and End-of-Life Branches

Goal: After a wave of CVEs, answer "is this host actually getting fixes?" from the repository alone, before you build anything. Read every pin in `flake.lock` with its date. Check whether each pin is behind the branch it tracks. Notice the case that `nix flake update` can never fix: a branch that has stopped moving. Find the second copy of nixpkgs that another input brought along.

The chapter before this one (`15-read-an-update-before-switching`, if your copy has it) covers what to do *after* you update: diff the closures and explain each change. This chapter covers the step before that: deciding whether there is anything to update, and which line to change.

All output below is genuine. It comes from Nix **2.35.3** on 2026-10-31, run against a small host flake. Nothing was built or activated; the commands only read lock files, resolve refs and evaluate package versions.





Thesis

"Stale" means three different things, and each has a different fix:

1. **The pin is old.** Read `lastModified` in `flake.lock`. Age is only a hint. Some inputs, such as `nix-systems/default`, never move, so a three-year-old pin is still correct.
2. **The pin is behind its branch.** The branch has newer commits than the lock. `nix flake update <input>` fixes this.
3. **The branch itself is dead.** The lock sits exactly at the tip of `nixos-25.11`, but that branch stopped receiving security fixes on 2026-06-30. `nix flake update` succeeds, prints nothing, and leaves you unpatched. The only fix is to edit the ref in `flake.nix`.

There is also a fourth problem that is easy to miss. If another input carries its own nixpkgs and you never added `follows`, then the lock holds a **second nixpkgs** that your updates do not touch.

The flake under test

This is a typical host flake as it looks four months after someone last edited it:

```
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-25.11";
    home-manager.url = "github:nix-community/home-manager/release-25.11";
    flake-utils.url = "github:numtide/flake-utils";
```

```
  };
  outputs = { self, ... }: { };
}
```

`nix flake lock` wrote this lock (`narHash` query strings trimmed):

```
warning: creating lock file "/tmp/fl/host/flake.lock":
• Added input 'flake-utils':
    'github:numtide/flake-utils/b7a79d3fea1e214f6d668d521cecec0c01e26544 (2026-10-13)
• Added input 'flake-utils/systems':
    'github:nix-systems/default/da67096a3b9bf56a91d16901293e51ba5b49a27e (2023-04-09)
• Added input 'home-manager':
    'github:nix-community/home-manager/3ee51fbdac8c8bdfe1e7e1fcaba6520a563f394f (2026-05-23)
• Added input 'home-manager/nixpkgs':
    'github:NixOS/nixpkgs/687f05a9184cad4eaf905c48b63649e3a86f5433 (2026-05-18)
• Added input 'nixpkgs':
    'github:NixOS/nixpkgs/b6018f87da91d19d0ab4cf979885689b469cdd41 (2026-06-30)
```

There are already two nixpkgs revisions in that list.

Check 1: how old is each pin?

Every node in `flake.lock` has a `locked` attribute set holding `rev` and `lastModified` (Unix seconds). It also has an `original` set, which is what you wrote in `flake.nix`. An entry in a node's `inputs` is either a node name or, for `follows`, a list path. This short script walks that graph from `root`, so nested inputs print as `parent/child`:

```
#!/usr/bin/env python3
"""lock-age.py: list every locked input in flake.lock with its commit date and age."""
import json, sys, time
from datetime import datetime, timezone

lock = json.load(open(sys.argv[1] if len(sys.argv) > 1 else "flake.lock"))
nodes, now = lock["nodes"], time.time()

def paths(node="root", prefix=""):
    """Walk the input graph so nested inputs print as parent/child."""
    for name, ref in nodes[node].get("inputs", {}).items():
        if isinstance(ref, list):                       # a 'follows': it reuses another node
            yield prefix + name, None, "/".join(ref)
            continue
        yield prefix + name, ref, None
        yield from paths(ref, prefix + name + "/")

print(f"{'input':28} {'source':28} {'rev':9} {'commit date':11} {'age':>5}")
```

```
for path, key, follows in paths():
    if follows:
        print(f"{path:28} follows {follows}"); continue
    l = nodes[key]["locked"]
    src = f"{l.get('owner', '')}/{l.get('repo', '')}" if l["type"] == "github" else l["type"
    when = datetime.fromtimestamp(l["lastModified"], timezone.utc)
    age = int((now - l["lastModified"]) // 86400)
    print(f"{path:28} {src:28} {l.get('rev', '')[:8]:9} {when:%Y-%m-%d} {age:>4}d")
```

```
$ python3 lock-age.py
input                        source                       rev       commit date   age
flake-utils                  numtide/flake-utils          b7a79d3f  2026-10-13   17d
flake-utils/systems          nix-systems/default          da67096a  2023-04-09 1300d
home-manager                 nix-community/home-manager   3ee51fbd  2026-05-23  160d
home-manager/nixpkgs         NixOS/nixpkgs                687f05a9  2026-05-18  165d
nixpkgs                      NixOS/nixpkgs                b6018f87  2026-06-30  122d
```

How to read it:

- `flake-utils/systems` at 1300 days is fine. That repository is a list of system names and has not needed a commit since 2023.
- `nixpkgs` at 122 days is worth a closer look. On a supported NixOS branch, the channel normally advances every few days.
- `home-manager/nixpkgs` is the fourth problem from the thesis. It is a separate nixpkgs, older than yours.

`nix flake metadata` prints the same tree with dates. The script is useful because its output is plain columns you can sort, diff, or run in CI across many repositories.

Check 2: is each pin behind its branch?

To know whether a newer commit exists, resolve each input's `original` ref the way `nix flake update` would, and compare revisions. `nix flake metadata --json <flakeref>` does that resolution and changes nothing:

```
#!/usr/bin/env python3
"""lock-behind.py: for each direct input, compare the locked rev with what its
original ref (branch) resolves to today. Needs network; changes nothing."""
import json, subprocess, sys
from datetime import datetime, timezone

lock = json.load(open("flake.lock"))
nodes = lock["nodes"]
day = lambda t: datetime.fromtimestamp(t, timezone.utc).strftime("%Y-%m-%d")
```

```
for name, key in nodes["root"]["inputs"].items():
    if isinstance(key, list):
        continue                                          # 'follows': nothing locked here
    n = nodes[key]
    o, l = n["original"], n["locked"]
    if o["type"] != "github":
        continue
    url = f"github:{o['owner']}/{o['repo']}" + (f"/{o['ref']}" if "ref" in o else "")
    tip = json.loads(subprocess.run(["nix", "flake", "metadata", "--json", url],
                     capture_output=True, text=True, check=True).stdout)
    tip_rev, tip_t = tip["locked"]["rev"], tip["locked"]["lastModified"]
    if tip_rev == l["rev"]:
        verdict = f"at branch tip; branch last moved {day(tip_t)}"
    else:
        verdict = f"behind: tip {tip_rev[:8]} from {day(tip_t)}, {(tip_t - l['lastModified']
    print(f"{name:13} {url:45} locked {l['rev'][:8]} ({day(l['lastModified'])})  {verdict}")
```

To see the "behind" case, the lab rolled `flake-utils` back to an older commit with `nix flake lock --override-input flake-utils github:numtide/flake-utils/c1dfcf08…`. That command keeps `original` as the plain `github:numtide/flake-utils`, so the lock looks like one that simply has not been updated for two years:

```
$ python3 lock-behind.py
flake-utils   github:numtide/flake-utils                     locked c1dfcf08 (2024-09-17)  be
home-manager  github:nix-community/home-manager/release-25.11 locked 3ee51fbd (2026-05-23)
nixpkgs       github:NixOS/nixpkgs/nixos-25.11               locked b6018f87 (2026-06-30)  at
```

Updating only the input that is behind leaves every other pin alone. That keeps the change small and easy to review:

```
$ nix flake update flake-utils
warning: updating lock file "/tmp/fl/ovr/flake.lock":
• Updated input 'flake-utils':
    'github:numtide/flake-utils/c1dfcf08411b08f6b8615f7d8971a2bfa81d5e8a (2024-09-17)
  → 'github:numtide/flake-utils/b7a79d3fea1e214f6d668d521cecec0c01e26544 (2026-10-13)
```

One caveat: Nix caches branch resolution for `tarball-ttl` seconds (3600 by default). A check run straight after a lock can report "at tip" from the cache. Pass `--refresh` to `nix flake metadata` when the answer has to be current.

Check 3: is the branch still alive?

Look again at the `nixpkgs` line: **at branch tip; branch last moved 2026-06-30**. Nothing is behind, so the full update has nothing to do:

```
$ nix flake update
$ echo $?
0
```

It exited 0, printed no lock changes and changed nothing. (In the lab's uncommitted Git tree, Nix also printed its usual `Git tree ... is dirty` warning.) The date explains why. The NixOS 25.11 release announcement says the release gets fixes "for seven months (up until 2026-06-30)". The 26.05 announcement says 25.11 "will reach its end-of-life and stop receiving security updates after 2026-06-30". The lock is not out of date. It points at a branch that was retired.

The same is true of `home-manager/release-25.11`, which last moved on 2026-05-23.

A simple rule for CI: if an input's ref is a NixOS release branch (`nixos-YY.MM`, `nixpkgs-YY.MM`, `release-YY.MM`) and the branch tip is more than about two weeks old, fail with "branch may be end-of-life". Then check the release schedule. Two releases a year (May and November), each supported for seven months, leaves only about a month of overlap to move.

The fourth problem: a second nixpkgs

`home-manager` declares its own `nixpkgs` input. Without `follows`, the lock gives it a separate node, `687f05a9` from 2026-05-18. Anything home-manager builds from that copy keeps the older openssl, openssh and curl, however often you bump your top-level `nixpkgs`. `follows` makes it reuse yours.

The fix, and what it changes

Edit the refs, add `follows`, and lock again:

```
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    home-manager.url = "github:nix-community/home-manager/release-26.05";
    home-manager.inputs.nixpkgs.follows = "nixpkgs";
    flake-utils.url = "github:numtide/flake-utils";
  };
  outputs = { self, ... }: { };
}
```

```
$ nix flake lock
warning: updating lock file "/tmp/fl/host/flake.lock":
• Updated input 'home-manager':
    'github:nix-community/home-manager/3ee51fbdac8c8bdfe1e7e1fcaba6520a563f394f (2026-05-23)
  → 'github:nix-community/home-manager/24f2190587513b9c2f73db112a17844ae912d965 (2026-10-24)
• Updated input 'home-manager/nixpkgs':
    'github:NixOS/nixpkgs/687f05a9184cad4eaf905c48b63649e3a86f5433 (2026-05-18)
  → follows 'nixpkgs'
```

```
• Updated input 'nixpkgs':
    'github:NixOS/nixpkgs/b6018f87da91d19d0ab4cf979885689b469cdd41 (2026-06-30)
  → 'github:NixOS/nixpkgs/6f06d204d8f2d8761d9da78778745974a3216381 (2026-10-29)

$ python3 lock-age.py
input                       source                       rev       commit date   age
flake-utils                 numtide/flake-utils          b7a79d3f  2026-10-13   17d
flake-utils/systems         nix-systems/default          da67096a  2023-04-09 1300d
home-manager                nix-community/home-manager   24f21905  2026-10-24    6d
home-manager/nixpkgs        follows nixpkgs
nixpkgs                     NixOS/nixpkgs                6f06d204  2026-10-29    1d

$ python3 lock-behind.py
flake-utils   github:numtide/flake-utils                     locked b7a79d3f (2026-10-13)  at
home-manager  github:nix-community/home-manager/release-26.05 locked 24f21905 (2026-10-24)
nixpkgs       github:NixOS/nixpkgs/nixos-26.05               locked 6f06d204 (2026-10-29)  at
```

Does the new lock carry the fix?

An advisory names a fixed version. Evaluating one attribute on each locked revision answers "does this lock have it?" without building anything:

```
for r in 687f05a9184cad4eaf905c48b63649e3a86f5433 \
         b6018f87da91d19d0ab4cf979885689b469cdd41 \
         6f06d204d8f2d8761d9da78778745974a3216381; do
  printf '%s ' ${r:0:8}
  for p in openssl openssh glibc curl; do
    printf '%s=%s ' $p "$(nix eval --raw github:NixOS/nixpkgs/$r#legacyPackages.x86_64-linux
  done; echo
done
```

```
687f05a9 openssl=3.6.2 openssh=10.3p1 glibc=2.40 curl=8.19.0
b6018f87 openssl=3.6.2 openssh=10.3p1 glibc=2.40 curl=8.20.0
6f06d204 openssl=3.6.5 openssh=10.6p1 glibc=2.42 curl=8.23.0
```

The first row is home-manager's private nixpkgs, the second is the end-of-life 25.11 tip, and the third is 26.05 today. The private copy was a curl release behind even the dead branch.

A version string is a first check, not proof. nixpkgs often backports a fix as a patch without changing the version. For a specific CVE, check `pkgs.<name>.patches` or the package's `meta`, or search the nixpkgs pull requests for the CVE id against the target branch.

Moving from 25.11 to 26.05 is a **release upgrade**, not a routine bump: the glibc version changes, and module options can be renamed. Read the 26.05 release notes. Then build and diff the closure before you switch, as in `15-read-an-update-before-switching`.

Put it in CI

Both scripts only need `python3`, plus `nix` for the second one. They make no changes. A weekly job that runs them and fails on any of the following catches all four problems before a CVE wave does:

- any input reported `behind` by more than N days;
- any release-branch input whose tip has not moved in 14 days;
- more than one `NixOS/nixpkgs` source in `lock-age.py` output, unless it is intentional (for example `nixpkgs-unstable` for a few packages).

Treat an intentional second nixpkgs like any other input: it needs the same checks, and its own bump.

Pitfalls

- **Ages from `lastModified` are commit times, not lock times.** A lock written today against a dead branch still shows the branch's last commit date. That is exactly what makes check 1 useful.
- **Pinned revisions have no branch.** If `flake.nix` says `github:NixOS/nixpkgs/<sha>`, `original` holds the sha. Check 2 will always say "at tip", and only check 1 can flag it.
- **`--override-input` is not an update.** It changes the lock while `original` keeps the branch. That is handy for a lab or for rolling back, but a later `nix flake update` moves the input forward again.
- **`follows` can break an input** that needs a different nixpkgs than yours, for example a home-manager release that does not match your NixOS release. Keep the two on the same release: `nixos-26.05` with `release-26.05`.

Sources

- Nix manual, `nix flake`: the lock file format (`locked`, `original`, `inputs`, `follows`), https://nix.dev/manual/nix/stable/command-ref/new-cli/nix3-flake.html
- Nix manual, `nix flake update`, including updating a single input: https://nix.dev/manual/nix/stable/command-ref/new-cli/nix3-flake-update.html
- Nix manual, `nix flake metadata`: https://nix.dev/manual/nix/stable/command-ref/new-cli/nix3-flake-metadata.html
- Nix manual, the `tarball-ttl` setting: https://nix.dev/manual/nix/stable/command-ref/conf-file.html#conf-tarball-ttl
- NixOS 25.11 release announcement (supported until 2026-06-30): https://nixos.org/blog/announcements/2025/nixos-2511/
- NixOS 26.05 release announcement (25.11 end of life, 26.05 supported until 2026-12-31): https://nixos.org/blog/announcements/2026/nixos-2605/

Internals

Internals and currency

Internals and currency

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by a calendar. Read linearly when building depth; skip and search when you already operate a flake host and need a reference answer.

Prerequisites. You should already rebuild a flake-defined NixOS host, write or heavily consume modules, pin inputs with `flake.lock`, and have at least one real service path (SSH, proxy, or data) under declarative control. Internals *last* is deliberate: evaluator and daemon vocabulary is most useful after you have scars from rebuilds, not before.

What this part is for. After host, layout, secrets, packaging, and fleet chapters, the remaining skill is **currency under change**: explain why eval hangs, why a type error appears only on merge, who may write the store, how releases and upgrades actually work, and how to keep an architecture honest. Capstone proof (wipe/rebuild, rollback, tests) depends on this literacy even when Capstone code lives in the next part.

Learning outcomes

When you leave this part you should be able to:

1. Force evaluation intentionally, diagnose infinite recursion, and use `trace` / `tryEval` / `seq` / `deepSeq` without cargo-culting them.

2. Choose module types, merge with `mkIf` / `mkMerge` / priorities, and fail bad configs at **eval** time with assertions.

3. Explain multi-user trust, GC roots, sandbox isolation, and substituter signature policy on a live machine.

4. Measure rebuild time (eval vs substitute vs build), tune `max-jobs` / `cores` / builders, and reject "sandbox off for speed."

5. Describe input-addressed vs content-addressed derivations and keep production hosts on default store semantics until criteria say otherwise.

6. Harden `nixosTest` / flake `checks` so waits are unit/port based, not blind `sleep`.

7. Sample the adjacent ecosystem **once**, with a keep/drop decision that protects Capstone main.

8. Steelman Guix, image-based OS families, and Ansible against *your* workloads.

9. Read official release notes into an IMPACT checklist; plan a staged upgrade with rollback gates.

10. Inject and recover lab failures into a durable playbook; freeze architecture and gaps before Capstone thrash.

Chapter map

Chapter	Theme	You can leave when…
The evaluator	Thunks, WHNF, cycles, trace toolkit	You can reproduce and fix three recursion shapes and force one real flake leaf safely
One language, several evaluators	CppNix, Lix, Determinate Nix, Snix, shared tests	You can name the evaluator you pin, and what a conformance `.exp` does and does not lock
Module system in depth	Types, merge, priorities, assertions	One production module is typed; bad configs fail at eval
Store and daemon	Multi-user trust, roots, sandbox, signatures	You map live roots and annotate a sandbox build log
What evaluation sends to the daemon	Eval-time store writes, `--read-only`, eval store	You measured cold vs read-only traffic for one real attribute
How Nix computes a store path	Fingerprints, NAR hashing, ATerm, `hashDerivationModulo`	Your Python reimplementation matches `nix eval` / `nix store add` paths and `.drv` bytes
Nix performance	Jobs/cores, eval cost, cache, builders	Before/after timings exist with justified settings
CA derivations (awareness)	IA vs CA, experimental flags	Status memo or lab experiment; Capstone stays default store
nixosTest mastery	Driver API, flakes, multi-node, CI	Suite inventory; sleep-only waits removed from critical path
Ecosystem map	devenv / Snowfall / nix-darwin (pick one)	`COMPARE.md` keep/drop without rewriting Capstone main
Comparative landscape	NixOS vs Guix vs images vs Ansible	Honest scorecard + essay with "when *not* NixOS"

Chapter	Theme	You can leave when…
Reading release notes	26.05 method, IMPACT tags, stateVersion	Annotated checklist against *your* flake
Upgrade planning	26.05→next runbook, gates, rollback	Dry-run upgrade document with abort criteria
Troubleshooting drills	Inject/recover on disposable VM	2 playbook entries with diagnose/fix/prevent
Architecture review	Map repo → Capstone A gaps	`ARCHITECTURE.md` + prioritized `GAPS.md`

How this part connects

```
Language / modules (earlier parts)

  Evaluator + types     ← debug eval failures at the source

  Store / daemon / perf ← trust, isolation, time budget

  CA literacy + tests   ← experimental awareness; proof harness

  Ecosystem + compare    ← adopt tools and OS choices deliberately

  Release + upgrade     ← currency under nixpkgs cadence

  Troubleshoot + arch   ← playbooks + freeze before Capstone

  Capstone part         ← implement, prove, recover
```

Earlier part	What you reuse here
Concepts	Store paths, derivations, modern CLI
NixOS host	Rebuild modes, custom modules as consumer
Home & layout	Flake layout, locks, checks

Earlier part	What you reuse here
Services & security	Secrets, firewall, proxy consumers
Packaging	FODs, caches, remote builders vocabulary
Ops & fleet	nixosTest intro, CI, GC, backups, deploy tools

Suggested study modes

Mode	Approach
Linear depth	Chapters in table order; one lab workspace per chapter under `~/lab/nixos/internals/<topic>/`
Reference	Jump via symptoms: infinite recursion → evaluator; silent option loss → module priorities; GC surprise → store/daemon
Pre-Capstone sprint	Architecture review first if your repo is chaotic; then evaluator + modules + tests; upgrade notes last if pin is already stable
Ops only	Store/daemon, performance, troubleshooting, release notes, upgrade planning

> **Tip**
>
> Prefer disposable VMs and `nixos-rebuild build` / `nix build …toplevel` over thrashing a daily driver. Console access before firewall experiments is non-negotiable.

Lab workspace convention

Use topic paths (not calendar folders):

```
mkdir -p ~/lab/nixos/internals/{evaluator,modules,store-daemon,perf,ca,tests,ecosystem,compa
```

Artifacts worth keeping in git (your flake or a notes repo):

Artifact	From
`NOTES.md` / personal gotchas	Every chapter
`TYPES.md`, `PRIORITIES.md`	Modules
`ROOTS.md`, `TRUST.md`	Store/daemon

Artifact	From
`PERF.md`, `BUILDERS.md`	Performance
`CA-MEMO.md` or experiment logs	CA awareness
`SUITE.md`	nixosTest mastery
`COMPARE.md`	Ecosystem
`SCORECARD.md`, `ESSAY.md`	Comparative landscape
`IMPACT.md`	Release notes
`UPGRADE-*.md`	Upgrade planning
`playbook/PL-*.md`	Troubleshooting
`ARCHITECTURE.md`, `GAPS.md`, `CAPSTONE-A.md`	Architecture review

Acceptance bar for the part

You do **not** need every optional experiment green. You **do** need:

- ☐ One real infinite-recursion fix documented
- ☐ One module redesigned with real types + 2 assertions
- ☐ Live GC-root classification and sandbox left **on**
- ☐ Before/after rebuild timing with justified `max-jobs`/`cores`
- ☐ Explicit decision: Capstone on default (input-addressed) store
- ☐ At least one flake check/`nixosTest` without blind `sleep` as the only wait
- ☐ Keep/drop decision for one ecosystem trial **without** rewriting Capstone main
- ☐ Comparative essay that includes when you would *not* choose NixOS
- ☐ `IMPACT.md` for 26.05 (or your pin) with ACTION/VERIFY items
- ☐ Upgrade runbook skeleton with rollback commands
- ☐ 2 playbook entries from real inject/recover drills
- ☐ Architecture + gap list ready to drive Capstone

Baseline and policy

Item	Book default
Nix	**2.35.x** class (2.35.3 at time of writing) — verify with `nix --version`
nixpkgs / NixOS	**26.05** pin in Capstone and lab hosts
Store semantics	Input-addressed default; CA is literacy only
Secrets	Paths only in modules; sops-nix primary
Experimental features	Enable only with revert plan on disposable builders

> **!** Important
>
> **No day-calendar framing** in this book part. Chapter order is pedagogical, not a schedule. Study pacing is yours.

Common traps in this part

Trap	Better move
Enabling CA on the only builder "for speed"	Memo + criteria; keep Capstone default
`deepSeq` everywhere to "fix" laziness	Redesign option dependencies
`mkForce` on every host option	Fix module defaults and boundaries
Tool tourism (devenv + Snowfall + darwin in one weekend)	Try **one**; score; decide
Upgrade Friday night with six input bumps	Branch, nixpkgs first, gates, lab first
Architecture doc never opened again	Link from `CAPSTONE-A.md`; gaps with acceptance tests

---

Next

→ **The evaluator** — thunks, force, and recursion diagnostics.

When architecture and gaps are frozen, continue to **Capstone and recovery**.

The evaluator

The evaluator

Goal: Explain thunks, force evaluation intentionally, reproduce and fix infinite recursion, and use `trace` / `tryEval` / `seq` / `deepSeq` without cargo-culting them.

> **Note**
>
> This chapter is **language/evaluator** depth. Prefer a disposable `nix repl` and tiny pure expressions before touching your host flake. A bad recursive module option can hang eval of the whole system.

Why this chapter exists

By the time you reach internals you already *use* laziness every rebuild: modules only force what activation needs. When eval explodes—`infinite recursion encountered`, mysterious `null`, or a multi-minute wait before any build starts—the bug is often a **thunk cycle** or accidental full-tree force, not a broken package.

Internals last (syllabus rule) means you can now map symptoms → mechanisms and fix the design rather than spraying `deepSeq` everywhere.

Theory 1 — Evaluation is a graph of thunks

Values vs thunks

Nix does not compute every subexpression when you write it. An unevaluated expression is a **thunk**: a promise to compute later, holding the expression body plus an environment of free variables.

Concept	Meaning
Thunk	Deferred computation; may be forced 0 or 1 times (then memoized)
WHNF	Weak head normal form: outer constructor known (`{…}`, `[…]`, lambda, int, …) without forcing nested fields
NF	Full normal form: nested values forced recursively

Concept	Meaning
Memoization	After force, the thunk is replaced by the result (shared)

```nix
let
  x = builtins.trace "forcing x" (1 + 1);
  y = { a = x; b = x; };
in
  y.a
```

You should see `"forcing x"` **once**. Both `a` and `b` share the same thunk identity until forced.

Why modules love laziness

```nix
# Sketch of the idea - not the real module system
config = {
  services.foo.enable = true;
  # huge attrset of options you never touch...
};
# Only options that are *demanded* by evaluation of the final system get forced.
```

If every option were strict, evaluating a NixOS config would always pay the full cost of every default expression, including unused services.

Eager edges you already know

Construct	Eager?	Notes
`import ./file.nix`	Path resolve + parse/eval of the file	File body still may return lazy values
Function application `f x`	Forces `f` to a function; forces args per function body	Body of `f` decides strictness
`&&` / `||`	Short-circuit	Second arm may never force
`if c then a else b`	Forces `c`	Only one branch forces
`builtins.seq a b`	Forces `a` to WHNF, returns `b`	Classic "strictness injection"
`builtins.deepSeq a b`	Forces `a` fully, returns `b`	Expensive; tests and asserts
`builtins.trace msg v`	Forces nothing about `v` except returns it	Side effect for debugging
`builtins.map f list`	Forces list spine as consumed	Element force depends on `f`

Construct	Eager?	Notes
String interpolation `"${x}"`	Forces `x` to a stringable value	Common accidental force site

---

Theory 2 — Infinite recursion patterns

Pattern A — Direct self-reference

```
let x = x + 1; in x
# error: infinite recursion encountered
```

Pattern B — Mutual recursion without a base case

```
let
  a = b + 1;
  b = a + 1;
in a
```

Pattern C — Attrset self-reference (legal *shape*, dangerous force)

```
rec {
  a = 1;
  b = a + 1;      # OK
  # c = c;        # infinite if forced
}
```

`rec` is not evil; **forcing a cycle** is.

Pattern D — Module / option cycles (production shape)

Classic failure:

```
# module A
config.services.foo.settings.bar = config.services.baz.settings.qux;

# module B
config.services.baz.settings.qux = config.services.foo.settings.bar;
```

Or subtler:

```nix
options.my.paths.dataDir = lib.mkOption {
  type = lib.types.path;
  default = config.my.paths.root + "/data";  # forces config while defining defaults
};
```

Rule of thumb: defaults and `apply` functions should depend on **options** (or pure inputs), not on deep `config` that re-enters the same option.

Pattern E — `with` + shadowing confusion

```nix
let pkgs = { hello = "ok"; }; in
with pkgs;
let hello = hello; in  # which hello?
hello
```

Prefer explicit attr access in teaching examples; `with` is a readability tax on recursion debugging.

Pattern F — `lib.fix` / fixed points gone wrong

```nix
# conceptual - when extends and self-reference collide badly
lib.fix (self: {
  a = self.b;
  b = self.a;
})
```

Fixed points are powerful for overlays and module-ish composition; cycles still blow up when forced. Prefer small pure functions until you need `fix`.

Theory 3 — Debugging toolkit

`builtins.trace`

```nix
builtins.trace "here" value
```

Prints to stderr when the `trace` expression itself is forced (i.e. when something needs `value` through the trace).

`builtins.tryEval`

```
builtins.tryEval (throw "boom")
# → { success = false; value = false; }  # value is dummy on failure

builtins.tryEval 42
# → { success = true; value = 42; }
```

Use for soft probing—not for swallowing production errors silently. Infinite recursion may **not** be caught the way you hope; catalog behavior on your Nix version.

`seq` vs `deepSeq`

```
builtins.seq { a = throw "late"; } "outer-ok"
# → "outer-ok"   # only WHNF of the attrset

builtins.deepSeq { a = throw "late"; } "never"
# → error: late
```

`lib.debug.traceVal` / `traceSeq` / `traceValSeq` (nixpkgs)

When you have `lib` from nixpkgs:

```
lib.debug.traceVal someAttr
lib.debug.traceValSeqN 2 nested
```

Prefer library helpers over ad-hoc nested `trace` once you are in a flake with `pkgs`/`lib`.

`NIX_SHOW_STATS` and evaluator metrics

```
NIX_SHOW_STATS=1 nix eval --impure --expr '1+1'
```

Look for counts of thunks, allocations, and GC—useful when comparing "eval is slow" hypotheses, not as a daily dashboard.

`nix-instantiate --eval` / `nix eval`

```
nix eval --expr 'let x = x; in x'          # expect infinite recursion
nix eval --json .#nixosConfigurations.lab.config.networking.hostName
```

For module cycles, force **one** leaf option rather than the entire `config`.

Bisect strategy for real flakes

```
1. Force smallest leaf (hostname, one service enable)
2. Binary-search modules list (comment half of imports)
3. Within suspect module, force each option default path
4. Draw DAG of option dependencies on paper
5. Only then add temporary trace
```

Remove all temporary traces before merge.

Theory 4 — When to force (and when not to)

Goal	Tool	Risk
Ensure side-effect config is checked early	`assertions` / `warnings` in modules	Good default
Fail fast if a secret path missing	assert in module, or activation script	Prefer module-time when pure
Make a function strict in an arg	`seq` at the start of the body	Can defeat sharing
Serialize entire config for tests	`deepSeq` or `lib.generators.toJSON`	Huge cost if done casually
"Fix" mysterious laziness bugs with deepSeq everywhere	**Don't**	Hides design issues; tanks eval

> ⚠ Warning
>
> `deepSeq` is a diagnostic scalpel and a test hammer—not a seasoning for every module.

Theory 5 — Eval vs build vs activation (where time goes)

Phase	What runs	Failure shape
Eval	Nix language → derivation graph / config attrset	Type errors, infinite recursion, missing options
Build / substitute	Realize store paths	Compile errors, hash mismatch, sandbox
Activation	Switch system generation	systemd failures, scripts, secret decrypt

Symptoms that feel like "Nix is broken" are often **eval**. Timing with `time nix eval …` vs `time nix build …` separates the budget before you tune `max-jobs` (see Nix performance).

Worked example — Construct, diagnose, fix

Broken

```
# broken-cycle.nix
rec {
  root = "/var/lib/app";
  data = "${data}/files";   # oops: meant root
}
```

```
nix eval -f broken-cycle.nix data
# infinite recursion encountered
```

Fixed

```
rec {
  root = "/var/lib/app";
  data = "${root}/files";
}
```

Module-shaped fix

Prefer:

```
{ lib, config, ... }:
{
  options.my.app.root = lib.mkOption {
    type = lib.types.path;
    default = "/var/lib/app";
  };
  options.my.app.dataDir = lib.mkOption {
    type = lib.types.path;
    default = "${config.my.app.root}/files";  # depends on sibling option via config - OK if
  };
}
```

If `root`'s default ever depended on `dataDir`, you create a cycle. Draw a DAG of option dependencies when stuck.

Force a leaf, not the world

```
# Good: small demand
nix eval .#nixosConfigurations.lab.config.networking.hostName

# Expensive: full system derivation
time nix eval .#nixosConfigurations.lab.config.system.build.toplevel --raw >/dev/null
```

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/evaluator
cd ~/lab/nixos/internals/evaluator
```

Use pure files here before editing Capstone modules.

Lab 1 — Watch memoization

Create `thunks.nix`:

```
let
  expensive = builtins.trace "EXPENSIVE" (builtins.foldl' (a: b: a + b) 0 (builtins.genList
  bag = { x = expensive; y = expensive; };
in
{
  first = bag.x;
  second = bag.y;
}
```

```
nix eval -f thunks.nix first
nix eval -f thunks.nix second
nix eval -f thunks.nix
```

Record how many times `EXPENSIVE` prints in each case. Explain sharing in one paragraph in `NOTES.md`.

Acceptance: You can predict print counts before running, then verify.

Lab 2 — Infinite recursion zoo

Create `cycles.nix` with **three** named attributes that each fail differently when forced:

1. Direct `let` cycle
2. Mutual `a`/`b` cycle
3. `rec` set cycle

Force each with `nix eval -f cycles.nix <name>` and paste the error fingerprints into `NOTES.md`.

Then fix all three; leave the broken versions commented for teaching.

Acceptance: Three broken + three fixed forms committed; each error reproduced once.

Lab 3 — `seq` vs `deepSeq`

```
# force.nix
let
  t = {
    a = builtins.trace "a" 1;
    b = builtins.trace "b" (throw "b-fail");
  };
in {
  onlyWhnf = builtins.seq t "ok-whnf";
  full = builtins.deepSeq (builtins.removeAttrs t ["b"]) "ok-deep";
  # fullBoom = builtins.deepSeq t "never";
}
```

Evaluate `onlyWhnf` and `full`. Uncomment `fullBoom` once and capture the error. Write when WHNF is enough for module work.

Lab 4 — Trace a tricky real eval (your flake)

Pick **one** leaf from your lab host:

```
cd /path/to/your/flake
nix eval .#nixosConfigurations.<host>.config.networking.hostName
# or a service option you care about
```

```
time nix eval .#nixosConfigurations.<host>.config.system.build.toplevel --raw >/dev/null
```

If eval is slow, binary-search:

1. Comment out recently added modules (or use a slimmed `modules = [ ... ]` copy).
2. Force smaller slices with `nix eval …config.services.<name>.enable`.
3. Add a temporary `lib.trace` only around the suspect option default.

Document the path of the investigation—even if the conclusion is "eval cost is nixpkgs baseline."

Acceptance: Written investigation path with wall-clock numbers for leaf vs toplevel.

Lab 5 — Intentional soft failure with `tryEval`

```
# try.nix
{
  ok = builtins.tryEval (1 + 1);
  bad = builtins.tryEval (1 + "x");
}
```

```
nix eval -f try.nix --json
```

Note: type errors vs `throw` vs infinite recursion behave differently under `tryEval`—catalog what you observe on **your** Nix version.

Lab 6 — Module-shaped cycle (optional, high value)

Create a tiny `lib.evalModules` demo with two options whose defaults reference each other. Force eval; capture the error. Redesign so defaults form a DAG (one root option, one derived).

```
# module-cycle-demo.nix - sketch; expand with lib.evalModules
# Show broken defaults, then fixed defaults from a pure root path.
```

Acceptance: One paragraph on why "defaults depending on config" is a design smell when it re-enters the same option.

Lab 7 — Stats snapshot

```
NIX_SHOW_STATS=1 nix eval --impure --expr 'builtins.length (builtins.attrNames (import <nixp
  2>&1 | tee stats-attrnames.txt
```

Compare (roughly) against evaluating a tiny expression. Do **not** over-interpret; note that large attrset demand is expensive.

Failure modes (quick reference)

Symptom	Likely class	First move
`infinite recursion encountered`	Cycle	Force smaller attr; draw option DAG
Eval hangs, no error	Cycle or huge force	Interrupt; bisect modules
Trace never prints	Not forced	Wrap the *used* path
Trace floods	Mapped over huge list	Narrow value; use `traceValSeqN`
`tryEval` "lies"	Uncatchable error class	Catalog; don't rely for cycles
Multi-minute silence then build	Eval cost	`time nix eval` leaf vs toplevel

---

Common gotchas

Symptom	Theory link
`infinite recursion encountered` with no file	Cycle in options/`rec`; force a smaller attr
Trace never prints	Expression never forced; wrap the *used* value
Trace prints thousands of times	You forced a list map; narrow the traced value
Eval hangs, no error	Likely cycle; interrupt and bisect modules
`deepSeq` "fixed" a bug	You hid a lazy design smell; redesign dependencies
`with pkgs;` + name collision	Shadowing; drop `with` while debugging
Module default uses `config` of same option	Define default from other options or pure constants
Temporary traces left in production modules	Grep before merge; CI can ban `builtins.trace` in `modules/`

Checkpoint

- ☐ Explain thunk / WHNF / memoization in your own words
- ☐ Three infinite-recursion examples constructed and fixed
- ☐ `seq` vs `deepSeq` demo committed
- ☐ Trace notes on one real flake option
- ☐ `NOTES.md` with three personal gotchas
- ☐ No permanent `deepSeq` left in production modules
- ☐ Lab workspace under `~/lab/nixos/internals/evaluator`

Commit

```
git add .
git commit -m "lab(nixos): evaluator thunks, cycles, and trace discipline"
```

Write three personal gotchas before continuing.

Next

→ **Module system in depth** — types, merge, priorities, and assertions that fail at eval.

One language, several evaluators

One language, several evaluators

Goal: Name the Nix implementations you can actually pin in 2026, see where they diverge (builtins, string context, float printing, error text, flakes), and test a portable expression against more than one of them with a shared conformance suite.

> **!** Important
>
> This writing box has **no `nix` binary** and this chapter installs nothing. Every expected value below is copied from a file that was read: upstream `tests/functional/lang/` at Nix tags **2.34.8** and **2.35.3**, or `lang/` / `flake/` in numtide/nix-conformance. The command line is the one those repositories' runners use. Re-run the files on a host before you treat a diff as yours.

Version note, re-checked **2026-10-17**:

Implementation	What it is	Version you can cite today
CppNix (upstream Nix, `NixOS/nix`)	The C++ evaluator the language still follows	Tag **2.35.3**, annotated 2026-10-13. The file `.version` at that tag is the string `2.35.3`.
Lix	Community fork of Nix (C++ with a growing Rust portion)	**2.96.0 "Bingsu"**, announced 2026-10-13. The same day's security fixes are also in **2.94.3** and **2.95.4**.
Determinate Nix	A distribution of upstream Nix with extra product surface	**3.23.3** (2026-10-16), release notes: "Based on upstream Nix 2.35.3."
Snix (formerly Tvix)	Rust reimplementation, library-first	No stable release tag on snix.dev. The project says it is early and not a drop-in `nix` binary.

Two package-set aliases are easy to misread. On nixpkgs branch `nixos-26.05`, `pkgs/tools/package-management/n` sets `nixVersions.nix_2_35` to **2.35.3** and `nixVersions.latest` to that package, while `nixVersions.stable` is still `nix_2_34` at **2.34.9**. The same branch's `lix/default.nix` sets `lixPackageSets.lix_2_94` to **2.94.3**, `lix_2_95` to **2.95.4**, `stable` to 2.94, and `latest` to 2.95. On nixpkgs `master` (read the same day) `lixPackageSets.lix_2_96` is **2.96.0**, and `latest` / `stable` still alias `lix_2_95` at **2.95.4**. Pin the numbered attribute.

Partial cousins, so they stay out of the table: hnix is a Haskell parser and evaluator (repository pushed 2026-03-17) whose own README still shells out to `nix-shell` and `nix-store`. nix-community/go-nix is Go libraries for NAR and store paths (pushed 2026-03-02), not an evaluator. A different language that happens to be declarative is a different chapter.

Thesis

> There is one Nix language in the files you commit, and several programs that claim to evaluate them. A shared suite matters because the language is still "what one C++ binary prints," and that binary is no longer the only one your CI might install.

Mental model

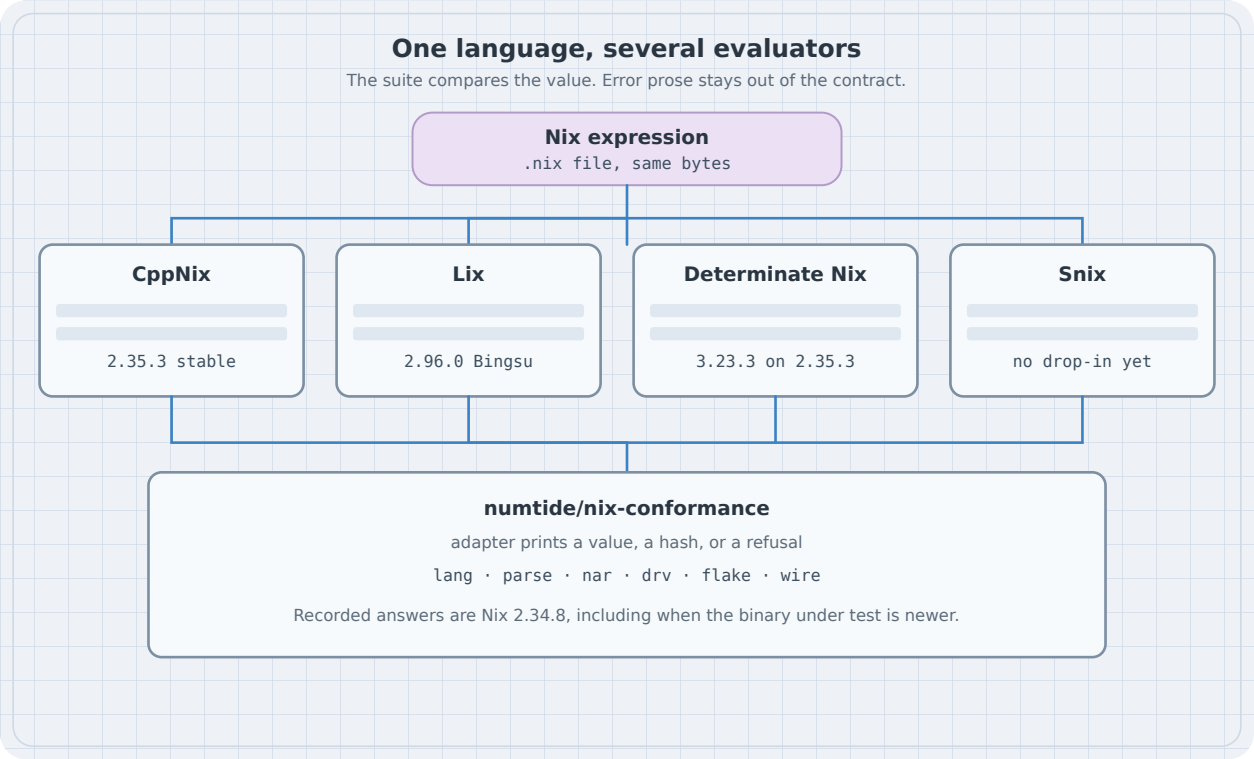


Figure 1: One Nix expression checked by several evaluators against one conformance suite

Snix's own eval notes say the quiet part: in the absence of a language standard, what the language *is* is prescribed by the behaviour of the C++ implementation. CppNix, Lix, and Determinate Nix are three builds of that lineage (Lix forked; Determinate tracks upstream and adds features). Snix is a bytecode interpreter written to agree with nixpkgs, and it documents the C++ corners it does not want to copy.

The practical standard, as of October 2026, is a test suite rather than an RFC. numtide/nix-conformance (created 2026-10-08, last push 2026-10-12, LGPL-2.1-or-later) records answers from **Nix 2.34.8**. Its README and `flake.nix` description both say so. Current stable CppNix is **2.35.3**. The suite is the contract for "same as 2.34.8," and a newer binary is something you run *through* the suite to see drift. It is not, by itself, a proof that 2.35.3 and 2.34.8 agree.

An implementation joins by shipping one adapter program. The runner calls `ADAPTER OP FILE` with `NIX_PATH=lang/dir3:lang/dir4`, `NIX_CONFIG="experimental-features = nix-command flakes"`, `HOME=/fake-home`, and `TEST_VAR=foo`. Exit code 3 means "this implementation does

not do that op," and the case is skipped. The reference adapter (`adapters/nix`) is bash: `parse`, `nar`, `drv`, and `wire` exec oracles linked against that Nix, and a missing oracle is a skip; `eval` is `nix-instantiate --eval --strict --read-write-mode` inside a throwaway `local?root=` store. The dummy store is refused for evaluation on purpose (`nix-bugs/dummy-store` in the suite): a drvPath's string context would not carry the real closure.

Op	Pass condition, from `SPEC.md`
`eval` on `eval-okay-*`	Strict printout equals the `.exp` file
`eval` on `eval-fail-*`	Non-zero exit. The `.err.exp` text is Nix's and is **not** compared
`parse`	`ok TREE` or `error LINE:COL`. The message is not part of the answer
`flake`	JSON of output `result`, or failure for `fail-*`. The message is not compared
`nar`, `drv`, `wire`	A hash line, a store path, or `error` — facts, not sentences

Upstream's own runner is stricter about prose. `tests/functional/lang.sh` at tag 2.35.3 diffs stderr of every `eval-fail-*.nix` against `lang/<name>.err.exp`. That file is a characterisation of **CppNix's** wording. Copying it into a cross-implementation CI tests the sentence, which Lix and Snix are free to change and which the shared suite deliberately ignores (`COVERAGE.md`: "A case tests what is specified, not how Nix words it").

Counts from a full git tree listing on 2026-10-17 (`truncated: false`):

Tree	`eval-okay-*.nix`	`eval-fail-*.nix`	all `*.nix` under the lang tests
`NixOS/nix` tag 2.34.8, `tests/functional/lang`	143	109	322
`NixOS/nix` tag 2.35.3, same path	144	269	483
`numtide/nix-conformance` `lang/`	532	274	893

The suite is larger than the 2.34.8 tree because it adds cases (string context, derivation output paths, regex and `fromTOML` corpora, float-range). Tag 2.35.3 grew a large pile of `eval-fail` files that a suite pinned to 2.34.8 does not automatically absorb. `lib.adapterFor` exists so a newer Nix can be run as one more implementation before those answers are promoted.

Worked examples

Case 1 — Addition, and the float that prints like an int

Upstream file `tests/functional/lang/eval-okay-builtins-add.nix`. The blob is identical at tags 2.34.8 and 2.35.3 (git blob `f678f640f129d966dc308fece3fe622882dcc667`) and the suite carries the same `.exp` (blob `0350b518a7ec2e41eb48e82bfd99b34685a10d95`).

```
# eval-okay-builtins-add.nix
# Source: NixOS/nix tests/functional/lang/eval-okay-builtins-add.nix
# tags 2.34.8 and 2.35.3, and numtide/nix-conformance lang/
# Not run here (no nix). Upstream command, from tests/functional/lang.sh:
#   nix-instantiate --eval --strict lang/eval-okay-builtins-add.nix
[
  (builtins.add 2 3)
  (builtins.add 2 2)
  (builtins.typeOf (builtins.add 2 2))
  ("t" + "t")
  (builtins.typeOf (builtins.add 2.0 2))
  (builtins.add 2.0 2)
]
```

Expected stdout, from `eval-okay-builtins-add.exp` (same blob at both tags and in the suite):

```
[ 5 4 "int" "tt" "float" 4 ]
```

`builtins.add 2.0 2` has type `"float"` and prints as `4`. That is the printer, not a cast back to int. In tag 2.35.3, `using NixFloat = double` (`src/libexpr/include/nix/expr/value.hh`) and `printFloat` does `output << v.fpoint()` with no precision set (`src/libexpr/print.cc`). Default iostream formatting of a double is what `.exp` locks.

The same printer shows up in assertion text, which the shared suite does **not** lock. `assertEqValues` formats a mismatch as `"%s with value '%s'"` through `ValuePrinter` (`src/libexpr/eval.cc`). `showType` of a float is `"a float"`. So CppNix's own `eval-fail-assert-equal-floats.err.exp` at tag 2.35.3 ends with:

```
error: a float with value '1' is not equal to a float with value '1.01'
```

for the expression `assert { b = 1.0; } == { b = 1.01; };`. The left-hand `1.0` is rendered as `1`. A second evaluator that prints `1.0` or `1.000000` still implements `==` if the assertion fails. It fails the upstream characterisation diff, and it passes `nix-conformance`.

Case 2 — Ordinary float arithmetic, then the extreme range

`tests/functional/lang/eval-okay-float.nix`, same `.exp` blob at 2.34.8 and 2.35.3 (`3c50a8adce861f615611328de1c01fdd6e6680ae`).

```
# eval-okay-float.nix
# Source: NixOS/nix tests/functional/lang/eval-okay-float.nix (tags 2.34.8 and 2.35.3)
# Not run here. Command:
#   nix-instantiate --eval --strict lang/eval-okay-float.nix
[
  (1.1 + 2.3)
  (builtins.add (0.5 + 0.5) (2.0 + 0.5))
  ((0.5 + 0.5) * (2.0 + 0.5))
  ((1.5 + 1.5) / (0.5 * 4.0))
]
```

Expected stdout, from `eval-okay-float.exp`:

```
[ 3.4 3.5 2.5 1.5 ]
```

The suite adds a case upstream 2.35.3 does not have (`lang/eval-okay-float-range.nix` is absent from that tag). The `.exp` is Nix 2.34.8's printout, per the suite README:

```
# eval-okay-float-range.nix
# Source: numtide/nix-conformance lang/eval-okay-float-range.nix
# Not in NixOS/nix tests/functional/lang at tag 2.35.3.
# Not run here. The suite's eval op is:
#   nix-instantiate --eval --strict --read-write-mode lang/eval-okay-float-range.nix
[ 2.2250738585072014e-308 0.0 0.0e-400 0.0e5 1.7976931348623157e308 1.5 ]
```

Expected stdout, from `lang/eval-okay-float-range.exp`:

```
[ 2.22507e-308 0 0 0 1.79769e+308 1.5 ]
```

The recorded line keeps six digits (`2.22507e-308`, `1.79769e+308`), prints `0.0` as `0`, and underflows `0.0e-400` to `0`. `printFloat` sets no precision, so this spelling is the stream default, not a format string in Nix. Rust's default `Display` for `f64`, Python `repr`, and a `printf("%.16g")` dump of the same IEEE bits do not match this line. Parse trees in the suite are stricter than this printer: `SPEC.md` records a float literal as IEEE-754 bits, so `1.5` is `(float 3ff8000000000000)`. Evaluation output and parse output are different contracts. A portable expression does not put the rendered form of a float into a derivation name, a lock, or a golden file unless that file was produced by the evaluator you pin.

Case 3 — `eval-fail` means "refuse", and `|>` is still experimental

```
# eval-fail-abort.nix
# Source: NixOS/nix tests/functional/lang/eval-fail-abort.nix
# same file at tags 2.34.8 and 2.35.3. Not run here.
# Upstream: nix-instantiate --eval --strict --show-trace lang/eval-fail-abort.nix
# must exit non-zero. Conformance compares the exit status only.
if true then abort "this should fail" else 1
```

CppNix's characterised stderr (`eval-fail-abort.err.exp`, same blob at both tags) begins `error:` and contains `evaluation aborted with the following error message: 'this should fail'`. Quote it when you are debugging CppNix. Do not paste it into a test that Lix or Snix must satisfy.

Pipe operators are the syntax trap. Tag 2.35.3 still ships this as a failing test, and `src/libutil/experimental-features.cc` describes the feature `pipe-operators` as "Add `|>` and `<|` operators to the Nix language."

```
# eval-fail-pipe-operators.nix
# Source: NixOS/nix tests/functional/lang/eval-fail-pipe-operators.nix at tag 2.35.3
# Not run here.
#   nix-instantiate --eval --strict --show-trace lang/eval-fail-pipe-operators.nix
1 |> 2
```

The characterised CppNix message at that tag is:

```
error: experimental Nix feature 'pipe-operators' is disabled; add '--extra-experimental-feat
```

Lix's stable manual still marks `|>` and `<|` experimental, and the flag it documents is `--extra-experimental-features pipe-operator` (the feature id on that page is `pipe-operator`). CppNix 2.35.3's table and its error text use the plural id `pipe-operators`. The conformance runner's `NIX_CONFIG` turns on `nix-command` and `flakes` only, so `1 |> 2` stays an `eval-fail` for the reference. The portable spelling is function application, or `lib.pipe` from nixpkgs, which is a function and needs no language flag.

Case 4 — String context is part of the value

A Nix string can carry a context: store paths that must exist when the string is interpolated into a derivation. Drop the context and the build sandbox, or a later garbage collection, sees a missing input. The suite's `lang/eval-okay-context-introspection.nix` builds a two-output derivation, interpolates its `drvPath` and `outPath` together with a path, and checks the eta rule `getContext` / `appendContext` / `unsafeDiscardStringContext`. Expected stdout, from `lang/eval-okay-context-introspection.exp`:

```
[ true true true true true true true true true true true true ]
```

`SPEC.md` says those `eval-okay-context-*` and `eval-okay-drv-*` cases use system `x86_64-linux` and a real local store, so the paths in the answer are stable across hosts. An implementation with no store can still be skipped (exit 3). It cannot honestly print `true` for this file.

Snix's builtins table (page "Builtins", `article:modified_time` 2025-12-24) marks `derivation` / `derivationStrict` / the fetchers / `toFile` / `storePath` as blocked on the store, and `placeholder` as blocked on context. `getContext` and `hasContext` are listed without a blocker on that snapshot. The companion note "Build references in derivations" describes C++ string context as the mechanism Snix still has to line up, and "Nix Language Issues" says Snix targets nixpkgs compatibility and can ignore edges nixpkgs does not hit. Treat that table as the published snapshot from December 2025, not as a census of `git.snix.dev` on the day you read this. The way you find out today's gap is `--known`, below.

`builtins.readFile` feeding arbitrary file bytes into a Nix string is a separate, already-sharp edge of the same string type. Keep large and binary files on the path side of that choice.

Case 5 — A flake the suite can check without the network

Flakes are on in the runner's `NIX_CONFIG`, and they are still an experimental feature of CppNix. The suite does not fetch. `flake/okay-follows-root/flake.nix`:

```
# flake/okay-follows-root/flake.nix
# Source: numtide/nix-conformance flake/okay-follows-root/flake.nix
# Not run here. Adapter command (adapters/nix):
#   nix eval --json --no-update-lock-file path:<dir>#result
{
  inputs.B.url = "path:./B";
  inputs.B.inputs.top.follows = "";
  outputs = { B, ... }: { name = "root"; result = [ B.inputs.top.name ]; };
}
```

Expected stdout, from `flake/okay-follows-root.exp`:

```
["root"]
```

`fail-*` flake directories must fail. The message is not compared. A lock the reference would have to rewrite is a failure: the implementation must not lock again and must not fetch. Determinate Nix adds flake schemas and `builtins.wasm` on top of this surface; those are product extensions. A flake that must evaluate on upstream Nix 2.35.3 stays off both. Snix's December 2025 builtins table still lists `fetchTree` as `todo`, which is the builtin flakes evaluation sits on. Snix is not a second flake client you can drop into CI yet.

How to write a portable expression

Stay inside the intersection the suite can already see:

1. **Integer arithmetic, strings, lists, attrsets, `let`, `if`, `assert`, ordinary calls.** Case 1's first three elements are the shape. Their printout `[ 5 4 "int" … ]` is stable from 2.34.8 through 2.35.3.
2. **No experimental syntax.** `|>` and `<|` need `pipe-operators` on CppNix 2.35.3 and `pipe-operator` on current Lix. Write `g (f x)` or `lib.pipe`.
3. **No implementation-only builtins.** `builtins.wasm` is Determinate Nix. A module shared with stock Nix and with Lix does not call it.
4. **Do not branch on `builtins.nixVersion` to paper over a real difference.** Versions differ on purpose across this table. A branch that is load-bearing belongs in the conformance `--known` list, with a deletion condition.
5. **Floats are `double`, and their spelling is not portable.** Compare them inside the expression if you must (`builtins.abs` is not the tool; subtract and bound), or keep the golden file next to one pinned evaluator. Do not splice a printed float into a store name.

6. **Context-bearing strings and derivations need a store and a system.** The suite fixes `x86_64-linux` for those cases. A pure function of integers does not.
7. **Assert outcomes, not paragraphs.** `abort` and `throw` must fire. The trace layout is CppNix's.

```
# portable-sum.nix
# The integer half of eval-okay-builtins-add.nix, inlined so it has no import.
# Not run here. Same command as that upstream file:
#   nix-instantiate --eval --strict portable-sum.nix
# Expected pieces, taken from eval-okay-builtins-add.exp (not a separate run):
#   builtins.add 2 3           → 5
#   builtins.add 2 2           → 4
#   typeOf (builtins.add 2 2)  → "int"
let
  total = builtins.add (builtins.add 2 3) (builtins.add 2 2);
in
{
  total = total;
  kind = builtins.typeOf total;
}
```

The file above is a rearrangement, so it has **no** `.exp` of its own. The three facts it relies on are the ones in `eval-okay-builtins-add.exp`. On a machine with Nix, the strict value of this rearrangement is what you record, and you record it from that machine.

Pinning more than one evaluator

The suite's `flake.nix` (read from `main` on 2026-10-17) exposes the check. It only defines `x86_64-linux` and `aarch64-linux`. The nixpkgs input is pinned (`github:NixOS/nixpkgs/ef34387ddd751e1ab885`). `packages.adapter` is built from **that** pin's `pkgs.nix`, which the suite treats as 2.34.8. `lib.${system}.adapterFor nix` rebuilds the adapter and the oracles against another Nix. An oracle that does not compile against that Nix's API is omitted and its op is skipped.

```
# ci-conformance.nix
# Shape copied from numtide/nix-conformance flake.nix and README.
# Not instantiated here: no nix binary on this box.
{
  inputs.nix-conformance.url = "github:numtide/nix-conformance";

  outputs = { self, nix-conformance, ... }:
  let
    system = "x86_64-linux";
    lib = nix-conformance.lib.${system};
  in
  {
    checks.${system} = {
```

```
      # A binary you built. Exit 3 skips an op. --known lists accepted gaps.
      conformance = lib.check {
        name = "my-implementation";
        adapter = "${self.packages.${system}.my-adapter}/bin/my-adapter";
        flags = [ "--known" "${./known-failures}" ];
      };
    };
  };
}
```

`known-failures` is one repo-relative path per line. A listed failure counts as known. A listed case that starts passing counts as a failure, so the list cannot rot in silence.

To put a second upstream Nix through the same answers, the README's command uses the **suite's** nixpkgs, not whatever `nixVersions` your 26.05 lock happens to have:

```
# Documented in numtide/nix-conformance README.md. Not run here.
nix build --impure --expr 'let f = builtins.getFlake "github:numtide/nix-conformance";
  p = f.inputs.nixpkgs.legacyPackages.x86_64-linux;
  in f.lib.x86_64-linux.adapterFor p.nixVersions.git'
nix run github:numtide/nix-conformance -- result/bin/* drv
```

What that gives you, if the oracle build succeeds:

Binary you pass to `adapterFor`	What the adapter actually executes
A `nix` that still looks like CppNix's CLI (`nix-instantiate`, `nix eval`)	`eval`, `eval-xml`, `check`, and `flake`
Headers the suite's `oracle/` still compiles against	`parse`, `nar`, `drv`, `wire` as well
Anything else	Those ops exit 3 and show up as skips

Lix installs a `nix` CLI, so the `nix-instantiate` ops are the ones a Lix package can answer without an oracle rebuild. Whether current Lix 2.96 headers still satisfy `oracle/` was not compiled here. Determinate Nix 3.23.3 is a `nix` CLI on top of 2.35.3; extra builtins do not show up unless a case calls them. Snix needs its own adapter: there is no `nix-instantiate` to wrap. Start it with exit 3 on `drv` / `wire` / `flake`, and a `--known` file for the language cases the December 2025 builtins table still marks `todo`, `store`, or `context`.

A smaller CI, when you only have two CLIs and you do not want the oracles, is the upstream loop from `lang.sh` with the conformance rule for failures: diff stdout of `eval-okay-*.nix` against the `.exp`, and require non-zero for `eval-fail-*.nix` **without** diffing `.err.exp`. Vendor the files you mean. Point each matrix leg at a numbered package (`nixVersions.nix_2_35`, `lixPackageSets.lix_2_96`), not at `stable`.

The October 2026 security advisory fixed Nix **2.35.3** (also 2.34.9, 2.31.6, 2.30.6, 2.28.8) and Lix **2.96.0**, **2.95.4**, and **2.94.3**. A conformance pin is not a reason to stay on an unfixed evaluator. Run the suite on the patched build.

The trap

Treating "it evaluates on my laptop" as the language. The laptop has one binary, one `experimental-features` line, and one iostream. The next job installs Lix, or Determinate Nix with flakes already on, or a Snix REPL, and a golden file full of error paragraphs and `1.0` rendered as `1` goes red for a reason that is not a behaviour change.

Trap	What to pin instead
`nixVersions.stable` on nixos-26.05	That alias is **2.34.9**. Upstream's current release is **2.35.3** (`nixVersions.nix_2_35`).
`lixPackageSets.latest`	On nixpkgs `master` this was still **2.95.4** after **2.96.0** existed as `lix_2_96`.
Diffing `.err.exp` across implementations	Upstream `lang.sh` does this for CppNix. The shared suite does not.
`|>` in a shared module	Experimental on CppNix 2.35.3 and on Lix. It fails the reference suite.
`builtins.wasm` or a custom flake schema in a portable flake	Determinate Nix surface. Stock Nix 2.35.3 will not grow it because your CI image did.
Assuming Snix matches because the pure subset feels small	Store builtins and string context are the nixpkgs-sized gap. Use `--known`.

The boring rule

One expression, one `.exp`, one numbered evaluator per CI leg. Failures are exit codes. Sentences are logs. The oracle in the shared suite stays Nix **2.34.8** until that repository says otherwise; your production pin can still be **2.35.3** or Lix **2.96.0**, and `adapterFor` is how you see the difference before users do.

Try this

On a machine that actually has Nix (this one does not):

1. Check out tag `2.35.3` of `NixOS/nix` and run the two commands in cases 1 and 2. Diff stdout against the `.exp` files. Then run `eval-fail-pipe-operators.nix` and confirm a non-zero exit **with the feature left off**.
2. Clone `numtide/nix-conformance` and read `adapters/nix` next to one `eval-okay-context-*.nix`. Note the scratch store. That is why a context test is not `nix-instantiate --eval` against `dummy://`.
3. Add five lines to `known-failures` on purpose, run with `--known`, then delete one line and watch that case turn red. The list is part of the contract.
4. In your own flake, replace any `nixVersions.stable` input with the numbered attribute you meant, and write the version string into the CI log.

Sources

- NixOS/nix tag 2.35.3 — `.version`, annotated tag date 2026-10-13. Also tag 2.34.8 (2026-07-06), the suite's oracle.
- NixOS/nix `tests/functional/lang.sh` at 2.35.3 — `eval-okay` diffs `.exp` and `.err.exp`; `eval-fail` diffs `.err.exp`.
- Test files quoted above, same blobs at 2.34.8 and 2.35.3 unless noted: `tests/functional/lang/eval-okay-bu` `eval-okay-float.nix`, `eval-fail-abort.nix`, `eval-fail-pipe-operators.nix`, `eval-fail-assert-equa`
- Printer and types at tag 2.35.3: `src/libexpr/print.cc` (`printFloat`), `src/libexpr/include/nix/expr/va` (`NixFloat = double`), `src/libexpr/eval.cc` (`assertEqValues`, `showType`), `src/libutil/experimental-f` (`pipe-operators`).
- numtide/nix-conformance — `README.md`, `SPEC.md`, `COVERAGE.md`, `flake.nix`, `adapters/nix`, `lang/eval-okay-float-range.nix`, `lang/eval-okay-context-introspection.nix`, `flake/okay-follows-root/`.
- Lix 2.96 "Bingsu" and the operators manual (pipe operators still experimental).
- Determinate Nix 3.23.3 release notes — based on upstream Nix 2.35.3, 2026-10-16.
- Snix about, builtins status, Nix language issues, string context. Rename announcement: Announcing Snix (2025-03-16).
- nixpkgs `nixos-26.05`: `pkgs/tools/package-management/nix/default.nix`, `pkgs/tools/package-manage` nixpkgs `master`: `lix/2.96.nix` (`version = "2.96.0"`) and the `latest` / `stable` aliases in `lix/default.nix`.
- Security advisory for Nix and Lix, October 2026 — patched Nix 2.35.3 and Lix 2.94.3 / 2.95.4 / 2.96.0.
- hnix, go-nix — partial, not drop-in evaluators.

Module system in depth

Module system in depth

Goal: Use a real **type catalog**, compose with `mkIf` / `mkMerge` / priorities correctly, add assertions, and redesign one production-shaped module so bad configs fail at *eval* time.

> **Note**
>
> You already *consumed* modules (NixOS host part) and wrote a custom one (custom modules chapter). This chapter is the **types + merge** pass—not another "enable nginx" tutorial.

Why this chapter exists

Untyped `attrs` options accept anything until activation or first request fails. Priority mistakes silently lose settings (`mkDefault` overridden by accident, or `mkForce` used as a hammer). Capstone hosts need modules that **reject nonsense early** and **merge predictably** across roles.

Theory 1 — Anatomy of a module (recall with precision)

A module is a function (or attrset) producing some of:

```
{ config, lib, pkgs, ... }:
{
  imports = [ ./peer.nix ];
  options = { /* option declarations */ };
  config = { /* option definitions */ };
}
```

Field	Role
`imports`	DAG of other modules
`options`	Schema: types, defaults, descriptions
`config`	Values / definitions (often under `mkIf`)
freeform / `_module`	Advanced; respect unless you know why

Evaluation merges **all** definitions according to type + priority, then checks types and assertions.

Freeform modules (awareness)

Some modules accept undeclared attrs (freeform types). That is convenient for "pass settings through" and dangerous for typos. Prefer explicit options for anything Capstone treats as policy.

Theory 2 — Type catalog (practical reference)

Import `lib.types` as `types` in real modules. Below is the catalog you should be able to **choose from**, not memorize constructor source.

Scalars & paths

Type	Accepts	Notes
`types.bool`	`true`/`false`	Prefer over "stringly" enables
`types.int` / `types.ints.positive` / `types.ints.u16` …	Integers with bounds	Ports, counts
`types.port`	Port range	Prefer over bare int for listeners
`types.float`	Floats	Rare in OS config
`types.str`	String	Most text
`types.nonEmptyStr`	Non-empty string	Hostnames, user names
`types.singleLineStr`	No newlines	Avoid injection in unit files
`types.lines` / `types.commas`	Multi-line / comma merge	Distinct merge semantics
`types.path`	Path-like	Coerces some strings; store paths common
`types.package`	Derivation/package	For `environment.systemPackages` style
`types.shellPackage`	Package with shell	Specialized
`types.passwdEntry types.str`	Shadow-safe strings	Passwords (still don't put secrets in store)
`types.strMatching "regex"`	Pattern	Domains, tokens shape
`types.enum [ "a" "b" ]`	Closed set	Modes
`types.either t1 t2`	Union	e.g. `bool` or `str`
`types.oneOf [ t1 t2 … ]`	Multi union	Prefer small sets
`types.nullOr t`	`null` or `t`	Optional values

Type	Accepts	Notes
`types.uniq t`	Unique merge	Only one definition wins (strict)
`types.anything`	Escape hatch	Avoid in public modules
`types.raw`	Untyped raw	Advanced / internal

Collections

Type	Merge behavior (intuition)
`types.listOf t`	Concatenate definitions
`types.attrsOf t`	Deep-merge by attr name
`types.lazyAttrsOf t`	Lazier attrs (perf / recursion)
`types.attrs`	Freeform attrs (weak)
`types.attrsWith { … }`	Configurable attrs (modern)
`types.loaOf t`	**Deprecated** list-or-attrs legacy

Structure

Type	Use
`types.submodule { options = …; }`	Nested option trees
`types.submoduleWith { modules = …; }`	Modular submodules
`types.deferredModule`	Module value delayed
`types.optionType`	Type of types (rare)

Function-ish / paths to callables

Type	Use
`types.functionTo t`	Options that are functions
`types.raw` + convention	When types can't express it

Useful helpers

```
types.addCheck t predicate
types.coercedTo from coerceFn to
lib.mkOption {
  type = types.submodule {
    options.enable = lib.mkEnableOption "this feature";
    options.port = lib.mkOption {
      type = types.port;
      default = 8080;
```

```
    };
  };
  default = { };
}
```

`lib.mkEnableOption "desc"` → bool option, default false, standard description.

Choosing types under pressure

Situation	Prefer
On/off feature	`mkEnableOption` / `types.bool`
Port	`types.port`
Hostname / domain	`types.strMatching` or `nonEmptyStr` + assertion
Multi-instance services	`attrsOf (submodule …)`
Free-form upstream JSON	`attrs` **or** structured submodule if you own schema
Secret material	**Never** as string in store; path option only

---

Theory 3 — Definitions: `mkIf`, `mkMerge`, priorities

`mkIf`

```
config = lib.mkIf config.services.myapp.enable {
  users.users.myapp = { isSystemUser = true; group = "myapp"; };
  # ...
};
```

False conditions **drop** definitions (they do not define `null` everywhere).

`mkMerge`

```
config = lib.mkMerge [
  (lib.mkIf cfg.enable { … })
  (lib.mkIf cfg.tls { … })
  { assertions = [ … ]; }
];
```

Prefer `mkMerge` over huge nested `//` trees when multiple independent conditionals apply.

Priority ladder (high level)

Helper	Intent
`mkOptionDefault`	Lowest-ish option default layer
`mkDefault`	Soft default; user config should win
`mkOverride n`	Numeric priority (`mkDefault` 1000, normal 100, …)
`mkForce`	High priority "I really mean this"
`mkFixStrictly` / order-related	Advanced ordering

Discipline:

1. Modules ship **`mkDefault`** for opinionated defaults.
2. Hosts use plain assignments or selective `mkForce`.
3. If you need `mkForce` everywhere, your module boundaries are wrong.

Example conflict

```nix
# module
services.openssh.enable = lib.mkDefault true;

# host
services.openssh.enable = false;  # wins over mkDefault
```

```nix
# role module
networking.firewall.enable = lib.mkForce true;

# host tries
networking.firewall.enable = false;  # loses unless higher override
```

Debugging silent loss

When a setting "does nothing":

1. Search all definitions of that option path.
2. Check for `mkDefault` vs plain vs `mkForce`.
3. Confirm the defining module is actually `imports`-ed.
4. Confirm `mkIf` condition is true.

5. Confirm type merge didn't drop invalid values (eval error should usually surface).

Theory 4 — Assertions & warnings

```
config = {
  assertions = [
    {
      assertion = !(cfg.enable && cfg.package == null);
      message = "myapp: enable requires package";
    }
    {
      assertion = cfg.port != 22 || !config.services.openssh.enable;
      message = "myapp: port 22 conflicts with OpenSSH on this host pattern";
    }
  ];
  warnings = lib.optional cfg.legacyMode ''
    myapp.legacyMode is deprecated; migrate to settings.v2
  '';
};
```

Assertions fail **evaluation**—perfect for Capstone CI (`nix flake check`).

Mechanism	Fails eval?	Use
Type check	Yes	Shape of values
`assertions`	Yes	Cross-option policy
`warnings`	No	Deprecations, soft policy
Activation scripts	No (too late for CI purity)	Runtime environment

---

Theory 5 — Submodules & `attrsOf` services

Pattern for multi-instance:

```
options.services.myapp.instances = lib.mkOption {
  type = types.attrsOf (types.submodule ({ name, ... }: {
    options = {
      enable = lib.mkEnableOption "instance ${name}";
```

```
      port = lib.mkOption { type = types.port; };
    };
  }));
  default = { };
};
```

Then map instances to systemd units with `lib.mapAttrs'`.

```
# sketch
systemd.services = lib.mapAttrs' (name: inst:
  lib.nameValuePair "myapp-${name}" (lib.mkIf inst.enable {
    description = "myapp ${name}";
    # …
  })
) cfg.instances;
```

Watch for name collisions and firewall port unique assertions across instances.

Theory 6 — Import graphs and role modules

Pattern	When
`hosts/lab` imports `profiles/server.nix`	Role defaults
`profiles/server` imports `modules/ssh.nix` + `modules/hardening.nix`	Composition
Host sets `my.role = "edge"` and one module switches	Fewer imports; more logic

Prefer **shallow import graphs** you can draw. Deep diamond imports with conflicting `mkForce` are how priorities become superstition.

Worked example — Redesign sketch

Before (weak)

```
{ config, lib, ... }:
{
  options.my.proxy = lib.mkOption {
    type = lib.types.attrs;
    default = { };
```

```
  };
  config = {
    # hope the attrs are right...
  };
}
```

After (typed)

```
{ config, lib, pkgs, ... }:
let
  inherit (lib) types mkIf mkOption mkEnableOption;
  cfg = config.my.proxy;
in {
  options.my.proxy = {
    enable = mkEnableOption "edge reverse proxy front door";
    domain = mkOption {
      type = types.strMatching "[a-z0-9.-]+";
      example = "lab.example.com";
      description = "Primary public hostname";
    };
    upstream = mkOption {
      type = types.str;
      example = "http://127.0.0.1:8080";
    };
    listenPort = mkOption {
      type = types.port;
      default = 443;
    };
    acme = mkOption {
      type = types.bool;
      default = true;
    };
  };

  config = mkIf cfg.enable {
    assertions = [
      {
        assertion = cfg.listenPort != 0;
        message = "my.proxy.listenPort must be non-zero";
      }
      {
        assertion = !(cfg.acme && cfg.listenPort == 80);
        message = "my.proxy: ACME mode expects non-80 exclusive conflict check - adjust to y
      }
    ];
```

```
    # wire services.nginx / caddy / traefik here with mkDefault
  };
}
```

Bad configs that must fail

```
# wrong type
my.proxy.listenPort = "443";

# failed assertion
my.proxy.enable = true;
my.proxy.listenPort = 0;
```

Save stderr as CI fixtures later (Capstone rebuild proof).

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/modules
cd ~/lab/nixos/internals/modules
```

Copy or reference **one** real module from your lab flake (proxy, app, or secrets-adjacent).

Lab 1 — Type catalog cheat sheet (you write it)

In `TYPES.md`, list **at least 15** types from the catalog with:

- one-line purpose
- one good example value
- one bad value that should fail

Do **not** paste the table from this chapter blindly—rewrite in your words.

Acceptance: 15 rows in your own language; at least 3 collection/structure types.

Lab 2 — Redesign one module

1. Identify untyped `attrs`, raw strings for ports, or missing `mkEnableOption`.
2. Rewrite options with proper types + descriptions.
3. Move conditionals under `mkIf` / `mkMerge`.
4. Add 2 assertions (conflict + required field).
5. Ensure host still evaluates:

```
nix eval .#nixosConfigurations.<host>.config.system.build.toplevel --raw >/dev/null
# or
nixos-rebuild build --flake .#<host>
```

Acceptance: Diff shows types + assertions; toplevel builds.

Lab 3 — Priority drill

Create a tiny three-file demo (can be pure `lib.evalModules`):

```
# eval-demo.nix - sketch
{ lib ? (import <nixpkgs> { }).lib }:
lib.evalModules {
  modules = [
    ({ lib, ... }: {
      options.value = lib.mkOption { type = lib.types.int; };
      config.value = lib.mkDefault 1;
    })
    ({ lib, ... }: {
      config.value = 2; # plain
    })
    ({ lib, ... }: {
      # toggle these in experiments:
      # config.value = lib.mkForce 3;
    })
  ];
}
```

```
nix eval -f eval-demo.nix config.value
```

Record the winner for: default vs plain vs force. Paste results into `PRIORITIES.md`.

Lab 4 — Intentional bad config

Against your redesigned module, write `bad-host.nix` snippets that **must** fail eval (wrong type, failed assertion). Run them and save stderr excerpts—these become CI fixtures later.

Acceptance: Two distinct failure classes with captured stderr.

Lab 5 — Multi-instance sketch (optional)

If your Capstone has only one service instance, still sketch `attrsOf submodule` for a fictional second instance and document how ports would be asserted unique.

Lab 6 — Description quality audit

Pick five options you own. Ensure each has a `description` a stranger could use. Missing descriptions are how "options search" fails future you.

```
# explorative: use nixos-option on a live system or docs
```

Failure modes

Symptom	Fix
Option used but not defined	Declare option or fix path typo
Settings silently ignored	Priority war; search all definitions
Infinite recursion	Defaults cycle (evaluator chapter)
`listOf` duplicates explode unit names	`lib.unique` or set-like design
Assertion message useless	Put **what** and **where** in message

---

Common gotchas

Symptom	What to do
`The option … is used but not defined`	Missing option decl or wrong path
Infinite recursion in modules	Option default depends on own config cycle (evaluator)
Settings silently ignored	Lost priority war; check `mkDefault`/`mkForce`
`listOf` duplicates explode	Dedup with `lib.unique` or design set-like options
Submodule names vs `name` arg	Use `{ name, ... }:` pattern carefully
Secrets in typed options	Types don't encrypt; still use sops/agenix
Over-using `types.attrs`	Prefer `attrsOf submodule` or explicit options
Assertions only on happy path	Add at least one negative fixture

---

Checkpoint

- ☐ `TYPES.md` with 15 types in your words
- ☐ One production module redesigned with real types
- ☐ 2 assertions
- ☐ Priority demo understood (`PRIORITIES.md`)
- ☐ Bad configs fail at eval with clear messages
- ☐ Host/toplevel still builds
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): module types catalog and redesigned module"
```

Write three personal gotchas before continuing.

Next

→ **Store and daemon** — multi-user trust, GC roots, sandbox, signatures.

Store and daemon

Store and daemon

Goal: Explain multi-user Nix trust boundaries, GC roots, sandbox isolation on Linux, and substituter signature policy—then prove it by reading a real sandbox build log and mapping live roots on your machine.

Why this chapter exists

"It came from the cache" and "sandbox off fixed my build" are not strategies. Capstone CI, remote builders, and disaster recovery all assume you understand **who can write the store**, **what a root pins**, and **what isolation actually guarantees**.

Theory 1 — Single-user vs multi-user

Mode	Builder identity	Typical install	Risk profile
Single-user	Your UID owns `/nix/store`	Rare on servers; some dev installs	User compromise store compromise
Multi-user	`nix-daemon` (root) builds; clients speak to daemon	Default multi-user installer / NixOS	Trusted users can influence builds; untrusted cannot freely poison store

On NixOS you almost always run **multi-user** with `nix-daemon.service`.

```
systemctl status nix-daemon
ps aux | grep '[n]ix-daemon'
```

Clients (`nix build`, `nixos-rebuild`) send build goals to the daemon over a Unix socket (`/nix/var/nix/daemon-socket/socket`).

Trusted users

```
nix.settings.trusted-users = [ "root" "@wheel" ];
# or classic nix.conf: trusted-users = root @wheel
```

Trusted clients may set extra substituters, import unsigned paths (depending on config), and change certain build parameters. **Do not** put random CI runners in `trusted-users` without understanding that.

Allowed users

`allowed-users` gates who may talk to the daemon at all. Default is often `*`.

Threat sketches

Actor	Without trusted	With trusted
Unprivileged shell user	Can request builds of known paths; limited conf changes	May configure extra caches, more build power
Compromised CI job as trusted	Can push/pull trust boundaries	Treat as root-adjacent for store policy
Stolen laptop user session	Profile + user roots	Same + any trusted abilities

---

Theory 2 — Store paths, validity, and roots

Path anatomy (recall)

```
/nix/store/<hash>-<name>
```

The hash fingerprints the derivation inputs (input-addressed world—the default you live in). The CA derivations chapter contrasts **content-addressed** experimental paths.

Validity

A path is **valid** if the daemon registered it (built or substituted). Query:

```
nix path-info /nix/store/…-hello-…
nix path-info -r /nix/store/…-hello-…   # closure
nix path-info -S /nix/store/…           # size
nix path-info --json /nix/store/… | head
```

GC roots

Garbage collection deletes **unreferenced** valid paths. Roots include:

Root class	Examples
Profiles	`/nix/var/nix/profiles/system`, per-user profiles
Generations	System generation symlinks
Result symlinks	`./result`, `./result-1` from `nix build`
Runtime roots	Running processes' mapped files (when gcroots scanning enabled)
Explicit roots	`nix-store --add-root`, GC root dirs under `/nix/var/nix/gcroots`

```
nix-store --gc --print-roots | head -50
# modern:
nix store gc --dry-run   # careful; prefer print-roots first
ls -la /nix/var/nix/gcroots/
ls -la /nix/var/nix/profiles/system* | head
```

Teaching point: deleting a git repo does **not** free its build products if a profile or `result` link still roots them.

Disk full playbook (operator)

```
df -h /nix
nix-store --gc --print-dead | head
# review old system generations before deleting
sudo nix-collect-garbage -d   # aggressive; know what you destroy
```

Prefer generation retention policy on lab hosts over panic GC mid-build.

Theory 3 — Sandbox (Linux)

Why sandbox

Builds should see:

- declared inputs only
- no network (except fixed-output derivations with hashed output)

- clean env, controlled `/dev`, empty `/build`

So "works on my laptop" impurities become hard errors.

Mechanisms (Linux)

Rough stack (details evolve; concepts stable):

Mechanism	Role
User namespaces / uid remap	Build user isolation
Mount namespaces	Private `/`, bind store inputs
PID / IPC / UTS namespaces	Process isolation
Network namespace	No ambient net
Seccomp / syscall filters	Reduce kernel attack surface (where enabled)
`sandbox-paths`	Extra allowed paths (dangerous if wide)

Config knobs

```
# NixOS
nix.settings.sandbox = true;          # true | false | relaxed
nix.settings.extra-sandbox-paths = [ ];
# nix.settings.sandbox-fallback = false;
```

Value	Meaning
`true`	Full sandbox (preferred)
`relaxed`	Some impurities allowed (debug)
`false`	Off — only for local debugging, never as silent prod default

```
nix show-config | grep -i sandbox
# or
nix config show | grep -i sandbox
```

Fixed-output derivations (FOD)

Fetchers (`fetchurl`, etc.) may have network **if** the output hash is fixed. Wrong hash → fail. This is intentional purity for source fetching.

Nested virt / KVM note

`nixosTest` and some builds need `kvm` features. That is **not** the same as disabling sandbox. Keep sandbox on; provision KVM correctly on the Elitebook/lab host (see front-matter labs).

Theory 4 — Substituters and signatures

Flow

1. Eval produces derivation goals.
2. Daemon checks local store.
3. Queries substituters (HTTP binary caches).
4. Verifies signatures against **trusted public keys**.
5. Registers paths if valid.

Trust policy

```
nix.settings = {
  substituters = [
    "https://cache.nixos.org"
    "https://myorg.cachix.org"
  ];
  trusted-public-keys = [
    "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
    "myorg.cachix.org-1:…"
  ];
};
```

Misconfiguration	Outcome
Substituter without key	Paths refused (or require trust elevation)
Key without care	You accept binaries from that signer as “as good as build”
`require-sigs = false`	Dangerous; only for isolated experiments

On NixOS, `nix.settings.require-sigs` should stay **true** in normal life.

Signing your own cache (awareness)

CI often builds → signs → pushes to Cachix/Attic/Harmonia. Capstone hardening wires this into CI; this chapter only requires **why** the public key is in your flake/host config.

Theory 5 — Reading build logs

```
nix build nixpkgs#hello --rebuild   # force local rebuild if substitutes exist
nix log /nix/store/…-hello-….drv
# or after failure:
nix log .#packages.x86_64-linux.default
```

Sandbox-related failure fingerprints:

Log signal	Likely cause
`Network access disabled` / connection errors in non-FOD	Impure net need
`No such file or directory` for `/usr/...`	Hardcoded FHS path
Permission denied under `/nix/store`	Trying to mutate inputs
Hash mismatch	FOD impurity or wrong expected hash
`cannot build … because … is not valid`	Missing input; corrupted store

`strace` on daemon builds is advanced and rarely needed; prefer log literacy first.

Theory 6 — Daemon lifecycle on NixOS

```
systemctl cat nix-daemon.service
systemctl restart nix-daemon   # after nix.conf / nix.settings changes when needed
journalctl -u nix-daemon -e --no-pager | tail -100
```

When settings appear "ignored": multi-user means **daemon** config, not only user `~/.config/nix/nix.conf` (unless allowed). Prefer declarative `nix.settings` on NixOS hosts.

Worked example — Trust interview (self)

Answer in writing:

1. Who can talk to the daemon?
2. Who is trusted?
3. Which substituters and which keys?
4. Is sandbox on?
5. What is your oldest system generation still rooted?
6. What would you change before exposing a shared builder on the LAN?

This interview becomes Capstone `TRUST` notes.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/store-daemon
cd ~/lab/nixos/internals/store-daemon
```

Work on a machine with multi-user Nix (NixOS lab VM ideal).

Lab 1 — Map live roots

```
nix-store --gc --print-roots 2>/dev/null | tee roots.txt | head -80
readlink -f /run/current-system || true
ls -la /nix/var/nix/profiles/ | tee profiles.txt
```

In `ROOTS.md`, classify **10 roots** into: system generation, user profile, result link, gcroots explicit, other. Explain what would free a path you care about.

Acceptance: 10 classified roots; one "how I would free X" paragraph.

Lab 2 — Daemon & trust surface

```
systemctl cat nix-daemon.service | tee daemon-unit.txt
nix config show | tee nix-config.txt
grep -E 'trusted|substitut|sandbox|require-sigs|allowed' nix-config.txt
```

Answer in `TRUST.md`:

1. Who is trusted?
2. Which substituters and keys?
3. Is sandbox on?
4. What would you change before exposing a shared builder?

Lab 3 — Sandbox log literacy

Force a small local build and capture the log:

```
nix build nixpkgs#hello --rebuild -L 2>&1 | tee hello-rebuild.log
# find drv / log:
nix log nixpkgs#hello | tee hello-nix-log.txt
```

If substitutes always win and `--rebuild` isn't available in your version, build a tiny local derivation:

```
# tiny.nix
{ pkgs ? import <nixpkgs> { } }:
pkgs.runCommand "lab-tiny" { } ''
  echo hello > $out
''
```

```
nix build -f tiny.nix -L 2>&1 | tee tiny.log
```

Annotate 5 lines in the log (phases, sandbox mentions, store paths).

Acceptance: Annotated log snippet in notes.

Lab 4 — Optional impurity demo (VM only)

> ⚠ Warning
>
> Do this only on a disposable VM. Re-enable sandbox afterward.

1. Note current `sandbox` setting.
2. Temporarily set `sandbox = false` via `nix.settings` or a one-off documented experiment.
3. Observe that impure references may start "working."
4. **Restore `sandbox = true`** and rebuild the host.
5. Record the moral in `TRUST.md`.

Prefer reading docs + logs over leaving sandbox off.

Lab 5 — Signature refusal mental model

Write a short scenario (no need to actually serve malware):

> Cache C serves path P signed by key K_bad. Your host only trusts K_official.

What should the daemon do? What if you are a `trusted-user` and force import? Document answers—tie to Capstone CI signing later.

Lab 6 — Elitebook / KVM lab acceptance

On the lab host (metal or primary VM):

```
# free space for store + images
df -h / /nix 2>/dev/null || df -h /
# nested virt if host is itself a VM
# egrep -i 'vmx|svm' /proc/cpuinfo | head
```

Acceptance criteria (lab readiness):

- ☐ `nix-daemon` active

☐ `sandbox = true`

☐ `require-sigs` true (or documented exception)

☐ 15% free on store filesystem before heavy rebuilds

☐ You know how to open hypervisor console if SSH dies

Common gotchas

Symptom	What to do
`cannot connect to daemon`	`nix-daemon` down; socket perms; single-user mismatch
GC deleted something "needed"	Missing root; reinstall profile / rebuild system
Build needs network mid-compile	Package not pure; fix derivation, don't disable sandbox forever
`untrusted substituter`	Key missing or not trusted for your user
Rootful Docker "fixed" Nix	Different problem; don't conflate
Filling disk	Roots + old generations; GC hygiene chapter
Settings ignored	Restart daemon; use declarative `nix.settings`

Checkpoint

☐ Multi-user vs single-user explained

☐ `ROOTS.md` with classified roots

☐ `TRUST.md` with trusted-users, keys, sandbox

☐ One build log annotated

☐ Sandbox left **enabled** on lab host

☐ Lab free-space / daemon acceptance ticked

☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): store daemon sandbox and substituter trust"
```

Write three personal gotchas before continuing.

Next

→ **Nix performance** — measure eval vs build, tune jobs/cores, justify settings.

What evaluation sends to the daemon

What evaluation sends to the daemon

Goal: Measure, byte by byte, what a Nix client pushes over the daemon socket while it is *only evaluating*, account for every byte class (derivations, `${./path}` imports, `builtins.toFile`), and know the three switches that make that traffic vanish when you do not need it.

Version note: every number below was measured with **Nix 2.35.3** against **nixos-26.05** (`github:NixOS/nixpkgs/6fcc1603…`, which evaluates `hello` to `hello-2.12.3`). The box was an unprivileged x86-64 Linux machine with no system `/nix`, so the "daemon" is `nix daemon --stdio` over a chroot store, fronted by a small counting proxy. The protocol on the wire is the same one `/nix/var/nix/daemon-socket/socket` speaks on a NixOS host; the absolute timings are this box's, not yours.

Why this chapter exists

The store and daemon chapter established that on a multi-user install the client never writes `/nix/store` itself. What it leaves implicit is that this is true *during evaluation too*. "Evaluation" sounds like pure computation inside the client. It is not: every derivation the evaluator instantiates is a `.drv` file that must become a valid store path, and every `${./some/file}` in nixpkgs is a source file that must be copied into the store before its path can be interpolated into a string. On a multi-user host, each of those writes is a request to the daemon.

That is invisible until it isn't: slow `nixos-rebuild` evaluations on a loaded builder, CI jobs that only need a `drvPath` but pay for megabytes of writes, or a remote store (`--store ssh-ng://…`) where each round trip costs real latency.

Mental model

```
nix eval / nix build / nixos-rebuild           nix-daemon (root)
                    socket
  evaluator
   derivation { … }    write    AddToStore    /nix/store/….drv
   "${./patch.diff}"   copy     AddToStore    /nix/store/…-patch
   builtins.toFile     write    AddToStore    /nix/store/…-name
   isValidPath? / queryPath     queries     sqlite db
```

Three byte classes go up the socket while evaluating:

Class	Source construct	Store content address
Derivations	every `derivation`/`mkDerivation` that gets instantiated	`text:` (a `.drv` is a text file)
Path imports	`"${./file}"`, `src = ./dir;` coerced to a string	`fixed:r:sha256` (NAR-hashed)
`builtins.toFile`	small generated scripts and configs	`text:`

The evaluator cannot skip these by default: a derivation's output path is computed from the store paths of its inputs, and the `.drv` is written so that a later `nix build` (possibly by a different process) can realise it.

The lab: a counting proxy

The proxy listens on a Unix socket, starts one `nix daemon --stdio` per client connection, and counts bytes in each direction. On a real host you would instead forward to `/nix/var/nix/daemon-socket/socket`; here the daemon serves a throwaway chroot store so every run can start cold.

```
# countproxy.py: Unix-socket proxy that spawns `nix daemon --stdio` per client
# and counts bytes in each direction.
import os, socket, subprocess, sys, threading, time
SOCK, NIX, STORE = sys.argv[1], sys.argv[2], sys.argv[3]
if os.path.exists(SOCK): os.unlink(SOCK)
srv = socket.socket(socket.AF_UNIX); srv.bind(SOCK); srv.listen(8)
def pump(src_read, dst_write, key, tot):
    while True:
        b = src_read(65536)
        if not b: break
        tot[key] += len(b); dst_write(b)
def handle(c):
    t0 = time.time()
    p = subprocess.Popen([NIX, "daemon", "--stdio", "--force-trusted", "--extra-experimental
                          stdin=subprocess.PIPE, stdout=subprocess.PIPE, bufsize=0)
    tot = {"up": 0, "down": 0}
    def w(b): p.stdin.write(b); p.stdin.flush()
    def up():
        pump(c.recv, w, "up", tot); p.stdin.close()   # client hung up -> daemon exits
    a = threading.Thread(target=up); a.start()
    pump(p.stdout.read, c.sendall, "down", tot); a.join()
    print(f"conn client->daemon={tot['up']} B daemon->client={tot['down']} B "
          f"secs={time.time()-t0:.2f}", flush=True)
    p.wait(); c.close()
while True:
```

```
    c, _ = srv.accept(); threading.Thread(target=handle, args=(c,), daemon=True).start()
```

`--force-trusted` (behind the `daemon-trust-override` experimental feature) makes the stdio daemon treat the connection as trusted, which is what root talking to its own daemon gets anyway. Start it against an empty store:

```
FR=/tmp/nixfresh; rm -rf $FR
python3 countproxy.py /tmp/nixproxy.sock "$(command -v nix)" "local?root=$FR" > proxy.log &
NP=github:NixOS/nixpkgs/6fcc160353f63555b560f25588925133c8f61f2a
nix eval --no-eval-cache --store unix:///tmp/nixproxy.sock --raw $NP#hello.drvPath
```

```
/nix/store/v4c5h94di88ccdd9isyzhivdw84k81nh-hello-2.12.3.drv
```

(On the lab box the client itself ran inside `nix shell --store <chroot>` so that it could see a `/nix/store`; on a normal install it already can.) `--no-eval-cache` matters: with the flake evaluation cache warm, a repeated `drvPath` lookup never touches the evaluator or the daemon at all.

Case 1: cold store, three runs

Each run recreated the empty daemon store, evaluated once cold, then once more against the now-populated store:

```
cold 1
wall=1.175s
conn client->daemon=3489309 B daemon->client=936600 B secs=1.25
warm 1
wall=0.475s
conn client->daemon=365513 B daemon->client=177728 B secs=0.45
cold 2
wall=1.159s
conn client->daemon=3489309 B daemon->client=936600 B secs=1.27
warm 2
wall=0.498s
conn client->daemon=365513 B daemon->client=177728 B secs=0.48
cold 3
wall=1.174s
conn client->daemon=3489309 B daemon->client=936600 B secs=1.24
warm 3
wall=0.472s
conn client->daemon=365513 B daemon->client=177728 B secs=0.45
```

Evaluating one small package from a cold store pushed **3.33 MiB** up the socket and received 0.89 MiB of replies. The byte counts are identical across runs: evaluation is deterministic, so so is its traffic. The warm run still sends 357 KiB. Those are validity queries and path registrations the client cannot avoid without already knowing what the daemon has.

Case 2: what the bytes were

Every path the cold run created is still in the throwaway store, so classify them by content address:

```
nix path-info --store "local?root=$FR" --json --all > fresh-pathinfo.json
python3 - <<'PY'
import json
d = json.load(open('fresh-pathinfo.json'))
items = d.items() if isinstance(d, dict) else [(x['path'], x) for x in d]
cat = {}
for p, i in items:
    ca = i.get('ca') or ''
    if isinstance(ca, dict): ca = ca.get('method', '') + ':'
    k = 'drv' if p.endswith('.drv') else ('toFile (text)' if str(ca).startswith('text') else
    c = cat.setdefault(k, [0, 0]); c[0] += 1; c[1] += i['narSize']
for k, (n, s) in cat.items(): print(f"{k:20s} {n:5d} paths  {s/1024:8.0f} KiB NAR")
PY
```

```
path import (nar)      149 paths       831 KiB NAR
drv                    961 paths      1801 KiB NAR
toFile (text)           10 paths         6 KiB NAR
```

So about **2.6 MiB of payload**: 961 derivations (the whole bootstrap chain behind `hello`, down to `hex0-seed`), 149 source files copied out of the nixpkgs tree (patches, setup hooks such as `make-binary-wrapper.sh`), and ten `toFile` texts. The other ~0.7 MiB on the wire is framing, NAR headers and the queries in between. Notice what is *not* there: the nixpkgs source tree itself. Nix 2.35 evaluates the flake input from its fetcher cache and only materialises the individual files a derivation actually interpolates.

Case 3: what that costs

The same cold evaluation, with the client opening the chroot store directly instead of going through a daemon:

```
direct cold wall=0.761s
direct cold wall=0.847s
direct cold wall=0.856s
```

and with `--read-only` through the proxy (more below):

```
read-only
wall=0.322s
conn client->daemon=632 B daemon->client=184 B secs=0.30
```

On this box, the daemon round trips add roughly 0.3–0.4 s to a ~0.8 s cold evaluation: about **30 %** of the wall time is spent shipping and registering store writes rather than evaluating. Skipping instantiation entirely brings it down to 0.32 s and **816 bytes** of total traffic (the handshake).

The three switches

Need	Switch	Effect
Just the `drvPath`/`outPath`, or an attribute value	`nix eval --read-only …` (classic: `nix-instantiate --readonly-mode`)	Paths are computed but no `.drv` or source is written; output path is identical
Repeated identical evaluations of a flake	leave the eval cache on (it is, by default)	No evaluation, so no traffic
Evaluate against a cheap local store, build elsewhere	`--eval-store <url>` with `--store ssh-ng://builder`	Instantiation writes go to the eval store; only the closure needed to build crosses to the remote

`--read-only` changes what the evaluator does, not what it returns. Its help text is exact: "Do not instantiate each evaluated derivation." The `drvPath` it prints is the same path, but no file exists there afterwards. A later `nix build` will instantiate it again (and pay the traffic then).

The trap

> "Eval is slow, so the daemon must be slow at building."

On a builder that is busy realising other derivations, the daemon is also the process registering your evaluation's 961 `.drv` writes, each a database transaction. Evaluation latency rises with daemon load even when you build nothing. The symptom is `nixos-rebuild dry-build` or `nix flake check` that are fast on an idle laptop and slow on the CI builder.

A second, quieter trap: measuring with the eval cache warm. Repeat `nix eval .#x.drvPath` twice and the second run looks free because it never evaluated. Use `--no-eval-cache` for any measurement in this chapter.

The boring rule

If you only need a path or a value, evaluate read-only. If you need to build, accept the writes, and keep evaluation on the machine that will build. For CI matrix jobs that compute `drvPath`s to decide what to build, `--read-only` is the default you want; the actual build job instantiates once.

Try this

1. Run the proxy against your own daemon socket and evaluate your NixOS host's `config.system.build.toplevel.drvPath` cold. How many derivations, and how many MiB? (Expect far more than `hello`: a whole system closure.)
2. Repeat with `--read-only` and compare the printed path. It should be byte-identical.
3. Add one `"${./big-file}"` to a flake output and watch the path-import class grow by that file's NAR size on the next cold run.
4. Point `--store` at `ssh-ng://` a builder over a high-latency link and time the cold evaluation with and without `--eval-store auto`.

Sources

- Nix 2.35 manual: `nix eval` (`--read-only`, `--eval-store`), common options, store URL types (`unix://`, `local?root=`), `nix daemon --stdio` and `--force-trusted`, https://nix.dev/manual/nix/latest/
- Nix manual, "Derivations" and "String interpolation" (path coercion copies to the store), https://nix.dev/manual/nix/latest/language/string-interpolation
- Nix manual, `builtins.toFile`, https://nix.dev/manual/nix/latest/language/builtins#builtins-toFile

builtins.readFile: file bytes become Nix strings (and pay at eval)

builtins.readFile: file bytes become Nix strings (and pay at eval)

Goal: Know exactly what `builtins.readFile` does during evaluation: it loads the whole file into a Nix string (byte-oriented, with one hard refusal), optionally attaches store references from the file's context, and makes that cost show up *before* any build runs. Prefer path coercion or `builtins.path` when you need the file in the store, not its contents in the evaluator.

Version note: the semantics below match the **Nix Reference Manual** built-ins for current stable Nix (**2.34.x / 2.35+** line; NixOS **26.05**). This writing box has no `nix` binary and installs nothing; listings are the documented forms and the C++ primop behaviour from the Nix source (`prim_readFile` in `src/libexpr/primops.cc`). Re-run the examples on your host with `nix --version` and `nix-instantiate --eval`.

Thesis

> `readFile` is not "import a text config". It is "pay the full file into the evaluator as a string". That string is not a pure UTF-8 text type — it is a byte sequence that happens to live in a Nix string — and anything you interpolate from it can land in a derivation.

Mental model

```
path on disk / in store


realisePath    read entire file into memory

        contains NUL byte?    EvalError (cannot be a Nix string)

        otherwise    Nix string (byte length = stringLength)

                                if path is in the store: rescan
                                 references that actually appear
                                 in the bytes → string context

                               ${thatString} in a drv    bytes in the build
```

Construct	What the evaluator does	What you usually want
`builtins.readFile ./f`	Whole file → string in memory	Small text you truly need as data
`"${./f}"` / `src = ./f;`	Copy path into the store; string is a store path	Ship the file; do not parse it at eval
`builtins.path { path = ./f; … }`	Controlled copy (name/filter/hash)	Same, with hashing/filtering knobs
`builtins.hashFile "sha256" ./f`	Hash the file; return hex string	Integrity check without keeping contents

Worked examples

Case 1: The contract (from the manual)

The manual entry is one sentence: *Return the contents of the file path as a string.* Aliases in nixpkgs (`lib.readFile`, `lib.strings.readFile`) are the same primop.

```
# readfile-hello.nix - documented form; run on a host with Nix 2.35+
# nix-instantiate --eval --strict -E 'builtins.readFile ./hello.txt'
# (create hello.txt with: printf 'hi\n' > hello.txt)
builtins.readFile ./hello.txt
```

Expected shape of the result: a Nix string whose `builtins.stringLength` equals the **byte** length of the file (the manual defines `stringLength` in bytes, not Unicode code points).

Case 2: NUL is the hard stop

The implementation refuses files that contain a zero byte. From `prim_readFile`:

```
// excerpt from Nix src/libexpr/primops.cc (prim_readFile)
auto s = path.readFile();
if (s.find((char) 0) != std::string::npos)
    state.error<EvalError>(
      "the contents of the file '%1%' cannot be represented as a Nix string",
      path).atPos(noPos).debugThrow();
```

So "binary" is not a single category:

- Bytes that are not valid UTF-8 text **can** still become a Nix string via `readFile` (zimbatm's conformance finding: Nix strings are not "UTF-8 only" once you go through this path).
- A **NUL** anywhere in the file still aborts evaluation.

If you need an arbitrary blob in the store, do **not** `readFile` it — pass the path (or `builtins.path`) so the store holds the NAR-hashed file and the string in the expression is a store path, not the payload.

Case 3: Eval cost and string context

Two costs show up at evaluation time, not at build time:

1. **Memory / CPU:** the whole file is read into the evaluator process. A multi-megabyte JSON "config" embedded with `readFile` is paid on every `nix eval` / `nixos-rebuild` dry-run that touches that expression, even if you never build.
2. **Store context:** if the path is already in the Nix store, `readFile` queries path info, then **re-scans** the file bytes for which store references actually occur, and attaches those as the string's context. That is correct for provenance — and it is more work than "just" reading text.

```
# readfile-vs-path.nix - prefer path when you want the file, not its text
let
  # BAD pattern for large / binary / secret-shaped files:
  # payload = builtins.readFile ./big.bin;  # entire file in eval memory

  # GOOD pattern: keep bytes out of the evaluator
  big = ./big.bin;                 # path value; copy happens when coerced
  # or: builtins.path { path = ./big.bin; name = "big.bin"; }
in
{
  # drv that needs the file on disk during the build:
  # src = big;
  tip = "use readFile only for small text you parse at eval";
}
```

Case 4: When `readFile` *is* the right tool

Use it for small, trusted text that the expression must interpret: a tiny TOML/JSON fragment, a pinned version file, a short script you then pass through `builtins.toFile`, or a hash you compare with `builtins.hashFile`. Keep the file size in the kilobytes, not the megabytes, and never treat the resulting string as "always UTF-8 text safe to print".

The trap

Embedding a large or binary-looking file with `readFile` because "I need it in the derivation". You needed the **file in the store**. You paid for a **string in the evaluator**, then often interpolated that string into another file — doubling the cost and smuggling non-text bytes into places that assume UTF-8.

The boring rule

`readFile` = small text for the evaluator. Path / `builtins.path` = bytes for the store. If `stringLength` would shock you, you chose the wrong primitive.

Try this

On a machine with Nix 2.35+ / NixOS 26.05:

```
printf 'abc' > /tmp/t.txt
nix-instantiate --eval -E 'builtins.stringLength (builtins.readFile /tmp/t.txt)'
# → 3

printf 'a\0b' > /tmp/nul.txt
nix-instantiate --eval -E 'builtins.readFile /tmp/nul.txt'
# → error: … cannot be represented as a Nix string

# Prefer this for binaries:
nix-instantiate --eval -E 'builtins.toString ./somefile.bin'   # or coerce "${./somefile.bin
```

Sources

- Nix Reference Manual — Built-ins (`readFile`)
- Noogle — `builtins.readFile` (type + primop excerpt)
- Nix source: `src/libexpr/primops.cc` (`prim_readFile`)
- Related historical discussion: NixOS/nix#2932 (NUL / binary representation)

How Nix computes a store path

How Nix computes a store path

Goal: Recompute `/nix/store/<32 characters>-<name>` by hand for each kind of store object: a `builtins.toFile` text, a source tree added by NAR hash, a flat fixed-output file, an input-addressed derivation output, and the `.drv` file itself. Then confirm that the reimplementation agrees with Nix byte for byte. Once you can do that, cache misses, "why did this rebuild?", and "why did nothing rebuild when I changed the URL?" have exact answers.

Version note: written against **Nix 2.35.3**, the latest upstream tag at the time of writing (nixos.org's download page still lists 2.35.2), and **NixOS 26.05**. Every `nix` command below really ran, using a statically linked Nix 2.35.3 binary on a box with no `/nix`, no daemon, and no root. It ran against a *chroot store* (`--store /tmp/nixlab/store`), whose logical store directory is still `/nix/store`, so every path printed is the path a NixOS host would print. On your own NixOS machine, drop `--store "$S"` (with `nix-command` enabled, as it usually is). The reimplementation uses only the Python standard library and was run with Python 3.13.5.

Thesis

A store path is not a random ID and not the hash of "the package". It is the SHA-256 of a short **fingerprint** string, `type:sha256:<inner digest>:<store dir>:<name>`, folded to 160 bits and written in Nix's own base-32 alphabet. Everything Nix means by "input-addressed" and "content-addressed" lives in two parts of that fingerprint: the **type**, and **what the inner digest is a hash of**. That can be file bytes, a NAR, a derivation's ATerm with fixed-output inputs factored out, or a `fixed:out:` string. The algorithm is specified in the Nix manual's store path calculation. It is about 150 lines of Python, and it explains several things people normally learn by surprise: renaming a source changes its path, mirrors of the same tarball share an output path, and a store at `/opt/nix/store` can never share binaries with `cache.nixos.org`.

Mental model

Store object	`type` in the fingerprint	Inner digest is SHA-256 of
`builtins.toFile`, every `.drv` file	`text` + `:<ref>` for each reference	the file's bytes
`./src`, `builtins.path`, `nix store add` (NAR, SHA-256)	`source` + `:<ref>`… (+ `:self`)	the NAR serialisation
Flat fixed-output (`fetchurl`, `--mode flat`)	`output:out`	the string `fixed:out:sha256:<hex>:`

Store object	`type` in the fingerprint	Inner digest is SHA-256 of
Input-addressed derivation output `out`	`output:out`	the derivation's ATerm, **modulo fixed-output derivations**

- **Two hashes, then a fold.** `sha256(fingerprint)` is 32 bytes. Nix XOR-folds it to 20 (`out[i % 20] ^= h[i]`) and encodes the 160 bits as 32 characters in base-32, using an alphabet without `e`, `o`, `u` or `t`.
- **The name and the store directory are hashed too.** The manual says so explicitly, "to make sure that changes to either of those are reflected in the hash". The same bytes at `/gnu/store` or under another name get a different digest.
- **NAR is the canonical form of a file tree.** It records file type, contents, the executable bit, symlink targets, and sorted directory entries. It records nothing else: no mtimes, owners, or group/other permission bits. That is why `nix hash path` is reproducible across machines (Nix Archive format).
- **"Modulo" means fixed-output inputs are replaced by their promised output.** When Nix hashes a derivation to name its outputs, every input `.drv` path is replaced by *that* derivation's modulo hash. For a fixed-output input, that hash comes only from its declared output hash and path, not from its URL or builder (input-addressing). Case 5 is the payoff.

The code

Two files, standard library only. `storepath.py` holds the fingerprint, the fold, base-32, NAR, and the three content-addressed path kinds:

```
# storepath.py - how Nix turns content and recipes into /nix/store/<hash>-<name>
# Standard library only. Mirrors the algorithm in the Nix manual's
# "Store Path" and "Content-Addressing Store Objects" pages.
import hashlib
import os
import stat

STORE_DIR = "/nix/store"
ALPHABET = "0123456789abcdfghijklmnpqrsvwxyz"   # Nix base-32: no e, o, u, t


def nix32(digest: bytes) -> str:
    """Nix's base-32: 5 bits at a time, starting from the END of the digest."""
    n = (len(digest) * 8 - 1) // 5 + 1
    out = []
    for k in range(n - 1, -1, -1):
        b = k * 5
        i, j = divmod(b, 8)
        c = digest[i] >> j
        if i + 1 < len(digest):
```

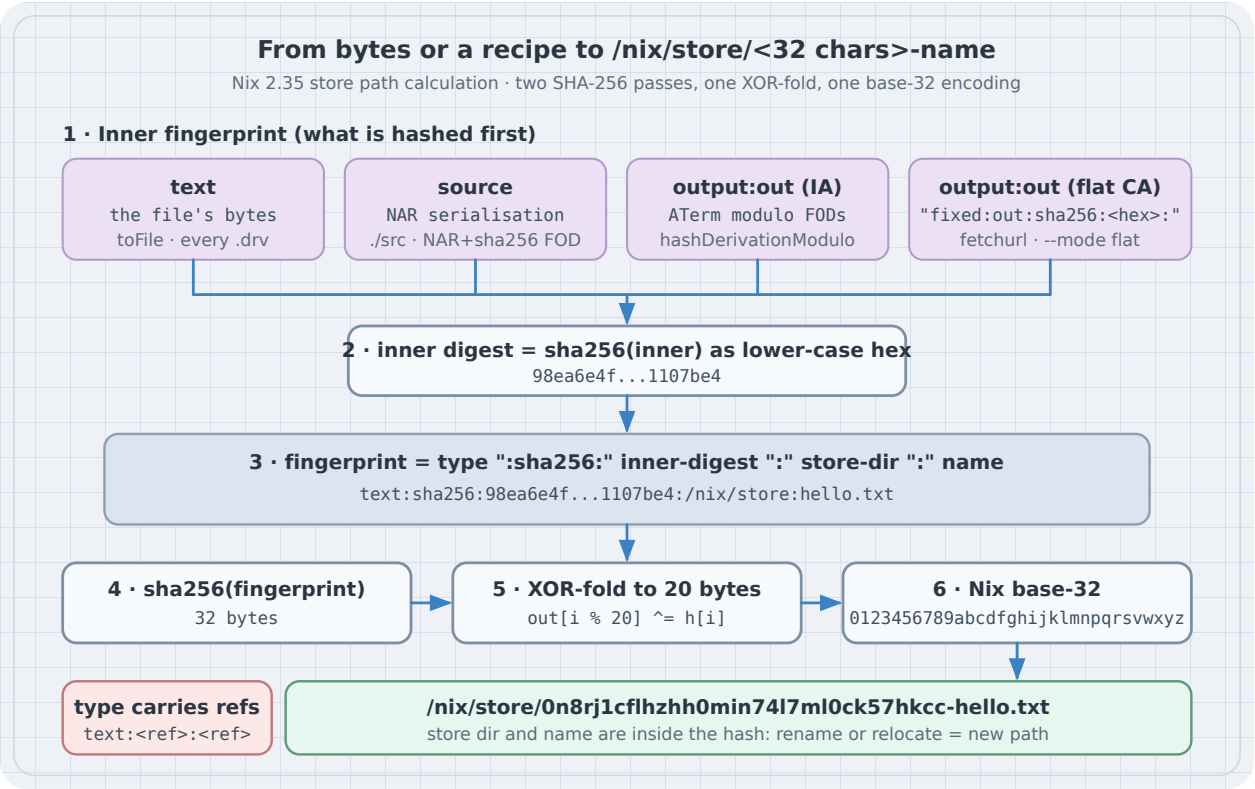


Figure 1: Inner fingerprint, inner digest, fingerprint, SHA-256, XOR-fold, Nix base-32

```
            c |= digest[i + 1] << (8 - j)
        out.append(ALPHABET[c & 0x1F])
    return "".join(out)


def compress(digest: bytes, size: int = 20) -> bytes:
    """XOR-fold a 32-byte SHA-256 down to 20 bytes (160 bits)."""
    out = bytearray(size)
    for i, b in enumerate(digest):
        out[i % size] ^= b
    return bytes(out)


def make_store_path(kind: str, inner_hex: str, name: str) -> str:
    store_dir = STORE_DIR                          # read at call time, so callers can change i
    fingerprint = f"{kind}:sha256:{inner_hex}:{store_dir}:{name}"
    digest = hashlib.sha256(fingerprint.encode()).digest()
    return f"{store_dir}/{nix32(compress(digest))}-{name}"


def refs(kind: str, references=()) -> str:
```

```
    return kind + "".join(":" + r for r in sorted(references))


# --- NAR: the canonical serialisation of a file system object -------------

def _str(b: bytes) -> bytes:
    return len(b).to_bytes(8, "little") + b + b"\0" * (-len(b) % 8)


def nar(path: str) -> bytes:
    def node(p):
        st = os.lstat(p)
        out = _str(b"(")
        if stat.S_ISLNK(st.st_mode):
            out += _str(b"type") + _str(b"symlink") + _str(b"target") + _str(os.readlink(p).
        elif stat.S_ISDIR(st.st_mode):
            out += _str(b"type") + _str(b"directory")
            for entry in sorted(os.listdir(p)):
                out += _str(b"entry") + _str(b"(") + _str(b"name") + _str(entry.encode())
                out += _str(b"node") + node(os.path.join(p, entry)) + _str(b")")
        else:
            out += _str(b"type") + _str(b"regular")
            if st.st_mode & 0o100:
                out += _str(b"executable") + _str(b"")
            out += _str(b"contents") + _str(open(p, "rb").read())
        return out + _str(b")")
    return _str(b"nix-archive-1") + node(path)


# --- the three ways a store path is born ----------------------------------

def text_path(name: str, contents: bytes, references=()) -> str:
    """builtins.toFile, and every .drv file."""
    return make_store_path(refs("text", references), hashlib.sha256(contents).hexdigest(), n


def source_path(path: str, name: str, references=()) -> str:
    """Files added to the store by NAR hash: ./src, builtins.path, nix store add."""
    return make_store_path(refs("source", references), hashlib.sha256(nar(path)).hexdigest()


def fixed_output_path(name: str, sha256_hex: str, recursive: bool) -> str:
    """Fixed-output derivations (fetchurl, ...) and `nix store add --mode flat`."""
    if recursive:
        return make_store_path("source", sha256_hex, name)
    inner = hashlib.sha256(f"fixed:out:sha256:{sha256_hex}:".encode()).hexdigest()
    return make_store_path("output:out", inner, name)
```

`drv.py` holds derivations: the ATerm serialisation (derivation ATerm format), `hashDerivationModulo`, and what `derivation { … }` does at evaluation time. It covers single-output input-addressed derivations and flat fixed-output derivations, which is enough for every case below:

```
# drv.py - derivations: ATerm, hashDerivationModulo, output and .drv paths
import hashlib

from storepath import make_store_path, text_path


def _s(x: str) -> str:
    return '"' + x.replace("\\", "\\\\").replace('"', '\\"').replace("\n", "\\n") \
                  .replace("\r", "\\r").replace("\t", "\\t") + '"'


def _list(items) -> str:
    return "[" + ",".join(items) + "]"


def aterm(d: dict, input_drvs: dict) -> str:
    """Serialise like a .drv file. input_drvs maps a key (a .drv path, or a
    hash when computing the modulo) to the list of outputs used."""
    outs = _list("(" + ",".join(_s(v) for v in (o, *d["outputs"][o])) + ")" for o in sorted(
    drvs = _list("(" + _s(k) + "," + _list(_s(o) for o in sorted(v)) + ")" for k, v in sorte
    srcs = _list(_s(p) for p in sorted(d["input_srcs"]))
    args = _list(_s(a) for a in d["args"])
    env = _list("(" + _s(k) + "," + _s(v) + ")" for k, v in sorted(d["env"].items()))
    return f"Derive({outs},{drvs},{srcs},{_s(d['system'])},{_s(d['builder'])},{args},{env})"


def is_fixed_output(d: dict) -> bool:
    return d["outputs"].keys() == {"out"} and d["outputs"]["out"][2] != ""


def hash_modulo(d: dict) -> str:
    """hashDerivationModulo: the hash that names a derivation's outputs."""
    if is_fixed_output(d):
        path, algo, h = d["outputs"]["out"]
        return hashlib.sha256(f"fixed:out:{algo}:{h}:{path}".encode()).hexdigest()
    # Replace every input .drv path by *its* modulo hash, then hash the ATerm.
    masked = {hash_modulo(dep): outs for dep, outs in d["input_drv_objs"]}
    return hashlib.sha256(aterm(d, masked).encode()).hexdigest()


def finalise(name: str, system: str, builder: str, args, env: dict, input_drv_objs=(), input
             fixed=None):
    """What `derivation { ... }` does at evaluation time (input-addressed or flat FOD)."""
```

```
    from storepath import fixed_output_path
    env = {"name": name, "system": system, "builder": builder, **env}
    d = {"system": system, "builder": builder, "args": list(args), "input_srcs": list(input_
         "input_drv_objs": list(input_drv_objs)}
    if fixed:                                          # fixed=("sha256", hex): outputHashMo
        algo, h = fixed
        out = fixed_output_path(name, h, recursive=False)
        d["outputs"] = {"out": (out, algo, h)}
        d["env"] = {**env, "out": out}
    else:
        d["outputs"] = {"out": ("", "", "")}           # 1. blank the output path ...
        d["env"] = {**env, "out": ""}                  #    ... everywhere it appears
        out = make_store_path("output:out", hash_modulo(d), name)   # 2. name it by the modu
        d["outputs"] = {"out": (out, "", "")}          # 3. write it back
        d["env"]["out"] = out
    real_drvs = {dep["drv_path"]: outs for dep, outs in d["input_drv_objs"]}
    d["aterm"] = aterm(d, real_drvs)
    d["out_path"] = out
    d["drv_path"] = text_path(name + ".drv", d["aterm"].encode(),
                              references=[*d["input_srcs"], *real_drvs])
    return d
```

Set up the lab once. On a NixOS host, skip `S` and every `--store "$S"`:

```
mkdir -p /tmp/nixlab && cd /tmp/nixlab     # storepath.py, drv.py and the case files live he
S=/tmp/nixlab/store                         # chroot store; logical store dir is still /nix/
nix --version
python3 --version
```

```
nix (Nix) 2.35.3
Python 3.13.5
```

Worked examples

Case 1: `builtins.toFile`, one step at a time

```
# case1.py - builtins.toFile "hello.txt" "hi\n", by hand
import hashlib
from storepath import nix32, compress, text_path

contents = b"hi\n"
inner = hashlib.sha256(contents).hexdigest()
fingerprint = f"text:sha256:{inner}:/nix/store:hello.txt"
digest = hashlib.sha256(fingerprint.encode()).digest()
```

```
print("fingerprint :", fingerprint)
print("sha256      :", digest.hex())
print("xor-fold 20 :", compress(digest).hex())
print("nix32       :", nix32(compress(digest)))
print("store path  :", text_path("hello.txt", contents))
```

```
python3 case1.py
nix --store "$S" eval --raw --expr 'builtins.toFile "hello.txt" "hi\n"'; echo
```

Output:

```
fingerprint : text:sha256:98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4:/
sha256      : 0e898c0e1e74df971c7c03b1083fa48e0599910582c4f4c47af409d4d5cdb2b3
xor-fold 20 : 8c4d78ca6480d643c9b1b102083fa48e05999105
nix32       : 0n8rj1cflhzhh0min74l7ml0ck57hkcc
store path  : /nix/store/0n8rj1cflhzhh0min74l7ml0ck57hkcc-hello.txt
/nix/store/0n8rj1cflhzhh0min74l7ml0ck57hkcc-hello.txt
```

The first five lines are the algorithm and the last line is Nix, and they match. Notice what the inner digest is: `98ea6e4f…` is simply `sha256("hi\n")`. A text object is addressed by its contents plus its name. Change either one and the path changes.

Case 2: A directory: NAR bytes, NAR hash, `source` path

Build a small tree with one executable, one plain file, and a symlink, then serialise and hash it in Python.

```
# case2.py - NAR-hash a directory and predict its store path
import base64
import hashlib
import sys

from storepath import nar, nix32, source_path

tree = sys.argv[1]
blob = nar(tree)
digest = hashlib.sha256(blob).digest()
open("tree.py.nar", "wb").write(blob)

print("NAR bytes    :", len(blob))
print("sha256 hex   :", digest.hex())
print("sha256 SRI   :", "sha256-" + base64.b64encode(digest).decode())
print("sha256 nix32 :", nix32(digest))
print("store path   :", source_path(tree, "tree"))
```

```
mkdir -p tree/bin
printf '#!/bin/sh\necho hello\n' > tree/bin/hello && chmod 755 tree/bin/hello
printf 'hi\n' > tree/README && chmod 644 tree/README
ln -s bin/hello tree/run
python3 case2.py tree
nix nar pack tree > tree.nix.nar && cmp tree.py.nar tree.nix.nar && echo "NAR bytes identica
nix hash path tree
nix hash path --base32 tree
nix --store "$S" store add tree
```

Output:

```
NAR bytes    : 896
sha256 hex   : 76089773df8aecb93056dbf6e3e015abb02b03433441699c061aa6988fcf894e
sha256 SRI   : sha256-dgiXc9+K7LkwVtv24+AVq7ArAOMOQWmcBhqmmI/PiU4=
sha256 nix32 : 0kl9ry7ri9hs0sf6jh9l8c1jpc5b2phf7xnvaqqbkv4avxrrf23n
store path   : /nix/store/zb37ng1sjjd4gilsna650k7mankgdjc9-tree
NAR bytes identical
sha256-dgiXc9+K7LkwVtv24+AVq7ArAOMOQWmcBhqmmI/PiU4=
0kl9ry7ri9hs0sf6jh9l8c1jpc5b2phf7xnvaqqbkv4avxrrf23n
/nix/store/zb37ng1sjjd4gilsna650k7mankgdjc9-tree
```

The 896 NAR bytes are identical to Nix's, so the hash is identical in all three spellings (hex, SRI base-64, Nix base-32), and so is the `source:` store path. Now change what NAR ignores, and then what it keeps:

```
touch -d 2001-01-01 tree/README && chmod g+w tree/README   # mtime and group bits: not in NA
nix hash path tree
chmod -x tree/bin/hello                                    # executable bit: in NAR
nix hash path tree
chmod +x tree/bin/hello
```

Output:

```
sha256-dgiXc9+K7LkwVtv24+AVq7ArAOMOQWmcBhqmmI/PiU4=
sha256-mQ8ZVZA7F6E7KJMY4dq0jELaewFiaRllv114SJv3CzY=
```

Case 3: One file, three addresses

The same `hi.txt` added three ways: as a flat file, as a NAR, and as a NAR under a different name.

```
# case3.py - one file, three addresses: flat, recursive (NAR), and a different name
import hashlib
import sys
```

```
from storepath import fixed_output_path, nar

path = sys.argv[1]
flat = hashlib.sha256(open(path, "rb").read()).hexdigest()
rec = hashlib.sha256(nar(path)).hexdigest()
print("sha256(file bytes):", flat)
print("sha256(NAR)       :", rec)
print("flat      hi.txt  :", fixed_output_path("hi.txt", flat, recursive=False))
print("recursive hi.txt  :", fixed_output_path("hi.txt", rec, recursive=True))
print("recursive hello   :", fixed_output_path("hello", rec, recursive=True))
```

```
printf 'hi\n' > hi.txt
python3 case3.py hi.txt
nix --store "$S" store add --mode flat hi.txt
nix --store "$S" store add --mode nar hi.txt
nix --store "$S" store add --mode nar --name hello hi.txt
```

Output:

```
sha256(file bytes): 98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4
sha256(NAR)       : 10551daed7fa467d0b58826e7eb9a0f2af7d693de319f2ef775fffcda244b9f6
flat      hi.txt  : /nix/store/a1qbm6bia1qj42ganmyz01knaai6d59p-hi.txt
recursive hi.txt  : /nix/store/qs2vcw1sd6lzsvgvq6gggnw1v31qrm28-hi.txt
recursive hello   : /nix/store/a5izqk5bgpxcrrnrm1m8n1fvyq2jlc52-hello
/nix/store/a1qbm6bia1qj42ganmyz01knaai6d59p-hi.txt
/nix/store/qs2vcw1sd6lzsvgvq6gggnw1v31qrm28-hi.txt
/nix/store/a5izqk5bgpxcrrnrm1m8n1fvyq2jlc52-hello
```

Three different paths for one 3-byte file. A flat SHA-256 is wrapped in `fixed:out:sha256:<hex>:` and goes through the `output:out` type. A NAR SHA-256 uses `source` directly. The manual's historical note explains that NAR + SHA-256 "always uses the `source:...` construction, regardless of how it was produced", which is why a `fetchzip`-style recursive fixed-output derivation and a manual `nix store add` of the same tree land on the same path. Renaming changes the path even though the hash of the content does not.

Case 4: An input-addressed derivation, byte for byte

```
derivation {
  name = "hello";
  system = "x86_64-linux";
  builder = "/bin/sh";
  args = [ "-c" "echo hi > $out" ];
}
```

```
# case4.py - predict hello.nix's .drv bytes, outPath and drvPath
from drv import finalise

hello = finalise("hello", "x86_64-linux", "/bin/sh", ["-c", "echo hi > $out"], {})
print("outPath :", hello["out_path"])
print("drvPath :", hello["drv_path"])
open("hello.py.drv", "w").write(hello["aterm"])
```

```
python3 case4.py
nix --store "$S" eval --raw -f hello.nix outPath; echo
drv=$(nix --store "$S" eval --raw -f hello.nix drvPath); echo "$drv"
cat "$S$drv"; echo
cmp hello.py.drv "$S$drv" && echo ".drv bytes identical"
```

Output:

```
outPath : /nix/store/mjs27ix6ig2bkbi3s3sm470vrv4lf7ic-hello
drvPath : /nix/store/76w21n1f03fs5kw8fnffphx7qrqffw6r-hello.drv
/nix/store/mjs27ix6ig2bkbi3s3sm470vrv4lf7ic-hello
/nix/store/76w21n1f03fs5kw8fnffphx7qrqffw6r-hello.drv
Derive([("out","/nix/store/mjs27ix6ig2bkbi3s3sm470vrv4lf7ic-hello","","")],[],[],"x86_64-lin
.drv bytes identical
```

Nix does three things here, in this order. (1) It builds the derivation with every output path set to `""`, both in the outputs list and in the `out` environment variable, then serialises that to ATerm and hashes it. (2) It names the output `make_store_path("output:out", that_hash, "hello")`. (3) It writes the path back in, serialises the final ATerm, which is the `.drv` file, and stores *that* as a `text` object named `hello.drv`, with its input sources and input derivations as references. The output path is computed **before** the `.drv` exists, so the output cannot depend on the `.drv` path. That is the circularity "modulo" breaks.

Case 5: The "modulo": two mirrors, one output path

A fixed-output "fetch" (the builder just prints a URL; nothing is built) and an app that copies it. Mirror `a` and mirror `b` differ only in the URL.

```
# app.nix - one fixed-output source, two different "mirrors", one consumer
let
  greeting = url: derivation {
    name = "greeting.txt";
    system = "x86_64-linux";
    builder = "/bin/sh";
    args = [ "-c" "echo fetching ${url}; printf 'hi\\n' > $out" ];
    outputHashMode = "flat";
    outputHashAlgo = "sha256";
    outputHash = "98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4";
```

```
  };
  app = src: derivation {
    name = "app";
    system = "x86_64-linux";
    builder = "/bin/sh";
    args = [ "-c" "cp ${src} $out" ];
  };
in {
  a = app (greeting "https://a.example/greeting.txt");
  b = app (greeting "https://b.example/greeting.txt");
}
```

```
# case5.py - the "modulo" in hashDerivationModulo
from drv import finalise

SHA = "98ea6e4f216f2fb4b69fff9b3a44842c38686ca685f3f55dc48c5d3fb1107be4"


def greeting(url):
    return finalise("greeting.txt", "x86_64-linux", "/bin/sh",
                    ["-c", f"echo fetching {url}; printf 'hi\\n' > $out"],
                    {"outputHashMode": "flat", "outputHashAlgo": "sha256", "outputHash": SHA
                    fixed=("sha256", SHA))


def app(src):
    return finalise("app", "x86_64-linux", "/bin/sh", ["-c", f"cp {src['out_path']} $out"],
                    input_drv_objs=[(src, ["out"])])


for mirror in ("a", "b"):
    src = greeting(f"https://{mirror}.example/greeting.txt")
    top = app(src)
    print(f"{mirror}: greeting.drvPath {src['drv_path']}")
    print(f"   greeting.outPath {src['out_path']}")
    print(f"   app.drvPath      {top['drv_path']}")
    print(f"   app.outPath      {top['out_path']}")
```

```
for x in a b; do for k in drvPath outPath; do
  echo "$x.$k $(nix --store "$S" eval --raw -f app.nix $x.$k)"; done; done
python3 case5.py
```

Output:

```
a.drvPath /nix/store/ah3ma3iq16cc07b5a1q31h7zxjpalmfk-app.drv
a.outPath /nix/store/gxiksvnwx12gg9nlvcnqw884mfp2ppfk-app
b.drvPath /nix/store/zhd4fazximx9kzrich1rk4qfcy3a7y7s-app.drv
b.outPath /nix/store/gxiksvnwx12gg9nlvcnqw884mfp2ppfk-app
a: greeting.drvPath /nix/store/1f76makgrz9w9s6qsghadmvfv333fnzx-greeting.txt.drv
   greeting.outPath /nix/store/0cd9wfphx5i9sly07cr00rzqn22d3lbj-greeting.txt
   app.drvPath      /nix/store/ah3ma3iq16cc07b5a1q31h7zxjpalmfk-app.drv
   app.outPath      /nix/store/gxiksvnwx12gg9nlvcnqw884mfp2ppfk-app
b: greeting.drvPath /nix/store/mpd6qm01vvislikzq4marrgwacpr272c-greeting.txt.drv
   greeting.outPath /nix/store/0cd9wfphx5i9sly07cr00rzqn22d3lbj-greeting.txt
   app.drvPath      /nix/store/zhd4fazximx9kzrich1rk4qfcy3a7y7s-app.drv
   app.outPath      /nix/store/gxiksvnwx12gg9nlvcnqw884mfp2ppfk-app
```

Read it from the bottom up. The two `greeting.txt.drv` files differ, because the URL is in their args. Their output path is the same, because a flat fixed-output path depends only on `name` and `outputHash`. The two `app.drv` files differ, because each one names a different input `.drv`. Yet `app`'s **output path is identical**: when Nix hashed `app` to name its output, it replaced each input `.drv` path with `sha256("fixed:out:sha256:<hash>:<outPath>")`, and that string is the same for both mirrors. This is why moving a `fetchurl` to a mirror, or changing its `curl` flags, does not rebuild the world, and why a wrong `outputHash` is caught at fetch time rather than silently accepted.

The trap

Two things in the fingerprint are not your bytes, and both bite in practice. One is the recipe: an input-addressed path is named by the ATerm, not by what the build produces. The content-addressed internals chapter shows the rebuild this causes. The other is the store directory itself.

```
# trap.py - two things in the fingerprint that are not your bytes
import storepath
from drv import finalise

v1 = finalise("hello", "x86_64-linux", "/bin/sh", ["-c", "echo hi > $out"], {})
v2 = finalise("hello", "x86_64-linux", "/bin/sh", ["-c", "echo hi > $out  # same output"], {
print("recipe v1          :", v1["out_path"])
print("recipe v2 (comment):", v2["out_path"])

storepath.STORE_DIR = "/opt/nix/store"
v1_opt = finalise("hello", "x86_64-linux", "/bin/sh", ["-c", "echo hi > $out"], {})
print("v1 in /opt/nix/store:", v1_opt["out_path"])
```

```
python3 trap.py
nix --store "$S" eval --raw --expr 'derivation { name = "hello"; system = "x86_64-linux"; bu
nix --store 'local?store=/opt/nix/store&real=/tmp/alt/store&state=/tmp/alt/state&log=/tmp/al
```

Output:

```
recipe v1          : /nix/store/mjs27ix6ig2bkbi3s3sm470vrv4lf7ic-hello
recipe v2 (comment): /nix/store/52vkb7gk02c7kdgmfzbzr1z860vsk9id-hello
v1 in /opt/nix/store: /opt/nix/store/9nyvidn6yc9hq5kcvw9iwk3hd71kzb6v-hello
/nix/store/52vkb7gk02c7kdgmfzbzr1z860vsk9id-hello
/opt/nix/store/9nyvidn6yc9hq5kcvw9iwk3hd71kzb6v-hello
```

A shell comment that cannot change the output still gives a new path, because the ATerm changed. Moving the store to `/opt/nix/store`, a common wish on machines where `/` is small or read-only, changes **every** hash. The binary cache is keyed by `/nix/store/<hash>`, so a store with a different directory gets no substitutes from `cache.nixos.org` and builds everything from source, including the compiler. If `/nix` is the problem, move the bytes rather than the name: a bind mount or a dedicated filesystem mounted at `/nix`.

The boring rule

1. Keep the store directory `/nix/store`. Relocate storage with mounts, never with `store=`.
2. Treat a store path as `hash(type, inner digest, store dir, name)`. When a path changes unexpectedly, ask which of the four changed.
3. Use `nix hash path` (NAR) for trees and `nix hash file` (flat) for single files, and know that they give different store paths for the same bytes.
4. Pin sources with fixed-output hashes. Mirrors and fetch flags then stay out of every downstream output path.
5. When debugging a rebuild, diff the two `.drv` files (`nix derivation show` on both), not the outputs. The output path is a pure function of the ATerm modulo fixed-output inputs.

Try this

1. Add a reference to `text_path`: compute `builtins.toFile "b" "${builtins.toFile "a" "x"}"` in Python (the reference goes into the type as `text:/nix/store/…-a`) and compare with `nix eval --raw --expr`.
2. Give `hello.nix` a second output (`outputs = [ "out" "dev" ];`). Extend `finalise` so both outputs are blanked before hashing and each is named `output:<id>` with the name `hello` or `hello-dev`. Check against `nix eval .dev.outPath`.
3. Change Case 5's fixed-output derivation to `outputHashMode = "recursive"`. Which line of `fixed_output_path` now applies, and does `app`'s output path change?
4. Write the inverse of `nix32()` in Python (decode 52 characters back to 32 bytes) and check it against `nix hash convert --hash-algo sha256 --to base16 0kl9ry7ri9hs0sf6jh9l8c1jpc5b2phf7xnv` from Case 2.
5. Compute the path of a store object that refers to itself (`source:...:self`). Which kind of build produces such an object, and why must the path be computed with the self-reference masked?

Sources

- Nix Reference Manual (2.35): Store path calculation (fingerprint grammar, `text` / `source` / `output:` types, `fixed:out:` inner fingerprint, historical note on `source:`), Store path, Content-addressing store objects, Nix Archive (NAR) format, Derivation ATerm file format, Input-addressing derivation outputs.
- Nix command reference: `nix eval`, `nix store add`, `nix hash path`, `nix nar pack`, `nix derivation show` (`nix <command> --help` on 2.35.3).
- Eelco Dolstra, *The Purely Functional Software Deployment Model* (PhD thesis, 2006), section 5.1, cited by the manual for the hash compression.
- Nix 2.35 release notes and the NixOS download page for the current versions (Nix 2.35.x, NixOS 26.05).

Nix performance

Nix performance

Goal: Measure where time goes (eval vs download vs build), tune `max-jobs` / `cores` / remote builders / caches, and record a before/after rebuild timing with settings you can justify.

Why this chapter exists

NixOS rebuilds feel "slow" for different reasons. Cranking `--max-jobs 128` on a laptop with 8GB RAM and a spinning disk can thrash harder than serial builds. Capstone CI budgets and lab iteration speed both require **measurement first**.

Theory 1 — The rebuild time budget

A flake host rebuild roughly pays:

```
eval (modules + nixpkgs)  →  realize (substitute or build)  →  activate
```

Phase	Dominated by	Knobs
Eval	CPU single-thread-ish, memory, module count, IFD	eval-cache, avoid IFD, slim modules
Substitute	Network, cache hit rate, compression	binary caches, closer mirrors
Build	CPU cores, jobs, IO, remote builders	`max-jobs`, `cores`, builders
Activate	Service restarts, switch strategy	`switch` vs `boot` vs `test`

```
# rough wall clock
/usr/bin/time -v nixos-rebuild build --flake .#<host>
# or
time nix build .#nixosConfigurations.<host>.config.system.build.toplevel
```

Separate **cold** (empty download cache / first eval) vs **warm** timings or you will lie to yourself.

Elitebook 16 GB reality

Workload	Expectation
Leaf `nix eval` hostname	Seconds
Warm toplevel after small edit	Often substitute-heavy; minutes if many new builds
Cold after lock bump	Long; plan coffee + cache
Parallel `max-jobs=8` link storms	OOM risk; prefer 2–4 jobs

Theory 2 — Jobs vs cores

Definitions

Setting	Meaning
`max-jobs`	How many derivations realized in parallel
`cores`	How many cores advertised to each derivation (`NIX_BUILD_CORES` / `enableParallelBuilding`)

```
nix.settings = {
  max-jobs = 4;     # or "auto"
  cores = 2;        # per-job parallelism
};
```

CLI overrides:

```
nix build -j 4 --cores 2 .#…
nixos-rebuild build --flake .#host --max-jobs 4 --cores 2
```

Choosing numbers (rules of thumb)

Hardware	Starting point
4c/8t laptop, 16GB	`max-jobs = 2-4`, `cores = 2`
16c desktop, 64GB	`max-jobs = 8`, `cores = 2-4`
Many small builds	higher `max-jobs`, lower `cores`
Few huge compiles (gcc/llvm)	lower `max-jobs`, higher `cores`
Tiny rootfs / low RAM	**lower** both; watch OOM

Constraint: `max-jobs * cores` should not massively exceed physical cores + RAM headroom, especially with linking-heavy packages.

Remote builders

```
nix.distributedBuilds = true;
nix.buildMachines = [ {
  hostName = "builder";
  system = "x86_64-linux";
  maxJobs = 8;
  speedFactor = 2;
  supportedFeatures = [ "nixos-test" "benchmark" "big-parallel" "kvm" ];
  mandatoryFeatures = [ ];
} ];
```

Offload large builds; keep eval local unless you know your setup.

Feature	Why list it
`kvm`	nixosTest VMs
`big-parallel`	large compiles allowed
`nixos-test`	test driver builds

Theory 3 — Eval performance

Eval cache

Modern Nix can cache evaluation results for flakes when inputs unchanged.

```
nix config show | grep -i eval
# features / options evolve; check your version docs for eval-cache
```

Practical wins:

- Stable `flake.lock` → better cache reuse
- Avoid unnecessary `--impure`
- Avoid Import From Derivation (IFD) when possible

IFD cost

```
# expensive pattern: build something at eval time
let pkg = import (pkgs.fetchFromGitHub { … }) { };
in pkg.something
```

IFD forces realization during eval—serial and painful in CI. Prefer flake inputs and generated files committed or produced in build phase.

Module count

Every module imported has a cost. Shared modules should be **focused**, not a megamodule imported everywhere with giant `mkIf` trees if you can split roles.

Memory

Large evals can exceed default stack/heap. Symptoms: evaluator OOM, thrashing. Fix structure before only raising limits.

```
# rough: watch RSS while evaluating toplevel
/usr/bin/time -v nix eval .#nixosConfigurations.lab.config.system.build.toplevel --raw >/dev
```

Theory 4 — Download & store IO

Lever	Effect
Hit `cache.nixos.org` + project cache	Avoid local compiles
`http2` / modern curl stack	Throughput (usually already fine)
Keep disk free	Avoid GC mid-build
`min-free` / `max-free` auto-GC	Prevent disk full mid-CI
SSD vs HDD	Link farms and unpack dominate on HDD

```
nix.settings = {
  min-free = 1073741824;   # 1 GiB
  max-free = 5368709120;   # 5 GiB
};
```

On Elitebook labs with small disks, aggressive auto-GC can surprise you mid-ISO build—know the knobs.

Theory 5 — What *not* to do

Anti-pattern	Why
`sandbox = false` for speed	Trust/purity regression
Disable binary caches “to force builds” in daily driver	Wastes hours
Infinite `max-jobs` on laptop	OOM + thermal throttle
Micro-optimizing before cache hits	Measure substitute rate first
Blindly copying someone else’s `nix.conf`	Hardware differs
Timing only after `switch`	Includes service downtime noise

Theory 6 — CI budget design

Goal	Technique
PR feedback $<$ N minutes	Cache push from main; build only changed hosts if matrixed
Avoid rebuild world	Pin lock; share `/nix/store` cache between jobs carefully
nixosTest cost	Nightly `checks.slow-*`; PR runs fast subset
Fail fast	Eval/checks before ISO builds

Write target budgets in `PERF.md` even for homelab CI.

Worked example — Measurement protocol

```
# 1) Note config
nix config show | egrep 'max-jobs|cores|substituters' | tee before-config.txt

# 2) Warm caches optionally with a cheap build
nix build nixpkgs#hello

# 3) Wall-clock system build (no switch)
/usr/bin/time -p nix build \
  .#nixosConfigurations.lab.config.system.build.toplevel \
  2>&1 | tee time-before.txt

# 4) Change nix.settings via configuration.nix / flake; rebuild once to apply daemon setting

# 5) Repeat timing → time-after.txt
```

Daemon-level settings change may need:

```
sudo systemctl restart nix-daemon
```

(On NixOS after a config switch that changes `nix.settings`.)

Interpreting results

Pattern	Interpretation
Eval leaf fast, toplevel slow, lots of `building`	Real compile cost; builders/cores
Many `copying path`	Substitute/network; cache placement
Eval alone dominates	IFD/modules; not max-jobs
After setting change wall clock worse	Thrash; lower parallelism

---

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/perf
cd ~/lab/nixos/internals/perf
```

Use your Capstone/lab flake host.

Lab 1 — Baseline timings

Record in `PERF.md`:

Run	Command	Wall seconds	Notes (cold/warm)
A	eval hostname only		
B	`nix build` toplevel		
C	`nixos-rebuild build`		

```
time nix eval .#nixosConfigurations.<host>.config.networking.hostName
time nix build .#nixosConfigurations.<host>.config.system.build.toplevel --print-build-logs
```

Capture `nix config show` snapshot as `config-before.txt`.

Acceptance: Three timed runs with cold/warm labels.

Lab 2 — Tune jobs/cores

Pick **one** hypothesis, e.g. "too many parallel link jobs thrash RAM."

Apply via flake:

```
nix.settings.max-jobs = 2;
nix.settings.cores = 4;
```

Switch config (or set user `nix.conf` on non-NixOS carefully), restart daemon if needed, re-time the **same** target as Lab 1.

Document winner settings for **this machine only**.

Lab 3 — Cache hit awareness

```
# After a toplevel build, approximate download vs build by reading logs (-L)
# Count "copying path" vs "building '" lines from a log snippet
```

Optional: temporarily misconfigure a wrong extra substituter (unreachable) and observe timeout cost—then remove it. Note CI implications.

Lab 4 — Eval hygiene checklist

In `PERF.md`, audit your flake for:

- ☐ IFD sites (`import` of derivation results)
- ☐ Accidental `--impure` scripts
- ☐ Oversized `imports` on every host
- ☐ Dev shells pulled into system eval unnecessarily

Fix **one** real issue if found (or document "none found" with evidence).

Lab 5 — Remote builder decision (write-only if no hardware)

Write `BUILDERS.md`: when you would add a remote builder vs buy a better cache. Include `supportedFeatures` needed for `nixosTest` (kvm).

Lab 6 — Memory pressure drill (lab only)

While a parallel build runs:

```
# another terminal
free -h
# watch -n2 free -h
```

If swap thrash appears, lower `max-jobs` and re-time. Record decision.

Acceptance (Elitebook): Documented settings that do not OOM a 16 GB host during a typical host toplevel build.

Common gotchas

Symptom	What to do
Settings in `nix.conf` ignored	Multi-user: daemon config; restart daemon
Faster jobs, slower wall clock	RAM thrash / disk queue; lower max-jobs
Eval dominates always	Module/IFD issue, not cores
CI OOM on matrix	Cap jobs per runner
`auto` max-jobs surprises	Pin explicit numbers for servers
Timing after `switch` includes downtime	Use `build` for fair comparison
Auto-GC mid-ISO	Raise free space; schedule GC windows

Checkpoint

- ☐ Before/after timings in `PERF.md`
- ☐ Final `max-jobs` / `cores` recorded with rationale

- ☐ Cache vs build awareness notes
- ☐ One eval-hygiene audit completed
- ☐ No sandbox disabled “for speed”
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): rebuild performance measurement and tuning"
```

Write three personal gotchas before continuing.

Next

→ **Content-addressed derivations (awareness)** — IA vs CA without migrating Capstone.

Content-addressed derivations (awareness)

Content-addressed derivations (awareness)

Goal: Explain input-addressed vs content-addressed derivations, know the **experimental** status and feature flags, run a tiny optional experiment **or** produce a documented status memo with primary-source links—without putting production Capstone (or **26.05** lab hosts) on CA.

> **!** Important
>
> Content-addressed (CA) derivations remain an **experimental** path. This chapter is literacy and a sandbox experiment—not a mandate to migrate your host. Capstone A stays on default store semantics.

Why this chapter exists

Default Nix paths are **input-addressed**: the store hash fingerprints the build plan (derivations + inputs), not only the output bytes. Two builds that produce identical bytes can still get different paths if input metadata differs. CA aims at **output-hash identity**—powerful for early cutoff / better sharing—but the feature surface, UX, and nixpkgs readiness evolve. Experts need an honest mental model before blog hype.

You will hear CA in RFCs, release notes, and conference talks. Literacy prevents cargo-cult `experimental-features` on prod builders.

Theory 1 — Input-addressed world (today's default)

```
inputs + build recipe  →  hash  →  /nix/store/<hash>-name
```

Consequences you already rely on:

Property	Implication
Rebuild with same inputs	Same path (reproducible *address*)
Different compiler flags	Different path even if binary identical
FODs	Special: output hash fixed; may network
Substitutes	Trust signatures of builders/caches

Trust is "did we run this recipe (or accept a signed substitute for it)?"

FODs vs CA (do not conflate)

	Fixed-output derivation	CA derivations (experimental)
Classic use	fetches, vendoring	general builds
Hash of	declared output	content-addressed store model
Maturity	Everyday	Experimental

FODs already teach "hash the result" for a *class* of builds. CA generalizes the idea; it is not "FODs for everything tomorrow."

Theory 2 — Content-addressed idea

```
build → output bytes → hash → /nix/store/<hash>-name
```

Aspirations:

Goal	Intuition
Early cutoff	If output already exists with same content hash, skip dependent rebuilds more aggressively
Better dedup	Same bytes converge to same path across "recipes" that don't matter
FODs generalized	More builds behave like "hash the result"

Challenges:

- Self-references in outputs (files embedding their own store path) need rewriting schemes
- Granularity (per-output CA, etc.)
- Cache/protocol/nixpkgs compatibility
- Debugging mental model shift
- Tooling and UX still moving

Theory 3 — Experimental feature surface

Exact names and defaults **change across Nix versions**. Always check:

```
nix --version
nix config show | grep -i experimental
man nix.conf | less   # search ca-derivations / content-address
```

Historically you will see flags in the family of:

- `ca-derivations`
- related: `dynamic-derivations`, `cgroups`, etc. (orthogonal but often discussed together)

Enable only on a **lab** profile:

```
# ephemeral lab - not Capstone default
nix.settings.experimental-features = [
  "nix-command"
  "flakes"
  "ca-derivations"  # if supported by your Nix
];
```

Or local:

```
# illustrative - confirm flag name for your version
nix build --experimental-features 'nix-command flakes ca-derivations' ...
```

Reading status (primary sources)

Prefer in this order:

1. Nix release notes for your exact version
2. Official Nix manual sections on store / experimental features
3. nixpkgs issues/PRs tagged CA (for package readiness)
4. RFCs / discourse RFCs (design intent)

Blogs are optional commentary after primary sources. Version-skew blogs are how people enable the wrong flag.

Theory 4 — What CA does *not* magically fix

Claim	Reality
"CA = perfect reproducibility"	Reproducibility is about *determinism* of builds; CA is about *addressing*
"CA replaces sandbox"	Isolation still required
"Turn on CA, everything faster tomorrow"	Needs package + toolchain readiness; may slow or break
"Signatures no longer matter"	You still decide trust for substitutes
"Capstone should adopt CA"	**No** for this book's Capstone A

Theory 5 — Relationship to Capstone and 26.05

Capstone should stay on **stable, default** derivation mode unless you explicitly document a lab-only branch. DR wipe/rebuild proofs should not depend on experimental store semantics.

Your **nixos-26.05** pin is about *module and package* stability. CA is a **Nix daemon / store** feature layer—orthogonal but coupled in operational risk if enabled fleet-wide.

Optional: track CA as a **roadmap** item in the retrospective—revisit when your Nix pin upgrades and primary sources look greener.

Theory 6 — Trust and substitutes under CA thinking

Even with CA aspirations, operations still answer:

Question	Still true?
Who is allowed to write to my store?	Yes
Which substituters do I trust?	Yes
Do I verify signatures?	Yes (policy)
Can a malicious builder hurt me?	Yes

CA is not a substitute for binary-cache trust discipline or remote-builder trust.

Theory 7 — Operational decision template

```
## Store semantics decision
- Mode: input-addressed (default)
- CA experimental: enabled? no
- Revisit when: <Nix version + release notes criteria>
- Owner: <you>
- Rollback: remove feature flag; rebuild hosts on default
```

Paste into Capstone docs so future-you does not "just try CA" mid-freeze.

Worked example — Conceptual comparison card

Dimension	Input-addressed	Content-addressed (experimental)
Path hash from	Recipe/inputs	Output content (design)
Same bytes, different flags	Different paths	*May* converge
Ecosystem maturity	Default everywhere	Experimental / partial
Debugging familiarity	High (this book)	Lower; fewer blog runbooks
Prod recommendation (this book)	**Yes**	**Not yet** for Capstone

---

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/ca
cd ~/lab/nixos/internals/ca
```

Lab 1 — Version & feature inventory (required)

```
nix --version | tee nix-version.txt
nix config show > nix-config.txt
grep -iE 'experimental|ca-|content' nix-config.txt || true
```

Open official docs for **your** version; save URLs in `SOURCES.md` (at least 2 primary links).

Write a 10–15 line **status summary**: what CA means, whether your Nix binary advertises the feature, and any warnings in the manual.

Lab 2 — Choose a track

Track A — Optional tiny experiment (if feature available)

1. Enable `ca-derivations` (or current equivalent) **only** in a disposable VM or user `nix.conf` overlay you can revert.
2. Build a trivial derivation twice with intentional non-semantic input noise if docs show how CA modes are requested for derivations (API differs by version—follow current manual).
3. Compare store paths and `nix path-info` output.
4. Capture logs in `experiment/`.
5. **Disable** experimental CA for daily lab work if it complicates Capstone.

If the manual's API is unclear or broken on your pin, stop and switch to Track B—do not thrash for hours.

Track B — Status memo without enabling (always acceptable)

Write `CA-MEMO.md` (1 page) covering:

1. Input- vs content-addressed definitions
2. Problems CA wants to solve
3. Current experimental flags for your Nix version
4. Known limitations (self-refs, ecosystem)
5. Decision: **not** enabling on Capstone host, with revisit date (e.g. after next release notes)

6. Primary source links
7. Difference from FODs in your own words

Acceptance: Either Track A logs + revert proof, or Track B memo complete.

Lab 3 — Teach-back diagram

In `NOTES.md`, draw (ASCII is fine):

```
[user config] → eval → derivations → (IA path | CA path?) → substitute/build → store
```

Annotate where trust/signatures apply.

Lab 4 — Nixpkgs awareness scan

Search your local nixpkgs (or GitHub) for mentions:

```
# if you have nixpkgs checkout
rg -n "contentAddressed|__contentAddressed|ca-derivations" -g '*.nix' | head
```

Note whether anything in *your* critical path packages advertises CA—likely little for a lab host. Record findings.

Lab 5 — Explicit non-adoption

Add to Capstone docs (or lab notes):

```
## Store semantics
- Mode: input-addressed (default)
- CA: deferred until <date/criteria>
- Owner: <you>
```

This prevents a well-meaning future PR from flipping experimental flags mid-freeze.

Lab 6 — Analogy card (teach others)

Write three sentences max for a teammate:

1. **Input-addressed:** "The address is the recipe's fingerprint."
2. **Content-addressed:** "The address wants to be the loaf's fingerprint."
3. **Why Capstone waits:** "Experimental + ecosystem partial; DR should not depend on it."

If you cannot explain without jargon, revise `CA-MEMO.md` until you can.

Interaction with other internals chapters

Chapter	Link to CA literacy
Evaluator	Eval still produces plans; addressing is store-side
Store/daemon	Feature flags live with daemon policy
Performance	Early cutoff is the performance *story*—measure before enabling
Release notes	Watch Nix + nixpkgs notes for CA maturity signals

Do not enable CA "for speed" without a benchmark and a rollback plan for the builder.

Common gotchas

Symptom / mistake	What to do
Blog for Nix 2.X vs your 2.Y	Trust `nix --version` + matching manual
Enabled CA on shared prod builder	Revert; isolate experiments
Assuming path equality bit equality always	Still verify; trust model differs
Spending the whole session debugging experimental UX	Fall back to Track B memo
Mixing CA talk with FOD confusion	FODs already hash outputs for *fetch* class
Capstone depends on CA	Redesign; default store only

Checkpoint

- ☐ `nix-version.txt` + feature inventory
- ☐ `SOURCES.md` with primary links
- ☐ Track A experiment notes **or** Track B `CA-MEMO.md`
- ☐ Explicit Capstone decision: stay input-addressed
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): CA derivations experimental awareness"
```

Write three personal gotchas before continuing.

Next

→ **nixosTest mastery** — harden checks for Capstone.

nixosTest mastery

nixosTest mastery

Goal: Harden a real `nixosTest` (or flake `checks`) suite: replace brittle `sleep` waits, use `waitForUnit` / port / file helpers correctly, handle multi-node networking, and leave CI-ready tests for Capstone.

Why this chapter exists

Ops chapters introduced nixosTest. Capstone A **requires** at least one test that catches a real regression. Flaky tests train you to ignore CI—worse than no tests. This chapter is production hygiene for the test layer.

Theory 1 — What nixosTest really runs

`pkgs.nixosTest` / `testers.runNixOSTest` (names vary by nixpkgs era) builds QEMU VMs from NixOS configs, wires a Python test driver, and runs scripted interactions.

```
# sketch - adjust to your nixpkgs helpers
pkgs.nixosTest {
  name = "lab-proxy";
  nodes.machine = { config, pkgs, ... }: {
    imports = [ ../modules/proxy.nix ];
    my.proxy.enable = true;
    # …
  };
  testScript = ''
    machine.start()
    machine.wait_for_unit("multi-user.target")
    machine.wait_for_unit("nginx.service")
    machine.succeed("curl -f http://127.0.0.1/")
  '';
}
```

Wire into flakes:

```
checks = {
  lab-proxy = pkgs.nixosTest (import ./tests/lab-proxy.nix { inherit pkgs; });
```

```
};
```

```
nix build .#checks.x86_64-linux.lab-proxy -L
nix flake check
```

> **Note**
>
> nixpkgs has migrated helpers (`pkgs.testers.runNixOSTest`, `lib.nixos.runTest`, classic `nixosTest`). Use whatever your **26.05** pin documents; the **shape** (nodes + testScript) remains the skill.

Theory 2 — Flake (flaky) sources

Cause	Symptom	Fix
Fixed `sleep 10`	Passes on fast hosts, fails on CI	Wait on unit/port/log
Race before network	DNS/curl fail intermittently	`wait_for_open_port`, `wait_until_succeeds`
Wall-clock timeouts too tight	CI under load fails	Budget timeouts; retry patterns in driver
Shared mutable state	Order-dependent	Independent nodes; pure configs
Missing `kvm`	Slow TCG or fail	Ensure nested virt / `supportedFeatures`
Non-determinism in app	Intermittent HTTP 500	Fix app or assert eventually

---

Theory 3 — Driver API patterns (Python DSL)

Exact methods evolve; concepts stable. Prefer:

```
machine.start()
machine.wait_for_unit("network-online.target")  # if appropriate
machine.wait_for_unit("myapp.service")
machine.wait_for_open_port(8080)
machine.wait_until_succeeds("curl -f http://127.0.0.1:8080/health")
machine.succeed("systemctl is-active myapp.service")
machine.fail("curl -f http://127.0.0.1:8080/admin")  # expect fail
```

```
machine.copy_from_vm("/var/log/myapp.log", "out")
print(machine.succeed("journalctl -u myapp -n 50 --no-pager"))
```

Anti-pattern

```
machine.succeed("sleep 30")
machine.succeed("curl …")  # still racy if app slower than 30s under load
```

`sleep` as a **backoff inside** `wait_until_succeeds` is different from a single blind sleep—prefer the wait helpers.

Multi-node

```
nodes = {
  server = { … };
  client = { … };
};
```

```
server.wait_for_unit("myapp.service")
client.wait_until_succeeds("curl -f http://server/health")
```

Hostname resolution between nodes is provided by the test network—use node names as hostnames per driver docs.

Theory 4 — Systemd-aware testing

Good services for tests:

- `Type=notify` when possible (ready = really ready)
- Explicit `After=`/`Wants=` for network
- Health endpoint or `ExecStartPost` readiness

If a unit is `active` but still binding ports, wait on **port** not only unit.

```
machine.wait_for_unit("nginx.service")
machine.wait_for_open_port(80)
```

Theory 5 — Secrets and tests

Never require production sops keys for unit tests. Patterns:

Pattern	Use
Test-only plaintext in VM	OK for non-sensitive lab secrets
`sops-nix` with age key generated in test	Heavier; sometimes worth it
Mock module `mkIf config.my.testMode`	Injects dummy secrets
Skip TLS in test, test TLS in one dedicated check	Split concerns

Capstone should have **at least one** test without human interactive decryption.

Theory 6 — CI integration

```
# sketch GitHub Actions
- uses: cachix/install-nix-action@…
- run: nix build .#checks.x86_64-linux.lab-proxy -L
```

Needs:

- KVM available **or** accept TCG slowness with longer timeouts
- Binary cache for nixpkgs to avoid rebuilding QEMU world every time
- Artifacts: fail logs on failure

```
nix log .#checks.x86_64-linux.lab-proxy
```

Elitebook / KVM lab acceptance

Check	Command / note
KVM usable	`/dev/kvm` exists; user in `kvm` group if required
Nested virt if host is VM	BIOS + hypervisor CPU flags
Test finishes	`nix build .#checks… -L` exits 0 twice
Disk headroom	Leave space for VM images under store

```
ls -l /dev/kvm || echo "no kvm"
# groups | tr ' ' '\n' | grep -E 'kvm|libvirt' || true
```

Theory 7 — Machine objects and multi-node patterns

```
# conceptual multi-node
start_all()
server.wait_for_unit("nginx.service")
client.wait_until_succeeds("ping -c1 server")
client.succeed("curl -sf http://server | grep -q ok")
```

Node names become hostnames in the test network. Use them deliberately; avoid hard-coded IPs unless the test network docs say so.

Theory 8 — Snapshotting vs rebuild cost

Long tests that rebuild the world each run train you to skip them. Techniques:

Technique	Effect
Smaller module under test	Faster eval/build
Shared fixture attrs	Less duplication
Separate `checks.slow-*`	CI nightly vs PR

Mastery is green **and** runnable.

Worked example — systemd journal assertion

```
machine.wait_for_unit("lab-tick.service")
machine.succeed("journalctl -u lab-tick.service -n 20 | grep -q tick")
```

Worked example — regression that fails when firewall wrong

```
machine.wait_for_open_port(443)
# backend should not be world-open if design is proxy-only
machine.fail("curl -f --max-time 2 http://127.0.0.1:8080/")
```

Adjust ports to your design; the skill is asserting **policy**, not only liveness.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/tests
cd ~/lab/nixos/internals/tests
```

Prefer extending your **real** flake checks; if missing, create a minimal test module now.

Lab 1 — Inventory existing tests

```
nix flake show
# list checks
rg -n "nixosTest|runNixOSTest|testScript" -g '*.nix' /path/to/flake
```

In `SUITE.md`, list each check: purpose, flake attribute, known flakes.

Lab 2 — Eliminate sleep-only waits

Find `sleep` in test scripts:

```
rg -n "sleep" -g '*test*' /path/to/flake
```

Replace with `wait_for_unit` / `wait_for_open_port` / `wait_until_succeeds`. Re-run:

```
nix build .#checks.<system>.<name> -L
```

Run **twice** consecutively—tests that pass once and fail once are not done.

Acceptance: Critical path has no blind `sleep` as sole wait; two consecutive greens.

Lab 3 — Add one regression that would have bitten you

Pick a real past bug (wrong firewall port, typo in upstream, module `mkIf` inverted). Write a test that fails if that bug returns.

Lab 4 — Multi-node or client/server (stretch)

If single-node only so far, add a second node that curls the first **or** document why single-node is enough for Capstone A and add a second assertion class instead (journal + HTTP).

Lab 5 — Failure UX

Break the test on purpose (wrong unit name). Confirm the log shows journal excerpts. Add `machine.succeed("journalctl -u …")` on failure paths if helpful. Record debugging time in `SUITE.md`.

Lab 6 — Flake check wiring audit

```
nix flake show
nix build .#checks.$(nix eval --raw --impure --expr 'builtins.currentSystem') -L 2>/dev/null
  nix flake check -L
```

List every check attr and its purpose in one table.

Lab 7 — Failure screenshot discipline

When a test fails, save:

1. Full command
2. Last 50 log lines
3. Whether rerun without change flakes

Flaky → fix waits; deterministic → fix system under test.

Common gotchas

Symptom	What to do
`error: unable to start guest`	KVM permissions, nested virt off
Hangs at `wait_for_unit`	Unit failed; print `systemctl status` / journal
Works interactively, fails CI	Timeouts; resource; sandbox/kvm
TLS/ACME tests fail offline	Don't use real ACME in VM; use self-signed or http-only mode
Eval of test is huge	Share modules carefully; don't import desktop profiles
Flake check runs *all* checks	Tag slow tests; `nix build` specific attr in iteration
Secrets block CI	Dummy secrets / testMode

Checkpoint

- ☐ Suite inventory in `SUITE.md`
- ☐ No critical path depends on blind `sleep` alone
- ☐ Test run twice green
- ☐ At least one regression-oriented assertion
- ☐ Notes on secrets strategy in tests
- ☐ KVM/lab acceptance noted
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): harden nixosTest suite for capstone"
```

Write three personal gotchas before continuing.

Next

→ **Ecosystem map** — try one adjacent tool; keep/drop without Capstone rewrite.

Ecosystem map

Ecosystem map

Goal: Map the adjacent tooling landscape, **try exactly one** of devenv / Snowfall / nix-darwin in a disposable trial, and write a keep/drop decision against **your** flake layout—without rewriting Capstone mid-flight.

> **Note**
>
> Syllabus rule: **try one, don't collect all.** Tool tourism is how deep curricula die mid-book.

Why this chapter exists

By now your layout is intentional: flake hosts, modules, HM, deploy, tests. The ecosystem constantly offers frameworks that promise less boilerplate. Some are worth it; many are expensive rewrites. Experts sample with a **comparison table**, not a weekend migration of Capstone main.

Theory 1 — Layers of "ecosystem"

Layer	Examples	Problem they target
Devshell UX	devenv, devbox, `nix develop`, direnv	Fast project environments
Flake structure frameworks	flake-parts, Snowfall, digga (legacy)	Scale modules/systems/homes
Darwin / non-NixOS	nix-darwin, HM standalone	Same declarative story on macOS
Deploy	deploy-rs, Colmena, nixinate	Ops & fleet part
Secrets	sops-nix, agenix	Services & security part
Caches	Cachix, Attic, Harmonia	Packaging / ops
Install / metal	nixos-anywhere, disko, images	Ops & fleet
UX wrappers	nh, nixos-rebuild-ng experiments	Faster day-2 commands
Editors / templates	templates repos, blueprint flakes	Speed of scaffolding

Today's forced choice is among **devenv**, **Snowfall**, **nix-darwin**—three different layers. Pick the one that matches *your* pain:

If your pain is…	Prefer trying
Polyglot project shells, services in dev	**devenv**
Many hosts/homes, naming convention fatigue	**Snowfall** (or double-down on flake-parts you may already use)
You live on macOS half the time	**nix-darwin**
None of the above	Still pick one; learn by contrast

Theory 2 — devenv (project environments)

What it is: A Nix-powered dev environment tool (CLI + modules) focused on languages, processes, pre-commit, and local services—closer to "compose for dev" than to NixOS hosts.

Typical win: New contributors run one tool and get DB + language toolchain.

Typical cost: Another module system dialect; CI must speak devenv; overlap with `devShells` you already have.

```
# illustrative - follow current devenv docs for install
# nix profile install nixpkgs#devenv
# devenv init
# devenv shell
```

Compare to house default

Concern	`nix develop` + direnv	devenv
Mental model	Flake `devShells`	devenv modules + CLI
Services (postgres…)	Manual / process-compose DIY	First-class modules often
Capstone impact	Low if already working	Medium if dual standards

---

Theory 3 — Snowfall (flake library / structure)

What it is: A convention-heavy library for laying out flakes: systems, modules, packages, homes under standard paths, auto-discovery.

Typical win: Multi-host monorepos with consistent naming and less flake.nix glue.

Typical cost: Framework lock-in; debugging indirection; migration of a working hand layout.

Compare to flake-parts

Concern	Hand layout / flake-parts	Snowfall
Structure	Explicit imports	Convention + library
Learning	Your code is visible	Magic paths
Capstone rewrite cost	Low if already structured	Often high mid-flight

Snowfall and flake-parts both attack "structure," differently. Trying Snowfall does **not** require abandoning flake-parts forever—decision is local.

Theory 4 — nix-darwin

What it is: Nix modules for macOS system configuration—spiritual cousin of NixOS modules, not a Linux VM.

Typical win: One repo language for laptop (Darwin) + servers (NixOS) + Home Manager.

Typical cost: Darwin modules NixOS modules; GUI/casks world; CI harder; not a substitute for your lab VM.

```
# only on a Mac you own - follow current nix-darwin + flakes docs
# darwin-rebuild switch --flake .#hostname
```

If you have no Mac, you may still **read** a nix-darwin sample config and write a comparison—or pick devenv/Snowfall for a hands-on trial instead.

Shared idea	Divergence
Module options, activation	Different option trees
HM integration common	GUI apps / Homebrew friction
Flakes	Same pin discipline

Theory 5 — Decision framework (keep/drop)

Score 1–5 for your context:

Criterion	Weight idea
Reduces *your* real weekly pain	High
Clear primary docs	High
Plays with flakes + lock discipline	High
Compatible with sops/deploy/tests	High
Rewrite cost of Capstone	**Critical penalty**
Bus factor / maintenance	Medium
Fashion / social media mindshare	**Zero**

Default Capstone policy: adopt only if score says "yes" **and** migration is a branch, not a rewrite of main before disaster recovery proof.

```
decision  { keep-current, adopt-later, trial-only-drop }
```

Theory 6 — Ecosystem postcard (vocabulary)

You need names for comparative conversations even without installs:

Name	One-line purpose
flake-parts	Modular flake output composition
nixos-anywhere	Remote install from existing SSH
nh	Friendlier rebuild/search UX
Attic	Self-hosted binary cache
digga	Legacy flake framework (historical literacy)
blueprint / templates	Scaffold speed
nixos-generators	Image formats from NixOS configs
Lanzaboote	Secure Boot UKI/signing path

15 lines total in notes is enough—depth belongs to the **one** trial.

Theory 7 — Integration risks matrix

Trial tool	Risk to secrets	Risk to deploy	Risk to tests
devenv	Low	Low	Low (CI may need devenv)
Snowfall	Medium (path moves)	Medium	Medium (check wiring)
nix-darwin	Low on Linux lab	N/A for NixOS hosts	Separate CI

Never "finish" a trial by force-pushing Capstone restructure without a branch review.

Worked example — COMPARE.md skeleton

```
# COMPARE.md - devenv vs house layout (example)

## Context
- Capstone flake with nixosConfigurations.lab, sops-nix, deploy-rs
- Pain: polyglot shells inconsistent across repos

## Scores (1-5)

| Criterion | House | devenv | Weight |
|-----------|-------|--------|--------|
| Weekly pain relief | 2 | 4 | 5 |
| Docs clarity | 4 | 4 | 4 |
| Flake/lock fit | 5 | 4 | 5 |
| sops/deploy/tests | 5 | 3 | 5 |
| Capstone rewrite cost | 5 | 2 | 5 |

## Decision
We will **not** adopt before Capstone freeze because rewrite cost outweighs shell pain;
revisit after Capstone freeze if side projects still hurt.

## Evidence from trial/
- commands run:
- what broke:
- time spent:
```

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/ecosystem/trial
cd ~/lab/nixos/internals/ecosystem
```

Do **not** trial on the only copy of Capstone; branch or separate directory.

Lab 1 — Map your current layout (30 min)

In `LAYOUT.md`:

```
flake.nix outputs:
  nixosConfigurations: …
  homeConfigurations / HM-as-module: …
  devShells: …
  checks: …
  packages: …
  deploy / colmena: …
modules/: …
hosts/: …
secrets/: …
```

List top 3 pains (e.g. “shells inconsistent”, “host boilerplate”, “no Mac story”).

Lab 2 — Pick one tool and trial

If devenv

1. `devenv init` in `trial/app` or import docs’ minimal flake.
2. Add one language toolchain you actually use.
3. Optional: one service (postgres/redis) if docs make it easy.
4. Enter shell; run a real command (tests/build).
5. Note direnv integration if any.
6. Time spent + friction log.

If Snowfall

1. Scaffold a **tiny** Snowfall flake (one system stub, one module).
2. Do not migrate all hosts.
3. Trace: where does a host definition live? How is it discovered?
4. Compare to your `hosts/lab/default.nix` pattern.
5. Eval something small; capture errors as learning.

If nix-darwin

1. On Mac: flake with one `darwinConfigurations`; apply to a **throwaway** aspect if possible.
2. Or read-only deep dive: clone an example, map options vs NixOS cousins.
3. Document what cannot be shared as-is with NixOS modules.

Timebox: 2–4 hours max for trial hands-on.

Lab 3 — Comparison table (required)

Complete `COMPARE.md` per Theory 5–worked example. Include explicit paragraph:

"We will / will not adopt before Capstone freeze because..."

Lab 4 — Ecosystem postcard

In 15 lines, define in your words: flake-parts, nixos-anywhere, nh, Attic. No install required.

Lab 5 — Capstone protection check

```
# Capstone main should be untouched by trial frameworks
cd /path/to/capstone-flake
git status
git log --oneline -5
```

If you "accidentally" started migrating main, revert or move work to a branch named `experiment/…` and restore main to last known good.

Acceptance: `git status` clean on Capstone main regarding trial frameworks; trial lives under `trial/` or `experiment/*` branch.

Lab 6 — Teach-back (5 bullets)

Explain to a peer:

1. What pain you tried to solve
2. What you tried
3. Score highlight
4. Decision
5. Revisit criteria

Common gotchas

Mistake	What to do
Trying all three "a little"	Violates the chapter; pick one
Rewriting Capstone main branch	Use `trial/` only
Adopting for aesthetics	Score against pain list
devenv *and* heavy custom shells forever duplicated	Choose a house standard
Assuming Snowfall NixOS	It's flake structure, not an OS
nix-darwin as lab substitute	Still need Linux NixOS for Capstone
Infinite template shopping	Timebox trial to hours, not days

Checkpoint

- ☐ `LAYOUT.md` + top pains
- ☐ One tool trial notes or read-only darwin depth
- ☐ `COMPARE.md` with keep/drop decision
- ☐ Ecosystem postcard vocabulary
- ☐ Capstone main layout untouched (or only trivial fixes)
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): ecosystem trial and keep/drop decision"
```

Write three personal gotchas before continuing.

Next

→ **Comparative landscape** — steelman alternatives to NixOS.

Comparative landscape

Comparative landscape

Goal: Write an honest, concrete trade-off note comparing NixOS to Guix System, image-based/immutable distros, and Ansible-style config management—grounded in **your** workloads, not tribal slogans.

Why this chapter exists

If you cannot say when NixOS is the wrong tool, you do not understand it. Capstone commitment is real effort; this chapter is the "steelman the alternatives" work so your retrospective is adult, not fanfic.

Theory 1 — Comparison dimensions

Score systems on dimensions that matter to operators:

Dimension	Question
Reproducibility	Can two machines converge from the same artifact/definition?
Atomic rollback	Bad update → previous generation quickly?
Abstraction	Packages + services + users in one language?
Learning curve	Time to first safe production change
Ecosystem breadth	Packaging coverage, cloud images, vendor support
Secrecy & compliance	Secrets, audit, SBOM stories
Day-2 ops	Observability, patches, multi-node
Escape hatches	Can you drop to imperative when burning?
Team skill fit	What does your org already know?
Mac/desktop story	Not everything is a Linux server
Migration cost	From *your* current estate

No system wins all columns. Weighted scores beat vibes.

Theory 2 — NixOS (your baseline)

Strengths you have felt (cite your own chapters):

- Declarative modules + generations
- Flake pins for inputs
- Same language from package to OS
- Binary caches + remote builders
- Strong "rebuild from repo" story
- sops-nix / deploy-rs / Colmena / nixosTest ecosystem

Weaknesses you should admit:

- Steep language + module learning curve
- Eval/build complexity; footguns (IFD, priorities, infinite recursion)
- Docs fragmented across eras (channels vs flakes)
- Some proprietary/vendor agents painful to package
- GUI desktop polish varies by investment
- Experimental features (CA derivations, etc.) not free wins
- Team hiring: fewer "NixOS natives" than Ansible generalists

Write two scars from **your** lab, not from memes.

Theory 3 — Guix System

What it is: GNU Guix—functional package manager + Guix System; Scheme (Guile) instead of Nix language; strong free-software ethos; similar store/generation ideas.

Topic	Guix-ish shape	NixOS-ish shape
Language	Scheme	Nix
Init	shepherd	systemd
Channels / pins	channels	flakes/channels

Topic	Guix-ish shape	NixOS-ish shape
FHS / binary blob culture	Often stricter free software	Pragmatic unfree options common
Community size	Smaller	Larger
Commercial surface	Smaller	Larger blog/job footprint

When Guix may win: Scheme fluency, GNU purity goals, shepherd preference, research alignment.

When NixOS may win for you: systemd ecosystem, larger package set, more operational write-ups, your already-built Capstone investment.

You do **not** need to install Guix today—read primary docs enough to avoid strawmen. If you already know Scheme, say so in the essay.

Theory 4 — Image-based / immutable distros

Examples of the *family* (not a full product review): Fedora CoreOS / Silverblue-style, Bottlerocket, Talos, some appliance OS designs—**image A/B**, ostree or similar, workloads often in containers.

Topic	Image-based	NixOS
Unit of deploy	OS image	Config evaluation → system closure
App delivery	Containers/k8s common	Packages + modules + optional containers
Rollback	Previous image slot	Generations
Host customization	Minimal by design	Extreme (sometimes too much)
"Pets"	Discouraged	Possible (homelab pets common)
Provisioning	Ignition/cloud-init class tools	Disko, nixos-anywhere, images

When images win: Large fleets of identical minimal nodes, Kubernetes workers, locked-down appliances, org already standardized on container platforms.

When NixOS wins: Highly specialized hosts, cross-cutting host services, desire for one repo to express weird topology without baking images every change (though NixOS can also build images).

Hybrid reality: NixOS **build** of images + k8s workers is a valid architecture—not purity cosplay.

Theory 5 — Ansible (and friends)

What it is: Imperative/convergent config management over SSH; YAML playbooks; enormous module library; agentless common. Cousins: Salt, Puppet, Chef—this comparison centers Ansible as the industry default many teams already run.

Topic	Ansible	NixOS
Model	Converge state with tasks	Specify desired system; build closure
Partial apply	Easy, sometimes too easy	Whole-system generations
Drift	Easy without discipline	Harder if you only use nix
Secrets	Vault, etc.	sops/agenix + store discipline
Non-NixOS estate	Excellent	Weak (HM/darwin partial)
Rollback	You build it	Built-in generations
Network gear / mixed OS	Strong ecosystem	Not the point

Hybrid reality: Many orgs use Terraform/cloud for create, Ansible for classic distros, NixOS for greenfield highly automated slices.

When Ansible wins: Heterogeneous fleets (Ubuntu + network gear + weird appliances), existing playbook investment, teams without Nix skills, "change this one file on 50 boxes" urgency.

When NixOS wins: Greenfield Linux hosts you fully own, strong need for rebuildable closures, rollback as a product feature, package+service unity.

Theory 6 — Honesty checklist (avoid tribal takes)

Bad argument	Better
"Ansible is YAML spaghetti, Nix is pure."	"Our fleet is 200 Ubuntu boxes with existing roles; migration cost exceeds benefits this year."
"Guix is just Nix with Scheme."	"Shared functional ideas; different language, init, and community trade-offs."
"Immutable OS means no SSH ever."	"SSH policy is orthogonal; image distros still need day-2 access models."
"NixOS can't do containers."	"NixOS can; sometimes image-first platforms optimize for that path harder."

Bad argument	Better
“Nix solves secrets.”	“sops-nix helps; key custody and policy still human.”

Theory 7 — Decision time-bounds

Prefer time-bounded decisions:

```
Through next nixpkgs release: NixOS Capstone lab remains source of truth for learning hosts.
Ubuntu estate at $work: Ansible remains unless a greenfield slice appears.
Re-evaluate Guix: only if Scheme project requires it.
```

Eternal loyalty is not an engineering output.

Worked example — weighted scorecard fragment

```
Context: solo homelab + one day-job Ubuntu fleet I don't control.

| Dimension | Weight | NixOS | Guix | Image-family | Ansible |
|-----------|--------|-------|------|--------------|---------|
| Reproducibility | 5 | 5 | 5 | 4 | 2 |
| Rollback | 5 | 5 | 5 | 4 | 2 |
| Learning curve | 3 | 2 | 2 | 3 | 4 |
| Team skill (job) | 4 | 1 | 1 | 3 | 5 |
| Specialized hosts | 5 | 5 | 4 | 2 | 3 |
```

Compute crude weighted sums; note uncertainty bands (“±1 on Guix—I read, didn't run”).

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/compare
cd ~/lab/nixos/internals/compare
```

Lab 1 — Primary sources (skimming with intent)

Collect links in `SOURCES.md` (official or project docs, not only Reddit):

1. Guix System manual intro / features
2. One image-based OS docs page (e.g. CoreOS update model or Talos architecture)
3. Ansible "how it works" / idempotence intro

Spend ~45–60 min reading, not installing everything. Optional: one throwaway VM if time remains—**not** required.

Lab 2 — Scorecard for *your* context

`SCORECARD.md` — rate NixOS / Guix / Image-family / Ansible 1–5 on Theory 1 dimensions. Add "Weight for me." Compute weighted sum. Note uncertainty.

Define **your** context in 5 bullets (homelab, job, course-only, multi-OS, etc.).

Lab 3 — Essay (required)

`ESSAY.md` — 400–800 words:

1. Where NixOS clearly wins for you (with concrete examples from this book).
2. Where it loses or is overkill.
3. One scenario you would pick Ansible or image-based instead.
4. Whether Guix is interesting to you personally (even if "not now").
5. What you will *not* rewrite after this essay (protect Capstone).

Tone: steelman alternatives; no meme warfare.

Lab 4 — Elevator version

Compress the essay into a 90-second talk outline (bullet list) for a skeptical SRE. Save as `ELEVATOR.md`.

Include one sentence: "Here's when I would **not** use NixOS…"

Lab 5 — Capstone implication

One short paragraph in `IMPLICATIONS.md`: does this change Capstone A scope? (Default: **no**—essay is clarity, not a rewrite trigger.)

Lab 6 — Scenario cards (forced choice)

Write three short cards (5–8 sentences each) in `SCENARIOS.md`. For each, pick a primary tool and justify with dimensions from Theory 1:

Card A — Homelab "weird topology"

One edge proxy, one data host, sops secrets, occasional GPU box. You already have Capstone muscle.

Card B — Employer fleet

80 Ubuntu VMs, existing Ansible roles, two network engineers who refuse new DSLs, compliance questionnaire next quarter.

Card C — Kubernetes worker pool

Stateless nodes, image A/B updates preferred by platform team, apps only in containers, host should be minimal.

Do not use these cards to reopen Capstone A scope. They exist to prove you can **switch criteria with context**.

Lab 7 — Quote hygiene

From your primary sources, copy **one** short quotation or paraphrased claim (with URL) that surprised you for each of: Guix, image-based, Ansible. In `SOURCES.md`, note whether you agree after Capstone experience.

This blocks empty essays that restate social media without contact with docs.

Worked example — elevator closer

```
NixOS wins for me when I fully own the Linux host and need rebuildable
closures plus rollback. I would not force it onto our Ubuntu estate this
year-the migration cost dwarfs the benefit. Image-based OS wins for
identical k8s workers. Guix is intellectually close but not my ops path.
Questions?
```

Adapt to your scorecard; keep under 90 seconds spoken.

Common gotchas

Mistake	Fix
Comparing marketing to your scars	Use Capstone experience as data
Installing four OSes today	Read + score; optional one VM if time
Declaring eternal loyalty	Prefer time-bounded decisions
Ignoring team skills	Skills are a real dimension
Strawmanning Guix without reading	Primary sources in SOURCES.md
Using this chapter to restart layout thrash	Protect Capstone; ecosystem chapter already covered tools

---

Checkpoint

- ☐ `SOURCES.md` with primary links
- ☐ Weighted `SCORECARD.md`
- ☐ `ESSAY.md` 400–800 words, honest trade-offs
- ☐ `ELEVATOR.md` including when *not* NixOS
- ☐ Capstone protection note
- ☐ Three scenario cards
- ☐ One primary-source surprise note per alternative
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): comparative landscape essay and scorecard"
```

Write three personal gotchas before continuing.

Next

→ **Reading release notes** — IMPACT method on official notes.

Reading release notes (26.05)

Reading release notes (26.05)

Goal: Read the **official** NixOS 26.05 "Yarara" release notes (and related nixpkgs/Nix notes as needed), then produce an annotated impact checklist for **your** flake—not a blog summary.

> **!** Important
>
> Baseline for this book: NixOS / nixpkgs **26.05**. Primary source beats social media threads. If a note conflicts with a blog, trust the release notes + your eval.

Why this chapter exists

Upgrades fail when people skip release notes and only chase build errors one by one. The upgrade planning chapter plans 26.05→next; this chapter teaches the **method** on the release you should already be targeting, and capture breakages that still lurk in your modules.

Reading notes is an operator skill equal to writing modules.

Theory 1 — What "a NixOS release" actually is

Artifact	Role
nixpkgs branch `nixos-26.05`	Package + module tree for the release
Release notes	Human-facing breaking changes, renames, deprecations
State version `system.stateVersion`	Migration behavior for stateful options
ISO / channel artifacts	Install media (flakes pin git/github instead)
Nix version shipped/tested	Sometimes separate from nixpkgs cadence
flake.lock	Your actual pinned revisions—not the marketing branch name alone

Flakes pin **revisions**. Your `flake.lock` may already be on 26.05 while your brain still follows older blog habits.

```
nix flake metadata
# note nixpkgs locked rev / ref / lastModified
```

```
nix eval --raw .#nixosConfigurations.lab.pkgs.lib.version 2>/dev/null || true
# or inspect lock:
# jq '.nodes.nixpkgs' flake.lock
```

Theory 2 — How to read release notes like an operator

Do not read linearly once and forget. Use passes:

Pass A — Inventory (30–45 min)

Skim sections:

- Breaking changes
- Notable renames / option moves
- Service-specific notes for software **you** run
- Deprecated options
- Kernel / boot / systemd highlights
- Security-relevant defaults
- NixOS module system / installer notes

Pass B — Personal filter (45–60 min)

For each note, tag:

Tag	Meaning
`N/A`	You don't use it
`WATCH`	Indirect risk
`ACTION`	Must change config
`DONE`	Already handled
`VERIFY`	Needs rebuild/test to confirm

Pass C — Prove with eval/build

```
nixos-rebuild build --flake .#<host>
# or
nix build .#nixosConfigurations.<host>.config.system.build.toplevel -L
```

Search your repo for renamed options:

```
rg -n "oldOption|services\.foo" -g '*.nix'
```

Pass D — Warnings are future breakages

```
evaluation warning: …
trace: obsolete option …
```

Paste warnings into `IMPACT.md` as VERIFY/ACTION. Next release often hard-errors them.

Theory 3 — `stateVersion` is not a fashion number

```
system.stateVersion = "26.05"; # example - only change with understanding
```

`stateVersion` tells modules how to migrate **on-disk state defaults**. Blindly bumping it to "match release" can break stateful services (Postgres data dirs, user settings, etc.). Release notes and module comments matter more than aesthetics.

Rule for this book: record your current `stateVersion`; do **not** bump it casually while reading notes. If notes require a migration path, put it on the upgrade planning runbook.

Action	When
Leave unchanged	Default
Bump with migration plan	Notes + module docs say so; backups ready
Mismatch with nixpkgs branch	Allowed and common; understand why

---

Theory 4 — Cross-check Nix itself

Your syllabus pins modern Nix (book baseline mentions **2.34.x** class—**verify yours**). Release notes for **Nix** (the evaluator/daemon) are separate from NixOS notes.

```
nix --version
nix config show | rg -i 'experimental|system|substituter' || true
```

If CI uses a different Nix than your laptop, note the skew in the checklist—flake eval can differ on experimental defaults and sandbox settings.

Surface	Record
Laptop Nix	version
CI Nix	installer action / version
Lab host Nix	from `nix --version` on host

Theory 5 — Module renames and aliases (kinds of change)

Common historical patterns (examples of *kinds*, not a complete 26.05 dump—you must read the real notes):

Kind	Example pattern	Response
Option rename	`services.foo` → `services.foo-ng`	Update; don't live on aliases forever
Nested `settings`	freeform attr moved under `settings`	Follow module type
Package rename	overlay breaks	Fix overlay attr
Default flip	`enable` default changed	Explicit `mkForce`/set
Kernel package set	boot fails on old nvidia/out-of-tree	Pin or update module
Removed service	module gone	Replace or package yourself

When notes say an option is renamed, prefer the new path. Aliases expire.

Theory 6 — Inputs beyond nixpkgs

Your flake may pin:

Input	Why check notes/changelogs
home-manager	`release-26.05` pairing
sops-nix	module option moves
deploy-rs / colmena	activation quirks

Input	Why check notes/changelogs
disko	device option changes
lanzaboote	SB flow changes

Release notes for NixOS are necessary but not sufficient. Skim changelogs for **your** inputs when IMPACT is large.

```
nix flake metadata
# for each input: note ref + whether it follows nixpkgs
```

Theory 7 — Severity and scheduling

Tag	SLA for lab Capstone
ACTION (eval broken)	Fix before next switch
ACTION (runtime risk)	Fix before DR proof
VERIFY	Schedule rebuild this week
WATCH	Note in upgrade seeds
N/A	Drop from working set

Do not turn notes reading into a two-week refactor of modules you do not run.

Worked example — IMPACT.md row

```
| Note / change | Tag | Repo paths | Action | Owner/date |
|---------------|-----|------------|--------|------------|
| services.foo renamed to services.foo-ng | ACTION | modules/foo.nix:12 | rename option; reb
| postgres default major | WATCH | hosts/lab/db.nix | verify stateVersion; backup | upgrade
| plasma something | N/A | - | no desktop | - |
| evaluation warning: obsolete bar | VERIFY | modules/common.nix | rebuild log line 480 | to
```

Aim for **every ACTION and VERIFY** item to reference a file path or explicit N/A reason.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/release-notes/notes-raw
cd ~/lab/nixos/internals/release-notes
```

Lab 1 — Capture current pin

```
cd /path/to/your/flake
nix flake metadata | tee ~/lab/nixos/internals/release-notes/metadata.txt
nix --version | tee ~/lab/nixos/internals/release-notes/nix-version.txt
rg -n "stateVersion" -g '*.nix' | tee ~/lab/nixos/internals/release-notes/stateVersion-hits.
```

Optional lock peek:

```
jq -r '.nodes.nixpkgs.original,.nodes.nixpkgs.locked' flake.lock \
  | tee ~/lab/nixos/internals/release-notes/nixpkgs-lock.txt
```

Lab 2 — Official notes intake

1. Open the official NixOS 26.05 release notes (nixos.org manual / nixpkgs `nixos/doc/manual` release notes for 26.05).
2. Save the URL in `SOURCES.md`.
3. Export or copy key sections into `notes-raw/` if you want offline annotation (optional).
4. Create `IMPACT.md` with the table from the worked example.

Also add:

- URL for any Nix release notes you consulted
- HM release notes if you use HM

Lab 3 — Repo audit scripts

```
cd /path/to/your/flake
rg -n "mkEnableOption|services\.|programs\." -g '*.nix' | head -n 100 \
  | tee ~/lab/nixos/internals/release-notes/options-sample.txt
# search for deprecated names you found in notes:
# rg -n "deprecatedOptionName" -g '*.nix'
```

For each ACTION, either:

- fix now on a branch, or
- file as upgrade-planning backlog with severity (blocker / nice)

Avoid giant drive-by refactors while annotating—**annotate first**; fix blockers only.

Lab 4 — Build verification

```
nix build .#nixosConfigurations.<host>.config.system.build.toplevel -L \
  2>&1 | tee ~/lab/nixos/internals/release-notes/build.log
```

If warnings appear, paste them into `IMPACT.md` as VERIFY/ACTION.

Run relevant checks:

```
nix build .#checks.<system>.<name> -L \
  2>&1 | tee ~/lab/nixos/internals/release-notes/check.log
```

Acceptance: Build log captured; every evaluation warning triaged.

Lab 5 — Personal "gotchas from 26.05"

Write **5 bullets** in `IMPACT.md` under "What I almost missed." These feed troubleshooting and the retrospective.

Also write `stateVersion` policy in one sentence:

```
## stateVersion policy
We run stateVersion = "…" on host lab. We will not bump it without: backup, notes citation,
```

Lab 6 — Feed upgrade planning

Create `UPGRADE-SEEDS.md` with:

Item	Why it matters for next upgrade
ACTION leftovers	Must clear or plan
WATCH services	Higher test priority
CI Nix skew	Align before upgrade PR

Do not perform the upgrade in this chapter unless you deliberately expand scope.

Common gotchas

Mistake	Fix
Only reading unofficial summaries	Start from official notes
Bumping `stateVersion` for fun	Read migration semantics first
Assuming lock is 26.05 without checking	`nix flake metadata`
Ignoring warnings that aren't errors	Warnings become breakages next release
Giant drive-by refactors while annotating	Annotate first; fix blockers only
Forgetting HM/sops inputs	Check paired release branches
Treating blog tips as canonical	Primary notes + your build log

---

Checkpoint

- ☐ `metadata.txt` + `nix-version.txt`
- ☐ Official 26.05 URL in `SOURCES.md`
- ☐ `IMPACT.md` tagged N/A/WATCH/ACTION/DONE/VERIFY
- ☐ Build (and checks) log captured
- ☐ `stateVersion` policy recorded
- ☐ Five "almost missed" bullets

- ☐ Upgrade seeds noted
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): annotate 26.05 release notes against flake"
```

Write three personal gotchas before continuing.

Next

→ **Upgrade planning** — dry-run runbook to the next release.

Upgrade planning

Upgrade planning

Goal: Produce a complete **dry-run** upgrade runbook from 26.05 → 26.11 (next cadence release): input bumps, test gates, staged rollout, rollback, and risk register—**without** requiring a production upgrade today.

> **Note**
>
> NixOS releases target roughly **May/November** cadence (`.05` / `.11`). 26.11 may be upcoming, in beta, or stable depending on when you read this. The deliverable is the **plan**; execute only on lab when ready.

Why this chapter exists

Release-note literacy without a runbook still produces Friday-night lock bumps. Upgrades are projects: branch, gates, lab first, rollback written cold.

Theory 1 — What actually changes in a flake upgrade

```
flake.nix inputs (nixpkgs url/ref)
    ↓
flake.lock (revs, narHashes)
    ↓
eval modules against new nixpkgs
    ↓
realize new toplevel
    ↓
switch / reboot if kernel
    ↓
stateful service migrations (stateVersion-aware)
```

Step	Failure mode
Lock bump	Unrelated input moves (HM, sops, deploy-rs) break together
Eval	Option removed/renamed; type errors
Build	Package broken; FODs; your overlay

Step	Failure mode
Switch	Service fails activation
Runtime	Data migration, socket changes, API versions
Reboot	Bootloader/kernel/initrd issues

Theory 2 — Staging strategy

Environments

Stage	Purpose
Dev laptop / lab VM	First eval/build/test
CI on branch	Checks + cache populate
Non-prod server	Real networking/secrets
Prod	Last; maybe canary host

Git strategy

```
main (26.05 pin)
    branch upgrade/26.11
         bump nixpkgs only
         fix eval
         bump companion inputs (HM, etc.)
         PR with checklist
```

Avoid one commit that bumps six inputs and refactors modules.

Companion inputs

Home Manager, sops-nix, deploy-rs, disko, agenix—each has branches/tags tracking nixpkgs. Read their upgrade notes. Mismatched HM vs nixpkgs is a classic footgun.

Theory 3 — Tests-first gate

Minimum gates before `switch` on anything important:

```
nix flake check
nix build .#nixosConfigurations.<host>.config.system.build.toplevel
nix build .#checks.<system>.<integration>
# deploy to lab only:
# deploy .#lab -- or colmena apply --on lab
```

Optional: `nixos-rebuild build-vm` / smoke VM.

Policy: red checks no promote.

Theory 4 — Rollback is part of the plan

Layer	Mechanism
Boot	Previous generation in bootloader
Runtime	`nixos-rebuild switch --rollback` / `rollback` generation
Git	Revert lock on branch
Secrets	Keys unchanged; ciphertext may need re-encrypt only if tooling changes
Data	DB dumps / zfs snapshots **before** upgrade

Document **RTO** goals for lab vs prod. Capstone disaster recovery practices wipe/rebuild and rollback drills.

Concrete rollback commands (fill for your path)

```
# local rebuild host
sudo nixos-rebuild switch --rollback
# list generations
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
# deploy-rs / colmena: redeploy previous known-good commit
git revert <lock-bump-commit>
# then redeploy
```

Theory 5 — `stateVersion` and stateful services

Upgrading nixpkgs automatically bumping `stateVersion`.

Plan section required:

- Current `stateVersion`
- Whether next release notes require migration
- Snapshot/backup steps for PostgreSQL, Vaultwarden, etc. you actually run
- Who verifies app-level health post-switch

Theory 6 — Communication & windows

Even for homelab:

- Maintenance window
- What breaks for family/users
- Where logs live
- Abort criteria (e.g. “if proxy down >15m, rollback”)

Abort criteria template

Signal	Action
Eval/build red on CI	Do not deploy
`systemctl --failed` non-empty after switch	Investigate 15m then rollback
Proxy healthcheck fail	Rollback
No console / no SSH	Prefer prevention; fix access before upgrade

---

Constraint	Plan adjustment

Theory 7 — Elitebook / small-lab constraints

Constraint	Plan adjustment
16 GB RAM	Lower `max-jobs` during upgrade builds
Limited disk	GC old generations **after** successful upgrade, not mid-build; ensure free space for two closures
Single physical host	Prefer guest upgrade first; snapshot VM
Nested virt	Confirm KVM still works after kernel bump

Runbook template (you will fill this)

Create `UPGRADE-26.11.md` with these sections:

0. Metadata

- Author, date, target hosts
- Current pins (`flake.lock` nixpkgs rev)
- Target ref (`github:NixOS/nixpkgs/nixos-26.11` or unstable interim)

1. Preconditions

☐ Backups verified restorable

☐ CI green on main

☐ Disk space for two closures

☐ Access to console/IPMI if remote

☐ sops/age keys available offline

2. Release notes

- Links to 26.11 notes (when published) + diff against your 26.05 `IMPACT.md` method
- ACTION items list

3. Technical steps (lab first)

```
git checkout -b upgrade/26.11
# edit flake.nix input for nixpkgs
nix flake lock --update-input nixpkgs
# then carefully update HM / sops / etc.
nix flake check
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L
# deploy lab
# health checks
```

4. Health verification

- HTTP endpoints
- `systemctl --failed`
- Specific app probes
- Logs

5. Rollback

Exact commands for your deploy path (deploy-rs/colmena/rebuild).

6. Risks

Risk	Likelihood	Impact	Mitigation
…			

7. Decision

Go / no-go criteria for prod.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/upgrade
cd ~/lab/nixos/internals/upgrade
```

Lab 1 — Inventory pins

```
cd /path/to/flake
nix flake metadata --json > ~/lab/nixos/internals/upgrade/metadata.json
cp flake.lock ~/lab/nixos/internals/upgrade/flake.lock.baseline
```

List every input and its tracking branch/tag in `INPUTS.md`.

Lab 2 — Write the full runbook

Complete `UPGRADE-26.11.md` from the template. Mark unknown 26.11 specifics as `TBD when notes ship` but keep the **structure** solid. Reuse the release notes method for notes intake.

Acceptance: All seven template sections present; rollback commands are copy-pasteable for *your* path.

Lab 3 — Dry-run mechanics on lab (optional execution)

If 26.11 exists and you have time:

1. Branch
2. Update nixpkgs **only**
3. Eval and fix until `toplevel` builds
4. **Do not** require prod switch

If 26.11 is not out: dry-run against `nixos-unstable` **on a disposable branch/VM only**, or stop at lock-file rehearsal:

```
# rehearsal of commands without pushing
nix flake lock --update-input nixpkgs --output-lock-file /tmp/test.lock
```

Document what you did / skipped.

Lab 4 — Test plan matrix

`TEST-PLAN.md`:

Check	Command	Blocks promote?
flake check	`nix flake check`	Yes
host toplevel	`nix build …`	Yes
nixosTest X	…	Yes
manual proxy curl	…	Yes for edge host
backup restore drill	…	Prod only

Lab 5 — Abort & rollback tabletop

Without breaking prod, write the exact rollback commands for:

1. Failed switch, system still booting old generation
2. Booted new generation, services bad
3. Git revert of lock after partial deploy

Acceptance: Three scenarios with commands; no "I'll figure it out live."

Lab 6 — Disk space math

```
df -h /nix
nix path-info -S /run/current-system 2>/dev/null || true
```

Record free space and estimated second closure headroom in the runbook preconditions.

Common gotchas

Mistake	Fix
Big-bang all inputs	Stage nixpkgs first
No backup verification	Test restore once
Forgetting companion inputs	HM/sops/disko matrix
Upgrading prod first	Lab → CI → prod
No abort criteria	Write them cold
Bumping stateVersion with release	Separate decision
GC mid-upgrade	Free space first; GC after success

Checkpoint

- ☐ `INPUTS.md` + lock baseline
- ☐ Complete `UPGRADE-26.11.md` runbook
- ☐ `TEST-PLAN.md` with blockers
- ☐ Rollback tabletop written
- ☐ Explicit statement: prod upgraded or **not** (expected: not required)
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): 26.05 to 26.11 upgrade runbook dry-run"
```

Write three personal gotchas before continuing.

Next

→ **Troubleshooting drills** — inject, recover, playbook.

Troubleshooting drills

Troubleshooting drills

Goal: Inject 2–3 realistic failures on a **disposable lab VM**, recover them, and write durable playbook entries covering boot, build/eval, service, and network classes.

> ⚠ Warning
>
> Do **not** run injection drills on a daily driver or the only copy of important data. Snapshot the VM first.

Why this chapter exists

Theory without broken systems is cosplay. Capstone DR and rollback assume you already have a personal playbook. This chapter builds it under time pressure.

Theory 1 — Failure class matrix

Class	Symptoms	First tools
Eval	`nixos-rebuild` dies before build; option errors; infinite recursion	`nix eval`, bisect modules, evaluator
Build	Compile/hash/sandbox failures	`nix log`, store and daemon
Boot	Unbootable, emergency shell, wrong generation	Console, bootloader generations, `stage-1` logs
Activation	Switch fails mid-script	`nixos-rebuild` output, systemd activation logs
Service	Unit failed, restart loop	`systemctl`, `journalctl`
Network	No SSH, DNS, firewall drops	`ip`, `nft`/`iptables`, `ss`, console
Secrets	Service can't read age/sops path	Permissions, sops keys, activation ordering
Disk/state	Full disk, wrong mount, impermanence surprise	`df`, `mount`, disko layout

For each incident, write:

```
Symptom → Class → Hypothesis → Command evidence → Fix → Prevention
```

Theory 2 — Golden command palette

```
# system health
systemctl --failed
journalctl -p err -b --no-pager | tail -100
journalctl -u <unit> -e --no-pager

# nix
nix log /nix/store/….drv
nix-store --verify --check-contents   # heavy; rare
nixos-rebuild dry-activate --flake .#host

# network
ip a; ip r
ss -lntp
ping -c2 1.1.1.1
resolvectl status || cat /etc/resolv.conf
nft list ruleset | head   # if nft

# boot / generations
ls -la /nix/var/nix/profiles/system*
# previous generation boot from bootloader menu
```

Remote when SSH dies

- Hypervisor console
- `deploy-rs` / colmena won't save you if network is wrong—console will

Elitebook lab specifics

Resource	Note
Console	virt-manager console / `virsh console` / Incus console
Nested virt	Needed for nixosTest, not for basic SSH drills
Free space	Disk-full injections need headroom to recover

Theory 3 — Injection catalog (choose carefully)

Pick **2–3** from different classes:

#	Injection	Recovery skill
1	Break a module option type / typo in flake host	Eval literacy
2	Firewall deny SSH (on VM with console!)	Network + console
3	Stop/mask a critical unit or bad `ExecStart`	Service debug
4	Wrong proxy upstream	Service + HTTP
5	Mis-permission a secret path	Secrets activation
6	Fill disk with a large dummy file	Disk hygiene
7	Bad `fileSystems` entry on **snapshot** VM only	Boot recovery

Never inject disk/boot failures without a hypervisor snapshot.

Theory 4 — Time discipline

For each incident, timebox:

Phase	Budget
Detect	5 min
Classify	5 min
Fix	20–40 min
Write playbook entry	10 min

If stuck past budget, snapshot rollback and write "what I needed to learn" instead of thrashing.

Theory 5 — Playbook entry format

`playbook/PL-XXX.md`:

```
# PL-003 - nginx 502 after deploy

## Symptoms
- curl localhost returns 502
- nginx active, upstream down

## Class
```

```
Service / proxy

## Diagnose
systemctl status myapp
journalctl -u myapp -n 100
curl -v upstream

## Fix
…

## Prevention
- nixosTest curl through proxy
- healthcheck in CI

## Related
TLS front door; nixosTest mastery
```

Theory 6 — Rollback as a diagnostic tool

```
sudo nixos-rebuild switch --rollback
# or boot previous generation from loader
```

Rollback answers: "Is this a config regression?" If symptoms vanish, bisect generations / git. If symptoms remain, suspect state/disk/network outside the generation.

Theory 7 — Layered isolation order

When unsure, isolate in this order:

```
1. Physical/virt console access?
2. Network path (ping/DNS/SSH)?
3. Boot/generation?
4. Eval of flake?
5. Activation/systemd?
6. App/data layer?
```

Jumping to "rewrite the flake" during a network outage wastes the drill clock.

Worked example — playbook stub

```
## Symptom
SSH hangs after rebuild

## Class
network / firewall

## Diagnose
ss -tlnp; nft list ruleset; journalctl -u sshd

## Fix
re-add allowedTCPPorts = [ 22 ]; console rebuild

## Prevention
nixos-rebuild test first; console attached
```

Lab 0 — Snapshot

```
# hypervisor UI or:
# virsh snapshot-create-as lab troubleshoot-pre --disk-only --atomic
# incus snapshot create nixlab troubleshoot-pre
```

Confirm console access works **before** breaking SSH.

```
mkdir -p ~/lab/nixos/internals/troubleshoot/playbook
cd ~/lab/nixos/internals/troubleshoot
```

Acceptance gate before inject: snapshot exists; console proven; you know wipe boundaries.

Lab 1 — Baseline health

On healthy lab:

```
systemctl --failed | tee baseline-failed.txt
curl -fsS http://127.0.0.1/ || true
nixos-rebuild dry-activate --flake .#<host> | tee baseline-dry.txt
```

Lab 2 — Inject & recover #1 (eval or build)

Example eval injection:

```
# deliberate typo in host config
networking.hostName = 123; # type error
```

```
time nixos-rebuild build --flake .#<host> 2>&1 | tee incident1.log
# fix
# re-build green
```

Write `playbook/PL-001.md` with timing notes.

Lab 3 — Inject & recover #2 (service or secrets)

Example:

- Point systemd `ExecStart` at wrong binary via your module, or
- Break a runtime secret path after backup of working config

Recover using journal + unit status. Playbook `PL-002.md`.

Lab 4 — Inject & recover #3 (network) — console required

Example:

```
networking.firewall.allowedTCPPorts = [ ]; # removed 22
# or extra iptables drop - prefer declarative so rebuild can fix
```

From **console**, restore SSH, verify from client. `PL-003.md`.

If you cannot safely do network injection, substitute a proxy misconfig incident and document why.

Acceptance: At least two distinct classes recovered; network class only with console.

Lab 5 — Compile master index

`playbook/README.md` table of all PL entries + class tags. This file is Capstone ops gold.

Lab 6 — Timeboxed chaos #4 (state)

Optional fourth injection: delete a non-critical state file (not `/etc/ssh` host keys unless snapshot). Recover from backup notes. Emphasize **state generation**.

Lab 7 — Master index cross-links

Ensure `docs/playbooks/index.md` links to rebuild/generations rollback, firewall, sops, build debug. Troubleshooting is a graph, not a single page.

Lab 8 — End-state health

```
systemctl --failed
curl -fsS http://127.0.0.1/ || true
# leave host better than you found it
```

Acceptance: Host healthy; snapshot can be deleted only after health confirmed.

Common gotchas

Mistake	Fix
Breaking SSH without console	Snapshot + console first
Only reading theory	Actually inject
Fixing without writing playbook	Entry is the deliverable
Multiple simultaneous injections	One at a time
Leaving host broken overnight	Revert snapshot if needed
Forgetting prevention section	Tests/asserts become prevention

---

Checkpoint

- ☐ VM snapshot taken
- ☐ 2 incidents injected and fixed
- ☐ 2 playbook entries with diagnose/fix/prevent
- ☐ Master playbook index
- ☐ Host healthy at end of drills
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): troubleshooting drill and ops playbook"
```

Write three personal gotchas before continuing.

Next

→ **Architecture review** — freeze map and gaps before Capstone thrash.

Architecture review

Architecture review

Goal: Map earlier parts of this book onto your actual repository, produce `ARCHITECTURE.md`, and a **prioritized gap list** that drives Capstone—before writing more features ad hoc. Assume Capstone hosts pin **NixOS 26.05**.

> Tip
>
> Capstone is implementation and proof, not discovery. Teams that skip architecture review thrash: secrets half-done, tests theater, deploy only from a laptop. This chapter freezes an honest map of what exists vs Capstone A requirements.

Why this chapter exists

You have skills across concepts through ops. Capstone fails when those skills never land in **one** coherent flake. Architecture review is the integration gate before code thrash: map, gaps, sequence—then implement.

Theory 1 — Capstone A bar (recall)

Required:

1. 1 NixOS host role understood end-to-end
2. Home Manager **or** minimal service-user story
3. Secrets: sops-nix **or** agenix
4. Reverse-proxy / edge pattern
5. Deploy path: deploy-rs **and/or** Colmena
6. 1 nixosTest or flake check catching real regressions
7. CI builds and preferably pushes a cache

Disaster recovery and rollback chapters add wipe/rebuild + rollback proof. Gaps from this chapter should make Capstone sequencing trivial.

Item	Your status (fill now)
Host role	solid / partial / missing
HM or service user	
Secrets	
Edge/proxy	
Deploy tool	
Tests	
CI + cache	

Theory 2 — Coverage matrix (topic parts)

Part	Theme	Evidence in repo (examples)
Concepts	Foundations, flake shell	`devShells`, language notes
NixOS host	First host, users, net, systemd	`nixosConfigurations.*`, host modules
Home & layout	Layout, HM, checks	`home-manager`, flake structure
Services & security	Secrets, harden, proxy, data	sops, firewall, nginx/caddy
Packaging	Overlays, caches	custom packages, cachix config
Ops & fleet	Disko, deploy, images, tests, CI	deploy configs, workflows, tests
Internals	Evaluator, upgrades	notes artifacts; not all need code

Mark each: **solid / partial / missing / N/A**. Partial without a demo command is missing in disguise.

Theory 3 — Architecture views (write these)

A. Context

Who uses the system? Laptop only? Lab server? Family services? Threat model one paragraph.

B. Containers of code

```
flake.nix
hosts/
modules/
homes/
secrets/
tests/
deploy/
.github/ or .forgejo/
docs/
```

C. Runtime view

```
[client] → [proxy:443] → [app:8080]
               ↓
            [sops secrets]
               ↓
            [deploy-rs/colmena from CI or admin]
```

D. Trust view

- Who can decrypt secrets?
- Who is `trusted-users`?
- Which caches are trusted?
- Where do age master keys live offline?

E. Failure view

- Boot rollback
- Deploy failure

- Secret loss (DR needs offline keys)
- Cache outage (can you still build?)

F. Data view

- What is stateful?
- Where are backups?
- Impermanence list if any?

Theory 4 — Prioritization

Score gaps:

Priority	Definition
P0	Capstone A impossible without it
P1	Required for DR proof quality
P2	Hardening / UX polish
P3	Nice / Capstone B

Sequence Capstone as:

Capstone chapter	Suggested focus
Scope and plan	P0 host + module skeleton gaps
Implement	Secrets + proxy E2E
Harden and document	Deploy + CI + cache + docs
Prove rebuild	Tests + freeze
Optional multi-node	Capstone B bar only if A solid
Disaster recovery	Wipe + rebuild timed
Rollback and tests	Rollback drill + suite green
Retrospective	Honest close-out

Adjust if your matrix differs—**write the plan**. A plan you do not write will be renegotiated under panic every session.

Theory 5 — Non-goals for Capstone A

Explicitly list what you will **not** do (protects freeze):

- Multi-region mesh
- Full k3s
- Every self-hosted app
- Capstone B fleet (unless ahead)
- Experimental CA store
- Perfect HA for lab toys

Non-goals are features, not apologies. Scope control is expertise.

Theory 6 — Evidence standards

Claim	Acceptable evidence
"We have sops"	Decrypt path works on host; secrets.nix in git
"We have CI"	Green URL + workflow file
"We have tests"	Check that fails when you break a module
"We deploy"	Log of remote activation this month
"Architecture exists"	`ARCHITECTURE.md` with diagrams

Screenshots of dashboards without repo paths do not count for Capstone A.

Theory 7 — Dossier structure (feeds Capstone)

Recommended docs layout:

```
docs/
  ARCHITECTURE.md
  GAPS.md
  CAPSTONE-A.md
  RUNBOOK.md          # deploy/rollback
  TRUST.md            # keys, caches, trusted-users
```

```
playbooks/          # from troubleshooting
UPGRADE-26.11.md    # from upgrade planning
```

Capstone chapters will tick `CAPSTONE-A.md` and deepen RUNBOOK/DR.

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/internals/architecture
cd ~/lab/nixos/internals/architecture
```

Work against the real Capstone flake path.

Lab 1 — Automated inventory

```
cd /path/to/flake
find . -name '*.nix' | head -200 | tee ~/lab/nixos/internals/architecture/nix-files.txt
nix flake show 2>&1 | tee ~/lab/nixos/internals/architecture/flake-show.txt
rg -n "sops|agenix|deploy-rs|colmena|home-manager|nixosTest|cachix" -g '*.nix' \
  | tee ~/lab/nixos/internals/architecture/feature-hits.txt
ls -la .github/workflows 2>/dev/null || ls -la .forgejo/workflows 2>/dev/null || echo "no CI
```

Skim `flake-show` for dead attrs and surprise outputs.

Lab 2 — Coverage matrix

Fill `COVERAGE.md`:

Part	Status	Paths	Demo command	Notes
Concepts				
…				
Capstone A items 1–7	each row			

Be brutal. “I watched a video about sops” ≠ solid.

Lab 3 — `ARCHITECTURE.md` (required)

Minimum sections:

1. Purpose & scope
2. Host roles
3. Module map
4. Secrets model
5. Edge/proxy
6. Deploy & CI
7. Test strategy
8. Trust & keys
9. Backup/DR pointers
10. Gap list summary (link)
11. Non-goals
12. Pin / release (26.05) policy

Diagrams: ASCII is enough.

Acceptance: A stranger can find rebuild, secrets, and deploy paths in under five minutes of reading.

Lab 4 — Prioritized backlog

`GAPS.md`:

ID	Gap	Pri	Capstone chapter	Acceptance test
G01	No CI cache push	P0	Harden/document	workflow uploads
…				

Every P0/P1 needs an acceptance test you could demo in under five minutes.

Lab 5 — Checklist freeze seed

Create `CAPSTONE-A.md` checklist (boxes empty/partial/done) that Capstone will tick. Include wipe/rebuild and rollback rows even though execution is later.

```
# Capstone A checklist

## Core
- [ ] Host role rebuilds from flake
- [ ] HM or service users
- [ ] Secrets E2E
- [ ] Reverse proxy edge
- [ ] Deploy path
- [ ] nixosTest/check
- [ ] CI (+ cache preferred)

## Proof
- [ ] Wipe + rebuild timed
- [ ] Rollback drill + suite green

## Docs
- [ ] ARCHITECTURE.md current
- [ ] RUNBOOK.md deploy/rollback
```

Lab 6 — Risk register (short)

Three risks that could sink Capstone A:

Risk	Likelihood	Impact	Mitigation
Only age key on lab disk			Offline copy now
No console on remote			Enable before firewall games
Tests don't fail on breaks			Capstone tests priority

Lab 7 — Sequence plan

Write `SEQUENCE.md`: ordered list of Capstone sessions with intended gap IDs closed. Keep it one page.

Common gotchas

Mistake	Fix
Optimistic “partial = done”	Require demo command proof
Skipping docs because “I know it”	Capstone is proof for future you
40 P0s	Reclassify; cut scope
Planning Capstone B now	Park in non-goals
No acceptance tests on gaps	Unverifiable work
Architecture file never opened again	Link it from CAPSTONE-A.md

---

Checkpoint

- ☐ Inventory artifacts (`flake-show`, feature hits)
- ☐ `COVERAGE.md` part matrix
- ☐ `ARCHITECTURE.md` complete
- ☐ `GAPS.md` prioritized with Capstone mapping
- ☐ `CAPSTONE-A.md` checklist seeded
- ☐ Risk register + sequence
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "lab(nixos): architecture review and capstone gap plan"
```

Write three personal gotchas before continuing.

Next

→ **Capstone and recovery** — implement Capstone A and prove rebuildability.

How Nix Normalizes Metadata for Reproducibility

How Nix Normalizes Metadata for Reproducibility

Thesis

Two machines evaluating the same flake should compute the **same store paths**. Nix gets there by **normalizing inputs** — including timestamps and other filesystem metadata — so "whatever my laptop's `stat` said" does not leak into hashes.

Mental model

```
source tree → Nix hasher

       file contents  (primary)
       normalized metadata
          timestamps → fixed / ignored per algorithm
          permissions → canonicalised where required
```

Content-addressed derivations go further (hash of result), but even input-addressed builds need stable inputs.

Worked example

`nix` is **not installed** on this box; the following is from Nix manuals / reproducible-builds practice.

```
# On a Nix-enabled host:
nix flake metadata .          # locked inputs, lastModified
nix hash path ./src           # path hash with Nix's nar serialization
```

NAR (Nix ARchive) serialization is the stable fingerprint: directory entries sorted, executable bit preserved, timestamps zeroed in the NAR model historically used for `nix-hash` / `hashPath`. Flake lockfiles pin `lastModified` and `narHash` for Git inputs so re-evaluation does not depend on `git clone` time.

Trap

Embedding `builtins.currentTime` or unpinned `fetchTarball` without a hash. You reintroduce wall-clock and CDN drift. Likewise, tooling that packs `mtime` into tarballs outside NAR rules will disagree across machines.

Boring rule

Pin flakes (`flake.lock`), hash fixed-output derivations, and treat NAR hashing as the source of truth for "same tree". Do not chase `mtime` equality on the raw checkout.

Try this

1. `nix flake metadata` before/after touching a file without content change — path hashes should follow content, not touch time, when going through Nix hashing.
2. Read the NAR format notes in the Nix manual.
3. Compare `nix hash path` vs `sha256sum` on a single file (different!) vs a NAR dump.

Sources

- Nix manual: reproducible builds / `nix hash` / flakes locking
- reproducible-builds.org (broader context)
- Nix NAR discussion in Nixpkgs / Nix internals docs

Capstone

Capstone and recovery

Capstone and recovery

This part of the **standalone NixOS book** is organized by **topic** (normal book chapters), not by a calendar. Complete Capstone A before optional multi-node work. Study pacing is yours.

Prerequisites. Architecture review complete (`ARCHITECTURE.md`, `GAPS.md`, seeded `CAPSTONE-A.md`). You can rebuild a flake host, manage secrets without store leakage, and run at least a stub flake check. Internals literacy (eval, modules, store, tests) is assumed when debugging Capstone failures.

What this part is for. Capstone A is not "another service." It is **proof** that your repository rebuilds a real host role end-to-end: secrets, edge, deploy, tests, CI, wipe/rebuild, and rollback—with documentation an exhausted future-you can follow.

Capstone A vs optional bar

Track	Required?	Bar
Capstone A	Yes	Single host role (or primary lab guest) with full checklist
Multi-node / freeze discipline	Strongly recommended before DR	Feature freeze + tag; optional second node only if A green
Capstone B ideas	No	Fleet patterns—park in retrospective roadmap

> **!** Important
>
> **Capstone A (required):** flake host · HM **or** service users · sops/agenix · reverse proxy · deploy-rs/colmena · nixosTest · CI + cache preferred · wipe/rebuild · rollback + green suite · architecture + runbook docs.

Learning outcomes

When you leave this part you should be able to:

1. Close P0 gaps into a building host skeleton with honest checklist ticks.
2. Wire secrets E2E and a reverse-proxy front door without plaintext secrets in git.
3. Deploy remotely and run CI that builds (and preferably caches) Capstone artifacts.
4. Freeze a tagged baseline with known waivers.
5. Wipe and rebuild from flake + offline keys alone; time the recovery.
6. Roll back a bad generation and keep the full test suite green.
7. Write a retrospective and teach plan without starting a rewrite.

Chapter map

Chapter	Theme	Deliverable
Scope and plan	Close structural P0s	Green host + users + module hygiene
Implement	Secrets + proxy E2E	Encrypted secrets + front door proof
Harden and document	Deploy + CI + cache	One-command deploy + CI green
Prove rebuildability	Tests + runbook + diagrams	Sufficient suite + RUNBOOK
Optional multi-node bar	Feature freeze (+ optional node)	Annotated freeze tag
Disaster recovery	Wipe + rebuild timed	DR log + fixed docs
Rollback and tests	Generation rollback + suite	Rollback log + green CI
Retrospective	Teach + roadmap	Retro notes; no rewrite

Suggested sequence

```
Architecture review (internals)
```

```
Scope & plan  →  Implement secrets/proxy

Harden deploy/CI  →  Prove tests/runbook

Freeze tag  →  Wipe/rebuild DR

Rollback + full suite  →  Retrospective
```

Do **not** wipe before freeze. Do **not** start multi-region mesh mid-Capstone A.

Dossier structure (canonical)

Keep these paths (or equivalent) current throughout Capstone:

```
docs/
  ARCHITECTURE.md
  CAPSTONE-A.md          # live checklist
  GAPS.md                # remaining / waived
  RUNBOOK.md             # deploy, rollback, health
  SECRETS.md             # key custody
  TLS.md                 # ACME vs lab mode
  DEPLOY.md              # remote path
  DR-PLAN.md             # wipe/rebuild steps
  TRUST.md               # caches, trusted-users
  POST-FREEZE.md         # ideas deferred after freeze
  playbooks/             # PL-*.md from drills
  RETRO.md               # retrospective
```

Doc	Owner chapter
CAPSTONE-A.md	All; start in scope
SECRETS / TLS	Implement
DEPLOY / CI notes	Harden
RUNBOOK / diagrams	Prove rebuild
Freeze tag notes	Multi-node / freeze
DR-PLAN + timing	Disaster recovery
Rollback log	Rollback and tests
RETRO	Retrospective

Live Capstone A checklist (copy into repo)

```
# CAPSTONE-A.md
- [ ] Host role rebuilds from flake
- [ ] HM user OR service user story
- [ ] sops-nix OR agenix wired; service reads secret
- [ ] Reverse proxy edge with TLS or lab HTTP mode documented
- [ ] deploy-rs OR colmena path works from admin machine
- [ ] nixosTest/check catches a real regression
- [ ] CI builds toplevel (and checks)
- [ ] CI pushes to binary cache (preferred)
- [ ] ARCHITECTURE.md current
- [ ] RUNBOOK.md for deploy/rollback
- [ ] Feature freeze tag (recommended before wipe)
- [ ] Wipe + rebuild timed from flake + offline keys
- [ ] Rollback drill + suite green
```

Tick only with **proof commands** recorded (build log path, curl, deploy log).

Lab workspace convention

```
mkdir -p ~/lab/nixos/capstone/{scope,implement,harden,prove,freeze,dr,rollback,retro}
```

Prefer a single Capstone git branch (`capstone-a`) over calendar-named branches.

Acceptance bar for Capstone A

- ☐ Every required checklist row has a demo command
- ☐ Secrets never appear as Nix strings in the store graph
- ☐ Front door health probe documented
- ☐ Remote deploy works from admin machine (or documented exception with local-only waiver)
- ☐ At least one behavioral check fails when you break a module
- ☐ DR wipe/rebuild timed with doc fixes committed

☐ Rollback proven after intentional bad change

☐ Freeze tag (or explicit waiver why not) before DR

Elitebook / KVM lab notes

Concern	Capstone guidance
16 GB RAM	Capstone A on **one** primary guest; optional second node only if headroom
Snapshots	Snapshot before implement breaks, freeze, and DR
Console	Required before firewall / wipe experiments
Disk	Free space for two system closures + installer ISO
KVM	Needed for nixosTest in prove/rollback chapters

Policy reminders

Policy	Rule
Pin	nixpkgs **26.05** for Capstone A
Store	Default input-addressed; no experimental CA on Capstone
Secrets	Offline private key backup before wipe
Scope	Non-goals stay non-goals until retrospective roadmap
Day framing	**None**—these are topic chapters

Next

→ **Capstone: scope and plan** — close structural P0s and make the checklist real.

Capstone: scope and plan

Capstone: scope and plan

Goal: Close the highest **P0** gaps from the architecture review so the Capstone A skeleton is real: flake host role, module layout, HM or service users, and a live checklist that tracks proof—not vibes.

> **!** Important
>
> **Capstone A (required):** flake host · HM **or** service users · sops/agenix · reverse proxy · deploy-rs/colmena · nixosTest · CI + cache.
> Later Capstone chapters implement secrets/proxy, harden deploy/CI, prove rebuild, optional freeze, DR wipe, rollback, and retrospective.

Why this chapter exists

Architecture review without commits is stationery. This chapter turns `GAPS.md` P0s into code on a branch, with rebuilds you can demonstrate.

Theory — Working agreements for Capstone

Rule	Why
One change → build → commit	Bisectable; safer switch
Lab VM first	Protect daily driver
Checklist is law	`CAPSTONE-A.md` updated same session
No drive-by rewrites	Freeze before DR
Secrets never in git plaintext	Even “temporary”

Suggested repo shape (adapt, don’t dogma)

```
flake.nix
flake.lock
hosts/<name>/configuration.nix
modules/{proxy,common,users,…}.nix
homes/<user>.nix          # if HM
```

```
secrets/secrets.yaml      # sops-encrypted
tests/*.nix
deploy.nix or colmena hive
.github/workflows/ci.yml
docs/{ARCHITECTURE,CAPSTONE-A,RUNBOOK,GAPS}.md
```

Capstone A checklist (live)

Copy into repo if missing; tick as you go:

```
# CAPSTONE-A.md
- [ ] Host role rebuilds from flake
- [ ] HM user OR service user story
- [ ] sops-nix OR agenix wired; service reads secret
- [ ] Reverse proxy edge with TLS or lab HTTP mode documented
- [ ] deploy-rs OR colmena path works from admin machine
- [ ] nixosTest/check catches a real regression
- [ ] CI builds toplevel (and checks)
- [ ] CI pushes to binary cache (preferred)
- [ ] ARCHITECTURE.md current
- [ ] RUNBOOK.md for deploy/rollback
- [ ] Disaster recovery: wipe + rebuild timed
- [ ] Rollback drill + suite green
```

Target scope for this chapter

From `GAPS.md`, implement **P0s that unblock structure**, typically:

1. `nixosConfigurations.<lab>` evaluates and builds
2. Host modules split (common + role)
3. Users: interactive HM **or** system service user + data dir
4. Firewall baseline + SSH
5. Placeholder modules for proxy/secrets if not ready (interfaces only)—prefer real stubs that `mkEnableOption` cleanly

Defer full proxy TLS and CI cache to implement/harden chapters if needed—but leave option shapes ready.

Explicit non-goals (this chapter)

Non-goal	Why
Multi-node mesh	Optional bar later
Perfect ACME	Document lab mode later
Full observability stack	P2+
CA store experiments	Internals literacy only

Worked example — Minimal host flake skeleton

```
# flake.nix (sketch)
{
  description = "capstone lab";
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    # home-manager.url = "github:nix-community/home-manager/release-26.05";
    # home-manager.inputs.nixpkgs.follows = "nixpkgs";
    # sops-nix.url = "github:Mic92/sops-nix";
    # sops-nix.inputs.nixpkgs.follows = "nixpkgs";
  };
  outputs = { self, nixpkgs, ... }@inputs: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      specialArgs = { inherit inputs; };
      modules = [
        ./hosts/lab/configuration.nix
        ./modules/common.nix
        # inputs.sops-nix.nixosModules.sops
        # inputs.home-manager.nixosModules.home-manager
      ];
    };
  };
}
```

```
# modules/common.nix
{ lib, pkgs, ... }:
{
  time.timeZone = "UTC";
  networking.firewall.enable = true;
  services.openssh.enable = true;
  environment.systemPackages = with pkgs; [ vim git curl ];
}
```

```
# hosts/lab/configuration.nix
{ config, pkgs, ... }:
{
  imports = [ ./hardware.nix ];
  networking.hostName = "lab";
  # users / HM / roles
  system.stateVersion = "26.05"; # your real value
}
```

Theory — Capstone A success definition (this chapter)

Done	Not done
Green host from flake	Perfect theming
Users + SSH story	Every optional service
Module hygiene applied	Multi-region fleet
Architecture doc updated	Final polish

Scope control is part of the grade.

Theory — Branch discipline

```
git checkout -b capstone-a
# work
# merge only when smoke passes
```

Avoid committing broken `main` mid-capstone if you use the same repo for the lab host.

Worked example — smoke script

```
#!/usr/bin/env bash
set -euo pipefail
hostname
systemctl is-active sshd
```

```
curl -sf -o /dev/null -w '%{http_code}\n' http://127.0.0.1:… || true
nix flake metadata
```

Keep it boring; run after every significant rebuild.

Rebuild proof steps (skeleton → switch)

Step	Command	Pass means
1	`nix flake show`	Host attr visible
2	`nix build .#nixosConfigurations.lab.config.system.build.toplevel -L`	Exit 0
3	`nixos-rebuild dry-activate --flake .#lab`	No activation surprises
4	`sudo nixos-rebuild switch --flake .#lab` (lab only)	SSH still works
5	`systemctl --failed`	Empty or known waivers

Lab 0 — Branch & checklist

```
cd /path/to/capstone-flake
git checkout -b capstone-a
mkdir -p docs ~/lab/nixos/capstone/scope
# seed docs/CAPSTONE-A.md from architecture review if needed
```

Lab 1 — Build green host

```
nix flake show
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L \
  2>&1 | tee ~/lab/nixos/capstone/scope/build.log
```

Fix until green. Prefer `build` over `switch` until config is sane; then:

```
sudo nixos-rebuild switch --flake .#lab
# or deploy path if already wired
```

Acceptance: toplevel green; hostname/SSH story demoable.

Lab 2 — Users story

Track HM (workstation-ish)

- Enable HM as NixOS module
- Manage shell + one dotfile
- Prove with `home-manager generations` or files in place

Track service users (server-ish)

```
users.users.myapp = {
  isSystemUser = true;
  group = "myapp";
  home = "/var/lib/myapp";
  createHome = true;
};
users.groups.myapp = { };
```

Document which track in `docs/CAPSTONE-A.md`.

Lab 3 — Module hygiene

Ensure at least one custom module uses:

- typed options
- `mkEnableOption`
- `mkIf`
- one assertion

Even if proxy is stubbed:

```
options.my.capstone.proxy.enable = lib.mkEnableOption "edge proxy";
```

Lab 4 — Update architecture & gaps

```
# edit docs/ARCHITECTURE.md paths if moved
# mark GAPS.md items closed with PR/commit refs
```

Write `~/lab/nixos/capstone/scope/NOTES.md`: what shipped, what slips to implement.

Lab 5 — Smoke

```
systemctl --failed
hostname
# ssh localhost or from client
```

Lab 6 — Gap register freeze for next chapter

Write `docs/capstone-gaps.md` with only items that remain for implement/harden. Anything critical for a bootable admin host must be closed now.

Lab 7 — Teach-back outline (5 bullets)

1. How to rebuild
2. How to roll back
3. Where secrets will live
4. What state to back up
5. What implement will add

Failure modes

Symptom	Fix
Hardware config missing	Import hardware or disko
Infinite recursion after split	Option cycles; evaluator chapter
HM + system package duplication	Pick layers deliberately
Switching daily driver	Stop; use VM
Checklist not updated	Same commit as code

Common gotchas

Symptom	What to do
Hardware config missing in flake	Import `hardware-configuration.nix` or disko
Infinite recursion after module split	Dependency cycles
HM + system package duplication	Pick layers deliberately
Switching daily driver	Stop; use VM
Checklist not updated	Same commit as code

Checkpoint

- ☐ `nix build` toplevel green for lab host
- ☐ HM **or** service user story demoable
- ☐ Modules structured; 1 typed custom module
- ☐ `CAPSTONE-A.md` updated honestly
- ☐ `GAPS.md` P0s reduced or re-dated
- ☐ Commits on capstone branch

Commit

```
git add .
git commit -m "capstone: host skeleton and user story"
```

Write three personal gotchas before continuing.

Next

→ **Capstone: implement** — secrets E2E and reverse-proxy front door.

Capstone: implement

Capstone: implement

Goal: Complete **secrets E2E** (sops-nix **or** agenix) and the **edge reverse-proxy** path so a service is reachable through the front door without plaintext secrets in git.

> **!** Important
>
> Use **lab-only** tokens. Never commit production credentials. Prefer **sops-nix** as house default; agenix is fine if Capstone already standardized on it—do not run both permanently.

Why this chapter exists

Host skeletons without secrets train bad habits. Proxy without a TLS story (or documented lab HTTP mode) is not an edge pattern. This chapter closes Capstone A items for **secrets + reverse proxy** with proof commands you can paste into the checklist.

Theory 1 — Secrets E2E definition of done

Step	Done means
Encrypt	`secrets/*.yaml` (or age files) committed **encrypted**
Keys	Machine age/ssh key or admin key decrypts on host
NixOS wiring	`sops.secrets.*` or `age.secrets.*` → path on disk
Consumer	systemd `LoadCredential` / `EnvironmentFile` / path read
Negative proof	Repo clone alone cannot read secret without keys
Test story	nixosTest uses dummy secret or test key
Offline custody	Private keys exist **off** the lab disk

Threat model (short)

Bad outcome	Control
Secret in git plaintext	SOPS/age encrypt before commit
Secret in Nix string → store	Reference **paths**, never secret values in modules

Bad outcome	Control
World-readable decrypted file	`owner` / `mode` / service user match
Lost keys after wipe	Offline backup of age/ssh private material

---

Theory 2 — sops-nix sketch (preferred)

```
# host module fragment - follow current sops-nix README for key paths
{ config, ... }:
{
  sops.defaultSopsFile = ./secrets/secrets.yaml;
  # common patterns: age ssh host key and/or dedicated age key file
  sops.age.sshKeyPaths = [ "/etc/ssh/ssh_host_ed25519_key" ];
  # sops.age.keyFile = "/var/lib/sops-nix/key.txt";

  sops.secrets."myapp/api_token" = {
    owner = "myapp";
    group = "myapp";
    mode = "0400";
    restartUnits = [ "myapp.service" ];
  };

  systemd.services.myapp = {
    description = "Capstone sample app";
    wantedBy = [ "multi-user.target" ];
    serviceConfig = {
      User = "myapp";
      Group = "myapp";
      EnvironmentFile = config.sops.secrets."myapp/api_token".path;
    };
  };
}
```

`.sops.yaml` creation rules (idea)

Encrypt lab secrets to **host** recipient + **admin** recipient so you can edit on a workstation and the machine can decrypt at activation.

```
sops secrets/secrets.yaml
```

Theory 3 — agenix sketch (alternative)

```
age.secrets.apiToken = {
  file = ../secrets/apiToken.age;
  owner = "myapp";
  mode = "0400";
};
# service references config.age.secrets.apiToken.path
```

Pick **one** house default for Capstone A.

Theory 4 — Proxy E2E definition of done

Step	Done means
Upstream	App listens **localhost** (or internal-only) port
Proxy	nginx/caddy/traefik terminates edge
Firewall	Only designed ports public (often 80/443 + SSH)
TLS	ACME **or** lab self-signed/internal CA **or** HTTP lab mode **documented**
Probe	`curl -f` through proxy hostname/port
Negative	Upstream port not world-open unless intentional
Regression	Test or script fails if upstream link breaks

nginx sketch

```
{ config, lib, ... }:
{
  services.nginx = {
    enable = true;
    recommendedProxySettings = true;
    recommendedTlsSettings = true;
    virtualHosts."lab.example.com" = {
      # forceSSL = true;
      # enableACME = true;
      locations."/" = {
        proxyPass = "http://127.0.0.1:8080";
        proxyWebsockets = true;
      };
    };
  };
  networking.firewall.allowedTCPPorts = [ 80 ]; # lab HTTP mode example
```

```
}
```

TLS modes (pick and document in `docs/TLS.md`)

Mode	When
ACME / public DNS	Real hostname, ports reachable
Self-signed / internal CA	Lab realism without public DNS
HTTP-only lab	Acceptable for Capstone if **prod path** is written down

---

Theory 5 — Ordering and dependencies

```
secret decrypt (activation)
    → app service start (reads secret path)
    → proxy start / reload
    → health probe via front door
```

Use `restartUnits` on secrets, systemd `After=`/`Requires=` where needed, and in tests `wait_for_unit` for **both** app and proxy.

Store discipline reminder

```
# WRONG - leaks into world-readable store
environment.variables.API_TOKEN = "supersecret";

# RIGHT - path only
# EnvironmentFile = config.sops.secrets."myapp/api_token".path;
```

Theory 6 — Module shape for Capstone

```
# modules/proxy.nix (sketch)
{ lib, config, ... }:
let cfg = config.roles.proxy; in
{
  options.roles.proxy = {
```

```
    enable = lib.mkEnableOption "edge reverse proxy";
    upstream = lib.mkOption {
      type = lib.types.str;
      default = "http://127.0.0.1:8080";
    };
    hostName = lib.mkOption {
      type = lib.types.str;
      default = "lab.example.com";
    };
  };
  config = lib.mkIf cfg.enable {
    # services.nginx.virtualHosts.${cfg.hostName} = …
  };
}
```

Concrete checklists

Secrets checklist

- ☐ Lab token generated (not production)
- ☐ Ciphertext committed; plaintext never staged
- ☐ Host decrypts after switch
- ☐ Unit user can read path; others cannot
- ☐ Offline private key backup exists
- ☐ Negative: `git grep` no raw token

Proxy checklist

- ☐ Upstream localhost-only (or documented)
- ☐ Front door probe succeeds
- ☐ Firewall matches design
- ☐ `docs/TLS.md` written
- ☐ Negative: external upstream port closed if required

Worked example — Proof commands

```
sudo ls -la /run/secrets 2>/dev/null || sudo ls -la /run/secrets.d 2>/dev/null
systemctl status myapp.service --no-pager
curl -fsS http://127.0.0.1:8080/ | head
curl -fsS -H "Host: lab.example.com" http://127.0.0.1/ | head
git grep -n "the-real-lab-token-value" && echo "LEAK" || echo "ok-no-plaintext"
```

Lab 0 — Branch

```
mkdir -p ~/lab/nixos/capstone/implement
cd /path/to/capstone-flake
git checkout capstone-a  # continue Capstone branch
```

Lab 1 — Secrets path E2E

1. Ensure age/ssh keys exist for the lab host; **backup private material offline**.
2. Generate a non-production random token.
3. Encrypt with sops-nix or agenix workflow; commit **ciphertext only**.
4. Wire module → service consumer via path.
5. Switch/deploy; prove service healthy.

```
sudo systemctl restart myapp.service
systemctl is-active myapp.service
journalctl -u myapp.service -n 50 --no-pager
head -n 5 secrets/secrets.yaml
```

Acceptance: Service active; secret path exists; git has ciphertext only.

Lab 2 — Proxy path E2E

1. Enable proxy module with typed options if still weak.
2. Upstream app on localhost only.
3. Firewall ports match design.
4. Choose TLS mode; write `docs/TLS.md`.
5. Prove front door with curl.

Lab 3 — Failure injection (mini)

1. Wrong secret path or mode → service fails → fix; note in playbook.
2. Wrong upstream port → 502 → fix.
3. Leave host **green** at end.

Lab 4 — Tests stub toward prove-rebuild chapter

```
machine.wait_for_unit("myapp.service")
machine.wait_for_unit("nginx.service")
machine.wait_for_open_port(80)
machine.succeed("curl -f http://lab/")
```

```
nix build .#checks.x86_64-linux.<test> -L \
  2>&1 | tee ~/lab/nixos/capstone/implement/test.log
```

If full test must wait, leave a **failing checklist note** with reason—not a silent gap.

Lab 5 — Docs & checklist

Doc	Content
`docs/ARCHITECTURE.md`	Secrets + edge data flow
`docs/SECRETS.md`	Who can decrypt; key locations; rotation sketch
`docs/TLS.md`	ACME vs lab mode
`docs/CAPSTONE-A.md`	Tick secrets + proxy with proof commands

Lab 6 — Offline key custody proof

```
# Document location of offline age/ssh private material
# Do NOT commit private keys
# Verify you can open the offline backup medium
```

Write path (physical) in `docs/SECRETS.md` offline section.

Common gotchas

Symptom	What to do
Secret in Nix string	Use sops/age paths only
ACME fail on lab	Document offline/lab TLS; don't block Capstone
Proxy works, app ignores secret	Prove unit environment loads file
Permission denied on secret	owner/mode/group match unit `User=`
Encrypted file looks empty	Confirm sops metadata headers

Checkpoint

- ☐ Secrets E2E with negative git proof
- ☐ Proxy front door probe
- ☐ `docs/SECRETS.md` + `docs/TLS.md`
- ☐ Offline keys backed up

- ☐ CAPSTONE-A ticks for secrets + proxy with commands
- ☐ Host green

Commit

```
git add .
git commit -m "capstone: secrets and reverse-proxy E2E"
```

Write three personal gotchas before continuing.

Next

→ **Capstone: harden and document** — remote deploy, CI, cache.

Capstone: harden and document

Capstone: harden and document

Goal: Make **remote deploy** (deploy-rs **and/or** Colmena) the normal path, and wire **CI** that builds Capstone artifacts and **preferably pushes a binary cache**.

> **Note**
>
> Baseline: flakes on **nixos-26.05**, secrets already on a path. This chapter is remote delivery + automation—not new app features.

Why this chapter exists

`nixos-rebuild --flake` only on the box is not a deploy story. Capstone A requires a deploy path and CI that would catch breakage before you SSH in panic. Cache push turns CI minutes into laptop seconds.

Theory 1 — Deploy definition of done

Item	Done means
Tool chosen	deploy-rs **or** Colmena (both OK; **one** is enough for A)
Config in repo	`deploy.nodes` or Colmena hive committed
Auth	SSH key or deploy user documented
Magic	One command updates lab host from admin machine
Failure UX	Partial failure doesn't leave mystery state; rollback known
Docs	`docs/DEPLOY.md` with command + prerequisites + rollback

deploy-rs sketch

```
# flake outputs fragment - follow current deploy-rs docs for 26.05-era usage
{
  inputs.deploy-rs.url = "github:serokell/deploy-rs";
  outputs = { self, nixpkgs, deploy-rs, ... }: {
    nixosConfigurations.lab = /* … */;
    deploy.nodes.lab = {
```

```
      hostname = "lab.local"; # or IP
      profiles.system = {
        user = "root";
        path = deploy-rs.lib.x86_64-linux.activate.nixos self.nixosConfigurations.lab;
      };
    };
  };
}
```

```
deploy .#lab
# or: nix run github:serokell/deploy-rs -- .#lab
```

Colmena sketch

```
# hive.nix idea
{
  meta.nixpkgs = import (builtins.getFlake "…").inputs.nixpkgs { };
  lab = { … imports … };
}
```

```
colmena apply --on lab
```

Pick one primary path; document the other as optional.

Theory 2 — CI definition of done

Item	Done means
Workflow file	`.github/workflows/ci.yml` (or Forgejo/Gitea equivalent)
Triggers	PR + push to main (or Capstone branch policy)
Build	`nix build` toplevel and/or `nix flake check`
Logs	Failures visible in CI UI
Cache (preferred)	Push to Cachix/Attic/etc. with secrets in CI vault
Pin	Nix installer version documented

Minimal workflow sketch

```yaml
name: ci
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
        with:
          nix_path: nixpkgs=channel:nixos-26.05
      # - uses: cachix/cachix-action@v15
      #   with: { name: mycache, authToken: '${{ secrets.CACHIX_AUTH_TOKEN }}' }
      - run: nix build .#nixosConfigurations.lab.config.system.build.toplevel -L
      - run: nix flake check -L
```

Adapt for KVM runners if nixosTest requires nested virt; otherwise split fast checks vs nightly tests.

Theory 3 — Cache push value

Without project cache	With project cache
Every agent rebuilds your overlays	Shared substitutes
Laptop recompiles CI work	Laptop substitutes CI outputs
Slow feedback	Minutes matter less

If you cannot push a cache yet, **document waiver** and still build in CI. Preferred: push.

Theory 4 — SSH deploy auth checklist

Check	Note
Deploy key or agent	Non-interactive preferred for CI deploy (optional)
`root` vs sudo user	Match deploy tool docs
Host key verification	known_hosts managed deliberately
Network path	Admin machine can reach lab

Check	Note
Firewall	SSH open; no accidental lockout without console

Theory 5 — RUNBOOK / DEPLOY docs minimum

`docs/DEPLOY.md`:

1. Prerequisites (keys, network, flake path)
2. Happy path one-liner
3. Dry-run / build-only variant
4. Failure symptoms
5. Rollback pointer (`docs/RUNBOOK.md`)
6. Who is allowed to deploy

Concrete checklists

Deploy checklist

- ☐ Tool chosen and committed
- ☐ One-command deploy from admin machine
- ☐ Deploy log saved
- ☐ Rollback path written
- ☐ `docs/DEPLOY.md` complete

CI checklist

- ☐ Workflow runs on PR
- ☐ Toplevel or checks green
- ☐ Cache push **or** explicit waiver
- ☐ Nix version noted
- ☐ Badge or URL in CAPSTONE-A

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/harden
cd /path/to/capstone-flake
```

Snapshot lab VM before first remote deploy if deploy is new.

Lab 1 — Wire deploy tool

1. Add deploy-rs or Colmena input.
2. Point at lab host.
3. Deploy once from admin machine.
4. Capture log:

```
deploy .#lab 2>&1 | tee ~/lab/nixos/capstone/harden/deploy.log
```

Acceptance: Remote activation succeeds; host healthy after.

Lab 2 — CI workflow

1. Add workflow file.
2. Push branch; observe CI.
3. Fix until green.
4. Optionally add Cachix/Attic push.

```
# local parity
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L
nix flake check -L
```

Lab 3 — Hardening pass (light)

Do **not** start new apps. Optionally:

- SSH: password auth off if not already
- Firewall review
- `systemctl --failed` clean
- Fail2ban only if already planned

Document changes in ARCHITECTURE.

Lab 4 — Docs

Doc	Update
`docs/DEPLOY.md`	Full path
`docs/RUNBOOK.md`	Deploy + rollback stubs
`docs/CAPSTONE-A.md`	Tick deploy + CI
`docs/TRUST.md`	CI cache keys / trusted-users if changed

Lab 5 — Failure tabletop

Write without executing production:

1. Deploy fails mid-activation → what do you do?
2. CI red on main → who unblocks?
3. Cache token leaked → rotation steps

Lab 6 — Elitebook admin path

If admin machine is the Elitebook host and target is guest:

```
# from host
ping -c1 <guest>
ssh root@<guest> true
deploy .#lab   # or colmena
```

Acceptance: Works over libvirt network without manual `nixos-rebuild` on guest.

Common gotchas

Symptom	Fix
SSH permission denied	Keys, user, PermitRootLogin policy
deploy evaluates wrong system	system/attr mismatch
CI OOM	lower jobs; cache; split matrix
nixosTest needs KVM on GA	nightly self-hosted or skip in PR
Cache auth fails	secrets scope; trusted-public-keys on consumers

---

Checkpoint

- ☐ Remote deploy one-liner works
- ☐ CI builds Capstone artifacts
- ☐ Cache push or waiver documented
- ☐ DEPLOY + RUNBOOK updated
- ☐ CAPSTONE-A ticks with proof
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "capstone: deploy path and CI hardening"
```

Write three personal gotchas before continuing.

Next

→ **Capstone: prove rebuildability** — tests, runbook, diagrams.

Capstone: prove rebuildability

Capstone: prove rebuildability

Goal: Leave Capstone with a **sufficient test suite**, a **RUNBOOK** an exhausted future-you can follow, and architecture diagrams that match reality—operability, not feature sprawl.

> **Note**
>
> Baseline: NixOS **26.05**, flakes, sops-nix (or documented agenix), deploy-rs **or** Colmena. This chapter hardens proof and human interface—not new roles.

Why this chapter exists

Code without operable docs fails disaster recovery. Tests without assertions that match the proxy/secrets design fail rollback drills. This chapter closes Capstone A quality on **tests + runbook + diagrams**: the difference between a demo host and a rebuildable system.

Theory 1 — "Sufficient" tests for Capstone A

Minimum bar:

Test	Intent
Eval/build in CI	Configuration still composes
1 nixosTest or strong VM-less check	Behavioral regression
Secret-safe	No prod keys required in CI
Proxy/app path	Front door works as designed
Optional negative	Firewall deny or failed assertion fixture

Stretch (only if already green):

- Multi-node client→server
- Deploy dry-run / eval in CI
- Dead-man checks for deprecated options

Flake outputs shape

```
# flake fragment - adapt to your layout
{
  checks.x86_64-linux.proxy = pkgs.nixosTest { /* … */ };
  # packages / apps optional
}
```

```
nix build .#checks.x86_64-linux.proxy -L
nix flake check -L
```

Theory 2 — RUNBOOK audience

Write for **tired you at 2am** after a bad switch:

Section	Content
Identity	Host name, role, IP/DNS
Health probes	curl, systemctl, logs
Deploy	One-liner + prerequisites
Rollback	Generation + git
Secrets	Where keys live; what not to do
Escalation	Console path; snapshot restore

Avoid tutorial prose. Use numbered steps and copy-paste commands.

Theory 3 — Diagrams that match reality

Minimum diagrams (ASCII OK):

1. **Runtime:** client → proxy → app → secret path
2. **Deploy:** admin → tool → host activation
3. **Trust:** who decrypts; which caches

If the diagram shows ACME but lab is HTTP-only, fix diagram or TLS mode.

Theory 4 — Rebuild proof steps (pre-wipe)

You are not wiping yet. Prove *paper* rebuildability:

Step	Evidence
Fresh clone builds	CI or clean machine `nix build`
Docs list install path	DR-PLAN draft exists
Keys offline	SECRETS.md offline section
stateVersion recorded	ARCHITECTURE / host module
Disk layout documented	disko or hardware notes

Theory 5 — Assertion fixtures

Keep at least one **bad config** snippet that must fail eval (from module deep dive). CI can later evaluate it expecting failure, or you re-run manually before freeze.

```
# expect non-zero
nix eval -f tests/bad-host.nix config 2>&1 | tee bad-host.err
```

Concrete checklists

Tests checklist

- ☐ Suite inventory (`SUITE.md` or RUNBOOK section)
- ☐ Critical path without blind `sleep`
- ☐ Twice-green local run
- ☐ CI runs checks or documents KVM waiver
- ☐ Regression assertion present

Docs checklist

- ☐ RUNBOOK complete
- ☐ ARCHITECTURE matches runtime
- ☐ Diagrams updated
- ☐ CAPSTONE-A ticks with log paths

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/prove
cd /path/to/capstone-flake
```

Lab 1 — Harden tests

1. Inventory checks.
2. Remove sleep-only waits.
3. Add/verify proxy + app assertions.
4. Run twice:

```
nix build .#checks.<system>.<name> -L 2>&1 | tee ~/lab/nixos/capstone/prove/check1.log
nix build .#checks.<system>.<name> -L 2>&1 | tee ~/lab/nixos/capstone/prove/check2.log
```

Acceptance: two consecutive greens; no sole blind sleep on critical path.

Lab 2 — RUNBOOK

Write or finish `docs/RUNBOOK.md` with deploy, health, rollback, secrets pointers.

Read it once **without** the repo open on a second machine if possible—note gaps.

Lab 3 — Diagrams

Update ARCHITECTURE with three diagrams. Paste into CAPSTONE notes.

Lab 4 — Paper DR draft

Create `docs/DR-PLAN.md` draft (steps only; wipe later):

1. Confirm target disk
2. Boot installer
3. Partition (disko command or manual)
4. Install flake
5. Place keys
6. Deploy/switch
7. Health probes
8. Timing log template

Lab 5 — Freeze readiness review

List P0s remaining. If any block health, fix now. Else schedule freeze chapter.

Lab 6 — Negative eval fixture

Commit `tests/fixtures/bad-*.nix` that fails type or assertion; document expected error fingerprint.

Common gotchas

Mistake	Fix
Docs describe yesterday's layout	Diff ARCHITECTURE vs flake show
Tests only curl localhost app	Include front door
RUNBOOK assumes muscle memory	Numbered commands
CI never runs tests	Wire or waive with reason
Secret keys in CI	Dummy secrets / testMode

Checkpoint

- ☐ Sufficient tests green twice
- ☐ RUNBOOK exhausted-you ready
- ☐ Diagrams match reality
- ☐ DR-PLAN draft exists
- ☐ CAPSTONE-A updated
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "capstone: tests runbook and rebuild proof docs"
```

Write three personal gotchas before continuing.

Next

→ **Optional multi-node bar** — feature freeze (and optional second node).

Capstone: optional multi-node bar

Capstone: optional multi-node bar

Goal: Feature freeze the Capstone A MVP—no new services or toys—only bugfixes, doc fixes, and checklist honesty—so disaster recovery tests a **stable, tagged** artifact. Optionally add a second node **only if** Capstone A is already green and hardware allows.

> **!** Important
>
> After the freeze tag, the only legitimate code changes before wipe are **P0 blockers** that make rebuild or health checks impossible. Everything else goes in `docs/POST-FREEZE.md`.

Why this chapter exists

Wipe/rebuild against a moving target is chaos. Freeze creates a known baseline: annotated git tag, green CI, known gaps **explicitly waived** with risk. Without freeze, disaster-recovery drills measure luck, not process.

Experts freeze early relative to demos. Amateurs keep "just one more module" until the disk wipe and then invent history.

Theory 1 — What "freeze" means

Allowed	Forbidden
Bugfixes for P0/P1 with issue id	New apps / node roles / hostnames
Doc corrections that match code	"Quick" plugin or package installs
Test stability fixes (flakes, waits)	Refactors without bug id
Checklist updates (honest ticks)	Starting Capstone B
Dependency bumps **only** if build broken	Casual `nix flake update`
Typo-level RUNBOOK fixes	New sops keys without re-documenting DR

Freeze is a social contract with yourself

If Capstone is a solo lab, freeze still matters: during DR you will be tired and tempted to "fix while rebuilding." The tag is the line between **recovering a known system** and **redesigning under fire**.

Tag shape

```
git tag -a capstone-a-freeze -m "Capstone A feature freeze"
git push origin capstone-a-freeze  # if remote exists
git rev-parse capstone-a-freeze
```

Record tag SHA in `docs/CAPSTONE-A.md`.

Theory 2 — Waiver register

Every unticked Capstone A row at freeze needs one of:

Disposition	Meaning
Done	Proof command recorded
Waived	Explicit risk + why DR still valid
Blocker	Must fix before freeze tag

```
# docs/POST-FREEZE.md / freeze waivers
| Item | Disposition | Risk | Owner |
|------|-------------|------|-------|
| CI cache push | Waived | Slower rebuilds | you |
```

Theory 3 — Optional multi-node bar

Capstone A is **single-node sufficient**. Multi-node is optional prestige:

Optional	Description
Second guest	`deploy-a` or similar with shared modules
Client/server test	nixosTest multi-node
Distinct roles	edge vs data

Config	Advice

Elitebook 16 GB guidance

Config	Advice
Host + 1 guest 4–6 GB	Capstone A default
Host + 2 guests	Only if you drop desktop/k3s toys
Nested heavy tests	Prefer serial; lower max-jobs

Do **not** add a second node if it risks freeze honesty or disk pressure before DR.

Shared modules pattern

```
modules/common.nix
modules/roles/edge.nix
modules/roles/data.nix
hosts/lab/…
hosts/deploy-a/…   # optional
```

Avoid copy-paste hosts that drift.

Theory 4 — Freeze gate commands

```
nix flake metadata
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L
nix flake check -L   # or specific checks
# deploy once more if remote path is Capstone-critical
systemctl --failed   # on lab
```

All green → tag. Any red → fix or waive with eyes open.

Concrete freeze checklist

- ☐ CAPSTONE-A honest ticks
- ☐ Waivers written
- ☐ CI green on freeze commit
- ☐ RUNBOOK matches freeze tag
- ☐ Offline keys verified present
- ☐ VM snapshot `pre-dr` taken
- ☐ `git tag -a capstone-a-freeze`
- ☐ POST-FREEZE.md for deferred ideas

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/freeze
cd /path/to/capstone-flake
```

Lab 1 — Honesty pass on CAPSTONE-A

For each box: done with proof, waived, or blocker. No “partial” without a sentence.

Lab 2 — Green suite at freeze

```
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L \
  2>&1 | tee ~/lab/nixos/capstone/freeze/toplevel.log
nix flake check -L 2>&1 | tee ~/lab/nixos/capstone/freeze/check.log
```

Lab 3 — Tag

```
git status
git tag -a capstone-a-freeze -m "Capstone A feature freeze $(date -I)"
git show capstone-a-freeze --stat | tee ~/lab/nixos/capstone/freeze/tag.show
```

Lab 4 — Optional second node (only if A green)

1. Snapshot first.
2. Add host with shared modules.
3. Deploy or rebuild.
4. Document network and secrets recipients.
5. If it destabilizes freeze, **revert** and leave as POST-FREEZE.

Lab 5 — Snapshot for DR

```
# virsh snapshot-create-as lab pre-dr --disk-only --atomic
# or hypervisor UI
```

Confirm you can revert without using DR path (sanity).

Lab 6 — Freeze announcement (to yourself)

Write 10 lines in NOTES: what is in, what is out, tag SHA, next step = DR wipe.

Common gotchas

Mistake	Fix
Tagging red CI	Fix or waive first
Freeze then “one more feature”	POST-FREEZE.md
Multi-node thrash on 16 GB	Defer
No snapshot before DR	Snapshot now
Lost tag SHA	Record in CAPSTONE-A

Checkpoint

- ☐ Honest CAPSTONE-A + waivers
- ☐ Green build/check logs
- ☐ `capstone-a-freeze` annotated tag
- ☐ pre-dr snapshot
- ☐ Optional multi-node either done or deferred
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "capstone: feature freeze readiness"
# tag after this commit if clean
```

Write three personal gotchas before continuing.

Next

→ **Disaster recovery** — wipe and rebuild from flake + keys.

Disaster recovery notes

Disaster recovery notes

Goal: Wipe the lab VM/disk and **rebuild** to a working Capstone A host using **only** git/flake + offline keys + your docs—time the recovery and fix documentation gaps that the wipe reveals.

> ⚠ Warning
>
> This chapter **destroys** the lab system disk by design. Confirm target identity three times. Do **not** wipe a daily driver, NAS, or the disk holding your only key copies. Capstone freeze tag + **NixOS 26.05**-aligned installer when possible.

Why this chapter exists

Syllabus completeness: *prove rebuildability.* "It worked once" is not NixOS expertise. DR is the exam for host, secrets, deploy, images, and docs—under stress.

Teams that skip wipe drills ship folklore. You will find the missing step that only lived in your head.

Theory 1 — What "from flake + keys alone" means

Allowed inputs	Disallowed crutches
Git remote / USB bundle of repo	Old disk "just mount and copy /etc"
`flake.lock` pins	Random unpinned `nix-env` debris
Offline age/sops **private** keys	Re-typing secrets from memory into git
Installer ISO / nixos-anywhere / disko	Undocumented manual partitioning folklore
Your `docs/DR-PLAN.md` + RUNBOOK	Chat scrollback
Freeze tag (`capstone-a-freeze`)	"Whatever was on main yesterday"

Ideal proof: new empty disk → installer → disko or manual partitions per docs → install flake → decrypt secrets → deploy/services healthy.

Theory 2 — Recovery paths (pick what you documented)

Path A — Interactive installer + flake

1. Boot NixOS installer ISO (**26.05**-aligned if possible).
2. Network + partition (disko or manual per DR-PLAN).
3. `nixos-install --flake /path/to/flake#lab` (or documented equivalent).
4. Reboot; place keys if not on disk yet.
5. Switch/deploy; health probes.

Path B — nixos-anywhere / remote install

Follow your ops docs; still need keys + freeze tag.

Path C — Image rehydrate

If Capstone ships generators/ISO product path, use only if documented as DR path.

Theory 3 — Timing and RTO

Record wall clock:

Phase	Start	End	Minutes
Confirm target + backups of keys			
Wipe / reinstall base			
First flake build/substitute			
Secrets decrypt + services up			
Health green			
Total			

Lab RTO target: pick a number (e.g. < 2–4 hours for solo lab) and note whether you met it.

Theory 4 — What wipe proves / does not prove

Proves	Does not prove
Docs completeness	Multi-region HA
Key custody	Perfect security
Pin/lock sufficiency	Infinite disk
Deploy path after install	Capstone B

Theory 5 — Data vs system

Layer	DR handling
System closure	Rebuild from flake
Secrets private keys	Offline restore
App state (DB)	Restore from backups chapter artifacts **if any**
Machine-id / SSH host keys	May change; update sops recipients if needed

If host SSH key is a sops recipient, **re-encrypt** or use dedicated age key that you restore from offline.

Concrete DR checklist

Pre-wipe (mandatory)

- ☐ Freeze tag exists and is the intended recover point
- ☐ Offline keys verified readable
- ☐ Repo pushed / USB clone available
- ☐ DR-PLAN printed or on second device
- ☐ Target disk identity confirmed **three times**
- ☐ Snapshot optional safety net (know if you will refuse to use it)
- ☐ Second machine for docs/timer

Post-rebuild health

- ☐ `hostname` correct
- ☐ SSH works
- ☐ `systemctl --failed` clean
- ☐ Secret path exists; app active
- ☐ Proxy front door probe
- ☐ Deploy path still works from admin

Lab 0 — Confirm freeze and keys

```
git rev-parse capstone-a-freeze
# open offline key medium
# read docs/DR-PLAN.md on a second device
mkdir -p ~/lab/nixos/capstone/dr
```

Lab 1 — Execute wipe per DR-PLAN

Follow **your** plan. Example shape only:

```
# ON INSTALLER - example placeholders
# lsblk   # confirm disk
# disko or manual partition
# nixos-install --flake github:you/capstone?ref=capstone-a-freeze#lab
```

Log every deviation from DR-PLAN in `~/lab/nixos/capstone/dr/DEVIATIONS.md`.

Lab 2 — Time the recovery

Fill the timing table. Be honest about waiting on downloads.

Lab 3 — Secrets rehydrate

```
# restore age key to documented path if needed
# rebuild/switch
systemctl is-active myapp.service
```

If SSH host key recipients broke sops, document fix in SECRETS.md and commit on a **post-DR** branch carefully.

Lab 4 — Health + deploy proof

```
curl -fsS http://127.0.0.1/ || true
# from admin:
deploy .#lab 2>&1 | tee ~/lab/nixos/capstone/dr/redeploy.log
```

Lab 5 — Fix docs (the real deliverable)

Every DEVIATIONS item becomes a DR-PLAN/RUNBOOK patch. Commit doc fixes.

```
git checkout -b capstone/dr-doc-fixes
# edit docs
git commit -m "docs: fix DR gaps found during wipe rebuild"
```

Lab 6 — Optional: refuse snapshot

If you took `pre-dr` snapshot, either:

- **Hard mode:** do not restore it; live with rebuilt system
- **Teaching mode:** note that snapshot was available but unused

Document which mode you chose.

Common gotchas

Mistake	Fix
Wiped wrong disk	Triple-check; hypervisor snapshot on host only
Keys only on wiped disk	Offline first—always
Installer channel freeze pin	Prefer flake install with lock
sops fails after new host keys	Dedicated age key or re-encrypt
“Fixed” by mounting old root	Invalidates proof; redo
No timing log	Fill table now

Checkpoint

- ☐ Wipe executed on correct target
- ☐ Rebuild to healthy Capstone A
- ☐ Timing table filled
- ☐ DEVIATIONS → doc commits
- ☐ CAPSTONE-A DR row ticked with time
- ☐ Three personal gotchas

Commit

```
git add docs/
git commit -m "docs: disaster recovery proof and gap fixes"
```

Write three personal gotchas before continuing.

Next

→ **Rollback and tests** — generation rollback + full suite green.

Rollback and tests

Rollback and tests

Goal: Prove **generation rollback** works after a bad change, run the **full** test/CI suite green, and finalize the **troubleshooting checklist** that ties playbooks to Capstone reality (post-DR host on **NixOS 26.05**).

> **Note**
>
> Disaster recovery proved rebuild from zero. This chapter proves you need not always go to zero: generations + tests + a crisp checklist cover the common middle ground. Do the **real** break—theater fails the exercise.

Why this chapter exists

Most production incidents are not "disk gone." They are "bad deploy an hour ago." Without rollback muscle and a short checklist, you re-image for a typo. Capstone A is incomplete until both DR *and* rollback are proven with logs.

Theory 1 — Rollback layers

Layer	When to use	How
Bootloader generation	Unbootable / broken after reboot	Select previous generation at boot
Runtime rollback	System boots but config bad	`nixos-rebuild switch --rollback` or activate previous profile
Git revert	Want source truth back	`git revert` / deploy freeze tag
Deploy tool	Remote	Redeploy known-good attr/tag
Full DR	Disk/keys/corruption	Disaster recovery path

Capstone must document **which** you try first for remote lab (console vs SSH).

Commands (lab)

```
# list generations
sudo nix-env -p /nix/var/nix/profiles/system --list-generations
ls -la /nix/var/nix/profiles/system*

# rollback switch
sudo nixos-rebuild switch --rollback

# boot previous (if needed)
# use bootloader menu; or:
# sudo /nix/var/nix/profiles/system-<n>-link/bin/switch-to-configuration switch
```

Theory 2 — Bad change protocol

1. Record current generation number.
2. Introduce **one** deliberate bad change (firewall locks SSH only with console, or break proxy upstream).
3. Deploy/switch.
4. Observe failure with checklist.
5. Rollback.
6. Confirm health.
7. Revert git if needed so main is clean.

Good bad-change	Bad bad-change
Wrong proxy upstream	`rm -rf /`
Typo in unit ExecStart	Disk wipe
Assertion-triggering option	Destroying offline keys

Theory 3 — Full suite definition

Layer	Command
Local checks	`nix flake check -L` or explicit check attrs
Toplevel	`nix build .#nixosConfigurations.lab.config.system.build.toplevel -L`
CI	Green workflow on post-DR commit
Manual probes	RUNBOOK health section

All must be green **after** rollback drill.

Theory 4 — Troubleshooting checklist (finalize)

One page `docs/TROUBLESHOOT.md` (or RUNBOOK section):

```
1. Console access?
2. systemctl --failed
3. journalctl -p err -b
4. Generation / rollback?
5. Eval flake?
6. Secrets path permissions?
7. Firewall / ss -lntp?
8. Playbook index?
```

Link PL entries from internals troubleshooting.

Theory 5 — State vs generation (again)

Rollback restores **system configuration**, not necessarily:

- Database contents
- Accidental deletes under `/var`
- Lost age keys

If the bad change was data, use backups—not only generations.

Concrete checklists

Rollback drill

- ☐ Generation number before break recorded
- ☐ Bad change applied and failed health
- ☐ Rollback restored health
- ☐ Log saved
- ☐ Git left clean (or intentional fix commit)

Suite

- ☐ flake check / tests green
- ☐ CI green
- ☐ Manual probes green
- ☐ CAPSTONE-A rollback + tests rows ticked

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/rollback
cd /path/to/capstone-flake
# confirm post-DR host healthy
```

Lab 1 — Record baseline generation

```
sudo nix-env -p /nix/var/nix/profiles/system --list-generations \
  | tee ~/lab/nixos/capstone/rollback/generations-before.txt
readlink -f /run/current-system | tee ~/lab/nixos/capstone/rollback/current-system.txt
```

Lab 2 — Intentional break

Example (adjust to your stack):

```
# temporarily wrong upstream
# roles.proxy.upstream = "http://127.0.0.1:1";
```

```
sudo nixos-rebuild switch --flake .#lab 2>&1 | tee break.log
curl -fsS http://127.0.0.1/ || echo "expected fail"
```

Lab 3 — Rollback

```
sudo nixos-rebuild switch --rollback 2>&1 | tee ~/lab/nixos/capstone/rollback/rollback.log
# health
curl -fsS http://127.0.0.1/ | head
systemctl --failed
```

Acceptance: health restored without full reinstall.

Lab 4 — Git hygiene

```
git checkout -- path/to/broken.nix   # or revert commit
# ensure freeze tag still meaningful
```

Lab 5 — Full suite green

```
nix flake check -L 2>&1 | tee ~/lab/nixos/capstone/rollback/check.log
nix build .#nixosConfigurations.lab.config.system.build.toplevel -L \
  2>&1 | tee ~/lab/nixos/capstone/rollback/toplevel.log
```

Confirm CI green if remote.

Lab 6 — Finalize troubleshooting checklist

Write `docs/TROUBLESHOOT.md`; link playbooks; tick CAPSTONE-A.

Lab 7 — Bootloader path (optional)

If safe, reboot and select previous generation once. Document for remote hosts with console.

Common gotchas

Mistake	Fix
Theater rollback without real break	Do real break
Rollback then leave bad git committed	Clean source
Expecting DB undo from generation	Backups
Skipping CI after DR	Run suite
No generation listed	Profile permissions; use sudo

Checkpoint

- ☐ Rollback log with before/after health
- ☐ Full suite green
- ☐ TROUBLESHOOT checklist finalized
- ☐ CAPSTONE-A complete for rollback+tests
- ☐ Three personal gotchas

Commit

```
git add .
git commit -m "capstone: rollback drill and suite green"
```

Write three personal gotchas before continuing.

Next

→ **Retrospective** — teach plan and roadmap without rewrite thrash.

Retrospective

Retrospective

Goal: Close Capstone with an honest retrospective, a **teach plan** for a friend, and a personal roadmap (next nixpkgs release, nixpkgs contribution, optional Capstone B)—without starting a new rewrite today.

> **Note**
>
> This chapter is reflection and direction, not another feature. Protect the freeze tag, keys, and runbook—they outrank any blog post.

Why this chapter exists

Expertise compounds when you **teach** and **schedule** the next loop. Without a retrospective, months of labs become a blur. Without a roadmap, Capstone B starts as thrash. Without a teach plan, you never test whether your mental model transfers.

Theory 1 — What you should now be able to do

By Capstone A completion you can:

Skill	Book roots
Explain store, generations, rollback	Concepts, host, internals, DR/rollback
Own a flake NixOS host	NixOS host + layout
Compose modules with types/assertions	Host + internals modules
Manage user or service identity	Host + HM
Keep secrets out of the store	Services + Capstone implement
Front door proxy pattern	Services + Capstone
Deploy remotely (deploy-rs/Colmena)	Ops + Capstone harden
Test with nixosTest	Ops + test mastery + Capstone
CI + cache discipline	Packaging/ops + Capstone
GC/store hygiene	Ops
Backup/restore state	Ops backups
Read release notes / plan upgrades	Internals

Skill	Book roots
Rebuild from zero	Capstone DR
Debug with a playbook	Internals troubleshoot + Capstone
Steelman alternatives	Comparative landscape

Mark each: **solid / shaky / missing**. Shaky items feed the roadmap.

Theory 2 — Retrospective structure (blameless)

Section	Prompt
Goals	What Capstone A required
Done	What you actually shipped (with tag SHA)
Waived	What you consciously skipped
Surprises	What docs/tests/DR revealed
Pain	Top 3 time sinks
Pride	Top 3 artifacts
Would change	Process, not fantasy rewrites
Thanks	Upstream/docs/people

Tone: operator journal, not marketing.

Theory 3 — Teach plan (1 session)

Design a **half-day or 1-day** session for a friend who knows Linux but not Nix:

Block	Minutes	Activity
Why Nix exists	20	Store path demo
Rebuild + rollback	40	Live lab guest
Flake tour	30	Your Capstone tree
Secrets story	20	Paths not strings
Q&A	30	Their context

Deliverable: `docs/TEACH-PLAN.md` with commands that work on **your** freeze tag.

Theory 4 — Roadmap options (pick few)

Option	When it makes sense
Track 26.11 upgrade with your runbook	After notes ship; lab first
nixpkgs PR (package or module fix)	You hit a real bug/gap
Capstone B multi-node	Hardware + freeze honesty
devenv/Snowfall adopt	Only if COMPARE said yes
Secure Boot / Lanzaboote	Hardware allows
Write public post	After teach plan works once

Time-bound items ("by date X") beat eternal backlogs.

Theory 5 — What not to do today

Do not	Why
Rewrite flake layout	Freeze protection
Enable CA on Capstone	Internals decision
Add five services	Scope creep
Delete DR logs	Evidence
Un-tick Capstone for aesthetics	Honesty

Lab 0 — Workspace

```
mkdir -p ~/lab/nixos/capstone/retro
cd /path/to/capstone-flake
```

Lab 1 — Skills matrix

Fill `docs/RETRO.md` skills table (solid/shaky/missing) from Theory 1.

Lab 2 — Full retrospective writeup

Write sections from Theory 2. Include:

- Freeze tag SHA
- DR total minutes
- Rollback proof one-liner
- Top waiver

Acceptance: 1 page honest prose + tables.

Lab 3 — Teach plan

Write `docs/TEACH-PLAN.md`. Dry-run the commands yourself once more on lab.

Lab 4 — Roadmap

`docs/ROADMAP.md`: 3–7 items max, each with trigger date or criterion. Link POST-FREEZE.md ideas selectively.

Lab 5 — Capstone A final sign-off

```
## Sign-off
- Capstone A: complete / complete-with-waivers
- Tag: capstone-a-freeze @ <sha>
- Date:
- Operator:
```

Paste into CAPSTONE-A.md.

Lab 6 — Celebrate without thrash

Optional: export PDF of docs, or demo for a friend. **Do not** open a refactor branch "while it's fresh."

Common gotchas

Mistake	Fix
Retro as victory lap only	Include waivers and pain
Teach plan with broken commands	Dry-run
Infinite roadmap	Cap at 7 items
Starting Capstone B tonight	Schedule
Deleting evidence logs	Keep `~/lab/nixos/capstone/**`

Checkpoint

- ☐ `docs/RETRO.md` complete
- ☐ Teach plan dry-runnable
- ☐ Roadmap time-bounded
- ☐ Capstone A sign-off
- ☐ Freeze/docs/keys protected
- ☐ Three personal gotchas

Commit

```
git add docs/
git commit -m "capstone: retrospective teach plan and roadmap"
```

Write three personal gotchas—and stop building for a day.

Next

Return to earlier parts as reference, or explore **Projects** for packaging and distro-branding depth. Capstone A, once signed off, is a **stable base**—not a perpetual construction site.

CI/CD

Setting Up Binary Caches

Setting Up Binary Caches

Compiling llvm on every laptop is not a culture. The boring default is: **a signed substituter (cache.nixos.org plus one team cache), public keys in `nix.conf`, and CI that uploads what it built**.

Mental model

When Nix needs `/nix/store/<hash>-pkg` it:

1. Looks in the local store.
2. Queries each **substituter** for a `.narinfo` signed with a **trusted public key**.
3. Downloads the NAR if the signature matches.
4. Builds from the `.drv` only if nobody had the bytes.

Piece	Role
Substituter URL	`https://cache.nixos.org`, `https://desk.cachix.org`, `http://attic.desk.internal`
Trusted public key	`cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQF`
Signing private key	Lives on CI / the cache host. Never in git.

Cachix is the hosted boring choice. Attic or Harmonia is the self-hosted one. The NixOS module is the same shape: extra substituters + extra keys.

```
laptop  --query-->  desk-cache  --miss-->  cache.nixos.org  --miss-->  build
CI      --build-->  sign  --put-->  desk-cache
```

A substituter **without** its public key is silently skipped (or refused). You compile the world and blame the network.

`nix.settings.substituters` **replaces** the default list if you assign it — keep `https://cache.nixos.org` in the list. `extra-substituters` **appends**. Same for keys: `trusted-public-keys` vs `extra-trusted-public-keys`. `require-sigs` stays true (the default). `nixConfig.extra-substituters` inside `flake.nix` only applies when the user/daemon `accept-flake-config` (or a trusted user); it is not a substitute for `/etc/nix/nix.conf`. Never put a signing secret in `nixConfig`.

Worked examples

Case 1: Consume a team cache on NixOS

Save as `cache_config.nix`:

```
# cache_config.nix
{
  nix.settings = {
    substituters = [
      "https://cache.nixos.org"
      "https://desk-cache.cachix.org"
    ];
    trusted-public-keys = [
      "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
      "desk-cache.cachix.org-1:aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa="
    ];
    require-sigs = true;
  };
}
```

Replace the `desk-cache` key with the real key Cachix showed when you created the cache. A truncated key in this book is **not** trusted; Nix will ignore that substituter.

```
sudo nixos-rebuild switch
nix show-config | grep -E 'substituters|trusted-public-keys'
```

On a multi-user machine the **daemon** reads `/etc/nix/nix.conf`. `~/.config/nix/nix.conf` is ignored for daemon builds.

Untrusted users cannot add substituters. If a laptop must use the team cache without being root:

```
# extraTrusted
{
  nix.settings.trusted-users = [ "root" "deskadmin" ];
}
```

Prefer listing the cache in the **system** `nix.settings` (Case 1) so every uid gets it. `trusted-users` is also who may import unsigned paths — keep it short.

Case 2: Push from a workstation (once)

```
nix build .#desk-api
cachix watch-exec desk-cache -- nix build .#desk-api
```

or:

```
nix build .#desk-api
cachix push desk-cache ./result
```

`CACHIX_AUTH_TOKEN` is an environment variable from the Cachix dashboard. It is a secret. It is not a flake input. Laptops pull; CI pushes. Do not require every laptop to upload.

Case 3: Verify a hit versus a miss

```
nix build .#desk-api --print-build-logs 2>&1 | tee /tmp/desk-build.log
grep -E 'copying path|building' /tmp/desk-build.log | head
```

`copying path '…' from 'https://desk-cache.cachix.org'` is a hit. `building '…drv'` is a miss. A team that still builds llvm locally has a key mismatch or is pushing different hashes (different nixpkgs lock).

```
nix build nixpkgs#hello --max-jobs 0
```

`--max-jobs 0` forbids a local compile. If hello errors, the substituter list is empty or unsigned.

```
nix show-config | grep -E 'substituters|trusted-public-keys'
curl -fsS https://desk-cache.cachix.org/nix-cache-info | head
```

`StoreDir: /nix/store` in `nix-cache-info` is a live cache. HTTP 404 here is why every laptop rebuilds llvm.

Case 4: Serve a tiny cache with `nix copy`

Without Cachix, an S3 bucket or a directory works:

```
nix copy --to file://$PWD/desk-nar .#desk-api
nix copy --from file://$PWD/desk-nar --to ssh://deskadmin@desk-vm .#desk-api
```

`file://` is a lab. Production is HTTP + signatures (`nix-serve`, Harmonia, Attic). Unsigned `file://` on a USB stick is fine for air-gap day; it is not a fleet cache.

Case 5: Generate a signing key for a self-hosted cache

```
nix key generate-secret --key-name desk-cache > /run/secrets/desk-cache.secret
nix key convert-secret-to-public < /run/secrets/desk-cache.secret
```

Put the **public** half in `trusted-public-keys`. Put the secret on the builder only (sops). `nix copy --to 's3://desk-cache?secret-key=/run/secrets/desk-cache.secret'` signs on upload.

The trap

The trap is **adding a substituter without the public key**. Nix will skip it. You will compile the world and blame the network.

The other trap is putting the **private** signing key in `flake.nix` or in a GitHub Actions log (`echo $CACHIX_AUTH_TOKEN`). Rotate immediately.

A third trap: `substituters` in `~/.config/nix/nix.conf` on a multi-user machine. The daemon reads `/etc/nix/nix.conf`.

A fourth: assigning `substituters = [ "https://desk-cache.cachix.org" ];` and **dropping** `cache.nixos.org`. Every `hello` is now a local compile unless your team cache mirrors the world.

The boring rule

- `cache.nixos.org` plus **one** team cache. Not five random community caches you do not audit. Keep cache.nixos.org in the list.
- Public keys next to substituter URLs, in the daemon's `nix.conf`. `require-sigs = true`.
- CI pushes. Laptops pull. Do not require every laptop to upload.
- Never commit signing secrets. `nixConfig` in the flake is not the daemon.
- Confirm a hit with "copying path from …" before you call the cache done.

Try this

1. `nix show-config | grep -E 'substituters|trusted-public-keys'` and identify which keys match which URLs.
2. `nix build nixpkgs#hello --max-jobs 0` (no local build). It should substitute. If it errors, your substituter list is empty.
3. Create a `file://` cache of `hello` (Case 4) and copy it into a second store (`nix copy --from …`).
4. Intentionally typo the public key, rebuild a package you do not have locally, and read the error. Restore the key.

GitHub Actions with Nix

GitHub Actions with Nix

`actions/setup-go` plus `actions/setup-python` plus a matrix of cache keys is how CI drifts from laptops. The boring default is: **install Nix on the runner, `nix flake check` / `nix build`, and push results to the team binary cache**.

Mental model

The runner starts empty. You give it Nix, the same `flake.lock` as the laptop, and a cache token. The graph that `nix develop` used locally is the graph CI builds. There is no second `Dockerfile` for CI.

Step	Job
checkout	The flake and the lock
install Nix	Daemon + flakes
cache auth	Cachix (or magic-nix-cache for GHA's own cache)
`nix flake check`	Eval + checks + tests
`nix build`	The artifacts you ship

Use **pinned action SHAs or release tags**, not `@main`, once the workflow is real. Examples below use tags for readability.

Worked examples

Case 1: Check and build, push to Cachix

Save as `.github/workflows/ci.yml`:

```yaml
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
```

```
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
        with:
          extra_nix_config: |
            extra-substituters = https://desk-cache.cachix.org
            extra-trusted-public-keys = desk-cache.cachix.org-1:aaaaaaaaaaaaaaaaaaaaaaaaaaaa
      - uses: cachix/cachix-action@v15
        with:
          name: desk-cache
          authToken: ${{ secrets.CACHIX_AUTH_TOKEN }}
          skipPush: ${{ github.event_name == 'pull_request' }}
      - name: Check and build
        run: |
          nix flake check -L
          nix build --print-build-logs
          git diff --exit-code flake.lock
```

The flake lock already pins nixpkgs — `nix_path: nixpkgs=channel:…` does nothing for `nix flake check`. Skip it. `extra_nix_config` is how the **runner daemon** sees the team cache (same keys as `/etc/nix/nix.conf` on laptops).

`CACHIX_AUTH_TOKEN` is a repository secret. Pull requests from forks should **not** receive a write token (GitHub's default is correct: `pull_request` from forks cannot read org secrets). `skipPush` on `pull_request` is belt-and-suspenders when the workflow still sees the token on internal PRs.

Do not add `actions/setup-go` next to this job. The flake already has `go`.

If CI rewrote the lock, `git diff --exit-code flake.lock` fails. The lock in git is the pin; the runner does not get a vote.

```
# .github/workflows/ci.yml fragment
permissions:
  contents: read
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
```

`pull_request_target` is still forbidden with a write token (secrets chapter).

Case 2: `nix flake check` is the gate

Save as a flake fragment (merge into `outputs`):

```
# checks.nix - conceptually part of flake outputs
{
  checks.x86_64-linux.fmt = pkgs.runCommand "fmt" { } ''
    echo ok > $out
```

```
    '';
}
```

A failing check fails the job. You do not add a second "lint" job with a different Python.

```
nix flake check
```

Output on success:

```
evaluating flake...
checking flake...
```

Case 3: Magic Nix Cache when you do not have Cachix yet

```
# .github/workflows/ci-magic.yml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: DeterminateSystems/nix-installer-action@v16
      - uses: DeterminateSystems/magic-nix-cache-action@v8
      - run: nix build
```

This uses GitHub's cache API. It is per-repository and evicts. It is a start. A team cache (Cachix/Attic) is what laptops share with CI.

Case 4: Pin the installer, pin nixpkgs

The flake lock already pins nixpkgs. The installer action should not float `@main`:

```
- uses: cachix/install-nix-action@v27
```

When you bump, bump in a dedicated commit. Same discipline as `flake.lock`.

Case 5: Free disk on the runner

GitHub's Ubuntu image is busy. If `nix build` dies with `No space left on device`:

```
- name: Free disk
  run: |
    sudo rm -rf /usr/share/dotnet /opt/ghc /usr/local/share/boost
    df -h
```

Then GC at the end of self-hosted runners only (`nix-collect-garbage -d`). Hosted runners die anyway.

The trap

The trap is **`nix-env -iA nixpkgs.go` in CI** after installing Nix, then `go test`. You just rebuilt the old world: an unpinned go, no flake, no cache sharing with the laptop.

The other trap is `cachix-action` with a write token on `pull_request_target`. A fork PR can then push malware NARs to the team cache. Write tokens: `push` to `main` and trusted workflows only.

A third: `nix_path: nixpkgs=channel:nixos-unstable` next to a 26.05 flake, then a step that uses `<nixpkgs>` — two pins, two worlds. A fourth: installing Nix twice (Cachix **and** Determinate) in one job.

The boring rule

- One install-Nix action, then `nix flake check` and `nix build`. `git diff --exit-code flake.lock`.
- Same `flake.lock` as developers. Team cache via `extra_nix_config`, not a channel `nix_path`.
- Push to the team cache from trusted branches (`skipPush` on `pull_request`).
- Pin action versions. One installer, not two.
- Do not `setup-go` / `setup-python` next to Nix "just in case."

Try this

1. Add Case 1 to a throwaway repo with a `flake.nix` that outputs `packages.x86_64-linux.default = pkgs.hello`. Confirm the Actions log shows a substitute or a build, then a second run that hits the cache.
2. Break `flake.nix` syntax, push, and confirm the job fails at eval, not at a later custom script.
3. List repository secrets and confirm `CACHIX_AUTH_TOKEN` is not printed in logs (`echo ${{ secrets.CACHIX_AUTH_TOKEN }}` is a firing offence).
4. Time `nix flake check` locally versus in CI. If CI is 10× slower, you are compiling; fix substituters (`extra_nix_config` keys).
5. Confirm the workflow YAML has no `pull_request_target` and no `nix_path` channel.

GitLab CI Integration

GitLab CI Integration

GitLab jobs are containers. The boring default is: **a Nix image (or a runner that already has Nix), flakes enabled, `nix flake check` / `nix build`, and a cache that is not `GIT_STRATEGY: none` plus hope**.

Mental model

Runner kind	How Nix appears
Shared SaaS runner	Job image `nixos/nix` (or `gitpod/workspace-nix`)
Self-hosted with Nix	`image:` optional; use the host store + a mounted `/nix`
Kubernetes executor	Persistent `/nix` volume or you download the world every job

`NIX_CONFIG` in `variables:` is the reliable way to turn on flakes inside `nixos/nix` images that ship a conservative `nix.conf`.

The same binary cache as GitHub Actions applies. Cachix works from GitLab. `CACHIX_AUTH_TOKEN` is a GitLab CI/CD variable, masked, protected if it can write.

Worked examples

Case 1: Official Nix image, flakes on

Save as `.gitlab-ci.yml`:

```yaml
# .gitlab-ci.yml
image: nixos/nix:2.35.2

variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"

stages:
  - test
  - build
```

```
test-job:
  stage: test
  script:
    - nix flake check

build-job:
  stage: build
  script:
    - nix build --print-build-logs
    - nix path-info -Shr ./result
```

Pin the image tag (`2.35.2`), not `latest`.

Case 2: Cachix from GitLab

```
# .gitlab-ci.yml
image: nixos/nix:2.35.2

variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"

before_script:
  - nix-env -iA nixpkgs.cachix
  - cachix use desk-cache
  - echo "$CACHIX_AUTH_TOKEN" | cachix authtoken --stdin

build:
  script:
    - nix flake check
    - nix build
    - cachix push desk-cache ./result
```

`nix-env -iA` for **cachix the client** on a throwaway image is acceptable. Do not use it to install the application under test.

A cleaner variant: `nix run github:NixOS/nixpkgs/nixos-26.05#cachix -- use desk-cache`.

Pin the image by digest when the runner can pull it:

```
image: nixos/nix:2.35.2@sha256:aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

`GIT_STRATEGY: clone` plus a shallow clone is fine. `GIT_STRATEGY: none` plus a mystery workspace is how you test last week's `flake.lock`.

Case 3: Cache `/nix` on a self-hosted runner

On a runner VM that keeps its disk:

```yaml
# .gitlab-ci.yml
default:
  tags: [nix-runner]
  image: nixos/nix:2.35.2
  cache:
    key: nix-store
    paths:
      - /nix/store
```

GitLab's `cache: paths` on `/nix/store` is **slow and incomplete** (no `db.sqlite`). Prefer a **persistent runner** with Nix installed on the host and:

```yaml
# .gitlab-ci.yml
default:
  tags: [nix-host]
  # no image: - job runs on the host
variables:
  NIX_CONFIG: "experimental-features = nix-command flakes"
build:
  script:
    - nix flake check
    - nix build
```

The host store **is** the cache. GC with `--delete-older-than 7d` in a nightly cron.

Case 4: `rules` so forks do not push

```yaml
# .gitlab-ci.yml
push-cache:
  script:
    - nix build
    - cachix push desk-cache ./result
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Write tokens stay on the default branch.

Case 5: Show what you shipped

```yaml
# .gitlab-ci.yml
build:
  script:
    - nix build
    - nix path-info -Shr ./result
    - nix-store -q --tree ./result | head -40
```

```
  artifacts:
    paths:
      - result
    expire_in: 1 day
```

The `result` symlink as an artifact is **not** a closure. It is a pointer that will not work on a machine without `/nix`. Use `nix copy --to file://$PWD/nar` and artifact `nar/` if humans must download.

The trap

The trap is **`image: ubuntu:24.04` plus `curl | sh` the Nix installer on every job**. You pay minutes and get a different Nix version than last week. Use a Nix image or a Nix-equipped runner.

The other trap is caching only `result` in GitLab artifacts and calling it a binary cache. Artifacts are not signed NARs.

The boring rule

- Pin `nixos/nix:<version>` (digest if you can) or use a self-hosted runner that already has Nix.
- `nix run nixpkgs#cachix`, not `nix-env` for the app under test.
- `NIX_CONFIG` enables flakes inside the job.
- Same team cache as GitHub Actions and laptops.
- Write tokens only on the default branch.
- `nix path-info -Shr` in the log so a fat closure is visible in review.

Try this

1. Run Case 1 against a flake that builds `hello`. Confirm the job log contains `Hello` or a store path for hello.
2. Pin the image digest (`nixos/nix:2.35.2@sha256:…`) and record why that is stricter than a moving tag.
3. Add `nix path-info -Shr ./result` and paste the closure size into the merge request template once.
4. On a self-hosted runner, run `nix-collect-garbage --delete-older-than 7d` after a week and note `df -h /nix`.

Build Caching Strategies

Build Caching Strategies

CI that recompiles llvm because someone edited `README.md` is not a pipeline, it is a heater. The boring default is: **substituters first, a persistent team cache second, GitHub's cache API only as a bump, and `src` filters so docs do not bust compiles**.

Mental model

Nix does not cache "step 3 of a Dockerfile." It caches **store paths**. If the `.drv` hash is unchanged, the path is substituted. If `src` includes the whole git tree, every commit is a new `.drv`.

Layers, cheapest first:

Layer	Lifetime	Who uses it
Local `/nix/store`	Until GC	This runner / laptop
Team cache (Cachix / Attic)	Months	Every laptop + every CI job
`magic-nix-cache` (GHA)	Days, per repo	GitHub-hosted runners only

Docker layer cache busts everything after a changed line. Nix rebuilds only the node whose inputs moved, plus dependents. A docs derivation and a Go derivation are siblings, not a stack.

```
README.md change
      hello-docs.drv   rebuilt
      desk-api.drv     same hash → substitute
```

Worked examples

Case 1: Team cache is the real cache

The binary-cache chapter already added substituters. CI must **push** what it built:

```yaml
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
    branches: [main]
```

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - uses: cachix/cachix-action@v15
        with:
          name: desk-cache
          authToken: ${{ secrets.CACHIX_AUTH_TOKEN }}
      - run: nix build --print-build-logs
```

A laptop with the same `flake.lock` then substitutes. `magic-nix-cache` alone does **not** help the laptop.

Case 2: magic-nix-cache as a spare tyre

```
# .github/workflows/ci-magic.yml
name: CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: DeterminateSystems/nix-installer-action@v16
      - uses: DeterminateSystems/magic-nix-cache-action@v8
      - run: nix build
```

Fine for a personal repo. Evicts. Does not sign NARs for other machines. Graduate to Cachix when a second human waits on CI.

Case 3: `lib.cleanSource` / gitignore

Save as `package.nix`:

```
# package.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.stdenv.mkDerivation {
  pname = "desk-banner";
  version = "1.0.0";
  src = pkgs.lib.cleanSource ./.;
  buildPhase = "echo ok > $out";
  installPhase = "true";
```

```
}
```

`cleanSource` drops `.git`, `result`, and editor junk. Stricter:

```
src = pkgs.lib.cleanSourceWith {
  src = ./.;
  filter = path: type:
    baseNameOf path != "README.md"
    && baseNameOf path != "docs";
};
```

Or `pkgs.nix-gitignore.gitignoreSource [ ] ./.` with a `.gitignore` you actually maintain.

```
nix-build package.nix
echo "# note" >> README.md
nix-build package.nix
```

If the second build is "checking paths" / a cache hit, the filter worked. If it rebuilds, `README.md` is still in `src`.

Case 4: Vendor once, compile often

Save as `go.nix`:

```
# go.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.buildGoModule {
  pname = "desk-api";
  version = "1.0.0";
  src = pkgs.lib.cleanSource ./.;
  vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

`vendorHash` is a FOD. Changing `main.go` does not re-fetch modules. Changing `go.mod` does. Probe the hash once; do not set `vendorHash = null` in CI "to make it work."

Case 5: See what would rebuild

```
nix build --dry-run
nix path-info -Shr ./result | tail -1
```

`--dry-run` lists derivations that are not in the store or substituters. If that list includes `gcc` after a comment-only change, `src` is too wide or an input leaked (`currentTime`, impure env).

```
nix build .#desk-api --rebuild --dry-run 2>&1 | head
```

`--rebuild` pretends the path is missing. Use it to see the **graph**, not to heat the room. `nix why-depends` after a surprise rebuild tells you which input moved (often `flake.lock` or an unfiltered `src`).

The trap

The trap is **`src = ./.`** on a monorepo root for a leaf package. A frontend CSS change rebuilds the Go API. Filter to `./cmd/desk-api` or `fileset` (`lib.fileset.gitTracked`).

The other trap is treating GHA cache as the team cache. A new contributor's first `nix build` still compiles the world.

The boring rule

- Push to a signed team cache from trusted CI. Laptops pull.
- Filter `src`. Docs and `.git` are not compile inputs.
- Language vendor hashes (Go, Cargo, npm) are FODs. Pin them.
- `--dry-run` after a trivial edit. If gcc is in the list, stop and fix inputs.
- magic-nix-cache is optional. Substituters are not.
- `--dry-run` / `why-depends` before you blame the cache. Filtered `src` before you add another cache.

Try this

1. `nix path-info -Shr ./result` before and after a README-only commit. The store path of the binary should be identical if `src` is filtered.
2. `--dry-run` twice in a row with no edits. Second list should be empty (or only substitutes).
3. Break `vendorHash` by one character, watch the FOD mismatch, restore it.
4. Time `nix build` on a clean store with the team cache configured versus `nix build --option substituters ''`. The delta is the cache earning its keep.

Testing in CI Pipelines

Testing in CI Pipelines

Tests that need the office Wi-Fi are not tests. The boring default is: **hermetic `checkPhase` / `go test` inside the derivation, `nix flake check` on every PR against nixpkgs 26.05, and NixOS VM tests when a unit must listen**.

Mental model

The sandbox has no network. `go test` that downloads modules fails. `vendorHash` (FOD) already fetched modules. `npm test` that hits staging fails. Mock it.

Kind	Where
Unit	`doCheck` / `checkPhase` on the package
Lint	`checks.<system>.lint`
Integration	`testers.runNixOSTest` (needs KVM)
Gate	`nix flake check -L` in CI

GitHub Actions: install Nix 2.35 (or the 26.05-pinned Nix), then `nix flake check`. No `setup-go`. Staging probes are **deploy** jobs, not `checks`.

Worked examples

Case 1: Hermetic Go tests

Save as `service.nix`:

```
# service.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.buildGoModule {
  pname = "desk-service";
  version = "1.0.0";
  src = pkgs.lib.cleanSource ./.;
  vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  doCheck = true;
}
```

`buildGoModule` already runs `go test` when `doCheck` is true. Probe `vendorHash`. `null` is not a pin.

```
nix-build service.nix
```

Case 2: Flake checks

Save as `flake.nix`:

```
# flake.nix
{
  description = "desk-service checks";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
      pkg = pkgs.callPackage ./service.nix { };
    in
    {
      packages.${system}.default = pkg;
      checks.${system}.unit = pkg;
      checks.${system}.vet = pkgs.runCommand "vet" {
        nativeBuildInputs = [ pkgs.go ];
        src = ./.;
      } ''
        cd $src
        GOPROXY=off go vet ./...
        touch $out
      '';
    };
}
```

`GOPROXY=off` proves you are not downloading. If vet needs modules, use the same `buildGoModule` env.

```
nix flake check --print-build-logs
```

Case 3: CI workflow

Save as `.github/workflows/check.yml`:

```yaml
# .github/workflows/check.yml
name: check
on: [pull_request, push]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - uses: cachix/cachix-action@v15
        with:
          name: desk-cache
          authToken: ${{ secrets.CACHIX_AUTH_TOKEN }}
          skipPush: ${{ github.event_name == 'pull_request' }}
      - run: nix flake check -L
```

Push cache on `main` only. `CACHIX_AUTH_TOKEN` is a GitHub secret, not a flake input.

Case 4: VM test job (KVM)

```nix
# test-nginx.nix fragment in checks
{
  checks.x86_64-linux.nginx = pkgs.testers.runNixOSTest {
    name = "desk-nginx";
    nodes.machine = { services.nginx.enable = true; };
    testScript = ''
      machine.wait_for_unit("nginx.service")
      machine.succeed("curl -f http://127.0.0.1/")
    '';
  };
}
```

```
nix build .#checks.x86_64-linux.nginx -L
```

GitHub hosted runners often lack nested KVM. Use a self-hosted runner with `/dev/kvm` or skip VM tests on GHA and run them on the builder.

Case 5: Fail the gate

```
# break a test, then:
nix flake check -L
echo $?
```

Non-zero. The PR does not merge. Logs are in `nix log` / `-L`, not in a screenshot. Run check twice: the second should substitute.

The trap

The trap is `go test that hits https://staging.desk.internal`. Sandbox: connection refused. "Works on my laptop" because your laptop is not a sandbox. `doCheck` must be hermetic. Integration against staging is a **deploy** job with secrets, not `flake check`.

The boring rule

- `nix flake check -L` is the PR gate. 26.05 pin. Nix 2.35 on the runner.
- `vendorHash` / `cargoHash` so tests do not download.
- VM tests need KVM; do not pretend GHA ubuntu has it.
- No prod credentials in check.
- `-L` so failures are in the log.

Try this

1. Flip an assertion, `nix flake check`, restore.
2. Add `curl https://example.com` to `checkPhase` and confirm sandbox failure.
3. `nix flake check` twice; second should substitute.
4. Document whether VM tests run on GHA or on the builder.

Secrets Management in CI

Secrets Management in CI

The store is world-readable. CI logs are copied. Fork pull requests are hostile. The boring default is: **no secret bytes in Nix; `access-tokens` for private inputs; Age/sops keys only on trusted jobs; never `pull_request_target` plus a write token**.

Mental model

Kind of secret	Where it lives	How Nix sees it
GitHub PAT for private flakes	Runner `nix.conf` `access-tokens`	Fetch, not a derivation input
Age private key for sops	Runner env / `SOPS_AGE_KEY`	Decrypt at **eval/deploy** time
DB password for the app	Encrypted in git; decrypted on the **host**	Path `/run/secrets/…` only
Cachix write token	Actions secret	`cachix push`, not `flake.nix`

Builds should not need production passwords. If `nix flake check` requires Vault, the check is not hermetic.

```
trusted push to main
    → runner gets tokens
    → nix build   (no app secrets)
    → deploy      (decrypt on target, not in the NAR)
```

Worked examples

Case 1: Private flake inputs without putting the PAT in Nix

Save as `.github/workflows/deploy.yml`:

```yaml
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]

jobs:
```

```
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
        with:
          extra_nix_config: |
            access-tokens = github.com=${{ secrets.GH_PAT_FOR_PRIVATE_MODULES }}
      - run: nix flake check
      - run: nix build
```

`access-tokens` is daemon config. The token is not hashed into `desk-api`. Fork PRs must **not** receive this secret (GitHub's default for `pull_request` from forks).

Do **not** put the PAT in `flake.nix`:

```
# flake.nix - do not
{
  nixConfig.access-tokens = { github.com = "ghp_…"; };
}
```

That file is world-readable in git and in the eval. `nixConfig.extra-substituters` without matching `trusted-public-keys` is a different bug (unsigned cache, or compile-the-world).

Prefer GitHub **OIDC** (`id-token: write`) to a Cachix/cloud role over a long-lived PAT in Actions secrets, when the other side supports it. The PAT in Case 1 is for private flake inputs that still require one.

```
# .github/workflows/deploy.yml fragment
permissions:
  contents: read
  id-token: write
```

Default `GITHUB_TOKEN` is enough to checkout **this** repo. A PAT is only for **other** private GitHub flake inputs. Scope it `repo:read` on those repos, not `admin:org`.

Case 2: Eval sops without decrypting values into the store

```
# .github/workflows/sops-eval.yml
name: Sops eval
on:
  push:
    branches: [main]
jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - name: Eval secret *paths*
        env:
          SOPS_AGE_KEY: ${{ secrets.CI_AGE_PRIVATE_KEY }}
        run: nix eval .#nixosConfigurations.desk-server.config.sops.secrets
```

The eval prints **paths** (`/run/secrets/db-password`), not password strings. If the log shows a password, the module leaked it into config (usually `hashedPassword = "plaintext"` or `environment.FOO = config.sops.secrets.x.path` is fine; interpolating `builtins.readFile` is not).

Case 3: The leak, on purpose, once

Save as `leak.nix`:

```
# leak.nix
{ pkgs ? import <nixpkgs> {} }:

pkgs.writeText "database.conf" ''
  password = SuperSecretPassword123
''
```

```
nix-build leak.nix
cat result
nix path-info result
```

Output:

```
password = SuperSecretPassword123
/nix/store/…-database.conf
```

Anyone with `nix copy` of this closure has the password. Delete the file, GC, **rotate** the password. Then never do this.

Case 4: Cachix write only on main

```
# .github/workflows/push-cache.yml
name: Push cache
on:
  push:
    branches: [main]
jobs:
  push:
    runs-on: ubuntu-latest
```

```yaml
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - uses: cachix/cachix-action@v15
        with:
          name: desk-cache
          authToken: ${{ secrets.CACHIX_AUTH_TOKEN }}
      - run: nix build
```

Do not use `pull_request_target` with this token. That event runs **your** workflow file from the base branch with **secrets**, but checks out (if you are not careful) or executes code from the fork. A fork PR can `cachix push` garbage — or steal the token from the log / a `curl | sh` step.

Event	Secrets	Runs whose YAML?
`pull_request` (fork)	No (GitHub default)	Merge of base + fork
`push` to `main`	Yes	Trusted
`pull_request_target`	**Yes**	Base workflow, attacker-controlled checkout if you `ref: ${{ github.event.pull_request.head.sha }}`

`pull_request_target` plus `actions/checkout` of the PR head plus a write token is a compromise of the cache and of any deploy key. Use `pull_request` for build checks; deploy only on `push` to `main`.

Case 5: `nix flake check` without credentials

```
unset SOPS_AGE_KEY CACHIX_AUTH_TOKEN GH_TOKEN
nix flake check
```

If this fails because sops cannot decrypt, split checks: eval that only looks at **paths** and types, versus a deploy job that has keys. Checks that need production secrets will fail on contributor laptops too.

The trap

The trap is `builtins.readFile ./prod.key` in a module. Eval copies the bytes into the store. `nix why-depends /run/current-system` will show a path whose `cat` is the key.

The other trap is printing secrets in `set -x` bash. `echo "$SOPS_AGE_KEY"` in a workflow is a rotation event.

A third is `pull_request_target` + **Cachix write / deploy key**. A fourth is `nixConfig.access-tokens` committed in the flake.

The boring rule

- Store is public. GitHub logs are public-ish. Treat both as hostile.
- `access-tokens` in **runner** `nix.conf` (Actions `extra_nix_config`), never in `flake.nix`.
- Encrypted files in git for app secrets. Decrypt on the **host**.
- Write tokens (Cachix, deploy keys) only on `push` to `main`. Never `pull_request_target`.
- `nix flake check` without production credentials.
- After any leak, rotate. GC does not un-copy a cache.

Try this

1. Case 3 on a throwaway password. `cat $(nix-build leak.nix --no-out-link)`. Then GC and confirm you still understand copies in the cache if you had pushed.
2. `nix-instantiate --eval -E 'builtins.readFile ./test.txt'` with a dummy file. Note that the string is in the eval result.
3. Confirm `nix flake check` on a clone with no `SOPS_AGE_KEY`.
4. Open the Actions secret list and tick "review logs for accidental echo" on the last deploy job.
5. Search the repo for `pull_request_target` and for `nixConfig.access-tokens`. Both should be empty.

Remote Builders

Remote Builders

Binary caches move **results**. Remote builders move **work**. The boring default is: **evaluate on the laptop, realise on a machine that has CPU, KVM, and the right `system`, then push the result to the team cache**.

A laptop should not compile Chromium. A Darwin laptop cannot natively produce `x86_64-linux` test VMs. Both problems are remote builders, not "more `-j`".

Mental model

	Binary cache	Remote builder
Needs the path to already exist?	Yes	No
What moves first	NAR (result)	`.drv` + input paths, then NAR back
Trust	Signing keys	SSH user on the builder
Typical role	Everyone pulls	One beefy box (or a queue) builds

```
laptop  --drv-->  builder  --push-->  cache  --substitute-->  fleet
```

`nix.buildMachines` (NixOS) or `--builders` (CLI) lists machines:

- `hostName`, `sshUser`, `sshKey`
- `system` (`x86_64-linux`, `aarch64-linux`, …)
- `protocol` (`ssh-ng` preferred)
- `maxJobs`, `speedFactor`
- `supportedFeatures` (`kvm`, `big-parallel`, `nixos-test`)

If a derivation requires `kvm` and the builder does not advertise it, Nix builds locally (or fails). Mysterious local compiles are often a missing feature flag.

`ssh-ng` is the Nix 2 protocol (one SSH session, multiplexed). Plain `ssh` (legacy) copies NARs over a new connection per path — slower, more fragile. Prefer `protocol = "ssh-ng"`. `mandatoryFeatures` is the strict sibling of `supportedFeatures`: the derivation **must** have those features or Nix will not pick the machine. Use it so a `big-parallel` Chromium job cannot land on a 2-core builder that merely listed `kvm`.

Worked examples

Case 1: NixOS client configuration

Save as `builders.nix`:

```
# builders.nix
{
  nix.distributedBuilds = true;
  nix.settings.builders-use-substitutes = true;

  nix.buildMachines = [
    {
      hostName = "builder.desk.internal";
      system = "x86_64-linux";
      protocol = "ssh-ng";
      sshUser = "nixbuilder";
      sshKey = "/root/.ssh/id_builder";
      maxJobs = 8;
      speedFactor = 4;
      supportedFeatures = [ "nixos-test" "big-parallel" "kvm" ];
    }
  ];
}
```

`builders-use-substitutes = true` lets the **builder** download from the team cache instead of asking the laptop to upload gcc.

The SSH key is a host key for root (the daemon builds as root's SSH). Protect it with sops. `nixbuilder` on the far side must be a trusted Nix user.

```
sudo nixos-rebuild switch
```

Case 2: One-shot without a module

```
nix build .#desk-api --max-jobs 0 \
  --builders 'ssh-ng://nixbuilder@builder.desk.internal x86_64-linux - 8 1 kvm,big-parallel,
```

`--max-jobs 0` means "do not build locally." If the builder is down, the command fails instead of melting the laptop.

Case 3: Builder host — NixOS

Save as `builder-host.nix`:

```
# builder-host.nix
{ pkgs, ... }:

{
  users.users.nixbuilder = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    openssh.authorizedKeys.keys = [
      "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskLaptopRootBuilderKey desk-laptop"
    ];
  };

  nix.settings.trusted-users = [ "root" "nixbuilder" ];
  nix.settings.max-jobs = 8;

  services.openssh.enable = true;
}
```

`trusted-users` is required for `nixbuilder` to start builds as the daemon. Do not put your whole team in `trusted-users` on a laptop; do it on the builder.

Test SSH **as the key the daemon will use**:

```
sudo ssh -i /root/.ssh/id_builder nixbuilder@builder.desk.internal nix-store --version
```

If this asks for a password, the daemon will hang.

Case 4: Darwin → Linux

An Apple silicon laptop producing Linux containers:

```
# darwin-builders.nix
{
  nix.distributedBuilds = true;
  nix.buildMachines = [
    {
      hostName = "linux-builder.desk.internal";
      system = "aarch64-linux";
      protocol = "ssh-ng";
      sshUser = "nixbuilder";
      sshKey = "/var/root/.ssh/id_builder";
      maxJobs = 8;
      supportedFeatures = [ "kvm" "big-parallel" "nixos-test" ];
    }
  ];
}
```

`linux-builder` can be a VM on the Mac (`nix.linux-builder.enable` on nix-darwin) or a real ARM server. Prefer a real server for `nixosTest` (KVM). The VM is fine for `GOARCH=arm64` CGO-off Go; it is a poor Hydra.

Root on the laptop does not read `~deskadmin/.ssh/config`. Put `Host builder` in `/root/.ssh/config` or use an IP in `hostName`.

Case 5: Features — why tests ran locally

```
nix build .#checks.x86_64-linux.desk-vm --max-jobs 0
```

If the check is a `nixosTest`, the builder **must** list `kvm` and `nixos-test`. Missing those, Nix tries the local machine; on Darwin that is a slow VM or a failure.

```
nix show-config | grep builders
```

Confirm the live config, not just the `.nix` file you meant to switch.

The trap

The trap is **pointing `buildMachines` at a machine that is also a substituter with a different nixpkgs**. You upload inputs, it rebuilds everything because its store is on 23.11. Same `flake.lock` everywhere; builder substitutes from the team cache (`builders-use-substitutes`).

The other trap is user SSH config (`~/.ssh/config` Host aliases) that root's daemon does not read. Use `hostName` IPs or put the alias in `/root/.ssh/config`.

The boring rule

- Cache for results, builders for first compiles and foreign `system`.
- `ssh-ng`, `builders-use-substitutes`, `--max-jobs 0` when you mean "remote only."
- Advertise `kvm` / `nixos-test` / `big-parallel` honestly.
- Daemon SSH keys, not your interactive `ssh-agent`, unless you designed for that.
- Builder nixpkgs + cache = laptop flake lock + team cache.

Try this

1. `sudo ssh` from Case 3 until `nix-store --version` prints without a password.
2. `nix build nixpkgs#hello --max-jobs 0 --builders '…'` and watch the builder's `nix-daemon` log (`journalctl -fu nix-daemon`).
3. Remove `kvm` from `supportedFeatures`, run a `nixosTest`, and note where it builds. Put `kvm` back.
4. After a remote build, `nix path-info -Shr ./result` on the laptop — the NAR should have been copied back. Then `cachix push` so the next laptop never builds it.

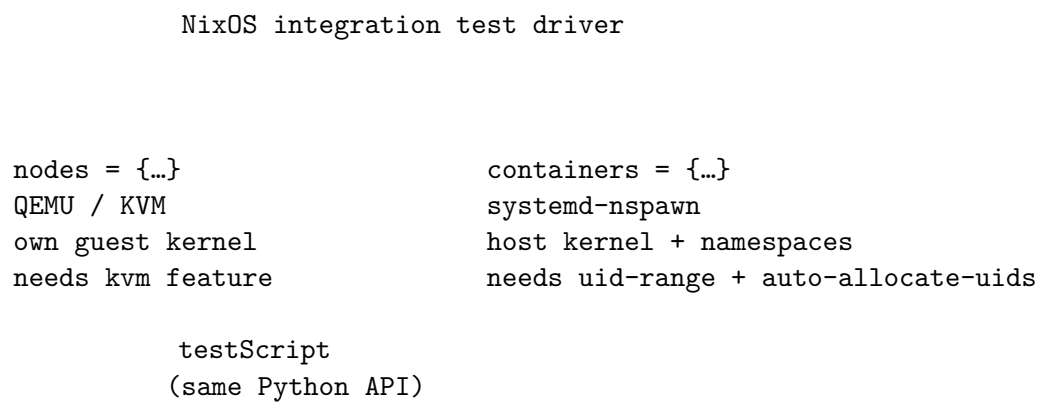
systemd-nspawn Test Backend

systemd-nspawn Test Backend

NixOS 26.05 "Yarara" added **systemd-nspawn** as an alternative backend to QEMU for the integration test driver (nixpkgs #470248, #478109, #479968). Most service and network tests do not need a full guest kernel. The boring default is: **declare those machines under `containers` instead of `nodes`, keep the same `testScript` API, configure the host Nix daemon for `uid-range` / auto-allocate-uids / cgroups once, and reserve QEMU (`nodes`) for kernel, graphics, specialisations, SUID, and restrictive systemd hardening.**

Containers share the host kernel, boot faster, use less memory, run on nested VM builders without KVM, and can bind-mount host devices (GPU/CUDA). That is a different isolation story than QEMU — not a drop-in replacement for every test.

Mental model



Piece	Role
`nodes`	QEMU VMs — full hardware model, guest kernel
`containers`	systemd-nspawn — fast, light, host kernel
`testScript`	Unchanged Python driver API (`start_all`, `.succeed`, …)
`nix.settings` on the **builder host**	Daemon knobs so nspawn can allocate UIDs and cgroups
`/dev/net` in the sandbox	Only when containers and VMs share a VLAN

Flip the declarative attrset. Do not rewrite the script. Prefer containers for "does this unit listen / can these hosts ping / does NAT work." Prefer QEMU when the claim under test is about the

kernel, a display, a specialisation, a setuid wrapper, or hardening options the sandbox will not honor inside nspawn.

Worked examples

Case 1: Host daemon settings for container tests

On the **builder** (desk laptop, CI runner, or remote Nix machine that will evaluate and run the test), enable the features the nspawn backend requires. Save as `nspawn-builder.nix` and import it on that host:

```
# nspawn-builder.nix
{
  # Required so systemd-nspawn tests can allocate UIDs and manage cgroups.
  nix.settings = {
    auto-allocate-uids = true;
    experimental-features = [ "auto-allocate-uids" "cgroups" ];
    extra-system-features = [ "uid-range" ];

    # Only when a single test mixes containers + VMs that share a VLAN:
    # extra-sandbox-paths = [ "/dev/net" ];
  };
}
```

Apply and confirm the daemon sees the features (exact list may include other features you already advertise):

```
sudo nixos-rebuild switch --flake .#builder
nix show-config | rg -e 'auto-allocate-uids|system-features|experimental-features|sandbox-pa
```

Representative snippets:

```
auto-allocate-uids = true
experimental-features = auto-allocate-uids cgroups nix-command flakes
system-features = … uid-range …
```

Without `uid-range` / auto-allocate-uids / cgroups, container tests fail at build time or when nspawn tries to start — not with a friendly "flip this option" banner. Fix the host once; every `containers = { … }` test on that builder benefits.

Case 2: Minimal two-machine ping with `containers`

Migrate the classic two-host ping by renaming `nodes` → `containers`. The `testScript` stays the same:

```
# ping-containers.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.testers.runNixOSTest {
  name = "ping-containers";

  # systemd-nspawn backend - not QEMU
  containers = {
    machine1 = { };
    machine2 = { };
  };

  testScript = ''
    start_all()

    machine1.succeed("ping -c 1 machine2")
    machine2.succeed("ping -c 1 machine1")
  '';
}
```

Build it (host must already have Case 1 settings):

```
nix-build ping-containers.nix -L
```

Representative success shape (timings vary; container boots are typically a few seconds for a trivial ping):

```
machine1 # ping -c 1 machine2
… 1 packets transmitted, 1 received …
machine2 # ping -c 1 machine1
… 1 packets transmitted, 1 received …
/nix/store/…-vm-test-run-ping-containers
```

Or as a flake check against the 26.05 pin:

```
# flake.nix fragment
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    checks.x86_64-linux.ping-containers =
      nixpkgs.legacyPackages.x86_64-linux.testers.runNixOSTest {
        name = "ping-containers";
        containers = {
          machine1 = { };
          machine2 = { };
        };
```

```
        testScript = ''
          start_all()
          machine1.succeed("ping -c 1 machine2")
          machine2.succeed("ping -c 1 machine1")
        '';
      };
  };
}
```

```
nix flake check -L
# or:
nix build .#checks.x86_64-linux.ping-containers -L
```

No `/dev/kvm` required. Nested builders that cannot offer KVM can still run this check.

Case 3: Keep QEMU `nodes` when the claim needs a real VM

Containers are wrong when the test asserts something about a guest kernel, a graphical session, a specialisation, setuid in the sandbox, or systemd hardening the sandbox strips. Keep (or restore) `nodes`:

```
# kernel-module.nix - must stay on QEMU
{ pkgs ? import <nixpkgs> { } }:

pkgs.testers.runNixOSTest {
  name = "desk-kernel-module";

  nodes.machine = { pkgs, ... }: {
    boot.kernelModules = [ "dummy" ];
    # … load / unload a module, assert lsmod, etc.
  };

  testScript = ''
    machine.wait_for_unit("multi-user.target")
    machine.succeed("lsmod | grep -q '^dummy'")
  '';
}
```

Prefer `containers`	Prefer `nodes` (QEMU)
Unit start / listen / curl	Kernel modules, custom kernels
Multi-host IP / NAT / DNS	Graphical apps / screenshots
Cheap CI without KVM	Specialisations
GPU bind-mount experiments	SUID wrappers under the Nix sandbox
Fast feedback loops	Restrictive `ProtectSystem=`-style hardening many modules enable

```
# Still needs kvm on the builder:
nix-build kernel-module.nix -L
```

If a PR "migrates everything to containers" and green-washes a kernel or GUI claim, the suite got faster by testing less. Split the suite: container jobs for services, QEMU jobs for the narrow kernel/graphics set.

Case 4: Mixed containers + VMs sharing a VLAN need `/dev/net`

One test can declare both attrsets. Shared VLAN bridging needs `/dev/net` inside the sandbox:

```
# mixed-vlan.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.testers.runNixOSTest {
  name = "mixed-vlan";

  containers.edge = { ... }: {
    virtualisation.vlans = [ 1 ];
  };

  nodes.core = { ... }: {
    virtualisation.vlans = [ 1 ];
  };

  testScript = ''
    start_all()
    edge.wait_for_unit("network.target")
    core.wait_for_unit("network.target")
    edge.succeed("ping -c 1 core")
  '';
}
```

On the builder, uncomment the sandbox path from Case 1:

```
# nspawn-builder-mixed.nix
{
  nix.settings = {
    auto-allocate-uids = true;
    experimental-features = [ "auto-allocate-uids" "cgroups" ];
    extra-system-features = [ "uid-range" ];
    # Bridge nspawn  QEMU over TAP
    extra-sandbox-paths = [ "/dev/net" ];
  };
}
```

One-shot without editing the host config:

```
nix-build mixed-vlan.nix -L --option extra-sandbox-paths /dev/net
```

Representative failure without the path: the driver cannot set up the TAP bridge between backends. Fix the sandbox, not the `testScript`. Pure-container tests (Case 2) do not need `/dev/net`.

Case 5: Bind-mount a host device (GPU sketch) — still not a kernel test

Containers can pass host device nodes with nspawn options when those paths are visible in the sandbox. Sketch only — adjust device paths and sandbox mounts for your desk:

```
# cuda-sketch.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.testers.runNixOSTest {
  name = "cuda-sketch";

  containers.worker = { ... }: {
    virtualisation.systemd-nspawn.options = [
      "--bind=/dev/nvidia0"
      "--bind=/dev/nvidiactl"
    ];
    # environment.systemPackages = [ pkgs.cudaPackages.cudatoolkit ];
  };

  testScript = ''
    start_all()
    worker.wait_for_unit("multi-user.target")
    worker.succeed("test -e /dev/nvidia0")
  '';
}
```

Host still needs Case 1 settings, plus whatever `sandbox-paths` / `programs.nix-required-mounts` make `/dev/nvidia*` visible to the build. This is why containers help GPU/CUDA **service** tests: exclusive PCI passthrough into QEMU is a different, heavier path. It does **not** mean you can validate a custom kernel module inside nspawn — you are still on the host kernel.

The trap

Trap A — rename `nodes` → `containers` for a kernel, graphics, specialisation, SUID, or hardening test. The suite gets faster and quietly stops testing the claim. Keep those on QEMU.

Trap B — run container tests without host `uid-range` / auto-allocate-uids / cgroups. Failures look like driver or nspawn bugs. Configure the builder first (Case 1).

Trap C — mix containers and VMs on one VLAN without `/dev/net` in the sandbox. Bridging fails; pure-container tests did not need that path, so the "it worked yesterday" story is usually "we added a QEMU node."

Trap D — assume container isolation equals VM isolation. Same host kernel, shared device bind-mounts, sandbox limits on SUID and many `Protect*` settings. Use containers for speed and hardware sharing; use QEMU when the threat model or the assertion needs a guest kernel.

```
# Anti-pattern - "containers everywhere"
{
  containers.machine = { pkgs, ... }: {
    boot.kernelPackages = pkgs.linuxPackages_latest; # does not give you that kernel
    services.xserver.enable = true;                  # no screenshots / GUI driver path
  };
}
```

```
# Boring fix - split by claim
{
  containers.api = { services.nginx.enable = true; };  # fast service check
  nodes.gui = { services.xserver.enable = true; };     # still QEMU when needed
}
```

The boring rule

- On NixOS 26.05+, prefer `containers` for service and network integration tests; keep `testScript` unchanged.
- Configure the builder once: `auto-allocate-uids`, experimental `auto-allocate-uids` + `cgroups`, `extra-system-features = [ "uid-range" ]`.
- Add `extra-sandbox-paths = [ "/dev/net" ]` only when containers and VMs share a VLAN in one test.
- Keep `nodes` for kernel, graphics, specialisations, SUID-under-sandbox, and restrictive hardening cases.
- Use container bind-mounts for host devices (e.g. GPU) when that is the point; never confuse that with testing a guest kernel.
- Pin nixpkgs to `nixos-26.05` (or newer) before relying on the backend in CI.

Try this

1. On a disposable 26.05 builder, apply Case 1 and confirm `uid-range` appears in `nix show-config`.
2. `nix-build` Case 2's ping test. Note wall time. Re-run the same script with `nodes` instead of `containers` and compare (needs KVM).
3. Break Case 2 by commenting out `extra-system-features` on the builder; capture the failure mode; restore it.
4. Add a throwaway QEMU `nodes` machine sharing a VLAN with a container; run once without `/dev/net`, once with Case 4's sandbox path.
5. Grep the desk flake for `runNixOSTest` / `nodes =` and list which checks could move to `containers` vs which must stay on QEMU. Open tickets; do not mass-rename blindly.

Containers and Kubernetes

Creating Container Images with Nix (dockerTools)

Creating Container Images with Nix (dockerTools)

`FROM debian:latest` plus `apt-get` is a different closure every Tuesday. The boring default is: **`pkgs.dockerTools.buildLayeredImage` (or `streamLayeredImage`) from nixpkgs 26.05, no daemon at build time, `cacert` if you speak TLS**.

Mental model

Nix builds an OCI tarball from store paths. Layers split by derivation so glibc stays cached when `desk-api` changes. No `/bin/bash` unless you listed `pkgs.bash`.

Function	Output
`buildLayeredImage` `streamLayeredImage`	`.tar.gz` you `docker load` / `podman load` a **script** that writes the tarball to stdout (`./result | podman load`) — smaller NAR in CI

`contents` is **deprecated**. Use `copyToRoot`. `fakeNss` is `/etc/passwd` + `/etc/group` + nsswitch (root + nobody). Without it, `User = "65534"` fails with "no such user". `extraCommands` runs with the image root as cwd (`mkdir -p tmp && chmod 1777 tmp`). `pkgs.iana-etc` if you see `getProtocolByName: no such protocol name: tcp`.

Podman and Docker both load the archive. Building does **not** need root or a daemon.

```
desk-api drv  +  cacert  →  layered OCI  →  docker load / podman load
```

Worked examples

Case 1: Layered image

Save as `image.nix`:

```
# image.nix
{ pkgs ? import <nixpkgs> {} }:

let
```

```
  deskApp = pkgs.writeShellApplication {
    name = "desk-api";
    text = ''
      echo "desk-api listening (fake)"
    '';
  };
in
pkgs.dockerTools.buildLayeredImage {
  name = "desk-api";
  tag = "1.0.0";
  copyToRoot = [
    deskApp
    pkgs.cacert
    pkgs.dockerTools.fakeNss
  ];
  extraCommands = ''
    mkdir -p tmp
    chmod 1777 tmp
  '';
  config = {
    Cmd = [ "${deskApp}/bin/desk-api" ];
    ExposedPorts."8080/tcp" = { };
    Env = [ "SSL_CERT_FILE=${pkgs.cacert}/etc/ssl/certs/ca-bundle.crt" ];
    User = "65534:65534";
  };
}
```

```
nix-build image.nix
podman load < result
podman run --rm desk-api:1.0.0
```

Output (shape):

```
desk-api listening (fake)
```

Prefer a version tag over `latest`. `User = "65534:65534"` is nobody from `fakeNss`. Numeric IDs skip a name lookup; still ship `fakeNss` so glibc `getpwuid` does not panic. The process cannot write `/` — that is the point. Writable dirs (`/tmp`, maybe a volume for `/var/lib/desk-api`) come from `extraCommands`. `extraCommands` runs at **image build** time as root in that fake root, not as `User` at runtime.

`streamLayeredImage` takes the **same** attr set as `buildLayeredImage`. The derivation is a script: `nix build .#imageStream && ./result | podman load`. CI prefers the stream so the NAR is the script, not a multi-hundred-megabyte tarball sitting in the cache twice.

Case 2: `vendorHash` for a real Go module

```
# image-go.nix
{ pkgs ? import <nixpkgs> {} }:

let
  deskApp = pkgs.buildGoModule {
    pname = "desk-api";
    version = "1.0.0";
    src = pkgs.lib.cleanSource ./.;
    vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  };
in
pkgs.dockerTools.buildLayeredImage {
  name = "desk-api";
  tag = "1.0.0";
  copyToRoot = [ deskApp pkgs.cacert pkgs.tzdata pkgs.dockerTools.fakeNss ];
  config = {
    Cmd = [ "${deskApp}/bin/desk-api" ];
    User = "65534:65534";
  };
}
```

Probe `vendorHash` once. `null` is not a pin.

Case 3: Inspect layers

```
tar -tf result | head
nix path-info -Shr result | tail -1
```

The image closure should be the app + libc + certs, not gcc. If gcc is in `path-info -r`, you copied `buildInputs` into `copyToRoot`. `contents = [ pkgs.stdenv ];` is the same bug under the old name.

Case 4: Flake package

One file returns the **attr set**, not an image. Both builders take that set:

```
# image-args.nix
{ pkgs, deskApp }:

{
  name = "desk-api";
  tag = "1.0.0";
  copyToRoot = [ deskApp pkgs.cacert pkgs.dockerTools.fakeNss ];
```

```
  extraCommands = ''
    mkdir -p tmp
    chmod 1777 tmp
  '';
  config = {
    Cmd = [ "${deskApp}/bin/desk-api" ];
    User = "65534:65534";
    Env = [ "SSL_CERT_FILE=${pkgs.cacert}/etc/ssl/certs/ca-bundle.crt" ];
  };
}
```

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
      deskApp = pkgs.writeShellApplication {
        name = "desk-api";
        text = ''echo "desk-api listening (fake)"'';
      };
      imageArgs = import ./image-args.nix { inherit pkgs deskApp; };
    in
    {
      packages.${system}.image = pkgs.dockerTools.buildLayeredImage imageArgs;
      packages.${system}.imageStream = pkgs.dockerTools.streamLayeredImage imageArgs;
    };
}
```

```
nix build .#image
skopeo copy docker-archive:./result docker://registry.desk.internal/desk-api:1.0.0
```

```
nix build .#imageStream
./result | podman load
# CI: stream to a temp archive, then skopeo (stdin support varies)
./result > /tmp/desk-api.tar
skopeo copy docker-archive:/tmp/desk-api.tar docker://registry.desk.internal/desk-api:1.0.0
```

CI does not start dockerd. `skopeo` talks to the registry. Pin the tag; do not push `latest` from `main` unless that is an explicit contract.

Case 5: Nobody is home in the image

```
podman run --rm --entrypoint /bin/sh desk-api:1.0.0
```

Output:

```
Error: no such file or directory
```

That is success for a production image. Debug with a **separate** image that includes `pkgs.busybox`. Do not ship busybox in the API tag.

The trap

The trap is **forgetting `cacert`**. HTTPS fails with `x509: certificate signed by unknown authority`. Include `pkgs.cacert` and `SSL_CERT_FILE` (or `dockerTools.caCertificates`).

The other trap is `copyToRoot = [ pkgs.stdenv ];` (or deprecated `contents`) so "debug is easy." You just shipped a compiler.

A third: **root in the image** because `fakeNss` was omitted and `User` was left unset. A fourth: no `/tmp` (mode `1777`) so the Go runtime cannot create scratch files.

The boring rule

- `copyToRoot`, not deprecated `contents`. `buildLayeredImage` or `streamLayeredImage` from the **same** args.
- Pin tags. `cacert` + `fakeNss` + `/tmp`. `User` is not root.
- `vendorHash` is a FOD. Probe once.
- Build in CI without dockerd. `skopeo copy` after.
- Debug images are a different output, not extra packages in prod.

Try this

1. Build Case 1, `tar -tf result | wc -l`.
2. Drop `cacert`, run a one-liner that `curl https://example.com` (add curl), read the TLS error, put cacert back.
3. `nix path-info -Shr result` and confirm gcc is absent.
4. Tag `1.0.0` and `1.0.1` after a script change; confirm the base layers' hashes stayed when only the top layer moved.
5. `podman inspect desk-api:1.0.0 --format '{{.Config.User}}'` — `65534:65534`. Drop `fakeNss`, run, read the getpwuid error, put it back.

Kubernetes Manifests as Nix Modules

Kubernetes Manifests as Nix Modules

Hand-written YAML that drifts from the image tag in CI is how Saturday pages start. The boring default is: **generate manifests from Nix (same pin as `.#image`), commit the generated YAML if the cluster cannot eval Nix, and skip Helm until a chart is forced on you**.

Mental model

Nix can produce JSON/YAML for `kubectl apply`. Libraries (`nixidy`, `kubenix`, or a 40-line `writeText`) all do the same job: **one evaluation, one tag, one registry**.

The cluster still runs kubelet. Nix does not replace Kubernetes. It replaces the YAML copypaste.

```
flake  →  image tag + digest
       →  Deployment YAML with that digest
       →  kubectl apply --server-side
```

If the platform already speaks Helm only, next chapter. If you own the app, raw manifests are enough. Secrets still do not belong in ConfigMaps (store-is-public).

Worked examples

Case 1: A Deployment as JSON

Save as `k8s.nix`:

```
# k8s.nix
{ pkgs ? import <nixpkgs> { } }:

let
  image = "registry.desk.internal/desk-api@sha256:aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
  deploy = {
    apiVersion = "apps/v1";
    kind = "Deployment";
    metadata = { name = "desk-api"; namespace = "desk"; };
    spec = {
      replicas = 2;
      selector.matchLabels.app = "desk-api";
      template = {
```

```
        metadata.labels.app = "desk-api";
        spec.containers = [
          {
            name = "desk-api";
            inherit image;
            ports = [ { containerPort = 8080; } ];
            resources.requests = { cpu = "50m"; memory = "64Mi"; };
            resources.limits = { memory = "128Mi"; };
            securityContext = {
              runAsNonRoot = true;
              runAsUser = 65534;
              allowPrivilegeEscalation = false;
              readOnlyRootFilesystem = true;
            };
            livenessProbe.httpGet = { path = "/healthz"; port = 8080; };
            readinessProbe.httpGet = { path = "/healthz"; port = 8080; };
          }
        ];
      };
    };
  };
  svc = {
    apiVersion = "v1";
    kind = "Service";
    metadata = { name = "desk-api"; namespace = "desk"; };
    spec = {
      selector.app = "desk-api";
      ports = [ { port = 8080; targetPort = 8080; } ];
    };
  };
in
pkgs.writers.writeJSON "desk-api.json" [ deploy svc ]
```

```
nix-build k8s.nix
python3 -m json.tool result | head
```

`writers.writeJSON` is the 26.05 helper (pretty JSON, one derivation). Pin **digest**, not `:latest`. `image: registry/desk-api:1.0.0@sha256:…` (tag + digest) is accepted by kubelet and still human-readable. Every node may otherwise run a different blob.

`runAsUser = 65534` must match the image (`fakeNss` nobody). A probe on `/healthz` that the binary does not serve is a permanent `NotReady`.

Case 2: Same flake as the image

Save as `flake.nix`:

```
# flake.nix
{
  description = "desk-api image + manifests";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
    in
    {
      packages.x86_64-linux.manifests = pkgs.callPackage ./k8s.nix { };
    };
}
```

```
nix build .#manifests
```

Bump the digest in **one** place and rebuild image + manifests together.

Case 3: Apply

```
kubectl apply --server-side --dry-run=server -f result
kubectl apply --server-side -f result
kubectl -n desk rollout status deploy/desk-api
```

`--server-side` plays nicer with fields you do not own (HPA replicas, kubectl last-applied). Dry-run first on a shared cluster. `--force-conflicts` is for when you **mean** to take a field back from another manager — not a default.

Case 4: ConfigMap from Nix, not from kubectl create

```
# configmap.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.writeText "desk-cm.json" (builtins.toJSON {
  apiVersion = "v1";
  kind = "ConfigMap";
  metadata = { name = "desk-api"; namespace = "desk"; };
  data.WINDOW = "A";
})
```

`WINDOW` is not a password. Use External Secrets / sealed-secrets / CSI for those — or do not put the password in git, same rule as the Nix store.

Case 5: When the cluster cannot run Nix

```
nix build .#manifests
cp --no-preserve=mode "$(readlink -f result)" generated/desk-api.json
git add generated/desk-api.json
git diff --exit-code generated/
```

CI diffs `generated/` so humans cannot `kubectl edit` without updating the flake. GitOps (Argo/Flux) applies the committed JSON.

The trap

The trap is **templating the image as `latest`**. Every node may run a different digest. Pin tag **and** digest (`image: repo:tag@sha256:…`) when the registry supports it.

The other trap is a Secret manifest with `data: { password: hunter2 }` in the flake. That is Case 1 of the secrets chapter, in YAML.

A third: `runAsNonRoot` on an image that still runs as uid 0 and has no `fakeNss` — the pod is `CreateContainerConfigError`. Build the image and the manifest from the **same** flake.

The boring rule

- Manifests and image from one flake pin (nixpkgs 26.05). `writers.writeJSON`.
- Digest (and a tag), not `latest`. Probes and `runAsNonRoot` match the image.
- Apply `--server-side`. Dry-run first.
- No passwords in ConfigMaps or Secret YAML in git.
- Generate-and-commit if GitOps tools cannot eval Nix.
- Helm next chapter, only when the vendor ships a chart.

Try this

1. Case 1, `jq '.[0].spec.replicas' result`.
2. Change replicas to 3, rebuild, `diff` the JSON.
3. Put `image: …:latest` in a lab, list why you will not ship it.
4. Add the Service (already in Case 1) and `kubectl apply --dry-run=server`.

Helm Charts Generated from Nix

Helm Charts Generated from Nix

Helm is the cluster's package manager whether you like it or not. The boring default is: **pin the chart *and* the values in Nix (vendored `Chart.yaml` tarball + hash), render with `helm template`, and never `helm install` from floating `latest`**.

Mental model

A chart is a tarball plus values. Unpinned `helm repo add && helm upgrade` is `npm install` on the control plane.

Pin	How
Chart version	`fetchurl` of the `.tgz` + hash (FOD)
Values	Nix attrset → YAML
Image inside values	Same digest as `.#image`

If the app is yours, prefer raw manifests. Helm when the vendor only ships a chart (ingress-nginx, cert-manager). `helm repo update` in CI is how Monday's job is not Friday's chart.

Worked examples

Case 1: Vendor a chart tarball

```
helm pull ingress-nginx --version 4.11.0 --repo https://kubernetes.github.io/ingress-nginx
nix hash path ingress-nginx-4.11.0.tgz
```

Save as `chart.nix`:

```
# chart.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.fetchurl {
  url = "https://github.com/kubernetes/ingress-nginx/releases/download/helm-chart-4.11.0/ing
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

```
nix-build chart.nix
```

Probe the hash (`got:`). That tarball is now as pinned as `hello`. Commit the hash, not a live repo index.

Case 2: Render values from Nix

Save as `values.nix`:

```
# values.nix
{
  controller = {
    replicaCount = 2;
    image.tag = "v1.11.2";
    service.type = "ClusterIP";
  };
}
```

Helm wants YAML. Do not hope JSON-as-values is always accepted — write YAML from Nix:

```
# values-file.nix
{ pkgs, ... }:

pkgs.writers.writeYAML "values.yaml" (import ./values.nix)
```

```
nix-build -E 'with import <nixpkgs> {}; callPackage ./values-file.nix {}'
helm template desk-ing ./ingress-nginx-4.11.0.tgz \
  --namespace desk \
  --include-crds \
  --kube-version 1.31.0 \
  -f result \
  > generated/ingress.yaml
```

`--include-crds` is how cert-manager actually installs. Without it you apply Deployments against CRDs the cluster does not have. `--kube-version` pins the Capabilities the chart sees so CI and the cluster do not render different `apiVersions`. `--namespace` must match GitOps `targetNamespace`.

Commit `generated/ingress.yaml` if Argo cannot run Helm+Nix. The image tag must match the digest your flake built, not `latest`.

Case 3: Wrapper script in a flake

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk helm render";
```

```
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
      chart = pkgs.fetchurl {
        url = "https://github.com/kubernetes/ingress-nginx/releases/download/helm-chart-4.11
        hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
      };
    in
    {
      packages.x86_64-linux.render-ing = pkgs.writeShellApplication {
        name = "render-ing";
        runtimeInputs = [ pkgs.kubernetes-helm ];
        text = ''
          helm template desk-ing ${chart} \
            --namespace desk \
            --include-crds \
            --kube-version 1.31.0 \
            -f ${./values.yaml}
        '';
      };
    };
}
```

`values.yaml` can be `builtins.toJSON` written by a derivation so it is not hand-edited. The chart path is a store path — no `helm repo add`.

Case 4: Apply the bits you reviewed

```
nix run .#render-ing | kubectl apply --server-side -f -
```

or `helm upgrade --install desk-ing ./ingress-nginx-4.11.0.tgz -f values.yaml` **from the vendored tgz**, not from a live repo index. GitOps (Argo/Flux) applies the rendered YAML; it does not call `helm repo update`.

Case 5: Diff in CI

```
nix run .#render-ing > /tmp/new.yaml
git diff --no-index generated/ingress.yaml /tmp/new.yaml
```

If the diff is unexpected (chart author pushed a hook), you see it before the cluster does. Non-zero diff on an unreviewed chart bump is a failed PR, not an automatic apply.

Pin the **image** inside values to a digest, same rule as raw manifests:

```nix
# values.nix fragment
{
  controller.image = {
    registry = "registry.k8s.io";
    image = "ingress-nginx/controller";
    tag = "v1.11.2";
    digest = "sha256:aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa";
  };
}
```

The trap

The trap is **`helm repo update` in CI** with no version. Monday's job is not Friday's chart. Version + hash, like any FOD. The other trap is pinning the chart and leaving `image.tag: latest` in values.

A third: `helm template` **without** `--include-crds` for a CRD chart. A fourth: omitting `--kube-version` so a laptop on Helm 3.16 and CI on 3.17 emit different YAML for the same chart.

The boring rule

- Helm for vendor charts. Nix manifests for your apps.
- Chart tarball in the store (`fetchurl` + hash) or git-vendor.
- Values from Nix (`writers.writeYAML`). Images from the same flake (digest).
- `helm template --include-crds --kube-version … --namespace …`. Render in CI; apply the bits you reviewed.
- No `latest` chart, no `latest` image. No `helm repo update`.

Try this

1. `helm pull` a tiny chart, `nix hash path` the tgz, put it in `fetchurl`.
2. Change `replicaCount`, re-render, `diff`.
3. Break the hash by one character and read the FOD error.
4. Search CI for `helm repo update` and delete it.
5. Render twice, once with `--include-crds` and once without; `diff` — know what you would have skipped.

GitOps with Argo CD and Flux

GitOps with Argo CD and Flux

SSH-from-laptop `kubectl apply` does not scale and does not audit. The boring default is: **the cluster pulls generated YAML from git (Flux or Argo), and Nix only runs in CI to refresh that YAML**.

Mental model

Tool	Pull model
Flux	controllers reconcile `GitRepository` + `Kustomization`
Argo CD	Application CR points at a git path

Both want **plain YAML** in the repo (or a Helm release they render). They do not eval your flake on the control plane. CI:

```
nix build .#manifests  →  commit generated/  →  Flux applies
```

Pick **one**. Running Argo and Flux on the same cluster is a fight.

Worked examples

Case 1: Generated tree layout

```
generated/
  desk/
    namespace.yaml
    deploy.yaml
    svc.yaml
```

```
nix build .#manifests
rsync -a --delete result/ generated/desk/
git add generated && git diff --cached
```

Human reviews the YAML, not the Nix, at merge time — or reviews both.

Case 2: Flux GitRepository (illustrative)

Save as `generated/flux/gitrepo.yaml`:

```yaml
# gitrepo.yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: desk
  namespace: flux-system
spec:
  interval: 1m
  url: https://git.desk.internal/platform/desk.git
  ref:
    branch: main
```

The URL is your repo. Auth is a Secret Flux already knows, not a token in Nix.

Case 3: Kustomization pointing at generated YAML

```yaml
# kustomization-flux.yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: desk-api
  namespace: flux-system
spec:
  interval: 5m
  sourceRef:
    kind: GitRepository
    name: desk
  path: ./generated/desk
  prune: true
  wait: true
  timeout: 5m
  targetNamespace: desk
```

`prune: true` deletes objects you removed from git. That is the point. Test in **staging** first: delete `deploy.yaml` in a branch that Flux watches on the staging cluster, `flux reconcile kustomization desk-api --with-source`, confirm the Deployment is gone, then merge the same prune policy to prod.

`wait: true` blocks the Kustomization until health checks pass. Without it, Flux marks the apply done while pods are still `CrashLoop`. `targetNamespace` keeps generated namespaced objects out of `flux-system`.

```
flux diff kustomization desk-api --path generated/desk
```

That diff is what on-call reads. If it wants to delete a CRD or a Namespace you still need, `prune` is too wide — split those objects into a Kustomization with `prune: false`.

Case 4: CI gate

```
# .github/workflows/manifests.yml
name: manifests
on: [pull_request]
jobs:
  gen:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - run: nix build .#manifests
      - run: diff -ru generated/desk result
```

If someone edited YAML by hand, the PR fails. Edit Nix, regenerate, commit both. `diff` against `result/` (the store path), not against a dirty working tree. Empty diff on a no-op commit is the contract.

Optional: `kubeconform` / `kubectl --dry-run=server apply -f` against staging as a second job. Nix generating YAML does not prove the API server will accept it.

Case 5: Argo Application (if the org standardised on Argo)

```
# application.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: desk-api
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://git.desk.internal/platform/desk.git
    targetRevision: main
    path: generated/desk
  destination:
    server: https://kubernetes.default.svc
    namespace: desk
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

```
    syncOptions:
      - CreateNamespace=true
```

Same generated path. Do not point Argo at `helm/` with unpinned deps. `selfHeal: true` reverts `kubectl edit` on the cluster — that is the GitOps contract, not a bug. Pin `targetRevision` to a **tag** for production if you need a freeze; `main` is a moving apply.

Turn prune on in the staging Application first. Argo's prune of a shared `Namespace` or a CRD is the same footgun as Flux.

Secrets in that tree stay encrypted. If you use KSOPS / Flux `sops-secrets`, the **cipher** is in git; the controller decrypts in-cluster. Plain `Secret` YAML in `generated/` is the store-is-public bug again.

```
# do not
apiVersion: v1
kind: Secret
stringData:
  password: hunter2
```

The trap

The trap is **the laptop as GitOps**: `flux reconcile` after `kubectl apply` after `nix build` that never got pushed. Git is the desired state. If it is not in `main`, the cluster should not have it.

The other trap is **`prune: true` first applied in production**. A deleted YAML is a deleted Deployment. Staging cluster, then prod. A third: committing plaintext `Secret` YAML (Case 5). A fourth: skipping the CI `diff` so generated/ and Nix drift.

The boring rule

- One GitOps controller. Generated YAML in git. Nix in CI.
- `diff -ru generated/ result` on every PR. Empty on no-ops.
- `prune` + `wait` in **staging** before production. Split CRDs/namespaces if prune would eat them.
- Secrets still not in that YAML (sops + KSOPS / External Secrets).
- Pin git `targetRevision` to a tag for prod if you need freeze.

Try this

1. `diff -ru` two `nix build .#manifests` trees with no edits — empty.
2. Change a replica count in Nix, regenerate, read the YAML diff as if you were on-call.
3. Write a one-page "we use Flux xor Argo" note in the desk repo.
4. Confirm `kubectl` on your laptop is **read** against prod, not write (RBAC).

5. On a lab cluster, enable prune, delete one generated manifest, reconcile, confirm the object is gone. Then restore it from git.

Service Mesh and Observability

Service Mesh and Observability

A mesh is another control plane. The boring default is: **Prometheus + logs + a NetworkPolicy before Istio; add a mesh only when mTLS between 20+ services is a written requirement**.

Mental model

Need	Boring tool
CPU/RAM/HTTP metrics	`prometheus` + `node_exporter` / app `/metrics`
Logs	journald on NixOS; Loki if you already have it
mTLS inside the cluster	Mesh (Istio/Linkerd) **or** app TLS
Traffic split	Ingress / Service, then mesh

NixOS can run Prometheus on the **ops** host without Kubernetes. In-cluster, same rule: pin charts, generate YAML.

This chapter does not install Istio. It tells you what to measure first so you do not. A mesh will not invent RED metrics for an app that has no `/metrics`.

Worked examples

Case 1: App metrics without a mesh

Save as `metrics.nix` (NixOS `ops-01`):

```
# metrics.nix
{
  services.prometheus = {
    enable = true;
    listenAddress = "10.100.0.30";
    port = 9090;
    scrapeConfigs = [
      {
        job_name = "desk-api";
        static_configs = [ { targets = [ "10.100.0.11:8080" ]; } ];
      }
      {
```

```
        job_name = "node";
        static_configs = [
          { targets = [ "10.100.0.11:9100" "10.100.0.12:9100" ]; }
        ];
      }
    ];
  };
  services.prometheus.exporters.node.enable = true;
  networking.firewall.interfaces.wg0.allowedTCPPorts = [ 9090 9100 ];
}
```

The Go service exposes `/metrics` on `8080`. Scrape only on `wg0`. Public 9090 is how you donate metrics to the internet.

Case 2: Probe the endpoint

```
curl -sS http://10.100.0.11:8080/metrics | head
```

Output (shape):

```
# HELP go_goroutines Number of goroutines
# TYPE go_goroutines gauge
go_goroutines 7
```

No output → fix the app, do not add Envoy. Then:

```
curl -sS http://10.100.0.30:9090/api/v1/targets | jq '.data.activeTargets[].health'
```

`up` is the boring dashboard. Grafana is optional.

Save a recording rule only when you have a query you already run by hand:

```
# prometheus-rules.nix fragment
{
  services.prometheus.rules = [
    ''
      groups:
        - name: desk-api
          rules:
            - alert: DeskApiDown
              expr: up{job="desk-api"} == 0
              for: 2m
    ''
  ];
}
```

An alert with no runbook in `docs/threats.md` is noise. Do not add twenty.

Case 3: NetworkPolicy before sidecar

Save as `netpol.yaml`:

```
# netpol.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: desk-api
  namespace: desk
spec:
  podSelector:
    matchLabels:
      app: desk-api
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: desk-ing
      ports:
        - port: 8080
```

Generate this from Nix like any manifest. Most "we need a mesh" tickets are "we need NetworkPolicy." Apply, then `curl` from a pod that is **not** `desk-ing` and confirm it fails.

Case 4: If you must mesh

Pin the Linkerd or Istio **chart version + hash** (Helm chapter). Inject sidecars in **one** namespace first (`desk`). Measure CPU on the sidecar versus the app. Roll back with GitOps prune.

Do not enable the mesh globally on day one. Do not run Istio **and** Linkerd. Two meshes is two CNI races.

Case 5: Logs on NixOS already exist

```
journalctl -u desk-api -n 50 --no-pager
journalctl -u prometheus -n 20 --no-pager
```

Ship them with `services.promtail` or `systemd-journal-upload` if you have a Loki. A mesh access log is extra, not a substitute for application logs. `desk-api` returning 500 with a perfect Envoy dashboard is still a 500.

The trap

The trap is **installing Istio to "get observability."** You get a second failure domain, CNI races, and a dashboard. Start with `/metrics` and NetworkPolicy. The other trap is Prometheus on `0.0.0.0:9090` without a firewall — scrape configs are not a secret, but they describe your fleet.

The boring rule

- `/metrics` on the app. Prometheus scrape config in Nix. Listen on `wg0`.
- NetworkPolicy before sidecars.
- Mesh is optional and pinned. One namespace first. One mesh.
- journalctl is a valid log system.
- A mesh does not fix bad schema, slow SQL, or missing auth.

Try this

1. Hit `/metrics` on a desk service or add a one-line Prometheus handler in a lab binary.
2. Write a NetworkPolicy that allows only the ingress pods; prove a sidecar-less deny.
3. List three problems a mesh does **not** fix (bad schema, slow SQL, missing auth).
4. `nixos-option services.prometheus.scrapeConfigs` on 26.05 and add one job.

IaC and Cloud

Managing Terraform Providers with Nix

Managing Terraform Providers with Nix

`terraform init` downloading whatever the registry serves this morning is a supply chain. The boring default is: **`pkgs.opentofu` or `pkgs.terraform.withPlugins` from nixpkgs 26.05 so providers are store paths**.

OpenTofu is the next chapter; the wrapper pattern is the same. Prefer OpenTofu on a new desk. This chapter names Terraform because that is the API people google.

Mental model

`withPlugins` builds a Terraform/OpenTofu **wrapper** whose plugin directory is in `/nix/store`. `init` does not need the network for providers. State still lives where you put it (S3, local) — Nix does not replace the backend.

```
nix develop  →  tofu/terraform in PATH
                 plugins = /nix/store/…-terraform-providers
terraform init -plugin-dir=…  →  offline providers
```

`which terraform` must be a store path. Distro `terraform` on PATH is how CI still hits the registry.

Worked examples

Case 1: Shell with pinned providers

Save as `terraform-env.nix`:

```
# terraform-env.nix
{ pkgs ? import <nixpkgs> { } }:

let
  tf = pkgs.terraform.withPlugins (p: [
    p.aws
    p.random
    p.local
  ]);
in
pkgs.mkShell {
```

```
  packages = [ tf ];
}
```

```
nix-shell terraform-env.nix --run "terraform version"
which terraform
```

Output (shape):

```
Terraform v1.x.x
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws v…
+ provider registry.terraform.io/hashicorp/random v…
```

Versions are whatever 26.05 pinned. To freeze harder, overlay a specific provider derivation.

Case 2: Flake devShell

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk Terraform (prefer OpenTofu for new work)";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
      tf = pkgs.terraform.withPlugins (p: [ p.random p.local ]);
    in
    {
      devShells.x86_64-linux.default = pkgs.mkShell { packages = [ tf ]; };
    };
}
```

```
nix develop --command terraform version
```

Case 3: Offline init

```
nix develop --command terraform init -get=false
# PR job: no backend credentials required
nix develop --command terraform init -input=false -backend=false
nix develop --command terraform validate
```

If providers are already in the wrapper, init should not download. If it tries the registry, the wrapper is not on `PATH` (you ran distro terraform). Commit `.terraform.lock.hcl`. `-plugin-dir` is already baked into the wrapper; do not also set `TF_PLUGIN_CACHE_DIR` in `$HOME`.

```
readlink -f "$(which terraform)"
```

Must start with `/nix/store`.

Case 4: Tiny plan

Save as `main.tf`:

```
# main.tf
resource "random_id" "desk" {
  byte_length = 4
}

output "hex" {
  value = random_id.desk.hex
}
```

```
nix develop --command terraform init
nix develop --command terraform plan
nix develop --command terraform apply -auto-approve
```

No AWS credentials required. Proves plugins execute. Add `*.tfstate*` to `.gitignore`.

Case 5: CI without `setup-terraform`

Save as `.github/workflows/tf.yml`:

```
# .github/workflows/tf.yml
name: tf
on: [pull_request]
jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cachix/install-nix-action@v27
      - run: nix develop --command terraform fmt -check
      - run: nix develop --command terraform init -input=false -backend=false
      - run: nix develop --command terraform validate
      - run: nix develop --command terraform plan -input=false
        env:
          TF_IN_AUTOMATION: "1"
```

Same flake as laptops. No `hashicorp/setup-terraform`. Apply is a protected job on `main`, not this PR workflow.

The trap

The trap is **`terraform init` in CI with a clean `$HOME` and no `withPlugins`**. Registry outage or a yanked version becomes your outage.

The other trap is wrapping Terraform **and** OpenTofu on the same PATH. State gets written by the wrong binary. One CLI.

The boring rule

- Providers from nixpkgs (26.05 pin), not the registry at apply time. `withPlugins` + lockfile.
- `which terraform` is a store path.
- PR: `init -backend=false` + `validate`. Plan/apply need the backend.
- State backend is still your problem (lock table, encryption). Credentials are sops / env, not Nix strings.
- Prefer OpenTofu for new work (next chapter). One CLI on PATH.
- Overlay only when you must pin a provider the channel does not have.

Try this

1. Case 1, `which terraform`, confirm `/nix/store`.
2. `terraform init` with network blocked (`unshare -n` if you can) to confirm no download.
3. Add `p.null` to the plugin list, enter a new shell, `terraform version`.
4. `git grep setup-terraform` and delete it from CI.

OpenTofu Integration

OpenTofu Integration

Terraform's license change is why OpenTofu exists. The boring default for a **new** desk is: **`pkgs.opentofu` with `withPlugins` on nixpkgs 26.05, `.tf` files, same backend discipline**.

Mental model

OpenTofu is a Terraform-compatible CLI. Providers from nixpkgs attach the same way: `withPlugins` builds a wrapper whose plugin directory is in `/nix/store`. `tofu init` does not need the network for those providers. State still lives where you put it (S3, local) — Nix pins the **binary**, not the AWS account.

```
nix develop  →  tofu on PATH
                 plugins = /nix/store/…-opentofu-plugins
tofu init    →  offline providers
tofu plan    →  state from the backend
```

Commands: `tofu init|plan|apply|fmt`. Do not install both Terraform and OpenTofu on `PATH` unwrapped. State files from Terraform 1.x often import; test in a **clone** of state first.

Worked examples

Case 1: devShell with pinned plugins

Save as `flake.nix`:

```nix
# flake.nix
{
  description = "Desk OpenTofu";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
      tofu = pkgs.opentofu.withPlugins (p: [
        p.aws
        p.random
        p.local
```

```
      ]);
    in
    {
      devShells.x86_64-linux.default = pkgs.mkShell {
        packages = [ tofu ];
      };
    };
}
```

```
nix develop --command tofu version
```

Output (shape):

```
OpenTofu v1.x.x
on linux_amd64
```

The path is under `/nix/store`. `which tofu` must not be `~/.local/bin/tofu`.

`withPlugins` is the provider **binary**. You still commit `.terraform.lock.hcl` (hashes of those plugins). `tofu init -input=false` in this shell should **not** hit the registry; if it does, PATH is not the wrapper. `required_providers` in `.tf` documents versions; the store pin is what actually runs. Do not also set `TF_PLUGIN_CACHE_DIR` to a home directory that then races the wrapper.

Case 2: Same `main.tf` as Terraform

Save as `main.tf`:

```
# main.tf
resource "random_pet" "desk" {
  length = 2
}

output "name" {
  value = random_pet.desk.id
}
```

```
nix develop --command tofu init
nix develop --command tofu plan
nix develop --command tofu apply -auto-approve
```

No AWS required. Local state appears as `terraform.tfstate` — add `*.tfstate*` to `.gitignore`.

Case 3: Backend still not in Nix

Save as `backend.tf`:

```
# backend.tf
terraform {
  backend "s3" {
    bucket = "desk-tf-state"
    key    = "desk/opentofu.tfstate"
    region = "eu-central-1"
  }
}
```

Credentials for the backend come from the environment or sops (`AWS_PROFILE`, `AWS_ACCESS_KEY_ID`), not from a string in the flake. Nix does not substitute your account id.

```
nix develop --command tofu init -input=false
```

Case 4: CI splits plan and apply

```
nix develop --command tofu fmt -check
nix develop --command tofu init -input=false -backend=false   # fmt/validate, no state
nix develop --command tofu init -input=false                  # plan needs the backend
nix develop --command tofu validate
nix develop --command tofu plan -input=false -out=plan.bin
```

`-backend=false` is the PR job that must not need AWS credentials. Apply is a separate, protected job on `main` (environment protection, two reviewers) **with** the backend. Commit `plan.bin` nowhere. Artifact the plan between jobs if you must, with a short TTL. `TF_IN_AUTOMATION=1` turns prompts into errors — set it in CI.

Case 5: Do not mix CLIs

```
# in a desk-only shellHook, if humans still type terraform:
# alias terraform=tofu
# better: teach tofu
```

A shell that has both unwrapped is how state gets written by the wrong binary. `tofu state pull` after a surprise `terraform apply` is a long afternoon.

```
nix develop --command bash -c 'which tofu; command -v terraform || echo no-terraform'
```

`which tofu` is `/nix/store/…`. `terraform` should be missing, or a wrapper that *is* tofu. Two real binaries is the incident.

State migration: copy the backend key (`desk/opentofu.tfstate.migratetest`), `tofu init`, `tofu plan` against the copy, then cut. Not Friday in production.

The trap

The trap is **migrating state on Friday in production** without `tofu plan` against a copied state. The other trap is `terraform init` / `tofu init` downloading providers from the registry after you already pinned them in `withPlugins` — that means PATH is not the wrapper (`which tofu` is `~/.local` or a distro package).

A third: both `pkgs.terraform` and `pkgs.opentofu` on the same `PATH`. A fourth: committing `plan.bin` or `*.tfstate`.

The boring rule

- New work: OpenTofu from 26.05. `withPlugins`. Commit `.terraform.lock.hcl`.
- One CLI on PATH. Store path, not `~/.local`. No Terraform next to it.
- State credentials are secrets (sops / env), not Nix strings.
- PR: `fmt` + `init -backend=false` + `validate`. Apply on main with protection.
- Test state migration on a copy of the backend key.

Try this

1. `nix develop --command tofu version` and confirm a store path (`which tofu`).
2. `tofu plan` on `random_pet` without AWS.
3. `tofu fmt -check` on a badly indented file, fix, re-run.
4. Write down where state lives for the desk (bucket + key) in the repo README — not the credentials.
5. `nix develop --command tofu init -input=false` with `NIX_SSL_CERT_FILE` set and network blocked (`unshare -n` in a lab) — it must still find providers.

Ansible Provisioning with Nix

Ansible Provisioning with Nix

Ansible mutates hosts. NixOS **replaces** that for machines you own. The boring default is: **do not Ansible a NixOS box; if you must drive a fleet of Ubuntu leftovers, pin `pkgs.ansible` in a 26.05 devShell and keep inventory in git**.

Mental model

Host OS	Tool
NixOS	`nixos-rebuild` / Colmena / nixos-anywhere
Legacy Ubuntu/Debian you cannot reinstall this quarter	Ansible from a Nix shell

Using both on the same NixOS host (`ansible ad-hoc apt`) fights the next `switch`. The playbook that `copy`s a `configuration.nix` and then `shell: nixos-rebuild` is two sources of truth.

Collections are FODs (or nixpkgs packages), not `ansible-galaxy install` on Monday morning on the runner. Inventory lives in git. Ad-hoc `ansible all` is how a leftover gets a surprise reboot.

Plan the reinstall to NixOS. Ansible is a bridge with a retirement date.

Worked examples

Case 1: Pinned ansible CLI

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk Ansible (legacy only)";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
    in
    {
      devShells.x86_64-linux.default = pkgs.mkShell {
```

```
        packages = [ pkgs.ansible ];
      };
    };
}
```

```
nix develop --command ansible --version
which ansible
```

`which` must be a store path. `~/.local/bin/ansible` is the unpinned CLI you just tried to retire.

Case 2: Inventory in git

Save as `inventory.ini`:

```
# inventory.ini
[legacy]
ubuntu-leftover.desk.internal

[nixos]
app-01.desk.internal
```

```
nix develop --command ansible legacy -i inventory.ini -m ping
```

Do not put NixOS hosts in the group you run playbooks against. The `[nixos]` group exists so Case 5 can refuse them.

Case 3: Collections pinned, not Galaxy at runtime

If nixpkgs has the collection, use it. If not, vendor the tarball as a fixed-output derivation and point `ANSIBLE_COLLECTIONS_PATH` at it.

Save as `collections.nix`:

```
# collections.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.stdenv.mkDerivation {
  pname = "desk-ansible-collections";
  version = "1.0.0";
  src = pkgs.fetchurl {
    url = "https://example.invalid/community-general-9.0.0.tar.gz";
    hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  };
  installPhase = ''
    mkdir -p $out/ansible_collections
```

```
    tar -C $out/ansible_collections -xzf $src
  '';
}
```

```
# in the devShell
{
  packages = [ pkgs.ansible collections ];
  ANSIBLE_COLLECTIONS_PATH = "${collections}";
}
```

Do not `ansible-galaxy install` on the runner every job. That is `terraform init` downloading providers again.

Case 4: Playbook that only talks to leftover Ubuntu

Save as `ping.yml`:

```
# ping.yml
- hosts: legacy
  gather_facts: false
  tasks:
    - ping:
```

```
nix develop --command ansible-playbook -i inventory.ini ping.yml
```

If this playbook grows `apt: name=nginx`, you are postponing the NixOS reinstall. Put a date in `README.md`.

Case 5: Refuse NixOS in the inventory

Save as `assert.yml`:

```
# assert.yml
- hosts: all
  gather_facts: true
  tasks:
    - name: refuse NixOS
      ansible.builtin.fail:
        msg: "Use nixos-rebuild / Colmena, not Ansible"
      when: ansible_facts['os_family'] | default('') == 'NixOS'
```

```
nix develop --command ansible-playbook -i inventory.ini assert.yml
```

A NixOS host in `all` must fail the play. That failure is the boring default working.

The trap

The trap is **Ansible `copy` of a `configuration.nix` then `shell: nixos-rebuild`**. You now have two sources of truth, and the one that wins is whoever ran last. SSH to the host and rebuild from the flake, or Colmena from `ops-01`.

The boring rule

- NixOS hosts: no Ansible. Fail the play if `os_family` is NixOS.
- Legacy hosts: Ansible from a Nix-pinned CLI + vendored collections.
- Inventory in git. No ad-hoc `ansible all`.
- Collections are FODs, not galaxy-on-Monday.
- Plan the reinstall to NixOS; Ansible is a bridge with a date.

Try this

1. `which ansible` inside `nix develop` — store path.
2. Add a fake inventory host, `ansible-inventory --list`.
3. `git grep ansible` on NixOS modules — should be empty.
4. Write the retirement date for the last Ubuntu leftover in the desk README.

Cloud Provider Configuration

Cloud Provider Configuration

Cloud consoles are mutable. The boring default is: **NixOS images from nixos-generators (or official AMIs), cloud-init only for SSH keys at first boot, then the flake owns the instance**.

Mental model

Piece	Owner
AMI / qcow / gce image	`nixos-generators` on 26.05, or official NixOS AMIs
Instance size, disk, VPC	OpenTofu `withPlugins`
OS config	`nixosConfigurations` in the same repo
First-boot SSH	cloud-init / metadata **once**

Do not bake AWS keys into the image. Do not grow `user_data` into a second `configuration.nix`. After first boot, Colmena / `nixos-rebuild --flake` takes over.

The NixOS firewall and the cloud security group must tell the same story. `0.0.0.0/0` on 22 "for debug" is the story of a scanner.

Worked examples

Case 1: Official AMI lookup (OpenTofu)

Save as `ami.tf`:

```
# ami.tf
data "aws_ami" "nixos" {
  owners      = ["427812963091"]
  most_recent = true
  filter {
    name   = "name"
    values = ["nixos/26.05*"]
  }
  filter {
    name   = "architecture"
    values = ["x86_64"]
```

```
  }
}
```

Owner ID is the NixOS AMI account as documented on nixos.org/download. Confirm it when you copy. `most_recent` plus a name glob is a moving target — pin a **specific AMI id** once you have tested it:

```
# ami-pin.tf
variable "nixos_ami" {
  type    = string
  default = "ami-xxxxxxxxxxxxxxxxx"
}
```

Case 2: Image you built

```
nix build .#amazon
ls -l result
# upload / AMI import per current AWS docs for the VHD
```

The generator produces the artifact. This chapter only consumes the AMI id in tofu. Do not hand-edit an AMI from 24.11 after the flake moved to 26.05.

Case 3: user-data is SSH keys, not packages

Save as `userdata.tf`:

```
# userdata.tf
resource "aws_instance" "app_01" {
  ami           = var.nixos_ami
  instance_type = "t3.medium"
  user_data     = <<-EOT
    #cloud-config
    users:
      - name: deskadmin
        sudo: ALL=(ALL) NOPASSWD:ALL
        ssh_authorized_keys:
          - ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI… deskadmin@ops-01
  EOT
  tags = {
    Name    = "desk-app-01"
    Flake   = "git.desk.internal/platform/desk"
    HostKey = "app-01"
  }
}
```

After first boot:

```
ssh deskadmin@203.0.113.11
sudo nixos-rebuild switch --flake git+ssh://git.desk.internal/platform/desk#app-01
```

Do not keep adding packages to `user_data`. They vanish on the next rebuild anyway if the flake does not declare them.

Case 4: Security group matches the firewall

Save as `sg.tf`:

```
# sg.tf
resource "aws_security_group" "gw" {
  name = "desk-gw"
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 51820
    to_port     = 51820
    protocol    = "udp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["203.0.113.0/24"] # office VPN, not the world
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

On the host, `networking.firewall.allowedTCPPorts = [ 443 ];` and `allowedUDPPorts = [ 51820 ];`. If the SG is wider than NixOS, the SG is the bug. If NixOS is wider than the SG, the module is the bug.

Case 5: Tags are how you find the box

When DNS is wrong:

```
aws ec2 describe-instances --filters Name=tag:HostKey,Values=app-01 \
  --query 'Reservations[].Instances[].InstanceId'
```

Tags are not secrets. Do not put the restic password in a tag.

The trap

The trap is **an AMI from 24.11 still in tofu** after the flake moved to 26.05. The instance boots old software, then the first rebuild is a surprise kernel. Pin AMI name `nixos/26.05*` **or** a specific AMI id you tested — and bump it on purpose.

The boring rule

- Images: 26.05 generator or official 26.05 AMI, then pin the id.
- tofu owns cloud objects. Flake owns the OS.
- user-data: SSH keys only. Then Colmena.
- SG + NixOS firewall agree. No `0.0.0.0/0` on 22.
- No cloud keys in the image.

Try this

1. `aws ec2 describe-images --owners 427812963091 --filters Name=name,Values=nixos/26.05*` (if you have AWS) and record an AMI id.
2. Compare that id to what tofu would pick with `most_recent`.
3. List ports in the SG vs `networking.firewall.allowedTCPPorts` for one host — they should match.
4. Boot from user-data keys, then rebuild from the flake; confirm a package you did **not** put in user-data is present only because the flake says so.

Multi-Cloud Deployment Patterns

Multi-Cloud Deployment Patterns

Two clouds is two IAM models, two images, two outages. The boring default is: **one cloud until you have a written reason; if you must span, one flake with two `nixosConfigurations` and two tofu roots — not a unified abstraction layer**.

Mental model

NixOS **closures** are cloud-agnostic. **Images and metadata** are not. An AWS AMI is not a GCP image. `nixos-generators` emits per-format artifacts from the **same** modules.

```
modules/desk-api.nix
      nixosConfigurations.desk-aws   → amazon image
      nixosConfigurations.desk-gcp   → gce image
tofu/aws/   state in s3://desk-tf-state/aws
tofu/gcp/   state in gcs://desk-tf-state/gcp
```

Do not share state files across clouds. Do not `for_each {aws, gcp}` in one root. A lock in the wrong cloud is an outage in the right one.

WireGuard between clouds is a mesh, not a reason to invent a "cloud-agnostic" Nix wrapper.

Worked examples

Case 1: Shared module, two hosts

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk two-cloud (only if you must)";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      mk = name: extra: nixpkgs.lib.nixosSystem {
        system = "x86_64-linux";
        modules = [
```

```
            ./modules/common.nix
            extra
            { networking.hostName = name; }
          ];
        };
    in
    {
      nixosConfigurations = {
        desk-aws = mk "desk-aws" ./hosts/aws.nix;
        desk-gcp = mk "desk-gcp" ./hosts/gcp.nix;
      };
    };
}
```

Save as `modules/common.nix`:

```
# modules/common.nix
{
  system.stateVersion = "26.05";
  services.openssh.enable = true;
  services.openssh.settings.PasswordAuthentication = false;
  networking.firewall.enable = true;
  # desk-api service lives here
}
```

`aws.nix` / `gcp.nix` own disks, metadata, and the provider's idea of a NIC. `common.nix` owns users, sshd, desk-api.

Case 2: Same birth, two clouds

`stateVersion` is birth, not channel. If both hosts are new on 26.05:

```
# already in common.nix
{ system.stateVersion = "26.05"; }
```

If `desk-aws` was born on 24.11, leave **that** host at `"24.11"` and only set `"26.05"` on the new GCP box. Copy-pasting `stateVersion` across clouds is how you skip a migration.

Case 3: Images from the generator, not a conversion script

```
nix build .#amazon
nix build .#gce
```

Typical artifacts (names follow your generator pin):

```
result/nixos-amazon-image-26.05.vhd
result/nixos-image-26.05-x86_64-linux.raw.tar.gz
```

Do not convert an AMI to GCE with a random script if the generator already has `gce` / `googleComputeImage`. Two formats, one module set.

Case 4: Separate tofu directories

```
tofu/aws/main.tf
tofu/aws/backend.tf    # bucket desk-tf-state, key desk/aws
tofu/gcp/main.tf
tofu/gcp/backend.tf    # bucket desk-tf-state, key desk/gcp
```

Save as `tofu/aws/backend.tf`:

```
# tofu/aws/backend.tf
terraform {
  backend "s3" {
    bucket = "desk-tf-state"
    key    = "desk/aws/terraform.tfstate"
    region = "eu-central-1"
  }
}
```

Different backends. Different credentials. Apply from `tofu/aws` never loads GCP state.

Case 5: WireGuard between clouds, not a service mesh

If the only reason for two clouds is "HA," a WireGuard mesh (`10.100.0.0/24`, `/32` peers) plus Colmena is enough. `desk-aws` is `10.100.0.11`, `desk-gcp` is `10.100.0.12`. UDP 51820 on each public NIC. Postgres still only on `wg0`.

Multi-cloud Kubernetes is not a first project. It is a second control plane, a second IAM, and a second way to lose DNS.

```
ls tofu/aws tofu/gcp
test ! -f tofu/terraform.tfstate && echo 'no shared state at tofu/ (good)'
```

If a single `terraform.tfstate` sits at `tofu/`, you already merged the roots. Split them before the next apply.

The trap

The trap is **a "cloud-agnostic" Nix wrapper** that hides AMI vs GCE differences until `tofu apply`. Keep the differences in `aws.nix` / `gcp.nix` where you can read them. The other trap is one terraform state that `for_each`s both clouds.

The boring rule

- One cloud first. Write the sentence that justifies the second.
- Same NixOS modules; different image formats and tofu roots.
- `stateVersion` per host birth — not "whatever the other cloud has."
- No shared terraform state.
- Mesh if you need routing; not a service mesh across clouds on day one.

Try this

1. List what actually differs between two clouds for the desk (image, IP, disk, IAM).
2. Draw two tofu dirs on paper. Do not merge them.
3. Build `.#amazon` and `.#gce` from one module set.
4. Write the one-sentence reason you need the second cloud — or delete the work.

Platform Engineering

The Internal Developer Platform Pattern

The Internal Developer Platform Pattern

Fifty repos each pinning their own nixpkgs is fifty llvm builds. The boring default is: **one platform flake on nixpkgs 26.05 that exports `devShells`, `templates`, and `nixosModules`, and product flakes `follows` it**.

Mental model

```
git.desk.internal/platform     nixpkgs 26.05
       devShells.backend / frontend
       templates.go-service
       nixosModules.baseline
git.desk.internal/desk-api
    inputs.platform.url = …
    inputs.nixpkgs.follows = "platform/nixpkgs"
```

Upgrading Go or sshd defaults is a platform bump, then a lock bump in each consumer (or a bot). Not a meeting. Tag `platform-26.05.3`; consumers pin the tag, not `main`. A broken main at 09:01 must not break every laptop.

Keep the platform **thin**: shells, templates, twenty lines of sshd. An overlay that replaces half of nixpkgs is a fork you cannot debug.

Worked examples

Case 1: Platform flake

Save as `flake.nix` in the platform repo:

```
# flake.nix
{
  description = "Desk platform";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" ];
      forAll = f: nixpkgs.lib.genAttrs systems (s: f nixpkgs.legacyPackages.${s});
```

```
    in
    {
      devShells = forAll (pkgs: {
        backend = pkgs.mkShell {
          packages = [ pkgs.go pkgs.gopls pkgs.postgresql ];
        };
        frontend = pkgs.mkShell {
          packages = [ pkgs.nodejs pkgs.pnpm ];
        };
      });

      templates.go-service = {
        path = ./templates/go-service;
        description = "Desk Go service";
      };

      nixosModules.baseline = ./modules/baseline.nix;
    };
}
```

Case 2: Init from template

Save as `templates/go-service/flake.nix` so `init` already `follows`:

```
# templates/go-service/flake.nix
{
  description = "Desk Go service";

  inputs.platform.url = "git+ssh://git.desk.internal/platform.git?ref=refs/tags/platform-26.
  inputs.nixpkgs.follows = "platform/nixpkgs";

  outputs = { self, nixpkgs, platform }:
    let
      pkgs = nixpkgs.legacyPackages.x86_64-linux;
    in
    {
      devShells.x86_64-linux.default = platform.devShells.x86_64-linux.backend;
      packages.x86_64-linux.default = pkgs.hello;
    };
}
```

```
nix flake init -t git+ssh://git.desk.internal/platform.git#go-service
```

Case 3: Consumer lock bump

Save as the product `flake.nix` (desk-api):

```
# flake.nix
{
  description = "desk-api";

  inputs.platform.url = "git+ssh://git.desk.internal/platform.git?ref=refs/tags/platform-26.
  inputs.nixpkgs.follows = "platform/nixpkgs";

  outputs = { self, nixpkgs, platform }: {
    devShells.x86_64-linux.default =
      platform.devShells.x86_64-linux.backend;
    nixosModules.default = { imports = [ platform.nixosModules.baseline ]; };
  };
}
```

```
nix flake update platform
nix flake metadata
```

`nix flake metadata` must show **one** nixpkgs. Two nixpkgs means a `follows` is missing.

Do not `nix flake update` of everything on Friday. Update `platform`, then only what broke.

Case 4: Baseline NixOS module

Save as `modules/baseline.nix`:

```
# modules/baseline.nix
{ lib, ... }:
{
  services.openssh.enable = lib.mkDefault true;
  services.openssh.settings.PasswordAuthentication = false;
  networking.firewall.enable = true;
  system.stateVersion = lib.mkDefault "26.05";
}
```

Hosts import `platform.nixosModules.baseline`. Hosts may `mkForce` in an emergency; review that in the PR. `mkDefault` is how a host opts into a different `stateVersion` if it was born earlier — birth still wins.

Case 5: Version the platform

```
git tag -a platform-26.05.3 -m "26.05.3 shells + sshd"
git push origin platform-26.05.3
```

Consumers pin the tag in `inputs.platform.url`. Floating `main` is for the platform team's laptops, not for production `desk-api`.

The trap

The trap is **a platform that overlays half of nixpkgs**. Consumers cannot debug `go` anymore: is it nixpkgs, the overlay, or the product? Keep the platform thin. One overlay for *one* pin, documented in `README.md`.

The boring rule

- One nixpkgs: 26.05 via the platform flake.
- Templates + `follows`. `nix flake metadata` shows a single nixpkgs.
- `nix flake update platform` is the upgrade API.
- Tags, not floating main, for production consumers.
- Thin modules. No mega-overlay.

Try this

1. `nix flake init -t` from a local path template.
2. `nix flake metadata` on a consumer and confirm a single nixpkgs.
3. Add a `devShells.docs` with `pkgs.pandoc` on the platform; consume it from desk-api.
4. Tag a fake `platform-26.05.3` and pin it in a consumer lock.

Monorepo Strategies with Nix

Monorepo Strategies with Nix

A monorepo with `src = ./.;` on every package rebuilds the world per commit. The boring default is: **one flake, many `packages.*`, `lib.fileset` (or `cleanSourceWith`) per leaf, and CI that builds `.#checks` — not `.#everything` — on a docs-only PR**.

Mental model

Layout	Nix
`apps/desk-api`	`packages.desk-api` with `src` filtered to that tree
`apps/desk-web`	sibling package; a docs change does not rebuild api
`modules/`	`nixosModules`
`flake.nix`	one lock, nixpkgs 26.05

`nix flake check` should be cheap enough to run on every PR. Heavy images are extra jobs or nightly. Recursive flakes (`inputs.foo.url = "path:./apps/foo"`) multiply locks — one lock at the root.

Nix caching still helps if you rebuild the world, but eval time and `vendorHash` churn do not. Filter first.

Worked examples

Case 1: fileset `src` per leaf

Save as `pkgs/desk-api.nix`:

```
# pkgs/desk-api.nix
{ pkgs, lib }:

pkgs.buildGoModule {
  pname = "desk-api";
  version = "1.0.0";
  src = lib.fileset.toSource {
    root = ../.;
    fileset = lib.fileset.unions [
```

```
      ../apps/desk-api
      ../go.mod
      ../go.sum
    ];
  };
  vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

A README typo under `apps/desk-web` is not in this fileset. `nix build .#desk-api` should be a cache hit.

Case 2: One flake, many packages

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk monorepo";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in
    {
      packages.${system} = {
        desk-api = pkgs.callPackage ./pkgs/desk-api.nix { };
        desk-web = pkgs.callPackage ./pkgs/desk-web.nix { };
        default = self.packages.${system}.desk-api;
      };
      checks.${system} = {
        api = self.packages.${system}.desk-api;
        web = self.packages.${system}.desk-web;
      };
    };
}
```

```
nix build .#desk-api
nix flake check -L
```

Case 3: PR path filter (CI)

Save as `.github/workflows/leaf.yml`:

```yaml
# .github/workflows/leaf.yml
name: leaf
on:
  pull_request:
    paths:
      - "apps/desk-api/**"
      - "pkgs/desk-api.nix"
      - "go.mod"
      - "go.sum"
jobs:
  api:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: nix build .#desk-api
```

A second workflow watches `flake.lock` and runs `nix flake check`. Nix will still cache; the path filter saves eval time on huge repos.

Case 4: `linkFarm` for the nightly smoke set

```nix
# in flake outputs
{
  packages.x86_64-linux.desk-smoke = pkgs.linkFarm "desk-smoke" [
    { name = "api"; path = self.packages.x86_64-linux.desk-api; }
    { name = "web"; path = self.packages.x86_64-linux.desk-web; }
  ];
}
```

```
nix build .#desk-smoke
```

Nightly builds the farm. PRs build the leaf. Do not make `default` the farm if humans run `nix build` on every save.

Case 5: Do not use recursive flakes

```nix
# do not
{
  inputs.desk-api.url = "path:./apps/desk-api";
  inputs.desk-web.url = "path:./apps/desk-web";
}
```

Each path flake wants its own `flake.lock`. You will update nixpkgs in one leaf and not the other. `callPackage` plus filesets is the boring layout.

The trap

The trap is `src = ./.;` at repo root for every `callPackage`. A README typo rebuilds Go, invalidates `vendorHash` work, and CI is red for twenty minutes. Filter. The other trap is a recursive flake per app.

The boring rule

- One flake, one 26.05 lock.
- Per-package `lib.fileset` (or `cleanSourceWith`) `src`.
- Leaf builds on leaf PRs; `flake.lock` PRs run full check.
- No recursive flakes.
- `vendorHash` / `cargoHash` per package, not one hash for the repo.

Try this

1. `nix build .#desk-api`, then touch a web-only file; `nix build .#desk-api --dry-run` (should be empty if filtered).
2. Pick `lib.fileset` or `cleanSource` — one style per repo.
3. Add a `checks` attr that is `linkFarm` of two packages; run it only nightly.
4. Time `nix flake show` on the monorepo; if it is minutes, you eval too much at the top.

Multi-Platform Development and Distribution

Multi-Platform Development and Distribution

`x86_64-linux` CI does not prove `aarch64-darwin`. The boring default is: **`lib.genAttrs` over the systems humans use, native remote builders for foreign `system`, and no QEMU-on-the-laptop as the production path**.

Mental model

What	How
Same shell on Mac + Linux	`devShells.${system}` from nixpkgs 26.05
Linux binary from a Mac	Remote Linux builder (`ssh-ng`)
Raspberry Pi image	`aarch64-linux` builder or `nixos-generators` `sd-aarch64`

`pkgsCross` is **cross**. A remote builder is **native**. Native is faster and hits the team cache. Cross is for when you do not have the hardware (Go `GOOS`/`GOARCH` is a third path — still not QEMU).

```
each system  →  packages.${system}.desk-api
             →  devShells.${system}.default
foreign linux  →  --max-jobs 0  +  ssh-ng builder
```

Two archs are two cache objects. `nix copy` each one.

Worked examples

Case 1: Enumerate systems without flake-utils

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk multi-system";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
```

```
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" ];
      forAll = f: nixpkgs.lib.genAttrs systems (system: f {
        inherit system;
        pkgs = nixpkgs.legacyPackages.${system};
      });
  in
  {
    packages = forAll ({ pkgs, ... }: {
      default = pkgs.hello;
      desk-api = pkgs.hello; # replace with callPackage
    });
    devShells = forAll ({ pkgs, ... }: {
      default = pkgs.mkShell { packages = [ pkgs.go pkgs.gopls ]; };
    });
  };
}
```

`flake-utils` is optional sugar. One extra input is one extra `follows` to forget. `lib.genAttrs` is enough.

```
nix flake show
```

You should see three systems. Skip `x86_64-darwin` for new work: nixpkgs 26.11 drops it. 26.05 still builds it until EOL.

Case 2: Native remote builder, not qemu

On the Darwin laptop, do not emulate `aarch64-linux` as the release path:

```
nix build .#packages.aarch64-linux.desk-api --max-jobs 0
```

`--max-jobs 0` forbids local realisation. Nix must use `ssh-ng` to an aarch64 (or x86_64-linux) builder. If you have no builder:

```
error: a 'aarch64-linux' with features {} is required to build '…', but I am a 'aarch64-darwin'
```

That error is correct. Add a builder; do not reach for `qemu-binfmt` for the artefact you ship.

Save as `builders.nix` on the Darwin (or laptop) NixOS/nix-darwin host:

```
# builders.nix
{
  nix.buildMachines = [{
    hostName = "builder.desk.internal";
    system = "x86_64-linux";
    protocol = "ssh-ng";
    maxJobs = 8;
```

```
    speedFactor = 2;
    supportedFeatures = [ "kvm" "nixos-test" "big-parallel" ];
  }];
  nix.distributedBuilds = true;
  nix.settings.max-jobs = 0;
}
```

`supportedFeatures` must include `kvm` if you offload `runNixOSTest`. A missing flag is why tests compile on the laptop anyway.

Case 3: CI matrix for the OS humans use

```
# .github/workflows/build.yml
name: build
on: [pull_request]
jobs:
  build:
    strategy:
      matrix:
        os: [ubuntu-latest, macos-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - run: nix build .#packages.${{ matrix.os == 'macos-latest' && 'aarch64-darwin' || 'x8
```

Each job `nix build`. Cache per system. Do not emulate `aarch64-linux` on GHA ubuntu unless you accept 10× time and flaky KVM.

Case 4: Release two closures

```
nix build .#packages.x86_64-linux.desk-api -o result-x86
nix build .#packages.aarch64-linux.desk-api -o result-arm
nix path-info -Shr ./result-x86
nix path-info -Shr ./result-arm
nix copy --to ssh://cache.desk.internal ./result-x86 ./result-arm
```

Two closures. Two cache keys. Goreleaser is optional; `nix build` + `nix copy` is enough for internal desk tools.

Case 5: `file` the toolchain, not your hopes

On Apple silicon:

```
nix develop --command bash -c 'file "$(which go)"'
```

You want `arm64`, not `x86_64`. A hardcoded `system = "x86_64-linux"` in a flake used on a Mac may still eval; `go` is then the wrong ELF.

The trap

The trap is **`system = "x86_64-linux"` hardcoded** in a flake used on Apple silicon, or `builtins.currentSystem` inside a flake (impure). Enumerate systems. Use `builtins.currentSystem` only inside impure `nix-shell`, not in flakes.

The boring rule

- Enumerate systems with `lib.genAttrs`. 26.05 pin.
- Native remote builders for foreign Linux. Not laptop QEMU for releases.
- CI matrix for the OS humans actually use.
- Do not plan on `x86_64-darwin` past 26.05.
- Two archs = two cache objects = two `nix copy`s.

Try this

1. `nix flake show` and list systems.
2. On a Mac, `file $(nix develop --command which go)` — arm64 on Apple silicon.
3. `nix build .#packages.aarch64-linux.default --max-jobs 0` without a builder and read the error.
4. Note the 26.11 darwin x86_64 drop in the version appendix; do not add that system to new flakes.

Build Optimization and Performance Tuning

Build Optimization and Performance Tuning

Waiting on eval is not a personality trait. The boring default is: **substituters first, `auto-optimise-store`, filtered `src`, honest `max-jobs`, and a profiler only when `nix develop` takes minutes to *start*.**

Mental model

Slow	Fix
Download	Team cache, `builders-use-substitutes`
Compile	Remote builder; do not rebuild gcc locally
Eval	No IFD; `fileset`; one nixpkgs
Disk	GC window + `--optimise`

Laptops: default `max-jobs` is fine. An 8-core CI runner should say `max-jobs = 8`. A 64G builder with `max-jobs = 1` is a mis-copied gist.

`cores = 0` lets each job use all CPUs and oversubscribe. Prefer a number on a shared builder. Cache hits make `cores` irrelevant — check substituters before you tune jobs.

Worked examples

Case 1: NixOS 26.05 knobs

Save as `nix-build.nix`:

```
# nix-build.nix
{
  nix.settings = {
    max-jobs = 8;
    cores = 4;
    substituters = [
      "https://cache.nixos.org"
      "https://desk-cache.cachix.org"
    ];
    trusted-public-keys = [
      "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
      "desk-cache.cachix.org-1:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA="
```

```
    ];
    auto-optimise-store = true;
    builders-use-substitutes = true;
  };
}
```

```
nix show-config | grep -E 'max-jobs|cores|substituters|auto-optimise'
```

You should see the team cache, not only `cache.nixos.org`. Missing `trusted-public-keys` is why substitutes are silently ignored.

Case 2: No IFD on the hot path

Bad — eval must **build** before it can parse:

```
# do not
{
  deskLib = import "${pkgs.fetchFromGitHub {
    owner = "desk";
    repo = "lib";
    rev = "abc123";
    hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
  }}/default.nix";
}
```

Fix — flake input or a vendored file:

```
# flake.nix
{
  inputs.desk-lib.url = "github:desk/lib/abc123";
  outputs = { self, nixpkgs, desk-lib }: {
    # import desk-lib as an input, not a fetch inside eval
  };
}
```

IFD is serial and hostile to flakes. `callPackage ./vendored.nix` is also fine.

Confirm you are not IFDing by accident:

```
nix flake show 2>&1 | grep -i 'building.*fetch' || echo 'no fetch during eval (good)'
```

A `building '/nix/store/…-source.drv'` line during `flake show` is IFD. Move that fetch to a flake input.

Case 3: Measure eval

```
time nix flake show
time nix eval .#packages.x86_64-linux.desk-api --raw
```

If `flake show` is minutes, you import too much at the top. Split packages; avoid reading the whole monorepo in `outputs`. Filter `src` with `lib.fileset` so a README does not re-eval Go.

Case 4: `--dry-run` after a docs edit

```
nix build .#desk-api --dry-run
echo x >> README.md
nix build .#desk-api --dry-run
```

Second list must not include gcc. If it does, `src` is unfiltered (`src = ./.;` at repo root).

Case 5: Heavy compiles belong on the builder

Chromium, Firefox, LLVM: `nix.buildMachines` + cache. Confirm with:

```
nix build nixpkgs#hello --max-jobs 0 --dry-run
```

`--max-jobs 0` means "do not build locally." If hello is not substituted, the cache is wrong — fix that before tuning `cores`. A laptop compiling LLVM is a missing substituter, not a missing `-j`.

```
nix build nixpkgs#hello --print-build-logs 2>&1 | grep -E 'copying path|building' | head
```

`copying path` from `cache.nixos.org` (or the team cache) is the win. `building` hello on a workstation means substituters are empty or unsigned.

```
free -h
nproc
```

Set `max-jobs` from RAM and cores you actually have during a **real** build, not from a blog.

The trap

The trap is **turning `max-jobs` down to 1 on a big box** because a blog mentioned RAM. Watch `free -h` during a real build; then set jobs. The other trap is eval-profiler cargo-cult without reading `--dry-run`. Filter `src` first.

The boring rule

- Cache, then jobs, then eval hygiene.
- No IFD on the hot path.
- `--dry-run` after trivia. Filtered `src`.
- Heavy derivations on the builder. `--max-jobs 0` on the laptop.
- `auto-optimise-store` on 26.05.

Try this

1. `nix show-config | grep max-jobs` and the substituter list.
2. Case 4 on a filtered package.
3. `time nix flake show` vs `time nix build nixpkgs#hello` when hello is cached.
4. Enable `auto-optimise-store`; `systemctl status nix-optimise.timer` if you also set the timer.

Debugging and Troubleshooting Production Issues

Debugging and Troubleshooting Production Issues

"Works on my laptop" is a missing `nix log`. The boring default is: **`nix log` → `journalctl` → `why-depends` → `--show-trace` → rollback — before editing a live unit**.

Mental model

Symptom	First tool
Build failed	`nix log /nix/store/…drv`
Eval failed	`--show-trace`, `nix repl .#`
Unit dead after switch	`systemctl status`, `journalctl -u`
Closure huge	`nix path-info -Shr`, `why-depends`
Will not boot	Previous generation in systemd-boot

`vim /etc/systemd/system/foo.service` is undone by the next switch (or by impermanence). Put the flag in Nix. Live drop-ins from `systemctl edit` are unmanaged state.

The log of a failed build lives in the store. CI can `nix log` the same drv hash the laptop saw. That is the whole point of a content-addressed failed build: the bytes of the log are the same everywhere.

Worked examples

Case 1: Build log

Save as `fail.nix`:

```
# fail.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.runCommand "desk-fail" { } ''
  echo boom >&2
  false
''
```

```
nix-build fail.nix
nix log $(nix-instantiate fail.nix)
```

```
nix build .#desk-api --print-build-logs
```

You should see `boom` in the log even after the build is gone from the terminal scrollback. On CI:

```
nix log /nix/store/…-desk-fail.drv
```

Case 2: Eval trace

```
nix eval .#nixosConfigurations.app-01.config.networking.hostName --show-trace
```

Read the **last** frame first. Infinite recursion is usually `rec` + `//` or `final.pkg.overrideAttrs` calling `prev` the wrong way.

```
error: infinite recursion encountered
       at /nix/store/…/overlays.nix:4:5
```

`nix repl .#` then `:lf .` and tab-complete the attr you meant. Do not add `builtins.trace` to production modules and leave it there.

Case 3: Unit after switch

```
systemctl status desk-api --no-pager
journalctl -u desk-api -n 80 --no-pager
systemctl cat desk-api
```

`systemctl cat` shows a store path:

```
# /nix/store/…-unit-desk-api.service/desk-api.service
[Service]
ExecStart=/nix/store/…-desk-api/bin/desk-api
```

A drop-in from `systemctl edit` appears as `/etc/systemd/system/desk-api.service.d/override.conf`. Delete it and encode the change in the flake. On an ephemeral root it is already gone after reboot — which is why "it worked until reboot" is this bug.

Case 4: Unexpected python in the OS closure

```
nix path-info -Shr /run/current-system | tail
nix why-depends /run/current-system nixpkgs#python3
```

Output (shape):

```
/nix/store/…-nixos-system-app-01-26.05
  /nix/store/…-desk-wrapper
      /nix/store/…-python3-3.12.x
```

Cut the edge (a wrapper, `propagatedBuildInputs`, a shebang). Re-switch. Measure `path-info -Shr` before and after. gcc in the **runtime** closure of `desk-api` is the same class of bug.

Case 5: Rollback then revert git

```
sudo nixos-rebuild list-generations | tail
sudo nixos-rebuild switch --rollback
git log -1 --oneline
git revert HEAD
colmena apply --dry-run --on app-01
```

Rollback without a git revert means the next Colmena apply re-breaks the box. The generation on metal and the flake in git must agree.

If the box will not boot, pick the previous generation in systemd-boot. Then SSH in and revert.

Diff two system closures when "what changed?" is the ticket:

```
nix-diff /run/current-system /nix/var/nix/profiles/system-43-link
```

Or, without `nix-diff`:

```
nix store diff-closures /run/booted-system /run/current-system
```

You should see unit files and store paths, not a wall of llvm.

If `switch` failed halfway:

```
systemctl --failed --no-pager
journalctl -b -p err --no-pager | tail
```

A failed unit after a successful eval is not a Nix bug — it is the unit. Fix the module, `switch` again. Do not `systemctl reset-failed` and walk away.

The trap

The trap is **`systemctl edit` + reboot "to see."** Unmanaged state. Impermanence eats it; a persistent root lies to the next apply. The other trap is screenshotting a red CI log instead of `nix log` on the drv.

The boring rule

- Store logs + journald. Not screenshots.
- `--show-trace` for eval. `why-depends` for size.
- Rollback first; bisect the flake second.
- No live drop-ins. `systemctl cat` must be a store path only.
- 26.05 `nix log` / `path-info` are enough to start.

Try this

1. Case 1: `runCommand` with `false`; `nix log` the failed drv.
2. `systemctl cat sshd` — find `/nix/store`.
3. `--rollback` after a hostname change on a lab VM, then revert git.
4. `why-depends hello` vs gcc — gcc must not be in the **runtime** closure.

Testing Strategies for Nix Configurations

Testing Strategies for Nix Configurations

`nixos-rebuild switch` on prod is not a test. The boring default is: **`nix flake check` on every PR, `pkgs.testers.runNixOSTest` for services that listen, and a VM that is not the laptop**.

Mental model

Level	Command	What it proves
Eval	`nix eval`, module types	The expression is a value
Build	`nix build .#desk-api`	The derivation realises
NixOS VM	`runNixOSTest`	A unit actually listens
Full gate	`nix flake check`	Every `checks.*` attr

A module that is `mkIf false` evals forever. A VM test that `curl`s the port catches the empty `wantedBy`.

VM tests need `kvm` (and usually `nixos-test`) on the builder. Darwin laptops offload them to a Linux remote builder. `checkPhase` inside a derivation cannot hit the network — mock or vendor fixtures.

Keep PR tests under a few minutes. Nightly runs the farm.

Worked examples

Case 1: Cheap flake checks

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk checks";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
```

```
    in
    {
      packages.${system}.desk-api = pkgs.hello;

      checks.${system}.fmt = pkgs.runCommand "desk-fmt" { } ''
        echo ok > $out
      '';

      checks.${system}.api = self.packages.${system}.desk-api;
    };
}
```

```
nix flake check -L
```

`-L` prints logs. Failures must be visible without SSH to the runner. Replace `hello` with the real `desk-api` package.

Case 2: NixOS VM test

Save as `test-desk.nix`:

```
# test-desk.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.testers.runNixOSTest {
  name = "desk-nginx";
  nodes.server = {
    services.nginx.enable = true;
    networking.firewall.allowedTCPPorts = [ 80 ];
    system.stateVersion = "26.05";
  };
  nodes.client = {
    system.stateVersion = "26.05";
  };
  testScript = ''
    start_all()
    server.wait_for_unit("nginx.service")
    client.succeed("curl -f http://server/")
  '';
}
```

```
nix-build test-desk.nix
```

Needs KVM. Wire the same attr into `checks.${system}.nginx` so `nix flake check` runs it in CI.

A failing assertion:

```
error: client failed: curl -f http://server/
```

Non-zero exit. That is the gate.

Case 3: CI job

Save as `.github/workflows/check.yml` (shape):

```yaml
# .github/workflows/check.yml
name: check
on: [pull_request]
jobs:
  flake:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: nix flake check -L
```

The runner must have Nix and, for Case 2, KVM (self-hosted Linux builder with `supportedFeatures = [ "kvm" "nixos-test" ]`). GitHub-hosted ubuntu without nested virt skips the VM check or offloads it.

Case 4: Hermetic `checkPhase`

```
# desk-api.nix fragment
{
  doCheck = true;
  checkPhase = ''
    # no curl to api.desk.internal
    go test ./...
  '';
}
```

If the test needs a fixture, vendor it in `src`. A check that needs prod credentials is not a check — it is a smoke probe you run from `ops-01` after deploy.

Case 5: What not to run on every PR

A 20-node mesh test on every docs typo is how people skip CI. Split:

When	What
Every PR	`fmt`, `.#desk-api`, one small `runNixOSTest`
Nightly	`linkFarm` of images, multi-node mesh
After Colmena	live `curl` from `ops-01`

The trap

The trap is **eval-only CI**. `nix flake check` that only type-checks modules never sees `wantedBy = [ ]`. Pair eval with at least one VM test per listening service, or you will discover the empty unit on prod.

The boring rule

- `flake check -L` on every PR. VM tests when a unit listens.
- KVM on the Linux builder. 26.05 `runNixOSTest`.
- No network in `checkPhase`.
- Small PR tests; big nightly.
- A test that needs prod credentials is a probe, not a check.

Try this

1. `nix flake check -L` on the desk flake.
2. Run Case 2 (needs KVM). Confirm `curl` succeeds.
3. Add `client.succeed("false")` and confirm non-zero exit.
4. Add a `checks` attr that is just `self.packages.${system}.desk-api` so a broken Go build fails the PR without a VM.

Maintenance, Upgrades, and Migration

Maintenance, Upgrades, and Migration

`nix flake update` on Friday without a plan is a weekend. The boring default is: **pin `nixos-26.05`, bump in a dedicated PR, `flake check` + one lab host, then Colmena — never bump `stateVersion` in that same PR**.

Mental model

Move	What changes
`nix flake update nixpkgs`	Same train, newer commit
`nixos-25.11` → `nixos-26.05`	Release upgrade; read the notes
`stateVersion`	Almost never

25.11 is EOL. New hosts: `"26.05"`. Old hosts: change the **input**, keep their birth `stateVersion`.

The 26.05 **channel** ships Nix **2.34**; this book's CLI baseline is **2.35+** via the installer or `nix.package`. Do not confuse those in the upgrade PR: bumping the channel does not magically bump the daemon.

Home Manager's `release-26.05` follows the same train. HM 24.11 + nixpkgs 26.05 is eval pain.

Worked examples

Case 1: Lock bump inside 26.05

Save as the existing `flake.nix` input (already on 26.05):

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
}
```

```
nix flake update nixpkgs
nix flake metadata
nix flake check -L
sudo nixos-rebuild dry-activate --flake .#desk-lab
```

Read which units restart. Switch the lab. Then Colmena `--on app-01`, then the rest. One commit, one PR, changelog the rev.

Case 2: Release upgrade

```
# flake.nix - was nixos-25.11
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  inputs.home-manager.url = "github:nix-community/home-manager/release-26.05";
  inputs.home-manager.inputs.nixpkgs.follows = "nixpkgs";
}
```

`git diff flake.lock` is huge. Run VM tests. Stage-1 systemd is **default** on 26.05 — if you still set `boot.initrd.systemd.enable` by hand, read that chapter before the fleet apply.

```
nix flake check -L
nixos-rebuild dry-activate --flake .#desk-lab
```

Case 3: `stateVersion` stays

Save as `hosts/app-01.nix` for a box born on 25.11:

```
# hosts/app-01.nix
{
  system.stateVersion = "25.11"; # born on 25.11; input is 26.05
}
```

Still correct. Do not “tidy” it to `"26.05"` in the lock-bump PR. Database dirs and module defaults then migrate on a surprise.

New metal:

```
# hosts/app-03.nix
{
  system.stateVersion = "26.05";
}
```

Birth, not channel.

Case 4: Home Manager follows the same train

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  inputs.home-manager.url = "github:nix-community/home-manager/release-26.05";
  inputs.home-manager.inputs.nixpkgs.follows = "nixpkgs";
}
```

```
nix flake metadata
```

One nixpkgs. `home.stateVersion` is also birth (`"25.11"` on an old user, `"26.05"` on a new one). Same rule as the system.

Case 5: Record the lock

Save as a changelog line, not a wiki:

```
# CHANGELOG.md
- 2026-09-10: nixpkgs 26.05.<rev> on desk-lab, then app-01 (Colmena).
  stateVersion unchanged. Nix CLI still 2.35+ via nix.package.
```

If you cannot name the lock in prod, you cannot roll back with confidence.

```
git show HEAD:flake.lock | jq -r '.nodes.nixpkgs.locked.rev'
```

Paste **that** rev, not "we updated nixpkgs." A Colmena rollback without this line in git is a generation the next apply will overwrite.

Do not bump Home Manager in the same PR as nixpkgs unless you must. Two diffs, two rollbacks:

```
nix flake update nixpkgs
nix flake check -L
# merge, then:
nix flake update home-manager
nix flake check -L
```

HM `release-26.05` already `follows` nixpkgs; updating HM alone is usually a module-fix, not a stdenv rebuild.

```
nix flake metadata --json | jq -r '.locks.nodes.nixpkgs.locked.rev'
```

Paste that rev into the changelog.

The trap

The trap is **changing `stateVersion` to match the channel**. Channel bump stateVersion bump. The other trap is `nix flake update` with no arguments on Friday, which also bumps every other input you forgot you had.

The boring rule

- 26.05 train. Lock PRs, not surprise `update`.
- Lab, then canary, then fleet.

- `stateVersion` / `home.stateVersion` is birth.
- HM release matches nixpkgs release. `follows`.
- Changelog the rev. Channel Nix 2.34 CLI 2.35+.

Try this

1. `nix flake metadata` — write the locked date and rev.
2. `dry-activate` after a lock bump; list units that would restart.
3. `git grep stateVersion` — none "updated" to 26.05 on old hosts without notes.
4. Skim 26.05 release notes for one module you use (Caddy, postgres, sshd).

Enterprise Operational Checklists

Enterprise Operational Checklists

A wiki of "remember to" is how generations rot. The boring default is: **scripts in the repo that run `flake check`, `restic`, and `colmena apply --dry-run` — not a PDF**.

Mental model

If it is not a command, it will not happen on Friday. Map each ritual to something you already have: GC, restic, sops rekey, Colmena, the 26.05 lock.

Commit `scripts/desk-daily.sh` and a systemd timer on `ops-01` if humans forget. The timer is a NixOS module, not a crontab someone added by hand.

```
ops-01 timer  →  scripts/desk-daily.sh
weekly timer  →  GC + restic snapshots
human         →  dry-run + canary before fleet
```

Worked examples

Case 1: Daily

Save as `scripts/desk-daily.sh`:

```
# scripts/desk-daily.sh
#!/usr/bin/env bash
set -euo pipefail
nix flake check -L
systemctl --failed --no-pager
df -h /nix /boot /persist
```

```
chmod +x scripts/desk-daily.sh
./scripts/desk-daily.sh
```

Non-zero `flake check` means do not deploy. Wire the same script as a CI job so ops and GitHub disagree less.

Save as `modules/desk-daily.nix` on `ops-01`:

```
# modules/desk-daily.nix
{
  systemd.services.desk-daily = {
    serviceConfig.Type = "oneshot";
    serviceConfig.ExecStart = "/persist/desk/scripts/desk-daily.sh";
  };
  systemd.timers.desk-daily = {
    wantedBy = [ "timers.target" ];
    timerConfig.OnCalendar = "09:00";
  };
}
```

Case 2: Weekly disk

Save as `scripts/desk-weekly.sh`:

```
# scripts/desk-weekly.sh
#!/usr/bin/env bash
set -euo pipefail
sudo nix-collect-garbage --delete-older-than 14d
sudo nix-store --optimise
systemctl status restic-backups-desk --no-pager || true
sudo restic snapshots | tail
```

Save as `modules/boot-limit.nix`:

```
# modules/boot-limit.nix
{
  boot.loader.systemd-boot.configurationLimit = 8;
  nix.gc.automatic = true;
  nix.gc.dates = "weekly";
  nix.gc.options = "--delete-older-than 14d";
}
```

Fourteen days of generations is enough to roll back a bad Friday. The ESP fills if you skip `configurationLimit`. The weekly script must `git -C /persist/desk pull --ff-only` or it checks last month's flake.

Case 3: Before fleet apply

```
git log -1 --oneline
nix flake metadata
nix flake check -L
colmena apply --dry-run --on app-01
colmena apply --on app-01
```

```
# watch journalctl -u desk-api on app-01
colmena apply --on @app
```

Never `@all` for a kernel bump. `db-01` is serial, after a restic snapshot.

Case 4: Advisory day

```
nix flake update nixpkgs
nix flake check -L
sudo nixos-rebuild dry-activate --flake .#desk-lab
# lab switch, then Case 3
```

Do not wait for the quarterly bump if nixpkgs already has the fix. Do not `nix flake update` with no arguments (that bumps every input). Changelog the rev.

Case 5: New host (checklist that is also git)

Save as `docs/new-host.md` in the same repo:

```
# docs/new-host.md
- Disko by-id + LUKS + @persist
- sops recipient + rekey
- stateVersion = "26.05"  (new metal only)
- WireGuard IP in the mesh module
- restic paths
- nixos-anywhere once
- Colmena targetHost + tag
```

Tick by merging a PR that adds the host — not by editing a spreadsheet. The PR is the checklist completing.

```
git grep -n 'stateVersion' -- '*.nix'
```

Every **new** host in that PR must be `"26.05"`. Old hosts keep their birth value. The checklist is wrong if the PR "tidies" `stateVersion` to match the channel.

The trap

The trap is **a 90-line Confluence page** and no `scripts/`. Five commands in git beat a beautiful table nobody runs. The other trap is a timer that runs `flake check` against a dirty `/persist` checkout nobody pulls.

The boring rule

- Commands in git. 26.05. sops. restic. Colmena.
- Dry-run, canary, then fleet. Never `@all` for kernels.
- 14-day GC. ESP `configurationLimit`.
- New hosts born on `"26.05"`.
- Advisories: lock bump the same day, changelog the rev.

Try this

1. Add Case 1; run it in CI as well as on ops.
2. Time `nix flake check`; if >10 minutes, split `checks`.
3. `restic snapshots` — newest < 48h.
4. Walk Case 5 on a throwaway VM and file the PR that adds it.

Fleet Deployments with Colmena

Fleet Deployments with Colmena

`for h in hosts; do nixos-rebuild --flake .#$h --target-host …; done` forgets a flag and leaves one box on last week's generation. The boring default is: **Colmena with one hive in the flake, SSH from `ops-01`, `--on` for canaries, and the same nixpkgs 26.05 lock the laptop uses**.

Mental model

Colmena evals a hive (host → modules + `deployment.targetHost`), realises each closure (locally or on a builder), copies it over SSH, and activates. Rollback is the previous NixOS generation on that host — not a restore CD.

```
ops-01  colmena apply --on app-01
                build + copy

        app-01  switch-to-configuration
```

Tags (`@app`, `@db`) are how you canary a role without typing every hostname. Parallel apply is for **stateless** boxes. Serial the node that owns the database.

`deployment.buildOnTarget = true` compiles on the host. The boring default is the opposite: build on `ops-01` (or the remote builder), copy the result. Laptops do not compile kernels.

Worked examples

Case 1: Hive in the flake

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk fleet hive";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    colmena = {
      meta = {
```

```
        nixpkgs = import nixpkgs { system = "x86_64-linux"; };
      };

      defaults = { pkgs, ... }: {
        system.stateVersion = "26.05";
        environment.systemPackages = [ pkgs.vim ];
        services.openssh.enable = true;
        services.openssh.settings.PasswordAuthentication = false;
        networking.firewall.enable = true;
      };

      app-01 = { name, ... }: {
        deployment.targetHost = "10.100.0.11";
        deployment.targetUser = "root";
        deployment.tags = [ "app" ];
        networking.hostName = name;
        imports = [ ./hosts/app-01.nix ];
      };

      app-02 = { name, ... }: {
        deployment.targetHost = "10.100.0.12";
        deployment.targetUser = "root";
        deployment.tags = [ "app" ];
        networking.hostName = name;
        imports = [ ./hosts/app-02.nix ];
      };

      db-01 = { name, ... }: {
        deployment.targetHost = "10.100.0.21";
        deployment.targetUser = "root";
        deployment.tags = [ "db" ];
        networking.hostName = name;
        imports = [ ./hosts/db-01.nix ];
      };
    };
  };
}
```

```
nix shell nixpkgs#colmena --command colmena eval -E '{ nodes }: builtins.attrNames nodes'
```

You should see `app-01`, `app-02`, `db-01`. Exact attr names follow the Colmena pin on 26.05 (`colmena --help`). Newer Colmena also accepts `colmenaHive` + `colmena.lib.makeHive` — if `colmena eval` says the flake attr moved, use that shape. The idea is still host → SSH target.

```
# hive extras
{
```

```
  deployment.allowLocalDeployment = true; # ops-01 applying itself
  deployment.keys."wg-priv" = {
    keyFile = "/run/secrets/wireguard/private";
    destDir = "/run/keys";
  };
}
```

Prefer sops-nix on the **target** over Colmena `deployment.keys` when you already have sops. Two secret planes is two rekeys.

Case 2: Canary, then the rest of the tag

```
colmena apply --on app-01
# watch desk-api metrics / journalctl -u desk-api
colmena apply --on @app
```

Do not `--on @all` for a kernel bump. `db-01` is a separate `--on db-01` after the apps are healthy.

Case 3: Dry-run before activate

```
colmena apply --dry-run --on app-01
```

Eval + build, no `switch`. If the dry-run fails, git is wrong — not the host.

```
[app-01] Built /nix/store/7q1a…-nixos-system-app-01-26.05
[app-01] (dry run; not activating)
```

Case 4: Build where the CPU is

Keep `deployment.buildOnTarget = false` (the default). Point the hive machine at the same remote builder as `nix.buildMachines` on `ops-01`. Confirm:

```
nix show-config | grep -E 'builders|max-jobs'
```

`--max-jobs 0` on the laptop forces remote realisation. A canary that compiles LLVM on `app-01` is a production incident.

Case 5: Activation failure is a generation, not a brick

```
[app-01] Built /nix/store/7q1a…-nixos-system-app-01-26.05
[app-01] Activation failed
```

The previous generation is still the boot default unless you passed a reboot-into-new-generation flag (`reboot` / `apply-local --reboot` on your pin). On the host:

```
ssh root@10.100.0.11 'systemctl is-system-running; nixos-rebuild list-generations | tail'
```

Roll back on the box (`nixos-rebuild switch --rollback` or Colmena's rollback flag on your pin), then revert the git commit so the hive and the metal agree.

The trap

The trap is **applying every host in parallel while a shared Postgres migration runs**. Stateless `desk-api` boxes can fan out. The stateful node is serial, after the migration is in the module and the backup snapshot exists.

The boring rule

- One hive, one 26.05 lock, SSH from `ops-01`.
- Canary `--on app-01`, then `--on @app`. Never `@all` for kernels.
- Dry-run first. Build off the target.
- Stateful hosts serial. Stateless hosts tagged.
- Rollback is the previous generation, then a git revert so the hive matches.
- One secret system (sops on the host). Colmena `deployment.keys` only if you do not have sops yet.

Try this

1. `nix shell nixpkgs#colmena --command colmena --help`.
2. Apply to a single lab VM with `--dry-run`, then without.
3. Break a unit on purpose, apply, roll back on the VM, revert git.
4. Write `docs/fleet.md`: which tags may run in parallel, which host is serial.

Lightweight MicroVMs with microvm.nix

Lightweight MicroVMs with microvm.nix

Full libvirt guests take minutes to iterate. The boring default for **local** multi-node tests is: **microvm.nix (or `runNixOSTest`) on nixpkgs 26.05; production still gets real machines or the capstone VMs**.

Mental model

`microvm.nix` is a NixOS module that boots a guest in hundreds of milliseconds, often sharing the host store via virtiofs. It is a **lab accelerator**, not a tenant isolator. Do not put untrusted workloads there without reading the project's trust model.

```
host flake
  nixosConfigurations.desk-micro
    microvm.hypervisor = "qemu"   # or cloud-hypervisor
    microvm.mem / vcpu
    volumes you listed - nothing else persists
```

`runNixOSTest` is the CI twin: headless, assertions, no SSH session. Keep both only when they answer different questions (human poke vs PR gate).

`/dev/kvm` must exist on the host. Nested virt on a cloud VM is often off — that is a missing feature, not a Nix bug.

Worked examples

Case 1: Guest in the flake

Save as `flake.nix`:

```nix
# flake.nix
{
  description = "Desk microVM lab";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    microvm.url = "github:astro/microvm.nix";
    microvm.inputs.nixpkgs.follows = "nixpkgs";
  };
```

```
  outputs = { self, nixpkgs, microvm }: {
    nixosConfigurations.desk-micro = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        microvm.nixosModules.microvm
        {
          microvm.hypervisor = "qemu";
          microvm.mem = 1024;
          microvm.vcpu = 2;
          networking.hostName = "desk-micro";
          services.openssh.enable = true;
          services.openssh.settings.PasswordAuthentication = false;
          users.users.root.openssh.authorizedKeys.keys = [
            "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI… deskadmin@ops-01"
          ];
          system.stateVersion = "26.05";
        }
      ];
    };
  };
}
```

`follows` keeps the guest on the same 26.05 as the host flake.

Case 2: Run the declared runner

```
nix run .#nixosConfigurations.desk-micro.config.microvm.declaredRunner
```

(The exact run attr is in microvm.nix docs for your pin.) You should get a console. Log in, then:

```
hostname
systemctl is-system-running
systemctl status sshd --no-pager
halt
```

If QEMU exits immediately, check `ls -l /dev/kvm` and that your user is in `kvm`.

```
ls -l /dev/kvm
id -nG | tr ' ' '\n' | grep -x kvm || echo 'not in kvm group - log out after extraGroups'
```

Nested virt on a cloud workstation: the hypervisor UI must enable it. `ls /dev/kvm` missing is not a Nix eval error.

Case 3: Persist nothing unless listed

A default microVM is as ephemeral as a tmpfs root. If a test needs `/persist`, declare a volume in the module:

```
# volumes.nix
{
  microvm.volumes = [
    {
      image = "persist.img";
      mountPoint = "/persist";
      size = 1024;
    }
  ];
}
```

Otherwise the next `nix run` starts clean — which is what you want for a lab.

Case 4: Host virtualisation (workstation)

Save as `hosts/ops-lab.nix` (fragment of the workstation):

```
# hosts/ops-lab.nix
{
  virtualisation.libvirtd.enable = true;
  users.users.deskadmin.extraGroups = [ "libvirtd" "kvm" ];
}
```

```
ls -l /dev/kvm
groups
```

`kvm` in the group list, `crw-rw----` on `/dev/kvm`. Nested virt on a cloud workstation: enable it in the hypervisor UI or pick a metal builder.

Case 5: microVM vs `runNixOSTest`

Need	Tool
SSH in, poke Caddy, read journals	microVM on the laptop
PR must fail if nginx does not listen	`pkgs.testers.runNixOSTest` in `checks`
Production desk-api	Colmena to real hosts

Do not maintain both for the same scenario without a reason. CI does not SSH into your laptop guest.

The trap

The trap is **running production `desk-api` in a microVM on someone's laptop**. Prod is Colmena to real hosts (or the capstone VMs). microVMs die when the lid closes.

The boring rule

- 26.05 + `microvm.inputs.nixpkgs.follows`.
- QEMU/KVM for labs. `runNixOSTest` for CI.
- No production tenants on a laptop hypervisor.
- `stateVersion = "26.05"` on new guests.
- Share the store only when you understand the trust model. Volumes you did not list do not exist.

Try this

1. Enable KVM, run Case 1, `hostname` inside the guest.
2. `systemctl status sshd` inside it, then `halt`.
3. Compare wall-clock boot to a libvirt NixOS guest on the same box.
4. Add a `checks.x86_64-linux runNixOSTest` that covers the same `sshd` enablement so CI does not need the laptop.

Content-Addressed Nix Internals

Content-Addressed Nix Internals

Everyday Nix is **input-addressed**: the `.drv` hash names the output. **Content-addressed** (CA) derivations hash the **bytes**. The boring default on 26.05 is: **input-addressed builds, FODs (`fetchurl`, `vendorHash`) for the CA you already use, `nix-store --optimise` for disk — not experimental CA on the laptop**.

Mental model

Kind	Output path depends on	Daily
Input-addressed	The `.drv` (builder, env, input paths)	Packages, NixOS
FOD	Declared `outputHash`	Fetchers, cargo/go vendor
Experimental CA	Hash of the built bytes	Hydra labs

A comment in a `runCommand` changes the `.drv` even if `cat $out` is identical. That wastes cache. It also makes "why did this rebuild?" answerable. Live with it.

Nix 2.35 lazy flake copy is **eval** performance. It is not CA builds. Do not mix them in an incident doc. The 26.05 channel's Nix 2.34 vs this book's CLI 2.35+ does not change the addressing model you ship.

Worked examples

Case 1: FOD — identity is the hash

Save as `fod.nix`:

```
# fod.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.fetchurl {
  url = "https://ftp.gnu.org/gnu/hello/hello-2.12.1.tar.gz";
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

```
nix-build fod.nix
```

Mismatch prints `got:`. Paste that into `hash`. Same `hash`, different URL (mirror) → same store path. That is the CA you already depend on: `vendorHash`, `cargoHash`, `fetchurl`.

`outputHashMode` is `"flat"` (one file, `fetchurl`) or `"recursive"` (a directory NAR, `fetchFromGitHub`, `vendorHash`). Mixing them is a hash mismatch that looks like a wrong URL. If `got:` is `sha256-` of a NAR and you expected a single-file hash, you used the wrong mode — not a bad mirror.

Case 2: Comment rebuilds an input-addressed path

Save as `banner.nix`:

```
# banner.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.runCommand "desk-banner" { } ''
  echo desk > $out
  # lab note
''
```

```
nix-instantiate banner.nix
# change the comment, instantiate again
nix-instantiate banner.nix
```

Two `.drv` hashes. `diff` the two `$out` files after `nix-build` — same bytes, different names. Normal for input-addressed builds. Do not "fix" this with `__contentAddressed`.

Case 3: `nix show-derivation`

```
nix show-derivation $(nix-instantiate -E '(import <nixpkgs> {}).hello') | head
nix show-derivation $(nix-build fod.nix --no-out-link) | jq '.[].outputs'
```

FODs have `outputHash` / `outputHashAlgo`. `hello` does not. If you are staring at a path and do not know which family it is, this is the check.

```
"outputs": {
  "out": {
    "hash": "…",
    "hashAlgo": "r:sha256"
  }
}
```

Case 4: Disk without CA — optimise

```
sudo nix-store --optimise
df -h /nix
```

Hard-link identical **files** inside distinct input-addressed paths. No experimental flags.

Save as `nix-optimise.nix`:

```
# nix-optimise.nix
{
  nix.settings.auto-optimise-store = true;
  nix.optimise.automatic = true;
}
```

14-day GC plus this is the disk plan. Experimental CA is not.

Case 5: Do not enable CA on the desk

```
# do not
{
  nix.settings.experimental-features = [ "ca-derivations" ];
}
```

unless the whole team **and** the cache agree. nixpkgs 26.05 still assumes input-addressed for almost everything you ship. Substituters miss, support assumes input-addressed, and a one-line stdenv change becomes a debugging career.

A recursive FOD (directory) looks like this — still not experimental CA:

```
# fod-git.nix
{ pkgs ? import <nixpkgs> { } }:

pkgs.fetchFromGitHub {
  owner = "NixOS";
  repo = "nixpkgs";
  rev = "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa";
  hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
}
```

```
nix-build fod-git.nix
nix show-derivation $(nix-build fod-git.nix --no-out-link) | jq '.[].outputs.out.hashAlgo'
```

You want `r:sha256` (recursive). `fetchurl` of a `.tar.gz` is usually flat. Same `got:` paste workflow.

The trap

The trap is **`__contentAddressed = true` on a production module** "to save disk." Use GC + `--optimise`. The other trap is calling Nix 2.35 lazy trees "content-addressed Nix" in a postmortem — different layer.

The boring rule

- FODs for fetches. Input-addressed for builds.
- `--optimise` + 14-day GC for disk.
- `show-derivation` when a path's kind is unclear.
- 2.35 lazy copy CA.
- No `ca-derivations` on workstation flakes.

Try this

1. Case 3: compare `hello` vs a `fetchurl` for `outputHash`.
2. Case 2: two instantiations; `diff` the `.drv` files.
3. `git grep contentAddressed` / `ca-derivations` — empty on the desk.
4. `nix-store --optimise` on a lab; note `df` delta.

AI Agent Sandboxes, Loop Engineering, and Universal Overlays

AI Agent Sandboxes, Loop Engineering, and Universal Overlays

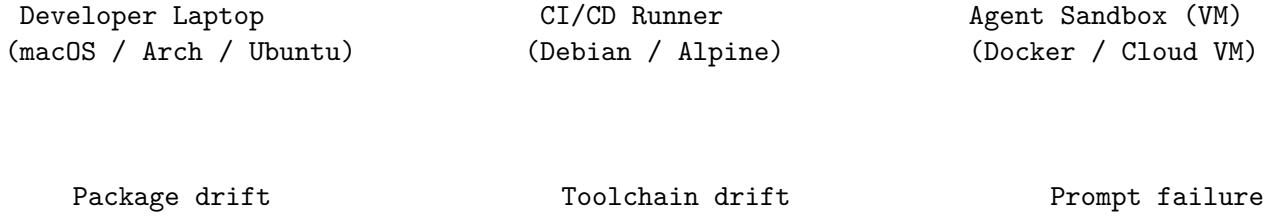
Giving an autonomous AI coding agent unrestricted `sudo` on an imperative Linux distro is reckless; giving it `sudo` on NixOS is routine engineering. The boring default is: **one declarative source of truth for developer laptops, CI runners, and ephemeral agent sandboxes (`devenv` or flake `devShells`), capability truncation and security hotfixes via universal overlays, and automated loop engineering that puts the entire operating system under test with `runNixOSTest`.**

Mental model

Autonomous AI development agents (like Claude Code, Codex, Cursor, and custom agent harnesses) thrive on mathematical determinism. While the Nix language has historically had a steep learning curve, modern LLMs excel at generating and reasoning over declarative Nix expressions. This shifts Nix from an esoteric power tool to the foundation of the AI-augmented software factory.

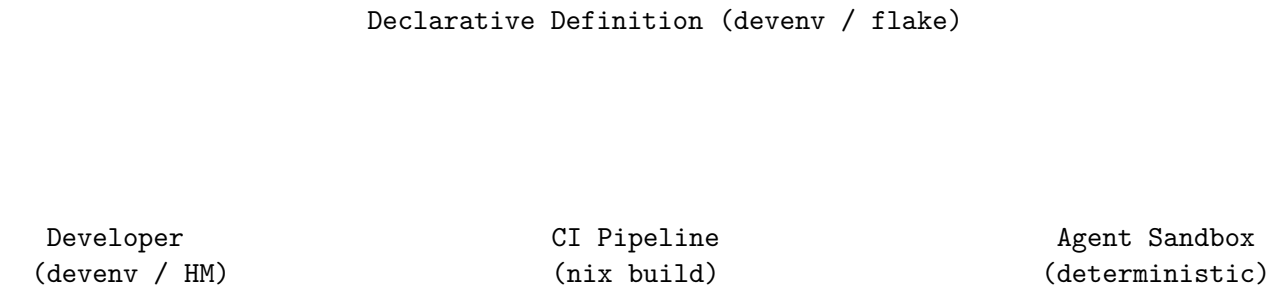
The three-way drift crisis

Most organizations suffer from configuration drift between local developer workstations and CI/CD runners. Introducing ephemeral AI agent sandboxes turns this into a three-way divergence:



When an agent hallucinates or encounters missing system headers, it attempts ad-hoc package installation (`apt-get install`, `brew install`, `pip install --break-system-packages`), destroying reproducibility.

Nix collapses all three environments into a single source of truth:



Why the operating system matters for agents

On mutable distributions (Debian, Fedora, Ubuntu), granting an autonomous agent `sudo` access risks bricked packages, poisoned `/etc`, and conflicting shared libraries. On NixOS:

1. **System state is read-only**: `/nix/store` is immutable.
2. **Generations are atomic**: Every system change is a new generation in the bootloader and system profile.
3. **Instant rollback**: If an agent corrupts a service configuration, `nixos-rebuild switch --rollback` restores the known good state instantaneously.
4. **Agent backpressure**: Pre-commit hooks, linters (`rustfmt`, `ruff`), and verification harnesses provide deterministic feedback (backpressure) directly to the agent's context window.

Universal overlays: "Fix everything at the root"

In traditional architectures, patching a critical library (such as OpenSSL) or altering upstream binary behavior requires: 1. Forking the upstream repository on GitHub. 2. Setting up dedicated build pipelines and container registries. 3. Hosting patched artifacts in private binary managers (Artifactory, Nexus). 4. Reconfiguring package managers across all machines.

In Nix, an **overlay** redefines the fixed-point package set (`pkgs`). With a few lines of code, you can patch a CVE or truncate an unsafe tool capability across your entire software graph—from dev shells and Docker images to full NixOS virtual machines.

Worked examples

Case 1: Declarative agent sandbox with devenv and backpressure

Save as `devenv.nix` in the project root. This file specifies the language toolchain, required services, and formatting hooks that enforce backpressure on the agent:

```
# devenv.nix
{ pkgs, ... }:

{
  # 1. Exact packages required for the project
  packages = [
    pkgs.git
    pkgs.ripgrep
    pkgs.fd
    pkgs.jq
  ];

  # 2. Declarative language runtimes
  languages.rust = {
    enable = true;
    channel = "stable";
  };

  # 3. Ephemeral local background services (managed via process-compose)
  services.postgres = {
    enable = true;
    package = pkgs.postgresql_16;
    initialDatabases = [
      { name = "desk_dev"; }
    ];
    listen_addresses = "127.0.0.1";
    port = 5432;
  };

  # 4. Agent backpressure: enforce git hooks before commits are accepted
  git-hooks.hooks = {
    rustfmt.enable = true;
    clippy.enable = true;
  };

  # 5. Environment variables exposed to both humans and agents
  env = {
    DATABASE_URL = "postgres://postgres@127.0.0.1:5432/desk_dev";
  };
}
```

Spawn the complete environment with a single command:

```
devenv up -d
devenv shell
```

The human developer, the CI test job, and the AI agent all execute against the exact same PostgreSQL binaries and Rust toolchain, completely eliminating runtime divergence.

Case 2: Minimal layered container from the same package expression

Rather than writing a `Dockerfile` with multi-stage builds and `apt-get` cache churn, compose the package into an OCI container image directly from Nix using `pkgs.dockerTools.buildLayeredImage`:

```
# docker-image.nix
{ pkgs ? import <nixpkgs> { } }:

let
  app = pkgs.writeShellApplication {
    name = "agent-worker";
    runtimeInputs = [ pkgs.curl pkgs.jq ];
    text = ''
      echo "Starting agent worker on $(uname -s)..."
      curl -s https://httpbin.org/get | jq .
    '';
  };
in
pkgs.dockerTools.buildLayeredImage {
  name = "agent-worker";
  tag = "latest";

  # Each store path forms an independent OCI layer; base dependencies stay cached
  contents = [
    (pkgs.buildEnv {
      name = "image-root";
      paths = [ app pkgs.cacert ];
      pathsToLink = [ "/bin" "/etc" ];
    })
  ];

  config = {
    Cmd = [ "/bin/agent-worker" ];
    Env = [
      "PATH=/bin"
      "SSL_CERT_FILE=/etc/ssl/certs/ca-bundle.crt"
    ];
  };
}
```

Build the container image without requiring root or a running Docker daemon:

```
nix-build docker-image.nix -o result-image.tar.gz
podman load < result-image.tar.gz
```

Case 3: Universal overlay for capability truncation (preventing agent force-pushes)

When autonomous agents operate inside git repositories, a common safety failure is an agent executing `git push --force` or destroying uncommitted state during error recovery loops.

Rather than trying to intercept command-line invocations through fragile shell aliases or wrapper scripts, an overlay patches the `git` binary itself at the source level. This guarantees that `push --force` physically cannot execute, regardless of how the binary is invoked.

Save as `overlays/no-force-push-git.nix`:

```
# overlays/no-force-push-git.nix
final: prev: {
  git = prev.git.overrideAttrs (oldAttrs: {
    pname = "git-no-force";

    # Apply a patch to builtin/push.c stripping the force options
    postPatch = (oldAttrs.postPatch or "") + ''
      substituteInPlace builtin/push.c \
        --replace-fail 'OPT_BIT('\'''f'\'', "force"' '// OPT_BIT('\'''f'\'', "force"'
    '';
  });
}
```

When this overlay is added to `nixpkgs.overlays`:

```
# flake.nix
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs {
        inherit system;
        overlays = [ (import ./overlays/no-force-push-git.nix) ];
      };
    in
    {
      devShells.${system}.default = pkgs.mkShell {
        packages = [ pkgs.git ];
      };
    };
}
```

Inside this shell, invoking `git push --force` returns `error: unknown option 'force'`, providing physical safety guarantees against rogue agent loops.

Case 4: Universal security hotfix across the entire dependency graph

Suppose an urgent OpenSSL vulnerability is announced. In traditional infrastructures, identifying and updating every binary linked against OpenSSL can take weeks.

With a Nix overlay, you re-pin OpenSSL at the root of the dependency graph:

```
# overlays/openssl-cve-hotfix.nix
final: prev: {
  openssl = prev.openssl.overrideAttrs (old: rec {
    version = "3.0.14";
    src = prev.fetchurl {
      url = "https://www.openssl.org/source/openssl-${version}.tar.gz";
      hash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
    };
  });
}
```

Because Nixpkgs uses a fixed-point evaluation (`final` and `prev`), every package in your flake that depends on `openssl` (curl, python3, nginx, postgresql) is automatically rebuilt against the patched OpenSSL derivation. No forked repositories, no third-party artifact hosting, and zero manual rebuild coordination.

Case 5: Loop engineering — putting the entire operating system under test

The pinnacle of agent loop engineering is not merely running unit tests inside a container, but putting the **entire operating system and network topology** under test.

NixOS provides `runNixOSTest` (or `testers.runNixOSTest`), which spins up headless QEMU nodes across isolated virtual networks (VLANs), validates firewall filtering, tests systemd service lifecycles, and executes Python-driven assertions.

Save as `tests/network-topology-test.nix`:

```
# tests/network-topology-test.nix
# Multi-node VM test: HAProxy frontend routing to an isolated backend web service.
{ lib, pkgs, ... }:

let
  frontendVlan = 1;
```

```
  backendVlan = 2;

  haproxyFrontendIp = "192.168.${toString frontendVlan}.1";
  haproxyBackendIp  = "192.168.${toString backendVlan}.1";
  haproxyUntrustedIp = "192.168.${toString backendVlan}.50";
  webIp             = "192.168.${toString backendVlan}.2";

  frontendPort = 80;
  webPort      = 8080;
  helloBody    = "hello from backend service";
in
{
  name = "agent-loop-network-test";

  nodes = {
    # Node 1: HAProxy reverse proxy bridging VLAN 1 (external) and VLAN 2 (internal)
    haproxy = { pkgs, ... }: {
      virtualisation.vlans = [ frontendVlan backendVlan ];
      virtualisation.memorySize = 512;
      environment.systemPackages = [ pkgs.curl ];

      networking.interfaces."eth${toString backendVlan}".ipv4.addresses = [
        { address = haproxyUntrustedIp; prefixLength = 24; }
      ];

      networking.firewall = {
        enable = true;
        interfaces."eth${toString frontendVlan}".allowedTCPPorts = [ frontendPort ];
      };

      services.haproxy = {
        enable = true;
        config = ''
          defaults
            mode http
            timeout connect 5s
            timeout client 10s
            timeout server 10s

          frontend fe_http
            bind ${haproxyFrontendIp}:${toString frontendPort}
            default_backend be_hello

          backend be_hello
            server web ${webIp}:${toString webPort} check
        '';
      };
```

```
  };

  # Node 2: Backend Python HTTP server locked down with strict host firewall
  web = { pkgs, ... }: {
    virtualisation.vlans = [ backendVlan ];
    virtualisation.memorySize = 512;

    networking.firewall = {
      enable = true;
      allowedTCPPorts = [ ]; # Closed to the public; only HAProxy allowed via iptables
      extraCommands = ''
        # Allow inbound traffic on port 8080 ONLY from HAProxy's authorized backend IP
        iptables -A INPUT -s ${haproxyBackendIp} -p tcp --dport ${toString webPort} -j ACC
        # Prevent backend from initiating new outbound connections
        iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
        iptables -A OUTPUT -j REJECT --reject-with icmp-host-prohibited
      '';
    };

    systemd.services.backend-service = {
      description = "Backend python server";
      wantedBy = [ "multi-user.target" ];
      after = [ "network.target" ];
      serviceConfig = {
        ExecStart = "${pkgs.python3}/bin/python3 -c '\nimport http.server\nclass H(http.se
        DynamicUser = true;
        ProtectSystem = "strict";
      };
    };
  };
};

# Python test script executed by the test driver
testScript = ''
  start_all()

  haproxy.wait_for_unit("haproxy.service")
  web.wait_for_unit("backend-service.service")
  web.wait_for_unit("firewall.service")

  with subtest("HAProxy frontend successfully proxies to protected backend"):
      haproxy.wait_until_succeeds(
          "curl -fsS http://${haproxyFrontendIp}/ | grep -F '${helloBody}'"
      )

  with subtest("Backend rejects unauthorized source IP on the same VLAN"):
```

```
        haproxy.fail(
            "curl -fsS --max-time 3 --interface ${haproxyUntrustedIp} http://${webIp}:${toSt
        )

    with subtest("Backend is strictly forbidden from initiating outbound connections"):
        web.fail("curl -fsS --max-time 3 http://${haproxyBackendIp}:${toString frontendPort}
  '';
}
```

Run this full multi-machine integration test in CI or during an agent verification loop:

```
nix build .#checks.x86_64-linux.agent-loop-network-test -L
```

If the agent needs to diagnose a failure interactively, launch the Python interactive test driver:

```
nix run .#checks.x86_64-linux.agent-loop-network-test.driverInteractive
```

Inside the driver REPL:

```
>>> start_all()
>>> haproxy.shell("curl -I http://192.168.1.1/")
>>> web.shell("iptables -L -n -v")
```

Bare metal fleet economics in the AI era

As AI models commoditize software generation, business margins compress. Running expansive fleets of dynamic virtual instances across hyperscalers (AWS, GCP) introduces punishing operational overhead.

Declarative NixOS unlocks extraordinary leverage: - A small engineering team can operate hundreds of bare-metal machines with the same reproducibility typically expected of container clusters. - Because hardware nodes boot from declarative flakes, fleet provisioning via tools like `disko`, `nixos-anywhere`, and `colmena` eliminates configuration drift across thousands of CPU and GPU cores.

Antipatterns

Antipattern	Why it fails	Boring replacement
Giving agents `sudo` on mutable distros	Agent runs `apt-get`, `pip`, or edits `/etc`, corrupting host state beyond repair	Run agents inside NixOS generations with atomic rollback (`--rollback`)
Forking upstream repos for minor patches	Massive maintenance burden, merge conflicts, separate CI and Artifactory registries	Apply a 5-line Nix overlay using `overrideAttrs`
Three separate environment files	Shell scripts for dev, Dockerfiles for CI, bespoke AMIs for agent runners	One `devenv.nix` or flake `devShell` acting as the single source of truth
Testing only the app binary in isolation	Fails in staging due to network, firewall, or systemd permission mismatches	Put the operating system under test with `runNixOSTest`
Overriding half of Nixpkgs globally	Causes massive cache invalidation and infinite evaluation loops	Keep overlays surgical; target leaf packages or specific shared libraries

---

The boring rule

- **One truth**: Developer workstations, CI runners, and agent sandboxes share the exact same declarative toolchain expression.
- **Backpressure**: Enforce formatting, linting, and tests through git hooks and strict type boundaries to keep autonomous agents grounded.
- **Overlays for deltas**: When upstream lacks a feature, has a CVE, or needs capability truncation, write an overlay instead of maintaining a fork.
- **OS-level testing**: For network-aware services, put the operating system under test using headless QEMU nodes in `runNixOSTest`.

Verification checklist

1. Verify `devenv up` or `nix develop` boots all declared services without host dependency contamination.
2. Confirm that patched packages in `overlays/` propagate to both `devShells` and container outputs.
3. Validate that capability truncation patches (e.g. `git-no-force`) produce the expected error when forbidden options are executed.
4. Run `nix flake check -L` to ensure all multi-node NixOS integration tests pass hermetically in isolated QEMU environments.

Production Labs

Reading NixOS.org Infra as a Pattern Source

Reading NixOS.org Infra as a Pattern Source

The public tree at `github:NixOS/infra` is how the NixOS project runs hydra, ofborg, builders, DNS, and terraform. The boring default is: **steal the *shapes* (keys file, host factory, critical/non-critical split, restricted builder SSH, channel table) and re-express them on the desk fleet — do not clone the repo and do not deploy their hosts**.

This part is labs. The capstone already has five desk roles. Here we add the org-scale habits that tree actually uses in 2026.

Mental model

Path in NixOS/infra	What it is	Desk equivalent
`flake.nix`	Live contract: 26.05-small, `follows`, extra substituter	One lock, one cache
`keys.nix`	People and machines as SSH/age groups	`keys.nix` for desk-ops
`builders/`	`mkNixOS` factory + common modules	`lib/mkHost.nix`
`non-critical-infra/`	Separate Colmena hive + sops	`hive-apps` vs `hive-core`
`channels.nix`	Hydra job → channel name + status	Attic/Cachix "tested" attr
`terraform/`	OpenTofu `withPlugins` for the cloud envelope	IaC chapters, same wrapper
`checks/`	Group host toplevels by arch for CI	`checks.x86_64-linux.hosts`
`docs/inventory.md`	**Stale** (NixOps, Packet, DataDog)	Do not copy; flake wins

The README still says "managed using NixOps." The flake says Colmena, Disko, sops-nix, agenix, nix-darwin 26.05, OpenTofu. **Trust the flake.** Inventory docs rot; `flake.lock` does not.

They use **flake-parts**. We do not. The factory is a function in `flake.nix`. Same hosts, no extra DSL.

They pin `https://channels.nixos.org/nixos-26.05-small/nixexprs.tar.zst` (the *small* channel). The desk default stays `github:NixOS/nixpkgs/nixos-26.05` unless you have measured that you only need the small jobset.

```
NixOS/infra (read)
    keys, factory, split hives, restricted SSH, channel table
            rewrite

desk flake (write)  -  26.05, Colmena, sops, Disko
```

Worked examples

Case 1: Map the live flake, ignore the stale README

On a throwaway clone (read-only; you will not `colmena apply`):

```
git clone --depth 1 https://github.com/NixOS/infra.git /tmp/nixos-infra
cd /tmp/nixos-infra
rg -n 'nixos-26.05|colmena|nixops|disko|sops' flake.nix README.md | head
```

Output (shape):

```
flake.nix:    nixpkgs.url = "https://channels.nixos.org/nixos-26.05-small/nixexprs.tar.zst";
flake.nix:    colmena = { url = "github:zhaofengli/colmena"; ...
flake.nix:    disko = { url = "github:nix-community/disko"; ...
flake.nix:    sops-nix = { url = "github:Mic92/sops-nix"; ...
README.md:All the hosts are currently managed using NixOps.
```

README lost. Flake won. That is the first lab skill: **the deploy tool is the flake output, not the first paragraph of the README**.

```
nix flake metadata --json 2>/dev/null | jq -r '.locks.nodes.nixpkgs.original.url // .locks.n
```

You should see a 26.05-small tarball or its `narHash`, not `nixpkgs-unstable` as the default OS pin. Unstable is a **second** input (`nixpkgs-unstable`) for tools that need it.

Case 2: Extra substituter on the flake, daemon still needs the key

They set:

```
# flake.nix fragment - pattern only
{
  nixConfig.extra-substituters = [ "https://nixos-infra-dev.cachix.org" ];
  nixConfig.extra-trusted-public-keys = [
    "nixos-infra-dev.cachix.org-1:OvwhqPPs81cInrtRAX0K7dG6lw8wXcQEX4xyp4AnSXw="
  ];
}
```

`nixConfig` in a flake is a **hint**. Untrusted users need `accept-flake-config` or the same keys in `/etc/nix/nix.conf`. The desk already puts the team cache in `nix.settings`. Do not copy their Cachix name onto the desk fleet; copy the *habit*: one extra cache, key next to URL, never a signing secret in `nixConfig`.

Case 3: `follows` is how a 20-input flake stays on one nixpkgs

Their `colmena`, `disko`, `sops-nix`, `srvos`, `agenix` all `inputs.nixpkgs.follows = "nixpkgs"`. Darwin follows `nixpkgs-darwin`. That is the same `follows` rule as the tooling-flake chapter, at org scale.

```
# in the clone
rg 'follows = "nixpkgs"' flake.nix | wc -l
```

A two-digit count is success. An input that does **not** follow is a second stdenv. Hunt those on purpose (hydra, ofborg) — they are the exceptions, documented by existing.

Case 4: Desk flake that *looks* like infra without flake-parts

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk fleet - shapes from NixOS/infra, not their hosts";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    disko.url = "github:nix-community/disko";
    disko.inputs.nixpkgs.follows = "nixpkgs";
    sops-nix.url = "github:Mic92/sops-nix";
    sops-nix.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, disko, sops-nix }:
    let
      mkHost = name: extraModules:
        nixpkgs.lib.nixosSystem {
          system = "x86_64-linux";
          modules = [
            { networking.hostName = name; system.stateVersion = "26.05"; }
            disko.nixosModules.disko
            sops-nix.nixosModules.sops
          ] ++ extraModules;
        };
    in
    {
```

```
      nixosConfigurations = {
        gw-01  = mkHost "gw-01"  [ ./hosts/gw-01.nix ];
        app-01 = mkHost "app-01" [ ./hosts/app-01.nix ];
        ops-01 = mkHost "ops-01" [ ./hosts/ops-01.nix ];
      };
    };
}
```

`mkHost` is their `mkNixOS` without flake-parts. Three hosts are enough for the labs in this part. Capstone still has five.

```
nix flake show
```

```
nixosConfigurations
    app-01
    gw-01
    ops-01
```

Case 5: What we will not copy

NixOS/infra	Why not on the desk
Hydra of all of nixpkgs	A queue-runner for the world. Desk CI is `nix flake check`.
ofborg eval fleet	GitHub PR eval for nixpkgs. Not your app.
`srvos` as a required input	Fine later; not the first module.
flake-parts	Extra DSL. A function is enough.
Their `keys.nix` values	Public SSH keys of *their* operators. Never paste into yours.
`isoImage.contents` of `key.txt`	Private keys stay off git.
NixOps	The README; not the flake.

Open `keys.nix` in the clone, **look at the shape** (`ssh.users`, `ssh.groups`, `ssh.machines`, `age.recipients`), then close it. Next chapter you write *your* keys.

The trap

The trap is **`git clone` + `colmena apply` against `staging-hydra.nixos.org`**. That hive is not yours. Read-only clone. Rewrite patterns into the desk flake.

The other trap is treating `docs/inventory.md` as current. Packet.net, NixOps, and DataDog in that file are archaeology. The flake and `non-critical-infra/` are the live map.

The boring rule

- Flake is the contract. README and inventory can lie.
- Steal shapes: keys file, host factory, hive split, restricted builder SSH, channel table.
- Do not steal hosts, secrets, flake-parts, or Hydra-for-nixpkgs.
- `follows` on every input that can. Extra substituter + key, never a signing secret.
- Desk pin stays `nixos-26.05` unless you have a reason for `-small`.

Try this

1. Clone NixOS/infra at depth 1. Diff `README.md` vs `flake.nix` for the deploy tool.
2. Count `follows = "nixpkgs"` in `flake.nix`.
3. `rg NixOps` the repo; note which files still say it.
4. Write `mkHost` as in Case 4 in a throwaway dir; `nix flake show`. Do not add their Cachix.

Operator Keys and Groups

Operator Keys and Groups

NixOS/infra keeps every operator and machine pubkey in one `keys.nix`: users, groups (`infra-core`, `infra`, `ofborg`), machine host keys, age recipients. The boring default is: **the same file for the desk — public keys only — and `users.mutableUsers = false` so a leftover `authorized_keys` on disk cannot add a human**.

Do not copy their key strings. Copy the attrset.

Mental model

```
keys.nix
  ssh.users.alice    = [ "ssh-ed25519 AAAA… alice@laptop" ]
  ssh.groups.desk-core = alice ++ bob
  ssh.machines.gw-01 = "ssh-ed25519 AAAA…"   # host key, for knownHosts
  age.recipients.alice = [ "age1…" ]


users.users.root.openssh.authorizedKeys.keys = keys.ssh.groups.desk-core;
```

Groups are concatenations of user lists. Adding alice to `desk-core` is one line; every host that imports the group gets her key on the next switch. Removing her is the same line, plus a rebuild — not `sed` on five boxes.

`mutableUsers = false` means NixOS owns `/etc/passwd` and authorized keys. A `useradd` on the box dies at next activation.

Age groups in their file mix SSH groups with YubiKey `age1yubikey1…` recipients. Desk: sops `.sops.yaml` creation rules point at the same people. One list of humans.

Worked examples

Case 1: Desk `keys.nix`

Save as `keys.nix`:

```
# keys.nix
rec {
  ssh = {
    users = {
```

```
      alice = [
        "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskAliceExampleKeyOnlyNotReal alice@laptop"
      ];
      bob = [
        "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskBobExampleKeyOnlyNotReal bob@laptop"
      ];
      hydra-queue = [
        "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskHydraExampleKeyOnlyNotReal hydra-queue@ops
      ];
    };

    groups = with ssh.users; {
      desk-core = alice ++ bob;
      desk-builders = desk-core;
      desk-ci = desk-core ++ hydra-queue;
    };

    machines = {
      gw-01 = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskGw01HostKeyExampleOnly";
      app-01 = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskApp01HostKeyExampleOnly";
      ops-01 = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDeskOps01HostKeyExampleOnly";
    };
  };

  age = {
    recipients = {
      alice = [ "age1qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq2w4k7f" ];
    };
    groups = {
      desk-core = ssh.groups.desk-core;
    };
  };
}
```

Replace every `AAAA…` / `age1…` with **your** pubkeys (`ssh-keygen -y`, `ssh-to-age < ~/.ssh/id_ed25519.pub`). The strings above are placeholders; they must not be committed as if they opened a box.

```
nix eval --file keys.nix 'ssh.groups.desk-core' --apply builtins.length
```

```
2
```

Case 2: Root and a named admin from the group

Save as `modules/users-from-keys.nix`:

```
# modules/users-from-keys.nix
{ pkgs, ... }:

let
  keys = import ../keys.nix;
in
{
  users.mutableUsers = false;

  users.users.root.openssh.authorizedKeys.keys = keys.ssh.groups.desk-core;

  users.users.deskadmin = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    openssh.authorizedKeys.keys = keys.ssh.groups.desk-core;
  };

  security.sudo.wheelNeedsPassword = false; # lab; prod: password or SSH-only + sudoers
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "prohibit-password";
    };
  };
}
```

NixOS/infra uses `users.extraUsers.root` (legacy alias). On 26.05 write `users.users.root`. Same option.

```
nix eval .#nixosConfigurations.ops-01.config.users.users.root.openssh.authorizedKeys.keys --
```

Must equal the length of `desk-core`.

Case 3: `knownHosts` from `ssh.machines`

Save as `modules/known-hosts.nix`:

```
# modules/known-hosts.nix
{
  programs.ssh.knownHosts = let
    keys = import ../keys.nix;
  in
    builtins.mapAttrs (_name: publicKey: { inherit publicKey; }) keys.ssh.machines;
}
```

Colmena/SSH from `ops-01` then does not prompt on first connect. Rotate a host key → change `keys.nix` → switch `ops-01`. Leaving `StrictHostKeyChecking=no` in Colmena is how you accept a MITM once.

Host keys on ephemeral roots must be **persisted** (`/etc/ssh/ssh_host_ed25519_key`) or this file and the metal disagree every boot.

Case 4: Drop a human in one commit

```
# keys.nix fragment after alice leaves
groups = with ssh.users; {
  desk-core = bob;          # alice removed
  desk-ci = bob ++ hydra-queue;
};
```

```
nix eval --file keys.nix 'ssh.groups.desk-core' --apply 'xs: builtins.any (k: builtins.match
```

```
false
```

Then `colmena apply --on @core`. There is no "disable the account on the bastion wiki." The next generation has no alice key. Rotate anything she could have copied (sops recipients, age, Cachix tokens).

Case 5: sops creation rules share the same people

Save as `.sops.yaml` (desk, not their file):

```
# .sops.yaml
keys:
  - &alice age1qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq2w4k7f
  - &bob   age1qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqp9z7n0h
creation_rules:
  - path_regex: secrets/core/.*
    key_groups:
      - age: [*alice, *bob]
  - path_regex: secrets/apps/.*
    key_groups:
      - age: [*alice, *bob]
```

Need-to-have: `secrets/apps/` does not need a laptop that only ships CI. NixOS/infra's non-critical README says the same: add your key on a PR, someone with access runs `sops updatekeys`. Desk: PR to `keys.nix` **and** `.sops.yaml`, then `sops updatekeys secrets/core/*.yaml`.

The trap

The trap is **pasting NixOS/infra `keys.nix` into the desk repo**. Those keys open *their* machines. Generate yours. The other trap is `mutableUsers = true` plus this file: a human `ssh-copy-id`s a third key and Nix never removes it.

A third: listing alice in `desk-core` but forgetting `.sops.yaml` — she can SSH and cannot decrypt. Or the reverse: she decrypts on the laptop and cannot log in.

The boring rule

- One `keys.nix`. Public keys only. Groups are concatenations.
- `mutableUsers = false`. Authorized keys come from the group.
- `knownHosts` from `ssh.machines`. Persist host keys on tmpfs roots.
- Drop a human in `keys.nix` + sops recipients + apply. Then rotate.
- Never commit private keys. Never copy another org's pubkeys as if they were yours.

Try this

1. Put two of *your* pubkeys in `keys.nix`. `nix eval` the group length.
2. Import `users-from-keys.nix` on a lab VM; `ssh -i` as each key; remove one; switch; confirm the removed key is refused.
3. `ssh-keyscan` a lab host; put it in `ssh.machines`; rebuild `ops-01`; `ssh` without a prompt.
4. `git grep -n 'ssh-ed25519' keys.nix` — every line is a key you can name.

Critical vs Non-Critical Hives

Critical vs Non-Critical Hives

NixOS/infra splits **builders / hydra / ofborg** from `non-critical-infra/` (community hosts, staging hydra, a second Colmena hive, its own `.sops.yaml` and `devShells.non-critical-infra`). The boring default is: **two Colmena tags (or two flakes) so a botched app host cannot share sops keys or apply waves with `gw-01`.**

The capstone hive is one file. This lab splits it the way a real org does once "the website" and "the CA / builders" must not share a blast radius.

Mental model

```
flake.nix
  colmena.meta
  defaults            →  ssh keys, firewall, stateVersion
  @core  gw-01 db-01 ops-01     secrets/core/
  @apps  app-01 app-02          secrets/apps/
```

NixOS/infra goes further: a **directory** with its own sops and `colmena.sh`. Desk: one flake, two tags, two secret trees. Two flakes when the team that owns apps must not eval the core modules.

Their non-critical README: secrets are need-to-have; add your key on a PR; someone who already has access runs `updatekeys`. That is the onboarding ritual. Not a Slack DM of `key.txt`.

Worked examples

Case 1: Tags on the existing hive

Save as `flake.nix`:

```nix
# flake.nix
{
  description = "Desk hive with core vs apps";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    colmena = {
      meta.nixpkgs = import nixpkgs { system = "x86_64-linux"; };
```

```
      defaults = { ... }: {
        system.stateVersion = "26.05";
        deployment.targetUser = "root";
        imports = [ ./modules/users-from-keys.nix ];
      };

      gw-01 = {
        deployment.targetHost = "203.0.113.10";
        deployment.tags = [ "core" "edge" ];
        imports = [ ./hosts/gw-01.nix ];
      };
      db-01 = {
        deployment.targetHost = "203.0.113.21";
        deployment.tags = [ "core" "db" ];
        imports = [ ./hosts/db-01.nix ];
      };
      ops-01 = {
        deployment.targetHost = "203.0.113.30";
        deployment.tags = [ "core" "ops" ];
        imports = [ ./hosts/ops-01.nix ];
      };
      app-01 = {
        deployment.targetHost = "203.0.113.11";
        deployment.tags = [ "apps" ];
        imports = [ ./hosts/app-01.nix ];
      };
      app-02 = {
        deployment.targetHost = "203.0.113.12";
        deployment.tags = [ "apps" ];
        imports = [ ./hosts/app-02.nix ];
      };
    };
  };
}
```

```
nix shell nixpkgs#colmena --command colmena eval -E '{ nodes }: builtins.attrNames nodes'
```

Five names. Apply waves:

```
colmena apply --on @core --dry-run
colmena apply --on @apps --dry-run
```

Never `--on @all` for a kernel bump on `@core`. Apps can fan out. `db-01` stays serial inside `@core` (capstone DR).

Case 2: Two sops trees

```
secrets/
  core/
    luks.yaml          # recipients: desk-core only
    restic.yaml
  apps/
    db-password.yaml   # recipients: desk-core + app runtime? no - hosts decrypt
```

App hosts decrypt `db-password` with the **host** age key, not alice's laptop. Alice's key is for editing the YAML. That is the same split as NixOS/infra: humans in `.sops.yaml`, machines as extra recipients.

Save as `modules/sops-core.nix`:

```
# modules/sops-core.nix
{ config, ... }:
{
  sops.defaultSopsFile = ../secrets/core/luks.yaml;
  sops.age.sshKeyPaths = [ "/etc/ssh/ssh_host_ed25519_key" ];
}
```

Save as `modules/sops-apps.nix`:

```
# modules/sops-apps.nix
{
  sops.defaultSopsFile = ../secrets/apps/db-password.yaml;
  sops.age.sshKeyPaths = [ "/etc/ssh/ssh_host_ed25519_key" ];
}
```

Import `sops-core.nix` only on `@core`. Import `sops-apps.nix` on `@apps` (and `db-01` if the DB password lives there). An app box that can decrypt LUKS material is the wrong blast radius.

Case 3: Named devShells, not one kitchen sink

NixOS/infra exposes `devShells.builders` (agenix) and `devShells.non-critical-infra` (colmena, sops, ssh-to-age) and `devShells.terraform` (OpenTofu `withPlugins`). Desk:

```
# flake.nix fragment
{
  devShells.x86_64-linux.core = pkgs.mkShell {
    packages = [ pkgs.colmena pkgs.sops pkgs.ssh-to-age ];
  };
  devShells.x86_64-linux.apps = pkgs.mkShell {
    packages = [ pkgs.colmena pkgs.sops ];
  };
  devShells.x86_64-linux.tf = pkgs.mkShell {
```

```
    packages = [
      (pkgs.opentofu.withPlugins (p: [ p.aws p.local ]))
    ];
  };
}
```

The app team's direnv is `use flake .#apps`. They do not get the OpenTofu wrapper or core sops files in `$PWD` if those live in another worktree. Need-to-have.

Case 4: Staging is a host, not a branch of secrets

Their Colmena list includes `staging-hydra.targetHost = "staging-hydra.nixos.org"` **in the non-critical hive**. A hydra change is exercised there before the production hydra input moves.

Desk equivalent:

```
# hosts/staging-app.nix
{
  imports = [ ./app-01.nix ];
  networking.hostName = "staging-app";
  # acmeCA staging; smaller RAM; same modules
}
```

```
colmena apply --on staging-app
curl -sf https://staging.api.desk.internal/healthz
colmena apply --on @apps
```

Do not "staging" by decrypting prod sops on a laptop and hoping. Same modules, different host, different ACME, different tag.

Case 5: `colmena.sh` is a one-liner, not a platform

NixOS/infra's `non-critical-infra/colmena.sh` is a thin wrapper so humans do not forget `--flake`. Desk:

```
# colmena-apps.sh
#!/usr/bin/env bash
set -euo pipefail
exec nix shell nixpkgs#colmena --command colmena "$@" --on @apps
```

```
chmod +x colmena-apps.sh
./colmena-apps.sh apply --dry-run
```

No extra DSL. The wrapper only *restricts* the tag. A wrapper that defaults to `@all` is worse than none.

The trap

The trap is **one sops file for LUKS, restic, and the app DB password**. A contractor who needs `desk-api` then decrypts disk keys. Split trees. Split tags.

The other trap is applying `@core` and `@apps` in one command because "it's faster." Core is serial and canary. Apps are tagged.

The boring rule

- `@core` vs `@apps` (or two flakes). No `@all` for kernels.
- `secrets/core/` vs `secrets/apps/`. Host age keys decrypt; humans rekey.
- Staging is a host with the same modules.
- Named `devShells` per team. Need-to-have sops.
- A 10-line `colmena-*.sh` that pins the tag is enough.

Try this

1. Add tags to the capstone hive; `colmena eval` and list `@core` vs `@apps`.
2. Move a dummy secret into `secrets/apps/` and confirm a `--dry-run` on `@core` does not need it.
3. Write `colmena-apps.sh`; run `--dry-run`; confirm it refuses to take `--on gw-01` or wrap so that `--on` is ignored.
4. Add `staging-app` as a sixth host; apply it before `@apps`.

Builder Factory and Restricted Store SSH

Builder Factory and Restricted Store SSH

NixOS/infra's `builders/` is a function `mkNixOS system config` that always imports the same common modules (hardening, ssh, nix, hydra-queue-builder, users), then a per-box instance (CPU, disks). The hydra queue runner may SSH in **only** as `nix-store --serve --write` — not a shell. The boring default is: **the same factory for desk builders, and a forced SSH command so CI cannot `rm -rf`**.

The remote-builders chapter listed `nix.buildMachines`. This lab is the **other** side of that SSH: the box that receives the `.drv`.

Mental model

```
ops-01 / hydra-queue
    ssh build@builder-01
         forced command: nix-store --serve --write


         builder-01  (kvm, big-parallel)
                NAR

         team cache
```

`users.users.build` (uid 2000 in their tree) holds **only** that authorized key, wrapped:

```
command="NIX_SSL_CERT_FILE=…/ca-bundle.crt  /nix/store/…-nix/bin/nix-store --serve --write" ssh
```

`authorizedKeys command=` is OpenSSH forced command. Agent forwarding and `pty` will not get you bash. That is the point.

Common modules stay boring: UTC, `mutableUsers = false`, nftables, `openssh`, nix flakes, node-exporter. Instance files set Disko `by-id` and `system`.

Worked examples

Case 1: `mkHost` factory with common modules

Save as `lib/mk-builder.nix`:

```
# lib/mk-builder.nix
{ nixpkgs, disko }:

system: extraModules:
nixpkgs.lib.nixosSystem {
  inherit system;
  modules = [
    disko.nixosModules.disko
    ../builders/common/system.nix
    ../builders/common/ssh.nix
    ../builders/common/nix.nix
    extraModules
  ];
}
```

Save as `builders/common/system.nix`:

```
# builders/common/system.nix
{ pkgs, ... }:
{
  time.timeZone = "UTC";
  users.mutableUsers = false;
  environment.systemPackages = [ pkgs.git pkgs.jq ];
  system.stateVersion = "26.05";
}
```

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk builders";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
    disko.url = "github:nix-community/disko";
    disko.inputs.nixpkgs.follows = "nixpkgs";
  };

  outputs = { self, nixpkgs, disko }:
    let
      mkBuilder = import ./lib/mk-builder.nix { inherit nixpkgs disko; };
    in
    {
      nixosConfigurations = {
        builder-01 = mkBuilder "x86_64-linux" [ ./builders/instances/builder-01.nix ];
        builder-arm = mkBuilder "aarch64-linux" [ ./builders/instances/builder-arm.nix ];
      };
```

```
    };
}
```

Two archs, one factory. NixOS/infra's comments list cores and RAM next to each instance — do that in a comment, not in a wiki.

Case 2: Forced `nix-store --serve --write`

Save as `builders/common/ssh.nix`:

```
# builders/common/ssh.nix
{ config, lib, pkgs, ... }:

let
  keys = import ../../keys.nix;
  authorizedNixStoreKey = key:
    let
      env = "NIX_SSL_CERT_FILE=${pkgs.cacert}/etc/ssl/certs/ca-bundle.crt";
      nixStore = "${config.nix.package}/bin/nix-store";
    in
    ''command="${env} ${nixStore} --serve --write" ${key}'';
in
{
  services.openssh.enable = true;
  services.openssh.settings.PasswordAuthentication = false;
  services.openssh.settings.KbdInteractiveAuthentication = false;

  users.users.root.openssh.authorizedKeys.keys = keys.ssh.groups.desk-core;

  users.users.build = {
    isNormalUser = true;
    uid = 2000;
    # no extraGroups. no lingering.
    openssh.authorizedKeys.keys =
      map authorizedNixStoreKey keys.ssh.users.hydra-queue;
  };

  nix.settings.trusted-users = [ "root" "build" ];
}
```

`trusted-users` is required for `--serve --write` to import paths. Keep it `root` + `build`, not `@wheel`.

```
nix eval .#nixosConfigurations.builder-01.config.users.users.build.openssh.authorizedKeys.ke
```

Output (shape):

`command="NIX_SSL_CERT_FILE=/nix/store/…`

If the line is a bare `ssh-ed25519`, the force is missing and CI has a shell.

Case 3: Nix daemon knobs the factory always sets

Save as `builders/common/nix.nix`:

```
# builders/common/nix.nix
{
  nix.settings = {
    experimental-features = [ "nix-command" "flakes" ];
    cores = 0;           # one job may use all CPUs; pair with max-jobs
    max-jobs = 8;        # set from nproc / RAM on the instance
    sandbox = true;
    builders-use-substitutes = true;
    extra-substituters = [ "https://cache.nixos.org" ];
    extra-trusted-public-keys = [
      "cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY="
    ];
  };
  nix.gc.automatic = true;
  nix.gc.dates = "weekly";
  nix.gc.options = "--delete-older-than 7d";
  boot.binfmt.emulatedSystems = [ ]; # add aarch64-linux only if you meant qemu
}
```

NixOS/infra sets `cores = 0` in `modules/common.nix`. On a shared builder, `max-jobs` is the throttle; `cores = 0` lets a single Chromium use the box. Do not copy `max-jobs = 96` from their Epyc comment onto a 4-core lab VM.

Case 4: Client `buildMachines` matches the forced user

Save as `modules/use-builder-01.nix` on `ops-01`:

```
# modules/use-builder-01.nix
{
  nix.buildMachines = [{
    hostName = "builder-01.desk.internal";
    sshUser = "build";
    sshKey = "/run/secrets/ops-builder-ssh";
    system = "x86_64-linux";
    protocol = "ssh-ng";
    maxJobs = 8;
    speedFactor = 4;
    supportedFeatures = [ "kvm" "nixos-test" "big-parallel" ];
```

```
  }];
  nix.distributedBuilds = true;
  nix.settings.builders-use-substitutes = true;
}
```

The key in `sshKey` is the **private** half of `keys.ssh.users.hydra-queue`, on sops, mode `0400`. The public half is what Case 2 wrapped in `command=`. If you use alice's laptop key here, alice can `--serve --write` **and** she still has a root shell from `desk-core` — two roles, two keys.

```
nix build .#nixosConfigurations.app-01.config.system.build.toplevel --max-jobs 0
```

`--max-jobs 0` forbids a local compile. The drv must go to `builder-01`. If it builds on `ops-01`, the feature flags or SSH forced command are wrong (`journalctl` on the builder, `ssh -v build@builder-01`).

Case 5: Instance file is disks + identity, not a second OS

Save as `builders/instances/builder-01.nix`:

```
# builders/instances/builder-01.nix
{ lib, ... }:
{
  networking.hostName = "builder-01";
  # Epyc-class in prod; lab: one disk.
  disko.devices.disk.main.device = lib.mkDefault "/dev/vda";
  imports = [ ../disk-layouts/ext4.nix ];

  nix.settings.max-jobs = 8;
  nix.settings.system-features = [ "kvm" "nixos-test" "big-parallel" ];
}
```

Hardware comments belong here ("48C/96T, 256 GB") so the next human does not schedule Chromium on a cloud burstable. The OS is the factory. The instance is numbers.

The trap

The trap is **a builder account with a login shell and `wheel`**. The queue runner then has root. Forced `command=` + `trusted-users = [ "build" ]` is the whole privilege.

The other trap is wrapping `command=` around **root**'s keys so you cannot SSH in to repair. Root stays `desk-core` without a force. `build` is the restricted user.

A third: `boot.binfmt.emulatedSystems = [ "aarch64-linux" ]` on an x86 builder so "we don't need builder-arm." Qemu is not a 96-thread Ampere. Separate `system` in the factory.

The boring rule

- One `mkBuilder` function. Common modules always. Instance = disks + jobs + features.
- `build` user: forced `nix-store --serve --write`, `trusted-users`, no `wheel`.
- Root: `desk-core` keys, no force.
- Client `sshUser = "build"` + sops private key. `--max-jobs 0` to prove it.
- `system-features` advertised must match what you put in `supportedFeatures` on the client.

Try this

1. Eval Case 2; confirm the `build` key starts with `command="`.
2. On a lab VM, add the `build` user; `ssh build@vm` — must not get a shell; `nix store ping --store ssh-ng://build@vm` (or `nix ping-store`) from a client that has the matching key.
3. `nix build nixpkgs#hello --max-jobs 0` from `ops-01` with Case 4; `journalctl -u nix-daemon` on the builder shows the compile (or a substitute).
4. Add `builder-arm` with `system = "aarch64-linux"`; `nix flake show`. Do not binfmt it onto builder-01 for the lab.

Channel Promotion and CI Host Groups

Channel Promotion and CI Host Groups

NixOS/infra's `channels.nix` is a table: channel name → Hydra job, variant (`primary` / `small` / `darwin`), status (`rolling` / `stable` / `unmaintained`). Their `checks/` flake groups host **toplevels** by architecture so CI can `nix-fast-build` one arch without evaluating every Darwin Mac. The boring default is: **the same two ideas at desk scale — a promotion table in git, and `checks` that list host closures by arch — not a wiki "we shipped 26.05."**

You will not run Hydra for nixpkgs. You *will* have a `tested` job that may update a Cachix/Attic alias.

Mental model

```
nix flake check          →  hosts eval + tests
nix build .#tested       →  the closure you are willing to name
cachix push / attic push →  cache
channels.nix status      →  stable | staging | unmaintained
```

NixOS maps `nixos-26.05` to `nixos/release-26.05/tested` on hydra.nixos.org. Desk maps `desk-stable` to `.#nixosConfigurations.app-01.config.system.build.toplevel` **after** `@apps` canary. The table is data. The job is CI.

`25.11` in their file is `status = "unmaintained"`. That is how you tell humans to stop following a pin — not a Slack message.

CI groups (their `flake.ciSystems.ofborg-x86_64-linux = [ core01 build01 … ]`) exist so one runner does not eval aarch64-darwin. List names **explicitly**. Do not `mapAttrs` every `nixosConfigurations` if that forces Mac eval on a Linux runner.

Worked examples

Case 1: A desk channel table

Save as `channels.nix`:

```
# channels.nix
{
  channels = {
    desk-unstable = {
      job = "flake.checks.x86_64-linux.default";
```

```
      variant = "primary";
      status = "rolling";
    };
    desk-26.05 = {
      job = "nixosConfigurations.app-01.toplevel";
      variant = "primary";
      status = "stable";
    };
    desk-26.05-small = {
      job = "nixosConfigurations.app-01.toplevel";
      variant = "small";
      status = "stable";
    };
  };
}
```

```
nix eval --file channels.nix 'channels.desk-26.05.status'
```

```
"stable"
```

When 26.11 exists, set `desk-26.05.status = "deprecated"` in the same PR that bumps the flake input. The table is the announcement.

Case 2: `tested` as an explicit package

Save as `flake.nix` fragment:

```
# flake.nix fragment
{
  packages.x86_64-linux.tested = pkgs.releaseTools.aggregate {
    name = "desk-tested";
    constituents = [
      self.nixosConfigurations.app-01.config.system.build.toplevel
      self.nixosConfigurations.app-02.config.system.build.toplevel
      self.checks.x86_64-linux.fmt
    ];
  };
}
```

If `releaseTools.aggregate` feels heavy, a simpler pin:

```
# flake.nix fragment
{
  packages.x86_64-linux.tested =
    self.nixosConfigurations.app-01.config.system.build.toplevel;
}
```

CI:

```
nix build .#tested
cachix push desk-cache ./result
```

Humans `nixos-rebuild switch --flake .#app-01` only after `.tested` is green. That is a channel. Hydra's UI is optional.

Case 3: Group host toplevels by arch (no surprise Darwin eval)

Save as `ci-hosts.nix`:

```
# ci-hosts.nix
{ self, lib }:

let
  nixos = names:
    lib.genAttrs names (n: self.nixosConfigurations.${n}.config.system.build.toplevel);
in
{
  desk-x86_64-linux = nixos [ "gw-01" "app-01" "app-02" "db-01" "ops-01" ];
  desk-aarch64-linux = nixos [ "builder-arm" ];
}
```

In `outputs`:

```
# flake.nix fragment
{
  checks.x86_64-linux = (import ./ci-hosts.nix { inherit self; lib = nixpkgs.lib; }).desk-x8
}
```

```
nix flake check
```

A Linux GHA runner evals five x86_64 toplevels. It does **not** eval `builder-arm` unless you add a matrix job for `aarch64-linux`. NixOS/infra's comment: list hosts explicitly so CI does not force every configuration just to learn `system`.

Case 4: OpenTofu envelope, NixOS OS — two shells

Their `terraform/flake-module.nix` is a `mkShellNoCC` with `opentofu.withPlugins` (AWS, Fastly, Netlify). DNS and IAM live there. Machines live in Colmena.

```
# flake.nix fragment
{
  devShells.x86_64-linux.tf = pkgs.mkShell {
    packages = [
```

```
      pkgs.awscli2
      (pkgs.opentofu.withPlugins (p: [ p.aws p.random ]))
    ];
  };
}
```

```
# dns.tf
resource "aws_route53_record" "api" {
  zone_id = "ZXXXXXXXX"
  name    = "api.desk.internal"
  type    = "A"
  ttl     = 60
  records = ["203.0.113.10"]
}
```

The IP is the Colmena `targetHost`. Terraform does not install NixOS (Disko / nixos-anywhere does). Two tools, two state files, one README that says which.

```
nix develop .#tf --command tofu fmt -check
```

Case 5: Promote, then deprecate

```
# channels.nix
{
  channels = {
    desk-26.05 = {
      job = "nixosConfigurations.app-01.toplevel";
      variant = "primary";
      status = "deprecated";
    };
    desk-26.11 = {
      job = "nixosConfigurations.app-01.toplevel";
      variant = "primary";
      status = "stable";
    };
    desk-25.11 = {
      job = "nixosConfigurations.app-01.toplevel";
      variant = "primary";
      status = "unmaintained";
    };
  };
}
```

Unmaintained means: delete the GHA job, leave the table row so `git log` explains why `flake.lock` must not point there. NixOS/infra keeps 25.11 rows as `unmaintained` instead of pretending the files never existed.

```
nix eval --file channels.nix 'channels' --apply builtins.attrNames
```

The trap

The trap is **`cachix push` from every PR** and calling that a channel. A channel is a **named, reviewed** closure (`tested`) plus a status in git. PR pushes belong behind `skipPush`.

The other trap is `checks = mapAttrs every nixosConfigurations` on a laptop flake that also has `aarch64-darwin`. The Linux runner then tries to eval Darwin and fails (or downloads a world). List names.

A third: Terraform `remote-exec` of `nixos-rebuild` so "we only have one tool." That is how state and generations diverge. OpenTofu for DNS/IAM; Colmena for the OS.

The boring rule

- `channels.nix` is a table: job, variant, status. Status changes in git.
- `.tested` (or aggregate) is what CI pushes to the team cache.
- `checks` list host toplevels **by arch**, names explicit.
- OpenTofu `withPlugins` for the cloud envelope. Colmena for NixOS.
- Deprecated deleted. Unmaintained still in CI.

Try this

1. Add `channels.nix` with `desk-26.05.status = "stable"`; `nix eval` it.
2. `nix build .#tested` where `tested` is `app-01`'s toplevel.
3. Write `ci-hosts.nix` with two names; `nix flake check`; add a fake Darwin host name and confirm you did **not** put it on the Linux checks attr.
4. `nix develop .#tf --command which tofu` — store path, plugins attached.
5. In a branch, set `desk-26.05.status = "deprecated"` and read the PR diff as if you were on-call.

Production Capstones

Production Architecture and Threat Model

Production Architecture and Threat Model

Toy VMs taught the pieces. The capstone is one **desk fleet** on NixOS 26.05: five roles, WireGuard, ephemeral workers, sops-nix, Colmena. The boring default is: **write the threat model first, then the flake — not a mesh of tools looking for a problem**.

Mental model

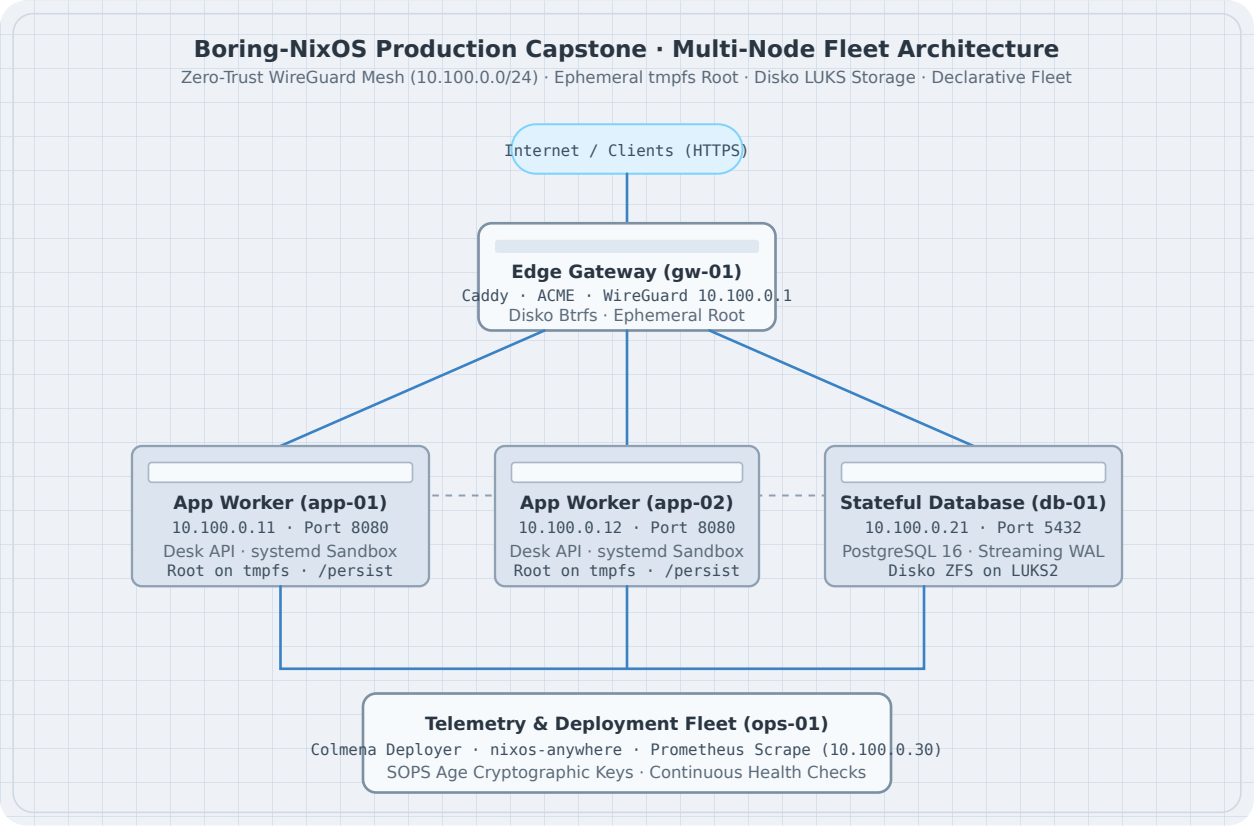


Figure 1: Production Capstone Fleet Architecture

Host	Role	Persist
`gw-01`	Caddy, ACME, nftables; WireGuard `10.100.0.1`	certs, Caddy state
`app-01`, `app-02`	desk-api; tmpfs root; `DynamicUser`	nothing (ephemeral)

Host	Role	Persist
db-01	PostgreSQL; Disko LUKS + Btrfs	/persist
ops-01	Colmena, Prometheus, sops key broker	hive checkout, keys

Invariants:

1. No secret bytes in git or /nix/store.
2. Store is read-only. Payloads in /etc die on reboot (workers).
3. WireGuard for all east-west. No DB port on the public NIC.
4. flake.lock on **nixos-26.05**; rollback is a generation, not a restore CD.
5. Backups of /persist (restic), not of /nix/store.

Public surface: TCP 443 on gw-01 (and UDP 51820 on every peer). SSH from the office VPN or the ops-01 jump only.

Worked examples

Case 1: Inventory in the flake

Save as flake.nix:

```
# flake.nix
{
  description = "Desk production fleet";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      mk = name: modules: nixpkgs.lib.nixosSystem {
        system = "x86_64-linux";
        modules = [
          { networking.hostName = name; system.stateVersion = "26.05"; }
        ] ++ modules;
      };
    in
    {
      nixosConfigurations = {
        gw-01  = mk "gw-01"  [ ./hosts/gw-01.nix ];
        app-01 = mk "app-01" [ ./hosts/app-01.nix ];
        app-02 = mk "app-02" [ ./hosts/app-02.nix ];
        db-01  = mk "db-01"  [ ./hosts/db-01.nix ];
        ops-01 = mk "ops-01" [ ./hosts/ops-01.nix ];
```

```
      };
    };
}
```

Five names. Five `system.stateVersion = "26.05"` because they are born here. Do not copy a `stateVersion` from a laptop that was installed on 24.11.

```
nix flake show
```

You should see five `nixosConfigurations`.

Save as `hosts/gw-01.nix` (shape — later chapters fill Caddy and WireGuard):

```
# hosts/gw-01.nix
{
  imports = [ ../modules/base-system.nix ../modules/wireguard-mesh.nix ];
  networking.firewall.allowedTCPPorts = [ 80 443 ];
  networking.firewall.allowedUDPPorts = [ 51820 ];
  # services.caddy … in the hardened-network chapter
}
```

If a port is not in `docs/threats.md`, it is not in this module.

Case 2: Trust boundary in git

Save as `docs/threats.md`:

```
# docs/threats.md
Public NIC: 443/tcp on gw-01, 51820/udp on every peer.
SSH: office VPN or ops-01 jump. No 22/tcp from 0.0.0.0/0.
Postgres: 5432/tcp on 10.100.0.21, wg0 only.
desk-api: 8080 on wg0; Caddy reverse-proxies from gw-01.
Attacker with a stolen app-01 disk: tmpfs, no secrets.
Attacker with a stolen db-01 disk: LUKS; restic passphrase is not on the disk.
```

If a port is not in this file, the firewall and the cloud SG must not open it.

```
git grep -n 'allowedTCPPorts\|allowedUDPPorts' -- '*.nix'
```

Every port in that grep lands in `docs/threats.md` or it is a bug. `0.0.0.0/0` on 22 is a bug even if the module lists 22.

Case 3: Secret inventory

Secret	Mechanism	Lives
WireGuard private keys	sops-nix → `/run/secrets`	encrypted in git

Secret	Mechanism	Lives
Postgres password	sops-nix `EnvironmentFile`	encrypted in git
ACME account / certs	Caddy module + `/persist`	`gw-01` persist
Colmena SSH key	`ops-01` persist	**not** the flake
restic password	sops-nix on `db-01` / `ops-01`	encrypted in git

A secret that is only in a password manager is a secret that will not survive the next hire.

Case 4: Failure drill list

Failure	Response
Lose `app-01`	traffic already on `app-02`; nixos-anywhere a replacement; Colmena
Lose `db-01`	restic restore onto a new Disko disk; Colmena; check `pg_isready` on `wg0`
Bad generation on `app-01`	previous generation / Colmena rollback; revert git
Lose `gw-01`	ACME + persist restore; DNS TTL is why you kept it short
Stolen laptop with ops checkout	sops age key is **not** on the laptop; rotate the Colmena SSH key

Write the restic repository URL in `docs/threats.md`. Not the password.

Case 5: What this capstone is not

Not multi-region Kubernetes. Not Vault HA. Not a service mesh. Those are later, optional, and they each add a new trust boundary. If the five hosts rebuild from git + restic, you passed.

Order of the remaining capstone chapters: Disko + impermanence, WireGuard, application stack, Colmena + restic. Do not start with Istio.

The trap

The trap is **starting with three clouds and a mesh** before SSH keys persist across reboot. Finish Disko, impermanence, sops, WireGuard, Colmena, restic. In that order. A threat model that lists “Zero Trust” and does not list `/persist` is a slide, not an ops doc.

The boring rule

- 26.05 lock. Five hosts. One mesh. One secret system (sops).
- Public surface is 443 (+ 22 only from a tight SG / VPN).
- `/persist` is the backup. Store is the cache.
- Rollback is a generation.
- Write the threat model in git next to the flake.

Try this

1. Copy the host table into `docs/threats.md`. Fill module file names.
2. List every port you think is open; compare to firewall + cloud SG in later chapters.
3. Name the restic repository in `docs/threats.md`.
4. Confirm `stateVersion` will be `"26.05"` on new metal — not copied from an old laptop.

Disko Storage and Ephemeral Base System

Disko Storage and Ephemeral Base System

Every node in our production fleet must deploy onto clean physical or cloud storage without manual partitioning tools. In this chapter, we create the unified **Disko** storage specification and configure the **ephemeral root filesystem with Impermanence**.

Mental model

1. **Unified disk module (`disk-layout.nix`):** Disko formats the physical NVMe or virtual disk into an EFI system partition (`/boot`) and a LUKS2 encrypted partition. Metal uses `/dev/disk/by-id/…`; `lib.mkDefault "/dev/nvme0n1"` is a lab fallback, not a fleet device.
2. **Btrfs subvolumes on LUKS:** Inside the decrypted container (`/dev/mapper/<luks.name>`), Btrfs subvolumes are created for `/nix` (the store), `/persist` (stateful data), and `/swap`. The LUKS **name** is the Stage-1 mapper name. Disko `name = "crypted"` must match `boot.initrd.luks.devices.crypted` (Disko usually emits this). A mismatch and systemd Stage 1 waits on `/sysroot` forever.
3. **RAM root (`tmpfs`):** The root filesystem (`/`) is mounted in RAM (`mode=755`, size you can afford). Unmounted state is discarded on power-down.
4. **Impermanence persistence:** System state required across reboots (SSH host keys, machine-id, WireGuard identity, sops keys) is bound into `/persist`. `/persist` is `neededForBoot = true` so Stage 1 mounts it before activation tries to link `machine-id`.

Worked examples

Case 1: The Production Disko Module

Save as `modules/disk-layout.nix`:

```
# modules/disk-layout.nix
{ lib, ... }:

{
  disko.devices = {
    disk.main = {
      type = "disk";
      # Lab fallback. Per-host modules override with /dev/disk/by-id/nvme-….
      device = lib.mkDefault "/dev/nvme0n1";
      content = {
        type = "gpt";
```

```
      partitions = {
        ESP = {
          priority = 1;
          name = "ESP";
          start = "1M";
          end = "1024M";
          type = "EF00";
          content = {
            type = "filesystem";
            format = "vfat";
            mountpoint = "/boot";
            mountOptions = [ "umask=0077" ];
          };
        };
        luks = {
          size = "100%";
          content = {
            type = "luks";
            name = "crypted";
            settings.allowDiscards = true;
            content = {
              type = "btrfs";
              extraArgs = [ "-f" ];
              subvolumes = {
                "/nix" = {
                  mountpoint = "/nix";
                  mountOptions = [ "compress=zstd" "noatime" ];
                };
                "/persist" = {
                  mountpoint = "/persist";
                  mountOptions = [ "compress=zstd" "noatime" ];
                };
                "/swap" = {
                  mountpoint = "/swap";
                  swap.swapfile.size = "8G";
                };
              };
            };
          };
        };
      };
    };
  };

  nodev = {
    "/" = {
```

```
        fsType = "tmpfs";
        mountOptions = [ "defaults" "size=4G" "mode=755" ];
      };
    };
  };

  # Disko emits fileSystems."/persist" from the subvolume mountpoint.
  # Stage 1 must mount it before activation links machine-id / SSH keys.
  fileSystems."/persist".neededForBoot = true;
}
```

`ls -l /dev/disk/by-id/` on the box (or rescue). Paste `nvme-eui.*` or `ata-*`, not a `-partN` suffix and not a `wwn-` alias you did not verify. Override per host:

```
# hosts/app-01/disk.nix
{
  imports = [ ../../modules/disk-layout.nix ];
  disko.devices.disk.main.device =
    "/dev/disk/by-id/nvme-VENDOR_MODEL_SERIAL";
}
```

LUKS `name = "crypted"` is `/dev/mapper/crypted` in Stage 1. Do not rename it in one file and leave `boot.initrd.luks.devices` on the old name.

Case 2: The Impermanence Persistent State Contract

Save as `modules/base-system.nix`:

```
# modules/base-system.nix
{ inputs, pkgs, ... }:

{
  imports = [
    inputs.impermanence.nixosModules.impermanence
    ./disk-layout.nix
  ];

  boot.loader.systemd-boot.enable = true;
  boot.loader.efi.canTouchEfiVariables = true;
  boot.loader.systemd-boot.configurationLimit = 8;

  fileSystems."/persist".neededForBoot = true;

  # Persistent state mapping - if it is not listed, it dies at reboot.
  environment.persistence."/persist" = {
    hideMounts = true;
```

```
    directories = [
      "/var/log"
      "/var/lib/nixos"
      "/var/lib/systemd/coredump"
      "/var/lib/systemd/timers"
      "/var/lib/sops-nix"
    ];
    files = [
      "/etc/machine-id"
      "/etc/ssh/ssh_host_ed25519_key"
      "/etc/ssh/ssh_host_ed25519_key.pub"
    ];
  };

  # SSH configuration
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      KbdInteractiveAuthentication = false;
      PermitRootLogin = "prohibit-password";
    };
  };

  system.stateVersion = "26.05";
}
```

Validating the base configuration:

```
nix eval .#nixosConfigurations.app-01.config.disko.devices.nodev."/".fsType
```

```
"tmpfs"
```

A new SSH host key every boot means sops cannot decrypt. Persist the ed25519 host key **and** `/var/lib/sops-nix`. WireGuard private keys belong in sops at `/run/secrets`, not as leftover files on tmpfs.

Case 3: `neededForBoot` is a Stage-1 mount, not a comment

```
nix eval .#nixosConfigurations.app-01.config.fileSystems.\"/persist\".neededForBoot
nix eval .#nixosConfigurations.app-01.config.boot.initrd.luks.devices.crypted.device
```

```
true
"/dev/disk/by-partlabel/disk-main-luks"
```

(Exact LUKS device path depends on Disko's GPT labels.) If `neededForBoot` is `false` or missing, activation looks for `/persist/etc/machine-id` before the subvolume is mounted. Emergency shell. Set it in the **same** module that owns Disko so a host cannot import the layout and forget the flag.

Case 4: Wrong disk, wrong mapper

```
ls -l /dev/disk/by-id/ | rg 'nvme|ata' | rg -v part
lsblk -o NAME,SIZE,MODEL,SERIAL,MOUNTPOINT
```

`/dev/nvme0n1` on rescue is often the installer USB. `by-id` includes the serial. Two hosts sharing `disk-layout.nix` without a per-host `device` override will format whichever disk is `nvme0n1` on that firmware.

Rename LUKS in Disko (`name = "cryptroot"`) only if **every** reference follows: `fileSystems."/nix".device`, Stage-1 `systemd-cryptsetup@cryptroot`, and any `crypttab` extra. The Stage-1 chapter uses `/sysroot` inside the initrd — a oneshot that writes `/persist` in Stage 1 is writing the initrd overlay unless `/persist` is already mounted under `/sysroot/persist`.

Case 5: Format is once; mounts are every boot

```
# lab VM only - destroys the disk
nix run github:nix-community/disko -- --mode destroy,format,mount ./modules/disk-layout.nix
mount | rg 'tmpfs on / | /nix | /persist | /boot'
```

`--mode mount` is the non-destructive path (installer / recovery). `--mode destroy,format,mount` is `nixos-anywhere`'s Disko phase. The running system never re-runs format: NixOS just mounts what Disko already described in `fileSystems.*`.

The trap

The trap is **failing to mark `/persist` as `neededForBoot = true`**. During early boot, the initramfs must mount `/persist` before systemd starts generating symlinks for machine-id and SSH keys. Always ensure `/persist` is mounted before root pivot.

The other trap is **one `device = "/dev/nvme0n1"` for the fleet**. Disk names move. Per-host `by-id`. The third is a LUKS mapper rename that Stage 1 still calls `crypted`.

The boring rule

- Model all partition tables and filesystems in Disko. Same file for `nixos-anywhere` and for `fileSystems`.
- `by-id` on metal. `mkDefault` + per-host override. Never a shared `nvme0n1`.
- LUKS `name` is the Stage-1 mapper. Do not drift.
- Keep `/` on tmpfs. `neededForBoot = true` on `/persist`.

- Explicitly list persistent files and directories (host keys, sops, logs, nixos uid map).

Try this

1. `nix eval fileSystems."/persist".neededForBoot` on `app-01`. Must be `true`.
2. `nix eval` the LUKS mapper name and confirm it is `crypted` (or the name you chose everywhere).
3. `ls -l /dev/disk/by-id` on a lab box; write the id you would put in `hosts/app-01/disk.nix`. Do not format that disk.
4. Run `nix run github:nix-community/disko -- --mode mount ./modules/disk-layout.nix` only in a test VM. `mount | grep tmpfs`.

Hardened Networking and WireGuard Mesh

Hardened Networking and WireGuard Mesh

Public IPs should not see Postgres. The boring default is: **WireGuard mesh `10.100.0.0/24`, keys via sops on `/run/secrets`, nftables so `5432` exists only on `wg0`.**

Mental model

Node	wg0
gw-01	10.100.0.1
app-01 / app-02	10.100.0.11 / .12
db-01	10.100.0.21
ops-01	10.100.0.30

Public: UDP **51820** on every peer; TCP **80/443** only on `gw-01`. Everything else: `wg0` or drop.

Private keys never enter `/nix/store`. `privateKeyFile` points at sops.

`persistentKeepalive = 25` keeps NAT mappings alive toward cloud UDP.

Worked examples

Case 1: Shared mesh module

Save as `modules/wireguard-mesh.nix`:

```
# modules/wireguard-mesh.nix
{ config, lib, ... }:

{
  networking.firewall.allowedUDPPorts = [ 51820 ];

  networking.wireguard.interfaces.wg0 = {
    ips = [ config.cluster.nodeIp ];
    listenPort = 51820;
    privateKeyFile = config.sops.secrets."wireguard/private".path;
    peers = [
      {
        publicKey = "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
        allowedIPs = [ "10.100.0.1/32" ];
```

```
        endpoint = "203.0.113.10:51820";
        persistentKeepalive = 25;
      }
      {
        publicKey = "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB=";
        allowedIPs = [ "10.100.0.11/32" ];
        endpoint = "203.0.113.11:51820";
        persistentKeepalive = 25;
      }
      {
        publicKey = "CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC=";
        allowedIPs = [ "10.100.0.21/32" ];
        endpoint = "203.0.113.21:51820";
        persistentKeepalive = 25;
      }
    ];
  };
}
```

Replace `AAA…` with real **public** keys. Private keys stay in sops.

Each peer must include **itself** in `ips` and every **other** node in `peers` with `/32` `allowedIPs`. A missing self IP is a silent blackhole. `endpoint` is the **public** UDP address; omit `endpoint` only for peers that only receive (roaming laptop) and then they need `persistentKeepalive` toward a fixed peer.

```
# cluster option (tiny)
{ lib, ... }:
{
  options.cluster.nodeIp = lib.mkOption {
    type = lib.types.str;
    description = "This host's wg0 CIDR, e.g. 10.100.0.11/24";
  };
}
```

Case 2: Database host — Postgres only on `wg0`

Save as `hosts/db-01.nix`:

```
# hosts/db-01.nix
{
  imports = [
    ../modules/base-system.nix
    ../modules/wireguard-mesh.nix
  ];
```

```
  cluster.nodeIp = "10.100.0.21/24";

  networking.firewall = {
    enable = true;
    interfaces.wg0.allowedTCPPorts = [ 5432 ];
    interfaces.eth0.allowedTCPPorts = [ ];
  };
}
```

```
nix eval .#nixosConfigurations.db-01.config.networking.firewall.interfaces.wg0.allowedTCPPor
```

Output:

```
[ 5432 ]
```

Case 3: Gateway — HTTP public, API only over mesh

```
# hosts/gw-01.nix fragment
{
  cluster.nodeIp = "10.100.0.1/24";
  networking.firewall.allowedTCPPorts = [ 80 443 ];
  networking.firewall.interfaces.wg0.allowedTCPPorts = [ ];
}
```

Caddy binds `0.0.0.0:443`. It **proxies to** `10.100.0.11:8080`, which is not open on the public NIC of `app-01`.

Case 4: App node does not open 8080 publicly

```
{
  cluster.nodeIp = "10.100.0.11/24";
  networking.firewall.interfaces.wg0.allowedTCPPorts = [ 8080 ];
}
```

Health checks from Caddy use `10.100.0.11`. A scanner on the cloud IP should not see 8080.

Case 5: Prove the path

On `ops-01` (once deployed):

```
sudo wg show
ping -c 1 10.100.0.21
nc -zv 10.100.0.21 5432
```

From a laptop **not** on the mesh, `nc` to the public IP port 5432 should fail. That is the test, not a green `wg show` alone.

```
sudo wg show wg0 dump
ip route get 10.100.0.21
```

Handshake age of minutes is NAT; `persistentKeepalive = 25`. Handshake never: wrong pubkey, UDP 51820 blocked, or `allowedIPs` does not contain that `/32`.

The trap

The trap is **`privateKey = "…" in Nix`**. World-readable store. `privateKeyFile` + sops. Public keys in git are fine.

The other trap is `allowedIPs = [ "0.0.0.0/0" ]` turning the mesh into a default route. Use `/32` per peer unless you **mean** to route the internet through `gw-01`.

The boring rule

- Mesh IPs in git. Private keys in sops → `/run/secrets`.
- `5432` / `8080` on `wg0` only.
- Public: 80/443 on gateway, UDP 51820 on peers.
- `/32` `allowedIPs` unless you are building a router.
- `persistentKeepalive = 25` behind NAT.

Try this

1. Eval Case 2; confirm `eth0` TCP list is empty.
2. Generate a keypair with `wg genkey | tee priv | wg pubkey`; put **only** the pubkey in the module.
3. Two VMs, one peer each; `ping` over `wg0`.
4. `ss -lntp` on `db-01` — Postgres on `10.100.0.21:5432`, not `0.0.0.0`.

Distributed Application Stack and Secrets

Distributed Application Stack and Secrets

Storage and the mesh exist. Now the workload: **Caddy + ACME on `gw-01`, sandboxed `desk-api` on the apps, PostgreSQL on `db-01`, sops-nix for the password**. The boring default is: **TLS at the edge, `EnvironmentFile` for secrets, `DynamicUser` + `ProtectSystem=strict` on the API**.

Mental model

```
Internet --443--> Caddy (gw-01)
                      reverse_proxy 10.100.0.11:8080, .12:8080

                desk-api  --wg0-->  Postgres 10.100.0.21

                      /run/secrets/db_password  (sops, tmpfs)
```

Caddy talks HTTP to the apps **on the mesh**. Apps do not terminate TLS. Postgres does not listen on the public NIC (previous chapter).

`MemoryDenyWriteExecute` is fine for `CGO_ENABLED=0` Go. Drop it if you add a JIT.

Worked examples

Case 1: Caddy on the gateway

Save as `hosts/gw-01.nix`:

```
# hosts/gw-01.nix
{
  services.caddy = {
    enable = true;
    email = "ops@desk.internal";
    # Lab: Let's Encrypt staging so you do not burn production rate limits.
    # Production: acmeCA = null; (26.05 default) so Caddy can fall back issuers.
    acmeCA = "https://acme-staging-v02.api.letsencrypt.org/directory";
    virtualHosts."api.desk.internal".extraConfig = ''
      reverse_proxy 10.100.0.11:8080 10.100.0.12:8080 {
        lb_policy round_robin
        health_uri /healthz
```

```
        health_interval 5s
        health_timeout 2s
      }
    '';
  };

  networking.firewall.allowedTCPPorts = [ 80 443 ];

  # Ephemeral root: ACME private keys live here. Miss this and every reboot
  # is a new account + a new cert (and a rate-limit event).
  environment.persistence."/persist".directories = [ "/var/lib/caddy" ];
}
```

ACME HTTP-01 needs 80/443 reachable on `gw-01`. DNS `api.desk.internal` → that public IP. `reverse_proxy` to **mesh** IPs (`10.100.0.11/12`), not `127.0.0.1` (the API is not on the gateway).

Caddy's ACME is **not** `security.acme`. Do not enable both for the same name unless you set `virtualHosts.<name>.useACMEHost` and point Caddy at lego's files. The boring path is Caddy's built-in ACME + persist `dataDir` (`/var/lib/caddy`). DNS-01 (Cloudflare etc.) belongs in `globalConfig` with `{$API_TOKEN}` and `services.caddy.environmentFile` from sops — never the token in the Caddyfile store path.

After the lab cert looks right in `curl -vI https://api.desk.internal`, set `acmeCA = null;` and switch. A staging cert in a browser is "not trusted"; that is success for the lab.

Case 2: Sandboxed API

Save as `modules/desk-service.nix`:

```
# modules/desk-service.nix
{ config, pkgs, lib, ... }:

let
  appPkg = pkgs.buildGoModule {
    pname = "desk-api";
    version = "2.0.0";
    src = lib.cleanSource ./src;
    vendorHash = "sha256-AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=";
    env.CGO_ENABLED = "0";
  };
in
{
  sops.secrets.db_password = { };

  systemd.services.desk-api = {
    description = "Desk API";
    after = [ "network-online.target" ];
```

```
    wants = [ "network-online.target" ];
    wantedBy = [ "multi-user.target" ];
    environment = {
      PORT = "8080";
      DB_HOST = "10.100.0.21";
      DB_USER = "desk_user";
      DB_NAME = "desk_production";
    };
    serviceConfig = {
      ExecStart = "${appPkg}/bin/desk-api";
      DynamicUser = true;
      # dotenv (DB_PASSWORD=…). Alternative: LoadCredential=db:/run/secrets/…
      # and read CREDENTIALS_DIRECTORY/db in the app - never argv.
      EnvironmentFile = config.sops.secrets.db_password.path;
      ProtectSystem = "strict";
      ProtectHome = true;
      PrivateTmp = true;
      PrivateDevices = true;
      NoNewPrivileges = true;
      RestrictRealtime = true;
      RestrictAddressFamilies = [ "AF_INET" "AF_INET6" "AF_UNIX" ];
      MemoryDenyWriteExecute = true;
    };
  };
}
```

`db_password` file is dotenv (`DB_PASSWORD=…`), not a CLI flag. For `DynamicUser`, leave the secret **root-owned** `0400` and readable via `EnvironmentFile` / `LoadCredential` — do not `owner = "desk-api"` on a user that does not exist at decrypt time.

Case 3: Postgres on `db-01`

```
# hosts/db-01.nix fragment
{ config, pkgs, ... }:

{
  services.postgresql = {
    enable = true;
    package = pkgs.postgresql_16;
    enableTCPIP = true;
    settings.listen_addresses = "10.100.0.21";
    authentication = pkgs.lib.mkForce ''
      local all all peer
      host  desk_production desk_user 10.100.0.11/32 scram-sha-256
      host  desk_production desk_user 10.100.0.12/32 scram-sha-256
    '';
```

```
    ensureDatabases = [ "desk_production" ];
    ensureUsers = [{
      name = "desk_user";
      ensureDBOwnership = true;
    }];
  };

  # Password is not a Nix string. A oneshot reads sops at runtime (peer as postgres).
  systemd.services.desk-pg-password = {
    after = [ "postgresql.service" ];
    wants = [ "postgresql.service" ];
    wantedBy = [ "multi-user.target" ];
    serviceConfig = {
      Type = "oneshot";
      User = "postgres";
      PrivateTmp = true;
      LoadCredential = "dbpw:${config.sops.secrets.db_password.path}";
      ExecStart = pkgs.writeShellScript "desk-pg-password" ''
        set -euo pipefail
        pw=$(cat "$CREDENTIALS_DIRECTORY/dbpw")
        umask 077
        # Dollar-quote so a password with ' does not break SQL. Not argv.
        printf "ALTER USER desk_user PASSWORD \$\$%s\$\$;\n" "$pw" > /tmp/alter.sql
        ${config.services.postgresql.package}/bin/psql -v ON_ERROR_STOP=1 -f /tmp/alter.sql
        rm -f /tmp/alter.sql
      '';
    };
  };

  # Do not open 5432 on the public NIC. Mesh only.
  networking.firewall.interfaces.wg0.allowedTCPPorts = [ 5432 ];

  environment.persistence."/persist".directories = [ "/var/lib/postgresql" ];
}
```

Do **not** `pkgs.writeText` an `initialScript` that interpolates the password (or even a sops placeholder). `initialScript` is a store path; placeholders only expand in **sops templates**, not in `writeText`. `LoadCredential` + `psql` as `postgres` (peer on the local socket) keeps the bytes out of the NAR. `journalctl` of that oneshot must not use `set -x`.

`listen_addresses = "10.100.0.21"` is the mesh IP of `db-01`, not `*`. Not `postgres` trust on `0.0.0.0/0`.

`ensureUsers` does **not** set a password. `scram-sha-256` without `ALTER USER` is a login that never succeeds. Data dir is `/var/lib/postgresql` — persist it; the store does not hold tables. Dump **before** restic (backups chapter): a running cluster's files without `pg_dumpall` are crash-consistent maybe.

Case 4: Eval the unit path

```
nix eval .#nixosConfigurations.app-01.config.systemd.services.desk-api.serviceConfig.ExecSta
```

Output (shape):

`"/nix/store/…-desk-api-2.0.0/bin/desk-api"`

No `--password=` in that string.

Case 5: Security score

On a deployed app node:

```
systemd-analyze security desk-api --no-pager | tail
curl -fsS http://127.0.0.1:8080/healthz
```

`curl` from the **node** (localhost) or via Caddy. From the public internet, `/healthz` on port 8080 should not answer — only 443 on `gw-01`.

The trap

The trap is **`--password=$DB_PASSWORD` on `ExecStart`**. `ps` shows it. `EnvironmentFile` from sops.

The other trap is `DynamicUser` plus a secret `owner = "desk-api"` that does not exist at decrypt time. For DynamicUser, leave the secret root-owned `0400` and `LoadCredential=` / group, or use a **named** user instead of DynamicUser when the secret must be that uid.

A third: **production Let's Encrypt on the first lab boot** (`acmeCA` left at the public CA while DNS still points at a laptop). Staging first. A fourth: not persisting `/var/lib/caddy` on tmpfs root — new cert every reboot, then a rate limit. A fifth: `listen_addresses = "*"` on postgres.

The boring rule

- TLS only on Caddy. Staging `acmeCA` in lab; `null` in prod. Persist `/var/lib/caddy`.
- Apps HTTP on `wg0`. `buildGoModule` + `CGO_ENABLED=0` + sandbox.
- DB: `listen_addresses` = mesh IP, `pg_hba` `/32`s, 5432 on `wg0` only. Persist the data dir. Dump before restic.
- Secrets: sops file / `LoadCredential`, not argv. `ensureUsers` is not a password.
- Health checks from Caddy over WireGuard.

Try this

1. `systemd-analyze security desk-api` on a lab VM with Case 2.
2. `grep -R password /nix/store/…-desk-api*` should miss the production password (rotate if it hits).
3. From a third VM not on the mesh, `curl` the public IP `:8080` — expect fail; `:443` — expect Caddy.
4. Fail a healthz on `app-01`; Caddy should still serve `app-02`.
5. `nix eval .#nixosConfigurations.gw-01.config.services.caddy.acmeCA` in the lab — staging URL. After cutover, `null`.
6. `ss -lntp | rg 5432` on `db-01` — only `10.100.0.21`.

Automated Fleet Rollout and Disaster Recovery

Automated Fleet Rollout and Disaster Recovery

A production architecture is only as dependable as its playbook. The boring default is: **Colmena canary (`--on app-01`), health probe, then tagged apply; rebuild a dead box with `nixos-anywhere` from the same 26.05 flake — not a restore CD**.

Mental model

The flake is the cluster. Colmena pushes closures. Canary is one host, then a tag. Disaster recovery is Disko + `nixos-anywhere` + restic of `/persist` (not of `/nix/store`).

```
git  →  colmena apply --on app-01  →  curl /healthz
     →  colmena apply --on @app
     →  serial db-01 after restic snapshot
```

Public IPs in the hive are the SSH targets. East-west traffic stays on `wg0` (`10.100.0.0/24`). Rollback is the previous generation, then a git revert so the hive matches metal.

Worked examples

Case 1: Hive for the five-host fleet

Save as `flake.nix`:

```
# flake.nix
{
  description = "Desk production fleet";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    colmena = {
      meta.nixpkgs = import nixpkgs { system = "x86_64-linux"; };

      defaults = { ... }: {
        system.stateVersion = "26.05";
        deployment.targetUser = "root";
      };
```

```
      gw-01 = {
        deployment.targetHost = "203.0.113.10";
        deployment.tags = [ "edge" ];
        imports = [ ./hosts/gw-01.nix ];
      };
      app-01 = {
        deployment.targetHost = "203.0.113.11";
        deployment.tags = [ "app" ];
        imports = [ ./hosts/app-01.nix ];
      };
      app-02 = {
        deployment.targetHost = "203.0.113.12";
        deployment.tags = [ "app" ];
        imports = [ ./hosts/app-02.nix ];
      };
      db-01 = {
        deployment.targetHost = "203.0.113.21";
        deployment.tags = [ "db" ];
        imports = [ ./hosts/db-01.nix ];
      };
      ops-01 = {
        deployment.targetHost = "203.0.113.30";
        deployment.tags = [ "ops" ];
        imports = [ ./hosts/ops-01.nix ];
      };
    };
  };
}
```

```
nix shell nixpkgs#colmena --command colmena eval
```

Five nodes. One 26.05 lock. `follows` on disko/sops if those inputs exist. If `colmena eval` says the flake attr moved, the Colmena pin wants `colmenaHive` + `colmena.lib.makeHive` — same hosts, different wrapper (see the Colmena chapter). `deployment.allowLocalDeployment = true` on `ops-01` if that box applies itself.

Case 2: Canary, probe, then the tag

```
colmena apply --dry-run --on app-01
colmena apply --on app-01
curl -sf https://api.desk.internal/healthz
colmena apply --on @app
```

```
[app-01] Built /nix/store/7q1a…-nixos-system-app-01-26.05
[app-01] Activation successful
```

```
HTTP/2 200
```

Non-200: do not apply `@app`. Roll back `app-01` (`nixos-rebuild switch --rollback` on the host, or Colmena's rollback flag), revert git.

Never `--on @all` for a kernel bump. `db-01` is serial, after `restic snapshots` shows a fresh copy.

Case 3: Zero-touch replace of `app-02`

Boot replacement metal into rescue (or a NixOS installer ISO). Disko's `passwordFile` is a path **on the installer** after kexec. Copy the LUKS key there, generate hardware config into git, and do **not** reboot stateful boxes until `/persist` is restored.

From `ops-01`:

```
# LUKS: remote dest (Disko passwordFile) then local source
nix run github:nix-community/nixos-anywhere -- \
  --flake .#app-02 \
  --disk-encryption-keys /tmp/luks.key /run/secrets/luks/app-02 \
  --generate-hardware-config nixos-generate-config ./hosts/app-02/hardware-configuration.nix
  --extra-files /tmp/app-02-seed \
  --no-reboot \
  root@203.0.113.99
```

```
[nixos-anywhere] Connected to rescue over SSH.
[nixos-anywhere] kexec NixOS installer...
[nixos-anywhere] Copied disk encryption key → /tmp/luks.key
[nixos-anywhere] Formatting disks via Disko...
[nixos-anywhere] Copying closure …-nixos-system-app-02-26.05
[nixos-anywhere] Copied extra files onto the new root
[nixos-anywhere] Installation complete. (--no-reboot)
```

`--disk-encryption-keys <remote> <local>` runs after kexec, before Disko. Repeat the flag per disk. The key is **not** in the flake.

`--generate-hardware-config nixos-generate-config <path>` writes initrd modules and NIC names back to the laptop so the next commit has a real hardware file. Backend can be `nixos-facter` instead; pick one.

`--extra-files <dir>` is a **tree rooted at `/`** of the new install (`etc/ssh/ssh_host_ed25519_key`, `persist/…`). Prepare it with `cp --parents`. This is how sops still decrypts on first boot (same host key). `--copy-host-keys` only copies keys **from the machine you are wiping**, which a blank replacement does not have.

`--no-reboot` leaves disks mounted so Case 4 can restore. Default phases are `kexec,disko,install,reboot`. Already on a NixOS installer ISO: `--phases disko,install` (skip kexec). `--vm-test` exercises Disko without touching metal.

Then reboot (or omit `--no-reboot` on **stateless** `app-*`). The node joins WireGuard (`10.100.0.12`). Ephemeral workers have nothing to restic. Stateful `db-01` needs a snapshot **before** you wipe, and a restore **before** you reboot into postgres.

Case 4: Restore `/persist` on `db-01` (not the store)

```
# still in the installer (or chroot) after --no-reboot, before first Stage 2 postgres:
sudo restic -r s3:s3.amazonaws.com/desk-restic snapshots
sudo restic -r s3:s3.amazonaws.com/desk-restic restore latest --target /mnt/persist
# then reboot into the new generation
sudo umount -R /mnt
sudo reboot
```

After boot:

```
sudo systemctl start postgresql
sudo -u postgres pg_isready
```

`--target /persist` vs `/mnt/persist` depends on whether you have already switched root. Wrong prefix dumps the tree next to the subvolume and postgres starts empty. The restic password is sops on the box (or `RESTIC_PASSWORD_FILE`), not argv. `/nix/store` comes from the flake + cache. Restoring a NAR of the old store is not DR.

If you omitted `--no-reboot`, postgres may already have `initdb`'d an empty cluster on the new persist. Restore **onto** that is a fight. Stop postgres, empty the data dir, restore, start. Better: never let it boot empty.

Case 5: Quarterly drill is the playbook

Save as `docs/dr-drill.md`:

```
# docs/dr-drill.md
1. Snapshot restic on db-01.
2. Wipe staging-app in the lab (not prod).
3. nixos-anywhere --flake .#staging-app root@…
4. curl /healthz from gw.
5. Time it. File the minutes in this repo.
```

If the drill needs a human in a web console past step 3, the ISO / Disko / sops recipients are incomplete. Fix the flake, not the wiki.

The trap

The trap is **not testing disaster recovery until a production incident**. The other trap is applying `db-01` in the same parallel wave as `@app` while a migration runs. Serial the stateful node.

Canary the stateless ones.

Rollback without a git revert means the next Colmena apply re-breaks the box.

A third trap is **rebooting `nixos-anywhere` before `/persist` is restored** on a stateful host — or omitting `--disk-encryption-keys` so Disko waits on a passphrase nobody will type in rescue. A fourth is generating hardware config and never committing it: the next install misses a kernel module.

The boring rule

- One hive, one 26.05 lock. SSH from `ops-01`.
- `--dry-run`, `--on app-01`, healthz, then `--on @app`. Serial `db-01`.
- Dead metal: `nixos-anywhere` + Disko. `--disk-encryption-keys`, `--generate-hardware-config`, `--extra-files` for host keys. `--no-reboot` until persist is restored on stateful nodes.
- Persist: restic. Store: substituters.
- Rollback is a generation, then git revert.
- DR drill in git. Time it.

Try this

1. `colmena eval` and list tags.
2. Dry-run `--on app-01`; read which units would restart.
3. Write the restic repository URL (not the password) next to the drill doc.
4. Wipe a lab VM and `nixos-anywhere` it with `--no-reboot`; restore a dummy file into `/mnt/persist`; reboot; confirm it survived. `wg show` has peers.

Conclusion: The Path Forward

Conclusion: The Path Forward

Goal: Synthesize the complete engineering journey from zero to production systems at scale, establish the cultural and operational mindset required to maintain mathematical determinism in engineering teams, examine the evolving frontiers of the Nix ecosystem, and chart your continuing roadmap as a production systems architect.

> **Note**
>
> This chapter marks the culmination of **From Zero to Production — Reproducible Systems at Scale**. Having designed, built, hardened, and deployed multi-region production infrastructure across the preceding chapters, we reflect on what has changed and how to lead organizations toward declarative maturity.

The Synthesis of Reproducible Systems

When you began this book, infrastructure was likely viewed through the conventional industry lens: an imperative collection of servers, dynamic configuration scripts, opaque container images, and fragile CI runners prone to "works on my machine" anomalies.

Across these fifteen parts, you disassembled that fragile model and replaced it with an unbroken mathematical chain of custody:

```
The Complete Reproducible Chain of Custody:

  1. Foundations: Hashed /nix/store, Content Addressing & Closure Graphs
  2. Tooling: Pure Flakes, Lockfiles, and Nix Evaluator Semantics
  3. Developer UX: Hermetic Polyglot DevShells & Declarative Direnv
  4. Workstations: Home Manager Shells, Neovim LSP, and Dotfiles
  5. Packaging: Stdenv Phases, FODs, Language Builders & Cross-Compile
  6. Host OS: Declarative NixOS Modules, Ephemeral Disko & LUKS Initrd
  7. Fleet Ops: WireGuard Meshes, Distributed Apps, & Colmena Deploys
  8. Enterprise: Vault Dynamic Secrets, GitOps with Argo, & MicroVMs
```

You proved that a complete multi-node production infrastructure—including disk partitions, systemd units, network topologies, firewall policies, encrypted dynamic secrets, containerized microservices, and monitoring stacks—can be expressed as a pure, version-controlled mathematical graph.

Wiping a production machine to bare metal is no longer a catastrophic event; it is a routine verification drill. You clone the repository, provide decryption keys, and rebuild the identical system state from cryptographic first principles.

The Production Mindset: From Pet Servers to Deterministic Graphs

Adopting NixOS at scale requires an organizational shift in perspective. It replaces tribal knowledge and ad-hoc operational heroism with structural guarantees:

The Old World (Imperative Operations)	The Nix World (Deterministic Engineering)
Servers accumulate state over months and years	Servers are ephemeral realizations of a Git commit hash
Upgrades are stressful mutations executed in-place	Upgrades are atomic symlink swaps with instant rollback
Dependencies are installed globally via `apt`, `pip`, or `brew`	Dependencies are hermetic store paths with zero collisions
Secrets are manually copied or baked into container images	Secrets are dynamically injected into volatile RAM at runtime
Disaster recovery relies on opaque VM snapshots or tape backups	Disaster recovery is a cold rebuild from pure code and keys
Testing requires deploying to expensive staging environments	Testing is executed via automated `nixosTest` VM suites in CI

When an operational failure occurs in production, your response is never to `ssh` into the node and run imperative commands. Instead, you locate the bug in your Nix expressions, write an automated regression test, commit the fix to Git, and let your automated deployment pipeline roll out the new generation atomically.

Overcoming Skepticism and Leading Cultural Adoption

Introducing Nix and NixOS to an engineering organization frequently encounters skepticism. Colleagues may perceive Nix as having an intimidating learning curve or esoteric syntax.

To lead successful organizational adoption:

1. Lead with Developer Pain Points (The Trojan Horse)

Do not begin by trying to rewrite the company's production Kubernetes cluster in NixOS on day one. Begin where developers suffer the most: **broken local development environments**. - Introduce a clean `flake.nix` with a polyglot `devShell` for a frustrating project. - Demonstrate that onboarding a new developer changes from a two-day ordeal of installing compilers and debugging PATH conflicts to a single command: `direnv allow`. - Once developers experience the speed and reliability of native devShells, they become champions of the model.

2. Provide High-Level Abstractions

Application developers should not need to understand `stdenv` override phases or fixed-point attrset recursion to deploy a service. - Build clean, internal flake templates and custom NixOS modules with intuitive options (e.g. `services.internal-api.enable = true;`). - Encapsulate corporate boilerplate (monitoring agents, logging pipelines, identity providers) into shared organization modules.

3. Maintain Absolute Lockfile Discipline

Treat `flake.lock` with the same reverence as production application lockfiles. Never allow floating pins or unversioned inputs in CI. Upgrades should occur on a predictable cadence via automated pull requests with green test suites.

Frontiers of the Nix Ecosystem

The Nix ecosystem continues to evolve rapidly. As you move forward, keep a close watch on these advancing frontiers:

1. Content-Addressed (CA) Derivations

Traditional Nix is **input-addressed**: if a comment in a base C library changes, its derivation hash changes, triggering rebuilds of every downstream dependency even if the compiled machine code is identical. Content-Addressed Nix computes the store path from the cryptographic hash of the **build output**. If the compiled binary matches, downstream rebuilds are completely skipped, eliminating cascading recompilations across massive monorepos.

2. Flake Schemas and Modern Standards

Flake schemas provide structured typing and validation for flake outputs, paving the way for standardized interfaces across packages, modules, shells, and templates.

3. Native MicroVM and Lightweight Virtualization

Projects like `microvm.nix` enable running hundreds of lightweight NixOS instances on a single hypervisor with sub-second boot times, combining the isolation of virtual machines with the density of containers.

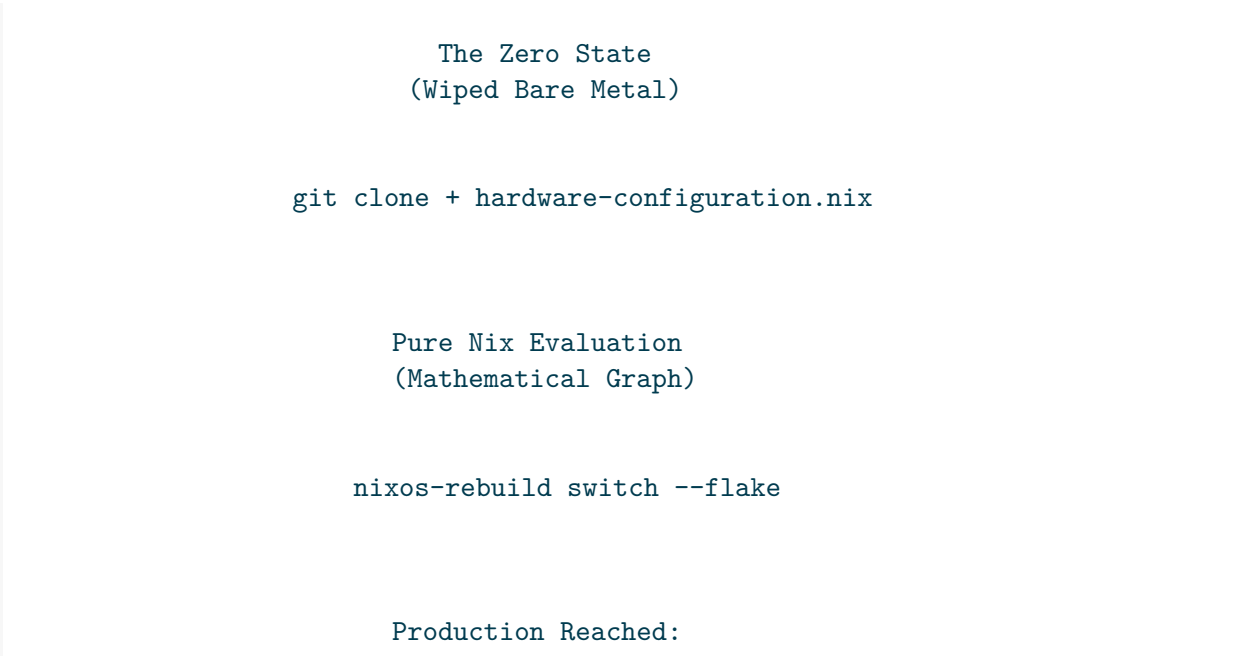
4. AI Coding Agents and Hermetic Toolchains

As autonomous coding agents become standard members of software teams, non-deterministic sandboxes become unacceptable liabilities. Nix provides the mathematical rails that allow AI agents to synthesize code, execute builds, run tests, and generate modules inside hermetically sealed environments without risking system corruption.

The Path Forward: Your Continuing Roadmap

Mastery of reproducible systems is a journey of continuous refinement. As you leave this book and return to your daily systems engineering work, commit to this professional cadence:

1. **Audit Existing Infrastructure:** Identify the snowflakes in your organization. Map their dependencies, isolate their mutable state, and begin encapsulating them into declarative NixOS host definitions.
2. **Contribute to Upstream Nixpkgs:** When you discover a missing package, an outdated version, or a broken build hook, submit a pull request to `NixOS/nixpkgs`. Contributing to the largest, most active package repository in the world sharpens your packaging craft and strengthens the entire community.
3. **Automate Everything with VM Tests:** Never ship a critical service module without a companion `nixosTest`. Let virtual machines verify that ports open, certificates renew, databases migrate, and recovery scripts function before code ever touches production hardware.
4. **Practice Disaster Recovery Drills:** Schedule regular bare-metal rebuild exercises. Prove to yourself and your organization that your infrastructure is truly immortal—reproducible anywhere, anytime, from zero to production.



```
                    Hermetic, Hardened,
                    Immutable, at Scale
```

You now possess the foundational knowledge, architectural patterns, and production engineering practices to build systems that endure. Go forth and make your infrastructure mathematically reproducible.

Projects

Projects index

Projects: packaging and your own distro

Hands-on projects that turn this book's theory into **store paths you own**. Prefer a **disposable VM** (or a dedicated partition), flakes-first workflows, and commits after each green checkpoint.

Baseline	Value
Nix	**2.34** family (`nix --version`)
NixOS / nixpkgs	**26.05** (`nixos-26.05` flake input)
Style	Flakes-first; modern `nix` CLI
Safety	Lab machines only; no production secrets in trees

These projects sit **after** (or alongside late stages of) the main syllabus. They do not replace the stdenv phases chapter–56 packaging theory—they force you to **ship** small packages and then a branded ISO product.

How to use this part

Learning loop

```
read goal → declare flake/module → rebuild/build
    → observe store / VM → fix → commit slice → next project
```

1. **Read the goal and constraints** before opening an editor. Know what "done" means.
2. **Type configs yourself.** Copy-paste once to unstick, then retype the critical attrs so they stick.
3. **Break one thing on purpose** (wrong hash, missing `runtimeInputs`, overlay not applied). Roll back with generations or `git checkout`.
4. **Commit a working flake slice** before starting the next project. Tag optional: `project-01-green`.
5. **Write a short lab note** (even five bullets). Future-you will forget which hash field mattered.

Lab environment checklist

Check	Why
Flakes + nix-command enabled	All project flakes assume modern CLI
20–40 GiB free on `/nix` volume	ISO builds and language deps are fat
KVM / nested virt available	Project 6–7 boot tests
Git installed	Lockfiles and project slices
No production credentials in the tree	Habits transfer to real fleets

```
nix --version
# expect something in the 2.34.x line (or newer compatible)

# optional: isolate lab work
mkdir -p ~/lab/nixos-projects
cd ~/lab/nixos-projects
```

How chapters are structured

Every project chapter follows roughly the same skeleton:

Section	Purpose
Goal	One-sentence outcome
Why	What skill this forces
Prerequisites	Book stages / prior projects
Theory	Concepts you must be able to explain
Lab tree	Suggested layout
Step-by-step	Commands and Nix you type
Acceptance criteria	Binary pass/fail
Pitfalls	Common failure modes
Stretch goals	Optional depth after green

Project path

Track A — Derivations & packaging

Build confidence from the smallest honest package up to language builders and overlays.

#	Chapter	Outcome
1	Hello CLI (`stdenv`)	`nix run` a shell/C tool you packaged

#	Chapter	Outcome
2	Script tool + wrap	Declared runtime deps (`jq`, …) via wrappers
3	Language package	`buildGoModule` **or** `buildPythonApplication` end-to-end
4	Overlay: pin / patch	Change nixpkgs locally without a hard fork

Track B — Custom NixOS-based distro

Name a product, brand it, build an ISO, ship a multi-edition flake.

#	Chapter	Outcome
5	Distro identity & branding	`PRODUCT.md` + identity module + assets
6	ISO + themes	Bootable installer/live image with visible brand
7	Complete product flake	Editions, defaults, install story, DEMO.md

Sample assets (lab-ready): `branding/` — `logo.svg` (CC0), `os-release` fragment example, Plymouth notes. Copy into your distro flake; **do not** reuse official NixOS marks as your product logo.

```
Track A (packaging)              Track B (distro product)

01 hello CLI                     05 identity + artwork
02 script + wrap                 06 ISO + themes
03 language builder              07 full product flake
04 overlay pin/patch
```

Suggested order

- **Linear:** 1 → 2 → 3 → 4 → 5 → 6 → 7 (recommended first pass).
- **Packaging only:** stop after 4 if you only need nixpkgs craft.
- **Distro-first:** you *can* start at 5 if you already completed Stage V packaging days—but Projects 1–2 still teach `$out` muscle memory that ISO work assumes.
- **Parallel with syllabus:** Project 1 after the stdenv phases chapter; Project 4 after the overlays chapter + 49; Project 6 after the images/generators chapter.

Time and difficulty (rough)

Project	Lab time	Difficulty	Heavy disk?
1 Hello CLI	1–2 h	Easy	No
2 Script tool	1–3 h	Easy–medium	No
3 Language pkg	2–5 h	Medium	Moderate (Go/Python deps)
4 Overlay	2–4 h	Medium	Low
5 Identity	2–4 h	Medium	Low
6 ISO + themes	3–8 h	Hard	**Yes** (multi-GB)
7 Product	4–12 h	Hard	**Yes**

ISO builds can pull large closures. Plan cache space and coffee accordingly.

Repository hygiene (all projects)

```
good project slice
  flake.nix
  flake.lock          # committed
  pkg.nix / modules/  # small, reviewable
  NOTES.md or REPORT.md
  no secrets, no /etc copies, no private keys
```

Rule	Reason
Commit `flake.lock`	Reproducible classmate / future rebuilds
Prefer `nixos-26.05` over `nixos-unstable` for labs	Stable teaching target for this book
One concern per commit	Easy bisect when Project 6 ISO fails
Document the **command that worked**	ISO flags and QEMU lines rot in memory

```
# habit after each green project
git add -A
git status   # re-check: no secrets
git commit -m "project-NN: green checkpoint"
```

Cross-links into the main book

When you are stuck on…	Revisit
Phases, `$out`, hooks	the stdenv phases chapter (stdenv phases)
Fetchers / FOD hashes	the fetchers/FOD chapter
Go / Python ecosystems	the language ecosystems A chapter–48
Patches & overrides	the patching and overrides chapter
Overlays (light)	the overlays chapter
Images / generators	the images/generators chapter
Module options	the modules as consumer chapter–19, 72
Capstone polish	the capstone scope chapter–90

Projects are **deliberate practice**. Syllabus days teach; these chapters demand a demoable artifact.

Done means (global bar)

You are "done" with this part when **all** of the following hold:

- ☐ You can explain every **store path** your Project 1–3 packages produce (at least one sentence each).
- ☐ Every lab flake has a **committed** `flake.lock`.
- ☐ **No secrets** in any project repo (tokens, age keys, Wi-Fi PSKs).
- ☐ Project 4 overlay change is **observable** (`nix run` / version / patched string).
- ☐ Distro ISO (Project 6) **boots in a VM** and shows **your** branding on 2 surfaces.
- ☐ Project 7 README explains build → boot → install/upgrade in your own words.

Trademark and ethics (read once)

"NixOS" is a trademark of the **NixOS Foundation**. Building a learning derivative is encouraged; **impersonating** official media is not.

- Choose a **distinct product name** (e.g. `YararaOS`, `MyLabOS`).

- Do not ship official snowflake artwork as *your* logo.
- When docs mention upstream, say "NixOS-based" honestly.
- See NixOS community / branding guidance when publishing publicly.

Quick start: create the lab root

```
mkdir -p ~/lab/nixos-projects
cd ~/lab/nixos-projects
git init

# after Project 1 skeleton exists:
# nix flake new hello-cli   # or hand-write flake.nix as in the chapter
```

Work **one directory per project** (or one monorepo with subdirs). Avoid mixing ISO builds into the tiny hello-cli tree—different GC pressure and mental models.

When ready, open Project 1 — Hello CLI.

Project 1: Hello CLI derivation

Project 1 — Package a tiny CLI with `stdenv.mkDerivation`

Goal: Write a reproducible derivation for a small shell or C "hello" tool, expose it from a flake, and run it via `nix build` / `nix run`.

Field	Value
Baseline	Nix **2.34**, nixpkgs `nixos-26.05`
Time	~1–2 hours
Depends on	the derivations chapter–8 (derivations/flake), the stdenv phases chapter (phases) helpful
Artifact	`packages.<system>.default` + working `apps` optional

Why this project exists

If you only install packages from nixpkgs, you never own the **build graph**. This is the smallest honest package: no network in the build, no language ecosystem, no overlay tricks—just **src** → `$out/bin`.

After this project you should be able to say, without hand-waving:

- What lands in `$out` and why the store path hash changed when you edited the script.
- Why `callPackage` injects `stdenv` and `lib`.
- Why `nix path-info -r` lists more than one path (the **closure**).

Prerequisites

- Flakes enabled (`experimental-features = nix-command flakes`).
- Comfortable with `nix build`, `nix run`, `nix flake metadata`.

- Optional: a C compiler via stdenv (already in the sandbox for C variant).

Theory

Derivation vs package vs app

Term	Meaning in this lab
Derivation	Build plan (inputs + builder) realized into a store path
Package	Usually the derivation's primary output (`$out`)
Flake `packages`	Attrset of packages per system
Flake `apps`	Named runnable (`type = "app"` + `program` absolute path)

Phases you will skip on purpose

Shell-only packaging often sets:

```
dontConfigure = true;
dontBuild = true;
```

That is honest: there is nothing to `./configure` or `make`. You still run **`installPhase`** (and hooks). Prefer `runHook preInstall` / `postInstall` so later overlays and stdenv hooks compose.

`$out` is the contract

Everything a dependent might need must appear under `$out` (here: `$out/bin/hello-cli-lab`). Scripts that assume files next to the source tree **outside** `$out` will break for users of your package.

`callPackage` dependency injection

```
pkgs.callPackage ./pkg.nix { }
```

opens `pkg.nix` as a function `{ stdenv, lib }: …` and fills arguments from `pkgs`. You rarely pass `stdenv` by hand.

Purity / sandbox

The build sandbox blocks network by default. Project 1 must **not** need `fetchurl` if `src = ./.`. That keeps the first win fast and debuggable.

Lab tree

```
~/lab/nixos-projects/hello-cli/
  flake.nix
  flake.lock
  pkg.nix
  hello.sh          # primary path in this chapter
  hello.c           # optional stretch
  NOTES.md          # your one-paragraph phase note
```

```
mkdir -p ~/lab/nixos-projects/hello-cli
cd ~/lab/nixos-projects/hello-cli
git init
```

Step 1 — Source: shell tool

`hello.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
echo "hello from $(basename "$0") - Nix derivation lab"
```

Make it executable in the **repo** for local smoke tests (the install step will set modes in the store either way):

```
chmod +x hello.sh
./hello.sh
```

> **Note**
>
> A working script on your PATH is **not** a Nix package yet. The derivation must install a copy into `$out/bin`.

Step 2 — Derivation

`pkg.nix`:

```
{ stdenv, lib }:

stdenv.mkDerivation {
  pname = "hello-cli-lab";
  version = "0.1.0";

  src = ./.;

  dontConfigure = true;
  dontBuild = true;

  installPhase = ''
    runHook preInstall
    mkdir -p $out/bin
    install -m755 hello.sh $out/bin/hello-cli-lab
    runHook postInstall
  '';

  meta = with lib; {
    description = "Tiny lab CLI packaged with stdenv";
    license = licenses.mit;
    platforms = platforms.all;
    mainProgram = "hello-cli-lab";
  };
}
```

Optional: clean the source filter

Without filtering, dirty files (`.git`, `result`, editor swap) can change hashes or bloat `src`. Prefer:

```
src = lib.cleanSource ./.;
```

or a tighter `lib.fileset` / `builtins.path` filter once you outgrow `cleanSource`.

Step 3 — Flake

`flake.nix`:

```
{
  description = "hello-cli-lab - Project 1";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
```

```
  outputs = { self, nixpkgs }:
    let
      systems = [
        "x86_64-linux"
        "aarch64-linux"
        "aarch64-darwin"
        "x86_64-darwin"
      ];
      forAll = nixpkgs.lib.genAttrs systems;
    in {
      packages = forAll (system:
        let
          pkgs = nixpkgs.legacyPackages.${system};
        in {
          default = pkgs.callPackage ./pkg.nix { };
          hello-cli-lab = pkgs.callPackage ./pkg.nix { };
        });

      apps = forAll (system: {
        default = {
          type = "app";
          program = "${self.packages.${system}.default}/bin/hello-cli-lab";
        };
      });

      checks = forAll (system: {
        hello-runs = nixpkgs.legacyPackages.${system}.runCommand "hello-cli-check" {
          nativeBuildInputs = [ self.packages.${system}.default ];
        } ''
          hello-cli-lab | grep -q "Nix derivation lab"
          touch $out
        '';
      });
    };
}
```

Tip

`meta.mainProgram` helps `nix run` discover the binary even without a custom `apps` output on newer nixpkgs helpers. Keeping an explicit `apps.default` is still clear for labs.

Step 4 — Build, run, inspect

```
nix flake lock
nix build -L
./result/bin/hello-cli-lab

nix run
nix run .#hello-cli-lab

# closure: what else is needed to run this?
nix path-info -rS ./result | tail -n 20
nix path-info -Sh ./result

# optional check output
nix build .#checks.$(nix eval --raw --impure --expr builtins.currentSystem).hello-runs
```

Record in `NOTES.md`:

1. Full store path of `./result` (resolve symlink).
2. Why more than one path appears in `-r`.
3. Phases you skipped and why.

```
readlink -f result
nix-store -q --references "$(readlink -f result)"
```

Step 5 — C variant (stretch, same flake)

Optional: `hello.c` + `buildPhase` with `$CC hello.c -o hello-cli-c`, install to `$out/bin`, expose as `packages.*.hello-c`. Compare `nix path-info -Sh` vs the shell package (bash vs dynamic linker story).

Theory checkpoints (self-test)

Topic	You should explain out loud
`$out`	Install prefix for this package's primary output
`pname` / `version`	How default `name` is formed

Topic	You should explain out loud
`callPackage`	Dependency injection of `stdenv` / `lib`
Closure	Runtime dependency graph of store paths
Purity	Why the sandbox blocks network by default
`result`	GC root symlink from `nix build`
`dontBuild`	Skipping a phase vs empty `buildPhase`

Pitfalls

Symptom	Likely cause	Fix
`permission denied` running store binary	Mode not set in install	`install -m755` or `chmod +x` in `installPhase`
Hash changes every commit	Unfiltered `src = ./.` includes junk	`lib.cleanSource` / fileset
`nix run` cannot find binary	Wrong path in `apps` or missing `mainProgram`	Align program name with `$out/bin/...`
Build sees no `hello.sh`	Wrong `src` root or file not committed	Check tree; `src` must contain the script
Darwin vs Linux surprises	`platforms` too narrow/wide	Set `meta.platforms` honestly; test the OS you claim
Infinite recursion in flake	`self` used before packages defined incorrectly	Keep `callPackage` on `pkgs`, not circular `self.packages` in same attr

---

Acceptance criteria

- ☐ `nix build` produces `./result/bin/hello-cli-lab` (or your chosen name).

- ☐ `nix run` prints the hello line.
- ☐ `flake.lock` exists and is committed.
- ☐ You can point at the **store path** and explain `$out/bin`.
- ☐ `NOTES.md` has one paragraph: phases skipped and why.
- ☐ `lib.cleanSource` (or equivalent) is used, **or** you documented why not.

Stretch goals

1. Add a trivial `installCheckPhase` / `doInstallCheck` that runs the binary.
2. Multi-output experiment: put a README in `$out/share/doc/...` (still one output is fine).
3. Build on a second system attribute (remote builder or second machine).
4. Replace `installPhase` body with `install -D` one-liner and confirm hooks still matter.
5. Add `formatter` / `nix fmt` later; not required for Project 1 green.

Checkpoint commit message (example)

```
project-01: package hello-cli-lab with stdenv and flake apps
```

Next: Project 2 — Script tool with runtime deps.

Project 2: Script tool derivation

Project 2 — Real script tool with deps and wrap

Goal: Package a Bash (or tiny Python) script that **depends on other programs** (`jq`, `curl`, …) so the **runtime closure is correct**—using `writeShellApplication` or `stdenv` + `wrapProgram`.

Field	Value
Baseline	Nix **2.34**, nixpkgs `nixos-26.05`
Time	~1–3 hours
Depends on	Project 1 (or equivalent `stdenv` comfort)
Artifact	Runnable tool whose `references` include runtime inputs

Why this project exists

Raw `install` of scripts often shebang-points at `/usr/bin/env bash` and assumes `jq` is on `PATH`. That works on your fat laptop and **fails in the sandbox, on servers, and for users who `nix shell` only your package**.

Nix's rule: **runtime tools are dependencies**, not ambient hope. This project makes that rule visceral: remove `jq` from `runtimeInputs`, watch the tool fail closed, restore it, and see the store reference reappear.

Prerequisites

- Project 1 green (you know `$out/bin` and `nix run`).
- Rough idea of *build-time* vs *run-time* inputs.
- Optional reading: the stdenv phases chapter phases; wrappers are a packaging pattern, not a new language.

Theory

Ways to ship script tools with dependencies

Helper	Best for	Runtime deps
`writeShellApplication`	Shell scripts with linting	`runtimeInputs` → PATH wrap + shellcheck
`pkgs.writers.writePython3Bin`	Python scripts with Python & system deps	`libraries` (PYTHONPATH) + `makeWrapperArgs` (PATH) + pyflakes
`writeShellScriptBin`	One-liners / tiny glue	Manual or none
`stdenv.mkDerivation` + `makeWrapper`	Multi-file layouts, custom share/	`wrapProgram`

Prefer **`writeShellApplication`** for shell tools and **`pkgs.writers.writePython3Bin`** for Python tools unless you need a multi-file tree.

What the wrapper actually does

Inspect the built binary:

```
nix build
head -40 ./result/bin/weather-lab
```

You will typically see a shell wrapper that **prefixes `PATH`** with store paths of `jq`, `curl`, etc., then `exec`s the real script. The script's shebang is rewritten to a store bash when using the write* helpers—another purity win.

Closure vs references

```
nix-store -q --references ./result     # direct refs
nix path-info -r ./result              # full closure walk
nix path-info -rS ./result | sort -nk2 # size-aware
```

If `jq` is a true runtime input, its store path appears in the graph. If you only *mention* jq in a comment, it will not.

Build-time vs run-time (again)

Kind	Example	In this project
`nativeBuildInputs`	`makeWrapper`, compilers	stdenv+wrap path
`buildInputs`	libs to link	rare for pure scripts
`runtimeInputs`	`jq`, `curl`	writeShellApplication

Kind	Example	In this project
propagated*	infect dependents	**avoid** in this lab

Lab tree

```
~/lab/nixos-projects/script-tool/
  flake.nix
  flake.lock
  weather.nix          # writeShellApplication path
  weather-wrap.nix     # optional stdenv + wrapProgram
  weather.sh           # only if using wrap path
  NOTES.md
```

```
mkdir -p ~/lab/nixos-projects/script-tool
cd ~/lab/nixos-projects/script-tool
```

Step 1 — Preferred path: `writeShellApplication`

`weather.nix`:

```
{ writeShellApplication, jq, curl }:

writeShellApplication {
  name = "weather-lab";
  runtimeInputs = [ jq curl ];
  text = ''
    set -euo pipefail

    usage() {
      echo "usage: weather-lab [--live URL]" >&2
      echo "  default: offline demo JSON (CI-friendly)" >&2
    }

    if [[ "''${1:-}" == "-h" || "''${1:-}" == "--help" ]]; then
      usage
      exit 0
    fi
```

```
    # Default: no network required for reproducible demos / CI.
    if [[ "''${1:-}" == "--live" ]]; then
      url="''${2:-https://httpbin.org/json}"
      curl -fsSL "$url" | jq -c .
    else
      echo '{"temp":21,"city":"lab"}' | jq -r '"\(.city): \(.temp)C"'
    fi
  '';
}
```

> **!** Important
>
> In real Nix strings, `''${...}` escapes `${...}` so Bash—not Nix—expands the variable. When you type the file, keep that escaping for any Bash `${...}` inside `'' ... ''` multiline strings.

Flake

`flake.nix`:

```
{
  description = "weather-lab - Project 2";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAll = nixpkgs.lib.genAttrs systems;
    in {
      packages = forAll (system:
        let pkgs = nixpkgs.legacyPackages.${system};
        in {
          default = pkgs.callPackage ./weather.nix { };
          weather-lab = pkgs.callPackage ./weather.nix { };
        });

      apps = forAll (system: {
        default = {
          type = "app";
          program = "${self.packages.${system}.default}/bin/weather-lab";
        };
      });
    };
}
```

Build and inspect

```
nix flake lock
nix build -L
./result/bin/weather-lab
# expect: lab: 21C

head -30 ./result/bin/weather-lab
nix-store -q --references ./result
nix path-info -rS ./result | rg 'jq|curl|bash' || true
nix run
```

Step 2 — Fail closed (required demo)

1. Edit `weather.nix`: remove `jq` from `runtimeInputs` (leave `curl` or empty).
2. Rebuild and run:

```
nix build --rebuild -L
./result/bin/weather-lab
# should fail: jq: command not found (or similar)
```

3. Restore `jq`, rebuild, confirm success.
4. In `NOTES.md`, paste **before/after** `nix-store -q --references` lines showing jq appear/disappear.

This single demo is the educational core of Project 2.

Step 3 — Alternative: `stdenv` + `wrapProgram`

Use when you already have a multi-file tree or want to see `makeWrapper` explicitly.

`weather.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
echo '{"temp":21,"city":"lab"}' | jq -r '"\(.city): \(.temp)C"'
```

`weather-wrap.nix`:

```
{ stdenv, lib, makeWrapper, jq, curl }:

stdenv.mkDerivation {
  pname = "weather-lab";
  version = "0.1.0";
  src = lib.cleanSource ./.;

  nativeBuildInputs = [ makeWrapper ];

  dontConfigure = true;
  dontBuild = true;

  installPhase = ''
    runHook preInstall
    mkdir -p $out/bin
    cp weather.sh $out/bin/weather-lab
    chmod +x $out/bin/weather-lab
    wrapProgram $out/bin/weather-lab \
      --prefix PATH : ${lib.makeBinPath [ jq curl ]}
    runHook postInstall
  '';

  meta = with lib; {
    description = "weather-lab via makeWrapper";
    license = licenses.mit;
    mainProgram = "weather-lab";
    platforms = platforms.all;
  };
}
```

```
# point flake default at weather-wrap.nix once, build, compare wrapper text
head -40 ./result/bin/weather-lab
```

Note: `wrapProgram` renames the original to `.weather-lab-wrapped` (or similar) and installs a wrapper—inspect both.

Step 4 — Optional live mode (network at runtime)

`--live` uses network when **you run** the tool, not when Nix **builds** it. That is allowed and normal: purity applies to the build sandbox, not to every program you ship.

Still:

- Default offline path keeps CI and classrooms network-free.

- Do not `curl` during `installPhase` without a FOD fetcher—that fights the sandbox.
- Document that live mode needs network + DNS.

```
./result/bin/weather-lab --live https://httpbin.org/json || echo "network optional"
```

Lab tasks

1. Build with `writeShellApplication` and offline JSON path.
2. Remove `jq` from `runtimeInputs`; show runtime failure.
3. Restore; show references include jq.
4. Document store path of the wrapper (`readlink -f result` + `head`).
5. (Optional) Port once to `wrapProgram`; note differences in `NOTES.md`.

Pitfalls

Symptom	Likely cause	Fix
Works in dev shell, fails via `nix run`	Ambient PATH only	Declare `runtimeInputs` / wrap
Bash `${var}` eaten by Nix	Unescaped `${` in `''` string	Use `''${var}`
`curl` missing only on live path	Not in runtimeInputs	Declare it
Dirty rebuild loops	Outputs under `src`	cleanSource; keep result out

---

Acceptance criteria

☐ `nix run` prints `lab: 21C` (or equivalent).

- ☐ Missing-dep experiment fails closed; notes capture the lesson.
- ☐ Closure/references include runtime tools (`jq` at minimum).
- ☐ Wrapper inspected with `head`.
- ☐ `flake.lock` committed; `NOTES.md` compares wrapper vs plain install.

Stretch goals

1. `--version` from installed metadata.
2. Second binary in the same derivation.
3. `checks.weather` asserting offline output.
4. `symlinkJoin` "kit" metapackage; compare closure.
5. **Python tool with dual dependencies (`pkgs.writers.writePython3Bin`):** Port `weather-lab` or an article processor to Python. Use `libraries` for `requests` and `makeWrapperArgs = [ "--prefix" "PATH" ":" (lib.makeBinPath [ jq curl ]) ];` to ensure CLI binaries are present on `PATH`. Inspect `./result/bin/<name>` with `head` to verify how `PYTHONPATH` and `PATH` are wrapped together.

Theory recap

Question	Target answer
Why not `/usr/bin/jq`?	Not in the closure; not reproducible
Who rewrites shebangs?	write* helpers / patchShebangs
Runtime network impure for Nix?	Build purity runtime policy
When wrapProgram?	Custom layout, multi-bin

Next: Project 3 — Language package.

Project 3: Language package derivation

Project 3 — Package a Go or Python app with language helpers

Goal: Use a **language-specific builder** (`buildGoModule` or `buildPythonApplication`) so fetch/build/test idioms match nixpkgs conventions. Finish **one** language end-to-end with real hashes (not `fakeHash` left behind).

Field	Value
Baseline	Nix **2.34**, nixpkgs `nixos-26.05`
Time	~2–5 hours
Depends on	Projects 1–2; the language ecosystems A chapter–48 helpful
Artifact	`packages.<system>.default` + `REPORT.md`

Why this project exists

`stdenv.mkDerivation` can package anything, but language ecosystems already have **conventions**: Go module vendor hashes, Python `pyproject` hooks, test integration. nixpkgs helpers encode those conventions so your package looks like every other good citizen in the tree.

This project teaches:

- How **fixed-output-style hashes** pin dependency graphs (`vendorHash`, cargo-like patterns, FOD src).
- How to read the **first failed build** that prints the expected hash.
- How language tests map onto `doCheck` / hooks.

Pick **Option A (Go)** *or* **Option B (Python)** and complete it. Doing both is excellent stretch, not required for green.

Prerequisites

- Project 1 flake layout comfort.
- Toolchain on the **host** optional for scaffolding (`go` / `python`); pure Nix builds should still work after hashes are set.
- Network **once** for module download / first hash discovery (unless you vendor offline).

Theory map

Concern	Go (`buildGoModule`)	Python (`buildPythonApplication`)
Manifest	`go.mod` / `go.sum`	`pyproject.toml` / `setup.cfg`
Dep pin	`vendorHash` or `vendor/`	Package set + optional FODs
Entry	compiled binary → `$out/bin`	`console_scripts` / scripts
Tests	`doCheck`, `checkFlags`	`pytestCheckHook`, `unittest`
Common footgun	wrong `vendorHash` / proxy	missing `format`/`pyproject`, native deps

Why helpers beat raw stdenv here

Language builders set:

- Correct `nativeBuildInputs` (compilers, `python`, hooks).
- Dependency fetch phases that honor hashes.
- Install layouts that match ecosystem expectations.

You still need to understand stdenv phases—the helper **fills** them.

Hash discipline

Field	Meaning
`lib.fakeHash` / `pkgs.lib.fakeSha256`	Placeholder that **must** be replaced
Hash in build error	Authoritative for that input content
Changing `src` or deps	**Must** recompute related hashes

Never commit a known-fake hash as “done.”

Lab tree (shared idea)

```
~/lab/nixos-projects/lang-pkg/
  flake.nix
  flake.lock
  pkg.nix
  REPORT.md
  # Go:
  go.mod go.sum main.go
  # or Python:
  pyproject.toml hi_py/__init__.py hi_py/__main__.py
```

```
mkdir -p ~/lab/nixos-projects/lang-pkg
cd ~/lab/nixos-projects/lang-pkg
```

Option A — Go (`buildGoModule`)

A1. Scaffold module

```
go mod init example.com/hi
```

`main.go`:

```
package main

import "fmt"

func main() {
    fmt.Println("hi from buildGoModule")
}
```

Optional dependency so `vendorHash` is non-trivial (recommended):

```
package main

import (
    "fmt"
    "github.com/google/uuid"
)

func main() {
    fmt.Println("hi from buildGoModule")
```

```
    fmt.Println(uuid.NewString())
}
```

```
go get github.com/google/uuid@v1.6.0
go mod tidy
```

A2. Package expression

pkg.nix:

```
{ buildGoModule, lib }:

buildGoModule {
  pname = "hi";
  version = "0.1.0";
  src = lib.cleanSource ./.;

  # First build: set to lib.fakeHash and read the error.
  # After vendor/ present, some pins use vendorHash = null - follow your nixpkgs docs.
  vendorHash = lib.fakeHash;

  meta = {
    description = "Project 3 Go lab binary";
    license = lib.licenses.mit;
    mainProgram = "hi";
  };
}
```

A3. Flake

```
{
  description = "lang-pkg Go - Project 3";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAll = nixpkgs.lib.genAttrs systems;
    in {
      packages = forAll (system:
        let pkgs = nixpkgs.legacyPackages.${system};
        in {
          default = pkgs.callPackage ./pkg.nix { };
```

```
        });

      apps = forAll (system: {
        default = {
          type = "app";
          program = "${self.packages.${system}.default}/bin/hi";
        };
      });
    };
}
```

A4. Hash fix loop

```
nix build 2>&1 | tee /tmp/go-build.log
# Look for: specified: sha256-AAAA... got: sha256-REAL...
# Paste REAL into vendorHash

# rebuild until green
nix build -L
nix run
./result/bin/hi
```

A5. Vendor / tests (optional)

```
go mod vendor   # then set vendorHash per nixpkgs 26.05 docs (often null when vendored)
```

Light test: `main_test.go` + `doCheck = true` so a failing test fails the derivation.

Option B — Python (`buildPythonApplication`)

Layout:

```
pyproject.toml
hi_py/__init__.py
hi_py/__main__.py
```

`pyproject.toml` (setuptools; hatchling also fine if hooks match your pin):

```
[build-system]
requires = ["setuptools>=61"]
```

```
build-backend = "setuptools.build_meta"

[project]
name = "hi-py"
version = "0.1.0"
description = "Project 3 Python lab"
requires-python = ">=3.11"
dependencies = []

[project.scripts]
hi-py = "hi_py.__main__:main"

[tool.setuptools.packages.find]
include = ["hi_py*"]
```

hi_py/__main__.py:

```
def main() -> None:
    print("hi from buildPythonApplication")


if __name__ == "__main__":
    main()
```

pkg.nix:

```
{ python3Packages, lib }:

python3Packages.buildPythonApplication {
  pname = "hi-py";
  version = "0.1.0";
  src = lib.cleanSource ./.;

  pyproject = true;
  build-system = [ python3Packages.setuptools ];

  meta = {
    description = "Project 3 Python lab";
    license = lib.licenses.mit;
    mainProgram = "hi-py";
  };
}
```

> ⚠ Warning
>
> Python attrs evolve (`pyproject = true` vs older `format`). **Match a simple package on your 26.05 pin** if eval drifts. Goal: helper lifecycle, not one frozen attr name.

```
nix build -L && nix run && ./result/bin/hi-py
```

Recommended: add one dep (`click` in `pyproject.toml` **and** `dependencies = [ python3Packages.click ];`), rebuild, confirm with `nix path-info -r ./result | rg -i click`.

Shared finish

```
nix flake show
nix path-info -rS ./result | tail
nix path-info -Sh ./result
```

REPORT.md (required)

```
# Project 3 report
## Language + helper (why)
## Hashes / pins (what each field fingerprints)
## Closure (`nix path-info -Sh`)
## One failure + fix
## Offline rebuild story after FODs populate
```

Pitfalls

Symptom	Likely cause	Fix
Hash mismatch forever	Edited `src` / `go.sum` after hash set	Re-run hash loop
Go wants network in sandbox	Bad vendor / `vendorHash`	Fix per pin docs
Python missing module at runtime	Dep not declared in Nix	Add to `dependencies`
`nix run` misses binary	Name `mainProgram` / scripts	Align entry names

Symptom	Likely cause	Fix
Dirty `src`	`.venv`, `result`, `__pycache__`	`cleanSource` + gitignore

Acceptance criteria

- ☐ `nix run` works for the chosen language binary.
- ☐ Hash fields are **real** (no committed `fakeHash`).
- ☐ `REPORT.md` complete (helper, hashes, closure, one failure).
- ☐ Flake exposes `packages.<system>.default`.
- ☐ Offline rebuild after FODs are populated (documented).
- ☐ `flake.lock` committed.

Stretch goals

1. Ship **both** Go and Python in one flake.
2. Real unit test under `doCheck`.
3. Rust track: `rustPlatform.buildRustPackage` + `cargoHash`.
4. Diff your `pkg.nix` against a similar nixpkgs package (three style notes).

Self-check

1. What does `vendorHash` fingerprint?
2. Why both Python `build-system` and runtime `dependencies`?
3. Where does the runnable live under `$out` for each option?

4. How would you pin a bad upstream version until a fix? (→ Project 4)

Next: Project 4 — Overlay pin and patch.

Project 4: Overlay pin and patch

Project 4 — Overlay: pin or patch a package

Goal: Change nixpkgs **without forking the tree**: apply a patch and/or pin behavior via an **overlay**, consume it from a flake package output and a `devShell`, and write a short comparison of overlay vs **extra flake input**.

Field	Value
Baseline	Nix **2.34**, nixpkgs `nixos-26.05`
Time	~2–4 hours
Depends on	the overlays chapter (overlays light), the patching and overrides chapter (patching); Projects 1–3 helpful
Artifact	Working overlay + `NOTES.md` (overlay vs input)

Why this project exists

Sooner or later you need:

- A one-line fix before upstream merges.
- A temporary pin while 26.05 carries a broken minor.
- The **same** patched tool in a devShell **and** a NixOS host.

Overlays modify the `pkgs` fixed-point. Done lightly, they are elegant. Done heavily, they make upgrades impossible. This project practices the light path with an observable demo (patched `hello` greeting or equivalent).

> **!** Important
>
> Overlays are sharp tools. Prefer **module options**, **single-site `overrideAttrs`**, or **separate flake inputs** when they suffice. Learn overlays so you are not helpless—not so every problem becomes one.

Prerequisites

- Can `import nixpkgs { overlays = [ ... ]; }` from a flake.
- Comfortable reading `overrideAttrs (old: { ... })`.
- Optional: `nix build nixpkgs#hello` works on your machine.

Theory

Overlay shape

```
final: prev: {
  # attrs to add or replace
}
```

Argument	Meaning
`prev`	Package set **before** this overlay
`final`	Fixed-point **after** all overlays—use when your new pkg depends on another overridden pkg

```
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or [ ]) ++ [ ./patches/hello-msg.patch ];
  });
}
```

Use **`prev.hello`** for the package you are overriding. Use **`final`** when composing packages that should see other overlays (e.g. `final.myLib`).

Three layers of change (recap)

Mechanism	Scope	Maintenance
`override` / `overrideAttrs` at call site	One use	Lowest
Overlay	Whole `pkgs` view for flake/host	Medium
Fork / replace input	Anything	Highest

```
nixpkgs  →  overlay(s)  →  pkgs.hello (patched)
                ↑
          overrideAttrs / patches
```

Overlay vs separate flake input

Approach	When it shines	Cost
Overlay patch on 26.05	Tiny delta, same channel	Patch may fail on bump
Second input at a rev	Need many packages from another commit	Two evaluations; version skew
`pkgs.hello.overrideAttrs` only in one module	Single consumer	Easy to forget other consumers

You will document this in `NOTES.md` after Lab B.

Infinite recursion classic

```
# BAD: final.hello depending on final.hello via careless composition
final: prev: {
  hello = final.hello.overrideAttrs (_: { }); # boom
}
```

Prefer `prev` for the base package you modify.

Lab tree

```
~/lab/nixos-projects/overlay-lab/
  flake.nix
  flake.lock
  overlay.nix
  patches/
    hello-msg.patch
  NOTES.md
```

```
mkdir -p ~/lab/nixos-projects/overlay-lab/patches
cd ~/lab/nixos-projects/overlay-lab
```

Step 1 — Flake wiring (overlay applied once)

`flake.nix`:

```
{
  description = "overlay-lab - Project 4";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      systems = [ "x86_64-linux" "aarch64-linux" "aarch64-darwin" "x86_64-darwin" ];
      forAll = nixpkgs.lib.genAttrs systems;

      pkgsFor = system:
        import nixpkgs {
          inherit system;
          overlays = [ (import ./overlay.nix) ];
        };
    in {
      packages = forAll (system: {
        default = (pkgsFor system).hello;
        hello = (pkgsFor system).hello;
      });

      devShells = forAll (system:
        let pkgs = pkgsFor system;
        in {
          default = pkgs.mkShell {
            packages = [ pkgs.hello pkgs.jq ];
          };
        });

      # Optional: prove NixOS would see the same pkgs if you import similarly
      # nixosConfigurations.lab = ... nixpkgs.overlays = [ (import ./overlay.nix) ];
    };
}
```

> **Tip**
>
> `legacyPackages` does **not** automatically apply your overlay. Construct `pkgs` with `import nixpkgs { overlays = ...; }` (or `nixpkgs.legacyPackages` + `extend`) deliberately and **reuse** that `pkgs` everywhere.

Step 2 — Lab A: message patch on `hello`

A1. Discover the string to patch

```
nix build nixpkgs/nixos-26.05#hello -o /tmp/hello-stock
/tmp/hello-stock/bin/hello
# note the exact greeting line on your pin
```

You need a patch against the **source** of that package version. Approaches:

1. **Proper patch file** against upstream `hello` sources (best practice).
2. **Lab-only** `postPatch` sed (acceptable for learning; document as lab-only).

A2. Lab-fast path: `postPatch` (allowed for green)

`overlay.nix`:

```
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    pname = old.pname;
    postPatch = (old.postPatch or "") + ''
      # Lab-only: force a recognizable greeting if source layout matches GNU hello.
      # If this fails, switch to a real patches = [ ./patches/hello-msg.patch ];
      if [ -f src/hello.c ]; then
        substituteInPlace src/hello.c \
          --replace-fail "Hello, world!" "Hello, overlay-lab!"
      fi
    '';
  });
}
```

> **Note**
>
> `substituteInPlace` / `--replace-fail` behavior depends on stdenv helpers on your pin. If flags differ, use `sed -i` carefully or a unified diff patch. The acceptance bar is **observable changed output**, not a perfect GNU patch hobby.

A3. Proper patch path (preferred write-up)

1. Unpack hello sources (from the derivation or upstream tarball matching the pin).
2. Edit the greeting.

3. `diff -u` → `patches/hello-msg.patch`.

4. Overlay:

```
final: prev: {
  hello = prev.hello.overrideAttrs (old: {
    patches = (old.patches or [ ]) ++ [ ./patches/hello-msg.patch ];
  });
}
```

A4. Prove it

```
nix build -L
./result/bin/hello
# must show your modified greeting

nix develop -c hello
# same binary lineage from overlayed pkgs

nix path-info -Sh ./result
```

If output is unchanged, the overlay was not applied or `postPatch` did not match sources—fix, do not proceed to Lab B until this is green.

Step 3 — Lab B: pin via extra input (document, implement lightly)

Sometimes you need an older/newer **package graph**, not a one-line patch.

`flake.nix` sketch (second input):

```
{
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";
  # Example only - replace <rev> with a real commit after you pick one:
  # inputs.nixpkgs-tool.url = "github:NixOS/nixpkgs/<rev>";
  # inputs.nixpkgs-tool.inputs = {}; # not always needed

  outputs = { self, nixpkgs /*, nixpkgs-tool */ }:
    let
      system = "x86_64-linux";
      pkgs = import nixpkgs {
        inherit system;
        overlays = [ (import ./overlay.nix) ];
      };
```

```
      # pkgsTool = import nixpkgs-tool { inherit system; };
    in {
      packages.${system} = {
        default = pkgs.hello;           # patched on 26.05
        # toolFromPin = pkgsTool.curl;  # whole package from another rev
      };
    };
}
```

In `NOTES.md`, answer:

1. When is a **second input** cleaner than an overlay?
2. What skew risk appears when `pkgs` and `pkgsTool` both provide libraries?
3. Which approach would you choose for (a) one-line crash fix (b) need cmake 3.x from a different commit?

You do **not** must ship a second input for green if Lab A works—but the written comparison is required.

Step 4 — Optional NixOS consumer + upgrade drill

On a VM: set `nixpkgs.overlays = [ (import ./overlay.nix) ];` and `environment.systemPackages = [ pkgs.hello ];`, rebuild, confirm the patched string.

In `NOTES.md`, write recovery if 26.05 → next stable breaks the patch: refresh patch, drop overlay if fixed upstream, switch to input pin, add `passthru.tests`.

Gotchas

Issue	Notes
Overlay not applied	`legacyPackages` without `extend` / separate `import`
Infinite recursion	`final.pkg` self-ref; use `prev`
Patch fails on bump	Comment + CI build of the package
Two nixpkgs copies	Missing `follows`; closure duplication
`nix shell nixpkgs#hello` unpatched	Different package set—expected

Acceptance criteria

- ☐ Overlay changes **observable** behavior.
- ☐ Visible via flake `packages` **and** `devShell`.
- ☐ `NOTES.md`: overlay vs extra input (2 scenarios) + drop-after-upstream plan.
- ☐ Patch file **or** labeled lab-only `postPatch`.
- ☐ `flake.lock` committed; no infinite recursion.

Stretch goals

1. `passthru.tests` asserting the new string.
2. Second package depending on patched `hello` via `final`.
3. Wire overlay into a tiny `nixosConfigurations.demo`.
4. `lib.composeManyExtensions` with two overlays.

Self-check

1. `override` vs `overrideAttrs`?
2. When is `final` required?
3. When is a second input *better* than an overlay?
4. Prove an overlay is active in 30 seconds.

Next: Project 5 — Distro identity and branding.

Project 5: Distro identity and branding

Project 5 — Name your distro: identity, versioning, branding assets

Goal: Define a **product identity** for a NixOS-based derivative: codename, version scheme, artwork, and modules that surface branding (`os-release`, issue/banner, optional loader labels, shipped logo).

This is the "what is this OS called?" project—**before** ISO polish (Project 6) and product packaging (Project 7).

Field	Value
Baseline	Nix **2.34**, NixOS / nixpkgs **26.05**
Time	~2–4 hours
Depends on	Stage II host flake comfort; the modules as consumer chapter–19 modules
Artifact	Distro flake slice + `PRODUCT.md` + `branding/`

Why this project exists

A "custom distro" that still says `NAME=NixOS` everywhere is only a config repo. Identity work forces you to:

- Pick a **distinct name** (trademark-safe).
- Encode version/codename policy.
- Express branding as **NixOS modules**, not ad-hoc files after install.
- Treat artwork and licenses as first-class inputs.

Projects 6–7 reuse this module graph. Do not skip `PRODUCT.md`.

> **!** Important
>
> **Trademark:** "NixOS" is a trademark of the NixOS Foundation. Use a distinct product name; say "NixOS-based" when describing heritage. Do not ship official snowflake art as *your* logo. See community branding guidance.

Prerequisites

- A NixOS VM or machine you can rebuild (prefer disposable).
- Flake-based `nixosConfigurations` experience (the flake hosts chapter+).
- Optional: copy book sample assets from `content/99-projects/branding/`.

Theory

Identity surfaces

Surface	User-visible?	Typical mechanism
`/etc/os-release`	Tools, neofetch, support scripts	NixOS options / etc files
`/etc/issue`	TTY login banner	`environment.etc.issue`
System label	Boot loader generations	`system.nixos.label`
`/etc/<distro>/logo.svg`	Docs, greeter hooks later	`environment.etc`
Pretty hostname motd	SSH banners	`users.motd` / issue.net

Not every surface must be perfect on day one. Pick **3** and make them consistent.

Version scheme for derivatives

Track **base nixpkgs** explicitly:

```
<distro-version> = <nixpkgs-release>.<distro-patch>[-label]
example: 26.05.0-lab1
```

Component	Meaning
`26.05`	Upstream release train you track
`.0`	Your product revision on that train
`-lab1`	Optional channel/codename tag

Record the **flake input rev** in `PRODUCT.md` when you cut a "release."

Modules as product API

```
options.myDistro.*   →  config that applies branding
```

Downstream hosts set:

```
myDistro.enable = true;
myDistro.name = "YararaOS";
```

instead of copy-pasting os-release text into every host.

os-release merge reality

NixOS already generates os-release-related content. On **26.05**, prefer **documented options** when they exist (names evolve—verify on your pin), for example patterns around distro id / pretty name if available. Overwriting `environment.etc."os-release"` can fight the module system.

Lab rule: after rebuild, `cat /etc/os-release` is the source of truth. Adjust the module until **your name** is visible without breaking activation.

Product sheet (`PRODUCT.md`)

Create and fill:

```
# PRODUCT.md

| Field | Value |
|-------|-------|
| Distro name | |
| Version | 26.05.0-lab1 |
| Codename | |
| Homepage | |
| Bug tracker | |
| Base | nixos-26.05 |
| Default edition | minimal |
| Trademark notes | distinct name; NixOS-based |
| Flake input rev (optional) | |
```

Example names: `YararaOS`, `MyLabOS`, `RiverNix`—anything **not** "NixOS" alone.

Repo layout

```
my-distro/
  flake.nix
  flake.lock
  PRODUCT.md
  branding/
    logo.svg
    README.md          # license for assets
  modules/
    identity.nix
  hosts/
    lab.nix            # disposable VM/host
    iso.nix            # stub for Project 6
```

Copy sample branding from this book

```
# adjust path to your checkout of this monorepo
cp -r /path/to/books/NixOS/content/99-projects/branding ./branding
# remix logo.svg; keep license notes honest
```

Book kit:

File	Purpose
`logo.svg`	Sample mark (CC0)
`os-release.fragment.example`	Field checklist
`plymouth-theme-notes.md`	Splash strategy (Projects 6–7)
`README.md`	License + trademark notes

Step 1 — Identity module

`modules/identity.nix`:

```
{ config, lib, pkgs, ... }:
let
  cfg = config.myDistro;
in {
  options.myDistro = {
    enable = lib.mkEnableOption "distro identity defaults";

    name = lib.mkOption {
```

```
      type = lib.types.str;
      default = "MyLabOS";
      description = "Pretty distro name (distinct from NixOS trademark).";
    };

    id = lib.mkOption {
      type = lib.types.str;
      default = "mylabos";
      description = "Lowercase ID-like token for os-release ID=";
    };

    version = lib.mkOption {
      type = lib.types.str;
      default = "26.05.0-lab1";
    };

    codename = lib.mkOption {
      type = lib.types.str;
      default = "lab";
    };

    homeUrl = lib.mkOption {
      type = lib.types.str;
      default = "https://example.invalid/";
    };
  };

  config = lib.mkIf cfg.enable {
    system.nixos.label = "${cfg.name}-${cfg.version}";

    # Validate on 26.05: prefer first-class options if present on your pin.
    # The etc approach is a lab lever-adjust when it conflicts with generators.
    environment.etc."os-release".text = lib.mkDefault ''
      NAME="${cfg.name}"
      ID=${cfg.id}
      ID_LIKE=nixos
      VERSION="${cfg.version}"
      VERSION_CODENAME=${cfg.codename}
      PRETTY_NAME="${cfg.name} ${cfg.version} (NixOS-based)"
      HOME_URL="${cfg.homeUrl}"
      DOCUMENTATION_URL="${cfg.homeUrl}docs"
      SUPPORT_URL="${cfg.homeUrl}support"
      BUG_REPORT_URL="${cfg.homeUrl}issues"
    '';

    environment.etc.issue.text = lib.mkDefault ''
      ${cfg.name} ${cfg.version} \n \l
```

```
    '';

    users.motd = lib.mkDefault ''
      Welcome to ${cfg.name} ${cfg.version} (${cfg.codename})
      Base: NixOS / nixpkgs 26.05 track - lab derivative
    '';

    environment.etc."${cfg.id}/logo.svg".source = ../branding/logo.svg;

    # Optional: make support tooling discover the brand easily
    environment.variables.DISTRIB_ID = cfg.id;
  };
}
```

> **Warning**
>
> If activation or other modules fight `environment.etc."os-release"`, switch to the **documented 26.05 options** for distro naming and keep `issue` + logo + `system.nixos.label` as your guaranteed branding. Record what worked in `PRODUCT.md`.

Use the field checklist in `branding/os-release.fragment.example` when editing.

Step 2 — Lab host + flake

`hosts/lab.nix`:

```
{ config, lib, pkgs, ... }: {
  imports = [ ../modules/identity.nix ];

  myDistro.enable = true;
  myDistro.name = "MyLabOS";
  myDistro.id = "mylabos";
  myDistro.version = "26.05.0-lab1";

  networking.hostName = "mylabos-lab";

  # Minimal disposable VM assumptions - adapt to your hardware config:
  # boot.loader, fileSystems, users.users.... from your existing lab host.
}
```

`flake.nix` (sketch):

```
{
  description = "MyLabOS - Project 5 identity";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.lab = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux"; # or your arch
      modules = [
        ./hosts/lab.nix
        # ./hardware-configuration.nix  # real disk/boot
      ];
    };
  };
}
```

```
nix flake lock
sudo nixos-rebuild switch --flake .#lab
# or: nixos-rebuild build-vm --flake .#lab && ./result/bin/run-*-vm
```

Step 3 — Verify identity

On the running system:

```
cat /etc/os-release
cat /etc/issue
cat /etc/motd 2>/dev/null || true
ls -la /etc/mylabos/   # or your cfg.id
hostnamectl 2>/dev/null || true
```

Capture outputs into `lab-evidence.md` or paste under `PRODUCT.md` "Evidence" section.

Optional screenshot: TTY login showing `/etc/issue`.

Step 4 — Artwork + consistency

1. Sample or custom `logo.svg`; license note in `branding/README.md`.
2. No official NixOS snowflake as *your* mark.

3. Prefer SVG; `rsvg-convert -w 256 branding/logo.svg -o branding/logo.png` if a greeter needs PNG later.

4. Confirm **name/version** match across options defaults, os-release, issue, `system.nixos.label`, motd—drift multiplies in Project 6.

Pitfalls

Symptom	Likely cause	Fix
Still says NixOS everywhere	Module not imported / disabled	Import + `enable`
os-release reverts	Lost merge to generator	Supported options; less force
Logo missing	Bad relative path	Path relative to module file
Trademark risk	Named only "NixOS …"	Distinct `NAME` / `ID`
Rebuild fails after clean eval	Missing boot/fs modules	Add real hardware config

---

Acceptance criteria

☐ `PRODUCT.md` complete (name, version, base, trademark note).

☐ `branding/logo.svg` + license note.

☐ Identity module: clean rebuild of lab host/VM.

☐ os-release and/or issue show **your** name (document option path if needed).

☐ Logo under `/etc/<id>/` (or equivalent).

☐ `flake.lock` committed; no official NixOS marks as product logo.

Stretch goals

1. `myDistro.edition` option varying motd.
2. Doc package installing `PRODUCT.md` under `$out/share/doc/...`.
3. `hosts/iso.nix` stub importing only identity (ready for Project 6).

Self-check

1. Why put `26.05` in the product version string?
2. `NAME` vs `ID` in os-release?
3. Why modules instead of editing files after first boot?
4. Which surfaces survive headless installs?

Next: Project 6 — ISO and themes.

Project 6: Distro ISO and themes

Project 6 — Installer ISO with themes (Plymouth / greeter / console)

Goal: Produce a **bootable installer ISO** (or live image) for your branded distro using NixOS installer profiles (`isoImage`) and/or `nixos-generators`, with **visible theming**: console colors/fonts, optional Plymouth splash, greeter/wallpaper hooks for a graphical profile, and ISO volume metadata.

Field	Value
Baseline	Nix **2.34**, NixOS **26.05**
Time	~3–8 hours (downloads + iteration)
Depends on	Project 5 identity module; the images/generators chapter images helpful
Artifact	`.iso` that boots in QEMU + branding checklist

Why this project exists

A distro people can **boot** beats a distro that only exists as `nixosConfigurations.laptop`. ISO generation connects:

- Module graph (identity + themes + installer profile)
- Image format (`system.build.isoImage`)
- Runtime verification (QEMU)

You will also learn why **theme options differ by display manager** and why console branding is the reliable baseline.

Prerequisites

- Project 5 green (`myDistro` identity, logo, PRODUCT.md).
- Disk space: plan for multi-GB store growth.
- KVM recommended:

```
ls /dev/kvm && echo "KVM ok"
```

- Patience: first ISO eval/build is the slowest.

Theory / architecture

```
flake.nix
  nixosConfigurations.iso

          installer profile (minimal or graphical)
          modules/identity.nix     (Project 5)
          modules/themes.nix       (this project)


  config.system.build.isoImage


  result/iso/*.iso  →  QEMU / VirtualBox / USB
```

Approach	Notes
`modulesPath` installer CD profiles	Classic; well documented
`nixos-generators format = "iso"`	Same modules, format abstraction (the images/generators chapter)
Graphical Calamares/GNOME profiles	Heavier; better greeter experiments

Theme layers (cheap → expensive)

Layer	Cost	Reliability in QEMU
Console font/colors + issue	Low	High
ISO volume label / isoName	Low	High
Plymouth stock theme	Medium	Medium (needs quiet splash path)
Custom Plymouth package	High	Medium

Layer	Cost	Reliability in QEMU
GDM/SDDM logos	Medium	Option names move across pins

Lab strategy: nail console + os-release + volume label first; add Plymouth/greeter after the ISO boots cleanly.

Lab tree (extends Project 5)

```
my-distro/
  PRODUCT.md
  flake.nix
  flake.lock
  branding/
    logo.svg
    README.md
  modules/
    identity.nix
    themes.nix
  hosts/
    lab.nix
    iso.nix
  NOTES-ISO.md          # build time, size, QEMU line
```

Step 1 — ISO host configuration

`hosts/iso.nix`:

```
{ config, pkgs, lib, modulesPath, ... }: {
  imports = [
    "${modulesPath}/installer/cd-dvd/installation-cd-minimal.nix"
    # Graphical experiment (much larger) - pick one profile only:
    # "${modulesPath}/installer/cd-dvd/installation-cd-graphical-calamares-gnome.nix"
    ../modules/identity.nix
    ../modules/themes.nix
  ];

  myDistro.enable = true;
  myDistro.name = "MyLabOS";
```

```
  myDistro.id = "mylabos";
  myDistro.version = "26.05.0-iso1";

  # Live image ergonomics
  networking.wireless.enable = lib.mkForce false; # avoid iwd/wpa fights on minimal ISOs if
  services.openssh.enable = true;

  # Installer images often define a `nixos` user - keep passwords per template docs.
  # Do not bake production secrets into public ISO flakes.

  # ISO metadata - confirm option names on your 26.05 pin if eval fails
  isoImage.volumeID = lib.mkDefault "MYLABOS_2605";
  isoImage.isoName = lib.mkDefault "mylabos-26.05-x86_64.iso";
}
```

> **Note**
>
> Exact attribute names under `isoImage.*` can differ slightly by release. If eval errors, `nixos-option` / source grep on your pin and adjust. Volume label is a branding surface either way.

Step 2 — Themes module

`modules/themes.nix`:

```
{ config, lib, pkgs, ... }: {
  # --- Console branding (do this first) ---
  console = {
    font = "Lat2-Terminus16";
    colors = [
      # 16-color palette, hex without leading #
      "1d1e28" "ff5555" "50fa7b" "f1fa8c"
      "bd93f9" "ff79c6" "8be9fd" "f8f8f2"
      "6272a4" "ff6e6e" "69ff94" "ffffa5"
      "d6acff" "ff92df" "a4ffff" "ffffff"
    ];
  };

  # --- Plymouth (optional; enable after minimal ISO works) ---
  # boot.plymouth.enable = true;
  # boot.plymouth.theme = "spinner";
  # boot.plymouth.themePackages = [ ]; # custom later
```

```
  # --- Graphical greeter hooks (only on graphical installer profiles) ---
  # Option paths vary by DM + nixpkgs revision. Verify before relying on them.
  # services.displayManager.sddm… / services.xserver.displayManager.gdm…
  # services.xserver.displayManager.gdm.logo = ./../branding/logo.png;

  environment.systemPackages = with pkgs; [
    git
    vim
    # fastfetch or neofetch - nice for DEMO screenshots
  ];
}
```

Read book notes: branding/plymouth-theme-notes.md.

> **Tip**
>
> If greeter logo options moved on your pin, **do not block** the project. Ship console + os-release + volume label + logo under `/etc/<id>/`, and document greeter as stretch.

Step 3 — Flake output

```
{
  description = "MyLabOS ISO - Project 6";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.iso = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [ ./hosts/iso.nix ];
    };

    # Optional convenience package-like output via apps/scripts in README
  };
}
```

Build

```
nix build -L .#nixosConfigurations.iso.config.system.build.isoImage
ls -lh result/iso/
```

Record in `NOTES-ISO.md`:

- Wall-clock time
- ISO size
- nixpkgs rev from `flake.lock`
- Exact build command

Alternative: nixos-generators

```
# sketch - pin nixos-generators and follows nixpkgs
# packages.x86_64-linux.iso = nixos-generators.nixosGenerate {
#   system = "x86_64-linux";
#   format = "iso";
#   modules = [ ./hosts/iso.nix ];
# };
```

Prefer **one** path for the lab so debugging stays sane.

Step 4 — Boot test (QEMU)

```
ISO=$(ls result/iso/*.iso | head -1)
qemu-system-x86_64 \
  -m 2048 \
  -cdrom "$ISO" \
  -boot d \
  -enable-kvm \
  -serial mon:stdio
```

Without KVM, drop `-enable-kvm` and expect slower boots; raise `-m` if the graphical profile OOMs.

VirtualBox / UTM

Import the ISO as an optical drive, boot order CD first. Same verification checklist.

Verification checklist (live session)

Check	Pass?	Notes
Boots to installer/live environment		
`cat /etc/os-release` shows product name		

Check	Pass?	Notes
`cat /etc/issue` branded		
Console palette/font applied		
Logo present under `/etc/<id>/`		
SSH or local login works		
ISO volume label visible (where shown)		
Plymouth (if enabled)		optional

```
# inside live system
cat /etc/os-release
cat /etc/issue
ls /etc/mylabos
```

Step 5 — Branding matrix

Copy into `NOTES-ISO.md` and fill status:

Surface	Status	How verified
os-release / issue		
Console colors/font		
ISO volumeID / isoName		
Logo on live FS		
Plymouth	skipped / ok	
Installer wallpaper/logo	skipped / ok	

Acceptance for Project 6: branding visible on **2 surfaces** in the live environment, ISO boots, build command documented.

Step 6 — Iterate without tears

ISO rebuilds are expensive. Iteration tips:

1. Test **identity module** on `build-vm` / normal host first (Project 5).
2. Change **one** theme knob per ISO rebuild when debugging.

3. Keep a `hosts/iso-minimal.nix` and `hosts/iso-graphical.nix` if you experiment—do not thrash a single file blindly.

4. Use binary caches; do not disable substitutes "for purity theater" on ISO day.

5. Tag git when an ISO is good: `iso-26.05.0-iso1-green`.

```
# eval-only sanity (faster than full ISO when checking option typos)
nix eval .#nixosConfigurations.iso.config.networking.hostName
nix eval .#nixosConfigurations.iso.config.myDistro.name
```

Plymouth (optional after minimal ISO is green)

```
boot.plymouth.enable = true;
boot.plymouth.theme = "spinner";
# later: themePackages + custom theme derivation; test only in a fat VM
```

Timebox custom Plymouth—failures are expensive.

Pitfalls

Symptom	Likely cause	Fix
Out of disk	`/nix` full	Free tens of GB; careful GC
QEMU hang	Wrong arch / low RAM	Match `system`; raise `-m`
Unbranded live	Modules not on iso host	Fix `imports`
`isoImage.*` unknown	Pin difference	Grep installer modules
Graphical ISO never ends	Huge closure	Minimal profile for green

---

Acceptance criteria

☐ ISO builds (`system.build.isoImage` or generators).

☐ VM boots live/installer environment.

- ☐ Branding on **2** surfaces.
- ☐ Build command, size, time in `NOTES-ISO.md`.
- ☐ ISO version tag (`-iso1`) noted; no production secrets.

Stretch goals

1. Graphical installer + greeter logo.
2. Custom Plymouth from `logo.svg`.
3. Generators: qcow2 **and** iso from same modules.
4. Serial QEMU smoke grepping os-release.

Self-check

1. Why console theming first?
2. Which derivation path built the ISO?
3. How does Project 5 prevent ISO-only drift?
4. What do you drop first under a 30-minute CI budget?

Next: Project 7 — Complete distro product.

Project 7: Complete custom distro product

Project 7 — Ship a complete NixOS-based distro flake

Goal: Assemble a **product-shaped** flake: branded identity, at least two editions (minimal + workstation **or** server), installer ISO, pinned nixpkgs **26.05**, documented install path, first-boot defaults, and a demo script—your own distro in **engineering** terms (not a legal claim on the NixOS trademark).

Field	Value
Baseline	Nix **2.34**, NixOS / nixpkgs **26.05**
Time	~4–12 hours (integration)
Depends on	Projects 5–6; packaging Projects 1–4 optional but enriching
Artifact	Public-ready README + DEMO.md + multi-output flake

> **!** Important
>
> **Trademark:** "NixOS" is a trademark of the NixOS Foundation. Your derivative must use a **distinct name** and follow NixOS branding guidelines when mentioning the base. This project teaches reproducible OS assembly—not unofficial branding abuse.

Why this project exists

Projects 5–6 create identity and an ISO. Productization means:

- **Editions** as module bundles (not copy-pasted hosts).
- **Channels of truth**: flake lock + docs, not tribal memory.
- **Install + upgrade story** a stranger could follow.
- Honest boundary: what is **your** policy vs upstream NixOS.

Treat this like a tiny open-source OS product even if only you use it.

Product definition of done

#	Requirement
1	Distinct distro name + version in `PRODUCT.md`
2	`flake.lock` pins `nixos-26.05` (or documented stable choice)
3	`nixosConfigurations` for `iso` **and** at least one installed host profile
4	Editions: e.g. `minimal` and `desktop` **or** `server` module bundles
5	Branding on ISO + installed system path (Projects 5–6)
6	Defaults module: flakes-on, explicit unfree policy, nix settings
7	README: build ISO, install notes, upgrade story
8	`DEMO.md` with commands + evidence list
9	Optional: CI stub (`nix flake check` / ISO build in GA)

Target tree

```
my-distro/
  PRODUCT.md
  README.md
  DEMO.md
  LICENSE                 # for *your* modules/assets
  flake.nix
  flake.lock
  branding/
    logo.svg
    README.md
  modules/
    identity.nix
    themes.nix
    defaults-nix.nix
    edition-minimal.nix
    edition-desktop.nix   # or edition-server.nix
  hosts/
    iso.nix
    demo-vm.nix
  packages/
    default.nix           # optional distro toolkit
  checks/                 # optional
    default.nix
```

Merge everything you already built in Projects 5–6; refactor names for consistency.

Theory: editions vs hosts

```
modules/edition-*.nix   =  product SKUs (what kind of system)
hosts/*.nix             =  concrete machines / images (where it runs)
```

Layer	Example
Edition	minimal SSH appliance defaults
Host	`demo-vm` imports minimal + disk/boot for QEMU
Image	`iso` imports minimal + installer profile + themes

Do **not** put disk device paths inside edition modules.

Step 1 — Defaults module (flakes-first)

`modules/defaults-nix.nix`:

```nix
{ lib, ... }: {
  nix = {
    settings = {
      experimental-features = [ "nix-command" "flakes" ];
      # trusted-users: keep tight on real products
      # substituters / trusted-public-keys: document if you run a cache
      auto-optimise-store = lib.mkDefault true;
    };
    # gc.automatic is a product policy - enable only with thought
  };

  # Product policy: be explicit
  nixpkgs.config.allowUnfree = lib.mkDefault false;

  # Prefer flake-pinned nixpkgs over channels in docs and defaults
  # (channel-less mental model for users of your distro)
}
```

Document in README: “This product is flakes-first; classic channels are not the support path.”

Step 2 — Edition modules

Minimal

`modules/edition-minimal.nix`:

```
{ lib, pkgs, ... }: {
  imports = [
    ./identity.nix
    ./defaults-nix.nix
  ];

  myDistro.enable = lib.mkDefault true;

  services.openssh.enable = lib.mkDefault true;
  networking.firewall.enable = lib.mkDefault true;

  environment.systemPackages = with pkgs; [
    vim
    git
    curl
  ];

  # Users: force consumers to define real users on hosts
}
```

Desktop (or server)

`modules/edition-desktop.nix`:

```
{ lib, pkgs, ... }: {
  imports = [
    ./edition-minimal.nix
    ./themes.nix
  ];

  # Pick ONE desktop stack and stick to it for the lab
  services.xserver.enable = true;
  services.displayManager.gdm.enable = true;   # verify attr path on 26.05
  services.desktopManager.gnome.enable = true; # or plasma6 / something you know

  networking.networkmanager.enable = true;

  environment.systemPackages = with pkgs; [
    firefox
  ];
```

```
}
```

> **Warning**
>
> Display manager option paths have moved across releases (`services.xserver.displayManager` vs `services.displayManager`, GNOME/Plasma module names). **Eval on your pin** and adjust. The product lesson is edition composition, not memorizing one GNOME attr forever.

Server alternative: skip GUI; add fail2ban posture notes, unattended-upgrade policy **or** flake-based upgrade docs only.

Step 3 — Hosts

Demo VM / installed profile

`hosts/demo-vm.nix`:

```
{ lib, modulesPath, ... }: {
  imports = [
    (modulesPath + "/virtualisation/qemu-vm.nix") # if using build-vm patterns
    ../modules/edition-minimal.nix
    # ../modules/edition-desktop.nix
  ];

  myDistro.enable = true;
  myDistro.name = "MyLabOS";
  myDistro.version = "26.05.0-prod1";

  networking.hostName = "mylabos-demo";

  users.users.demo = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
    # initialPassword only for disposable lab VMs - never for public ISOs if weak
    initialPassword = "demo"; # LAB ONLY
  };

  # boot.loader + fileSystems: required for real metal; qemu-vm module helps for demos
}
```

ISO host

Keep Project 6 `hosts/iso.nix` importing identity, themes, installer profile, and **edition-minimal** (not full desktop unless you accept huge ISOs).

Step 4 — Flake product surface

```
{
  description = "MyLabOS - NixOS-based lab distro";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }:
    let
      system = "x86_64-linux";
      pkgs = nixpkgs.legacyPackages.${system};
    in {
      nixosConfigurations = {
        iso = nixpkgs.lib.nixosSystem {
          inherit system;
          modules = [ ./hosts/iso.nix ];
        };
        demo-vm = nixpkgs.lib.nixosSystem {
          inherit system;
          modules = [ ./hosts/demo-vm.nix ];
        };
      };

      packages.${system} = {
        # optional toolkit
        mylabos-tools = pkgs.callPackage ./packages/default.nix { };
      };

      # formatter, checks, devShells - add as you harden the product
      checks.${system}.demo-vm-eval =
        self.nixosConfigurations.demo-vm.config.system.build.toplevel;
    };
}
```

```
nix flake show
nix build -L .#nixosConfigurations.demo-vm.config.system.build.toplevel
nix build -L .#nixosConfigurations.iso.config.system.build.isoImage
nix flake check   # if checks are wired
```

Step 5 — Optional metapackage (toolkit)

`packages/default.nix`:

```
{ symlinkJoin, hello, git, vim }:

symlinkJoin {
  name = "mylabos-tools";
  paths = [ hello git vim ];
  meta.description = "Convenience toolkit metapackage for MyLabOS demos";
}
```

Ship only if it clarifies the product; empty packages/ is fine.

Step 6 — Install story (document, then test once)

Write this in README, then execute **once** on a disposable VM disk:

1. Boot ISO (Project 6).
2. Partition (manual or Disko—advanced).
3. Install from your flake:

```
# Conceptual - exact installer flow varies by profile and version.
# Prefer copying the flake onto the live system or using a reachable git remote.

sudo nixos-install --flake /path/to/my-distro#demo-vm

# or, if published:
# sudo nixos-install --flake github:you/my-distro#demo-vm
```

4. Reboot into installed system.
5. Verify branding (`os-release`, issue) and flakes defaults.
6. Exercise upgrade path:

```
# on a machine that tracks your flake
cd /etc/nixos   # or your flake path
nix flake update  # policy: when and how you allow this
```

```
sudo nixos-rebuild switch --flake .#demo-vm
```

Document **failure modes**: wrong host attr, missing boot loader, network for first download.

Step 7 — README + DEMO.md (required)

README minimum: what it is (NixOS-based, trademark-safe), quick start, build ISO, QEMU, install, editions, upgrade policy, branding/license, upstream vs yours, limitations. Paste the **exact** commands that worked.

DEMO.md minimum:

1. Build ISO command
2. QEMU one-liner
3. Evidence list (boot, issue, `os-release`, optional fetch)
4. Upgrade: bump flake input → rebuild
5. Limitations (hardware, Secure Boot, blobs, not a Foundation release)

Run the demo once end-to-end before calling Project 7 green.

Step 8 — Channels of truth

Source	Content
`flake.lock`	Exact nixpkgs (and other inputs)
`PRODUCT.md`	Name, version, release notes
`README.md`	Operator docs
`modules/*`	Behavior
CI (optional)	Prevents silent breakage

Avoid a second brain of classic channels. Binary caches are **optimization**, not identity.

Pitfalls

Symptom	Likely cause	Fix
ISO branded, install not	Host forgot identity import	Fix import chain
Desktop never evals	DM option names on 26.05	Align to pin
`nixos-install` misses flake	Path/remote	Copy flake into live image
Weak password on public ISO	Lab password leaked into iso host	Demo-only secrets
Scope explosion	Out-featuring NixOS	Two editions only

Acceptance criteria

- ☐ Two editions as modules (minimal + desktop/server).
- ☐ ISO **and** demo host configs build.
- ☐ Branding on install path **or** dual proof (live ISO + demo VM) with evidence.
- ☐ README + DEMO.md + PRODUCT.md complete.
- ☐ Trademark-safe naming; flakes on; unfree policy explicit.
- ☐ `flake.lock` committed.
- ☐ Note: what you would "sell" vs what remains upstream NixOS.

Stretch goals

1. Calamares-branded graphical installer.
2. Binary cache + documented substituters.
3. `deploy-rs` / Colmena for two demo VMs.
4. `nixosTests` grepping `PRETTY_NAME`; optional GA `nix flake check`.

5. Ship Project 1–4 packages under `packages/`.

Capstone reflection (½ page)

In `PRODUCT.md` / `RETRO.md`: what is yours vs forever-upstream; what breaks first on the next stable bump; would you trust this on a travel laptop tomorrow?

Self-check

1. Edition vs host vs image in *your* tree.
2. Upgrade channel of truth?
3. How ISO and installed systems avoid drift?
4. Fifteen-minute demo for a friend?

Done with the projects track

Return to the Projects index and tick the global bar. Optional: fold into capstone days 83–90 or publish the flake as a portfolio piece labeled as a lab derivative.

Branding

Sample distro branding assets (lab only)

These files are **teaching placeholders** for Projects 5–7 (custom NixOS-based distro).

File	Use
`logo.svg`	System `environment.etc`, greeter/docs mockups, ISO branding notes
`os-release.fragment.example`	Fields to merge into identity module experiments
`plymouth-theme-notes.md`	How to think about splash themes without vendoring huge assets

License

Original simple geometry in `logo.svg` is provided for this monorepo as **CC0-1.0** (public domain dedication) so you can remix freely for personal labs.

Trademark

Do **not** reuse official NixOS snowflake artwork as your distro mark. Use a **distinct name** (e.g. `MyNixOS`, `YararaLab`) and follow NixOS branding guidance when mentioning upstream.

Wire into a flake

```
environment.etc."mynixos/logo.svg".source = ./branding/logo.svg;
```

Or copy into your own repo:

```
cp -r path/to/books/NixOS/content/99-projects/branding ~/src/my-distro/branding
```

Plymouth / greeter theme notes (lab)

Goal

Visible **product identity** on boot/login without shipping megabytes of third-party themes you do not license.

Practical path on 26.05

1. **Console first** — `console.colors`, `console.font`, `/etc/issue` (Projects 5–6). Cheap and reliable in QEMU.
2. **Plymouth** — `boot.plymouth.enable = true` with a **stock** theme (`spinner`, etc.) while learning. Custom themes = package a theme derivation later.
3. **Greeter logo** — option names depend on GDM/SDDM/LightDM and nixpkgs revision. Verify on your pin; if missing, set wallpaper via Home Manager on installed systems.
4. **ISO volume label** — `isoImage.volumeID` / `isoImage.isoName` when using installer profiles.

Custom Plymouth (stretch)

- Theme = directory of script + images under the Plymouth theme path.
- Package with `stdenv.mkDerivation` installing into `$out/share/plymouth/themes/...`.
- Reference via `boot.plymouth.themePackages` + `boot.plymouth.theme`.
- Test only in a VM with enough RAM; Plymouth failures are painful on metal.

Assets in this folder

Use `logo.svg` as a greeter/docs mark. Convert to PNG if a greeter requires bitmap:

```
# if rsvg-convert available
rsvg-convert -w 256 logo.svg -o logo.png
```

Appendices

Glossary of Terms

Glossary of Terms

A concise reference for Nix and NixOS terminology used in this book.

Derivation (`.drv`)

The build specification Nix evaluates to before any compiler runs. An immutable file in `/nix/store` listing the builder, environment, and input paths required to produce an output.

Nix Store (`/nix/store`)

Read-only tree of hashed paths. Identity is the hash; the name-version suffix is a label. Nothing is overwritten in place.

Closure

The store path plus every path it references, transitively. The **runtime** closure is what you copy to production. The **build-time** closure includes compilers and is not what you deploy.

Reference scanning

Nix greps output files for other store hashes. A path mentioned in a binary becomes a runtime reference; gcc used only during the compile does not.

Flake

`flake.nix` + `flake.lock`: hermetic evaluation, pinned inputs, standard outputs (`packages`, `devShells`, `nixosConfigurations`, `checks`).

DevShell

An isolated environment from `nix develop` / `mkShell` with exactly the tools the project listed.

Binary cache / substituter

HTTP service (`cache.nixos.org`, Cachix, Attic, Harmonia) that serves signed NARs. Nix queries substituters before building.

NAR

Nix ARchive: the serialisation of a store path used for copy and substitute.

GC root

A pointer outside the store (`/run/current-system`, `~/.nix-profile`, `result`, `.direnv/flake-profile`) that keeps a closure alive. Unreachable paths are garbage.

Home Manager

NixOS module system aimed at `$HOME`: user packages, dotfiles, user services.

Overlay

Function `final: prev: { … }` that patches `nixpkgs` without editing upstream.

Module system

Typed `options` + merging `config`. Powers NixOS and Home Manager.

Fixed-output derivation (FOD)

A derivation allowed to use the network whose **output hash** you declared. Fetchers (`fetchurl`, `fetchFromGitHub`) are FODs. Identity is the hash, not the URL.

RUNPATH

ELF field pointing at store library directories. Replaces a global `/lib` search.

nix-ld

NixOS compatibility loader at `/lib64/ld-linux-…` so vendor ELFs can find Nix-provided libraries without a global `LD_LIBRARY_PATH`.

FHS env (`buildFHSEnv`)

Bubblewrapped filesystem that looks like `/usr` for vendor installers that refuse to run on a NixOS host.

Remote builder

A machine that realises `.drv` files over SSH (`ssh-ng`). Caches move results; builders move work.

stateVersion

Birth release of a NixOS or Home Manager config. Not an upgrade knob. Leave it until a release note says otherwise.

Generation

One successful `nixos-rebuild switch` or `home-manager switch`. The bootloader (and `home-manager generations`) lists them for rollback.

Impermanence

Root filesystem thrown away at boot; state lives on `/persist`. Requires backups of `/persist`, not of `/nix/store`.

Command Cheat Sheet

Command Cheat Sheet

Daily commands for developing, building, and operating Nix and NixOS.

Flakes and development

Command	Purpose
`nix develop`	Default project shell
`nix develop .#backend`	Named shell
`nix flake init`	Skeleton flake
`nix flake check`	Eval + `checks.*`
`nix flake update`	Refresh every input in `flake.lock`
`nix flake update nixpkgs`	Refresh one input
`nix flake show`	Outputs in this flake
`nix flake metadata`	Locked revisions
`nix fmt`	Formatter if `formatter` is set

Build, run, eval

Command	Purpose
`nix build .#pkg`	Build; symlink `./result`
`nix run .#app`	Build and run
`nix eval .#attr`	Print a value
`nix log ./result`	Build log
`nix repl .#`	REPL on this flake

Inspect closures

Command	Purpose
`nix path-info -Shr ./result`	Closure size (the number that matters)
`nix path-info ./result`	Output path only
`nix-store -q --references P`	Direct runtime edges
`nix-store -q --requisites P`	Full runtime closure
`nix-store -q --referrers P`	Who points at `P`

Command	Purpose
`nix-store -q --tree P`	ASCII graph
`nix why-depends A B`	Why `A`'s closure contains `B`
`nix-tree P`	Interactive tree (`nix run nixpkgs#nix-tree`)
`nom build .#pkg`	`nix build` with a progress table

Fetchers

Command	Purpose
`nix-prefetch-url URL`	Hash a file (packed)
`nix-prefetch-url --unpack URL`	Hash an unpacked archive
`nix hash convert --to sri HASH`	Hex → SRI `sha256-…`

Copy and caches

Command	Purpose
`nix copy --to file://$PWD/nar P`	Directory cache
`nix copy --to ssh://host P`	Copy closure over SSH
`cachix push desk-cache ./result`	Upload to Cachix
`nix show-config | grep substituters`	Which caches this daemon queries

Remote builders

Command	Purpose
`nix build .#pkg --max-jobs 0 --builders 'ssh-ng://user@host system'`	Realise only remotely
`sudo ssh -i /root/.ssh/id_builder user@host nix-store --version`	Daemon SSH must work without a password

NixOS

Command	Purpose
`sudo nixos-rebuild switch --flake .#host`	Build, activate, set boot default
`sudo nixos-rebuild test --flake .#host`	Activate without changing the boot default
`sudo nixos-rebuild boot --flake .#host`	Next reboot only

Command	Purpose
`sudo nixos-rebuild dry-activate --flake .#host`	Show unit changes, do nothing
`sudo nixos-rebuild switch --rollback`	Previous generation
`sudo nix-env --list-generations --profile /nix/var/nix/profiles/system`	Generation list
`nixos-version`	Running release

Home Manager

Command	Purpose
`home-manager switch --flake .#user`	Standalone user env
`home-manager generations`	User generations
`home-manager switch --rollback`	Previous user generation (standalone only)

GC and disk

Command	Purpose
`nix-store --gc --print-roots`	What is keeping paths alive
`nix-collect-garbage`	Delete unreferenced paths; keep generations
`nix-collect-garbage --delete-older-than 14d`	Drop old generations, then orphans
`sudo nix-collect-garbage -d`	All old generations (incident)
`nix-store --optimise`	Hard-link identical files
`nix-store --verify --check-contents`	Rehash after disk trouble
`df -h /nix /boot`	Store vs ESP — two different full-disk incidents

Backup (restic)

Command	Purpose
`sudo systemctl start restic-backups-desk`	Run the NixOS restic unit now
`restic snapshots`	List snapshots (needs `-r` and password file)
`restic restore latest --target /tmp/drill`	Restore drill

Nix and NixOS Version Reference

Nix and NixOS Version Reference

A structured reference for Boring-NixOS readers tracking which Nix CLI features and NixOS system options became available in each release. Coverage spans **Nix 2.18 – 2.35** and **NixOS 23.05 – 26.05**. This book's examples target **Nix 2.35** and **NixOS 26.05 "Yarara"**. Earlier rows are history so you can read old flakes.

Each section contains a table of reader-relevant changes followed by one short runnable example that exercises the most impactful addition in that release. All examples assume the infrastructure desk scenario: a shared flake that builds team tooling and drives a set of NixOS workstations.

Nix CLI Releases

Nix 2.18

Area	Change
`nix flake metadata`	Prints flake description, inputs, and their resolved URLs; last-modified timestamp now shown per input.
`--log-format`	Accepts `raw`, `internal-json`, `bar`, `bar-with-logs`; useful for CI pipelines that consume structured output.
Evaluator	`builtins.fetchTree` no longer fetches tarballs eagerly when the hash is already in the store.
Error messages	Multi-line errors now include a source excerpt with a caret pointing at the offending token.

Key feature — `nix flake metadata`

Save as `show-meta.sh`:

```
# show-meta.sh
nix flake metadata \
  --log-format bar \
  github:NixOS/nixpkgs/nixos-26.05
```

Run:

```
bash show-meta.sh
```

Output:

```
Resolved URL:  github:NixOS/nixpkgs/nixos-26.05
Locked URL:    github:NixOS/nixpkgs/a1b2c3d4e5f6...
Description:   Nix Packages collection & NixOS
Path:          /nix/store/xxxxxxxx-source
Last modified: 2024-11-30 12:00:00
Inputs:
  nixpkgs: (this flake)
```

Nix 2.19

Area	Change
`nix store gc`	Replaces `nix-collect-garbage` progressively; supports `--max-freed <bytes>` to cap space reclaimed per run.
`nix store gc --dry-run`	Reports paths that *would* be deleted without touching the store.
`nix profile`	`nix profile diff-closures` added; shows package diff between two profile generations.
Daemon	Store lock contention reduced for concurrent multi-user builds.

Key feature — `nix store gc --dry-run`

Save as `gc-preview.sh`:

```
# gc-preview.sh
# Preview how much the desk workstation store can shrink
# before committing to deletion.
nix store gc --dry-run --max-freed $((10 * 1024 * 1024 * 1024))
```

Run:

```
bash gc-preview.sh
```

Output:

```
Would delete: /nix/store/aaa...-old-toolchain-2023
Would delete: /nix/store/bbb...-devshell-2023-11
...
Would free 7.4 GiB (limit was 10.0 GiB)
```

Nix 2.20

Area	Change
`nix flake archive`	Copies all flake inputs into a binary cache or a local store, enabling fully offline evaluation afterwards.
`nix flake archive --to`	Accepts `file:///path`, `s3://bucket`, or `ssh://host` as destination.
`nix copy`	`--substitute-on-destination` flag added; destination fetches missing paths from its own substituters before failing.
Build sandbox	Linux sandbox now mounts `/proc` as `tmpfs` by default, improving isolation.

Key feature — `nix flake archive` to a local mirror

Save as `archive-inputs.sh`:

```
# archive-inputs.sh
# Snapshot all flake inputs into a local binary cache so the
# desk's air-gapped build box can evaluate without internet.
nix flake archive \
  --to file:///srv/nix-cache \
  /home/ops/desk-infra
```

Run:

```
bash archive-inputs.sh
```

Output:

```
archiving input 'nixpkgs'
  /nix/store/xxxxxxxx-source → file:///srv/nix-cache
archiving input 'home-manager'
  /nix/store/yyyyyyyy-source → file:///srv/nix-cache
done - 2 inputs archived
```

Nix 2.21

Area	Change
`nix eval --apply`	Applies a Nix function to the evaluated result before printing; enables one-liner transformations.
`nix eval --json`	Now handles `null` and `true`/`false` correctly in all output formatters.
`nix flake update`	`--commit-lock-file` flag commits the updated `flake.lock` with a machine-generated commit message.
Repl	`:reload` re-reads the current file without restarting the session.

Key feature — `nix eval --apply`

Save as `list-packages.sh`:

```
# list-packages.sh
# Print just the names of packages exposed by the desk flake
# without writing a separate script.
nix eval \
  /home/ops/desk-infra#packages.x86_64-linux \
  --apply 'pkgs: builtins.attrNames pkgs' \
  --json
```

Run:

```
bash list-packages.sh
```

Output:

```
["desk-tools","lint-runner","release-packager"]
```

Nix 2.22

Area	Change
`nix build --rebuild`	Rebuilds a derivation even if its output is in the store, then compares the new result to the cached one — a fast reproducibility check.
`nix copy --from` / `--to`	Both flags marked stable; no longer experimental.
`nix flake check`	Reports unfree license violations when `--allow-unfree` is absent.

Area	Change
Evaluator	`builtins.readFileType` added: returns `"regular"`, `"directory"`, `"symlink"`, or `"unknown"`.

Key feature — `nix build --rebuild`

Save as `verify-repro.sh`:

```
# verify-repro.sh
# Rebuild the desk tool and confirm the output hash is stable.
# A mismatch here means a non-deterministic build is lurking.
nix build \
  /home/ops/desk-infra#packages.x86_64-linux.desk-tools \
  --rebuild
```

Run:

```
bash verify-repro.sh
```

Output (reproducible):

```
warning: performing a check build
[3/0/1 built] building desk-tools-1.4.0...
/nix/store/abc123...-desk-tools-1.4.0 (deterministic )
```

Output (non-reproducible):

```
warning: performing a check build
error: derivation '/nix/store/...-desk-tools-1.4.0.drv' may not
       be deterministic: output '/nix/store/abc123...' differs
       from '/nix/store/def456...'
```

Nix 2.23

Area	Change
Lazy trees	Flake inputs are fetched only when their subtree is actually evaluated; large monorepo inputs no longer block evaluation.
`nix fmt`	Declared stable; runs the formatter specified in `flake.nix` `formatter` output.

Area	Change
`nix repl :doc`	Prints the docstring attached to any Nix function or built-in.
`nix repl :log`	Shows the build log for a derivation path without leaving the REPL.
`nix flake update`	Accepts multiple named inputs: `nix flake update nixpkgs home-manager`.

Key feature — `nix fmt` enforced in CI

Save as `fmt-check.sh`:

```
# fmt-check.sh
# Fail CI if any Nix file in the desk flake needs reformatting.
nix fmt /home/ops/desk-infra -- --check
```

Run:

```
bash fmt-check.sh
```

Output (clean):

```
All Nix files already formatted.
```

Output (needs formatting):

```
Would reformat: /home/ops/desk-infra/modules/workstation.nix
error: 1 file would be reformatted
```

Nix 2.24

Area	Change
`nix path-info --json`	Output is now a structured JSON array with `path`, `narHash`, `narSize`, `references`, `registrationTime`, and `deriver` fields.
`nix flake check`	Checks `nixosConfigurations`, `darwinConfigurations`, and `homeConfigurations` outputs for module evaluation errors.
`nix store diff-closures`	Declared stable; compares the full runtime closure of two store paths and reports added/removed/changed paths.

Area	Change
`nix build --print-build-logs`	Default changed to `true` in interactive terminals.

Key feature — `nix store diff-closures`

Save as `diff-release.sh`:

```
# diff-release.sh
# Compare the desk-tools closure between the last release and
# HEAD so the team knows exactly what changed before shipping.
OLD=$(nix build \
  "/home/ops/desk-infra?ref=v1.3.0#packages.x86_64-linux.desk-tools" \
  --no-link --print-out-paths)

NEW=$(nix build \
  /home/ops/desk-infra#packages.x86_64-linux.desk-tools \
  --no-link --print-out-paths)

nix store diff-closures "$OLD" "$NEW"
```

Run:

```
bash diff-release.sh
```

Output:

```
desk-tools: 1.3.0 → 1.4.0
openssl: 3.2.1 → 3.2.2 (+12 KiB)
python3: 3.11.8 → 3.12.3 (+1.1 MiB)
curl: (unchanged)
added: zlib 1.3.1
```

NixOS Releases

NixOS 23.05 — Stoat

Area	Change
systemd	253; `systemd-oomd` enabled by default on desktop profiles.
Python	3.11 is the default interpreter; `python3` symlink resolves to `python3.11`.

Area	Change
Nix	Ships Nix 2.13; flake support via `nix.settings.experimental-features` is opt-in stable.
Module system	`config.system.stateVersion` warning added for stale values.
Security	`services.fail2ban` updated to 1.0; new `ignoreIP` list option.

Key feature — enabling flakes on a Stoat workstation

Save as `configuration.nix`:

```
# configuration.nix
{ pkgs, ... }:
{
  # Enable the two features that make flakes and the new nix
  # command available on every desk workstation running 23.05.
  nix.settings.experimental-features = [
    "nix-command"
    "flakes"
  ];

  # Pin the channel so every workstation evaluates identically.
  nix.registry.nixpkgs.flake = import <nixpkgs> { };

  environment.systemPackages = [ pkgs.git ];
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
building the system configuration...
activating the configuration...
setting up /etc...
the following new units were started: nix-daemon.service
```

NixOS 23.11 — Tapir

Area	Change
GNOME	45; Mutter gains explicit sync support for tear-free rendering on variable-refresh monitors.
PHP	8.2 is default; `services.phpfpm` pools now set `pm.max_spare_servers` automatically.
Kernel	6.1 LTS; default for most hardware profiles.
`services.avahi`	`openFirewall` now defaults to `false`; must be set explicitly.
Nix	Ships Nix 2.18.

Key feature — explicit Avahi firewall declaration

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # 23.11 changed the default: openFirewall is now false.
  # Desk printers and mDNS service discovery need this set
  # explicitly or discovery will silently fail.
  services.avahi = {
    enable = true;
    nssmdns4 = true;   # allow getaddrinfo to resolve .local
    openFirewall = true;
  };
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
activating the configuration...
setting up /etc...
reloading the following units: firewall.service
```

NixOS 24.05 — Uakari

Area	Change
systemd	255; `systemd-boot` gains UKI (Unified Kernel Image) signing support.

Area	Change
Python	3.12 is default; `python3.11` still available as explicit attribute.
Nix	Ships Nix 2.22 — `nix build --rebuild` and stable `nix copy` available out of the box.
`system.etc.overlay`	Experimental option; mounts `/etc` as an overlay for impermanence workflows.
`boot.initrd.systemd`	Available as an opt-in Stage-1; **not** the channel default yet (that lands in 26.05).
`services.postgresql`	`enableJIT` now defaults to `true` for 14+.

Key feature — opt-in systemd initrd (pre-26.05)

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # On 24.05 this is opt-in. 26.05 flips the default to true and
  # deprecates the scripted Stage 1. Prefer the systemd path early
  # if the desk host uses LUKS / TPM2 unlock.
  boot.initrd.systemd.enable = true;
  boot.initrd.luks.devices."cryptroot" = {
    device = "/dev/disk/by-uuid/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx";
    allowDiscards = true;
  };

  # Pin Nix to 2.22 features in this release.
  nix.settings.experimental-features = [
    "nix-command"
    "flakes"
  ];
}
```

Run:

```
sudo nixos-rebuild boot --flake /etc/nixos#desk-build
```

Output:

```
building the system configuration...
updating GRUB 2 menu...
Done. The new configuration will be used when the machine is rebooted.
```

NixOS 24.11 — Vicuna

Area	Change
GNOME	47; default font is now Cantarell 11; GNOME Color Manager updated.
Kernel	6.6 LTS; `amdgpu` gains firmware for RDNA 3.5 integrated graphics.
Nix	Ships Nix 2.24 — `nix path-info --json`, `nix store diff-closures` stable.
`services.openssh.settings`	New sub-options: `MaxSessions`, `PermitUserEnvironment`, `AllowGroups`.
`security.apparmor`	`enableCache` option added; profile compile cache speeds boot on profile-heavy machines.
Python	3.12 remains default; 3.13 available as `python313`.

Key feature — locked-down SSH with `services.openssh.settings`

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # 24.11 exposes AllowGroups directly in the settings attrset,
  # eliminating the need for extraConfig string concatenation.
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      PermitRootLogin = "no";
      MaxSessions = 4;
      AllowGroups = [ "ops" "developers" ];
    };
  };

  # AppArmor with compile cache - useful on desk machines that
  # load many profiles at boot.
  security.apparmor = {
    enable = true;
    enableCache = true;
  };
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
activating the configuration...
reloading the following units: sshd.service apparmor.service
```

Nix 2.28 – 2.34 (bridge)

Area	Change
Fetchers	Stricter NAR hash mismatches; prefer SRI `hash = "sha256-…"` over legacy `sha256`.
Flakes	Source trees copied into the store more lazily (completed in 2.35).
`nix store`	`diff-closures`, `copy`, `ping` remain the daily ops set.

Pin `nixos-25.11` only to read old machines. Do not start new desk hosts there: it reached EOL on 2026-06-30.

Nix 2.35

Area	Change
Flake sources	Copied to the store lazily; unused flake files are not hashed into evaluation unless needed.
Sandbox	FreeBSD jail sandboxing; Linux sandbox unchanged.
Installer	Current upstream installer ships 2.35.2 (2026-08).
Security	Recursive-nix advisory fix from the 2.35.0 series.

Key feature — current CLI on a 26.05 host

Stock 26.05 may print `nix (Nix) 2.34.x`. For this book's **2.35+** baseline, install the upstream Nix installer on foreign Linux/macOS, or set `nix.package = pkgs.nixVersions.latest;` (or `nix_2_35`) on NixOS. Then:

```
nix --version
nix flake metadata github:NixOS/nixpkgs/nixos-26.05
```

Output (shape, after pinning 2.35):

```
nix (Nix) 2.35.2
Resolved URL:  github:NixOS/nixpkgs/nixos-26.05
```

NixOS 25.05 / 25.11

25.05 and 25.11 are previous stables. 25.11 "Xantusia" is **deprecated** (EOL 2026-06-30). If `system.stateVersion` is `"25.11"`, **leave it** and move the flake input to `nixos-26.05`.

NixOS 26.05 — Yarara

Area	Change
Support	Bugfix/security until 2026-12-31.
Initrd	Stage-1 systemd initrd is the default (not the old scripted initrd).
Toolchain	GCC 15; GNOME 50 on desktop ISOs.
Nix	Channel ships Nix **2.34**; book CLI baseline remains **2.35+** (installer or `nix.package`).
Modules	New options; confirm names with `nixos-option` if a 24.11 snippet fails eval.

Key feature — pin 26.05 in the desk flake

Save as `flake.nix`:

```nix
# flake.nix
{
  description = "Desk workstation on NixOS 26.05";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.desk-workstation = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        {
          networking.hostName = "desk-workstation";
          system.stateVersion = "26.05";
        }
      ];
    };
  };
```

```
}
```

```
nix flake metadata
sudo nixos-rebuild dry-build --flake .#desk-workstation
```

Quick-Reference Matrix

The table below maps each NixOS release to the Nix CLI it ships and highlights the single most operationally significant change per pair.

NixOS Release	Nix Version Shipped	Most Impactful Addition
23.05 Stoat	2.13	Flakes opt-in stable via `experimental-features`
23.11 Tapir	2.18	`nix flake metadata` improvements; `--log-format` for CI
24.05 Uakari	2.22	`nix build --rebuild` reproducibility check; systemd initrd opt-in
24.11 Vicuna	2.24	`nix store diff-closures` stable; `services.openssh.settings.AllowGroups`
25.05	2.28	`nix flake check` / fetch-tree hardening; module cleanups
25.11 Xantusia	2.31	EOL 2026-06-30; last train before Yarara
26.05 Yarara	2.34 (pin 2.35+)	Current baseline; systemd stage-1 initrd default; GCC 15

Nix 2.19–2.34 intermediate releases are available via `pkgs.nixVersions` even when the channel ships another version. Pin only when you need a CLI feature the channel does not have yet:

```
# configuration.nix  (pin Nix version independently of channel)
{ pkgs, ... }:
{
  nix.package = pkgs.nixVersions.latest; # or pkgs.nixVersions.nix_2_35
}
```

Nix and NixOS Version Reference

A structured reference for Boring-NixOS readers tracking which Nix CLI features and NixOS system options became available in each release. Coverage spans **Nix 2.18 – 2.35** and **NixOS 23.05 – 26.05**. This book's examples target **Nix 2.35** and **NixOS 26.05 "Yarara"**. Earlier rows are history so you can read old flakes.

Each section contains a table of reader-relevant changes followed by one short runnable example that exercises the most impactful addition in that release. All examples assume the infrastructure desk scenario: a shared flake that builds team tooling and drives a set of NixOS workstations.

Nix CLI Releases

Nix 2.18

Area	Change
`nix flake metadata`	Prints flake description, inputs, and their resolved URLs; last-modified timestamp now shown per input.
`--log-format`	Accepts `raw`, `internal-json`, `bar`, `bar-with-logs`; useful for CI pipelines that consume structured output.
Evaluator	`builtins.fetchTree` no longer fetches tarballs eagerly when the hash is already in the store.
Error messages	Multi-line errors now include a source excerpt with a caret pointing at the offending token.

Key feature — `nix flake metadata`

Save as `show-meta.sh`:

```
# show-meta.sh
nix flake metadata \
  --log-format bar \
  github:NixOS/nixpkgs/nixos-26.05
```

Run:

```
bash show-meta.sh
```

Output:

```
Resolved URL:  github:NixOS/nixpkgs/nixos-26.05
Locked URL:    github:NixOS/nixpkgs/a1b2c3d4e5f6...
Description:   Nix Packages collection & NixOS
Path:          /nix/store/xxxxxxxx-source
Last modified: 2024-11-30 12:00:00
Inputs:
  nixpkgs: (this flake)
```

Nix 2.19

Area	Change
`nix store gc`	Replaces `nix-collect-garbage` progressively; supports `--max-freed <bytes>` to cap space reclaimed per run.
`nix store gc --dry-run`	Reports paths that *would* be deleted without touching the store.
`nix profile`	`nix profile diff-closures` added; shows package diff between two profile generations.
Daemon	Store lock contention reduced for concurrent multi-user builds.

Key feature — `nix store gc --dry-run`

Save as `gc-preview.sh`:

```
# gc-preview.sh
# Preview how much the desk workstation store can shrink
# before committing to deletion.
nix store gc --dry-run --max-freed $((10 * 1024 * 1024 * 1024))
```

Run:

```
bash gc-preview.sh
```

Output:

```
Would delete: /nix/store/aaa...-old-toolchain-2023
Would delete: /nix/store/bbb...-devshell-2023-11
...
Would free 7.4 GiB (limit was 10.0 GiB)
```

Nix 2.20

Area	Change
`nix flake archive`	Copies all flake inputs into a binary cache or a local store, enabling fully offline evaluation afterwards.
`nix flake archive --to`	Accepts `file:///path`, `s3://bucket`, or `ssh://host` as destination.
`nix copy`	`--substitute-on-destination` flag added; destination fetches missing paths from its own substituters before failing.
Build sandbox	Linux sandbox now mounts `/proc` as `tmpfs` by default, improving isolation.

Key feature — `nix flake archive` to a local mirror

Save as `archive-inputs.sh`:

```
# archive-inputs.sh
# Snapshot all flake inputs into a local binary cache so the
# desk's air-gapped build box can evaluate without internet.
nix flake archive \
  --to file:///srv/nix-cache \
  /home/ops/desk-infra
```

Run:

```
bash archive-inputs.sh
```

Output:

```
archiving input 'nixpkgs'
  /nix/store/xxxxxxxx-source → file:///srv/nix-cache
archiving input 'home-manager'
  /nix/store/yyyyyyyy-source → file:///srv/nix-cache
done - 2 inputs archived
```

Nix 2.21

Area	Change
`nix eval --apply`	Applies a Nix function to the evaluated result before printing; enables one-liner transformations.
`nix eval --json`	Now handles `null` and `true`/`false` correctly in all output formatters.
`nix flake update`	`--commit-lock-file` flag commits the updated `flake.lock` with a machine-generated commit message.
Repl	`:reload` re-reads the current file without restarting the session.

Key feature — `nix eval --apply`

Save as `list-packages.sh`:

```
# list-packages.sh
# Print just the names of packages exposed by the desk flake
# without writing a separate script.
nix eval \
  /home/ops/desk-infra#packages.x86_64-linux \
  --apply 'pkgs: builtins.attrNames pkgs' \
  --json
```

Run:

```
bash list-packages.sh
```

Output:

```
["desk-tools","lint-runner","release-packager"]
```

Nix 2.22

Area	Change
`nix build --rebuild`	Rebuilds a derivation even if its output is in the store, then compares the new result to the cached one — a fast reproducibility check.
`nix copy --from` / `--to`	Both flags marked stable; no longer experimental.
`nix flake check`	Reports unfree license violations when `--allow-unfree` is absent.

Area	Change
Evaluator	`builtins.readFileType` added: returns `"regular"`, `"directory"`, `"symlink"`, or `"unknown"`.

Key feature — `nix build --rebuild`

Save as `verify-repro.sh`:

```
# verify-repro.sh
# Rebuild the desk tool and confirm the output hash is stable.
# A mismatch here means a non-deterministic build is lurking.
nix build \
  /home/ops/desk-infra#packages.x86_64-linux.desk-tools \
  --rebuild
```

Run:

```
bash verify-repro.sh
```

Output (reproducible):

```
warning: performing a check build
[3/0/1 built] building desk-tools-1.4.0...
/nix/store/abc123...-desk-tools-1.4.0 (deterministic )
```

Output (non-reproducible):

```
warning: performing a check build
error: derivation '/nix/store/...-desk-tools-1.4.0.drv' may not
       be deterministic: output '/nix/store/abc123...' differs
       from '/nix/store/def456...'
```

Nix 2.23

Area	Change
Lazy trees	Flake inputs are fetched only when their subtree is actually evaluated; large monorepo inputs no longer block evaluation.
`nix fmt`	Declared stable; runs the formatter specified in `flake.nix` `formatter` output.

Area	Change
`nix repl :doc`	Prints the docstring attached to any Nix function or built-in.
`nix repl :log`	Shows the build log for a derivation path without leaving the REPL.
`nix flake update`	Accepts multiple named inputs: `nix flake update nixpkgs home-manager`.

Key feature — `nix fmt` enforced in CI

Save as `fmt-check.sh`:

```
# fmt-check.sh
# Fail CI if any Nix file in the desk flake needs reformatting.
nix fmt /home/ops/desk-infra -- --check
```

Run:

```
bash fmt-check.sh
```

Output (clean):

```
All Nix files already formatted.
```

Output (needs formatting):

```
Would reformat: /home/ops/desk-infra/modules/workstation.nix
error: 1 file would be reformatted
```

Nix 2.24

Area	Change
`nix path-info --json`	Output is now a structured JSON array with `path`, `narHash`, `narSize`, `references`, `registrationTime`, and `deriver` fields.
`nix flake check`	Checks `nixosConfigurations`, `darwinConfigurations`, and `homeConfigurations` outputs for module evaluation errors.
`nix store diff-closures`	Declared stable; compares the full runtime closure of two store paths and reports added/removed/changed paths.

Area	Change
`nix build --print-build-logs`	Default changed to `true` in interactive terminals.

Key feature — `nix store diff-closures`

Save as `diff-release.sh`:

```
# diff-release.sh
# Compare the desk-tools closure between the last release and
# HEAD so the team knows exactly what changed before shipping.
OLD=$(nix build \
  "/home/ops/desk-infra?ref=v1.3.0#packages.x86_64-linux.desk-tools" \
  --no-link --print-out-paths)

NEW=$(nix build \
  /home/ops/desk-infra#packages.x86_64-linux.desk-tools \
  --no-link --print-out-paths)

nix store diff-closures "$OLD" "$NEW"
```

Run:

```
bash diff-release.sh
```

Output:

```
desk-tools: 1.3.0 → 1.4.0
openssl: 3.2.1 → 3.2.2 (+12 KiB)
python3: 3.11.8 → 3.12.3 (+1.1 MiB)
curl: (unchanged)
added: zlib 1.3.1
```

NixOS Releases

NixOS 23.05 — Stoat

Area	Change
systemd	253; `systemd-oomd` enabled by default on desktop profiles.
Python	3.11 is the default interpreter; `python3` symlink resolves to `python3.11`.
Nix	Ships Nix 2.13; flake support via `nix.settings.experimental-features` is opt-in stable.
Module system	`config.system.stateVersion` warning added for stale values.
Security	`services.fail2ban` updated to 1.0; new `ignoreIP` list option.

Key feature — enabling flakes on a Stoat workstation

Save as `configuration.nix`:

```
# configuration.nix
{ pkgs, ... }:
{
  # Enable the two features that make flakes and the new nix
  # command available on every desk workstation running 23.05.
  nix.settings.experimental-features = [
    "nix-command"
    "flakes"
  ];

  # Pin the channel so every workstation evaluates identically.
  nix.registry.nixpkgs.flake = import <nixpkgs> { };

  environment.systemPackages = [ pkgs.git ];
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
building the system configuration...
activating the configuration...
setting up /etc...
the following new units were started: nix-daemon.service
```

NixOS 23.11 — Tapir

Area	Change
GNOME	45; Mutter gains explicit sync support for tear-free rendering on variable-refresh monitors.
PHP	8.2 is default; `services.phpfpm` pools now set `pm.max_spare_servers` automatically.
Kernel	6.1 LTS; default for most hardware profiles.
`services.avahi`	`openFirewall` now defaults to `false`; must be set explicitly.
Nix	Ships Nix 2.18.

Key feature — explicit Avahi firewall declaration

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # 23.11 changed the default: openFirewall is now false.
  # Desk printers and mDNS service discovery need this set
  # explicitly or discovery will silently fail.
  services.avahi = {
    enable = true;
    nssmdns4 = true;   # allow getaddrinfo to resolve .local
    openFirewall = true;
  };
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
activating the configuration...
setting up /etc...
reloading the following units: firewall.service
```

NixOS 24.05 — Uakari

Area	Change
systemd	255; `systemd-boot` gains UKI (Unified Kernel Image) signing support.
Python	3.12 is default; `python3.11` still available as explicit attribute.
Nix	Ships Nix 2.22 — `nix build --rebuild` and stable `nix copy` available out of the box.
`system.etc.overlay`	Experimental option; mounts `/etc` as an overlay for impermanence workflows.
`boot.initrd.systemd`	Available as an opt-in Stage-1; **not** the channel default yet (that lands in 26.05).
`services.postgresql`	`enableJIT` now defaults to `true` for 14+.

Key feature — opt-in systemd initrd (pre-26.05)

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # On 24.05 this is opt-in. 26.05 flips the default to true and
  # deprecates the scripted Stage 1. Prefer the systemd path early
  # if the desk host uses LUKS / TPM2 unlock.
  boot.initrd.systemd.enable = true;
  boot.initrd.luks.devices."cryptroot" = {
    device = "/dev/disk/by-uuid/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx";
    allowDiscards = true;
  };

  # Pin Nix to 2.22 features in this release.
  nix.settings.experimental-features = [
    "nix-command"
    "flakes"
  ];
}
```

Run:

```
sudo nixos-rebuild boot --flake /etc/nixos#desk-build
```

Output:

```
building the system configuration...
updating GRUB 2 menu...
Done. The new configuration will be used when the machine is rebooted.
```

NixOS 24.11 — Vicuna

Area	Change
GNOME	47; default font is now Cantarell 11; GNOME Color Manager updated.
Kernel	6.6 LTS; `amdgpu` gains firmware for RDNA 3.5 integrated graphics.
Nix	Ships Nix 2.24 — `nix path-info --json`, `nix store diff-closures` stable.
`services.openssh.settings`	New sub-options: `MaxSessions`, `PermitUserEnvironment`, `AllowGroups`.
`security.apparmor`	`enableCache` option added; profile compile cache speeds boot on profile-heavy machines.
Python	3.12 remains default; 3.13 available as `python313`.

Key feature — locked-down SSH with `services.openssh.settings`

Save as `configuration.nix`:

```
# configuration.nix
{ ... }:
{
  # 24.11 exposes AllowGroups directly in the settings attrset,
  # eliminating the need for extraConfig string concatenation.
  services.openssh = {
    enable = true;
    settings = {
      PasswordAuthentication = false;
      PermitRootLogin = "no";
      MaxSessions = 4;
      AllowGroups = [ "ops" "developers" ];
    };
  };

  # AppArmor with compile cache - useful on desk machines that
  # load many profiles at boot.
  security.apparmor = {
    enable = true;
    enableCache = true;
  };
}
```

Run:

```
sudo nixos-rebuild switch --flake /etc/nixos#workstation
```

Output:

```
activating the configuration...
reloading the following units: sshd.service apparmor.service
```

Nix 2.28 – 2.34 (bridge)

Area	Change
Fetchers	Stricter NAR hash mismatches; prefer SRI `hash = "sha256-…"` over legacy `sha256`.
Flakes	Source trees copied into the store more lazily (completed in 2.35).
`nix store`	`diff-closures`, `copy`, `ping` remain the daily ops set.

Pin `nixos-25.11` only to read old machines. Do not start new desk hosts there: it reached EOL on 2026-06-30.

Nix 2.35

Area	Change
Flake sources	Copied to the store lazily; unused flake files are not hashed into evaluation unless needed.
Sandbox	FreeBSD jail sandboxing; Linux sandbox unchanged.
Installer	Current upstream installer ships 2.35.2 (2026-08).
Security	Recursive-nix advisory fix from the 2.35.0 series.

Key feature — current CLI on a 26.05 host

Stock 26.05 may print `nix (Nix) 2.34.x`. For this book's **2.35+** baseline, install the upstream Nix installer on foreign Linux/macOS, or set `nix.package = pkgs.nixVersions.latest;` (or `nix_2_35`) on NixOS. Then:

```
nix --version
nix flake metadata github:NixOS/nixpkgs/nixos-26.05
```

Output (shape, after pinning 2.35):

```
nix (Nix) 2.35.2
Resolved URL:  github:NixOS/nixpkgs/nixos-26.05
```

NixOS 25.05 / 25.11

25.05 and 25.11 are previous stables. 25.11 "Xantusia" is **deprecated** (EOL 2026-06-30). If `system.stateVersion` is `"25.11"`, **leave it** and move the flake input to `nixos-26.05`.

NixOS 26.05 — Yarara

Area	Change
Support	Bugfix/security until 2026-12-31.
Initrd	Stage-1 systemd initrd is the default (not the old scripted initrd).
Toolchain	GCC 15; GNOME 50 on desktop ISOs.
Nix	Channel ships Nix **2.34**; book CLI baseline remains **2.35+** (installer or `nix.package`).
Modules	New options; confirm names with `nixos-option` if a 24.11 snippet fails eval.

Key feature — pin 26.05 in the desk flake

Save as `flake.nix`:

```nix
# flake.nix
{
  description = "Desk workstation on NixOS 26.05";

  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-26.05";

  outputs = { self, nixpkgs }: {
    nixosConfigurations.desk-workstation = nixpkgs.lib.nixosSystem {
      system = "x86_64-linux";
      modules = [
        {
          networking.hostName = "desk-workstation";
          system.stateVersion = "26.05";
        }
      ];
    };
  };
```

```
}
```

```
nix flake metadata
sudo nixos-rebuild dry-build --flake .#desk-workstation
```

Quick-Reference Matrix

The table below maps each NixOS release to the Nix CLI it ships and highlights the single most operationally significant change per pair.

NixOS Release	Nix Version Shipped	Most Impactful Addition
23.05 Stoat	2.13	Flakes opt-in stable via `experimental-features`
23.11 Tapir	2.18	`nix flake metadata` improvements; `--log-format` for CI
24.05 Uakari	2.22	`nix build --rebuild` reproducibility check; systemd initrd opt-in
24.11 Vicuna	2.24	`nix store diff-closures` stable; `services.openssh.settings.AllowGroups`
25.05	2.28	`nix flake check` / fetch-tree hardening; module cleanups
25.11 Xantusia	2.31	EOL 2026-06-30; last train before Yarara
26.05 Yarara	2.34 (pin 2.35+)	Current baseline; systemd stage-1 initrd default; GCC 15

Nix 2.19–2.34 intermediate releases are available via `pkgs.nixVersions` even when the channel ships another version. Pin only when you need a CLI feature the channel does not have yet:

```
# configuration.nix  (pin Nix version independently of channel)
{ pkgs, ... }:
{
  nix.package = pkgs.nixVersions.latest; # or pkgs.nixVersions.nix_2_35
}
```